数据结构与算法：Python 讲算法，C++ 讲实现

基于张铭、王腾蛟、赵海燕《数据结构与算法》重编
高等教育出版社 2008 年版教学内容的现代化讲义

目录

写给学生 i
如何使用本教程 i
两条阅读路径 i
两种语言 ii
文件组织 iii
怎样保留原书 iii
语言版本与实现状态 iv
现代 C++ 能力地图 iv
术语速查 v
属性和接口约定 vi
各章实现状态 vi
每章验证边界 vii
各章一览 vii
阅读约定 viii
验证与边界 viii

第 1 章 概论 1
1.1 问题求解 1
1.1.1 问题描述：股市的传言 1
1.1.2 问题分析和抽象 1
为什么要取 “最慢的一项” 3
1.1.3 数据结构和算法设计 4
实现导读 6
现代实现 8
1.2 数据结构 10
1.2.1 数据的逻辑结构 10
1.2.2 数据的存储结构 12
1.2.3 抽象数据类型 14
1.3 算法 15
1.3.1 算法的概念 16
1.3.2 算法设计 17
1.4 算法分析 18
1.4.1 渐进分析方法 19
1.4.2 最佳、最差和平均情况 25
1.4.3 时间和空间的折衷 27
1.4.4 求解问题时数据结构的选择和评价 27
本章小结 28
习题 29

补充证明练习（参考课程第 1 章） 29
正文习题 29
上机题 30

第 2 章 线性表 31
先跑一遍 31
2.1 线性表的概念 33
2.2 顺序表 35
为什么这一节没有 Python 版 35
教学版：完整实现 35
2.2.1 顺序表的类定义 39
2.2.2 顺序表的运算实现 40
2.2a 进阶（选读）：从教学版到工程版 43
与原书的对照 46
2.3 链表 47
为什么这一节没有 Python 版 47
教学版：完整实现 47
2.3.1 单链表 52
2.3.2 双链表 56
2.3.3 循环链表 63
2.3a 进阶（选读）：从教学版到工程版 64
2.4 线性表实现方法的比较 66
本章小结 67
习题 67
正文习题 67
上机题 68

第 3 章 栈与队列 69
先跑一遍 69
3.1 栈 70
为什么顺序栈和链式栈只有 C++ 70
3.1.1 栈的抽象数据类型 71
3.1.2 顺序栈 72
3.1.2a 进阶（选读）：从教学版到工程版 76
3.1.3 链式栈 84
3.1.3a 进阶（选读）：从教学版到工程版 88
3.1.4 表达式求值 90
3.1.5 栈与递归 97
运行栈有多大：三个档位，三种结果 99
反直觉的那一条：-O2 为什么不崩 100
递归吃运行栈，显式栈吃堆 100

原书这三个版本共同的问题：不查溢出 . . . 102
一个更复杂的例子：背包问题 . . . 103
与原书的对照 . . . 112
3.2 队列 . . . 113
为什么这一节没有 Python 版 . . . 113
3.2.1 队列的抽象数据类型 . . . 114
3.2.2 顺序队列 . . . 114
3.2.3 链式队列 . . . 116
3.2.3a 进阶（选读）：从教学版到工程版 . . . 121
3.3 栈与队列的深入讨论 . . . 124
3.3.1 顺序栈与链式栈的比较 . . . 124
3.3.2 顺序队列与链式队列的比较 . . . 124
3.3.3 限制存取点的表 . . . 125
出栈序列能不能得到：拿顺序栈模拟一遍 . . . 125
本章小结 . . . 126
习题 . . . 127
补充算法设计题（参考课程第 3 章） . . . 127
正文习题 . . . 127
上机题 . . . 128

第 4 章 字符串 131
先跑一遍 . . . 131
4.1 字符串的基本概念 . . . 132
4.1.1 字符编码 . . . 133
4.1.2 字符的编码顺序 . . . 134
4.2 字符串的存储结构和实现 . . . 135
为什么这一节没有 Python 版 . . . 135
4.2.1 字符串的顺序存储 . . . 135
4.2.2 字符串类 class String 的存储结构 . . . 137
4.2.3 字符串运算的实现 . . . 143
4.2a 进阶（选读）：从教学版到工程版 . . . 144
4.3 字符串的模式匹配 . . . 145
4.3.1 朴素的模式匹配算法 . . . 145
4.3.2 字符串的特征向量 . . . 150
4.3.3 KMP 模式匹配算法 . . . 154
KMP 的另一个用途：求最小循环节 . . . 156
与原书的对照 . . . 159
本章小结 . . . 160
习题 . . . 160
正文习题 . . . 160
上机题 . . . 161

第 5 章 二叉树 163
5.1 二叉树的概念 . . . 163
5.1.1 二叉树的定义和基本术语 . . . 163
5.1.2 满二叉树、完全二叉树、扩充二叉树 . . . 165
5.1.3 二叉树的主要性质 . . . 167
5.2 二叉树的周游 . . . 169
5.2.1 二叉树的抽象数据类型 . . . 169
5.2.2 深度优先周游二叉树 . . . 171
5.2.3 广度优先周游二叉树 . . . 181
5.3 二叉树的存储结构 . . . 183
为什么这一节没有 Python 版 . . . 183
5.3.1 二叉树的链式存储结构 . . . 184
5.3.2 完全二叉树的顺序存储结构 . . . 184
教学版：完整实现 . . . 185
5.4 二叉搜索树 . . . 195
为什么这一节没有 Python 版 . . . 195
5.5 堆与优先队列 . . . 198
为什么这一节没有 Python 版 . . . 198
5.5.1a 建堆为什么是 $O(n)$ 而不是 $O(n \log n)$ . . . 209
5.5.2 优先队列 . . . 211
5.6 Huffman 树及其应用 . . . 213
为什么这一节没有 Python 版 . . . 213
5.6.2 Huffman 编码 . . . 217
5.6a 进阶（选读）：从教学版到工程版 . . . 219
本章小结 . . . 220
习题 . . . 221
补充证明题（参考课程第 5 章） . . . 221
原书习题 . . . 222
上机题 . . . 224
原书上机题 . . . 224

第 6 章 树 227
6.1 树的定义和基本术语 . . . 227
6.1.1 树和森林 . . . 227
6.1.2 森林与二叉树的等价转换 . . . 229
6.1.3 树的抽象数据类型 . . . 230
6.1.4 树的周游 . . . 230
6.2 树的链式存储结构 . . . 231
6.2.1 “子结点表” 表示方法 . . . 231
6.2.2 静态 “左子/右兄” 表示法 . . . 232
6.2.3 动态表示法 . . . 233

6.2.4 动态 “左子/右兄” 二叉链表表示法 233
6.2.5 父指针表示法和在并查集中的应用 240
6.3 树的顺序存储结构 245
6.3.1 带右链的先根次序表示 246
6.3.2 带双标记的先根次序表示 246
6.3.3 带度数的后根次序表示 249
6.3.4 带双标记的层次次序表示 250
6.4 K 叉树 251
本章小结 251
习题 252
补充证明与算法题（参考课程第 6 章） 252
原书习题 252
上机题 254
原书上机题 254

第 7 章 图 257
7.1 图的定义和基本术语 257
7.2 图的抽象数据类型 261
7.3 图的存储结构 263
7.3.1 相邻矩阵 263
7.3.2 邻接表 264
7.3.3 十字链表 270
7.4 图的周游 274
先跑一遍 274
7.4.1 深度优先周游 275
7.4.2 广度优先周游 276
7.4.3 拓扑排序 278
7.5 最短路径 281
7.5.1 单源最短路径 281
7.5.2 每对顶点之间的最短路径 287
7.6 最小生成树 290
7.6.1 Prim 算法 291
7.6.2 Kruskal 算法 294
本章小结 296
习题 297
补充算法设计题（参考课程第 7 章） 297
正文习题 298
原书习题 298
上机题 301
原书上机题 302

第 8 章 内部排序 305
8.1 排序问题的基本概念 305
8.2 插入排序 306
8.2.1 直接插入排序 307
8.2.2 Shell 排序 310
8.3 选择排序 312
8.3.1 直接选择排序 312
8.3.2 堆排序 313
8.4 交换排序 316
8.4.1 冒泡排序 316
8.4.2 快速排序 318
8.5 归并排序 324
归并排序的副产品：逆序对计数 327
8.6 分配排序和索引排序 330
8.6.1 桶式排序 330
8.6.2 基数排序 333
8.6.3 索引排序 340
8.7 排序算法的时间代价 343
8.7.1 简单排序算法的时间代价 343
8.7.2 排序算法的理论和实验时间 344
8.7.3 排序问题的下限 346
本章小结 347
习题 350
补充复杂度题（参考课程第 8 章） 350
正文习题 350
原书习题 350
上机题 354
原书上机题 354

第 9 章 文件管理与外部排序 357
9.1 主存储器和外存储器 357
先建立成本模型 357
9.2 文件的组织和管理 358
9.2.1 文件组织 358
9.2.2 C++ 的流文件 359
记录怎样装进页 360
9.3 外排序 360
9.3.1 置换选择排序 361
9.3.2 二路外排序 366
9.3.3 多路归并——选择树 367
本章小结 377

习题 . . . 377
补充外排序题（参考课程第 9 章） . . . 377
正文习题 . . . 377
原书习题 . . . 378
上机题 . . . 379
原书上机题 . . . 379

第 10 章 检索 381
10.1 基于线性表的检索 . . . 381
10.1.1 顺序检索 . . . 381
10.1.2 二分检索 . . . 382
10.1.3 分块检索 . . . 384
教学版：完整实现 . . . 386
10.2 集合的检索 . . . 391
10.2.1 集合的数学特性 . . . 392
10.2.2 计算机中的集合 . . . 392
10.3 散列方法 . . . 393
10.3.1 散列函数 . . . 395
10.3.2 开散列方法 (拉链法) . . . 397
10.3.3 闭散列方法 (开地址法) . . . 399
10.3.4 闭散列表的算法 . . . 402
10.3.4a 进阶（选读）：工程版差在哪 . . . 408
10.3.5 散列方法的效率分析 . . . 408
10.3.6 散列方法的应用 . . . 410
本章小结 . . . 411
习题 . . . 413
补充检索题（参考课程第 10 章） . . . 413
正文习题 . . . 413
原书习题 . . . 413
上机题 . . . 415
原书上机题 . . . 416

第 11 章 索引技术 419
11.1 线性索引 . . . 419
11.2 静态索引——多分树 . . . 422
11.3 倒排索引 . . . 423
11.3.1 基于属性的倒排 . . . 424
11.3.2 对正文文件的倒排 . . . 424
11.4 动态索引 . . . 427
11.4.1 B 树 . . . 427
11.4.2 B+ 树 . . . 431

11.4.3 B 树的性能分析 436
11.4.4 动态索引和静态索引性能的比较 438
11.5 位索引技术 438
位图、签名和红黑树 438
练习路径 445
本章小结 445
Python 实现的证据链 445
习题 449
补充索引题（参考课程第 11 章） 449
正文习题 449
原书习题 449
上机题 453
原书上机题 453

第 12 章 高级数据结构 455
12.1 多维数组 455
12.1.1 多维数组的存储 456
12.1.2 特殊矩阵 456
12.1.3 稀疏矩阵 457
12.2 广义表和存储管理 459
为什么这一节没有 Python 版 459
12.2.1 广义表的定义和存储结构 459
12.2.2 可利用空间表 463
12.2.3 存储的动态分配和回收 466
12.2.4 失败处理策略和无用单元回收 468
12.3 Trie 结构和 Patricia 树 469
为什么这一节的存储侧只有 C++ 469
12.4 改进的二叉搜索树 474
12.4.1 最佳二叉搜索树 474
12.4.2 平衡的二叉搜索树 481
12.4.3 伸展树 488
本章小结 492
Python 实现的证据链 492
习题 496
补充概念与上机题（参考课程第 12 章） 496
正文习题 497
原书习题 497
上机题 499
原书上机题 499

习题、上机题与参考答案 503

第 1 章 概论 503
补充题参考答案 503
正文习题答案 503
正文上机题参考 505
课程作业与 ref_DSA 参考答案 505
课程上机参考 506
第 2 章 线性表 506
正文习题答案 506
正文上机题参考 507
课程作业与 ref_DSA 参考答案 508
课程上机参考 508
第 3 章 栈与队列 508
补充题参考答案 508
正文习题答案 508
正文上机题参考 510
课程作业与 ref_DSA 参考答案 510
课程上机参考 511
第 4 章 字符串 512
正文习题答案 512
正文上机题参考 512
课程作业与 ref_DSA 参考答案 513
课程上机参考 515
第 5 章 二叉树 515
补充题参考答案 515
正文习题答案 515
正文上机题参考 516
课程作业与 ref_DSA 参考答案 516
课程上机参考 517
原书习题答案 517
原书上机题参考 519
第 6 章 树 520
补充题参考答案 520
正文习题答案 520
正文上机题参考 521
课程作业与 ref_DSA 参考答案 521
课程上机参考 522
原书习题答案 522
原书上机题参考 524
第 7 章 图 524
补充题参考答案 524
正文习题答案 524

正文上机题参考 . . . 525
课程作业与 ref_DSA 参考答案 . . . 525
课程上机参考 . . . 525
原书习题答案 . . . 525
原书上机题参考 . . . 528
第 8 章 内部排序 . . . 528
补充题参考答案 . . . 528
正文习题答案 . . . 528
正文上机题参考 . . . 529
课程作业与 ref_DSA 参考答案 . . . 529
课程上机参考 . . . 530
原书习题答案 . . . 530
补充题参考答案 . . . 536
正文习题答案 . . . 536
正文上机题参考 . . . 536
课程作业与 ref_DSA 参考答案 . . . 537
课程上机参考 . . . 537
第 10 章 检索 . . . 538
补充题参考答案 . . . 538
正文习题答案 . . . 538
正文上机题参考 . . . 539
课程作业与 ref_DSA 参考答案 . . . 539
课程上机参考 . . . 539
第 11 章 索引技术 . . . 540
2021 书面作业补充题 . . . 540
补充题参考答案 . . . 540
正文习题答案 . . . 540
正文上机题参考 . . . 541
课程作业与 ref_DSA 参考答案 . . . 541
课程上机参考 . . . 541
第 12 章 高级数据结构 . . . 541
2021 书面作业补充题 . . . 541
补充题参考答案 . . . 542
正文习题答案 . . . 542
正文上机题参考 . . . 543
课程作业与 ref_DSA 参考答案 . . . 543
课程上机参考 . . . 544
参考答案的使用边界与 OJ 迁移 . . . 544
OJ 题型迁移与校正 . . . 544
来源文件登记 . . . 546
第 9 章 外部排序（原书题补充答案） . . . 547

原书习题答案 547
第 10 章 检索（原书题补充答案） 548
原书习题答案 548
第 11 章 索引（原书题补充答案） 551
原书习题答案 551
原书习题答案 557

期中复习题库（第 1—6 章） 561
第 1 章 概论 561
一、填空题 561
二、选择题 562
三、辨析与简答 563
四、算法设计与分析 564
五、数学证明 566
第 2 章 线性表 568
一、填空题 568
二、选择题 569
三、辨析与简答 570
四、算法设计与分析 571
五、数学证明 575
第 3 章 栈与队列 576
一、填空题 576
二、选择题 577
三、辨析与简答 577
四、算法设计与分析 579
五、数学证明 586
第 4 章 字符串 588
一、填空题 588
二、选择题 590
三、辨析与简答 591
四、算法设计与分析 593
五、数学证明 597
第 5 章 二叉树 598
一、填空题 598
二、选择题 600
三、辨析与简答 601
四、算法设计与分析 603
五、数学证明 613
第 6 章 树 615
一、填空题 615
二、选择题 617

三、辨析与简答 618
四、算法设计与分析 620
五、数学证明 629
模拟卷（120 分钟，100 分） 631
一、填空题（每题 2 分，第 1 题两空各 1 分，共 20 分） 631
二、选择题（每题 2 分，共 16 分） 632
三、辨析与简答题（每题 6 分，共 24 分） 633
四、算法设计与分析题（每题 10 分，共 30 分） 633
五、证明题（10 分） 636
考前速查表 636
渐进分析 636
线性表 637
栈与队列 637
字符串 638
二叉树 639
树与森林 639
答题格式提醒 640

期末复习题库（第 7—12 章） 641
大纲对照表 641
第 7 章 图 642
一、填空题 643
二、选择题 644
三、辨析与简答 646
四、算法设计与分析 647
五、证明题 649
第 8 章 内排序 650
一、填空题 650
二、选择题 652
三、辨析与简答 653
四、算法填空与设计分析 654
五、证明题 657
第 9 章 文件管理和外排序 658
一、填空题 658
二、选择题 659
三、辨析与简答 659
四、算法设计与分析 661
五、证明题 662
第 10 章 检索 663
一、填空题 663
二、选择题 664

三、辨析与简答 . . . 665
四、算法设计与分析 . . . 667
五、证明题 . . . 669
第 11 章 索引技术 . . . 670
一、填空题 . . . 670
二、选择题 . . . 672
三、辨析与简答 . . . 672
四、算法设计与分析 . . . 673
五、证明题 . . . 679
第 12 章 高级数据结构 . . . 680
一、填空题 . . . 680
二、选择题 . . . 682
三、辨析与简答 . . . 683
四、算法设计与分析 . . . 684
五、证明题 . . . 688
模拟卷（120 分钟，100 分） . . . 690
一、填空与选择（每空 2 分，共 24 分） . . . 690
二、辨析与简答（每题 6 分，共 24 分） . . . 690
三、算法填空（每空 2 分，共 12 分） . . . 691
四、算法设计与分析（每题 10 分，共 20 分） . . . 692
五、证明题（20 分，每题 10 分） . . . 694
考前速查表 . . . 694
图 . . . 694
内排序 . . . 695
外排序 . . . 696
检索 . . . 696
索引 . . . 697
高级数据结构 . . . 698
答题格式提醒 . . . 699

《数据结构与算法》MOOC 答案 & 解析 701
第一章 概论 . . . 701
1. 逻辑结构与存储结构 . . . 701
2. 线性结构 . . . 701
3. 算法特性 . . . 701
4. 时间复杂度 . . . 701
5. 大 O 表示法性质 . . . 702
第二章 线性表 . . . 703
2. 线性表存储性质 . . . 704
第三章 栈与队列 . . . 705
1. 中缀表达式转后缀 . . . 705

2. 栈容量计算 705
6. 卡特兰数应用 706
第四章 字符串 706
1.KMP 算法 next 数组（特征向量） 706
2-5. 字符串基本概念与运算 707
6. 互补回文串 708
9-11. 字符串变换与运算 709
10. 利用给定的字母映射表进行加密。 709
第五章 二叉树（上） 710
1-2. 二叉树性质 710
3-4. 完全二叉树高度 711
5-7. 二叉树遍历 711
10. 度数定理 712
第五章 二叉树（下） 712
1-4. 搜索树、堆与 Huffman 712
12-14. 状态栈模拟非递归遍历: 715
15. 三叉最小带权外部路径长度（WPL） 723
16. Huffman 节约比特计算 723
第六章 树 724
1-5. 森林与二叉树转换 724
6. 2-3 树非叶结点数 725
7-10. 树的逻辑结构与度数分析 725
11-12. 双标记法转换 726
15-16. 二叉树转森林 726
17-18. 并查集重载合并规则 727
第七章 图 727
1-5. 图论基础与最短路径 727
第八章 内排序（上） 729
7. 冒泡排序 730
13. 快速排序一次划分 731
第八章 内排序（下） 731
1-4. 排序算法选择 731
第九章 外排序 733
3-4. 败者树（Loser Tree） 733
5-7. 置换-选择排序段数 734
第十章 检索 736
3. 极值 ASL 计算 736
第十一章 索引 739
10-12. B+ 树容量计算 742
第十二章 高级数据结构（上） 743
4-6. 稀疏因子 744

第十二章 高级数据结构（下） 746
第一步：插入 2 747
第二步：插入 5 747
第三步：插入 6 747
第四步：插入 4 747
第五步：插入 1 748
第六步：插入 3 749
第七步：删除 6 750

原书勘误 757
怎么读 757
编译不过 757
能编译、跑起来是错的 759
书内自相矛盾 759
正文说法有误 759
反复出现、但单独一条通常还能「跑两下」 764
作者代码包里有没有 765
曾经误判、已撤回 770
还没收进本表的 770
参考勘误表补充（前六章） 770

考场代码包 773
它和书上印的不是同一份 773
用之前要知道的三件事 773
清单在包里的位置 774
今天的 g++ 编译得过吗 781
包里的坑 786

写给学生

基于《数据结构与算法》（张铭、王腾蛟、赵海燕编著，高等教育出版社，2008）重编。

这是 `dsa_raw.md` 的可读替代稿：保留原书的章节脉络、数据结构与算法思想，移除 OCR 噪声，并把所有示例统一为可编译、可测试的 C++17；讲算法的章节另给一份同样跑过测试的 Python 实现（见下文「两种语言」）。原 OCR 底稿继续只作为考证材料保留，不应作为学习文本阅读。

如何使用本教程

两条阅读路径

这本书的每个数据结构都有**两份实现**，写给两种不同的读者：

	教学版 `teaching.hpp`	工程版 `modern.hpp`
给谁看	第一次学这个数据结构的人	要写自己的容器、自己的库的人
在书里	**正文主线**，整个文件一次印全，抄下来就能编译	标了「进阶（选读）」的小节，按片段印
资源管理	**三法则**：析构 + 拷贝构造 + 拷贝赋值	**五法则**：再加移动构造 + 移动赋值
异常安全	不作承诺：分配与拷贝不抛时正确，`T` 拷贝抛出时可能漏内存（见下）	强异常保证（失败时容器像没动过）
类型约束	交给模板实例化时报错	`static_assert` 在编译期先拦一道
注解	不写 `[[nodiscard]]` / `noexcept`	写全

初学者只读教学版就够了。它不是「简化到不对」的版本——它一样进闸门，一样在 `-Werror` + AddressSanitizer/UBSan 与 `-O2` 两档下真编译真运行，一样有专门的断言测试。

但它的正确性有一条明确的边界，这里说清楚：教学版在**内存分配和元素类型 `T` 的拷贝都不抛异常**时是正确的——元素是 `int`、`std::string`、指针这类东西时就是如此，也就是教材里的全部场景。一旦 `T` 的拷贝真的抛出，几个手写裸指针的实现会**漏掉已经申请的内存**（顺序栈扩容中的新缓冲区、链式栈拷贝到一半的结点、二叉树递归克隆的半棵树）。工程版用 `try/catch` 补上了这一段，那正是各章「进阶（选读）」要讲的**强异常保证**。

换句话说：教学版少的是**工程加固**，而工程加固里有一项是**异常路径上的正确性**。也因为会漏内存，教学版连「基本异常保证」都谈不上——基本保证要求出异常后不泄漏资源、对象仍然可用，漏掉一块缓冲区就已经不满足了。不写成「教学版完全正确」，是因为那句话在 `T` 会抛的前提下不成立。

标着「进阶（选读）」的小节可以整节跳过，等到你需要写自己的容器时再回来。这个分层的理由与代价写在 `collab/DECISION_LOG.md` 的 D-012。

哪些结构有教学版：凡是「自己管着内存」的手写数据结构都有。

章节	教学版	正文位置	进阶节
2.2 顺序表	`ch02/array_list`	2.2 开头	2.2a
2.3 单链表	`ch02/linked_list`	2.3 开头	2.3a
2.3.2 双链表	`ch02/doubly_linked_list`	2.3.2	2.3a
3.1.2 顺序栈	`ch03/array_stack`	3.1.2	3.1.2a
3.1.3 链式栈	`ch03/linked_stack`	3.1.3	3.1.3a
3.2 队列	`ch03/queue`	3.2.3	3.2.3a
4.2 字符串类	`ch04/string_class`	4.2 开头	4.2a
5.3–5.4 二叉树与 BST	`ch05/binary_tree`	5.3	5.6a
5.5–5.6 堆与 Huffman	`ch05/heap_huffman`	5.5	5.6a
7.3.3 十字链表	`ch07/orthogonal_graph`	7.3.3	7.3.3 后半「现代实现」
10.1–10.3 检索与散列	`ch10/search_hash`	10.1.3 之后	10.3.4a

其余章节只有一份实现，因为它们讲的是**算法**而不是存储管理：排序（第 8 章）、KMP（4.3）、图算法（第 7 章）、背包（3.1.5）、外排序（第 9 章）、索引（第 11 章）、高级结构（第 12 章）。那些代码本来就是教学形态——没有五法则、没有异常安全的取舍，只有算法本身。再分一层只会多一份要同步的东西。

两种语言

这本书还有第二条分界，和上面那条**互相垂直**：讲**存储管理**的章节只有 C++，讲**算法**的章节 C++ 与 Python 各给一份（D-025）。

	讲什么	实现
存储侧	存储布局、所有权、拷贝与移动语义、异常安全	只有 C++
算法侧	算法本身的正确性与代价	C++ + Python

存储侧不给 Python，不是偷懒。顺序栈写成 `list`、链表写成 `list`、字符串类写成 `str`，三法则、五法则、强异常保证、对齐义务在 Python 里**根本没有对应物**——那不是换一种语言讲同一课，是把课删掉再换个语言讲别的。这和本书不用 `std::stack` 包装顺序栈是同一条理由。

两份实现不是逐行翻译。策略是同一份，代价不是同一份，每一处差异都在正文点名，并且都有断言钉着。第 8 章的三处最典型：

- **递归深度**：算法 8.6 的快排在 C++ 里只是慢，在 Python 里**跑不完**——CPython 递归上限默认 1000 层，两千个有序元素直接 `RecursionError`。于是算法 8.7 的「只递归短侧」在 C++ 里是省栈，在 Python 里是能不能跑（8.4.2）。
- **定长整数**：算法 8.11 基数排序在 C++ 里靠「异或最高位」一趟处理负数，那依赖 `int` 是 32 位定长补码；Python 的整数是任意精度，**没有**「**最高位**」，只能改用整体平移——代价多一次扫描，换来对 10^{30} 一样成立（8.6.2）。
- **稳定性可不可观测**：C++ 那份 `std::vector<int>` 的归并排序**无法验证稳定性**，两个相等的 3 换不换位置排完一模一样；Python 那份不改一个字就能排任何可比较对象，于是稳定性变成可以断言的东西（8.5）。

Python 代码与 C++ 代码受同一条契约管：书上印的每一段都标着 `file=code/.../modern.py# 锚点`，与那个文件逐字一致，并且在 `python3` 默认档与 `python3 -X dev -W error` 两档下真跑过。`-O` 档故意不跑——它会把 `assert` 整个剥掉，那一档下测试恒绿。

文件组织

每一段印在书上的 C++ 都与配套源码逐字一致。建议按第 1 至第 12 章顺序阅读：先理解问题，再运行该章的示例程序，最后对照实现。

想在浏览器里读，用 `python3 tools/build_site.py` 生成的网页版：打开 `book/site/index.html` 即可，左侧是全书目录、右侧是本章小节，代码块可一键复制。网页由这些 Markdown 渲染而来，不是另一份稿子。离线阅读还有 `book/pdf/数据结构与算法.pdf`。

怎样保留原书

这不是把 2008 年的文字逐段删掉再换一套新名词。原书中定义、推导、例题、图和算法策略只要仍然正确，就优先保留在同号章节。正文与课件用的插图**全部直接裁自 2008 年原版扫描件**；底稿里那 292 张 OCR 图库另存一份原书插图作对照（网页版与仓库里有，学生 PDF 不再重复收录）。OCR 把一幅带 `(a)`、`(b)` 面板的**同一原图**错切成多张时，正文可以把这些裁块还原为一个完整图号，但不会把原本独立的整图切开后再拼作新图。

遇到历史材料，取舍会写在相应位置，而不是悄悄删去：经典方法保留并说明今天的适用边界；接口、设备或实现已经变化的内容，保留其要解决的问题，换成当前写法并标出差异；不属于数据结构教学目标 或没有继续保留价值的内容，在 `collab/section_gaps.json` 留下理由、负责人和日期。第 9.2.2 节的 C++ 文件流就是一个例子：流、缓冲和定位仍然要讲，`<fstream.h>`、裸字节对象读写和手工 `close()` 则明确作为历史写法说明，不作为正文方案。

建议的学习顺序是第 1 至第 10 章；第 11 章把索引技术做成可运行的页模拟（线性索引、倒排、B+ 树、位图）；第 12 章收束广义表的共享回收、前缀树与动态规划。每章先理解抽象模型，再阅读实现，最后运行对应单元的测试。所有提取操作（例如 `pop`、`dequeue`）都用 `std::optional` 表达正常的空状态；参数、容量或权重不合法则抛标准异常。

```
# 编译并运行全书的现代实现与测试
python3 tools/check_code.py --allow-degraded

# 检查书稿中的代码与源码是否仍逐字一致
python3 tools/check_doc.py
```

单章可以先跑那个单元的 `demo.cpp`。第 1 章的编译命令在 ch01-adt.md 里逐项解释过；后面各章的命令形状相同，只改 `-I` 路径和源文件。

语言版本与实现状态

正文代码固定使用 C++17，以保证 GCC、Clang 和 MSVC 的可用性。C++20/23 的 concepts、ranges 和 views 不作为正文前置条件，只在需要时作为延伸阅读。每章内容按实现状态区分：

- **实现并测试**：有 `code/` 源码、`demo.cpp` 和测试，并由闸门编译运行。
- **概念导读**：解释结构、表示法或算法取舍，但没有把未经验证的代码印进正文。
- **练习实现**：给出接口、公式或题目，由读者实现并自行验证。

现代 C++ 能力地图

能力	首次出现	本书内的学习入口	路径
`std::optional` 表达「可能没有」	第 1 章 `best_source()`	1.1.3 初见，2.2.2 顺序表的检索有完整对照	正文
RAII、析构与三**法则**	第 2 章顺序表	2.2.1 存储与所有权	正文
五法则与移动语义	第 2、3 章	2.2a、3.1.2a「进阶」	选读

能力	首次出现	本书内的学习入口	路径
异常与**强异常保证**	第 2、3、5、10 章	2.2a、3.1.2a「进阶」	选读
`static_assert` 与 `<type_traits>`	第 2、3 章	2.2a、3.1.2a「进阶」	选读
迭代器与 range-for	第 2 章顺序表	2.2.1 末尾的 `begin()/end()`	正文
STL 算法	第 8、10 章	第 8 章排序、第 10 章检索	正文
`[[nodiscard]]`	第 1 章起	1.1.3 的 `best_source()`、2.2.2 与下面的“属性和接口约定”	选读
concepts、ranges、views	延伸阅读	不作为正文前置条件	选读

术语速查

正文里反复出现的几个词，第一次撞上时可以回来查。每条只讲两件事：**是什么**，以及 **不这样做会出什么事**——后者才是它们存在的理由。

`std::optional<T>`：一个「可能装着 T 的盒子」。表示「可能没有结果」的返回值，取值前必须先判断盒子空不空。它替代的是老写法「返回 `bool` 表示成败 + 用引用参数带出结果」—— 那种写法下调用方忘了看 `bool`，就会读到一个从没被写过的变量，程序不崩，只是默默按错误数据继续跑。第 2.2.2 节有完整对照。

三法则（Rule of Three）：一个类只要自己管着资源（这里是 `new` 出来的缓冲区或结点），就得同时写全**析构函数、拷贝构造、拷贝赋值**这三个函数。少写任何一个，编译器都会替你生成一个「逐成员照抄」的版本——照抄指针的结果是两个对象指向同一块内存，各自析构一次，**同一块内存被释放两次**。原书的 `arrStack` 和 `arrList` 都缺了这几个，第 3 章用 ASan 报告复现了这个二次释放。**教学版守到这一条为止**—— 在分配与 `T` 的拷贝都不抛异常的前提下，这已经够了（见上面「两条阅读路径」里的边界）。

五法则（Rule of Five）：在三法则之上再加**移动构造、移动赋值**。它们不修正确性，只修性能：把一个马上要销毁的临时对象赋给别人时，没必要深拷贝一遍，直接把指针「偷」过来即可。工程版写全五个，属于「进阶（选读）」。

RAII：把「资源的生命周期」绑在「对象的生命周期」上——构造时拿到，析构时自动还回去。好处是无论函数怎么退出（正常返回、提前 `return`、抛异常），析构都一定会跑，所以不会漏掉释放。`std::unique_ptr` 就是 RAII 的典型；本书在少数几处刻意不用它，理由和代价见 2.3.1 节的「所有权工具怎么选」。

`noexcept`：写在函数后面，承诺这个函数不会抛异常。它不只是文档——标准库和本书的容器 都会**查询**这个承诺来决定策略（例如扩容时是「移动元素」还是「拷贝元素」）。承诺了却真的抛，程序直接 `std::terminate`，没有补救余地。

强异常保证：一个操作要么完全成功，要么**像没发生过一样**，绝不留下半成品状态。典型做法是「先在新地方做好，全部成功了再换过去」。反例是原书扩容：先 `delete[]` 旧缓冲区 再搬元素，中途抛异常就停在一个既不是旧状态、也不是新状态的地方，对象从此不可用。第 3.1.2a 节（顺序栈的进阶小节）有可运行的对照测试。

摊还代价（amortized cost）：单次操作偶尔很贵，但把总代价分摊到所有操作上后每次很便宜。顺序表扩容是标准例子：绝大多数插入是 O(1)，偶尔一次翻倍要搬走全部 n 个元素；但容量是翻倍长的，上一次扩容（容量从 n/2 到 n）之后已经插入了 n/2 个元素，把这次搬迁分摊给它们，每个元素只摊到 2 次搬运。从空表一路插入 n 个元素，历次搬运合计 $1+2+4+\cdots<2n$ 次，所以每次插入的摊还代价仍是 O(1)。注意它说的是**平均**，不是保证——对延迟敏感的场合，那一次搬迁仍然会实打实地卡住。

属性和接口约定

代码中的 `[[nodiscard]]` 是 C++17 的标准属性，意思是 “调用者不应无意丢弃这个返回值”。它特别 适合查找结果、`std::optional`、是否成功以及新建对象等接口：

```
[[nodiscard]] std::optional<std::size_t> find(int key) const;
```

如果调用者只写 `table.find(42);`，编译器可能给出警告，提醒程序员检查返回值。它不是运行时检查、不是异常，也不会改变函数的返回类型；确实要忽略时可以显式写 `(void)table.find(42);`。本教程的 编译命令使用 `-Wall -Wextra -Werror`，因此把这类疏忽尽早变成编译错误。

各章实现状态

章节	状态	验证入口或边界
第 1 章 概论	实现并测试	`code/ch01/adt`；Floyd 示例与测试
第 2 章 线性表	实现并测试	`code/ch02/array_list`、`linked_list`、`doubly_linked_list`、`ownership`
第 3 章 栈与队列	实现并测试	`code/ch03` 下 7 个单元
第 4 章 字符串	实现并测试	`code/ch04`；朴素匹配与 KMP

章节	状态	验证入口或边界
第 5 章 二叉树	实现并测试	`code/ch05`；深退化树有栈风险
第 6 章 树	实现并测试	`code/ch06/general_tree`
第 7 章 图	实现并测试	`code/ch07/graph` 邻接矩阵、`code/ch07/adjacency_list` 邻接表，两者逐项对拍；`code/ch07/orthogonal_graph` 十字链表
第 8 章 内部排序	实现并测试	`code/ch08/sorting`；手写排序
第 9 章 外部排序	实现并测试	`code/ch09/external_sort`；内存模拟外存
第 10 章 检索	实现并测试	`code/ch10/search_hash`；散列与墓碑
第 11 章 索引技术	实现并测试	`code/ch11` 下 4 个单元；内存里的页模拟，不做真实磁盘 I/O
第 12 章 高级数据结构	混合	广义表、Trie/Patricia、稀疏矩阵、动态分配、标记–清除、AVL/伸展树与最优 BST 均实现并测试；只有多维数组的定位公式仍为概念导读

每章验证边界

“实现并测试” 表示对应 `code/` 单元在 g++ 与 clang++ 下各编两档——`-Werror` 加 ASan/UBSan 的调试档与 `-O2` 档——都编译并运行通过，并不表示所有平台、线程场景或极端 输入都已证明。macOS 环境若无法启动 ASan/UBSan，检查脚本会明确报告降级；递归结构的栈深度也 随编译器、优化级别和进程栈大小变化。概念导读章节不伪造未验证的实现，练习代码需要读者自行 编译和测试。

各章一览

1. 概论：1.1 问题求解，1.2 数据结构，1.3 算法，1.4 算法分析。
2. 线性表：2.1 概念，2.2 顺序表，2.3 链表，2.4 比较。
3. 栈与队列：3.1 栈，3.2 队列，3.3 比较。
4. 字符串：4.1 概念，4.2 存储与实现，4.3 模式匹配。

5. 二叉树：5.1 概念，5.2 周游，5.3 存储，5.4 BST，5.5 堆，5.6 Huffman。
6. 树：6.1 定义，6.2 链式存储，6.3 顺序存储，6.4 K 叉树。
7. 图：7.1–7.3 概念与存储，7.4 周游，7.5 最短路，7.6 最小生成树。
8. 内部排序：8.1 概念，8.2–8.6 各类排序，8.7 时间代价。
9. 文件管理与外部排序：9.1–9.2 外存与文件，9.3 置换选择与选择树。
10. 检索：10.1 线性表检索，10.2 集合，10.3 散列。
11. 索引技术：11.1–11.5 线性/多分树/倒排/B 树与 B+ 树/位索引；原书 11.6 的红黑树并在 11.5 末尾讲。
12. 高级数据结构：12.1 多维数组，12.2 广义表与存储，12.3 Trie，12.4 改进 BST。

对照纸书时，编译级硬伤与算法错见 原书勘误；底稿的 OCR 图库见 插图（仅网页版）。各章习题、上机题及参考答案见 习题与参考答案；按期中复习提要（第 1—6 章）编排的自测题与答案见 期中复习题库，按期末复习大纲（第 7—12 章）编排的见 期末复习题库；题目来源和验证状态见 参考资料逐项追踪矩阵。MOOC 题目的逐题解析作为来源参考附录收录，见 MOOC 题解附录；其中答案 需以本书正文、代码测试和追踪矩阵为准，不替代本书的自拟习题答案。

阅读约定

- **算法思想优先**。没有用标准库容器或排序算法替代本章要教的核心结构或算法。
- **资源所有权显式**。手写数组和结点链使用 RAII、析构、深复制和移动语义管理寿命。
- **测试是正文的一部分**。每个实现目录的 `test.cpp` 都覆盖其声明的原书清单；运行测试比 阅读一段未经执行的伪代码更可靠。
- **风险如实保留**。递归树周游与析构仍适合教学，但极深退化结构存在栈溢出风险；详见 `collab/UNVERIFIED-RISKS.md`。

验证与边界

本教程覆盖原书 105 条算法/代码清单：104 条做了现代化实现，1 条（代码 4.2，标准库 `basic_string` 的空声明骨架）经登记退场，理由见 `collab/exclusions.json`。`code/` 下的每个单元都有对应的 `unit.json`、现代实现、原书问题证据和可执行测试；书稿的 C++ 代码由工具逐字同步。完整的边界条件、未覆盖故障路径与 递归深度风险见 `collab/UNVERIFIED-RISKS.md`。

本教程采用 C++17，侧重数据结构的实现与算法语义。它不提供文件系统、数据库页缓存、并发控制 或跨平台性能承诺；这些属于使用数据结构的系统层设计。

第 1 章 概论

数据结构刻画信息及其关系，算法描述求解过程。二者互相依赖：结构选错了，算法再巧也补不回来。本章先用股市传言把问题说到能写程序，再补回逻辑结构、存储映射、抽象数据类型和渐进分析——这些原书没错，是后面十一章共用的词汇。

1.1 问题求解

程序是指令的组合，算法是求解过程的逻辑抽象，数据结构是数据及其关系在计算机里的反映。Wirth 的「程序 = 数据结构 + 算法」说的就是这件事。求解路径通常是：把现实问题抽象成模型，选定逻辑结构与存储方式，再设计算法并写成可运行的程序。

1.1.1 问题描述：股市的传言

本章把原书 1.1 的 “股市传言” 问题写成一个可直接调用的程序。它不是在找 “边最多” 的经纪人，而是在找一个起点：从这个人出发能把消息传给**所有**人，并且最慢的那条最短传播路径尽可能短。

1.1.2 问题分析和抽象

把 B1 到 B5 看成五个顶点。一条 `Bi -> Bj` 边表示消息能从 `Bi` 直接传给 `Bj`；边权是传播 时间。没有路径就记为 ∞，表示消息永远传不到那里。B3 是唯一可以到达全部经纪人的起点。

表 1.1　经纪人之间消息传递的关系（有向边，时间为权；原书按「经纪人 → (接收方, 时间)」逐行列出，这里每条边占一行）：

起点	终点	时间
B1	B4	4
B1	B2	8
B1	B5	3
B2	B5	8
B3	B1	6
B3	B2	7
B3	B4	10
B3	B5	2
B5	B1	5
B5	B2	5

B3 指出四条边，是唯一能到达其余所有人的起点。原书把这张关系表画成一个有向带权图：

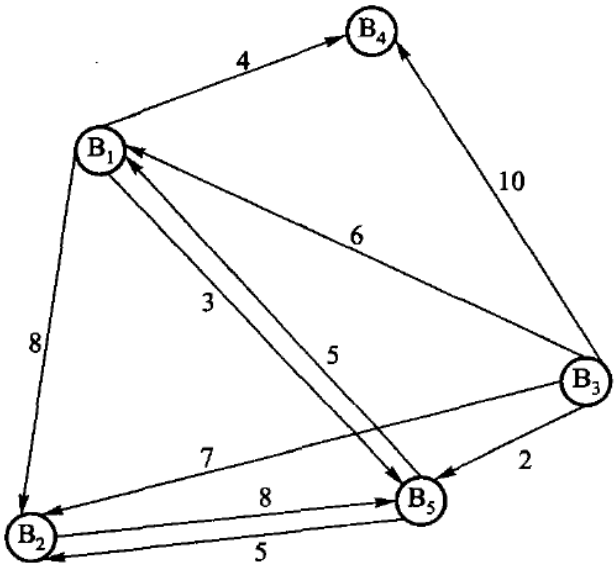


图 1.1　经纪人间的消息传递图。顶点 B_i 是第 i 个经纪人，有向边 $\langle \mathrm{B}_i, \mathrm{B}_j \rangle$ 表示 i 能把消息传给 j，边上的数是所需时间。B_i 到 B_j 与 B_j 到 B_i 的时间不一定相同——图中 $\mathrm{B}_1 \to \mathrm{B}_5$ 是 3，$\mathrm{B}_5 \to \mathrm{B}_1$ 是 5。

于是问题变成：在有向带权图里找一个顶点，从它出发能以最短时间把消息送到其余所有顶点。两点之间可以绕道——B_5 到 B_4 要经 B_1 中转，顶点序列 $\mathrm{B}_5\mathrm{B}_1\mathrm{B}_4$ 就是一条路径，路径长度是各边权之和；两点之间最短的那条路径叫最短路径。

本题要算任意两点之间的最短路径，所以图用相邻矩阵存：第 i 行第 j 列是边 $\langle \mathrm{B}_i, \mathrm{B}_j \rangle$ 的权，没有边记 ∞。图 1.1 对应的相邻矩阵就是原书的图 1.2。

	B1	B2	B3	B4	B5
B1	0	8	∞	4	3
B2	∞	0	∞	∞	8
B3	6	7	0	10	2
B4	∞	∞	∞	0	∞
B5	5	5	∞	∞	0

图 1.2　经纪人间消息传递的相邻矩阵，即算法的输入。底稿里这张表被 OCR 打坏——∞ **大量被认成** 8，所以它不能照抄；这里的取值以表 1.1 和图 1.1 为准，并与图 1.3 的结果矩阵逐格对得上。

原书算法 1.1 的工作可以拆成三步：

1. Floyd 算法算出任意两人间的最短传播时间。
2. 对每一个候选起点，找出 “从它出发到每个人的最短时间” 中最慢的一项。

3. 从这些最大值中选最小的一个；如果某行仍有不可达的 ∞，该起点不能胜任。

原书中的 `D` 对应现代代码里的 `shortest` 矩阵，`max[i]` 对应每一行的 `largest_distance`，`pos` 对应 `best_source()` 的返回值。下标从 0 开始，因此 B3 的返回值是 2。

为什么要取 “最慢的一项”

假设从 B3 开始，Floyd 算出的最短传播时间为：到 B1 是 6，到 B2 是 7，到自身是 0，到 B4 是 10，到 B5 是 2。消息要 “传遍所有人”，必须等最慢的 B4 收到消息，因此 B3 的完成时间是 这五个数里的最大值 10，而不是它们的和，也不是最小值。

把每个人都当一次起点，结果如下。∞ 表示至少有一人永远收不到消息，所以这一行没有可用的 完成时间。

起点	到 B1	到 B2	到 B3	到 B4	到 B5	最慢的最短时间	是否可选
B1	0	8	∞	4	3	∞	否
B2	13	0	∞	17	8	∞	否
B3	6	7	0	10	2	10	是
B4	∞	∞	∞	0	∞	∞	否
B5	5	5	∞	9	0	∞	否

这张表就是原书的图 1.3——Floyd 算出的结果矩阵：

	B_1	B_2	B_3	B_4	B_5
B_1	0	8	∞	4	3
B_2	13	0	∞	17	8
B_3	6	7	0	10	2
B_4	∞	∞	∞	0	∞
B_5	5	5	∞	9	0

图 1.3　Floyd 算完之后的矩阵。每一行是从 B_i 出发到各点的最短时间；第 3 行最大的一项是 10，其余各行都含 ∞。图 1.3 在底稿里是一张图片，没被 OCR 碰过，因此它是核对图 1.2 取值的证据：第 1 行第 2 列印的是 8 而不是 ∞，说明 B_1 确实能到达 B_2、最短时间为 8。注意这一格单凭结果矩阵还分不出是哪条

路——直连边是 8，绕道 $B_1 \to B_5 \to B_2$ 也是 $3+5=8$；$B_1 \to B_2$ 确有一条权为 8 的边，要看图 1.1 和表 1.1。

因此答案是 B3。这个目标常称为“最小化最大距离”：不是选离某一个人最近的起点，而是选让 **最后一个**收到消息的人也尽可能早收到消息的起点。

1.1.3 数据结构和算法设计

下面是完整可运行的程序。`RumorNetwork(5)` 创建 B1 至 B5；`add_route(from, to, cost)` 添加一条 **有向**传播路径，三个参数均从 0 开始编号。`best_source()` 返回 `std::optional<std::size_t>`——可以先理解成一个「可能装着下标的盒子」：有解时盒子里是起点下标，无解时盒子是空的（`std::nullopt`）。取值前必须先判断盒子空不空，所以「无解」这种情况不可能被漏掉。为什么不用「返回 `bool` + 引用参数带出下标」的老写法，第 2.2.2 节有完整对照。

源码文件在这里：可运行示例、数据结构实现、测试用例。`demo.cpp` 负责输入/输出；`modern.hpp` 只负责数据和计算，这就是把“怎么展示”与“怎么算”分开。

```cpp
#include "modern.hpp"

#include <iostream>

int main() {
    // B1 ... B5 对应下标 0 ... 4。
    dsa::adt::RumorNetwork network(5);
    network.add_route(0, 3, 4);   // B1 -> B4, 耗时 4
    network.add_route(0, 4, 3);   // B1 -> B5, 耗时 3
    network.add_route(0, 1, 8);   // B1 -> B2, 耗时 8
    network.add_route(1, 4, 8);   // B2 -> B5, 耗时 8
    network.add_route(2, 0, 6);   // B3 -> B1, 耗时 6
    network.add_route(2, 1, 7);   // B3 -> B2, 耗时 7
    network.add_route(2, 3, 10);  // B3 -> B4, 耗时 10
    network.add_route(2, 4, 2);   // B3 -> B5, 耗时 2
    network.add_route(4, 0, 5);   // B5 -> B1, 耗时 5
    network.add_route(4, 1, 5);   // B5 -> B2, 耗时 5

    const auto source = network.best_source();
    if (source) {
        std::cout << " 最佳传播起点是 B" << *source + 1 << '\n';
    } else {
        std::cout << " 不存在能到达全部经纪人的起点\n";
    }
}
```

在仓库根目录运行：

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch01/adt code/ch01/adt/demo.cpp -o /tmp/
↪  rumor-demo
/tmp/rumor-demo
```

第一行是 “编译”，第二行是 “运行刚刚编译出的程序”。逐项解释如下：

片段	含义
`c++`	C++ 编译器命令。在 macOS 上通常指 Clang；Linux 上也可能指 GCC。
`-std=c++17`	按 C++17 语言规则编译；本教程的 `std::optional` 需要 C++17。
`-Wall -Wextra`	打开常见和额外的编译器警告，例如未使用变量或可疑转换。
`-Werror`	把警告当作错误；用于学习时可及早发现问题。初学调试时可暂时去掉它。
`-Icode/ch01/adt`	加一个头文件搜索目录，因此 `#include "modern.hpp"` 能找到 `code/ch01/adt/modern.hpp`。
`code/ch01/adt/demo.cpp`	要编译的源文件。它包含 `main()`，所以可以成为可执行程序。
`-o /tmp/rumor-demo`	指定输出文件名为 `/tmp/rumor-demo`。`/tmp` 是临时目录，重启后文件可能消失。
`/tmp/rumor-demo`	运行该可执行文件，打印程序结果。

如果系统提示 `c++: command not found`，需要先安装 C++ 编译器；如果想把程序留在当前目录，可以把 `-o /tmp/rumor-demo` 改成 `-o rumor-demo`，再运行 `./rumor-demo`。

输出是：

```
最佳传播起点是 B3
```

注意，只删 B3 的一条出边并不能让 B3 失效。例如删掉 `network.add_route(2, 1, 7)`，B3 仍可经 B5 到达 B2（$2 + 5 = 7$），答案和最慢时间都不变——逐条删去这 10 条边中的任意一条，结果都还是 B3、10。要让 B3 真正到不了某人，得把通往那人的路全部切断，例如同时删掉 `add_route(2, 3, 10)` 与 `add_route(0, 3, 4)`：这样谁也到不了 B4。B3 没有入边，别人都到不了它，于是不存在任何 能覆盖全部顶点的起点，`best_source()` 返回空值，程序会输出 “不存在能到达全部经纪人的起点”。

可以先只改边和权值，再运行同一条命令观察结果。例如把 `B3 -> B4` 的时间从 10 改为 1，答案 仍是 B3，只是它的最慢传播时间从 10 缩短为 7。

实现导读

先只关注三个公开操作：构造函数建立距离矩阵，`add_route` 填入直接边，`best_source` 计算答案。`best_source` 内部复制一份矩阵，因而多次调用不会改变原始网络。下面按代码出现顺序解释。

预处理、头文件与命名空间

`#pragma once` 告诉编译器：同一个头文件即使被间接包含多次，也只处理一次，避免类定义重复。

头文件	本程序用它做什么
`<cstddef>`	提供 `std::size_t`，表示非负的大小或下标。
`<cstdint>`	提供 `std::int64_t`，即下面 `distance_type` 用的 64 位整数。
`<limits>`	提供 `std::numeric_limits<int>::max()`，取得 `int` 可表示的最大值。
`<optional>`	提供 `std::optional`，表达“可能没有答案”。
`<stdexcept>`	提供 `std::invalid_argument`，用于报告非法参数。
`<vector>`	提供动态数组 `std::vector`，这里用它保存二维距离矩阵。

`namespace dsa::adt { ... }` 把名字放入 `dsa::adt` 命名空间。完整类名因此是 `dsa::adt::RumorNetwork`，可避免项目中另一个人也定义 `RumorNetwork` 时发生重名。代码块中的 `// >>> adt` 与 `// <<< adt` 只是本书稿的同步标记，不参与 C++ 逻辑。

类、常量和构造函数

`class RumorNetwork` 定义一种新类型。`public:` 以下是调用者可以使用的接口；`private:` 以下是 实现细节，调用者不能直接改写。

```
static constexpr int infinity = std::numeric_limits<int>::max() / 4;
```

`static` 表示 `infinity` 属于类本身，而不是每个对象各存一份；`constexpr` 表示编译期常量。`infinity` 只用来表示「两人之间没有直接路线」，并规定每条路线的时间都必须小于它。**最短时间是若干段之和**，可能超过 `int` 能表示的范围，所以

求最短路时另用 64 位整数 distance_type,「到不了」则用它的最大值 unreachable 表示，不再借用 infinity—— 否则一条总长恰好等于 infinity 的合法路径会被误读成「到不了」（D-039）。

```
explicit RumorNetwork(std::size_t people)
    : distance_(people, std::vector<int>(people, infinity)) { ... }
```

这是构造函数：RumorNetwork network(5) 会调用它。冒号后的部分叫**成员初始化列表**，先创建 distance_，再进入花括号。它构造一个 5 行、每行 5 列的整数矩阵，初始全为 infinity。随后 循环把对角线改为 0，因为一个人到自己不需要传播时间。矩阵的第 from 行、第 to 列总是表示 从 from 到 to 的当前已知最短时间。

添加一条边

```
void add_route(std::size_t from, std::size_t to, int cost)
```

这三个参数分别是起点编号、终点编号和直接传播时间。第一段 if 检查编号是否越界、时间是否为 负、时间是否不小于 infinity（那样的值会和「没有路线」混淆）；不满足题目定义时抛出 std::invalid_argument，调用者可用 try/catch 处理。第二段 if 只在 新边更短时更新矩阵，所以即使重复添加同一方向的边，也保留较小的时间。

Floyd 三重循环

shortest 是一个与 distance_ 同样大小、元素为 distance_type（64 位整数）的矩阵：有直接路线的位置抄入路线时间，没有的位置记为 unreachable。

三层循环的含义不是 “随便循环三次”，而是按中转站逐步扩大可用路径：

```
for each via:       允许路径经过 via
  for each from:    固定起点 from
    for each to:    固定终点 to
      比较 原来的 from->to 与 from->via->to
```

条件 shortest[from][via] != unreachable && shortest[via][to] != unreachable 先确认两段都可达。若 from -> via -> to 的总时间更小，就更新 shortest[from][to]。例如 B2 到 B4 没有直接边，但 B2 → B5 是 8、B5 → B1 是 5、B1 → B4 是 4，所以 Floyd 最终得到 B2 → B4 是 8 + 5 + 4 = 17。

从最短路矩阵选答案

```
std::optional<std::size_t> result;
distance_type smallest_eccentricity = unreachable;
```

默认构造的 `result` 是空值，表示 “还没有找到合格起点”。`smallest_eccentricity` 保存目前见过的 最小完成时间。这里的 eccentricity（离心率）就是某起点到全部顶点的最短距离中的最大值。

外层循环每次处理一行，即一个候选起点；内层循环扫描该行。三目表达式 `distance > largest_distance ? distance : largest_distance` 等价于 “若新值更大就采用新值，否则保留 旧值”，最终得到这行的最大值。若它比当前最佳值小，就同时更新最佳时间与 `result`。

任何一行含 `unreachable` 时，该行最大值也是 `unreachable`，不会优于有限答案；若所有行都含 `unreachable`，`result` 保持为空，函数返回 `std::nullopt`。`[[nodiscard]]` 提醒编译器：调用者不应 无意丢弃这个可能为空的重要返回值。

私有数据成员

```
std::vector<std::vector<int>> distance_;
```

外层 `vector` 是行，内层 `vector<int>` 是一行中的列，合起来是二维矩阵。末尾下划线是本教程的 约定，表示私有成员；它与 `best_source` 里的循环变量 `distance` 或局部变量 `shortest` 不会混淆。

时间复杂度为 O(V^3)，空间复杂度为 O(V^2)。这适合顶点数较少、需要比较所有起点的示例；大图 通常应根据图的稀疏度和查询需求选择其他最短路径算法。

现代实现

```
#pragma once

#include <cstddef>
#include <cstdint>
#include <limits>
#include <optional>
#include <stdexcept>
#include <vector>

namespace dsa::adt {

// >>> adt
```

```cpp
class RumorNetwork {
public:
    static constexpr int infinity = std::numeric_limits<int>::max() / 4;
    /// 最短时间是若干段之和，可能超过 int；用 64 位，「到不了」另用 unreachable（D-039）。
    using distance_type = std::int64_t;
    static constexpr distance_type unreachable =
    ↪  std::numeric_limits<distance_type>::max();

    explicit RumorNetwork(std::size_t people)
        : distance_(people, std::vector<int>(people, infinity)) {
        for (std::size_t person = 0; person < people; ++person) {
            distance_[person][person] = 0;
        }
    }

    void add_route(std::size_t from, std::size_t to, int cost) {
       // cost 不小于 infinity 时，下面的 Floyd 会把这条路线当成「没有路线」，结果静默出错。
       if (from >= distance_.size() || to >= distance_.size() || cost < 0 || cost >=
        ↪  infinity) {
            throw std::invalid_argument("route");
        }
        if (cost < distance_[from][to]) {
            distance_[from][to] = cost;
        }
    }

    [[nodiscard]] std::optional<std::size_t> best_source() const {
        // >>> best-source-floyd
        const std::size_t people = distance_.size();
        std::vector<std::vector<distance_type>> shortest(
            people, std::vector<distance_type>(people, unreachable));
        for (std::size_t from = 0; from < people; ++from) {
            for (std::size_t to = 0; to < people; ++to) {
                if (distance_[from][to] != infinity) {
                    shortest[from][to] = distance_[from][to];
                }
            }
        }
        for (std::size_t via = 0; via < people; ++via) {
            for (std::size_t from = 0; from < people; ++from) {
                for (std::size_t to = 0; to < people; ++to) {
                    if (shortest[from][via] != unreachable &&
                        shortest[via][to] != unreachable &&
                      shortest[from][to] > shortest[from][via] + shortest[via][to]) {
                        shortest[from][to] = shortest[from][via] + shortest[via][to];
                    }
                }
```

```
            }
        }
        // <<< best-source-floyd

        // >>> best-source-pick
        std::optional<std::size_t> result;
        distance_type smallest_eccentricity = unreachable;
        for (std::size_t from = 0; from < people; ++from) {
            distance_type largest_distance = 0;
            for (distance_type distance : shortest[from]) {
                largest_distance = distance > largest_distance ? distance :
↪  largest_distance;
            }
            if (largest_distance < smallest_eccentricity) {
                smallest_eccentricity = largest_distance;
                result = from;
            }
        }
        return result;
        // <<< best-source-pick
    }

private:
    std::vector<std::vector<int>> distance_;
};
// <<< adt

}  // namespace dsa::adt
```

1.2 数据结构

数据结构描述的是：按一定逻辑关系组织起来的数据，它们在计算机里怎么存放，以及定义在其上的运算。三件事要分开看——逻辑结构、存储结构、运算——后面各章都在这三层之间做选择。

1.2.1 数据的逻辑结构

逻辑结构是从具体问题里抽出来的模型，只谈「有哪些结点、结点之间是什么关系」，不谈它们占几个字节。1.1 节经纪人之间的传播路径就是一个有向图：结点是人，关系是「消息能直接传给谁」。

从集合论的观点来看，数据的逻辑结构可以用一个二元组 $B = (K, R)$ 来表示。其中 K 是数据结点（node）组成的有穷集合，每一个结点代表一个数据或一组有明确结构的数据；关系集 R 是定义在集合 K 上的一组二元关系（binary relation），其中每个关系 $r \in R$ 都是 K 上的二元关系，用来描述数据结点之间的逻辑关系。K 上的二元组是 K 中元素的有序对，记为 $\langle k, k' \rangle$（$k, k' \in K$）。

K 上的一个关系即为 K 上的一些二元组所组成的集合，K 上不同的二元组集合构成不同的关系。

若 $r \in R$，有 $k, k' \in K$，$\langle k, k' \rangle \in r$，则称 k 为 k' 在关系 r 上的**前驱**，k' 为 k 在关系 r 上的**后继**。没有前驱的结点称为**开始结点**，没有后继的结点称为**终止结点**。

教学计划是另一个常见例子。每门课是一个结点，全部必修课组成 K；「必须先修」是 K 上的一个关系。< 程序设计，数据结构 > 表示必须先上程序设计，再上数据结构。没有先修约束的课可以并行开设。

结点的类型。数据结构重点研究的是数据间的结构关系，因此把组成结构的那些元素都视为结点；结点是数据结构中数据的基本单位。结点的数据类型既可以是基本数据类型，也可以是复合数据类型。基本数据类型是指数据结点所包含的元素被作为整体看待和处理的类型，类似于常见的程序设计语言中提供的基本数据类型，例如整数类型（integer）、实数类型（real）、布尔类型（bool）、字符类型（char）和指针类型（pointer）等。复合类型是由基本数据类型组合而成的数据结构类型，不同的组合方式形成不同的类型；像程序语言中的数组类型、结构（记录）类型、对象类型等都属于复合数据类型，而复合数据类型本身，又可以参与定义结构更为复杂的结点类型。总而言之，结点的类型不限于基本数据类型，可以根据应用的需要来灵活定义。数据结构课关心的是结点之间的关系，所以通常把结点当作黑盒。

结构的分类。讨论逻辑结构 (K, R) 的结构分类，一般以关系集 R 的分类为主。在本书讨论的数据结构中，R 中一般只包含一个关系。假定关系集 R 仅含一个关系 r，即 $R = \{r\}$，此时记 (K, R) 为 (K, r)。根据 r 所刻画的数据结构的性质来对逻辑结构进行分类。

线性结构（linear structure）中的关系 r 称为线性关系，也称前驱关系。结点集合 K 中的每个结点在关系 r 上最多只有一个前驱结点和一个后继结点。具体而言，结点集合 K 在关系 r 上存在一个唯一的开始结点，它没有前驱、但有唯一的后继；也存在一个唯一的终止结点，具有一个唯一的前驱而没有后继；其他结点均为内部结点，在关系 r 上既有一个唯一的前驱，也有一个唯一的后继。线性结构在程序设计中应用最多，例如数组、链表、栈和队列等。

树形结构（tree structure）简称树结构或层次结构，其关系 r 一般称为层次关系。结点集合 K 中有且仅有一个结点 $k_0 \in K$，在关系 r 中没有前驱，该结点称为树根（root）。除结点 k_0 外，K 中所有结点都有且仅有一个前驱，但后继的数目不加限制。同时，K 中除结点 k_0 外的任何结点 $k \in K$，都存在一个结点序列 $k_0, k_1, \cdots, k_s$，使得 k_0 是树根，且 $k_s = k$，有序对 $\langle k_{i-1}, k_i \rangle \in r$ $(1 \leq i \leq s)$，这样的结点序列称为从根到结点 k 的一条**路径**（path）。树形结构存在很多形态，如二叉树结构、有序树结构等，它们都有着各自独特的应用。UNIX 和 DOS 中的文件系统就是一个典型的树结构。

图结构（graph structure）有时也称为网络结构。图结构的关系 r 对集合 K 中结点的前驱和后继数目不加任何约束。例如，交通网络就是一个非常复杂的典

型图结构；1.1 节的传言网也是图。

从数学上看，树形结构和图结构的基本区别就是「每个结点是否仅仅从属于一个直接前驱」，而线性结构和树形结构的基本区别是「每个结点是否仅有一个直接后继」。

以上这种结构分类揭示出枯燥数据之间的相互关系，给出了关系本身的一般性质。这对于理解数据结构以及正确设计算法都很重要，后面各章都建立在这三种关系上。

1.2.2 数据的存储结构

存储结构回答的是：同一套逻辑关系，在计算机的存储器里怎么落下来。主存按字节编址，相邻单元的地址连续，并且可以按地址随机访问、访问时间基本相同。

计算机的主存储器具有「空间相邻」和「随机访问」的特点：基本的存储单元是字节，用非负整数对存储地址编码，提供对存储空间上相邻的单元集合的随机访问；计算机指令具有按地址随机访问存储空间内任意单元的能力，访问不同地址所需的访问时间基本相同。

对于逻辑结构 (K, r) 而言，其数据的存储结构就是建立一种由逻辑结构到物理存储空间的映射。一方面，需要建立一个从结点集合 K 到存储器 M 的映射 $K \to M$，其中每一个结点 $k \in K$ 都对应一个唯一的连续存储区域 $c \in M$；另一方面，还要把每一个关系元组 $\langle k_i, k_j \rangle \in r$（其中 $k_i, k_j \in K$ 是结点）映射为相应的存储单元的地址间的关系（顺序关系，或指针的地址指向关系）。

同一种逻辑结构可以采用不同的表示方式，即采用不同的映射关系来建立数据的逻辑结构到存储结构的转换。例如，线性结构既可以采用顺序的方式来存储，形成顺序表；也可以采用链接的方式，得到链表。

常用的四种基本方法如下。

顺序方法把一组结点存放在一片地址相邻的存储单元中，结点间的逻辑关系用存储单元间的自然顺序关系来表达。顺序存储法为使用整数编码访问数据结点提供了便利，按下标访问因此是 $O(1)$；程序设计语言内提供的数组就是顺序方法的一个具体实例。

顺序存储结构通常也被称为**紧凑存储结构**：其紧凑性是指其存储空间除了存储数据本身之外，没有存储其他附加的信息。紧凑性可以用**存储密度**来度量——所存储的「数据」占用的存储空间，和该结构（包括附加信息）占用的整个存储空间大小之比。显然，存储密度太小的存储结构，其空间效率比较低。

除线性结构外，对于部分非线性数据结构也可以采用顺序存储方法。例如树形结构也可采用顺序方式来存储，前提是同时存储一些附加信息来表示结点之间的逻辑关系：完全二叉树按层编号后，孩子下标可以用 $2i+1$、$2i+2$ 算出来（第 5 章）。

链接方法在每个结点里附一个（或多个）指针域，专门存放后继的地址。结点因此分成数据域和指针域。需要一个结点同时挂多个后继时，就放多个指针。链接适合经常增删、长度事先不知道的结构。代价是：不知道指针时，只能从链

头一个一个比下去；按位置访问不再是 $O(1)$。

索引方法是顺序存储的一种推广：通过建造一个由整数域 Z 映射到存储地址域 D 的索引函数 $Y : Z \to D$，把整数索引值 z 映射到结点的存储地址 $d \in D$，从而形成一个存储了一组指针的索引表，每个指针指向存储区域的一个数据结点。索引表的存储空间是**附加在结点存储空间之外**的，每一个元素就是指向相应数据结点的指针，即结点存储单元的开始地址。

作为一种存储机制，索引的主要作用是提高检索的效率。当数据量很大时，对数据的检索可能涉及大量读/写磁盘的操作，会影响效率；通过索引则可以降低读/写的数据量——根据检索码确定了被检索数据的存储地址之后，再进行相应的读/写。

原书特意指出，索引函数一般**不是**数组那样的简单线性函数——数据结点长度不等时它就不可能是线性的。图 1.4 画的正是这种情况：

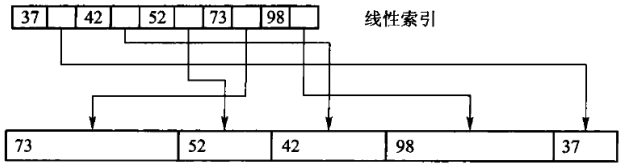


图 1.4 索引示例

把它抄成文字，两行数字对照着看：

```
存储区的数据 ——物理无序，而且每块长度不等

┌──────────┬──────────┬──────────┬──────────┬──────────┐
│ [1] 73   │ [2] 52   │ [3] 42   │ [4] 98   │ [5] 37   │
└──────────┴──────────┴──────────┴──────────┴──────────┘

线性索引 ——附加在数据之外的一张小表

┌──────┬──────┬──────┬──────┬──────┐
│  37  │  42  │  52  │  73  │  98  │   关键码：升序 ——逻辑有序
├──────┼──────┼──────┼──────┼──────┤
│ [5]  │ [3]  │ [2]  │ [1]  │ [4]  │   指向第几块：乱序 ——物理无序
└──────┴──────┴──────┴──────┴──────┘
```

这两行的错位就是索引的全部意义。上面一行升序，所以可以在索引里二分；下面一行完全乱序，说明数据一块都没有搬动过。要找关键码 52，先在索引里二分定位到第 3 列，读出 `[2]`，再直接去存储区取第 2 块——**只读了一块，没有扫过其余四块**。

顺带看一眼每块的宽度：五块长度各不相同。正因为长度不等，“第 k 块在哪”就没法用 `起始地址 + k × 块长` 算出来，只能查表——这才是原书说”索引函数并非数组那样的简单线性函数”的意思。

散列方法是索引法的一种延伸和扩展：散列法利用散列函数进行索引值的计算，然后通过索引表求出结点的指针地址。其主要思想是根据结点的关键码值来

确定其存储地址——利用一种称为散列函数（hash function）的关系，把关键码的值映射到存储空间的地址，然后把结点存入此存储单元中。散列技术的关键问题是如何选择和设计恰当的散列函数、构造散列表、研究构造散列表时的碰撞解决方案等。

一个散列函数把一个给定的关键码映射为一个小于 K 的非负整数，K 的大小取决于具体的应用。散列函数应该满足一些重要的散列性质：散列函数计算出的地址尽可能均匀地分布在构造的散列表地址空间；散列函数的计算应该简单化，以便提高地址计算速度。冲突怎么处理、装载因子多高会退化，是第 10 章的内容。

在一个具体的应用中，可以根据需要选用以上 4 种存储方法或其组合。例如，树形结构的「子结点表」表示方法就是顺序和链接的结合（第 6 章）。另外，一个逻辑结构可以有多种不同的存储方案，因此在选择存储方法时，还要综合考虑定义在其上的运算及其算法的实现——要不要随机访问、会不会频繁插入、数据在内存还是外存。

1.2.3 抽象数据类型

抽象意味着同一个概念可以有多种实现。一旦抽象问题被解决，许多同类问题就不必从头再来。用户自定义的类型，通常就是对「多种可能的存储和实现」做一次抽象。

抽象数据类型（abstract data type，ADT）这个词是逐步形成的。二十世纪六七十年代，人们把用户自己定义的类型叫做抽象数据类型。更早的算法语言往往不提供这种机制。六十年代后期，为了把算法细节和数据的内部结构藏起来，Simula 67 引入了类，随后出现了模块：模块式给出外部看得见的运算接口，模块体存放对外不可见的私有数据和实现。接口与实现分离之后，用户才能真正自定义类型，达到数据抽象和信息隐蔽。在这个意义下，模块所定义的数据类型就叫做抽象数据类型。

作为描述数据结构的理论工具，ADT 可以看成「定义了一组操作的抽象模型」。它把数据和操作封在一起，使用方只能通过这些操作访问数据，不能直接去翻内部表示。这样，讨论一种结构时可以先不管它是数组还是链表，只问它提供哪些运算、这些运算有什么约定。整数连同加、减、乘、除，就是一个最简单的 ADT。

一般写成三元组 (D, R, P)：

```
ADT 名字 {
    数据对象 D：结点是什么
    数据关系 R：结点之间有什么关系
    基本操作 P：对外提供哪些运算
}
```

D 和 R 给出数据模型，是对数据的抽象；P 给出这个模型能做什么，是对操作的抽象。

在面向对象语言里，一组操作常常先写成接口，不透露实现。C++ 用类（以及类模板）的声明表示 ADT，用类的定义去实现它。因此，一个写好的 C++ 类同时承担两件事：它是某种存储结构，也是这组运算在该存储上的实现。

这两层不要混：概念层上的 ADT 描述「问题需要什么运算」；实现层上的类描述「这些运算在内存里怎么做」。类本身还是一种普通类型，可以拿来定义变量，也就是对象。所以 ADT 也可以说成：数据的逻辑结构，加上定义在这个逻辑结构上的一组抽象运算。

原书【代码 1.2】用下面这个空模板来提醒读者「先写接口」：

```
template <class Type>
class ClassName {
private:
    Type data;          // 取值类型和存储
public:
    void method();      // 对外运算
};
```

原书习惯先写一个只有函数声明、没有任何实现的抽象基类（例如 `List<T>`），再让 `arrList` 继承它。本书各章不这么做，原因有两条：

- **它给不了多态**。基类之所以有用，是因为可以「拿基类指针指向派生对象」，写一份代码同时适用于顺序表和链表。但本书各章的容器是**模板**，元素类型在编译期就定了，彼此之间根本不需要互相替换；这个基类从头到尾没有一处用到。
- **它还会引入一个陷阱**。只要基类的析构函数不是 `virtual` 的，一旦有人写出 `List<int>* p = new ArrayList<int>; delete p;`，标准规定这是**未定义行为**——编译器不保证会发生什么：常见后果是只析构了基类那一层，派生类里 `new` 出来的缓冲区 没人释放，内存就漏了；也可能直接崩。「未定义行为」这个词在本书里会反复出现，它的意思始终是「标准不再对程序的行为作任何承诺」，而不是「结果不确定但还能用」。

所以运算直接写在实现类上。对元素类型的要求，正文里用一句话说清即可：例如顺序栈、顺序表底层是 `new T[n]`，会把整块槽位都默认构造出来，所以 `T` 必须能默认构造。

1.3 算法

算法的研究可以追溯到公元前 300 多年。算法的中文名称出自《周髀算经》，而英文名称 algorithm 则源于 9 世纪波斯数学家比阿勒·霍瓦里松的名字 al-Khwarizmi，他首先在数学上提出了算法这个概念。算法的第一次编写是 Ada Byron 于 1842—1843 年间为巴贝奇分析机编写计算伯努利数的程序（原书作「求解伯努利方程」；Ada 的注释 G 算的是伯努利数，这里按史实更正），因此 Ada Byron 被大多数人认为是世界上第一位程序员；由于查尔斯·巴贝奇（Charles Babbage）未能完成他的分析机，这个算法最终未能在机器上执行。20 世纪的英国数学家图

灵（Turing）提出了著名的图灵论题，并提出一种假想的计算机的抽象模型，这个模型被称为图灵机。图灵机的出现解决了算法定义的难题，对算法的发展起到了重要的作用，使得大多数算法都可以转换成程序交给计算机执行，原来认为靠人力难以完成的算法也变得可行。由此也拉开了算法研究和实用的帷幕。

从应用范围来看，算法可分成数值算法和非数值算法两大类；从工作方式看，算法可分成串行算法和并行算法两大类。早在 20 世纪 70 年代，D. E. Knuth 就指出，计算机科学就是研究算法的学问。本书后面各章几乎都是非数值的、串行的算法。

1.3.1 算法的概念

简单来说，算法是对特定问题求解过程的描述，是指令的有限序列，即为解决某一特定问题而采取的具体而有限的操作步骤。程序是算法的一种实现，计算机按照程序逐步执行算法，实现对问题的求解。

求最大公因子的辗转相除法，以及求解联立线性方程组的主元素消除法，都是算法的典型例子。求解一个问题，通常用该问题的**输入数据类型**和该问题所要求解的**结果（算法的输出数据）所应遵循的性质**来描述。以求最大公因子算法为例，它的输入是整数类型，输入数据是任意给定的两个正整数 n 和 m，而算法的输出则是既能整除 n 又能整除 m 的所有公因子中的最大的非负整数。对于求解联立线性方程组，它的输入数据是方程的系数矩阵和方程等式右侧的常数向量，而其输出结果数据是方程变元的一组取值，它们代入方程应该满足所给的等式。

算法一般具有以下性质。

1. **算法的通用性**。对于那些符合输入数据类型的**任意**输入数据，都能根据算法进行问题求解，并保证计算结果的正确性。不是只对书上那一组样例成立。
2. **算法的有效性**。算法是有限条指令组成的指令序列，其中每一条指令都必须能够被人或机器所确切执行。指令的类型应该明确规定，仅限于若干明确无误的指令动作，是一个有限的指令集。例如辗转相除法，其基本动作（指令）只包括整数相除求余数和商数以及判断余数是否为 0 等几个指令。这些基本指令不仅能够被准确无误地执行，而且每一步的执行结果具有确定的类型。算法的有效性不仅指出了每一步指令能够被有效执行，而且也规定了指令的执行结果，其结果应具有确定的数据类型，是可预期的。
3. **算法的确定性**。算法每执行一步之后，关于下一步怎么执行，应该有明确的指示。下一步动作可以是条件判断、分支指令、顺序执行一条指令，或者指示整个算法的结束等。算法的确定性就是要保证每一步之后都有关于下一步动作的指令，不能缺乏下一步指令（即「被锁住」状态），或仅含有模糊不清的指令。
4. **算法的有穷性**。算法的执行必须在有限步内结束，换言之，算法不能含有死循环。注意，**算法由有限条指令所组成这一事实本身并不能保证算法执行的有穷性**：循环条件写错，有限条指令也可以转个没完。设计算法时，应该关注算法的结束条件。

总之，了解这些基本性质有助于人们更确切地理解算法的概念。

1.3.2 算法设计

算法设计与算法分析是计算机科学的核心问题。算法设计是求解问题时必须要考虑的，其任务是对各类具体问题设计求解的方法和过程。常用的算法设计方法有穷举法（enumeration）、回溯法（backtrack）、分治法（divide and conquer）、递归法（recursion）、贪心法（greedy）和动态规划法（dynamic programming）等。下面分别就这些算法设计方法予以简单介绍。

1. **穷举法**。穷举法也称枚举法，其基本思想是将问题空间中的所有求解对象一一列举出来，然后逐一加以分析、处理，并验证结果是否满足给定的条件。穷举完所有对象后，问题将最终得以解决。穷举法具有以下一些特点：

1. 对象应该是有限的，有明显的穷举范围；
2. 有穷举规则，可按某种规则列举对象；
3. 一时找不出解决问题的更好途径。

一般而言，对一个问题空间进行全局的穷举，往往很费时间，效率上难以满足要求，但在问题的局部采用穷举法常很有效。例如，C/C++ 语言中常用的分支语句 `switch` 就是一种穷举。

2. **回溯法**。回溯法也称试探法，该方法的基本思想是将问题的候选解按某种顺序逐一枚举和检验，来寻找一个满足预定条件的解。当发现当前候选解不可能是解时，就退回到上一步重新选择下一个候选解；如果当前候选解还不满足问题规模要求，但是满足所有其他要求，那么继续扩大当前候选解的规模，并继续试探；如果当前候选解满足包括问题规模在内的所有要求时，该候选解就是问题的一个解。在回溯法中，放弃当前候选解、寻找下一个候选解的过程称为**回溯**；扩大当前候选解的规模、以继续试探的过程称为**向前试探**。

第 5、6、7 章将要介绍的深度优先搜索就是典型的回溯法实例；第 3 章的背包问题也是。

3. **分治法和递归法**。分治法的设计思想很朴素，其要点是在遇到一个难以直接解决的大问题时，将其分割成一些规模较小的子问题，以便各个击破、分而治之，然后把各个子问题的解合并起来，得出整个问题的解。分治法是一种**自顶向下**的设计方法。

如果原问题可分割成若干个子问题，这些子问题都可解，并且可利用这些子问题的解求出原问题的解，那么这种分治法就是可行的。由分治法产生的子问题往往是原问题的较小模式，在这种情况下，反复应用分治手段，可以使问题的规模不断缩小，最终缩小到很容易直接求出其解的规模，这自然会导致递归过程的产生。**分治与递归如同一对孪生兄弟**，经常同时应用在算法设计之中，并由此产生许多高效算法。

本书第 8 章的快速排序、归并排序，以及第 10 章的二分检索，都是分治法的应用实例。（原书此处写的是「第 8、9 章」，但二分检索讲在第 10 章，这里按本书的实际编号更正。）

4. **贪心法和动态规划法**。贪心法的基本思路是从问题的初始状态出发，依据某种贪心标准，通过若干次的贪心选择而得出最优值（或较优解）。贪心法并不

是从整体上考虑问题，它所做出的选择只是在某种意义上的局部最优解，寄希望于由局部的最优解构建最终的全局最优解。**选择能产生问题最优解的最优度量标准，是使用贪心法的核心问题**。

比贪心法更为一般的动态规划法通常也用于求解具有某种最优性质的问题。当一个问题的解可以看成一系列判定的结果时，可以利用动态规划方法设计其求解算法。在这类问题中，可能会有许多可行解，希望从中找到具有最优值的解。动态规划法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题，先求解子问题，然后从这些子问题的解得到原问题的解。**与分治法不同的是**，适合于用动态规划求解的问题，经分解得到的子问题往往不是相互独立的。若用分治法来解这类问题，则分解得到的子问题数目太多，有些子问题被重复计算了很多次。如果能够保存已解决的子问题答案，而在需要时利用这些已求得的答案，就可以避免大量的重复计算，节省时间。一般用一张表来记录所有已解的子问题的答案：不管该子问题以后是否被用到，只要它被计算过，就将其结果填入表中——这就是动态规划法的基本思路。

第 7 章介绍的最小生成树的 Prim 算法和 Kruskal 算法、求最短路径的 Dijkstra 算法和 Floyd 算法，其中 Floyd 算法是比较典型的动态规划，而其他 3 种算法是贪心算法。第 12 章的最佳二叉搜索树也是动态规划。

可以选择和组合这些基本算法设计方法，根据问题的资源约束和要求，设计出相关数据结构和算法来求解问题。运用算法的常见方式有以下 4 种：

1. 算法的组合；
2. 经典算法的变形与推广；
3. 研究困难问题的特殊情况；
4. 探索新的算法。

回到 1.1 节的传言问题：它要的是任意两点之间的最短路径，并且顶点数很小，所以选 Floyd，而不是对每个起点各跑一遍 Dijkstra。后面各章会反复回到这几种方法。

1.4 算法分析

一般而言，同一类问题总是存在着多种解决方案，即针对一个问题可以有多个算法。这些算法也许各有其优缺点以及适用的场景。那么如何在多个算法中按照某种原则进行取舍？这就需要用到算法分析技术来评价算法的效率。

算法分析的任务是利用数学工具，对每一个具体算法讨论其各种复杂度，以探讨某种具体算法适用于哪类问题，或某类问题宜采用哪种算法。其目的在于从解决同一问题的不同算法中选择出比较合适的一种，或者是对原有的算法进行改造、加工，使其更优。

为比较不同算法的效率，Juris Hartmanis 和 Richard E. Stearns 提出了一种称为计算复杂性（computational complexity）的方法来度量算法的难度。计算复杂性分析显示了实施一个算法所需的努力或代价。对于一个算法而言，其复杂性的高低体现在运行该算法所需要的计算机资源的多少上：所需的资源越多，表

明该算法的复杂性越高；反之越低。计算机资源中最重要的是时间资源（即处理器）和空间资源（即存储器），因而算法的复杂性通常体现为时间复杂性和空间复杂性两个指标。本书后文说的「时间代价」「空间代价」，指的就是这两项。（这两个人名在原书里印作 “Juris Hartmanishe” 与 “Richars E. Stearns”，此处按学界通行拼法更正。）

不言而喻，对于任意给定的问题，设计出复杂性尽可能低的算法是人们追求的一个重要目标；另一方面，当给定的问题已有多种算法时，选择其中复杂性最低者，也是选用算法时应遵循的一个重要准则。因此，复杂性分析对算法的设计或选用有着重要的指导意义和实用价值。

对于算法的复杂性，首先需要明确两个问题：

1. 怎样**表达**一个算法的复杂性；
2. 怎样**计算**一个算法的复杂性。

影响程序运行时间的因素有很多。一个运行在 Cray 巨型机上的算法，放在 PC 上肯定会慢很多；反过来，一个效率较差的算法运行在 Cray 上，也可能比一个效率极好的算法运行在 PC 上还要快许多。即使运行在同样的机器上，所采用的程序语言不同，运行时间也可能不同：采用编译型的语言往往要比解释型的语言快许多——原书举的例子是，采用 C 语言编写的程序，比用 LISP 语言编写的、采用同样算法的程序快将近 20 倍。

综上所述，评估算法的效率不能采用诸如微秒、纳秒这样的真实时间单位。对算法分析来说，最重要的不是具体的运行时间，而是**算法与输入规模之间的关系**：用一定「规模（size）」的数据作为输入时，程序运行所需的「基本操作（basic operation）」数来描述时间效率。例如，如果数据规模 n 和运行时间 t 之间存在线性关系 $t_1 = cn_1$，那么数据规模增加到原来的 5 倍后，运行时间也相应地增加同样的倍数，即 $n_2 = 5n_1$，则 $t_2 = 5t_1$。同理，如果 $t_1 = \log_2 n_1$，则规模加倍只意味着时间增加一个单位：$n_2 = 2n_1$ 时 $t_2 = \log_2(2n_1) = \log_2 n_1 + 1 = t_1 + 1$。

此处「规模」和「基本操作」的关系视具体的算法而定。「规模」一般指输入量的数目，例如在排序问题中，问题的规模即为待排序元素的个数；而完成一个「基本操作」所需的时间应该与被操作的具体数值无关，例如两个整数相加的操作时间，与这两个整数的具体取值没有关系。

1.4.1 渐进分析方法

一般情况下，用来表示数据规模和时间关系的函数都相当复杂。计算这样的函数时通常只考虑大的数据，而那些不能显著改变函数量级的部分都可以忽略掉；其结果是原函数的一个近似值，这个近似值在数据规模很大时会足够接近原值。这种方法称为渐进算法分析（asymptotic algorithm analysis）方法，简称渐进分析。例如下面这个函数：

$$f(n) = n^2 + 100n + \log_{10} n + 1000$$

在 n 的取值很小时，例如为 1 时，最后的常数项 1000 对函数值的贡献最大；

当 $n = 10$ 时，第 2 项（$100n$）和最后的 1000 具有相同的贡献；当 n 达到 100 时，前两项的贡献相同；但当 n 大于 100 后，第 2 项的贡献就小于第 1 项，而且 n 越大，第 2 项对函数值而言就越微不足道。由于第 1 项是二次增长的，当 n 取值很大时，函数的取值主要依赖于第 1 项的贡献。

渐进分析是对资源开销的一种不精确的估计，它提供的是对算法所需资源开销进行评估的简化模型，以便把注意力集中在最重要的部分。应该注意的是，**并非任何情况下都可以忽略常数部分**：当算法要解决的问题规模很小时，各项系数和常数项就会起到举足轻重的作用，如同上面的函数所示。因此，用于上万个数的排序算法，也许并不适用于仅对 10 个数的排序。

大 O 表示法。渐进分析最常用的表示方法是估计函数的增长趋势，采用由 Paul Bachmann 于 1894 年引入的大 O 表示法。设 f 和 g 为从自然数到非负实数集的两个函数。

定义 1　如果存在正数 c 和 N，使得对任意的 $n \geq N$，都有 $f(n) \leq c\,g(n)$，则称 $f(n)$ 在集合 $O(g(n))$ 中，或简称 $f(n)$ 是 $O(g(n))$ 的。

该定义说明了函数 f 和 g 之间的关系：既可以表达成函数 $g(n)$ 是函数 $f(n)$ 取值的上限（upper bound），也可以说函数 f 的增长最终至多趋同于 g 的增长。

因此，大 O 表示法提供了一种表达函数增长率上限的方法。换言之，若某种算法在 $O(g(n))$ 中，只是表明了该算法**最多会差到何种程度**。当然，一个函数增长率的上限可能不止一个：一个在集合 $O(n)$ 中的函数也一定在 $O(n^2)$ 中，同时也在 $O(n^3)$ 中。大 O 表示法给出的是所有上限中最小的那个上限。准确地说，这是**使用上的约定**，不是定义 1 的要求：按定义，说一个 $O(n)$ 的函数「是 $O(n^2)$ 的」并不算错，只是不够紧；分析算法时人们约定报出能证明的最紧（最小）的那个上限，否则这个上限就没有多少信息量。

下面列出大 O 表示法的一些有益特性，可在计算算法效率时使用（其中 a 表示不依赖于 n 的任意常数）：

1. 如果 $f(n)$ 是 $O(g(n))$ 的，$g(n)$ 是 $O(h(n))$ 的，则 $f(n)$ 是 $O(h(n))$ 的；
2. 如果 $f(n)$ 是 $O(h(n))$ 的，$g(n)$ 是 $O(h(n))$ 的，则 $f(n)+g(n)$ 是 $O(h(n))$ 的；
3. 函数 an^k 是 $O(n^k)$ 的；
4. 若 $f(n) = c\,g(n)$，则 $f(n)$ 是 $O(g(n))$ 的；
5. 对于任何正数 a 和 b（$a \neq 1$，$b \neq 1$；原书只写了 $b \neq 1$，但底数为 1 的对数没有定义），函数 $\log_a n$ 是 $O(\log_b n)$ 的——即任何对数函数无论底数为何值，都具有相同的增长率；
6. 对任何正数 $a \neq 1$，都有 $\log_a n$ 是 $O(\log_2 n)$ 的。本书把 $\log_2 n$ 简写为 $\log n$。

常见的上限 $g(n)$ 有以下若干种（$\text{rate}_{n\to\infty} f(n)$ 表示 n 趋于无限大时函数 $f(n)$ 的增长率）：

$g(n)$	名称	原书给的例子
1	常数函数	不依赖于数据规模 n
$\log n$	对数函数	比线性函数 n 增长慢
n	线性增长	$\text{rate}(n\text{ 个}a\text{ 相加}) = \text{rate}(n \times a) = O(n)$
$n \log n$	阶数低于二阶、高于一阶	$\text{rate} \sum_{i=1}^{\log n} \sum_{j=1}^{n} a = O(n \log n)$，一般出现在树形结构的算法中
n^2	二阶增长	$\text{rate}(1 + 2 + \cdots + n) = \text{rate}(n(n+1)/2) = O(n^2)$
a^n	指数增长	往往出现在递归定义的函数计算中

值得注意的是，指数增长的渐进式比任何高次的多项式函数（如 n^3、n^4）都增长得更快。

这些渐进式的函数曲线，在 n 很大的时候其大小差别非常大。令时间单位为 $1\,\mu$s，在 n 等于 1000 时：线性增长率 $T(n) = n$ 为 $1000\,\mu$s，即 1 ms；二次增长率 $T(n) = n^2$ 的时间开销是 $10^6\,\mu$s，即 1 s；三次增长率 $T(n) = n^3$ 的时间开销就是 1000 s（约 16 分钟）；而指数型的增长率 $T(n) = 2^n$ 所花费的时间约为 10^{286} 年，迄今地球的年龄还不超过 10^{10} 年。（细算：$2^{1000} \approx 1.07 \times 10^{301}$ μs $\approx 1.07 \times 10^{295}$ s，一年约 3.16×10^7 s，所以是约 3.4×10^{287} 年——原书的 10^{286} 少算了一个多数量级，但「天文数字」这个结论不受影响。）具有指数增长率的算法简称指数爆炸型算法，应用时需要谨慎。

原书用表 1.2 把这些量级换算成现实世界里的速度，好让「差一个数量级」有一个能感知的尺度：

表 1.2　各个量级的实例

m/s	英制度量衡	现实世界的例子
10^{-11}	1.2 英寸/世纪	钟乳石的生长速度
10^{-10}	1.2 英寸/十年	大陆板块的漂移速度
10^{-9}	1.2 英寸/年	指甲的生长速度
10^{-8}	1 英尺/年	头发的生长速度
10^{-7}	1 英尺/月	杂草的生长速度
10^{-6}	3.4 英寸/天	冰河的流动速度
10^{-5}	1.4 英寸/时	表的分针转动速度
10^{-4}	1.2 英尺/时	胃肠的蠕动速度

m/s	英制度量衡	现实世界的例子
10^{-3}	2 英寸/分	蜗牛的速度
10^{-2}	2 英尺/分	蚂蚁的速度
10^{-1}	20 英尺/分	巨龟的速度
1	2.2 英里/时	人类的散步速度
10^{1}	22 英里/时	人类疾跑速度
10^{2}	220 英里/时	螺旋桨飞机的速度
10^{3}	37 英里/分	喷气式飞机的速度
10^{4}	370 英里/分	宇宙飞船的速度
10^{5}	3700 英里/分	流星撞击地球的速度
10^{6}	620 英里/秒	地球自转速度
10^{7}	6200 英里/秒	卫星从洛杉矶到纽约的速度
10^{8}	62000 英里/秒	光速的三分之一

表里个别「现实世界的例子」与速度对不上，读时只取量级之意：地球赤道的自转线速度约 465 m/s，介于 10^2 与 10^3 之间，远不到 10^6；近地卫星的速度约 7.9×10^3 m/s，也不是 10^7。各行的米制与英制换算本身是对得上的。

图 1.5 把常用函数的增长趋势画在一起——n 稍大，曲线之间就拉开天文数字的差距：

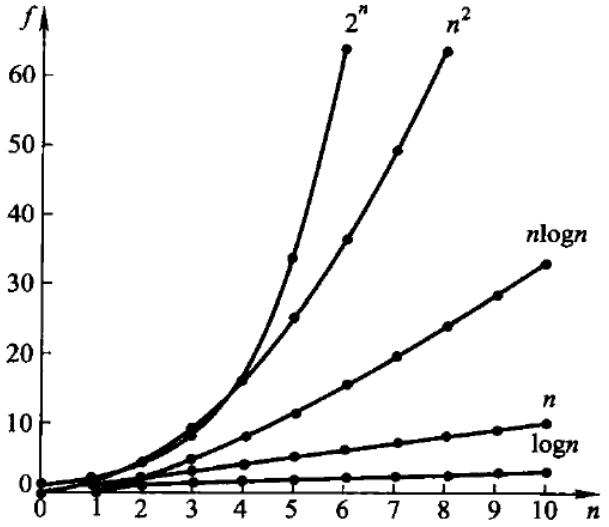


图 1.5　常用函数的增长趋势。

同一组 n 用数字排出来是这样：

阶	$n=16$	$n=256$	直觉
1	1	1	常数
$\log n$	4	8	二分
n	16	256	扫一遍
$n\log n$	64	2048	快排 / 归并
n^2	256	65536	双重循环
2^n	65536	—	穷举子集

Ω 表示法。大 O 表示法给出了函数增值率的上限。与此相对，还有 Ω 表示法（读做「欧米伽」）：

定义 2　如果存在正数 c 和 N，使得对所有的 $n \geq N$，都有 $f(n) \geq c\,g(n)$，则称 $f(n)$ 在集合 $\Omega(g(n))$ 中，或简称 $f(n)$ 是 $\Omega(g(n))$ 的。

此定义说明了 $c\,g(n)$ 是函数 $f(n)$ 取值的下限（lower bound），也可以说函数 $f(n)$ 的增长最终至少是趋同于函数 $g(n)$ 的增长。大 O 表示法和 Ω 表示法的唯一区别在于不等式的方向不同；正如大 O 取的是所有上限中最小的那个，Ω 取的是所有下限中**最大**的那个——同样是使用约定：按定义 2，n^2 也是 $\Omega(n)$ 的，只是这个下限太松。

Θ 表示法。大 O 和 Ω 描述了某一函数增长的上限和下限。当上、下限相同时，则可用 Θ（读做「希塔」）表示法：如果一个函数既在集合 $O(g(n))$ 中，又在集合 $\Omega(g(n))$ 中，则称其为 $\Theta(g(n))$。即存在正常数 c_1、c_2 以及正整数 N，使得对于任意的正整数 $n > N$，下列两不等式同时成立：

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

此定义具体给出了下限估计和上限估计两种估计式。例如 $f(n) = 100 \times n^2 + 5 \times n + 500$，令 $g(n) = n^2$，此时存在常数 $c_1 = 100$、$c_2 = 105$、$N = 10$，当 $n > N$ 时上式成立，因此可以说 $f(n)$ 为 $\Theta(n^2)$。

渐进分析的实例。通过对算法所需的时间和空间代价的估算，渐进分析方法可以衡量算法的复杂度。实际上，大多数情况下，算法的时间复杂度是根据算法执行过程中需要实施的赋值、比较等基本运算数目的多少来衡量的。

先从一个简单的例子开始分析。下面是一段对数组中的各个元素求和的代码：

```
for (i = sum = 0; i < n; i++)
    sum += a[i];
```

其中主要的操作为赋值运算，因此该算法的时间代价主要体现在赋值操作的数目上。在循环开始之前有两次赋值，分别对 `i` 和 `sum` 进行；循环进行了 n 次，每次循环中执行两次赋值，分别对 `sum` 和 `i` 进行更新操作。总共有 $2 + 2n$ 次赋

值操作，其渐进复杂度为 $O(n)$。

在循环中嵌套循环，其复杂度会相应增大。例如下面这段代码依次求出给定数组的所有子数组中各元素之和：

```
for (i = 0; i < n; i++) {
    for (j = 1, sum = a[0]; j <= i; j++)
        sum += a[j];
    cout << "sum for subarray 0 through " << i << " is " << sum << endl;
}
```

输出那一行原书印作 `"sum for subarray 0 through" + i + " is " + sum`。在 C++ 里字符串字面量加整数是**指针算术**（把字符指针往后挪 `i` 位），不是拼接，所以这里按本意改用 `<<` 逐项输出；它不影响下面的赋值计数。

循环开始前有一次对 `i` 的赋值操作。之后，外层循环共进行 n 次，每个循环中包含一个内层循环以及对 `i`、`j`、`sum` 分别进行赋值的操作；每个内层循环执行 2 个赋值操作（分别更新 `sum` 和 `j`），共执行 i 次（$i = 1, 2, \cdots, n-1$）。因此，整个程序总共执行的赋值操作为

$$1+3n+\sum_{i=1}^{n-1} 2i = 1+3n+2(1+2+\cdots+n-1) = 1+3n+n(n-1) = O(n)+O(n^2) = O$$

一般情况下，循环中嵌套循环会增加算法的复杂度，但也并非总是如此。例如在上面的实例中，如果只对每个子数组的前 5 个元素求和，则相应的代码可采用下面的方式（看代码，求的其实是每个子数组 `a[0..i]` **末尾**的 5 个元素 `a[i-4]`…`a[i]` 之和；「前」字容易读偏，以代码为准）：

```
for (i = 4; i < n; i++)
    for (j = i - 3, sum = a[i - 4]; j <= i; j++)
        sum += a[j];
```

此时，外层循环进行 $n-4$ 次。对每个 i 而言，内层循环只执行 4 次，每次的操作次数与 i 的大小无关：8 次赋值操作。外加对 `i` 的初始化，整个代码总共进行 $O(1) + 8(n-4) = O(n)$ 次赋值操作。（若按上一个例子的口径把外层每轮对 `i`、`j`、`sum` 的 3 次赋值也算上，精确值是 $1 + 11(n-4)$；原书只数了内层的 8 次。常数不同，结论一样是 $O(n)$。）**尽管存在嵌套循环，但算法的整体时间复杂度依然呈线性增长**。

另外，对于排序算法而言，其主要时间开销体现在比较和交换（或移动）等操作上，具体可参考第 8 章的各算法分析。

1.4.2 最佳、最差和平均情况

对于某些算法，即使问题规模相同，如果输入数据不同，其时间复杂度也不同。换言之，算法的渐进分析往往无法独立于输入数据的状态而进行。这是因为算法实际执行的操作往往依赖于算法中分支条件的走向，而这些分支走向又取决于输入数据的取值。为此，对算法进行渐进分析时会根据输入数据的取值分情况进行，以便确切了解各种算法所适用的情况，以及能否在规定的响应时间内完成。

例如,求一个数组的所有有序子数组中最长的一个。在数组 $[1,8,1,2,5,0,11,9]$ 中，这个最长的有序子数组为 $[1,2,5]$，长度为 3。可用以下代码实现：

```
for (i = 0, length = 1; i < n - 1; i++) {
    for (j1 = j2 = k = i; k < n - 1 && a[k] < a[k + 1]; k++, j2++);
    if (length < j2 - j1 - 1)
        length = j2 - j1 + 1;
}
```

上面这段是原书印出来的样子，**照抄它会算错**：内层循环走完后这一段升序的长度是 `j2 - j1 + 1`，而判断条件写的却是 `length < j2 - j1 - 1`，两处的常数项差了 2。用原书自己的例子 $[1,8,1,2,5,0,11,9]$ 跑一遍，答案会是 1 而不是 3：长为 L 的升序段要通过判断，得满足 `length < L - 2`，初值 `length = 1` 时只有 $L \geq 4$ 的段才进得去，而这个数组里最长的段只有 3。这里保留原样是为了让读者对照纸书，分析长度时按 `j2 - j1 + 1` 读。

这段代码的时间代价和数组 `a` 中元素的实际取值状态相关。根据这些元素的初始状态可以分成以下几种情况：

1. 如果数组 `a` 的所有元素是以**降序**方式输入的，那么外层循环执行 $n-1$ 次，每次内层循环只执行 1 次，整个时间开销为 $O(n)$。
2. 如果数组 `a` 的所有元素是以**升序**方式输入的，那么外层循环执行 $n-1$ 次，对每个 $i\,(i=0,1,\cdots,n-2$；原书写到 $n-1$，但外层条件是 `i < n - 1`)，内层循环需要执行 $(n-1-i)$ 次，合计 $(n-1)+(n-2)+\cdots+1=n(n-1)/2$ 次，整个时间开销为 $O(n^2)$。
3. 在大多数情况（即**平均情况**）下，数组的元素是无序的，既不按升序也不按降序输入。

一般而言，计算平均情况的复杂度应该考虑算法的所有输入情况，确定针对每种输入情况算法所需的操作数目。在简单情况下，如果每种输入出现的概率相同，可把针对每种输入的操作数目依次相加，再除以输入的总数目，来得到平均的开销。但是，每种输入的出现概率并非总是相同，此时分析平均情况的复杂度就需要把每种输入出现的概率作为权值加以考虑，即

$$C_{\text{avg}} = \sum_i p(\text{input}_i)\,\text{steps}(\text{input}_i)$$

此处假设可以事先得知输入的概率分布情况。$p(\text{input}_i)$ 为第 i 种输入的出

现概率，$\text{steps}(\text{input}_i)$ 为算法处理第 i 种输入时所需的操作或步骤数。所有的概率均为非负数，且满足 $\sum_i p(\text{input}_i) = 1$。

尽管平均情况的复杂度是算法在输入规模为 n 时的典型表现，但平均情况的分析并不总是可行的，因为这需要了解算法的实际输入在所有可能的输入集合中的分布状况。例如上面例子中获取数组元素的平均分布情况就并不很容易。

以从一个规模为 n 的一维数组中找出一个给定的 K 值为例（假设该数组中有且仅有一个元素的值为 K），顺序检索法将从第一个元素开始，依次检查每一个元素，直到找到 K 为止。

- **最佳情况**下，数组中第 1 个元素就是 K，此时只要检查一个元素即可。
- **最差情况**下，K 是数组的最后一个元素，此时算法需要检查数组中所有的 n 个元素才能找到。
- **平均情况**下，如果 K 出现在数组中每个位置上的概率相等，即出现在每个单元中的概率均为 $1/n$，平均需要 $\frac{1+2+\cdots+n}{n} = \frac{n+1}{2}$ 次比较才能找到。

概率不相等时，结论会明显改变。例如出现在第 1 个位置的概率为 $1/2$，第 2 个位置上的概率为 $1/4$，而出现在其他位置的概率相等，即为 $\frac{1-1/2-1/4}{n-2} = \frac{1}{4(n-2)}$，则查找 K 平均需要

$$\frac{1}{2} + \frac{2}{4} + \frac{3+\cdots+n}{4(n-2)} = 1 + \frac{n(n+1)-6}{8(n-2)} = 1 + \frac{n+3}{8}$$

次比较，比等概率的 $\frac{n+1}{2}$ 快将近 4 倍。

由此可见，第 1 种情况是最佳的，所需的步骤最少；第 2 种情况所需的步骤最多，是最差的情形；第 3 种则介于最佳和最差之间。那么分析一种算法时，应该研究最佳、最差还是平均情况？

一般而言，**最佳情况发生的概率太小**，并不能作为算法性能的代表，但它可以帮助算法设计者或使用者了解某个算法在何种情况下适用。而**最差情况可以让人了解一个算法至少能做多快**，这一点在实时系统中尤其重要：例如在空运处理系统中，一个绝大部分情况下能管理 n 架飞机的算法，如果不能在规定时间内管理 n 架飞机，则该算法是不可接受的。此外，对于多数算法而言，最坏情况和平均情况的时间开销公式虽然不同，但往往只是常数因子大小的区别，或者常数项大小的区别。

上面 3 种情况的复杂度都相对比较简单，可以得到很精确的函数关系，但复杂度与数据规模之间的关系并非总是一目了然。尤其是平均情况的复杂性分析往往需要复杂的计算，此时前面介绍的大 O、Ω 和 Θ 等渐进分析法便可以派上用场。

只报一个数字、不说针对哪种输入，这个数字就没有意义。本书后面每讲一个算法，都会分别交代它的最佳、最差与平均情况——例如第 8 章的快速排序平均是 $O(n \log n)$，而在取末元素作轴值（本书的写法）时，输入已经有序就退化成 $O(n^2)$。

1.4.3 时间和空间的折衷

对于空间开销，也可以采用类似的渐进分析方法。但是，很多常见算法所使用的数据结构是**静态的存储结构**：所谓静态，是指算法所使用的存储空间在算法执行过程中并不发生变化。一旦确定了输入数据和问题规模，其数据结构大小也就确定下来。对于这种静态数据结构，空间开销的估算比较容易，往往与所涉及的问题规模成正比（空间开销为线性增长），或者不随问题的规模而增大（空间开销为常数）。本书后续章节对这类使用静态数据结构的算法，一般将仅限于讨论其时间代价。

当然，也经常会遇到一些使用**动态数据结构**的算法，它们的存储空间是变化的，在算法运行过程中有时会有数量级的增大或缩小。对于这种情况，空间开销的分析和估计是十分必要的。

在算法设计分析中，还涉及一个「时空资源的折衷原理」。这个原理可以简述如下：对于同一个问题求解，一般会存在多种算法，而这些算法在时间和空间开销上的优劣往往表现出「时空折衷」的性质。所谓时空折衷是指，为了改善一个算法的时间开销，往往可以增大空间开销为代价；反之亦然。

例如，为了从公司黄页数据表中快速查询公司电话（假设每个公司的名称、地址和电话都存储在该数据表中），可以附加一个散列式索引表，其散列函数可以将公司名称快速变换为索引值，通过这个索引值可以读出一个指针，指向存储该公司电话的存储单元。这种散列法附加了索引表的空间开销，但节省了时间：用散列函数的简单计算代替了原本耗时的在黄页中逐项进行公司名称的比较操作，时间开销从线性增长改善为常数时间，而空间开销方面则增添了一个线性增长率的存储空间。在设计算法时经常采用这种以空间开销换取时间开销的办法；为此，就需要对原有存储结构做出修改，或者想出全新的、更适合逻辑数据结构使用的存储方案。

当然，有时也可以牺牲计算机的运行时间，通过增大时间开销来换取存储空间的节省，例如树的顺序存储。第 6.3 节把一棵树压成一串「带右链」「带度数」的序列，省下了大部分乃至全部指针域的空间（带右链的表示只留一条右链，带度数的表示一个指针也不存），代价是恢复结构时必须重新扫描一遍；第 12 章的稀疏矩阵、第 5 章的 Huffman 编码也都是同一个取舍的不同侧面。**没有免费的加速：空间紧、时间松，和空间松、时间紧，选的结构会不一样**。

1.4.4 求解问题时数据结构的选择和评价

如同 1.1 节所述，利用计算机求解一个给定问题时，一个关键任务是确立问题模型所采用的数据结构。一旦确定了数据结构，算法的设计便水到渠成。

根据应用的具体情况，设计或选择数据结构可以遵循以下一些原则：

1. 仔细分析所要解决的问题，特别是求解问题所涉及的数据类型和数据间的逻辑关系。
2. 在算法设计之前往往先进行数据结构的初步设计。
3. 注意数据结构的**可扩展性**，包括考虑当输入数据的规模发生改变时，数据结

构是否能够适应。同时，数据结构应该适应求解问题的演变和扩展。

4. 数据结构的设计和选择也要比较算法的时空开销的优劣。

数据结构的选择和评价是复杂的，需要考虑的因素也不限于上述几条，一般要具体情况具体分析。本书会针对具体的问题进一步阐述和检验这些原则。

落到本章的例子上：1.1 节要的是任意两点之间的最短路径，并且只有五个经纪人，所以相邻矩阵加 Floyd 是合适的——实现简单，三重循环在 $n = 5$ 时可以忽略。如果顶点很多、边很少，而且只问单源最短路，就应该换成邻接表加 Dijkstra（第 7 章）。再往下，还要看数据在内存还是外存：一次磁盘 I/O 往往比一次比较贵几个数量级，这是第 9 章与第 11 章反复出现的分界线。后面每一章都会再做一次这样的取舍。

本章小结

本章介绍了「数据结构与算法」以及本书要用到的一些预备知识和相应的设计原则：从求解问题的角度 开始，引入数据结构的基本概念、算法的概念和基本性质、算法设计的基本方法，以及衡量算法效率的 方法。

利用计算机求解问题的过程，是一个问题抽象、数据抽象和算法抽象的过程。首先从分析问题的要求、明确需求开始；然后对问题进行分析和抽象，建立问题的抽象模型；在此基础上确定该模型所采用的数据 结构，设计求解该问题的算法；最后选定某种程序设计语言来编码实现。

数据结构描述的是按照一定逻辑关系组织起来的待处理数据元素的表示及相关操作，涉及数据的逻辑 结构、数据的存储结构和数据的运算。

数据的逻辑结构是从具体问题抽象出来的数学模型，反映了事物的组成结构及事物之间的逻辑关系。数据结构重点研究的是数据间的结构关系，组成结构的所有元素都看做结点；结点是数据结构中数据的 基本单位，其数据类型既可以是基本数据类型，也可以是复合数据类型。根据结点间关系 r 所刻画的 性质，可把逻辑结构分为**线性结构、树形结构和图结构**三大类。

数据的存储结构所要解决的问题，是各种逻辑结构在计算机中的物理存储表示，即从逻辑结构到存储 结构的一种映射。同一种逻辑结构可以采用不同的映射关系来建立到存储结构的转换，从而形成不同的 存储表示方式；常用的基本存储映射方法有**顺序方法、链接方法、索引方法和散列方法**。

作为描述数据结构的一种理论根据，**抽象数据类型 ADT 可以看做是定义了一组操作的一个抽象模型**，在很大程度上可以帮助人们独立于程序的实现细节来理解数据结构的主要性质和约束条件，从而达到数据 抽象、信息隐蔽的目的。

算法是对特定问题求解过程的描述，是指令的有限序列。程序是算法的一种实现，计算机按照程序逐步 执行算法，实现对问题的求解。算法应具备通用性、有效性、确定性、有穷性等基本性质。

算法设计与算法分析是计算机科学的核心问题。算法设计的任务是对各类具体问题设计良好的算法，并研究设计算法的规律和方法，常用的方法有穷举法、回溯法、分治法、递归法、贪心法和动态规划法等；算法分析的任务是利用数学

工具，对每一个具体算法讨论其各种复杂度，以探讨各种具体算法的适用场景，目的在于从解决同一问题的不同算法中选择出比较合适的一种，或者对原有的算法进行改造、加工使其更优。分析不看墙上的秒数，而看基本操作次数随规模 n 怎么长，用大 O、Ω、Θ 说话，并分清最好、最坏和平均。

针对同一个问题一般会存在多种算法，这些算法在时间和空间开销上的优劣往往表现出「时空折衷」的性质，需要根据应用的具体情况，在设计或选择数据结构时进行适当的权衡。

习题

补充证明练习（参考课程第 1 章）

1. 精确计算下列双重循环中赋值语句 `a[i][j] = 1` 的执行次数，再给出数量级：`for (i=1; i<n; ++i) for (j=n; j>=i; --j) a[i][j]=1;`
2. 对非负渐进函数证明 `max(f(n), g(n)) = Theta(f(n)+g(n))`。
3. 求解并用数学归纳法证明 `T(n)=2T(floor(n/2))+n` 的渐进阶。

正文习题

1. 设计一个算法，从大到小依次输出顺序读入的三个整数 X、Y、Z。
2. 举例说明什么是数据结构。
3. 数据有哪些存储方式？各有什么特点。
4. 分析下面程序段中循环体执行多少次。

```
int i = 0, s = 0, n = 100;
do {
    i = i + 1;
    s = s + 10 * i;
} while ((i < n) && (s < n));
```

5. 指出算法 A 的功能和时间复杂度。h、g 是同一个单循环链表上的两个结点指针。

    text original-listing=" 原书习题给出的 A/B 函数待学生分析，不属于本书可运行实现" void B(Node* s, Node* q) { Node* p = s; while (p->next != q) p = p->next; p->next = s; } void A(Node* h, Node* g) { B(h, g); B(g, h); }

6. 设 n 是偶数，计算下列程序段结束后 m 的值，并给出时间复杂度。

```
m = 0;
for (i = 1; i <= n; i++)
    for (j = 2 * i; j <= n; j++)
        m = m + 1;
```

7. 把下列运行时间写成大 O：$T_1(n) = 1000, T_2(n) = n^2 + 1000n, T_3(n) =$

$3n^3 + 100n^2 + n + 1$。

8. 给出下面两个算法的时间复杂度：(1) 一层循环 $i = 1..n$ 做 $x = x + 1$；(2) 两层循环 $i, j = 1..n$ 做 $x = x + 1$。
9. 给出求两个整数最大公因数的算法，并分析时间和空间复杂度。
10. 计算机每秒 10^{10} 次操作，要求一小时内算完。下列复杂度各能处理多大的 n：n^2，n^3，$100n^2$，$n\log_{10} n$，2^n，2^{2^n}。
11. 已知 $f(n)$ 是 $O(g(n))$。判断并证明或举反例：(1) $\log_2 f(n)$ 是 $O(\log_2 g(n))$；(2) $2^{f(n)}$ 是 $O(2^{g(n)})$；(3) $f(n)^2$ 是 $O(g(n)^2)$。
12. 计算下列递推的复杂度：(1) $T(n) = T(\lceil n/2 \rceil) + 1$；(2) $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$；(3) $T(n) = 2T(\lfloor n/2 \rfloor) + n$。

上机题

1. m、n 是自然数。m 可以写成若干个不超过 n 的自然数之和，编写 $f(m, n)$ 计算有多少种写法。例如 $f(5, 3) = 5$：$3 + 2$，$3 + 1 + 1$，$2 + 2 + 1$，$2 + 1 + 1 + 1$，$1 + 1 + 1 + 1 + 1$。
2. 试用 C++ 的类声明定义「复数」的抽象数据类型。要求：
    1. 在复数内部用浮点数定义其实部和虚部；
    2. 实现 3 个构造函数——默认的构造函数没有参数；第二个构造函数将双精度浮点数赋给复数的实部、虚部置为 0；第三个构造函数将两个双精度浮点数分别赋给复数的实部和虚部；
    3. 定义获取和修改复数的实部和虚部，以及 +、-、*、/ 等运算的成员函数；
    4. 定义重载的流函数来输出一个复数。

第 2 章 线性表

线性表把一串同类型元素排成唯一的先后次序。顺序表把元素连续放在数组里，能 O(1) 按下标访问；链表用链接保存相邻关系，能在已知前驱结点时 O(1) 插入或删除。关键是区分「按位置找元素」和「已知结点后改链接」这两类操作。

源码：顺序表·教学版、顺序表·工程版、顺序表示例、链表·教学版、链表·工程版、链表示例、双链表·教学版、双链表·工程版。

先跑一遍

```
// 第 2 章「先跑一遍」：用教学版 ArrayList 走一遍 append / insert / find / remove。
// 编译运行：
//   g++ -std=c++17 -I code/ch02/array_list code/ch02/array_list/demo.cpp -o demo &&
↪  ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    ArrayList<int> values;
    values.append(10);
    values.append(30);
    values.insert(1, 20);

    std::cout << " 顺序表:";
    for (int value : values) {   // 有 begin()/end()，range-for 直接可用
        std::cout << ' ' << value;
    }

    // find 返回 optional：有值才解引用
    if (auto pos = values.find(20)) {
        std::cout << "\n查找 20 的下标: " << *pos << '\n';
    }

    std::cout << " 删除位置 1 得到 " << values.remove(1) << "，剩余:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch02/array_list \
    code/ch02/array_list/demo.cpp -o /tmp/list-demo
```

```
/tmp/list-demo
```

```
顺序表: 10 20 30
查找 20 的下标: 1
删除位置 1 得到 20，剩余: 10 30
```

`insert(1, 20)` 要把后面的元素右移一位，这是顺序表的固有代价。链表同一组操作只改两条链接，但按位置找前驱仍是 O(n)：

```
// 第 2 章「先跑一遍」：用教学版 LinkedList 走一遍 append / insert / remove。
// 编译运行：
//   g++ -std=c++17 -I code/ch02/linked_list code/ch02/linked_list/demo.cpp -o demo
↪  && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    LinkedList<int> values;
    values.append(10);
    values.append(30);
    values.insert(1, 20);

    std::cout << " 链表:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << "\n删除位置 0 得到 " << values.remove(0) << "，剩余:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << "\nappend 之后尾元素是 " << values.at(values.size() - 1) << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch02/linked_list \
    code/ch02/linked_list/demo.cpp -o /tmp/link-demo
/tmp/link-demo
```

```
链表: 10 20 30
删除位置 0 得到 10，剩余: 20 30
append 之后尾元素是 30
```

append 经尾指针 O(1) 接链，不必再从头走到尾。

2.1 线性表的概念

线性表（linear list）是由称为元素（element）的数据项组成的一种有限且有序的序列，这些元素也可称结点或表目。用第 1 章的二元组 $B=(K,R)$ 写出来就是

$$K=\{k_0,k_1,\cdots,k_{n-1}\},\quad R=\{r\},\quad r=\{\langle k_i,k_{i+1}\rangle,0\le i\le n-2\}$$

结点集合 K 中的元素个数 n 称为线性表的**长度**：$n=0$ 时称空表，$n>0$ 时称非空表；k_0 称开始结点或**表首**，k_{n-1} 称终止结点或**表尾**。线性关系 r 刻画元素间的前驱、后继关系——k_i 是 k_{i+1} 的前驱，k_{i+1} 是 k_i 的后继，所以线性关系也称前驱关系或 后继关系。**相邻关系 r 具有反对称性**；它本身只含相邻有序对，例如 $\langle k_0,k_1\rangle$、$\langle k_1,k_2\rangle\in r$ 而 $\langle k_0,k_2\rangle\notin r$，因此严格说并不传递——具有传递性的是由它导出的「排在前面」关系，即 r 的传递闭包。由定义可知线性表的逻辑特征是：K 中的每个结点在关系 r 上最多只有一个前驱结点和一个后继结点。

线性结构有两个结构特点。**均匀性**：虽然不同线性表的数据元素可以各种各样，但同一线性表中 的各数据元素必定具有相同的数据类型和长度。**有序性**：各数据元素在表中都有自己的位置，元素之间的相对位置是线性的。本章讨论的简单线性表因此约定所有元素同类型——概念上并不反对 元素类型不同（例如第 12 章的广义表），但那属于高级线性结构。

同一种线性结构在不同场合有不同的称谓：顺序表、链表、串、栈、顺序文件；其中的元素也相应地 叫表目、结点或记录。**严格说它们不是同义词**：顺序表与链表首先指两种存储方式，串、栈、顺序 文件则各自带有不同的元素类型、运算限制或存储介质；这里沿用原书的说法，只强调它们都属于「线性结构」这一族。

在线性表上可以实施的运算依赖于具体应用，但一般不外乎两大类：一类是**对整个表**的操作—— 创建或置空一个线性表、合并两个线性表、判断表是否为空或为满；另一类是**对表中元素**的操作 ——查找满足一定条件的元素、在表中插入或删除指定的元素。

按照特性，线性表的运算还可以细分为 5 类：① 创建线性表的一个实例；② 析构，消除实例并释放 所占空间；③ 获取当前线性表的信息（由内容寻找位置、由位置读取元素内容），不改变表的内容；④ 访问并改变表的内容或结构，例如更新指定元素、添加元素、删除元素、清空线性表；⑤ 辅助管理操作，例如求表的当前长度。

原书【代码 2.1】用一个类模板 `List<T>` 来写这组运算。那份清单有两处今天必须改：

1. 它声明了 `bool delete(const int p);`——`delete` **是 C++ 关键字**，不能作函数名。整章的删除操作都建立在这个编译不过的名字上。

2. 它写的是 `class List { void clear(); ... };`，**通篇没有** `public:`。`class` 的默认访问权限是 private，于是这个抽象数据类型的每一个运算都调不到。（同一本书第 3 章的代码 3.1 是写了 `public:` 的。）

第 2 点尤其值得停一下：抽象数据类型的意义就是**对外**给出一组运算。一个所有运算都私有的 ADT，在语法上成立、在语义上是空的。

原书在【代码 2.1】之外还给出一组「当前位置（position）」运算：`setPos` 设置当前下标，`setStart` / `setEnd` 把当前下标移到表头或表尾，`prev` / `next` 让它左右移动一位；配合 `getValue` 就能依次处理每个元素。本书用迭代器（`begin()` / `end()` 加 range-for）达到同一 目的——「先跑一遍」那段 `for (int value : values)` 就是原书那段 `setStart` / `next` 循环的现代写法，好处是不必在容器里额外存一个「当前位置」状态。

线性表运算的具体实现与它在计算机中的物理存储结构密切相关，效率也与存储结构相关。线性表的存储结构本质上是**逻辑结构到存储空间的映射**：不仅要为结点集合到存储器单元建立 一个映射，同时还要为元素之间的线性关系到相应存储单元地址间的关系建立映射。主要有两类：

1. **定长的顺序存储结构**，简称顺序表。程序中通过创建数组来建立，为线性表分配一块连续的 存储空间，元素顺序地存储在这些地址连续的空间中，以「物理位置相邻」来表示元素之间的关系。定长存储结构的不足之处是限制了线性表的长度变化。见 2.2 节。
2. **变长的线性存储结构**，也称链接式存储结构，简称链表。它使用指针来表示元素之间的线性 关系，利用前驱和后继关系将各个元素用指针链接起来；对表长不加限制，需要加入新元素时可以 方便地申请空间并链接到合适的位置上。见 2.3 节。

本书把这组运算直接定义在 `ArrayList<T>` 上，不再另设一个基类—— 理由与第 3 章相同：那样的空基类给不了多态，却会带来非虚析构的未定义行为。要定下来的是**这张表**：

运算	含义	时间代价
`at(i)` / `set(i, x)`	按下标取值、改值	O(1)
`find(x)`	按内容查位置；找不到返回「没有」	O(n)
`insert(i, x)`	在位置 i 插入	O(n)
`append(x)`	在表尾追加	摊还 O(1)
`remove(i)`	删除位置 i 上的元素并把它带回来	O(n)
`size()` / `empty()` / `clear()`	长度、判空、置空	O(1)

2.2 顺序表

为什么这一节没有 Python 版

这里故意只给 C++。顺序表的重点不是 “把几个值放进一个序列”，而是容量、对象生命周期和扩容失败时旧数组是否仍然有效。若直接写成 Python `list`，扩容、元素搬迁和析构都由解释器隐藏；读者得到的是一个可用的容器，却看不到三法则、移动语义和强异常保证为何存在。算法侧可以把同一思想用 Python 重讲，存储侧若也用 `list`，反而会把本节要教的课删掉。

按顺序方式存储的线性表称为顺序表 (array-based list)，又称向量 (vector)，通过数组建立。

假设每个元素占用 L 个存储单元，顺序表的开始结点 k0 的存储位置记为 b = loc(k0)，称为首地址；则下标为 i 的元素 ki 的存储位置为

$$\mathrm{loc}(k_i) = b + i \times L$$

每个元素的存储位置都与起始位置相差一个与位序成正比的常数。只要确定了基地址，表中任一元素的地址都能直接算出——**顺序表因此是一种随机存取的存储结构**，按下标取值的时间代价为 O(1)。物理相邻表示了逻辑相邻。

(a) **线性表的逻辑结构**

数据元素	k0	k1	…	ki	…	kn-1	…	
逻辑地址	0	1	…	i	…	$n-1$	…	$\mathrm{maxSize}-1$

(b) **线性表的顺序存储结构**

数据元素	k0	k1	…	ki	…	kn-1	…
存储地址	b	b+L	…	b+i·L	…	b+(n − 1)·L	…

图 2.1 顺序表的示意图

教学版：完整实现

下面是一份**完整的、能直接编译运行的**顺序表。一个文件、一个类，没有省略号。它保留原书【代码 2.2】【算法 2.3】【算法 2.4】【算法 2.5】要教的全部内容—— 连续存储、按下标 O(1) 随机存取、插入/删除要搬 O(n) 个元素——只把原书那几处编译不过或会崩的写法换掉。后面 2.2.1、2.2.2 两节就是把它拆开逐段讲——检索、插入、删除都在 2.2.2 里。

```
// 顺序表 ArrayList ——教学版。
//
```

```cpp
// 一个文件、一个类、能直接编译运行，给「第一次读这一节」的人看。
// 它保留原书【代码 2.1】【代码 2.2】【算法 2.3】【算法 2.4】【算法 2.5】要教的全部内容——
// 连续存储、按下标 O(1) 随机存取、插入/删除要搬 O(n) 个元素——
// 只把原书那几处编译不过或会崩的写法换掉。
//
// 与 modern.hpp（工程版）的分工：
//   教学版  遵守 ** 三法则 **（析构 + 拷贝构造 + 拷贝赋值），正确，但拷贝多一点；
//   工程版  在此之上补齐移动语义、强异常保证、编译期类型约束。
// 两份都在闸门里真编译真运行。先读这一份，2.2a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <optional>
#include <stdexcept>

template <typename T>
class ArrayList {
public:
    using value_type = T;
    using size_type = std::size_t;

    explicit ArrayList(size_type initial_capacity = 8)
        : data_(new T[initial_capacity]), capacity_(initial_capacity), size_(0) {}

    ~ArrayList() { delete[] data_; }

    // 三法则：自己管着 new 出来的数组，就得自己写拷贝构造和拷贝赋值。
    // 不写的话编译器会照抄指针，两个表指向同一块内存，各析构一次 → 二次释放。
    ArrayList(const ArrayList& other)
        : data_(new T[other.capacity_]), capacity_(other.capacity_),
        ↪  size_(other.size_) {
        for (size_type i = 0; i < size_; ++i) {
            data_[i] = other.data_[i];
        }
    }

    ArrayList& operator=(const ArrayList& other) {
        if (this == &other) {
            return *this;
        }
        T* fresh = new T[other.capacity_];
        for (size_type i = 0; i < other.size_; ++i) {
            fresh[i] = other.data_[i];
        }
        delete[] data_;
        data_ = fresh;
        capacity_ = other.capacity_;
```

```
        size_ = other.size_;
        return *this;
    }

    bool empty() const { return size_ == 0; }
    size_type size() const { return size_; }
    size_type capacity() const { return capacity_; }

    // 清空只把长度归零，已经申请的数组留着复用。
    void clear() { size_ = 0; }

    // 按下标读取，O(1)——这是顺序表相对链表的看家本领。
    // 下标非法是 ** 调用方的错误 **，不是可预期状态，所以抛异常而不是返回 optional。
    const T& at(size_type index) const {
        if (index >= size_) {
            throw std::out_of_range("ArrayList::at: 下标越界");
        }
        return data_[index];
    }

    T& at(size_type index) {
        if (index >= size_) {
            throw std::out_of_range("ArrayList::at: 下标越界");
        }
        return data_[index];
    }

    void set(size_type index, const T& value) {
        if (index >= size_) {
            throw std::out_of_range("ArrayList::set: 下标越界");
        }
        data_[index] = value;
    }

    // 按内容查找，O(n)。找到返回下标，没找到返回空 optional。
    // 原书【算法 2.3】用 `bool getPos(int& p, const T value)`：忘了看返回值，
    // 就会读到一个从没被写过的 p。这里「找没找到」是返回值类型的一部分。
    std::optional<size_type> find(const T& value) const {
        for (size_type i = 0; i < size_; ++i) {
            if (data_[i] == value) {
                return i;
            }
        }
        return std::nullopt;
    }

    // 在位置 pos 插入，pos 可以等于 size()（追加到表尾）。
```

```
    // 代价 O(n)：pos 之后的元素都要右移一位。这正是顺序表与链表要对比的地方。
    void insert(size_type pos, const T& value) {
        if (pos > size_) {
            throw std::out_of_range("ArrayList::insert: 插入位置非法");
        }
        if (size_ == capacity_) {
            grow();
        }
        for (size_type i = size_; i > pos; --i) {
            data_[i] = data_[i - 1];   // 从后往前搬，否则会自己覆盖自己
        }
        data_[pos] = value;
        ++size_;
    }

    void append(const T& value) { insert(size_, value); }

    // 删除 pos 上的元素并返回它。代价同样是 O(n)：后面的元素都要左移一位。
    T remove(size_type pos) {
        if (pos >= size_) {
            throw std::out_of_range("ArrayList::remove: 下标越界");
        }
        T removed = data_[pos];
        for (size_type i = pos; i + 1 < size_; ++i) {
            data_[i] = data_[i + 1];
        }
        --size_;
        return removed;
    }

    // 有了 begin/end，range-for 就能用了：for (auto& x : list) { ... }
    //
    // 原书是在类里放一个 `int position` 游标，配 setPos/next/prev 来依次处理元素。
    // 那种设计把「遍历到哪了」这个状态塞进了容器：const 对象没法遍历，
    // 两处代码不能同时遍历，嵌套遍历直接互相踩。游标挪到容器外面，这些问题一起消失。
    T* begin() { return data_; }
    T* end() { return data_ + size_; }
    const T* begin() const { return data_; }
    const T* end() const { return data_ + size_; }

private:
    // 扩容：申请两倍大的新数组，搬过去，再释放旧的。
    // 翻倍而不是加一，才能让 append 的摊还代价保持 O(1)。
    void grow() {
        size_type next = (capacity_ == 0) ? 1 : capacity_ * 2;
        T* fresh = new T[next];
        for (size_type i = 0; i < size_; ++i) {
```

```
            fresh[i] = data_[i];
        }
        delete[] data_;        // 先搬完再释放旧的，顺序反了就会读到已释放的内存
        data_ = fresh;
        capacity_ = next;
    }

    T* data_;              // 指向底层数组
    size_type capacity_;   // 数组能放多少个
    size_type size_;       // 现在放了几个
};
```

编译运行：

```
g++ -std=c++17 -Wall -Wextra demo.cpp -o demo && ./demo
```

2.2.1 顺序表的类定义

原书【代码 2.2】用四个成员表示顺序表：`T* aList`、`int maxSize`、`int curLen`、`int position`。前三个对应缓冲区、容量、当前长度，教学版一一对应（改用 `std::size_t`，因为「负下标」不该在类型层面存在）。

第四个 `position` 是个**当前处理位置游标**，配合正文提到的 `setPos / setStart / next / prev` 用来「依次处理元素」。这个设计今天不能要：遍历状态一旦住进容器，`const` 对象就没法遍历（游标要改），两处代码不能同时遍历，嵌套遍历直接互相踩。本书删掉这个成员，改为提供 `begin()/end()`，把遍历状态交回调用方——range-for 因此可以直接用在顺序表上。（顺带一提：原书那个 `position`，在书中展示的所有算法里一次都没被用到。）

缓冲区是**裸指针**。于是这个类自己管着资源，就必须遵守三**法则**：一个类只要写了**析构函数、拷贝构造、拷贝赋值**中的任意一个，通常这三个都得写。理由很直白——你之所以要写析构函数，是因为你在管资源；既然在管资源，编译器那份「逐成员照抄」的拷贝就一定是错的：照抄一个指针成员的结果是 **两个对象指向同一块内存**，各自析构时各释放一次，同一块内存被释放两次。

原书 `arrList` 正是如此：有析构函数，却没有拷贝构造与拷贝赋值。一句普通的 `arrList<int> b = a;` 就会二次释放——与第 3 章 `arrStack` 是同一个错误，同一份 ASan 报告可以复现。

这里用裸指针而不是 `std::unique_ptr<T[]>`，是因为顺序表的存储管理正是本节的教学内容：换成 `std::unique_ptr<T[]>` 后，析构和两个移动操作都由编译器生成，学生看不到它们为什么必须存在（拷贝构造和拷贝赋值则被禁用，要拷贝仍得手写）。判据见 2.3.1 节的「所有权工具怎么选」。

另外原书的 `clear()` 是这样写的：

```
text original-listing=" 原书 clear 清单缺少三法则与异常安全，作为缺陷原样
```

引用" void clear() { delete [] aList; curLen = position = 0; aList = new T[maxSize]; }

释放整块再重新分配。既没必要（把长度归零即可，容量留着复用），也不是异常安全的：若 new 抛异常，对象就停在「指针已释放、长度已归零」的破碎状态，之后析构还会再 delete[] 一次。教学版的 clear() 只把长度归零，一行。

2.2.2 顺序表的运算实现

顺序表的检索

顺序表上的检索分按位置和按内容两类。按位置的检索直接由地址公式算出，O(1)—— 就是上面 at() 和 set() 那几行：先查下标合不合法，然后直接 data_[index]，没有循环。原书的 getValue 用「出参 + bool」，越界时打印一行再返回 false；教学版把越界当**错误**处理，抛 std::out_of_range。

按内容的检索是把待查值与表中元素依次比较, O(n)。这里要专门讲一下「**没找到**」**怎么告诉调用方**—— 这是全书反复出现的一个问题，第一次遇到值得说透。

原书【算法 2.3】的签名是：

```
bool getPos(int& p, const T value);
```

一次调用要带回两件事：**找没找到**（函数的返回值 bool），和**在第几个位置**（引用参数 p，函数在里面写结果，这种参数习惯上叫「出参」）。找到了就把下标写进 p 并返回 true，没找到就返回 false、p 不动。

用起来是这样：

```
int p;                          // 此刻 p 是未初始化的，里面是内存里的残留值
if (list.getPos(p, 20)) {       // ← 这一行的判断是必须的
    use(p);
}
```

问题就在那句注释上：**这个判断没有任何东西强制你写**。漏掉它，代码照样编译、照样运行：

```
int p;
list.getPos(p, 20);             // 忘了看返回值
use(p);                         // ← 用的是一个从没被写过的 p
```

p 里是什么？没找到时 getPos 根本没碰它，所以它还是那块内存原来的残留值——可能是 0，可能是上一次调用留下的旧下标，也可能是任意数字。程序不会崩，只会**默默地按错误的下标继续跑**。这类 bug 极难查，因为出错的地方和崩溃的地方往往隔着很远。

现代写法把这两件事合成一**个**返回值——就是上面 find() 的 std::optional<size_type>：

循环体一模一样，差别只在「没找到」怎么表达。

`std::optional<size_type>` 可以理解成一个「可能装着下标的盒子」：找到了，盒子里是下标；没找到，盒子是空的（`std::nullopt`）。关键在于**取值必须先开盒**，而开盒这一步绕不过去：

```
if (auto pos = list.find(20)) {   // 先问盒子空不空
    use(*pos);                     // 确认非空后才取值
}
```

对空盒子直接 `*` 取值是未定义行为,用 `.value()` 取则会抛 `std::bad_optional_access`。两条路都不会像 `int p` 那样**悄悄给你一个看似正常的垃圾值**。

工程版还在 `find()` 上加了一个 `[[nodiscard]]` 属性，再补一道：连返回值都丢掉不用，编译器直接报错。

```
$ g++ -std=c++17 -Wall -Wextra -Wpedantic -Werror …
error: ignoring return value of ‘std::optional<...> dsa::ArrayList<T>::find(const
↪  T&) const’,
       declared with attribute ‘nodiscard’ [-Werror=unused-result]
```

教学版没有写这个属性——它是纯粹的工程加固，与顺序表这一节要讲的东西无关。

一句话概括这个改动：**原来靠调用方自觉，现在靠类型系统和编译器**。「找没找到」这件事 从「一个你可以忽略的 `bool`」变成了「不处理就取不出数据的盒子」。全书凡是「可能没有结果」的接口——查找、出栈、取队首——都按这条办。

（顺带一提，算法 2.3 那段按印刷原样是编译不过的：循环写的是 `for (i = 0; i < n; i++)`，而 `n` 从未声明，按上下文应为 `curLen`。）

检索的时间代价体现在比较次数上。最好情况是第 1 个元素即为所求，比较 1 次；最差情况是表中没有该元素，比较 n 次。假设待查值出现在每个位置的概率相同，均为 $p = 1/n$，它在下标 $i-1$（第 i 个元素）处时要比较 i 次，于是查找成功时的平均比较次数为

$$\sum_{i=1}^{n} p \times i = \frac{1}{n}(1 + 2 + \cdots + n) = \frac{n+1}{2}$$

即平均需要检查表中一半的元素，时间开销为 O(n)。

顺序表的插入

插入要在指定位置腾出一个空位：从表尾起，把 pos 之后的元素逐个右移一位。教学版 `insert()` 里那个**倒着走**的循环就是在做这件事：

```
for (size_type i = size_; i > pos; --i) {
    data_[i] = data_[i - 1];
}
```

方向不能反。若改成从 pos 起递增地 `data_[i + 1] = data_[i]`，第一次搬完就把 `data_[pos + 1]` 覆盖掉了，后面搬的全是同一个值。

两处与原书不同：

- **容量不足时自动翻倍**，而不是打印 `"The list is overflow"` 然后返回 false。扩容策略与第 3 章算法 3.3 相同。
- **位置非法抛** `std::out_of_range`，而不是打印一行再返回 false。按公约，可预期的空状态用 `optional`，真正的错误抛异常——插到表外是错误。

插入位置 p 处腾空位的过程如下（图 2.2）：p 及其之后的元素整体右移一位，空出的 p 位置写入新元素。

aList	k0	k1	...	value	kp	...	kn-1	...	
下标	0	1	...	p	p+1	...	n	...	maxSize − 1

图 2.2 顺序表元素插入示意图

时间代价仍然是 O(n)。最好情况是插在表尾，移动 0 次；最差情况是插在表头，n 个元素全要移动；一般地，插在位置 i 要把其后的 $n-i$ 个元素右移。可插入的位置有 $0, 1, \cdots, n$ 共 $n+1$ 个，假设在各位置插入的概率相等，均为 $p = 1/(n+1)$，则平均移动次数为

$$\sum_{i=0}^{n} p \times (n-i) = \frac{1}{n+1} \sum_{i=0}^{n} (n-i) = \frac{n}{2}$$

扩容没有改变这个结论：翻倍带来的搬迁**摊还**到每次插入是 O(1)，而元素右移本身仍是 O(n)。

「摊还」是指：单次操作偶尔很贵，但把总代价分摊到所有操作上之后，每次仍然很便宜。这里绝大多数插入不触发扩容，偶尔一次翻倍要搬走全部 n 个元素；但容量 n 是上一次翻倍（从 n/2 到 n）得来的，两次扩容之间至少发生了 n/2 次插入，把这一次搬迁的 n 次复制分摊下去，每次插入平摊不超过 2 次，仍是 O(1)。累计起来，从空表连续追加 N 个元素，扩容搬迁的总次数 是 $8 + 16 + \cdots$ 这样一个等比级数，末项不超过 N，总和小于 2N。注意它说的是**平均**而不是保证—— 那一次搬迁真的会卡住，对延迟敏感的场合要单独考虑。

这一点是 2.4 节拿顺序表与链表对比的依据，不能被优化掉。

顺序表的删除

删除是插入的镜像：把 pos 之后的元素逐个左移一位，这次循环**正着走**。教学版的 `remove()` 还多做一件事——先把被删的元素存下来，最后返回给调用方。

空表上删除必然越界，所以不需要原书那句单独的空表检查：「表空」在这里不是一种可预期状态，而就是下标非法的一个特例。

删除位置 p 上的元素后，其后的元素整体左移一位（图 2.3）：

删除前：

aList	k0	k1	k2	...	kp	...	kn-1	...
下标	0	1	2	...	p	...	n − 1	...

删除后：

aList	k0	k1	k2	...	kp+1	...	kn-1	...
下标	0	1	2	...	p	...	n − 2	...

图 2.3 顺序表元素删除示意图

原书【算法 2.5】的函数名是 `delete`——C++ 关键字，编译不过；本书叫 `remove`。另外它返回 `bool`，被删掉的值就此丢失；这里把它返回给调用方，「删除并取用」不必先 `getValue` 再 `delete` 两步走。

删除的时间代价分析与插入相似，平均为 O(n)。具体地，删除位置 i 上的元素要把其后的 $n-1-i$ 个 元素左移；可删除的位置只有 $0, 1, \cdots, n-1$ 共 n 个，等概率（$p = 1/n$）时平均移动次数为

$$\sum_{i=0}^{n-1} p \times (n-1-i) = \frac{1}{n}\sum_{i=0}^{n-1}(n-1-i) = \frac{n-1}{2}$$

比插入少半次：删表尾不必移动，而插入多了「插在表尾之后」这个位置。

2.2a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。上面那份教学版在**分配和 `T` 的拷贝都不抛异常**时是正确的（元素是 `int`、`std::string` 这类东西时就是如此），跑得起来，日常用例下 ASan 也干净。这一节讲的是把它变成一个**工业级容器**还要补哪些东西。等你哪天要写自己的容器时再回来读。工程版在 `code/ch02/array_list/modern.hpp`，与教学版一样进闸门、一样在 g++ 与 clang++ 下各按两档编译运行。

一、三法则补成五法则：移动语义

教学版把一个临时表赋给别人时，会老老实实深拷贝一遍——而那个临时对象下一行就要销毁，这次拷贝纯属浪费。工程版补上**移动构造**与**移动赋值**：直接把指针「偷」过来，再把对方置空，O(1) 完事。

```
/// 在位置 pos 插入元素，pos 可以等于 size()（即追加到表尾）。
/// 位置非法抛 std::out_of_range；容量不足自动翻倍。
///
/// 时间代价仍是 O(n)——pos 之后的元素都要右移一位。这是顺序表的固有代价，
/// 也是第 2.4 节要拿它和链表对比的地方，没有被" 优化" 掉。
void insert(size_type pos, const T& value) {
    make_gap(pos);
    data_[pos] = value;
    ++size_;
}

void insert(size_type pos, T&& value) {
    make_gap(pos);
    data_[pos] = std::move(value);
    ++size_;
}

void append(const T& value) { insert(size_, value); }
void append(T&& value) { insert(size_, std::move(value)); }
```

`insert` 因此有了两个重载:左值走拷贝,右值走 `std::move`。`append(std::string("很长的字符串"))` 于是不再复制那串字符，只搬指针。

二、扩容中途抛异常：强异常保证

教学版的 `grow()` 是「申请 → 搬 → 释放旧的」三步。它对 `int`、`std::string` 这类元素完全够用，但如果某个元素的拷贝**中途抛了异常**，新申请的 `fresh` 就漏掉了。工程版把这一段包进 `try/catch`，并保证失败时原表**原封不动**——这叫**强异常保证**：一个操作要么完全成功，要么像没发生过一样。

```
void ensure_capacity() {
    if (size_ < capacity_) {
        return;
    }
    constexpr size_type kMax = std::numeric_limits<size_type>::max();
    if (capacity_ > kMax / 2) {
        throw std::overflow_error("ArrayList: 容量翻倍会溢出");
    }
    const size_type next = capacity_ == 0 ? kInitialCapacity : capacity_ * 2;
```

```
    T* fresh = new T[next];
    try {
        for (size_type i = 0; i < size_; ++i) {
            // 判据同第 3 章（DECISION_LOG D-005）：看的是 ** 移动赋值 ** 抛不抛，
            // 不是 std::move_if_noexcept 检查的移动构造。两者可以不同，
            // 用错会让搬到一半的失败把原表掏空。
            if constexpr (std::is_nothrow_move_assignable<T>::value) {
                fresh[i] = std::move(data_[i]);
            } else {
                fresh[i] = data_[i];
            }
        }
    } catch (...) {
        delete[] fresh;
        throw;
    }
    delete[] data_;
    data_ = fresh;
    capacity_ = next;
}
```

搬迁到底用移动还是拷贝，判据必须落在**实际执行的那个操作**（移动赋值）上，而不是惯用的 `std::move_if_noexcept`（它看的是移动**构造**）。这个坑第 3 章 3.1.2a 有完整的复现与解释，此处不重复。

三、编译期的类型约束

工程版类头上有四条 `static_assert`，在编译期检查放进来的 `T` 合不合格：必须能默认构造（`new T[n]` 会构造整块槽位）、必须能移动赋值、不可拷贝的 `T` 必须能无异常移动赋值、不能是引用类型。

写在类里而不是文档里，是因为**编译期报错永远比运行期谜案便宜**。教学版没有它们：`T` 不合格时报错发生在模板实例化的深处，信息难看，但同样报错。这是可读性与报错质量之间的一次交换，教学版选了前者。

四、其它工程细节

工程版	教学版	差在哪
`check_index()` / `make_gap()` 抽成私有辅助函数	检查写在每个函数里	少一层跳转，读者不必来回翻

工程版	教学版	差在哪
查询函数标 `[[nodiscard]]` 与 `noexcept`	都不标	前者拦住「忘了看返回值」，后者让标准库能查询这个承诺
放在 `namespace dsa` 里	无命名空间	真实项目要防名字冲突
自由函数 `swap(a, b)`	无	ADL 找得到，标准库算法会用

与原书的对照

原书	现在	为什么
`bool delete(const int p)`	`T remove(size_type pos)`	`delete` 是关键字，原样编译不过；顺带把被删元素还给调用方
`class List { ... } public:` 无	不设基类，运算直接定义在 `ArrayList<T>` 上（工程版对 `T` 的要求写成 `static_assert`）	所有运算私有的 ADT 在语义上是空的；空基类给不了多态
`for (i = 0; i < n; i++)`	`i < size_`	原书 `n` 从未声明，编译不过
`bool getPos(int& p, T v)`	`std::optional<size_type> find(const T&)`	「找没找到」交给类型系统，忽略返回值会告警
`bool getValue(int p, T& v)`	`const T& at(size_type)`，越界抛异常	同上；越界是错误，不是可预期状态
`int maxSize / curLen`，满了拒绝	`size_t capacity_ / size_`，自动翻倍	`int` 溢出是未定义行为；「负下标」不该存在；容量不该是使用者的负担
`int position` 游标 + `next/prev`	`begin()/end()`	遍历状态住在容器里，`const` 不能遍历、不能嵌套遍历
`cout << "The list is overflow"`	不做任何 I/O	数据结构不该和标准输出耦合
`clear()` 释放后重新分配	`clear() noexcept` 只归零长度	原写法没必要，且 `new` 抛异常会留下破碎对象

刻意没改的：连续存储、按下标 O(1) 随机存取、插入与删除搬动 O(n) 个元素。这三条正是 2.4 节拿顺序表和链表做对比的全部依据，一条都不能优化掉。

完整实现见 `code/ch02/array_list/modern.hpp`，测试见同目录 `test.cpp`（47 项断言，覆盖上表每一行；用 `python3 tools/check_code.py` 在 `-Werror` + ASan/UBSan 与 `-O2` 两种构建下、g++ 与 clang++ 两个编译器上共跑四遍）。

2.3 链表

为什么这一节没有 Python 版

链表这一节要看的是结点所有权：谁负责 `new` 出来的结点、拷贝时是否深复制、异常和析构时怎样沿链释放。Python 的 `list` 既不是这条链，也不要求读者实现拷贝构造、拷贝赋值、移动构造和析构；即便手写 Python 结点，垃圾回收仍会把所有权边界变得不可观察。因此这里保留 C++ 的裸指针与五法则，Python 版只会成为另一种容器写法。

顺序表用物理相邻表示逻辑相邻，所以按下标读取是 O(1)，但中间插入和删除需要搬动 后续元素。链表把逻辑相邻写进结点的链接域：结点可以散落在内存中；给定一个前驱结点后，插入或删除只改常数条指针。代价也必须如实保留：要按位置找到那个前驱，仍须从表头循链，所以按位置访问、查找、插入和删除的总时间仍是 O(n)。

每个结点只存一根指向后继的指针时，叫单链表 (singly linked list)。

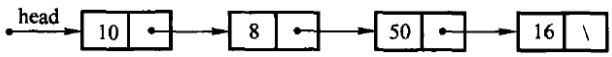


图 2.4　单链表示例。结点分成两部分：`data` 域存真正的数据（图中的 10、8、50、16），`next` 域存后继结点的地址。终止结点没有后继，`next` 是空指针，图里用「」表示。结点在内存中不必两两相邻——这正是链表能以常数条指针完成插入删除的原因。

访问只能从表头开始顺着 `next` 走：图 2.4 里 `head->next->data` 是 8，`head->next->next->data` 是 50。表越长，这条链越长。为了让「在表尾追加」不必每次走到底，另设一个指向尾结点的变量 `tail`：

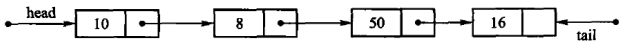


图 2.5　具有头、尾两指针的单链表。有了 `tail`，`append()` 是 O(1)。

教学版：完整实现

下面是一份**完整的、能直接编译运行的**单链表。一个文件、一个类。后面 2.3.1、2.3.2 两节就是把它拆开逐段讲。

```
// 单链表 LinkedList ——教学版。原书【代码 2.6】【代码 2.7】【算法 2.8】-【算法 2.11】。
//
// 一个文件、一个类、能直接编译运行，给「第一次读这一节」的人看。
// 保留链表要教的全部内容：结点分散存储、带头结点、尾指针让 append 保持 O(1)、
// 按位置查找必须循链 O(n)、插入删除在定位之后只改常数条链接。
//
```

```cpp
// 与 modern.hpp（工程版）的分工：
//   教学版  三法则（析构 + 拷贝构造 + 拷贝赋值），头结点里放一个不用的 T；
//   工程版  补齐移动语义，并把头结点做成不含 T 的哨兵基类，
//           这样 T 就不必可默认构造——代价是多一层继承和一堆 static_cast。
// 两份都在闸门里真编译真运行。先读这一份，2.3a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <optional>
#include <stdexcept>

template <typename T>
class LinkedList {
private:
    // 结点（node）：一个数据域，加一根指向后继的链接。原书【代码 2.6】。
    // 它是实现细节，所以放在 private 里——调用方拿不到指针，就改不坏链。
    struct Node {
        T value;
        Node* next;
    };

public:
    using value_type = T;
    using size_type = std::size_t;

    // 头结点（head node）是一个 ** 不存放数据 ** 的哨兵结点，永远排在第一个真元素前面。
    // 它的作用是消掉「在表头插入 / 删除表头」这个特例：有了它，任何位置的插入
    // 都变成「找到前驱，改它的 next」，一套代码走遍全表。
    LinkedList() : head_(new Node), tail_(head_), size_(0) {
        head_->next = nullptr;
    }

    ~LinkedList() {
        clear();
        delete head_;      // 头结点是构造时 new 的，最后要还回去
    }

    // 三法则：这个类管着一串 new 出来的结点，拷贝必须自己写。
    LinkedList(const LinkedList& other) : head_(new Node), tail_(head_), size_(0) {
        head_->next = nullptr;
        for (Node* source = other.head_->next; source != nullptr; source =
        ↪  source->next) {
            append(source->value);
        }
    }

    LinkedList& operator=(const LinkedList& other) {
```

```
    if (this == &other) {
        return *this;
    }
    clear();
    for (Node* source = other.head_->next; source != nullptr; source =
    ↪  source->next) {
        append(source->value);
    }
    return *this;
}

bool empty() const { return size_ == 0; }
size_type size() const { return size_; }

// 删掉全部真元素，头结点留着。
void clear() {
    Node* current = head_->next;
    while (current != nullptr) {
        Node* dying = current;
        current = current->next;
        delete dying;
    }
    head_->next = nullptr;
    tail_ = head_;          // 表空了，尾指针退回头结点
    size_ = 0;
}

// 在表尾追加。** 因为存了 tail_，这里是 O(1)**；没有它就得每次从头走到尾。
void append(const T& value) {
    Node* fresh = new Node;
    fresh->value = value;
    fresh->next = nullptr;
    tail_->next = fresh;
    tail_ = fresh;
    ++size_;
}

// 在位置 pos 插入，pos 可以等于 size()（追加到表尾）。
// 两步：① 循链找到 pos 的 ** 前驱 **，O(n)；② 改两条链接，O(1)。
// 这就是链表与顺序表的分工——链表的插入不搬元素，但定位要走。
void insert(size_type pos, const T& value) {
    Node* predecessor = predecessor_at(pos);
    Node* fresh = new Node;
    fresh->value = value;
    fresh->next = predecessor->next;
    predecessor->next = fresh;
    if (predecessor == tail_) {   // 插在表尾，尾指针要跟上
```

```
            tail_ = fresh;
        }
        ++size_;
    }

    // 删除位置 pos 上的元素并返回它。同样是「先定位前驱，再改一条链接」。
    T remove(size_type pos) {
        if (pos >= size_) {
            throw std::out_of_range("LinkedList::remove: 下标越界");
        }
        Node* predecessor = predecessor_at(pos);
        Node* dying = predecessor->next;
        T value = dying->value;
        predecessor->next = dying->next;
        if (dying == tail_) {          // 删的是最后一个，尾指针退回前驱
            tail_ = predecessor;
        }
        delete dying;
        --size_;
        return value;
    }

    // 按位置取值：链表 ** 不是 ** 随机存取的，只能从头一个一个数过去，O(n)。
    // 这一条是 2.4 节拿链表和顺序表对比的关键。
    const T& at(size_type pos) const { return node_at(pos)->value; }
    T& at(size_type pos) { return node_at(pos)->value; }

    // 按内容查找，O(n)。找到返回位置，没找到返回空 optional。
    std::optional<size_type> find(const T& value) const {
        size_type pos = 0;
        for (Node* current = head_->next; current != nullptr; current =
        ↪  current->next) {
            if (current->value == value) {
                return pos;
            }
            ++pos;
        }
        return std::nullopt;
    }

    // 链表没有连续下标，range-for 要靠迭代器。这个迭代器就是一根被包起来的
    // 结点指针：`*it` 取值，`++it` 顺着 next 走一步，走到 nullptr 就是结束。
    class Iterator {
    public:
        explicit Iterator(Node* node) : node_(node) {}
        T& operator*() const { return node_->value; }
        Iterator& operator++() {
```

```
            node_ = node_->next;
            return *this;
        }
        bool operator!=(const Iterator& other) const { return node_ != other.node_; }

    private:
        Node* node_;
    };

    Iterator begin() { return Iterator(head_->next); }   // 从第一个 ** 真 ** 元素开始
    Iterator end() { return Iterator(nullptr); }

private:
    // 返回位置 pos 的前驱结点。pos == 0 时前驱就是头结点——
    // 这正是头结点存在的意义：表头不再是特例。
    Node* predecessor_at(size_type pos) const {
        if (pos > size_) {
            throw std::out_of_range("LinkedList: 下标越界");
        }
        Node* predecessor = head_;
        for (size_type i = 0; i < pos; ++i) {
            predecessor = predecessor->next;
        }
        return predecessor;
    }

    Node* node_at(size_type pos) const {
        if (pos >= size_) {
            throw std::out_of_range("LinkedList::at: 下标越界");
        }
        Node* current = head_->next;
        for (size_type i = 0; i < pos; ++i) {
            current = current->next;
        }
        return current;
    }

    Node* head_;          // 头结点（哨兵，不存数据）
    Node* tail_;          // 最后一个结点；表空时等于 head_
    size_type size_;
};
```

编译运行：

```
g++ -std=c++17 -Wall -Wextra demo.cpp -o demo && ./demo
```

2.3.1 单链表

单链结点与头结点

【代码 2.6】单链表的结点定义。

结点 (node) 就是上面那个 `Node`：**一个数据域，加一根指向后继的链接域**。原书把它写成独立的 `Link<T>`；这里放在 `LinkedList` 的 private 区里，因为调用方拿不到结点指针，就没法改坏链。

【代码 2.6 结束】

`LinkedList<T>` 还有一个**头结点** (head node)：一个不存放数据的哨兵，永远排在第一个真元素前面。它等价于原书的" 第 -1 个结点"。有了它，表头插入和删除都变成" 修改某个前驱的 `next`"，空表也不必另写一套分支—— 这就是 `predecessor_at(0)` 能直接返回头结点的原因。

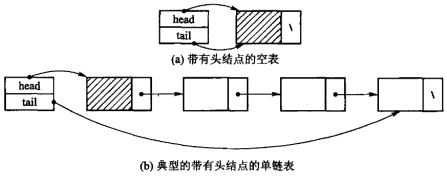


图 2.6　引入头结点的单链表：(a) 带头结点的空表，(b) 一个典型的带头结点的单链表。带阴影的那个结点就是头结点，它的数据域不算表中元素，只是为了让空表和表头不再是特例。

尾指针 `tail_` 指向最后一个实际结点，空表时回指头结点，所以 `append` 不需要从头寻找末尾，是 O(1)。

【代码 2.7】单链表的类型定义。

见上面教学版清单的类头：三个成员——头结点、尾指针、长度。

【代码 2.7 结束】

构析与循链定位

【算法 2.8】带有头结点的单链表构造函数与析构函数。

教学版的构造函数 `new` 出头结点，析构函数先 `clear()` 掉全部真元素、再把头结点还回去。`clear()` 沿 `next` **逐个循环释放**，不是递归——链长十万级时递归释放会耗尽运行栈，本节稍后的「所有权工具怎么选」和第 3 章 3.1.3a 都有实测数字。

两处一漏就出事，教学版的测试各有一条断言盯着：

- `clear()` 之后 `tail_` 必须退回头结点。不退，下一次 `append` 就写进已释放的内存，AddressSanitizer 报 `heap-use-after-free`。
- 析构里那句 `delete head_` 不能漏。漏了，每个链表对象泄漏一个结点，LeakSanitizer 会报。

【算法 2.8 结束】

【算法 2.9】寻找链表的第 i 个结点。

定位从头结点开始逐步走 next，不新建任何结点，O(n)。原书的 setPos(-1) 是处理 头结点的技巧；教学版不暴露 -1 这个特殊位置，predecessor_at(0) 直接返回头结点。

返回"前驱"而不是"结点本身"，是因为插入和删除都要改前驱的 next——单链表从一个结点走不回它的前一个。一个函数同时服务两处，这是头结点带来的简化。

【算法 2.9 结束】

所有权工具怎么选

读到这里会有一个很自然的疑问：既然现代 C++ 有 std::unique_ptr，为什么 clear() 还在手写 delete？把 next 声明成 std::unique_ptr<Node> 不是更省事吗——没有 delete，没有析构函数，五法则一条都不用写。

小规模下这么写确实看不出问题。代价藏在编译器替你生成的析构里：

```
/// ** 教学反例。** 用 `std::unique_ptr` 把结点串成链，看起来最干净：没有 `delete`，
/// 没有析构函数，五法则一条都不用写。
///
/// 代价藏在编译器替你生成的析构里：`~RecursiveNode` 要析构 `next`，`next` 的析构又要
/// 析构它的 `next`……链有多长，栈就压多深。链表通常正是「元素很多」的结构，
/// 于是一次普通的析构就能把栈压穿——而且崩溃阈值随优化级别变，debug 崩、release 过。
///
/// 实测数字与复现命令见本单元的 `legacy.md`。
struct RecursiveNode {
    int value = 0;
    std::unique_ptr<RecursiveNode> next;
};

class RecursiveChain {
public:
    void push_front(int value) {
        auto node = std::make_unique<RecursiveNode>();
        node->value = value;
        node->next = std::move(head_);
        head_ = std::move(node);
        ++size_;
    }

    [[nodiscard]] std::vector<int> to_vector() const {
        std::vector<int> out;
        for (const RecursiveNode* cursor = head_.get(); cursor != nullptr;
             cursor = cursor->next.get()) {
            out.push_back(cursor->value);
```

```
        }
        return out;
    }

    [[nodiscard]] std::size_t size() const noexcept { return size_; }

private:
    std::unique_ptr<RecursiveNode> head_;
    std::size_t size_ = 0;
    // 析构函数是编译器生成的——问题就出在这里：它是递归的，而且看不见。
};
```

~RecursiveNode 要析构 next，next 的析构又要析构它的 next——**链有多长，栈就压多深**。而链表恰恰是「元素很多」的结构。本机 8 MB 栈上实测：

构建档	最大安全结点数	崩溃于
-O0 -g	约 57,625	58,601
-O2	约 523,329	524,306

两档差了九倍。这才是最难受的地方：同一份代码，debug 下五万多个结点就段错误，release 下五十多万才崩。学生在 debug 里调出段错误，切到 release 又复现不了。递归深度不该由优化级别决定。

自己管所有权，多写一个析构和一个 clear()，换来的是**栈深度与链长无关**：

```
void clear() noexcept {
    NodeBase* current = head_.next;
    while (current != nullptr) {
        NodeBase* following = current->next;
        delete static_cast<Node*>(current);
        current = following;
    }
    head_.next = nullptr;
    tail_ = &head_;
    size_ = 0;
}
```

同样 -O0，五百万个结点一次段错误都没有。这几行不是仪式，是这个结构能不能处理大数据的分界线。

但这不等于「本书不用智能指针」。判据是结构形态，不是个人偏好：

结构形态	该用什么	理由
链（结点串成一条线）	裸指针 + 迭代释放	`unique_ptr` 的析构是递归的，深度正比于链长
树（孩子唯一所有权）	`unique_ptr`	释放同样递归，但深度等于树高——B+ 树这类平衡树是 $O(\log n)$，Trie 是最长键的长度，都远小于结点数（退化成一条链的树则另当别论）；第 11 章的 B+ 树、第 12 章的 Trie 用的就是它
一整块缓冲区	裸 `T*` + 五法则	换成 `unique_ptr<T[]>` 只省掉一句 `delete[]`——它只能移动，拷贝构造仍须手写，五法则并没有消失（见 2.2.1）
共享（一个结点多个父）	手写引用计数	`unique_ptr` 语义上不成立；12.2 的广义表要教的正是「谁来回收」

`code/ch02/ownership` 把两种写法并排放着，复现命令和完整数字在该目录的 `legacy.md`。写工程代码时默认用智能指针是对的；本书在少数几处不用，是因为**那几处的所有权本身就是要教的内容**，而且换过去有可测的代价。

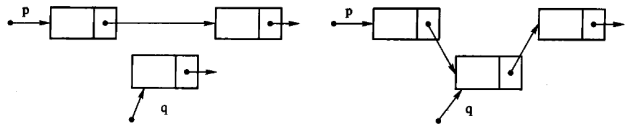
插入与删除

【算法 2.10】插入单链表的第 i 个结点。

教学版的 `insert(pos, value)` 分两步：① 循链找到 pos 的**前驱**，O(n)；② 改两条链接，O(1)。

这就是链表与顺序表的分工：顺序表的 `insert` 不用找（下标直接算），但要搬 O(n) 个元素；链表不搬任何元素，但要走 O(n) 步去找。两者的 O(n) 是不同的东西——一个在搬数据，一个在走指针。

`append` 单列一个函数而不是转调 `insert(size_)`，正是因为有 `tail_`：尾插不需要定位，直接接在 `tail_` 后面，O(1)。转调 `insert` 就白白退化成 O(n) 了。插在表尾时 `tail_` 必须跟着往后挪，教学版的测试有一条专门验它。



(a) 准备新结点，定位待插入点　　(b) 成功插入后指针链接情况

图 2.7　插入示意：新结点的 next 先接上前驱原来的后继（图中的 ①），再把前驱的 next 改指向新结点（②）。两步的次序不能反——先改前驱就再也找不到原来的后继了。

【算法 2.10 结束】

【算法 2.11】单链表的删除算法。

删除同样是「定位前驱 → 改一条链接」，但有两件事不能漏：

1. **被删结点必须** `delete`，只摘链不释放就是内存泄漏；
2. **删的若是尾结点，** `tail_` **必须回退到前驱**。不回退，下一次 append 就写进已释放的内存——教学版的测试用「删到空表再 append」把这条钉死了，去掉那两行，AddressSanitizer 立刻报 `heap-use-after-free`。

位置不合法统一抛 `std::out_of_range`，容器不打印任何提示。

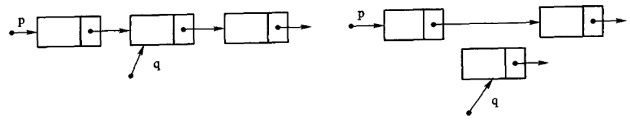


图 2.8　删除示意：左边是删除前，p 是前驱、q 是待删结点；右边是把 `p->next` 跨过 q 之后的样子。q 这时已经脱链，但内存还在，必须 `delete` 掉。

【算法 2.11 结束】

2.3.2 双链表

双链结点

【代码 2.12】双链表的结点定义和实现。

双链结点比单链结点多一根 `prev`。多出来的那个指针（64 位机上 8 字节）只买到一件事，但这件事很值：**已知一个结点时，删除它是 O(1)**。单链表要做同一件事得先从头走到它的前驱，O(n)——因为单链表从一个结点走不回前一个。

原书在此只给出结点定义，没有给出完整的双链表算法。

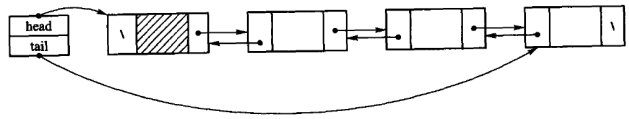


图 2.9　加入表头结点的双链表。带斜线阴影的是头结点；head 指向这个头结点（它排在表首结点之前），tail 指向表尾结点。

注意下面的教学版**没有**头结点：它的 `head_` 直接指向第一个真元素，空表时 `head_` 与 `tail_` 都是 `nullptr`。代价是表头、表尾各成一个特例，所以 `push_front`、

push_back、erase_node 里都有判空指针的分支——这正好与带头结点的单链表形成对照。



图 2.10　双链表的结点：一个数据域，两根指针——prev 指前驱，next 指后继。

【代码 2.12 结束】

【本书补充实现】完整的双链表。

```
// 双链表 DoublyLinkedList ——教学版。原书【代码 2.12】的双链结点。
//
// 一个文件、一个类、能直接编译运行，给「第一次读这一节」的人看。
//
// 双链表比单链表每个结点多存一根 `prev`。多出来的空间买到的是一件事：
// ** 已知结点位置时，插入和删除都是 O(1)**——不必像单链表那样先循链找前驱。
// 这是本节唯一值得多花那份空间的理由，全部教学内容都围绕它。
//
// 与 modern.hpp（工程版）的分工：
//   教学版  三法则，插入/删除各写成一段直白的改链；
//   工程版  补齐移动语义，并把「表头 / 表中 / 表尾」三种情形合并进一个
//           insert_before(pos)——省代码，但初读时那几个 nullptr 分支很绕。
// 两份都在闸门里真编译真运行。先读这一份，2.3a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <stdexcept>

template <typename T>
class DoublyLinkedList {
private:
    // 双链结点：数据 + 前驱链接 + 后继链接。
    struct Node {
        T value;
        Node* prev;
        Node* next;
    };

public:
    using value_type = T;
    using size_type = std::size_t;
```

```
    DoublyLinkedList() : head_(nullptr), tail_(nullptr), size_(0) {}

    ~DoublyLinkedList() { clear(); }

    // 三法则：这个类管着一串 new 出来的结点，拷贝必须自己写。
    DoublyLinkedList(const DoublyLinkedList& other)
        : head_(nullptr), tail_(nullptr), size_(0) {
        for (Node* source = other.head_; source != nullptr; source = source->next) {
            push_back(source->value);
        }
    }

    DoublyLinkedList& operator=(const DoublyLinkedList& other) {
        if (this == &other) {
            return *this;
        }
        clear();
        for (Node* source = other.head_; source != nullptr; source = source->next) {
            push_back(source->value);
        }
        return *this;
    }

    bool empty() const { return size_ == 0; }
    size_type size() const { return size_; }

    void clear() {
        Node* current = head_;
        while (current != nullptr) {
            Node* dying = current;
            current = current->next;
            delete dying;
        }
        head_ = tail_ = nullptr;
        size_ = 0;
    }

    // 表头插入，O(1)。要改的链接一共三处：新结点的 next、原表头的 prev、head_。
    void push_front(const T& value) {
        Node* fresh = new Node;
        fresh->value = value;
        fresh->prev = nullptr;
        fresh->next = head_;
        if (head_ != nullptr) {
            head_->prev = fresh;
        } else {
            tail_ = fresh;      // 原来是空表，新结点同时也是表尾
```

```
        }
        head_ = fresh;
        ++size_;
    }

    // 表尾插入，O(1)。与 push_front 完全对称。
    void push_back(const T& value) {
        Node* fresh = new Node;
        fresh->value = value;
        fresh->prev = tail_;
        fresh->next = nullptr;
        if (tail_ != nullptr) {
            tail_->next = fresh;
        } else {
            head_ = fresh;      // 原来是空表，新结点同时也是表头
        }
        tail_ = fresh;
        ++size_;
    }

    T pop_front() {
        if (empty()) {
            throw std::out_of_range("DoublyLinkedList::pop_front: 表空");
        }
        return erase_node(head_);
    }

    T pop_back() {
        if (empty()) {
            throw std::out_of_range("DoublyLinkedList::pop_back: 表空");
        }
        return erase_node(tail_);
    }

    // 在位置 pos 插入。定位仍是 O(n)——双链表省掉的是「找前驱」，不是「找位置」。
    void insert(size_type pos, const T& value) {
        if (pos > size_) {
            throw std::out_of_range("DoublyLinkedList::insert: 位置非法");
        }
        if (pos == 0) {
            push_front(value);
            return;
        }
        if (pos == size_) {
            push_back(value);
            return;
        }
```

```
        Node* successor = node_at(pos);          // 新结点要插在它前面
        Node* predecessor = successor->prev;     // O(1)：prev 现成的，不用循链
        Node* fresh = new Node;
        fresh->value = value;
        fresh->prev = predecessor;
        fresh->next = successor;
        predecessor->next = fresh;
        successor->prev = fresh;
        ++size_;
    }

    T erase(size_type pos) { return erase_node(node_at(pos)); }

    const T& at(size_type pos) const { return node_at(pos)->value; }
    T& at(size_type pos) { return node_at(pos)->value; }

    // 迭代器：一根被包起来的结点指针。双链表的迭代器可以 `--`，单链表的不行——
    // 这也是那根 prev 买到的东西。
    class Iterator {
    public:
        explicit Iterator(Node* node) : node_(node) {}
        T& operator*() const { return node_->value; }
        Iterator& operator++() {
            node_ = node_->next;
            return *this;
        }
        Iterator& operator--() {
            node_ = node_->prev;
            return *this;
        }
       bool operator!=(const Iterator& other) const { return node_ != other.node_; }

    private:
        Node* node_;
    };

    Iterator begin() { return Iterator(head_); }
    Iterator end() { return Iterator(nullptr); }

private:
    // 摘掉一个已知的结点，O(1)。** 这是双链表的看家本领 **：
    // 单链表要做同一件事，得先从头走到它的前驱，O(n)。
    T erase_node(Node* node) {
        T value = node->value;
        if (node->prev != nullptr) {
            node->prev->next = node->next;
        } else {
```

```
            head_ = node->next;      // 删的是表头
        }
        if (node->next != nullptr) {
            node->next->prev = node->prev;
        } else {
            tail_ = node->prev;      // 删的是表尾
        }
        delete node;
        --size_;
        return value;
    }

    Node* node_at(size_type pos) const {
        if (pos >= size_) {
            throw std::out_of_range("DoublyLinkedList: 下标越界");
        }
        Node* current = head_;
        for (size_type i = 0; i < pos; ++i) {
            current = current->next;
        }
        return current;
    }

    Node* head_;
    Node* tail_;
    size_type size_;
};
```

三处值得停一下：

- **插入和删除都要同时改两侧链接**。少改一根，表在**正向**遍历时看起来完全正常，只有反着走或者删中间结点时才炸。教学版的测试因此正着走一遍、倒着走一遍，断言两个结果互为逆序——漏掉 `successor->prev = fresh;` 这一条就红。
- **首尾是特例，但只有两个**：插入时若新结点成了表头/表尾，`head_`/`tail_` 要跟着改；删除时若删掉的是表头/表尾，同样要改。四个分支写清楚就没有别的坑了。
- **迭代器可以 `--`**。单链表的不行。这也是那根 `prev` 买到的东西。

原书用两幅图讲清了这四根指针怎么改。删除 `p` 所指结点，要同时接好前驱和后继两条链：

text original-listing="原书 2.3.2 节删除双链结点的指针操作，作为原书记录原样引用" p->prev->next = p->next; p->next->prev = p->prev; p->next = NULL; p->prev = NULL; delete p;

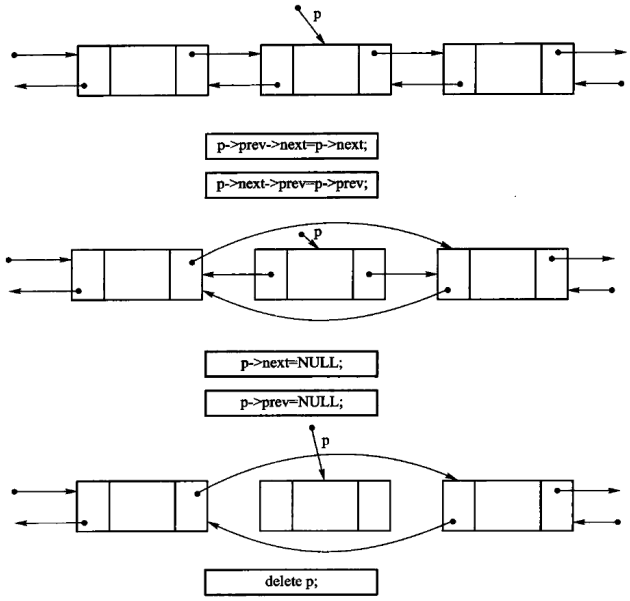


图 2.11　双链表的删除操作示意。原书在摘链之后先把 p 自己的两根指针置空再释放；这两句对正确性没有作用——delete 之后那块内存本来就不能再读，编译器甚至可以把这两次写入当作无用存储删掉，所以教学版的 erase_node 不写它们。

还要注意，这三行假定 p **既有前驱也有后继**。带头结点时前驱总是存在，但若 p 是表尾，p->next 为空指针，第二行就是对空指针解引用。教学版的 erase_node 因此在表头、表尾各留一个分支。

在 p 之后插入新结点 q，先让 q 认好左右邻居，再让邻居认 q：

text original-listing=" 原书 2.3.2 节插入双链结点的指针操作，作为原书记录原样引用" q->prev = p; q->next = p->next; p->next = q; q->next->prev = q;

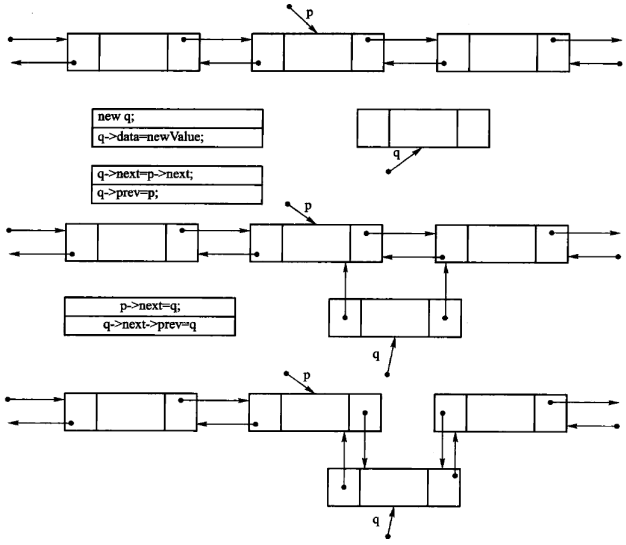


图 2.12　双链表插入操作示意。四条赋值的次序有一处不能反：`q->next = p->next` 必须在 `p->next = q` 之前——先改 `p->next` 就丢了原来的后继；而最后一句 `q->next->prev = q` 要借 `q->next` 找到后继，也必须排在 `q->next` 填好之后。稳妥的做法就是原书的次序：先让新结点指向两侧，再改两侧指向新结点。

同样，最后一句假定 `p` 不是表尾：`p` 是表尾时 `q->next` 为空指针，这一句要换成更新 `tail`。教学版 `insert` 把这种情形交给 `push_back` 处理。

双链表比单链表多一根指针的空间开销，换来的是给定元素时能在 $O(1)$ 时间内找到它的前驱。

【本书补充实现结束】

2.3.3 循环链表

循环链表把尾结点的 `next` 接回首结点，因而没有 `nullptr` 作为终点。它适合轮转调度、循环缓冲区等“处理完最后一个又回到第一个”的场景。空表（没有任何结点时）仍用 `tail == nullptr` 表示——这是唯一出现 `nullptr` 的情形；非空时首结点是 `tail->next`。

原书举的例子是进程轮转：多个进程在一段时间内访问同一个资源，为了让每个进程公平分享，把它们串成一个环，用一个 `current` 指针指向下一个要激活的进程，指针往前走一步就轮到下一个。

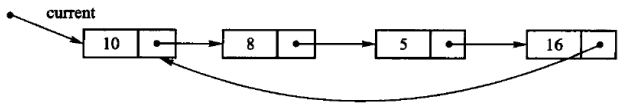


图 2.13　循环链表示意图。`current` 指向将要激活的结点，走一圈回到自己。

只要把单链表最后一个结点的 `next` 指回表首，就得到循环链表；不多花任何存储，却让「从任一结点都能访问到其余全部结点」成为可能。

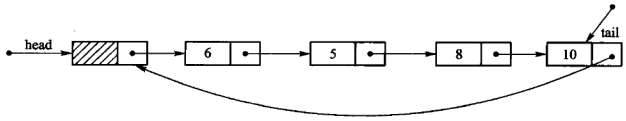


图 2.14　带头结点的循环链表示意图。

把双链表的首尾也接起来，就是循环双链表，反向遍历同样闭合。

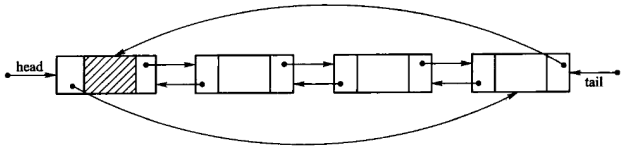


图 2.15　循环双链表示意图。

只保存 `tail` 就够了：取首结点是 `tail->next`，尾插只需改两根链接，均为 $O(1)$。从任意结点开始遍历时必须保存起点，并在再次遇到起点时停止，不能写成“走到 `nullptr` 为止”。删除唯一剩下的那个结点后要把 `tail` 清空；删除首结点则把 `tail->next` 跳过被删结点。循环双链表再把首结点的 `prev` 接到尾结点，反向遍历同样闭合。

循环链表的代价是边界更容易写错：空表、单结点表和多结点表的链接规则不同。测试至少应覆盖三种长度，并验证从每个结点出发恰好走一整圈；否则漏接或误接都可能表现为无限循环。

2.3a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。工程版（`code/ch02/linked_list/modern.hpp` 与 `code/ch02/doubly_linked_list/modern.hpp`）在教学版之上补三件事。

一、头结点不再存 `T`。教学版的头结点是一个完整的 `Node`，里面那个 `value` 从头到尾没人用——代价是 `T` 必须可默认构造。工程版把结点拆成两层：

```
/// 带头结点、尾指针的单链表。
///
/// 原书的 `setPos(-1)` 返回头结点；本实现把这个实现细节留在 predecessor_at，
/// 对外位置统一为 [0, size()]。按值查找返回 optional，位置错误抛 out_of_range。
template <typename T>
class LinkedList {
    // 头结点只保存链接，不保存 T：哨兵不代表元素，因此不要求 T 默认构造。
    // Node 继承 NodeBase 后，定位和接链逻辑只需操作 next，结点布局也不重复。
    struct NodeBase {
        NodeBase* next{nullptr};
    };
    struct Node final : NodeBase {
        T value;

        template <typename U>
        explicit Node(U&& item, NodeBase* successor = nullptr)
```

```
            // 完美转发保留左值拷贝、右值移动两条路径，避免无谓的 T 临时对象。
            : NodeBase{successor}, value(std::forward<U>(item)) {}
    };

public:
    using value_type = T;
    using size_type = std::size_t;
```

NodeBase 只有链接域，头结点用它；真正存数据的 Node 继承它。于是头结点不含 T，LinkedList< 不可默认构造的类型 > 也能用了。代价是链上走的是 NodeBase*，取值时到处要写 static_cast<Node*>：

```
void clear() noexcept {
    NodeBase* current = head_.next;
    while (current != nullptr) {
        NodeBase* following = current->next;
        delete static_cast<Node*>(current);
        current = following;
    }
    head_.next = nullptr;
    tail_ = &head_;
    size_ = 0;
}
```

这是一次实打实的交换：多一层继承和一堆强制转换，换 T 少一条约束。教学版选了前者，工程版选了后者。

二、拷贝到一半失败时要自己收拾。构造函数抛出时，这个对象的析构函数**不会运行**——它还没构造完，语言不认为它存在。所以工程版的拷贝构造把循环包进 try/catch，失败时先 clear() 掉已经接上的半截链再重新抛出。教学版没有这一段。

三、移动语义、[[nodiscard]]、noexcept。与 2.2a 说过的完全一样，不再重复。

双链表这边，工程版还把「表头 / 表中 / 表尾」三种插入情形合并进一个 insert_before(pos)——省代码，但初读时那几个 nullptr 分支很绕：

```
template <typename U>
Node* insert_before(Node* pos, U&& value) {
    // 先构造结点；构造失败时原链完全未改变。
    Node* inserted = new Node(std::forward<U>(value));
    inserted->next = pos;
    inserted->prev = pos != nullptr ? pos->prev : tail_;
    if (inserted->prev != nullptr) {
        inserted->prev->next = inserted;
    } else {
```

```
        head_ = inserted;
    }
    if (pos != nullptr) {
        pos->prev = inserted;
    } else {
        tail_ = inserted;
    }
    ++size_;
    return inserted;
}
```

```
T erase_node(Node* node) {
    if (node == nullptr) throw std::out_of_range("DoublyLinkedList: empty");
    T value = std::move(node->value);
    if (node->prev != nullptr) node->prev->next = node->next;
    else head_ = node->next;
    if (node->next != nullptr) node->next->prev = node->prev;
    else tail_ = node->prev;
    delete node;
    --size_;
    return value;
}
```

工程版的测试覆盖头/中/尾插入、删尾后的尾指针修复、深拷贝、移动、元素构造异常和 move-only 元素；变异自检还确认” 删尾不回退 tail” 会在后续尾插崩溃，“复制构造失败不清理” 会留下元素对象。原书逐条证据见 `code/ch02/linked_list/legacy.md`。

2.4 线性表实现方法的比较

顺序表是最简单的数据组织方法，因此大多数程序设计语言都提供了数组这种机制来实现它。除具有 易用、空间开销较小等特点之外，顺序表还提供了对任意元素进行**随机访问**的能力，适合于二分 检索及快速排序等运算，是存储静态数据的理想选择。按下标访问是 $O(1)$，代价是插入删除要搬动 后续元素，平均 $O(n)$。

链表除了适用于频繁插入或删除内部元素的线性表之外，还适合于管理那些**长度经常变化、或事先 无法确定长度**的线性表。它在已知前驱结点时插入删除是 $O(1)$，但按位置找到那个前驱仍是 $O(n)$。

作为基本的数据结构，线性表有着广泛的应用：存储管理本质上就是利用线性表管理可利用空间（第 12 章）；线性表还可以作为基本成分构建复杂的数据结构，例如散列方法就是把顺序表和链表 结合起来的一种数据结构（第 10 章）。在实际应用中具体采用何种实现方法，取决于应用中线性表 数据元素的统计特征和操作特点。取舍时有两个因素值得注意。

1. **不要使用顺序表的场合**。线性表中经常要插入/删除内部元素时，不宜使用

顺序表，因为顺序 表的插入/删除操作平均情况下需要移动表中一半的元素。此外，无法确定线性表长度的最大值时，也不宜采用顺序表。

2. **不要使用链表的场合**。经常对线性表进行按位置的访问，而且按位读操作比插入/删除操作 频繁时，不宜使用链表，因为顺链表扫描浏览比按顺序表下标读元素要费时。此外，**指针本身的 存储开销也需要考虑**：如果与结点内容所占空间相比，指针所占的比例较大（超过 1:1），应该慎重 选择。

本章小结

本章在介绍线性表基本概念和抽象数据类型的基础上，描述了线性表的两种实现方法。

线性结构是最简单且最常用的一种数据结构，其基本特点是结构中的元素之间满足线性关系，按这个 关系可以把所有元素排成一个线性序列。线性表是由若干数据元素组成的有限序列，是一种比较简单的 数据结构；线性表通常有顺序和链式两种存储方式，**在两种存储结构上，线性表各运算的实现效率各有 千秋**。

顺序表是组织数据的最简单方法，具有易用、空间开销较小以及对元素随机访问等特点，是存储静态 数据的理想选择。**而链表不仅可以适用于那些频繁增删结点的应用，还适用于处理事先无法确定长度的 线性表**。具体选哪一种，看访问统计和操作特点。

习题

正文习题

1. 设顺序表 a 中的数据元素递增有序，试设计一个算法，将 x 插入到顺序表的适当位置，以保持该 表的有序性。
2. 设 $A=(a_1,\cdots,a_m)$ 和 $B=(b_1,\cdots,b_n)$ 均为顺序表，A' 和 B' 分别为 A 和 B 中 除去最大共同前缀后的子表。若 $A'=B'=$ 空表，则 $A=B$；若 $A'=$ 空表而 $B'\neq$ 空表，或者 两者均不为空表且 A' 的表首元素小于 B' 的表首元素，则 $A<B$；否则 $A>B$。试写一个比较 A、B 大小的算法。
3. 设线性表中的数据元素以值递增排列，并以单链表作为存储结构。设计一个高效的算法，删除表中 所有值大于 min 且小于 max 的元素，同时释放被删除结点的空间，并分析算法的时间复杂度。
4. 假设有两个按元素值递增有序排列的线性表 A 和 B，均以单链表作为其存储结构。试设计一个 算法，将 A 和 B 归并成一个按元素值递减有序排列的线性表 C，**并要求利用原表（即表 A 和 表 B）的结点空间构造表 C**。
5. 已知由一个链表表示的线性表中含有三类字符的数据元素（字母字符、数字字符和其他字符），试设计 一个算法，将该线性链表分割为 3 个循环链表，其中每个循环链表中均只含一类字符。
6. 试设计一个算法，删除一个顺序表中从第 i 个元素开始的 k 个元素。
7. 已知线性表中元素以值递增有序排列，并以单链表作为存储结构。试设计一个算法，删除表中值相同 的多余元素，使得操作后表中的所有元素值均不相

同，同时释放被删除的结点空间。

8. 已知两个元素按值递增有序排列的线性表 A 和 B，且同一表中的元素值各不相同。试构造一个 线性表 C，其元素为 A 和 B 中元素的交集，且表 C 中的元素也按值递增有序排列。设计对 A、B、C 都是顺序表时的算法。
9. 要求同第 8 题。设计当 A、B、C 都是单链表时的算法。
10. 设表达式以字符形式已存入数组 $E[n]$ 中，# 为表达式的结束符，试设计判断表达式中括号（和）是否配对的算法。

上机题

1. 试设计一个非递归算法，在 $O(n)$ 时间内将一个含有 n 个元素的单链表逆置，要求其辅助空间为 常量。
2. 给定一个单向链表，试设计一个既节省时间又节省空间的算法来找出该链表的倒数第 m 个元素。实现这个算法，并对可能出现的特殊情况做相应处理。自行设计链表的数据结构。（「倒数第 m 个 元素」的含义：当 $m = 0$ 时，链表的最后一个元素将被返回。）
3. 设有 n 个人围坐在一个圆桌周围，现从第 s 个人开始报数，数到第 m 的人出列，然后从出列的 下一个人重新开始报数，数到第 m 的人又出列，如此反复直到所有的人全部出列为止。Josephus **问题**是：对于任意给定的 n、s、m，求出按出列次序得到的 n 个人员的序列。试在计算机上 模拟 Josephus 问题的求解过程。

第 3 章 栈与队列

栈是「最后放入、最先取出」，适合递归调用、括号匹配和表达式计算；队列是「先放入、先取出」，适合排队服务和广度优先搜索。空栈、空队列是正常状态，现代接口以 `std::optional` 返回。

源码：顺序栈·教学版、顺序栈·工程版、栈示例、链式栈·教学版、链式栈·工程版、表达式求值、背包、队列·教学版、队列·工程版、队列示例。

先跑一遍

```
// 第 3 章「先跑一遍」：用教学版 ArrayStack 走一遍 push / top / pop。
// 编译运行：
//   g++ -std=c++17 -I code/ch03/array_stack code/ch03/array_stack/demo.cpp -o demo
↪  && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    ArrayStack<int> stack;
    stack.push(1);
    stack.push(2);
    stack.push(3);

    // top() 返回 optional：有值才解引用，空栈不会崩
    if (auto value = stack.top()) {
        std::cout << " 栈顶是 " << *value << '\n';
    }

    std::cout << " 依次弹出:";
    while (auto value = stack.pop()) {
        std::cout << ' ' << *value;
    }
    std::cout << "\n空栈再弹? " << (stack.pop() ? " 有值" : " 空") << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch03/array_stack \
    code/ch03/array_stack/demo.cpp -o /tmp/stack-demo
/tmp/stack-demo
```

```
栈顶是 3
依次弹出: 3 2 1
```

```
空栈再弹？空
```

后进先出：最后压入的 3 最先出来。空栈上 `pop()` 返回空 optional，不打印、不崩溃。

循环队列牺牲一个槽位区分空与满，所以逻辑容量 3 实际申请 4 个槽：

```cpp
// 第 3 章「先跑一遍」：用教学版 ArrayQueue 走一遍 enqueue / dequeue，
// 顺便看看「牺牲一个槽位」的效果——逻辑容量 3 就真的只装得下 3 个。
// 编译运行：
//   g++ -std=c++17 -I code/ch03/queue code/ch03/queue/demo.cpp -o demo && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    ArrayQueue<int> queue(3);
    if (!queue.enqueue(1) || !queue.enqueue(2) || !queue.enqueue(3)) {
        std::cout << " 入队失败\n";
        return 1;
    }
    std::cout << " 逻辑容量 3 时再入队？" << (queue.enqueue(4) ? " 成功" : " 已满") <<
    ↪  '\n';
    std::cout << " 依次出队：";
    while (auto value = queue.dequeue()) {
        std::cout << ' ' << *value;
    }
    std::cout << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch03/queue \
    code/ch03/queue/demo.cpp -o /tmp/queue-demo
/tmp/queue-demo
```

```
逻辑容量 3 时再入队？已满
依次出队： 1 2 3
```

3.1 栈

为什么顺序栈和链式栈只有 C++

顺序栈和链式栈在本章承担的是存储管理课：数组容量如何扩展、元素如何构造和销毁、链式结点由谁拥有，以及扩容失败时如何保持原栈不变。Python `list.append` 会把这些决定全部藏起来，手写一个 Python 栈也不会让三法则、五

法则或强异常保证变得可观察，所以这两个存储结构不提供 “把 C++ 逐行翻译成 Python” 的版本。

这条只管存储侧。本节 3.1.5 的背包问题是算法侧,两种语言各有一份,Python 版就印在那一小节里——同一节里两种做法并存, 正是 D-025 那条分工线的样子。

3.1.1 栈的抽象数据类型

栈 (stack) 是限定仅在一端进行插入和删除运算的线性表，该端称为**栈顶** (top), 另一端称为**栈底** (bottom)。栈的元素按后进先出 (LIFO, last in first out) 的次序访问：最后压入的元素最先弹出。

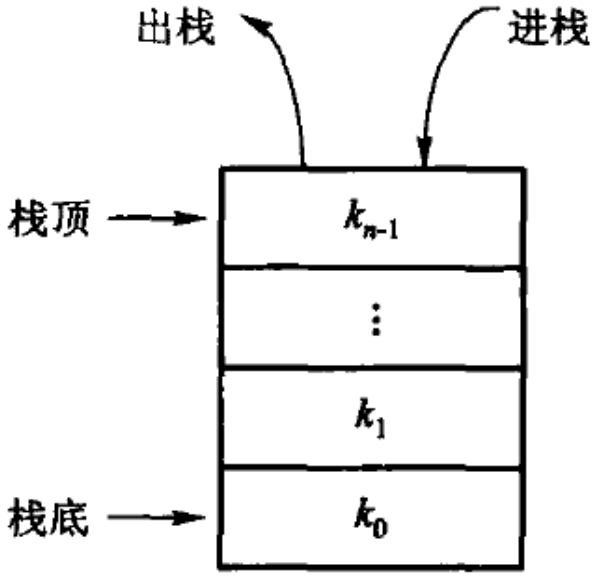


图 3.1　栈的示意图。进和出都在栈顶这一端，栈底不动。

基于栈的特性，栈的抽象数据类型包含进栈 push、出栈 pop、读栈顶 top 等常用操作，以及判断栈是否为空的边界操作。根据抽象和封装的原则，只可通过抽象数据类型 定义的运算来操作栈。

原书【代码 3.1】用一个成员函数既非纯虚、析构函数也非 virtual 的空基类 `Stack<T>` 来表达这层抽象。那样的基类给不了任何多态能力：成员函数既不是纯虚的、也没有定义，派生类” 实现” 它们并不构成覆盖；反而埋下了「通过 `Stack<T>*` 删除派生对象」这一未定义行为。

抽象数据类型描述的是「一组运算」, 不是「一个基类」。在 C++ 里，模板本身已经承担了这层抽象——`ArrayStack<T>` 提供哪些运算，由它的接口决定，不需要继承任何东西。我们不关心它是否继承自某个 `Stack<T>`，只关心它有没有 `push`、`pop`、`top` 这几个运算。所以这一节要定下来的是**这张表**:

运算	含义	时间代价
`push(x)`	把 x 压到栈顶	摊还 O(1)
`pop()`	弹出栈顶并把它带回来；空栈返回「没有」	O(1)
`top()`	只看栈顶，不弹出；空栈返回「没有」	O(1)
`empty()`	栈里还有没有元素	O(1)
`size()`	栈里有几个元素	O(1)
`clear()`	清空	O(1)（教学版只把长度归零；3.1.2a 的工程版要逐个析构元素，O(n)）

表里有两处「空栈返回『没有』」。这四个字在 C++17 里有一个精确的表达方式，下一节的代码就是这么写的。

3.1.2 顺序栈

采用顺序存储结构的栈称为顺序栈 (array-based stack)，需要一块连续的区域存储栈中元素。

对元素数目为 n 的栈，首先要确定数组的哪一端表示栈顶。如果把数组的第 0 个位置作为栈顶，所有的插入和删除都在第 0 个位置进行，每次 push 或 pop 都要把栈中所有元素后移或前移 一个位置，时间代价为 O(n)。反之，把最后一个元素的位置作为栈顶，新元素添加在表尾、出栈也只删除表尾元素，每次操作的时间代价仅为 O(1)。图 3.2 所示为按后一种方案实现的栈。

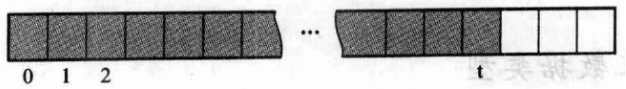


图 3.2　栈的顺序存储结构示意：一块连续空间，栈顶在元素多的那一端。

用本书的记法写出来是这样——下标 0 是栈底，`size_` 是元素个数，也是下一个空位：

```
下标     0     1     2     3     4     …
内容    [A]   [B]   [C]    ?     ?     …
              ▲
           size_ = 3    （栈顶是 C，下一个 push 写到 3）
```

原书用的是另一种记法：成员 `top` 直接存**栈顶元素所在的位置**，空栈时 `top = -1`，push 先加一再写、pop 先取再减一。图 3.3 画的就是 `mSize` 为 6 的顺序栈的四个瞬间。

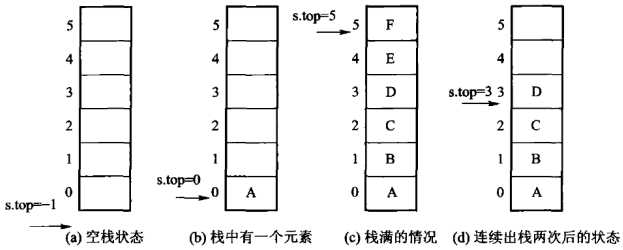


图 3.3　顺序栈的存储结构：(a) 空栈，`s.top = -1`；(b) 栈中有一个元素，`s.top = 0`；(c) 栈满，`s.top = mSize - 1`；(d) 在 (c) 上连续出栈两次之后。

两种记法只差一个常数：`top == size_ - 1`。本书选 `size_`，因为它同时就是长度，`empty()` 与 `size()` 都不必再换算，也避开了「−1 是合法下标吗」这种要靠约定记住的事。栈里已有 `mSize` 个元素还要 `push` 叫**上溢** (overflow)，空栈上 `pop` 叫**下溢** (underflow)：前者本书用扩容化解，后者是可预期状态，返回 `std::nullopt`。

教学版：完整实现

下面是一份**完整的、能直接编译运行的**顺序栈。一个文件、一个类，没有省略号，没有「此处略去若干行」。它保留原书【代码 3.2】【算法 3.3】要教的全部内容—— 连续数组、栈顶在表尾、满了就把容量翻倍——只把原书那几处会崩的写法换掉。

```
// 顺序栈 ArrayStack ——教学版。
//
// 这一份是给「第一次读这一节」的人看的：一个文件、一个类、能直接编译运行。
// 它保留原书【代码 3.2】【算法 3.3】要教的全部内容——连续数组、栈顶在表尾、
// 满了就把容量翻倍——只把原书那几处会崩的写法换掉。
//
// 与 modern.hpp（工程版）的分工：
//   教学版  遵守 ** 三法则 **（析构 + 拷贝构造 + 拷贝赋值），正确，但拷贝多一点；
//   工程版  在此之上补齐移动构造/移动赋值、强异常保证、编译期类型约束。
// 两份都在闸门里真编译真运行。先读这一份，3.1.2a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <optional>

template <typename T>
class ArrayStack {
public:
    using value_type = T;
    using size_type = std::size_t;

    // 构造：先要一小块数组。容量不够时会自动翻倍，所以初值给多少都不影响正确性。
    explicit ArrayStack(size_type initial_capacity = 8)
```

```
    : data_(new T[initial_capacity]), capacity_(initial_capacity), size_(0) {}

// 析构：数组是 new[] 来的，就得 delete[] 回去。
~ArrayStack() { delete[] data_; }

// 拷贝构造：** 必须自己写 **。
// 不写的话编译器生成的版本会把 data_ 这根指针照抄一份，于是两个栈指向同一块
// 内存，各析构一次 ——同一块内存被释放两次。原书 arrStack 正是漏了这个。
ArrayStack(const ArrayStack& other)
    : data_(new T[other.capacity_]), capacity_(other.capacity_),
    ↪  size_(other.size_) {
    for (size_type i = 0; i < size_; ++i) {
        data_[i] = other.data_[i];
    }
}

// 拷贝赋值：同理。注意三件事的顺序——先把新数组备好，再释放旧的，最后接管。
ArrayStack& operator=(const ArrayStack& other) {
    if (this == &other) {   // 自己赋值给自己，什么都不用做
        return *this;
    }
    T* fresh = new T[other.capacity_];
    for (size_type i = 0; i < other.size_; ++i) {
        fresh[i] = other.data_[i];
    }
    delete[] data_;
    data_ = fresh;
    capacity_ = other.capacity_;
    size_ = other.size_;
    return *this;
}

// 入栈。满了就翻倍，所以不会有「栈满溢出」这回事。
void push(const T& value) {
    if (size_ == capacity_) {
        grow();
    }
    data_[size_] = value;
    ++size_;
}

// 出栈并把元素带回来。空栈返回空的 optional，不是错误，也不打印任何东西。
std::optional<T> pop() {
    if (empty()) {
        return std::nullopt;
    }
    --size_;
```

```
        return data_[size_];
    }

    // 只看栈顶，不弹出。空栈同样返回空 optional。
    std::optional<T> top() const {
        if (empty()) {
            return std::nullopt;
        }
        return data_[size_ - 1];
    }

    bool empty() const { return size_ == 0; }
    size_type size() const { return size_; }
    size_type capacity() const { return capacity_; }

    // 清空：把长度归零就行，已经申请的数组留着接着用。
    void clear() { size_ = 0; }

private:
    // 扩容：申请一块两倍大的，把老元素搬过去，再把老的还回去。
    // 每个元素在均摊意义下只被搬运常数次，所以 push 的摊还代价仍是 O(1)。
    void grow() {
        size_type next = (capacity_ == 0) ? 1 : capacity_ * 2;
        T* fresh = new T[next];
        for (size_type i = 0; i < size_; ++i) {
            fresh[i] = data_[i];
        }
        delete[] data_;        // 先搬完再释放旧的，顺序反了就会读到已释放的内存
        data_ = fresh;
        capacity_ = next;
    }

    T* data_;              // 指向底层数组
    size_type capacity_;   // 数组能放多少个
    size_type size_;       // 现在放了几个，同时也是下一个空位的下标
};
```

把它存成 `teaching.hpp`，配上本章开头那个 `demo.cpp`，一条命令就能跑：

```
g++ -std=c++17 -Wall -Wextra demo.cpp -o demo && ./demo
```

关键要点

1. **空栈上的 pop/top 返回空 optional。**原书的写法是 `bool pop(T & item)`——用出参带值、用返回值带成败。调用方一旦忘了检查 返回值，读到的就是那个从

没被修改过的 `item`：程序不崩，只是默默按错误数据继续跑。`std::optional<T>` 把「有没有值」搬进了**类型**里，取值之前必须先判断。**空栈不是错误，是一种可预期的状态**，所以它返回空盒子而不是抛异常；真正的错误（参数非法、容量溢出）才抛异常。

2. **拷贝构造和拷贝赋值必须自己写——这是原书最硬的一处错**。原书的 `arrStack` 有析构函数，却没有拷贝构造，也没有拷贝赋值。于是一句 `arrStack<int> b = a;` 会走编译器生成的拷贝——**把指针照抄一份**，之后 `a` 和 `b` 持有**同一根指针**，各析构一次，**同一块内存被释放两次**。这类错误的可怕之处在于它安静：`-Wall -Wextra -Wpedantic` 一句警告都不给，小数据量下往往也不当场崩。本书用 AddressSanitizer 把它抓了出来，报告抄在 `code/ch03/array_stack/legacy.md` 里。

规则叫三**法则**：一个类只要自己写了析构函数、拷贝构造、拷贝赋值中的**任意一个**，通常这三个都得写。原因很直白——你之所以要写析构函数，是因为你在管资源；既然在管资源，编译器那份「逐成员照抄」的拷贝就一定是错的。

3. **翻倍扩容让 `push` 的摊还代价保持 O(1)**。`grow()` 每次把容量乘 2，而不是加 1。差别不是常数：加 1 的话，push n 个元素总共要搬 $1+2+\cdots+n=O(n^2)$ 次；翻倍的话，总搬运次数是 $1+2+4+\cdots+n<2n$，平摊到每次 push 就是常数。这就是原书【算法 3.3】的策略，一个字都没改。

顺带一提 `grow()` 里那三行的**顺序**：先申请新数组、再搬元素、最后才 `delete[]` 旧的。反过来写（先释放再搬）就是读已经还回去的内存，ASan 当场报 `heap-use-after-free`。

4. `new T[n]` **有一条限制，值得知道**。它会把整块槽位**默认构造**出来。所以 `T` 必须能默认构造——一个只有带参构造函数的类 放不进这个栈。这是原书 `new T[mSize]` 就有的限制，本书的教学版没有改善它。标准库容器的做法是先申请**未初始化**的原始内存，push 时再用 placement new 在指定位置构造对象；那要手写内存对齐和逐个析构，属于另一个话题。

5. **容器里一个 `cout` 都没有**。原书在空栈出栈时直接 `cout << " 栈空"`。那样做把一个数据结构和标准输出焊死了：它没法在库里复用，提示无法本地化，失败路径也无从测试。**数据结构负责数据结构，报错交给调用方**。本书的测试里有一条专门重定向 `cout`/`cerr` 并断言它们为空。

3.1.2a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。上面那份教学版在**分配和 `T` 的拷贝都不抛异常**时是正确的（元素是 `int`、`std::string` 这类东西时就是如此），跑得起来，日常用例下 ASan 也干净。这一节讲的是把它变成一个**工业级容器**还要补哪些东西—— 移动语义、异常安全、编译期类型约束。等你哪天要写自己的容器时再回来读。

完整的工程版在 `code/ch03/array_stack/modern.hpp`，与教学版一样进闸门、一样双档编译运行。下面按主题一段一段拆开看。

一、存储与所有权：把三法则补成五法则

先把原书【代码 3.2】的问题列全。它用三个成员表示一个顺序栈：`int mSize`（容量）、`int top`（栈顶位置）、`T * st`（裸指针数组）。这套写法有三处在今天必须改：

1. `int top` 与成员函数 `bool top(T&)` **重名**，导致这个类按印刷原样根本编译不过；
2. 无参构造只设了 `top = -1`，`mSize` 与 `st` 都没初始化，析构时 `delete[]` 一个不确定指针；
3. 有析构函数却没有拷贝构造与拷贝赋值，一次 `arrStack<int> b = a;` 就会二次释放。

第 2、3 条是未定义行为，编译器开到 `-Wall -Wextra -Wpedantic` 也**一句警告都不给**。教学版已经把这三条都修掉了：`size_` 不与任何成员函数重名，三个成员全部有初值，三法则写全。

工程版在这之上再补两个函数——**移动构造**与**移动赋值**，合称**五法则**。它们不修正确性，只修性能：教学版把一个临时栈赋给别人时会老老实实深拷贝一遍，而那个临时对象下一行就要销毁了，这次拷贝纯属浪费。移动版直接把指针「偷」过来，再把对方置空，O(1) 完事。

这里保留裸指针 `T* data_`——顺序栈的存储管理正是本节要教的内容，把它换成 `std::unique_ptr<T[]>` 固然更省事，但五法则也就随之退化成走过场（编译器生成的版本就够用了）。留着裸指针，这五个函数才是承重的：少写一个，AddressSanitizer 立刻能复现出崩溃。

```
// 三/五法则：原书只写了析构函数，没有拷贝构造与拷贝赋值，
// 于是 `arrStack<int> b = a;` 之后两个对象持有同一根指针，各析构一次 → 二次释放。
//
// 这里用的是 ** 裸指针 **，所以这五个函数是承重的，不是仪式——
// 少写一个，ASan 立刻能复现出崩溃（见 legacy.md 缺陷 4 的实测输出）。
/// 逐个 ** 拷贝构造 ** 进新槽位。构造到一半抛异常时，已经建好的那几个要自己析构掉——
/// `new T[n]` 版本不用写这一段（数组 new 会替你回滚），裸存储要自己负责。
ArrayStack(const ArrayStack& other)
    : capacity_(other.capacity_), top_index_(0), data_(allocate(other.capacity_)) {
    try {
        for (; top_index_ < other.top_index_; ++top_index_) {
            construct_at(top_index_, other.data_[top_index_]);
        }
    } catch (...) {
        destroy_range(data_, top_index_);
        deallocate(data_, capacity_);
        throw;
    }
}
```

```
/// 拷贝并交换：先构造完整副本再与自身交换，天然自赋值安全，
/// 且拷贝失败时原对象不受影响。
ArrayStack& operator=(const ArrayStack& other) {
    if (this != &other) {
        ArrayStack copy(other);
        swap(copy);
    }
    return *this;
}

ArrayStack(ArrayStack&& other) noexcept { swap(other); }

ArrayStack& operator=(ArrayStack&& other) noexcept {
    if (this != &other) {
        ArrayStack moved(std::move(other));  // other 交出所有权
        swap(moved);                         // 自己原来的缓冲区随 moved 析构释放
    }
    return *this;
}

/// 先析构还活着的元素，再还存储。顺序反了就是对已释放内存调析构。
~ArrayStack() {
    destroy_range(data_, top_index_);
    deallocate(data_, capacity_);
}
```

拷贝赋值用的是**拷贝并交换** (copy-and-swap)：先构造一个完整副本，再与自身交换。它自然地做到了自赋值安全，也在拷贝失败时保证原对象不受影响。

二、读栈顶的第二个接口：`peek()`

`optional` 那部分教学版已经有了，这里补的是它的**代价**：`std::optional<T> top()` 返回的是一份**副本**。副本要求 `T` 可拷贝，而且确实拷了一次。对 `std::unique_ptr` 这类只能移动的元素，这个接口根本用不了。

所以工程版另给一个零拷贝的观望接口 `peek()`：

```
/// 出栈。空栈返回 std::nullopt——原书是「返回 false + 往 cout 打一行中文」，
/// 调用方既没法在库里复用，也容易忽略返回值。
[[nodiscard]]
std::optional<T> pop() {
    if (empty()) {
        return std::nullopt;
    }
    // 三步的次序是承重的：先把值搬出来（可能抛），成功了才缩短逻辑长度，
    // 最后 ** 显式析构 ** 那个槽位。若把 --top_index_ 提到前面，移动一抛，
```

```
    // 那个元素就既不在栈里、也没人析构它了。
    std::optional<T> value(std::move(data_[top_index_ - 1]));
    --top_index_;
    data_[top_index_].~T();
    return value;
}

/// 读栈顶但不弹出，返回 ** 副本 **。空栈返回 std::nullopt，不是未定义行为。
/// 要求 T 可拷贝；move-only 元素请用 peek()。
std::optional<T> top() const {
    if (empty()) {
        return std::nullopt;
    }
    return std::optional<T>(data_[top_index_ - 1]);
}

/// 只读观望栈顶：不拷贝、不弹出。空栈返回 nullptr。
///
/// 与 top() 的分工——top() 给你一份可以带走的副本（安全，但要求 T 可拷贝，
/// 而且确实拷了一次）；peek() 零拷贝，move-only 元素也能用，代价是
/// ** 返回的指针在下一次 push / pop / clear 之后即失效 **（扩容会换掉整块缓冲区）。
/// 生命周期由调用方负责，这一点必须写在文档里，不能靠使用者猜。
const T* peek() const noexcept {
    return empty() ? nullptr : &data_[top_index_ - 1];
}
```

「读栈顶」有两种正当需求，因此这里给了两个接口，**不要把它们合并成一个**：

- `top()` 返回 `std::optional<T>`，是一份可以带走的**副本**。安全，但要求 `T` 可拷贝，而且确实拷贝了一次。
- `peek()` 返回 `const T*`，空栈为 `nullptr`，**零拷贝**。`std::unique_ptr` 这类只能移动的元素只能用它来观望。

代价也是一对：`optional` 的安全靠拷贝换来，`peek()` 的零拷贝则把生命周期交给了 调用方——**它返回的指针在下一次 `push` / `pop` / `clear` 之后即失效**，因为扩容会换掉整块缓冲区。这条失效契约必须写在文档里，不能靠使用者猜。

两种代价都真实存在。把任何一种藏起来，都是替读者做了本该由读者做的选择。

三、扩容中途抛异常：强异常保证

翻倍扩容的策略教学版已经照原书【算法 3.3】实现了。工程版在同一段代码上多做一件事：**保证搬到一半失败时，原来的栈仍然完好**。

原书【算法 3.3】那段代码本身有两个问题：循环变量 `i` 从未声明（编译不过），以及先 `delete[] st` 再赋新指针，搬运中途若抛出异常，栈就停在半新半旧

的状态。教学版修掉了前者，也把顺序改成「先搬后释放」；但它没有处理「搬到一半抛异常」—— 那时新申请的 `fresh` 会漏掉。教学版接受这个代价（元素类型是 `int`、`std::string` 这类东西时它根本不会发生），工程版不接受。

下面这段实现里会反复出现两个词，先说清楚：

- **强异常保证**：一个操作要么完全成功，要么**像没发生过一样**——绝不留下半成品状态。做法就是「先在新缓冲区上把事情做完，全部成功了再换过去」。原书的写法没有这个保证：旧缓冲区已经 `delete[]` 掉了，搬到一半抛异常，栈既回不到旧状态也到不了新状态。
- RAII：把资源的生命周期绑在对象的生命周期上，析构时自动释放。用了 RAII 的类型（例如 `std::unique_ptr`）不需要手写下面那段 `try/catch` 清理——这里用裸指针，所以清理要自己写。这份多出来的代价是本节要看见的东西之一。

```
static constexpr size_type kInitialCapacity = 4;

void ensure_capacity() {
    if (top_index_ < capacity_) {
        return;
    }
    constexpr size_type kMax = std::numeric_limits<size_type>::max();
    if (capacity_ > kMax / 2) {
        throw std::overflow_error("ArrayStack: 容量翻倍会溢出");
    }
    const size_type next = capacity_ == 0 ? kInitialCapacity : capacity_ * 2;
    T* fresh = allocate(next);
    size_type built = 0;
    try {
        for (; built < top_index_; ++built) {
            // 搬迁现在是 ** 构造 **，于是 std::move_if_noexcept 恰好就是对的工具：
            // 它问的正是「移动 ** 构造 ** 抛不抛」，而这里执行的正是移动构造。
            //
            // 2026-08-12 第一版把这件事做在 ** 赋值 ** 语义上，两者的异常规格可以不同，
            // 红队 T-002 就是从这个缝里打进来的（legacy.md 缺陷 11），
            // 当时只能手写「看移动赋值抛不抛」的判据。换成裸存储后这道题消失了：
            //   移动构造 noexcept → 移动，不抛，也不白白深拷贝；
            //   否则              → 复制，抛了也动不了原栈（上面的 static_assert 保证可复
            ↪  制构造）。
            construct_at_in(fresh, built, std::move_if_noexcept(data_[built]));
        }
    } catch (...) {
        // 裸存储的代价：搬迁失败要自己析构已经建好的那几个、再还掉新存储。
        // 此时 data_/capacity_ 一个字节都没动，所以原栈完好——这就是强异常保证。
        destroy_range(fresh, built);
        deallocate(fresh, next);
        throw;
```

```
    }
    destroy_range(data_, top_index_);
    deallocate(data_, capacity_);
    data_ = fresh;
    capacity_ = next;
}
```

四处值得注意：

- **先建后换**。新缓冲区搬完才动 `data_`，中途抛异常则原栈原封不动，这就是**强异常保证**。原书先 `delete[] st` 再赋新指针，一旦搬运途中抛出，栈就停在一个既不是旧状态也不是新状态的地方。
- `try/catch` **是裸指针的代价**。搬迁失败要自己把新缓冲区收掉再重新抛出；如果底层换成 `std::unique_ptr<T[]>`，这一段可以整个删掉。取舍在这里是显式的：本节要教存储管理，就得连这份代价一起看见。
- **搬迁用移动还是拷贝，判据必须选对**。这一条值得单独讲，见下一小节。
- **容量用** `std::size_t`。原书用 `int`，`mSize * 2` 溢出是未定义行为；这里在翻倍前先判界并抛 `std::overflow_error`。

这不是纸面推理。测试里有一个「第 3 次拷贝构造必定抛异常」的元素类型，用它撑满栈再触发扩容，然后逐项断言：长度不变、容量不变、原有元素逐个完好、栈之后仍能继续使用；另一条用例让扩容时的存储分配本身抛 `std::bad_alloc`（测试里替换了全局的对齐版 `operator new`），同样逐项断言。把「先建后换」改回原书的「先释放旧的」，Debug 构建下 UBSan 当场报空指针引用，Release 构建直接段错误。

四、存储层：分配存储 ≠ 构造对象

教学版用 `new T[n]` 拿到数组——一行就有，讲「顺序栈就是一块连续数组」够用了。但这一行做了两件事：**申请存储**，以及**把整块槽位默认构造一遍**。第二件是白做的。

先量，再改。用一个数构造次数的探针：

```
① 构造 ArrayStack<Counted>(1000)，一次 push 都没有：构造了 1000 个对象
```

预留 1000 的容量，就先造出 1000 个没人要的对象。原书 `new T[mSize]` 一模一样。而且这一行还顺手给使用者加了一条限制——`T` 必须可默认构造：

```
error: static assertion failed: ArrayStack<T>: T 必须可默认构造（底层 new T[n] 会构造整
↪  块槽位）
```

栈没有任何理由要求元素能默认构造。这条限制不是设计出来的，是 `new T[n]` 漏出来的。

还有第三笔账。教学版的 `clear()` 只写 `size_ = 0`，槽位里的对象一个也没析构。用一个每实例自带 200 字节堆内存的探针量：

```
满栈 1000：存活 1024 个，持有 204800 字节
clear() 之后：存活 1024 个，仍持有 204800 字节（逻辑长度已经是 0）
再 push 3 个：存活 1024 个——只有 3 个是栈里的，其余 1021 个谁也够不着
```

注意是 1024 **而不是** 1000：多出来的 24 个是扩容时默认构造、从未被用过的空槽位。

工程版把这两件事拆开：**存储只是字节，对象在真正入栈时才构造，出栈时立刻析构**。

```
// 存储层：分配存储 / 构造对象 / 析构对象 / 归还存储，四件事分开。
//
// 用 `::operator new(bytes, align_val_t)` 而不是 `new T[n]`：后者会顺手把
// 整块槽位默认构造一遍。** 代价是对齐要自己管 **——`new T[n]` 替你保证的
// 对齐，这里必须显式传 alignof(T)，否则 `alignas(64)` 的元素会落在错地方。
// 这是裸存储换来的自由所对应的那份义务。
[[nodiscard]] static T* allocate(size_type count) {
    if (count == 0) {
        return nullptr;
    }
    if (count > std::numeric_limits<size_type>::max() / sizeof(T)) {
        throw std::overflow_error("ArrayStack: 请求的字节数溢出");
    }
    void* raw = ::operator new(count * sizeof(T), std::align_val_t{alignof(T)});
    return static_cast<T*>(raw);
}

// 归还用不带大小的对齐版：带大小的 `operator delete(p, bytes, align_val_t)`
// 只在编译器开了 sized deallocation 时才有声明，clang 18 及以前默认不开，
// 那样这一行直接编译不过（2026-09-18 Codex 复核 T-079 时撞出）。大小只是给分配器的提示。
static void deallocate(T* block, size_type /*count*/) noexcept {
    if (block == nullptr) {
        return;
    }
    ::operator delete(static_cast<void*>(block), std::align_val_t{alignof(T)});
}

/// 在第 index 个槽位上就地构造一个 T。槽位在此之前是 ** 生存储，不是对象 **。
template <typename U>
void construct_at(size_type index, U&& value) {
    construct_at_in(data_, index, std::forward<U>(value));
}

template <typename U>
```

```
static void construct_at_in(T* block, size_type index, U&& value) {
    ::new (static_cast<void*>(block + index)) T(std::forward<U>(value));
}

/// 逆序析构 [0, count) 这些元素。只析构，不还存储。
static void destroy_range(T* block, size_type count) noexcept {
    for (size_type i = count; i-- > 0;) {
        block[i].~T();
    }
}
```

于是 push 是**构造**而不是赋值，pop 在搬走值之后显式调 `~T()`，clear 逐个析构。同一组测量变成：

```
① 构造 ArrayStack<Counted>(1000)：构造了 0 个对象
满栈 1000：存活 1000 个；clear() 之后：存活 0 个；再 push 3 个：存活 3 个
```

这份自由对应一份义务：对齐要自己管。 `new T[n]` 替你保证的对齐，`::operator new(bytes)` 不管，所以 `allocate()` 必须显式传 `alignof(T)`。把那个参数去掉，`alignas(64)` 的元素当场越界——ASan 报 heap-buffer-overflow，`-O2` 档直接段错误。教学版不必操心这件事，因为它没拿走这份自由。

一个副产品：一道旧题消失了。早先这里有个坑——搬迁元素时写 `std::move_if_noexcept(data_[i])` 几乎是条件反射，但它的判据是**移动构造**抛不抛，而 `new T[n]` 版本的槽位已经构造好了，搬迁执行的是**移动赋值**。两者是不同的函数，可以有不同的异常规格：移动构造 `noexcept`、移动赋值却会抛的类型是存在的，本书测试里就有一个，它当场打穿了第一版实现——被移动走的元素已经掏空，强异常保证就此破裂。当时只能手写判据：看移动赋值抛不抛。

换成裸存储之后，搬迁执行的**就是移动构造**，`move_if_noexcept` 问的正是那个问题。工具和语境对上了，手写判据可以删掉。这不是「顺手变好看了」——**它说明当初那个坑的根源不是用错了工具，而是容器把对象构造和存储分配捆在了一起。**

守门用例仍然是两条，缺一不可：移动构造会抛的类型，扩容必须一**次移动都不用**（数出来是 0 次移动、4 次拷贝）；移动构造 `noexcept` 的类型，扩容一**次都不许拷贝**（否则 `std::string` 每次扩容都变深拷贝，算法 3.3 摊还 O(1) 的分析就打了折扣）。这两种写法都能通过其他所有断言，只有这两条能把它们分开。

五、push 的两个重载

进栈本身随之简化为「保证容量、写入、长度加一」，但工程版给了**两个**重载：

```
/// 入栈。容量不足时按算法 3.3 的策略翻倍。
/// 强异常保证：搬迁在新缓冲区上完成，中途抛异常则原栈原封不动。
/// 注意是 ** 构造 ** 而不是赋值：那个槽位此前根本没有对象。
/// 构造失败时 top_index_ 还没加，栈原样不动。
void push(const T& item) {
    ensure_capacity();
    construct_at(top_index_, item);
    ++top_index_;
}

void push(T&& item) {
    ensure_capacity();
    construct_at(top_index_, std::move(item));
    ++top_index_;
}
```

两个重载分别处理左值和右值。教学版只写了 `push(const T&)`——正确，但压入一个临时对象时会多拷贝一次。原书的 `bool push(const T item)` 更糟：按值传参，顶层 `const` 对调用方没有意义，却强制了一次拷贝，而且 `std::unique_ptr` 这类只能移动的类型根本传不进去。

六、对元素类型的编译期约束

工程版的类头上还有三条 `static_assert`。它们不参与运行，只在编译期检查「你放进来的这个 `T` 合不合格」：必须能移动或复制构造（`push` 要把元素构造进槽位）、移动构造要么不抛、要么 `T` 可复制构造（扩容的强异常保证靠这条）、不能是引用类型。

原本还有第四条：必须能默认构造。上一小节把存储层换掉之后，那条限制连同 `new T[n]` 一起消失了——少一条对使用者的要求，是这次改动最实在的收益之一。

这几条写在类里而不是文档里，是因为**编译期报错永远比运行期谜案便宜**。教学版没有它们：`T` 不合格时报错会发生在模板实例化的深处，信息难看，但同样报错。这是可读性与报错质量之间的一次交换，教学版选了前者。

3.1.3 链式栈

采用链式存储结构的栈称为链式栈。结点分散在堆上，压栈只是接一个新结点，**不需要连续空间，也没有" 栈满" 这回事**——这正是它与顺序栈的核心差别。

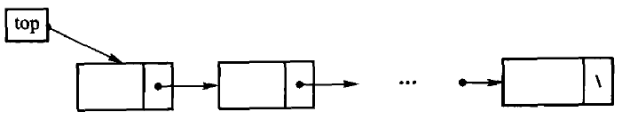


图 3.4　链式栈示意。栈顶就是链表的第一个结点，`push` 在链头插入、`pop` 删

链头，两者都是 O(1)；把栈顶放在链尾就得从头走到尾，退化成 O(n)。

教学版：完整实现

接口形状和 3.1.2 的 `ArrayStack` 刻意保持一致——同一个抽象数据类型，换一种存储结构，两者才好拿来对比。

```
// 链式栈 LinkedStack ——教学版。
//
// 一个文件、一个类、能直接编译运行，给「第一次读这一节」的人看。
// 本节要教的是「同一个 ADT 换一种存储结构」：顺序栈用连续数组，链式栈用结点串联。
// 所以接口形状与 ArrayStack 刻意保持一致，两者才好拿来对比。
//
// 与 modern.hpp（工程版）的分工：
//   教学版  三法则（析构 + 拷贝构造 + 拷贝赋值），正确，但拷贝多一点；
//   工程版  在此之上补齐移动语义、拷贝失败时的清理、零拷贝的 peek()。
// 两份都在闸门里真编译真运行。先读这一份，3.1.3a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <optional>

template <typename T>
class LinkedStack {
public:
    using value_type = T;
    using size_type = std::size_t;

    // 链式栈 ** 不需要预设容量 **，所以构造函数没有参数。
    // 原书的 `lnkStack(int defSize)` 是从顺序栈那边照抄过来的，那个参数一次都没用过。
    LinkedStack() : top_(nullptr), size_(0) {}

    ~LinkedStack() { clear(); }

    // 三法则：这个类自己管着一串 new 出来的结点，拷贝必须自己写。
    // 不写的话两个栈会共享同一串结点，各析构一次 → 二次释放。
    // 原书 lnkStack 有析构函数却没有这两个，与顺序栈是同一个错误。
    LinkedStack(const LinkedStack& other) : top_(nullptr), size_(0) {
        copy_from(other);
    }

    LinkedStack& operator=(const LinkedStack& other) {
        if (this == &other) {
            return *this;
        }
        clear();
```

```
        copy_from(other);
        return *this;
    }

    // 入栈：造一个新结点，让它指向原来的栈顶，再让栈顶指向它。
    // ** 没有「栈满」这回事 **——这正是链式栈相对顺序栈最大的差别。
    void push(const T& value) {
        Node* fresh = new Node;
        fresh->value = value;
        fresh->next = top_;
        top_ = fresh;
        ++size_;
    }

    // 出栈：把栈顶结点摘下来，取走它的值，再释放它。空栈返回空 optional。
    std::optional<T> pop() {
        if (empty()) {
            return std::nullopt;
        }
        Node* dying = top_;
        T value = dying->value;
        top_ = dying->next;
        delete dying;
        --size_;
        return value;
    }

    std::optional<T> top() const {
        if (empty()) {
            return std::nullopt;
        }
        return top_->value;
    }

    bool empty() const { return top_ == nullptr; }
    size_type size() const { return size_; }

    // 逐个释放结点。** 用循环，不要用递归 **——链长十万级时递归析构会把运行栈撑爆。
    // 第 5 章有实测数字。
    void clear() {
        while (top_ != nullptr) {
            Node* dying = top_;
            top_ = top_->next;
            delete dying;
        }
        size_ = 0;
    }
```

```
private:
    struct Node {
        T value;
        Node* next;
    };

    // 拷贝一串结点：原栈是「顶 → 底」，新栈也要按同样次序串起来，
    // 所以从原栈的顶开始走，每次把新结点接到上一个新结点的后面。
    void copy_from(const LinkedStack& other) {
        Node** tail = &top_;              // 指向「下一个新结点该挂在哪」
        for (Node* source = other.top_; source != nullptr; source = source->next) {
            Node* fresh = new Node;
            fresh->value = source->value;
            fresh->next = nullptr;
            *tail = fresh;
            tail = &fresh->next;
            ++size_;
        }
    }

    Node* top_;           // 栈顶结点；空栈时是 nullptr
    size_type size_;      // 结点个数
};
```

三处值得对着上一节看：

- **没有 `capacity`，也没有 `grow()`**。链式栈不需要连续空间，压多少个都不用扩容。顺序栈那一节大半篇幅在讲扩容，这里整段消失了。
- **代价在别处**：每个元素多带一根 `next` 指针（64 位机上 8 字节），而且结点分散在堆上，缓存局部性不如顺序栈的连续数组。3.3.1 节把这笔账算完。
- **`clear()` 用循环，不用递归**。链式结构最自然的写法是递归释放，但深链会耗尽运行栈。教学版的测试用 80 万个结点压栈再整体析构，正是为这条兜底：把 `clear()` 换成递归释放，AddressSanitizer 当场报 `stack-overflow`，回溯指到递归那一行。

原书【代码 3.4】的 `lnkStack` 有一处与顺序栈**完全相同**的错误：成员 `Link<T>* top` 与成员函数 `bool top(T&)` 重名，整个类编译不过。同一本书在两个存储结构上犯了同一个命名错误，两处都没有被编译器验证过。

另有两处值得指出：

- **构造函数是 `lnkStack(int defSize)`，而那个参数从未被使用**。链式栈不需要预设容量，这个参数是从 `arrStack(int size)` 照抄来的；更麻烦的是它没有默认构造函数，使用者被迫为一个无意义的参数编个数出来。
- **有 `~lnkStack(){ clear(); }` 却没有拷贝构造与拷贝赋值**。按本书章节顺序，这是第四次遇到同一个错误（第 2 章的顺序表、链表，本章的顺序栈，再加

上这里的链式栈；第 4 章的字符串还会再遇到一次）。教学版按三法则补齐了这两个。

3.1.3a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。工程版（`code/ch03/linked_stack/modern.hpp`）在教学版之上补三件事：移动语义、拷贝到一半失败时的清理、零拷贝的 `peek()`。

```
// 原书有 `~lnkStack(){ clear(); }` 却没有拷贝构造与拷贝赋值：
// 一次 `lnkStack<int> b = a;` 之后两个栈共享同一串结点，各自析构一次 → 二次释放。
// 与顺序栈、顺序表、链表、字符串是同一个错误，按章节顺序这是本书第四次遇到它。
LinkedStack(const LinkedStack& other) {
    // 用尾指针按原序逐个复制，不用递归拷贝（深链会爆栈，见 UNVERIFIED-RISKS.md）
    Node* source = other.top_;
    Node** tail = &top_;
    try {
        while (source != nullptr) {
            *tail = new Node(source->value, nullptr);
            tail = &(*tail)->next;
            ++size_;
            source = source->next;
        }
    } catch (...) {
        clear();  // 半截链必须自己收拾：构造函数抛出时析构函数不会运行
        throw;
    }
}

LinkedStack& operator=(const LinkedStack& other) {
    if (this != &other) {
        LinkedStack copy(other);
        swap(copy);
    }
    return *this;
}

LinkedStack(LinkedStack&& other) noexcept
    : top_(std::exchange(other.top_, nullptr)), size_(std::exchange(other.size_, 0))
    ↪  {}

LinkedStack& operator=(LinkedStack&& other) noexcept {
    if (this != &other) {
        clear();
        top_ = std::exchange(other.top_, nullptr);
        size_ = std::exchange(other.size_, 0);
    }
    return *this;
```

```
}

~LinkedStack() { clear(); }
```

比教学版多出来的那段 `try/catch` 值得说一句：**构造函数抛出时，这个对象的析构函数不会运行**——它还没构造完，语言不认为它存在。所以拷贝到一半失败时，已经接上去的那半截链必须在 `catch` 里自己收拾掉，否则就漏了。教学版没有这一段，代价是「元素拷贝抛异常时会漏结点」，与 3.1.2a 里顺序栈那条是同一笔账。

压栈与出栈的工程版：

```
/// 入栈：接一个新结点。** 没有" 栈满" 这回事 **——这正是链式栈相对顺序栈的差别，
/// 原书顺序栈那边要判 `top == mSize - 1` 并打印" 栈满溢出"。
void push(const T& item) { top_ = new Node(item, top_); ++size_; }
void push(T&& item) { top_ = new Node(std::move(item), top_); ++size_; }

/// 出栈。空栈返回 std::nullopt（D-001 §3c：空是可预期状态，不抛异常）。
[[nodiscard]] std::optional<T> pop() {
    if (top_ == nullptr) {
        return std::nullopt;
    }
    Node* dying = top_;
    std::optional<T> result(std::move(dying->value));
    top_ = dying->next;
    delete dying;
    --size_;
    return result;
}

/// 取栈顶副本；零拷贝的观望用 peek()（D-001 §3b）。
[[nodiscard]] std::optional<T> top() const {
    return top_ == nullptr ? std::nullopt : std::optional<T>(top_->value);
}

[[nodiscard]] const T* peek() const noexcept {
    return top_ == nullptr ? nullptr : &top_->value;
}
```

工程版另给了一个 `peek()`，与 3.1.2a 里的分工完全一样：`top()` 返回可以带走的副本，`peek()` 零拷贝但指针会失效。教学版只有 `top()`。

两版共同的一处取舍：`clear()` 与析构都写成**迭代**而非递归。链式结构最自然的写法是递归释放，但深链会耗尽运行栈——本机实测，把 `clear()` 换成递归后 40 万结点仍能过、80 万结点 ASan 报 `stack-overflow`；迭代版在 200 万结点下无恙。教学版的测试就把门槛定在 80 万。

3.1.4 表达式求值

后缀（逆波兰）表达式不需要括号，也不需要优先级规则：求值时**遇操作数压栈，遇操作符弹出两个算完再压回**，读到末尾时栈里剩下的唯一元素就是结果。这条主线本书一字未改，用的还是本章自己的 `ArrayStack<double>`。

先回顾人是怎么算中缀式的。原书给出三条计算次序：① 先算括号内、后算括号外，多层括号逐层去掉，左右括号必须配对；② 无括号或同层括号时，先乘除、后加减；③ 同一层次有多个乘除（或多个加减）时，自左至右依次执行。按这三条，`23 + (34 * 45) / (5 + 6 + 7)` 的计算过程是：① 34 × 45 = 1530；② 5 + 6 + 7 = 18；③ 1530 / 18 = 85；④ 23 + 85 = 108。（`34 * 45` 外的括号只是为了贴合原书的语法公式，去掉它语义不变。）

后缀式把这套次序直接写进了记号的排列里：操作数的先后与中缀式完全相同，运算符则按**实际计算的先后**排在它的两个操作数之后，括号随之消失。上式的后缀式是 `23 34 45 * 5 6 + 7 + / +`，求值过程如图 3.5。

步骤	待处理的后缀表达式（左→右）	栈的状态（底→顶）	进行的运算
1	`23 34 45 * 5 6 + 7 + / +`		
2	`34 45 * 5 6 + 7 + / +`	23	
3	`45 * 5 6 + 7 + / +`	23 34	
4	`* 5 6 + 7 + / +`	23 34 45	
5	`5 6 + 7 + / +`	23 1530	34 × 45 = 1530 入栈
6	`6 + 7 + / +`	23 1530 5	
7	`+ 7 + / +`	23 1530 5 6	
8	`7 + / +`	23 1530 11	5 + 6 = 11 入栈
9	`+ / +`	23 1530 11 7	
10	`/ +`	23 1530 18	11 + 7 = 18 入栈
11	`+`	23 85	1530 / 18 = 85 入栈
12		108	23 + 85 = 108 入栈

图 3.5　后缀表达式的计算过程。第 11 步要留意次序：先弹出的是**右**操作数（18），后弹出的才是左操作数（1530），所以算的是 1530 / 18 而不是 18 / 1530。减法和除法都不可交换，弹出次序写反，结果就错。

```
/// 对后缀（逆波兰）表达式求值。记号之间用空白分隔，例如 "3 4 + 2 *" → 14。
///
/// 与原书 `class Calculator` 的差别，逐条对应它的三个问题：
///
/// 1. ** 原书 `Run()` 直接从 `cin` 读、往 `cout` 写 **，算法与终端焊死：没法写测试、
///    没法在库里复用、没法处理来自别处的表达式。这里接受一个 `string_view`、返回结果。
/// 2. ** 原书 `GetTwoOperands` 在第二个操作数缺失时已经把第一个弹掉了 **，
///    返回 false 时栈已被破坏（legacy.md 缺陷 1）。
```

```
///    但要说准确：真正让这个 bug 无害的，** 不是 ** 下面那句" 先查够不够"，
///    而是 ** 出错即抛出、整次求值随即作废 **——栈根本没有机会被下一步用到。
///    预检查只是让错误信息更早、更准，属于锦上添花。
///    原书的 bug 之所以要命，是因为它 `cerr` 一行之后 ** 继续跑 **。
/// 3. ** 原书出错时 `cerr` 打一行然后 `s.clear()` 继续跑 **，调用方拿不到任何信号。
///    这里抛 `std::invalid_argument`（表达式不合法）或 `std::domain_error`（除零）。
[[nodiscard]] inline double evaluate_postfix(std::string_view expression) {
    ArrayStack<double> operands;
    std::size_t i = 0;

    const auto pop_two = &operands {
        // 先确认有两个再弹。注意这不是" 修复" 原书 bug 的关键（见函数注释第 2 条），
        // 而是让报错更早、更准。
        if (operands.size() < 2) {
            throw std::invalid_argument(" 后缀表达式：操作数不足");
        }
        right = *operands.pop();  // 先弹出的是右操作数
        left = *operands.pop();
    };

    while (i < expression.size()) {
        const char c = expression[i];
        if (c == ' ' || c == '\t' || c == '\n') {
            ++i;
            continue;
        }

        // 操作符：+ - * / 各弹两个。注意 '-' 也可能是负号，靠后面是不是数字来区分。
        const bool is_sign = (c == '-' || c == '+') && i + 1 < expression.size()
                             && (std::isdigit(static_cast<unsigned char>(expression[i
                               ↪  + 1]))
                                 || expression[i + 1] == '.');
        if (!is_sign && (c == '+' || c == '-' || c == '*' || c == '/')) {
            double left = 0.0;
            double right = 0.0;
            pop_two(left, right);
            switch (c) {
                case '+': operands.push(left + right); break;
                case '-': operands.push(left - right); break;
                case '*': operands.push(left * right); break;
                default:
                    // 原书写 `if (operand1 == 0.0)`，正文又说这样比较浮点数不对、
                    // 该用阈值。两者都值得推敲：除以 1e-300 是 ** 合法 ** 的，
                    // 结果是个很大的数或 inf，用阈值把它当成错误是另一个决定。
                    // 这里只拦精确的 0.0（含 -0.0），并把这个取舍写在明面上。
                    if (right == 0.0) {
                        throw std::domain_error(" 后缀表达式：除数为零");
```

```
                    }
                    operands.push(left / right);
                    break;
                }
                ++i;
                continue;
            }

            // 操作数：交给标准库解析，顺便拿到它吃掉了多少字符
            std::size_t consumed = 0;
            double value = 0.0;
            try {
                value = std::stod(std::string(expression.substr(i)), &consumed);
            } catch (const std::exception&) {
                throw std::invalid_argument(std::string(" 后缀表达式：无法识别的记号 '") +
                ↪  c + "'");
            }
            operands.push(value);
            i += consumed;
        }

        if (operands.size() != 1) {
            // 空表达式、操作数多余、操作符不足，都落在这里
            throw std::invalid_argument(" 后缀表达式：不是一个完整的表达式");
        }
        return *operands.pop();
    }
```

原书【算法 3.5】把它写成一个 `class Calculator`，有三处今天必须改。

一、算法与终端焊死。 `Run()` 直接 `cin >> c` 读、`cout << res` 写，出错时 `cerr` 打一行。后果是这个算法没法写测试、没法在库里复用、也没法处理来自别处的表达式。本书改为接受一个字符串、返回结果、出错抛异常。

二、`GetTwoOperands` 在第二个操作数缺失时，第一个已经被弹掉了。栈就此少一个元素。但要说准确——**真正让这件事变成 bug 的是调用方继续跑**：`Compute` 拿到 `false` 之后清空栈然后接着读下一个记号，`Run()` 的循环照转不误。

这一点改变了”怎样才算修好”：本书的实现**出错即抛出、整次求值作废**，栈根本没有机会被下一步用到。（写这一节时验证过：把实现改回”先弹一个再查”，全部断言依然通过——因为在抛异常即作废的设计里，栈被破坏与否观测不到。于是补了一条测试，钉住”上一次失败不给下一次留残留”这条真正可测的性质。）

三、除零判断。原书正文自己写道：

> 在实际编写程序时，浮点数是否为 0 不能直接与 0 进行相等比较，而是要通过某个很小的阈值来判断，例如采用 `if(abs(operand1) < 1E-7)`

但印出来的代码仍然是 `if (operand1 == 0.0)`。书知道更好的写法，却没有

印它。

不过原书这条建议本身也值得推敲：**除以 1e-300 是合法的**，结果是一个极大的有限值或无穷，把它当作错误是另一个决定，不是" 更正确"。本书只拦精确的零，并把这个取舍写在代码注释里；测试中有一条断言专门钉住 "除以 1e-300 得到极大值而非报错"。

中缀表达式到后缀表达式的转换

上面那个求值器有个前提：**表达式已经是后缀式了**。可人写出来的是中缀式 `23 + (34 * 45) / (5 + 6 + 7)`，中间隔着一步转换。这一步才是**栈最典型的应用**—— 括号和优先级造成的「先算什么」，全靠一把栈记住。编译器把算术式变成机器指令，走的就是同一条路。

原书把规则讲全了，却把代码留成了练习：「上面仅给出了算法的梗概和思路，其程序实现 涉及字符符号读入、语法检查以及语法错误处理等细节，有兴趣的读者可作为练习给出具体的 算法」。**本书补上这个练习**，因为不补的话，第 3.1.4 节就只剩下半个应用。

原书的五条规则，从左到右扫描中缀式：

遇到	做什么
操作数	直接输出到后缀序列
`(`	入栈
`)`	反复弹出并输出，直到遇到 `(`；`(` 弹掉但**不输出**。没遇到 `(` 就是括号不匹配
运算符	当「栈非空 且 栈顶不是 `(` 且 **栈顶优先级不低于**当前」时反复弹出并输出；然后把当前运算符入栈
扫描结束	栈里剩下的依次弹出输出；若弹出 `(`，说明括号不匹配

第四条里「不低于」三个字是承重的。如果写成「高于」，同优先级时就不先弹，`1 - 2 - 3` 会转成 `1 2 3 - -`，求值得 $1-(2-3)=2$；而正确答案是左结合的 $(1-2)-3=-4$。测试里有一条专门算这个数，把 `<` 写成 `<=` 它立刻变红。

拿原书的例子 `23 + (34 * 45) / (5 + 6 + 7)` 逐个记号走一遍（栈只放运算符和开括号）：

读入	用到的规则	运算符栈（底→顶）	已输出的后缀序列
`23`	操作数，直接输出		`23`
`+`	栈空，直接入栈	`+`	`23`

读入	用到的规则	运算符栈（底→顶）	已输出的后缀序列
`(`	开括号入栈	`+ (`	`23`
`34`	操作数	`+ (`	`23 34`
`*`	栈顶是 `(`，停，入栈	`+ ( *`	`23 34`
`45`	操作数	`+ ( *`	`23 34 45`
`)`	弹出 `*` 输出，弹掉 `(`	`+`	`23 34 45 *`
`/`	栈顶 `+` 优先级低于 `/`，停，入栈	`+ /`	`23 34 45 *`
`(`	开括号入栈	`+ / (`	`23 34 45 *`
`5`	操作数	`+ / (`	`23 34 45 * 5`
`+`	栈顶是 `(`，停，入栈	`+ / ( +`	`23 34 45 * 5`
`6`	操作数	`+ / ( +`	`23 34 45 * 5 6`
`+`	栈顶 `+` 与当前同级（不低于），弹出输出；再遇 `(` 停，入栈	`+ / ( +`	`23 34 45 * 5 6 +`
`7`	操作数	`+ / ( +`	`23 34 45 * 5 6 + 7`
`)`	弹出 `+` 输出，弹掉 `(`	`+ /`	`23 34 45 * 5 6 + 7 +`
结束	依次弹出 `/`、`+` 输出		`23 34 45 * 5 6 + 7 + / +`

读入括号里第二个 `+` 的那一行（第 13 行）就是「不低于」在起作用：`5 + 6` 的 `+` 先于后来的 `+` 输出，保证了左结合。最后一行先弹出的是栈顶的 `/`，再弹出 `+`——除法确实要在最外层的加法之前做。

```
/// 把中缀表达式转换成等价的后缀表达式。例如 "23 + (34 * 45) / (5 + 6 + 7)"
/// → "23 34 45 * 5 6 + 7 + / +"。记号之间用单个空格分隔。
///
/// ** 原书把这个算法留成了练习 **（3.1.4 节那段：「上面仅给出了算法的梗概和思路，
/// 其程序实现涉及字符符号读入、语法检查以及语法错误处理等细节，有兴趣的读者可作为
/// 练习给出具体的算法」）。本书补上，因为它是 ** 栈最典型的一个应用 **：
/// 括号和优先级造成的「先算什么」全靠一把栈记住。
///
/// 算法就是原书那五条，逐条对应下面的分支：
///
```

```
///   (1) 操作数          → 直接输出到后缀序列
///   (2) 开括号 `(`      → 入栈
///   (3) 闭括号 `)`      → 反复弹出并输出，直到遇到开括号；开括号弹掉但不输出。
///                         没遇到开括号就说明括号不匹配
///   (4) 运算符          → 当「栈非空 且 栈顶不是开括号 且 栈顶优先级不低于当前」时，
///                         反复弹出并输出；然后把当前运算符入栈
///   (5) 扫描结束        → 栈里剩下的依次弹出输出；若弹出的是开括号，说明括号不匹配
///
/// 第 (4) 条那个「不低于」是关键：同优先级时也要先弹，这样 `a - b - c` 才会变成
/// `a b - c -`（左结合），而不是 `a b c - -`。
///
/// 与 evaluate_postfix 一样，出错抛 std::invalid_argument，不打印任何东西。
[[nodiscard]] inline std::string infix_to_postfix(std::string_view expression) {
    // 只有四则运算：+ - 同级，* / 同级且更高。开括号在栈里的优先级最低，
    // 这样第 (4) 条的循环碰到它自然会停——但仍要单独判，因为它不参与输出。
    const auto precedence = [](char op) -> int {
        return (op == '*' || op == '/') ? 2 : 1;
    };

    std::string output;
    ArrayStack<char> operators;

    const auto emit = &output {
        if (!output.empty()) {
            output.push_back(' ');
        }
        output.append(token);
    };

    std::size_t i = 0;
    bool expect_operand = true;  // 用来把 "-3" 的负号和二元减号区分开
    while (i < expression.size()) {
        const char c = expression[i];
        if (c == ' ' || c == '\t' || c == '\n') {
            ++i;
            continue;
        }

        if (c == '(') {                                    // (2)
            operators.push(c);
            expect_operand = true;
            ++i;
            continue;
        }

        if (c == ')') {                                    // (3)
            bool matched = false;
```

```
            while (const char* top = operators.peek()) {
                const char op = *operators.pop();
                if (op == '(') {
                    matched = true;
                    break;
                }
                emit(std::string_view(&op, 1));
                (void)top;
            }
            if (!matched) {
                throw std::invalid_argument(" 中缀表达式：右括号没有配对的左括号");
            }
            expect_operand = false;
            ++i;
            continue;
        }

        const bool is_sign = (c == '-' || c == '+') && expect_operand;
        if (!is_sign && (c == '+' || c == '-' || c == '*' || c == '/')) {   // (4)
            while (const char* top = operators.peek()) {
                if (*top == '(' || precedence(*top) < precedence(c)) {
                    break;
                }
                const char op = *operators.pop();
                emit(std::string_view(&op, 1));
            }
            operators.push(c);
            expect_operand = true;
            ++i;
            continue;
        }

        // (1) 操作数。用和 evaluate_postfix 同一套解析，两者才好对拍。
        std::size_t consumed = 0;
        try {
            (void)std::stod(std::string(expression.substr(i)), &consumed);
        } catch (const std::exception&) {
            throw std::invalid_argument(std::string(" 中缀表达式：无法识别的记号 '") +
            ↪  c + "'");
        }
        emit(expression.substr(i, consumed));
        expect_operand = false;
        i += consumed;
    }

    while (const char* top = operators.peek()) {                            // (5)
        if (*top == '(') {
```

```
            throw std::invalid_argument(" 中缀表达式：左括号没有配对的右括号");
        }
        const char op = *operators.pop();
        emit(std::string_view(&op, 1));
    }

    if (output.empty()) {
        throw std::invalid_argument(" 中缀表达式：空表达式");
    }
    return output;
}

/// 直接对中缀表达式求值：先转成后缀，再用【算法 3.5】求值。
/// 两步都在上面，这里只是把它们接起来——** 这正是转换算法存在的理由 **。
[[nodiscard]] inline double evaluate_infix(std::string_view expression) {
    return evaluate_postfix(infix_to_postfix(expression));
}
```

转换和求值接起来就是 `evaluate_infix`——**这正是转换算法存在的理由**。测试里有一组 10 个式子做三方对拍：转成后缀再求值、直接对中缀求值、手算的期望值，三者必须一致。这比记住某个具体输出串硬得多。

原书那个例子的结果也写进了断言：

```
中缀  23 + (34 * 45) / (5 + 6 + 7)
后缀  23 34 45 * 5 6 + 7 + / +
```

括号不匹配的两种情形（右括号多了、左括号多了）分别对应规则表里的第三条和第五条，各有一条用例；两者都抛 `std::invalid_argument`，不打印任何东西。

3.1.5 栈与递归

原书所说的 “活动记录” 在现代编译器与运行时资料中通常称为**栈帧**（stack frame）：其中保存返回地址、参数、局部变量及必要的保存寄存器。名称不同，描述的是同一类调用栈记录。

递归为什么非要有栈？因为被调函数的局部变量不能静态地占一块固定单元——每调用一次就得有一份，返回时随即释放。这种「执行到调用时才分配」叫动态分配，它需要内存里有一块足够大的**运行栈** (runtime stack)。运行时存储通常这样划分：

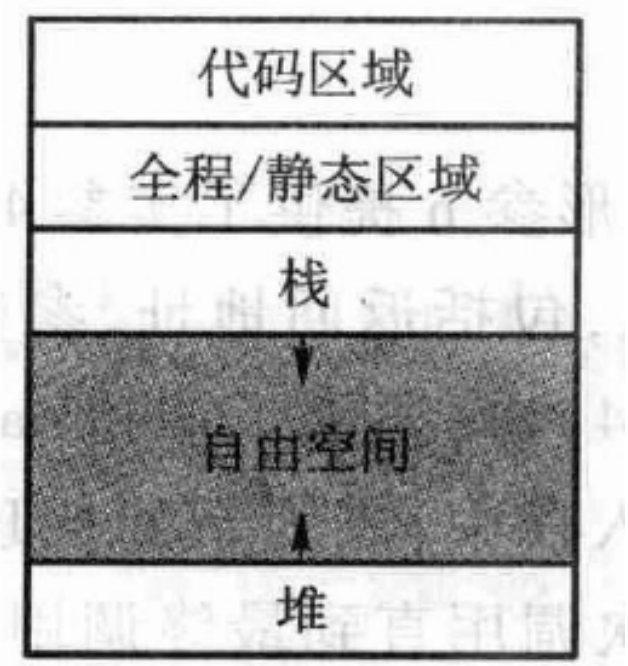


图 3.6　运行时存储器的组织形式：栈区放具有后进先出特征的数据（函数调用），堆区放不符合 LIFO 的动态分配（例如 new 出来的对象）。

栈区里一格一格压着的就是活动记录，它至少包含这些内容：

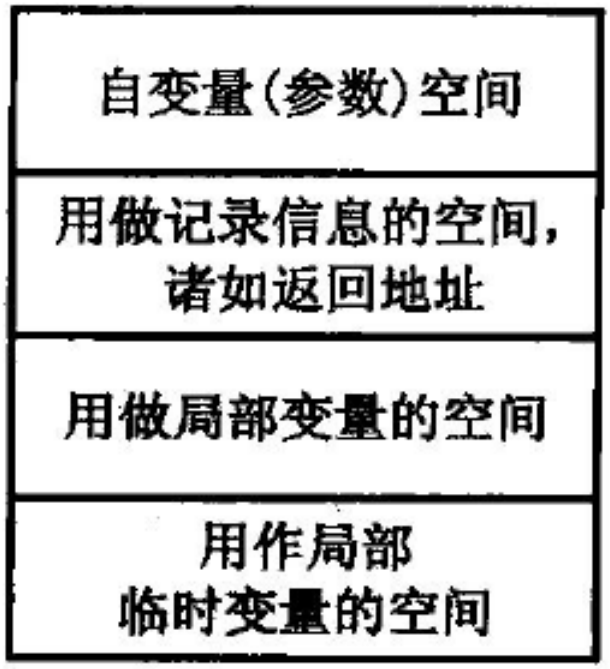


图 3.7　活动记录的内容。调用一次压一格，返回一次弹一格；被调函数里的变量地址因此都是相对于栈顶的相对地址。运行栈也因此又叫活动记录栈或调用栈 (call stack)。

概括起来，一次函数调用分三步：(1) 调用方发送调用信息——传给形式参数的实际参数值、返回地址等；(2) 为被调方分配局部数据区，存放它的局部变量、形参和返回地址，并接收调用信息；(3) 调用方暂停，控制转到被调函数的入口。返回时也分三步：(1) 传回返回信息，例如计算结果；(2) 释放被调方的数据区；(3) 按返回地址把控制转回调用方。调用的第 (2) 步就是一次压栈，返回的第 (2) 步就是一次弹栈。

同一个函数可以在栈上同时有很多活动记录，每个代表一次不同的调用——对递归函数来说，**递归深度就是它在运行栈里活动记录的个数**，同一个局部变量在不同层次被分到不同的存储空间。以 `factorial(4)` 为例：

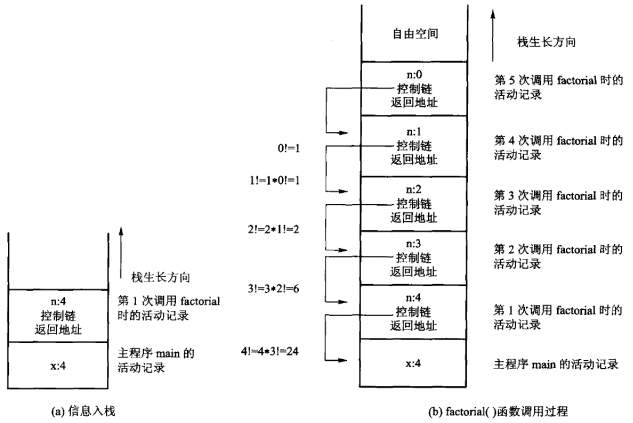


图 3.8　计算 `factorial(4)` 时运行栈的变化：(a) 逐层调用，`factorial(4)` → `factorial(3)` →...→ `factorial(0)` 依次压栈；(b) `factorial(0)` 满足出口条件直接返回，随后按压栈的反序逐层弹出，把结果和控制权一层层交回去，最后 `factorial(4)` 把 24 交还给 `main()`。

这张图也解释了下面那组实测数字：每深一层就多一格活动记录，深度一旦超过栈区的大小，程序不是「变慢」，而是直接崩。

> **本节开篇先摆一组实测数字**。原书正文说递归「需要在内存中开辟一个称为 运行栈 (runtime stack) 的**足够大**的动态区」。多大算足够大？这是可以量的，而量出来的结果里有一条相当反直觉。

运行栈有多大：三个档位，三种结果

同一份递归源码（每层做一次加法后返回），在一台 Linux 机器上（gcc 13.3，`ulimit -s` 为默认的 8 MB）逐档加深度：

构建档	20 万层	50 万层	100 万层
`-O0`（不优化）	通过	**段错误**	段错误

构建档	20 万层	50 万层	100 万层
`-O1` + ASan/UBSan	通过	stack-overflow	stack-overflow
`-O2`	通过	通过	**通过**

把同样的计算改成显式栈（数据压进本章的 `ArrayStack`，也就是放到堆上）：三个档位在 1000 **万层**都通过。

反直觉的那一条：`-O2` 为什么不崩

不是因为它栈更大，而是因为**编译器把递归消掉了**。查汇编可以确认：

```
-O0: recursive_sum 函数体内调用自己的次数 = 2
-O2: recursive_sum 函数体内调用自己的次数 = 0
```

`-O2` 下这个函数根本没有自调用——它被转成了循环。所以：

> **「这段递归会不会爆栈」不是源码单独决定的，是源码 × 编译器 × 优化档共同决定的。**

这件事对学习者的实际含义是：在 `-O2` 下跑通的递归程序，换成 `-O0` 调试、或者换个编译器、或者递归形状稍微复杂到无法被转换，就可能在同样的输入上崩掉。而崩掉时你看到什么，也取决于构建方式——开了 sanitizer 的构建会明确告诉你 `stack-overflow` 并给出递归回溯，`-O0` 的构建只有一个段错误，**一行解释都没有**。

递归吃运行栈，显式栈吃堆

这正是本节的正题。原书用阶乘作例子，给了三个版本：

【算法 3.6】递归——每一层的返回地址与局部变量都压在运行栈上，深度由进程栈上限决定：

```cpp
/// 【算法 3.6】递归实现。保留原书的递归形状——那正是本节要教的东西。
///
/// 加了原书没有的两道检查：负数是定义域错误，溢出是真错误（D-001 §3）。
/// 原书 `if (n <= 0) return 1;` 把负数静默当成 0 处理，返回 1。
[[nodiscard]] inline factorial_type factorial_recursive(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围（20! 是上限）");
    }
    if (n <= 1) {
        return 1;  // 递归出口
    }
    return static_cast<factorial_type>(n) * factorial_recursive(n - 1);
```

```
}
```

【算法 3.8】迭代——不用栈，也不占深度：

```
/// 【算法 3.8】迭代实现。不用栈，也不占运行栈深度。
[[nodiscard]] inline factorial_type factorial_iterative(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围（20! 是上限）");
    }
    factorial_type m = 1;
    for (long long i = 2; i <= n; ++i) {
        m *= static_cast<factorial_type>(i);
    }
    return m;
}
```

【算法 3.9】显式栈——把待处理的数据压进一个自己管理的栈，用原书的话说是「模拟编译系统处理递归的机制，使用栈等数据结构保存回溯点」：

```
/// 【算法 3.9】用显式栈模拟递归。
///
/// 这一版存在的意义不是" 更快"——它比迭代版慢——而是 ** 演示编译系统处理递归的机制 **：
/// 遇到递归规则就压栈，遇到递归出口就出栈返回。原书的话是
/// 「模拟编译系统处理递归的机制，使用栈等数据结构保存回溯点」。
///
/// 关键差别在 ** 数据放在哪 **：递归版把每层的返回地址与局部变量放在 ** 运行栈 ** 上，
/// 大小由进程栈上限决定；这一版把待处理的数据压进 ArrayStack，** 在堆上 **，
/// 只受内存限制。实测数字见书稿 3.1.5 节。
///
/// 原书写的是 `while (s.pop(&tmp))`——传的是 ** 指针 **，而同书代码 3.1 的栈 ADT
/// 声明的是 `bool pop(T& item)`（引用）。两处对不上（legacy.md 缺陷 3）。
[[nodiscard]] inline factorial_type factorial_with_explicit_stack(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围（20! 是上限）");
    }
    ArrayStack<factorial_type> pending;
    for (long long i = n; i > 1; --i) {  // 按递归规则压栈
        pending.push(static_cast<factorial_type>(i));
    }
    factorial_type m = 1;  // 递归出口的返回值
```

```
    while (auto top = pending.pop()) {   // 出栈即" 递归返回"
        m *= *top;
    }
    return m;
}
```

以 `factorial_with_explicit_stack(4)` 为例，栈的变化是这样的（循环条件是 `i > 1`，所以 1 不入栈——乘 1 不改变结果，`m` 的初值 1 已经代表了递归出口的返回值）：

阶段	操作	栈（底→顶）	m
压栈（相当于逐层递归调用）	push 4, push 3, push 2	4 3 2	1
出栈（相当于逐层返回）	pop 2，m = 1 × 2	4 3	2
	pop 3，m = 2 × 3	4	6
	pop 4，m = 6 × 4	空	24

压栈的次序与图 3.8(a) 逐层调用的次序相同，出栈的次序与图 3.8(b) 逐层返回的次序相同。

三个版本的差别不在快慢（显式栈版最慢），而在**数据放在哪**：递归版的数据在运行栈上，受进程栈上限约束；显式栈版的数据在堆上，只受内存约束；迭代版只用一个变量 `m`，根本不需要栈。上面那张表量的就是前两者的差别。

原书这三个版本共同的问题：不查溢出

三个版本都是 `long factorial(long n)`，既不检查负数也不检查溢出。64 位 `long` 装得下的最大阶乘是 20!，从 21 开始：

```
factorial(20) = 2432902008176640000
factorial(21) = -4249290049419214848      ← 负数
factorial(66) = 0                          ← 零
factorial(-5) = 1                          ← 负数静默当成 0 处理
```

而且这不只是" 答案错"——有符号整数溢出在 C++ 里是**未定义行为**：

```
runtime error: signed integer overflow: 21 * 2432902008176640000
               cannot be represented in type 'long int'
```

本书改用 `std::uint64_t` 并在入口显式判界：超过 20 抛 `std::overflow_error`，负数抛 `std::invalid_argument`。三个版本在边界上的行为完全一致—— 测试要求它们对同一输入给出相同答案，包括同样地抛出异常。

（另有一处书内不一致：算法 3.9 写 `s.pop(&tmp)` 传指针，而同章代码 3.1 的栈 ADT 声明的是 `bool pop(T& item)` 传引用，两处配不上。详见 `code/ch03/recursion_and_stack/legacy.md`。）

一个更复杂的例子：背包问题

阶乘只有一**条**递归规则，改写成循环几乎是显然的。原书接着给了一个有**两条**递归规则的例子——背包问题（更准确地说是子集和判定：能否从若干物品中选出一部分，使重量之和恰好等于背包承重）。

```cpp
/// 【算法 3.10】递归解法。两条递归规则、两个递归出口，原书的结构一字未改：
///   出口 1：承重恰为 0 → 有解（什么都不再选）
///   出口 2：承重为负，或承重为正但已无物品可选 → 无解
///   规则 1：选第 n-1 件 → 求解 knap(s - w[n-1], n-1)
///   规则 2：不选第 n-1 件 → 求解 knap(s, n-1)
///
/// 与原书的差别只有两处：
/// 1. ** 原书直接 `cout << w[n-1]`** 把选中的物品打印出来——算法与终端焊死，
///    调用方拿不到结果，也没法测试。这里把下标收集进 `chosen` 返回。
/// 2. ** 原书的 `w[]` 是一个从未声明的全局数组 ** (legacy.md 缺陷 3)。这里作参数传入。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_recursive(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);
    knapsack_solution chosen;

    // 返回 true 表示 weights0..n) 中存在一个子集，其和恰为 s
    const auto solve = [&weights, &chosen -> bool {
        if (s == 0) {
            return true;  // 递归出口 1
        }
        if (s < 0 || n == 0) {
            return false;  // 递归出口 2
        }
        if (self(self, s - weights[n - 1], n - 1)) {  // 规则 1：选它
            chosen.push_back(n - 1);
            return true;
        }
        return self(self, s, n - 1);  // 规则 2：不选它
    };

    return solve(solve, capacity, weights.size())
               ? std::optional<knapsack_solution>(std::move(chosen))
               : std::nullopt;
}
```

```python
def knapsack_recursive(capacity: int, weights: list[int]) -> list[int] | None:
    """ 算法 3.10：两条递归规则，返回选中物品的下标。"""
    _validate(capacity, weights)
    chosen: list[int] = []
```

```
    def solve(remaining: int, count: int) -> bool:
        if remaining == 0:
            return True
        if remaining < 0 or count == 0:
            return False
        if solve(remaining - weights[count - 1], count - 1):
            chosen.append(count - 1)
            return True
        return solve(remaining, count - 1)

    return chosen if solve(capacity, len(weights)) else None
```

两个递归出口、两条递归规则，本书一字未改。改的是两处：原书 `w[]` 是一个**从未声明的全局数组**，这里作参数传入；原书在选中物品时 `cout << w[n-1]` 直接打印，调用方拿不到解、正确性也无从检验，这里把下标收集起来返回——测试因此可以独立验算：**把返回的下标对应的重量加起来，必须恰好等于承重**。

把它机械地改写成循环

原书的做法是引入一个显式栈，每帧保存四个域：参数 s 与 n、**返回地址** rd、结果单元 k。返回地址是关键——它记的是”这一层算完之后该回到哪一步继续”，正是编译器为你做的那件事。原书用 `goto label0/1/2/3` 表达它。

本书保留这套机制，但把”返回地址”写成栈帧里的一个 `stage` 字段：

```
/// 【算法 3.11】把上面的递归机械地改写成显式栈驱动。
///
/// 原书用 `goto label0/1/2/3` 表示" 执行到哪一步"，本书把同一件事写成栈帧里的
/// 一个 `stage` 字段——** 语义完全对应 **，只是不用 goto：goto 跳进跳出会让编译器
/// 无法保证局部对象的构造与析构配对，在有 RAII 的 C++ 里不能这么写。
///
///   `Enter`      ↔ label0，递归调用入口：判出口，否则按规则 1 展开
///   `AfterRule1` ↔ label1，规则 1（选第 n-1 件）返回后的处理
///   `AfterRule2` ↔ label2，规则 2（不选第 n-1 件）返回后的处理
///
/// 每一帧存原书说的四个域：参数 s 与 n、返回地址（这里是 stage）、结果单元 k。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_with_explicit_stack(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);

    enum class Stage { Enter, AfterRule1, AfterRule2 };
    struct Frame {
        int s = 0;
        std::size_t n = 0;
        Stage stage = Stage::Enter;
    };
```

```cpp
    ArrayStack<Frame> stack;
    stack.push(Frame{capacity, weights.size(), Stage::Enter});
    knapsack_solution chosen;
    bool child_result = false;   // 下层刚刚返回的结果单元 k

    while (!stack.empty()) {
        Frame frame = *stack.pop();

        if (frame.stage == Stage::Enter) {
            if (frame.s == 0) {            // 递归出口 1
                child_result = true;
                continue;                  // 相当于 goto label3：直接向上返回
            }
            if (frame.s < 0 || frame.n == 0) {   // 递归出口 2
                child_result = false;
                continue;
            }
            frame.stage = Stage::AfterRule1;     // 记下" 回来时该走哪一步"
            stack.push(frame);
            stack.push(Frame{frame.s - weights[frame.n - 1], frame.n - 1,
↪   Stage::Enter});
            continue;
        }

        if (frame.stage == Stage::AfterRule1) {
            if (child_result) {            // 规则 1 成功：第 n-1 件被选中
                chosen.push_back(frame.n - 1);
                continue;                  // k 已是 true，继续上传
            }
            frame.stage = Stage::AfterRule2;     // 回溯，改用规则 2
            stack.push(frame);
            stack.push(Frame{frame.s, frame.n - 1, Stage::Enter});
            continue;
        }

        // Stage::AfterRule2：规则 2 的结果就是本层的结果，原样上传
    }

    return child_result ? std::optional<knapsack_solution>(std::move(chosen)) :
    ↪   std::nullopt;
}
```

```python
def knapsack_with_explicit_stack(capacity: int, weights: list[int]) -> list[int] |
↪   None:
    """ 算法 3.11：用栈帧中的返回地址机械模拟递归。"""
```

```
    _validate(capacity, weights)

    # ------------------------------------------------------------
    # 1. 状态定义（模拟 CPU 的“程序计数器” PC / 返回地址 Return Address）
    # enter      : 函数入口点（开始执行当前层的逻辑）
    # after_rule1: 从“选择当前物品”的子调用返回后的恢复点
    # after_rule2: 从“不选当前物品”的子调用返回后的恢复点
    # ------------------------------------------------------------
    enter, after_rule1, after_rule2 = range(3)

    # 显式模拟调用栈：
    # 栈帧与递归版 solve(remaining, count) 的参数一一对应，再加上返回地址：
    # (当前剩余容量 remaining, 可选物品数 count, 当前执行阶段 stage)
    stack = [(capacity, len(weights), enter)]

    # 记录最终被选中的物品索引（构成解的路径）
    chosen: list[int] = []

    # 模拟 CPU 的“返回值寄存器”（例如 x86 中的 RAX/EAX）
    # 子调用执行完毕后，将结果（True/False）写入此变量，交由父调用读取
    child_result = False

    while stack:
        # 弹出当前栈顶执行上下文，恢复该层的全部“局部变量”
        # count 表示当前还有前 count 个物品可选（对应下标 0 到 count - 1）
        remaining, count, stage = stack.pop()

        # ==================== 阶段 0：函数入口 ====================
        if stage == enter:
            # Base Case 1: 恰好凑齐容量，递归成功
            if remaining == 0:
                child_result = True
            # Base Case 2: 超过承重，或已无物品可选，递归失败
            elif remaining < 0 or count == 0:
                child_result = False
            else:
                # 分支 1：尝试“选择”第 (count - 1) 个物品
                # [压栈操作 1 - 保存断点]：子调用结束后，需回到 after_rule1 处继续
                stack.append((remaining, count, after_rule1))
              # [压栈操作 2 - 发起子调用]：扣减当前物品重量，可选物品数减 1，进入新的 enter
                stack.append((remaining - weights[count - 1], count - 1, enter))

        # ==================== 阶段 1：分支 1 返回 ====================
        elif stage == after_rule1:
            # 从“返回值寄存器”读取分支 1 的成败
            if child_result:
                # 分支 1 成功：说明拿该物品能凑齐，将其记录进结果集中
```

```
                chosen.append(count - 1)
                # 当前层任务完成，直接结束，准备弹出更上一层的断点
            else:
                # 分支 1 失败：回溯，转入分支 2——尝试“不选”第 (count - 1) 个物品
                # [压栈操作 1 - 保存断点]：分支 2 结束后恢复到 after_rule2
                stack.append((remaining, count, after_rule2))
                # [压栈操作 2 - 发起子调用]：容量不变，可选物品数减 1，发起新调用
                stack.append((remaining, count - 1, enter))

        # ==================== 阶段 2：分支 2 返回 ====================
        # elif stage == after_rule2:
        #     pass
        # 说明：分支 2 的结果已经直接留存在 child_result 里了。
        # 这里无需额外代码，当前帧直接随 pop() 销毁，继续回溯上层。

    # 整个搜索结束后，若根调用成功则返回结果集，否则返回 None
    return chosen if child_result else None
```

`Enter` / `AfterRule1` / `AfterRule2` 与原书的 label0 / label1 / label2 一一对应。**不用 goto 的理由是硬的**：goto 跳进跳出会让编译器无法保证局部对象的构造与析构 配对，在有 RAII 的 C++ 里不能这么写。

光看代码不容易想象栈怎样起落，拿一个小例子手算一遍：承重 $s = 5$，三件物品重量 `w = {2, 3, 4}`。每个栈帧写成 (s, n, stage)，stage 简记为 E（`Enter`）、R1（`AfterRule1`）、R2（`AfterRule2`）；k 列是栈外那个结果单元 `child_result`。

步	弹出的帧	做了什么	k	处理后的栈（底→顶）
1	(5,3,E)	规则 1：试选 w[2]=4	假	(5,3,R1) (1,2,E)
2	(1,2,E)	规则 1：试选 w[1]=3	假	(5,3,R1) (1,2,R1) (−2,1,E)
3	(−2,1,E)	出口 2：承重为负，无解	假	(5,3,R1) (1,2,R1)
4	(1,2,R1)	规则 1 失败，回溯改用规则 2：不选 w[1]	假	(5,3,R1) (1,2,R2) (1,1,E)
5	(1,1,E)	规则 1：试选 w[0]=2	假	(5,3,R1) (1,2,R2) (1,1,R1) (−1,0,E)

步	弹出的帧	做了什么	k	处理后的栈（底→顶）
6	(− 1,0,E)	出口 2：无解	假	(5,3,R1) (1,2,R2) (1,1,R1)
7	(1,1,R1)	规则 1 失败，改用规则 2：不选 w[0]	假	(5,3,R1) (1,2,R2) (1,1,R2) (1,0,E)
8	(1,0,E)	出口 2：已无物品，无解	假	(5,3,R1) (1,2,R2) (1,1,R2)
9	(1,1,R2)	规则 2 的结果原样上传	假	(5,3,R1) (1,2,R2)
10	(1,2,R2)	规则 2 的结果原样上传	假	(5,3,R1)
11	(5,3,R1)	规则 1 失败（选 4 凑不成），改用规则 2：不选 w[2]	假	(5,3,R2) (5,2,E)
12	(5,2,E)	规则 1：试选 w[1]=3	假	(5,3,R2) (5,2,R1) (2,1,E)
13	(2,1,E)	规则 1：试选 w[0]=2	假	(5,3,R2) (5,2,R1) (2,1,R1) (0,0,E)
14	(0,0,E)	出口 1：承重恰为 0，有解	真	(5,3,R2) (5,2,R1) (2,1,R1)
15	(2,1,R1)	规则 1 成功，记下下标 0	真	(5,3,R2) (5,2,R1)
16	(5,2,R1)	规则 1 成功，记下下标 1	真	(5,3,R2)
17	(5,3,R2)	规则 2 的结果原样上传	真	空

栈空时 k 为真，选中的下标是 {0, 1}，即 2 + 3 = 5；递归版、显式栈版、下

面的优化版三者给出的 都是这个答案。留意两件事：每一层在栈里**始终只占一帧**（进入时是 E，展开后换成 R1，回溯后换成 R2），所以栈深就是递归深度；第 4、7、11 步的「回溯」，正是递归版里 `self(self, s, n - 1)` 那一行。

优化：让每层要记的东西更少

原书接着指出两点可以省：

1. **结果单元 k 可以提到栈外**——一旦某层为真就逐层上传且不再变化，一个函数级变量就够了；
2. **参数 n 可以由栈深推出**——每递归一层 n 减 1、栈深加 1，所以 `n = n0 − 栈深`，这里的「栈深」指压在这一帧**下面**的帧数。上表可以直接验证：第 5 步之后栈里四帧自底向上 n 依次是 3、2、1、0，恰好等于 3 − 0、3 − 1、3 − 2、3 − 3。

于是栈帧从四个域缩到两个：

```cpp
/// 【算法 3.12】原书" 优化版" 的两点观察，本书照单实现：
///
/// 1. ** 结果单元 k 可以提到栈外 **——一旦某层为 true 就逐层上传且不再变化，
///    因此一个函数级变量即可，栈帧里的 `k` 域连同它的反复赋值、进出栈都省掉。
///    （上面那版其实已经这么做了：`child_result` 就在栈外。）
/// 2. ** 参数 n 可以由栈深推出 **——每递归一层 n 减 1、栈深加 1，
///    所以 `n = n0 - 栈深`，栈帧里的 `n` 域也能省掉。
///
/// 于是栈帧从四个域缩到两个（s 与 stage）。这是本节真正的" 优化":
/// 不是让它更快，而是 ** 让每层要记的东西更少 **——这正是手工模拟递归的意义。
///
/// 原书这一版另有一处致命问题：它同时把 `stack.top` 当 ** 数据成员 ** 用
/// （`t = stack.top;`）又当 ** 成员函数 ** 用（`stack.top(&tmp);`）。
/// 这在任何一种解释下都编译不过，而且它恰恰依赖代码 3.2/3.4 那个 `top` 重名缺陷。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_optimized(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);

    enum class Stage { Enter, AfterRule1, AfterRule2 };
    struct Frame {
        int s = 0;              // 只剩两个域
        Stage stage = Stage::Enter;
    };

    const std::size_t n0 = weights.size();
    ArrayStack<Frame> stack;
    knapsack_solution chosen;
    bool child_result = false;

    stack.push(Frame{capacity, Stage::Enter});
```

```cpp
    std::size_t depth = 1;      // 栈中帧数；当前帧的 n = n0 - (depth - 1)

    while (!stack.empty()) {
        Frame frame = *stack.pop();
        --depth;
        const std::size_t n = n0 - depth;   // 观察 2：n 由栈深推出，不再入栈

        if (frame.stage == Stage::Enter) {
            if (frame.s == 0) { child_result = true; continue; }
            if (frame.s < 0 || n == 0) { child_result = false; continue; }
            frame.stage = Stage::AfterRule1;
            stack.push(frame);
            stack.push(Frame{frame.s - weights[n - 1], Stage::Enter});
            depth += 2;
            continue;
        }

        if (frame.stage == Stage::AfterRule1) {
            if (child_result) { chosen.push_back(n - 1); continue; }
            frame.stage = Stage::AfterRule2;
            stack.push(frame);
            stack.push(Frame{frame.s, Stage::Enter});
            depth += 2;
            continue;
        }
        // AfterRule2：结果原样上传
    }

    return child_result ? std::optional<knapsack_solution>(std::move(chosen)) :
    ↪  std::nullopt;
}
```

```python
def knapsack_optimized(capacity: int, weights: list[int]) -> list[int] | None:
    """ 算法 3.12：栈帧只保存剩余承重和返回地址。"""
    _validate(capacity, weights)

    # ------------------------------------------------------------
    # 1. 状态定义（模拟 CPU 的“程序计数器” PC / 返回地址 Return Address）
    # enter       : 函数入口点（开始执行当前层的逻辑）
    # after_rule1: 从“选择当前物品”的子调用返回后的恢复点
    # after_rule2: 从“不选当前物品”的子调用返回后的恢复点
    # ------------------------------------------------------------
    enter, after_rule1, after_rule2 = range(3)

    # 显式模拟调用栈：
    # 栈帧中只保留绝对必需的两个状态：(当前剩余容量 remaining, 当前执行阶段 stage)
```

```
    stack = [(capacity, enter)]

    # 记录最终被选中的物品索引（构成解的路径）
    chosen: list[int] = []

    # 模拟 CPU 的“返回值寄存器”（例如 x86 中的 RAX/EAX）
    # 子调用执行完毕后，将结果（True/False）写入此变量，交由父调用读取
    child_result = False

    size = len(weights)

    # 全局调用深度计数器：
    # 【核心优化】：不需要在每个栈帧里都存“当前处理到第几个物品（count）”。
    # 递归深度与可选物品数是一一对应的，利用公式直接计算，极大节省了栈内存。
    depth = 1

    while stack:
        # 弹出当前栈顶执行上下文，深度减 1
        remaining, stage = stack.pop()
        depth -= 1

        # 通过全局深度逆向推导出当前正在处理的物品索引范围
        # count 表示当前还有前 count 个物品可选（对应下标 0 到 count - 1）
        count = size - depth

        # ==================== 阶段 0：函数入口 ====================
        if stage == enter:
            # Base Case 1: 恰好凑齐容量，递归成功
            if remaining == 0:
                child_result = True
            # Base Case 2: 超过承重，或已无物品可选，递归失败
            elif remaining < 0 or count == 0:
                child_result = False
            else:
                # 分支 1：尝试“选择”第 (count - 1) 个物品
                # [压栈操作 1 - 保存断点]：子调用结束后，需回到 after_rule1 处继续
                stack.append((remaining, after_rule1))
                # [压栈操作 2 - 发起子调用]：剩余容量扣减当前物品重量，进入新的 enter
                stack.append((remaining - weights[count - 1], enter))
                # 同时压入“返回断点”和“新调用”，栈深度增加 2
                depth += 2

        # ==================== 阶段 1：分支 1 返回 ====================
        elif stage == after_rule1:
            # 从“返回值寄存器”读取分支 1 的成败
            if child_result:
                # 分支 1 成功：说明拿该物品能凑齐，将其记录进结果集中
```

```
                chosen.append(count - 1)
                # 当前层任务完成，直接结束，准备弹出更上一层的断点
            else:
                # 分支 1 失败：回溯，转入分支 2——尝试“不选”第 (count - 1) 个物品
                # [压栈操作 1 - 保存断点]：分支 2 结束后恢复到 after_rule2
                stack.append((remaining, after_rule2))
                # [压栈操作 2 - 发起子调用]：容量不变，发起新调用
                stack.append((remaining, enter))
                depth += 2

        # ==================== 阶段 2：分支 2 返回 ====================
        # elif stage == after_rule2:
        #     pass
        # 说明：分支 2 的结果已经直接留存在 child_result 里了。
        # 这里无需额外代码，当前帧直接随 pop() 销毁，继续回溯上层。

    # 整个搜索结束后，若根调用成功则返回结果集，否则返回 None
    return chosen if child_result else None
```

这才是本节真正的”优化”：不是让它跑得更快，而是让每层要记的东西更少。手工模拟递归的全部意义就在这里——你必须想清楚”每一层到底需要记住什么”，而这正是编译器替你想过的那个问题。

> **一句实话**：写这个单元时，第一版显式栈实现我把”返回地址”塞进了位标志里，结果死循环、撞上构建超时；改写成与原书 label 一一对应的状态机之后才一次通过。手工模拟递归确实容易写错——而原书那份用 goto 的版本从未被编译器验证过。算法 3.12 里 `stack.top` 同时被当成数据成员（`t = stack.top;`）和成员函数（`stack.top(&tmp);`），任何一种解释下都编译不过。

与原书的对照

原书	现在	为什么
`class Stack<T>` 空基类	三条 `static_assert` + `<type_traits>`	空基类给不了多态，还带来非虚析构的未定义行为；对 `T` 的要求应当是编译期约束
`int mSize` / `int top` / `T* st`	`size_t capacity_` / `size_t top_index_` / `T* data_`	`int` 溢出是未定义行为；`top_index_` 避开与成员函数 `top()` 重名
只有 `~arrStack()`	五个特殊成员函数补齐	否则一次拷贝就二次释放

原书	现在	为什么
`bool pop(T& item)` `bool top(T& item)`	`[[nodiscard]] std::optional<T> pop()` `optional<T> top()` 取副本；`const T* peek()` 零拷贝	「有没有值」交给类型系统，忽略返回值会告警 两种正当需求，两种代价，都摆在明面上
`cout << " 栈满溢出"`	不做任何 I/O；越界抛 `std::out_of_range`	数据结构不该和标准输出耦合；可预期的空状态用 `optional`，真错误才抛异常
`delete[] st; st = newSt;`	先建后换 + `try/catch`，搬迁用 `std::move_if_noexcept`（移动构造 `noexcept` 才移动，否则拷贝）	搬迁中途抛异常不破坏原栈（强异常保证）；换成裸存储后搬迁执行的就是移动构造，判据与实际执行的操作一致（见 3.1.2a 第四小节）

刻意没改的：它仍然是一个手写的、自己管缓冲区的数组栈，缓冲区仍是裸的 `T* data_`，扩容仍是算法 3.3 的翻倍策略，`clear()` 仍然保留已分配容量、不归还存储（工程版会先逐个析构栈里的元素，教学版只把长度归零）。把它换成 `std::stack` 的薄封装固然更短，但这一节要教的正是这些实现细节本身。

完整实现见 `code/ch03/array_stack/modern.hpp`，测试见同目录 `test.cpp`（85 项断言，覆盖上表每一行；用 `python3 tools/check_code.py` 在 `-Werror` + ASan/UBSan 与 `-O2` 两档、g++ 与 clang++ 两种编译器、共四种构建下各跑一遍）。

3.2 队列

为什么这一节没有 Python 版

循环队列和链式队列的教学重点是环形下标、结点所有权以及空/满状态的表示，而不是调用一个现成队列。Python 的 `list` 或 `collections.deque` 会替调用方管理容量和对象寿命，既看不到环形存储的约束，也无法对应 C++ 的析构与异常安全；因此这里用 C++ 把存储不变量讲完整，算法章节再用 Python 展示队列的使用。

与栈类似，队列（queue）也是一种**限制访问点的线性表**：元素只能从表的一端插入，另一端删除。按照习惯，把只允许删除的一端称为队列的头，简称**队头**（front），删除操作本身称为**出队**（dequeue）；表的另一端为队列的尾，简称**队尾**（rear），这一端只能进行插入操作，称为**入队**（enqueue）。如同现实生活中的队列一样，在没有人为破坏的前提下，队列是按照到达的顺序来释放 元素的——先进入队列的元素会先离开，因此队列通常也被称为**先进先出**（first in

first out）表，简称 FIFO 表。空队列上的提取是可预期状态，返回 `std::nullopt`。

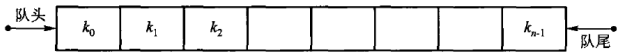


图 3.9　队列的示例。删除的一端叫队头 (front)，删除操作叫出队 (dequeue)；插入的一端叫队尾 (rear)，插入操作叫入队 (enqueue)。售票窗口前排队买票就是日常版的队列。

3.2.1 队列的抽象数据类型

根据数据抽象和封装的原则，对队列的操作只能通过队列的抽象数据类型所定义的运算来进行。原书【代码 3.13】用一个 C++ 类模板给出这组运算：

原书的运算	含义	本书对应
`clear()`	变为空队列	`clear()`
`enQueue(item)`	将 item 插入队尾，成功返回真	`enqueue(item)`
`deQueue(item)`	返回队头元素并把它从队列中删除	`dequeue()` 返回 `std::optional`
`getFront(item)`	返回队头元素，但不删除	`front()` 返回 `std::optional`
`isEmpty()`	队列已空则返回真	`empty()`
`isFull()`	队列已满则返回真	顺序实现才有 `full()`

根据具体应用的不同需要，可以适当增删抽象数据类型中所定义的运算——例如链式队列并不需要 判断队列是否满。队列抽象数据类型中运算的实现方法依赖于队列的存储结构，下面介绍常用的顺序 队列和链式队列两种。原书把这些运算写成一个假抽象基类，本书直接定义在 `ArrayQueue` 与 `LinkedQueue` 上，理由与 3.1 节相同。

3.2.2 顺序队列

用顺序存储结构来实现队列就形成了**顺序队列**（array-based queue）。与顺序表一样，顺序队列 需要分配一块连续的区域来存储队列的元素，**需要事先知道或估算队列的大小**。与栈类似，顺序 队列也存在溢出问题：队列满时入队会产生**上溢**，而队列为空时出队则会产生**下溢**；出现上溢时，如果需要，也可以考虑对队列进行适当的扩容。

队列两头都要动：入队动队尾，出队动队头。如果只是沿用顺序表的实现方法，就很难取得良好的 效率——假设队列中有 n 个元素、顺序表的实现要求它们都存在数组的前 n 个位置上：把队尾放在 位置 0，则出队是 $O(1)$（队列最前面的元素就是数组最后面的元素），而入队是 $O(n)$，因为必须把 当前所有元素都向后移动一个位置；反之把队尾放在位置 $n-1$，入队只需在最后添加，$O(1)$，而出队要移动剩余 $n-1$ 个元素以确保它们在表的前 $n-1$ 个位置，代价 $O(n)$。

两头都别固定：在保证队列元素连续性的同时允许队列的首尾位置在数组中移动，让 `front` 和 `rear` 都在数组里往后走，两个操作就都是 $O(1)$：

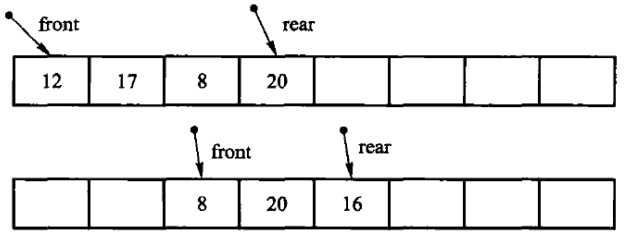


图 3.10　顺序队列的实现示意。出队让 `front` 后移，入队让 `rear` 后移；元素本身一个都不搬。

这样做会遇到**假溢出**：整个队列不断向数组尾部漂移，`rear` 一旦到了 `mSize - 1`，即使数组前端还空着一片，入队也会报满。解决办法是把数组在逻辑上看成一个环——下标 0 是下标 `mSize - 1` 的直接后继，位置 `x` 的后继是 `(x + 1) % mSize`，这就是**循环队列**（也叫环形队列）。

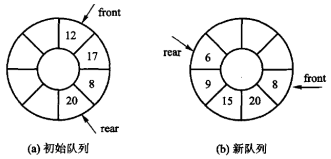


图 3.11　把数组看成环之后的样子：(a) 初始队列里有 12、17、8、20 四个元素；(b) 两次出队、三次入队之后，队列变成 8、20、15、9、6。入队增加 `rear`，出队增加 `front`，两者都在环上顺时针走。

先约定两个下标的含义（与原书【代码 3.14】和本书代码一致）：`front` 是**队头元素**的下标，`rear` 是**下一个入队元素要写的位置**，即队尾元素的后一格。于是入队写 `qu[rear]` 后 `rear = (rear + 1) % mSize`，出队取 `qu[front]` 后 `front = (front + 1) % mSize`，空队列就是 `front == rear`。

绕回之后有个麻烦：队列装满时 `rear` 恰好追上 `front`，又是 `front == rear`——空和满分不开。数一数就明白为什么：m 个位置的数组，若允许装满，队列有「空、1 个、2 个.....m 个」共 $m+1$ 种状态，而固定 `front` 后 `rear` 只有 m 种取值，必有两种状态撞在一起。一种办法是另记一个计数或布尔量；顺序队列更常用的是原书的办法——**牺牲一个槽位**：要装 n 个元素就把数组开成 $n+1$ 格，约定「rear 的下一格就是 front」时算满，于是两种状态就分得开了。这正是章首 demo 里「容量 3 时第 4 个入不进去」的原因。

记数组格数为 $m = n + 1$（代码里的 `slots_`），三个判定公式是：

- 空：`front == rear`；
- 满：`(rear + 1) % m == front`；
- 元素个数：`(rear - front + m) % m`（代码里为避免无符号数相减写成了 `rear >= front ? rear - front : m - front + rear`，两者相等）。

以章首 demo 为例（$n = 3$，$m = 4$）：依次入队 1、2、3 后 `front = 0`、`rear`

= 3，个数 $(3-0+4) \bmod 4 = 3$；再入队时 $(3+1) \bmod 4 = 0$，正好等于 front，判满，返回 false。此时下标 3 那一格空着，它就是被牺牲的槽位。

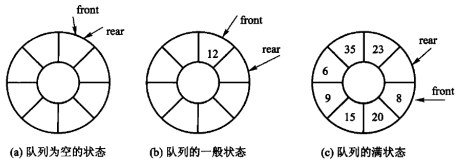


图 3.12　牺牲一格的循环队列（8 格数组最多放 7 个元素）：(a) 空，`front == rear`；(b) 一般状态；(c) 满，`(rear + 1) % (n + 1) == front`。

3.2.3 链式队列

链式队列（linked queue）是队列的链式实现，本质上是对链表的简化：链接的方向从队列的前端 指向队列的尾端，成员 `front` 和 `rear` 分别指向队首和队尾。入队只是把新元素放到链表的尾部 并修改 `rear` 使其指向新的结点；出队只是删除表中最前面的那个结点并修改 `front` 使其指向新的 队首。结点分散在堆上，所以**没有「队满」这回事**。

除了队头指针，队尾指针是必需的：没有它，每次入队都得从队头走到尾，$O(n)$；有了它，入队和 出队都是 $O(1)$。

为了减少访问队首结点的时间代价，这种实现**没有链表中的专用表头虚结点**，代价是两个边界要 单独处理：enqueue 必须单独处理「插入到空队列」的情况（此时 `front` 和 `rear` 都要指向新结点），dequeue 也需考虑「出队后队列变空」的情况。

后一条正是最容易漏的地方：**出队把队列摘空时，队尾指针必须一起置空**，否则它就成了一根指向已释放结点的野指针，下一次入队会写进已释放的内存。教学版的测试里有一条专门盯着它——去掉那两行，AddressSanitizer 立刻报 `heap-use-after-free`。

教学版：完整实现

两个类放在同一个文件里，正好对着看：同一个先进先出的抽象数据类型，一个用固定大小的连续数组，一个用堆上的结点。

```
// 队列 ——教学版。原书【代码 3.13】【代码 3.14】【代码 3.15】。
//
// 一个文件、两个类、能直接编译运行：
//   ArrayQueue   顺序队列（循环队列），元素放在一块固定大小的连续数组里；
//   LinkedQueue  链式队列，结点分散在堆上，队尾指针让入队保持 O(1)。
//
// 与 modern.hpp（工程版）的分工：
//   教学版  三法则（析构 + 拷贝构造 + 拷贝赋值），一行一句，正确但拷贝多一点；
//   工程版  在此之上补齐移动语义与 copy-and-swap。
// 两份都在闸门里真编译真运行。先读这一份，3.2.3a「进阶（选读）」再读那一份。
```

```cpp
#pragma once

#include <cstddef>
#include <optional>

// ---------------------------------------------------------------------------
// 顺序队列（循环队列）
//
// 队列两头都要动：入队动队尾，出队动队头。如果队头固定在下标 0，每次出队都要把
// 后面所有元素前移一位，O(n)。所以真正的做法是让队头也往后走——走到数组末尾就
// 绕回下标 0，数组被当成一个圈来用，这就是「循环队列」。
//
// 绕回之后有个麻烦：`front == rear` 既可能是空、也可能是满，分不开。
// 原书的办法是 ** 牺牲一个槽位 **：约定「rear 的下一格就是 front」时算满，
// 于是 n 个槽位最多装 n-1 个元素，两种状态就分得开了。本书照办。
// ---------------------------------------------------------------------------
template <typename T>
class ArrayQueue {
public:
    using value_type = T;
    using size_type = std::size_t;

    // capacity 是「最多能装几个元素」，所以内部要多要一个槽位。
    explicit ArrayQueue(size_type capacity)
        : slots_(capacity + 1), data_(new T[capacity + 1]), front_(0), rear_(0) {}

    ~ArrayQueue() { delete[] data_; }

    ArrayQueue(const ArrayQueue& other)
        : slots_(other.slots_), data_(new T[other.slots_]),
          front_(other.front_), rear_(other.rear_) {
        for (size_type i = front_; i != rear_; i = (i + 1) % slots_) {
            data_[i] = other.data_[i];
        }
    }

    ArrayQueue& operator=(const ArrayQueue& other) {
        if (this == &other) {
            return *this;
        }
        T* fresh = new T[other.slots_];
        for (size_type i = other.front_; i != other.rear_; i = (i + 1) %
        ↪  other.slots_) {
            fresh[i] = other.data_[i];
        }
        delete[] data_;
        data_ = fresh;
```

```
        slots_ = other.slots_;
        front_ = other.front_;
        rear_ = other.rear_;
        return *this;
    }

    bool empty() const { return front_ == rear_; }

    // 满的判据：rear 再往前走一格就撞上 front。那一格就是被牺牲掉的槽位。
    bool full() const { return (rear_ + 1) % slots_ == front_; }

    size_type size() const {
        return (rear_ >= front_) ? (rear_ - front_) : (slots_ - front_ + rear_);
    }

    // 入队。队满返回 false——顺序队列的容量是固定的，这是它与链式队列的核心差别。
    bool enqueue(const T& value) {
        if (full()) {
            return false;
        }
        data_[rear_] = value;
        rear_ = (rear_ + 1) % slots_;    // 走到末尾就绕回 0，取模不能漏
        return true;
    }

    // 出队。空队列返回空 optional，不是错误，也不打印任何东西。
    std::optional<T> dequeue() {
        if (empty()) {
            return std::nullopt;
        }
        T value = data_[front_];
        front_ = (front_ + 1) % slots_;
        return value;
    }

    // 看队头但不出队。
    std::optional<T> front() const {
        if (empty()) {
            return std::nullopt;
        }
        return data_[front_];
    }

    void clear() { front_ = rear_ = 0; }

private:
    size_type slots_;     // 数组格数 = 容量 + 1（多的那一格用来区分空和满）
```

```
    T* data_;
    size_type front_;     // 队头元素的下标
    size_type rear_;      // 下一个入队元素要写的下标
};

// ----------------------------------------------------------------------------
// 链式队列
//
// 结点分散在堆上，** 没有「队满」这回事 **。
// 除了队头指针，还要一个 ** 队尾指针 **：没有它，每次入队都得从队头走到尾，O(n)。
// 有了它，入队和出队都是 O(1)。
// ----------------------------------------------------------------------------
template <typename T>
class LinkedQueue {
public:
    using value_type = T;
    using size_type = std::size_t;

    LinkedQueue() : front_(nullptr), rear_(nullptr), size_(0) {}

    ~LinkedQueue() { clear(); }

    LinkedQueue(const LinkedQueue& other) : front_(nullptr), rear_(nullptr),
↪   size_(0) {
        for (Node* source = other.front_; source != nullptr; source = source->next) {
            enqueue(source->value);
        }
    }

    LinkedQueue& operator=(const LinkedQueue& other) {
        if (this == &other) {
            return *this;
        }
        clear();
        for (Node* source = other.front_; source != nullptr; source = source->next) {
            enqueue(source->value);
        }
        return *this;
    }

    // 入队：新结点接到队尾。队列原来是空的话，它同时也是队头。
    void enqueue(const T& value) {
        Node* fresh = new Node;
        fresh->value = value;
        fresh->next = nullptr;
        if (rear_ == nullptr) {
            front_ = rear_ = fresh;
```

```
        } else {
            rear_->next = fresh;
            rear_ = fresh;
        }
        ++size_;
    }

    // 出队：摘下队头结点。摘完若队列空了，队尾指针也必须置空，
    // 否则它就成了一根指向已释放内存的野指针。
    std::optional<T> dequeue() {
        if (empty()) {
            return std::nullopt;
        }
        Node* dying = front_;
        T value = dying->value;
        front_ = dying->next;
        if (front_ == nullptr) {
            rear_ = nullptr;
        }
        delete dying;
        --size_;
        return value;
    }

    std::optional<T> front() const {
        if (empty()) {
            return std::nullopt;
        }
        return front_->value;
    }

    bool empty() const { return front_ == nullptr; }
    size_type size() const { return size_; }

    // 用循环释放，不要用递归——长队列的递归析构会把运行栈撑爆。
    void clear() {
        while (front_ != nullptr) {
            Node* dying = front_;
            front_ = dying->next;
            delete dying;
        }
        rear_ = nullptr;
        size_ = 0;
    }

private:
    struct Node {
```

```
        T value;
        Node* next;
    };

    Node* front_;
    Node* rear_;          // 少了它，入队就要每次从头走到尾
    size_type size_;
};
```

3.2.3a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。工程版（`code/ch03/queue/modern.hpp`）与教学版的差别，和 3.1.2a、3.1.3a 里说过的完全是同一批：补齐移动构造与移动赋值、拷贝赋值改用 copy-and-swap、查询函数标 `[[nodiscard]]` 与 `noexcept`、读队头改为返回 `const T*`（零拷贝，但指针会随下一次修改失效）。

```
/// 循环队列。牺牲一个槽位区分「空」与「满」：逻辑容量 n 时实际申请 n+1 格。
template <typename T>
class ArrayQueue {
public:
    explicit ArrayQueue(std::size_t capacity)
        : slots_(capacity + 1), data_(allocator_.allocate(slots_)) {}

    ArrayQueue(const ArrayQueue& other)
        : slots_(other.slots_),
          front_(other.front_),
          rear_(other.rear_),
          data_(other.slots_ ? allocator_.allocate(other.slots_) : nullptr) {
        std::size_t constructed = 0;
        try {
            for (std::size_t i = front_; i != rear_; i = (i + 1) % slots_) {
                allocator_traits::construct(allocator_, data_ + i, other.data_[i]);
                ++constructed;
            }
        } catch (...) {
            for (std::size_t i = 0; i < constructed; ++i) {
                allocator_traits::destroy(allocator_, data_ + ((front_ + i) %
↪   slots_));
            }
            if (data_ != nullptr) allocator_.deallocate(data_, slots_);
            throw;
        }
    }

    ArrayQueue& operator=(const ArrayQueue& other) {
        if (this != &other) {
```

```
            ArrayQueue copy(other);
            swap(copy);
        }
        return *this;
    }

    ArrayQueue(ArrayQueue&& other) noexcept { swap(other); }

    ArrayQueue& operator=(ArrayQueue&& other) noexcept {
        if (this != &other) {
            ArrayQueue moved(std::move(other));
            swap(moved);
        }
        return *this;
    }

    ~ArrayQueue() {
        clear();
        if (data_ != nullptr) allocator_.deallocate(data_, slots_);
    }

    void swap(ArrayQueue& other) noexcept {
        using std::swap;
        swap(slots_, other.slots_);
        swap(front_, other.front_);
        swap(rear_, other.rear_);
        swap(data_, other.data_);
    }

    [[nodiscard]] bool empty() const noexcept { return front_ == rear_; }

    /// 满：rear 再往前一格就撞上 front。那一格就是被牺牲掉的槽位。
    [[nodiscard]] bool full() const noexcept {
        return slots_ != 0 && (rear_ + 1) % slots_ == front_;
    }

    [[nodiscard]] std::size_t size() const noexcept {
        return rear_ >= front_ ? rear_ - front_ : slots_ - front_ + rear_;
    }

    /// 入队。队满返回 false——顺序队列的容量是固定的。
    [[nodiscard]] bool enqueue(const T& value) {
        if (full()) {
            return false;
        }
        allocator_traits::construct(allocator_, data_ + rear_, value);
        rear_ = (rear_ + 1) % slots_;
```

```
        return true;
    }

    [[nodiscard]] bool enqueue(T&& value) {
        if (full()) {
            return false;
        }
        allocator_traits::construct(allocator_, data_ + rear_, std::move(value));
        rear_ = (rear_ + 1) % slots_;
        return true;
    }

    /// 出队。空队列返回 std::nullopt（D-001 §3c）。
    [[nodiscard]] std::optional<T> dequeue() {
        if (empty()) {
            return std::nullopt;
        }
        T value = std::move(data_[front_]);
        allocator_traits::destroy(allocator_, data_ + front_);
        front_ = (front_ + 1) % slots_;
        return value;
    }

    /// 零拷贝地看一眼队头。空队列返回 nullptr；指针在下一次修改后即失效（D-001 §3b）。
    [[nodiscard]] const T* front() const noexcept {
        return empty() ? nullptr : &data_[front_];
    }

    /// 清空：逐个销毁存活元素，释放其持有的资源；槽位本身保留为原始存储。
    void clear() noexcept {
        while (front_ != rear_) {
            allocator_traits::destroy(allocator_, data_ + front_);
            front_ = (front_ + 1) % slots_;
        }
        front_ = rear_ = 0;
    }

private:
    using allocator_type = std::allocator<T>;
    using allocator_traits = std::allocator_traits<allocator_type>;
    allocator_type allocator_;
    std::size_t slots_{0};   // 数组格数 = 容量 + 1
    std::size_t front_{0};
    std::size_t rear_{0};
    T* data_{nullptr};
};
```

这份工程版一度把每个函数压成一行，最长的一行 239 个字符——印在书上，读者要 横向找分号才知道函数在哪结束。2026-08-16 已把它排开，逻辑一字未改。**一行一个函数省的是纸，费的是读者的眼睛**，这也正是 D-012 要分层的理由之一。

3.3 栈与队列的深入讨论

3.3.1 顺序栈与链式栈的比较

栈的应用非常广泛，只要满足后进先出的特性都可以使用栈结构，例如函数调用和递归实现、深度 优先周游、表达式转换和求值等。

时间上难分伯仲，空间上各有代价。顺序栈和链式栈的基本操作都只需要常数时间，push/pop 都是 $O(1)$。从空间角度看：初始时顺序栈必须说明一个固定的长度，当栈不够满时，势必浪费一些 空间；链式栈的长度可根据需要而增减，但每个元素都需要一个指针域，从而产生**结构性开销**。

还有一处差别常被忽略：访问栈的内部元素。顺序栈可以根据元素与栈顶的相对位置快速定位并 读取内部元素，而链式栈则需要沿着指针链遍历才能访问内部元素。鉴于上述原因，顺序栈在实际中 更为常用一些。

多栈管理。在编译系统等计算机系统软件中，往往同时使用和管理多个栈，这时可充分利用顺序 栈单向延伸的特性：用一个数组来存储两个栈，让它们共享同一存储空间——把数组的两端设置为 两者各自的栈底，从两端开始向中间迎面延伸，这样浪费的空间相对少一些：

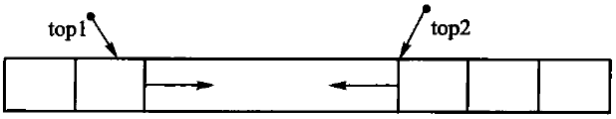


图 3.13　存储在同一个数组中的两个迎面增长的栈。两个栈的空间需求恰好相反（一个涨、另一个缩）时，这种做法很省空间；两个同时涨，中间那点余量很快就没了。队列做不到这件事——除非总有元素从一个队列转入另一个队列。

3.3.2 顺序队列与链式队列的比较

由于存储空间固定，顺序队列无法满足队列规模变化很大且最大规模无法预测的情况，而链式队列则 可以轻松应对这种类型的应用。另一方面，顺序队列在存取访问上很简单，可以适用那些对队列内部 元素（不仅限于队列的两端）有访问需求的应用；如果可以预测浪涌的统计特性，就可以大致估算出 缓冲队列的长度。

常用的入队和出队操作在两种实现里都是常数时间，**二者在时间效率上没有优劣之分**；空间代价上 与栈的情况类似。只是顺序队列不像顺序栈那样，**不能在一个数组中存储两个队列**——除非总有 数据项从一个队列转入另一个队列。

队列通常作为消息或数据缓冲器在很多领域得到广泛应用，只要满足先来先服务的特性即可。例如，计算机的硬件设备之间需要队列作为其数据通信的缓冲；操作系统中使用队列对内存、打印机等各种 资源进行管理，而且根据不同优先级

别的服务请求，按优先类别把服务请求分别组织成多个不同的 队列。另外，在第 5、6、7 章中将会看到，**队列是广度优先搜索中采用的主要中间数据结构**。

3.3.3 限制存取点的表

利用栈和队列的思想可以设计出一些变种的栈或队列结构。虽然它们的应用没有栈和队列那样广泛，但在一些特定情况下具有很好的应用价值。

1. **双端队列**：限制插入和删除在线性表的两端进行。栈和队列都是特殊的双端队列，对其存取 设置了限制。
2. **双栈**：两个底部相连的栈。双栈是一种添加限制的双端队列，它规定从 end1 插入的元素只能 从 end1 端删除，而从 end2 插入的元素只能从 end2 端删除。
3. **超队列**：一种删除受限的双端队列，删除只允许在一端进行，而插入可在两端进行。
4. **超栈**：一种插入受限的双端队列，插入只限制在一端，而删除允许在两端进行。

由此可见，这几种限制存取点的表都是某种受限的双端队列。C++ 标准库的基本序列中就有双端队列 `std::deque`，`std::stack` 与 `std::queue` 默认都是通过它实现的——**但那是使用者的视角**；本章手写这两种结构，是因为「存储怎么落、边界怎么守」本身就是这一章的教学内容。

出栈序列能不能得到：拿顺序栈模拟一遍

本章习题第 7 题问 5 辆列车开出车站的次序有多少种，第 8 题给出合法出栈序列的充分必要条件，补充算法设计题第 2 题要证明合法序列的个数是 Catalan 数。三道题背后是同一个操作层面的问题：**车辆 $1, 2, \ldots, n$ 依次进栈，给定一个出栈次序，它能不能得到？能的话，怎么进、怎么出？**

回答它不需要搜索。拿栈模拟一遍，每一步都没有选择余地——看下一辆要开出的车：

1. **正在栈顶**：必须现在就开出。再压进一辆，它就被盖住了；
2. **还没进栈**：只能把后面的车一辆辆压进去，直到它到达栈顶；
3. **已在栈里、却不在栈顶**：它上面压着别的车，这个次序不可能。

第 3 种情况不必单独判断：车全部进过栈了还等不到它，就是它。以 `3, 1, 2` 为例：3 要先开出，1、2 就必须都已进栈，而且 2 压在 1 上面；3 开走之后栈顶是 2，1 出不来。

```
enum class StackOp { Push, Pop };

/// 车辆 1, 2, …, n 依次进栈，途中随时可以出栈。问：出栈次序能否恰好是 pop_order？
/// 能则返回一串操作（按这个次序做 Push / Pop 就得到它），不能返回 std::nullopt。
///
/// 做法是 ** 拿本节的 ArrayStack 模拟一遍 **，每一步都没有选择余地：
///   下一个要出栈的车正在栈顶 ——必须现在就出（再压一辆它就被盖住了）；
///   还没进栈                 ——只能一辆辆往里压，直到它到栈顶；
```

```
///    已经在栈里、却不在栈顶    ——它上面压着别的车，这个次序不可能。
/// 第三种情况不必单独判断：压完所有车还等不到它，就是它。
///
/// pop_order 里出现重复、缺号或越界的编号时同样返回 std::nullopt：
/// 进栈的车两两不同，出栈次序只可能是 1..n 的一个排列。
[[nodiscard]] inline std::optional<std::vector<StackOp>> stack_operations_for(
        const std::vector<int>& pop_order) {
    const int n = static_cast<int>(pop_order.size());
    ArrayStack<int> stack;
    std::vector<StackOp> ops;
    int next_car = 1;  // 下一辆等着进栈的车
    for (int wanted : pop_order) {
        while (stack.empty() || *stack.peek() != wanted) {
            if (next_car > n) {
                return std::nullopt;  // 车都进过栈了，它还没到栈顶：被压在下面了
            }
            stack.push(next_car);
            ++next_car;
            ops.push_back(StackOp::Push);
        }
        (void)stack.pop();
        ops.push_back(StackOp::Pop);
    }
    return ops;
}
```

模拟用的就是本节的 `ArrayStack`,看栈顶用的是不拷贝的 `ArrayStack::peek`。「这个次序不可能」是可预期的结果，不是错误，所以返回 `std::nullopt` 而不是抛异常——与 `pop()` 在空栈上的口径一致。每辆车至多进栈、出栈各一次，整个判定是 $O(n)$ 的。

测试对 $n = 1, \dots, 7$ 穷举全部 5913 个排列，逐个核对三件事：合法序列的个数恰为 1、2、5、14、42、132、429，即 Catalan 数 $C_n = \frac{(2n)!}{(n+1)!\,n!}$；判定结果与习题第 8 题的 判据（不存在 $i < j < k$ 使 $p_j < p_k < p_i$）逐个一致；返回的操作序列用另一个栈独立重放，恰好得到目标次序。第 8 题的判据是一个 $O(n^3)$ 的三重循环，与这里的模拟过程不共用任何一行，所以两者一致才有分量。

本章小结

栈是一种限制访问端口的线性表，常被称为后进先出（LIFO）表。其元素的插入和删除都只能在表的 一端进行，该端称为栈的栈顶，另一端则叫做栈底。栈的特点是：每次取出（并被删除）的元素总是当前 栈内元素中最后压入的，而最先压入的元素则被放在栈的底部、要到最后才能取出。

栈的运算包括压栈、出栈、返回栈顶元素、判断栈是否为空等，**运算的时间**

代价均为常数时间。栈的实现通常有顺序栈和链式栈两种：顺序栈用数组实现；链式栈则用单链表方式存储，其中的指针方向 是从栈顶向下链接。**在实际中，顺序栈比链式栈使用得更为广泛**。

栈是一种很重要、应用非常广泛的数据结构，常见的应用包括表达式转换和求值、函数调用和递归实现、深度优先搜索等。**栈的一个很重要的应用在于对函数调用机制和递归实现的支持**——本章通过递归函数 的实现机制和递归算法到非递归算法的转换，揭示了递归的本质以及栈与递归的内在联系。

队列也是一种限制访问端口的特殊线性表。其元素的删除只限于队列头一端进行，而元素的插入则被 限制于队列尾一端。队列的特点是新来的成员总是加入到队的末尾，而每次取出的元素总是来自队列的 前端，通常称为先进先出（FIFO）表。队列的运算主要包括进队列、出队列、取队头元素、判断队列是否为 空、判断队列是否已满等，运算的时间代价均为常数时间。

队列的存储结构和实现方法主要分为顺序和链式两种。**顺序队列是比较普遍的一种实现方式**，采用环状 顺序表来存放队列元素，并用两个变量分别指向队列的头和尾，往队列中加入元素或取出元素时分别改变 队头和队尾这两个变量的计数；链式队列一般用单链表方式存储，其中链接指针的方向是从队列的前端向尾端链接。

顺序队列采用固定的存储空间，一方面在读访问时非常简单、在特殊应用中需要访问内部元素时也很方便；但另一方面，在某些无法事先估计其大小的情况下会出现溢出现象。**链式队列可以通过动态申请空间来 满足队列长度的大幅变动**。

队列也是一种应用很广泛的数据结构，常见的应用包括广度优先搜索、消息缓冲器、操作系统的各种资源 管理等——只要符合先进先出特性的应用均可运用队列。

习题

补充算法设计题（参考课程第 3 章）

1. 只使用两个栈的 `push`、`pop`、`empty`，设计队列的 `enqueue`、`dequeue` 和 `empty`，并分析摊还复杂度。
2. 编号为 `1..n` 的车辆依次进栈,求所有合法出栈序列的数量并证明其为 Catalan 数。
3. 用两个栈设计编辑器的撤销和恢复；说明新操作、撤销、恢复时两个栈如何变化。

正文习题

1. 如果允许在循环队列的两端都可以进行插入和删除操作，要求：(1) 写出循环队列的类型定义；
   (2) 写出「从队尾删除」和「从队头插入」的算法。
2. 如果用一个循环数组 $q[0 \ldots m-1]$ 表示队列时，该队列只有一个队列头指

针 front、不设队列尾 指针 rear，而设置计数器 count 用以记录队列中结点的个数。试编写算法实现队列的 3 个基本 运算：判空、入队、出队。

3. 试按以下的要求把栈 S 中的元素逆置：(1) 使用额外的两个栈；(2) 使用额外的一个队列；
   (3) 使用额外的一个栈，外加一些非数组的变量。
4. 试给出用栈所定义的队列。
5. 假设以带头结点的循环链表表示队列，并且只设一个指针指向队尾元素结点（**注意不设头指针**），试编写相应的队列初始化、入队列和出队列的算法。
6. 试在一个长度为 n 的数组中实现两个栈，使得二者在元素的总数目为 n 之前都不溢出，并且保证 push 和 pop 操作的时间代价为 $O(1)$。
7. 编号为 1、2、3、4、5 的 5 辆列车顺序开进栈式结构的站台，试问开出车站的顺序有多少种可能，并予以解释。
8. 证明:从初始输入序列 $1, 2, \cdots, n$ 可以利用一个栈得到输出序列 $p_1, p_2, \cdots, p_n$（它是 $1, 2, \cdots, n$ 的一种排列）的充分必要条件是：**不存在下标** $i < j < k$**，满足** $p_j < p_k < p_i$。
9. 用栈计算后缀表达式 $12\,8\,9\ *\ +$，写出每一步的栈。
10. 用栈把中缀 $a * (b * c - d) + e$ 转成后缀，写出每一步的栈。
11. 试写出一个按照下面定义计算整数 n 和 m 的最大公因子 $GCD(n, m)$ 的递归算法：$GCD(n, m) = m$（当 $m \le n$ 且 $n \bmod m = 0$）；$GCD(n, m) = GCD(m, n)$（当 $n < m$）；否则 $GCD(n, m) = GCD(m,\ n \bmod m)$。
12. 写递归和非递归算法，求 $1 + 1/2 - 1/3 + 1/4 - \cdots$ 的前 n 项和。

上机题

1. 利用两个栈 s_1、s_2 模拟一个队列时，如何用栈的运算来实现该队列的运算?
   (1) enqueue：插入一个元素；(2) dequeue：删除一个元素；(3) queue_empty：判定队列为空。
2. 双端队列 deque 是一种插入和删除操作在线性表的两端进行的数据结构，试给出利用数组实现的 deque 两端的插入、删除操作，**要求这 4 个操作的时间代价均为常数**。
3. 试利用非数组变量，按下述条件各设计一个相应的算法以使队列中的元素有序:
   (1) 使用两个辅助的队列；(2) 使用一个辅助的队列。
4. 试按下述条件各设计一个算法，把栈 s_1 中的元素转移到栈 s_2 中，**并保持栈中元素原来的顺序**:
   (1) 使用一个辅助栈；(2) 只使用一些辅助的非数组变量。
5. Fibonacci 序列 $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \cdots$ 中每个元素是前两个元素之和，可递归定义为 $fib(n) = n$（$n = 0, 1$）、$fib(n) = fib(n-2) + fib(n-1)$（$n \ge 2$）。试设计一个计算 $fib(n)$ 的 递归过程，并利用栈来模拟递归调用，将递归过程改写成一个非递归过程。

6. 已知 Ackermann 函数定义为：$Ack(m,n) = n+1\ (m=0)$；$Ack(m,n) = Ack(m-1,1)\ (m \neq 0, n=0)$；$Ack(m,n) = Ack(m-1, Ack(m,n-1))\ (m \neq 0, n \neq 0)$。
(1) 写出 $Ack(2,1)$ 的计算过程；(2) 写出计算 $Ack(m,n)$ 的非递归算法。

第 4 章 字符串

字符串是字符的有限序列。先区分「保存字符串」的类与「在文本中找模式」的算法：前者处理容量、复制和下标边界，后者处理比较顺序。朴素匹配在失配后移动模式；KMP 预先计算 next 信息，避免 重复比较已经知道相等的前缀。

源码：字符串类·教学版、字符串类·工程版、字符串示例、模式匹配、匹配示例。

先跑一遍

```
// 第 4 章「先跑一遍」：用教学版 String 走一遍 append / substr / find。
// 编译运行：
//   g++ -std=c++17 -I code/ch04/string_class code/ch04/string_class/demo.cpp -o
↪  demo && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    String text = "Hello";                      // 隐式转换：String(const char*)
    text.append(' ').append('C').append('+').append('+');
    std::cout << " 拼接后：" << text.c_str() << '\n';

    const String slice = text.substr(6, 3);
    std::cout << " 子串：" << slice.c_str() << '\n';

    // find 返回 optional：有值才解引用
    std::cout << " 首次出现 C 的下标：";
    if (const auto found = text.find('C')) {
        std::cout << *found << '\n';
    } else {
        std::cout << " 无\n";
    }
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch04/string_class \
    code/ch04/string_class/demo.cpp -o /tmp/str-demo
/tmp/str-demo
```

```
拼接后：Hello C++
子串：C++
首次出现 C 的下标：6
```

`String` 的缓冲区仍是手写的 `char*`：4.2 节要讲的正是变长串的分配、`'\0'` 结尾和深复制，若内部直接用 `std::string`，这些都被标准库藏起来了。`find` 找不到时返回空 optional，不用 `-1` 和位置 0 抢同一个数字。

图 4.12 自己的那对串，原书两个匹配算法都返回 11，正确答案是 10：

```cpp
#include "modern.hpp"

#include <iostream>

int main() {
    const char* text = "abcddabcababcdaabcababcdaabcabaa";
    const char* pattern = "abcdaabcab";
    const auto naive = dsa::naive_search(text, pattern);
    const auto kmp = dsa::kmp_search(text, pattern);
    std::cout << " 图 4.12 的串，正确起始下标是 10\n";
    std::cout << " 朴素: " << (naive ? static_cast<long>(*naive) : -1) << '\n';
    std::cout << "KMP:  " << (kmp ? static_cast<long>(*kmp) : -1) << '\n';
    std::cout << " 原书返回 11，一律差 1\n";
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch04/pattern_matching \
    code/ch04/pattern_matching/demo.cpp -o /tmp/kmp-demo
/tmp/kmp-demo
```

```
图 4.12 的串，正确起始下标是 10
朴素: 10
KMP:  10
原书返回 11，一律差 1
```

4.1 字符串的基本概念

字符串（string）是组成元素（结点）为单个字符的线性表，简称「串」。一个字符串可以是一个单词、一个句子、一篇文章，或者一个文件的内容。串中所包含的字符个数称为**串的长度**；长度为零的串 称为**空串**，不包含任何字符内容。

尽管字符串是一种特殊的线性表，但其逻辑结构与线性表相比存在两点区别：

1. 字符串的数据对象约束为**字符集**；
2. 线性表的基本操作大多以「单个元素」为操作对象，而字符串的基本操作通常以「**串的整体**」作为操作对象——拼接、复制、抽取子串、模式匹配，动的都是一整段。

子串。设 $s_1 = a_0a_1a_2\cdots a_{n-1}$、$s_2 = b_0b_1b_2\cdots b_{m-1}$ $(0 \le m \le n)$ 是两个 字符串，a_i 和 b_j 均为字符集中的字符。如果存在整数 i $(0 \le i \le n-m)$，使得对任意 $j = 0, 1, \cdots, m-1$ 都有 $b_j = a_{i+j}$ 同时成立，则称字符串 s_2 是

s_1 的**子串**，或称 s_1 包含 s_2。空字符串是所有字符串的子串，任何串都是其自身的子串；如果 `str` 的一个 子串非空且不为 `str` 自身，则称其为 `str` 的一个**真子串**。例如 `"quick"`、`"jump"`、`"fox"`、`"brown dog"` 均为 `"The quick brown dog jumps over the lazy fox"` 的真子串。4.3 节的模式匹配 问题，问的就是「一个串是不是另一个串的子串，如果是，从哪个位置开始」。

原书【代码 4.1】给出字符串的抽象数据类型。那份声明有三处今天必须改：

1. **类名是小写的** `string`。在任何 `using namespace std;` 的翻译单元里，它与 `std::string` 构成歧义。原书正文随后又改用大写 `String`，同一章里两个名字混用。
2. `int isEmpty();` 用 `int` 表达布尔，`int find(const char c, const int start);` 用 `-1` 表示" 没找到"——与" 位置 0" 只差一个符号，调用方漏判就会把 "没找到" 当成" 匹配在开头"。
3. **修改器按值返回**：`string append(const char c);`、`string concatenate(const char* s);` 都返回 `string` 而非引用。代码 4.1 只有声明没有函数体，所以不能断定原书会丢结果；**能断定的是签名含混**——调用方无从判断 `s.append('x');` 是改了 `s`，还是返回了一个新串而 `s` 原封不动。

本书的 `String` 把这三处分别改为：类名大写、`bool empty()`、`std::optional<size_type> find(...)`、修改器返回 `String&`。

4.1.1 字符编码

字符串的逻辑元素是字符，内存中却保存**编码单元**。计算机只能识别 0、1 组成的字节，因此实际 字符集的「字符」在内存中需要用「字节」来表示，这就是所谓的**字符编码**。

在 C/C++ 等程序设计语言中，字符类型是单字节的基本数据类型，采用 ASCII（American Standard Code for Information Interchange，美国标准信息交换码）编码标准，每个字符用一个字节表示，其中低 7 位表示字符，最高位均为 0。ASCII 编码集中包含 128 个字符：编号为第 0～32 号及第 127 号的 34 个字符为控制字符或通信专用字符，其余编号处于第 33～126 号之间的 94 个字符为 通用字符。控制符包括 LF（换行）、CR（回车）、FF（换页）、DEL（删除）、BEL（振铃）等，通信专用 字符则有 SOH（文头）、EOT（文尾）、ACK（确认）等；通用字符包括常用的 52 个大小写字母、10 个 数字以及若干标点符号和运算符号，其中大写字母的编号范围是 65～90，小写字母是 97～122，10 个数字是 48～57。（按 ASCII 标准本身的划分，控制字符是第 0～31 号和第 127 号共 33 个；第 32 号是**空格**，属于可打印字符，只是看不见。原书把它算进控制字符，于是得出 34 个。33～126 号这 94 个「通用字符」的说法不受影响，它们就是除空格以外的全部可见字符。）

制定于 20 世纪 60 年代的 ASCII 标准没有考虑日后容纳中文、阿拉伯文等多种国际语言文字符号的 统一编码。在发展过程中，不同的语言形成了各自独立的字符编码系统，**这些编码系统有可能使用 相同的编号来表示各自不同的字符**：中文、日文和韩文尽管均采用两个字节的编码方式，但在编码上 存在冲突；每种

语言都存在多种编码方式，例如中文的 GB2312-80（中文简体国标码，包括 6763 个 汉字）、BIG5（中文繁体编码，容纳 13053 个汉字），日文的 S-JIS、JIS 以及韩文 Wansung 码等。这就导致不同语言系统之间进行文档交流非常困难——计算机看到的只是编码后的数字，而两种编码 可能使用相同的数字代表两个不同的字符。

为了解决这些互通问题，产生了「通用文字符号编码标准」UNICODE。简单地说，UNICODE 是一种 可伸缩的编码：既可以容纳多种语言文字的大编码集，同时又可以缩减，允许用单字节表示常用的 ASCII 符号。今天最常见的落地形式是 UTF-8——它以 1～4 个字节编码一个 Unicode 码点，且与 ASCII 完全兼容。

由此产生一条 2008 年的书还讲不到的结论：C++ 的 `char` 是一个字节，**不等于**一个「人眼看到 的字符」。`std::string::size()` 返回的是字节数，不能用来数 Unicode 字符；要数字符就得按 UTF-8 解码，或使用明确的 Unicode 库。字面量 `u8"..."` 表示 UTF-8 字节序列。好在**编码本身不改变 字符串概念和操作的本质**：查找、拼接、抽取子串的接口语义与编码无关，所以本书与原书一样，以 ASCII 为主，基本不涉及多国语言混排的问题。

4.1.2 字符的编码顺序

为了便于进行字符串的比较和运算，字符编码表一般遵循约定俗成的「**偏序编码规则**」。这里所说的 字符偏序，是指根据字符的自然含义，某些字符间具有一定的次序，例如数字符号 $0, 1, 2, \cdots, 9$ 的大小次序；为了符合其自然次序，在对这些字符编码时使其在字符集中的编码顺序和自然次序一致。令函数 $\mathrm{encode}(x)$ 为符号 x 到其 ASCII 编码（序号）的映射函数，则有

$$\mathrm{encode}('0')+1 = \mathrm{encode}('1'),\quad \mathrm{encode}('1')+1 = \mathrm{encode}('2'),\quad \cdots,\quad \mathrm{encode}('8')+$$

即 10 个数字符号的编码是连续递增的。对于字母符号 $A, B, C, \cdots, Z$ 也遵循类似的偏序规则，$a, b, \cdots, z$ 同理。于是任意两个字符 `ch1` 和 `ch2` 均可用其编码值来直接比较大小；对于字母 而言，这种大小次序的含义就是按照字典编目次序。两个字符串按其构成的字符顺序比较大小，得到 的结果是：`"monday" < "sunday" < "tuesday"`，`"123" < "1234" < "23"`。**注意这种次序仅仅是 字符串的字典次序，与日常生活中人们对其理解的次序不同**——星期一在星期二之前，整数 123 比 23 要大。以上讨论的比较次序并不限于 ASCII 编码，也适合 GB2312 等其他语言文字在排序方面的 需要。

ASCII 保留了数字 `0`–`9`、大写字母和小写字母各自连续递增的区间，所以同一区间内可用编码值比较。但这不是通用的语言学排序规则：字符的编码值比较是按码元的字典序，大小写、重音、中文排序都可能需要额外的 locale 或排序键。字符串比较先比较第一个不同的编码单元；若一个是另一个的前缀，较短者更小。因此对 `std::string` 而言 `"123" < "1234" < "23"`，但这不是整数大小比较。比较前必须先约定编码和规范化方式，不能把 “字节序” 误当成 “自然语言顺序”。

这里有个 C++ 特有的坑，必须点破：上面那条结论只对 `std::string` 成立。两个**字符串字面量**之间写 `<`，比的是两个数组的**地址**，不是内容：

```
$ g++ -std=c++17 -Wall -Wextra litcmp.cpp
litcmp.cpp:7:25: warning: comparison between two arrays [-Warray-compare]
    7 |     std::cout << ("123" < "1234") << ' ' << ("1234" < "23") << '\n';
      |                   ~~~~~~^~~~~~~~
$ ./litcmp
true true      # ("123" < std::string("1234"))、(std::string("1234") < "23")
false true     # ("123" < "1234")、("1234" < "23")——第一个的答案反了
```

同一行代码，两边只要有一边是 `std::string`，比的就是内容；两边都是裸字面量，比的就是地址，而地址由编译器摆放决定。所以本书凡是比较字符串，一律先落到 `String`/`std::string` 上，再谈编码顺序。

4.2 字符串的存储结构和实现

选择字符串的存储结构时要考虑字符串的**变长特点**，同时需结合具体的应用分析各种存储方案的 利弊。某些应用中字符串的长度变化非常显著——短如单词，长为文件；从统计分布来看，字符串 长度分布的方差很大。**对于这种情况，用静态长度的向量作为存储结构是不恰当的**。字符串的变长 特点是无法回避的：拼接、查找、置换和模式匹配等操作本身都涉及变长的字符串操作，这些操作 时间开销大，必须精心设计算法、选择恰当的串存储结构。本节重点讨论程序执行过程中的字符串 **变长存储**问题。

为什么这一节没有 Python 版

本节的对象是一个自管理的字符缓冲区：长度、容量、结尾空字符、深复制和移动后的源对象状态都属于接口契约。Python `str` 是不可变的运行时对象，`list` 也不会让读者实现缓冲区的分配、释放和强异常保证；把它们当作 “字符串类的 Python 版” 会跳过本节的存储布局和所有权问题。因此算法 4.3 的模式匹配有 Python 版，4.2 的字符串类只保留 C++ 实现。

`String` 采用动态变长的存储结构：内部持有一块以 `'\0'` 结尾的字符数组和当前长度，构造时按初值长度分配，赋值时按新长度重新分配。这正是本节要教的内容，所以缓冲区是**裸** `char*`——换成 `std::string` 这一节就没了。

4.2.1 字符串的顺序存储

字符串用一段连续字符数组按字符的逻辑顺序存储，末尾保留 `\0` 作为 C 风格字符串的结束标志，并单独记录不含终止符的长度 `length`。因此容量至少要能容纳 `length + 1` 个字符；`\0` 不是字符串内容，不能计入长度。

顺序存储的字符串**适合访问字符串中单个字符或连续的一组字符**：按下标取第 i 个字符是 $O(1)$。但进行插入和删除（增减字符）操作就不是很方便，需要移动插入或删除点后面的所有字符；拼接和 抽取子串也要复制一段连续区域。若

容量不足，通常重新申请更大的数组、复制旧内容、释放旧数组，再更新指针和长度；这也是后面 `String` 类需要自己维护所有权、拷贝和析构的原因。

C/C++ 的标准字符串是顺序存储方案的典型代表：在程序中用 `char s[M];` 的形式定义字符串变量，其中 `M` 是整型常数，表示字符数组的长度。标准字符串需要在其末尾带一个结束标记 `'\0'` 来表示 串的结束，因此**字符串的长度不能超过最大长度** `M - 1`。原书图 4.1 画的就是

```
char s1[12] = "Hello world";
char s2[8]  = "2008";
char s3[6];
```

这三个数组在内存里的样子。**存储字符串的数组是静态定长的**，程序运行中一旦产生更长的字符串 就会造成数组溢出，这给编写和调试程序带来不便——这正是本节接下来要用一个类来解决的问题。

还有一处「字符串处理中非常容易犯的错误」值得单独点出：**赋值语句** `s1 = s2` **不能被理解为把** `s2` **的内容复制到** `s1`。原书的说法是「C++ 中的字符数组是用字符指针定义的，指向字符数组的 始址」，这不准确——数组不是指针，只是在多数表达式里退化为指向首元素的指针（`sizeof s1` 是 12，不是指针的大小）。于是（勘误 E23）：

- 若 `s1`、`s2` 是上面那样的字符数组，`s1 = s2` **根本编译不过**——数组不可赋值，g++ 报 `error: incompatible types in assignment of 'char [8]' to 'char [12]'`，clang++ 报 `error: array type 'char[12]' is not assignable`。复制内容要用 `strcpy`。
- 若它们是 `char*`，赋值能通过，但复制的只是**地址**：两个指针从此指向同一块内存，改一个，另一个跟着变。

第二种正是本书的 `String` 必须自己写拷贝赋值运算符的原因（4.2.2 节）：编译器默认的逐成员 复制只复制 `str` 指针，两个对象共用一块缓冲区，析构时重复释放。

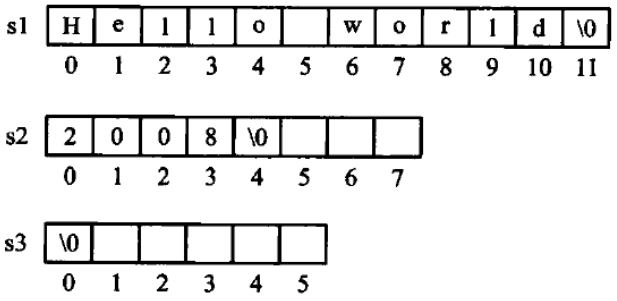


图 4.1　C 风格字符串的变量说明示意。`'\0'` 是 ASCII 里 8 位全 0 的 NULL 符，只作结束标记用，不算内容；说明变量时可以用字符串常量给初值，也可以不给——原书说图中的 `s3` 没给初值，「意味着它存储空字符串」。**这只对全局或** `static` **数组成立**（它们会被零初始化）；写在函数里的局部数组不给初值，内容

是不确定的，在写入之前把它当字符串读（例如 `strlen(s3)`）是未定义行为。要一个空串，就写 `char s3[6] = "";`。

标准库 `<string.h>`（C++ 里是 `<cstring>`）提供了若干处理字符串的常用函数，原书表 4.1 列出了 常用的几个：

表 4.1　标准串函数

函数名	功能说明
`size_t strlen(char* s)`	求字符串 `s` 的当前长度，不计结束符；空字符串的长度为 0
`char* strcpy(char* s1, const char* s2)`	将 `s2` 复制到 `s1`，并返回一个指向 `s1` 开始的指针
`char* strcat(char* s1, const char* s2)`	将 `s2` 拼接到 `s1` 尾部
`int strcmp(const char* s1, const char* s2)`	比较 `s1` 和 `s2`：全同返回 0，`s1` 大于 `s2` 返回正数，小于返回负数
`char* strchr(char* s, char c)`	定位到 `s` 中第一次出现字符 `c` 的位置，没有则返回空指针
`char* strrchr(char* s, char c)`	从尾部逆向定位，返回最后一次出现 `c` 的位置，没有则返回空指针

此外，输入/输出的处理也是标准串函数库中的主要组成部分，例如 `cin >> s1;` 从标准输入读取 字符串到串变量 `s1`，`cout << s1;` 把 `s1` 的内容输出到标准输出。

例如对图 4.1 中的 `s1` 调用 `strchr(s1, 'o')` 和 `strrchr(s1, 'o')`，将分别得到字符 `'o'` 的第一次和 倒数第一次出现位置：

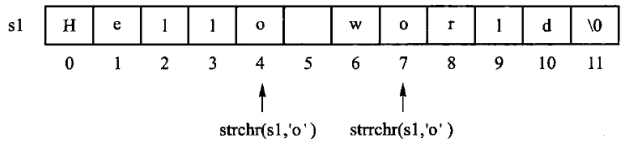


图 4.2　在字符串 `s1` 中定位字符 `'o'`。两个函数返回的都是指针，指向找到的那个字符；找不到返回空指针。本书的 `String` 用下标和 `std::optional` 表达同一件事——「没找到」不再靠一个特殊指针值来表示。

4.2.2 字符串类 class String 的存储结构

字符串的顺序存储方式简单易实现，C++ 的标准串及其相关的标准函数也提供了若干实用的函数，**但终归难以避免其静态定长的局限**；而在实际应用中，大多的字符串变量具有动态变化的长度。

那么为什么不用链式的变长结构? 因为**每一个链指针比一个字符所占的存储空间还大**——用链表 存字符, 指针域的开销会几倍于数据域（回到 2.4 节那条「指

针所占比例超过 1:1 就要慎重」的判据）。所以字符串的变长存储走的是另一条路：仍然顺序存放，但**把这块缓冲区的分配与释放交给一个类 去管**。

顺序存储只说了「字符放在哪」，还没说「这块内存归谁、什么时候还」。把它包成一个类之后，这两件事才有着落：类里存一个指向字符数组的指针和一个长度，构造时申请、析构时释放、拷贝时另开一份——本节的教学内容就是这套**变长存储管理**。

教学版：完整实现

下面是一份**完整的、能直接编译运行的**字符串类。一个文件、一个类。后面 4.2.2、4.2.3 两节就是把它拆开逐段讲。

```
// 字符串类 String ——教学版。原书【代码 4.1】【算法 4.3】【算法 4.4】【算法 4.5】。
//
// 一个文件、一个类、能直接编译运行，给「第一次读这一节」的人看。
//
// 本节的教学内容是 ** 字符串的变长存储管理 **：动态分配、按长度重新开辟、拷贝与释放。
// 所以这里是手写的 char* 缓冲区，不是 std::string——换成 std::string，这一节就没了。
//
// 与 modern.hpp（工程版）的分工：
//   教学版  三法则（析构 + 拷贝构造 + 拷贝赋值）；
//   工程版  补齐移动语义（移动之后 data_ 为空，读取路径要多一层 raw() 兜底）、
//           比较运算符全家、copy-and-swap。
// 两份都在闸门里真编译真运行。先读这一份，4.2a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <cstring>
#include <optional>
#include <stdexcept>

class String {
public:
    using size_type = std::size_t;

    // 空串。** 注意它仍然申请了 1 个字节 **，里面放一个 '\0'。
    // 这样 c_str() 永远返回一个合法的 C 字符串，调用方不必先判空指针。
    String() : data_(new char[1]), size_(0) {
        data_[0] = '\0';
    }

    // 从 C 字符串构造。参数是 `const char*` 而不是原书的 `char*`——
    // 原书那个签名让它自己书里的例子 `String s1 = "Hello";` 从 C++11 起就编译不过：
    // 字符串字面量的类型是 const char[6]，绑不到 char*。
    //
```

```
    // 这里 ** 故意不加 explicit**，为的就是保住 `String s1 = "Hello";` 这种写法。
    String(const char* s) {   // NOLINT(google-explicit-constructor)
        if (s == nullptr) {
            throw std::invalid_argument("String: 不能用空指针构造字符串");
        }
        size_ = std::strlen(s);
        data_ = new char[size_ + 1];
        std::memcpy(data_, s, size_ + 1);   // +1 把结尾那个 '\0' 也带上
    }

    ~String() { delete[] data_; }

    // 三法则：自己管着 new 出来的缓冲区，拷贝必须自己写。
    // 原书正文里描述过赋值时「必须释放 s1 的原有空间」，却没有把拷贝构造和
    // 拷贝赋值作为清单给出。只有析构没有这两个，一次 `String b = a;` 就是二次释放。
    String(const String& other) : data_(new char[other.size_ + 1]),
↪   size_(other.size_) {
        std::memcpy(data_, other.data_, size_ + 1);
    }

    String& operator=(const String& other) {
        if (this == &other) {
            return *this;
        }
        char* fresh = new char[other.size_ + 1];   // 先备好新的
        std::memcpy(fresh, other.data_, other.size_ + 1);
        delete[] data_;                            // 再释放旧的
        data_ = fresh;
        size_ = other.size_;
        return *this;
    }

    size_type size() const { return size_; }
    size_type length() const { return size_; }
    bool empty() const { return size_ == 0; }
    const char* c_str() const { return data_; }

    void clear() {
        char* fresh = new char[1];
        fresh[0] = '\0';
        delete[] data_;
        data_ = fresh;
        size_ = 0;
    }

    // 按下标取字符。越界抛异常，不是返回一个随便什么值。
    char at(size_type index) const {
```

```
    if (index >= size_) {
        throw std::out_of_range("String::at: 下标越界");
    }
    return data_[index];
}

// 在串尾添加一个字符。
//
// ** 变长存储的代价在这里看得最清楚 **：字符串长度变了，就得重新申请一块、
// 把老内容拷过去、再把老的还回去。所以 append 一个字符是 O(n)，不是 O(1)。
//
// 返回自身引用，于是 `s.append('a').append('b')` 可以连着写；
// 原书【代码 4.1】声明的是 ** 按值返回 **，调用方从签名看不出它改不改本串。
String& append(char c) {
    char* fresh = new char[size_ + 2];        // 老内容 + 新字符 + '\0'
    std::memcpy(fresh, data_, size_);
    fresh[size_] = c;
    fresh[size_ + 1] = '\0';
    delete[] data_;
    data_ = fresh;
    ++size_;
    return *this;
}

// 把 s 接在本串后面。同样是「重新申请、拷两段、释放旧的」。
String& concatenate(const char* s) {
    if (s == nullptr) {
        throw std::invalid_argument("String::concatenate: 空指针");
    }
    size_type extra = std::strlen(s);
    char* fresh = new char[size_ + extra + 1];
    std::memcpy(fresh, data_, size_);
    std::memcpy(fresh + size_, s, extra + 1);
    delete[] data_;
    data_ = fresh;
    size_ += extra;
    return *this;
}

// 【算法 4.5】从 pos 开始取长度至多 len 的子串。
//
// 原书在 `pos >= size` 时 `return NULL;`。那不是「返回空串」——
// NULL 会去走 String(char*) 构造函数，然后 strlen(nullptr) 当场段错误。
// 这里越界就抛异常，让错误停在发生的地方。
// pos == size() 是合法的，得到空串（「从末尾取 0 个字符」）。
String substr(size_type pos, size_type len) const {
    if (pos > size_) {
```

```
            throw std::out_of_range("String::substr: 起始位置越界");
        }
        size_type available = size_ - pos;
        size_type take = (len < available) ? len : available;   // 原书的 if (n >
        ↪ left) n = left
        String result;
        char* fresh = new char[take + 1];
        std::memcpy(fresh, data_ + pos, take);
        fresh[take] = '\0';
        delete[] result.data_;
        result.data_ = fresh;
        result.size_ = take;
        return result;
    }

    // 【算法 4.4】从 start 开始查找字符 c。找到返回下标，没找到返回空 optional。
    // 原书用 -1 表示没找到——与「位置 0」只差一个符号，忘了判就会读错位置。
    std::optional<size_type> find(char c, size_type start = 0) const {
        for (size_type i = start; i < size_; ++i) {
            if (data_[i] == c) {
                return i;
            }
        }
        return std::nullopt;
    }

    // 【算法 4.3】三路比较：负 / 零 / 正 表示 小于 / 等于 / 大于。
    //
    // 原书自己实现了一个 strcmp，返回值固定为 -1/0/1，并在正文里说
    // 「这与 C/C++ 语言中通常的大小比较习惯不一致」——其实不一致的是原书自己：
    // 标准 strcmp 返回的就是差值的符号，调用方只该看符号，不该看具体数值。
    int compare(const String& other) const {
        return std::strcmp(data_, other.data_);
    }

private:
    char* data_;        // 以 '\0' 结尾的字符数组，永远非空
    size_type size_;    // 字符个数，不含结尾的 '\0'
};

inline bool operator==(const String& a, const String& b) { return a.compare(b) == 0;
↪ }
inline bool operator!=(const String& a, const String& b) { return a.compare(b) != 0;
↪ }
inline bool operator<(const String& a, const String& b) { return a.compare(b) < 0; }
```

构造与所有权

原书【算法 4.4】的构造函数有两处问题。

第一处，`assert(str != '\0')` **本身就编译不过**：`'\0'` 是 `char` 而不是空指针常量，拿指针与它比较是 ill-formed（`error: ISO C++ forbids comparison between pointer and integer`）。而且即便改写成 `assert(str != nullptr)` 也是无效断言——`new` 失败时抛 `std::bad_alloc`，从不返回空指针；`assert` 更会在 `NDEBUG` 构建里整个消失。

第二处，参数类型是 `char*`。原书 4.2.2 节自己写下：

`String s1 = "Hello";` 隐含地调用构造函数 `String::String(char* s)`

而字符串字面量的类型是 `const char[6]`，转成 `char*` 在 C++11 起已被移除。GCC 默认降级为警告，在本书的 `-Werror` 构建下即是错误。

还有一处原书没有做的检查：构造函数对 `s == nullptr` 毫无防备——而 4.2.3 节的 `Substr` 恰好会喂给它一个空指针（见 4.2.3 节）。

原书的类里有 `~string()`，却从未把拷贝构造与拷贝赋值作为清单给出（正文描述过赋值时" 必须释放 s1 的原有空间"，但没有代码）。只要有析构而没有这两个，一次 `String b = a;` 就是二次释放——与第 2、3 章是同一个错误。

教学版按三**法则**补齐了这两个（析构 + 拷贝构造 + 拷贝赋值），拷贝赋值走的是「先备好新缓冲区，再释放旧的，最后接管」——顺序反过来，一旦 `new` 抛异常，对象就停在「指针已释放」的破碎状态。

原书用两张图说明这套变长管理。`String s1 = "Hello";` 会在动态存储区开一个长度为 6 的字符数组（5 个字符加结束符），把初值顺序存进去：

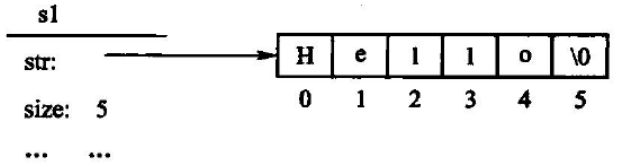


图 4.3　创建字符串：对象里只有一个指针和一个长度，字符本体在堆上。

再写 `String s2 = "Hello world"; s1 = s2;`，新内容比旧的长，装不下，于是必须先另开一块新数组、把内容复制过去，旧数组（图 4.4 中的灰格）释放掉，`str` 改指新数组：

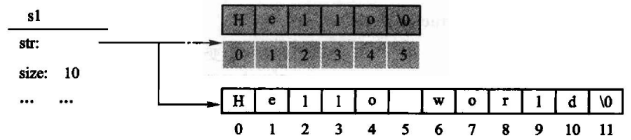


图 4.4　赋值运算：`String` 按当前串长动态调整空间，这就是「变长管理」的全部含义。注意图上那块灰色的旧空间——**它必须释放，而且必须在新空间准备好之后再释放**。原书正文说到了这一步，却没有给出拷贝赋值的代码；漏掉它，`String b = a;` 就是二次释放。

工程版还多两个**移动**操作，见 4.2a。

4.2.3 字符串运算的实现

追加与拼接

`append(char)` 与 `concatenate(const char*)` 的形状是一样的三步：**重新申请一块、把老内容拷过去、释放旧块**。所以追加一个字符是 O(n)，不是 O(1)——这就是变长串管理的代价，也是后面章节讨论"预留容量"的动机。

两者都返回 `String&`，于是 `s.append('a').append('b')` 可以连着写，而且调用方从签名一眼看出它改的是本串。原书【代码 4.1】声明的是**按值返回**，`s.append('x');` 到底改不改 `s`，从签名看不出来。

抽取子串

原书【算法 4.5】在起始位置越界时写 `return NULL;`。这里的返回类型是 `String`，所以 `NULL` 不是"空串"：它先转成 `char*`，再走 `String(char*)` 构造函数，于是 `strlen(nullptr)`。这段**能编译**，崩在运行期：

```
$ ./s7
准备调用 s.Substr(99, 1)——原书会走到 return NULL 那一支
runtime error: null pointer passed as argument 1, which is declared to never be null
AddressSanitizer:DEADLYSIGNAL
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000
```

本书越界就抛 `std::out_of_range`，让错误停在发生的地方，而不是变成调用方某处的段错误。

`pos == size()` 是**合法**的，得到空串——与"从末尾取 0 个字符"的直觉一致；`len` 超出剩余长度时**截断**而不报错，与原书的 `if (n > left) n = left;` 语义一致。

查找与比较

对应【算法 4.4】的 `find` 返回 `std::optional<size_type>`：找到是下标，没找到是空盒子。原书用 `-1` 表示没找到——与"位置 0"只差一个符号，调用方漏判就会把"没找到"当成"匹配在开头"。这与 2.2.2 节 `getPos` 那一处是同一个问题、同一个改法。

关于【算法 4.3】：原书自己实现了一个 `strcmp`，固定返回 −1/0/1，并在正文里说"这与 C/C++ 语言中通常的大小比较习惯 (0 和非 0) 不一致"。其实标准 `strcmp` 返回的就是差值的符号，调用方本来就只该看符号——不一致的是原书固定返回 ±1 的写法。本书保持标准语义，并据此提供关系运算符。

（另外补一句实测结论：原书那个与标准库同名同签名的 `strcmp` 定义，既能编译也能链接，不构成冲突——这是我们查过之后否掉的一个猜测，记在 `legacy.md` 第五节。）

4.2a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。工程版在 `code/ch04/string_class/modern.hpp`。它比教学版多的东西里，只有一件是这一章特有的：**移动之后的那个空壳怎么办**。

```
// 原书只在正文里描述了赋值时" 必须释放 s1 的原有空间 (delete [] s1.str)",
// 却没有把拷贝构造和拷贝赋值作为清单给出。只要有析构函数而没有这两个,
// 一次 `String b = a;` 就是二次释放——与第 2、3 章是同一个错误。
String(const String& other) : data_(new char[other.size_ + 1]), size_(other.size_) {
    // 注意读的是 other.raw() 不是 other.data_: 源可能是被移动过的对象 (data_ 为空),
    // 从空指针 memcpy 即便长度为 0 也是未定义行为。
    std::memcpy(data_, other.raw(), size_ + 1);
}

String& operator=(const String& other) {
    if (this != &other) {
        String copy(other);  // 拷贝并交换: 自赋值安全，且拷贝失败时原对象不受影响
        swap(copy);
    }
    return *this;
}

/// 移动 ** 不分配 **: 直接接管缓冲区，把对方置为 nullptr。
/// 被移动方仍是一个可用的空串——读取路径统一走 raw()，它在 data_ 为空时返回 ""。
///  (若在这里 new 一块空缓冲区来" 修复" 对方，移动就不再是 noexcept 的了。)
String(String&& other) noexcept : data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;
    other.size_ = 0;
}

String& operator=(String&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}

~String() { delete[] data_; }
```

移动操作声明为 `noexcept`，因此**不能在里面分配**——所以被移动方的 `data_` 只能置空，不能塞一块新的 1 字节缓冲区进去。可是教学版承诺过「`c_str()` 永远不是空指针」，这个承诺不能因为被移动过就作废。

工程版的解法是加一个私有的 `raw()`：`data_` 为空时返回一个静态空串。读取

路径全部走它，于是「被移动之后仍是可用的空串」这个保证**不花任何分配**就成立。教学版没有移动操作，也就不需要这一层。

其余的差别与前几章相同：`[[nodiscard]]`、`noexcept`、copy-and-swap、关系运算符全家（`< > <= >=` 都由 `compare()` 派生）。

4.3 字符串的模式匹配

字符串模式匹配是一种常用的运算。所谓**模式匹配**（pattern matching），简单地说就是在**文本**（text）中寻找一个给定的**模式**（pattern）：通常文本都很大，而模式则比较短小。典型的例子包括 文本编辑和 DNA 分析——编辑文本时人们经常使用「替换」命令对文中的某个字符串或语句进行替换，此时便需先找到要被替换的内容；在文本编辑器中，模式通常为一个单词，长度在 10 个字符左右，而文本的长度则从几百字到上百万字不等。在生物信息中，DNA 信息一般由 A、C、G、T 这 4 个符号 组成，基因一般也就是几百个字符的长度，而**人类染色体的长度却有 30 亿之多**。显而易见，这些 应用对匹配算法的效率要求很高。

模式匹配有精确匹配和近似匹配两类。**精确匹配**（exact matching）：如果在目标 T 中至少一处 存在模式 P，则称匹配成功，否则即使目标与模式只有一个字符不同也算匹配失败。想要找的词可以 是单选的（例如 `"set"`），也可以是多选的——例如模式 `"s?t"` 表示任何以 `s` 打头、以 `t` 结尾、中间夹一个字符的串都可与之匹配（`"sat"`、`"set"`、`"sit"` 都可以），模式中的符号 `?` 称为**通配符**；更复杂的模式可以采用正则表达式来表示。**近似匹配**（approximate matching）：如果模式 P 与 目标 T（或其子串）存在某种程度的相似，则认为匹配成功；常用的衡量字符串相似度的方法，是根据 一个串转换成另一个串所需的基本操作（插入、删除、替换）数目来确定的。本节与原书一样，只讨论 **精确匹配中的单选情况**，不涉及通配符、正则表达式与近似匹配。

给定目标串 T 和模式串 P，模式匹配要回答：P 是否作为 T 的连续子串出现，若出现，首次出现 在哪个位置。用原书的记号写出来：T_j、P_j 表示其在位置 j 上的字符，$|T|$、$|P|$ 表示长度，a^n 表示由 n 个字符 a 组成的字符串，**P 和 T 的第一个字符都从位置 0 开始**；模式匹配 就是要在 T 中找到一个 j，使得

$$P_0P_1P_2\cdots P_{m-2}P_{m-1} = T_jT_{j+1}T_{j+2}\cdots T_{j+m-2}T_{j+m-1}$$

否则称模式匹配失败。**记住「都从位置 0 开始」这句话**——4.3.1 节会看到，原书自己的两个算法 都没有遵守它。

本节的教学内容是**算法本身**，因此下面的实现用 `std::string_view` 接收输入：不拷贝、不拥有，把注意力留在匹配过程上。字符串容器的实现是 4.2 节的事。

4.3.1 朴素的模式匹配算法

朴素算法的思路直白：把模式的首字符对齐目标的每一个位置，逐字符比较；某次比较失败（称为一次 **失配**）时，就把模式对于 T 向右移动一个字符位置，重新开始下一趟匹配。如此不断重复，直到 某趟配串成功返回，或者比较到目标串

的结束也没有出现配串，则匹配失败。

时间效率分情况看。最佳情况是目标首位置开始的子串便是所要找的配串，只要通过 $|P|$ 次比较 即可完成匹配，代价 $O(|P|)$，与模式的长度成正比。如果最终匹配失败，其最佳情况是每次都在模式 的首字符比较时就不等、于是右移一位，直到在目标的 $|T|-|P|$ 位置处依然与模式的首字符不匹配，整个过程中至少比较了 $(|T|-|P|+1)$ 次，时间代价 $O(|T|-|P|+1)$。

最差情况需要 $O(|T|\cdot|P|)$：例如目标串 T 形如 a^n，而模式 P 形如 $a^{m-1}b$，每趟都 是在模式的最后一个字符处不匹配，即每趟都需要进行 $|P|$ 次比较，再把模式右移一位、再次从模式 头比较，最多需要 $(|T|-|P|+1)$ 趟，因此总共需要比较 $|P|(|T|-|P|+1)$ 次。**此类情况很少 出现在文本中，却经常出现在 DNA 信息和图像信息中**——这正是朴素算法在生物计算里不够用的原因。

平均代价同时依赖于目标和模式中字符的分布概率。假设一个字符串中只允许使用 2 种字符，那么使用 任一字符的概率为 $1/2$：对于第 j 趟扫描来说，仅比较一次的概率为 $1/2$，比较两次的概率为 $1/4$，比较 k 次（$k<|P|$，即前 $k-1$ 个字符相等、第 k 个不等）的概率为 2^{-k}；比较满 $|P|$ 次时，第 $|P|$ 个字符相等（配串成功）或不等都算 $|P|$ 次，概率为 $2^{-(|P|-1)}$——原书写成 $2^{-|P|}$，漏了配串成功的那一半，各情况的概率加起来不到 1。因此平均情况下第 j 趟扫描的比较次数为 $\sum_{k=1}^{|P|-1}k/2^k+|P|/2^{|P|-1}=2-2^{1-|P|}<2$（原书按 $\sum_{k=1}^{|P|}k/2^k$ 估算，同样小于 2，结论不变）。最多有 $|T|-|P|+1$ 趟，所以平均比较总次数小于 $2(|T|-|P|+1)<2|T|$ 次。字符种数为 $|A|$ 时同理，每趟平均不到 $|A|/(|A|-1)$ 次——字符种类越多，越早遇到失配。

原书接着说：「若运用马尔科夫链（Markov Chain）的有关理论，可以估算一个更好的比较次数 $2^{|P|+1}-2$；在通用的情况下，如果字符串中允许使用的字符数目为 $|A|$，则平均比较次数为 $(|A|^{|P|+1}-|A|)/(|A|-1)$。」**这两个式子算的不是比较次数**（勘误 E24）：式子里根本没有 $|T|$，又随 $|P|$ 指数增长，不可能比 $2|T|$「更好」。它们是另一个量——在随机目标串里逐个读入 字符，**模式第一次出现时平均读到了第几个字符**，而且只对 `aa…a` 这类自身重叠最多的模式成立。

以 2 种字符、`$P = $ "00"` 为例：读入时只需记住「已凑齐几个 0」，状态 S_0（没有进展）读到 0 进入 S_1、读到 1 留在 S_0；S_1 读到 0 即配串成功、读到 1 则退回 S_0。下一步只取决于当前 状态，这就是马尔科夫链。记 E_0、E_1 为从两个状态出发平均还要读入的字符数：

$$E_1=1+\frac{1}{2}\cdot 0+\frac{1}{2}E_0,\qquad E_0=1+\frac{1}{2}E_1+\frac{1}{2}E_0$$

解得 $E_0=6=2^{2+1}-2$，与原书的式子相符。换成 `$P = $ "01"`：S_1 读到 0 时这个 0 可以当新的 开头，仍留在 S_1，于是 $E_1=2$、$E_0=4$——同样长度，期望位置却不同。一般地，期望位置等于 $\sum|A|^k$，k 取模式所有「既是前缀又

是后缀」的长度（含模式本身）：`"000"` 是 $2+4+8=14$，`"001"` 只有 8。原书的通式 $(|A|^{|P|+1}-|A|)/(|A|-1)=|A|+|A|^2+\cdots+|A|^{|P|}$ 正是把每个长度都算上的最坏情形。前缀与后缀的这种重合，恰好也是 4.3.2 节特征向量要刻画的东西。

两个结论可以合起来用：如果找到第一次出现就停，要试的对齐位置大约是这个期望位置，每个位置平均 比较不到 2 次；如果匹配失败，就得把整个目标扫完，比较次数在 $|T|-|P|+1$ 与 $2(|T|-|P|+1)$ 之间。

问题出在哪里。尽管朴素算法思想简单易懂，但效率较低：一旦某趟匹配中发生失配，**无论模式的 具体情况如何，都采用模式右移一位开始下一趟的匹配**。这会导致很多冗余的比较，因为目标 T 中 的字符会多次与模式中的字符进行比较，造成**目标的回溯**。例如模式 `$P = $ "abacab"` 与目标 `$T = $ "abacaabaccabacabaa"` 的第 1 趟比较中，发现 $P_5 \neq T_5$ 时把模式右移一位、开始第 2 趟，使得 P_0 与 T_1 比较；其实之前的匹配中 T_1 已与 P_1 比较过而且相等，由于 P_1 与 P_0 不同，因此可知 P_0 与 T_1 肯定不等，**根本用不着重新比较**。换言之，利用模式本身的构成特征 以及上趟匹配的比较结果，就能知道 P_0 与 T_1 肯定不等。

一般来说，右移的位数越多效率就越好，但前提是要保证不能丢失 P 和 T 的任何可匹配子串。Knuth、Morris、Pratt 等人发现，**每次右移的位数存在，且与目标串无关，仅依赖于模式本身**，因此他们对朴素算法进行了改进：预先处理模式本身，分析其字符分布状况，为模式中的每一个字符 计算失配时应该右移的位数——此即 4.3.2 节所谓的**字符串的特征向量**，改进后的算法就是 4.3.3 节 的 KMP 算法。

```cpp
/// 朴素模式匹配：返回 pattern 在 text 中首次出现的 ** 起始下标 **；没有则 std::nullopt。
///
/// 与原书【算法 4.6】的关键差别是返回值：原书写的是 `return (j - pLen + 1);`，
/// 而在 0 起始的下标体系里正确的是 `j - pLen`——** 原书这里差了 1**。
/// 用书中自己的例子可以当场看出来：T="abcddabcab..."、P="abcdaabcab" 匹配始于下标 10，
/// 原书返回 11（证据见 legacy.md 缺陷 1）。
///
/// 空模式约定返回 0（与 std::string::find("") 一致）；原书用 assert(m>0) 把它挡在门外，
/// 而 assert 在 NDEBUG 下会被整个编译掉。
[[nodiscard]] inline std::optional<std::size_t> naive_search(std::string_view text,
                                                             std::string_view pattern) {
    const std::size_t n = text.size();
    const std::size_t m = pattern.size();
    if (m == 0) {
        return std::size_t{0};
    }
    if (n < m) {
        return std::nullopt;
    }
    std::size_t i = 0;  // 模式下标
    std::size_t j = 0;  // 目标下标
```

```
    while (i < m && j < n) {
        if (text[j] == pattern[i]) {
            ++i;
            ++j;
        } else {
            j = j - i + 1;  // 回退到本趟起点的下一个位置
            i = 0;
        }
    }
    return i >= m ? std::optional<std::size_t>(j - m) : std::nullopt;
}
```

```
def naive_search(text: str, pattern: str) -> int | None:
    """ 朴素匹配：返回首次出现的 0 起始下标。"""
    if not pattern:
        return 0
    i = 0
    j = 0
    while i < len(pattern) and j < len(text):
        if text[j] == pattern[i]:
            i += 1
            j += 1
        else:
            j = j - i + 1
            i = 0
    return j - len(pattern) if i == len(pattern) else None
```

失配时 `j = j - i + 1` 把目标下标退回本趟起点的**下一个**位置——这一步的"+1"是对的，它表示"换一个起点重来"。

原书用 `T = "abacaabaccabacabaa"`、`P = "abacab"` 走了一遍全过程：第 1 趟在目标的 `a` 与模式的 `b` 处失配，模式右移一位重来，如此直到第 11 趟在子串处配上，返回首位置 10。

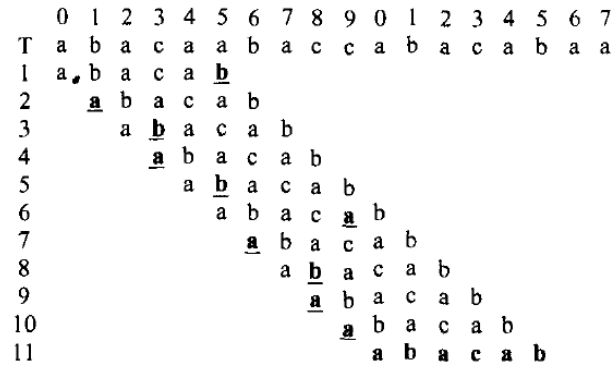


图 4.6　朴素匹配的示例。加粗带下划线的是当前失配的那一对字符。可以数一数目标里有多少字符被反复比较过——这些重复比较正是 KMP 要省掉的东西。

三种极端情况值得分开看。**最好**：目标开头那一段就是配串，比 $|P|$ 次就完事，$O(|P|)$。

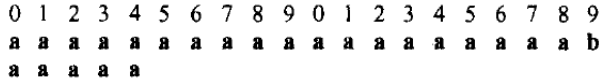


图 4.7　朴素匹配的最佳情况：第一趟就配上。

匹配失败中的最好情况：每趟都在模式首字符处就不等，右移一位再试，一共比较约 $|T|-|P|+1$ 次，$O(|T|-|P|+1)$。

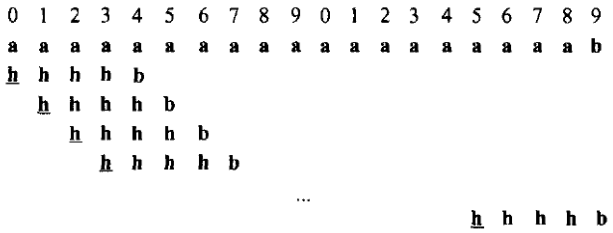


图 4.8　匹配失败的最佳情况：每趟只比一次就失配。

最坏：目标形如 a^n、模式形如 $a^{m-1}b$，每趟都要比到模式的最后一个字符才失配，$|P|(|T|-|P|+1)$ 次比较，即 $O(|T|\cdot|P|)$。

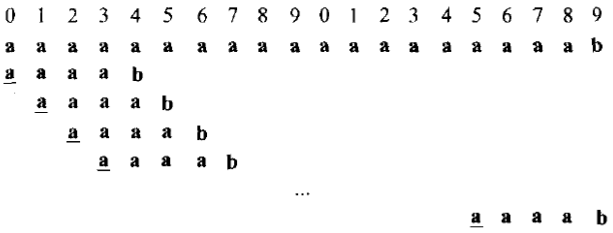


图 4.9　朴素匹配的最差情况。这种输入在自然语言文本里少见，在 DNA 序列和图像数据里却很常见——这正是需要 KMP 的实际理由。

平均情况依赖字符分布：字符等概率出现时，每趟平均比较不到 $|A|/(|A|-1)$ 次，两种字符时不到 2 次，全程不到 $2|T|$ 次。原书在这里给的 $(|A|^{|P|+1}-|A|)/(|A|-1)$ 算的不是比较次数，而是模式首次出现的期望位置（勘误 E24，见上文）。

一处必须指出的错误：原书的返回值差 1

原书【算法 4.6】与【算法 4.8】在匹配成功时都写：

```
if (i >= pLen) return (j - pLen + 1);
```

匹配成功时 j 已经走到匹配段的**末尾之后**，所以起始位置是 j - pLen。那个 +1 是多余的。把原书两段代码照抄进程序，拿标准库的 find 做参照：

```
T=abc                                  P=abc          原书朴素 =  1  正确答案 =  0
T=xabc                                 P=abc          原书朴素 =  2  正确答案 =  1
T=aaab                                 P=ab           原书朴素 =  3  正确答案 =  2
T=abcddabcababcdaabcababcdaabcabaa P=abcdaabcab   原书朴素 = 11  正确答案 = 10
```

每一组都恰好多 1，最后一组正是书中图 4.12 自己用来演示 KMP 的那对串。原书用它逐趟画了匹配过程，却没有给出返回值，这个错误因此在书里没有暴露。

这不是排版或 OCR 造成的，有三重佐证：j - pLen + 1 在两个算法里独立印出、写法一致；同一段代码里的 j = j - i + 1 说明作者用的就是 0 起始下标；而 4.3 节开头的约定更是把这一点写死了——

> P 和 T 的第一个字符都从位置 0 开始。

本书的实现返回 j - m，并且**每一条匹配用例都拿标准库的 find 逐个对拍**，再加 3000 组随机对拍。只断言"找到了"的测试，在原书那份实现下同样全绿——那样的测试等于没写。

4.3.2 字符串的特征向量

KMP 的想法是：失配时不必把模式退回从头开始，因为**已经匹配上的那一段本身携带了信息**。这句话抽象，用一个例子就清楚了。

设目标串 T = ababab d，模式 P = ababd，从 T 的位置 0 开始比：

```
位置：  0 1 2 3 4 5 6
T   :  a b a b a b d
P   :  a b a b d
               ↑ 这里失配（T[4]='a'，P[4]='d'）
```

失配前已经匹配上了 abab 这四个字符。**朴素算法**到这里会把模式整体右移一位、从 T[1] 重新 逐字符比起——但这其实浪费了信息：我们已经知道 T[0..3] 就是 abab。

KMP 的做法是问：abab 的**前缀**和**后缀**里，最长的相同的一段是多少？

```
abab 的前缀：a，ab，aba
abab 的后缀：b，ab，bab
最长相同的一段："ab"，长度 2
```

这个 2 的意义是：abab 的末尾两个字符 ab，恰好也是 abab 的开头两个字符。而模式的开头 也是 ab。所以模式可以**一次右移 4 − 2 = 2 位**直接对上去，并且移过去之后**前两个字符 不必再比**——它们必然相同：

```
位置:  0 1 2 3 4 5 6
T   :  a b a b a b d
P   :      a b a b d
             ↑ 这两个已知相同，从这里继续比
```

接着从 P[2] 与 T[4] 往下比，一路比到底，在位置 2 匹配成功。整个过程中 **T 的下标从来没有往回退过**，这正是 KMP 优于朴素算法的地方：朴素算法最坏要把 T 的每个位置都当一次起点重来。

把「每个位置失配时该退到哪里」对模式的每个位置预先算出来，就是特征向量（next 数组）。上面这个例子用本章的实现跑出来是：

```
next(ababd) = -1 0 -1 0 2
kmp_search("abababd", "ababd") = 2
std::string::find 对照         = 2
```

`next[4] = 2` 正是「在 P[4] 处失配时，模式的比较位置退到 2」——和上面手推的一致。（这里印的是原书【算法 4.7】的**优化版** next，所以中间会出现 −1；未优化版本的取值不同，但失配后的落点等价。）

```cpp
/// 计算模式的特征向量（next 数组），原书【算法 4.7】的" 优化版"。
///
/// 与原书的差别只有所有权：原书 `int* findNext(String P)` 用 `new int[m]` 返回裸数组，
/// 而书中 ** 从未展示过与之配对的 delete[]**——每调用一次泄漏一个数组。
/// 计算过程一字未改，包括 `next[i] = next[k]` 这一步优化。
///
/// 空模式返回空向量；原书是 `assert(m > 0)`，而 assert 在 NDEBUG 下会被编译掉，
/// 于是 release 构建里 `new int[0]` 加 `next[0] = -1` 就是一次越界写。
[[nodiscard]] inline std::vector<next_type> build_next(std::string_view pattern) {
    const std::size_t m = pattern.size();
    std::vector<next_type> next(m);
    if (m == 0) {
        return next;
    }
    next_type i = 0;
    next_type k = -1;
    next[0] = -1;
    while (i < static_cast<next_type>(m)) {
        while (k >= 0 && pattern[static_cast<std::size_t>(i)] !=
        ↪  pattern[static_cast<std::size_t>(k)]) {
            k = next[static_cast<std::size_t>(k)];  // 沿已算好的特征值回退
        }
        ++i;
        ++k;
        if (i == static_cast<next_type>(m)) {
            break;
```

```cpp
        }
        const auto ui = static_cast<std::size_t>(i);
        const auto uk = static_cast<std::size_t>(k);
        // P[i] 与 P[k] 相等时可以直接借用 next[k]，省掉一次注定失败的比较——这就是" 优化
        ↪ 版"。
        next[ui] = (pattern[ui] == pattern[uk]) ? next[uk] : k;
    }
    return next;
}
```

```python
def build_next(pattern: str) -> list[int]:
    """ 计算原书算法 4.7 的优化版 next 数组。"""
    if not pattern:
        return []
    next_values = [-1] * len(pattern)
    i = 0
    k = -1
    while i < len(pattern):
        while k >= 0 and pattern[i] != pattern[k]:
            k = next_values[k]
        i += 1
        k += 1
        if i == len(pattern):
            break
        next_values[i] = next_values[k] if pattern[i] == pattern[k] else k
    return next_values
```

`next[i] = next[k]` 那一步是原书所说的”优化”：如果 `P[i]` 与 `P[k]` 相同，那么用 `k` 作为回退目标必然会再失配一次，不如直接借用 `next[k]` 一步到位。

对模式 `"abcdaabcab"`，算法产生：

i	0	1	2	3	4	5	6	7	8	9
P[i]	a	b	c	d	a	a	b	c	a	b
next[i]	−1	0	0	0	−1	1	0	0	3	0

这与原书图 4.11 最后一行完全一致，也与原书正文两处印出的优化后 `next = {-1,0,0,0,-1,1,0,0,3,0}` 一致。原书在优化之前，还先按定义给出了未优化的 `next = {-1,0,0,0,0,1,1,2,3,1}`，并指出「可以优化的下标位置为 4、6、7、9」——这两条都可以在 下面的三表对照里逐项核对：未优化的那一行就是「第一步」，4、6、7、9 正是第二步里值发生变化的位置。

优化版 next 与常见的 lps 表是什么关系

很多教材和网上的 KMP 用的是另一张表：lps（longest proper prefix which is also suffix），`lps[i]` 是 `P[0..i]`（**含** `P[i]`）最长的相等真前后缀长度，初值全 0。它和本书的 `border_lengths`（见 4.3.3 节后面求最小循环节的部分）是同一张表。原书的优化版 next 与它并不矛盾，可以由它**两步**推出来：

```
i            0   1   2   3   4   5   6   7   8   9
P[i]         a   b   c   d   a   a   b   c   a   b
lps          0   0   0   0   1   1   2   3   1   2
next        -1   0   0   0   0   1   1   2   3   1    ← 第一步：右移一格
优化版 next  -1   0   0   0  -1   1   0   0   3   0    ← 第二步：原书算法 4.7 / 图 4.11
```

第一步：`next[i] = lps[i-1]`，`next[0] = -1`。两张表回答的问题不同：`lps[i]` 问的是「`P[0..i]` 的最长前后缀」；`next[i]` 问的是「`P[i]` 与目标失配时，模式下标退到哪里」。`P[i]` 刚失配，能利用的只有它前面已经配上的 `P[0..i-1]`，所以取 `lps[i-1]`——整张表右移一格。`P[0]` 前面什么都没有，退无可退，记作 −1，意思是「不必再比，目标前进一格」，对应 KMP 匹配里 `i == -1` 时的 `++i; ++j`。用 lps 表写匹配时要写 `i = lps[i-1]` 并单独处理 `i == 0`；原书把这个「减 1」和特判都折进了表里。

第二步：跳过注定失败的回退。设 `P[i]` 与 `T[j]` 失配，按未优化的 next 应退到 `k = next[i]`、接着比 `P[k]` 与 `T[j]`。若 `P[k] == P[i]`，这次比较不用做就知道结果——`T[j]` 刚和 `P[i]` 不等，也就一定和 `P[k]` 不等。于是直接借用 `next[k]`：这就是上面代码里的 `next[i] = (P[i] == P[k]) ? next[k] : k`。`next[k]` 本身已经优化过，所以一串相同字符会一步跳到底。对照上表：`i = 4` 处 `P[4] = P[0] = 'a'`，0 变成 `next[0] = -1`；`i = 6, 7, 9` 同理塌成 0；`i = 8` 处 `P[8] = 'a'` 而 `P[3] = 'd'`，不同，保留 3。

模式 `"aaaab"` 最能看出差别：

```
i            0   1   2   3   4
lps          0   1   2   3   0
next        -1   0   1   2   3
优化版 next  -1  -1  -1  -1   3
```

在 `i = 3` 处与一个不是 `'a'` 的字符失配时，未优化的 next 会依次退到 2、1、0、−1，白比三次 `'a'`；优化版一步退到 −1。

对照原书代码：里层 `while` 的 `k = next[k]` 就是 lps 写法里的 `length = lps[length - 1]`；`++i; ++k` 之后，`k` 恰是 `P[0..i-1]` 的最长前后缀长度，也就是未优化的 `next[i]`；最后那个 三目运算是第二步。所谓「初始化为 −1」，起作用的其实是 `next[0] = -1` 和 `k = -1` 两个初值—— 第一次 `++k` 后 `k` 正好是 0。回退时用的是**已优化**的 `next[k]`，结果照样正确：被跳过的位置 与 `P[k]` 是同一个字符，本来也会失配。本章测试在 {a,b} 上长度 1–10 的全部 2046 个串上验证了

「右移一格、再跳过同字符回退」与算法 4.7 逐项相同。

代价是：**优化版 next 只能拿来做匹配**，它的值已不再是前后缀长度（上表 `i = 4` 是 −1）。需要前后缀长度本身时——例如 4.3.3 节后面求最小循环节——必须用 lps / `border_lengths`。

关于所有权：原书 `findNext` 用 `new int[m]` 返回裸数组，而书中调用它的地方——算法 4.8 的 `N` 参数、正文的示例——**一次都没有出现配对的** `delete[]`。每匹配一个模式就漏一个数组。本书返回一个拥有所有权的容器，计算过程一字未改。

4.3.3 KMP 模式匹配算法

有了特征向量，匹配过程与朴素算法几乎一样，只有失配那一步不同：不再把模式退回开头，而是退到 `next[i]`。

```cpp
/// KMP 模式匹配。失配时不再把模式右移一位，而是按特征值决定右移多少。
///
/// 返回值与 naive_search 一致，也修正了原书【算法 4.8】同样的差一错误。
/// next 由调用方传入：同一个模式可以只算一次、多次匹配复用——
/// 这正是原书强调的性质，接口把它显式表达出来。
[[nodiscard]] inline std::optional<std::size_t> kmp_search(std::string_view text,
                                                           std::string_view pattern,
                                                           const std::vector<next_type>&
                                                               next) {
    const std::size_t n = text.size();
    const std::size_t m = pattern.size();
    if (m == 0) {
        return std::size_t{0};
    }
    if (next.size() != m) {
        throw std::invalid_argument("kmp_search: next 数组长度与模式不符");
    }
    if (n < m) {
        return std::nullopt;
    }
    next_type i = 0;    // 模式下标，可以退到 -1
    std::size_t j = 0;  // 目标下标，只增不减
    while (i < static_cast<next_type>(m) && j < n) {
        if (i == -1 || text[j] == pattern[static_cast<std::size_t>(i)]) {
            ++i;
            ++j;
        } else {
            i = next[static_cast<std::size_t>(i)];
        }
    }
    return i >= static_cast<next_type>(m) ? std::optional<std::size_t>(j - m) :
    ↪  std::nullopt;
```

```cpp
}

/// 便利重载：模式只用一次时，自己把 next 算掉。
[[nodiscard]] inline std::optional<std::size_t> kmp_search(std::string_view text,
                                                           std::string_view pattern) {
    return kmp_search(text, pattern, build_next(pattern));
}
```

```python
def kmp_search(text: str, pattern: str, next_values: list[int] | None = None) -> int
↪  | None:
    """KMP 匹配；目标串下标只向前移动。"""
    if not pattern:
        return 0
    if next_values is None:
        next_values = build_next(pattern)
    if len(next_values) != len(pattern):
        raise ValueError("kmp_search: next 数组长度与模式不符")
    i = 0
    j = 0
    while i < len(pattern) and j < len(text):
        if i == -1 or text[j] == pattern[i]:
            i += 1
            j += 1
        else:
            i = next_values[i]
    return j - len(pattern) if i == len(pattern) else None
```

时间代价的关键在于目标下标 `j` 只增不减。循环体里 `++j` 至多执行 $|T|$ 次，与它同处一条语句的 `++i` 也不超过 $|T|$ 次；能让 `i` 减少的只有 `i = next[i]`，而 `next[i] < i`，所以每执行一次 `i` 至少减 1；一旦减到 -1，下一步必然进入 `++i`，`++j` 分支。因此 `i = next[i]` 的执行次数不超过 `++i`，`++j` 的次数加 1，整个循环体至多执行 $2|T|+1$ 次——**与目标串长度成线性关系**。

加上计算 next 的 $O(|P|)$，KMP 整体为 $O(|P|+|T|)$。而同一个模式的特征向量 只需算一次即可跨多个目标复用，这一点接口里用「next 由调用方传入」显式表达。

本书的测试用朴素算法的最坏情况（T = 20 万个 ‘a’，P = 1999 个 ‘a’ 加一个 ‘b’）验证这条线性性质：KMP 在其上瞬间完成，而失配后不按特征值回退的实现会退化成 $O(|T|\cdot|P|)$，直接撞上构建闸门的超时。

原书用 P = `"abcdaabcab"`（特征向量 $\{-1,0,0,0,-1,1,0,0,3,0\}$）、T = `"abcddabcababcdaabcababcdaabcabaa"` 走了一遍完整过程：

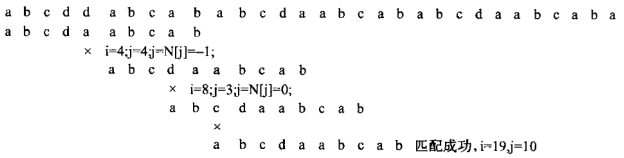


图 4.12　KMP 匹配示例。第 1 趟比到第 5 次发现 $P_4 \neq T_4$，P_4 的特征值是 −1，模式右移 $4-(-1)=5$ 位；第 2 趟到第 9 次比较时 $P_3 \neq T_8$，按特征值 0 右移 $3-0=3$ 位；第 3 趟第 12 次比较时 $P_2 \neq T_{10}$，右移 2 位；第 4 趟比到第 22 次匹配成功。**整个过程中目标下标一次都没有回退**——对照图 4.6 里被反复比较的那些字符，省下的就是这些。

KMP 的另一个用途：求最小循环节

原书讲特征向量只为匹配服务。但它算出来的那个量——前缀里「最长的相同前缀与后缀」——本身还回答 另一个问题：**一个串最短是由多长的一段反复拼成的？** 程序设计上机考试里常见的「循环串」「字符串乘方」「前缀中的周期」三类题，核心都是这一件事。

两个定义。串 s 长为 n。既是 s 的真前缀、又是 s 的真后缀的子串叫 s 的**真边界**（abab 的真边界有 ab 和空串）。正整数 p 叫 s 的**周期**，当且仅当对所有 $0 \le i < n-p$ 都有 $s_i = s_{i+p}$。

定理：s 有长为 b 的真边界，当且仅当 $n-b$ 是 s 的周期。所以最小周期 = n − 最长真边界的长度。证明只需把两边展开：长为 b 的真边界说的是 $s_0 s_1 \cdots s_{b-1} = s_{n-b} s_{n-b+1} \cdots s_{n-1}$，也就是对所有 $0 \le i < b$ 有 $s_i = s_{i+(n-b)}$；令 $p = n-b$，这恰好是「对所有 $0 \le i < n-p$ 有 $s_i = s_{i+p}$」——**同一组等式的两种读法**。边界越长周期越短，最长的边界就对应最小的周期。

三个例子：abcabcabc 的最长真边界是 abcabc，最小周期 $9-6=3$；ababa 的最长真边界是 aba，最小周期 $5-3=2$，但 2 不整除 5，它**不是**某个串的整数次重复；abcd 没有非空真边界，最小周期就是 4。

边界长度怎么算。 4.3.2 节的特征向量回答的正是「前缀的最长相同前后缀有多长」，只是原书印的是 **优化版**。求周期要用去掉那一步优化的版本——回退链一模一样，只是不做 `next[i] = next[k]` 的借用：

```
/// 未优化的失效函数：border[i] 是前缀 s[0..i] 的 ** 最长真边界 ** 长度——
/// 既是它的真前缀、又是它的真后缀的最长那一段有多长。
///
/// 这就是 build_next 去掉「优化」那一步之后的数，只是下标错开一位、没有 - 1：
/// 未优化的 next[i+1] == border[i]。回退 `k = border[k - 1]` 与 build_next 的
/// `k = next[k]` 是同一个动作。
///
/// ** 求周期不能拿 build_next 代替它。** 优化版的 `next[i] = next[k]` 专为匹配服务：
/// 当 P[i] == P[k] 时它跳过一个注定失配的落点，于是 next[i] 不再是边界长度。
/// 例如 "aaaa" 的优化版 next 是 {- 1, - 1, - 1, - 1}，而边界长度是 {0,1,2,3}。
[[nodiscard]] inline std::vector<std::size_t> border_lengths(std::string_view s) {
```

```
    const std::size_t n = s.size();
    std::vector<std::size_t> border(n);
    for (std::size_t i = 1; i < n; ++i) {
        std::size_t k = border[i - 1];  // 先试着把上一个前缀的最长边界延长一个字符
        while (k > 0 && s[i] != s[k]) {
            k = border[k - 1];  // 延长不了，退到「边界的边界」再试
        }
        if (s[i] == s[k]) {
            ++k;
        }
        border[i] = k;
    }
    return border;
}
```

```
def border_lengths(s: str) -> list[int]:
    """ 未优化的失效函数：border[i] 是前缀 s[0..i] 的最长真边界长度。

    求周期不能拿 build_next 代替：优化版 next[i] = next[k] 不再是边界长度，
    例如 "aaaa" 的优化版 next 是 [-1, -1, -1, -1]，边界长度是 [0, 1, 2, 3]。
    """
    border = [0] * len(s)
    for i in range(1, len(s)):
        k = border[i - 1]
        while k > 0 and s[i] != s[k]:
            k = border[k - 1]
        if s[i] == s[k]:
            k += 1
        border[i] = k
    return border
```

陷阱一：原书的优化版 next 不能拿来求周期。 `next[i] = next[k]` 是专为匹配做的：$P_i = P_k$ 时，退到 k 注定再失配一次，于是直接借用 `next[k]` 跳过这个落点。对匹配而言那个落点没用，对周期而言它恰恰是答案。aaaa 的优化版 next 是 $\{-1,-1,-1,-1\}$，而边界长度是 $\{0,1,2,3\}$；拿「前缀长 − next」去算前缀 aaa 的周期，得到 $3-(-1)=4$，比前缀本身还长，正确答案是 1。本书的测试在 $\{a,b\}$ 上长度不超过 10 的全部串、全部前缀上统计过：优化版给出错误周期的前缀 **数以千计**，这不是个别串的巧合。

```
/// 最小周期 p：满足 s[i] == s[i+p]（对所有 0 ≤ i < n − p）的最小正整数。
/// 定理：p = n − （整串的最长真边界长度）。空串约定返回 0。
///
/// 注意 p 不一定整除 n："ababa" 的最小周期是 2，但它不是某个串重复若干次。
[[nodiscard]] inline std::size_t minimal_period(std::string_view s) {
```

```cpp
    const std::size_t n = s.size();
    if (n == 0) {
        return 0;
    }
    return n - border_lengths(s)[n - 1];
}

/// s 能否写成某个 ** 更短 ** 的串重复至少两次（「循环串」问题）。
///
/// 两个条件缺一不可：
///   n % p == 0 ——周期不整除长度就拼不回整串（"ababa"）；
///   p < n      ——边界为 0 时 p == n，而 n % n == 0 ** 恒成立 **，
///                漏掉这一条会把 "abcd" 这种毫无重复的串也判成循环串。
[[nodiscard]] inline bool is_repetition(std::string_view s) {
    const std::size_t n = s.size();
    const std::size_t p = minimal_period(s);
    return p < n && n % p == 0;
}

/// 最大的 K，使 s 恰好是某个串重复 K 次（「字符串乘方」问题）。
/// 不是循环串时 K = 1（s 就是它自己重复一次）；空串约定返回 0。
[[nodiscard]] inline std::size_t repetition_count(std::string_view s) {
    if (s.empty()) {
        return 0;
    }
    return is_repetition(s) ? s.size() / minimal_period(s) : 1;
}
```

```python
def minimal_period(s: str) -> int:
    """ 最小周期 p = n - 整串的最长真边界长度；空串约定返回 0。"""
    if not s:
        return 0
    return len(s) - border_lengths(s)[-1]


def is_repetition(s: str) -> bool:
    """s 能否写成更短的串重复至少两次。

    p < n 不能省：边界为 0 时 p == n，而 n % n == 0 恒成立。
    """
    p = minimal_period(s)
    return p < len(s) and len(s) % p == 0


def repetition_count(s: str) -> int:
    """ 最大的 K，使 s 是某个串重复 K 次；不是循环串时为 1，空串为 0。"""
```

```
    if not s:
        return 0
    return len(s) // minimal_period(s) if is_repetition(s) else 1
```

陷阱二：边界为 0 时，「整除」不等于「循环」。判断 s 是否由一个更短的串重复而成，要同时满足 $n \bmod p = 0$ 与 $p < n$。边界为 0 时 $p = n$，而 $n \bmod n = 0$ **恒成立**——只写第一个条件，`abcd` 就被当成了循环串。「前缀中的周期」一类题要逐个前缀报出重复次数 $K > 1$ 的那些，漏了这一条，每个没有边界的前缀都会被报出来。

三类题怎么落到这组函数上。循环串就是 `is_repetition`；字符串乘方（最大的 K 使 s 是某串重复 K 次）就是 `repetition_count`；前缀中的周期，对 `border_lengths` 的每个下标 i，拿长为 $i+1$ 的前缀套同一条判断即可——整个串只算一遍失效函数，总代价 $O(n)$。

测试怎么证伪它。独立参照物是周期的定义本身：逐个试 $p = 1, 2, \ldots$ 的 $O(n^2)$ 暴力解。在 $\{a, b\}$ 上长度 0 到 10 的全部 2047 个串上，`minimal_period`、`repetition_count`、`is_repetition` 与暴力解逐个对拍，C++ 与 Python 两侧各跑一遍；两个陷阱各有专门的断言——把实现改成用优化版 next，或者删掉 `p < n`，测试当场变红。

与原书的对照

原书	现在	为什么
`return (j - pLen + 1)`	`return j - m`	**原书差 1**，四组数据逐个对拍证实
`int` 返回值，−1 表示没找到	`std::optional<std::size_t>`	−1 与” 位置 0” 只差一个符号，漏判就把未匹配当成匹配在开头
`int* findNext()` 返回 `new int[]`	返回拥有所有权的容器	书中从未展示配对的 `delete[]`，每次调用漏一个数组
`assert(m > 0)`	空模式返回 0	`assert` 在 NDEBUG 下整个消失，release 构建里是越界写
`assert(next != 0)`	删除	`new` 失败抛 `bad_alloc`，从不返回空指针，该断言永远为真

刻意没改的：朴素匹配仍是回溯式的；next 仍是原书的优化版（图 4.11 对的是这一版）；KMP 的 next 仍由调用方传入并可跨目标复用。这三条是本节全

部的教学内容。

完整实现见 `code/ch04/pattern_matching/modern.hpp`，测试见同目录 `test.cpp`（96 项断言，含 3000 组随机对拍与最小周期的 2047 串穷举；用 `python3 tools/check_code.py` 在 g++ 与 clang++ 下各按 `-Werror` + ASan/UBSan 与 `-O2` 两种构建跑一遍，共 4 种构建）。

本章小结

字符串是由零个或多个字符顺序排列组成的有限序列。它是一种特殊的线性表，其特殊性主要体现在 组成表的每个元素均为一个字符，以及与此相应的一些特殊操作。一个字符串中所包含的字符的个数为 串的长度，长度为零的字符串称为空字符串；主字符串 S_1 中若干个连续字符组成的**子串** S_2 被称为 S_1 的子串，而 S_1 被称为 S_2 的主串。

选择字符串的存储结构时要考虑字符串的变长特点。顺序存储使用类型为 `char` 的一维定长数组，访问字符串中的单个字符或连续的一组字符比较容易，但进行插入和删除（增减字符）操作就不太方便，因为需要移动相关的字符。**为了更方便地处理字符串、避免静态定长字符串数组的问题，有些字符串类的 存储结构和实现方案提供了动态的字符串存储空间管理**。

常用的字符串运算包括字符串的复制、求长度、比较、拼接、抽子串、寻找字符等；**根据存储结构的不同，这些运算的实现也有所不同**。

模式匹配是一个比较复杂的串操作，是子串（模式）在主串（目标串）中的定位操作。常用的模式匹配 算法有朴素的原始匹配算法和经过优化改进的无回溯算法。朴素的模式匹配比较直观、易于理解，但由于 回溯而使复杂度提高，**最坏情况下其时间代价与目标串长度和模式串长度的乘积成正比**。

无回溯的模式匹配中最具代表性的是 KMP **算法**：它对模式本身的字符分布特征进行分析，生成模式的 特征向量，并在匹配的过程中利用模式的特征向量，以提高模式匹配的效率；**其时间代价是目标串长度的 线性函数，计算模式特征向量的时间也与模式本身的长度成正比**。

本书在此之外还修正了原书返回值一律差 1 的错误（见 4.3.1 节）。

习题

正文习题

1. 已知 $s =$ `(xyz)+*`，$t =$ `(x+z)*y`。用连接、抽子串和置换把 s 变成 t。
2. 给出使 $s_1 + s_2 = s_2 + s_1$ 成立的所有可能条件（$+$ 为连接）。
3. 统计输入串中各合法字符（A–Z、0–9）的频度。
4. 把长为 n 的串 S 改造成：偶数位字符按原下标从大到小放后半，奇数位从小到大放前半。例如 `ABCDEFGHIJKL` 变成 `ACEGIKLJHFDB`。
5. 判断串是否对称，对称返回 1 否则 0。
6. 求包含在 s 中但不在 t 中的字符构成的新串，以及每个字符在 s 中第一次出

现的位置。

7. 线性时间判断 T 是否是 T' 的循环反转（如 `arc` 与 `car`）。
8. 从 s 中删除所有与 t 相同的子串。
9. 求 `BAAABBBAA` 的特征向量，并与目标 `BAAABBBCDDDCCHHHHBBBAAABBBAADD` 匹配，画出过程。

上机题

1. 实现 `atoi(x)`：把由数字和可选负号组成的串转成整数。
2. 统计输入串中的整数个数并输出它们（连续数字看成一个整数）。
3. 返回串中最长重复子串及其下标。例如 `abcdacdac` 的最长重复子串是 `cdac`，下标 2。
4. **字符串综合练习**：编写一个简单行编辑程序，对文本文件进行插入、删除等修改操作。可以是类似于 UNIX Vi 或 DOS Edlin 的简单行编辑，要求实现以下功能：(1) 行插入；(2) 行删除；(3) 改变当前行 指针；(4) 对于超过一屏的长文件进行分页显示；(5) 基于模式匹配算法进行查找和替换。
   要求和提示：**必须实现查找字符串的操作（用 KMP 或其他模式匹配算法），不允许用编程环境所提供 的查找算法**（可以用函数重载）；有能力的同学可以支持 `*`、`?` 等通配符；本题不要求做图形界面，建议实现普通的字符界面编辑器，注意界面简单友好；允许使用编程环境提供的图形包与字符串类；可以研究网上开源代码包，但最好不要直接采用，可以在详细说明自己引用了哪些包中哪些代码段的 情况下进行局部引用；统计自己编写的代码行数时，开源代码包、编程环境生成的代码框架等不能计算 在内。

第 5 章 二叉树

二叉树的每个结点最多有左、右两个孩子。周游回答「以什么顺序访问全部结点」；二叉搜索树加上左小右大；堆把最小（或最大）值放在根；Huffman 树不断合并两个最小权值。

源码：二叉树与二叉搜索树·教学版、二叉树与二叉搜索树·工程版、最小堆与 Huffman 树·教学版、最小堆与 Huffman 树·工程版、树的示例、堆与 Huffman 的示例。

5.1 二叉树的概念

二叉树由结点的有限集合构成：或者为空，或者由一个根和两棵互不相交的左、右子树组成。左右次序不能颠倒。根没有父结点；其余每个结点恰有一个父结点，至多两个孩子。没有孩子的是叶，其余是内部结点。从根到某结点的边数是该结点的层数，根在第 0 层。

按定义展开，二叉树只有 5 种基本形态：(a) 空树；(b) 只有一个根结点；(c) 右子树为空；(d) 左子树为空；(e) 左右子树均非空。

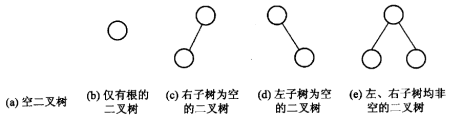


图 5.1　二叉树的 5 种基本形态。**(c) 和 (d) 是两棵不同的二叉树**——同样是一个根加一个孩子，挂左边和挂右边不能混为一谈。第 6 章的一般树没有这个区分，这正是二叉树不是「度为 2 的树」的原因。

5.1.1 二叉树的定义和基本术语

定义。二叉树（binary tree）由结点的有限集合构成，这个有限集合或者为空集（empty），或者为由 一个根结点（root）及两棵互不相交、分别称做这个根的左子树（left subtree）和右子树（right subtree）的二叉树组成的集合。这个**递归定义**刻画了二叉树的固有特性：二叉树既可以是一棵空树，也可以是 由一个根结点和分别为其左子树和右子树的互不相交的二叉树组成（其左、右子树也可以为空）。

值得注意的是，**二叉树的子树有左、右之分，其次序不能颠倒**——上面图 5.1 中的 (c) 和 (d) 分别 表示两棵不同的二叉树。二叉树的结点与表的元素类似，可以表示任何一种数据类型。

基本术语。二叉树是由唯一的起始结点引出的结点集合，这个起始结点称为**根**（root）。

- 二叉树中的任何非根结点都有且仅有一个前驱结点，称为该结点的**父结点**（或称双亲，parent）；根结点即为二叉树中唯一没有父结点的结点。
- 任何结点最多可能有两个后继结点，分别称为**左子结点**（左孩子、左子女，left

child）和**右子结点**（右孩子、右子女，right child）。具有相同父结点的结点之间互称**兄弟结点**（sibling）。

- 一个结点的子树数目称为该结点的**度**（degree）。没有子结点的结点称为**叶结点**（leaf，也称树叶 或终端结点），叶结点的度为 0；除叶结点以外的那些非终端结点称为**内部结点**（或分支结点，internal node）。
- 父结点 k 与子结点 k' 之间存在的一条有向连线 $\langle k, k' \rangle$ 称做**边**（edge）。
- 若二叉树中存在结点序列 $k_0, k_1, \cdots, k_s$，使得 $\langle k_0, k_1 \rangle$、$\langle k_1, k_2 \rangle$、$\cdots$、$\langle k_{s-1}, k_s \rangle$ 都是二叉树中的边，则称从结点 k_0 到结点 k_s 存在一条**路径**（path），该路径所经历的边的个数称为这条路径的**路径长度**（path length）。若有一条由 k 到达 k_s 的路径，则称 k 是 k_s 的**祖先**（ancestor），k_s 是 k 的**子孙**（descendant）。
- 切断一个结点与其父结点的连接，则该结点与其子孙构成的树就称为以该结点为根的**子树**（subtree）。从根结点到某个结点的路径长度称为结点的**层数**（level）：根结点为第 0 层，非根 结点的层数是其父结点的层数加 1。

除了严格区分左右子结点外，其他二叉树概念在第 6 章的一般树形结构中也适用。

下面这棵树：

```
    A
   / \
  B   C
 / \
D   E
```

这棵 5 个结点的小树是本章示例程序用的例子。5.2.2 讨论周游次序时会换成原书图 5.5 那棵 9 个结点的树——结点多一些，三种周游的差别才看得出来。

四种周游访问的是同一组结点，次序不同：

周游	顺序	本例
先序	根，左，右	A B D E C
中序	左，根，右	D B E A C
后序	左，右，根	D E B C A
层次	按离根的距离	A B C D E

递归版最贴合定义，也是 5.2 节要教的东西。极深的退化树会耗尽调用栈：Release 档大约在百万层段错误且**没有诊断**，ASan 档会打印 `stack-overflow` 并指到具体行。数字和复现程序见仓库中的未验证风险说明。

周游与析构在这件事上待遇不同，要分清楚：

- **周游**保留递归为主实现——它就是本节要教的东西，抹掉递归等于抹掉课程内容。迭代版按 D-001 §3d 作为补充并列给出，读者可以对照。

- **析构** `make_empty()` **与深拷贝**改成了迭代。理由是它们**不是教学内容，却会被隐式触发**：一次普通的作用域结束、一句 `auto copy = tree;`，调用方根本看不见自己正在递归。教学价值为零，风险却实打实。

析构用的是「右旋到没有左孩子，再沿右链逐个删」的经典办法：额外空间 O(1)，不分配内存，所以能保持 `noexcept`。深拷贝用堆上的显式栈代替调用栈。改完之后，纯左链 500 万结点的析构与深拷贝都不再崩（原先分别在 100 万、50 万段错误）；测试里有一个百万深链的用例专门守着这条，改回递归它就会报 `stack-overflow`。

5.1.2 满二叉树、完全二叉树、扩充二叉树

任何结点或者是叶，或者左右子树都非空，叫做满二叉树（full binary tree）。这个定义只要求**没有度为 1 的结点**，不要求叶在同一层，结点数也不固定——图 5.2(a) 的叶 C 在第 1 层、D 在第 2 层、F 和 G 在第 3 层。

注意口径：有些教材所说的「满二叉树」是指**每一层都排满**（英文 perfect binary tree），深度为 k 时恰有 $2^{k+1}-1$ 个结点、叶全在第 k 层。它是本书满二叉树的特例；答题时先说清用的是哪种定义。

完全二叉树（complete binary tree）不要求每个内部结点都有两个孩子，但**最多只有最下面两层的结点度数可以小于 2**（换句话说，除最后一层外每层都排满），且最下层的结点从左到右连续排列、没有空洞。由此可知它的叶只出现在最下两层；但反过来「叶只在最下两层」并不够——一条三个结点的左链 A → B → C 只有一片叶 C，却因第 1 层缺了 A 的右孩子而不完全。堆正是利用这个条件按层编号存进数组：编号关系见 5.1.3。满二叉树与完全二叉树**互不包含**：图 5.2(a) 是满的，但 C 没有孩子，第 2 层缺了两个位置，而下面还有第 3 层，所以不完全；图 5.2(b) 是完全的，但 F 只有一个孩子，所以不满。每层都排满的二叉树（perfect）才既满又完全。

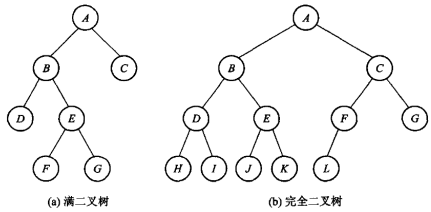


图 5.2　(a) 满二叉树；(b) 完全二叉树。

把普通二叉树的每个空子树位置都补成一个“外部叶”，所得树叫扩充二叉树（extended binary tree），原来的结点叫内部结点。若原树有 n 个内部结点，则扩充树有 $n+1$ 个外部叶。

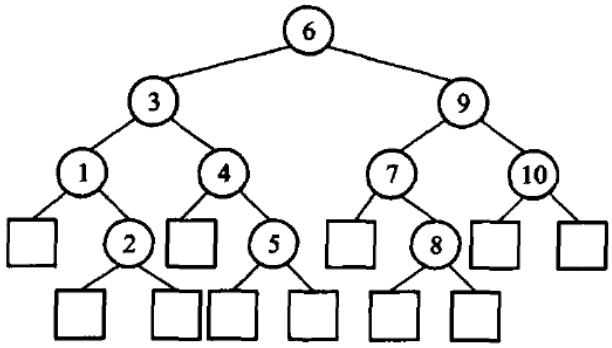


图 5.3　扩充二叉树: 方框是补出来的外部叶, 圆圈是原有的内部结点。补完之后每个内部结点都恰好有两个孩子，所以扩充二叉树一定是满二叉树——5.6 节算带权外部路径长度时用的就是这张图的形状。设内部路径长度 I 为根到所有内部结点的边数之和，外部路径长度 E 为根到所有外部叶的边数之和，则

$$E = I + 2n.$$

以图 5.3 为例（$n = 10$ 个内部结点、11 个外部叶），按深度逐个相加:

$$E = 3{+}4{+}4{+}3{+}4{+}4{+}3{+}4{+}4{+}3{+}3 = 39, \qquad I = 0{+}1{+}2{+}3{+}2{+}3{+}1{+}2{+}3{+}2$$

正好 $39 = 19 + 2 \times 10$。

直观的理由是：每个内部结点向下增加的两条分支，分别把一条路径贡献给外部叶; 沿着原树的每条边, 路径长度同时在内部路径和外部路径中各增加一次。等价地，对内部结点按度数计边，可得外部叶数为 $n + 1$。严格的证明用原书的办法，对内部结点数 n 归纳:

证明. $n = 1$ 时 $I = 0$、$E = 1 + 1 = 2$，等式成立。设等式对 n 个内部结点的扩充二叉树成立，考虑有 $n + 1$ 个内部结点的扩充二叉树。取一个**两个孩子都是外部叶**的内部结点（最深的内部结点就是这样的），设它到根的路径长度为 K，把它连同两个外部叶换成一个外部叶，得到有 n 个内部结点的扩充二叉树。内部路径长度少了 K: $I_n = I_{n+1} - K$; 外部路径长度少了两个长 $K+1$ 的、多了一个长 K 的: $E_n = E_{n+1} - 2(K+1) + K = E_{n+1} - K - 2$。由归纳假设 $E_n = I_n + 2n$,代入得 $E_{n+1} = I_n + 2n + K + 2 = I_{n+1} + 2(n+1)$。 ∎

这个关系是最优二叉树和 Huffman 树计算带权路径长度的基础。

5.1.3 二叉树的主要性质

第 i 层至多 2^i 个结点；深度为 k 的二叉树至多 $2^{k+1}-1$ 个结点。按层从 0 编号时，结点 i 的父是 $\lfloor (i-1)/2 \rfloor$，左右孩子是 $2i+1$ 与 $2i+2$——第 5.5 节的堆全靠这一条。n 个结点的完全二叉树高度为 $\lceil \log_2(n+1) \rceil$。这些性质原书没错，原样保留。

先说清两个容易混的词。**深度**是最大的层数（根在第 0 层，所以只有一个根的树深度为 0）；**高度**（height）是层数最大的叶结点的层数加 1，也就是「一共有几层」（只有一个根的树高度为 1）。二者恰好差 1。原书把上面几条依次编为：

- **性质 1**：第 i 层上最多有 2^i 个结点（$i \ge 0$）。对 i 归纳：第 0 层只有根；第 i 层每个结点至多两个孩子，所以第 $i+1$ 层至多 $2 \cdot 2^i$ 个。
- **性质 2**：深度为 k 的二叉树至多有 $2^{k+1}-1$ 个结点（$k \ge 0$）。把性质 1 对 $i = 0, \dots, k$ 求和：$1 + 2 + \cdots + 2^k = 2^{k+1} - 1$。
- **性质 3〜5** 见下文。
- **性质 6**：有 n 个结点（$n > 0$）的完全二叉树，高度为 $\lceil \log_2(n+1) \rceil$，深度为 $\lceil \log_2(n+1) \rceil - 1$（也等于 $\lfloor \log_2 n \rfloor$）。证明：设高度为 h，完全二叉树前 $h-1$ 层排满、第 h 层至少有一个结点，由性质 2 得 $2^{h-1}-1 < n \le 2^h - 1$，即 $2^{h-1} < n+1 \le 2^h$；取对数 $h - 1 < \log_2(n+1) \le h$，$h$ 是整数，所以 $h = \lceil \log_2(n+1) \rceil$。例如 $n = 10$（5.3 节图 5.9 去掉最后两个结点）时高度为 $\lceil \log_2 11 \rceil = 4$。
- **性质 7**：n 个结点的完全二叉树按层次、从左到右编号 $0 \sim n-1$，对任一结点 i：
  (1) $i = 0$ 是根；$i > 0$ 时父结点是 $\lfloor (i-1)/2 \rfloor$。
  (2) 当 $2i+1 \le n-1$ 时左孩子是 $2i+1$，否则没有左孩子；当 $2i+2 \le n-1$ 时右孩子是 $2i+2$，否则没有右孩子。
  (3) i 为偶数且 $0 < i < n$ 时左兄弟是 $i-1$；i 为奇数且 $i+1 < n$ 时右兄弟是 $i+1$。

  性质 7 (2) 的证明. 设结点 i 在第 j 层、是该层从左数第 p 个（从 0 数）。第 j 层之前共有 $1 + 2 + \cdots + 2^{j-1} = 2^j - 1$ 个结点，所以 $i = (2^j - 1) + p$。完全二叉树里第 j 层在它左边的 p 个结点都排满了孩子，共 $2p$ 个，排在第 $j+1$ 层的最前面；于是 i 的左孩子是第 $j+1$ 层第 $2p$ 个，编号 $(2^{j+1}-1) + 2p = 2\big[(2^j - 1) + p\big] + 1 = 2i + 1$，右孩子紧随其后是 $2i+2$。编号超过 $n-1$ 就说明那个孩子不存在。(1) 是 (2) 反过来解：$2i' + 1 = i$ 或 $2i' + 2 = i$，两种情况都有 $i' = \lfloor (i-1)/2 \rfloor$。(3) 由 (2) 直接得到：左孩子编号是奇数、右孩子是偶数。∎

  例：5.3 节图 5.9 那棵 $n = 12$ 的完全二叉树里，结点 5（F）的左孩子是 11（L），右孩子 12 超出了 $n - 1 = 11$，所以 F 没有右孩子；结点 11 的父结点是 $\lfloor 10/2 \rfloor = 5$。

其中三条值得单独写下来，因为后面反复要用。

性质 3：非空二叉树的叶结点数 n_0 等于度为 2 的结点数 n_2 加 1，即 $n_0 = n_2 + 1$。

证明. 设结点总数 n、度为 1 的结点数 n_1，则 $n = n_0 + n_1 + n_2$。换个角度数边：除根之外每个结点都恰好有一条边进入，所以边数 $e = n - 1$；而这些边都是从度为 1 或 2 的结点射出的，所以 $e = n_1 + 2n_2$。两式合起来得 $n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$，即 $n_0 = n_2 + 1$。∎

性质 4（满二叉树定理）：非空满二叉树的**树叶数目等于其分支结点数加 1**。

证明. 满二叉树里每个结点的度非 0 即 2，也就是 $n_1 = 0$，于是分支结点就是 n_2，由性质 3 直接得 $n_0 = n_2 + 1$。∎

性质 5（满二叉树定理推论）：非空二叉树的**空子树数目等于其结点数加 1**。

证明. 设二叉树为 T，把它所有的空子树都换成树叶，得到的扩充二叉树记为 T'。T' 是满二叉树，而 T 原来的每个结点在 T' 里都成了分支结点。由满二叉树定理，T' 的树叶数 = 分支结点数 + 1 = T 的结点数 + 1。而每片新添的树叶恰好对应 T 的一棵空子树，所以 T 的空子树数目等于它的结点数加 1。∎

性质 5 不是纸面游戏：它说明**一棵 n 个结点的二叉树里有 $n + 1$ 个空指针**。链式存储时这些空指针占的空间和结点本身同阶——线索二叉树、Trie 的压缩、第 12 章 PATRICIA 的动机都从这里来。

线索二叉树（threaded binary tree）通过复用空指针保存遍历前驱或后继，适合教学和 需要低额外空间遍历的专门场景；现代通用库通常直接使用显式栈、父指针或迭代器，因此不应把线索化当作默认的树表示。原书把它放在习题里，图是这样的：

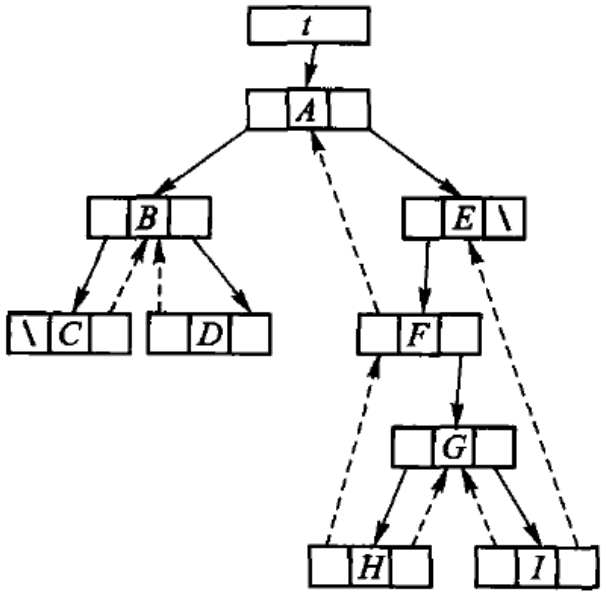


图 5.23　中序穿线（线索）二叉树。虚线是借空指针存下的中序前驱/后继，

结点里另加一位标志区分「这根指针是孩子还是线索」。有了它，中序遍历不用栈也不用递归。

5.2 二叉树的周游

5.1 节介绍了二叉树的逻辑结构，本节讨论二叉树的抽象数据类型和几种周游算法。

5.2.1 二叉树的抽象数据类型

一般情况下，需要在二叉树的各个结点中存储必要的信息，对二叉树的操作和运算也主要集中在访问 结点信息上：访问某个结点的左子结点、右子结点、父结点，或者访问结点存储的数据；从应用角度看，有时还需要遍历二叉树的每个结点。二叉树 ADT 的基本操作因此包括建树、判空、取得根和左右子树，以及先序、中序、后序和层次周游，在具体应用中可以此为基础进行扩充。

为了强调抽象数据类型与存储无关，这里并不规定该 ADT 的存储方式——二叉树结点的各种存储 方式在 5.3 节讨论，本节先从调用方观察这些操作。（原书【代码 5.1】【代码 5.2】分别给出二叉树 结点类和二叉树类，并把二叉树类声明为结点类的**友元类**，以便访问结点的私有成员。本书改用 另一种做法：结点是二叉树内部的实现细节，对外只暴露一组访问器，因此不需要友元声明。）

所谓二叉树的**周游**（或称遍历，traversal）是指按照一定顺序依次访问树中所有的结点，并使得每个 结点仅被访问一次。这里所说的「访问」是指**操作**，可以理解成对结点数据成员的处理，例如输出、修改结点的信息等。

对于线性结构来说周游是一个很容易的问题；但二叉树是一种非线性结构，每个结点可以有一个以上 的直接后继，因此需要把二叉树中的结点转化为顺序结构才能周游整棵树。**周游一棵二叉树的过程，实际上就是把二叉树的结点放入一个线性序列的过程，即对二叉树进行线性化**。下面介绍两类周游 方法：深度优先周游和广度优先周游。

先跑一遍

先建树并打印四种周游，再插一棵 BST：

```
// 第 5 章「先跑一遍」：用教学版 BinaryTree 与 BinarySearchTree 走一遍四种周游与增删查。
// 编译运行：
//   g++ -std=c++17 -I code/ch05/binary_tree code/ch05/binary_tree/demo.cpp -o demo
↪  && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    // 建一棵样例树：A 的左孩子 B（孩子 D、E），右孩子 C（叶子）
```

```
    BinaryTree<char> d, e, b, c, root;
    d.create_leaf('D');
    e.create_leaf('E');
    b.create_tree('B', d, e);       // d、e 的所有权转移给 b，之后两者变空
    c.create_leaf('C');
    root.create_tree('A', b, c);

    std::cout << " 先序: ";
    root.preorder([](char value) { std::cout << value; });
    std::cout << "\n中序: ";
    root.inorder([](char value) { std::cout << value; });
    std::cout << "\n后序: ";
    root.postorder([](char value) { std::cout << value; });
    std::cout << "\n层次: ";
    root.level_order([](char value) { std::cout << value; });
    std::cout << '\n';

    BinarySearchTree<int> tree;
    for (int key : {8, 3, 10, 1, 6, 14, 4, 7}) {
        (void)tree.insert(key);
    }
    std::cout << "BST 中序:";
    tree.inorder([](int key) { std::cout << ' ' << key; });   // 中序 = 升序
    std::cout << "\n含 6? " << (tree.contains(6) ? " 是" : " 否")
              << "  删 3 后含 3? ";
    (void)tree.remove(3);
    std::cout << (tree.contains(3) ? " 是" : " 否") << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch05/binary_tree \
    code/ch05/binary_tree/demo.cpp -o /tmp/tree-demo
/tmp/tree-demo
```

```
先序: ABDEC
中序: DBEAC
后序: DEBCA
层次: ABCDE
BST 中序: 1 3 4 6 7 8 10 14
含 6? 是  删 3 后含 3? 否
```

`create_tree(value, left, right)` 接管两棵子树。参数是右值引用，表示所有权被移走；`left` 和 `right` 在调用之后变空，不会和新建的树抢着析构同一批结点。

5.2.2 深度优先周游二叉树

二叉树的逻辑结构只有三个基本单元：根结点、左子树、右子树。

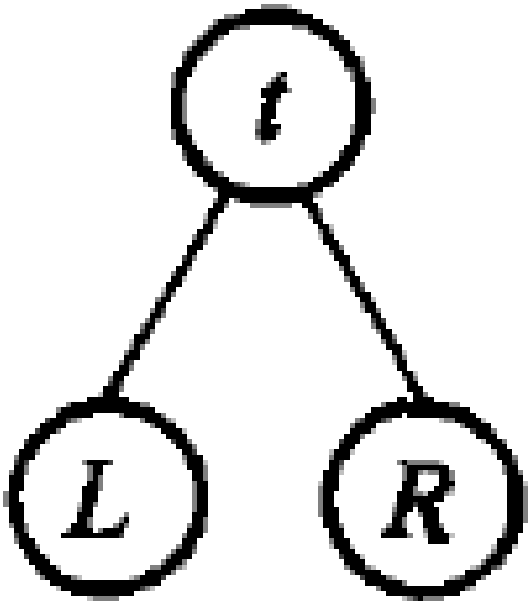
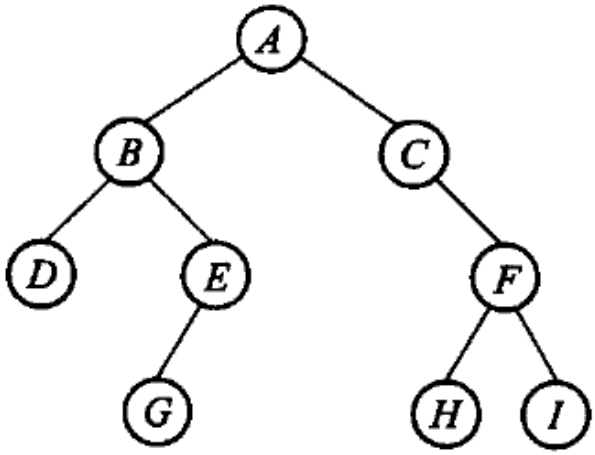


图 5.4　二叉树的基本结构。周游一棵树就是安排这三件事的先后：记访问当前结点为 t、周游左子树为 L、周游右子树为 R，六种排列里约定「先左后右」，剩下的就是前序 tLR、中序 LtR、后序 LRt 三种。

深度优先周游的思路是尽量往深处走：沿左链一路下降，遇到左子树为空就退回最近的、右子树尚未访问的分支结点，转向它的右孩子，再继续尽量往左。重复到没有结点可退为止，每个结点恰好被访问一次。

按图 5.4 说明里的记号，三种深度优先周游的递归定义是：

次序	名字	递归定义
tLR	前序（preorder）	访问根 → 前序周游左子树 → 前序周游右子树
LtR	中序（inorder）	中序周游左子树 → 访问根 → 中序周游右子树
LRt	后序（postorder）	后序周游左子树 → 后序周游右子树 → 访问根



前序（中序、后序）周游二叉树得到二叉树结点的一个有序序列，称为二叉树的前序（中序、后序）**序列**。对于图 5.5 所示的二叉树：

周游	序列
前序 tLR	A B D E G C F H I
中序 LtR	D B G E A C H F I
后序 LRt	D G E B H I F C A

二叉树这种非线性结构经过周游转化成线性序列，从而**可以指定某种周游次序下某结点的前驱和后继 结点**。以结点 E 为例：在前序序列中它的前驱是 D、后继是 G；中序序列中 E 的前驱是 G、后继是 A；后序序列中 E 的前驱是 G、后继是 B。同一个结点，在三种线性化下有三组不同的 邻居——这正是「周游次序」这件事本身的意义。

原书【算法 5.3】把三种周游写成三个递归函数，与存储结构无关。教学版照搬这个结构（完整实现见 5.3）：

```
template <typename Visitor>
static void preorder_impl(const Node* node, Visitor& visit) {
    if (node == nullptr) return;
    visit(node->value);                 // 根
    preorder_impl(node->left, visit);   // 左
    preorder_impl(node->right, visit);  // 右
}
```

```
template <typename Visitor>
static void inorder_impl(const Node* node, Visitor& visit) {
    if (node == nullptr) return;
    inorder_impl(node->left, visit);    // 左
```

```
    visit(node->value);                 // 根
    inorder_impl(node->right, visit);   // 右
}
```

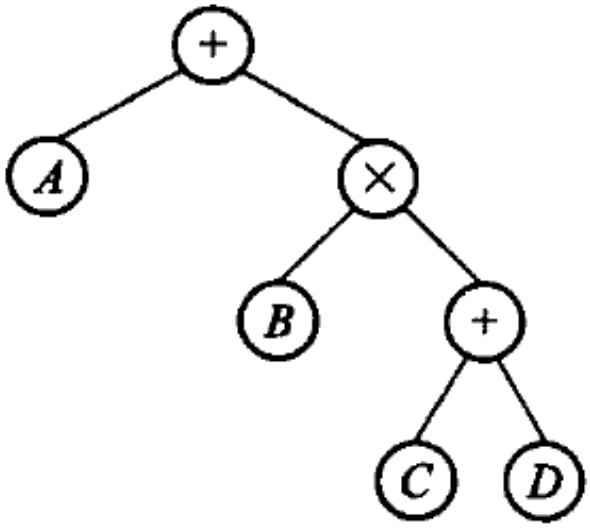
```
template <typename Visitor>
static void postorder_impl(const Node* node, Visitor& visit) {
    if (node == nullptr) return;
    postorder_impl(node->left, visit);  // 左
    postorder_impl(node->right, visit); // 右
    visit(node->value);                 // 根
}
```

三个函数的形状完全一样，只差 `visit` 那一行放在哪里。**递归在这一章的价值就在这里：代码的形状和定义的形状能一眼对上。**读的时候要把「访问」和「走进去」分清楚——`visit(node->value)` 是访问，`preorder_impl(node->left, visit)` 是走进去；报错的序列多半来自把这两件事混为一谈。

每个结点进出各一次，三种周游的时间代价都是 $O(n)$。空间代价是递归深度，等于树高：平衡时约 $\log_2 n$，退化成一条链时是 n，5.1.1 说的「压穿运行栈」就是这种树。工程版另给了三个用手写链式栈显式模拟调用栈的迭代版（`preorder_iterative`、`inorder_iterative`、`postorder_iterative`），按 D-001 §3d 只作补充，不替换递归主实现，见 5.6a。

表达式树：周游的一个用途

三种周游次序不是三种口味，它们各自对应一种实用的表达式写法。图 5.6 是表达式 $A + B \times (C + D)$ 的二叉树表示：运算符落在内部结点，运算对象落在叶结点，括号一个都不出现——树的形状已经把运算次序记下来了。



周游	得到	本例
前序	**前缀**表达式（波兰式）	+ A × B + C D
中序	中缀表达式（缺括号）	A + B × C + D
后序	**后缀**表达式（逆波兰式）	A B C D + × +

中序那一行值得多看一眼：它和原式长得像，但**括号丢了**，照它算出来的是 $A + B \times C + D$，已经不是原来的表达式。要印回可读的中缀式，必须在周游时按优先级补括号。前缀式和后缀式不需要括号——运算符出现的位置本身就定死了运算次序。

第 3.1.4 节那个后缀表达式求值器（原书【算法 3.5】）吃的正是这棵树后序周游的结果；那一节把中缀式转成后缀式的工作，换个说法就是「建出这棵表达式树，再后序周游一遍」。本章「原书上机题」第 2 题「表达式二叉树」要做的就是这件事：读入一种表达式，在机内建出这棵树，再按要求输出另外两种。

由周游序列重建二叉树

反过来问：给出周游序列，能不能把树建回来？一**个序列不行**——前序 A B C 既可以是一条左链，也可以是一条右链。**前序（或后序）加中序就够了**，前提是结点的关键码互不相同：后序的最后一个元素（前序的第一个元素）一定是根；在中序里找到根的位置 k，它左边的就是左子树的全部结点，右边的就是右子树；左子树的结点个数 k 一旦知道，前序或后序里哪一段属于左子树、哪一段属于右子树也就切开了。对两段各做一遍同样的事，直到区间为空。

以图 5.5 为例：后序 D G E B H I F C A 的末元素 A 是根；A 在中序 D B G E A C H F I 里左边有 4 个结点，于是后序的前 4 个 D G E B 是左子树的后序、接

下来的 H I F C 是右子树的后序，再各自对照中序的 D B G E 与 C H F I 递推下去。前序加后序则不够：前序 A B、后序 B A 说不清 B 是左孩子还是右孩子。

这是一道典型的上机题（本章上机题第 1 题就是「由先序和中序序列重建二叉树」），写法有两处常见的坑。第一，每一层都把子数组拷贝出来再递归，总代价会退化成 $O(n^2)$；只传**下标区间**就不必拷贝。第二，**输入不自洽时默默建出一棵树**。中序 1 2 3、后序 3 1 2 不对应任何一棵二叉树：根是 2，左子树只该有结点 1，后序里对应的却是 3。朴素写法在这里要么越界、要么建出一棵中序根本不是 1 2 3 的树。本书的处理与第 6 章由双标记先根序列重建森林（【算法 6.10】）时一样：**能当场指出输入错了，就不把错误合法化**。检查只要一条——每次取出的根必须落在当前的中序区间里。

教学版把上面的推演原样写成递归，只做「中序 + 后序」；「前序 + 中序」留给上机题。对外的入口先把树建在一棵临时树里，建成了才替换掉自己——中途发现序列对不上而抛出异常时，临时树的析构会释放已经建好的那一半，原来的树一个结点都不动：

```
// 由中序 + 后序序列重建二叉树（两个序列各 count 个元素，关键码互不相同）。
// 递归的主体在 build_impl；前序 + 中序的写法留作本章上机题第 1 题。
//
// 两个序列对不上（比如中序 1 2 3、后序 3 1 2）就抛 std::invalid_argument，
// 不悄悄建出一棵错树；这时本树保持原样。
void build_from_inorder_postorder(const T* inorder, const T* postorder, std::size_t
↪  count) {
    BinaryTree fresh;                       // 先建在一棵临时树里：中途抛异常，
    build_impl(fresh.root_, inorder, 0, count, postorder, 0);   // 由它的析构收拾半成品
    clear();                                // 建成了，再拆掉旧树、接过新树
    root_ = fresh.root_;
    fresh.root_ = nullptr;
}
```

递归主体的每一次调用，就是上面图 5.5 那段推演里的一步：参数里的两个区间是当前的中序段和后序段，key 是这一段的根，最后两行递归分别去建左子树和右子树：

```
// 把中序 inorder[in_begin, in_end) 与后序 postorder[post_begin, …) 建成一棵子树，挂到
↪  link 上。
// 两段一样长；只传下标，不拷贝子数组。
//
// 后序段的最后一个是根；在中序段里找到它（下标 k），左边 k - in_begin 个是左子树，
// 右边是右子树。左子树的大小一旦知道，后序段也就切成了「左子树 | 右子树 | 根」。
static void build_impl(Node*& link, const T* inorder, std::size_t in_begin,
↪  std::size_t in_end,
                       const T* postorder, std::size_t post_begin) {
    if (in_begin == in_end) {
        return;                                     // 空区间：空子树，link 保持 nullptr
```

```
    }
    const std::size_t size = in_end - in_begin;
    const T& key = postorder[post_begin + size - 1];    // 后序段的最后一个是根

    std::size_t k = in_begin;                           // 在中序段里从左往右找根
    while (k < in_end && inorder[k] != key) {           // 只找本段：根不在本段，说明两
    ↪  个序列对不上
        ++k;
    }
    if (k == in_end) {
        throw std::invalid_argument("build_from_inorder_postorder: root lies outside
        ↪  its inorder range");
    }

    link = new Node{key, nullptr, nullptr};             // 先挂上：子树里再抛异常，它也
↪  归临时树释放
    const std::size_t left_size = k - in_begin;
    build_impl(link->left, inorder, in_begin, k, postorder, post_begin);
    build_impl(link->right, inorder, k + 1, in_end, postorder, post_begin +
↪  left_size);
}
```

两处坑在这里各对应一行。参数只有下标，子数组一次也没有拷贝。找根的 `while` 只在本段 `[in_begin, in_end)` 里找——找不到就说明根不在它的中序区间里，中序 `1 2 3`、后序 `3 1 2` 正是在这里被拒：建左子树时，后序段只剩 `3`，中序段却只有 `1`。去掉 `k < in_end` 这个边界，`while` 就会越过本段一直往后读，这就是「要么越界」。

教学版的代价也一目了然：每一帧从左往右找根，最多看一遍本段，所以总时间是 $O(n \cdot h)$，h 为树高；树退化成一条链时就是 $O(n^2)$。递归深度等于树高，与本节的递归周游、5.6a 说的递归 `clone` 一样，退化成链的输入会压穿运行栈。教学版也不单独检查重复键：有重复键时它照样能建出一棵中序、后序都对得上的树，只是这棵树不唯一。

找根还能更快。上机题里的关键码多半是字符或小整数，这时可以直接拿关键码当数组下标，先把中序登记成一本「点名册」：`pos[c]` 记下字符 `c` 在中序里的下标，`-1` 表示中序里没有它；登记时发现这一格已经有人，就是中序里有重复键。下面两个函数写在教学版的测试文件里，测试拿它们与类里的两版逐棵对拍：

```
// 「点名册」：pos[c] = 字符 c 坐在中序第几号，-1 表示中序里没有。
// 登记时发现这一格已经有人，就是中序里有重复键。
void fill_position_table(int pos[256], const char* inorder, int count) {
    for (int c = 0; c < 256; ++c) {
        pos[c] = -1;
    }
    for (int i = 0; i < count; ++i) {
```

```
        const unsigned char c = static_cast<unsigned char>(inorder[i]);
        if (pos[c] != -1) {
            throw std::invalid_argument("duplicate key in inorder sequence");
        }
        pos[c] = i;
    }
}
```

```
// 与 build_impl 同一个递归，只把「从左往右数」换成查一次表：找根 O(1)，总代价 O(n)。
// 两棵子树先各自建成小树，再用 create_tree（【代码 5.2】）接到根上。
void build_by_table(BinaryTree<char>& out, const int pos[256], const char*
↪  postorder,
                    int in_begin, int in_end, int post_begin) {
    if (in_begin == in_end) {
        out.clear();
        return;
    }
    const char key = postorder[post_begin + (in_end - in_begin) - 1];   // 后序段的最
    ↪  后一个是根
    const int k = pos[static_cast<unsigned char>(key)];                 // 查表：它坐
    ↪  在中序第几号
    if (k < in_begin || k >= in_end) {                                  // 不在本段（-1
    ↪  也落在这里）
        throw std::invalid_argument("root lies outside its inorder range");
    }
    const int left_size = k - in_begin;
    BinaryTree<char> left, right;
    build_by_table(left, pos, postorder, in_begin, k, post_begin);
    build_by_table(right, pos, postorder, k + 1, in_end, post_begin + left_size);
    out.create_tree(key, left, right);
}
```

递归的形状与 `build_impl` 一样，只是那个从左往右数的 `while` 换成了查一次表：找根 $O(1)$，总代价 $O(n)$。`k < in_begin || k >= in_end` 就是那条「根必须落在本段」的检查，查不到时的 `-1` 也落在这里。两棵子树先各自建成小树，再用【代码 5.2】的 `create_tree` 接到根上。它只适用于关键码范围很小的情形——关键码是任意整数或字符串时，没法拿它当下标。

再看递归。递归深度等于树高；把递归改写成显式栈，是本章反复用到的手法（5.6a 的非递归周游、迭代深拷贝都是）。改写是机械的：一次递归调用的参数存成一帧，「调用」变成压栈，「返回」变成弹栈：

```
// 同一个重建，不递归：把每次递归调用的参数存成一帧，压进一条手写的链式栈。
// 与 build_impl 逐行对应——「递归调用」变成「压一帧」，「函数返回」变成「弹一帧」；
// 参数 Node*& link 变成帧里的 Node** link（引用存不进结构体，只能存指针）。
```

```
//
// 注意：教学版的析构 destroy 仍是递归的，所以一条很长的链就算这样建出来了，
// 析构时照样压穿运行栈。这里要学的是「递归 → 显式栈」的改写手法；
// 工程版把析构也改成了迭代，见 5.6a。
void build_from_inorder_postorder_iterative(const T* inorder, const T* postorder,
↪  std::size_t count) {
    struct Frame {
        Node** link;                    // 建好的子树挂到哪根指针上
        std::size_t in_begin, in_end;   // 中序段
        std::size_t post_begin;         // 后序段起点（与中序段一样长）
        Frame* next;                    // 链式栈：下面那一帧
    };
    // 栈也要有主人：它一离开作用域——正常建完、找不到根抛异常、
    // 甚至 new 本身失败——析构都会把剩下的帧还掉。与 fresh 是同一个想法。
    struct FrameStack {
        Frame* top = nullptr;
        FrameStack() = default;
        FrameStack(const FrameStack&) = delete;             // 三法则：拷贝出两份，
        FrameStack& operator=(const FrameStack&) = delete;  // 同一批帧就会被删两次
        ~FrameStack() {
            while (top != nullptr) {
                Frame* rest = top->next;
                delete top;
                top = rest;
            }
        }
    };

    BinaryTree fresh;
    FrameStack stack;
    stack.top = new Frame{&fresh.root_, 0, count, 0, nullptr};   // 第一次「调用」
    while (stack.top != nullptr) {
        const Frame f = *stack.top;     // 弹出一帧
        Frame* popped = stack.top;
        stack.top = stack.top->next;
        delete popped;

        if (f.in_begin == f.in_end) {
            continue;                   // 空区间：空子树
        }
        const std::size_t size = f.in_end - f.in_begin;
        const T& key = postorder[f.post_begin + size - 1];
        std::size_t k = f.in_begin;
        while (k < f.in_end && inorder[k] != key) {
            ++k;
        }
        if (k == f.in_end) {            // 剩下的帧由 stack 的析构还掉
```

```
            throw std::invalid_argument("build_from_inorder_postorder_iterative:
            ↪  root lies outside its inorder range");
        }

        Node* node = new Node{key, nullptr, nullptr};
        *f.link = node;                 // 先挂上：之后再抛异常，它也归临时树释放
        const std::size_t left_size = k - f.in_begin;
        // 先压右、再压左：栈后进先出，于是左子树先建，与递归的次序一致
        stack.top = new Frame{&node->right, k + 1, f.in_end, f.post_begin +
↪   left_size, stack.top};
        stack.top = new Frame{&node->left, f.in_begin, k, f.post_begin, stack.top};
    }
    clear();
    root_ = fresh.root_;
    fresh.root_ = nullptr;
}
```

对照 `build_impl` 逐行看：参数 `Node*& link` 变成帧里的 `Node** link`（引用存不进结构体，只能存指针）；先压右、再压左，是因为栈后进先出，这样左子树先建，次序与递归一致（反过来压也能建对，只是建的次序不同）。递归版不用操心「剩下的帧」：一出错，运行栈自动回收；自己管的栈就得有个主人。这里把栈包成局部对象 `FrameStack`，它的析构负责还帧——正常建完、找不到根抛异常、甚至某次 `new` 本身失败，剩下的帧都由它还掉，与入口里那棵临时树 `fresh` 是同一个想法。测试把 28 次 `new`（图 5.5 的 9 个结点加 19 帧）逐个注入失败，每一次都核对分配全部归还、原树不动。还要说清楚一点：教学版的析构 `destroy` 仍是递归的，一条很长的链即使这样建出来了，析构时照样压穿运行栈。这里要学的是改写的手法本身。

工程版把上面两步合在一起，并推广到任意类型 `T`。关键码不一定能当下标，点名册只好换成「把中序下标按键排序，再二分」，总代价 $O(n \log n)$，只要求 `T` 支持 `<`；显式栈换成本章那把手写的链式栈 `LinkedStack`，析构也是迭代的，所以 20 万结点的链从建到拆都不怕；重复键、键缺失单独报出，好让报错说对原因。它把两种重建做成 `BinaryTree` 的两个静态工厂，返回一棵拥有全部结点的新树：

```
/// 由中序 + 后序序列重建二叉树（两个序列各 count 个元素，** 关键码互不相同 **）。
///
/// 不自洽的输入一律抛 std::invalid_argument，绝不默默建出一棵错树：
/// 中序里有重复键、后序里出现中序没有的键、某个根落在当前中序区间之外。
/// 能建完就说明两个序列恰好是这棵树的中序与后序（归纳即得），不必再周游核对。
/// 要求 T 支持 operator<（用来排序查位置）。时间 O(n log n)，额外空间 O(n)。
static BinaryTree from_inorder_postorder(const T* inorder, const T* postorder,
↪   std::size_t count) {
    return rebuild(inorder, postorder, count, Order::postorder);
}
/// 由前序 + 中序序列重建（本章上机题第 1 题）。契约同上。
```

```
static BinaryTree from_preorder_inorder(const T* preorder, const T* inorder,
↪  std::size_t count) {
    return rebuild(inorder, preorder, count, Order::preorder);
}
```

主体只写一份。它**没有写成递归**（这是与教学版 `build_impl` 最大的不同）：待建的子树记成「中序区间 + 另一序列的起点 + 该挂到哪根指针上」这样一帧，压进本章那把手写链式栈。递归版的深度等于树高，退化成链的输入会和 5.6a 说的递归 `clone` 一样压穿运行栈；显式栈在堆上，测试里 20 万结点的纯左链照样重建。查位置用的是「把中序下标按键排序，再二分」，所以要求 `T` 支持 `<`，总代价 $O(n \log n)$：

```
/// 两种重建共用的主体。** 不递归 **：待建的子树用手写链式栈保存为「区间帧」，
/// 退化成链的输入也不会压穿调用栈。每一帧只记下标，不复制子数组。
static BinaryTree rebuild(const T* inorder, const T* other, std::size_t count, Order
↪  order) {
    BinaryTree result;
    if (count == 0) return result;
    if (inorder == nullptr || other == nullptr) {
        throw std::invalid_argument("rebuild: non-empty sequence given as null
        ↪  pointer");
    }
    // 位置查找表：中序下标按键排序，之后二分查位置。排好序顺便查出重复键。
    // D-001 §2 的边界：std::sort / std::lower_bound 只用在这张「键 → 中序下标」的查找表
    ↪  上，
    // 树结点、区间帧栈（LinkedStack）和挂接过程仍全部手写——没有拿 STL 容器替代本章要教的
    ↪  结构。
    std::unique_ptr<std::size_t[]> by_key(new std::size_t[count]);
    for (std::size_t i = 0; i < count; ++i) by_key[i] = i;
    const auto key_less = inorder { return inorder[a] < inorder[b]; };
    std::sort(by_key.get(), by_key.get() + count, key_less);
    for (std::size_t i = 1; i < count; ++i) {
        if (!(inorder[by_key[i - 1]] < inorder[by_key[i]])) {
            throw std::invalid_argument("rebuild: duplicate key in inorder
            ↪  sequence");
        }
    }
    const auto position_in_inorder = & {
        const std::size_t* hit = std::lower_bound(by_key.get(), by_key.get() +
        ↪  count, key,
            inorder { return inorder[index] < k; });
        if (hit == by_key.get() + count || key < inorder[*hit]) {
            throw std::invalid_argument("rebuild: key missing from inorder
            ↪  sequence");
        }
```

```
        return *hit;
    };

    // 一帧 = 「把中序 [in_begin, in_end) 与另一序列 [other_begin, …) 建成子树，挂到
    ↪  *link」。
    struct Frame { Node** link; std::size_t in_begin, in_end, other_begin; };
    LinkedStack<Frame> pending;
    pending.push(Frame{&result.root_, 0, count, 0});
    while (auto frame = pending.pop()) {
        const std::size_t size = frame->in_end - frame->in_begin;
        if (size == 0) continue;                          // 空子树: *link 已是 nullptr
        // 前序的根在区间最前，后序的根在区间最后。
        const std::size_t root_at = order == Order::preorder ? frame->other_begin
                                                             : frame->other_begin +
                                                             ↪  size - 1;
        const std::size_t k = position_in_inorder(other[root_at]);
        if (k < frame->in_begin || k >= frame->in_end) {
            throw std::invalid_argument("rebuild: root lies outside its inorder
            ↪  range");
        }
        Node* const node = new Node(other[root_at]);
        *frame->link = node;                              // 先挂上: 再抛异常时由 result
        ↪  统一释放
        const std::size_t left_size = k - frame->in_begin;
        // 左子树在另一序列里紧跟根（前序）或从区间头开始（后序），右子树接在左子树后面。
        const std::size_t left_other = order == Order::preorder ? frame->other_begin
        ↪  + 1
                                                               : frame->other_begin;
        pending.push(Frame{&node->right, k + 1, frame->in_end, left_other +
↪  left_size});
        pending.push(Frame{&node->left, frame->in_begin, k, left_other});
    }
    return result;
}
```

为什么「能建完就一定对」？对区间长度归纳：根取自区间的末端（后序）或首端（前序），左右两段恰好分走剩下的元素；若两段各自重建出的子树中序、后序（前序）与各自的区间一致，拼上根之后整棵树也一致。所以建完之后不必再周游一遍核对，教学版与工程版都是如此。测试用随机树做往返（工程版 400 棵、教学版 300 棵）：取序列、重建、再取四种序列（含层次序列，确认形状也一样）逐一比对。

5.2.3 广度优先周游二叉树

先把三种深度优先周游的代价算清楚。不管采用哪种周游方式，对于有 n 个结点的二叉树，周游完树的所有元素都需要 $O(n)$ 时间——只要对每个结点的处理

（`Visit` 的执行）时间是一个常数，周游二叉树就可以在线性时间内完成。所需要的**辅助空间为周游过程中栈的最大容量，即树的高度**；最坏情况下（每个结点只有一个孩子的「藤」），具有 n 个结点的二叉树高度为 n，所需空间复杂度 为 $O(n)$。写成递归时这个栈就是运行栈，因此那句「树退化成藤会爆栈」不是危言耸听—— 本书 README 的「闸门证明不了什么」一节量过这个边界。

深度优先是尽量往深走，广度优先是一层一层走：首先访问第 0 层（即根结点所在的层），然后从左到右 依次访问第 1 层的两个结点，依次类推——当第 i 层的所有结点访问完之后，再从左至右依次访问 第 $i+1$ 层的各个结点。对图 5.5 那棵树，结果是 A B C D E F G H I。

深度优先靠**栈**（写成递归时就是运行栈），广度优先靠**队列**：

```
把根入队
只要队列非空：
    出队一个结点，访问它
    它的左孩子入队
    它的右孩子入队
```

上层结点总是比下层结点先入队，队列又先进先出，于是出队次序自然就是逐层从左到右。

为什么不能像前三种那样写成递归？ 递归的形状天生是深度优先的：一层调用只能带着「当前子树」往下走，而层次周游要在兄弟子树之间横向跳——第 2 层的最后一个结点和第 3 层的第一个结点常常分属不同的子树。这个跨子树的次序只能由一个显式的队列记住，递归的调用栈替不了它。

原书【算法 5.7】在这里直接用了 `std::queue`。教学版改成在 `level_order` 里现写一条极简的链式 FIFO：本节要教的就是「队列在这里起了什么作用」，一行 `std::queue<Node*>` 会把它藏起来。队列本身第 3.2 节已经讲过，这里只是用它。

```cpp
// 【算法 5.7】层次周游：一层一层从左到右。
// 深度优先靠栈（这里是递归用的运行栈），广度优先靠 ** 队列 **。
template <typename Visitor>
void level_order(Visitor visit) const {
    // 一条极简的链式队列，只在这个函数里用
    struct Pending {
        const Node* node;
        Pending* next;
    };
    Pending* front = nullptr;
    Pending* rear = nullptr;

    if (root_ != nullptr) {
        front = rear = new Pending{root_, nullptr};
    }
    while (front != nullptr) {
```

```
        Pending* item = front;              // 出队
        front = front->next;
        if (front == nullptr) {
            rear = nullptr;
        }
        const Node* node = item->node;
        delete item;

        visit(node->value);

        if (node->left != nullptr) {        // 左右孩子依次入队
            Pending* fresh = new Pending{node->left, nullptr};
            if (rear == nullptr) { front = rear = fresh; } else { rear->next = fresh;
            ↪  rear = fresh; }
        }
        if (node->right != nullptr) {
            Pending* fresh = new Pending{node->right, nullptr};
            if (rear == nullptr) { front = rear = fresh; } else { rear->next = fresh;
            ↪  rear = fresh; }
        }
    }
}
```

每个结点入队、出队各一次，时间同样是 $O(n)$。空间由结点最多的那一层决定：最坏是每层都排满的二叉树（perfect），队列最长时装着最下面一整层，约 $(n+1)/2$ 个结点。这与深度优先的空间代价（正比于树高）正好互补——树越平衡，深度优先越省；树越接近一条链，广度优先越省。

5.3 二叉树的存储结构

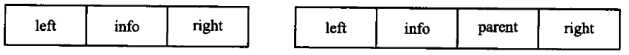
为什么这一节没有 Python 版

本节讲的是结点的动态存储和树的所有权：析构要遍历并释放每个结点，拷贝要深复制，移动要转移根指针，还要考虑递归实现的栈边界。Python 的对象引用和垃圾回收不会给出 C++ 五法则或强异常保证的对应练习；用嵌套 `list` 表示树更会把结点布局抹掉。因此这里的教学实现只提供 C++，算法层的周游和搜索才适合做双语言对照。

前面几节讨论的二叉树逻辑结构和抽象数据类型，都**不依赖于二叉树的物理存储方式**。二叉树的存储 实现方法有多种：既可以用链接存储结构实现，又可以用顺序存储结构实现；这些方法各有其特点和 适用范围，在应用中要根据情况决定采用哪种。

5.3.1 二叉树的链式存储结构

链式存储把二叉树的各结点随机地存储在内存空间，结点之间的关系用指针表示。由二叉树的定义 可知，每一个结点由一个数据元素和指向其左右子树的分支组成，所以表示二叉树的链表结点包含 3 个域：数据域和左、右指针域——用 `info` 域存储结点的数据元素，另外再设置两个指针域 `left` 和 `right`，分别指向结点的左子结点和右子结点；当结点的某个子结点为空时，相应的指针为空指针。



(a) 有2个指针域的结点结构　　(b) 有3个指针域的结点结构

图 5.7　二叉树结点的两种存储结构：(a) 数据域 + 左右指针，用它得到的是**二叉链表**（也叫 left-right 存储法）；(b) 再加一根指向父结点的 `parent`，得到的是**三叉链表**。二叉链表里找父结点得从根重新走一趟，三叉链表顺着 `parent` 一步就到——多一根指针买到的就是这件事。

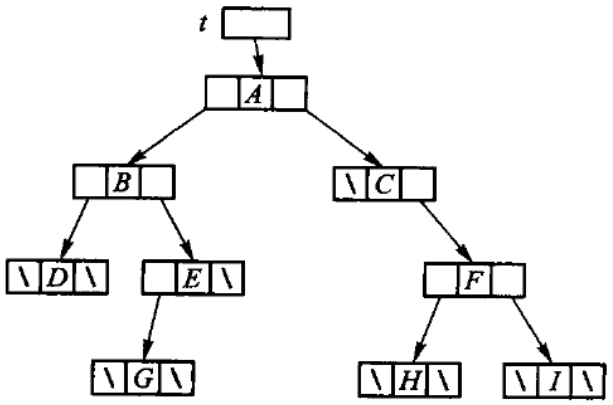


图 5.8　图 5.5 那棵树的二叉链表存储结构。数一数图里的空指针：n 个结点的二叉链表恰有 $n+1$ 个空链域（5.1.3 的性质 5），线索二叉树就是把它们利用起来。

本章的教学版用的正是二叉链表。`create_tree` 先 `new` 出新根，再把两棵子树的根指针挪过来，最后才清空自己原来的树。这样即使 `new` 抛异常，调用方的两棵子树也不动。

5.3.2 完全二叉树的顺序存储结构

顺序存储是指按照一定次序，用一组地址连续的存储单元存储二叉树上的各个结点元素。由于二叉树 是一种非线性结构，因此必须将结点排成一个线性序列，使得通过结点在这个序列中的相对位置就能 确定结点之间的逻辑关系；**但通常情况下，只通过相应位置不足以刻画整个树形结构**。

而对于一棵具有 n 个结点的**完全二叉树**，可以从根结点起自上而下、从左至右地把所有结点编号，得到一个足以反映整个二叉树结构的线性序列——采用这种方式，线性序列里存储的结点就是按照层次 周游二叉树得到的排列。按层次顺

序把结点从 0 到 $n-1$ 编号，编号 i 的结点就放在数组下标 i。

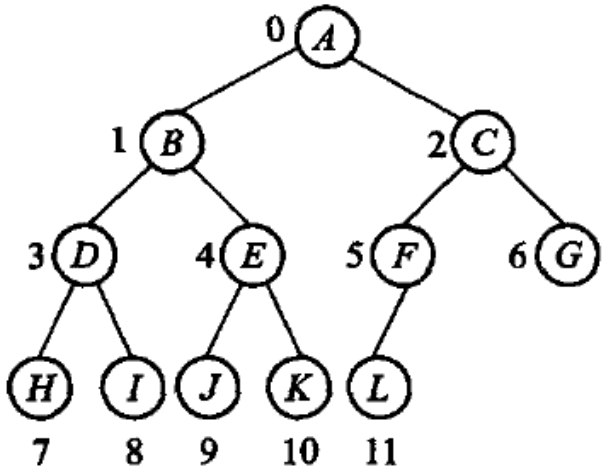


图 5.9　完全二叉树的结点编号。

按层次次序把图 5.9 的 12 个结点放进一维数组，就得到图 5.10：

下标	0	1	2	3	4	5	6	7	8	9	10	11
元素	A	B	C	D	E	F	G	H	I	J	K	L

图 5.10　完全二叉树的顺序存储结构（即原书图 5.10，本书改用表格给出）。

结点间的关系不再靠指针，而是**隐含在下标里**：下标 i 的左右孩子是 $2i+1$ 和 $2i+2$，父结点是 $\lfloor (i-1)/2 \rfloor$——正是 5.1.3 的性质。一个指针都不用存，这是完全二叉树最省空间的存法；5.5 节的堆和 8.3.2 节的堆排序都用它。**但它只对完全二叉树划算**：树一旦有空洞，数组里就要留出同样多的空位。

第 6 章会介绍一般情况下树的顺序存储；由于二叉树与树能够进行等价转换，因此一般情况下的二叉树 也可以采用类似的顺序存储结构方法。

在不同的存储结构中，不仅空间开销有差异，实现二叉树操作的方法也不同。因此在具体应用中 采取什么存储结构，除了根据二叉树的形态之外，还应该考虑时间、空间复杂度和算法的简洁性。

教学版：完整实现

下面是一份**完整的、能直接编译运行的**二叉树与二叉搜索树。一个文件、两个类。**递归是这一章的正题**，所以周游、释放、深拷贝这里全部写成递归—— 形状和定义一样，一眼就能对上。5.6a 会说明递归的代价与工程版的处理。

```
// 二叉树与二叉搜索树 ——教学版。
// 原书【代码 5.1】【代码 5.2】【算法 5.3】【算法 5.7】【算法 5.9】【算法 5.10】。
//
```

```cpp
// 一个文件、两个类、能直接编译运行，给「第一次读这一节」的人看。
//
//   BinaryTree          二叉链表：一个结点，两根指向孩子的链接；深度优先与层次周游；
//                       由中序 + 后序序列重建，递归与显式栈各一版（5.2.2，不是原书编号算法）。
//   BinarySearchTree    在二叉树上加一条「左小右大」的约束，于是查找变成一路往下走。
//
// ** 递归是这一章的正题 **，所以周游、释放、深拷贝这里全部写成递归——形状和定义一样，
// 一眼就能对上。代价是递归深度等于树高：退化成一条链的树（比如按升序插入 BST）
// 会把运行栈压穿。工程版把释放和深拷贝改成了迭代，见 5.6a「进阶（选读）」。
//
// 与 modern.hpp（工程版）的分工：
//   教学版   三法则 + 全递归；
//   工程版   五法则、迭代释放（右旋拉直）、迭代深拷贝、强异常保证、非递归周游、
//            O(n log n) 且不递归的重建。
#pragma once

#include <cstddef>
#include <stdexcept>

// ---------------------------------------------------------------------------
// 二叉树（二叉链表）
// ---------------------------------------------------------------------------
template <typename T>
class BinaryTree {
public:
    // 【代码 5.1】二叉树结点：一个数据域 + 左右两根链接。
    // 没有孩子就是 nullptr——这比原书用一个" 空结点" 表示要省事得多。
    struct Node {
        T value;
        Node* left;
        Node* right;
    };

    BinaryTree() : root_(nullptr) {}

    ~BinaryTree() { clear(); }

    // 三法则：树管着一堆 new 出来的结点，拷贝必须自己写（而且必须是 ** 深 ** 拷贝）。
    BinaryTree(const BinaryTree& other) : root_(clone(other.root_)) {}

    BinaryTree& operator=(const BinaryTree& other) {
        if (this == &other) {
            return *this;
        }
        Node* fresh = clone(other.root_);   // 先把新树建好
        clear();                            // 再拆掉旧树
        root_ = fresh;
```

```
        return *this;
    }

    bool empty() const { return root_ == nullptr; }
    const Node* root() const { return root_; }
    Node* root() { return root_; }

    // 造一棵新树：一个根，接上左右两棵子树。
    // 两棵子树的所有权 ** 转移 ** 给新树——传进来的那两棵随即变空，
    // 否则同一批结点会被两棵树各删一次。
    void create_tree(const T& value, BinaryTree& left, BinaryTree& right) {
        Node* fresh = new Node{value, left.root_, right.root_};
        left.root_ = nullptr;
        right.root_ = nullptr;
        clear();
        root_ = fresh;
    }

    void create_leaf(const T& value) {
        Node* fresh = new Node{value, nullptr, nullptr};
        clear();
        root_ = fresh;
    }

    // 由中序 + 后序序列重建二叉树（两个序列各 count 个元素，关键码互不相同）。
    // 递归的主体在 build_impl；前序 + 中序的写法留作本章上机题第 1 题。
    //
    // 两个序列对不上（比如中序 1 2 3、后序 3 1 2）就抛 std::invalid_argument，
    // 不悄悄建出一棵错树；这时本树保持原样。
    void build_from_inorder_postorder(const T* inorder, const T* postorder,
    ↪  std::size_t count) {
        BinaryTree fresh;                   // 先建在一棵临时树里：中途抛异常，
        build_impl(fresh.root_, inorder, 0, count, postorder, 0);   // 由它的析构收拾
↪  半成品
        clear();                            // 建成了，再拆掉旧树、接过新树
        root_ = fresh.root_;
        fresh.root_ = nullptr;
    }

    // 同一个重建，不递归：把每次递归调用的参数存成一帧，压进一条手写的链式栈。
    // 与 build_impl 逐行对应——「递归调用」变成「压一帧」，「函数返回」变成「弹一帧」；
    // 参数 Node*& link 变成帧里的 Node** link（引用存不进结构体，只能存指针）。
    //
    // 注意：教学版的析构 destroy 仍是递归的，所以一条很长的链就算这样建出来了，
    // 析构时照样压穿运行栈。这里要学的是「递归 → 显式栈」的改写手法；
    // 工程版把析构也改成了迭代，见 5.6a。
    void build_from_inorder_postorder_iterative(const T* inorder, const T*
    ↪  postorder, std::size_t count) {
```

```
        struct Frame {
            Node** link;                    // 建好的子树挂到哪根指针上
            std::size_t in_begin, in_end;   // 中序段
            std::size_t post_begin;         // 后序段起点（与中序段一样长）
            Frame* next;                    // 链式栈：下面那一帧
        };
        // 栈也要有主人：它一离开作用域——正常建完、找不到根抛异常、
        // 甚至 new 本身失败——析构都会把剩下的帧还掉。与 fresh 是同一个想法。
        struct FrameStack {
            Frame* top = nullptr;
            FrameStack() = default;
            FrameStack(const FrameStack&) = delete;             // 三法则：拷贝出两份，
            FrameStack& operator=(const FrameStack&) = delete;  // 同一批帧就会被删两次
            ~FrameStack() {
                while (top != nullptr) {
                    Frame* rest = top->next;
                    delete top;
                    top = rest;
                }
            }
        };

        BinaryTree fresh;
        FrameStack stack;
        stack.top = new Frame{&fresh.root_, 0, count, 0, nullptr};   // 第一次「调用」
        while (stack.top != nullptr) {
            const Frame f = *stack.top;     // 弹出一帧
            Frame* popped = stack.top;
            stack.top = stack.top->next;
            delete popped;

            if (f.in_begin == f.in_end) {
                continue;                   // 空区间：空子树
            }
            const std::size_t size = f.in_end - f.in_begin;
            const T& key = postorder[f.post_begin + size - 1];
            std::size_t k = f.in_begin;
            while (k < f.in_end && inorder[k] != key) {
                ++k;
            }
            if (k == f.in_end) {            // 剩下的帧由 stack 的析构还掉
                throw std::invalid_argument("build_from_inorder_postorder_iterative:
                ↪  root lies outside its inorder range");
            }

            Node* node = new Node{key, nullptr, nullptr};
            *f.link = node;                 // 先挂上：之后再抛异常，它也归临时树释放
```

```
            const std::size_t left_size = k - f.in_begin;
            // 先压右、再压左：栈后进先出，于是左子树先建，与递归的次序一致
            stack.top = new Frame{&node->right, k + 1, f.in_end, f.post_begin +
↪  left_size, stack.top};
            stack.top = new Frame{&node->left, f.in_begin, k, f.post_begin,
↪  stack.top};
        }
        clear();
        root_ = fresh.root_;
        fresh.root_ = nullptr;
    }

    // 【算法 5.3】深度优先周游的三种次序。三个函数只差 visit 那一行的位置：
    //   前序 根左右 ・中序 左根右 ・后序 左右根
    template <typename Visitor>
    void preorder(Visitor visit) const { preorder_impl(root_, visit); }

    template <typename Visitor>
    void inorder(Visitor visit) const { inorder_impl(root_, visit); }

    template <typename Visitor>
    void postorder(Visitor visit) const { postorder_impl(root_, visit); }

    // 【算法 5.7】层次周游：一层一层从左到右。
    // 深度优先靠栈（这里是递归用的运行栈），广度优先靠 ** 队列 **。
    template <typename Visitor>
    void level_order(Visitor visit) const {
        // 一条极简的链式队列，只在这个函数里用
        struct Pending {
            const Node* node;
            Pending* next;
        };
        Pending* front = nullptr;
        Pending* rear = nullptr;

        if (root_ != nullptr) {
            front = rear = new Pending{root_, nullptr};
        }
        while (front != nullptr) {
            Pending* item = front;              // 出队
            front = front->next;
            if (front == nullptr) {
                rear = nullptr;
            }
            const Node* node = item->node;
            delete item;
```

```
            visit(node->value);

            if (node->left != nullptr) {          // 左右孩子依次入队
                Pending* fresh = new Pending{node->left, nullptr};
                if (rear == nullptr) { front = rear = fresh; } else { rear->next =
                ↪  fresh; rear = fresh; }
            }
            if (node->right != nullptr) {
                Pending* fresh = new Pending{node->right, nullptr};
                if (rear == nullptr) { front = rear = fresh; } else { rear->next =
                ↪  fresh; rear = fresh; }
            }
        }
    }

    // 结点数与高度：两个最典型的「先算孩子、再算自己」的递归。
    std::size_t size() const { return count(root_); }
    std::size_t height() const { return depth(root_); }

    void clear() {
        destroy(root_);
        root_ = nullptr;
    }

private:
    //  【代码 5.8】后序释放：** 必须先删两个孩子，再删自己 **。
    // 反过来先 delete node，node->left 就成了读已释放内存。
    static void destroy(Node* node) {
        if (node == nullptr) {
            return;
        }
        destroy(node->left);
        destroy(node->right);
        delete node;
    }

    // 深拷贝：形状和 destroy 一样，只是把" 删" 换成" 建"。
    static Node* clone(const Node* node) {
        if (node == nullptr) {
            return nullptr;
        }
        return new Node{node->value, clone(node->left), clone(node->right)};
    }

    // 把中序 inorder[in_begin, in_end) 与后序 postorder[post_begin, …) 建成一棵子树，
    ↪  挂到 link 上。
    // 两段一样长；只传下标，不拷贝子数组。
```

```
    //
    // 后序段的最后一个是根；在中序段里找到它（下标 k），左边 k - in_begin 个是左子树，
    // 右边是右子树。左子树的大小一旦知道，后序段也就切成了「左子树 | 右子树 | 根」。
    static void build_impl(Node*& link, const T* inorder, std::size_t in_begin,
    ↪  std::size_t in_end,
                           const T* postorder, std::size_t post_begin) {
        if (in_begin == in_end) {
            return;                                          // 空区间：空子树，link 保持
            ↪  nullptr
        }
        const std::size_t size = in_end - in_begin;
        const T& key = postorder[post_begin + size - 1];     // 后序段的最后一个是根

        std::size_t k = in_begin;                            // 在中序段里从左往右找根
        while (k < in_end && inorder[k] != key) {            // 只找本段：根不在本段，说
        ↪  明两个序列对不上
            ++k;
        }
        if (k == in_end) {
            throw std::invalid_argument("build_from_inorder_postorder: root lies
            ↪  outside its inorder range");
        }

        link = new Node{key, nullptr, nullptr};              // 先挂上：子树里再抛异常，
↪  它也归临时树释放
        const std::size_t left_size = k - in_begin;
        build_impl(link->left, inorder, in_begin, k, postorder, post_begin);
        build_impl(link->right, inorder, k + 1, in_end, postorder, post_begin +
↪  left_size);
    }

    template <typename Visitor>
    static void preorder_impl(const Node* node, Visitor& visit) {
        if (node == nullptr) return;
        visit(node->value);                 // 根
        preorder_impl(node->left, visit);   // 左
        preorder_impl(node->right, visit);  // 右
    }

    template <typename Visitor>
    static void inorder_impl(const Node* node, Visitor& visit) {
        if (node == nullptr) return;
        inorder_impl(node->left, visit);    // 左
        visit(node->value);                 // 根
        inorder_impl(node->right, visit);   // 右
    }
```

```
    template <typename Visitor>
    static void postorder_impl(const Node* node, Visitor& visit) {
        if (node == nullptr) return;
        postorder_impl(node->left, visit);  // 左
        postorder_impl(node->right, visit); // 右
        visit(node->value);                 // 根
    }

    static std::size_t count(const Node* node) {
        return node == nullptr ? 0 : 1 + count(node->left) + count(node->right);
    }

    static std::size_t depth(const Node* node) {
        if (node == nullptr) return 0;
        std::size_t l = depth(node->left);
        std::size_t r = depth(node->right);
        return 1 + (l > r ? l : r);
    }

    Node* root_;
};

// ----------------------------------------------------------------------------
// 二叉搜索树
//
// 约束只有一条：** 左子树的键都小于根，右子树的键都大于根 **。
// 有了它，查找就不必遍历全树——每比较一次就砍掉一半（前提是树是平衡的）。
// ----------------------------------------------------------------------------
template <typename T>
class BinarySearchTree {
public:
    struct Node {
        T value;
        Node* left;
        Node* right;
    };

    BinarySearchTree() : root_(nullptr) {}

    ~BinarySearchTree() { clear(); }

    BinarySearchTree(const BinarySearchTree& other) : root_(clone(other.root_)) {}

    BinarySearchTree& operator=(const BinarySearchTree& other) {
        if (this == &other) {
            return *this;
        }
```

```
    Node* fresh = clone(other.root_);
    clear();
    root_ = fresh;
    return *this;
}

bool empty() const { return root_ == nullptr; }

// 【算法 5.9】插入。一路比较着往下走，走到空位就把新结点挂上去。
// 键已存在时返回 false——重复键是可预期状态，不是错误，所以不抛异常。
bool insert(const T& value) {
    Node** link = &root_;               // 指向「新结点该挂在哪根指针上」
    while (*link != nullptr) {
        if (value < (*link)->value) {
            link = &(*link)->left;
        } else if ((*link)->value < value) {
            link = &(*link)->right;
        } else {
            return false;               // 已经有了
        }
    }
    *link = new Node{value, nullptr, nullptr};
    return true;
}

// 查找：同样一路往下走。树高是 h，代价就是 O(h)。
bool contains(const T& value) const {
    const Node* current = root_;
    while (current != nullptr) {
        if (value < current->value) {
            current = current->left;
        } else if (current->value < value) {
            current = current->right;
        } else {
            return true;
        }
    }
    return false;
}

// 【算法 5.10】删除。键不存在返回 false（幂等，不是错误）。
//
// 难点只有一个：被删结点有两个孩子时，谁来顶替它？
// 答案是 ** 中序前驱 **——左子树里最大的那个。它顶上来之后，
// 「左小右大」仍然成立，因为它比左子树其余的都大、比右子树全部都小。
bool remove(const T& value) { return remove_impl(root_, value); }
```

```
    void clear() {
        destroy(root_);
        root_ = nullptr;
    }

    // 中序周游一棵 BST，得到的正是 ** 升序 **——这是「左小右大」的直接推论。
    template <typename Visitor>
    void inorder(Visitor visit) const { inorder_impl(root_, visit); }

private:
    static bool remove_impl(Node*& link, const T& value) {
        if (link == nullptr) {
            return false;
        }
        if (value < link->value) {
            return remove_impl(link->left, value);
        }
        if (link->value < value) {
            return remove_impl(link->right, value);
        }

        Node* removed = link;
        if (removed->left == nullptr) {          // 没有左孩子：右孩子直接顶上
            link = removed->right;
            delete removed;
            return true;
        }

        // 找中序前驱：从左孩子出发，一路向右走到底
        Node** predecessor_link = &removed->left;
        while ((*predecessor_link)->right != nullptr) {
            predecessor_link = &(*predecessor_link)->right;
        }
        Node* replacement = *predecessor_link;
        *predecessor_link = replacement->left;   // 前驱可能还有左孩子，先接走
        replacement->left = removed->left;
        replacement->right = removed->right;
        link = replacement;
        delete removed;
        return true;
    }

    static void destroy(Node* node) {
        if (node == nullptr) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
```

```
    }

    static Node* clone(const Node* node) {
        if (node == nullptr) return nullptr;
        return new Node{node->value, clone(node->left), clone(node->right)};
    }

    template <typename Visitor>
    static void inorder_impl(const Node* node, Visitor& visit) {
        if (node == nullptr) return;
        inorder_impl(node->left, visit);
        visit(node->value);
        inorder_impl(node->right, visit);
    }

    Node* root_;
};
```

5.4 二叉搜索树

为什么这一节没有 Python 版

二叉搜索树的查找算法可以用 Python 讲，但本章这一实现还要展示结点的拥有关系、删除时子树指针的转移、深拷贝和析构。Python 容器会替我们保管这些资源，无法让读者观察“删除一个结点后谁负责释放它”或强异常保证如何维持，所以存储实现不另写 Python 包装。

在实际应用中，经常会碰到这样的操作：在一组记录中检索一个记录，向其中插入一个记录，或者删除 一个记录。对于一个**无序线性表**，插入记录时只需要把该记录放在表的末端，但在表中查找一个特定 记录的检索时间却相对较慢；对于**有序线性表**，如果用二分查找法检索特定的记录，检索效率较高，但是遇到动态增减变化的情况（如插入或删除一个元素）时需要移动大量的元素。很多应用都需要一种 **插入、删除和检索记录效率都较高**的数据组织方法——二叉搜索树（binary search tree，BST，也称 二叉排序树、二叉查找树）就是这样一种高效的数据结构。

二叉搜索树是一类满足以下属性的特殊二叉树：树中的每个非空结点表示一个记录；若某结点左子树 不为空，则**左子树上所有结点的值均小于该结点的关键码值**；若其右子树不为空，则**右子树上所有 结点的值均大于该结点的关键码值**。二叉搜索树可以是一棵空树，且**任何结点的左右子树都是二叉 搜索树**。按照中序周游整个二叉树，可得到一个由小到大的有序排列；这个定义也表明**树中各结点的 关键码必须是唯一的**。原书用关键码集合 $K = \{50, 19, 35, 55, 20, 5, 100, 52, 88, 53, 92\}$ 建的树是这样的：

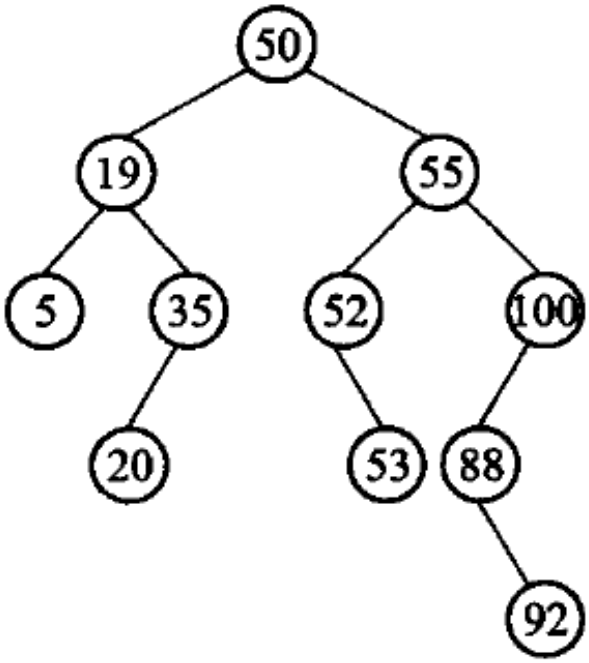


图 5.11　二叉搜索树：从空树起，按 50, 19, 35, 55, 20, 5, 100, 52, 88, 53, 92 的顺序逐个插入得到（中序周游正好是升序 5 19 20 35 50 52 53 55 88 92 100）。检索 key 时把它与根比较：小于就只找左子树，大于就只找右子树，每一步都能扔掉一整棵子树——这就是它比无序线性表快的原因。走到空子树还没找到，就说明 key 不在树里。

插入要先运用检索方法，查找待插入关键码是否在树中：如果存在则不允许插入重复关键码；如果 直到找到叶结点还没有发现重复关键码，则把新结点插入到待插入方向作为新的叶结点。它**不像在 有序线性表中插入元素那样要移动大量的数据**，只需改动某个结点的空指针；与查找一样，插入一个 新结点的时间复杂度是根到插入位置的路径长度，因此**在树形比较平衡时二叉搜索树的效率相当高**。对于给定的关键码集合，可以从一棵空的二叉搜索树开始，按照检索路径逐个插入到相应的叶结点位置，从而动态生成二叉搜索树。

删除比较复杂：要保持二叉搜索树的性质，就不能在树中留下一个空位置，因此需要用另一个结点来 填充这个位置并且保持性质。插入重复键、删除不存在的键都是可预期状态，返回 `false`，不抛异常。删除有左右孩子的结点时，用左子树里最右的前驱替换它：先把前驱从原位置摘下，再让它继承被删 结点的两棵子树，最后只 `delete` 被删结点一次。漏掉「先脱离原父」会形成环或二次释放。

二叉搜索树的实现就在上面那份教学版清单里的 `BinarySearchTree`。三处值得停一下：

- **插入用的是「指向指针的指针」** `Node** link`。它指向「新结点该挂在哪根指针上」，于是「挂到根」和「挂到某个孩子」写成同一句 `*link = new Node{...}`，

空树不必另开一个分支。

- **删除有两个孩子的结点时，用左子树里最右的那个（中序前驱）顶替**。它比左子树其余的都大、比右子树全部都小，顶上来之后「左小右大」仍然成立。
- **前驱自己可能还有一个左孩子**，顶替之前必须先把它接走（`*predecessor_link = replacement->left;`）。漏了这一句，树上会出现环—— 教学版的测试专门搭了一棵这样的树，去掉那句，中序周游立刻无限递归、ASan 报栈溢出。

删除分几种情形，原书的图把它们摆开了：

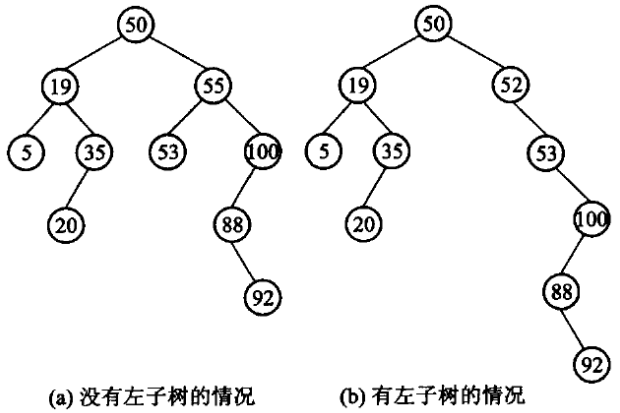


图 5.12　二叉搜索树的删除（都从图 5.11 出发，用下文的基本方案）：(a) 删 52——它没有左子树，直接让右子树的根 53 顶上；(b) 删 55——它有左子树 52 → 53，左子树中序最后的结点是 53，把 55 的右子树（以 100 为根）整个挂到 53 的右边，再让左子树的根 52 顶替 55。原树深度是 4（50 → 55 → 100 → 88 → 92），删完反而变成 5（50 → 52 → 53 → 100 → 88 → 92）。

原书在这里给了两种删除方案，本书实现的是后一种。（原书【算法 5.9】是上面讲的插入，基本删除方案只有文字描述，改进方案才写成【算法 5.10】。）基本方案是：若被删结点 `pointer` 有左子树，就在左子树里找到按中序周游的最后一个结点 `temppointer`，把 `temppointer` 的右指针置成指向 `pointer` 右子树的根，然后用 `pointer` 左子树的根代替被删结点。它是对的，但从图 5.12(b) 可以看到，**把 `pointer` 的右子树整个下降为 `temppointer` 的右子树后，树的高度 可能会增加**——删了一个结点，树反而更瘦长了。

改进方案（原书【算法 5.10】）换一个顺序：若 `pointer` 有左子树，先在左子树中找到中序周游的 最后一个结点 `temppointer`（即左子树中的最大结点），**并将其从二叉搜索树中删除**；由于 `temppointer` 没有右子树，删除它只需用它的左子树代替它；然后用 `temppointer` 结点代替待删除 的结点 `pointer`。这样接上去的是同一层的邻居，高度不会凭空长高。上面教学版实现里那三句 `*predecessor_link = replacement->left;` / `replacement->left = ...` / `replacement->right = ...`，逐句对应的就是这个改进方案。

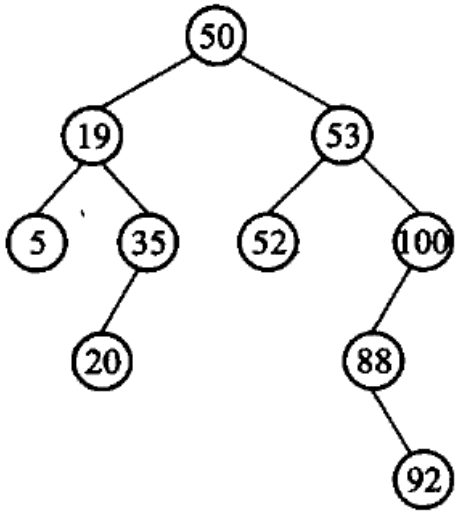


图 5.13　改进的删除办法：同样从图 5.11 删 55，先把左子树里最右的结点 53 摘下（它没有左孩子，原位置直接置空），再让 53 接过 55 的左子树 52 和右子树 100，顶替 55。删完深度仍是 4。用左子树里最右的那个结点（中序前驱）顶替被删结点。它比左子树其余全部都大、比右子树全部都小，顶上来之后「左小右大」原封不动，树也不会因为整棵子树搬家而变高。

5.5 堆与优先队列

为什么这一节没有 Python 版

这里的教学版堆把完全二叉树直接放进自有数组，并明确处理扩容、元素移动和对象销毁；优先队列则依赖这份存储不变量。Python `heapq` 一行就替代了整节，Python `list` 也不会呈现五法则与异常安全。为了不把存储课伪装成算法调用，本节保留 C++ 教学实现；算法侧的排序和动态规划另有 Python 双实现。

在现实应用中，经常会遇到频繁地在一组对象中查找最大值或最小值的情况。为了达到这个目的，可以 每次都先排序，然后从已排序的序列中找到其最大值或最小值——这种方法虽然可行，但时间开销比较 大。有没有一种结构能为这类特殊应用提供较高的效率？这就是**堆**。堆可分为最小堆和最大堆，本节 主要讨论最小堆，最大堆具有类似的性质。

最小堆（min-heap，最小值堆）是关键码序列 $\{K_0, K_1, \cdots, K_{n-1}\}$，它具有如下特性：

$$K_i \le K_{2i+1}, \quad K_i \le K_{2i+2} \quad (i = 0, 1, \cdots, \lfloor n/2 \rfloor - 1)$$

（下标超过 $n-1$ 的那一项不存在，就不必比较：例如 n 为偶数时，最后一个内部结点 $i = n/2 - 1$ 只有左孩子 $2i + 1 = n - 1$。）

从逻辑的角度来讲，堆是一种树形结构，而且是一种**特殊的完全二叉树**：此完全二叉树的每个结点 对应于序列中的一个关键码，根结点对应于 K_0，按层次从左至右依次类推。说其特殊，主要是因为 **堆序只是局部有序——**最小堆对应的完全二叉树中所有内部结点的值均不大于其左右子结点的关键 码值，且一个结点与其兄弟之间没有必然的联系。最小堆不像二叉搜索树那样实现了关键码的完全排序；相比较而言，只有当结点之间是父子关系时，才可以确定这两个结点关键码的大小关系。

根据最小堆的定义，堆对应的完全二叉树的根结点具有最小关键码值，即堆所对应序列的第一个元素 具有最小值。5.3.2 节已经介绍过用顺序存储结构实现完全二叉树的有效方法，因此可以把最小堆的 关键码存储在一维数组中：只需要计算简单的代数表达式，就能非常容易地查找某个结点的父结点和 子结点（下标 `i` 的孩子是 `2i+1` 和 `2i+2`），**既避免了使用指针来保持结构，又能有效地执行相应 操作**。当然，由于顺序存储的空间分配要求，必须预先知道堆的大小（本书的实现按需扩容）。

插入与删除。插入时新添的元素加入末尾；为了保持最小堆的性质，需要沿着其祖先的路径，自下 而上依次比较和交换该结点与父结点的位置，直到重新满足堆的性质为止。这样做会出现两种情况：要么新结点升到最小堆的顶端，要么到某一位置时发现父结点比新插入的结点关键码值小。这个自下 而上逐渐上升、最后停在满足最小堆性质的位置的过程，通常被称为「**筛选**」。删除操作的处理与 插入时方向相反：删除某个位置的元素后形成了一个空位置，首先把最末端的结点填入这个位置；同理，这样做也可能导致破坏堆序特性，末端元素需要与被删位置的子结点比较交换，直到过滤到该结点小于 最小子结点的正确位置为止。删除的若是**堆顶以外的任意位置**，还要多看一眼父结点：填进来的末端元素可能比新父结点还小（它原本来自另一棵子树），这时要改为**向上**筛选。例如堆 `1 10 2 11 12 3 4` 删除下标 3 的 11，填进来的 4 比父结点 10 小，就得上浮。原书【代码 5.11】的 `Remove(pos)` 正是先比父结点、再决定上浮还是下沉。

建堆。把所有关键码放到一维数组中，此时形成的完全二叉树并不具备最小堆的特性，但是**仅包含 叶子结点的子树已经是堆——**在有 n 个结点的完全二叉树中，当 $i > \lfloor n/2 \rfloor - 1$ 时，以 关键码 K_i 为根的子树已经是堆。这时，从含有内部结点数最少的子树（这种子树在完全二叉树的 倒数第二层，此时 $i = \lfloor n/2 \rfloor - 1$）开始，从右至左依次进行调整；对这一层调整完成之后，继续对上一层进行同样的工作，直到整个过程到达树根时，整棵完全二叉树就成为一个堆。

对当前结点 K_i，调整过程是从 K_i 开始向下筛选的过程：如果 $K_i \le K_{2i+1}$ 且 $K_i \le K_{2i+2}$，则以 K_i 为根的子树就已经是堆，不需要调整结点的位置；否则将 K_i 的值与 其子结点中关键码小的一个进行交换，交换后继续对以该子结点为根的子树进行重建，这样在最坏 情况下向下调整到树叶，然后依次调整以 $K_{i-1}, K_{i-2}, \cdots, K_0$ 为根的子树。完成了以 K_0 为根的树的筛选，就完成了建堆的过程。

最后一句提醒：最小堆只适合于查找最小值，**查找任意值的效率不高——**它

没有二叉搜索树那样的 全序，找一个中间大小的关键码只能线性扫描。类似地也可以定义最大堆，第 8.3.2 节的堆排序用的 就是最大堆。

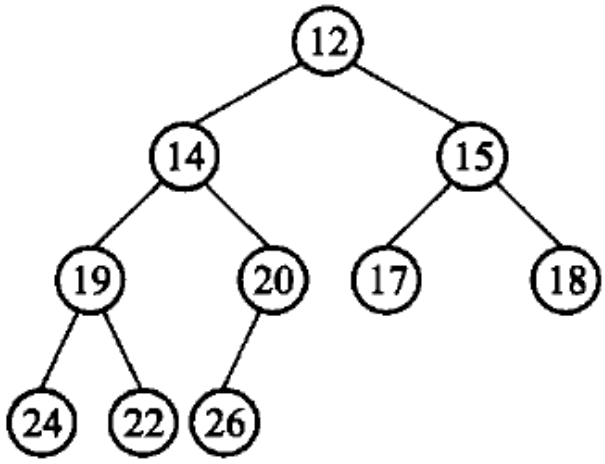


图 5.14　关键码序列 $K = \{12, 14, 15, 19, 20, 17, 18, 24, 22, 26\}$ 对应的最小堆。堆序是**局部有序**：只有父子之间才能断定大小，兄弟之间没有任何关系——这一点和二叉搜索树完全不同，也正是建堆能比排序更快的原因。根一定是最小的那个。

```
// 第 5 章「先跑一遍」：用教学版 MinHeap 与 HuffmanTree。
// 编译运行：
//   g++ -std=c++17 -I code/ch05/heap_huffman code/ch05/heap_huffman/demo.cpp -o
↪  demo && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    MinHeap<int> heap;
    for (int value : {5, 1, 4, 2}) {
        heap.insert(value);
    }
    std::cout << " 依次取出最小元:";
    while (auto value = heap.remove_min()) {   // 空堆返回 nullopt，循环自然结束
        std::cout << ' ' << *value;
    }
    std::cout << '\n';

    const int weights[] = {2, 3, 4, 7};
    const HuffmanTree tree(weights, 4);
    std::cout << " 权 2,3,4,7 的 Huffman 树根权 = " << tree.total_weight() << '\n';
    std::cout << " 带权路径长度 WPL = " << tree.weighted_path_length() << '\n';
```

```
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch05/heap_huffman \
    code/ch05/heap_huffman/demo.cpp -o /tmp/heap-demo
/tmp/heap-demo
```

```
依次取出最小元: 1 2 4 5
权 2,3,4,7 的 Huffman 树根权 = 16
带权路径长度 WPL = 30
```

教学版：完整实现

最小堆与 Huffman 树放在同一个文件里——因为 Huffman 的构造规则「反复取两个最小的合并」正是堆的第一个真实用途，两节内容在这里接上。

```
// 最小堆与 Huffman 树 ——教学版。原书【代码 5.11】【代码 5.12】。
//
// 一个文件、两个类、能直接编译运行，给「第一次读这一节」的人看。
//
//   MinHeap      完全二叉树用 ** 数组 ** 存：下标 i 的孩子是 2i+1 和 2i+2，父亲是 (i-1)/
↪  2。
//                不需要任何指针，这正是这一节最漂亮的地方。
//   HuffmanTree  反复「取两个最小的合并」，用最小堆来取——这是堆的第一个真实用途。
//
// 与 modern.hpp（工程版）的分工：
//   教学版   三法则、扩容不考虑异常、Huffman 构造不做溢出与失败清理；
//   工程版   五法则、对元素类型的 static_assert、构造中途失败时逐个回收裸结点、
//            权重相加的溢出检查。
// 两份都在闸门里真编译真运行。先读这一份，5.6a「进阶（选读）」再读那一份。
#pragma once

#include <cstddef>
#include <optional>
#include <stdexcept>
#include <string>

// ---------------------------------------------------------------------------
// 最小堆
//
// 堆是一棵 ** 完全二叉树 **（除最后一层外每层排满，最后一层靠左连续），
// 且每个结点都不大于它的孩子。完全二叉树可以按层次次序压进一个数组，
// 于是父子关系变成下标算术：
//
```

```
//   下标 i 的左孩子  2i + 1
//   下标 i 的右孩子  2i + 2
//   下标 i 的父亲    (i - 1) / 2        (i > 0)
//
// 一根指针都不用。
// ----------------------------------------------------------------------------
template <typename T>
class MinHeap {
public:
    using size_type = std::size_t;

    explicit MinHeap(size_type initial_capacity = 8)
        : data_(new T[initial_capacity]), capacity_(initial_capacity), size_(0) {}

    ~MinHeap() { delete[] data_; }

    // 三法则：管着 new 出来的数组，拷贝必须自己写。
    MinHeap(const MinHeap& other)
        : data_(new T[other.capacity_]), capacity_(other.capacity_),
        ↪  size_(other.size_) {
        for (size_type i = 0; i < size_; ++i) {
            data_[i] = other.data_[i];
        }
    }

    MinHeap& operator=(const MinHeap& other) {
        if (this == &other) {
            return *this;
        }
        T* fresh = new T[other.capacity_];
        for (size_type i = 0; i < other.size_; ++i) {
            fresh[i] = other.data_[i];
        }
        delete[] data_;
        data_ = fresh;
        capacity_ = other.capacity_;
        size_ = other.size_;
        return *this;
    }

    bool empty() const { return size_ == 0; }
    size_type size() const { return size_; }

    // 插入：先放到数组末尾（也就是完全二叉树的最后一个位置），
    // 再一路和父亲比较、必要时上浮。树高是 log n，所以代价是 O(log n)。
    void insert(const T& value) {
        if (size_ == capacity_) {
```

```
            grow();
        }
        data_[size_] = value;
        sift_up(size_);
        ++size_;
    }

    // 取走最小的那个（就是根，下标 0）。空堆返回空 optional。
    //
    // 手法是固定的：把 ** 最后一个 ** 元素搬到根上，长度减一，然后让它一路下沉。
    // 为什么是最后一个？因为只有拿掉最后一个位置，剩下的才仍然是一棵完全二叉树。
    std::optional<T> remove_min() {
        if (empty()) {
            return std::nullopt;
        }
        T smallest = data_[0];
        --size_;
        if (size_ > 0) {
            data_[0] = data_[size_];
            sift_down(0);
        }
        return smallest;
    }

private:
    // 上浮：只要比父亲小就换上去。
    void sift_up(size_type index) {
        while (index > 0) {
            size_type parent = (index - 1) / 2;
            if (!(data_[index] < data_[parent])) {
                break;                          // 已经不小于父亲，位置对了
            }
            T tmp = data_[index];
            data_[index] = data_[parent];
            data_[parent] = tmp;
            index = parent;
        }
    }

    // 下沉：和两个孩子里较小的那个比，比它大就换下去。
    // ** 必须和较小的那个换 **——跟较大的换会破坏「父亲不大于两个孩子」。
    void sift_down(size_type index) {
        for (;;) {
            size_type left = index * 2 + 1;
            size_type right = left + 1;
            size_type smallest = index;
            if (left < size_ && data_[left] < data_[smallest]) {
```

```
                smallest = left;
            }
            if (right < size_ && data_[right] < data_[smallest]) {
                smallest = right;
            }
            if (smallest == index) {
                return;                      // 父亲已经最小，停
            }
            T tmp = data_[index];
            data_[index] = data_[smallest];
            data_[smallest] = tmp;
            index = smallest;
        }
    }

    void grow() {
        size_type next = (capacity_ == 0) ? 1 : capacity_ * 2;
        T* fresh = new T[next];
        for (size_type i = 0; i < size_; ++i) {
            fresh[i] = data_[i];
        }
        delete[] data_;
        data_ = fresh;
        capacity_ = next;
    }

    T* data_;
    size_type capacity_;
    size_type size_;
};

// ---------------------------------------------------------------------------
// Huffman 树
//
// 构造规则只有一句：** 反复取出权最小的两棵树，合并成一棵新树放回去 **，
// 直到只剩一棵。「取最小的」正是最小堆的拿手好戏，两节内容在这里接上了。
// ---------------------------------------------------------------------------
class HuffmanTree {
public:
    struct Node {
        int weight;
        char symbol;      // 只有叶子有意义；内部结点是合并出来的，置 '\0'
        Node* left;
        Node* right;
    };

    HuffmanTree() : root_(nullptr) {}
```

```
    /// 只关心树形与 WPL 时用这个。
    HuffmanTree(const int* weights, std::size_t count) : HuffmanTree(nullptr,
↪   weights, count) {}

    /// 带上每个权重对应的字符，树就能拿来 ** 编码和译码 ** 了。
    HuffmanTree(const char* symbols, const int* weights, std::size_t count) :
↪   root_(nullptr) {
        if (count == 0) {
            return;
        }
        if (weights == nullptr) {
            throw std::invalid_argument("HuffmanTree: 权重数组是空指针");
        }

        // ** 先把参数全查一遍，再动手 new。** 顺序反过来的话，
        // 在第 k 个权重上发现非法值时前 k-1 个结点已经建好了，
        // 抛出去就全漏了——LeakSanitizer 会当场把它报出来（作者第一版正是如此）。
        for (std::size_t i = 0; i < count; ++i) {
            if (weights[i] < 0) {
                throw std::invalid_argument("HuffmanTree: 权重不能为负");
            }
        }

        MinHeap<ByWeight> heap;
        for (std::size_t i = 0; i < count; ++i) {
            char symbol = (symbols == nullptr) ? '\0' : symbols[i];
            heap.insert(ByWeight{new Node{weights[i], symbol, nullptr, nullptr}});
↪   // 每个权重先做成一棵单结点树
        }

        while (heap.size() > 1) {
            Node* left = heap.remove_min()->node;      // 最小的
            Node* right = heap.remove_min()->node;     // 次小的
           Node* parent = new Node{left->weight + right->weight, '\0', left, right};
            heap.insert(ByWeight{parent});
        }
        root_ = heap.remove_min()->node;
    }

    ~HuffmanTree() { destroy(root_); }

    // 这棵树不支持拷贝：结点是裸指针，深拷贝要写一整套，而 Huffman 树建好就只读。
    // 明确 `= delete` 好过让编译器悄悄生成一个会二次释放的版本。
    HuffmanTree(const HuffmanTree&) = delete;
    HuffmanTree& operator=(const HuffmanTree&) = delete;
```

```
    const Node* root() const { return root_; }

    // 根的权重就是所有叶子权重之和。
    int total_weight() const { return root_ == nullptr ? 0 : root_->weight; }

    // 带权路径长度 (WPL)：每个叶子的权重乘以它的深度，再求和。
    // Huffman 树的意义就在于它让这个数最小。
    int weighted_path_length() const { return wpl(root_, 0); }

    // ---- Huffman 编码与译码 ------------------------------------------------
    //
    // 编码规则：从每个结点引向 ** 左 ** 孩子的边标 0，引向 ** 右 ** 孩子的边标 1；
    // 从根走到某个叶子，一路上的 0/1 连起来就是那个字符的编码。
    //
    // 这样得到的一定是 ** 前缀码 **——任何字符的编码都不会是另一个字符编码的前缀。
    // 理由很直白：字符只住在 ** 叶子 ** 上，而一个叶子不可能在另一个叶子的路径上。
    // 前缀码是能译码的前提：不然 011 既可以读成 a+bb 也可以读成 c+b，无法分辨。

    /// 查一个字符的编码。不在树里返回空 optional。
    std::optional<std::string> code_of(char symbol) const {
        std::string path;
        if (find_code(root_, symbol, path)) {
            // 只有一个字符时，从根到叶的路径是空串——那样的编码没法传。
            // 约定给它一位 "0"。这是个真实的边界，不是抠字眼。
            return path.empty() ? std::string("0") : path;
        }
        return std::nullopt;
    }

    /// 把一段文字编成比特串。出现了树里没有的字符就返回空 optional。
    std::optional<std::string> encode(const std::string& text) const {
        std::string bits;
        for (char c : text) {
            auto code = code_of(c);
            if (!code) {
                return std::nullopt;
            }
            bits += *code;
        }
        return bits;
    }

    /// 把比特串译回文字。** 用的是同一棵树 **：从根出发，读 0 走左、读 1 走右，
    /// 走到叶子就吐出一个字符再回到根。比特串不合法（多出半截、出现非 0/1）
    /// 就返回空 optional。
    std::optional<std::string> decode(const std::string& bits) const {
        if (root_ == nullptr) {
```

```
            return bits.empty() ? std::optional<std::string>(std::string()) :
            ↪  std::nullopt;
        }
        // 单结点树是特例：整棵树只有一个字符，每一位都译成它。
        if (root_->left == nullptr && root_->right == nullptr) {
            std::string text;
            for (char b : bits) {
                if (b != '0') {
                    return std::nullopt;
                }
                text.push_back(root_->symbol);
            }
            return text;
        }

        std::string text;
        const Node* current = root_;
        for (char b : bits) {
            if (b == '0') {
                current = current->left;
            } else if (b == '1') {
                current = current->right;
            } else {
                return std::nullopt;          // 既不是 0 也不是 1
            }
            if (current == nullptr) {
                return std::nullopt;
            }
            if (current->left == nullptr && current->right == nullptr) {
                text.push_back(current->symbol);
                current = root_;              // 吐出一个字符，回到根
            }
        }
        if (current != root_) {
            return std::nullopt;              // 走到一半就没比特了：串不完整
        }
        return text;
    }

private:
    // 放进堆里的是「一棵树的根指针」，比较的是它的权重。
    struct ByWeight {
        Node* node;
        bool operator<(const ByWeight& other) const {
            return node->weight < other.node->weight;
        }
    };
```

```
    static void destroy(Node* node) {
        if (node == nullptr) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }

    /// 在树里找字符 symbol，把根到它的路径（左 0 右 1）写进 path。
    static bool find_code(const Node* node, char symbol, std::string& path) {
        if (node == nullptr) {
            return false;
        }
        if (node->left == nullptr && node->right == nullptr) {
            return node->symbol == symbol;
        }
        path.push_back('0');
        if (find_code(node->left, symbol, path)) {
            return true;
        }
        path.back() = '1';
        if (find_code(node->right, symbol, path)) {
            return true;
        }
        path.pop_back();                        // 这条路走不通，回溯
        return false;
    }

    static int wpl(const Node* node, int depth) {
        if (node == nullptr) return 0;
        if (node->left == nullptr && node->right == nullptr) {
            return node->weight * depth;        // 叶子
        }
        return wpl(node->left, depth + 1) + wpl(node->right, depth + 1);
    }

    Node* root_;
};
```

插入的手法也是固定的：**新元素先放在末尾**（这样仍是完全二叉树），再沿着祖先一路向上比较交换，直到父结点比它小、或者它升到了根。这个上升过程叫**筛选**。

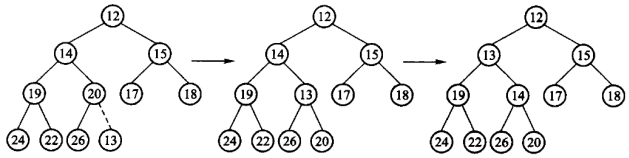


图 5.15　在图 5.14 的最小堆里插入 13：13 从末尾一路上浮，停在父结点比它小的地方。具体是：13 放在下标 10，父结点（下标 4）是 20，13<20，交换；新的父结点（下标 1）是 14，13<14，再交换；父结点成了根 12，12<13，停。数组变成 `12 13 15 19 14 17 18 24 22 26 20`，只交换了 2 次。

`remove_min` 的手法是固定的：**把最后一个元素搬到根上，长度减一，再让它下沉**。为什么是最后一个？因为只有拿掉最后一个位置，剩下的才仍然是一棵完全二叉树。下沉时**必须和两个孩子里较小的那个交换**——跟较大的换会破坏「父亲不大于两个孩子」。

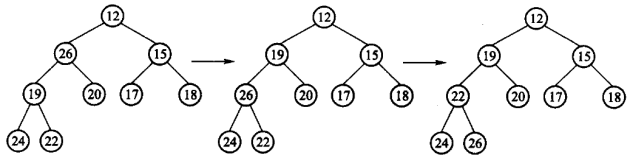


图 5.16　在图 5.14 的最小堆里删除 14：末端元素填进空出来的位置，再与较小的那个孩子逐层交换下沉，直到它不大于自己的孩子。具体是：14 在下标 1，末端的 26 填进来；它的孩子 19、20 里小的是 19，26>19，交换；再看孩子 24、22，小的是 22，26>22，交换；26 到了叶子，停。数组变成 `12 19 15 22 20 17 18 24 26`。

5.5.1a 建堆为什么是 $O(n)$ 而不是 $O(n \log n)$

把一个无序序列变成堆，做法是**从最后一个非叶结点起，倒着对每个结点做一次下沉**：

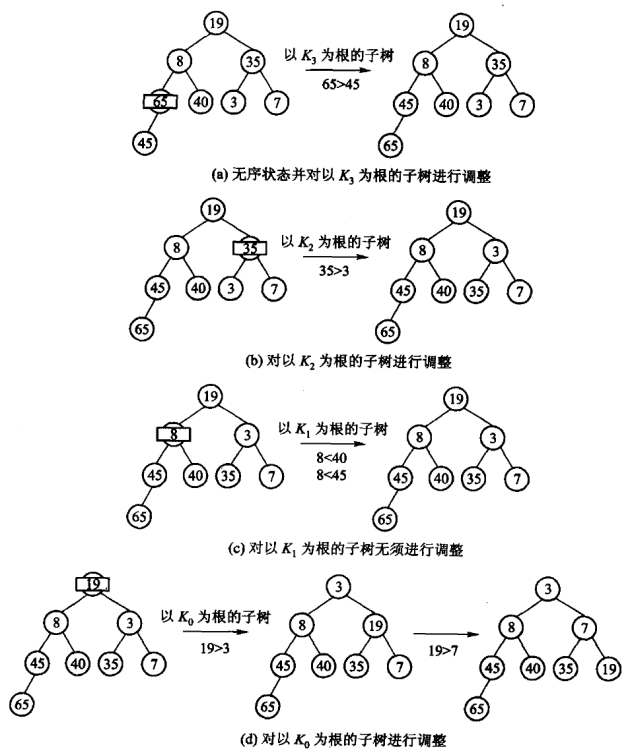


图 5.17　建堆过程示例（原书图 5.17 的筛选法）　。关键码集合 $K = \{19, 8, 35, 65, 40, 3, 7, 45\}$，$n = 8$，$\lfloor n/2 \rfloor - 1 = 3$，所以从 $K_3 = 65$ 开始倒着调整：(a) $i = 3$：65 的孩子只有 45，65>45，交换；(b) $i = 2$：35 的孩子 3、7 里小的是 3，交换；(c) $i = 1$：8 不大于孩子 45、40，不动；(d) $i = 0$：19 先与较小的孩子 3 交换，到了下标 2 再与较小的孩子 7 交换。最后得到堆 `3 8 7 45 40 35 19 65`。倒着做的理由是：对某个结点下沉时，它的两棵子树必须已经是堆——从最后一个非叶结点往前扫，正好保证这一点。叶结点本身就是堆，所以后一半结点一次都不用碰。

粗看这是 $O(n \log n)$：`sift_down` 一次最多走一条树高那么长的路，是 $O(\log n)$；非叶结点大约 $n/2$ 个，乘起来就是 $n/2 \cdot \log n$。**原书特意说这只是一个粗略的上界**。

细算要用到一件事：**下沉的代价不是 $\log n$，而是「这个结点到最底层还有多远」**。越靠近底层的结点越多，而它们能下沉的距离越短——两者正好互相抵消。

先把这个堆放大成一棵每层都排满的二叉树（perfect）。设堆的深度（最大层数，根在第 0 层）为 $h = \lfloor \log_2 n \rfloor$，见 5.1.3 的性质 6。深度同为 h、每层都排满的二叉树有 $2^{h+1} - 1 \geq n$ 个结点，而且每个结点能下沉的距离都不比原堆里对应位置的短。**所以在这棵满树上算出来的总代价，是原堆的一个上界**——这一步是必要的，因为一般的 n 下堆的最后一层未必满，层数也要取整，直接把「共 $\log n$ 层」当等式用是不严谨的。

满树的第 i 层**恰有** 2^i 个结点，其中的结点最多下沉 $h-i$ 层，于是

$$\text{建堆代价} \;\le\; \sum_{i=0}^{h} 2^i\,(h-i)$$

令 $j=h-i$ 代入：

$$\sum_{i=0}^{h} 2^i(h-i) \;=\; \sum_{j=0}^{h} 2^{h-j}\cdot j \;=\; 2^h \sum_{j=0}^{h} \frac{j}{2^j} \;<\; 2^h\cdot 2 \;=\; 2^{h+1} \;\le\; 2n$$

最后两步用的是 $\sum_{j\ge 0} j/2^j = 2$，以及 $h=\lfloor \log_2 n \rfloor$ 蕴含 $2^h \le n$。

所以**建堆的时间复杂度是** $O(n)$**——可以在线性时间内把一个无序序列变成堆**。

> **另一种写法**（不必先放大成满树）：n 个结点的堆里，往下到最远叶子还有 j 条边（这里把它叫做结点的「高度 j」，与 5.1.3 整棵树的高度口径不同）的结点**至多** $\lceil n/2^{j+1} \rceil$ 个，每个最多下沉 j 层，于是总代价 $\le \sum_{j\ge 0} \lceil n/2^{j+1} \rceil \cdot j = O(n)$。两种证法算的是同一件事：**越靠近底层的结点越多，能沉的距离越短**。

这个结论有实际后果：第 8.3.2 节的堆排序总代价是 $O(n \log n)$，但那 $\log n$ 全部来自**后面** n **次取最小值**，建堆那一步是白送的。反过来，如果一个个 `insert` 建堆—— 每次上浮 $O(\log n)$，n 次就是 $O(n \log n)$——就把这份便宜丢掉了。

堆建好之后，插入、删除最小元、删除任意元素（已知它在数组里的下标时）的平均与最差代价都是 $O(\log n)$，因为树高就是 $\log n$。但**最小堆只适合查最小值，查任意值的效率不高**—— 要找一个给定的键，除了从头扫没有更好的办法。这正是第 10 章要另起炉灶讲检索的原因。

5.5.2 优先队列

优先队列（priority queue）是一种有用的数据结构：它是 0 个或多个元素的集合，每个元素都有一个 关键码值，执行的操作有查找、插入和删除等。它的主要特点是支持从一个集合中**快速地查找并移出 具有最大值或最小值的元素——**最小优先队列适合查找和删除最小元素，最大优先队列适合查找和删除 最大元素。

优先队列的应用很广泛。例如，计算机的操作系统用一个优先队列来实现等候进程的调度管理：在一 系列等待执行的进程中，每一个进程可以用一个数值来表示它的优先级，优先级越高这个值就越小，优先级高的进程应该最先获得处理器。另一个例子是打印机的输出任务队列：对于先后到达的、打印 几百页和只有几页的任务，一个合理的方法是先打印页数少的任务，这样做就是按照文件的大小来 排列打印任务的优先顺序。

优先队列可以用多种方法实现，例如采用无序线性表或者有序线性表：无序表的插入可为 $O(1)$，但取最小值要 $O(n)$；有序表取最小值可为 $O(1)$，但插入要移动元素。**堆是一种很好的优先队列 实现方法**：一旦最小堆建成，其堆顶元素就满足关键码最小的要求，为快速查找和删除优先级最高的 元素创造条件；进行插入、删除等操作时，也能有效地保持堆的性质，从而确保优先队列的高效性。具体地说，查看堆顶为 $O(1)$，插入与移除堆顶为 $O(\log n)$，建堆为 $O(n)$。

本书的 `MinHeap<T>` 就是一个最小优先队列：`insert` 对应入队，`remove_min` 对应取出最高优先级任务，空队列以 `std::nullopt` 表示。Huffman 构造反复取两个最小权值，Dijkstra 和 Prim 则反复取当前距离或边权最小的候选；这些算法需要的是“下一项最优”，不需要把整个队列排序。若应用要求相同优先级保持到达顺序，还要把到达序号作为第二关键码显式存入元素。

对顶堆：动态维护中位数

优先队列的一个经典上机题：数据一个一个到来，每来一个就要报出当前的中位数。每次排序是 $O(n\log n)$；用两个堆可以做到插入 $O(\log n)$、查询 $O(1)$。

做法是把已到的数据劈成两半：**较小的一半放进最大堆，较大的一半放进最小堆**，并始终保持两件事——最大堆里的每个元素都不大于最小堆里的每个元素；两个堆一样大，或者最大堆多一个。于是中位数就是最大堆的堆顶。新元素不大于最大堆堆顶就进最大堆，否则进最小堆；进完之后若两堆的大小差出了规定，把多的那边的堆顶搬到另一边——一次搬一个就够，因为插入前两堆至多差 1。两个堆背靠背、堆顶相对，所以叫「对顶堆」。

这里需要一个**最大堆**。工程版没有再写一个类，而是给 `MinHeap` 加了一个比较器模板参数，默认 `std::less<T>`；`MinHeap<T, std::greater<T>>` 的堆顶就是最大元素。原书代码 5.11 那个堆的结构、上浮与下沉一行没变，只是把 `<` 换成了 `compare_`。另外加了一个只看不拿的 `MinHeap::peek`，与第 3 章栈的 `peek` 同一个约定：返回指针，空堆为 `nullptr`，下一次插入或删除后失效。

```
/// 对顶堆求动态中位数：较小的一半放进 ** 最大堆 ** lower_，较大的一半放进 ** 最小堆 **
↪  upper_。
///
/// 不变式：lower_ 的每个元素 ≤ upper_ 的每个元素，且
///         lower_.size() == upper_.size() 或 lower_.size() == upper_.size() + 1。
/// 于是中位数永远是 lower_ 的堆顶。元素个数为偶数时取 ** 下中位数 **（第 n/2 小，从 1 数），
/// 不做两数平均——那需要 T 支持除法，而且整数平均会截断，调用方要平均就自己取两个堆顶。
/// insert 为 O(log n)，median 为 O(1)。
template <typename T>
class RunningMedian {
public:
    void insert(const T& value) {
        if (lower_.empty() || !(*lower_.peek() < value)) lower_.insert(value);
        else upper_.insert(value);
```

```
        rebalance();
    }
    /// 空时 std::nullopt。
    [[nodiscard]] std::optional<T> median() const {
        if (const T* top = lower_.peek()) return *top;
        return std::nullopt;
    }
    [[nodiscard]] std::size_t size() const noexcept { return lower_.size() +
    ↪  upper_.size(); }

private:
    /// 每次插入后两堆大小至多差 2，搬一个堆顶就回到不变式。
    void rebalance() {
        if (lower_.size() > upper_.size() + 1) upper_.insert(*lower_.remove_min());
        else if (upper_.size() > lower_.size()) lower_.insert(*upper_.remove_min());
    }
    MinHeap<T, std::greater<T>> lower_;   // 较小的一半，堆顶是其中最大的
    MinHeap<T> upper_;                    // 较大的一半，堆顶是其中最小的
};
```

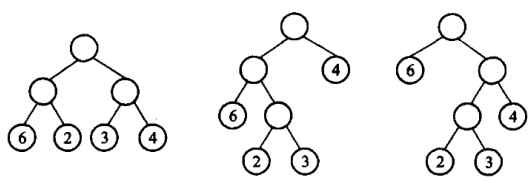
元素个数为偶数时，`RunningMedian::median` 返回**下中位数**（排好序后第 $n/2$ 个，从 1 数），不做两数平均：平均需要 `T` 支持除法，整数平均还会截断，要平均的调用方自己取两个堆顶即可。测试把每一次插入后的结果都与「复制一份排序取中间」对比，数据取自很小的值域，重复值很多；另有全升序、全降序两条，分别让新元素一直落进同一边、逼着每次都搬堆顶。

5.6 Huffman 树及其应用

为什么这一节没有 Python 版

本节的 Huffman 教学实现同时承担树结点的所有权、优先队列数组和异常路径的清理；例如权值校验必须在第一次 `new` 之前完成，否则中途抛异常会泄漏已经建立的结点。Python 的 `heapq` 与垃圾回收会把这两个存储问题藏掉，留下的只是一个算法结果。因此这里不提供 Python 版，避免读者误以为两种语言承担的是同一层教学内容。

同样是四个权为 6、2、3、4 的外部结点，树的形状不同，带权外部路径长度就不同：



(a) 外部路径为30　(b) 外部路径为31　(c) 外部路径为29

图 5.18　三棵外部结点权值同为 6、2、3、4 的扩充二叉树。(a) $6\times 2+$

$2\times2+3\times2+4\times2=30$；(b) $6\times2+2\times3+3\times3+4\times1=31$；(c) $6\times1+2\times3+3\times3+4\times2=29$。(c) 最小，它就是这组权值的 Huffman 树——**权越大的叶离根越近**，总长就越小。

Huffman 树反复取出两个最小权，合成它们的和，直到只剩一棵——这就是前缀编码的那棵树。根权等于全部叶子权之和。实现就在 5.5 那份教学版清单里的 `HuffmanTree`。

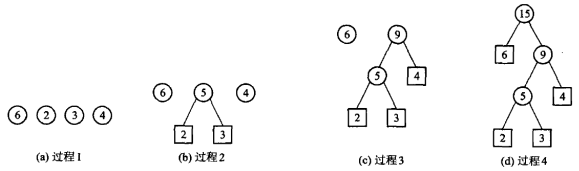


图 5.19　Huffman 树的构造过程：n 个权各自成为一棵单结点树；每次挑出根权最小的两棵，合成一棵新树、新根的权是两者之和；重复 $n-1$ 次，集合里只剩一棵。图中是图 5.18(c) 那棵树的构造：权 6、2、3、4 → 先合并最小的 2 和 3 得 5 → 再合并 4 和 5 得 9 → 最后合并 6 和 9 得 15。三个内部结点的权之和 $5+9+15=29$，正是图 5.18(c) 的带权外部路径长度（理由见下文「WPL 等于所有内部结点的权之和」）。

为什么用堆：每一轮都要「取当前最小的两棵」。用数组线性扫是 O(n) 一轮、总共 O(n²)；用最小堆是 O(log n) 一轮、总共 O(n log n)。这是 5.5 节那个数据结构 在这里换来的东西。

衡量一棵 Huffman 树好不好，用**带权路径长度** (WPL)：每个叶子的权重乘以它的深度再求和。以本章示例程序（5.5 节的 `demo.cpp`）用的权 2、3、4、7 为例，合并过程是 2+3=5 → 4+5=9 → 7+9=16，于是 2 和 3 落在第 3 层、4 在第 2 层、7 在第 1 层：

$$\mathrm{WPL}=2\times3+3\times3+4\times2+7\times1=30$$

Huffman 树的意义就是让这个数最小——权大的离根近，编码就短。教学版的测试把 30 这个数字直接写成断言：合并时若取的不是最小的两个，它立刻变红。

如果把 K 叉树定义为结点的有限集合——或者为空集，或者由一个根和 K 棵有序且不相交的 K 叉树组成——Huffman 算法可以推广到 K 叉树：每次取 K 个最小的合并。要注意一个细节：每合并一次，树的棵数减少 $K-1$，最后要恰好剩 1 棵，就要求 $(n-1)$ 能被 $K-1$ 整除；不能整除时先补上若干个权为 0 的虚叶子凑齐，否则最后一次合并凑不满 K 棵，根下留着空位，WPL 就不是最小（习题 21 用得上）。

教学版有一处值得单独说的写法：构造函数**先把全部权重检查一遍，再动手** `new`。顺序反过来的话，在第 k 个权重上发现负数时前 k − 1 个结点已经建好了，一抛就全漏。这不是纸上推演——本书作者第一版正是先检查边建，LeakSanitizer 当场把它报了出来：

```
==2738715==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 24 byte(s) in 1 object(s) allocated from:
    #1 HuffmanTree::HuffmanTree(int const*, unsigned long) teaching.hpp:180
```

权值成批给出：只求 WPL

另一类常见的上机题把输入换成「权 w_i 出现了 c_i 次」，c_i 可以到 10^9，只问最小的带权路径长度。结点根本建不出来，要换两个想法。

第一，WPL 等于所有内部结点的权之和。每个内部结点的权是它下面全部叶子的权之和，所以一个深度为 d 的叶子，它的权在它的 d 个祖先里各被算了一次，合起来恰好是「权 × 深度」。仍以示例程序的权 2、3、4、7 为例，三次合并得到的内部结点权是 5、9、16，$5+9+16=30$，与上面按深度算的一样。于是不必知道树的形状，只要把每次合并出的新权累加。

第二，同权的树可以整批合并。若当前最小的权是 w、有 c 棵，前 $\lfloor c/2 \rfloor$ 次合并取的一定都是两棵 w（合并出的 $2w$ 不比 w 小），一步就得到 $\lfloor c/2 \rfloor$ 棵权为 $2w$ 的树，WPL 加上 $2w\lfloor c/2 \rfloor$。c 为奇数时**会余下一棵** w，它要留在堆里，和下一小的树合并——这正是最容易漏掉的一步，漏了结果就偏小。堆里放的是「(权, 棵数)」这样的组，取出一组时顺手把堆顶同权的组并进来。10^9 棵等权叶子每整批合并一次棵数就减半，约 $\log_2 10^9 \approx 30$ 次减半就到底；不过奇数时余下的那一棵会和别的组拼出新的权值，派生出新的组，所以循环次数比 30 多一些——照下面的代码逐行复算，单组 10^9 棵权为 1 的叶子共循环 50 次。

```cpp
/// 一组同权叶子：weight 这个权出现了 count 次。
struct WeightGroup {
    std::uint64_t weight;
    std::uint64_t count;
};

/// 只求 Huffman 树的带权路径长度，不建树——叶子数可达 1e9 量级，结点根本放不下。
///
/// 依据：WPL 等于 ** 所有内部结点的权之和 **（每个叶子的权在它的每个祖先里各被加一次，
/// 祖先个数正是它的深度）。所以只要把每次合并出的新权累加起来。
/// 又因为同权的若干棵树谁先合并都一样，最小的一组有 c 棵时，一次就合并出 c/2 棵
/// 权为 2w 的树；c 为奇数时剩下的那一棵要和 ** 下一小 ** 的树合并，不能丢。
/// 堆里放的是「组」，一轮处理一个最小权的组。
///
/// 轮数：** 只证明了平凡上界 **——每轮至少造出一个内部结点，所以至多 n − 1 轮（n = 叶子总
↪ 数）；
/// n 可达 1e9，这个界没有实用价值。更紧的界 ** 没有证明 **。实测（T-075，Python 逐行移植计
↪ 数）：
/// 6 个组、count 取 1/2/3/≈ 1e9 的对抗混合最坏 352 轮，约「每组 log2n 轮」；
/// 单组 1e9 叶子 50 轮、1e18 叶子 101 轮。早先写在这里的 O(k log k + log n) 被这组实测
↪ ** 否定 ** 了，已撤掉。
```

```cpp
///
/// 溢出界：设总权 W = Σ weight·count、叶子数 n = Σ count，则 WPL ≤ W·⌈log2 n⌉。
/// 该乘积小于 2^64 时一定不溢出；否则中途任何一次加法/乘法溢出都抛 std::overflow_error，
/// 不返回回绕后的错数。count 为 0 的组忽略；叶子数 ≤ 1 时 WPL 为 0。
inline std::uint64_t huffman_wpl_batched(const WeightGroup* groups, std::size_t
↪  group_count) {
    if (group_count != 0 && groups == nullptr) {
        throw std::invalid_argument("huffman_wpl_batched: non-empty input requires
        ↪  groups");
    }
    struct ByWeight {
        std::uint64_t weight, count;
        bool operator<(const ByWeight& other) const noexcept { return weight <
        ↪  other.weight; }
    };
    const auto add = [](std::uint64_t a, std::uint64_t b) {
        if (a > std::numeric_limits<std::uint64_t>::max() - b) throw
        ↪  std::overflow_error("Huffman WPL overflows uint64");
        return a + b;
    };
    const auto mul = [](std::uint64_t a, std::uint64_t b) {
        if (b != 0 && a > std::numeric_limits<std::uint64_t>::max() / b) throw
        ↪  std::overflow_error("Huffman WPL overflows uint64");
        return a * b;
    };

    MinHeap<ByWeight> heap;
    for (std::size_t i = 0; i < group_count; ++i) {
        if (groups[i].count != 0) heap.insert(ByWeight{groups[i].weight,
        ↪  groups[i].count});
    }
    std::uint64_t wpl = 0;
    while (auto smallest = heap.remove_min()) {
        ByWeight group = *smallest;
        while (heap.peek() != nullptr && heap.peek()->weight == group.weight) {   //
        ↪  同权的组并成一组
            group.count = add(group.count, heap.remove_min()->count);
        }
        if (group.count == 1) {
            auto next = heap.remove_min();
            if (!next) break;                                  // 只剩一棵树：它就是根
            const std::uint64_t merged = add(group.weight, next->weight);
            wpl = add(wpl, merged);
            heap.insert(ByWeight{merged, 1});
           if (next->count > 1) heap.insert(ByWeight{next->weight, next->count - 1});
        } else {
            const std::uint64_t pairs = group.count / 2;
```

```
            const std::uint64_t merged = mul(group.weight, 2);
            wpl = add(wpl, mul(merged, pairs));           // pairs 个内部结点，各权 2w
            if (group.count % 2 == 1) heap.insert(ByWeight{group.weight, 1});   // 奇
             ↪ 数余下的一棵
            heap.insert(ByWeight{merged, pairs});
        }
    }
    return wpl;
}
```

结果用 64 位无符号数。设总权 $W=\sum w_i c_i$、叶子总数 $n=\sum c_i$，因为等权时尽量满的树深度不超过 $\lceil \log_2 n \rceil$，最优的 WPL 不超过 $W\lceil \log_2 n \rceil$；这个数小于 2^{64} 就一定不溢出。超出时，中途的每一次加法和乘法都先检查，溢出就抛 `std::overflow_error`，不返回回绕后的错数。测试把几百组随机的小输入逐个展开成单个叶子，交给 5.5 那份教学版 `HuffmanTree::weighted_path_length` 真建树核对；10^9 棵权为 1 的叶子则与闭式公式 $n\lfloor \log_2 n \rfloor + 2(n - 2^{\lfloor \log_2 n \rfloor})$ 核对（n 片等权叶子的最优树把叶子放在相邻两层，取 $k=\lfloor \log_2 n \rfloor$，每片叶子至少深 k，再有 $2(n-2^k)$ 片要深一层——它们两两挂在 $n-2^k$ 个第 k 层结点下）。

5.6.2 Huffman 编码

编码与译码

建出树只是半程。Huffman 树的**用途**是给字符编码，这一节把它走完。

问题从哪来。要传电文 `abbaaadc`。定长编码给每个字符 2 位（a=00, b=01, c=10, d=11），总共 16 位。想更短，就得让**出现次数多的字符用更短的码**。比如 a=0, b=1, c=01, d=10，电文变成 10 位——**但这样的电文没法译码**：收到 `011`，既可以读成 a b b，也可以读成 c b。

症结是「前缀」。`0` 是 `01` 的前缀，所以读到 `0` 时无法判断该停还是该再读一位。解决办法是要求任何字符的编码都不是另一个字符编码的前缀——这种编码叫**前缀码**。

Huffman **树天然给出前缀码**。规则：从每个结点引向**左**孩子的边标 0、引向**右**孩子的边标 1，从根走到某个叶子，一路的 0/1 连起来就是那个字符的编码。为什么一定无前缀冲突？因为**字符只住在叶子上**，而一个叶子不可能出现在另一个叶子的路径上。

原书的编码示例是这样一棵树：

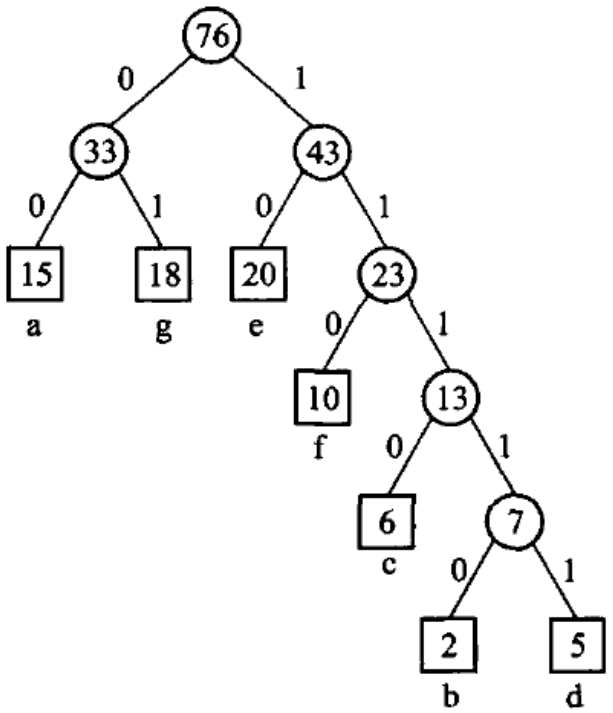


图 5.20　Huffman 编码示例：左边的边标 0、右边的边标 1，从根到叶一路读下来就是那个字符的编码。字符只住在叶子上，所以没有一个编码会是另一个的前缀。

这个例子的字符是 a、b、c、d、e、f、g，出现频率分别为 15、2、6、5、20、10、18。合并次序是 2+5=7 → 6+7=13 → 10+13=23 → 15+18=33 → 20+23=43 → 33+43=76，把每条到左子树的边标 0、到右子树的边标 1，得到的各字符编码为（教学版 `HuffmanTree::code_of` 对这组权值给出的正是这一组）：

字符	a	b	c	d	e	f	g
频率	15	2	6	5	20	10	18
编码	`00`	`11110`	`1110`	`11111`	`10`	`110`	`01`

频率最高的 e、g、a 只要 2 位，最低的 b、d 要 5 位。电文总长 $15\times2+2\times5+6\times4+5\times5+20\times2+10\times3+18\times2=195$ 位，也等于内部结点权之和 $7+13+23+33+43+76=195$；7 个字符若用定长编码，每个要 3 位，共 $76\times3=228$ 位。有了码表，编码就是查表替换；译码则从根出发逐位匹配边上的 0/1，走到叶子译出一个字符再回到根。

以 `abbaaadc` 为例，四个字符的频率是 a×4、b×2、c×1、d×1，建出的 Huffman 树给出

字符	频率	编码	位数
a	4	`0`	1
b	2	`10`	2
c	1	`110`	3
d	1	`111`	3

电文编成 14 位，比定长的 16 位短。**注意 14 正好等于这棵树的 WPL**——这不是巧合：WPL 是 $\sum$(权 × 深度)，而权就是出现次数、深度就是码长，两者算的是同一个和。5.6 开头说「Huffman 树让 WPL 最小」，翻译过来就是 **它让这段电文的编码总长最短**。

译码用的是同一棵树：从根出发，读 0 走左、读 1 走右，走到叶子就吐出一个字符，然后回到根接着读。前缀码保证这个过程不会有歧义。

教学版实现了 `code_of` / `encode` / `decode` 三个函数，就在 5.5 那份清单里。三处边界值得一提，测试各有一条用例守着：

- **比特串走到一半就没了**（比如传丢了最后一位）→ 返回空 `optional`，而不是吐出半个字符；
- **出现既不是 0 也不是 1 的字符** → 同样返回空 `optional`；
- **整棵树只有一个字符**：从根到叶的路径是空串，那样的编码根本没法传。本书约定给它一位 `"0"`。这是真实的边界，不是抠字眼。

还有一个反直觉的结论：**如果所有字符频率相同，Huffman 一位都省不下来**。8 个等权字符建出的是一棵平衡树，每个字符恰好 3 位——退化成定长编码。压缩的收益完全来自频率的不均匀。教学版的测试把这条也写成了断言。

（把这套编码器扩展成能压缩文件，需要再加两步：把码表本身也写进输出，以及处理最后不足一字节的那几位。原书的上机题正是这么布置的。）

5.6a 进阶（选读）：从教学版到工程版

这一节可以整节跳过。工程版在 `code/ch05/binary_tree/modern.hpp` 与 `code/ch05/heap_huffman/modern.hpp`，与教学版的差别有四处，第一处是本章特有的。

一、递归的深度限制，以及怎么绕开。教学版的 `destroy` 与 `clone` 都是递归的——形状和树的定义一样，好读。代价是**递归深度等于树高**：一棵退化成链的树（比如按升序往 BST 里插入）会把运行栈压穿。本机实测（Linux/gcc 13.3/8MB 栈）：纯左链 50 万结点，递归 `clone` 即段错误；100 万结点，递归 `destroy` 段错误。

工程版把两者都改成迭代。`destroy` 用的是一个漂亮的办法——**右旋到没有左孩子，再沿右链删**：每次旋转把左子树提上来，树被逐步拉直成一条右链，然后一个一个删。总代价仍是 O(n)，额外空间 O(1)，而且不分配内存，所以能保持 `noexcept`：

```
/// 释放整棵树。** 迭代实现 **，栈深度恒定。
///
/// 递归版 `destroy(left); destroy(right); delete node;` 在退化成链的树上会压穿栈——
/// 实测纯左链 100 万结点即段错误（`collab/UNVERIFIED-RISKS.md` 有复现方法）。
/// 这里用「右旋到没有左孩子，再沿右链删」的经典办法：每次旋转把左子树提上来，
/// 树被逐步拉直成一条右链，然后一个一个删。总代价仍是 O(n)，额外空间 O(1)，
/// 而且不分配内存，所以能保持 noexcept。
static void destroy(Node* node) noexcept {
    while (node != nullptr) {
        if (node->left != nullptr) {
            Node* const left = node->left;   // 右旋：左孩子成为新的根
            node->left = left->right;
            left->right = node;
            node = left;
        } else {
            Node* const right = node->right;
            delete node;
            node = right;
        }
    }
}
```

clone 则用一把**显式栈**代替调用栈——结点放在堆上，深度不再受线程栈限制；中途抛异常时回收已建好的部分，保持强异常保证。用的正是本章自己那把手写链式栈。

注意这条与 D-001 §3d **不冲突**：**周游**的递归版仍是主教学实现（递归结构正是要教的），被改成迭代的只有**释放与深拷贝**这两件与教学无关的杂务。原书【算法 5.4】–【算法 5.6】的非递归周游作为补充保留在工程版里。

二、MinHeap 对元素类型的编译期约束。工程版类头有一条 static_assert，要求 T 的移动构造与移动赋值都 noexcept—— 扩容搬迁靠的就是它们，判据与 3.1.2a 说过的是同一条。

三、Huffman 构造的两处防御。工程版在合并前检查 left->weight + right->weight 会不会溢出 int，并用层层 try/catch 保证「中途任何一步失败，已经取出来的裸结点都被回收」。教学版只做了参数预检查，这两处留给进阶。

四、其余与前几章相同：五法则、[[nodiscard]]、noexcept、copy-and-swap。

本章小结

本章介绍了二叉树的概念。与线性表、栈、队列一样，二叉树也是一种重要的数据结构，而且应用十分广泛。

在二叉树中，每个结点最多有两个子结点。由二叉树的逻辑结构给出了二叉树的递归定义：二叉树若不 为空，则由根结点和两棵互不相交的子树构成，其左右子树也是二叉树。随后给出了几种特殊的二叉树 结构，例如满二叉树、完全二

叉树和扩充二叉树。

二叉树周游是指按照一定顺序依次访问树中的所有结点，且使得每一个结点仅被访问一次。根据访问 结点和左右子树的次序不同，周游算法可以分为深度优先周游和广度优先周游；深度优先周游算法有前序、中序和后序 3 种，本章给出了它们的递归和非递归实现。

二叉树抽象数据类型的各个成员函数的实现，需要参考具体的存储结构和应用。**二叉树有两种主要的实现 方式：链式存储结构和顺序存储结构**。用指针实现二叉树是常用的方法，其中每个结点包含数据域和左、右指针域，分别用来存储结点本身的信息和指向左、右子结点的指针，这种存储方式称为**二叉链表**；顺序存储方式主要介绍了用数组实现完全二叉树，即一棵具有 n 个结点的完全二叉树按层次顺序把所有 结点从 0 到 $n-1$ 编号，按编号的次序将所有结点元素存储在一维数组中。

二叉搜索树（BST）是一种重要的索引结构，可以用于快速检索。 BST 每个结点关键码的值大于其左子树 上所有结点的关键码值，而小于其右子树上所有结点的值，且其左右子树分别为一棵二叉搜索树；因此，**按中序周游二叉搜索树可以得到关键码的正序序列**。在二叉搜索树中进行检索可以得到较高的效率，动态插入和删除结点时需要保持二叉搜索树的性质。

堆是一种特殊的完全二叉树，常用于实现优先队列。其特殊性主要体现为局部有序性，即最小堆对应的 完全二叉树中所有内部结点的值均不大于其左右子结点的关键码值，而一个结点与其兄弟之间没有必然的 联系。堆的插入和删除操作必须保持最小堆的堆序性质不变，因此在堆中插入和删除一个元素时需要调整父子结点的位置关系；**建堆过程是一个从下至上不断筛选的过程**。

结点的带权路径长度是指从根结点到该结点的路径长度与结点权值的乘积。Huffman 树采用贪心法 构造带权外部路径长度最小的二叉树，在信息编码中有广泛的应用。

习题

补充证明题（参考课程第 5 章）

1. 判断 “先序和后序遍历序列总能唯一确定二叉树” 是否成立；若不成立给出最小反例。
2. 设计 `O(n)` 算法检查数组是否为大顶堆，并说明为什么只需检查每个非根结点与父结点的关系。
3. 证明含 `n` 个结点的满二叉 Huffman 树叶子数为 `(n+1)/2`。

以下为本书正文习题：

1. 分别画出含 3 个结点的二叉树的所有不同形态。
2. 一棵二叉树的先序是 ABCDEFG、中序是 CBDAFEG，画出这棵树并写出后序。
3. 证明：有 n 个结点的二叉树，空指针域的个数是 $n+1$。
4. 完全二叉树按层编号，写出结点 i 的父、左孩子、右孩子公式，并证明。
5. 写出层次周游的算法，并说明为什么需要队列。

6. 在二叉搜索树中插入序列 50, 30, 70, 20, 40, 60, 80，画出结果；再删除 50，画出删除后的树。
7. 对权 2, 3, 4, 7, 8 构造一棵 Huffman 树，计算带权路径长度，并给出一组前缀编码。

原书习题

以下是原书第 5 章的习题,本轮按扫描件补回题面;**参考答案见** `book/习题与参考答案.md` **的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。

1. 对于 3 个结点 A、B、C，有多少棵不同的二叉树？试将其画出来。
2. 分别按前序、后序、中序列出图 5.21 所示二叉树的结点。

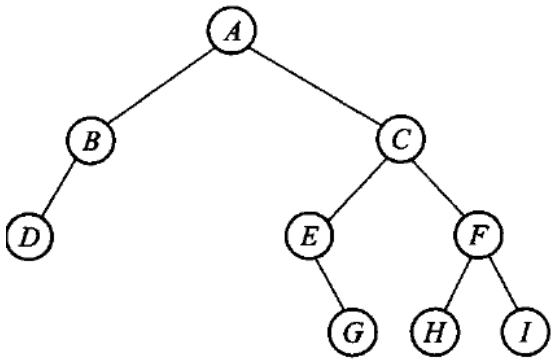


图 5.21　习题 2 的图例

3. 以下命题是否为真？若真请证明之：**一棵二叉树的所有终端结点（叶结点），在前序序列、中序序列 以及后序序列中都按相同的相对位置出现**。
4. 找出所有这样的二叉树，其结点在下列两种次序之下恰好都以同样的顺序出现：(1) 前序和中序；
   (2) 前序和后序；(3) 中序和后序。
5. 写一个递归函数计算二叉树的叶结点个数。
6. 写一个递归函数计算二叉树的高度（只有一个根结点的二叉树高为 1）。
7. 设计一个镜面映射算法，将一棵二叉树的每个结点的左、右子结点交换位置。
8. 给定结点类型为 `BinaryTreeNode` 的 3 个指针 p、q、rt，假设 rt 为根结点，求距离结点 p 和结点 q 最近的这两个结点的**共同祖先结点**。
9. 画出图 5.22(a) 所示二叉树的二叉链表存储表示图。

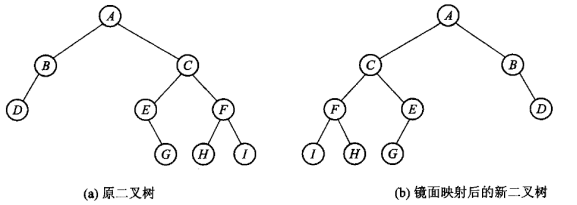


图 5.22　习题 7 的图例：(a) 原二叉树，(b) 镜面映射后的新二叉树

10. 对于 3 个关键码值 A、B、C，有多少个不同的二叉搜索树？试将其画出来。

11. 试证明：二叉搜索树结点的中序序列就是二叉搜索树结点按关键码值排序的序列。
12. 写出从二叉搜索树中删除一个关键码的递归算法。
13. 给出关键码序列 {wxw, wxz, wzw, wzy, wzz, yww, yyx, zww, zwx, zwy, zyw, zyx, zyy, zyz}。从空二叉搜索树开始，按照上述关键码出现的顺序依次插入，画出插入所有结点后的 BST。
14. 编写一个递归函数 `search()`，传入参数为一棵二叉树（不是二叉搜索树）和一个值 K，如果值 K 出现在树中则返回 `true`，否则返回 `false`。相应地，写出一个等价的非递归函数 `search()`。
15. 编写一个递归函数 `smallcount()`，传入一棵二叉搜索树的根和值 K，返回值小于或等于 K 的 结点数目。**函数应尽可能少地访问 BST 的结点**。相应地写出一个等价的非递归函数。
16. 编写一个递归函数 `printRange()`，传入一个 BST、一个较小的值和一个较大的值，按照顺序打印出 介于两个值之间的所有结点。函数应尽可能少地访问 BST 的结点。
17. 编写一个函数 `IsBST`，传入参数为一棵二叉树，如果这棵二叉树是 BST 则返回 `true`，否则返回 `false`。
18. 编写一个函数 `IsMinHeap()`，传入参数为一个数组，如果该数组中的值满足最小堆的定义则返回 `true`，否则返回 `false`。
19. 初始关键码序列为 $\{E, D, X, K, H, L, M, C, P\}$，试给出用筛选法所建的最小堆，并写出其相应的 序列。在建堆过程中，移位次数是多少？（提示：一次移位就是对关键码的一次赋值操作；例如交换 `Arr[i]` 与 `Arr[j]` 是 3 次移位。不考虑下标变量的赋值。）
20. 对于给出的一组权 $W = \{1, 4, 9, 16, 25, 36, 49, 64, 81, 100\}$，构造具有最小带权外部路径长度的 扩充二叉树，并求出它的带权外部路径长度。
21. 给定一组权 $W_0, W_1, \cdots, W_{n-1}$，说明怎样构造一个具有最小带权外部路径长度的扩充的 k 叉树。试对权集 1, 4, 9, 16, 25, 36, 49, 64, 81, 100 来具体构造一个这样的扩充三叉树。
22. 任何包括 n 个结点的二叉树的二叉链表表示中，$2n$ 个指针中都只有 $n-1$ 个用来指示结点的左右 子结点，而另外 $n+1$ 个为空。可以用指向结点按某种次序周游的前驱和后继的指针来代替这些空的 指针，这种附加的指针称做「**线索**」，加进了线索的二叉链表称做**穿线二叉树**（thread binary tree）。在穿线树中，如果结点有左子树，那么它的左指针 `left` 就指向其左孩子，否则 `left` 指向其 前驱结点；如果结点有右子树，那么它的右指针 `right` 就指向其右孩子，否则 `right` 就指向其后继 结点。为区分指针和线索，需要在每个结点中增加两个标志位，分别指示左右指针位置中存的是指针 还是线索，结点结构为 `[left | ltag | info | rtag | right]`。**试给出将二叉树以中序遍历使其变成 穿线树的线索化算法**。

上机题

1. 由先序和中序序列重建二叉树，再输出后序。
2. 实现二叉搜索树的插入、按键删除和中序输出，并用有序插入验证会退化成链。
3. 用最小堆对 n 个整数排序，并与直接插入比较运行时间。
4. 根据一组权值建 Huffman 树，输出每个权的编码。

原书上机题

同上，本轮只补题面，参考答案登记在 `collab/answer_gaps.json`。

1. **二叉树两个结点间的最小距离**。定义二叉树两个结点间的最小距离为：这两个结点的最近公共祖先 结点分别到这两个结点的路径长度之和。试设计一种方法，找出给定二叉树中任意两个结点之间的最小 距离，可以考虑以图形显示。
2. **表达式二叉树**。表达式可以用表达式二叉树来表示。对于简单的四则运算表达式，请实现以下功能：
   (1) 对于任意给出的前缀表达式（不带括号）、中缀表达式（可以带括号）或后缀表达式（不带括号），能够在计算机内部构造出一棵表达式二叉树，并且以图示显示出来（字符图或图形的形式）；
   (2) 对于构造好的内部表达式二叉树，按照用户的要求，输出相应的前缀表达式（不带括号）、中缀表达式（可以带括号，但不允许冗余括号）或后缀表达式（不带括号）。**所谓中缀表达式中的冗余括号，就是去掉 该括号不影响表达式的计算顺序**——例如 `(c + b) + a` 中的括号是冗余的，可以表示成 `c + b + a`。
3. **逻辑表达式推演**。一个逻辑表达式如果对于其变元的任一种取值都为真，则称为重言式；反之，如果 对于其变元的任一种取值都为假，则称矛盾式；否则既非重言式也非矛盾式。编写程序通过真值表判断 一个逻辑表达式属于哪一类。(1) 逻辑运算符包括 `|`、`&`、`~`，分别标识逻辑或、与、非，表达式可包含 括号；(2) 如果是既非重言式也非矛盾式，试按照用户给定变元的取值（可选）显示逻辑表达式的值；
   (3) 按照用户的要求可以打印真值表。
4. Huffman **编码**。完成根据代码 5.12 建立 Huffman 编码树的源代码，包括计算各个字母对应代码的 函数以及对信息进行编码与解码的函数。这个对象可以进一步扩展以支持对文件的压缩，为此必须增加 两个步骤：(1) 扫描整个文件以生成文件中各个字母的实际使用频率；(2) 在编码文件的开头存储 Huffman 树，以便解码函数使用。
5. **优先队列**。利用最大堆实现一个优先队列。对于队列的操作应该至少支持下列几种指令：`void enqueue(int ObjectID, int Priority)` 向优先队列中插入一个 ID 号为 `ObjectID`、优先级为 `Priority` 的新对象；`int dequeue()` 从优先队列中删除优先级最高的对象并返回该对象的 ID 号；`void changeweight(int ObjectID, int newPriority)` 将 ID 号为 `ObjectID` 的对象的优先级改为 `new-`

`Priority`。**需要建立一种机制来获取所需对象在堆中的位置**，还需要对堆的实现进行修改，以存储对象在数组中的位置，以便在辅助数组结构中记录针对堆中对象的修改。

第 6 章 树

一般树允许一个结点有任意多个孩子。本章用「左孩子、右兄弟」表示它：`child` 指向第一个孩子，`sibling` 指向下一个兄弟。并查集则回答另一类问题：若干元素分属哪些互不相交的集合。

源码：一般树和并查集、可运行示例、测试。

6.1 树的定义和基本术语

树形结构用分支关系定义层次：一个结点至多一个前驱，但可以有任意多个后继。家谱、机关编制、文件系统目录、编译器的句法树，都是树。和第 5 章的二叉树不同，一般树的结点不限制孩子个数。

6.1.1 树和森林

定义。树（tree）是包括 n（$n \geq 1$）个结点的有限集合 T，使得：

1. 有且仅有一个特定的称为**根**（root）的结点；
2. 除根以外的其他结点被分成 $m(m \geq 0)$个不相交的有限集合 $T_1, T_2, \cdots, T_m$，而每一个 集合又都是树。其中，树 $T_1, T_2, \cdots, T_m$ 称做这个根的**子树**（subtree）。

这个定义是**递归**的：在树的定义中又用到了树的概念——一棵树可分成几个分支，而每个分支也都是 一棵树。

树的逻辑结构也可以用第 1 章的二元关系来描述：树是包含 n（$n > 0$）个结点的有穷集合 K，且在 K 上定义了一个满足以下条件的二元关系 $R = \{r\}$：

1. 有且仅有一个结点 $k_0 \in K$，它对于关系 r 来说没有前驱，结点 k_0 称做树的根；
2. 除结点 k_0 外，K 中的每个结点对于关系 r 来说都有且仅有一个前驱；
3. 除结点 k_0 外的任何结点 $k \in K$，都存在一个结点序列 $k_0, k_1, \cdots, k_s$，使得 k_0 就是 树根，且 $k_s = k$，其中有序对 $\langle k_{i-1}, k_i \rangle \in r$ $(1 \leq i \leq s)$ ——这样的结点序列 称为从根 k_0 到结点 k 的一条**路径**。

除了定义的角度不同外，以上给出的逻辑结构描述和树的递归定义是等价的。以下面的图 6.1 为例，抽象出来的逻辑结构是结点集合 $K = \{A, B, C, D, E, F, G, H, I, J, K, L\}$,$K$ 上的关系 $r = \{\langle A, B \rangle, \langle A, C \rangle, \langle B, D \rangle, \langle B,$

基本术语（父结点、子结点、根、树叶、子孙、路径、层次等）都类似于二叉树中的相关概念：在一棵 树中，若存在结点 k 指向结点 k' 的连线，则称 k 是 k' 的父结点，而 k' 则是 k 的子结点，有向连线 $\langle k, k' \rangle$ 称做边；同一个父结点的子结点之间互称兄弟；树中没有父结点的结点 称为根，没有子结点的结点称为树叶。**结点的子树数目称为结点的度，树的度是树中各结点度的最大 值**（二叉树的度是 2）。若有一条由 k 到达 k_s 的路径，则称 k 是 k_s 的祖先、k_s 是 k 的子孙。结点的层数同样由树根开始定义：根结点为第 0 层，非根结点的层数是其父结点的层数加 1。

自然界中树的子结点次序没有必要加以区别，称为**无序树**；但计算机的存储是有序的，为方便计算机 处理，往往把子结点按从左到右的次序顺序编号，即把树作为**有序树**（ordered tree）看待。

注意：度为 2 的有序树并不是二叉树。因为有序树中在第一子结点被删除后，第二子结点自然顶替 成为第一子结点；**因此，度为 2 并且严格区分左右两个子结点的有序树才是二叉树**——二叉树必须能 表示「左空、右不空」这种左右不对称。

森林（forest）是零棵或多棵不相交的树的集合（通常是有序集合）。对于树中的每个结点，其子树 组成的集合就是森林；而加入一个结点作为根，森林就可以转化成一棵树。

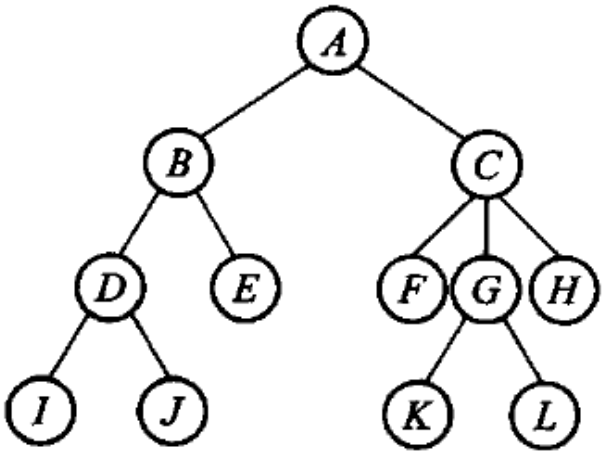


图 6.1　一般树的树形表示。

树形结构在客观世界中大量存在，有多种逻辑表示方法。**树形表示法**其实是根在上的倒挂树：用圆圈 结点表示树的数据元素，用连接两个结点的边表示数据元素的关系；尽管树中的边没有标明方向，但是 一般默认上面的结点表示前驱、下面的数据元素表示该结点的后继。**凹入表示法**表示出根结点及各个 子树的层次关系，线条表示各结点：树根结点对应的线条最长，各子结点对应的线条短于其父结点并置 于父结点线条的下方，各兄弟结点对应的线条长度相同——这种表示法类似于图书的目录表。**文氏图 表示法**用嵌套的圆表示树：每棵树对应于一个圆，树的根结点及其子树画在同一个圆内，同一个根结点 的任意两棵子树对应的圆互不相交。**嵌套括号表示法**是类似于广义表的一种表示方法：括号表示层次 关系，每棵树对应于一个以根结点作为表名的表，根结点写在左边，表由子树森林对应的表组成，各个 子树对应的表用逗号隔开，例如 `A(B(D(I,J),E),C(F,G(K,L),H))`。表示法多样，说明树在建模里用得广。

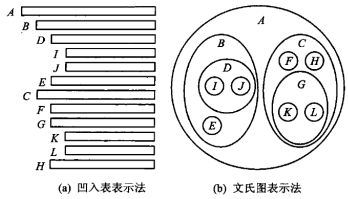


图 6.2　同一棵树的三种等价表示。凹入表适合按层次阅读，文氏图突出包含关系，嵌套括号则便于作为文本编码或输入。

例如 A 的孩子是 B、C、D，B 的孩子是 E、F。用左孩子 / 右兄弟画出来：

```
A
└ child → B ── sibling → C ── sibling → D
          └ child → E ── sibling → F
```

任意度的树只需两个指针域。代价是不能 $O(1)$ 取得「第 k 个孩子」，必须沿兄弟链走过去。

6.1.2 森林与二叉树的等价转换

树和森林都可以一对一地变成二叉树，相关操作也就都能转到二叉树上做。形象的做法是「连线、切线、旋转」三步：

1. **连线**：把兄弟结点用线连起来。
2. **切线**：只保留父结点到第一个孩子的连线，砍掉到其余孩子的连线。
3. **旋转**：以根为轴顺时针转一下，画面才像通常的二叉树。

转换后，一个结点的左孩子是它在原树（或森林）里的第一个孩子，右孩子是原来的下一个兄弟。左枝上是父子关系，右枝上是兄弟关系。单棵树的根没有兄弟，所以转成二叉树后根的右孩子一定为空。

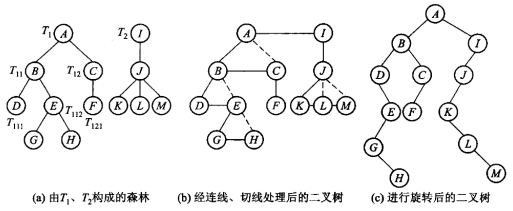


图 6.3　森林转换为二叉树的过程：先把兄弟连线，再切去多余的父子线，最后旋转整理方向。

形式地说：森林 $F = \{T_1, T_2, \dots, T_n\}$ 转成二叉树 $B(F)$——F 空则 $B(F)$ 空；否则 $B(F)$ 的根是 T_1 的根，$B(F)$ 的左子树是 T_1 的子树森林转成的二叉树，$B(F)$ 的右子树是 $\{T_2, \dots, T_n\}$ 转成的二叉树。

反过来是三步的逆：逆时针旋转；若 x 是 y 的左孩子，就把 x 以及 x 右侧整条右链上的结点都补连到 y；再删掉所有到右孩子的边。二叉树 B 对应的森林 $F(B)$：空树对应空森林；否则 $F(B)$ 是一棵以 B 的根为根、以 $F(B_L)$ 为子树森林的树，再加上 $F(B_R)$。这两种转换互逆。

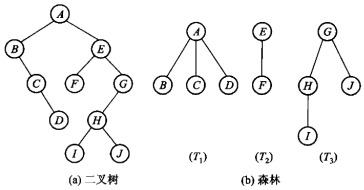


图 6.4　上述转换的逆过程：(a) 二叉树；(b) 它对应的森林。

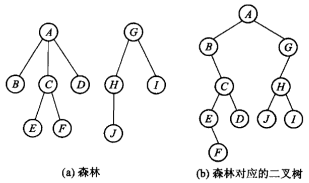


图 6.5　森林与对应二叉树的互相转换：(a) 森林；(b) 森林对应的二叉树。

6.1.3 树的抽象数据类型

和二叉树一样，树的 ADT 分结点类和树类。结点保存自身值，以及指向最左孩子、下一个兄弟和父结点的链接；树类保存根（森林时根还可以沿兄弟链相连）。对外运算包括：用一个值建立根；在某结点下插入第一个孩子；在某结点旁插入下一个兄弟；查询父结点；删除一棵子树；以及先根、后根、层次三种周游。插入和删除必须同时维护父链、孩子链和兄弟链，不能只改其中一条。原书【代码 6.1】【代码 6.2】只给出声明，本书把这些运算直接写在 `GeneralTree` 上，不再另设空基类。

6.1.4 树的周游

由树和森林的定义可以引出两种周游树或森林的方法：既可以按深度的方向周游，也可以按广度的方向 周游。**对于树或森林，一个结点可能具有多于两个子树，因此不能像二叉树的中序周游法那样给出树的 中根次序周游方式**，但是仍然可以考虑树或森林的前序和后序周游算法——仿照周游二叉树深度优先 算法的前序法和后序法，可以类似地定义**先根次序**和**后根次序**周游。

先根次序周游森林：① 访问森林中第一棵树的根结点；② 在先根次序下周游第一棵树根结点的子树 森林；③ 在先根次序下周游其他的树构成的森林。后根次序周游森林：① 在后根次序下周游第一棵树 根结点的子树森林；② 访问森林中第一棵树的根结点；③ 在后根次序下周游其他树构成的森林。

这两种次序与 6.1.2 节的等价转换正好对得上，值得单独记住：

- 按**先根次序**周游森林的序列，正好等同于其对应二叉树的**前序**序列；
- 按**后根次序**周游森林的序列，正好是该森林对应二叉树在**中序**次序周游下的结点序列。

原因就在转换关系里：森林 F 的第一棵树 T_1 的诸子树 $T_{11}, T_{12}, \cdots, T_{1m}$ 对应 $B(F)$ 的 左子树，而 F 中其余的树 $T_2, \cdots, T_n$ 对应于 $B(F)$ 的右子树；森林的后根次序周游过程是 先周游 $\{T_{11}, \cdots, T_{1m}\}$、再访问 T_1 的根结点、最后

周游 $\{T_2, \cdots, T_n\}$——这恰好是 二叉树 $B(F)$ 的中序周游过程。以图 6.5 那个森林为例：先根次序得到 `A B C E F D G H J I`，后根次序得到 `B E F C D A J H I G`。

广度优先（breadth-first）周游也称「宽度优先周游」或「层次周游」：从树的第 0 层（根结点）开始，自上至下逐层周游；在同一层中，则按照从左到右的顺序对结点逐一访问。同样以图 6.5(a) 的森林为例，按广度方向周游得到 `A G B C D H I E F J`。**注意森林的层次序列与其对应二叉树的层次次序不同**：按广度方向周游森林对应的二叉树时，不是简单地按照离二叉树根从近到远地遍历，而是**沿着二叉树 右链访问的过程，左指针起到承上启下的作用**。

对 6.1.1 节末尾那棵小树（A 的孩子是 B、C、D，B 的孩子是 E、F；6.2.4 节的 `demo.cpp` 建的就是它）：

周游	顺序	本例
先根	结点，再孩子子树	A B E F C D
后根	孩子子树，再结点	E F B C D A
层次	按离根的距离	A B C D E F

递归周游和递归销毁在极深的退化树上会耗尽调用栈，和第 5 章是同一类风险。

6.2 树的链式存储结构

在计算机中，树有多种存储方式：一类是链式结构，另一类是顺序结构（6.3 节）。但是，**无论在应用中 采取何种方式，都要求树的存储结构不但能存储各结点本身的数据信息，还要求能准确反映树中各结点 之间的逻辑关系**。一般树的存储比二叉树麻烦，因为度不固定；本节介绍四种链式存储方式，本章的 主实现是第四种。

6.2.1 “子结点表” 表示方法

「子结点表」（list of children）表示方法，是指每个分支结点的子结点按照从左至右的顺序形成一个 链表存储在该分支结点中，其**主体是一个存储了树中各结点信息的数组**。数组中的每个元素包括 3 个域，分别用来存放结点信息的值、其父结点指针以及指向其子结点表的指针——为了简明，这些「指针」实际上使用数组的下标值；最后一列存储的子结点链表中，子结点的顺序由左至右，且每个表项 均存储着指向下一个子结点的指针。

在这种表示法中，**最左子结点可由子结点链表的第一个表项直接找到，顺链可以找到所有子结点**。取第 k 个孩子若表用数组可为 $O(1)$，若用链表则要沿表走 k 步。

这种表示法的代价是：度不固定时数组表需要扩容，孩子序列中间插入还要搬动后面的指针；空间上，度为 d 的结点要留 d 个指针槽，孩子很少时会浪费。

本章随后用 “左子/右兄” 把每个结点的链接数固定为两个。

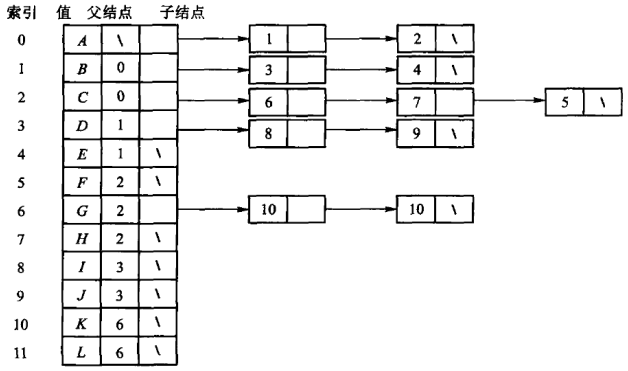


图 6.6　每个结点单独保存孩子表的表示法。

6.2.2 静态 “左子/右兄” 表示法

下面介绍上述表示方法的一个改进——静态「左子/右兄」（也称「左子结点/右兄弟结点」，left child / right sibling）表示法，**使得访问结点的右侧兄弟结点更加方便**。

图 6.7 所示为静态「左子/右兄」的一个实例，该方法仍然使用数组存储树中的各结点，数组下标代替 指针。每个结点元素包括 4 个域，分别用于存储结点的值、指向其父结点、以及指向最左子结点和右侧 兄弟结点的指针。**这种表示法比「子结点表」表示法的空间效率更高，而且每个结点的存储空间大小 固定**。

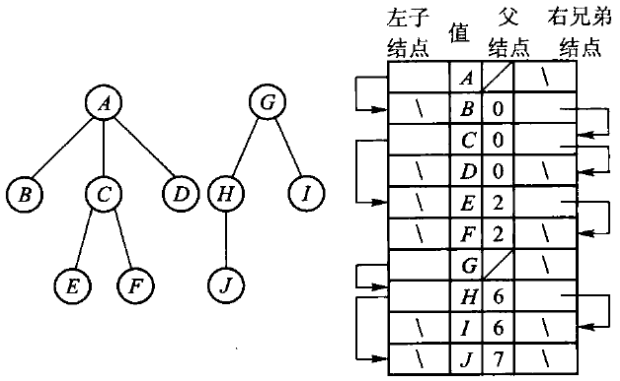


图 6.7　左孩子—右兄弟表示法把任意度的树压成两个链接域。

如果两棵树存储在同一个数组中，把其中一棵树添加为另一棵树的子树就很简单。把图 6.7 中以 A 为根的树变成 H 的最左子树，合并结果如图 6.8 所示。结点数组中只需调整 3 个链接：将 H 的最左子结点指向 A，将 A 的父结点指向 H，再将 A 的右侧兄弟指向 J。其余结点的值和链接都不变。

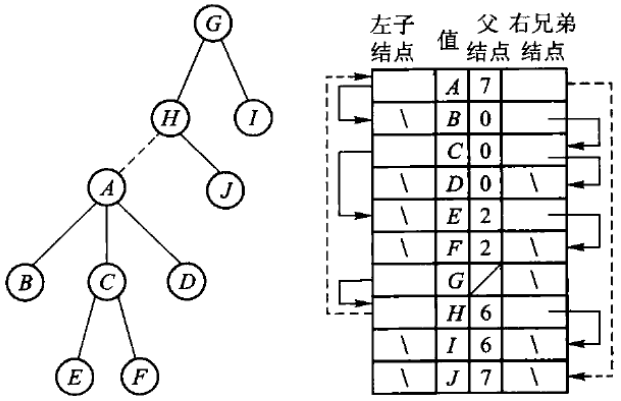


图 6.8　静态数组中的左子、右兄链接和树的归并。

6.2.3 动态表示法

问题的根源在于「度不固定」。树中的结点可以有任意的度数，并且各结点的子结点数随时可能发生 变化，这种动态性给树的实现带来了困难。当然，可以采取一种不太灵活的方式，即限定树的度数，在实现树的时候只需要给树的每一个结点分配确定数目的指针域即可；很明显，**这种存储方法对度数 小的结点会浪费存储空间，而超过限制时增加结点会非常不方便**。

另外一种实现方法是**为每个结点分配可变的存储空间**：每一个结点都存储一个子结点指针表，其子 结点的数目也存储在该结点中。使用这样的存储方式可以避免上述不足——即使子结点数目发生变化，也只需给该结点重新分配一个大小合适的存储空间即可，因此这种途径更具有灵活性。这种表示法 **本质上与「子结点表」表示法相同，但是它可以动态地分配结点空间，而不是把所有结点分配在同一个 数组中**；树的大小事先未知、会频繁长出新结点时更合适。图 6.9(a) 是树形结构，图 6.9(b) 展开了 每个结点的动态孩子表。

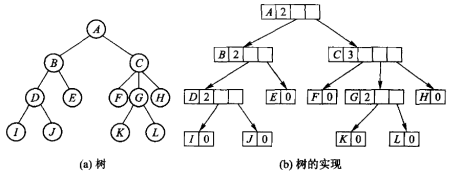


图 6.9　树的动态表示法：(a) 的每个结点在 (b) 里都是一块单独分配的空间，里面存着孩子数和一张长度可变的孩子指针表。

6.2.4 动态 “左子/右兄” 二叉链表表示法

6.1.2 节介绍了森林和树的等价转换，6.2.2 节的静态「左子/右兄」可以看成是这种方法的**静态链表 实现**。既然如此，就可以直接采用第 5 章二叉树链式存储的做法，实现**动态的「左子/右兄」二叉链表 表示法**（dynamic “left child / right sibling”，往往简称为「左子/右兄」表示法，或二叉链表表示法）：左子结点在树中是结点的最左子结点，右子结点是结点原来的右侧兄弟结点，而**根的右链就是**

森林中 每棵树的根结点。这种方法应用最广泛，也是本章的主实现。

具体地说，任意度的树只需两个指针域：`child` 指向第一个孩子，`sibling` 指向下一个兄弟。再加一个 `parent` 便于向上走。代价是取第 k 个孩子必须沿兄弟链走 k 步。

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::GeneralTree<char> tree;
    tree.create_root('A');
    auto* b = tree.insert_first(tree.root(), 'B');
    auto* c = tree.insert_next(b, 'C');
    tree.insert_next(c, 'D');
    tree.insert_first(b, 'E');
    tree.insert_next(tree.root()->child->child, 'F');

    std::cout << " 先根: ";
    tree.preorder([](char value) { std::cout << value; });
    std::cout << "\n后根: ";
    tree.postorder([](char value) { std::cout << value; });
    std::cout << "\n层次: ";
    tree.breadth_first([](char value) { std::cout << value; });
    std::cout << '\n';

    dsa::DisjointSet sets(5);
    sets.unite(0, 1);
    sets.unite(1, 2);
    sets.unite(3, 4);
    std::cout << "0 与 2 同集合: " << (sets.same(0, 2) ? " 是" : " 否") << '\n';
    std::cout << "0 与 3 同集合: " << (sets.same(0, 3) ? " 是" : " 否") << '\n';
}
```

在仓库根目录运行：

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch06/general_tree \
    code/ch06/general_tree/demo.cpp -o /tmp/tree-demo
/tmp/tree-demo
```

输出是：

```
先根: ABEFCD
后根: EFBCDA
层次: ABCDEF
0 与 2 同集合: 是
```

```
0 与 3 同集合：否
```

`insert_first(parent, value)` 把新结点插到孩子链的最前面；`insert_next(node, value)` 插到该结点的下一个兄弟。先插入 B，再 `insert_next(B, C)`、`insert_next(C, D)`，孩子从左到右就是 B、C、D。

`create_root` 清空旧树并新建根。`insert_first` 先让新结点的 `sibling` 指向原来的第一个孩子，再把它接到 `parent->child` 上——所以后插入的孩子会出现在兄弟链前端。`delete_subtree` 沿着父的孩子链或森林的根兄弟链找到指向它的指针，改写成下一个兄弟，再递归销毁。

孩子-兄弟树：

```
template <typename T>
class GeneralTree {
public:
    struct Node {
        T value;
        Node* child{nullptr};
        Node* sibling{nullptr};
        Node* parent{nullptr};

        explicit Node(const T& value) : value(value) {}
    };

    GeneralTree() = default;

    GeneralTree(const GeneralTree& other) : root_(clone(other.root_, nullptr)) {}

    GeneralTree& operator=(const GeneralTree& other) {
        if (this != &other) {
            GeneralTree copy(other);
            swap(copy);
        }
        return *this;
    }

    GeneralTree(GeneralTree&& other) noexcept : root_(other.release()) {}

    GeneralTree& operator=(GeneralTree&& other) noexcept {
        if (this != &other) {
            clear();
            root_ = other.release();
        }
        return *this;
    }
```

```
    ~GeneralTree() { clear(); }

    void swap(GeneralTree& other) noexcept {
        using std::swap;
        swap(root_, other.root_);
    }

    [[nodiscard]] Node* root() noexcept { return root_; }
    [[nodiscard]] const Node* root() const noexcept { return root_; }

    void create_root(const T& value) {
        clear();
        root_ = new Node(value);
    }

    Node* insert_first(Node* parent, const T& value) {
        if (parent == nullptr) {
            throw std::invalid_argument("parent must not be null");
        }
        Node* node = new Node(value);
        node->sibling = parent->child;
        node->parent = parent;
        parent->child = node;
        return node;
    }

    Node* insert_next(Node* node, const T& value) {
        if (node == nullptr) {
            throw std::invalid_argument("node must not be null");
        }
        Node* next = new Node(value);
        next->sibling = node->sibling;
        next->parent = node->parent;
        node->sibling = next;
        return next;
    }

    [[nodiscard]] Node* parent_of(Node* node) const noexcept {
        return node == nullptr ? nullptr : node->parent;
    }

    void delete_subtree(Node* node) {
        if (node == nullptr) {
            return;
        }

        Node** link = node->parent == nullptr ? &root_ : &node->parent->child;
```

```
        while (*link != nullptr && *link != node) {
            link = &(*link)->sibling;
        }
        if (*link != node) {
            throw std::invalid_argument("node is not part of this tree");
        }

        *link = node->sibling;
        node->sibling = nullptr;
        destroy(node);
    }

    void clear() noexcept {
        destroy(root_);
        root_ = nullptr;
    }

    template <class Visitor>
    void preorder(Visitor&& visitor) const {
        pre(root_, visitor);
    }

    template <class Visitor>
    void postorder(Visitor&& visitor) const {
        post(root_, visitor);
    }

    // >>> dual-tag
    /// 【算法 6.10】带双标记位的先根次序表示 → 「左子/右兄」链式树。
    ///
    /// 顺序表示里每个结点只带两个标志位：`has_child`（原书 ltag == 0）和
    /// `has_sibling`（原书 rtag == 0）。光靠先根次序 + 这两位就能把链恢复出来，
    /// 靠的是先根次序的一条性质：** 任何结点的子树都紧跟在它后面 **，
    /// 子树排完才轮到它的下一个兄弟。
    ///
    /// 于是「谁是某个结点的右兄弟」这件事要等它整棵子树扫完才知道——用栈记着：
    /// 扫到 `has_sibling` 的结点就压栈；扫到没有孩子的结点（子树到头了）就弹一个出来，
    /// 把刚建的结点接成它的右兄弟。
    struct DualTagNode {
        T value;
        bool has_child;    ///< 原书 ltag == 0
        bool has_sibling;  ///< 原书 rtag == 0
    };

    [[nodiscard]] static GeneralTree from_dual_tag(const DualTagNode* nodes,
    ↪  std::size_t count) {
        GeneralTree tree;
```

```
        if (count == 0) {
            return tree;
        }
        if (nodes == nullptr) {
            throw std::invalid_argument("from_dual_tag: 结点数组是空指针");
        }

        // 原书用 `stack<TreeNode<T>*> aStack`，这里用 vector 当栈（见 unit.json 豁免）。
        std::vector<Node*> waiting;  // 已扫到、还等着接右兄弟的结点
        Node* current = new Node(nodes[0].value);
        tree.root_ = current;

        for (std::size_t i = 0; i + 1 < count; ++i) {
            if (nodes[i].has_sibling) {
                waiting.push_back(current);
            }
            Node* fresh = new Node(nodes[i + 1].value);
            if (nodes[i].has_child) {
                current->child = fresh;
                fresh->parent = current;
            } else {
                // 子树到头了：刚建的结点属于栈顶那个结点的右兄弟。
                //
                // 原书这里直接 `aStack.top()`，** 没有判空 **。标志位不自洽的输入
                // （例如全是 has_child=false、has_sibling=false）会让它对空栈取顶，
                // 那是未定义行为（证据见 legacy.md 缺陷 4）。这里判空并抛异常。
                if (waiting.empty()) {
                    delete fresh;
                    throw std::invalid_argument("from_dual_tag: 标志位不自洽，右兄弟无
                    ↪  处安放");
                }
                Node* owner = waiting.back();
                waiting.pop_back();
                owner->sibling = fresh;
                fresh->parent = owner->parent;  // 兄弟与它共享同一个父结点
            }
            current = fresh;
        }
        // 先根次序里最后一个结点必是叶子，** 而且没有下一个兄弟 **——
        // 它的孩子和它的右兄弟都只能排在它后面，而它已经是最后一个了。
        // 按标记的定义，末结点必然 `ltag == 1 且 rtag == 1`。
        // 循环只走到 count-2，所以末结点的两个标志位都得在这里单独查。
        //
        // ** 不自洽就拒绝，不做「尽量还原」**：压栈（有兄弟）与出栈（子树到头）
        // 必须一一配对，配不上的序列不对应任何森林的编码。见 legacy.md 缺陷 4。
        if (nodes[count - 1].has_child || nodes[count - 1].has_sibling ||
        ↪  !waiting.empty()) {
```

```
            throw std::invalid_argument("from_dual_tag: 标志位不自洽，序列没有正常收
            ↪ 尾");
        }
        return tree;
    }
    // <<< dual-tag

    template <class Visitor>
    void breadth_first(Visitor&& visitor) const {
        std::vector<Node*> queue;
        for (Node* node = root_; node != nullptr; node = node->sibling) {
            queue.push_back(node);
        }
        for (std::size_t index = 0; index < queue.size(); ++index) {
            visitor(queue[index]->value);
            for (Node* child = queue[index]->child; child != nullptr;
                 child = child->sibling) {
                queue.push_back(child);
            }
        }
    }

private:
    // Recursive destruction and traversals preserve the textbook presentation.
    // They have a Stack Overflow Risk for a pathologically deep tree.
    static void destroy(Node* node) noexcept {
        if (node == nullptr) {
            return;
        }
        destroy(node->child);
        destroy(node->sibling);
        delete node;
    }

    static Node* clone(const Node* node, Node* parent) {
        if (node == nullptr) {
            return nullptr;
        }
        Node* copy = new Node(node->value);
        copy->parent = parent;
        try {
            copy->child = clone(node->child, copy);
            copy->sibling = clone(node->sibling, parent);
        } catch (...) {
            destroy(copy);
            throw;
        }
```

```
        return copy;
    }

    template <class Visitor>
    static void pre(Node* node, Visitor& visitor) {
        for (; node != nullptr; node = node->sibling) {
            visitor(node->value);
            pre(node->child, visitor);
        }
    }

    template <class Visitor>
    static void post(Node* node, Visitor& visitor) {
        for (; node != nullptr; node = node->sibling) {
            post(node->child, visitor);
            visitor(node->value);
        }
    }

    Node* release() noexcept {
        Node* result = root_;
        root_ = nullptr;
        return result;
    }

    Node* root_{nullptr};
};
```

6.2.5 父指针表示法和在并查集中的应用

并查集要防的是树退化成一条链。原书给了两条改进，本书都实现了。

等价类和并查集

并查集（union/find）是一种由若干互不相交子集组成的集合抽象。它的两个基本操作是：`find` 判断两个元素是否属于同一集合，`union` 把两个集合归并为一个集合。它常用来维护“已经连通”“属于同一组”这类关系。

形式地说，集合 S 上的关系 R 是等价关系，当且仅当满足：对所有 x 有 $(x,x)\in R$（自反性）；若 $(x,y)\in R$，则 $(y,x)\in R$（对称性）；若 $(x,y)\in R$ 且 $(y,z)\in R$，则 $(x,z)\in R$（传递性）。若 $(x,y)\in R$，称 x、y 等价；元素 x 的等价类是

$$[x]_R=\{y\in S:(x,y)\in R\}.$$

所有等价类互不相交，它们的并正好是 S，因此等价关系就是对 S 的一种划

分。

给定 n 个元素和若干等价偶对 (x, y)，划分过程从 n 个单元素集合开始。依次读入每个偶对，先查找 x、y 所在子集；若两个根不同，就把这两个子集合并成一个。最后剩下的非空子集就是由这些偶对确定的等价类。实现这一过程需要三种操作：为每个元素建立独立集合；找到元素所在子集的标识（根）；把两个子集归并。

结点索引	0	1	2	3	4	5	6	7	8	9	10	11
值	A	B	C	D	E	F	G	H	I	J	K	L
父结点索引	\	0	0	1	1	2	2	2	3	3	6	6

图 6.10　用每个结点的父指针表示树。

用父指针表示时，把每个子集看成一棵树，森林中的每棵树代表一个子集，树根就是该集合的标识符。`find` 沿父链找到根，`union` 只需把一棵树的根指向另一棵树的根；因此不必搬动集合中的所有元素。例如图 6.11(a)、(b) 的两棵树分别表示子集 $S_1 = \{1, 3, 5, 7\}$ 和 $S_2 = \{2, 4, 6, 8\}$，(c) 把 S_1 的根 1 指向 S_2 的根 2，就得到 $S_3 = S_1 \cup S_2$。（这两个子集一样大，挂法本可任选；本书下面的实现在并列时把值大的根挂到值小的根下，会得到 2 挂在 1 下，与图相反。）

每次合并前都要做两次查找。合并本身只改一个指针，是 $O(1)$；查找的时间则由树的高度决定。最坏情况下，合并会让 n 个结点的树退化成一条链：把图 6.12(a) 的 n 个单元素集合依次合并，若每次都让含元素多的根指向含元素少的根，经过 $n-1$ 次合并后树高为 n（按第 5 章的口径数层数），如图 6.12(b) 所示——这时对最底下的结点做一次 `find` 要走 $n-1$ 步。为了防止树退化为单链，应该让每个结点到根的距离尽可能小，可以做下面两种改进。

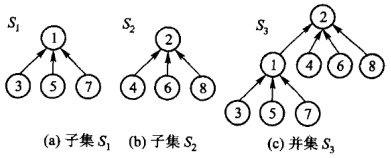


图 6.11　集合的父指针表示与并集：(a) 子集 S_1、(b) 子集 S_2 各是一棵树，(c) 合并后 S_1 的根挂到了 S_2 的根下。

第一条是【重量权衡合并规则】(weighted union rule)：合并时看两个集合的**元素个数**，「令含元素少的子集的树根指向含元素多的子集的根」。原书【代码 6.8】的结点里那个 `int nCount; //子树元素数目` 就是为它准备的。小树挂到大树下，能把整体深度限制在 $O(\log n)$，每次 `find` 也就只需 $O(\log n)$ 时间。理由要盯住**某一个结点** x 看：x 到根的距离只在它所在的树 作为**较小**（或一样大）的一方被挂到另一棵树下时才加 1；而这时 x 所在集合的元素个数至少翻倍。集合最多 n 个元素，翻倍至多 $\lfloor \log_2 n \rfloor$ 次，所以任何结点到根的距离都不超过 $\lfloor \log_2 n \rfloor$。这个上界是达得到的：2^k 个元素两两合并、再把得到的两元素集合两两合并……最后最深的结点离根恰好 k 步。

注意别和「按秩合并」混了。按秩比的是**树高**，按重量比的是**元素个**

数。两者复杂度同阶，但在同一组等价对上会长出**形状不同**的树。本书按原书口径用重量，课程第 6 章习题 8 要求「使用重量权衡合并规则与路径压缩」并画出父指针数组，换成按秩就对不上答案。并列时本书让**值大的根挂到值小的根下**——原书没规定这一条，这个口径取自那道习题的原话，好让书里的实现能直接用来核对。

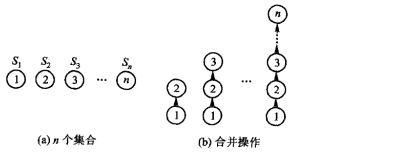


图 6.12　不采用重量权衡时，反复合并可能形成极端退化的树：(a) 起初是 n 个单元素集合，(b) 每次都把大树挂到小树下，最后退化成一条长链。

第二条是路径压缩：`find` 在返回根之前，把沿途每个结点的父指针都直接改成根。

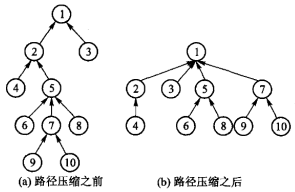


图 6.13　查找根结点时把沿途父指针直接改到根：(a) 是压缩之前的树，(b) 是对结点 7 调用一次 `find` 之后的样子——路径 $7 \to 5 \to 2 \to 1$ 上的 7 和 5 都被直接改挂到根 1 下（2 本来就是根的孩子），而 9、10 仍留在 7 下面。

图 6.13(a) 是依次合并集合 $\{7,9,10\}$、$\{5,6,8\}$、$\{2,4\}$ 和 $\{1,3\}$ 后形成的树。注意它为了造出一条够长的路径，是把先合并出的大树挂到后来集合的根下面，**并没有遵守重量权衡规则**；按重量权衡合并同样这些集合，树不会这么深。路径压缩本身与合并规则无关，所以拿它演示仍然合适。

原书【算法 6.9】用递归实现带路径压缩的查找（`FindPC`）：结点没有父结点就返回自己；否则先递归找到父结点的根，把自己的父指针改成这个根，再返回它。递归返回的途中，路径上每个结点都被改指到根。下面 `DisjointSet::find` 里标着【算法 6.9】的那一行就是这个做法。

```
/// 【代码 6.8】树的父指针表示与 union/find。
///
/// 合并用原书的 ** 重量权衡合并规则 **(weighted union rule):
/// 「令含元素少的子集的树根指向含元素多的子集的根」。原书结点里那个
/// `int nCount; //子树元素数目` 就是为它准备的，这里对应 `size_`。
///
/// ** 不要换成「按秩合并」**：按秩比的是树高，按重量比的是元素个数，两者
/// 在同一组等价对上会长出 ** 不同形状 ** 的树。原书与课程习题都按重量口径出题
/// （课程第 6 章习题 8 要求「使用重量权衡合并规则与路径压缩」并给出父指针数组），
/// 换成按秩会让读者对不上答案。
class DisjointSet {
```

```
public:
    explicit DisjointSet(std::size_t count) : parent_(count), size_(count, 1) {
        for (std::size_t index = 0; index < count; ++index) {
            parent_[index] = index;
        }
    }

    std::size_t find(std::size_t index) {
        if (index >= parent_.size()) {
            throw std::out_of_range("disjoint-set index");
        }
        if (parent_[index] != index) {
            parent_[index] = find(parent_[index]);  // 【算法 6.9】路径压缩
        }
        return parent_[index];
    }

    bool unite(std::size_t left, std::size_t right) {
        left = find(left);
        right = find(right);
        if (left == right) {
            return false;  // 已经同类：幂等的可预期失败（D-001 §3c）
        }
        // 重量权衡：小树挂到大树下。并列时把 ** 值大的根 ** 挂到值小的根下——
        // 原书没有规定并列怎么办，这个口径取自课程第 6 章习题 8 的原话
        // 「当两棵树规模同样大时，使结点值较大的根结点作为值较小的根结点的子结点」，
        // 这样书里的实现能直接用来核对那道题的答案。
        if (size_[left] < size_[right] || (size_[left] == size_[right] && left >
        ↪  right)) {
            std::swap(left, right);
        }
        parent_[right] = left;
        size_[left] += size_[right];
        return true;
    }

    [[nodiscard]] bool same(std::size_t left, std::size_t right) {
        return find(left) == find(right);
    }

    /// 某个元素所在集合的大小。原书 `nCount` 的对外读法，也让「重量」这件事可测。
    [[nodiscard]] std::size_t set_size(std::size_t index) { return
    ↪  size_[find(index)]; }

    /// 当前的父指针数组——课程习题要求画出的正是它。
    [[nodiscard]] const std::vector<std::size_t>& parents() const noexcept { return
    ↪  parent_; }
```

```
private:
    std::vector<std::size_t> parent_;
    std::vector<std::size_t> size_;  // 原书 nCount：子树元素数目
};
```

两条改进合起来有多快。路径压缩大大加速了 `find`。合并时总是小树挂到大树下、查找时再做路径压缩，可以证明（Tarjan，1975）：在 n 个元素上执行 m 次（$m \geq n$）交错的合并和查找，总时间是 $O(m\,\alpha(m,n))$；原书把它简写成「n 次 `find` 至多需要 $O(n\alpha(n))$ 时间」。其中 α 是 Ackermann 函数的反函数，增长速度比 $\log n$ 慢得多，但又不是常数；对实际能遇到的任何 n，$\alpha(n)$ 都不超过 4（原书写作「往往小于 4」，具体取值随 Ackermann 函数定义的版本略有出入）。因此，执行一串交错的合并和查找操作所需的时间，几乎与操作次数成线性关系——这是**均摊**意义上的 结论，单次 `find` 仍可能走 $O(\log n)$ 步。重量权衡加路径压缩是目前最有效的并查集实现方法。另外，由于重量权衡已把树深限制在 $O(\log n)$，上面递归版 `find` 的递归深度也不超过 $\log_2 n$，不会有第 5 章那种栈溢出的风险。

顺带数出连通分量和连通点对

并查集最常见的用法是**动态连通性**：图的边一条条加进来，每加一条就要回答「现在有几个连通分量」「现在有多少对顶点互相连通」。两个数都不必重新数，合并时顺手维护即可：

- **连通分量数**从 n 开始，每次**成功**合并减 1；两端本来就在同一集合里的冗余边不改变它。
- **连通点对数**从 0 开始。把规模为 s_1、s_2 的两个集合并成一个，新连通的无序点对恰好是 $s_1 \cdot s_2$ 对：一边任取一个、另一边任取一个。

这里最容易错的是 s_1、s_2 **从哪里读**。重量记在根上（原书的 `nCount` 只对根有意义），所以必须先 `find` 到根再读规模；拿边的端点直接去读 `size_`，读到的是某棵子树当年的旧规模。另一种错法是合并之后再读，那时两个规模已经加在了一起。下面这一层把两个规模都在合并**之前**经 `DisjointSet::set_size` 从根上读出。点对数用 64 位无符号整数：n 个元素最多 $n(n-1)/2$ 对，32 位在 n 约 9.3 万时就会溢出。

`ComponentCounter` 组合一个 `DisjointSet`，没有去改 `DisjointSet::unite`：合并规则保持上面的样子，计数是外面这一层的事。内部的并查集是私有的，调用方没办法绕过计数直接合并，两个数也就不会失真。`ComponentCounter::connect` 返回这条边新连通的点对数，冗余边返回 0；`ComponentCounter::components` 和 `ComponentCounter::connected_pairs` 都是 $O(1)$ 的查询。测试在 $n \leq 30$ 的随机边序列上每连一条边就用 BFS 暴力重数一遍，逐步对拍这两个数。第 7 章 Kruskal 算法一节的末尾有一个用到 $s_1 \cdot s_2$ 的例子。

```cpp
/// 并查集的一个常见用法：边一条条加进来，随时回答「现在有几个连通分量」
/// 「现在有多少对顶点互相连通」。两个数都在合并成功时 O(1) 维护。
///
/// 合并规模为 s1、s2 的两个集合，新连通的无序点对恰好是 s1 * s2 对——
/// ** 两个规模都要在合并之前、从根上读 **：读非根元素的 size_ 或读合并后的规模，都是错的。
/// 用 `std::uint64_t` 装点对数：n 个元素最多 n(n-1)/2 对，n 到 2^32 量级才会溢出。
///
/// 这里组合一个 `DisjointSet` 而不改动它：`unite` 保持原书的样子，计数是外面这一层的事。
/// 内部的并查集是私有的，所以不存在「绕过计数直接合并」把两个数弄脏的途径。
class ComponentCounter {
public:
    explicit ComponentCounter(std::size_t count) : sets_(count),
    ↪  component_count_(count) {}

    /// 连一条边 (left, right)，返回因此 ** 新 ** 连通的点对数；两端本已连通时返回 0。
    std::uint64_t connect(std::size_t left, std::size_t right) {
        const std::uint64_t pairs =
           static_cast<std::uint64_t>(sets_.set_size(left)) * sets_.set_size(right);
        if (!sets_.unite(left, right)) {
            return 0;  // 冗余边：分量数不变，点对数不变
        }
        --component_count_;
        pair_total_ += pairs;
        return pairs;
    }

    [[nodiscard]] std::size_t components() const noexcept { return component_count_;
    ↪  }
    [[nodiscard]] std::uint64_t connected_pairs() const noexcept { return
    ↪  pair_total_; }

private:
    DisjointSet sets_;
    std::size_t component_count_;
    std::uint64_t pair_total_{0};
};
```

6.3 树的顺序存储结构

顺序存储表示法存储树，要求把树中的结点按照一定的顺序存储到一片连续的存储单元中；为了能够 显示出树的逻辑结构，需要在结点表中包含足够的结构信息。链式存储每个结点单独分配，指针占空间，也不利于整块读写；而把结点按某一种周游次序排进数组，再用很少的附加信息记下「谁是下一个兄弟」或「有几个孩子」，就能在连续空间里还原整棵树。

下面介绍与树和森林的**遍历次序**相关的几种典型的顺序存储结构，都以图

6.5(a) 中所示的森林为例。原书给了四种，思路都对。除 6.3.2 节的【算法 6.10】（双标记先根次序 → 左子/右兄链表）有本书的实现和测试外，其余三种本章不另写一套未验证的实现，只把还原办法说清楚。

6.3.1 带右链的先根次序表示

图 6.5(a) 所示森林的先根次序序列是 `A B C E F D G H J I`。**在先根序列中，任何结点的子树的所有 结点都直接跟在该结点之后，任何一个分支结点后面紧跟的是其第一个子结点**——这条性质是下面几种 表示法共同的基础。

在「带右链的先根次序表示」中，每个结点包括结点本身数据以及附加的两个表示结构信息的域 `ltag` 和 `rlink`，结点的形式为 `[ltag | info | rlink]`。其中 `info` 是结点的数据；`rlink` 是右指针，指向结点的下一个兄弟，即树或森林所对应的二叉树中结点的右子结点；`ltag` 是一个标记位，当结点是叶结点（即对应的二叉树中没有左子结点）时 `ltag` 为 1，否则为 0。

为什么用 `ltag` 而不是再存一个左链？ 为了节省存储单元，「带右链的先根次序表示」用 `ltag` 代替 了 `llink`：从结点的次序和 `ltag` 的值完全可以推知 `llink`——`ltag` 为 0 的结点有左子结点，其 `llink` 就指向存储区中该结点的下一个结点；`ltag` 为 1 的结点没有左子结点，`llink` 为空。所以顺着数组往下走、再靠右链跳过整棵子树，就能还原所有父子和兄弟关系。

对图 6.5(a) 的森林，三行内容如下（下标从 0 起；`rlink` 写成兄弟结点的下标，`-` 表示空）：

```
下标      0  1  2  3  4  5  6  7  8  9
先根次序  A  B  C  E  F  D  G  H  J  I
ltag      0  1  0  1  1  1  0  0  1  1     0 = 有孩子
rlink     6  2  5  4  -  -  -  9  -  -     A → G、B → C、C → D、E → F、H → I
```

例如 `C` 的 `ltag` 为 0，所以它的第一个孩子是紧跟其后的 `E`；`C` 的 `rlink` 为 5，越过它的整棵子树（`E`、`F`）直接指到兄弟 `D`。

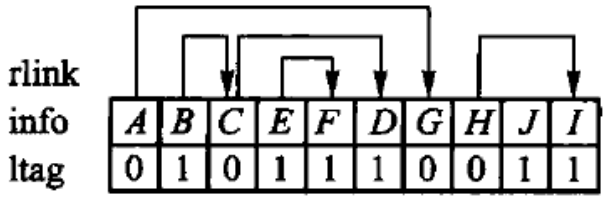


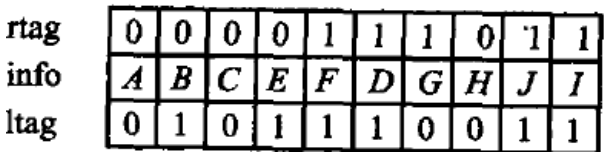
图 6.14　先根序列配合右链下标还原树形。

6.3.2 带双标记的先根次序表示

右链要用一个整型下标。若改成两个布尔标记——`ltag`「有没有孩子」、`rtag`「有没有下一个兄弟」—— 理论上只需两位信息，更省。**但普通 C++ `bool` 成员通常按字节存储，并不保证每位只占 1 bit**；只有显式位压缩或位域实现才可讨论这种

空间节省。它是早期教材常见的顺序编码，今天主要用于理解 树的序列化思想，不是通用工程接口。

以原书图 6.5(a) 那片森林为例，它的双标记先根次序表示（图 6.15）是：



rtag	0	0	0	0	1	1	1	0	1	1
info	A	B	C	E	F	D	G	H	J	I
ltag	0	1	0	1	1	1	0	0	1	1

图 6.15　双标记先根次序表示法的原书示例。

```
先根次序   A  B  C  E  F  D  G  H  J  I
ltag       0  1  0  1  1  1  0  0  1  1     0 = 有孩子
rtag       0  0  0  0  1  1  1  0  1  1     0 = 有下一个兄弟
```

> **照抄这张图很容易把两行读反**。图 6.15 的三行从上到下是 `rtag` / `info` / `ltag`—— `ltag` 在**最下面**，不在最上面。按定义核一遍就知道哪行是哪行：`B` 在图 6.5(a) 里是叶结点，它的 `ltag` 必须是 1；`B` 有兄弟 `C`，它的 `rtag` 必须是 0。两行一旦互换，还原出来的是**另一片森林**（`C` 会变成 `B` 的孩子而不是兄弟），而它的**先根序列恰好还是** `A B C E F D G H J I`——所以只对先根序列是查不出来的，得同时对后根序列（`B E F C D A J H I G`）。本书的测试两条都对。

光靠这三行就能把链恢复出来，靠的是先根次序的一条性质：任何结点的子树都紧跟在它后面，子树排完才轮到它的下一个兄弟。于是「谁是某个结点的右兄弟」要等它整棵子树扫完才知道—— 用一把栈记着：扫到 `rtag == 0` 的结点就压栈，扫到 `ltag == 1` 的结点（没有孩子，说明子树到头）就弹一个出来，把刚建的结点接成它的右兄弟。

【算法 6.10】带双标记位先根次序树构造算法。

```cpp
/// 【算法 6.10】带双标记位的先根次序表示 → 「左子/右兄」链式树。
///
/// 顺序表示里每个结点只带两个标志位：`has_child`（原书 ltag == 0）和
/// `has_sibling`（原书 rtag == 0）。光靠先根次序 + 这两位就能把链恢复出来，
/// 靠的是先根次序的一条性质：** 任何结点的子树都紧跟在它后面 **，
/// 子树排完才轮到它的下一个兄弟。
///
/// 于是「谁是某个结点的右兄弟」这件事要等它整棵子树扫完才知道——用栈记着：
/// 扫到 `has_sibling` 的结点就压栈；扫到没有孩子的结点（子树到头了）就弹一个出来，
/// 把刚建的结点接成它的右兄弟。
struct DualTagNode {
    T value;
    bool has_child;    ///< 原书 ltag == 0
    bool has_sibling;  ///< 原书 rtag == 0
};
```

```
[[nodiscard]] static GeneralTree from_dual_tag(const DualTagNode* nodes, std::size_t
↪  count) {
    GeneralTree tree;
    if (count == 0) {
        return tree;
    }
    if (nodes == nullptr) {
        throw std::invalid_argument("from_dual_tag: 结点数组是空指针");
    }

    // 原书用 `stack<TreeNode<T>*> aStack`，这里用 vector 当栈（见 unit.json 豁免）。
    std::vector<Node*> waiting;  // 已扫到、还等着接右兄弟的结点
    Node* current = new Node(nodes[0].value);
    tree.root_ = current;

    for (std::size_t i = 0; i + 1 < count; ++i) {
        if (nodes[i].has_sibling) {
            waiting.push_back(current);
        }
        Node* fresh = new Node(nodes[i + 1].value);
        if (nodes[i].has_child) {
            current->child = fresh;
            fresh->parent = current;
        } else {
            // 子树到头了：刚建的结点属于栈顶那个结点的右兄弟。
            //
            // 原书这里直接 `aStack.top()`，** 没有判空 **。标志位不自洽的输入
            //  （例如全是 has_child=false、has_sibling=false）会让它对空栈取顶，
            // 那是未定义行为（证据见 legacy.md 缺陷 4）。这里判空并抛异常。
            if (waiting.empty()) {
                delete fresh;
                throw std::invalid_argument("from_dual_tag: 标志位不自洽，右兄弟无处安
                ↪  放");
            }
            Node* owner = waiting.back();
            waiting.pop_back();
            owner->sibling = fresh;
            fresh->parent = owner->parent;  // 兄弟与它共享同一个父结点
        }
        current = fresh;
    }
    // 先根次序里最后一个结点必是叶子，** 而且没有下一个兄弟 **——
    // 它的孩子和它的右兄弟都只能排在它后面，而它已经是最后一个了。
    // 按标记的定义，末结点必然 `ltag == 1 且 rtag == 1`。
    // 循环只走到 count-2，所以末结点的两个标志位都得在这里单独查。
    //
```

```
    // ** 不自洽就拒绝，不做「尽量还原」**：压栈（有兄弟）与出栈（子树到头）
    // 必须一一配对，配不上的序列不对应任何森林的编码。见 legacy.md 缺陷 4。
    if (nodes[count - 1].has_child || nodes[count - 1].has_sibling ||
    ↪  !waiting.empty()) {
        throw std::invalid_argument("from_dual_tag: 标志位不自洽，序列没有正常收尾");
    }
    return tree;
}
```

【算法 6.10 结束】

原书这段代码有一处会崩：`ltag == 1` 分支里直接写 `pointer = aStack.top();`，**没有判空**。标志位不自洽的输入（比如两个结点都声称「没有孩子、也没有下一个兄弟」）会让它对空栈取顶——未定义行为。本书判空并抛 `std::invalid_argument`，另加一条收尾检查：**最后一个结点的两个标记都必须是 1**。测试里三种不自洽输入各有一条用例。

为什么是「拒绝」而不是「尽量还原」，值得单独说一句，因为初学者常想「能修就修」：

- 标记的定义就是 `ltag == 1` 表示无孩子、`rtag == 1` 表示无兄弟。先根序列的最后一个结点 后面已经没有结点了，孩子和右兄弟都无处安放，所以它**必然**是 `1, 1`。
- 扫描过程里，「有兄弟」入栈与「子树到头」出栈是一一配对的。对空栈出栈，说明这个序列违反了这种配对关系。

也就是说，这三类输入不是合法的边界情况，而是**不对应任何森林的编码**——多半是标记抄错了一位，或者序列被截断了。这时抛异常是在替使用者指出输入有问题；默默「还原」出一棵树，只会把错误往后传。

6.3.3 带度数的后根次序表示

图 6.5(a) 森林的后根次序序列是 `B E F C D A J H I G`。**在后根次序序列中，任何一棵子树的所有结点都聚集在一起，并且以该子树的根作为最后一个结点**。在「带度数的后根次序表示」中，每个结点有两个域：`info` 存放结点的数据，`degree` 存储结点的度数，结点形式为 `[info | degree]`。

这种表示法不包括指针，但它仍能反映树的结构，因此把它转化成森林的逻辑结构时，只需要从左至右进行扫描：度数为零的结点是叶子结点（也可以看做是一棵子树的根）；当遇到度数非零（设为 k）的结点时，则排在该结点之前且离它最近的 k 个子树的根就是该结点的 k 个子结点。例如上述森林中结点 C 的度数为 2，那么排在 C 之前的两个结点 E 和 F 就是 C 子树的根结点，从而结点 C、E 以及 F 就构成了一棵树，C 是这棵子树的根；同理，A 的度数为 3，则排在 A 之前且离 A 最近的三棵子树的根结点就是 A 的子结点，即 B、C 和 D。如此进行下去，就可以得到原来的森林。实现时同样需要用到栈：每读到一个度数为 d 的结点，就从栈顶弹出 d 棵子树做它的孩子，再把这棵新树压回去；

扫完数组，栈里剩下的就是整棵树（或森林）。

info	*B*	*E*	*F*	*C*	*D*	*A*	*J*	*H*	*I*	*G*
degree	0	0	0	2	0	3	0	1	0	2

图 6.16　结点度数随后根次序一起保存。

6.3.4 带双标记的层次次序表示

0	0	1	0	1	0	1	1	1	1
A	*G*	*B*	*C*	*D*	*H*	*I*	*E*	*F*	*J*
0	1	0	0	1	0	1	0	1	1

图 6.17　层次次序配合双标记记录孩子和兄弟关系。

图 6.5(a) 所示森林的层次次序序列为 `A G B C D H I E F J`。**这种层次次序的序列也同样表现了森林 的一些结构信息**：森林中处于同一层的结点在序列中都聚集在一起，任何一个结点只要它有右兄弟，那么在序列中它的右兄弟就排在该结点后面。

类似于带双标记的先根次序表示，引入带双标记的层次次序表示法，结点形式为 `[ltag | info | rtag]`：当结点没有左子结点时 `ltag` 为 1，否则为 0；当结点没有下一个兄弟时 `rtag` 为 1，否则为 0。

对图 6.5(a) 的森林，各结点的标记如下：

```
层次次序   A  G  B  C  D  H  I  E  F  J
ltag       0  0  1  0  1  0  1  1  1  1     0 = 有孩子
rtag       0  1  0  0  1  0  1  0  1  1     0 = 有下一个兄弟
```

> **图 6.17 的行序和图 6.15 相反**。图 6.17 没有标行名，按定义核对可知它的三行从上到下是 `ltag` / `info` / `rtag`（`B` 是叶结点，上行为 1；`G` 是最后一棵树的根、没有下一个兄弟，下行为 1），而图 6.15 是 `rtag` 在上、`ltag` 在下。两张图对照着看时别套用同一种读法。

怎么还原。由层次次序的性质可知，任何结点的子结点都排在该结点的所有兄弟结点后面，并且任何 结点的最后一个兄弟结点都不会再有下一个兄弟，即最后一个兄弟的 `rtag` 为 1。`rlink` 的值很容易 确定：结点的 `rtag` 为 1 则 `rlink` 为空，`rtag` 为 0 则 `rlink` 指向该结点紧邻的下一个结点。`llink` 要靠队列来定：顺序扫描层次序列，若结点的 `ltag` 值为 1 则置其 `llink` 为空，否则把该结点 放入队列；遇到 `rtag` 为 1 的结点时，说明这一条兄弟结点链已经扫描完毕，下一个结点就是队头结点 的最左子结点——**队列在这里起的正是「承上启下」的作用**。适合需要按层处理的外部存储。

拿上表走一遍：`A`（`ltag` 0）入队，`rtag` 0，下一个 `G` 是它的兄弟；`G` 入队，`rtag`

1——两棵树的根 排完了，下一个结点 B 是队头 A 的最左孩子，A 出队；B、C、D 依次是兄弟，其中 C 入队，D 的 rtag 为 1，于是 H 是队头 G 的最左孩子；H 入队、I 的 rtag 为 1，E 成为队头 C 的最左 孩子；F 的 rtag 为 1，J 成为队头 H 的最左孩子。扫完时恰好还原出图 6.5(a)。

现代视角：紧凑树编码。如果目标是工程中的紧凑存储或高速导航，通常会考虑 LOUDS、平衡括号（balanced parentheses）等 succinct tree 编码，而不是直接照搬本节的双标记结构。它们同样把树压成位串，但会明确规定位级布局、秩/选择（rank/select）操作和随机访问边界。本节的右链、双标记、带度数表示应视为经典编码对照，用来理解「结构信息如何随周游序列保存」即可。

6.4 K 叉树

有些应用里每个结点的孩子数有固定上限 K，例如三子棋的博弈树、某些 B 树的内存模拟。这时不必走「任意度 + 兄弟链」，可以规定每个结点至多 K 个孩子，叫做 K 叉树。

满 K 叉树、完全 K 叉树的定义与二叉树平行：满 K 叉树的每个结点要么是叶，要么恰好 K 个孩子；完全 K 叉树只有最下两层的度可以小于 K，且最下层靠左对齐。按层从 0 编号时，结点 i 的孩子们是 $Ki+1, \dots, Ki+K$，父结点是 $\lfloor (i-1)/K \rfloor$。$K=2$ 就回到二叉树。

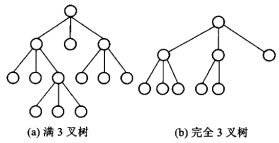


图 6.18　$K=3$ 时的 (a) 满 3 叉树与 (b) 完全 3 叉树。K 叉树的分支结点孩子数是定死的，所以链式和顺序两种存法都好实现；完全 K 叉树按编号存进数组，和完全二叉树是同一套办法。计算机图形学里的 4 叉树、8 叉树就是常见的应用。

本章主实现仍是任意度的左子 / 右兄，不单独做一份 K 叉数组。需要固定 K、又想顺序存放时，用上面的编号公式即可。

本章小结

本章介绍了树和森林的概念、术语、表示方法及重要的算法。

对于树的概念，本书给出了两种等价的定义方式：(1) 从递归来看，树可以看做是由根结点和子树构成，而每一棵子树也是树；(2) 从二元组的逻辑结构来看，树是结点的有穷集合以及在该集合上定义的一个 二元关系。在树的基本概念中，还给出了树及森林的相关概念，例如树叶、根结点、边、层数、度数、有序树 等——**这些概念基本上沿用二叉树一章中的相关术语**。

树与二叉树、森林与二叉树之间存在一一对应的关系，本章介绍了它们相互转化的方法。

树和森林的周游是十分重要的运算。按深度优先周游树和森林包括先根次序

和后根次序周游: **按先根次序周游森林等同于按前序法周游树或森林对应的二叉树，按后根次序周游森林等同于按中序法 周游对应的二叉树**。广度优先周游树，是指从树的第零层（根结点）开始，自上至下逐层周游；在同一层 中，按照从左到右的顺序对结点逐一进行访问，直到访问完最下一层的所有结点。

本章重点介绍了树的存储结构，联系到具体的应用可以选择不同的存储方式。常用的链式存储结构有「子结点表」、静态「左子/右兄」、动态表示法、「左子/右兄」二叉链表表示法和父指针表示法等:「**左子/右兄**」**二叉链表是采用得最多的表示法**；父指针表示法可以方便地实现并查集，常用于维护由 一些不相交子集构成的集合，判断两个结点是否在同一个集合中以及归并两个集合（路径压缩与重量权衡 合并让大量操作接近常数)。

顺序存储表示法把树中结点按照一定的顺序存储到一片连续的存储单元中。「**带双标记的先根次序表示**」、「**带度数的后根次序表示**」**和**「**带双标记的层次次序表示**」**都是与树/森林遍历次序相关的顺序存储结构**，根据存储方法的性质加入了必要的附加信息。

在本章的最后还介绍了 K 叉树的概念及性质: K 叉树是由一个根结点及 K 棵互不相交的 K 叉树 构成；二叉树的许多性质可以推广到 K 叉树，通常用数组实现完全 K 叉树。

习题

补充证明与算法题（参考课程第 6 章）

1. 高度为 `h` 的满 `k` 叉树按层编号，推导第 `l` 层结点数、结点 `i` 的第 `m` 个孩子编号及右兄弟条件。
2. 用并查集判断变量方程组 `a==b`、`a!=b` 是否有解，并分别分析无优化、按秩合并和路径压缩的复杂度。
3. 给定一棵树的左孩子/右兄弟表示, 写出其森林的先根序列和带度数后根序列。

以下为本书正文习题:

1. 画出三棵树组成的森林转换成的二叉树，再转换回去，验证互逆。
2. 对用 6.1.1 节嵌套括号表示法给出的树 `A(B(E,F),C(G),D(H,I,J))`，写出先根、后根、层次序列。
3. 度为 2 的有序树和二叉树差在哪里？举一个「左空右不空」的例子。
4. 用带度数的后根次序表示一棵小树，并说明扫描时栈如何弹出孩子。
5. 对元素 $0..6$ 依次 `unite(0,1)`、`unite(2,3)`、`unite(0,2)`（按本书 `DisjointSet` 的重量权衡口径，规模相等时值大的根挂到值小的根下），画出父指针，再 `find(3)` 后画出路径压缩的结果。
6. 完全 3 叉树按层编号，写出结点 4 的父和孩子们。

原书习题

以下是原书第 6 章的习题,本轮按扫描件补回题面;**参考答案见** `book/` **习题与参考答案**`.md` **的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。

1. 给出一棵树的逻辑结构$T=(K,R)$,其中$K=\{A,B,C,D,E,F,G,H,I,J,K,L$
$R=\{\langle A,B\rangle,\langle A,C\rangle,\langle A,D\rangle,\langle B,E\rangle,\langle B,F\rangle,\langle B,G\rangle,\langle C,H\rangle,\langle D,I\rangle,\langle D,J\rangle,\langle$
面照原书扫描件抄录。注意$\langle D,J\rangle$与$\langle I,J\rangle$让J有两个父结点，而M不出现在任何 有序对里——严格说这个R不构成树。做题时先指出这一矛盾，再说明自己如何修正后作答。）试用树形表示法画出此树，并按照递归定义将树划分为子树，指出哪些结点是根、哪些结点是树叶，确定各结点的层数、度数，并将该树转化为对应的二叉树。
2. 3 个结点可以构造出多少个不同的有序树？试将其画出来。
3. 将图 6.19 中的二叉树转换为对应的森林，分别按照先根次序、后根次序列出其结点。

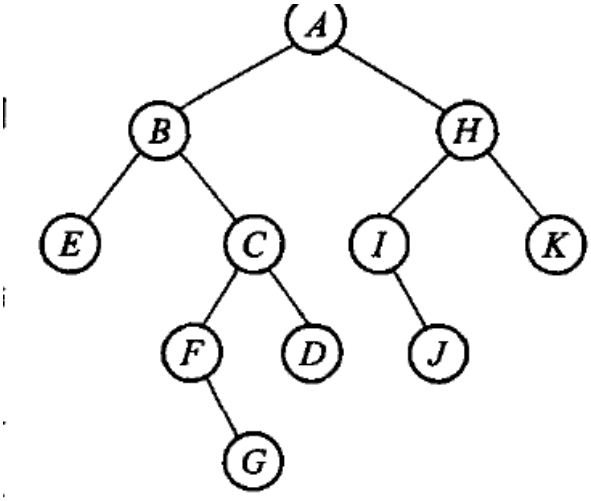


图 6.19　习题 3 的图示

4. 编写出一个算法来判断两棵树是否相同。**尽可能提高算法效率**，并分析算法的运行时间代价。
5. 编写出一个算法来判断两棵树在**不考虑子树顺序**的前提下是否相等。尽可能提高算法效率，分析算法的运行时间代价。
6. 编写一个函数，以一棵树为输入，返回树的结点数目。
7. 分析「子结点表」表示法、「左子/右兄」二叉链表表示法的**结构性开销比率**。相比之下它们各自的 空间效率如何?
8. 使用重量权衡合并规则与路径压缩，对下列从 $0 \sim 15$ 之间的数的等价对进行归并，并给出所得树的 父指针表示法的数组表示。在初始情况下，集合中的每个元素分别在独立的等价类中；当两棵树规模 同样大时，使结点值较大的根结点作为值较小的根结点的子结点。$(0,2)\ (1,2)\ (3,4)\ (3,1)\ (3,5)\ (9,11)\ (12,14)\ (3,9)\ (4,14)\ (6,7)\ (8,10)\ (8,7)$
9. 编写函数，以一棵树的根为输入，返回一棵对应的二叉树。
10. 一个高度为L的K叉树, 如果按层次顺序对树中所有结点从 1 开始编号, 问: (1) 编号为N的 结点的父结点（若存在）编号是多少? (2) 编号为N的结点

的第 i 个子结点（若存在）编号是多少?

(3) 编号为 N 的结点有右兄弟的条件是什么，其编号是多少?

11. 画出图 6.20 所示二叉树对应森林的左子/右兄法存储表示、带右链的先根次序存储表示、带左链的 层次次序存储表示和带度数的后根次序存储表示。

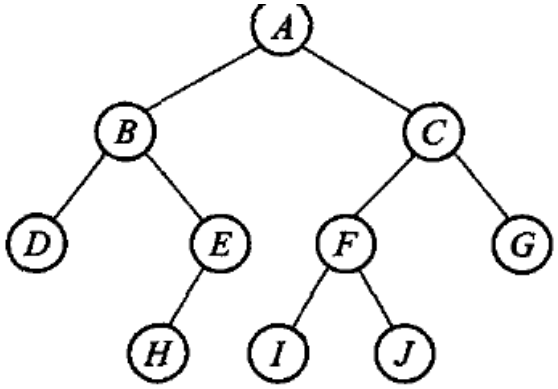


图 6.20　习题 11 的图示

12. 设计一个算法，将一个用带度数的后根次序法表示的森林转换为左子/右兄法表示。
13. 设计一个算法，将一个用带右链的先根次序法表示的森林转换为用带度数的后根次序法表示。

上机题

1. 实现森林与二叉树的互相转换，并用先根序列对拍。
2. 用并查集判断无向图是否连通，并统计连通分量个数。
3. 比较带路径压缩与不带路径压缩的 `find` 在退化链上的时间。

原书上机题

同上，本轮只补题面，参考答案登记在 `collab/answer_gaps.json`。

1. 使用带子结点指针数组的树的链接表示法，实现 6.1.3 节中定义的树类。**此实现必须能支持结点的 子结点数目的变化**：在生成时，结点应该只被分配只够存储其初始子结点集的空间；当一个新的子结点 添加到结点上以致于数组溢出时，则从可利用空间中分配一个为原来数组两倍大的数组。
2. 使用父指针表示法，实现 6.1.3 节中定义的树类。
3. 若用大写字母标识树的结点，则可用**带标号的广义表形式**表示一棵树，其语法为「树 → 大写字母 [(树 { , 树 })]」（图 6.21）。编写递归算法：(1) 将这种表示方法的字符序列 转化成树的左子/右兄表示法；(2) 将用左子/右兄表示法表示的树转化为这种表示法的字符序列。

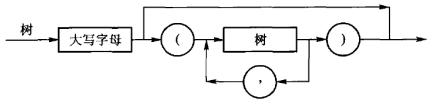


图 6.21　上机题 3 的图示：广义表形式的语法图

4. 另一种表示二维数组的方法是 4 **叉树表示法**，其中一个结点的 4 个儿子分别表示数组的 NE、SE、SW、NW 4 个分块（图 6.22）。这种表示二维数组的 4 叉树称为**金字塔**。

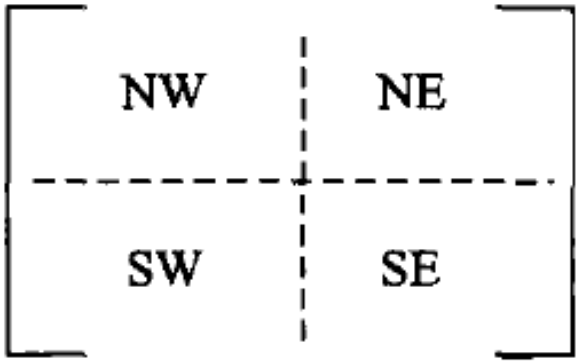


图 6.22　上机题 4 的图示：一个结点的 4 个子结点对应数组的 4 个分块

(1) 设计一个基于金字塔表示法的矩阵乘法算法；(2) 对于给定的 $2^p \times 2^p$ 矩阵，设计一个构造 表示这个矩阵的二叉树算法，并将这个算法推广到可表示任意 $M \times N$ 矩阵的情形，算法效率如何?

第 7 章 图

图由顶点和边组成。有向边表示单向关系，无向边表示双向关系，带权边表示代价。先弄清题目属于哪一种，再选算法。

源码：图与七个算法、可运行示例、交叉验证测试。

7.1 图的定义和基本术语

图比线性结构和树更一般：结点之间的关系可以是任意的。按第 1 章的分类——线性结构唯一前驱、唯一后继；树唯一前驱、多个后继；图对前驱和后继的个数都不加限制。线性和树都可以看成受限的图。

一个典型问题：要在几个城市之间建通信网，使每两个城市都能直接或间接通话，并且总造价尽量低。用顶点代表城市，边代表线路，边旁的数代表造价，问题就变成：在带权图里找一棵连通全部顶点、边权之和最小的树。这就是后面的最小生成树。

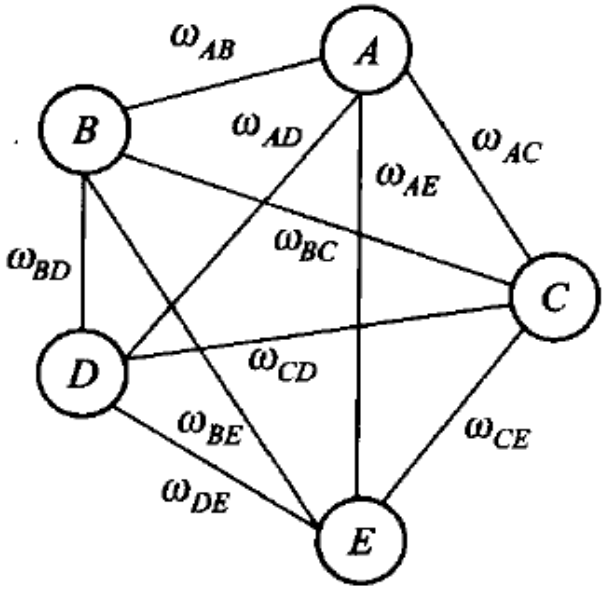


图 7.1　用图描述通信网络。顶点是城市，边是可以架设的线路，边旁的数是造价。要求选出一组线路，把 5 个城市全连起来且总造价最小。

图记作 $G = \langle V, E \rangle$。V 是顶点的有穷非空集合；E 是边的集合，每条边是一对顶点。边没有方向时叫无向图，无序对写成 (v_1, v_2)，(v_1, v_2) 与 (v_2, v_1) 是同一条边。边有方向时叫有向图，有序对写成 $\langle v_1, v_2 \rangle$，也叫弧：v_1 是弧尾（起点），v_2 是弧头（终点）；$\langle v_1, v_2 \rangle$ 与 $\langle v_2, v_1 \rangle$ 是两条不同的弧。图 7.2 给出两个例子。(a) 是无向图 G_1，(b) 是有向图 G_2，写成集合就是

$$G_1 = \langle V(G_1), E(G_1) \rangle, \quad V(G_1) = \{v_0, v_1, v_2, v_3, v_4\}$$

$$E(G_1) = \{(v_0, v_2), (v_0, v_3), (v_1, v_3), (v_1, v_4), (v_2, v_3), (v_2, v_4)\}$$

$$G_2 = \langle V(G_2), E(G_2) \rangle, \quad V(G_2) = \{v_0, v_1, v_2, v_3\}$$

$$E(G_2) = \{\langle v_0, v_1 \rangle, \langle v_0, v_2 \rangle, \langle v_2, v_3 \rangle, \langle v_3, v_0 \rangle\}$$

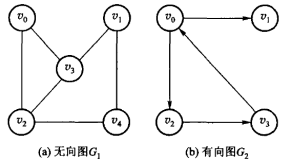


图 7.2　图的示例：(a) 无向图 G_1，(b) 有向图 G_2。本章后面反复拿这两张图举例。

边或弧上可以附带权，表示距离、时间或代价。每条边都带权的图叫带权图。本书不考虑多重边（同一对顶点之间多于一条边）和自环（顶点到自己的边）。

用 n 表示顶点数，e 表示边数。无向图 e 的范围是 $0 \sim n(n-1)/2$，有向图是 $0 \sim n(n-1)$。边相对少的叫稀疏图，相对多的叫稠密图。任意两个顶点之间都有边的图叫完全图，它取到边数的上限。

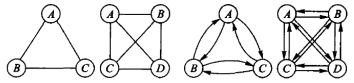


图 7.3　完全图：(a) 是 3 个和 4 个顶点的无向完全图，(b) 是顶点数相同的有向完全图。无向完全图每对顶点之间一条边，有向完全图每对之间两条方向相反的弧。

若 V' 是 V 的子集，E' 是 E 的子集，且 E' 里的边只连 V' 里的顶点，则 $G' = \langle V', E' \rangle$ 是 G 的子图。一条边所连的两个顶点互为邻接点；这条边与这两个顶点相关联。有向图里还要分清「邻接到」和「邻接自」。

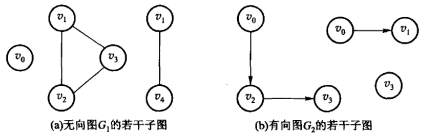


图 7.4　图 7.2 的若干子图：(a) 的两幅取自无向图 G_1，(b) 的两幅取自有向图 G_2。只保留一部分顶点和一部分边，剩下的仍是原图的子图。

顶点的度是与它相关联的边数。有向图再分成入度（指进来的弧）和出度（指出去的弧）。度为 0 的顶点是孤立点。

从 u 出发、沿边走到 v 的顶点序列叫路径；边数是路径长度。有向图必须顺着弧走。起点和终点相同、且至少含一条边的路径叫回路（环）。**顶点不重复出现的路径叫简单路径**；回路中除起点和终点外其余顶点不重复，叫简单回路。

无向图中，若任意两点之间都有路径，称图连通；极大的连通子图叫连通分量。有向图中，若任意两点 u、v 都存在 u 到 v 和 v 到 u 的有向路径，称强连通；只要求把有向边看成无向边后连通，叫弱连通。

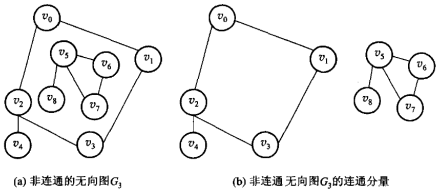


图 7.5　(a) 非连通的无向图 G_3；(b) G_3 的两个连通分量。「极大」是指再添任何一个顶点，子图就不连通了。

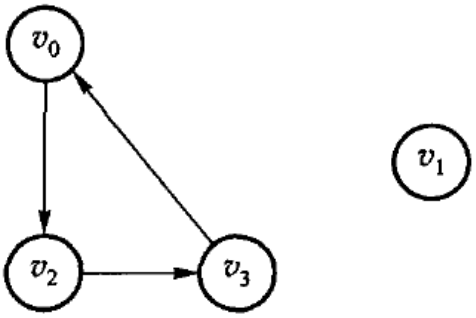


图 7.6　图 7.2(b) 中 G_2 的两个强连通分量：左边是 $\{v_0, v_2, v_3\}$（$v_0 \to v_2 \to v_3 \to v_0$ 成环），右边是只含 v_1 的分量。G_2 本身不是强连通的——v_1 只有入弧，从 v_1 回不到 v_0。

无向连通图的生成树，是包含全部顶点、有 $n-1$ 条边、因而没有环的连通子图。带权连通图里边权之和最小的生成树，就是最小生成树。最短路的「短」是边权总和最小，不一定是经过的边数最少。

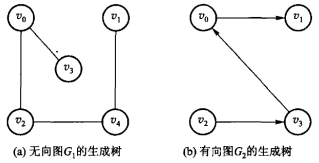


图 7.7　图 7.2 中 (a) 无向图 G_1 和 (b) 有向图 G_2 的一棵生成树。生成树上再加一条边必成环；边数少于 $n-1$ 则必不连通。

不带简单回路的连通无向图叫自由树，它同样有 $n-1$ 条边。带权的连通图叫网络，图 7.8 的 G_4 就是一个网络——7.5 节的最短路径和 7.6 节的最小生成树都在网络上讨论。

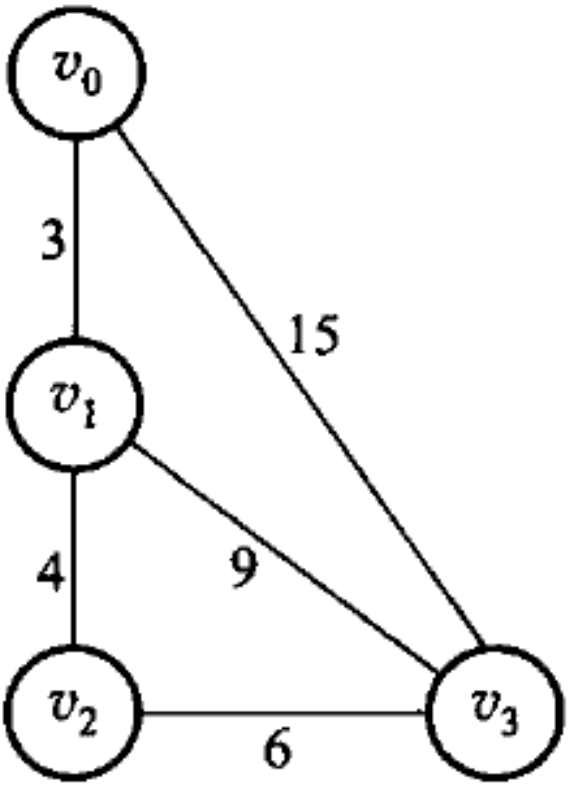


图 7.8　网络实例 G_4：带权的连通图。

有向图里也有对应的说法：只有一个顶点入度为 0、其余顶点入度均为 1 的有向图叫有向树；若干棵有向树，弧互不相交、顶点合起来正好是原图的全部顶点，就构成一个生成森林。图 7.9 是一个例子。

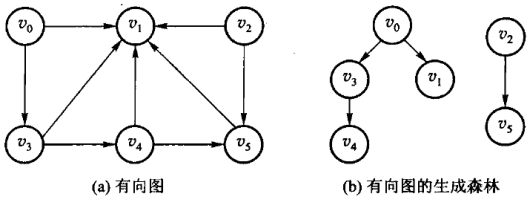


图 7.9　(a) 一个有向图；(b) 它的生成森林——两棵有向树，弧互不相交，顶点合起来是原图的全部顶点。

选算法之前，先分清题目：

题目	前提	结果
从一个点能走到哪些点	任意图	DFS 或 BFS 的访问序列
课程或任务的可行顺序	有向无环图	拓扑序；有环则无解

题目	前提	结果
一个源点到各点的最短路	边权非负	Dijkstra 的距离数组
任意两点最短路	顶点数较小	Floyd 的距离矩阵
连通所有点且总权最小	无向连通图	Prim 或 Kruskal 的边集

本单元用邻接矩阵。没有边记为 `infinity`（`int` 最大值的四分之一），边权必须小于它；最短路径的长度是若干条边之和，另用 64 位的 `Graph::distance_type` 存，到不了记为 `Graph::unreachable`，不与「无边」共用一个值（D-039）。环、非连通等正常失败用 `optional` 表达。DFS 仍是递归，深图有栈溢出风险。

7.2 图的抽象数据类型

和线性表、树一样，先说清楚图对外提供哪些运算，再谈怎么存。

原书 7.2 节先强调了一件容易被忽略的事：**图的顶点之间本来没有次序**。从逻辑结构看，任何一个顶点都可以当作「第一个」顶点，一个顶点的若干邻接点之间也没有先后。可是一旦按某种存储结构把图建起来，次序就被存储结构定死了——邻接矩阵里的次序是列号，邻接表里的次序是链表的插入次序。所以下文所有「第一条边」「下一条边」说的都是存储结构给出的次序，不是图本身的性质；同一张图换个存法，DFS 的访问序列就可能不同。7.3 节把两种存储结构逐项对拍，正是把这点不确定性钉死：同一张图、同样的建图次序，两种实现必须给出同一个序列。

原书用【代码 7.1】把运算列成一个抽象类（下面是修掉 OCR 断字、补齐花括号后的样子，运算和注释照原书）：

```
text original-listing=" 原书【代码 7.1】的图 ADT 声明，OCR 把标识符切断、类尾缺右花括号，作为原书记录原样引用" class Graph {                    // 图的抽象数据类型 public:  int VerticesNum();              // 返回图的顶点个数  int EdgesNum();  // 返回图的边数  Edge FirstEdge(int oneVertex);   // 返回依附于顶点 oneVertex 的第一条边  Edge NextEdge(Edge preEdge);      // 返回与 preEdge 有相同顶点的下一条边  bool setEdge(int fromVertex, int toVertex, int weight);  // 添加边  bool delEdge(int fromVertex, int toVertex);                // 删除边 bool IsEdge(Edge oneEdge);       // oneEdge 是否是边  int FromVertex(Edge oneEdge);     // 返回 oneEdge 的始点  int ToVertex(Edge oneEdge);       // 返回 oneEdge 的终点  int Weight(Edge oneEdge);         // 返回 oneEdge 的权 };
```

这份清单里真正的设计决定是 `Edge`：原书不把「顶点 v 的邻居」整个交出去，而是给一对游标运算，让上层算法写成

```
text original-listing=" 原书各周游算法遍历一个顶点出边的固定写法，出自算法 7.5–7.11，作为原书记录原样引用" for (Edge e = G.FirstEdge(v); G.IsEdge(e); e = G.NextEdge(e)) { /* 处理 e */ }
```

于是 DFS、拓扑、Dijkstra 都只依赖这四个运算，不依赖矩阵还是链表——这是原书用一套算法讲两种存储结构的办法。代价有两处：换一种存法就要重写这对游标；`IsEdge` 同时兼任「这是不是一条边」和「游标是否走到头」两种含义。

原书代码 7.2 给出这个类的基类实现，它的可复核问题记在 `code/ch07/graph/legacy.md` 里：`Mark`、`Indegree` 两个裸数组在构造里 `new`、只在析构里 `delete[]`，没有拷贝构造和赋值运算符，按值传一次图就会二次释放；`numVertex`、`numEdge` 是 public 数据成员，谁都能改坏；`IsEdge` 用 `oneEdge.weight > 0` 判断边是否存在，于是**权为 0 的合法边被判成不存在**。加上标识符被 OCR 空格切断（`num Vertex = num Vert`），这两段清单都不能照抄。

本书因此不复刻这个抽象基类，直接把运算定义在两个可运行的类上：`Graph`（邻接矩阵，`code/ch07/graph`）和 `GraphList`（邻接表，`code/ch07/adjacency_list`）。对应关系如下。

原书运算	干什么	本书写法
`Graph(int numVert)`	指定顶点数建一张空图	`Graph::Graph`、`GraphList::GraphList`
`VerticesNum()`	返回顶点个数	`Graph::vertices`、`GraphList::vertices`
`setEdge(f, t, w)`	添加一条边	`Graph::add_edge`、`GraphList::add_edge`（末参选有向/无向）
`IsEdge(e)` / `Weight(e)`	问某条边在不在、权多少	矩阵直接读 `adjacency_`；邻接表用 `GraphList::weight`，无边返回空
`FirstEdge` / `NextEdge`	逐条走一个顶点的出边	矩阵扫一行；邻接表用 `GraphList::neighbors` 返回该顶点的边表
—	周游、拓扑、最短路、最小生成树	`Graph::dfs`、`Graph::bfs`、`Graph::topological_sort`、`Graph::dijkstra`、`Graph::floyd`、`Graph::prim`、`Graph::kruskal`

`delEdge` 本书没有实现：后面七个算法都不删边，凭空加一个没有测试覆盖的运算，只会多一处无人验证的代码。需要时按 D-001 的规矩补，连测试一起补。

接口上还有一条贯穿全章的约定：「**有环**」「**不连通**」**不是异常，是可预期的结果**。所以 `topological_sort`（图里有环）、`prim` 和 `kruskal`（图不连通）返回

`std::optional`，空值本身就是答案；`GraphList::weight` 在两点之间没有边时同样返回空。真正的用法错误才抛异常——顶点下标越界抛 `std::out_of_range`，负权边抛 `std::invalid_argument`（Dijkstra 在负权上不成立，见 7.5.1 节），权不小于 `infinity` 的边同样抛——`infinity` 在矩阵里兼任「无边」，这么大的权若被收下，后面所有算法都会把它当成没有边，而且一声不吭。原书把这些情况打印到 `cout`，容器里做输出，调用者既拿不到结果也没法测试。

7.3 图的存储结构

图常用两种存法。邻接矩阵用 $n \times n$ 的表，$A[i][j]$ 表示 i 到 j 有没有边、权是多少，适合稠密图和需要 $O(1)$ 问「两点之间有没有边」的算法（Floyd、Prim）。邻接表为每个顶点挂一条出边表，适合稀疏图和只沿出边走的算法（DFS、BFS、Dijkstra、拓扑）。

两种都有可运行实现：邻接矩阵是 `code/ch07/graph`（代码 7.1–7.3），邻接表是 `code/ch07/adjacency_list`（代码 7.4）。两者**逐项对拍**——同一张图上 DFS/BFS 序列、拓扑序、Dijkstra 距离向量必须完全相同，外加 30 轮随机图对拍。换存储方式不该换答案，只该换代价。

7.3.1 相邻矩阵

本节沿用原书标题；现代教材通常称为**邻接矩阵**（adjacency matrix）。邻接矩阵是一个 $V \times V$ 的二维数组；第 `from` 行、第 `to` 列保存弧 `from` → `to` 的权值。构造时对角线为 0（到自己的距离是 0），其余位置为 `infinity` 表示没有边。无向边要同时写入 (u, v) 和 (v, u) 两格；`add_edge(..., directed=false)` 正是这样做的。零权边是合法的，因此不能再用 0 表示 “无边”。

不带权的图只需要记「有没有边」，即 $A[i, j] = 1$ 表示 $(v_i, v_j) \in E$，否则为 0。图 7.2 那两张图的相邻矩阵是：

$A(G_1)$	v_0	v_1	v_2	v_3	v_4
v_0	0	0	1	1	0
v_1	0	0	0	1	1
v_2	1	0	0	1	1
v_3	1	1	1	0	0
v_4	0	1	1	0	0

$A(G_2)$	v_0	v_1	v_2	v_3
v_0	0	1	1	0
v_1	0	0	0	0
v_2	0	0	0	1
v_3	1	0	0	0

图 7.10　(a) 无向图 G_1 与 (b) 有向图 G_2 的相邻矩阵。**无向图的矩阵一定对称**，顶点 v_i 的度就是第 i 行（或第 i 列）之和；有向图的矩阵不一定对称，第 i 行之和是出度，第 i 列之和是入度。

带权图把 1 换成权值，没有边的位置记 ∞。图 7.8 的网络 G_4 是：

$A(G_4)$	v_0	v_1	v_2	v_3
v_0	0	3	∞	15
v_1	3	0	4	9
v_2	∞	4	0	6
v_3	15	9	6	0

图 7.11　带权图 G_4 及其相邻矩阵。对角线是 0（到自己的距离为 0）。本书的实现用 `infinity` 而不是 0 表示无边——**因为权为 0 的边是合法的**，用 0 兼任「无边」会把它误判掉。

矩阵的空间是 $O(V^2)$，无论图中实际有多少条边；查询两点是否相邻为 $O(1)$，但扫描一个顶点的全部邻居要扫完整行，遍历全图通常为 $O(V^2)$。所以它适合稠密图，以及 Floyd、Prim 这类需要频繁随机访问矩阵元素的算法；稀疏图用邻接表会省下大量空间。

7.3.2 邻接表

邻接表为每个顶点存一张出边表，**只存实际存在的边**。

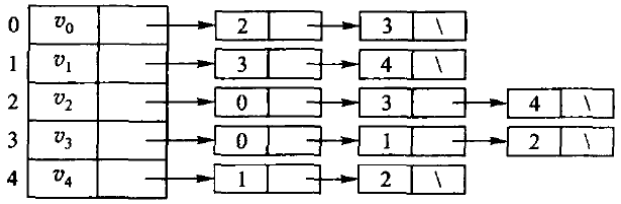


图 7.12　无向图 G_1 的邻接表表示。顶点数组的每一项挂一条链，链上是它的邻接点。无向图里一**条边出现两次**（(u,v) 在 u 的表里，也在 v 的表里），所以边表结点共 $2e$ 个。

有向图的一条弧只出现一次，顶点 v_i 的边表长度就是它的**出度**——因此邻接表也叫出边表。想知道入度就得扫遍整张表，于是有了**逆邻接表**（入边表）：把每条弧记在弧头那一侧。

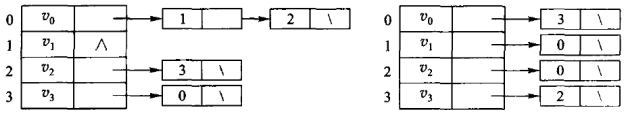


图 7.13　有向图 G_2 的 (a) 邻接表与 (b) 逆邻接表。逆邻接表里 v_i 的边表长度就是它的入度——7.4.3 节拓扑排序要的正是入度。

带权图在边表结点里多存一个权：

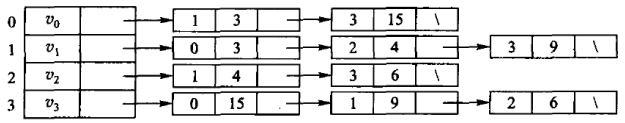


图 7.14　带权图 G_4 的邻接表表示。本书 `GraphList` 的边表结点（`Edge` 结构里的 `to` 与 `weight`）就是这两个域。

代价随之全变：

	邻接矩阵	邻接表
存储量	V^2	$V+E$
遍历某点的邻居	$O(V)$，要扫过整行	$O(\deg v)$
问「(u,v) 之间有没有边」	$O(1)$	$O(\deg u)$
DFS / BFS 全图	$O(V^2)$	$O(V+E)$
Dijkstra	$O(V^2)$	$O((V+E)\log V)$（配二叉堆）

注意第三行：**邻接表在这一项上是吃亏的**。没有哪种表示法总是更好，这正是本节要比较的东西。

结构上的差别只有一处——存的是什么：

```
/// 邻接表存图（原书【代码 7.4】）：每个顶点挂一条边表，只存 ** 实际存在 ** 的边。
///
/// 与同章 `Graph`（邻接矩阵）的区别只有一处——存储方式——但这一处决定了全部代价：
///
/// | | 邻接矩阵 | 邻接表 |
/// | --- | --- | --- |
/// | 存储量 | $V^2$ | $V + E$ |
/// | 遍历某点的邻居 | $O(V)$，要扫过整行 | $O(\deg v)$，只走这条边表 |
/// | 判断 (u,v) 是否有边 | $O(1)$ | $O(\deg u)$ |
/// | DFS / BFS 全图 | $O(V^2)$ | $O(V + E)$ |
///
/// 稀疏图（$E \ll V^2$）上邻接表快得多；稠密图或频繁问「这两点之间有没有边」时，
/// 矩阵反而合适。** 没有哪种表示法总是更好 **，这正是本章要比较的东西。
class GraphList {
public:
    /// 边权上限：0 <= weight < infinity，与邻接矩阵版 `Graph` 同口径。
    static constexpr int infinity = std::numeric_limits<int>::max() / 4;

    /// 路径长度 64 位、「到不了」用 unreachable，与 `Graph` 相同（D-039）。
    using distance_type = std::int64_t;
    static constexpr distance_type unreachable =
    ↪  std::numeric_limits<distance_type>::max();

    struct Edge {
        std::size_t from;
        std::size_t to;
        int weight;

        bool operator<(const Edge& other) const noexcept { return weight <
        ↪  other.weight; }
```

```
};

explicit GraphList(std::size_t count) : adjacency_(count) {}

/// 加边。重复加同一条 `from → to` 时 ** 覆盖权值 ** 而不是并存两条——
/// 与邻接矩阵的语义保持一致，两种表示法才可以逐项对拍。代价是每次加边 O(deg from)。
void add_edge(std::size_t from, std::size_t to, int weight, bool directed =
↪  true) {
    check_vertex(from);
    check_vertex(to);
    if (weight < 0) {
        throw std::invalid_argument("negative edge");
    }
    if (weight >= infinity) {  // 与邻接矩阵版同口径：infinity 在那边表示「无边」，两
    ↪  种表示法才能逐项对拍
        throw std::invalid_argument("edge weight must be below
        ↪  GraphList::infinity");
    }
    put(from, to, weight);
    if (!directed) {
        put(to, from, weight);
    }
}

[[nodiscard]] std::size_t vertices() const noexcept { return adjacency_.size(); }

/// 边表里的条目数。无向图一条边会在两端各存一次，这里如实返回条目数。
[[nodiscard]] std::size_t edge_entries() const noexcept {
    std::size_t total = 0;
    for (const auto& list : adjacency_) {
        total += list.size();
    }
    return total;
}

[[nodiscard]] std::size_t degree(std::size_t vertex) const {
    check_vertex(vertex);
    return adjacency_[vertex].size();
}

/// 某个顶点的边表。邻接表的核心能力：拿到邻居不必扫过不存在的边。
[[nodiscard]] const std::vector<Edge>& neighbors(std::size_t vertex) const {
    check_vertex(vertex);
    return adjacency_[vertex];
}

[[nodiscard]] std::optional<int> weight(std::size_t from, std::size_t to) const {
```

```
        check_vertex(from);
        check_vertex(to);
        for (const Edge& edge : adjacency_[from]) {
            ++scanned_;
            if (edge.to == to) {
                return edge.weight;
            }
        }
        return std::nullopt;  // 没有这条边是预期状态，不是错误
    }

    /// 存储量（按「一个整数算一格」计）。对照矩阵的 $V^2$，稀疏图上差距一目了然。
    [[nodiscard]] std::size_t storage_cells() const noexcept {
        return vertices() + edge_entries() * 2;  // 每条边存 to 和 weight
    }

    /// 已经检查过多少条边。教学计数器：用它把 $O(V+E)$ 和 $O(V^2)$ 的差别量出来。
    [[nodiscard]] std::size_t scan_steps() const noexcept { return scanned_; }
    void reset_scan_steps() const noexcept { scanned_ = 0; }
```

周游时的差别随之而来。矩阵版每出队一个顶点要扫过整行 V 格，邻接表只走这条边表：

```
/// 深度优先周游。只走边表，不看不存在的边——这是与矩阵版最直接的差别。
[[nodiscard]] std::vector<std::size_t> dfs(std::size_t source) const {
    check_vertex(source);
    std::vector<bool> seen(vertices());
    std::vector<std::size_t> result;
    visit_depth_first(source, seen, result);
    return result;
}

[[nodiscard]] std::vector<std::size_t> bfs(std::size_t source) const {
    check_vertex(source);
    std::vector<bool> seen(vertices());
    std::queue<std::size_t> pending;
    std::vector<std::size_t> result;
    pending.push(source);
    seen[source] = true;
    while (!pending.empty()) {
        const std::size_t from = pending.front();
        pending.pop();
        result.push_back(from);
        for (const Edge& edge : adjacency_[from]) {  // 矩阵版这里要扫 V 格
            ++scanned_;
            if (!seen[edge.to]) {
```

```
                seen[edge.to] = true;
                pending.push(edge.to);
            }
        }
    }
    return result;
}
```

差距是量出来的，不是推出来的。取一条 1000 个顶点的链（$E = V - 1$，典型的稀疏图）：

```
#include "modern.hpp"

#include <cstdio>

int main() {
    using dsa::GraphList;

    // 一条 1000 个顶点的链：典型的稀疏图（E = V - 1）。
    constexpr std::size_t n = 1000;
    GraphList sparse(n);
    for (std::size_t v = 0; v + 1 < n; ++v) {
        sparse.add_edge(v, v + 1, 1);
    }
    sparse.reset_scan_steps();
    const auto order = sparse.bfs(0);
    const std::size_t steps = sparse.scan_steps();
    std::printf(" 稀疏图 V=%zu, E=%zu\n", n, sparse.edge_entries());
    std::printf("  邻接表存储 %zu 格   ← 随 V+E 走\n", sparse.storage_cells());
    std::printf("  邻接矩阵存储 %zu 格 ← V*V\n", n * n);
    std::printf("  BFS 走遍 %zu 个顶点，只检查了 %zu 条边\n", order.size(), steps);
    std::printf("  邻接矩阵的 BFS 要扫 %zu 格（每出队一个顶点扫一整行）\n", n * n);

    // 教科书例子：最短路与最小生成树。
    GraphList graph(6);
    const int edges[][3] = {{0,1,7},{0,2,9},{0,5,14},{1,2,10},{1,3,15},
                            {2,3,11},{2,5,2},{3,4,6},{5,4,9}};
    for (const auto& e : edges) {
        graph.add_edge(static_cast<std::size_t>(e[0]),
↪   static_cast<std::size_t>(e[1]), e[2], false);
    }
    const auto distance = graph.dijkstra(0);
    std::printf("\n从 0 出发的最短距离:");
    for (const auto d : distance) {
        std::printf(" %lld", static_cast<long long>(d));
    }
```

```
    const auto mst = graph.prim(0);
    int total = 0;
    for (const auto& edge : *mst) {
        total += edge.weight;
    }
    std::printf("\n最小生成树 %zu 条边，总权 %d\n", mst->size(), total);
    return 0;
}
```

```
稀疏图 V=1000, E=999
  邻接表存储 2998 格   ← 随 V+E 走
  邻接矩阵存储 1000000 格 ← V*V
  BFS 走遍 1000 个顶点，只检查了 999 条边
  邻接矩阵的 BFS 要扫 1000000 格（每出队一个顶点扫一整行）

从 0 出发的最短距离: 0 7 9 20 20 11
最小生成树 5 条边，总权 33
```

存储差三百多倍（$1000000/2998 \approx 334$），BFS 的检查次数差约 1000 倍（$1000000/999$）。稠密图上这个差距会消失——E 接近 V^2 时，$V+E$ 也就是 V^2，而矩阵还省掉了存 to 的那一半空间。

Dijkstra 是这一章里差别最大的一处。矩阵版每轮要扫一遍全部顶点找最近的那个，是 $O(V^2)$；邻接表配一个最小堆之后，「找最近顶点」由堆负责，「松弛」只走实际存在的边：

```
/// Dijkstra，最小堆版。代价 $O((V+E)\log V)$。
///
/// 矩阵版每轮要扫一遍全部顶点找最近的那个，是 $O(V^2)$；换成邻接表 + 堆之后，
/// 「找最近顶点」由堆负责、「松弛」只走实际存在的边。** 稀疏图上这才是该用的组合 **。
/// 堆是第 5 章的教学内容，这里作为基础设施使用（见 unit.json 的 d001_exceptions）。
[[nodiscard]] std::vector<distance_type> dijkstra(std::size_t source) const {
    check_vertex(source);
    std::vector<distance_type> distance(vertices(), unreachable);
    distance[source] = 0;

    using Item = std::pair<distance_type, std::size_t>;  // (当前距离, 顶点)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> heap;
    heap.emplace(0, source);
    while (!heap.empty()) {
        const auto [dist, from] = heap.top();
        heap.pop();
        if (dist > distance[from]) {
            continue;  // 堆里的旧条目，已经被更短的路径取代
        }
```

```
        for (const Edge& edge : adjacency_[from]) {
            ++scanned_;
            const distance_type relaxed = dist + edge.weight;
            if (relaxed < distance[edge.to]) {
                distance[edge.to] = relaxed;
                heap.emplace(relaxed, edge.to);
            }
        }
    }
    return distance;
}
```

7.5.1 节给出的 $O(V^2)$ 是**矩阵版**的结论，别把它套到邻接表上；反过来，常见教材里那个 $O((V+E)\log V)$ 也不能套到矩阵版上。复杂度是「算法 + 存储结构」一起决定的。

7.3.3 十字链表

邻接表只沿出边组织数据；如果还要频繁查询 “哪些边指向这个顶点”，就得另存一份逆邻接表。十字链表把两种方向接在同一个弧结点上：`tailnextarc` 串起同一尾点的出边，`headnextarc` 串起同一头点的入边；顶点同时保存 `firstoutarc` 和 `firstinarc`。每条弧只分配一个结点，却能在 $O(\deg^+(v))$ 或 $O(\deg^-(v))$ 时间遍历出边或入边，空间为 $O(V+E)$。

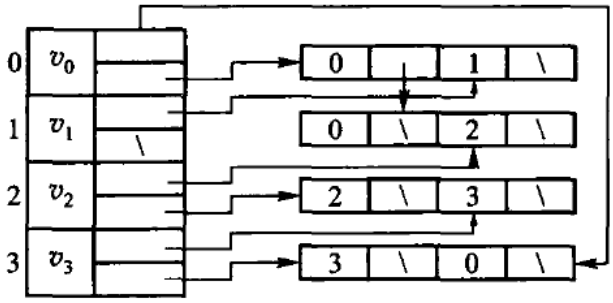


图 7.15　有向图的十字链表示例。每条弧只有一个结点，却同时挂在「同尾」和「同头」两条链上——邻接表与逆邻接表的信息合在了一起，不必存两份。

原书式教学实现：把两次挂接写在眼前

原书 7.3.3 本身只有图和字段说明，没有独立代码清单。这里保留截图所用的传统教材写法：`ArcBox`、`VexNode`、裸链接和表头插入。它不是把截图伪称为原书逐字代码，而是把图 7.15 的 表示法写成一份可以运行的教学版；为使其能安全结束生命周期，补上了析构并禁止浅拷贝。

```
struct ArcBox {
    std::size_t tailvex;       // 弧尾 u
    std::size_t headvex;       // 弧头 v
    ArcBox* tailnextarc;       // 下一条同尾弧
    ArcBox* headnextarc;       // 下一条同头弧
    int info;                  // 权值或其他边属性
};

struct VexNode {
    ArcBox* firstin = nullptr;   // 第一条指向本顶点的弧
    ArcBox* firstout = nullptr;  // 第一条从本顶点出发的弧
};
```

先读这两种结点。ArcBox 的前两个域说明“哪条弧”，后两个域说明“这条弧在两条链中各接向哪里”；VexNode 的两个表头则分别给出入链和出链的入口。重点是 firstin 与 firstout 指向的可以是同一个 ArcBox，并没有第二份边结点。

下面六行就是传统写法最值得保留的地方。先 new 出一个弧结点；tailnextarc 接住旧出链表头，再改 firstout；headnextarc 接住旧入链表头，再改 firstin。因为是表头插入，所以最新插入的弧最先被遍历到。

```
void add_edge(std::size_t tail, std::size_t head, int info = 1) {
    check_vertex(tail);
    check_vertex(head);
    auto* arc = new ArcBox{tail, head, vertices_[tail].firstout,
                           vertices_[head].firstin, info};
    vertices_[tail].firstout = arc;  // 接进 tail 的出边链
    vertices_[head].firstin = arc;   // 同一结点接进 head 的入边链
    ++arc_count_;
}
```

clear() 沿每个顶点的**出链**释放一次即可，因为每条弧恰好属于一个尾点；绝不能再沿入链释放。教学版删除复制构造和复制赋值，防止两个图对象析构同一批 ArcBox。这些资源边界不改变“一个结点、两条链”的教学结构，只把短例中通常省略的收尾补齐。

现代实现：所有权集中，双链仍显式

完整可编译类型在 code/ ch07/ orthogonal_graph/ modern.hpp。它用 std::vector<std::unique_ptr<Arc>> arcs_ 集中拥有每个弧结点；firstin、firstout、tailnextarc、headnextarc 仍是裸的**非拥有**链接，因此双链的形状没有被智能指针藏掉。

插入仍是与教学版相同的两次挂接，只是 owned 最后移入唯一所有者表：

```
void add_edge(std::size_t tail, std::size_t head, int info = 1) {
    check_vertex(tail);
    check_vertex(head);
    if (find_arc(tail, head)) {
        throw std::invalid_argument("duplicate edge");
    }

    auto owned = std::make_unique<Arc>(tail, head, info);
    Arc* arc = owned.get();
    arc->tailnextarc = vertices_[tail].firstout;
    vertices_[tail].firstout = arc;
    arc->headnextarc = vertices_[head].firstin;
    vertices_[head].firstin = arc;
    arcs_.push_back(std::move(owned));
}
```

删除最能检验两条链是否真的独立维护：先在尾点出链找到并摘下 `victim`，再在头点入链找到**同一地址**并摘下，最后从 `arcs_` 移除唯一所有者，析构自动释放结点。若少掉任一次摘链，测试会分别在出边或入边查询处失败。

```
bool remove_edge(std::size_t tail, std::size_t head) {
    check_vertex(tail);
    check_vertex(head);
    Arc** out_link = &vertices_[tail].firstout;
    while (*out_link && (*out_link)->head != head) {
        out_link = &(*out_link)->tailnextarc;
    }
    if (!*out_link) {
        return false;
    }

    Arc* victim = *out_link;
    *out_link = victim->tailnextarc;
    Arc** in_link = &vertices_[head].firstin;
    while (*in_link != victim) {
        in_link = &(*in_link)->headnextarc;
    }
    *in_link = victim->headnextarc;
    arcs_.erase(std::remove_if(arcs_.begin(), arcs_.end(),
                               victim {
                                   return owned.get() == victim;
                               }),
                arcs_.end());
    return true;
}
```

把一条弧拆开看

先只看一条弧 $u \to v$。它不是 “出边结点复制一份、入边结点再复制一份”，而是**同一个** 弧结点带着两根不同用途的链接：

```
顶点 u 的 firstoutarc  ->  [tailvex=u | headvex=v | tailnextarc=...]  ->  下一条从 u
↪  出发的弧
                                    ^
顶点 v 的 firstinarc   ->  同一个弧结点                          ->  下一条指向 v 的弧
                                    |
                              headnextarc=...
```

域	回答的问题	沿它走时保持不变的顶点
`tailvex`	这条弧从哪里出发?	弧尾，即出边表所属顶点
`headvex`	这条弧指向哪里?	弧头，即入边表所属顶点
`tailnextarc`	同一个尾点还有哪条下一弧?	`tailvex` 相同
`headnextarc`	同一个头点还有哪条下一弧?	`headvex` 相同
`info`	权值、容量或其他边属性是什么?	不参与两条链的连接

因此查 “从 u 能去哪里” 时，从 `u.firstoutarc` 开始，反复读当前结点的 `headvex`，然后跳 `tailnextarc`；查 “谁能到 v” 时，从 `v.firstinarc` 开始，反复读当前结点的 `tailvex`，然后跳 `headnextarc`。**读的是同一批弧结点，走的是两条不同的 next 链**。这正是 “十字” 的含义。

例如依次插入 $0 \to 1$、$0 \to 2$、$3 \to 2$，不管新弧接在表头还是表尾，两个查询的逻辑都是下表这样；差别只在读出的先后：本节两份实现都是表头插入，所以 0 的出边依次读到 2、1，2 的入边依次读到 3、0（表尾插入才是表中的 1、2 和 0、3）。

查询	起点	读取字段	下一步	得到的顶点
0 的出边	`firstoutarc[0]`	每个结点的 `headvex`	`tailnextarc`	1、2
2 的入边	`firstinarc[2]`	每个结点的 `tailvex`	`headnextarc`	0、3

这也解释了更新为何容易出错：插入 $u \to v$ 时，弧结点必须同时接进 `u` 的出链和 `v` 的入链；删除时也必须在两条链中各找到一次该结点再摘除。只更新其中一条，表面上 “从 u 出发” 可能仍正确，但统计 v 的入度或反向遍历时就会读到陈旧链接。

它适合需要同时维护入度和出度的有向图，例如拓扑排序、依赖图和删除一条弧后的双向更新。代价是插入、删除必须同时维护两条链，指针不变量比普通

邻接表多；只沿出边遍历的程序使用邻接表更简单。实现时删除弧要从尾点的出链和头点的入链各摘除一次，不能只改其中一条，否则下一次按另一方向遍历会访问已释放结点。

7.4 图的周游

图的周游（traversing graph）是指从图中的某一个顶点出发，按照一定的策略访问图中的每一个顶点，使得每一个顶点被访问且只被访问一次。**图的周游算法是求解图的连通性问题、拓扑排序和关键路径 等问题的基础**。

图的周游比树的周游复杂，因为**图中每一个顶点都可能和其他顶点相邻接**：某一个顶点被访问后，还可能经过其他路径又回到这个顶点。为了避免重复访问同一个顶点，在周游图的过程中应该记下顶点 是否已经被访问，若遇到已访问的顶点则不再访问——这就是下面每个算法里那个 `seen` 数组存在的 理由（原书写作 `Mark` 数组，取值 `VISITED` / `UNVISITED`）。

深度优先周游和广度优先周游是两种基本的周游方法，**对有向图和无向图都适用**。

先跑一遍

下面这张有向图：`0 → 1(2)`、`0 → 2(7)`、`1 → 2(1)`、`1 → 3(5)`、`2 → 3(1)`、`3 → 4(3)`。从 0 到 4 的最短路是 `0 → 1 → 2 → 3 → 4`，总权 $2+1+1+3=7$，不是先走那条权为 7 的直达边 `0 → 2` 再往后走（`0 → 2 → 3 → 4` 总权 11）。

```cpp
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::Graph graph(5);
    graph.add_edge(0, 1, 2);
    graph.add_edge(0, 2, 7);
    graph.add_edge(1, 2, 1);
    graph.add_edge(1, 3, 5);
    graph.add_edge(2, 3, 1);
    graph.add_edge(3, 4, 3);

    std::cout << "DFS(0):";
    for (std::size_t vertex : graph.dfs(0)) {
        std::cout << ' ' << vertex;
    }
    std::cout << "\nBFS(0):";
    for (std::size_t vertex : graph.bfs(0)) {
        std::cout << ' ' << vertex;
    }
```

```
    const auto topo = graph.topological_sort();
    std::cout << "\n拓扑序:";
    if (topo) {
        for (std::size_t vertex : *topo) {
            std::cout << ' ' << vertex;
        }
    } else {
        std::cout << " 无（有环）";
    }

    const auto distance = graph.dijkstra(0);
    std::cout << "\nDijkstra(0->4) = " << distance[4] << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch07/graph \
    code/ch07/graph/demo.cpp -o /tmp/graph-demo
/tmp/graph-demo
```

```
DFS(0): 0 1 2 3 4
BFS(0): 0 1 2 3 4
拓扑序: 0 1 2 3 4
Dijkstra(0->4) = 7
```

若再加上 `4 → 0` 形成环，`topological_sort()` 返回 `nullopt`。

7.4.1 深度优先周游

深度优先搜索（depth-first search，DFS）是树的先根次序周游的推广：进入一个顶点就立刻标记 已访问，再沿编号从小到大的出边递归下去；某个顶点的所有邻接点都访问过了，就回退到上一个顶点，换一条还没走过的边继续。

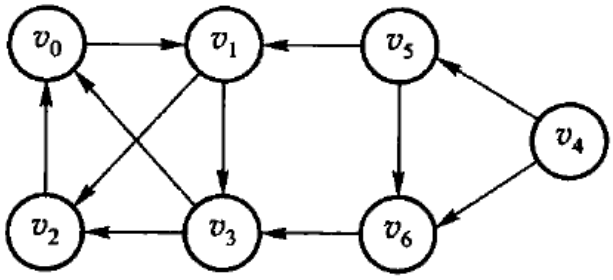


图 7.16　原书用来走深度优先周游的有向图 G。从 v_0 出发：访问 v_0，取它唯一的邻接点 v_1；在 v_1 的边表里 v_2 排在 v_3 前面，于是先进 v_2；v_2 除已访问的 v_0 外没有别的邻接点，回退到 v_1，再进 v_3；v_3 的邻接点都访问过了，一路回退到 v_0。此时还有顶点没访问，**再挑一个未访问顶点重新开始**，依次访问

v_4、v_5、v_6。

两点值得记住：一是**图不连通时一次 DFS 走不完**，要在外层对每个未访问顶点再起一次；二是**存储结构一旦定下、源点一旦定下，深度优先序列就是唯一的**——序列不同不是算法不同，是边表次序不同。

代价取决于存储结构，而不是取决于算法。周游图的过程实质上是搜索每个顶点的邻接点的过程，时间主要耗费在从该顶点出发搜索它的所有邻接点上。对于具有 n 个顶点和 e 条边的无向图或有向 图，深度优先周游算法对图中每个顶点至多调用一次 DFS 函数：**用相邻矩阵表示图时，共需检查 n^2 个矩阵元素，所需时间为 $O(n^2)$；用邻接表表示图时，找邻接点需将邻接表中所有边结点检查一遍，需要时间 $O(e)$，对应的深度优先搜索算法的时间复杂度为 $O(n+e)$**。这也是 7.3 节那张「稠密图用 矩阵、稀疏图用邻接表」的取舍表在算法侧的直接后果。

```cpp
[[nodiscard]] std::vector<std::size_t> dfs(std::size_t source) const {
    check_vertex(source);
    std::vector<bool> seen(vertices());
    std::vector<std::size_t> result;
    visit_depth_first(source, seen, result);
    return result;
}
```

```python
def dfs(self, source: int) -> list[int]:
    self._check_vertex(source)
    seen = [False] * self.vertices
    result: list[int] = []

    def visit(vertex: int) -> None:
        seen[vertex] = True
        result.append(vertex)
        for target in range(self.vertices):
            if self._adjacency[vertex][target] < self.infinity and not seen[target]:
                visit(target)

    visit(source)
    return result
```

7.4.2 广度优先周游

系统地访问图的所有顶点的另一个方法是**广度优先搜索**（breadth-first search, BFS）。其周游的过程 是：从图中的某个顶点 v 出发，访问并标记了顶点 v 之后，**横向**搜索 v 的所有邻接点 $u_1, u_2, \cdots, u_t$；在依次访问 v 的各个未被访问的邻接点之后，再从这些邻接点出发，依次访问 与它们邻接的所有未曾访问过的顶点。重复上述过程，直至图中所有与源点 v 有路径相通的顶点都被 访问过为止。若

G 是连通图则周游完成；否则，在图 G 中选一个尚未访问的顶点作为新源点继续 广度优先搜索。

例如对于图 7.16 所示的有向图 G：首先访问 v_0 和 v_0 的邻接点 v_1，然后访问顶点 v_1 的所有 邻接点 v_2 和 v_3；由于顶点 v_2 和 v_3 的所有邻接点都已经被访问过，选择未被访问过的顶点 v_4 继续广度优先搜索，即访问顶点 v_4 和 v_4 的邻接点 v_5、v_6。

广度优先搜索的过程类似于树的按层次次序周游（5.2.3 节），可以使用 FIFO 队列保存已访问过的 顶点，从而使得先访问的顶点的邻接点在下一轮被优先访问到：在搜索过程中，每访问到一个顶点后将 其入队，当队头元素出队时将其未被访问的邻接点入队，**每个顶点只入队一次**。

广度优先搜索实质上与深度优先相同，只是访问顺序不同而已，**二者的时间复杂度也相同**。

```cpp
[[nodiscard]] std::vector<std::size_t> bfs(std::size_t source) const {
    check_vertex(source);
    std::vector<bool> seen(vertices());
    std::queue<std::size_t> queue;
    std::vector<std::size_t> result;
    queue.push(source);
    seen[source] = true;
    while (!queue.empty()) {
        const std::size_t from = queue.front();
        queue.pop();
        result.push_back(from);
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity && !seen[to]) {
                seen[to] = true;
                queue.push(to);
            }
        }
    }
    return result;
}
```

```python
def bfs(self, source: int) -> list[int]:
    self._check_vertex(source)
    seen = [False] * self.vertices
    queue = [source]
    head = 0
    seen[source] = True
    result: list[int] = []
    while head < len(queue):
        vertex = queue[head]
        head += 1
```

```
            result.append(vertex)
            for target in range(self.vertices):
                if self._adjacency[vertex][target] < self.infinity and not seen[target]:
                    seen[target] = True
                    queue.append(target)
        return result
```

7.4.3 拓扑排序

有向图的边可以看做顶点之间制约关系的描述。在工程实践中，有些工程的进行经常受到一定条件的 约束，例如一个工程项目通常由若干个子工程组成，某些子工程完成之后另一些子工程才能开始。

一个无环的有向图称为**有向无环图**（directed acyclic graph，DAG），常用来描述一个过程或一个系统 的进行过程。对于有向无环图 $G = \langle V, E \rangle$，如果顶点序列满足「存在顶点 v_i 到 v_j 的 一条路径，那么在序列中顶点 v_i 必在顶点 v_j 之前」，顶点集合 V 的这种线性序列称做一个 **拓扑序列**（topological order）；根据有向图建立拓扑序列的过程称为**拓扑排序**（topological sorting）。对有向无环图的顶点进行拓扑排序，对应于实际问题就是对各项子工程排出一个线性顺序关系；如果 条件限制这些工作必须串行，就应该按拓扑次序安排依次执行。**拓扑排序可以解决先决条件问题**，即以某种线性顺序来组织多项任务，以便能够在满足先决条件的情况下逐个完成各项任务。

原书的例子是课程先修关系：顶点是课程，弧 $\langle c_i, c_j \rangle$ 表示 c_i 是 c_j 的先修课。c_0 高等数学、c_1 程序设计基础没有先修课；c_2 离散数学要先修 c_0、c_1；c_3 数据结构要先修 c_1、c_2；c_4 算法语言要先修 c_1；c_5 编译原理要先修 c_3、c_4；c_6 操作系统要先修 c_3、c_8；c_7 普通物理要先修 c_0；c_8 计算机原理要先修 c_7。

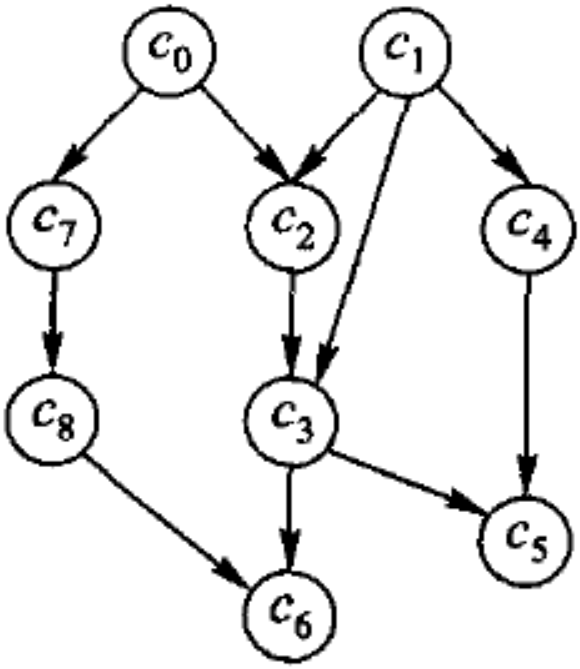


图 7.17　表示课程优先关系的有向无环图。它的拓扑序列可以是 $(c_0, c_1, c_2, c_3, c_4, c_5, c_7, c_8, c_6)$，也可以是 $(c_0, c_7, c_8, c_1, c_4, c_2, c_3, c_6, c_5)$——**拓扑序不唯一**，学生按其中任何一个排课都不会先修颠倒。

拓扑排序的方法是两步的反复：① 从有向图中选出一个没有前驱（入度为 0）的顶点并输出；② 删除图中该顶点和所有以它为起点的弧。不断重复这两个步骤，会出现两种情形：要么有向图中顶点 全部被输出，要么当前图中不存在没有前驱的顶点。当图中的顶点全部输出时，就完成了拓扑排序；**当图中还有顶点没有输出时，说明有向图中含有环——可见拓扑排序可以顺便检查有向图是否存在环**。

以图 7.17 为例：c_0 和 c_1 没有前驱，可以选其中任何一个输出。假设先输出 c_0，删除顶点 c_0 和弧 $\langle c_0, c_2\rangle$、$\langle c_0, c_7\rangle$ 之后，顶点 c_1 和 c_7 没有前驱，则不妨从中选择 c_1 输出，然后删去顶点 c_1 和弧 $\langle c_1, c_2\rangle$、$\langle c_1, c_3\rangle$、$\langle c_1, c_4\rangle$；依次类推，就可以得到有向无环图的拓扑有序序列。

具体实现时，用邻接表作为存储结构，每个顶点中加入一个存放该顶点入度的域（indegree），这样检查 顶点数组就可以方便地找出入度为 0 的顶点；删除该顶点及以它为尾的弧，即将边表中所有弧头顶点的 入度减 1。**为了减少查找入度为 0 的顶点的次数**，可以把入度为 0 的顶点构造成一个队列，使得每次 查找时只要从队列中取出第一个顶点即可，而不必检查整个顶点表；删除入度为 0 的顶点后，如果此时 某个顶点的入度减为 0，就将其插入队列。若最终取出的点数少于顶点数，图里有环。

按这个队列做法、每次先处理编号小的顶点，图 7.17 得到的是 $(c_0, c_1, c_7, c_2, c_4, c_8, c_3, c_5$

初始入度为 0 的 c_0、c_1 先入队；删去 c_0 后 c_7 入度变 0 入队，删去 c_1 后 c_2、c_4 入队；以此类推。这又是一个与图注里两个都不同的合法拓扑序。

代价同样取决于存储结构。用邻接表时，求全部入度扫一遍边表是 $O(n+e)$，每个顶点进出队列一次、每条弧让弧头入度减 1 一次，合计 $O(n+e)$。下面的实现用邻接矩阵，求入度和「删除以它为尾的弧」都要 扫整行，合计 $O(n^2)$。

```cpp
[[nodiscard]] std::optional<std::vector<std::size_t>> topological_sort() const {
    std::vector<std::size_t> indegree(vertices());
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (from != to && adjacency_[from][to] < infinity) {
                ++indegree[to];
            }
        }
    }
    std::queue<std::size_t> queue;
    for (std::size_t vertex = 0; vertex < vertices(); ++vertex) {
        if (indegree[vertex] == 0) {
            queue.push(vertex);
        }
    }
    std::vector<std::size_t> result;
    while (!queue.empty()) {
        const std::size_t from = queue.front();
        queue.pop();
        result.push_back(from);
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (from != to && adjacency_[from][to] < infinity && --indegree[to] == 0)
            ↪ {
                queue.push(to);
            }
        }
    }
    return result.size() == vertices() ?
    ↪ std::optional<std::vector<std::size_t>>(result)
                                       : std::nullopt;
}
```

```python
def topological_sort(self) -> list[int] | None:
    indegree = [0] * self.vertices
    for source in range(self.vertices):
        for target in range(self.vertices):
            if source != target and self._adjacency[source][target] < self.infinity:
                indegree[target] += 1
    queue = [vertex for vertex in range(self.vertices) if indegree[vertex] == 0]
```

```
    head = 0
    result: list[int] = []
    while head < len(queue):
        source = queue[head]
        head += 1
        result.append(source)
        for target in range(self.vertices):
            if source != target and self._adjacency[source][target] < self.infinity:
                indegree[target] -= 1
                if indegree[target] == 0:
                    queue.append(target)
    return result if len(result) == self.vertices else None
```

7.5 最短路径

旅客要从 A 城到 B 城，希望路程最短：顶点是城市、边是交通线、权是里程，问题就是找权值之和最小的那条路径。

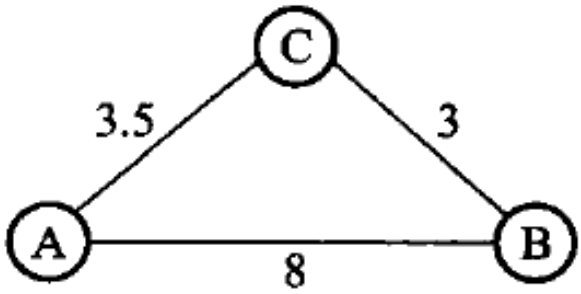


图 7.18　最短路径的示例：从 A 到 B 的最短路径是 A → C → B，**并不是边数最少的那条**。路径上的第一个顶点叫源点 (source)，最后一个叫汇点 (sink)。下面两小节分别讨论单源最短路径和每对顶点之间的最短路径。

7.5.1 单源最短路径

给定一个带权图 $G = \langle V, E \rangle$，其中每条边 (v_i, v_j) 上的权 $W[v_i, v_j]$ 是一个非负实数，另外给定 V 中的一个顶点 s 充当源点；要计算从源点 s 到所有其他各顶点的最短路径，这个问题 通常称为**单源最短路径**（single-source shortest paths）问题。

解决它的一个常用算法是 Dijkstra **算法**，由 E. W. Dijkstra 提出，是一种**按路径长度递增的次序 产生到各顶点最短路径的贪心算法**。基本思想是把图的顶点集合划分成两个集合 S 和 $V - S$：S 表示最短距离已经确定的顶点集，其余顶点放在 $V - S$ 中。初始时 S 只包含源点，即 $S = \{s\}$，此时只有源点到自己的最短距离是已知的。

设 v 是 V 中的某个顶点，把从源点 s 到顶点 v 且**中间只经过集合 S 中顶点**

的路径称为从源点 到 v 的**特殊路径**，并用数组 D 记录当前所找到的、从源点 s 到每个顶点的最短特殊路径长度。D 的初始状态是：如果从 s 到 v 有弧，则 $D[v]$ 记为弧的权值，否则置为无穷大。Dijkstra 算法 每次从尚未确定最短路径长度的集合 $V-S$ 中取出一个最短特殊路径长度最小的顶点 u，将 u 加入 集合 S，同时修改数组 D 中由 s 可达的最短路径长度：若加进 u 做中间顶点使得 v_i 的最短特殊 路径长度变短，则修改 v_i 的距离值（即当 $D[u]+W[u,v_i]<D[v_i]$ 时，令 $D[v_i]=D[u]+W[u,v_i]$）—— 这一步通常称为**松弛**。重复上述操作，一旦 S 包含了所有 V 中的顶点，D 中各顶点的距离值就 记录了从源点 s 到该顶点的最短路径长度。

要输出路径本身，还需要一个 `pre` 域：记录从源点到顶点 v 的最短路径上 v 前面经过的那个顶点。初始时对所有 $v\neq s$ 均设其前一个顶点为 s；更新最短路径长度时，只要 $D[u]+W[u,v]<D[v]$ 就设置 v 的前一个顶点为 u，否则不做修改。算法终止时，顺着 `pre` 一路回溯就得到完整路径。

例如对图 7.19 的有向图，从源点 v_0 出发，每一步的 S、距离 length 和前驱 pre 如表 7.2（原书表 7.2；带 * 的是这一步刚并入 S 的顶点，它的 length 从此不再改变）：

迭代步数	S	v_0	v_1	v_2	v_3	v_4
初始状态	$\{v_0\}$	0 / 0	10 / 0	∞ / 0	30 / 0	100 / 0
1	$\{v_0,v_1\}$	0 / 0	10* / 0	60 / 1	30 / 0	100 / 0
2	$\{v_0,v_1,v_3\}$	0 / 0	10 / 0	50 / 3	30* / 0	90 / 3
3	$\{v_0,v_1,v_3,v_2\}$	0 / 0	10 / 0	50* / 3	30 / 0	60 / 2
4	$\{v_0,v_1,v_3,v_2,v_4\}$	0 / 0	10 / 0	50 / 3	30 / 0	60* / 2

表 7.2　用 Dijkstra 算法的处理过程，源点为 v_0（每格写作「length / pre」）。第 1 步选中 v_1（10 最小），经 v_1 到 v_2 得 $10+50=60$；第 2 步选中 v_3（30），v_2 改为 $30+20=50$，v_4 改为 $30+60=90$；第 3 步选中 v_2（50），v_4 再改为 $50+10=60$；第 4 步选中 v_4，$S=V$，结束。顺着 pre 回溯，到 v_4 的最短路径是 $v_0\to v_3\to v_2\to v_4$，长度 60，比直达边的 100 短。

为什么这样得到的就是最短路径？ 设某一步选中的是 u，假如存在一条从 s 到 u、比 $D[u]$ 更短的 路径。这条路径从 S 出发，最终要到 S 外的 u，所以它一定有第一个落在 $V-S$ 里的顶点，记为 x；把这条路径上 s 到 x、x 到 u 两段的长度分别记为 $d(s,x)$、$d(x,u)$，全长 $d(s,u)=d(s,x)+d(x,u)$。s 到 x 这一段中间只经过 S 中的顶点，是一条特殊路径，故 $D[x]\le d(s,x)$。边权非负，所以 $d(s,x)\le d(s,x)+d(x,u)=d(s,u)<D[u]$，于是 $D[x]<D[u]$——这与「u 是 $V-S$ 中距离值最小的 顶点」矛盾。所以每次并入 S 的 u，它的距离值就是从 s 到 u 的最短路径长度。证明里「$d(x,u)\ge 0$」这一步正是非负权的用处；有负权边时，绕道反而可能更短，结论就不成立了。

本书实现的初始化与上面的描述写法不同、效果相同：`distance` 只把源点置

0、其余置为「到不了」，第一轮必然选中源点本身，松弛它的出边之后，D 就恰好是上面说的初始状态。

代价取决于用什么挑「最小的那个」。对于 n 个顶点 e 条边的图，图中的任何一条边都可能在最短 路径中出现，因此最短路径算法对每条边至少都要检查一次。原书【算法 7.8】采用**最小堆**来选择权值 最小的边，每次改变最短特殊路径长度时需要对堆进行一次重排，时间复杂度为 $O((n+e)\log e)$，**适合于稀疏图**；如果像 Prim 算法那样，通过直接比较 D 数组元素来确定代价最小的边，则需要总 时间 $O(n^2)$，取出顶点后修改最短特殊路径长度共需要 $O(e)$，因此共需 $O(n^2)$，**这种方法适合于 稠密图**。

本节下面的 `Graph::dijkstra` 是后一种：邻接矩阵加 `nearest_unvisited` 逐个比较，$O(n^2)$；堆版本是 7.3.2 节邻接表上的 `GraphList::dijkstra`。那里写的 $O((V+E)\log V)$ 与这里的 $O((n+e)\log e)$ 是同一个量级——没有多重边时 $e \le n^2$，$\log e \le 2\log n$。

边权必须非负。

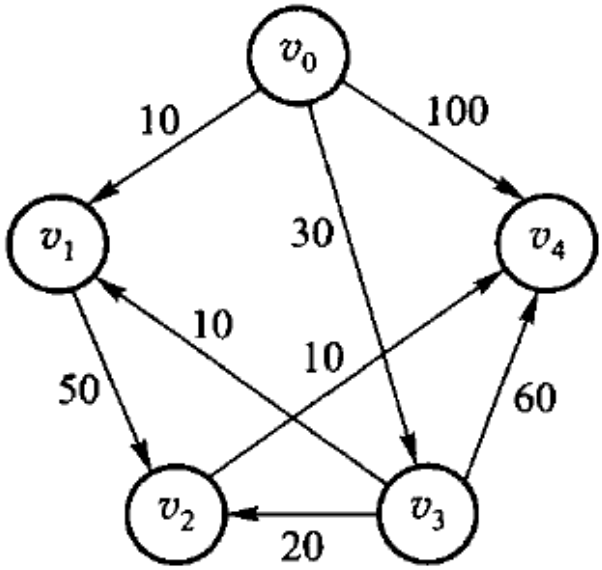


图 7.19　单源最短路径的示例。已确定的顶点集合每轮扩大一个：从「尚未确定」的顶点里挑距离最小的那个，它的距离**此后不会再变小**——因为任何绕道都要先经过一个距离不更小的顶点，而边权非负。这条论证正是 Dijkstra 要求非负权的地方，本书的实现因此在 `add_edge` 就拒绝负权。

```
[[nodiscard]] std::vector<distance_type> dijkstra(std::size_t source) const {
    check_vertex(source);
    std::vector<distance_type> distance(vertices(), unreachable);
    std::vector<bool> used(vertices());
    distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
```

```
        const std::size_t from = nearest_unvisited(distance, used);
        if (from == vertices() || distance[from] == unreachable) {
            break;
        }
        used[from] = true;
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity &&
                distance[to] > distance[from] + adjacency_[from][to]) {
                distance[to] = distance[from] + adjacency_[from][to];
            }
        }
    }
    return distance;
}
```

```
def dijkstra(self, source: int) -> list[int]:
    self._check_vertex(source)
    distance = [self.unreachable] * self.vertices
    used = [False] * self.vertices
    distance[source] = 0
    for _ in range(self.vertices):
        vertex = self._nearest(distance, used)
        if vertex is None or distance[vertex] == self.unreachable:
            break
        used[vertex] = True
        for target in range(self.vertices):
            candidate = distance[vertex] + self._adjacency[vertex][target]
            if self._adjacency[vertex][target] < self.infinity and candidate <
            ↪  distance[target]:
                distance[target] = candidate
    return distance
```

把路径本身取出来

上面的 `dijkstra` 只给距离。要输出路径，就像前文说的那样多记一个 `pre` 域，也就是原书 `Dist` 类里的 `pre`。`Graph::dijkstra_tree` 和 `dijkstra` 是同一个算法，只多了一句：**只在松弛成功时**把 `to` 的前驱设为 `from`。如果不管松没松弛都去改前驱，距离数组照样正确，但顺着前驱走出来的路径和距离对不上，甚至会走成环。源点没有前驱，到不了的顶点也没有前驱，所以前驱数组用 `std::optional`。原书把所有 $v \neq s$ 的 pre 初始化为 s（表 7.2 的初始状态就是这样）；对走得到的顶点两种写法结果相同——第一轮选中源点、松弛它的出边时前驱就被设成了 s——区别只在到不了的顶点上：原书会留下一个看似「从 s 直达」的假前驱，本书留空。

`Graph::shortest_path` 从汇点出发，顺着前驱倒着走回源点，再把序列反转过来。到不了时返回空；汇点就是源点时，路径只有源点一个顶点。前驱链断了或者成环，说明传进来的树已经坏了，这时抛 `std::invalid_argument`。有好几条等长的最短路径时，返回哪一条取决于顶点的扫描次序，所以测试不断言具体路径，只断言路径合法：首尾是源点和汇点，每一步都是图里真实存在的边，边权之和等于 Floyd 算出的最短距离。这项检查在随机小图上跑，边权取 0 到 4，并列和零权边都会大量出现。

关于溢出：边权是 `int`，必须小于 `Graph::infinity`（`INT_MAX/4`，在矩阵里表示「无边」）；路径长度是若干条边之和，用 64 位的 `Graph::distance_type` 存，「到不了」记为 `Graph::unreachable`。一条简单路径至多 $V-1$ 条边，总长小于 $V \cdot 2^{29}$；在 $V < 2^{34}$ 的实际规模内，既不会溢出 64 位，也不会撞上 `unreachable`。早先用 `int` 存距离、以 `infinity` 兼任「到不了」时，一条总长恰好等于 `infinity` 的合法路径会被误读成到不了——同一张图上 BFS 却说走得到（D-039）。

```
/// 原书 Dist 类的 `length` 与 `pre` 两个域：到各点的距离，和最短路径上的前一个顶点。
/// 源点自己、以及到不了的顶点，没有前一个顶点（`std::nullopt`）。
struct ShortestPathTree {
    std::size_t source;
    std::vector<distance_type> distance;
    std::vector<std::optional<std::size_t>> predecessor;
};

/// 与 `dijkstra` 同一个算法，多记一个 `pre`：** 只在松弛成功时 ** 改前驱。
/// 距离与 `dijkstra` 一样是 64 位，到不了是 `unreachable`（D-039）。
[[nodiscard]] ShortestPathTree dijkstra_tree(std::size_t source) const {
    check_vertex(source);
    ShortestPathTree tree{source, std::vector<distance_type>(vertices(),
↪   unreachable),
                          std::vector<std::optional<std::size_t>>(vertices())};
    std::vector<bool> used(vertices());
    tree.distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
        const std::size_t from = nearest_unvisited(tree.distance, used);
        if (from == vertices() || tree.distance[from] == unreachable) {
            break;
        }
        used[from] = true;
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity &&
                tree.distance[to] > tree.distance[from] + adjacency_[from][to]) {
                tree.distance[to] = tree.distance[from] + adjacency_[from][to];
                tree.predecessor[to] = from;
            }
        }
    }
```

```cpp
    return tree;
}

/// 顺着 `pre` 从汇点倒着走回源点，再把序列翻过来。到不了返回 `std::nullopt`；
/// 汇点就是源点时路径是 `{source}`。前驱链断了或成环，说明传进来的树是坏的。
[[nodiscard]] static std::optional<std::vector<std::size_t>> shortest_path(
    const ShortestPathTree& tree, std::size_t target) {
    const std::size_t count = tree.distance.size();
    if (target >= count || tree.source >= count || tree.predecessor.size() != count)
    ↪  {
        throw std::out_of_range("vertex");
    }
    if (tree.distance[target] == unreachable) {
        return std::nullopt;
    }
    std::vector<std::size_t> path{target};
    for (std::size_t vertex = target; vertex != tree.source;) {
        if (!tree.predecessor[vertex] || path.size() > count) {
            throw std::invalid_argument("broken predecessor chain");
        }
        vertex = *tree.predecessor[vertex];
        path.push_back(vertex);
    }
    std::reverse(path.begin(), path.end());
    return path;
}
```

```python
def dijkstra_tree(self, source: int) -> "ShortestPathTree":
    """ 与 dijkstra 同一个算法，多记原书 Dist 的 pre 域：只在松弛成功时改前驱。"""
    self._check_vertex(source)
    distance = [self.unreachable] * self.vertices
    predecessor: list[int | None] = [None] * self.vertices
    used = [False] * self.vertices
    distance[source] = 0
    for _ in range(self.vertices):
        vertex = self._nearest(distance, used)
        if vertex is None or distance[vertex] == self.unreachable:
            break
        used[vertex] = True
        for target in range(self.vertices):
            candidate = distance[vertex] + self._adjacency[vertex][target]
            if self._adjacency[vertex][target] < self.infinity and candidate <
            ↪  distance[target]:
                distance[target] = candidate
                predecessor[target] = vertex
    return ShortestPathTree(source, distance, predecessor)
```

```
    @staticmethod
    def shortest_path(tree: "ShortestPathTree", target: int) -> list[int] | None:
        """ 顺着 pre 从汇点走回源点再翻转；到不了返回 None，汇点即源点时是 [source]。"""
        count = len(tree.distance)
        if not 0 <= target < count or not 0 <= tree.source < count or
        ↪  len(tree.predecessor) != count:
            raise IndexError("vertex")
        if tree.distance[target] == Graph.unreachable:
            return None
        path = [target]
        vertex = target
        while vertex != tree.source:
            previous = tree.predecessor[vertex]
            if previous is None or len(path) > count:
                raise ValueError("broken predecessor chain")
            vertex = previous
            path.append(vertex)
        return path[::-1]
```

7.5.2 每对顶点之间的最短路径

给定带权图 $G = \langle V, E\rangle$，要求对任意的顶点有序对 $\langle v_i, v_j\rangle$ 找出从 v_i 到 v_j 的 最短路径，这个问题通常称为**所有顶点对之间的最短路径**（all-pairs shortest paths）问题。

解决它可以每次以一个顶点为源点、重复执行 Dijkstra 算法 n 次，时间复杂度为 $O(n^3)$。下面介绍 Floyd **算法**，它是一个典型的**动态规划法**：先自底向上分别求解子问题的解，然后由这些子问题的解 得到原问题的解；时间复杂度也是 $O(n^3)$，但形式上比较简单。

Floyd 算法用相邻矩阵 adj 表示带权有向图，基本思想是：初始化 $adj^{(0)}$ 为相邻矩阵 adj，在 $adj^{(0)}$ 上做 n 次迭代，递归地产生一个矩阵序列 $adj^{(1)}, \cdots, adj^{(k)}, \cdots, adj^{(n)}$。其中，**经过第 k 次迭代，$adj^{(k)}[i,j]$ 的值等于从顶点 v_i 到 v_j、中间顶点只取自前 k 个顶点 $v_0, \cdots, v_{k-1}$ 的最短路径长度**。原书说成「路径上所经过的顶点序号不大于 k」，那是把顶点从 1 数起的说法；本书的顶点从 v_0 编号，按「前 k 个」理解才与下面图 7.21 的数字对得上。$adj^{(0)}$ 不允许 任何中间顶点，正是相邻矩阵本身。

推导只有两种情况：进行第 k 次迭代时已求得矩阵 $adj^{(k-1)}$，这一次新开放的中转点是 v_{k-1}。那么从 v_i 到 v_j、中间顶点只取自前 k 个顶点的最短路径，要么**中间不经过顶点** v_{k-1}，此时 $adj^{(k)}[i,j] = adj^{(k-1)}[i,j]$；要么**中间经过** v_{k-1}，此时这条路径由两段组成——从 v_i 到 v_{k-1} 且中间顶点只取自前 $k-1$ 个顶点的最短路径，加上从 v_{k-1} 到 v_j 且中间顶点只取自前 $k-1$ 个顶点的最短路径（最短路径不会两次经过 v_{k-1}，所以两段里都不必再用它）。综合两种情

况：

$$adj^{(k)}[i,j] = \min\{adj^{(k-1)}[i,j],\ adj^{(k-1)}[i,k-1] + adj^{(k-1)}[k-1,j]\}$$

这样 $adj^{(n)}[i,j]$（全部 n 个顶点都可以当中转点）就是所要求的从 v_i 到 v_j 的最短路径长度。

要输出路径本身，可以设置一个 $n \times n$ 的矩阵 $path$：$path[i,j]$ 是由 v_i 到 v_j 的最短路径上 排在 v_j 前面的那个顶点。初始时 $path[i,i] = i$；有弧 $\langle v_i, v_j\rangle$ 时 $path[i,j] = i$；如果当前没有 路径，就将 $path[i,j]$ 置为 -1。第 k 次迭代若经 v_{k-1} 中转使 $adj[i,j]$ 变小，就置 $path[i,j] = path[k-1,j]$——新路径的后半段就是 v_{k-1} 到 v_j 的那段，v_j 前面的顶点自然也就是那段 路上 v_j 前面的顶点。原书正文写的是「当 k 使得 $adj^{(k)}[i,j]$ 达到最小值，就置 $path[i,j] = k$」，这只在中转点到 v_j 的那段恰好是一条弧时才对；原书【算法 7.9】的代码写的是 `D[i][j].pre = D[v][j].pre`，与这里 一致。反例：弧 $v_0 \to v_2$、$v_2 \to v_1$、$v_1 \to v_3$ 权都为 1，第 3 次迭代经 v_2 中转得到 $v_0 \to v_2 \to v_1 \to v_3$，$v_3$ 前面的顶点是 v_1，而「置为中转点」会写成 v_2。

具体到实现：Floyd 枚举中转点 `via`，比较 `from → to` 与 `from → via → to`。测试里五个源点的 Dijkstra 与 Floyd 逐项对拍。

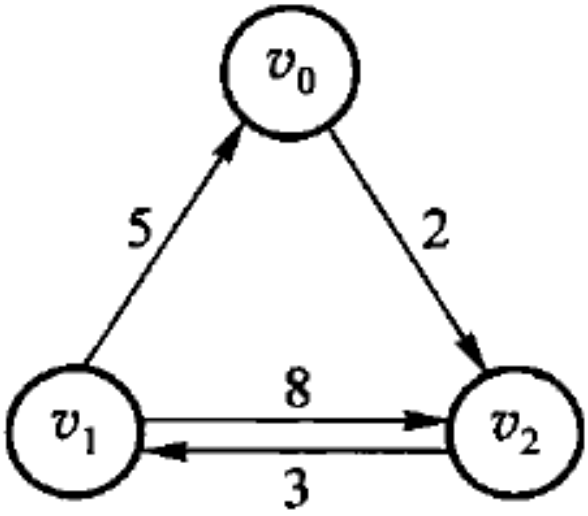


图 7.20　每对顶点间最短路径的示例。Floyd 从相邻矩阵 $adj^{(0)}$ 出发，第 k 轮新允许把 v_{k-1} 当中转点（代码里的 `via` 从 0 数起，即 `via` $= k-1$），得到 $adj^{(k)}$；n 轮之后每一格就是最短路径长度（逐轮结果见下面的图 7.21）。三重循环的次序不能换：**中转点必须在最外层**，否则 $adj^{(k)}$ 还没算完就被拿去用了。

例如图 7.20 的有向带权图（弧 $\langle v_0, v_2\rangle$ 权 2、$\langle v_1, v_0\rangle$ 权 5、$\langle v_1, v_2\rangle$ 权 8、

$\langle v_2, v_1\rangle$ 权 3），$adj^{(0)}$ 的初值和每次迭代的结果如图 7.21 所示（每格写作「adj / $path$」，加粗的是这一轮刚变小的格）：

迭代	新开放的中转点	第 0 行	第 1 行	第 2 行
$adj^{(0)}$	无	0/0　∞/−1 2/0	5/1　0/1 8/1	∞/−1　3/2 0/2
$adj^{(1)}$	v_0	0/0　∞/−1 2/0	5/1　0/1 7/0	∞/−1　3/2 0/2
$adj^{(2)}$	v_1	0/0　∞/−1 2/0	5/1　0/1 7/0	8/1　3/2 0/2
$adj^{(3)}$	v_2	0/0　5/2 2/0	5/1　0/1 7/0	8/1　3/2 0/2

图 7.21　每对顶点间的最短路径的 Floyd 算法迭代过程（原书印成矩阵，这里按行排成表）。第 1 次迭代经 v_0 中转：$v_1 \to v_0 \to v_2$ 长 $5+2=7<8$；第 2 次经 v_1：$v_2 \to v_1 \to v_0$ 长 $3+5=8<\infty$；第 3 次经 v_2：$v_0 \to v_2 \to v_1$ 长 $2+3=5<\infty$。最后一张表里 $path[2,0]=1$、$path[2,1]=2$，所以 v_2 到 v_0 的最短路径倒着读出来是 $v_0 \leftarrow v_1 \leftarrow v_2$，即 $v_2 \to v_1 \to v_0$。

```
[[nodiscard]] std::vector<std::vector<distance_type>> floyd() const {
    std::vector<std::vector<distance_type>> distance(
        vertices(), std::vector<distance_type>(vertices(), unreachable));
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity) {
                distance[from][to] = adjacency_[from][to];
            }
        }
    }
    for (std::size_t via = 0; via < vertices(); ++via) {
        for (std::size_t from = 0; from < vertices(); ++from) {
            for (std::size_t to = 0; to < vertices(); ++to) {
                if (distance[from][via] != unreachable && distance[via][to] !=
                ↪  unreachable) {
                    distance[from][to] = std::min(
                       distance[from][to], distance[from][via] + distance[via][to]);
                }
            }
        }
    }
    return distance;
```

```
}
```

```
def floyd(self) -> list[list[int]]:
    distance = [[weight if weight < self.infinity else self.unreachable for weight
↪   in row]
                for row in self._adjacency]
    for via in range(self.vertices):
        for source in range(self.vertices):
            for target in range(self.vertices):
                if distance[source][via] != self.unreachable and distance[via]
                ↪  [target] != self.unreachable:
                    candidate = distance[source][via] + distance[via][target]
                    distance[source][target] = min(distance[source][target],
↪   candidate)
    return distance
```

7.6 最小生成树

回到本章开始提到的例子：假设要在 n 个城市之间建立通信网络，要在最节省经费的情况下建立这个 网络，则联通 n 个城市只需要 $n-1$ 条线路——如何在这些可能的线路中选择 $n-1$ 条使得总花费 最小，就是**最小生成树问题**。目标不是任意两点最短，而是连通全部顶点且选中边的总权最小。

图 G 的**生成树**是一棵包含 G 的所有顶点的树，树上所有权值总和表示代价；在 G 的所有生成树中，代价最小的生成树称为图 G 的**最小生成树**（minimum-cost spanning tree，MST）。构造最小生成树 有多种算法，本节介绍 Prim 算法和 Kruskal 算法。

Prim 和 Kruskal 都是贪心算法，都靠同一条性质站住脚，MST **性质**：设 U 是顶点集 V 的非空真子集，若 (u,v) 是所有一端在 U、另一端在 $V-U$ 的边里权最小的一条，则一定存在一棵包含 (u,v) 的最小生成树。

反证：假设 G 的**任何一棵**最小生成树都不含 (u,v)，任取其中一棵 T。把 (u,v) 加进 T 必成一个回路；这个回路从 U 里的 u 走到 $V-U$ 里的 v 之后还要回到 u，所以回路上一定另有一条边 (u',v') 同样跨在 U 与 $V-U$ 之间（$u'\in U$，$v'\in V-U$）：

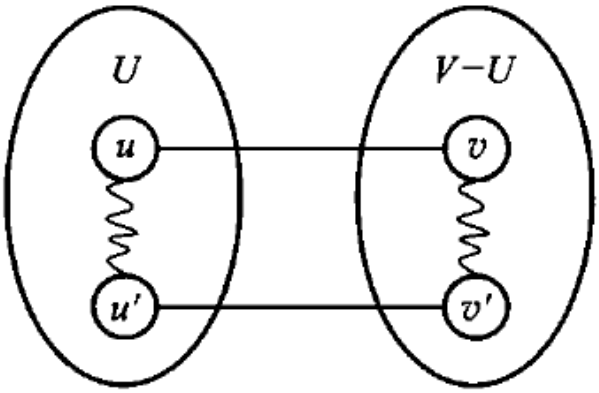


图 7.22　含 (u, v) 的回路。删掉 (u', v') 就消掉了回路，得到另一棵生成树 T'；因为 $W(u, v) \leq W(u', v')$，T' 的代价不比 T 大，于是 T' 也是最小生成树，且含 (u, v)——与假设矛盾。

这条性质就是两个算法「每一步都可以放心地取当前最小的那条跨界边」的依据。下面两小节都拿同一张带权图举例：

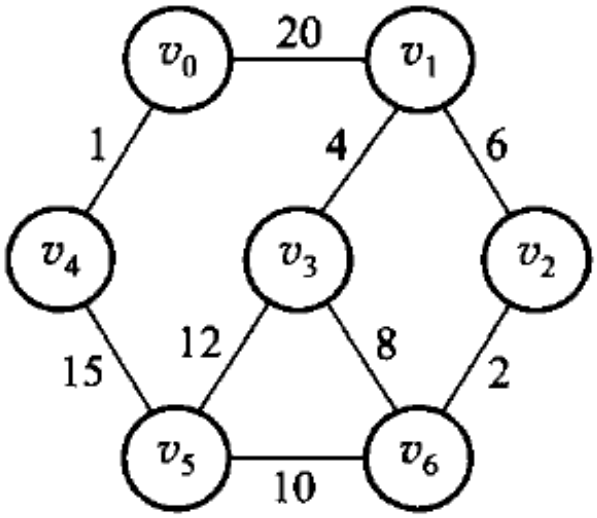


图 7.23　本节两个算法共用的带权图。

7.6.1 Prim 算法

设 $G = \langle V, E \rangle$ 是一个连通的带权图，V 是顶点的集合，E 是边的集合，TE 为最小生成树 的边的集合。则 Prim 算法通过以下步骤得到最小生成树：

1. **初始状态**：$U = \{u_0\}$，$TE = \{\}$，其中 u_0 是顶点集合 V 中的某一个顶点；
2. 在所有 $u \in U$、$v \in V - U$ 的边 $(u, v) \in E$ 中找一条权值最小的边 (u_0, v_0)，将这条边加进 集合 TE 中，同时将此边的另一顶点 v_0 并入 U；

3. 如果 $U = V$ 则算法结束，否则重复步骤 2。

算法结束时 TE 中包含了 G 中的 $n-1$ 条边，经过上述步骤选取到的所有边恰好就构成了图 G 的 一棵最小生成树。具体到实现：从指定源点生长，每次把离当前树最近的顶点加进来；非连通则返回 `nullopt`。

Prim **与** Dijkstra **非常像，差别只有一处**。两者都反复「从集合外挑一个距离值最小的顶点并入 集合」，但 Prim **的距离值不需要累积，直接采用「离集合最近的边距**」，而 Dijkstra 累积的是从源点 起的整条路径长度。时间复杂度也与 Dijkstra 相同：通过直接比较 D 数组元素确定代价最小的边需要 总时间 $O(n^2)$，取出权最小的顶点后修改 D 数组共需要 $O(e)$，因此共需 $O(n^2)$，**适合于稠密图**。

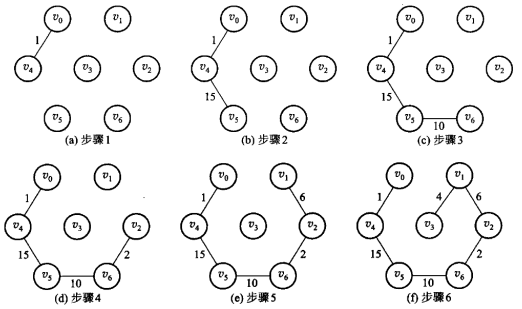


图 7.24　Prim 算法在图 7.23 上的六个步骤（原书印在相邻两页上，(a)(b)(c) 一页、(d)(e)(f) 一页，这里照原样分作两张）。注意每一步长出来的都是**一棵连着的树**——这正是它与 Kruskal 的区别。

逐步对照（从 v_0 出发，「候选」列出此刻所有跨在 U 与 $V-U$ 之间的边）：

步骤	U（并入前）	候选跨界边	选中
1	$\{v_0\}$	(v_0,v_1) 20, (v_0,v_4) 1	(v_0,v_4) 1
2	$\{v_0,v_4\}$	(v_0,v_1) 20, (v_4,v_5) 15	(v_4,v_5) 15
3	$\{v_0,v_4,v_5\}$	(v_0,v_1) 20, (v_5,v_3) 12, (v_5,v_6) 10	(v_5,v_6) 10
4	$\{v_0,v_4,v_5,v_6\}$	(v_0,v_1) 20, (v_5,v_3) 12, (v_6,v_3) 8, (v_6,v_2) 2	(v_6,v_2) 2
5	再加 v_2	(v_0,v_1) 20, (v_5,v_3) 12, (v_6,v_3) 8, (v_2,v_1) 6	(v_2,v_1) 6

步骤	U（并入前）	候选跨界边	选中
6	再加 v_1	(v_5, v_3) 12， (v_6, v_3) 8， (v_1, v_3) 4	(v_1, v_3) 4

6 步选出 $n-1=6$ 条边，总权 $1+15+10+2+6+4=38$。第 2 步取了权 15 的边，是因为此刻跨界边只有 20 和 15 两条——Prim 只看「离当前这棵树最近」，不看全局最小。

```cpp
[[nodiscard]] std::optional<std::vector<Edge>> prim(std::size_t source) const {
    check_vertex(source);
    std::vector<int> distance(vertices(), infinity);
    std::vector<std::size_t> predecessor(vertices());
    std::vector<bool> used(vertices());
    std::vector<Edge> result;
    distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
        const std::size_t from = nearest_unvisited(distance, used);
        if (from == vertices() || distance[from] == infinity) {
            return std::nullopt;
        }
        used[from] = true;
        if (from != source) {
            result.push_back({predecessor[from], from, distance[from]});
        }
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (!used[to] && adjacency_[from][to] < distance[to]) {
                distance[to] = adjacency_[from][to];
                predecessor[to] = from;
            }
        }
    }
    return result;
}
```

```python
def prim(self, source: int) -> list[Edge] | None:
    self._check_vertex(source)
    distance = [self.infinity] * self.vertices
    predecessor = [0] * self.vertices
    used = [False] * self.vertices
    result: list[Edge] = []
    distance[source] = 0
    for _ in range(self.vertices):
        vertex = self._nearest(distance, used)
```

```
        if vertex is None or distance[vertex] == self.infinity:
            return None
        used[vertex] = True
        if vertex != source:
            result.append(Edge(predecessor[vertex], vertex, distance[vertex]))
        for target in range(self.vertices):
            if not used[target] and self._adjacency[vertex][target] <
            ↪  distance[target]:
                distance[target] = self._adjacency[vertex][target]
                predecessor[target] = vertex
    return result
```

7.6.2 Kruskal 算法

构造最小生成树的另一个常用算法是 Kruskal 算法。**它使用的贪心准则是：从剩下的边中选择不会产生回路且具有最小权值的边，加入到生成树的边集中**。

给定含有 n 个顶点和 e 条边的无向连通带权图 $G = \langle V, E \rangle$，Kruskal 算法的构造思想是：首先将 G 中的 n 个顶点看成是独立的 n 个连通分量，这时的状态是有 n 个顶点而无边的森林，可以记为 $T = \langle V, \{\} \rangle$；然后在 E 中选择代价最小的边，**如果该边依附于两个不同的连通分支，那么将这条边加入到 T 中，否则舍去这条边而选择下一条代价最小的边**；依次类推，直到 T 中所有顶点都在同一个连通分量中为止，此时就得到图 G 的一棵最小生成树。

具体到实现：把边按权排序，用并查集跳过会形成环的边。Kruskal **算法的时间复杂度为 $O(e \log e)$，主要取决于边数，因此适合于构造稀疏图的最小生成树**——这正好与适合稠密图的 Prim 算法互补。连通图上两者得到相同总权、`n-1` 条边。

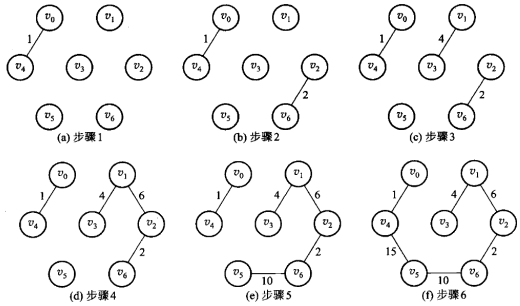


图 7.25　Kruskal 算法在同一张图 7.23 上的步骤（同样跨两页，(a)(b)(c) 与 (d)(e)(f) 各一张）。中间过程是一**片森林**，直到最后才连成一棵树；判断「这条边会不会成环」就是判断两个端点是不是已经在同一棵树里——这正是第 6 章并查集的用处。两个算法走的路不同，得到的总权相同。

把 9 条边按权排好逐条看：

边	权	两端是否已连通	处理
(v_0, v_4)	1	否	取，步骤 (a)
(v_2, v_6)	2	否	取，步骤 (b)
(v_1, v_3)	4	否	取，步骤 (c)
(v_1, v_2)	6	否	取，步骤 (d)
(v_3, v_6)	8	是：$v_3 - v_1 - v_2 - v_6$	舍去
(v_5, v_6)	10	否	取，步骤 (e)
(v_3, v_5)	12	是：$v_3 - v_1 - v_2 - v_6 - v_5$	舍去
(v_4, v_5)	15	否	取，步骤 (f)，已有 $n - 1 = 6$ 条边
(v_0, v_1)	20	是	舍去（实现里照样查一次，结果不变）

选中的 6 条边与 Prim 完全相同，总权同为 38，只是加入的次序不同。

```cpp
[[nodiscard]] std::optional<std::vector<Edge>> kruskal() const {
    std::vector<Edge> edges;
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = from + 1; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity) {
                edges.push_back({from, to, adjacency_[from][to]});
            }
        }
    }
    std::sort(edges.begin(), edges.end());
    std::vector<std::size_t> parent(vertices());
    for (std::size_t vertex = 0; vertex < vertices(); ++vertex) {
        parent[vertex] = vertex;
    }
    std::vector<Edge> result;
    for (const Edge& edge : edges) {
        const std::size_t from_root = find_root(parent, edge.from);
        const std::size_t to_root = find_root(parent, edge.to);
        if (from_root != to_root) {
            parent[from_root] = to_root;
            result.push_back(edge);
        }
    }
   return result.size() + 1 == vertices() ? std::optional<std::vector<Edge>>(result)
                                          : std::nullopt;
}
```

```python
def kruskal(self) -> list[Edge] | None:
    edges = [Edge(source, target, self._adjacency[source][target])
             for source in range(self.vertices) for target in range(source + 1,
             ↪  self.vertices)
```

```
            if self._adjacency[source][target] < self.infinity]
    for end in range(len(edges) - 1, 0, -1):
        for index in range(end):
            if edges[index].weight > edges[index + 1].weight:
                edges[index], edges[index + 1] = edges[index + 1], edges[index]
    parent = list(range(self.vertices))

    def root(vertex: int) -> int:
        while parent[vertex] != vertex:
            parent[vertex] = parent[parent[vertex]]
            vertex = parent[vertex]
        return vertex

    result: list[Edge] = []
    for edge in edges:
        left = root(edge.source)
        right = root(edge.target)
        if left != right:
            parent[left] = right
            result.append(edge)
    return result if len(result) + 1 == self.vertices else None
```

附：把 Kruskal 的过程倒过来用。有一类题目给出一棵带权树，要求补成一个完全图，使这棵树是该完全图**唯**一的最小生成树，并使完全图的总权最小（边权为整数）。思路是按边权从小到大重放 Kruskal 的合并过程。处理树边 (u, v, w) 时，u、v 所在的两个连通分量规模为 s_1、s_2，两个分量之间一共有 $s_1 \cdot s_2$ 对顶点。其中一对由这条树边连着，剩下的 $s_1 \cdot s_2 - 1$ 条边都必须比 w 严格大，否则 Kruskal 可以改选它们，最小生成树就不唯一了。所以这些边最少取 $w + 1$，答案是

$$\sum_{(u,v,w)} \left[w + (w+1)(s_1 s_2 - 1) \right]$$

其中 $s_1 s_2$ 正是第 6 章 `ComponentCounter::connect` 的返回值（见 6.2.5 节）。如果题目只要求「是一棵最小生成树」而不要求唯一，把 $w + 1$ 换成 w 即可。

本章小结

本章介绍了图的基本概念以及图的重要运算，**重点是图的存储结构、图的周游、拓扑排序、最短路径、最小生成树等相关算法**。

首先介绍了图的相关定义和抽象数据类型。图可以分为无向图和有向图，这两种图中含有一些对应的 概念，例如边和弧、度和出度/入度、连通分量和强连通分量等；在图的定义中还给出了子图、带权图等概念。

图有 3 种主要存储结构：相邻矩阵表示法、邻接表表示法和十字链表表示法。

相邻矩阵用数组表示顶点 间相邻关系的矩阵；图的邻接表由顶点表和边表组成，把依附于同一个顶点的边（或弧）存放在同一个边表 中；十字链表是有向图的另一种链式存储结构，可以看做有向图的邻接表和逆邻接表的结合。相邻矩阵 表示法与邻接表表示法有以下差异：

1. **相邻矩阵适合存储稠密图，邻接表适合存储稀疏图；**
2. 求无向图顶点的度时采用这两种存储结构都比较容易，求有向图顶点的**入度**时采用相邻矩阵更方便（原书此处印作「出度」。出度在邻接表里就是边表长度，$O(\deg v)$ 即得；入度在矩阵里是一列之和，$O(n)$，在邻接表里却要扫遍全部边表，$O(n+e)$——见 7.3.2 节逆邻接表的由来）；
3. 判断是否是图中的边，相邻矩阵更容易；
4. 求边数 e，相邻矩阵耗时为 $O(n^2)$、与 e 无关，邻接表的耗时为 $O(n+e)$。

图的周游是指从图中的某一个顶点出发，按照一定的策略访问图中的每一个顶点，使得每一个顶点被访问 且只被访问一次。**图的深度优先搜索（DFS）类似于树的先根次序周游**，采用的搜索方法的特点是尽可能 先对纵深方向进行搜索；**广度优先搜索（BFS）的基本思想是「先被访问的顶点的邻接点」先于「后被访问 的顶点的邻接点」被访问**。

对一个有向无环图进行拓扑排序，是将图中所有顶点排成一个线性序列，使得弧尾在弧头之前出现。算法的实现思想是：反复选择入度为 0 的顶点输出并删除该顶点涉及的所有边。

单源最短路径和所有顶点对之间的最短路径是两类在带权图上寻找最短路径的问题。解决单源最短路径 问题的常用算法是 Dijkstra 算法，求所有顶点对之间的最短路径使用 Floyd 算法；**其中 Floyd 算法是 比较典型的动态规划，而** Dijkstra **算法是贪心算法**。

构造最小生成树的算法有 Prim 算法和 Kruskal 算法。Prim 算法与 Dijkstra 算法类似，具体操作是从 图中的一个顶点开始，把这个顶点包括在 MST 中，然后反复寻找一个顶点已在 MST 中而另一个顶点还不 在 MST 中的最小权边，把新边和新结点加入到生成树中，Prim **算法适合于稠密图**；Kruskal 算法的 构造思想是将 G 中的 n 个顶点看成是独立的 n 个连通分量，然后在边集中选择代价最小且依附于 不同连通分支的边加入到生成树中，Kruskal **算法较适合于稀疏图**。

本章实现使用邻接矩阵，因此当前 Dijkstra 和 Prim 的直接实现是 $O(V^2)$，Floyd 是 $O(V^3)$，空间是 $O(V^2)$。改用邻接表并配合二叉堆之后，稀疏图上的 Dijkstra 达到 $O((V+E)\log V)$—— 这一条现在有实现和实测支撑，见 7.3.2 与 `code/ch07/adjacency_list`。两个结论各自对应 各自的存储结构，不能互相套用。

习题

补充算法设计题（参考课程第 7 章）

1. 在 DAG 中设计关键路径算法，计算每个顶点的最早发生时间和工程总工期。
2. 说明为什么不能给所有边统一加常数后继续使用 Dijkstra；给出负权边反例。

3. 证明 Dijkstra 得到的是最短路树而不一定是最小生成树。

正文习题

1. 分别画出 4 个顶点的无向完全图和有向完全图，并写出边数公式。
2. 对本章 demo 那张有向图，写出从 0 出发的一种 DFS 序列和一种 BFS 序列。
3. 给一个有环的有向图，说明拓扑排序如何发现环。
4. 在边权非负的图上用手算 Dijkstra，列出每一步选出的顶点和距离数组。
5. 说明为什么负权边不能直接用 Dijkstra。
6. 对同一张无向带权连通图分别做 Prim 和 Kruskal，验证边集可以不同、总权必须相同。
7. 邻接矩阵和邻接表各适合什么图、什么运算？给出空间代价。

原书习题

以下是原书第 7 章的习题，本轮按扫描件补回题面；**参考答案见 `book/习题与参考答案.md` 的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。题面里引用的图，一并从扫描件裁了回来。

1. 对于图 7.26 所示的带权有向图：(1) 写出其相邻矩阵；(2) 画出其邻接表表示；(3) 计算每个顶点的 入度和出度；(4) 如果每个指针需要 4 个字节、每个顶点的标号需要 2 个字节、每条边的权需要 2 个 字节，则此图采用哪种表示法所需要的空间较少？

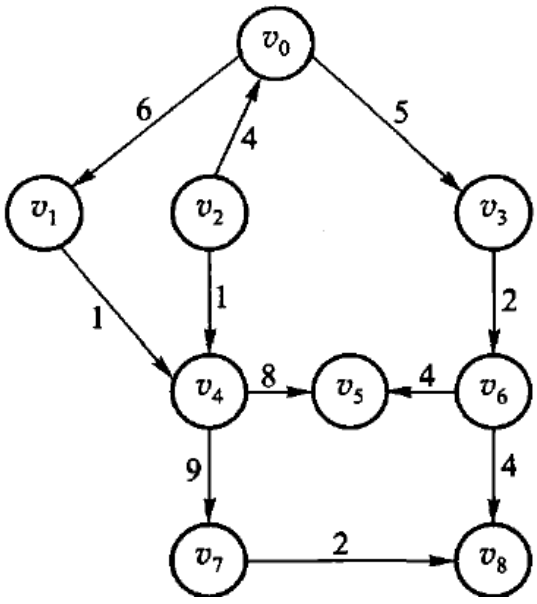


图 7.26　习题 1 的图例

2. 对于图 7.27 所示的有向图，从顶点 v_1 出发分别画出其深度优先搜索（DFS）和广度优先搜索（BFS）的生成森林。

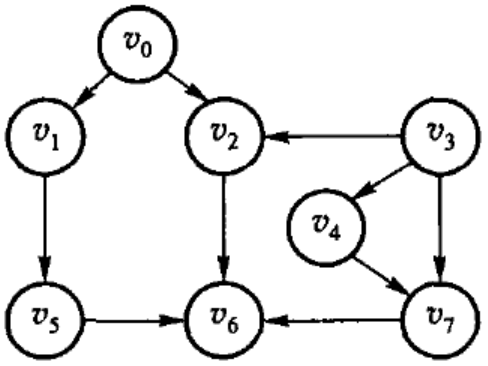


图 7.27　习题 2 的图例

3. 求图 7.28 所示的有向图中从顶点 v_4 到其他各顶点的全部最短路径及长度。

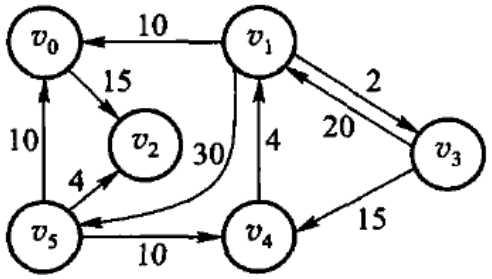


图 7.28　习题 3 的图例

4. **拓扑排序的结果不是唯一的**，对于图 7.29 所示有向图的顶点进行拓扑排序，能够得到多少个不同的 拓扑序列？试输出得到的所有拓扑序列。

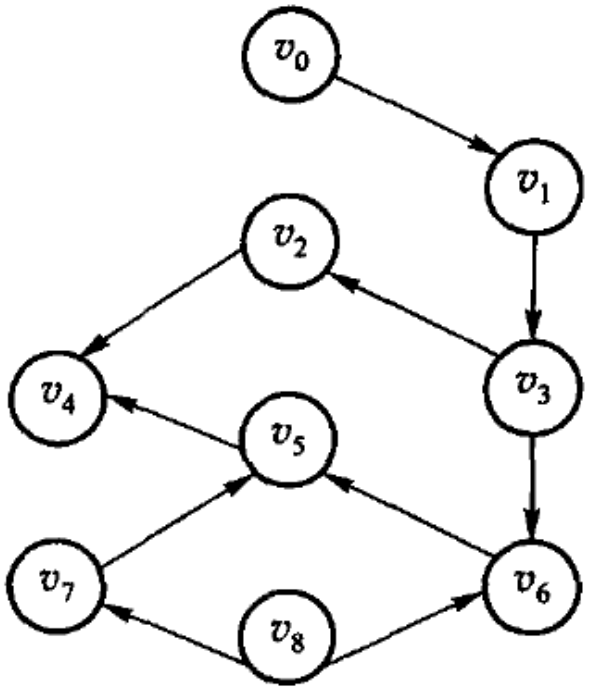


图 7.29　习题 4 的图例

5. 证明：只要适当排列顶点的次序，就能够使有向无环图的相邻矩阵中对角线以下元素全为 0。
6. 证明：(1) 一个没有简单回路的连通无向图 $G = \langle V, E\rangle$ 有 $n-1$ 条边；(2) 一个有 $n-1$ 条边的无环图一定是连通的。
7. 有向图 $G = \langle V, E\rangle$ 的**转置**是图 $G^T = \langle V, E^T\rangle$，其中边 $\langle u, v\rangle \in E^T$ 当且仅当 $\langle v, u\rangle \in E$，即 G^T 就是逆转 G 中所有的方向而 得到的图。试按照相邻矩阵和邻接表两种表示法写出从 G 计算 G^T 的有效算法，并确定算法的时间 复杂度。
8. 有向图 $G = \langle V, E\rangle$ 的**平方图**是图 $G^2 = \langle V, E^2\rangle$，其中边 $\langle u, v\rangle \in E^2$ 当且仅当存在一个顶点 $x \in V$ 使得 $\langle u, x\rangle \in E$ 且 $\langle x, v\rangle \in E$。试按照相邻矩阵和邻接表两种表示法写出从 G 产生 G^2 的有效算法，并确定算法的时间复杂度。
9. 写出一个算法确定一个有 n 个顶点 e 条边的图（有向图或者无向图）是否包含回路，所设计的算法 时间复杂度应该是 $O(n+e)$。
10. 说明并验证你所认为的每对顶点间最短路径问题的**最大可能下限**。
11. 证明：如果图 G 所有边的权值不相等时，它只存在一棵最小生成树（MST）。
12. Dijkstra 最短路径算法是否给出一棵生成树？是否给出一棵最小生成树（MST）？并证明你的结论。
13. 设计算法找图（有向图或无向图）的所有连通分量（对于有向图则是强连通分量）。提示：第一个连通 分量的所有顶点使用第一分量的标记，第二个连通

分量的所有顶点使用第二分量的标记，依次类推。

14. 证明：对于一个无向图 $G = \langle V, E \rangle$，若 G 中各顶点的度均大于或等于 2，则 G 中必有 回路。
15. 设有一个含 n 个顶点的有向无环连通图 G，试问 G 有多少条边?
16. 在有向图中，**源**是一个入度为 0 的顶点，试证明每个有向无环图（DAG）至少有一个源。
17. 什么样的有向无环图（DAG）具有唯一的拓扑排序?
18. 采用相邻矩阵表示一个有向图 G，写出一个算法确定 G 是否含有一个**漏**（即入度为 $n-1$、出度 为 0 的顶点），要求该算法的时间复杂度是 $O(n^2)$。
19. 对于一个具有 n 个顶点和 e 条边的有向图 $G = \langle V, E \rangle$，证明：求其强连通分量的 算法所需的时间复杂度是 $O(n+e)$。
20. 设计一个算法，在图（有向图或者无向图）的邻接表表示的基础上实现边的插入和删除。
21. 只要图中不存在权值为负数的边，Dijkstra 算法就可以使用。**如果一个图存在权值为负数的边，Dijkstra 算法是否继续可用?** 如果你认为不可用，是否有办法改进 Dijkstra 算法使其可用？试证明 你的结论；如果你改进了 Dijkstra 算法，也请证明所做改进的正确性。
22. 设一个带权有向图 $G = \langle V, E \rangle$，v 是 G 的一个顶点，v 的**偏心距**定义为 $\max\{$ 从 u 到 v 的最短路径的长度 $\}$，其中 $u \in V$（**最短路径的长度不是指边数，而是指 路径上的边所带的权的总和**）。将 G 中偏心距最小的顶点称为 G 的**中心**，试设计一个算法求 带权有向图的中心，并确定算法的时间复杂度。
23. 每棵树是一个有向无环图（DAG），但并不是所有 DAG 就是树。试设计一个程序以判断 DAG 是否 是树，并确定程序所采用算法的时间复杂度。
24. 设计算法找有向图的**根**：以一个有向图作为输入，如果它有根，则输出它的所有的根；如果没有，则输出空。试分析算法的时间复杂度。
25. 设计算法找有向无环图（DAG）每对顶点间的「**最长简单路径**」（所谓最长简单路径，是指该简单路径 包含的边数最多）。以一个有向无环图作为输入，对于每对顶点，如果它们之间存在简单路径，则输出 其中路径长度最长（边数最多）的简单路径，否则输出空。试分析算法的时间复杂度。
26. 设计一个程序，输入一个图（有向图或者是无向图）$G = \langle V, E \rangle$ 以及一对顶点 $v_i, v_j \in V$，输出的结果是：如果从 v_i 到 v_j 存在一条简单路径，则输出从 v_i 到 v_j 的 所有简单路径；如果不存在，则输出空。

上机题

1. 读入一个有向图，输出一种拓扑序；有环则报告。
2. 实现 Dijkstra，输出源点到各点的距离和一条最短路径。
3. 实现 Prim 与 Kruskal，对同一组随机连通图比较总权。
4. `code/ch07/adjacency_list` 已经用邻接表重做了 DFS/BFS 并与矩阵版对拍。再往前一步：给它加一个删边接口，并说明为什么删边在邻接表上是 $O(\deg u)$、在矩阵上是 $O(1)$。

原书上机题

原书上机题的参考答案见 book/习题与参考答案.md 的「原书上机题参考」，由 R14 逐题守着。

1. **套汇**是指利用汇率差异将一个单位的货币转换为大于一个单位的同种货币。例如，假设 1 美元兑换 7.51 人民币，1 元人民币兑换 0.07 英镑，1 英镑兑换 2.03 美元,那么如果一个人拿 1 美元先兑换成 人民币,再把人民币兑换成英镑,最后把英镑兑换成美元, 则他最后能够得到 $1\times 7.51\times 0.07\times 2.03=1.07$ 美元，从而获得 $1.07-1=0.07$ 美元的利润，这就是套汇。假设有 n 种货币 $v_1,v_2,\cdots,v_n$ 和有关汇率的 $n\times n$ 矩阵，其中 $A[i,j]$ 是一单位货币 v_i 兑换成货币 v_j 的单位数，要求设计一个程序判断是否存在一个货币序列 $v_{i1},v_{i2},\cdots,v_{ik}$ 使得 $A[i_1,i_2]\times A[i_2,i_3]\times\cdots\times A[i_k,i_1]>1$；如果存在则输出所有这样的货币序列，如果不存在则输出空，并确定算法的时间复杂度。
2. AOE **网络与关键路径**。有向无环图在工程计划和管理系统中具有广泛的应用,是描述一项工程或 系统进行过程的有效工具。如果用一个带权的有向无环图表示这种结构，其中顶点表示事件、有向边 表示活动、弧上的权值表示活动进行的时间，这样的有向图称做 AOE（activity on edge）**网络**。
   利用 AOE 网络可以对工程进行估算，分析完成整个工程至少需要的时间以及哪些活动是影响工程进度 的关键等问题。完成工程的最短时间是从开始点到结束点的最长路径长度，具有最长路径长度的路径 称为**关键路径**（critical path），关键路径上的活动称为**关键活动**：如果不按期完成关键活动就会 影响整个工程的完成时间，**而提前完成那些不在关键路径上的活动也不能加快整个工程的进度**。
   用 $e(i)$ 表示活动 a_i 的最早开始时间，用 $l(i)$ 表示活动 a_i 在不推迟整个工程的前提下允许的 最晚开始时间。若 a_i 对应弧 $\langle v_j,v_k\rangle$、所需时间为 $W(\langle v_j,v_k\rangle)$，则 $e(i)=ee(j)$、$l(i)=le(k)-W(\langle v_j,v_k\rangle)$。$\Delta t=l(i)-e(i)$ 是活动 a_i 的时间余量（机动时间）；**所谓关键活动就是那些 $l(i)=e(i)$ 的活动**。
   为了求活动的 $l(i)$ 和 $e(i)$，先定义 $ee(i)$ 为事件 v_i 的最早发生时间、$le(i)$ 为事件 v_i 允许发生的最晚时间。设网络有 n 个事件 $v_0,v_1,\cdots,v_{n-1}$，v_0 是起点、v_{n-1} 是终点，计算分两步进行：
   (1) 令 $ee(0)=0$，按拓扑序正向递推

$$ee(j)=\max\{ee(i)+W(\langle v_i,v_j\rangle)\},$$

   其中 i 取遍所有使 $\langle v_i,v_j\rangle$ 为边的顶点；
   (2) 令终点 $le(n-1)=ee(n-1)$，按逆拓扑序反向递推

$$le(i)=\min\{le(j)-W(\langle v_i,v_j\rangle)\},$$

其中 j 取遍所有使 $\langle v_i, v_j \rangle$ 为边的顶点。

求 $ee(j)$ 时，v_j 的所有前驱事件必须已经算完；求 $le(i)$ 时，v_i 的所有后继事件必须已经算完。因此事件序列必须按拓扑序处理，反向递推则按逆拓扑序处理。

了解了上面的知识后，试设计一个以文件方式存储事件结点网络的格式，从而实现从一个文件中读入一个 事件结点网络，并求出各活动可能的最早开始时间和允许的最晚完成时间、整个工程的最短完成时间、哪些活动是关键活动、提高哪些活动的速度能够使整个工程提前完成等，并将求出的结果写入到另一个 文件。图 7.30 是一个可以拿来做输入的例子。

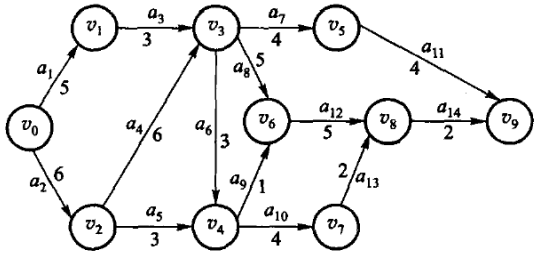


图 7.30　上机题 2 的 AOE 网络图例。v_0 是起点、v_9 是终点。

第 8 章 内部排序

排序把一个序列重排为非递减顺序。阅读时同时问四个问题：比较还是分配？是否稳定？额外空间多少？最好、平均、最坏时间是多少？

源码：全部手写排序、可运行示例、对拍测试。

8.1 排序问题的基本概念

在日常生活中，经常需要对收集到的各种数据信息进行处理，这些数据处理中经常用到的核心运算就是 **排序**（sorting）：图书管理员将书籍按照编号排序放置在书架上；打开资源管理器，可以选择按名称、大小、类型排列图标；搜索引擎把与检索词最相关的页面排在前面返回给用户。

排序主要分为两类。如果待排序的记录个数较少，整个排序过程中所有的记录都可以直接存放在内存中，这样的排序叫做**内排序**（internal sorting）——本章讨论的就是它；如果待排序记录数量太大、内存 无法容纳所有记录，在排序过程中还需要访问外存，这样的排序叫做**外排序**（external sorting），是第 9 章的内容。

几个术语先约定清楚。讨论排序时，习惯上用「记录」（record）或「元素」（element）代替前 7 章的「结点」（node）；记录是进行排序的基本单位，把所有待排记录称为「序列」（sequence），而不用「线性表」。每一个记录内都有一个**排序码**（sort key）域作为排序运算的依据。**注意排序码不一定是关键码**：关键码是唯一确定记录的一个或多个域；如果排序码不是关键码，就可能有多个记录具有同一个排序码，因此排序结果可能不唯一。为了简便，本章假设排序码为整数。

给定序列 $R = \{r_0, r_1, \cdots, r_{n-1}\}$,其排序码分别为 $k = \{k_0, k_1, \cdots, k_{n-1}\}$，排序后 形成新序列 R'，对应排序码满足 $k'_0 \le k'_1 \le \cdots \le k'_{n-1}$（**不减序列**）或 $k'_0 \ge k'_1 \ge \cdots \ge k'_{n-1}$（**不增序列**）；没有重复排序码时，前者称**升序**、后者称**降序**。如果待排序的序列正好符合排序要求，则称「**正序**」序列；如果把它逆转过来正好符合要求，则称「**逆序**」序列。本章的排序算法一般对记录数组进行不减排序。

稳定性。如果存在多个具有相同排序码的记录，经过排序后这些记录的相对次序仍然保持不变，这种 排序算法称为「**稳定的**」（stable），否则称为「不稳定的」。有些应用要求尽量不改变具有相同排序码的 记录的原始输入顺序，这时就需要采用稳定的排序算法——例如 8.6.2 节的低位优先基数排序 LSD，**第一趟分配和收集以后的其他排序步骤都要求采用稳定排序算法**，否则前面几趟的成果会被后面的 趟数打乱。

读每一种方法时同时问四件事：它靠元素之间比较，还是靠关键码的数字结构做分配？相等的键排完之后相对次序变不变（稳定）？除了原序列还要多少额外空间？最好、平均、最坏各是什么时间？「稳定」不是装饰：按成绩排序时，两名同分学生若仍按原来的姓名顺序出现，后续按姓名再排才有意义。

怎么衡量代价。评价一种排序算法的好坏主要通过空间代价和时间代价两方面，尤其是时间代价；如果有特殊的空间限制，则要注意采用辅助空间较小的算法。时间代价一般通过**记录的总比较次数和 总移动次数**来衡量：一次移动就是一次记录赋值，**一次** `swap` **交换算三次移动**——这条换算在 8.7.2 节解释「同样是 $\Theta(n^2)$，冒泡为什么比插入慢几十倍」时会再用到。记录的数量、排序码和记录的 大小、以及输入记录的原始有序程度，都会影响排序算法的相对运行时间。估算时间代价时需要分别 考虑 3 种情况：最小时间代价（最佳情况）、最大时间代价（最差情况）以及平均时间代价（平均情况）。**当序列中不含重复记录时，一个长度为 n 的序列共有 $n!$ 种排列；假定每种排列的出现都是等概率的，所有可能排列的平均运行时间就是排序算法的平均时间代价**。

本章的分类沿用原书：把直接插入排序、Shell 排序归为插入排序类（8.2）；把冒泡排序和快速排序 归为交换排序（8.4）；把直接选择排序、堆排序以及第 9 章的选择树归并排序划为选择排序（8.3）；分配排序中介绍桶排序、基数排序以及索引排序（8.6）。另一种常见分类是把插入、直接选择、冒泡归为「简单排序」，把快速排序和归并排序归为「分治排序」。

比较排序在最坏情况下至少需要 $\Omega(n \log n)$ 次比较，这是判定树给出的下界。插入、选择、冒泡是 $O(n^2)$；堆和归并最坏也是 $O(n \log n)$；快排平均 $O(n \log n)$，但取首/末元素作轴时（本书的实现就是取末元素），已经有序或全部相等的输入会让它退化成 $O(n^2)$。计数和基数不是比较排序，代价取决于值域或位数。

方法	平均时间	稳定性	主要特点
直接插入	O(n²)	稳定	小规模、近乎有序时简单有效
冒泡	O(n²)	稳定	教学直观，通常不作为通用选择
选择	O(n²)	不稳定	交换次数少
堆排序	O(n log n)	不稳定	最坏情况也有保证，原地排序
快速排序	O(n log n) 平均	不稳定	实务常用，坏划分时会退化
归并排序	O(n log n)	稳定	需要 O(n) 辅助空间
计数/基数	与值域或位数有关	可稳定	适合整数键，不是比较排序

本章所有实现都接受有符号整数，测试含重复值和负数；没有用 `std::sort` / `std::make_heap` 顶替任何一个手写算法——那样等于把这一章删掉。

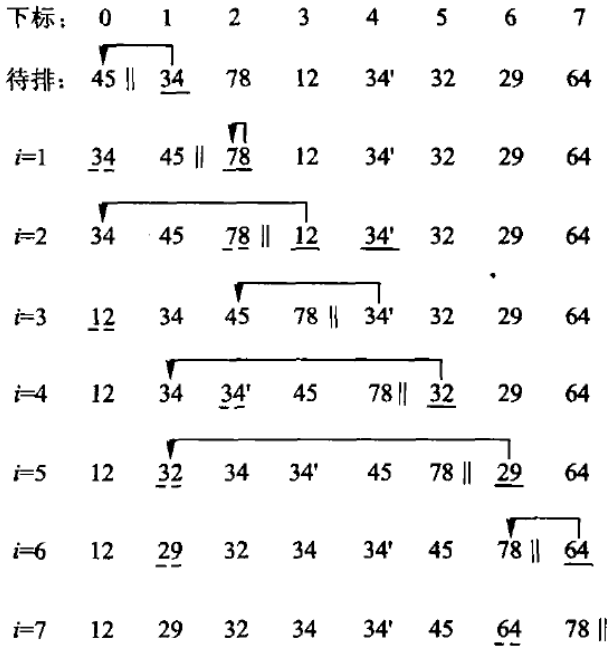
8.2 插入排序

先把四种排序跑一遍，看看它们交出什么。

插入排序（insert sorting）的算法思想十分简单：对待排序的记录逐个进行处理, 每个新记录与同组 那些已排好序的记录进行比较, 然后插入到适当的位置。关键在于如何将一个新记录 r_i 插入到已排序 序列 R' 中, 这涉及两个方面——**找到序列中应插入的位置**，以及**如何移动序列中那些已排好序的 元素以便插入新**

记录。插入排序的变种很多，例如本章上机题里的二分插入排序和交换插入排序。

8.2.1 直接插入排序




```cpp
#include "modern.hpp"

#include <iostream>
#include <vector>

namespace {
void print(const char* name, const std::vector<int>& values) {
    std::cout << name;
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << '\n';
}
}  // namespace

int main() {
    const std::vector<int> raw{3, -2, 7, 3, 0, -2, 9, 1};

    auto insertion = raw;
    dsa::sorting::insertion_sort(insertion);
    print(" 插入:", insertion);
```

```
    auto heap = raw;
    dsa::sorting::heap_sort(heap);
    print(" 堆排:", heap);

    auto quick = raw;
    dsa::sorting::quick_sort(quick);
    print(" 快排:", quick);

    auto radix = raw;
    dsa::sorting::radix_sort(radix);
    print(" 基数:", radix);
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch08/sorting \
    code/ch08/sorting/demo.cpp -o /tmp/sort-demo
/tmp/sort-demo
```

```
插入: -2 -2 0 1 3 3 7 9
堆排: -2 -2 0 1 3 3 7 9
快排: -2 -2 0 1 3 3 7 9
基数: -2 -2 0 1 3 3 7 9
```

四种算法交出同一条非递减序列。插入排序保持两个 `-2`、两个 `3` 的相对次序；堆排和快排不保证这一点。

直接插入排序通过线性搜索来确定待插入记录的位置。如果前面已经有若干个记录排成了不减序列，则对已排序记录按照从大到小依次逐个与新记录进行比较，直到找到第一个不大于新记录的值，这就是新 记录应该插入的位置；依次把新记录插入到逐步扩大的已排序子序列中，直到最后完全排好序。

对待排序序列 $R = \{r_0, r_1, \cdots, r_{n-1}\}$，插入排序按照下述步骤对 R 进行排序，排完序后的 有序序列为 $R' = \{r'_0, r'_1, \cdots, r'_{n-1}\}$：(1) 首先记 R' 为只含一个记录 r_0 的序列，此时 $r'_0 = r_0$；(2) 将记录 r_1 与序列 R' 中的 r'_0 进行比较，将 r_1 插入到序列中适当的位置—— 如果 $r_1 < r'_0$ 就将 r_1 插到 r'_0 前面，否则 r_1 和 r'_0 的相对位置保持不变；(3) 插入 r_i 时 $(1 < i < n)$，$\{r'_0, r'_1, \cdots, r'_{i-1}\}$ 已经有序，**将 r_i 暂时保存起来、腾出该 记录所占的数组元素位置**，从右向左依次将 r_i 与前面 j 位置的记录 r'_j 比较：如果 $r_i < r'_j$ 则 r'_j 向后移动一位；如果 $r_i \geq r'_j$ 则找到了正确位置，直接将 r_i 放到 $j+1$ 的位置上。最后得到 一个已经排好序的 R'。

具体到实现：直接插入把 `values[index]` 抽出来，向前挪动所有比它大的元素，把空位留给它。相等元素不越过彼此，所以**稳定**——每次插入只与临近记录逐个比较，直到找到第一个不大于新记录的值就停止；如果前面有 等于新记录排序码的记录，该记录仍然会在前方位置。图 8.1 中 $i = 4$ 那一行就没有改变两

个 34 （34 和 34′）的原始顺序，直到排序结束二者的相对顺序也没有改变。

代价分析。算法用到一个辅助存放待插入记录的临时变量，因此**空间代价是一个记录大小，即** $\Theta(1)$。时间代价复杂一些：程序体由两重循环组成，外层循环迭代 $n-1$ 次；内层循环的开始和 结束有两个记录的移动操作，迭代次数依赖于「在第 i 个记录前的 $i-1$ 个记录中有多少个小于第 i 个 记录」。

- **最佳情况**：数组正序排列时，每次第 i 个记录一进入内层循环就退出，迭代次数为 0，比较次数为 $n-1$；当前记录保存在临时变量中 $n-1$ 次、回填 $n-1$ 次，移动次数共为 $2(n-1)$。因此最小时间 代价为 $\Theta(n)$。
- **最差情况**：数组恰好按降序排列（逆序）。外层第 i 次迭代中内层需要进行 i 次循环、比较 i 次；当前记录存入临时变量 1 次移动、回填 1 次移动，前面序列顺序向后移动 i 次，共 $i+2$ 次。总比较次数为 $\sum_{i=1}^{n-1} i = n(n-1)/2 = \Theta(n^2)$，总移动次数为 $\sum_{i=1}^{n-1}(i+2) = (n-1)(n+4)/2 = \Theta(n^2)$。

平均情况要靠「逆置」来数。考虑序列 R 的一个排列 $P = \{p_0, p_1, \cdots, p_{n-1}\}$：如果 P 中 两个元素 p_i 和 p_j 满足 $p_i > p_j$ 但 $i < j$，那么 (p_i, p_j) 称为一个**逆置**（inversion）。P 的全逆置序列为 $P' = \{p_{n-1}, p_{n-2}, \cdots, p_0\}$。当 R 中不含重复元素时 P 共有 $n!$ 种排列；把 P 中的元素两两组成对，共有 $n(n-1)/2$ 个元素对，每个元素对都可能构成 P 或者 P' 的一个 逆置，因此按每个元素对出现的几率相等的原则，**平均有一半的逆置出现在** P **中，即** P **中平均有** $n(n-1)/4$ **个逆置**。

处理第 i 个记录时，内层循环的迭代次数依赖于该记录前面比它大的记录个数，也就是逆置的数目—— 例如图 8.1 中处理到 $i=3$ 时，记录 12 前面有 3 个比它大的数，存在 3 个逆置。**逆置的数目决定了 比较及移动的次数**，因此计算平均时间代价也就是要计算整个数组中平均有多少个逆置：按上面的推理，平均时间代价为 $n(n-1)/4$，即 $\Theta(n^2)$。

```cpp
// 算法 8.1：直接插入排序。相等元素不越过彼此，故稳定。
inline void insertion_sort(std::vector<int>& values) {
    for (std::size_t index = 1; index < values.size(); ++index) {
        const int value = values[index];
        std::size_t hole = index;
        while (hole != 0 && value < values[hole - 1]) {
            values[hole] = values[hole - 1];
            --hole;
        }
        values[hole] = value;
    }
}
```

本书讲**算法**的章节同时给一份 Python 实现，讲**存储管理**的章节只有 C++（D-025）。两份不是逐行翻译：策略是同一份，代价不是同一份——差在哪里，每处都会点名。

```python
# 算法 8.1：直接插入排序。相等元素不越过彼此，故稳定。
def insertion_sort(values: list[int]) -> None:
    for index in range(1, len(values)):
        value = values[index]
        hole = index
        # `hole > 0` 这个条件在 Python 里是 ** 承重的 **，不是防御性写法：
        # C++ 版越界会被 ASan 当场抓住，Python 的 values[-1] 却合法——
        # 它悄悄环绕到最后一个元素，把排序结果搅乱而不报任何错。
        while hole > 0 and value < values[hole - 1]:
            values[hole] = values[hole - 1]
            hole -= 1
        values[hole] = value
```

`hole > 0` 这个条件在 Python 里是**承重的**，不是防御性写法。C++ 版写漏了，下标越界会被 AddressSanitizer 当场抓住；Python 的 `values[-1]` 却完全合法——它环绕到最后一个元素，把结果悄悄排错而不报任何错。同一处笔误，一种语言当场崩，另一种语言交出一个看起来正常的错答案。

8.2.2 Shell 排序

直接插入的代价几乎全花在「一次只能挪一格」上：一个很小的元素落在末尾，就得一路挪回 $n-1$ 次。

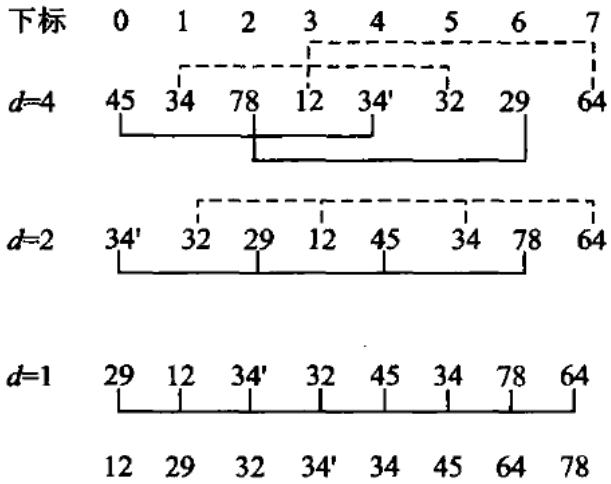


图 8.2　Shell 排序。同一种底纹标出的是同一个子序列。

8.2.1 节已经介绍过两条性质：初始序列为正序时，插入排序的最佳时间代价为 $\Theta(n)$；另外，对于 短序列的情况插入排序也比较有效。Shell **排序**（D. L. Shell，1959）**有效地利用了插入排序的这两个 性质**。

与直接插入排序不同的是，Shell 排序不是着眼于相邻记录之间的比较，而是**对那些不相邻的记录进行 比较和移动**：先将待排序序列分为若干个子序列，而且要保证子序列中的记录在原始数组中不相邻、且间距相同，分别对这些小子序

列进行插入排序；然后减少记录间的间距、减少小序列个数，将原始序列 分为更大、更有序的子序列，分别进行插入排序；重复下去，直到最后间距减少为 1（整个序列比较接近 于正序状态），然后对整个序列进行插入排序——此时序列已经**基本有序**，插入排序在这种输入上接近 $\Theta(n)$。

由于 Shell 排序按照不断缩小的增量来将原始序列分成若干个子序列，因此有时也称为**缩小增量排序法**（diminishing increment sorting）。本书按原书取「增量每次除以 2 递减」：代码里增量从 $\lfloor n/2 \rfloor$ 开始、每趟整除 2，直到 1；$n = 2^{k+1}$ 时增量序列就是 $(2^k, 2^{k-1}, \cdots, 2, 1)$。仍用上面那个序列（$n = 8$），两个 34 用 34 和 34′ 区分：

```
初始       45  34  78  12 34′  32  29  64
gap=4     34′  32  29  12  45  34  78  64
gap=2      29  12 34′  32  45  34  78  64
gap=1      12  29  32 34′  34  45  64  78
```

gap=4 那一趟把序列分成 4 个各含两个元素的子序列 {45,34′}、{34,32}、{78,29}、{12,64}，各自排好：29 从下标 6 一步搬到下标 2，34′ 从下标 4 一步搬到下标 0——**一步跨四格**，这是直接插入做不到的。代价是稳定性：34 与 34′ 落在不同的子序列里，34′ 在第一趟就越过了 34，最终结果里 34′ 排在 34 前面，与输入次序相反。Shell **排序不稳定**，原因正是「跨越式移动」，而稳定性恰恰依赖「只与相邻元素比较、绝不跳过相等元素」。

```cpp
// 算法 8.2：增量每次减半的 Shell 排序。
inline void shell_sort(std::vector<int>& values) {
    for (std::size_t gap = values.size() / 2; gap != 0; gap /= 2) {
        for (std::size_t index = gap; index < values.size(); ++index) {
            const int value = values[index];
            std::size_t hole = index;
            while (hole >= gap && value < values[hole - gap]) {
                values[hole] = values[hole - gap];
                hole -= gap;
            }
            values[hole] = value;
        }
    }
}
```

增量序列的选择直接决定复杂度，这是一个至今没有完全解决的问题：减半序列最坏 $\Theta(n^2)$，Hibbard 序列 $1, 3, 7, \ldots, 2^k - 1$ 最坏 $\Theta(n^{3/2})$，实测常用的 Sedgewick 序列更好。本书取减半，是因为它最容易讲清楚「增量」这件事本身。

为什么「除以 2 递减」效果有限。 Shell 排序的效率比直接插入排序要高——整个排序过程中，第一轮 循环中逆置个数比较多，最后几轮循环中子序列都是基本有序的，因此效率比起直接对整个数组做插入 排序要好。但是当选取「增量

每次除以 2 递减」时，**效率仍然为** $\Theta(n^2)$，并没有多大效果。问题 就在于这样选取的增量之间**并不互质**：间距为 2^{k-1} 的子序列都是由那些间距为 2^k 的子序列组成 的，而实际上上一轮循环中这些子序列都已经排序过，导致后面的处理效率不高。最坏情况下，所有比较 大的记录下标都是奇数、比较小的记录下标都是偶数：由于每次分出的子序列要么下标都是奇数、要么 都是偶数，因此每次对子序列排序后，大数仍然在奇数位置上、小数仍然在偶数位置上，整个序列没有达到基本有序状态，导致最后一次对整个序列做直接插入排序时效率大幅降低。

换增量序列能换来质变。Hibbard 提出了一种新的增量序列 $\{2^k-1, 2^{k-1}-1, \cdots, 7, 3, 1\}$，推理证明这种选取方式下 Shell 排序的效率可以达到 $\Theta(n^{3/2})$，而模拟实验表明甚至可以达到 $\Theta(n^{5/4})$，只是尚未得到理论上的证明；「增量每次除以 3 递减」的效率也是 $\Theta(n^{3/2})$。事实上，选取其他增量序列还可以进一步减少时间代价，甚至有的增量序列可以达到 $\Theta(n^{7/6})$，很接近 $\Theta(n \log n)$，比直接插入排序要快得多。

注意两条边界。一是**最后一趟的增量必须是** 1：如果采用其他增量序列，一定要注意人为地加上一个 间距为 1 的增量元素，否则整个序列不能保证有序。二是 Shell **排序不稳定**：子序列互相交错，而且 跨度比较大——上面那个例子里 34 和 34′ 的排序结果就是不稳定的。空间代价仍是 $\Theta(1)$。

8.3 选择排序

选择排序（selection sorting）的算法思想是逐个找出第 i 小的记录，并将其放到数组的第 i 个位置。**关键在于如何从剩余的未排序记录中找出最小（或最大）的那个记录**：本节介绍简单的线性查找方法（直接选择排序）和基于二叉树堆进行选择的方法（堆排序）。

8.3.1 直接选择排序

选择排序的思路是逐个「选出第 i 小的记录，一次交换到位」：第 i 轮从剩下的 $n-i$ 个记录里线性扫描出最小的那个，与下标 i 上的记录交换。

```cpp
// 算法 8.3：直接选择排序。
inline void selection_sort(std::vector<int>& values) {
    for (std::size_t first = 0; first < values.size(); ++first) {
        std::size_t minimum = first;
        for (std::size_t index = first + 1; index < values.size(); ++index) {
            if (values[index] < values[minimum]) minimum = index;
        }
        using std::swap;
        swap(values[first], values[minimum]);
    }
}
```

仍用原书图 8.3 的那个序列（两个 34 记作 34 和 34′，用来看稳定性）：

```
初始       45  34  78  12 34′  32  29  64
i=0        12  34  78  45 34′  32  29  64    12 与下标 3 交换
i=1        12  29  78  45 34′  32  34  64    29 与下标 6 交换
i=2        12  29  32  45 34′  78  34  64    32 与下标 5 交换
i=3        12  29  32 34′  45  78  34  64    34′ 与下标 4 交换
i=4        12  29  32 34′  34  78  45  64    34 与下标 6 交换
i=5        12  29  32 34′  34  45  78  64    45 与下标 6 交换
i=6        12  29  32 34′  34  45  64  78    64 与下标 7 交换
```

（i=7 那一轮内层一次也不比较，`minimum == first`，代码里的 swap 是一次自己与自己的交换，排列不变，表中省去。）

它不稳定，i=3 那一轮（第 4 轮）就看得出来：扫描是从前往后的，34′（下标 4）先于 34（下标 6）被选中，于是排完是 34′ 34，与输入次序相反。更本质的原因是那次交换的**跨度很大**—— i=1 那一轮把 34 从下标 1 一脚踢到了下标 6，越过了 34′。稳定的排序必须避免这种远距离交换。

代价分析很干脆：外层 $n-1$ 轮，第 i 轮内层比较 $n-1-i$ 次，

$$\sum_{i=0}^{n-2}(n-1-i)=\frac{n(n-1)}{2}=\Theta(n^2)$$

而且与输入顺序无关——已经排好序的输入照样扫这么多次，所以最好、平均、最坏都是 $\Theta(n^2)$。这一点和插入排序、冒泡排序都不同，后两者在有序输入上是 $\Theta(n)$。

但它有一个别人没有的优点：**真正改变位置的交换最多只有 $n-1$ 次**（每轮 1 次交换、3 次移动）。注意本书代码每轮都无条件调用一次 swap，`minimum == first` 时就是一次白做的自交换—— 原书习题 5 讨论的正是这一点。当记录很大（比如每条记录几百字节）而比较很便宜时，移动开销才是主导，这时选择排序反而合适。想把移动降到更低，见 8.6.3 节的索引排序。空间代价 $\Theta(1)$。

8.3.2 堆排序

直接选择排序每一轮都从头线性扫描剩余记录，**没有利用前一轮查找已经得到的比较信息**。堆排序（heapsort）用堆来保存「剩余记录之间谁大谁小」的信息，因而是更有效的选择排序。

这里用**最大堆**：每个父结点的值都大于或等于它的子结点，所以堆顶就是剩余记录中的最大者，次大者一定在堆顶的左或右孩子里。堆排序分两步：

1. 对所有记录建立最大堆（与 5.5 节的最小堆对称，只是把比较方向反过来）；
2. 把堆顶的最大记录与堆的末端记录交换——最大记录落在下标 $n-1$，原末端记录临时处于根；再把根向下筛选（`sift_down`），剩下的 $n-1$ 个记录重新成堆；然后把新堆顶与下标 $n-2$ 交换..... 不断重复，直到堆里只剩一个记录。这时数组正好按从小到大排好。

筛选时 `sift_down` 必须比较左右两个孩子、与**较大**的那个交换，否则换上来的孩子可能比它的兄弟小，堆性质就破了。

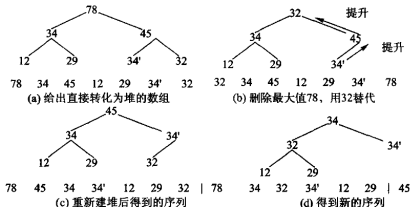


图 8.4　堆排序：(a) 待排序列建成最大堆；(b) 取走最大值 78，用末尾的 32 填上；(c) 重新筛选后又是一个堆；(d) 重复下去，每取一次就把一个最大值放到已排好的那一段。建堆是 $O(n)$（5.5.1a 证过），此后 $n-1$ 次筛选各 $O(\log n)$，合计 $O(n\log n)$——**而且不需要额外数组**。（扫描件 (c) 那一行开头多印了一个 78，正确的序列是 `45 34 34′ 12 29 32 | 78`。）

原书的待排数组是 $\{45, 29, 32, 12, 34, 34', 78\}$，按上面的代码建堆得到图 8.4(a) 的 $\{78, 34, 45, 12, 29, 34', 32\}$。此后每一趟「堆顶与堆尾交换、再从根筛选」的结果如下（竖线右边是已经排好的部分）：

```
建堆后      78  34  45  12  29 34′  32
第 1 趟       45  34 34′  12  29  32 | 78          图 8.4(c)
第 2 趟       34  32 34′  12  29 | 45  78          图 8.4(d)
第 3 趟      34′  32  29  12 | 34  45  78
第 4 趟       32  12  29 | 34′  34  45  78
第 5 趟       29  12 | 32  34′  34  45  78
第 6 趟       12 | 29  32  34′  34  45  78
```

第 2 趟筛选时根的两个孩子是 34 和 34′，相等，代码只在左孩子**严格小于**右孩子时才选右孩子，于是 34 被提上来；下一趟第三个出堆的就是 34，它被放到了 34′ 后面。最终结果是 `12 29 32 34′ 34 45 78`——输入里 34 在 34′ 之前，排完反了过来。**堆排序不稳定**：建堆和筛选都是父子之间的远距离移动，保证不了两个相等记录的原始次序。

代价分析。建堆 $\Theta(n)$，此后 $n-1$ 次「删除堆顶、重新筛选」每次 $O(\log n)$，总时间 $\Theta(n)+O(n\log n)=O(n\log n)$；关键码互不相同时，最佳、最差、平均都是 $\Theta(n\log n)$（全部相等这种极端输入下每次筛选立即返回，只要 $\Theta(n)$）。交换只用一个临时记录，空间代价 $\Theta(1)$。堆排序的优点是建堆很快、只需建一次就能反复取用，且最坏情况也有 $O(n\log n)$ 的保证；第 9 章还会看到堆在外排序（置换选择）里的用处。

```cpp
// 算法 8.4：手写最大堆筛选与堆排序，不委托 std::make_heap/sort_heap。
inline void sift_down(std::vector<int>& values, std::size_t root, std::size_t count)
↪  {
    while (root * 2 + 1 < count) {
        std::size_t child = root * 2 + 1;
```

```
        if (child + 1 < count && values[child] < values[child + 1]) ++child;
        if (values[root] >= values[child]) return;
        using std::swap;
        swap(values[root], values[child]);
        root = child;
    }
}

inline void heap_sort(std::vector<int>& values) {
    for (std::size_t root = values.size() / 2; root != 0; --root) {
        sift_down(values, root - 1, values.size());
    }
    for (std::size_t end = values.size(); end > 1; --end) {
        using std::swap;
        swap(values[0], values[end - 1]);
        sift_down(values, 0, end - 1);
    }
}
```

Python 版同样手写筛选，不借 heapq：

```
# 算法 8.4：手写最大堆筛选与堆排序，不借 heapq——那一行就是 5.5 节全节。
def sift_down(values: list[int], root: int, count: int) -> None:
    while root * 2 + 1 < count:
        child = root * 2 + 1
        if child + 1 < count and values[child] < values[child + 1]:
            child += 1
        if values[root] >= values[child]:
            return
        values[root], values[child] = values[child], values[root]
        root = child


def heap_sort(values: list[int]) -> None:
    for root in range(len(values) // 2 - 1, -1, -1):
        sift_down(values, root, len(values))
    for end in range(len(values) - 1, 0, -1):
        values[0], values[end] = values[end], values[0]
        sift_down(values, 0, end)
```

`heapq.heapify` 加 `heappop` 三行就能排完一个表，但那三行就是 5.5 节全节。本书的闸门有一条规则专查这件事——Python 实现文件里出现 `heapq`、`bisect`、`sorted`、`list.sort` 一律判红，要豁免得在 `unit.json` 里写明理由（D-025）。

8.4 交换排序

交换排序的基本思想是：**两两比较待排序记录的关键码，发现记录逆置则进行交换，直到没有逆置对 为止**。冒泡排序和快速排序是典型的交换排序算法。

8.4.1 冒泡排序

冒泡排序（bubble sorting，也称为起泡排序）不停地比较**相邻**两个记录，如果不满足排序要求就交换 相邻记录，直到所有的记录都已经排好序为止。一趟走完，最大（或最小）的那个必然被顶到一端，**就像一个气泡从水底慢慢地冒上来、最后浮出水面**——这就是冒泡排序的命名由来。

对于长度为 n 的待排序记录数组 $R=\{r_0, r_1, \cdots, r_{n-1}\}$，原书的步骤是：从数组末端开始，不断比较相邻记录，不满足排序要求就交换——首先比较 r_{n-1} 和 r_{n-2}，如果 $r_{n-1}<r_{n-2}$ 则两者 交换，然后依次对 r_{n-2} 和 r_{n-3}、…、r_1 和 r_0 比较处理；这样比较完一轮后，最小的那个 记录已经被推到数组的最左端 r_0 位置上。开始第二轮时由于 r_0 已经是最小的记录，因此只需要对 r_{n-1} 到 r_1 进行比较；依次类推，直到数组中所有记录都已经排好序为止。

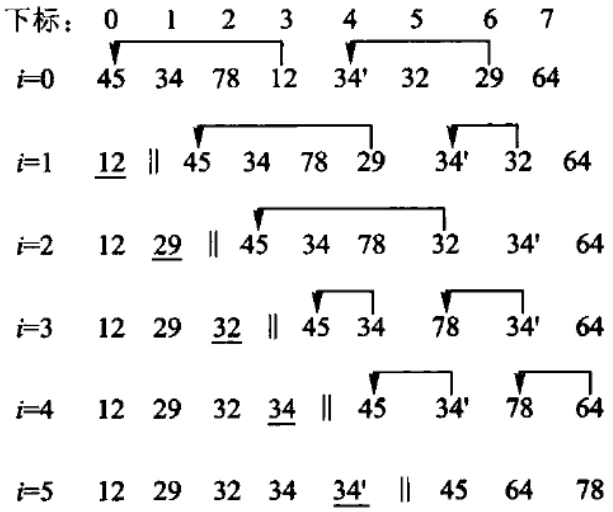


图 8.5　冒泡排序（原书的方向：从右往左冒，每趟把最小的推到左端）。第 i 行是第 i 趟冒泡**开始前**的序列（$i=0$ 即初始序列）；带箭头的连线表示尾部的某个记录在这一趟里将被逐次交换、一路移到箭头所指记录之前；带下画线的是上一趟冒到左端的那个记录，它和 ‖ 左边的记录都已经排好。$i=5$ 那一趟没有发生任何交换，算法结束。

原书是从数组末端往前比，每趟把最小的推到最左；本书从前往后比，每趟把最大的顶到最右——原书自己也写了「算法的具体实现取决于编程人员的个人喜好」，两种写法对称等价。

关键的一处优化在原书算法 8.5 里已经有了：**记一个「本趟有没有发生过交换」的标志，没有就说明整个序列已经有序，立刻结束**。

```cpp
// 算法 8.5：带“本趟无交换即结束”优化的冒泡排序。
inline void bubble_sort(std::vector<int>& values) {
    for (std::size_t end = values.size(); end > 1; --end) {
        bool changed = false;
        for (std::size_t index = 1; index < end; ++index) {
            if (values[index] < values[index - 1]) {
                using std::swap;
                swap(values[index], values[index - 1]);
                changed = true;
            }
        }
        if (!changed) return;
    }
}
```

仍是那个序列：

```
初始       45  34  78  12 34′  32  29  64
第 1 趟      34  45  12 34′  32  29  64  78
第 2 趟      34  12 34′  32  29  45  64  78
第 3 趟      12  34  32  29 34′  45  64  78
第 4 趟      12  32  29  34 34′  45  64  78
第 5 趟      12  29  32  34 34′  45  64  78
第 6 趟      12  29  32  34 34′  45  64  78    ← 本趟一次交换都没有，结束
```

本书的写法里大的记录往右走：每一趟从左往右扫，「到目前为止最大的那个」被一路换到右边；遇上不比它小的记录就停下，改由那个记录接着往右走。

它是稳定的：只有严格小于才交换，相等的两个元素永远不会互换位置——第 3 趟里 34 碰到右邻居 34′ 就停下（不严格小于，不交换），改由 34′ 越过 32 和 29 往右走；第 4 趟轮到 34 越过同样的 32 和 29，又停在 34′ 左边。两者始终保持「34 在前」。把实现里的 `<` 写成 `<=`，稳定性当场消失，这是最容易犯的一个错。

时间代价：最好情况是输入已经有序，第一趟没有交换就退出，$\Theta(n)$；最坏是完全逆序，比较与交换次数都是

$$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}=\Theta(n^2)$$

平均交换次数约为最坏的一半，仍是 $\Theta(n^2)$。空间 $\Theta(1)$。

需要提醒的是：$\Theta(n^2)$ 相同不代表快慢相同。8.7.2 节的实测里，同样 5 万个随机数，冒泡 5822 毫秒、插入 214 毫秒，**差 27 倍**——都是 $\Theta(n^2)$，差在常数上：冒泡每次逆序都要做一次三步交换，插入排序只做一次赋值搬移。

8.4.2 快速排序

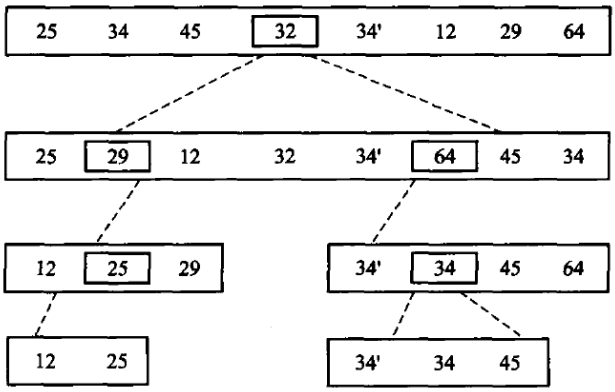


图 8.6　快速排序：选一个枢轴把序列分成「都不大于它」和「都不小于它」两段，枢轴落到它最终的位置上，再对两段各自递归。

快速排序就是基于分治法的排序算法：分治法（divide and conquer）的关键是分、治、合——将给定 问题分成若干个子问题（分），再对每个子问题求解（治），最后将所有子问题的解合并成一个综合的解（合），得到原始问题的解。

1962 年，伦敦 Elliot Brothers Ltd 公司的 Tony Hoare 发明了快速排序（quicksort）方法。**它几乎是 最快的排序算法，被评选为 20 世纪十大算法之一**，平均时间代价为 $\Theta(n \log n)$。快速排序之所以 很快，主要因为它在对数组进行分组时不是随便划分，而是**尽量将原数组划分为两半**。算法思想是：

1. 从待排序序列 S 中任意选择一个记录 k 作为**轴值**（pivot）；
2. 将剩余的记录**分割**（partition）成左子序列 L 和右子序列 R；
3. L 中所有记录都小于或等于 k，R 中记录都大于等于 k，因此 k 正好位于正确的位置；
4. 对子序列 L 和 R 递归进行快速排序，直到子序列中只含有 0 或 1 个元素，退出递归。

步骤 2、3 是「分」「治」过程，可以同时实现，完成序列的分割。**轴值 k 已经到位，而左序列 L 的记录 再也不会跑到 k 的右边，反之亦然，因此不需要明显的「合」过程**——这是它与归并排序（8.5 节）最大 的区别。如果把轴值看成子根、分割后的 L 和 R 分别看做轴值的左右子结点，则整幅图可以看成一棵 二叉树；对这棵二叉树进行中序遍历，收集轴值以及单个元素的叶结点，就得到最终的排序结果。

轴值怎么选，影响很大。轴值的选择应尽量使得序列可以据此划分为均匀的两半。最糟糕的情况莫过于 轴值恰好是第一个或者最后一个记录——这样分出的两个子序列中就会有一个为空，从而使分治法根本 起不到作用。最简单的办法是选择第一个记录（下标 `start`）或者最后一个记录（下标 `end`）作为轴值，**但弊端在于：当原始输入数组恰巧是正序或者恰好是逆序时，每次分割都会将剩余记录全都分到一个 序列中，而另一个序列却为空**。可以选取中间点 `(start + end)`

/ 2 的记录作为轴值，这种轴值在输入 数据为正序或逆序时可以平分序列，实验效果非常好。

本书的实现选区间末元素为枢轴，把更小的元素换到左侧，再递归两边——分割方案与下面原书的两种都不同，同节代码之后会用图 8.7 的数据把它逐步走一遍。全相等的输入必须也能结束（它确实能结束，但代价是退化，见代码后的说明）。

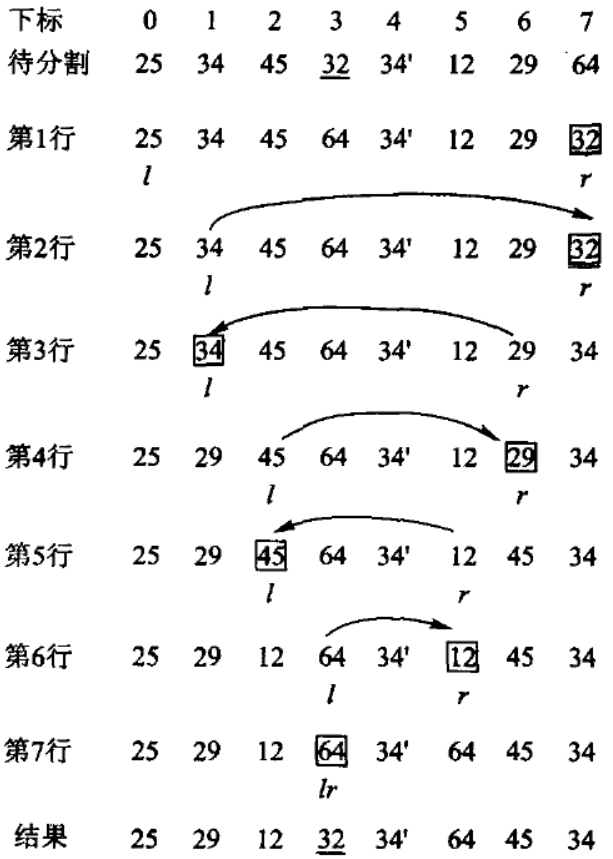


图 8.7　一趟分割的过程。分割是快排的全部工作量所在：它是 $O(n)$ 的一次线性扫描，而**分得均不均匀决定了递归有多深**——每次都平分是 $O(n \log n)$，每次只切下一个元素就退化成 $O(n^2)$。

原书给了两种分割写法。最简单的一种是 l、r 下标分别从序列的左端、右端向中间扫描：左边越过 那些小于等于 pivot 的值，停在第一个大于 pivot 的值 k_l；右边越过那些大于等于 pivot 的值，停在 第一个小于 pivot 的值 k_r；交换逆置记录 k_l 和 k_r；从交换后的位置继续从左右向中间扫描，发现并交换逆置记录对，直到 l、r 交叉而整个序列扫描完毕。一种改进的方法（图 8.7 画的就是它）是**从序列两端交替检查空闲位置，将逆置记录移动到空闲位置上，而不是直接交换逆置记录**：分割前 先将轴值与最后一个记录交换、并把轴值保存到一个临时变量中，此时序列中的最后一个位置就是空闲 位置。不同的分割方法所分出来的

子序列不同。

图 8.7 那趟分割逐行读下来是这样的。初始序列如第一行，轴值为 32；**分割前先将轴值与最后一个 记录交换，并将轴值保存到一个临时变量中**，此时序列中的最后一个位置就是空闲位置（图中用方框表示），l、r 分别用来扫描整个序列的左右指针、初始时分别位于序列两端。首先 l 指针从左边开始扫描，找到 34 大于轴值就停下；**将这个逆置记录放到空闲位置 r 上**，这时 34 原来所在位置 l 就空闲了、可以用来存放下一个逆置记录；然后从 r 前面的记录开始寻找小于轴值的逆置记录，找到 29 停下，**将 29 放到空闲位置 l 上**，这时 29 原来所在位置 r 变为空闲。不断循环下去，直到 l、r 相遇，表示整个序列已经扫描完毕；**此时的空闲位置正好就是轴值的正确位置，将轴值 32 回填回去**—— 可以看到轴值左边的记录都小于它、右边的记录都大于它，分割过程完毕。

等于轴值的记录怎么办? 由于快速排序是不稳定的，因此等于轴值的情况可以有多种处理方法。原书算法 8.6 里那句 `while (Array[l] <= TempRecord && r > l) l++;` 的注释写着「`<=` 也可以改写为 `<`，但增加记录移动」——**两种写法都对，差别在移动次数上**；重复值很多时还牵涉到分得均不均匀，本节代码之后用全相等的输入说明。

代价分析。最差情况下每次分割只切下一个元素，需要 $\Theta(n)$ 次分割，最大时间代价为 $\Theta(n^2)$；每次分割时递归算法都需要用到编译栈中一个临时记录来存储轴值，因此最大空间代价为 $\Theta(n)$。最好情况下每次分割恰好将记录分为两个长度相等的子序列，此时

$$T(n) = 2T(n/2) + cn$$

两边同除以 n 得 $T(n)/n = T(n/2)/(n/2) + c$, 逐层展开并叠加（递归共 $\log n$ 层, 每层的分割合计扫描约 n 个记录）得到 $T(n)/n = T(1)/1 + c\log n$, 即 $T(n) = cn\log n + n = \Theta(n\log n)$。

平均情况要对所有可能的情况求和再除以总情况数。假设每次分割时轴值处于结果数组中各位置的概率 是一样的，即轴值把数组分成长度为 0 和 $n-1$、1 和 $n-2$、$\cdots$ 的子序列的概率都是 $1/n$，则 $T(i)$ 和 $T(n-1-i)$ 的平均值均为 $\frac{1}{n}\sum_{k=0}^{n-1} T(k)$，代入递推式得到

$$T(n) = cn + \frac{2}{n}\sum_{k=0}^{n-1} T(k)$$

两边同乘 n 得 $nT(n) = cn^2 + 2\sum_{k=0}^{n-1} T(k)$；以 $n-1$ 代入同一式得 $(n-1)T(n-1) = c(n-1)^2 + 2\sum_{k=0}^{n-2} T(k)$；两式相减并忽略常数项，得 $nT(n) = (n+1)T(n-1) + 2cn$。再两边同除以 $n(n+1)$ 并逐层展开叠加:

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + 2c\sum_{i=3}^{n+1}\frac{1}{i} = \Theta(\log n)$$

因此 $T(n) = \Theta(n \log n)$。**快速排序平均时间代价 $\Theta(n \log n)$，平均递归深度（分割的层数）为 $\Theta(\log n)$，平均空间代价 $\Theta(\log n)$。**（原书写作「平均需要 $\Theta(\log n)$ 次分割」，指的是层数；分割的**次数**总是 $\Theta(n)$——每次分割都让一个轴值到位。）

```cpp
inline std::size_t partition(std::vector<int>& values, std::size_t first,
↪  std::size_t last) {
    const int pivot = values[last - 1];
    std::size_t boundary = first;
    for (std::size_t index = first; index + 1 < last; ++index) {
        if (values[index] < pivot) {
            using std::swap;
            swap(values[boundary], values[index]);
            ++boundary;
        }
    }
    using std::swap;
    swap(values[boundary], values[last - 1]);
    return boundary;
}

inline void quick_sort_range(std::vector<int>& values, std::size_t first,
↪  std::size_t last) {
    if (last - first < 2) return;
    const std::size_t middle = partition(values, first, last);
    quick_sort_range(values, first, middle);
    quick_sort_range(values, middle + 1, last);
}

// 算法 8.6：手写快排。
inline void quick_sort(std::vector<int>& values) { quick_sort_range(values, 0,
↪  values.size()); }
```

```python
def partition(values: list[int], first: int, last: int) -> int:
    """ 以 [first, last) 的末元素为轴划分，返回轴的落点。"""
    pivot = values[last - 1]
    boundary = first
    for index in range(first, last - 1):
        if values[index] < pivot:
            values[boundary], values[index] = values[index], values[boundary]
            boundary += 1
```

```
    values[boundary], values[last - 1] = values[last - 1], values[boundary]
    return boundary


def quick_sort_range(values: list[int], first: int, last: int) -> None:
    if last - first < 2:
        return
    middle = partition(values, first, last)
    quick_sort_range(values, first, middle)
    quick_sort_range(values, middle + 1, last)


# 算法 8.6：手写快排。
#
# ** 这一版在 Python 里会真的崩，而在 C++ 里只是慢 **：末元素当轴，遇到已排好序的
# 输入就退化成 n 层递归。C++ 有 8 MB 栈，几万层才炸；CPython 的递归上限默认是
# 1000 层，一千个有序元素就抛 RecursionError。同一个算法缺陷，两种语言的暴露
# 阈值差两个数量级——8.7 的「短侧递归」因此在 Python 里不是优化，是能不能跑。
# test.py 里有断言把这两件事都钉住。
def quick_sort(values: list[int]) -> None:
    quick_sort_range(values, 0, len(values))
```

本书的 `partition` 逐步走一遍。它不是图 8.7 那种「两端交替填空位」，而是单向扫描：`boundary` 左边是已经确认「严格小于轴值」的记录；`index` 从左往右扫，遇到小于轴值的就与 `values[boundary]` 交换、`boundary` 右移一格；扫完后把末尾的轴值与 `values[boundary]` 交换，`boundary` 就是轴值的最终位置。取图 8.7 第 1 行——轴值 32 已经换到末尾——作输入：

```
下标              0    1    2    3    4    5    6    7
输入             25   34   45   64  34′   12   29  [32]   轴值 32，boundary=0
index=0  25<32   25   34   45   64  34′   12   29  [32]   与自己交换，boundary=1
index=1..4       34、45、64、34′ 都不小于 32，跳过
index=5  12<32   25   12   45   64  34′   34   29  [32]   12 与下标 1 的 34 交换，
↪ boundary=2
index=6  29<32   25   12   29   64  34′   34   45  [32]   29 与下标 2 的 45 交换，
↪ boundary=3
轴值归位         25   12   29   32  34′   34   45   64    32 与下标 3 的 64 交换，返回 3
```

左段 25 12 29 都小于 32，右段 34′ 34 45 64 都不小于 32，与图 8.7 的结果 25 29 12 32 34′ 64 45 34 不同——**不同的分割方法分出的子序列不同**，但都满足「左小右大」。这一趟也顺便演示了不稳定：34 在 `index=5` 那次交换里被甩到 34′ 后面。比较次数是 $n-1=7$，与图 8.7 一样是一次线性扫描；代价是交换更多（等于把「小于轴值」的记录逐个搬到左边）。

等于轴值的记录一律留在右侧（判断是严格的 `<`）。这让全相等的输入退化：

每次 boundary 都停在 first，轴值落在最左端，右段只比原区间短 1，于是要分割 $n-1$ 层、做 $n(n-1)/2$ 次比较——与已排好序的输入一样是 $\Theta(n^2)$，递归深度也是 n。原书算法 8.6 照印的 `<=`/`>=` 写法同样如此（l 一路越过所有相等记录，轴值落在最右端）；把它改成 `<`/`>`、遇到相等就停下来搬动，全相等时轴值恰好落在正中——这正是原书注释里「增加记录移动」换来的东西。重复值很多时，习题 11 的三分数组是更彻底的办法。

这一版在 Python 里会真的崩，而在 C++ 里只是慢。未元素当枢轴，遇到已排好序的输入就退化成 n 层递归。C++ 默认 8 MB 栈，几万层才炸；CPython 的递归上限默认 1000 层，**一千个有序元素就抛** `RecursionError`（实测 995 个还能跑完，1000 个就抛；`test.py` 用两千个，留足余量）。同一个算法缺陷，两种语言的暴露阈值差两个数量级——下面那条「只对较短的一侧递归」的优化，在 C++ 里是省栈，在 Python 里是**能不能跑完**。

基础版有两处会出事。原书算法 8.7 只给了第二条的对策（小区间不递归、最后整体插入排序），两侧照样都递归——原书认为用栈消除递归在现代编译器下收益不明显；第一条对策是本书补上的：

- **递归太深。**每次只把区间切成两半再各自递归，划分不平衡时深度可达 n；已经排好序的输入配上「取末元素作枢轴」正是最坏情形。对策是**只对较短的一侧递归，较长的一侧改成循环**——这样递归深度被压到 $O(\log n)$，因为每次递归的区间至少减半。
- **小区间上递归不划算。**区间只剩十几个元素时，递归调用与划分的固定开销超过了收益，改用直接插入排序更快。本书的阈值取 16，而且是**每个小区间当场**做插入排序；原书则是先全部跳过、最后对整个数组做**一次**插入排序——两种做法的总代价同阶。原书的道理讲得更细：当子数组小于某个长度时不必继续递归，**那些子数组内部是无序的，但整块地看待这些子数组时它们一块块地有序**（左边子数组的排序码都小于右边数组的排序码），整个序列已经基本有序，正适合最后对整个数组做一次插入排序。原书按表 8.4 的实验取阈值 28；**阈值与算法、机器的软/硬件条件有关，在不同的环境下得到过 9、16、28 等最佳值**——本书在自己的机器上取 16，读者应当在自己的环境里重测。

```cpp
inline void quick_sort_optimized(std::vector<int>& values) {
    quick_sort_optimized_range(values, 0, values.size());
}
```

```python
def quick_sort_optimized(values: list[int]) -> None:
    quick_sort_optimized_range(values, 0, len(values))
```

注意这两条优化管的是**栈深与常数**，管不了**最坏时间**：枢轴仍取末元素，已排好序的输入照样是 $\Theta(n^2)$ 次比较。8.7.2 节的实测里，2 万个已排好序的数，优化快排要 105.9 毫秒，而插入排序只要 0.0 毫秒——**快排在这种输入上比 $\Theta(n^2)$ 的插入排序还慢**。想根治，得换枢轴策略（三数取中、随机枢轴），那是习题里

的事。

8.5 归并排序

归并排序（merge sorting）简单地将原始序列划分为两个子序列，然后分别对每个子序列递归排序，最后再将有序子序列合并。主要步骤为：① 将序列划分为两个子序列；② 分别对两个子序列递归进行 归并排序；③ 将这两个已排好序的子序列合并为一个有序序列，即**归并**过程。

归并排序也是一种基于分治法的排序，但重心与快速排序恰好相反。快速排序侧重于「分」（含「治」），即用轴值分割子序列的过程，没有明显的「合」；归并排序的「分」很简单（对半切），它侧重于「治」和「合」——每次比较子序列头，取出较小的进入结果序列，其后次小记录顶上来，继续比较。

区间长度小于 2 时已经有序，因此递归可以停止。每一层合并总共扫描 n 个元素，递推式为

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

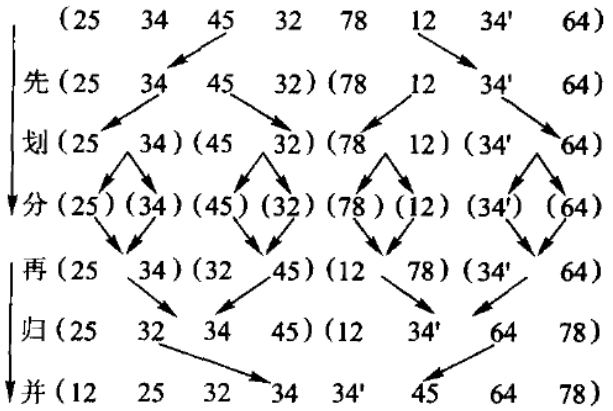


图 8.8　归并排序。**归并与快排正好相反**：快排把力气花在分（分割）上，合的时候什么都不用做；归并分得很随意（对半切），力气全花在合上。

归并排序所需时间主要包括划分时间、两个子序列的排序时间以及归并时间：**划分时间为常数，可以 忽略；归并时间随着数组长度 n 线性增长**。当数组长度为 1 时函数直接返回，$T(1) = 1$。由于**归并排序不依赖于原始数组的输入情况**，每次划分时两个子序列的长度都是基本一样的，因此 它的最大、最小以及平均时间代价均为 $\Theta(n \log n)$——这一点与快速排序不同，快排的最差情况会 退化到 $\Theta(n^2)$。

实现只分配一个与输入等长的缓冲区，并在所有递归层复用它；因此辅助空间为 $\Theta(n)$，递归 调用栈另占 $O(\log n)$。合并时使用「右侧严格更小才取右侧」的判断：相等元素先取左侧，从而保持稳定性（原书【算法 8.8】的注释写的是「为保证稳定性，相等时左边优先」，是同一件事）。

归并排序也有改进的余地。 R. Sedgewick 提出过一种优化：在把数组暂时复制到临时数组时，**将第二个子数组中元素的顺序颠倒一下**；这样两个子数组从两端开始处理、向中间推进，使得这两个 子数组的两端互相成为另一个数组的「**监视哨**」，从而不用像基本版那样在循环里反复检查子序列是否 已经结束。另外，当子数组小于某个长度（原书取 28）时也不继续递归，而是**当场对这个小子数组**做插入 排序。原书正文这里写的是「对整个序列使用插入排序」，像 8.4.2 节优化快排那样留到最后扫尾——但那对归并 行不通：归并要求两半**各自已经有序**，小子数组不排好就没法往上合并。原书算法 8.9 的代码本身是逐段做的（`else InsertSort(&Array[left], right - left + 1)`），以代码为准。

```cpp
inline void merge_ranges(std::vector<int>& values, std::vector<int>& buffer,
                         std::size_t first, std::size_t middle, std::size_t last) {
    std::size_t left = first;
    std::size_t right = middle;
    std::size_t output = first;
    while (left < middle && right < last) {
        buffer[output++] = values[right] < values[left] ? values[right++] :
↪   values[left++];
    }
    while (left < middle) buffer[output++] = values[left++];
    while (right < last) buffer[output++] = values[right++];
    for (std::size_t index = first; index < last; ++index) values[index] =
    ↪   buffer[index];
}

inline void merge_sort_range(std::vector<int>& values, std::vector<int>& buffer,
                             std::size_t first, std::size_t last) {
    if (last - first < 2) return;
    const std::size_t middle = first + (last - first) / 2;
    merge_sort_range(values, buffer, first, middle);
    merge_sort_range(values, buffer, middle, last);
    merge_ranges(values, buffer, first, middle, last);
}

// 算法 8.8：两路归并排序。
inline void merge_sort(std::vector<int>& values) {
    std::vector<int> buffer(values.size());
    merge_sort_range(values, buffer, 0, values.size());
}
```

```python
def merge_ranges(values: list[int], buffer: list[int],
                 first: int, middle: int, last: int) -> None:
    left, right, output = first, middle, first
    while left < middle and right < last:
        # `<` 而不是 `<=`：相等时取左边，稳定性就是从这一个符号来的。
```

```
        if values[right] < values[left]:
            buffer[output] = values[right]
            right += 1
        else:
            buffer[output] = values[left]
            left += 1
        output += 1
    while left < middle:
        buffer[output] = values[left]
        left += 1
        output += 1
    while right < last:
        buffer[output] = values[right]
        right += 1
        output += 1
    values[first:last] = buffer[first:last]


def merge_sort_range(values: list[int], buffer: list[int], first: int, last: int) ->
↪  None:
    if last - first < 2:
        return
    middle = first + (last - first) // 2
    merge_sort_range(values, buffer, first, middle)
    merge_sort_range(values, buffer, middle, last)
    merge_ranges(values, buffer, first, middle, last)


# 算法 8.8：两路归并排序。辅助空间 O(n)，一次开够，不在递归里反复申请。
def merge_sort(values: list[int]) -> None:
    buffer = [0] * len(values)
    merge_sort_range(values, buffer, 0, len(values))
```

`values[right] < values[left]` 里的这个 `<` 就是稳定性的全部来源：相等时取左边。写成 `<=` 就取右边，稳定性当场消失——**而 C++ 那份 `std::vector<int>` 的实现根本无法验证这一点**，两个相等的 3 换不换位置，排完一模一样。Python 这份不必改一个字就能排任何可比较的对象，于是测试里放一种「只按 key 比较、payload 不参与比较」的记录，排完检查 payload 是否还是入场顺序。把那个 `<` 改成 `<=`，这条断言会红。

空序列和单元素序列都能直接通过。

原书算法 8.9 的两处优化是上一段说的 Sedgewick 反向复制（省掉「子序列是否结束」的检查）和阈值 28 的小区间插入排序。本书的优化版（`merge_sort_optimized`）保留了后一处，前一处换成了另一个同样常见的技巧：

- **小区间改用直接插入排序**（阈值 16）。递归到只剩十几个元素时，划分与函

数调用的固定 开销超过了收益，而插入排序在这种规模上几乎是最快的。

- **归并前先看一眼左半段的末尾与右半段的开头**：若 `values[middle-1] <= values[middle]`，两段拼起来本来就有序，整趟归并直接跳过。对已经有序或接近有序的输入，这一句把实际搬移 从 $\Theta(n \log n)$ 压到接近 $\Theta(n)$；对随机输入它几乎不命中，代价只是每层多一次比较。

```
inline void merge_sort_optimized(std::vector<int>& values) {
    std::vector<int> buffer(values.size());
    merge_sort_optimized_range(values, buffer, 0, values.size());
}
```

归并排序的副产品：逆序对计数

8.2.1 节分析直接插入排序的平均代价时定义过**逆置**：$i < j$ 而 $p_i > p_j$ 的一对元素。那里用它来数代价——处理第 i 个记录时，内层循环向后挪位的次数恰好是它前面比它大的记录个数，所以**整趟直接插入排序的挪位次数就等于序列的逆置数**；再加上每趟暂存、回填各一次，总移动次数 是「逆置数 $+2(n-1)$」，逆序输入时正是 8.2.1 节的 $(n-1)(n+4)/2$。反过来，想**求**一个序列的 逆置数（常叫「逆序对数」），照着插入排序去数是 $\Theta(n^2)$。

归并排序能在 $\Theta(n \log n)$ 内数完，而且只需在合并里多写一句。逆置 (p_i, p_j) 要么两个元素 都在左半段，要么都在右半段，要么一左一右：前两类由递归数掉，第三类在合并时数。两段各自有序，当右段的 `values[right]` **严格小于**左段的 `values[left]` 时，左段里还没输出的 `[left, middle)` 全都比它大，一次记上 `middle - left` 个（写成闭区间 $[i, mid]$ 就是常见的 $mid - i + 1$）。相等时取左边、不计数：**相等元素不构成逆置**，这与归并排序稳定性来自同一个 `<`。

```
// 归并排序的副产品（原书无清单）：逆置（逆序对）计数，O(n log n)。
// 与 merge_ranges 同一个合并过程，只多一句计数：右段的 values[right] 严格小于
// 左段的 values[left] 时，左段还没输出的 [left, middle) 都比它大，一次记 middle - left
↪  个。
// 相等时取左边、不计数——相等元素不构成逆置，这与稳定性是同一个 `<`。
inline std::uint64_t merge_count_ranges(std::vector<int>& values, std::vector<int>&
↪  buffer,
                                        std::size_t first, std::size_t middle,
                                        ↪  std::size_t last) {
    std::uint64_t inversions = 0;
    std::size_t left = first;
    std::size_t right = middle;
    std::size_t output = first;
    while (left < middle && right < last) {
        if (values[right] < values[left]) {
            inversions += middle - left;
            buffer[output++] = values[right++];
        } else {
```

```
            buffer[output++] = values[left++];
        }
    }
    while (left < middle) buffer[output++] = values[left++];
    while (right < last) buffer[output++] = values[right++];
    for (std::size_t index = first; index < last; ++index) values[index] =
    ↪  buffer[index];
    return inversions;
}

inline std::uint64_t count_inversions_range(std::vector<int>& values,
↪  std::vector<int>& buffer,
                                            std::size_t first, std::size_t last) {
    if (last - first < 2) return 0;
    const std::size_t middle = first + (last - first) / 2;
    return count_inversions_range(values, buffer, first, middle)
         + count_inversions_range(values, buffer, middle, last)
         + merge_count_ranges(values, buffer, first, middle, last);
}

// 按值传参：函数在自己的副本上归并，调用方的序列原样不动（计数是查询，不该顺手排序）。
// 调用方若本来就要排序，写 count_inversions(std::move(v)) 即可省掉这次拷贝。
// 返回 64 位：逆置数最多 n(n-1)/2，n >= 65537 时就超过 INT_MAX，用 int 累加是有符号溢出。
inline std::uint64_t count_inversions(std::vector<int> values) {
    std::vector<int> buffer(values.size());
    return count_inversions_range(values, buffer, 0, values.size());
}
```

```
# 归并排序的副产品（原书无清单）：逆置（逆序对）计数，0(n log n)。
# 与 merge_ranges 同一个合并过程，只多一句计数：右段元素严格小于左段当前元素时，
# 左段还没输出的 [left, middle) 都比它大，一次记 middle - left 个。
def merge_count_ranges(values: list[int], buffer: list[int],
                       first: int, middle: int, last: int) -> int:
    inversions = 0
    left, right, output = first, middle, first
    while left < middle and right < last:
        # 与 merge_ranges 同一个 `<`：相等时取左边、不计数——相等元素不构成逆置。
        if values[right] < values[left]:
            inversions += middle - left
            buffer[output] = values[right]
            right += 1
        else:
            buffer[output] = values[left]
            left += 1
        output += 1
    while left < middle:
```

```
        buffer[output] = values[left]
        left += 1
        output += 1
    while right < last:
        buffer[output] = values[right]
        right += 1
        output += 1
    values[first:last] = buffer[first:last]
    return inversions


def count_inversions_range(values: list[int], buffer: list[int], first: int, last:
↪  int) -> int:
    if last - first < 2:
        return 0
    middle = first + (last - first) // 2
    return (count_inversions_range(values, buffer, first, middle)
            + count_inversions_range(values, buffer, middle, last)
            + merge_count_ranges(values, buffer, first, middle, last))


# 在副本上归并，调用方的序列原样不动。Python 整数没有宽度，
# C++ 版那个「n >= 65537 就溢出 int」的坑在这里不存在。
def count_inversions(values: list[int]) -> int:
    work = list(values)
    buffer = [0] * len(work)
    return count_inversions_range(work, buffer, 0, len(work))
```

两处设计值得点明：

- **计数不顺手排序。** C++ 版按值传参、在副本上归并，调用方的序列原样不动；本来就要排序的调用方 写 `count_inversions(std::move(v))` 省掉这次拷贝。Python 版同样先 `list(values)` 再归并。
- **结果必须是 64 位。**逆置数最多 $n(n-1)/2, n \geq 65537$ 时就超过 `INT_MAX`，而机考的 n 动辄 10^5。累加器写成 `int` 是有符号溢出：测试里 $n = 70000$ 的严格递减序列应得 2449965000，改成 `int` 后 UBSan 档当场报 `signed integer overflow`，`-O2` 档不报任何错、只是悄悄给出一个错数，由断言抓住；再加一条 $n = 100000$（4999950000 超过 32 位无符号上限），连「换成 `unsigned`」这种不触发未定义行为的截断也一并锁住。Python 整数没有宽度，这个坑不存在。在 OJ 上交 C++ 时，这一条就是「答案开 `long long`」。

变形题（留作练习）。求满足 $i < j$ 且 $a_i + a_j > b_i + b_j$ 的对数。移项得 $a_i - b_i > b_j - a_j$，令 $c_i = a_i - b_i$，条件变成 $c_i > -c_j$：仍是「前面的数比后面的某个量大」，套用上面的框架，只是合并时拿左段的 c_i 去比右段的 $-c_j$（先用双指针数完，再照常合并）。顺带可以注意到，这个条件等价于 $c_i + c_j > 0$，对 i、j 对称，所以「$i < j$」其实不起作用——把 c 排序后双指针也能数，可以

拿来对拍。

另一条路：离散化 + 树状数组。从左到右扫描，把每个值换成它在全体值中的名次（离散化），用树状数组维护「已扫过的值里各名次出现几次」；扫到 a_j 时，前面比它大的个数就是「已扫过的个数 − 名次不超过 a_j 的个数」，查询与更新都是 $O(\log n)$。相等元素名次相同、落在「不超过」一侧，因此同样不计入逆置。本书第 12 章正文目前没有树状数组一节，可测实现在 `code/ch12/fenwick`（`FenwickTree::add` 单点加、`FenwickTree::prefix_sum` 前缀和）。两种做法代价同为 $\Theta(n \log n)$；归并法不需要离散化，树状数组法则能在序列动态追加时继续回答。

8.6 分配排序和索引排序

本章要介绍的最后一种排序算法是**分配排序**（distribution sorting）。**这种排序算法的唯一特征是 不需要进行关键码之间的比较，但是需要事先知道记录序列的一些具体情况**。

三种情况对应三种做法：如果事先知道序列中记录的排序码都位于某个小区间段内，就可以用 8.6.1 的 **桶式排序**；当排序码值域 m 很大时，可以拆分为多个部分来进行比较，这就是基于桶式排序的 **基数排序**（radix sorting，8.6.2）；为减少记录的移动，可以采取基于静态链的基数排序，这本质上 是一种**索引排序**（index sorting）或称地址排序（address sorting，8.6.3）——索引排序后的结果应该 可以在 $\Theta(n)$ 的线性时间内整理为按照数组下标为序的有序序列。

8.6.1 桶式排序

前面每一种排序都靠**元素之间的比较**决定次序，而比较排序最坏至少要 $\Omega(n \log n)$ 次比较（证明见 8.7.3）。要突破这个下界，只能不再比较——转而利用关键码本身的数字结构，直接算出每个元素该去哪儿。这类方法叫**分配排序**。

桶式排序是其中最简单的一种：已知所有取值落在 $[0, m)$ 之间时，准备 m 个计数器，扫一遍序列数出每个取值出现几次，再把计数**累加**成「小于等于 i 的元素共有几个」，这就直接给出了每个取值在结果里的位置区间。

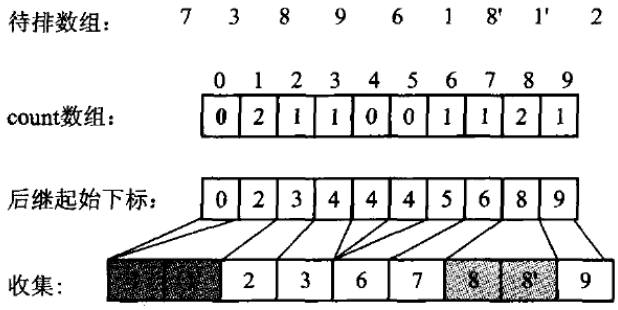


图 8.9　桶式排序示意图。值相同的记录进同一个桶，再按桶号依次收集。先数一遍、把计数累加成起始位置，就能预先给每个桶留出恰好够用的一段——这样只需要一个长度为 n 的辅助数组，而不是 m 条变长的链。

为什么要「先数一遍再累加」。假如知道某个长度为 n 的序列中所有记录的值都在 $0 \sim m-1$ 之间，如果事先知道每个桶中会有多少个元素，就可以使用一个长度为 n 的辅助数组：例如第 0 个桶将含有 3 个记录、第 1 个桶将接收 5 个记录，就可以预先将前 3 个位置空出来留给第 0 个桶使用，把接着的 5 个位置留给第 1 个桶。因此还需要用 m 个计数器分别来统计 $0, 1, 2, \cdots, m-1$ 这些数出现的 个数，将具有相同值的记录都分配到同一个桶中，然后依次按照编号从桶中取出记录，组成一个有序序列。

用原书的例子，$m = 10$，序列 {7, 3, 8, 9, 6, 1, 8′, 1′, 2}：

```
取值 i        0   1   2   3   4   5   6   7   8   9
出现次数      0   2   1   1   0   0   1   1   2   1
累加之后      0   2   3   4   4   4   5   6   8   9   ← count[i] 是「i 的结束位置」
```

count[8] == 8 的含义是：值 8 的记录占据结果数组下标 6、7 两格。**输出时必须从原序列的尾部往前扫**——先遇到 8′，把它放在下标 7；后遇到 8，放在下标 6。这样两个 8 的相对次序才保持不变。倒着扫是桶式排序稳定性的**唯**一来源，正着扫就不稳定了。

本书的实现有两处与原书不同，都写在这里：

```cpp
// 算法 8.10：桶式（计数）排序，支持负数但不适合巨大稀疏值域。
inline void counting_sort(std::vector<int>& values) {
    if (values.empty()) return;
    int low = values[0];
    int high = values[0];
    for (int value : values) { if (value < low) low = value; if (high < value) high =
    ↪  value; }
    const auto range = static_cast<unsigned long long>(static_cast<long long>(high)
    ↪  - low + 1);
    if (range > counting_range_limit) throw std::invalid_argument("counting sort
    ↪  value range is too sparse");
    std::vector<std::size_t> counts(static_cast<std::size_t>(range), 0);
    for (int value : values) ++counts[static_cast<std::size_t>(value - low)];
    std::size_t output = 0;
    for (std::size_t bucket = 0; bucket < counts.size(); ++bucket) {
        while (counts[bucket]-- != 0) values[output++] = static_cast<int>(bucket) +
        ↪  low;
    }
}
```

```python
# 算法 8.10：桶式（计数）排序，支持负数但不适合巨大稀疏值域。
#
# 与 C++ 版的一处 ** 实质 ** 差别：C++ 里 `high - low + 1` 本身就可能溢出 int，
# 那一版必须先转成 long long 再算。Python 的整数没有宽度，这个坑不存在——
# 但值域上限的检查一条都不能少，它挡的是内存而不是溢出。
```

```
def counting_sort(values: list[int]) -> None:
    if not values:
        return
    low = high = values[0]
    for value in values:
        if value < low:
            low = value
        if high < value:
            high = value
    span = high - low + 1
    if span > COUNTING_RANGE_LIMIT:
        raise ValueError("counting sort value range is too sparse")
    counts = [0] * span
    for value in values:
        counts[value - low] += 1
    output = 0
    for bucket, count in enumerate(counts):
        for _ in range(count):
            values[output] = bucket + low
            output += 1
```

C++ 版计算值域时必须先转成 `long long`：`high - low + 1` 在 `int` 里自己就会溢出。Python 的整数没有宽度，这个坑不存在——**但值域上限的检查一条都不能少**，它挡的是内存，不是溢出。

第一，**不要求下界为 0**。实现先扫出实际的 `[low, high]`，按 `value - low` 计数，于是负数不需要调用者先做平移——原书那个「所有记录都位于区间 $[0, \max)$」的前提，在真实数据上往往是不成立的。

第二，**值域太稀疏时直接抛异常**。桶式排序的时间与空间都是 $\Theta(m+n)$：m 是值域长度，不是元素个数。排 10 个数、值域却是 $[0, 10^9)$ 时，它会试图分配十亿个计数器。原书没有防线，本书在 $m > 10^7$ 时抛 `std::invalid_argument`——**让它当场失败，好过让它把机器拖垮**。判断能不能用桶式排序，靠的正是这个 m 与 n 的比值：$m = \Theta(n)$ 时总代价 $\Theta(n)$，是相对比较排序的一次飞跃；m 涨到 $\Theta(n \log n)$ 或 $\Theta(n^2)$，优势立刻消失，还倒贴 $\Theta(m+n)$ 的空间。

还有一处要说清楚：本书实现按计数**重建取值**（`values[output++] = bucket + low`），而不是像原书那样把原记录搬进结果数组。对纯整数，两者的输出完全一样，稳定性无从区分；但只要记录带上了卫星数据（学号、姓名），就必须回到原书的写法——**倒序扫描 + 搬移原记录**。这一点在把本节代码推广到真实记录时是个坑。

代价与适用范围。对于一个长度为 n 且值域区间长度为 m 的数组，桶式排序首先需要扫描一遍序列 以便进行统计计数，然后再统计小于等于 i（$i \in [0, m)$）的个数，输出有序序列时共循环 n 次，因此**总的时间代价为** $\Theta(m+n)$；由于算法中需要用到 m 个计数器，还需要一个长度为 n 的临时 数组，因此**空间代价**

为 $\Theta(m+n)$。

当 m 为 $\Theta(n)$ 数量级时，时间代价为 $\Theta(n)$——此时桶式排序的效率 相对于前面那些排序算法是一个飞跃。**但是当 m 为更高数量级（例如 $\Theta(n\log n)$ 或 $\Theta(n^2)$）时，时间代价就由 m 来决定**，也变成 $\Theta(n\log n)$ 或 $\Theta(n^2)$；此时桶式排序相比于前面那些排序 就没有什么优势，不仅如此，它还要额外的 $\Theta(m+n)$ 空间代价。**因此桶式排序只对 m 较小时才具有 实际意义**——这也正是下一节要把大值域拆成若干小值域的原因。

8.6.2 基数排序

当值域 m 很大时，可以对桶式排序做一些改进：**将排序码拆分为多个部分来进行比较**。例如要对 $0\sim 9999$ 之间的整数进行排序，可以先按千、百、十、个位拆分。这种将排序码按照其进制的**基数** 进行拆分排序的方法就是基数排序，是分配排序的一种特例。

先把记号定下来。假设一个长度为 n 的序列 $R=\{r_0,r_1,\cdots,r_{n-1}\}$，记录的排序码 K 包含 d 个子排序码 $K=(k_{d-1},k_{d-2},\cdots,k_1,k_0)$，称序列 R 对排序码有序，就是对于任意的两个 记录 R_i、R_j $(0\le i<j\le n-1)$ 都满足 $(k_{i,d-1},\cdots,k_{i,0})\le(k_{j,d-1},\cdots,k_{j,0})$；其中 k_{d-1} 称为最高位排序码，k_0 称为最低位排序码。

这 d 个排序码可以是不同类型。例如按照桥牌规则，扑克牌的排序码就是（花色，面值）的组合值：花色按黑桃（S）> 红心（H）> 方片（D）> 梅花（C）的顺序，面值按 A > K > Q > J > 10 > 9 > ⋯ > 3 > 2 的顺序。（原书印作「黑桃（S）> 红心（H）> 梅花（D）> 方片（C）」：把梅花、方片的英文字母配反了。桥牌规则里方片高于梅花；「黑、红、梅、方」是国内很多牌戏的习惯次序，原书第 8 章习题 22 用的就是它，两者不要混淆。下面例子的结果按 C < D < H < S 排，与这里的更正一致。）人们对整数的运算是十进制的，因此本章例子都选取 10 作为基数；其实对于由 0、1 代码组成的整数，还可以看做二进制、四进制、八进制、十六进制，而采用基数 $r=2,4,8,16$；**对于字符串，则可以采用 基数 $r=26$（英文字母个数）**。

按照排序码的先后顺序，分配排序分为两种。高位优先法（most significant digit first，MSD）：先对高位 k_{d-1} 排序，将序列分成若干子序列，每个子序列具有相同 k_{d-1} 值；然后对每个子序列按 次高位 k_{d-2} 排序，分成更小的子序列；依次重复，直到对 k_1 排序后，分成的极小子序列每个内部 含有相同的排序码；最后对每个子序列用最低位 k_0 排序，将所有子序列依次连接在一起，成为一个有序 序列——**这是一个分、分、⋯、分、收的过程，是一个递归分治问题**。**低位优先法**（least significant digit first，LSD）：从最低位 k_0 开始排序，对于排好的序列再用次低位 k_1 排序，依次重复，直至对最高位 k_{d-1} 排好序后整个序列成为有序的——**这是一个分、收；分、收；⋯；分、收的过程**。

以扑克牌 `S3 HJ C8 H9 S9 D3 CA D7` 为例：MSD **方法**先按花色分 4 堆、各堆中再按牌点分，最后把 4 堆合并（先按花色得 `C8 CA D3 D7 HJ H9 S3 S9`，再按面

值得 C8 CA D3 D7 H9 HJ S3 S9)；**LSD 方法**先按牌点分成顺序的 13 堆，再依次从这 13 堆中顺序取出 4 种花色（先按面值得 S3 D3 D7 C8 H9 S9 HJ CA，再按花色得 C8 CA D3 D7 H9 HJ S3 S9)。

在人们看起来似乎觉得 MSD 比较直观，一般都采用 MSD 方法；但是计算机常用 LSD 进行分配排序，因为它速度较快、便于统一处理——而 MSD 在分配之后还要处理各子集的排序问题，这通常是一个递归 过程，计算机处理比较复杂。**本节的算法都是讨论 LSD 方法**。

具体地说，基数排序把关键码拆成 d 位，每一位的取值只有 r 种（十进制数 $r=10$，字符串 $r=26$)，于是可以逐位做桶式排序。

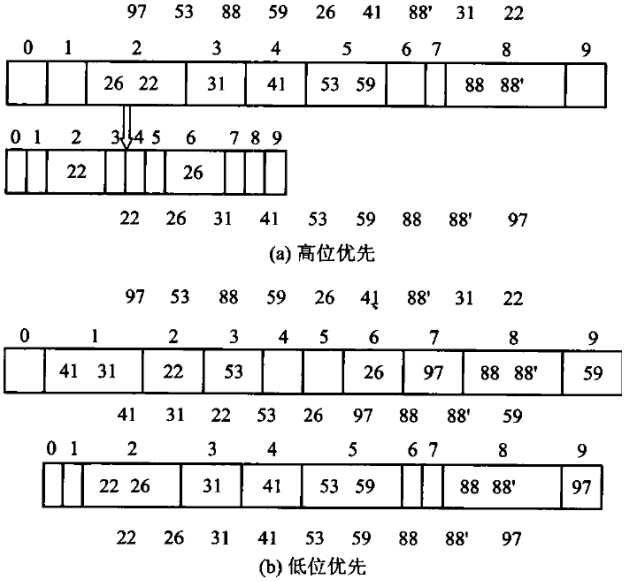


图 8.10　两位数的基数排序，(b) 低位优先：先按个位分成 10 个桶、收集，再按十位分桶、收集，两趟之后整个序列有序。(a) 高位优先则要先按十位分桶，再对**每个桶**分别按个位细分，两位数一共 100 个子桶——桶数随位数指数增长，所以实用的是低位优先。低位优先能成立的前提是每一趟都**稳定**：后一趟不能打乱前一趟已经排好的相对次序。

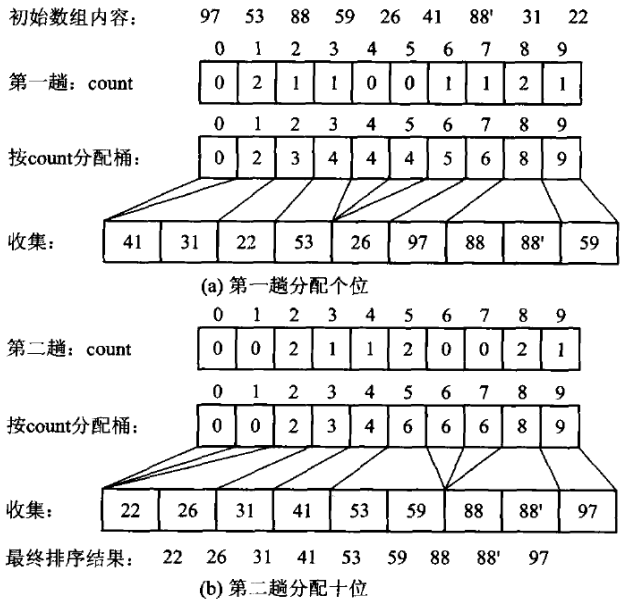


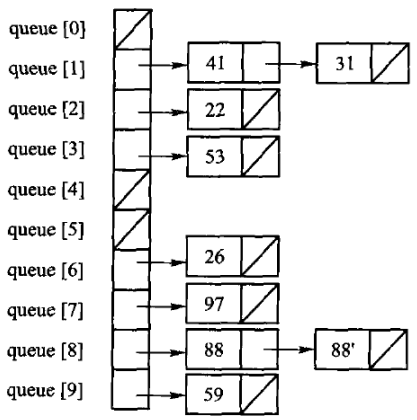
图 8.11　基于顺序存储的基数排序：每一趟用桶式排序把记录搬进辅助数组，再搬回来。

搬记录太贵时，可以改用静态链：不动记录，只改 next 索引，每趟只重排链。

原书图 8.12 用 $\{97, 53, 88, 59, 26, 41, 88', 31, 22\}$（$n = 9$、$d = 2$、$r = 10$）走了一遍，五个分图依次是：

(a) 初始链表内容

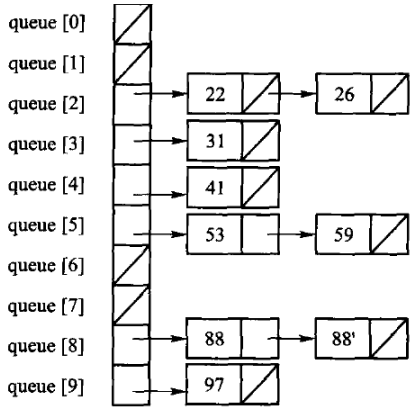
下标	0	1	2	3	4	5	6	7	8
关键码	97	53	88	59	26	41	88’	31	22



(b) 第一趟分配结果

(c) 第一趟收集结果

下标	0	1	2	3	4	5	6	7	8
关键码	41	31	22	53	26	97	88	88’	59



(d) 第二趟分配结果

(e) 第二趟收集结果

下标	0	1	2	3	4	5	6	7	8
关键码	22	26	31	41	53	59	88	88’	97

图 8.12　基于静态链的基数排序。两趟分配和收集之后，静态链已经串成有序次序，再顺链整理一遍（原书的 `AddrSort()`，$\Theta(n)$）就得到按下标有序的数组。注意 88 和 88’ 在 (e) 里仍保持原来的先后——基数排序的每一趟都必须稳定，否则低位排好的次序会被高位那趟打乱。

本书的实现把有符号 `int` 的符号位翻转后，按字节做 4 趟计数收集。否则负数会按无符号序排到最大。

```
// 算法 8.11：LSD 基数排序。翻转符号位使二补码有符号 int 按无符号序排序。
inline void radix_sort(std::vector<int>& values) {
    std::vector<int> buffer(values.size());
    for (unsigned shift = 0; shift < 32; shift += 8) {
        std::size_t counts[256]{};
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
            ++counts[(key >> shift) & 0xffU];
        }
        std::size_t offset = 0;
        for (std::size_t& count : counts) { const std::size_t old = count; count =
        ↪  offset; offset += old; }
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
```

```
            buffer[counts[(key >> shift) & 0xffU]++] = value;
        }
        values.swap(buffer);
    }
}
```

```python
# 算法 8.11：LSD 基数排序，每趟按 8 位分桶。
#
# **C++ 版那个「异或最高位」的技巧在 Python 里不成立 **，这是本章最值得看的一处
# 语言差异。C++ 的 int 是定长补码，把符号位翻过来，负数的位模式就整体排到正数
# 前面，一趟不用特殊处理。Python 的整数是任意精度、没有「最高位」这个东西，
# `-1` 的二进制是概念上无限长的 1。所以这里换一种同样只扫两遍的办法：
# ** 整体平移到非负区间 **，排完再平移回来。代价是多一次 min 扫描，
# 换来的是不依赖任何机器字长——这份实现对 10**30 一样成立。
def radix_sort(values: list[int]) -> None:
    if not values:
        return
    shift_base = min(values)
    keys = [value - shift_base for value in values]
    largest = max(keys)
    buffer = [0] * len(keys)
    shift = 0
    while (largest >> shift) > 0 or shift == 0:
        counts = [0] * RADIX_BUCKETS
        for key in keys:
            counts[(key >> shift) & (RADIX_BUCKETS - 1)] += 1
        offset = 0
        for bucket in range(RADIX_BUCKETS):
            counts[bucket], offset = offset, offset + counts[bucket]
        for key in keys:
            digit = (key >> shift) & (RADIX_BUCKETS - 1)
            buffer[counts[digit]] = key
            counts[digit] += 1
        keys, buffer = buffer, keys
        shift += RADIX_BITS
    for index, key in enumerate(keys):
        values[index] = key + shift_base
```

C++ **版那个「异或最高位」的技巧在** Python **里不成立**，这是本章最值得看的一处语言差异。C++ 的 `int` 是定长补码，把符号位翻过来，负数的位模式就整体排到正数前面，一趟不用特殊处理。Python 的整数是任意精度、**没有「最高位」这个东西**，`-1` 的二进制是概念上无限长的 1。所以 Python 版换成**整体平移到非负区间**，排完再平移回来：代价是多一次 `min` 扫描，换来的是不依赖任何机器字长——这份实现对 10^{30} 一样成立，测试里就是拿这个规模钉住的。

一趟按一个字节分配，4 趟覆盖 32 位，因此时间是 $\Theta(d(n+r))$：$d=4$

趟，每趟 $r = 256$ 个桶。与桶式排序相比，基数排序把「值域 m」换成了「位数 d 与基数 r」，于是 $[0, 2^{32})$ 这样的值域也排得动——代价是要扫 4 遍。

上面这版把桶做成了计数数组（先数、再累加、再放），桶是**隐式**的。原书算法 8.13 走的是另一条路：桶是**显式**的队列。不过原书的桶是**静态链队列**——代码 8.12 的 `StaticQueue` 只有 `head`、`tail` 两个下标，每条记录多一个 `next` 域，分配时把记录挂到第 k 个桶的链尾，收集时把 r 条非空链首尾相接，**记录本身一次都不搬**，两趟之后再用 `AddrSort()` 顺链整理（图 8.12 就是这个过程）。本书的 `radix_sort_linked_style` 保留了「桶是显式队列、按桶号依次出队」这个结构，把桶换成固定容量的环形队列、每趟把值 `push` 进 256 个队列再按桶号依次 `pop` 出来——它更接近「分配-收集」这个名字的字面意思，也更容易看懂；代价是每趟都要维护 256 个队列，而且值是真的搬了，**静态链「不搬记录」那一点没有演示出来**。本书的定长队列如下（编号沿用代码 8.12）：

```cpp
// 代码 8.12：固定容量 FIFO，是基数排序的桶而非通用 STL queue 替身。
template <typename T>
class StaticQueue {
public:
    explicit StaticQueue(std::size_t capacity) : data_(capacity),
    ↪  capacity_(capacity) {}
    [[nodiscard]] bool push(const T& value) {
        if (size_ == capacity_) return false;
        data_[(front_ + size_++) % capacity_] = value;
        return true;
    }
    [[nodiscard]] std::optional<T> pop() {
        if (size_ == 0) return std::nullopt;
        T value = data_[front_];
        front_ = (front_ + 1) % capacity_;
        --size_;
        return value;
    }
    [[nodiscard]] bool empty() const noexcept { return size_ == 0; }
private:
    std::vector<T> data_;
    std::size_t capacity_{0};
    std::size_t front_{0};
    std::size_t size_{0};
};
```

本章 Python 侧不给实现的清单一共三条，代码 8.12 是其中之一（另两条是代码 8.16 随机数据与 代码 8.17 计时）。理由写在 `code/ch08/sorting/unit.json` 的 `py_skip` 字段里：容量、环绕、下标回卷都是**存储管理**，而 Python 的 `list` 没有容量这个概念，照着写只会得到一层没有内容的包装。下面算法 8.13 的 Python 版改用普通 `list` 作桶，这一节要讲的东西——稳定性来自「桶内保持入桶顺序、

收集时按桶号从小到大」——一个字都没少。

它不是 `std::queue` 的替身，而是基数排序的桶：容量固定、不扩容，`push` 满了返回 `false`，`pop` 空了返回 `std::nullopt`（D-001 §3c：可预期的空状态用 `optional`，不是错误）。

```cpp
// 算法 8.13：以显式桶队列演示顺序收集的基数排序。
inline void radix_sort_linked_style(std::vector<int>& values) {
    std::vector<int> buffer(values.size());
    for (unsigned shift = 0; shift < 32; shift += 8) {
        std::vector<StaticQueue<int>> buckets;
        buckets.reserve(256);
        for (std::size_t bucket = 0; bucket < 256; ++bucket)
        ↪  buckets.emplace_back(values.size());
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
            (void)buckets[(key >> shift) & 0xffU].push(value);
        }
        std::size_t output = 0;
        for (auto& bucket : buckets) while (auto value = bucket.pop())
        ↪  buffer[output++] = *value;
        values.swap(buffer);
    }
}
```

```python
# 算法 8.13：以显式桶队列演示「分桶 —按桶序收集」的基数排序。
#
# C++ 版这里用的是【代码 8.12】那个固定容量环形队列。Python 侧按 D-025 §1
# 不实现 8.12：容量、环绕、下标回卷都是 ** 存储管理 **，而 list 没有容量这个概念，
# 照着写只会得到一层没有内容的包装。桶换成普通 list，这一节要讲的东西
# ——稳定性来自「桶内保持入桶顺序、收集时按桶号从小到大」——一个字都没少。
def radix_sort_linked_style(values: list[int]) -> None:
    if not values:
        return
    shift_base = min(values)
    keys = [value - shift_base for value in values]
    largest = max(keys)
    shift = 0
    while (largest >> shift) > 0 or shift == 0:
        buckets: list[list[int]] = [[] for _ in range(RADIX_BUCKETS)]
        for key in keys:
            buckets[(key >> shift) & (RADIX_BUCKETS - 1)].append(key)
        output = 0
        for bucket in buckets:
            for key in bucket:
                keys[output] = key
                output += 1
```

```
        shift += RADIX_BITS
    for index, key in enumerate(keys):
        values[index] = key + shift_base
```

两种写法排出来的结果逐字节相同，测试对拍了这一点。**分配排序天然稳定**：同一个桶里的记录按进桶顺序出桶，从头到尾没有任何一次比较能打乱它们。

基数排序到底是不是 $\Theta(n)$ 的？ 前面的分析给出基数排序的时间代价是 $\Theta(d(n+r))$。当 r 远小于 n 时（整数基数 $r=10$、字符串基数 $r=26$ 均为常数），可以忽略掉 r，时间代价为 $\Theta(d\cdot n)$，而 d 取决于数字的位数或者字符串的长度；如果 d 相对于数组长度 n 很小，时间代价就成了 $\Theta(n)$。**这样看来，基数排序似乎是最快的排序算法——但它不是。**

考虑 n 个互不相同的关键码：此时需要 n 个不同的编码来表示它们，也就是说 $d\geq \log_r n$，即 d 在 $\Omega(\log n)$ 中。因此，对 n 个不同的关键码值进行基数排序需要耗用 $\Omega(n\log n)$ 的 时间代价。**换句话说，如果 n 个记录的排序码均不重复，那么排序码会被拆分为 $\log_r n$ 位，这时 d 就不能再当作常数来处理，基数排序的时间代价变成 $\Theta(n\log_r n)$**——按二进制选基数 $r=2$ 时就是 $\Theta(n\log n)$。所以**基数排序实质上并不是 $\Theta(n)$ 的算法，与堆排序、快速排序、归并排序同属 $\Theta(n\log n)$ 复杂度量级**。

（实际应用中常用的 32 位计算机，其 `int` 的 32 个二进制位数几乎都比 $\log n$ 大，因此比较两个较长 整数的时间代价理论上是 $\Omega(\log n)$，但实际上人们认为整数的比较运算在常数时间内完成—— 这条「常数时间比较」的默认假设，正是上面那笔账能算出差别的原因。）

一处值得抄下来的工程优化：取排序码第 i 位的朴素写法是循环 `k = k / r` 做 i 次再取模。对正整数、而且采用 2 的 g 次幂 $r=2^g$ 作基数时，可以改用位操作 `k = ((k & (0xFF << (i*g))) >> (i*g)) % r;`——先把 `0xFF` 左移 $i\cdot g$ 位，与 `k` 按位与，消除 `k` 右边的 $i\cdot g$ 位，再右移回来。原书在同一测试环境下实测：1M 数据、$r=16$、$g=4$ 的 链式基数排序从 0.6570 秒降到 0.5532 秒。

8.6.3 索引排序

前面所有算法都在**搬记录**。当记录很大——一条学籍记录几百字节，一张图片几兆——搬移的代价 就压过了比较的代价。索引排序（也叫地址排序）的办法是：**让数组中的每一个元素存储指向该元素 记录的指针，在需要移动记录时只移动指针值（或索引地址）而不移动记录本身**。8.6.2 节中的静态链 基数排序实际上也是一种索引排序。

索引排序是典型的用空间换取时间的技巧：它需要一些空间来存放指针，但换来了更高的效率。需要说明的是，索引排序并不要求在待排序数组中就给出附加空间——可以通过申请新数组、通过下标 来进行对应。排序后，需要根据辅助的索引数组来调整原始的待排数组，使其按照下标有序地存放记录，**这些整理都应该在 $O(n)$ 时间之内完成**（否则前面省下的搬移就白省了）。

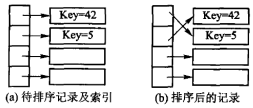


图 8.13　索引排序示意：(a) 待排序记录及索引；(b) 排序后——**记录一个都没动，动的是索引**。

排完之后手上有一个索引数组。原书表 8.1 给了两种约定，本节按第二种：`index[i]` 存的是「结果数组第 i 个位置应该从原数组的哪个下标取值」，也就是 `结果 [i] == 原数组 [index[i]]`。仍用原书那组数据：

```
原下标 i        0    1    2    3    4    5    6    7
排序码         29   25   34   64  34′   12   32   45
索引 index[i]   5    1    0    6    2    4    7    3
按索引取值     12   25   29   32   34  34′   45   64
```

排索引本身可以用任何排序算法，原书算法 8.14 用的是直接插入——比较的是 `values[index[...]]`，交换的是 `index[...]`：

```cpp
// 算法 8.14：排序索引，不移动原记录。
inline std::vector<std::size_t> insertion_index_sort(const std::vector<int>& values)
↪  {
    std::vector<std::size_t> indexes(values.size());
    for (std::size_t i = 0; i < indexes.size(); ++i) indexes[i] = i;
    for (std::size_t i = 1; i < indexes.size(); ++i) {
        const std::size_t index = indexes[i];
        std::size_t hole = i;
        while (hole != 0 && values[index] < values[indexes[hole - 1]]) {
        ↪  indexes[hole] = indexes[hole - 1]; --hole; }
        indexes[hole] = index;
    }
    return indexes;
}
```

```python
# 算法 8.14：排序索引，不移动原记录。记录大而键小时，搬索引比搬记录便宜得多。
def insertion_index_sort(values: list[int]) -> list[int]:
    indexes = list(range(len(values)))
    for i in range(1, len(indexes)):
        index = indexes[i]
        hole = i
        while hole > 0 and values[index] < values[indexes[hole - 1]]:
            indexes[hole] = indexes[hole - 1]
            hole -= 1
        indexes[hole] = index
    return indexes
```

注意这里**稳定性是免费的**：插入排序不越过相等元素，所以 34 与 34′ 的先后没变（index 里 2 排在 4 前面）。

如果最终还是要把记录本身排好，就得按索引把数组整理一遍，而且要在 $O(n)$ 时间内完成——否则前面省下的搬移又还回去了。做法是顺着**置换环**走：从下标 0 开始，0 该放 5 的值、5 该放 4 的值、4 该放 2 的值、2 该放 0 的值，一圈回到起点，这一环上的元素一次到位；每到位一个就把它的索引改成自己的下标，从此不再参与后续的环。

```cpp
// 算法 8.15：沿置换环把索引顺序落实为记录顺序。
inline void adjust_by_index(std::vector<int>& values, std::vector<std::size_t>&
↪  indexes) {
    for (std::size_t first = 0; first < values.size(); ++first) {
        if (indexes[first] == first) continue;
        std::size_t current = first;
        const int saved = values[first];
        while (indexes[current] != first) {
            const std::size_t source = indexes[current];
            values[current] = values[source];
            indexes[current] = current;
            current = source;
        }
        values[current] = saved;
        indexes[current] = current;
    }
}
```

```python
# 算法 8.15：沿置换环把索引顺序落实为记录顺序。
# 每个元素最多被搬一次，整趟 O(n) 次移动、O(1) 辅助空间。
def adjust_by_index(values: list[int], indexes: list[int]) -> None:
    for first in range(len(values)):
        if indexes[first] == first:
            continue
        current = first
        saved = values[first]
        while indexes[current] != first:
            source = indexes[current]
            values[current] = values[source]
            indexes[current] = current
            current = source
        values[current] = saved
        indexes[current] = current
```

这组数据一共两个环：

```
环 {0, 5, 4, 2}    12   25   29   64   34  34′   32   45
环 {3, 6, 7}       12   25   29   32   34  34′   45   64
```

不在正确位置上的元素每个恰好被搬一次（每个环另加一次存入、取出临时变量；本例下标 1 的 25 已经在位，根本不动），所以整理是 $\Theta(n)$ 的。（底稿里这一环印成了 `{0,5,4,1}`，与它自己列出的元素 `{29,12,34′,34}` 对不上——那组元素对应的下标是 `{0,5,4,2}`，多半是扫描损坏。）

代价与收益：多用 $\Theta(n)$ 的索引空间，换来「排序阶段一次记录搬移都没有」。8.6.2 节的静态链基数排序其实也是一种索引排序——它排的是 `next` 链，不是记录本身。

8.7 排序算法的时间代价

以下结论都以「元素比较次数」为模型，并假定一次比较是 $O(1)$。

8.7.1 简单排序算法的时间代价

三种 $\Theta(n^2)$ 的算法放在一起看，差别不在数量级，而在**它们对输入形态的反应**：

算法	最好	平均	最坏	稳定	交换/搬移次数
直接插入	$\Theta(n)$（已有序）	$\Theta(n^2)$	$\Theta(n^2)$（逆序）	稳定	搬移 $\Theta(n^2)$
冒泡（带无交换退出）	$\Theta(n)$（已有序）	$\Theta(n^2)$	$\Theta(n^2)$	稳定	交换 $\Theta(n^2)$
直接选择	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	**不稳定**	交换 $\le n-1$

选择排序那一行是关键：**它的最好情况也是 $\Theta(n^2)$**，因为无论输入什么样，它都要把剩余部分完整扫一遍才能确定最小值。代价是它换来了「交换次数最少」这个别人没有的性质。反过来，插入与冒泡都能在有序输入上退化到 $\Theta(n)$——前者的内层循环一次都不进，后者靠那个「本趟无交换」的标志立刻退出。

为什么这三种都快不起来，原因是同一个。插入和冒泡算法都有一个共同缺点：**只对相邻的两个记录 进行比较和移动**，因此记录只能一步步地向目标位置移动，效率低下。直接选择排序虽然没有这样，但是在选择第 i 小的记录时也是在剩下的记录中逐个地进行线性比较——**它改进了冒泡排序的交换 次数，但比较次数仍然没有降低**，因此总体效率依旧很低。

8.2.1 节分析过：插入排序的时间代价主要由记录移动到目标位置所需的步长之和决定，而步长又由序列 中的**逆置数**决定；由于一个长度为 n 的序列平均有 $n(n-1)/4$ 对逆置，因此**任何一种只对相邻记录 进行比较的排序算法，平均时间代价都是 $\Theta(n^2)$**。

这些算法效率太低，是因为它们在排序过程中的比较或者移动都是一步步进行的，实际上做了很多重复 的操作。**因此，如果想彻底从数量级上提高算法效率，就需要摆脱这种逐个对每个记录一步步操作的 算法思想**——例如 Shell 排序就跨越相邻元素进行分区插入排序，从而达到了 $\Theta(n^{1.5})$ 的效率。Shell 排序不在上表内：它的复杂度取决于增量序列，减半增量最坏 $\Theta(n^2)$，Hibbard 增量最坏 $\Theta(n^{3/2})$。

8.7.2 排序算法的理论和实验时间

同阶不等于同速。原书为此专门给了两个工具——随机数据生成（代码 8.16）与计时（代码 8.17）：

```cpp
// 代码 8.16：可复现随机数据；代码 8.17：单调时钟计时。
inline std::vector<int> random_values(std::size_t count, int upper_bound, unsigned
↪  seed = 1) {
    std::mt19937 engine(seed);
    std::uniform_int_distribution<int> distribution(0, upper_bound - 1);
    std::vector<int> values(count);
    for (int& value : values) value = distribution(engine);
    return values;
}
```

```cpp
class Stopwatch {
public:
    void start() noexcept { started_ = std::chrono::steady_clock::now(); }
    [[nodiscard]] double elapsed_seconds() const noexcept {
        return std::chrono::duration<double>(std::chrono::steady_clock::now() -
        ↪  started_).count();
    }
private:
    std::chrono::steady_clock::time_point started_{};
};
```

`random_values` 带默认种子，**同一个种子生成同一组数据**——这是能不能复现实验的前提；计时用 `steady_clock` 而不是 `system_clock`，因为后者会被系统对时改动。

用它们量一遍（g++ 13.3，`-O2`，随机数据，单位毫秒；上机题第 1 题要求的正是这张表）：

n	插入	选择	冒泡	堆排	快排	归并
1000	0.1	0.2	2.8	0.0	0.0	0.0
10000	7.9	18.9	162.4	0.7	0.4	0.5
50000	213.5	482.1	5821.9	3.0	2.2	3.1

两件事值得停下来看：

1. **同为 $\Theta(n^2)$，冒泡比插入慢 27 倍**（5822 对 214）。渐近记号把常数丢掉了，而这里的常数差异来自「冒泡每遇到一个逆序对就做一次三步交换，插入只做一次搬移」。Θ 告诉你 n 变大时谁会输，不告诉你在你关心的那个 n 上谁更快。
2. n 从 1 万涨到 5 万（5 倍），$\Theta(n^2)$ 的三个涨了 25～36 倍（理论值 $5^2 = 25$，冒泡偏高；渐近记号只管增长的阶，不保证每一档都正好是这个倍数），$\Theta(n \log n)$ 的三个只涨了 4～6 倍（理论值约 $5 \times 1.17 \approx 5.9$；零点几毫秒的计时本身误差就大）。**这就是量出来的数量级**。

再看输入形态的影响（2 万个数）：

	插入	冒泡	优化快排
已经排好序	0.0	0.0	105.9
随机	58.5	675.1	0.7

优化快排在已排好序的输入上比插入排序慢了不止 100 倍。原因在 8.4.2 讲过：枢轴取末元素，有序输入每次划分都最坏，$\Theta(n^2)$；那两处优化管的是栈深和常数，救不了最坏时间。这也是为什么实务中的快排一定要配三数取中或随机枢轴。

原书还量出了几条对数据分布的敏感性结论，值得一并记住：

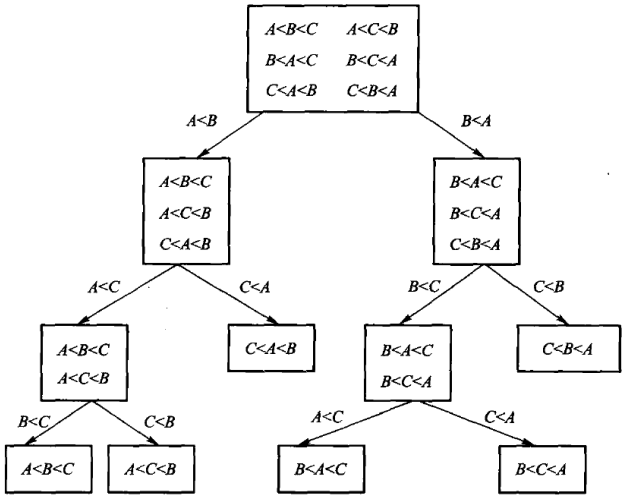
- **选择排序、归并排序、基数排序对数据分布情况不敏感**；插入排序、冒泡排序、Shell 排序都是正序 比逆序要快，而且在正序或基本有序等特殊情况下，插入排序、冒泡排序等简单的算法是最好的。
- **快速排序的正序和逆序速度一样**，都比随机分布要快——这是因为原书的算法选取中间点作为轴值，在输入数据为正序或逆序时正好平分序列。（本书实现取末元素作枢轴，所以在这两种输入上反而最慢，上面那张表就是证据。**同一个算法，选轴策略不同，对输入形态的反应可以完全相反**。）
- **堆排序逆序比正序稍微快一些**，因为算法采用最大堆，逆序时已经自然成堆，建堆速度比较快。
- 顺序基数排序比链式要快；但如果记录是比较大的组合类型，应该采用链式存储以减少排序过程中的 记录移动，从而提高速度。

标准库里的排序是怎么做的。 ANSI C 提供了 `qsort` 快速排序。C++ 标准库根据不同的需求提供了 不同的排序函数：`sort` 对给定区间所有元素进行排序，`stable_sort` 进行稳定排序，`partial_sort` 部分排序，`nth_element` 找出给定区间的某个位置对应的元素。`std::sort` **实际上是一种基于快速 排序的混合算法**（今天通称 introsort）：首先采用快速排序，其轴值为三点选中，当长度小于阈值（原书记为 $n < 20$）时停止递归，留下分块有序的子序列最后进行整个序列的插入排序；为了避免出现 性能退化为 $O(n^2)$ 的情况，还引入了一个递归计数——**当递归深度超过一定阈值（原书记为 $2\log n$）时，算法转入到代价为 $O(n \log n)$ 的堆排序**。这样最坏情况下也能达到接近 $O(n \log n)$ 的性能：最坏情况比堆排

序差一些，但比快速排序好得多，而平均性能与快速排序差不多。`stable_sort` 采用归并排序，`partial_sort` 采用堆排序。

这段话正好解释了本书为什么不用 `std::sort` 顶替这一章：它是把本章三种算法拼起来的产物，读懂它的前提恰恰是先把这三种都手写一遍。

8.7.3 排序问题的下限



直接插入排序、冒泡排序等算法的平均时间代价是 $\Theta(n^2)$，堆排序、快速排序、归并排序是更加有效的 $\Theta(n\log n)$ 算法；在某些特殊情况下基数排序的时间代价接近 $\Theta(n)$，但从实验数据中也可看到，基数排序在大部分时候远比 $\Theta(n\log n)$ 的快速排序慢得多。快速排序 几乎是所有这些算法中最快的排序，**那么还有没有更快的排序?**

这一小节分析排序问题本身的时间代价的上限和下限。**所谓排序问题的下限，就是解决排序问题所能 达到的最佳可能效率，即使尚未设计出算法；而上限则是指已知的最快算法所达到的最佳渐近效率**。如果问题的上限与下限相同，那么从渐近分析的意义上说，不可能有更有效的算法，因此没必要再去 花费大量时间精力去试图改进。

先看一条谁也绕不过去的下限：任何算法的时间代价都不可能小于它的 I/O 时间，因此没有哪种排序 算法能够将时间下限降到 $\Omega(n)$ 以下——算法至少要花 n 步来读入 n 个待排序的数据、输出 排序后的 n 个结果。实际上，排序的时间只可能在 $\Omega(n)\sim O(n\log n)$ 之间。已经知道所有 基于比较的排序算法的上限为 $O(n\log n)$，那么它的下限能不能降到 $\Omega(n)$?

答案是不能。上面所有比较排序，最坏时间没有一个低于 $\Theta(n\log n)$。这不是巧合：**任何基于比较的排序，最坏情况下至少需要 $\Omega(n\log n)$ 次比较**。

证明用判定树。把一次排序的执行过程画成一棵二叉树：每个内部结点是一次比较「$a_i < a_j$?」，两条边是两种结果，每片叶子是一种最终的输出排列。算法在某个输入上做了多少次比较，就等于从根走到对应叶子的路径长度。

n 个互不相同的元素有 $n!$ 种排列，每一种都必须能被这棵树区分出来，否则算法会对两个不同的输入给出同一个答案——所以**叶子至少有 $n!$ 片**。高度为 h 的二叉树最多有 2^h 片叶子，于是 $2^h \geq n!$,

$$h \geq \log_2(n!) = \Theta(n \log n)$$

最后一步不必用 Stirling 公式，原书用的初等估计就够：$n!$ 的 n 个因子里，较大的一半（$n, n-1, \cdots, \lceil n/2 \rceil$）每个都不小于 $n/2$，所以 $\log_2(n!) \geq \log_2 \left(\frac{n}{2}\right)^{n/2} = \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$；另一方面 $n! \leq n^n$，$\log_2(n!) \leq n \log_2 n$, 两边合起来就是 $\Theta(n \log n)$。而树的高度正是最坏情况下的比较次数。

这个下界**只约束比较排序**。桶式排序和基数排序不比较元素，它们直接用关键码的值算位置，所以能做到 $\Theta(m+n)$ 或 $\Theta(d(n+r))$——它们没有违反下界，是站在下界管辖范围之外。代价是对关键码的形态有额外要求：值域要够密，或者位数要够少。

本章小结

其实所谓排序，就是将线性表序列中的记录按照特定的顺序排列起来。**如果存在多个具有相同排序码的 记录，经过排序后这些记录的相对次序仍然保持不变，则这种排序算法称为「稳定的」**，否则称为「不稳定的」。

作为最常用的算法，**排序、检索和索引历来是数据结构讨论的重点问题**。检索是直接面向用户的，往往 要求比较好的性能，即以较快的检索速度把最相关的检索结果呈现给用户；为了提高检索速度往往需要 预先索引，而索引可能要求事先对数据进行排序，而且为了把最相关的检索结果呈现给用户，也需要把 检索结果按照相关性排序。

排序算法最能够体现算法设计和算法分析的魅力，其速度要求非常高：**内排序主要考虑的是怎样减少 关键码之间的比较次数和记录移动次数来提高排序速度；而外排序则考虑外存的特性，尽量减少访外操作**。

本章介绍的几种经典内排序算法，思想可以总结如下：

1. **直接插入排序**：依次对第 i 个记录进行排序，将当前无序区的第一个记录 `Array[i]` 插入到有序区 `Array[0..i-1]` 的合适位置。
2. **Shell 排序**：按照某种增量序列依次操作。在处理第 i 个增量 d 时，把整个待排数组中距离 d 倍数的那些记录放在同组子序列中，组内进行插入排序；逐渐扩大小序列的规模，而减少小序列个数；最后所有序列都合并在一个大序列中进行一趟插入排序。**注意，如果最后一个增量不是 1，则需要对 整个数列进行一次插入排序**。
3. **直接选择排序**：依次从剩余记录中找出第 i 小的记录，直接交换到正确位置（本趟待排子序列的 最前端）。
4. **堆排序**：对所有记录建立最大堆（也可以用最小堆），取出堆顶的最大记录，重新建堆；不断重复这一 操作，直到堆为空，得到完整的有序序列。

5. **冒泡排序**：依次比较相邻的两个记录，若逆序则交换，每趟把当前无序区中最大的记录“冒泡”到本趟的最末端。反复进行直到某一趟没有发生交换为止。
6. **快速排序**：首先从待排序序列中任意选择一个记录 k 作为轴值，然后将剩余的记录分割成左子序列 L 和右子序列 R；L 中的记录都小于或等于轴值 k，R 中的记录都大于或等于轴值 k，k 正好到位（等于 k 的记录分到哪一侧取决于分割写法：本书代码把它们全放进 R）。原问题分解为 L 和 R 这两个规模更小的同结构子问题，递归进行直到子序列中只含有 0 或 1 个元素时退出递归。
7. **归并排序**：将序列划分为两个子序列，对子序列递归；反复比较两个有序子序列的最小记录，取较小者 出列进入结果序列，合并为一个有序序列。
8. **桶式排序**：扫描一遍序列进行统计计数，然后再统计小于等于编号 i 的个数，给各桶预留适当的 空间；从序列尾部往前再一次进行扫描，将具有相同值的记录都分配到同一个桶中；最后依次按照编号 从桶中取出记录。
9. **基数排序**：将排序码拆分为 d 个子码，按照低位优先的顺序进行 d 趟桶式排序。有顺序存储和 静态链式存储两种实现方法。

插入排序、冒泡排序、归并排序、分配排序（桶式、基数）都是稳定的排序算法。若要证明一个排序算法 是稳定的，则需证明：对于任意两个关键码相同的记录 r_i、r_j（$K_i = K_j$，序号 $i < j$），排序后 它们的新位置为 i'、j'，其相对序号仍保持 $i' < j'$。**可以举反例来证明一个排序算法是不稳定的——直接选择排序、堆排序、Shell 排序、快速排序都是不稳定的排序算法**。

时间代价在排序算法中居于非常重要的地位，是衡量排序算法最重要的指标。排序总时间代价是比较和 移动次数之和；根据应用需要可以分别考察最小、平均、最大时间代价。对于记录比较大的情况，可以采取 减少移动记录的策略，例如索引排序法。

直接插入、直接选择、冒泡这 3 种排序算法的时间代价都很大，在平均和最差情况下的时间代价均为 $\Theta(n^2)$。**三者之中，插入排序的试验时间最好，而冒泡排序的性能最差**，因此插入排序经常被用来 与其他排序方法结合使用。快速排序、归并排序、堆排序、基数排序都是 $\Theta(n \log n)$ 量级的排序；**快速排序的确是最快的，比同级别的排序算法都要快一些，尤其是数组规模较大时更为明显**，因此快速 排序在实践中应用十分广泛。本章证明了任何基于比较的排序算法在最坏情况下都至少需要 $\Omega(n \log n)$ 次比较，而堆排序、归并排序 恰好达到 $O(n \log n)$，因此 $\Theta(n \log n)$ 就是（基于比较的）排序问题的时间代价。

空间代价考虑排序算法所需要的额外空间，包括编译栈消耗的辅助记录空间。直接插入排序、直接选择 排序、冒泡排序、Shell 排序、堆排序的空间代价为 $\Theta(1)$，快速排序平均为 $\Theta(\log n)$（最坏 $\Theta(n)$，只对短侧递归的优化版最坏也是 $O(\log n)$），归并排序为 $\Theta(n)$，桶式排序为 $\Theta(n+m)$，基数排序为 $\Theta(n+r)$。如果有特殊的空间限制，则要注意 采用辅助空间较小的算法。

把本章各算法的代价放进一张表（时间按比较、移动次数计，关键码互不相同；「本书实现」指同节代码）：

算法	最好	平均	最坏	辅助空间	稳定
直接插入	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	稳定
Shell（增量减半）	$\Theta(n\log n)$	取决于增量序列	$\Theta(n^2)$	$\Theta(1)$	不稳定
直接选择	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	不稳定
堆排序	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(1)$	不稳定
冒泡（带无交换退出）	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	稳定
快速排序（本书：末元素作轴）	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n^2)$	平均 $\Theta(\log n)$，最坏 $\Theta(n)$	不稳定
归并排序（基本版）	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n)$	稳定
桶式排序	$\Theta(n+m)$	$\Theta(n+m)$	$\Theta(n+m)$	$\Theta(n+m)$	稳定
基数排序（d 趟，基数 r）	$\Theta(d(n+r))$	$\Theta(d(n+r))$	$\Theta(d(n+r))$	$\Theta(n+r)$	稳定

Shell 排序的最好情况指输入已经有序：每一趟每个记录只比较一次，$\log n$ 趟共 $\Theta(n\log n)$。归并排序的优化版（合并前先比较两段交界）在已经有序的输入上只要 $\Theta(n)$。

在实际应用中，可以根据排序规模（n 的大小）、稳定性要求、待排数组有序状况、排序码的组合情况等来 选择合适的排序算法，或者对这些排序算法进行适当的组合。**可以遵循以下一些原则：**

1. 当待排序的关键码序列基本有序时，直接插入排序最快，冒泡排序速度也较快；
2. 归并排序对待排序关键码的初始排列不敏感，因此排序速度比较稳定；
3. 若待排序的记录个数 n 较小时，可采用直接插入或直接选择排序；
4. 若 n 较大时，则应采用时间代价为 $O(n\log n)$ 的快速排序、堆排序、归并排序或基数排序方法；
5. 当 n 较大、且输入顺序比较随机（即杂乱无序）时，如果没有稳定性要求，则采用快速排序效果最好；
6. **当 n 很大且关键码位数较少时，采用静态链的基数排序效果比较好**。

本章所有算法都是手写实现，不用标准库排序替代。

习题

补充复杂度题（参考课程第 8 章）

1. 用随机划分实现 Randomized-Select，求无序数组第 `k` 小元素，并说明期望复杂度。
2. 对字符串按长度分桶，再按字符位做稳定基数排序；证明总时间与所有字符串长度之和同阶。
3. 给出索引排序与直接排序的区别，并说明重复键时如何保持稳定性。

正文习题

1. 对序列 `{49, 38, 65, 97, 76, 13, 27}` 分别画出直接插入、冒泡、简单选择的第一趟结果。
2. 说明为什么直接插入稳定、堆排序不稳定。举一个使堆排序打乱相等键次序的例子。
3. 写出快速排序一次划分的过程，并指出最坏输入。
4. 对 8 个元素用手算堆排序：先建大顶堆，再写出每一趟交换堆顶之后的数组。
5. 二路归并需要多少额外空间？能否原地归并？代价是什么。
6. 基数排序为什么对有符号整数要先处理符号位。
7. 在什么情况下你会选插入而不是快排？选堆而不是归并？

原书习题

> 以下是原书第 8 章的习题（原书共 33 道，此处按扫描件补回题面）；**参考答案见 `book/`习题与参考答案`.md` 的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。

1. 试构造 5 个整数元素进行排序且最多只用 7 次比较的算法思想。
2. 写一个处理整数排序码的插入排序算法。算法要满足以下条件：**输入数据存放于栈中**（不是存放在 数组中），并且程序中只许用一定的整数及栈；算法结束时排序结果放在栈中，栈顶元素最小。最差 情况下算法的执行时间为 $O(n^2)$。
3. 试给出采用增量序列 $\{2^k-1, 2^{k-1}-1, \cdots, 7, 3, 1\}$ 的 Shell 排序的时间代价为 $\Theta(n^{3/2})$ 的推理过程。
4. 采用增量序列 $\{2^k-1, 2^{k-1}-1, \cdots, 7, 3, 1\}$ 重新对图 8.5 中的例子进行排序，试写出排序 各阶段时的数组状态，与原来的排序过程进行比较。
5. 直接选择排序的最少交换次数为 $\Theta(n)$，因为算法并不检查第 i 个元素是否已经在第 i 个位置，即有可能带来不必要的交换。(1) 改进算法使之没有不必要的交换；(2) 这种改进能加快速度吗？试说明 理由；(3) 编程验证一下原始排序算法和改进算法的运行时间，哪个算法实际上更快？
6. 考虑这样一种**计数排序**：对每个待排序记录，扫描整个序列统计比它小的记录个数 c，则 c 就是它 在序列中的正确位置。值得注意的是，**计数排序只适用于不存在重复记录的情况**。编程实现这种算法，并分析其时间代价，与直接选择排序进行比较。

7. 在堆排序中采用的堆是基于二叉树的，因此时间代价为 $\Theta(n\log n)$。如果采用基于三叉树的堆来 实现堆排序，时间代价是否会变成 $\Theta(n\log_3 n)$？试编写出基于三叉树的堆排序算法，并分析 时间代价。
8. 冒泡排序函数中有循环 `for (int j = n-1; j >= i; j--)`，考虑一下将其换成 `for (int j = n-1; j > 0; j--)` 的影响：新的算法是否还能正常执行？这种改变是否会影响到程序的 近似复杂性？此改变对时间代价有何影响？
9. 冒泡排序的一个变种就是所谓的**鸡尾酒混合排序**：与本章中介绍的冒泡排序一样，它对数组总共也要 进行 $n-1$ 趟排序，但是**相邻两趟的冒泡方向是相反的**——例如第一趟排序将最大的记录起跑到数组 的底部，而在第二趟排序中将最小的记录冒泡到数组的顶部。试编程实现该算法。
10. 在冒泡排序过程中，什么情况下排序码会朝向与排序相反的方向移动？试举例说明。在快速排序过程中 有这种现象吗？（提示：排序相反的方向是某个元素从它现在的位置向其最终的位置移动的方向。）
11. 通常，快速排序是将数组划分成两部分，当数组中有大量重复记录时，试将快速排序改为三**分数组**（将数组划分成大于、等于和小于轴值三部分），并讨论算法对包含 d 个不同值的长度为 N 的数组的 效率。
12. 本章中算法 8.6 给出的快速排序是通过递归来实现的。(1) 试用栈来代替递归，减少函数调用次数，优化该算法；(2) 在最差情况下栈有多深？(3) 怎样组织递归调用的顺序可以让栈的深度减到最小，可以减为多少？(4) 可以用队列，而不用栈来消除快速排序中的递归吗？
13. 一个非零数列中有些为正数、有些为负数。试编写一个算法，重新排列这些非零实数，使得负数排在 前面、正数排在后面。（提示：参考快速排序的分割算法。）
14. 设有一个仅由红、白、蓝 3 种颜色的条块组成的序列，各种色块的个数是随机的，但 3 种颜色色块的 总数为 n。试编写一个时间复杂度为 $\Theta(n)$ 的算法，使用辅助空间最少的算法，使得这些条块 按照红、白、蓝的顺序排好序，即排成**荷兰国旗图案**。试分析你的算法的时间和空间代价。
15. 定义「中位数」为长度为 n 的序列的第 $\lceil (n+1)/2 \rceil$ 个或第 $\lfloor (n+1)/2 \rfloor$ 个（若对该序列排序，就是排在该位置上的值；注意当 n 是偶数时中位数有两个）。试设计一个 $\Theta(n)$ 的算法求给定序列的中位数。
16. 设 A、B 是长为 n 的数表，已经按照非降顺序排好序。如果将这 $2n$ 个数全体排序，则处于第 n 个位置的数称为中位数。设计一个最坏情况下 $O(\log n)$ 时间的算法求 A 和 B 的中位数：
    (1) 描述算法的设计思想；(2) 证明算法的时间复杂性。
17. 为了从由 n 个元素构成的未排序序列 $S=\{s_1, s_2, \cdots, s_n\}$ 中选择出第 k 个最大的元素，考虑「中位数的中位数」算法：(a) 如果 $n \le 5$，对 S 排序且直接选出第 k 个最大元素；(b) 否则 把序列 S 分割成长度为 5 的子序列；(c) 对长度为 5 的子序列用任意一种方法进行排序；(d) 从每个 子序列中选取一个中位数，形成一个含 $\lfloor n/5 \rfloor$ 个中位数的序列 M；(e) 递归重复这一 选择过程，找出序列 M 中的中位数 m；(f) 将序列 S 分割成 3 个子序列 $\{L, E, G\}$，

使得 L 中的元素都比 m 小、E 中的元素都等于 m、G 中的元素都比 m 大；(g) 如果 $k \leq |L|$，递归重复这个过程选择 L 中的第 k 个最大元素；如果 $|L| < k \leq |L| + |E|$，结果就是 m；否则选择 G 中的第 $k - (|L| + |E|)$ 个最大元素。问：(1) 此算法的运行时间是多少？(2) 如果用这个算法选择轴值，试证明快速排序的最差运行时间是 $\Theta(n \log n)$。

18. 本章中介绍的是两路归并算法, 如果是 4 路或者 8 路归并, 应该如何改写? 试分析改写后的算法代价，并与原来的算法进行比较。
19. 有人说：「由于归并排序的时间代价为 $\Theta(n \log n)$，插入排序的时间代价为 $\Theta(n^2)$，因此 当对 256 个数字进行排序时，归并排序的速度将是插入排序的 64 倍!」**试问这种说法是否正确?** 并说明理由。
20. 本章中算法 8.8 给出的归并排序是通过递归来实现的。(1) 试用栈来代替递归, 编写一个非递归的 归并排序算法；(2) 不用栈、也不用队列，编写一个非递归的归并排序算法。
21. 本章中介绍的归并函数中，需要用到一个长度为 n 的辅助数组来保存合并后的记录。试对该函数进行 改进，在不增加算法的时间代价前提下尽量减小空间代价。
22. 编写一种对扑克牌进行排序的基数排序。花色为高位排序码，面值为低位排序码，大小顺序为：方块 < 梅花 < 红桃 < 黑桃，2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A。
23. 本章介绍了很多排序算法，分别列出哪些算法具有以下特性：(1) 不需要访问那些已排好序的记录；
    (2) 在整个排序过程中，算法每次都要访问序列中的所有记录。
24. 下面的各个操作中，哪一个最适于先进行排序处理? 对于这些操作分别简短地描述一个实现算法，并给出算法的渐近复杂性：(1) 找最小值；(2) 找最大值；(3) 计算算术平均值；(4) 找中位数；
    (5) 找出现次数最多的值。
25. 在下列情况下, 你会选择哪种排序算法? 并说明理由。(1) 需要对 1000 个数字进行排序, 程序只需运行 一次; (2) 只需要对 50 个数字进行排序, 程序只需运行一次；(3) 在编写一个很庞大的程序中需要编写 一个对 5 个记录进行排序的函数, 而且该函数将被反复调用很多次; (4) 需要对 1000 个大型的记录进行排序，记录本身存储在外存中，在内存中只保存了所有记录的排序码——排序码之间的比较非常快，但移动代价很大，因为一旦移动一个排序码，相应的外存中的记录也要移动, 将涉及上百个磁盘块的 移动; (5) 在图书馆里计算机类书籍区一共有 12 列书架，书架上的书本来都是按照编目号排列好的，但其中有些书被读者放错了地方，但通常不会偏离超过一个书架，试设计一个算法将这些书重新放回 正确的位置；(6) 需要将 500 张随机排列的图书卡片按照字母顺序排好序；(7) 已知一个包含了 5000 个 单词的列表已按字母顺序排好序，需要再进行一次检查，确保所有单词已经排好序。
26. 某整型数组 A 的 10 个元素值依次为 $\{6, 2, 9, 7, 3, 8, 4, 5, 0, 1\}$，用下列各排序方法将 A 中的 元素由小到大排序：(1) 取第一个元素值 6 作为分割数,

试写出快速排序第一次分隔后 A 中的结果；

(2) 用堆排序，试写出将第一个选出的数据放在 A 的最后位置上、将 A 调整成堆后的 A 中结果；

(3) 用基数为 3 的基数排序法，试写出第一次分配和收集后 A 中的结果。

27. 已知一组元素的排序码为 $(46, 74, 16, 53, 14, 26, 40, 53', 86, 65, 27, 34)$。

(1) 利用直接插入排序的方法写出每次向前面有序表插入一个元素后的排列结果；

(2) 利用直接选择排序方法写出每次选择和交换后的排列结果；

(3) 利用最大堆排序的方法写出在构成初始堆和利用堆排序的过程中，每次过筛（代码 5.11 中的函数 `SiftDown`）运算后的排列结果，并画出初始堆所对应的完全二叉树；

(4) 采用快速排序，每次都取子序列的最左元素为轴值，写出每一层划分后的排列结果，并画出由此 快速排序得到如图 8.6 所示的二叉搜索树；

(5) 利用归并排序的方法写出每一趟二路归并排序后的结果。

28. 如果只想在一个有 n 个元素的任意序列中得到其中最小的第 $k(k \ll n)$个元素之前的部分排序 序列，而且这个 n 是比较小的值，**采用什么排序方法能得到比用堆排序的比较次数更少?**例如序列 $\{503, 017, 512, 908, 170, 897, 275, 653, 612, 1$ 要得到其第 4 个元素之前的部分有序序列 $\{017, 094, 154, 170\}$，用所选择的排序算法（非堆排序）实现时要执行多少次比较？若用堆排序算法，要执行多少次比较?

29. 设 $S_1, S_2, \cdots, S_k$ 是 k 个集合，每个集合均由 n 个实数所组成。试设计一个算法将所有的和 $x_1 + x_2 + \cdots + x_k$（其中 $x_i \in S_i$，$i = 1, 2, \cdots, k$）从小到大排序，并分析算法的时间 代价。

30. 设 k 为给定常数，$A_1 = \langle x_1, x_2, \cdots, x_k \rangle$，$A_2 = \langle y_1, y_2, \cdots, y_k \rangle$ 是 k 维正整数序列。如果存在 $x_1, x_2, \cdots, x_k$ 的排列 $x'_1, x'_2, \cdots, x'_k$ 满足 $x'_1 < y_1, x'_2 < y_2, \cdots, x'_k < y_k$，则称 A_1 嵌入 A_2，记作 $A_1 < A_2$。现有 n 个 k 维序列 $A_1, A_2, \cdots, A_k$ 构成集合 B。(1) 说明 B 上的 嵌入关系有哪些性质；(2) 设计一个算法求使得 j 最大的 $A_{i1}, A_{i2}, \cdots, A_{ij}$，并且 $A_{i1} < A_{i2} < \cdots < A_{ij}$；(3) 说明算法的正确性并分析算法在最坏情况下的复杂性。

31. 像静态链基数排序那样，数组元素增加一个 `next` 域逻辑链表，对这个静态链表实现插入排序（只修改 `next` 静态指针，不移动元素），则称为「**表插入排序**」。(1) 实现这个表插入排序算法，算法执行后，这个长度为 n 的数组由其内部的静态链链指针 `next` 串成一个已排序序列（不减的 顺序）；(2) 设计一个只使用一个辅助结点的算法，在 $\Theta(n)$ 时间内，把这个由静态指针 `next` 串成的有序数组，整理成为从数组的 $0 \sim n-1$ 位置存放的不减的序列。原书在第 (1) 问之后给出的 示例（下标 0 是辅助头结点，`next[0]` 指向最小记录）：

下标	0	1	2	3	4	5	6	7	8
key	—	49	38	65	97	76	13	27	52
next	6	8	1	5	0	4	7	2	3

下标	0	1	2	3	4	5	6	7	8

32. 考虑由 3 个不同记录组成的序列 $\{A, B, C\}$，试分别画出表示下列排序算法的二叉判定树：直接插入排序、直接选择排序、冒泡排序。
33. 设计一个算法，将数组 $A(0..n-1)$ 中的元素循环右移 k 位。假设原数组序列为 $a_0, a_1, \cdots, a_{n-2}, a_{n-1}$，移动后的序列为 $a_{n-k}, a_{n-k+1}, \cdots, a_0, a_1, \cdots, a_{n-k-1}$。**要求只用一个元素大小的附加存储，元素移动或交换次数为 $O(n)$。**

上机题

1. 用同一组随机数据比较插入、堆、快排、归并的运行时间，画出 n 与时间的关系。
2. 构造使快排退化的输入，观察递归深度。
3. 实现带监视哨的插入排序，并与本章实现对拍。
4. 对含负数的整数序列验证基数排序与 `std::sort` 结果一致。

原书上机题

原书上机题的参考答案见 `book/习题与参考答案.md` 的「原书上机题参考」，由 R14 逐题守着。

1. 编程实现下述算法，并与教材上的算法进行时间效率比较，然后用理论分析支持你的试验结论。
   (1) 教材中是用插入排序算法线性搜索来寻找正确位置的；在插入第 i 个记录时，前面的记录已经 有序，因此可以考虑用二分法查找第 i 个记录的正确位置，这种算法称为**二分法插入排序**（binary insert sorting）；(2) **交换式插入排序**（swap insert sorting）改写教材中每一次插入时的 操作：当插入第 i 个记录时，如果该记录比它前面的那个记录小，则交换该记录与其前面的那个记录，即将该记录向前移动一个位置，然后继续向前比较，直到遇到一个小于或等于它的记录，本次插入才完成。
2. **奇偶交换排序**是另一种交换排序。它的第一趟对序列中的所有奇数项 i 进行扫描，第二趟对序列中 的所有偶数项 i 进行扫描；若 `A[i] > A[i+1]` 则交换它们。第三趟又对所有的奇数项，第四趟对所有 的偶数项，如此反复，直到整个序列全部排好序为止。(1) 这种排序方法结束的条件是什么？
   (2) 写出奇偶交换排序的算法，并实验测试其时间代价；(3) 当待排序排序码序列的初始排列是从小到大 有序，或从大到小有序时，在奇偶交换排序过程中的排序码比较次数是多少？分析奇偶交换排序的平均 时间代价。
3. 改写算法 8.6 中的快速排序算法（原书此处印作「算法 8.8」，8.8 是归并排序），采用一种不同的轴值选择方法：**选择序列中所有记录的平均值作为 轴值**。试设计并实现一个 `MeanPivotQuickSort` 类，使其能把要排列的序列分割成比平均值大和比 平均值小的元素。注意：需要修改原来的分割函数，因为现在轴值可能不是序列中的元素。试编程比较 两种算法的实际运行时间。

4. 试写一组英文单词按字典序排序的基数排序算法。单词区分大小写，设最大的单词有 n 个字母。本章中介绍的排序算法大都是基于数组实现的，试采用链式存储来改写本章的排序算法，并试验比较 算法的数组实现和链式实现效率。此时，可以发现大部分算法都能用链表来实现，**但有些特殊算法不适于链式存储，试分析为什么这些算法不适合链式存储**。

第 9 章 文件管理与外部排序

当数据大到内存装不下时，排序的瓶颈变成磁盘读写。外部排序先生成有序顺串，再进行多路归并。置换选择用最小堆尽可能延长当前顺串；竞赛树维护多路输入中当前最小的选手。

源码：置换选择与竞赛树、可运行示例、测试。

9.1 主存储器和外存储器

前面各章的结构基本上都在内存里，也叫内部数据结构。内存容量有限，程序一结束数据也就没了。大规模数据必须放到外存上，排序过程也就变成多次内存与外存之间的交换。

主存（RAM、cache、显存）在主板上，按单元连续编址，CPU 直接访问，一次存取时间可以看成很小的常数，单位是纳秒。常见主存还包括高速缓存和视频存储器；常见外存则包括硬盘、软盘、磁带、光盘和 U 盘等。主存中存放的是当前正在执行的程序和数据，程序结束后其中的状态随之消失；外存便宜、容量大，信息不易失，也便于携带：断电后数据仍在，并能在不同设备之间传递。但一次存取以毫秒甚至秒计，比内存慢几个数量级。

外存慢，一个原因是每次访问都要先定位再读写。磁盘要把磁头移到目标磁道，再等扇区转到磁头下，定位往往就要几毫秒到几十毫秒，远慢于真正把数据读出来。所以外存按固定大小的页块存取：一次定位读写一整页，减少定位次数。顺序扫描时再配合缓冲：一次读入一页或几页到内存，后面的访问尽量打在缓冲区里。

对外排序来说，目标首先是减少读写次数，而不是减少内存里的比较。一次磁盘 I/O 往往比一次比较贵几个数量级。

先建立成本模型

分析外排序时，不能再只数比较次数。设文件有 N 个记录，每页可放 B 个记录，顺序扫描一遍至少要读 $\lceil N/B \rceil$ 页。若这一遍还要把结果写回外存，I/O 量就是大约 $2\lceil N/B \rceil$ 页；内存中多做几次比较，通常远比多一次整文件读写便宜。

层次	访问单位	本章关心的性质
CPU cache / 主存	字或 cache line	可随机访问，比较和交换在这里完成
SSD / 磁盘	页块	定位与传输有固定成本，应顺序、批量读写
磁带等顺序介质	连续块	随机定位尤其昂贵，归并比原地交换合适

例如 1 亿条 16 字节记录约占 1.6 GB。若内存工作区只有 64 MB，任何 “把

全部记录放进数组再排序”的内部排序都不成立；外排序必须让大部分记录始终留在外存，只在内存里保存少量缓冲页和当前候选。

9.2 文件的组织和管理

文件是外存上的数据结构，由大量性质相同的记录组成。记录是有独立逻辑意义的一块数据，简单可以是一串字符，复杂则由若干字段组成。操作系统文件常常是连续字符流，结构不明显；数据库文件是有结构的记录集合，每条记录由若干不可再分的数据项组成。学生登记表——姓名、学号、性别、出生年月——就是后一种。

按记录长度，文件分定长和不定长：定长更好处理。按关键码个数，分单关键码和多关键码：多关键码文件除了主码还可以有若干次码。操作通常以记录为单位：顺序读、追加、按条件修改或删除。处理方式有实时（要求很快应答）和批量（允许较长反馈）。

用户看见的是逻辑文件：顺序定长、顺序变长、或按关键码存取。系统实现的是物理文件，常见几种：

1. **顺序文件**：记录按逻辑次序放进连续物理块，物理顺序与逻辑顺序一致。顺序扫描很快，按关键码插入、删除要搬很多块。
2. **索引文件**：主文件之外另造索引，先查索引再读记录。第 11 章专门讨论。
3. **散列文件**：用散列函数把关键码映射到桶或块。第 10 章的闭散列思想可以搬到外存，但冲突处理要按块设计。

本章不实现页缓存和文件句柄，只抽出外排序里两件与文件组织无关、却决定 I/O 次数的事：如何生成更长的初始顺串，以及如何在 k 路归并里选出当前最小。

9.2.1 文件组织

原书区分了逻辑文件和物理文件，这个区分仍然重要：前者是应用看见的记录顺序与字段，后者是 系统实际读写的页、块和地址。顺序、散列、索引、倒排四种组织方式也仍是本书第 9–11 章的共同 骨架，只是今天通常由文件系统、数据库或对象存储系统实现，而不是由业务程序直接搬动磁盘扇区。

组织方式	保留的核心取舍	当代说明
顺序文件	连续扫描快，按中间位置更新贵	日志、列式批处理和外排序的顺串都利用这种顺序 I/O。机械磁带是历史例子，原则仍适用于顺序读写 SSD、云对象和网络流。

组织方式	保留的核心取舍	当代说明
散列文件	按键定位快，范围扫描与有序输出差	不直接把内存散列表搬到磁盘；冲突、页满和缓存未命中都必须按页处理，相关基本思想见第 10 章。
索引文件	用较小目录把键映射到记录位置	现代数据库常用 B+ 树、LSM 等页级索引；本书第 11 章用页模拟讲 B+ 树。
倒排文件	用属性值反查记录集合	搜索系统仍以它为核心，只是词典、压缩和分片比原书时代复杂得多。

顺序文件的 “物理顺序与逻辑顺序一致” 不能理解成文件一定连续占用磁盘扇区。现代文件系统可能重排 块位置、使用写时复制或远端对象存储；对本章而言真正要守的是**读者可以顺序消费页，写者可以批量 产生页**。这正是归并比大量随机交换更适合外排序的原因。

定长记录能用 “第 i 条在第几页” 直接定位；变长记录通常需要槽目录，把稳定的槽号映射到会移动的 页内偏移。删除时立即压紧能让扫描紧凑，留下墓碑写入更快但会积累空洞。原书把这几种选择列为 文件组织；今天仍要做同一种权衡，只是实现者多半是数据库或存储引擎，而不是调用 `seekg` 的应用代码。

9.2.2 C++ 的流文件

原书的关键概念仍然成立：**文件**是持久化的数据对象，**流**是程序在某一时刻对它进行顺序读写的 通道；流通常配有用户态缓冲，读写位置决定下一次操作从哪里开始。外排序把输入、工作区和输出缓冲 分开（图 9.1），正是利用了这层抽象。

但原书的接口已是历史写法。`<fstream.h>` 不是标准 C++17 头文件；现在使用 `<fstream>` 中的 `std::ifstream`、`std::ofstream` 与 `std::fstream`。文件名应通过 `std::string` 或 `std::filesystem::path` 表达，位置使用 `std::streamoff` / `std::streampos`，而不是 `char*` 和 `int`。流对象离开作用域会自动关闭，显式 `close()` 只在需要尽早检查关闭或复用同一对象时才有意义。

原书操作	今天仍保留的意思	现代接口与边界
`open` / `close`	取得或结束文件访问	构造时打开或调用 `open()`；用流状态检查失败，RAII 负责通常的关闭。

原书操作	今天仍保留的意思	现代接口与边界
`read` / `write`	在当前位置传输字节	二进制块 I/O 仍可用，但不能把含指针、虚函数表或依赖字节序的 C++ 对象直接写入文件。定义稳定的字段格式或序列化协议。
`seekg` / `seekp`	移动读/写位置	只适用于支持随机访问的底层对象；管道、终端和许多网络流不能任意定位。

本章的 `external_sort` 实现故意以容器模拟输入与输出，不假装一段未测试的文件 I/O 就是外排序。它验证的是顺串、归并与选择树的算法不变量；真正连接文件时，调用方还必须处理打开失败、短读短写、编码/字节序、临时文件原子替换和崩溃恢复。它们是重要的工程课题，但不是把原书算法换成错误 I/O 示例的理由。

记录怎样装进页

定长记录最容易计算：页大小为 P 字节、记录长 R 字节时，一页最多装 $\lfloor P/R \rfloor$ 条，剩余空间是内部碎片。不定长记录通常在页尾保存槽目录，槽里记录每条记录的偏移和长度；记录在页内移动时，外部引用仍可通过 “页号 + 槽号” 找到它。

删除记录也有两种代价模型。立即压紧能保持扫描紧凑，却可能搬动大量记录；留下墓碑写入便宜，但查询必须跳过空洞，积累到一定程度仍要重组。第 10 章闭散列里的墓碑解决的是同一种矛盾，只是这里的搬动单位从数组槽变成了页块。

把逻辑顺序与物理位置分开，是后面索引章节的入口：索引保存 “关键码到页/槽” 的映射，主文件不必为了每次插入都整体移动。外排序则反过来，趁批处理窗口一次性重写主文件，用顺序 I/O 换取之后更快的扫描。

9.3 外排序

第 8 章介绍的内排序算法都假定被排序的数据完全存放在内存中。**如果数据存放在外存文件中，则需要 考虑外存的特点**——采用外存文件排序技术，简称**外排序**（external sort）。

外排序处理的数据对象非常多，而且在内存不能一次处理完成。因此，需要根据内存的大小，**将外存中 的数据文件划分成若干段，每次把其中一段读入内存并用内排序方法进行排序**；这些已排序的段或有序 的子文件称为**顺串**或**归并段**（run），顺串需要重新写回外存等待将来处理，让出的内存空间继续处理 其他文件段。

外排序所需要的时间由三部分组成：内部排序所需要的时间、外存信息读/写所需要的时间和内部归并 所需要的时间。外排序必须不断地在内存与外存之间传送数据，而**外存的读/写速度与内存相比要慢得 多，因此减少外存信息的读/写次数是提高外部排序效率的关键**。

对同一个文件而言，进行外排序所需读/写外存的次数与归并趟数有关：假设有 m 个初始顺串，每次对 k 个顺串进行归并，归并趟数为 $\lceil \log_k m \rceil$。因此，为了减少归并趟数，可以从两个方面着手 ——**一是减少初始顺串的个数** m（9.3.1 **的置换选择排序），二是增加归并的顺串数量** k（9.3.3 **的 多路归并**）。

外排序分成两个阶段：

1. **顺串生成**：把输入切成若干内部有序的段；
2. **归并**：反复合并这些段，直到只剩一条覆盖全文件的顺串。

若内存能放 M 条记录，普通分批排序只能产生长度至多 M 的初始顺串，顺串数约为 $r = \lceil N/M \rceil$。做 k 路归并需要 $\lceil \log_k r \rceil$ 趟；每一趟都要读完整文件再写完整文件，所以减少一趟，省下的是约 $2N/B$ 次页 I/O，而不是几个比较。

9.3.1 置换选择排序

置换选择排序（replacement selection sorting）是在**堆排序的基础上演化而来的**。顺串是磁盘上已经 有序的一段记录；内存一次只能装 M 条时，朴素做法是每批排成一条长为 M 的顺串——顺串数因此是 $\lceil N/M \rceil$，一个也少不了。

置换选择可以做得更好：输出堆顶之后，若下一条记录**不小于**刚输出的值，它还可以进入当前顺串；否则冻结到下一趟。**置换选择排序得到的顺串长度并不相等，但平均情况下可以形成长度为** $2M$ **的 顺串**——顺串数因此大约减半，而每减少一半顺串数，归并趟数就可能少一趟，省下的是一整轮完整的 读写。整个工作过程如图 9.1 所示。

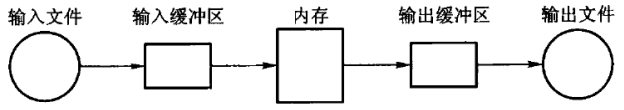


图 9.1　置换选择算法流程。输入缓冲空了才整页补入，输出缓冲满了才整页写回；工作区处理的是记录，真正与设备交换的是页。

原书图 9.2 的输入是 `50 49 35 45 30 25 15 60 16 27 1`，工作区 `M = 7`。前 7 个建成最小堆后，堆顶是 15，先输出它。接着读到 60——它比刚输出的 15 大，可以进当前堆；之后每输出一个堆顶再读一条：16、27、1 依次遇到刚输出的 25、30、35，都比它小，被冻结。第一顺串因此是

```
15 25 30 35 45 49 50 60
```

长度 8，已经超过 M。剩下的 `1 16 27` 构成第二顺串。

下面把图 9.2 的过程逐步摊开。活跃堆只容纳仍可能进入当前顺串的记录；冻结中的记录必须等下一条顺串。表中 “输出后读入” 表示先弹出最小值，再从输入缓冲补一条：

步	输出	输出后读入	去向	当前顺串	冻结
初始	-	前 7 条	建堆	-	-
1	15	60	$60 \ge 15$，活跃	15	-
2	25	16	$16 < 25$，冻结	15 25	16
3	30	27	$27 < 30$，冻结	15 25 30	16 27
4	35	1	$1 < 35$，冻结	15 25 30 35	16 27 1
5–8	45,49,50,60	输入耗尽	排空活跃堆	15 25 30 35 45 49 50 60	16 27 1
新顺串	1,16,27	-	冻结区重新建堆	1 16 27	-

这个表给出两个守门不变量：每条顺串内部非递减；全部顺串拼在一起包含输入的每条记录且恰好一次。第一条保证归并可以工作，第二条防止冻结时丢记录或重复输出。

输入次序决定顺串长度。单调递增输入会一直补进活跃堆，得到一条长顺串；单调递减输入每次读入都会被冻结，每条顺串恰好长 M（最后一条可能更短），与朴素分批没有区别。随机排列在常见独立分布假设下平均约为 $2M$，但这是平均结论，不是最坏保证。

回到图 9.1 的流程：工作区是最小堆。弹出堆顶后，下一条记录若不小于刚输出的值就入堆，否则冻结到下一趟。

若把整份输入推进一个堆再依次弹出，得到的是一条完全有序序列——那是堆排序，不是置换选择。旧实现曾经这样做，测试只断言 `{3,1,2}` → `{1,2,3}`，堆排序也能过。现在的接口返回**若干顺串**，并用原书这组数据守门。

多路归并时，每次要在 k 路的队首里选出最小者。赢者树的内部结点保存胜者下标；败者树的内部结点保存败者，另用一个冠军槽记录全局最小。替换一名选手后，两者都只需沿叶到根重赛。

赢者树和败者树统称**选择树**（tournament tree），是外部排序多路归并中的专用优化；内存内的一般 “取最小” 任务通常直接使用二叉堆或优先队列，只有需要反复从固定的多路输入中选冠军时才值得维护选择树。

先跑一遍：

```
#include "modern.hpp"

#include <iostream>
```

```
int main() {
    const std::vector<int> input{50, 49, 35, 45, 30, 25, 15, 60, 16, 27, 1};
    const auto runs = dsa::external_sort::replacement_selection(input, 7);

    std::cout << " 工作区 M=7，得到 " << runs.size() << " 个顺串\n";
    for (std::size_t index = 0; index < runs.size(); ++index) {
        std::cout << " 顺串 " << index + 1 << " (长度 " << runs[index].size() << ") :";
        for (int value : runs[index]) {
            std::cout << ' ' << value;
        }
        std::cout << '\n';
    }

    dsa::external_sort::LoserTree tree({20, 6, 8, 9, 11});
    std::cout << " 败者树当前冠军: " << *tree.winner() << '\n';
    tree.replace(1, 15);
    std::cout << " 替换后冠军: " << *tree.winner() << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch09/external_sort \
    code/ch09/external_sort/demo.cpp -o /tmp/extsort-demo
/tmp/extsort-demo
```

```
工作区 M=7，得到 2 个顺串
顺串 1 (长度 8) : 15 25 30 35 45 49 50 60
顺串 2 (长度 3) : 1 16 27
败者树当前冠军: 6
替换后冠军: 8
```

把 `M` 改成 11（装得下全部输入），会只得到一个顺串，内容是整表有序——这时置换选择退化为堆排序，正是「内存够用」的边界。

`replacement_selection(input, memory)` 先读入至多 `memory` 条建成最小堆。循环中弹出堆顶写入当前顺串；若还有输入，按「`incoming >= emitted` 则入堆，否则冻结」分流。当前堆空了，就把冻结区建成新堆，开始下一趟。

原书算法 9.1 不另开冻结数组：被冻结的记录 R 直接放到堆尾 LAST 位置，堆的规模减 1（LAST = LAST − 1），等 LAST < 0、整个数组都被冻结记录占满时，再对它重新建堆。这里的 `heap` 与 `frozen` 是同一块工作区的两部分：每弹出一条就至多补进一条，所以两者的记录数之和始终不超过 `memory`，内存占用没有超出 M。

```
// 算法 9.1：置换选择。memory 是内存工作区能容纳的记录数 M。
// 返回若干顺串：每个顺串内部有序，第一趟的平均长度约为 2M，而不是 M。
```

```cpp
// 不属于当前顺串的新记录被冻结，等当前堆耗尽后再建下一趟。
inline std::vector<std::vector<int>> replacement_selection(const std::vector<int>&
↪  input,
                                                           std::size_t memory) {
    if (memory == 0) {
        throw std::invalid_argument("replacement selection memory must be positive");
    }
    if (input.empty()) {
        return {};
    }

    std::size_t next = 0;
    std::vector<int> heap;
    heap.reserve(memory);
    while (next < input.size() && heap.size() < memory) {
        heap.push_back(input[next++]);
    }
    detail::heap_from(heap);

    std::vector<std::vector<int>> runs;
    std::vector<int> current_run;
    std::vector<int> frozen;

    while (!heap.empty() || next < input.size() || !frozen.empty()) {
        if (heap.empty()) {
            if (!current_run.empty()) {
                runs.push_back(std::move(current_run));
                current_run = {};
            }
            heap = std::move(frozen);
            frozen = {};
            while (next < input.size() && heap.size() < memory) {
                heap.push_back(input[next++]);
            }
            detail::heap_from(heap);
            if (heap.empty()) {
                break;
            }
        }

        const int emitted = detail::heap_pop(heap);
        current_run.push_back(emitted);

        if (next == input.size()) {
            continue;
        }
        const int incoming = input[next++];
```

```cpp
        if (incoming >= emitted) {
            detail::heap_push(heap, incoming);
        } else {
            frozen.push_back(incoming);
        }
    }
    if (!current_run.empty()) {
        runs.push_back(std::move(current_run));
    }
    return runs;
}
```

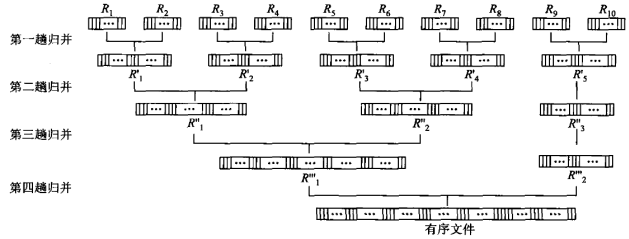
```python
def replacement_selection(values: list[int], memory: int) -> list[list[int]]:
    """ 算法 9.1：置换选择。内存里放得下 memory 个记录，产生长度可超过 memory 的顺串。"""
    if memory <= 0:
        raise ValueError("memory must be positive")
    runs: list[list[int]] = []
    heap: list[int] = []
    frozen: list[int] = []
    source = list(values)
    cursor = 0
    while cursor < len(source) and len(heap) < memory:
        heap.append(source[cursor])
        cursor += 1
    _heapify(heap)
    current: list[int] = []
    while heap or frozen:
        if not heap:
            # 工作区空了：这一趟顺串到此为止，冻结区整体解冻成下一趟的工作区。
            runs.append(current)
            current = []
            heap, frozen = frozen, []
            _heapify(heap)
            continue
        smallest = _heap_pop(heap)
        current.append(smallest)
        if cursor < len(source):
            nxt = source[cursor]
            cursor += 1
            if nxt < smallest:
                frozen.append(nxt)  # 比刚输出的还小，进不了本趟顺串
            else:
                _heap_push(heap, nxt)
    if current:
        runs.append(current)
```

```
    return runs
```

9.3.2 二路外排序

对 m 个顺串两两归并，趟数是 $\lceil \log_2 m \rceil$。置换选择把初始顺串变长，就是为了减小 m。

归并两条顺串至少需要三个缓冲页：两页分别保存两路尚未消费的记录，一页收集输出。比较两个输入缓冲的队首，把较小者移入输出缓冲；某个输入页耗尽就读该顺串下一页，输出页满则整页写回。这样每个输入页恰好读一次，每个输出页恰好写一次。



原书图 9.3 的例子有 3000 条记录，初始得到 10 条、每条 300 条的顺串：

趟次	输入顺串数	合并方式	输出顺串数	每条典型长度
0	10	初始顺串	10	300
1	10	两两归并	5	600
2	5	两对归并，1 条轮空	3	1200,1200,600
3	3	一对归并，1 条轮空	2	2400,600
4	2	最后归并	1	3000

因此确实需要 $\lceil \log_2 10 \rceil = 4$ 趟。

每一趟归并都要以页块为单位把参与归并的数据读一遍、写一遍。原书接着算了读/写次数：设每个页块可容纳 100 个记录，全文件共 30 页。第一趟所有顺串都参与，读 30 页（同时写 30 页）；第二趟 R_5' 轮空、不参与，只读写 24 页；第三趟 R_3''（即原来的 R_5'）仍不参与，又是 24 页；第四趟所有记录再扫描一遍，30 页。4 趟归并合计读 108 页、写 108 页。注意轮空的顺串不必复制，所以不是简单的 “4 趟 × 30 页 = 120 页”。

文件长度不变而初始顺串长度增加 1 倍（5 条、每条 600 个记录）时，趟数从 4 降为 $\lceil \log_2 5 \rceil = 3$，读/写各只需 24 + 24 + 30 = 78 次。因此，为一个待排文件创建尽可能长的初始顺串，可以大幅减少扫描遍数和外存读/写次数。这正是置换选择要把顺串平均拉长到 $2M$ 的动机。

归并顺序的安排也能影响读/写次数。把每条初始顺串的长度（页数）作为权，每条顺串被读写的次数等于它在归并树中的深度，总读页数就是归并树的带权路

径长度，其实质是 Huffman 树最优化问题（5.6 节）：每次挑当前**最短**的两条顺串归并。上例中第三趟原书归并的是两条 1200 的顺串（24 页），而 Huffman 策略会先把 600 与 1200 归并（18 页），总读页数由 108 降为 30 + 24 + 18 + 30 = 102。这样得到的归并树称为**最佳归并树**。k 路归并时要用 k 叉 Huffman 树，若 $(m-1) \bmod (k-1) \neq 0$，需先补 $(k-1)-(m-1) \bmod (k-1)$ 个长度为 0 的**虚段**，使每次合并都恰好凑满 k 路；例如 $m=8$、$k=3$ 时 $(8-1) \bmod 2 = 1$，要补 1 个虚段。

置换选择若把初始顺串数从 10 降到 5，就只需 3 趟，直接省掉一次完整读写。

“增加归并路数总会更快” 也不成立。k 越大，趟数越少，但至少要为每一路留一个输入缓冲，还要有输出缓冲；内存固定时，每路缓冲变小，补页更频繁。选择 k 是 “少趟数” 和 “每路有足够缓冲” 之间的折中。

9.3.3 多路归并——选择树

先算一笔账：为什么多路归并需要选择树。k 路归并是每次将 k 个顺串合并成一个排好序的顺串；对 m 个初始顺串进行 k 路归并时归并趟数为 $\lceil \log_k m \rceil$，**增加 k 可以减少归并趟数**。

但代价不是免费的。在 k（$k>2$）路归并中，为在 k 个初始顺串中的第一个记录中找出具有最小关键码值的记录，需要经过 $k-1$ 次两两比较；若得到含 u 个记录的归并段，则需要进行 $(k-1)(u-1)$ 次比较。对记录总数为 n 的文件进行外排序时，内部归并过程进行的总比较次数是

$$\lceil \log_k m \rceil \cdot (k-1) \cdot (n-1) = \left\lceil \frac{\log_2 m}{\log_2 k} \right\rceil \cdot (k-1) \cdot (n-1)$$

若 m 不变，随着 k 的增大，$(k-1)/\log_2 k$ 也将增大，**即内部归并的时间将随之增大，某种程度上削减了由于增大 k 而减少归并趟数带来的好处**。

选择树正是用来消掉这一项的。它从 k 个关键码中找出最小值所需的比较次数是树高 $\lceil \log_2 k \rceil$，于是内部归并的总比较次数变为

$$\lceil \log_k m \rceil \cdot \lceil \log_2 k \rceil \cdot (n-1) = \left\lceil \frac{\log_2 m}{\log_2 k} \right\rceil \cdot \lceil \log_2 k \rceil \cdot (n-1) \approx \log_2 m \cdot (n-1)$$

最后一步只是近似：原书在这里写的是等号 $= \lceil \log_2 m \rceil \cdot (n-1)$，但两次向上取整不能抵消。例如 $m=5$、$k=4$ 时左边是 $\lceil \log_4 5 \rceil \cdot \lceil \log_2 4 \rceil = 2 \times 2 = 4$，而 $\lceil \log_2 5 \rceil = 3$。去掉取整（或 m 恰为 k 的幂、k 恰为 2 的幂）时两边才相等。结论不受影响：主项是 $\log_2 m \cdot (n-1)$。

此时内部比较次数（主项）与 k 无关，不随 k 的增加而增加——这就是选择

树存在的全部理由。

选择树是完全二叉树，通常采用顺序存储；它也称为**竞赛树**，反映了一系列「淘汰赛」的结果，有两种 类型：赢者树和败者树。

赢者树。对于 n 名选手，赢者树是一棵含有 n 个外部结点、$n-1$ 个内部结点的完全二叉树，其中 每个内部结点记录了其左右子结点相应赛局的赢家。从树的最底层开始，每两个树叶之间进行比赛，输的选手被淘汰，赢的可以继续向上竞赛，**树根记录了整个比赛的胜者**。最小赢者树中分数小的选手 获胜。用数组作存储结构时，选手（叶结点）用数组 $L[1\cdots n]$ 表示，内部结点用 $B[1\cdots n-1]$ 表示，**数组 B 中实际存放的是数组 L 的索引**。

赢者树的一个优点是：如果一个选手 $L[i]$ 的分数值改变了，**只需要沿着从 $L[i]$ 到根结点的路径修改 二叉树，而不必改变其他比赛的结果**；为了使这个过程更高效，修改过程中如果爬到树根结点路径上的 某一场比赛结果没有改变，则不需要再进行其祖先结点的比赛。这样的修改次数在 $0\sim\log_2 n$ 之间。

要实现这些操作，必须能够确定每一个外部结点 $L[i]$ 所对应的内部结点 $B[p]$。当外部结点的数目为 n 时，最底层最左端的内部结点编号为 2^s，其中 $s=\lfloor\log_2(n-1)\rfloor$；因此最底层的内部 结点数为 $n-2^s$，最底层的外部结点数为内部结点数的 2 倍，记为 $LowExt=2(n-2^s)$。令 $offset=2^{s+1}-1$ 代表最底层外部结点之上的所有结点数目，则对于任何一个外部结点 $L[i]$，其内部结点 $B[p]$ 满足

$$p=\left\lfloor\frac{i+offset}{2}\right\rfloor\ (i\le LowExt),\qquad p=\left\lfloor\frac{i-LowExt+n-1}{2}\right\rfloor\ (i>LowEx$$

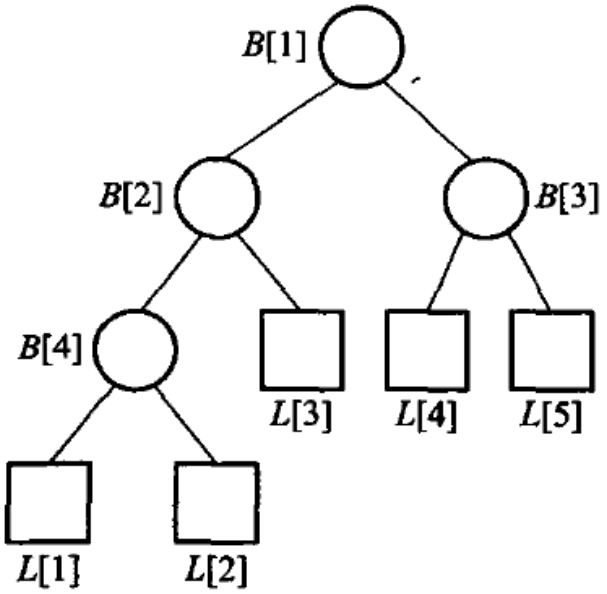
这里的除法是整数除法（C++ 的 /）。以图 9.4 的 $n=5$ 为例：$s=2$，最底层最左端的内部结点为 $B[4]$，该层只有 $n-2^s=1$ 个内部结点，$LowExt=2$，$offset=7$；于是 $L[1]$、$L[2]$ 的父结点是 $B[4]$，倒数第 2 层最左端的外部结点 $L[3]$（即 $L[LowExt+1]$）的父结点是 $B[2]$，$L[4]$、$L[5]$ 的父结点是 $B[3]$。

败者树。内部结点记录的是**败者**的下标，另外加入结点 $B[0]$ 来记录整个比赛的**冠军**。当某一个 选手 $L[i]$ 的分数改变时，同样只需沿着从 $L[i]$ 到根结点的路径修改：将新进入选择树的结点与其父 结点进行比赛，把败者的下标存放在父结点中，而赢者再与上一级的父结点进行比较，比赛沿着到根 结点的路径不断进行——**这正是败者树比赢者树少一次" 取兄弟" 访问的原因，也是外排序实现里更常见 的选择**。

本书的实现：赢者树用完全二叉树数组，叶存放选手下标，内部结点写 `better(左, 右)`；败者树在内部 结点写败者，另用 `champion_` 记全局胜者。替换后只沿叶到根重赛。

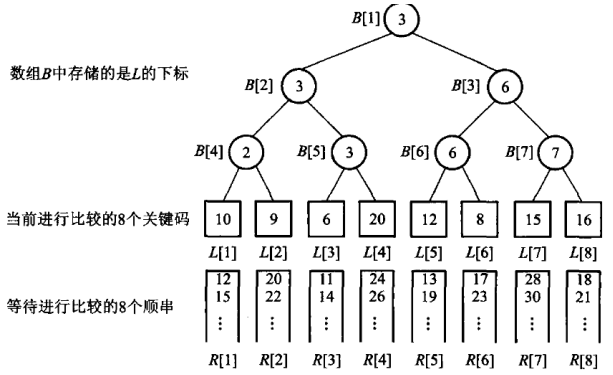
与原书的 L/B 布局不同，实现先把叶子数补齐到 2 的幂 `leaf_base_`（5 名选手补成 8 个叶位，空位记 `no_player`，永远输），于是第 i 名选手（0 起）的叶位

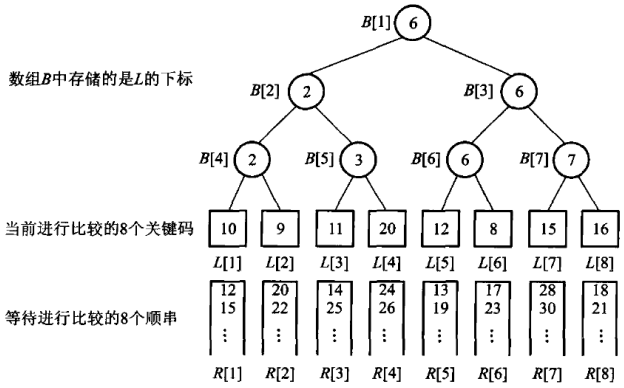
是 `leaf_base_ + i`，父结点就是 `(leaf_base_ + i) / 2`，不再需要 $LowExt$ 和 $offset$ 的分段公式，代价是至多多出约一倍的空叶位。另外，`LoserTree` 除了败者数组 `loser_`，还保存每个结点子树的胜者 `subtree_winner_`：重赛时读的是兄弟子树已记下的胜者，所以替换**任意**一名选手都正确；原书那种“只与路径上的败者比较”的做法只在替换当前冠军时成立——而 k 路归并恰好只替换冠军。



每个“选手”代表一条顺串的当前记录。直接扫描 k 个队首，每输出一条记录要比较 $k-1$ 次；选择树第一次建树同样要 $k-1$ 次比较，但之后只替换获胜顺串的队首，沿高度 $\lceil \log_2 k \rceil$ 的路径重赛。输出 N 条记录时，选择代价由 $O(Nk)$ 降为 $O(N \log k)$。

以当前值 `[20, 6, 8, 9, 11]` 为例，第一轮比赛的冠军是下标 1、值 6。输出 6 后，该路补入 15，只需重算“选手 1 到根”的比赛；8 成为新冠军。不在这条路径上的比赛（例如选手 2 与选手 3 那一场、选手 4 那一侧）结果没有变化，不应整棵树重建。





赢者树的父结点记胜者，所以更新时要找到沿途的对手；败者树把沿途败者直接留在父结点，替换冠军后，新选手一路与这些败者比较即可。两者渐近复杂度相同，区别在重赛时保存了什么信息。相同关键码还要规定稳定的决胜规则；本书实现让下标较小的路获胜，测试用两个值为 1 的选手固定这条规则。

四路归并手工演算

设四条输入顺串为：

```
R0:  2 12 20
R1:  4  9 25
R2:  1 11 18
R3:  6  8 30
```

初始选手是各路队首 `[2,4,1,6]`。冠军来自 R2；输出 1 后，只有 R2 前进到 11，选手变成 `[2,4,11,6]`。之后每一步都只替换刚获胜的那一路：

输出步	比赛中的 4 个当前值	冠军路	输出	该路补入
1	2,4,1,6	R2	1	11
2	2,4,11,6	R0	2	12
3	12,4,11,6	R1	4	9
4	12,9,11,6	R3	6	8
5	12,9,11,8	R3	8	30
6	12,9,11,30	R1	9	25
7	12,25,11,30	R2	11	18
8	12,25,18,30	R0	12	20
9	20,25,18,30	R2	18	耗尽
10	20,25,+ ∞,30	R0	20	耗尽
11	+ ∞,25,+ ∞,30	R1	25	耗尽
12	+ ∞,+ ∞,+ ∞,30	R3	30	耗尽

最终得到 `1 2 4 6 8 9 11 12 18 20 25 30`。某一路耗尽后，用 “正无穷” 参加余下比赛，它就不会再次获胜。工程实现不一定真的存 `INT_MAX`：若关键码本身允许取最大整数，哨兵会与合法数据冲突；更稳妥的做法是额外保存 “该路是否耗尽” 的状态。

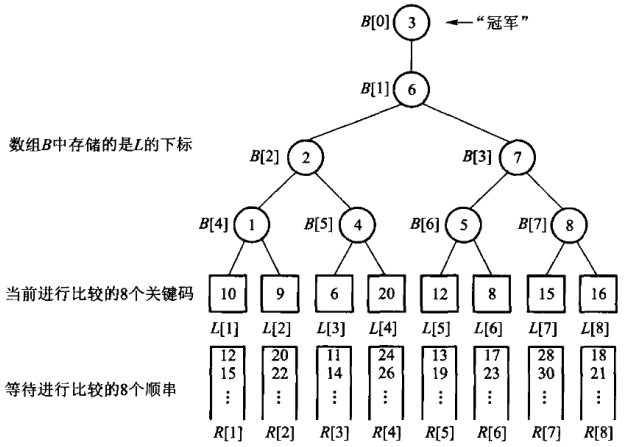
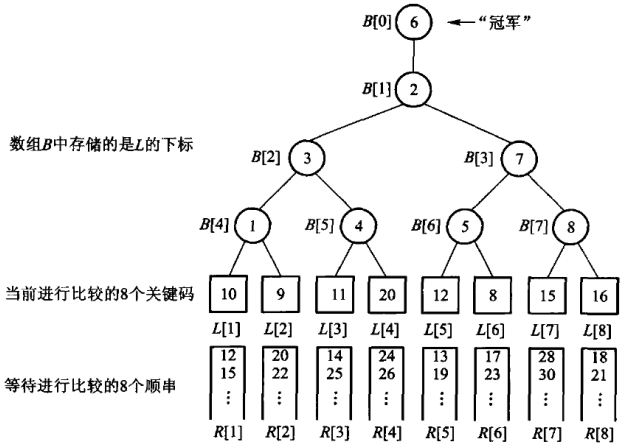


图 9.7 中内部结点保存败者，冠军另放在根外的槽位 $B[0]$。方形外部结点 $L[1..8]$ 是各路当前参与比较的关键码 `10 9 6 20 12 8 15 16`，$B[0]=3$ 指示最小关键码 6 在第三个顺串。替换冠军之后，新值只与路径上留下的败者依次比较。

具体地，顺串 3 的记录 6 输出后，下一个记录 11 进入败者树，引起调整：$L[3]$（11）先与 $B[5]$ 所指的 $L[4]$（20）比赛，败者的下标 4 留在 $B[5]$；赢者 11 继续与 $B[2]$ 所指的 $L[2]$（9）比赛，败者的下标 3 放进 $B[2]$；赢者 9 再与 $B[1]$ 所指的 $L[6]$（8）比赛，败者的下标 2 放进 $B[1]$，赢者的下标 6 放进 $B[0]$。整条路径只比了 3 次，从未访问兄弟子树。重构结果如图 9.8 所示。



稳定性和重复关键码

外排序处理的通常是完整记录，关键码相等不代表记录相同。若要求稳定排序，比较器必须把“原始先后次序”作为第二关键码：同一顺串内先出现的先输出；不同顺串之间，可用初始顺串号和顺串内位置决胜。只比较整数值虽然仍能得到非递减关键码，却可能打乱同关键码记录的先后。

本章 `WinnerTree` 和 `LoserTree` 在值相等时让下标较小的选手获胜，这保证比赛结果可重复，但它只等价于“路号优先”，并不自动等价于全文件稳定。若初始顺串本身由稳定排序生成，还要证明路号顺序与原始文件顺序一致；否则应把原始序号随记录一起参与比较。

三个常见错误

1. **把所有输入一次放进堆**。这要求内存容纳整个文件，已经退回内部堆排序。
2. **冻结后仍与当前堆比赛**。小于刚输出值的记录会让当前顺串倒序，破坏归并前提。
3. **一路耗尽就停止**。正确做法是让该路退出比赛，继续归并其余顺串，直到所有路都耗尽。

检查外排序程序不能只看“最后输出是否有序”。还要分别检查记录守恒、每条初始顺串有序、内存工作区不超过 M、耗尽一路后仍能继续，以及相等关键码的决胜规则。否则一个偷偷做整表排序的实现也会交出正确终值，却完全没有实现外排序。

```
// 代码 9.2：赢者树。内部结点保存两名选手比较后的胜者下标，根是全局最小。
class WinnerTree {
public:
    explicit WinnerTree(std::vector<int> players) : players_(std::move(players)) {
        if (players_.empty()) {
            return;
        }
        leaf_base_ = detail::TournamentOps::next_power_of_two(players_.size());
        tree_.assign(leaf_base_ * 2, detail::TournamentOps::no_player);
        for (std::size_t index = 0; index < players_.size(); ++index) {
            tree_[leaf_base_ + index] = index;
        }
        for (std::size_t node = leaf_base_ - 1; node > 0; --node) {
            tree_[node] = detail::TournamentOps::better(players_, tree_[node * 2],
                                                        tree_[node * 2 + 1]);
        }
    }

    [[nodiscard]] std::optional<std::size_t> winner_index() const {
        if (players_.empty() || tree_[1] == detail::TournamentOps::no_player) {
            return std::nullopt;
```

```
        }
        return tree_[1];
    }

    [[nodiscard]] std::optional<int> winner() const {
        const auto index = winner_index();
        return index ? std::optional<int>(players_[*index]) : std::nullopt;
    }

    void replace(std::size_t player, int value) {
        if (player >= players_.size()) {
            throw std::out_of_range("tournament player");
        }
        players_[player] = value;
        std::size_t node = leaf_base_ + player;
        tree_[node] = player;
        while (node > 1) {
            node /= 2;
            tree_[node] = detail::TournamentOps::better(players_, tree_[node * 2],
                                                        tree_[node * 2 + 1]);
        }
    }

private:
    std::vector<int> players_;
    std::vector<std::size_t> tree_;
    std::size_t leaf_base_{0};
};
```

```
class WinnerTree(_Tournament):
    """ 代码 9.2：内部结点记 ** 赢家 **，根就是全局最小的那一路。

    重建一个结点要看它两个孩子的赢家，所以替换选手后沿路每层各比一次。
    """

    def _winner_at(self, node: int) -> int:
        # 叶子层不占内部结点的位置：第 j 个选手就在 _size + j 上，它自己是自己的赢家。
        return node - self._size if node >= self._size else self._tree[node]

    def _build(self) -> None:
        for node in range(self._size - 1, 0, -1):
            self._tree[node] = self._better(self._winner_at(node * 2),
                                            self._winner_at(node * 2 + 1))

    def winner_index(self) -> int | None:
        if not self.players:
```

```
            return None
        return self._winner_at(1)

    def replace(self, player: int, value: int) -> None:
        self._check_player(player)
        self.players[player] = value
        node = (self._size + player) // 2
        while node >= 1:
            self._tree[node] = self._better(self._winner_at(node * 2),
                                            self._winner_at(node * 2 + 1))
            node //= 2
```

```
// 代码 9.3：败者树。内部结点保存败者下标，另用 champion_ 记录全局胜者。
// 替换一名选手时只需沿叶到根重赛，不必访问兄弟子树的内部结构。
class LoserTree {
public:
    explicit LoserTree(std::vector<int> players) : players_(std::move(players)) {
        if (players_.empty()) {
            return;
        }
        leaf_base_ = detail::TournamentOps::next_power_of_two(players_.size());
        loser_.assign(leaf_base_, detail::TournamentOps::no_player);
        subtree_winner_.assign(leaf_base_ * 2, detail::TournamentOps::no_player);
        for (std::size_t index = 0; index < players_.size(); ++index) {
            subtree_winner_[leaf_base_ + index] = index;
        }
        for (std::size_t node = leaf_base_ - 1; node > 0; --node) {
            replay_node(node);
        }
        champion_ = subtree_winner_[1];
    }

    [[nodiscard]] std::optional<std::size_t> winner_index() const {
        if (players_.empty() || champion_ == detail::TournamentOps::no_player) {
            return std::nullopt;
        }
        return champion_;
    }

    [[nodiscard]] std::optional<int> winner() const {
        const auto index = winner_index();
        return index ? std::optional<int>(players_[*index]) : std::nullopt;
    }

    [[nodiscard]] std::optional<std::size_t> loser_at(std::size_t node) const {
        if (node == 0 || node >= loser_.size() ||
```

```
            loser_[node] == detail::TournamentOps::no_player) {
            return std::nullopt;
        }
        return loser_[node];
    }

    void replace(std::size_t player, int value) {
        if (player >= players_.size()) {
            throw std::out_of_range("tournament player");
        }
        players_[player] = value;
        subtree_winner_[leaf_base_ + player] = player;
        for (std::size_t node = (leaf_base_ + player) / 2; node > 0; node /= 2) {
            replay_node(node);
        }
        champion_ = subtree_winner_[1];
    }

private:
    void replay_node(std::size_t node) {
        const std::size_t left = subtree_winner_[node * 2];
        const std::size_t right = subtree_winner_[node * 2 + 1];
        loser_[node] = detail::TournamentOps::worse(players_, left, right);
       subtree_winner_[node] = detail::TournamentOps::better(players_, left, right);
    }

    std::vector<int> players_;
    std::vector<std::size_t> loser_;
    std::vector<std::size_t> subtree_winner_;
    std::size_t leaf_base_{0};
    std::size_t champion_{detail::TournamentOps::no_player};
};
```

```
class LoserTree(_Tournament):
    """ 代码 9.3：内部结点记 ** 输家 **，另用 `_champion` 记全局胜者。

    与胜者树同一套重赛路径，差别只在留下什么痕迹：胜者树只留赢家，
    败者树把每一场的输家也记在结点上。外排序要这份痕迹——
    `loser_at(node)` 就是「这一路是在哪一层、被谁淘汰的」，
    而胜者树把这件事丢掉了。

    两个数组一起维护（`_subtree_winner` 与 `_loser`），所以替换 ** 任意 ** 一片叶子
    都成立，不只是替换当前冠军。只留输家数组的写法看着更省，
    但那样只有「替换冠军」这一种用法是对的——k 路归并恰好只用那一种，
    于是错误可以长期不被发现。这里不取那条捷径。
    """
```

```
    def __init__(self, players: list[int]) -> None:
        self._loser: list[int | None] = []
        self._subtree_winner: list[int] = []
        self._champion: int | None = None
        super().__init__(players)

    def _match(self, left: int, right: int) -> tuple[int, int]:
        """ 一场比赛，返回 (赢家, 输家)。一次比较，两个结果。"""
        winner = self._better(left, right)
        return winner, (right if winner == left else left)

    def _build(self) -> None:
        self._loser = [None] * self._size
        self._subtree_winner = [0] * (self._size * 2)
        for index in range(self._size):
            self._subtree_winner[self._size + index] = index
        for node in range(self._size - 1, 0, -1):
            self._replay_node(node)
        self._champion = self._subtree_winner[1]

    def _replay_node(self, node: int) -> None:
        winner, loser = self._match(self._subtree_winner[node * 2],
                                    self._subtree_winner[node * 2 + 1])
        self._loser[node] = loser
        self._subtree_winner[node] = winner

    def winner_index(self) -> int | None:
        if not self.players:
            return None
        return self._champion

    def replace(self, player: int, value: int) -> None:
        self._check_player(player)
        self.players[player] = value
        node = (self._size + player) // 2
        while node >= 1:
            self._replay_node(node)
            node //= 2
        self._champion = self._subtree_winner[1]

    def loser_at(self, node: int) -> int | None:
        """ 第 node 个内部结点上记着的输家。越界返回 None。"""
        if node <= 0 or node >= self._size:
            return None
        return self._loser[node]
```

本章小结

本章讨论了文件管理和外排序的相关内容。

首先介绍了内存和外存的特点：**内存具有存取速度快、造价高、存储容量小和断电后数据丢失等特点；而外存的特点有读/写速度慢、造价低廉、存储容量大，但信息不易失等**。外存按页存取，一次定位比一次 比较贵几个数量级，所以外排序首先要减少读写次数。

文件在外存上的组织方式称为文件的物理结构，几种常见的物理组织结构有顺序文件、散列文件、索引文件、倒排文件（原书此处把 “逻辑文件” 也列为物理结构，与 9.2 节的区分矛盾，已更正）。此外，还介绍了 C++ 语言中流的概念和有关的操作。

处理数据量大的文件时，待排记录存储在外存中、不能一次读入内存，需要采用外排序技术。外排序由 两个相对独立的阶段组成：根据内存的大小将外存中的数据文件排序成顺串，然后对这些顺串进行逐趟 归并，直到最后整个数据文件排列有序为止。

外排序算法是本章的重点，9.3 节中介绍了置换选择算法、二路归并算法和选择树排序算法。**置换选择排序的目标是生成最长的顺串，其核心思想是利用最小堆（或最大堆）对数据进行处理——** 它把不属于当前顺串的记录冻结，第一趟平均长度约 $2M$ 而不是 M。二路外排序的归并是逐步把顺串 两两合并，直至变为一个顺串为止，从而形成了已排序的文件。

在 K 路归并中，最直接的方法就是进行 $K-1$ 次关键码两两比较来找出最小的记录。**为了提高效率，采用选择树的方法来实现 K 路归并**。选择树是完全二叉树，有两种类型：赢者树和败者树。利用赢者树 进行归并时，将两个子结点中的赢者（关键码值较小者）上升到父结点，赢者树的根结点是树中的最终赢者的索引；利用败者树进行归并时，将两个子结点中的败者（关键码值较大者）记录在父结点中，赢者进入高一层 的比赛，同时加进一个结点以代表比赛的全局赢者的索引值。

习题

补充外排序题（参考课程第 9 章）

1. 给定内存容量 `M` 的最小堆，模拟置换选择生成全部顺串，并标出每次冻结的记录。
2. 给定页大小、记录大小和内存缓冲页数，计算一次归并的最大路数、顺串长度和访外次数。
3. 比较 winner tree、loser tree 与最小堆在多路归并中的更新代价。

正文习题

1. 说明为什么外存要按页读写，以及缓冲如何减少定位次数。
2. 对输入 `50 49 35 45 30 25 15 60 16 27 1`、$M=7$，写出置换选择的两个顺串，并指出哪些键被冻结。

3. 若 M 大到能装下全部输入，置换选择退化成什么。
4. 赢者树和败者树的内部结点各记什么？替换一名选手后为什么只需沿叶到根重赛。
5. m 个顺串做二路归并要多少趟？置换选择怎样减少 m。

原书习题

以下是原书第 9 章的习题，本轮按扫描件补回题面；**参考答案见** `book/习题与参考答案.md` **的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。

1. 假设一个记录长 32 B、一个块长 1024 B（因此每个块有 32 个记录），工作内存是 1 MB（还有用于 I/O 缓冲区、程序变量等的其他存储空间）。使用置换选择和多路归并，其中归并算法只允许扫描两遍，**预计能得到的文件最长为多少？**试解释是怎样得到这个结果的。
2. 假设原有一个计算机系统，能够处理一个大公司所有雇员记录的排序操作。由于公司合并，其雇员数目 扩大为原来的 100 倍，计算机在可接受的时间内不能对原来 100 倍的记录进行外排序。下面考虑用这 几种独立的方案对计算机系统进行升级，它们是否能够满足新的外排序要求？(1) CPU 的速度增长两倍；(2) 磁盘 I/O 时间缩短至原来的一半；(3) 内存存取时间缩短至原来的一半；(4) 内存大小增长两倍。
3. 如果某个文件经内排序得到 80 个初始归并段，试问：(1) 若使用多路归并并执行 3 趟完成排序，那么 应取的归并路数至少应为多少？(2) 如果操作系统要求一个程序同时可用的输入/输出文件的总数不超过 15 个，则按多路归并至少需要几趟可以完成排序？如果限定这个趟数，可取的最低路数是多少？
4. 有 8 个顺串，每个顺串的第一个记录的关键码分别为 14、22、24、15、16、11、100、18，而第二个记录的 关键码分别为 26、38、30、26、50、28、110、40。试画出对顺串开始 8 路归并时的败者树。从败者树输出 一个全局赢者（并有相应的一个记录进入败者树）后需对败者树进行重构，试画出输出第一个全局赢者并 进行重构后的败者树。
5. 代码 9.3 是一种败者树的实现方法（**原书这道题把这份清单叫成「算法」，而它在书里印的编号是「代码」，同一处东西两个名字**）。此外，还可以用另一种方法来初始化败者树：**先构造一棵赢者树，然后把这棵赢者树映射成它所对应的败者树**。这一种方法只需要修改败者树的两个成员函数 `Initialize()` 和 `Play()`——其中 `Play()` 用于在 p 处的比较，赢者索引放在 $B[p]$ 中，若 $B[p]$ 是右孩子则持续向上比赛；`Initialize` 先按选手索引顺序调用 `Play()`，生成赢者树之后再映射成 败者树。请读者实现这种方法。
6. 设输入文件包含下列记录（仅列出其关键码）：14、22、7、24、15、16、11、100、10、9、20、12、90、17、13、18、26、38、30、25、50、28、110、21、40、…，采用外部结点数为 8 的败者树方法对该文件生成初始 顺串。试画出最初产生的败者树和最终输出的所有初始顺串。

上机题

1. 实现置换选择，用原书图 9.2 的输入做守门测试。
2. 实现赢者树，随机替换选手并与每次扫描 k 路的朴素选最小对拍。
3. 模拟 k 路归并：输入是若干已排序向量，用败者树输出完整有序序列。
4. 比较 $M = 4, 8, 16$ 时置换选择产生的顺串个数。

原书上机题

> 原书上机题的参考答案见 book/习题与参考答案.md 的「原书上机题参考」，由 R14 逐题守着。

1. 在磁盘中，图片一般作为数据一行一行地存储。16 色图片可以用 4 位表示一个像素；如果允许每个 像素 8 位，则不需要解压缩像素的处理（由于 1 个像素对应 1 个字节，在大多数机器中字节是最低级的 寻址单位）；**如果把两个像素压缩到一个字节中，就能够节省空间，但是必须对像素进行解压缩**。对比每个像素 8 位和压缩为每个像素 4 位两种情况，哪种情况从磁盘中读取并访问图像中每个像素所花 的时间更多？对这两种情况进行编程，并比较需要的时间。
2. 根据本章描述的置换选择方法和多路归并算法实现一个外部排序。**针对大记录文件和小记录文件两种 情况测试你的程序**，考虑记录为多大时，应该首先采用关键码排序、然后再根据排序结果重排原数据文件。

第 10 章 检索

检索的目标是在一组记录中找到目标键。顺序检索不要求有序；二分检索要求有序，靠每次排除一半区间加速；散列表用散列函数直接定位槽位，冲突时继续探测。

源码：检索、集合和散列表·教学版、检索、集合和散列表·工程版、可运行示例、墓碑探测测试。

检索是在一组记录里定位关键码等于给定值的那一条，或属性满足条件的那些条。成功就是至少找到一条，失败就是没有。精确匹配查单个值，范围查询查一个区间。

为了衡量一个检索算法的效率，需要在时间和空间两方面进行权衡。检索运算的主要操作是关键码值的 比较，通常把检索过程中对关键码需要执行的**平均比较次数**称为**平均检索长度**（average search length，ASL），它是衡量检索算法优劣的时间标准，也是存储结构中元素总数 n 的函数：

$$\mathrm{ASL} = \sum_{i=1}^{n} P_i C_i$$

其中 P_i 为检索第 i 个元素（即给定值 K 与存储结构中第 i 个元素的关键码值相等）的概率，C_i 为找到第 i 个元素所需的关键码值与给定值的比较次数。举原书的例子：线性表为 (a, b, c)，检索 a、b、c 的概率分别为 0.4、0.1、0.5，则顺序检索的平均检索长度为 $0.4 \times 1 + 0.1 \times 2 + 0.5 \times 3 = 2.1$，即平均需要比较 2.1 次才能找到待查元素。「检索第 i 个元素的概率 P_i」可理解为：在很多次的检索中，检索第 i 个元素的次数占总次数的比例。**若已知各个元素检索概率的分布情况，则 ASL 可准确地反映检索算法的平均时间性能**。另外，衡量一个检索算法还要考虑算法所需的存储量、算法的复杂性等因素。

可以把检索分成四类：基于线性表、按关键码直接访问（含散列）、树形索引、基于属性（倒排）。本章做前两类；树形索引见第 5、11、12 章，倒排见第 11 章。

10.1 基于线性表的检索

数据放在数组或链表里，按给定值 K 与表中元素比较，直到命中或能确定不在表中。这一节不要求散列函数，也不建树，只靠线性结构本身。

10.1.1 顺序检索

顺序检索（sequential search）是一种最基本的检索方法：针对线性表里的所有记录逐个进行关键码和 给定值的比较，若某个记录的关键码和给定值相等，则检索成功；反之检索失败。**表中各数据元素可以 任意排列**——这是它唯一的、也是最宝贵的优点：不要求有序，实现最简单，链表和数组都能用。

最好情况目标在第一个位置，比较 1 次；最坏情况目标不在表里，比较 n 次。在顺序检索中通常假设 对于所有 i 有 $P_i = 1/n$（称为「**等概率假设**」），此时平均检索长度 ASL $= (n+1)/2$。

「**监视哨**」**是一个值得单独说的技巧**。原书的做法是把位置 0 空出来做监视哨，检索前先 `dataList[0]->setKey(K)`，然后从后往前逐个比较，循环条件只写「当前元素是否等于目标」。监视哨的作用是**保证 while 循环一定终止**：因为检索不成功时 i 会走到 0，而此时 `dataList[0]` 的关键码恰好等于 K，使得终止条件「被迫」成立。因此不必将 `i > 0` 写入循环检查 条件，从而使其得到简化。**监视哨方法利用了一个重要的「加速」原理：当程序的一个循环需要测试 两个或多个条件时，应该尽量减少测试条件**——比较次数的常数会小一点，阶不变。

对 `{8,3,8,1}` 查 8，第一次比较就命中下标 0；查 7 则依次比较 8、3、8、1，走完整张表才知道失败。重复关键码存在时，本书接口返回**第一个**匹配位置，这不是顺序检索自动保证的抽象性质，而是 “从左向右、命中即停” 这份实现的明确约定。

监视哨适合能临时修改的数组：先把目标值写到末尾的额外槽，再只问 “当前元素是否等于目标”。循环一定会停，停在原表范围内就是成功，停在哨兵槽就是失败。但只读输入、并发共享数组或没有额外槽时，不应为省一个边界判断去改数据。

10.1.2 二分检索

先说这个「有序」是什么意思。对于任何一个线性表，若其中的所有数据元素按关键码值的某种次序 进行不增或不降的排列，则称为**有序表**——有序表是一种特殊的线性表。查英语字典的过程就是有序表 检索的日常版：字典按照从 a 到 z 有序排列，查字典时大体上翻一下，b 打头就靠前翻、z 打头就翻到 后面；如果当前页的单词排在待查单词的前面，则往后翻，反之往前翻，直到待查单词位于当前页的首末 单词范围内。**这种查字典的方法体现了「先确定待查目标所在的范围，然后逐步缩小范围直到找到或找 不到该目标为止」的思想**。

由于有序表中所有元素按不增或不降的次序排列，把表中任一元素的关键码值与给定值 K 比较，可根据 3 种比较结果区分出 3 种情况（以递增为例）：相等则检索成功；表中元素大于 K 说明待查元素若在表中 一定排在它之前；小于 K 则一定排在它之后。**因此在一次比较之后，若没有找到待查元素，就可根据比较结果缩小进一步检索的区间**。

二分检索法（binary search）因此要求：表必须按关键码有序，并且能按下标随机访问，所以适合数组，不适合链表。每次取当前区间的中点：相等则停；目标更大则丢掉左半，否则丢掉右半。区间长度每次 至少减半，比较次数是 $O(\log n)$。**二分检索是比较典型的分治算法**，也可以改用递归来实现。

实现时用半开区间 `[first, last)`：中点是 `first + (last - first) / 2`，走左半时 `last = middle`，走右半时 `first = middle + 1`。这样不必写 `middle - 1`，无符号下标不会在 `middle == 0` 时下溢。循环条件是 `first < last`；区间空了还

没找到，就返回空。

以有序表 `{1,3,7,8,12,15,18,21}` 查 18 为例，按本书的半开区间写法：

轮次	候选区间 `[first,last)`	`middle`	中值	判断
1	`[0,8)`	4	12	$12 < 18$，令 `first=5`
2	`[5,8)`	6	18	命中下标 6

若改查 14，过程会是 `[0,8)` → `[5,8)` → `[5,6)` → `[5,5)`，区间为空后失败。每轮都维持同一个不变量：若目标存在，它一定还在 `[first,last)` 里；更新边界时必须把已经比较过且不可能命中的中点排除。

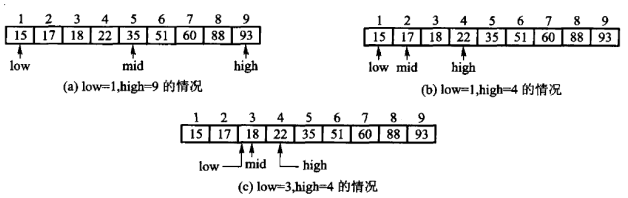


图 10.1 是原书的例子，数据不同：有序表 `(15,17,18,22,35,51,60,88,93)` 查 18，下标从 1 起、用闭区间 `[low,high]`，中点 `(low+high)/2`，走左半时 `high = mid - 1`。三轮依次是 `[1,9]` 中点 5（35，太大）→ `[1,4]` 中点 2（17，太小）→ `[3,4]` 中点 3（18，命中）。同一张表若用本书的半开区间、从 0 起，则是 `[0,9)` 中点 4（35）→ `[0,4)` 中点 2（18，命中），两轮就停——两种写法都对，但中点落点不同，手算时别混用。

图 10.1 把一次查找画成区间逐步缩小；图 10.2 则把所有可能的比较路径同时展开。**这棵决策树其实就是一棵 BST（二叉搜索树）**：检索某个值时正好走了一条从根结点到与该给定值对应结点的路径，所以 BST 中关键码与给定值的比较次数正好等于该值对应结点所在的层数加 1，因此二分检索到某个 元素所需的比较次数最多不超过这棵树的层数加 1（即 BST 的高度）。**注意：对同一长度的有序数组，采用相同的二分检索算法，所对应的决策树的形状是固定的**——不管数组中元素的数值是什么，数组 下标位置的 BST 都保持不变。

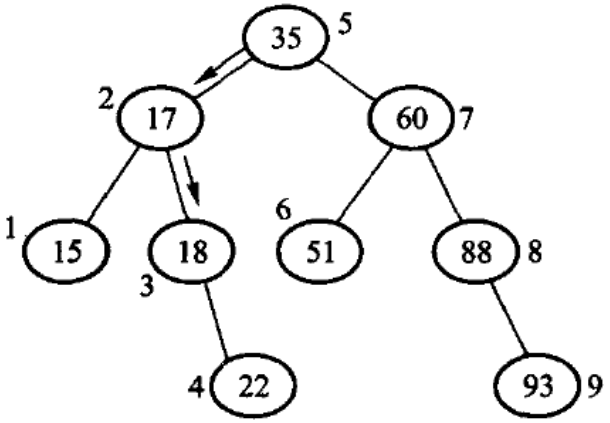
代价可以从代数角度精确算出来。只比较 1 次就成功检索的结点数为 $1 = 2^0$；只比较 2 次的为 $2 = 2^1$；比较 j 次才成功的结点个数为 2^{j-1}。若表中结点总数恰好为 $n = 2^0 + 2^1 + \cdots + 2^{j-1} = 2^j - 1$，则最大检索长度为 j；若 $2^j - 1 < n \le 2^{j+1} - 1$，则最大检索长度为 $j+1$。**总的来说，最大检索长度为** $\lceil \log_2(n+1) \rceil$。设检索每个结点的 概率相同，则在 $n = 2^j - 1$ 的情况下

$$\mathrm{ASL} = \frac{1}{n}\sum_{i=1}^{j} i \cdot 2^{i-1} = \frac{1}{n}(j \cdot 2^j - 2^j + 1) \approx \frac{n+1}{n}\log_2(n+1) - 1$$

当 n 较大时 $\mathrm{ASL} \approx \log_2(n+1) - 1$：**二分检索的平均检索长度与最大检**

索长度相近，因而效率较高。

但它有两条硬约束。一是要求被检索序列事先按关键码次序排列，而**排序本身是一种很费时的操作**；二是只适用于顺序存储结构，而在顺序结构中插入和删除都比较困难。**因此，二分检索特别适用于那种 一经建立就很少改动、而又需要经常检索的线性表。**



10.1.3 分块检索

把 n 个元素分成 b 块。块与块之间有序（后一块的最小关键码不小于前一块的最大关键码），块内部可以无序。另造一张块索引，每项记下该块的最大（或最小）关键码和块的起始位置。检索时先在索引上顺序或二分，确定目标可能在哪一块，再在那一块里顺序查。

分块检索又称索引检索，如果线性表既希望有较快的检索速度又需要动态变化，就可以采用它。索引表上的检索分两个阶段：确定待查元素所在的块；在块内检索待查的元素。**第二阶段只能采用顺序 检索法，而第一阶段既可以采用顺序检索法，也可以采取二分检索法。**多数情况下数据不需要完全填满 每个块，因此索引表还需要记录每块当前记录的数目。

分块检索的平均检索长度等于两个阶段各自的检索长度之和。设线性表中共有 n 个数据元素，将表均分 成 b 块、每块含 s 个元素（$n = s \times b$）。如果第一阶段以二分检索来确定块，则在等概率假定下

$$\mathrm{ASL}_{bs} \approx \log_2\left(\frac{n}{s}+1\right)+\frac{s-1}{2}$$

若第一阶段采用顺序检索，则

$$\mathrm{ASL}=\frac{b+1}{2}+\frac{s+1}{2}=\frac{n+s^2}{2s}+1$$

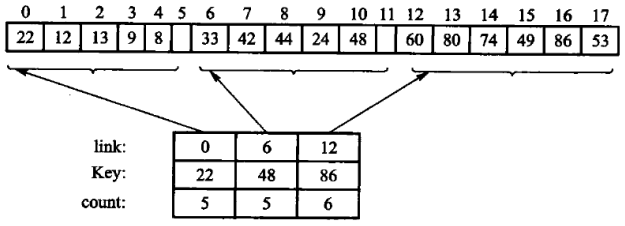
显然，当 s 取 $\sqrt{n}$ 时，顺序分块检索的 ASL 达到最小值 $\sqrt{n}+1$。原书给了

一个直观的 对照：如果要检索的线性表中有 10000 个结点，把它分成 100 个块、每块 100 个结点，分块检索平均 需要做 101 次比较，而顺序检索平均约做 5000 次比较，二分检索则最多需要 14 次比较——**分块检索 的时间性能高于顺序检索而低于二分检索**。

如果数据块（子表）存放在外存时，还会受到页块大小的制约，此时往往以外存一次 I/O 读取的数据（一页）作为一块。分块检索的主要代价是增加一个辅助索引数组的存储空间和将初始线性表分块排序 的运算；另外，**当大量地插入删除运算使块中结点数分布很不均匀时，检索速度将会下降**。

它的优点在插入删除上：在线性表中插入或删除一个结点时，只要找到该结点应属于的块，然后在块内 进行插入和删除运算即可；由于块内结点的存放是任意的，因此插入或删除比较容易，不需要移动大量的 结点——插入可以在块尾进行；如果待删除的记录不是块中最后一个记录时，可以将本块内最后一个记录移入被删除记录的位置。

总之，线性表检索的三种实现各有优缺点：顺序检索效率最低但限制最少；二分检索效率最高但限制较多；分块检索则介于二者之间。实际应用中可根据线性表的具体情况进行选择，需要综合考虑检索效率、插入删除频率等。本章不另写未验证的分块实现。



原书图 10.3 把主表分成三块。注意块与块之间留了空位（下标 5、11），供块内插入用，所以块的起始位置是 0、6、12，索引表另用 count 记每块当前有几个元素：

```
下标 0–4 : 22 12 13  9  8      (下标 5 空)    索引项 (link=0,  Key=22, count=5)
下标 6–10: 33 42 44 24 48      (下标 11 空)   索引项 (link=6,  Key=48, count=5)
下标 12–17: 60 80 74 49 86 53                   索引项 (link=12, Key=86, count=6)
```

查 44 时，先在索引中比较 22、48，确定只能在块 1；再顺序比较 33、42、44。查 23 时同样落到块 1，但扫完整块失败；不能因为 23 小于块最大值 48 就断言存在。块间有序只负责排除别的块，块内仍然无序。

若索引顺序查且每块等长，粗略代价是“查索引约 $b/2$ 次 + 查块内约 $n/(2b)$ 次”。令两项平衡得到 $b \approx \sqrt{n}$。索引改用二分后最优块数会变化，因此“固定分成根号 n 块”不是定义，只是特定成本模型下的选择。

教学版：完整实现

本章的四件东西——两种检索、集合、散列函数、闭散列表——放在同一个文件里。后面 10.2、10.3 各节就是把它拆开逐段讲。

```
// 检索、集合与散列 ——教学版。
// 原书【代码 10.1】【算法 10.2】–【算法 10.13】。
//
// 一个文件、能直接编译运行，给「第一次读这一章」的人看。
//
//   sequential_search / binary_search   线性表上的两种检索
//   IntSet                              不重复整数集合与集合运算
//   elf_hash                            ELF 散列函数
//   HashTable                           线性探测的闭散列表，含「墓碑」删除
//
// 与 modern.hpp（工程版）的分工：教学版把 `[[nodiscard]]`/`noexcept` 和压成一行的
// if 分支全部展开，其余逻辑一模一样。这一章没有手写存储管理，所以没有三法则/五法则
// 之分——`std::vector` 在这里只是「一块连续的槽位」，不是被它替换掉的教学内容
// （见 unit.json 的 d001_exceptions）。
#pragma once

#include <cstddef>
#include <cstdint>
#include <optional>
#include <stdexcept>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// 10.1 基于线性表的检索
// ---------------------------------------------------------------------------

// 【算法 10.2】顺序检索：从头比到尾。找到返回下标，没找到返回空 optional。
// 代价 O(n)。它对数据没有任何要求——这是它唯一的优点，也是全部优点。
inline std::optional<std::size_t> sequential_search(const std::vector<int>& values,
↪  int key) {
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (values[index] == key) {
            return index;
        }
    }
    return std::nullopt;
}

// 【算法 10.3】二分检索：要求 ** 已排好序 **，每比较一次砍掉一半，代价 O(log n)。
//
// 这里用的是 ** 半开区间 ** [first, last)：`last` 指向" 最后一个候选的下一个"。
// 为什么不用书上常见的闭区间 [low, high]？因为闭区间在没找到时要写 `high = mid - 1`，
```

```cpp
// 而下标是无符号的，`mid == 0` 时这一句会下溢成一个天文数字。
// 半开区间只写 `last = middle`，永远不减 1，这个坑就不存在。
inline std::optional<std::size_t> binary_search(const std::vector<int>&
↪  sorted_values, int key) {
    std::size_t first = 0;
    std::size_t last = sorted_values.size();
    while (first < last) {
        // 写成 first + (last - first) / 2 而不是 (first + last) / 2：
        // 后者在两个下标都很大时会溢出。
        std::size_t middle = first + (last - first) / 2;
        if (sorted_values[middle] == key) {
            return middle;
        }
        if (sorted_values[middle] < key) {
            first = middle + 1;    // 目标在右半边
        } else {
            last = middle;         // 目标在左半边
        }
    }
    return std::nullopt;
}

// ---------------------------------------------------------------------------
// 10.2 集合的检索
//
// 【代码 10.4】【算法 10.5】–【算法 10.7】：用一个不含重复元素的表表示集合。
// 所有运算都建立在「查一个元素在不在集合里」之上，而这里的查是顺序检索 O(n)——
// 所以交集是 O(n·m)。10.3 节的散列就是来把这个 O(n) 压成 O(1) 的。
// ---------------------------------------------------------------------------
class IntSet {
public:
    // 插入。已经有了就返回 false——集合不含重复元素，这是可预期状态，不是错误。
    bool insert(int value) {
        if (contains(value)) {
            return false;
        }
        values_.push_back(value);
        return true;
    }

    bool erase(int value) {
        std::optional<std::size_t> found = sequential_search(values_, value);
        if (!found) {
            return false;
        }
        values_.erase(values_.begin() + static_cast<std::ptrdiff_t>(*found));
        return true;
```

```
    }

    bool contains(int value) const {
        return sequential_search(values_, value).has_value();
    }

    // 交集：本集合里凡是对方也有的，都收进结果。
    IntSet intersection(const IntSet& other) const {
        IntSet result;
        for (int value : values_) {
            if (other.contains(value)) {
                (void)result.insert(value);
            }
        }
        return result;
    }

    // 包含：对方的每个元素本集合都得有。
    bool includes(const IntSet& other) const {
        for (int value : other.values_) {
            if (!contains(value)) {
                return false;
            }
        }
        return true;
    }

    std::size_t size() const { return values_.size(); }

private:
    std::vector<int> values_;
};

// ---------------------------------------------------------------------------
// 10.3 散列方法
// ---------------------------------------------------------------------------

// 【算法 10.8】ELF 散列：把字符串搅成一个整数。
//
// 逐字节读的是 `unsigned char` 而不是 `char`——`char` 在多数平台上是有符号的，
// 中文等非 ASCII 字节会变成负数，一进位运算就带出符号扩展，散列值随平台而变。
//
// 中间值用 32 位的 `std::uint32_t`：ELF 散列是按 32 位定义的。若用 64 位的
// `std::size_t`，`hash << 4` 再加一个字节可能进位到第 32 位，下面只清第 28–31 位，
// 那一位就永远留着，结果与经典 ELF 散列不同。
inline std::size_t elf_hash(const std::string& text) {
    std::uint32_t hash = 0;
```

```
    for (unsigned char character : text) {
        hash = (hash << 4U) + character;        // 左移 4 位，腾出位置放新字节
        std::uint32_t high_bits = hash & 0xF0000000U;   // 溢出到高 4 位的那部分
        if (high_bits != 0) {
            hash ^= high_bits >> 24U;           // 折回低位，别让它白白丢掉
        }
        hash &= ~high_bits;                     // 再把高 4 位清掉
    }
    return hash;
}

// 【算法 10.9】–【算法 10.13】线性探测的闭散列表。
//
// 「闭散列」是指所有元素都住在表里，不另开链表。key 先由散列函数算出一个
// ** 基地址 **(home)；那一格被占了就往后一格一格找，这就是 ** 线性探测 **。
//
// 删除是这里唯一的难点。直接把槽位标成「空」是 ** 错的 **：
//
//   假设 A 和 B 的基地址都是 3，A 占了 3，B 探测一格住进 4。
//   现在删掉 A，把 3 标成空。再查 B：从 3 开始，看到「空」就认定 B 不在表里——
//   可 B 明明就在 4 号槽。** 探测链被这个空格截断了。**
//
// 所以删除只能把槽位标成 ** 墓碑 **(tombstone)：查找路过它继续往下走，
// 插入则可以覆盖它。三种状态因此缺一不可。
class HashTable {
public:
    enum class SlotState { empty, used, tombstone };

    struct SlotView {
        int key;
        SlotState state;
    };

    explicit HashTable(std::size_t capacity) : slots_(capacity), size_(0) {
        if (capacity == 0) {
            throw std::invalid_argument("HashTable: 容量必须为正");
        }
    }

    // 插入。键已存在返回 false；表满且没有墓碑可用也返回 false。
    bool insert(int key) {
        std::optional<std::size_t> target = insertion_slot(key);
        if (!target) {
            return false;                       // 表满，插不进去
        }
        if (slots_[*target].state == SlotState::used) {
            return false;                       // 键已经在表里
```

```
        }
        slots_[*target].key = key;
        slots_[*target].state = SlotState::used;
        ++size_;
        return true;
    }

    bool contains(int key) const { return find_slot(key).has_value(); }

    // 删除：** 标墓碑，不标空 **。理由见上面类注释里那三行推演。
    bool erase(int key) {
        std::optional<std::size_t> found = find_slot(key);
        if (!found) {
            return false;
        }
        slots_[*found].state = SlotState::tombstone;
        --size_;
        return true;
    }

    std::size_t size() const { return size_; }
    std::size_t capacity() const { return slots_.size(); }

    // 让调用方（和测试）能看到每一格的状态，方便观察探测过程。
    SlotView slot_at(std::size_t index) const {
        if (index >= slots_.size()) {
            throw std::out_of_range("HashTable::slot_at: 下标越界");
        }
        return SlotView{slots_[index].key, slots_[index].state};
    }

private:
    struct Slot {
        int key = 0;
        SlotState state = SlotState::empty;
    };

    // 基地址：key 取绝对值再对表长取模。
    // 先转成 long long 再取绝对值，是因为 INT_MIN 的相反数在 int 里放不下。
    std::size_t home(int key) const {
        long long magnitude = (key >= 0) ? key : -static_cast<long long>(key);
        return static_cast<std::size_t>(magnitude) % slots_.size();
    }

    // 查找：从基地址起一格一格往后走。
    //   碰到「空」   → 探测链到头了，键不在表里；
    //   碰到「墓碑」→ 继续走（这正是墓碑存在的意义）；
```

```
    //   碰到「占用」且键相同 → 找到了。
    std::optional<std::size_t> find_slot(int key) const {
        for (std::size_t step = 0; step < slots_.size(); ++step) {
            std::size_t index = (home(key) + step) % slots_.size();
            if (slots_[index].state == SlotState::empty) {
                return std::nullopt;
            }
           if (slots_[index].state == SlotState::used && slots_[index].key == key) {
                return index;
            }
        }
        return std::nullopt;                    // 走遍全表也没有
    }

    // 找插入位置。比查找多做一件事：** 记住路上第一个墓碑 **。
    // 走到「空」时优先返回那个墓碑——回收墓碑，探测链才不会越来越长。
    // 但必须先走到「空」或找到同键才能停，否则会把一个已存在的键插第二遍。
    std::optional<std::size_t> insertion_slot(int key) const {
        std::optional<std::size_t> first_tombstone;
        for (std::size_t step = 0; step < slots_.size(); ++step) {
            std::size_t index = (home(key) + step) % slots_.size();
           if (slots_[index].state == SlotState::used && slots_[index].key == key) {
                return index;                   // 键已存在
            }
            if (slots_[index].state == SlotState::tombstone && !first_tombstone) {
                first_tombstone = index;
            }
            if (slots_[index].state == SlotState::empty) {
                return first_tombstone ? first_tombstone :
                ↪  std::optional<std::size_t>(index);
            }
        }
        return first_tombstone;                 // 全表没有空格，只能指望墓碑
    }

    std::vector<Slot> slots_;
    std::size_t size_;
};
```

10.2 集合的检索

检索也可以用来实现集合。集合只关心一个元素在不在里面，不保留重复，也不保证顺序。交、并、差、包含都可以建立在「是否属于」之上。

10.2.1 集合的数学特性

集合是由若干个确定的、相异的对象构成，这些对象称为元素；一个集合中不包含两个完全相同的元素。在问题求解中，集合是十分有用的工具。数学上集合（set）的元素可以是原子（属初等类型，不可再分解），也可以是一个集合，而且元素个数可以是有限的也可以是无限的。表示一个集合通常是把成员放在一对 花括号中、元素之间用逗号隔开，例如 $\{1, 4\}$；**$\{1, 4\}$ 和 $\{4, 1\}$ 代表的是同一个集合，因为集合 中元素的次序是无关紧要的**。当元素数目较多难以一一枚举时，一般采用 `{x | x 应满足的条件}` 这种形式，其中的条件是一个谓词，严格地定义了属于该集合中的元素。元素个数为零的集合称为**空集**，一般用 $\emptyset$ 表示。

几个基本关系与运算，本节的实现和测试都建立在它们之上：

- **成员关系**是最基本的：若 x 是集合 A 的元素，则称 x 属于 A，记作 $x \in A$。
- **子集与超集**：如果集合 A 的每个元素也都是集合 B 的元素，称 A 被 B 包含，记作 $A \subseteq B$，也称 A 是 B 的子集（subset），或称 B 是 A 的超集（superset）。**每个集合都是其本身的子集合，空集是任何集合的子集**。如果 A、B 互相包含，则两集合相等；A 是 B 的真子集必须满足 $A \subseteq B$ 且 $A \neq B$。
- **并、交、差**：由至少属于 A 和 B 之一的一切元素组成的集合称为并集，记作 $A \cup B$；由 A 和 B 的所有共同元素组成的集合称为交集，记作 $A \cap B$；由所有属于 A 但不属于 B 的 元素全体组成的集合称为差集，记作 $A - B$。例如 $A = \{1, 2, a, b, c\}$、$B = \{1, 3, b, d\}$ 时，$A \cup B = \{1, 2, 3, a, b, c, d\}$，$A \cap B = \{1, b\}$，$A - B = \{2, a, c\}$。
- **幂集**：一个集合 A 的所有子集合的全体也是一个集合，称为 A 的幂集，记作 $P(A)$。例如 $A = \{a, b, c\}$ 时 $P(A)$ 含 8 个元素，从 $\emptyset$ 到 $\{a, b, c\}$。

集合的元素互异：同一个值不能出现两次。「x 属于 S」是一个命题，不是一个位置。所以插入一个已经在里面的键、删除一个不在里面的键，都是正常、可预期的失败，应当返回「没做成」，而不是抛异常或打印一行。对任意元素都有 $x \notin \emptyset$；但子集关系方向不同，任何集合都包含空集，空集也包含自身。

集合运算应满足可拿来测试的代数性质：$A \cap B = B \cap A$，$A \cap A = A$，$A \cap \emptyset = \emptyset$；包含关系满足 $A \supseteq A$，且空集被任何集合包含。示例测试不只核对一次结果，还要用这些性质防止实现 “碰巧对一组输入”。

10.2.2 计算机中的集合

计算机所支持的集合，其基类型（basetype）**一般是有限、顺序类型**。被定义的集合类型称为与基类型 相联系的集合类型：**集合类型的值集是其基类型值集的幂集，集合类型的每个值是幂集的一个子集**。这句话解释了为什么下面那种「位表示」成立——既然每个集合值都是全集的一个子集，那么给全集的 每个元素分配一个二进制位，一个集合就正好是一个位串。

同一组集合运算，底下可以用不同结构：线性表加顺序检索（实现简单，适合很小的集合）、有序表加二分、二叉搜索树、散列表。规模上去以后，线性表的 $O(n)$ 检索会拖垮交和并。本章的 `IntSet` 故意用线性表，把接口先钉死：`insert`

/ `erase` 返回 `bool`，`contains` 只回答在不在，`intersection` 和 `includes` 用检索组合出来。换成散列表时，调用方不用改。

`IntSet` 的实现见上面那份清单。注意它的每一个运算最后都落到 `contains`，而 `contains` 是顺序检索 O(n)——所以交集是 O(n·m)。10.3 **节的散列就是来把这个 O(n) 压成 O(1) 的**，接口一个字都不用改。

有限全集上的位表示

若全集固定而且不大,集合可以直接表示成位向量。设全集为 `{a,b,c,d,e,f,g,h}`，从低位到高位依次对应这 8 个元素：

```
A = {a,c,f}     -> 00100101
B = {b,c,f,g}   -> 01100110
A ∩ B            -> 00100100 = {c,f}
A ∪ B            -> 01100111 = {a,b,c,f,g}
A \ B           -> 00000001 = {a}
```

交、并、差分别变成按位与、按位或、`A & ~B`，一次机器指令能并行处理 32 或 64 个元素。**这种表示方法称为位向量（bit vector）或者位图（bitmap）**：它很节省空间，而且对于「属于」「并」「交」「差」操作十分方便——**集合比数组的操作更加便捷**，例如对于数组的插入和删除都有大量的数据 移动，而集合类型的并、交、差运算只需要修改相应的比特位标记；要确定某个元素是否在集合中，只需要直接检查对应的位标志。如果集合大小在计算机的一个字长范围内，而且高级语言支持按位操作，就可以通过逻辑的位操作而完成集合的并、交、差运算。

原书举了一个可以口算的例子（例 10.1，计算 $0 \sim 15$ 之间的奇素数集合）：假设位向量从右到左 表示自然数，则素数集合 $\{13, 11, 7, 5, 3, 2\}$ 表示为 `0010100010101100`，奇数集合 $\{15, 13, 11, 9, 7, 5, 3, 1\}$ 表示为 `1010101010101010`；两者按位与得到 `0010100010101000`，表示 $0 \sim 15$ 之间的奇素数集合为 $\{13, 11, 7, 5, 3\}$。**一次按位与，就完成了两个集合的求交**。

代价是空间与全集大小成正比：学号全集若可能有十亿个值、实际只出现几十个，位向量就很浪费；这时散列表或树更合适。

位表示还要求先固定 “元素到位号” 的映射。若映射随运行改变，两个看起来同为 `00100101` 的位串可能代表不同集合，按位运算就没有意义。第 11 章位图索引会把同一思想扩展到数据库记录列。

10.3 散列方法

前面所介绍的检索，基本上都是**基于关键码比较**的检索：顺序检索和分块检索依赖于「等于」或「不等于」的判断，而二分检索和树形检索（BST、B 树等）依赖于「大于」「等于」「小于」这三种判断。**这些检索方法 的平均检索长度都与 n 有关**；检索是直接面向用户的操作，当问题规模 n 很大时，上述检索的时间 效率可能使得用户无法忍受。

最理想的情况是，根据关键码值直接找到记录的存储地址，而不需要把待查关键码与候选记录集合的某些记录进行逐个比较。例如，读取指定下标的数组元素，就是根据数组的起始存储地址以及数组下标值而直接计算出来的，所花费的时间是 $\Theta(1)$，与数组元素的个数 n 无关——受此启发，计算机科学家发明了散列方法。

散列方法的主要思想是根据结点的关键码值来确定其存储地址：以关键码值 K 为自变量，通过一定的函数关系 $h(K)$（称为**散列函数**，hash function）计算出对应的函数值，把这个值解释为结点的存储地址，将结点的关键码和属性数据一起存入到此存储单元中；检索时用同样的方法计算地址，然后到相应的单元中取出要找的结点。按散列存储方式构造的存储结构称为**散列表**（hash table），散列表中的一个位置称为一个**槽**（slot）。

原书的例 10.2 值得抄下来，因为它一句话说明了「冲突」是怎么来的。已知线性表关键码值集合为 $S=\{$and, array, begin, do, else, end, for, go, if, repeat, then, until, while, with$\}$，可设散列表为 `char HT[26][8]`。如果散列函数取关键码 K 的第一个字母在字母表中的序号，即 $h(K)=K[0]-$'a'，那么 (and, array)、(else, end)、(while, with) 这三对关键码会发生冲突。改成「首尾字母序号的平均值」，即 $h(K)=(K[0]+K[\text{last}]-2\times$'a'$)/2$，这一组关键码就分得开了——对应的散列表如图 10.5：

散列地址	关键码
0	
1	and
2	
3	end
4	else
5	
6	if
7	begin
8	do
9	
10	go
11	for
12	array

散列地址	关键码
13	while
14	with
15	until
16	then
17	
18	repeat
19	
20	
21	
22	
23	
24	
25	

图 10.5　例 10.2 中关键码对应的散列表。26 个槽装 14 个关键码，**空槽是有意留的**。

由此可以看出：一般情况下，散列表的空间必须比结点的集合大，此时虽然浪费了一定的空间，但换取的是检索效率。设散列表的空间大小为 M、填入表中的结点数为 N，则称 $\alpha=N/M$ 为散列表的**负载因子**（load factor，又称「装填因子」）。建立散列表时，若关键码与散列地址是一对一的关系，则检索时只需根据

散列函数对给定值进行某种运算即可得到待查结点的存储位置；但是，**散列函数可能 对于不相等的关键码计算出相同的散列地址，该现象称为冲突**（collision），**发生冲突的两个关键码称为 该散列函数的同义词**。在实际应用中，很少存在不产生冲突的散列函数，必须考虑发生冲突时的处理办法。

因此，采用散列技术时需要考虑的两个首要问题是：① 如何构造（选择）使结点「分布均匀」的散列 函数；② 一旦发生冲突，用什么方法来解决。当然，还需考虑散列表本身的组织方法。

散列不保持键的次序，不能按范围遍历；装载因子过高或散列不均匀时，冲突变多，会退化成接近线性。

10.3.1 散列函数

散列函数 h 把关键码映到 $\{0, 1, \dots, M-1\}$。**选择原则有三条**：运算尽可能简单；函数的值域必须 在散列表的范围内；尽可能使得结点均匀分布，即尽量让不同的关键码具有不同的散列函数值。需要 考虑的因素包括关键码长度、散列表大小、关键码分布情况、记录的检索频率等。下面是原书介绍的几种 常用构造方法。

1. **除余法**（modulo division method）。用关键码 x 除以 M（往往取散列表长度）并取余数作为散列 地址，$h(x) = x \bmod M$。它几乎是最简单的散列方法，**但挑选 M 值时要格外小心**：把 M 设置为 偶数，意味着 x 是偶数则 $h(x)$ 也是偶数、x 是奇数则 $h(x)$ 也是奇数——如果偶数关键码比奇数 出现的概率大，散列值就分布不匀。类似地，常有人把 M 设置为 2 的幂，此时 $h(x) = x \bmod 2^k$ 仅仅是 x 用二进制表示时最右边的 k 个位；若 M 是 10 的幂，则仅仅是最右边的 k 个十进制位。**这两个散列函数虽然易于计算，但它们不依赖于 x 的全部比特位，并不合乎要求**。因此通常选择一个 **质数**作为 M 值，那么 $x \bmod M$ 就依赖于 x 的所有位，这就增大了均匀分布的可能性。除余法的优点是实现简单、M 不必是常数（可以在运行时确定）、运行时间是常数；潜在的缺点是 **连续的关键码映射成连续的散列值**——虽然这能保证连续关键码不冲突，但也意味着要占据连续的数组 单元，在某些实现方法中可能导致检索性能的降低。

2. **乘余取整法**（multiplicative method）。先让关键码 K 乘上一个常数 A $(0 < A < 1)$，提取乘积 的小数部分，然后再用整数 n 乘以这个值并向下取整：$hash(K) = \lfloor n \times (A \times K \bmod 1) \rfloor$，其中 $A \times K \bmod 1$ 表示取小数部分。**这个方法的优点是对 n 的选择无关紧要**：若地址空间为 p 位就取 $n = 2^p$，所求出的散列地址正好是 $A \times K$ 小数点后最左 p 位值。Knuth 对常数 A 做过仔细研究，认为它可以取任何值，最佳选择与待散列的数据特征有关；**一般情况下取黄金分割 $A = (\sqrt{5} - 1)/2 \approx 0.6180339$ 最理想**。

3. **平方取中法**（mid-square method）。由于整数相除通常比相乘慢，有意识地避免除余法运算可以提高 运行时间。做法是**先求关键码的平方值以扩大相近数的差别，然后根据表长度取中间的几位数**（往往取 二进制的比特位）作为散列函数值——因为一个乘积的中间几位数与乘数的每一数位都相关，所以由此产生的散列地址比较均匀。

4. **数字分析法**（digit analysis method）。设有 n 个 d 位数、每一位可能有 r 种不同的符号；这 r 种符号在各位上出现的频率不一定相同，**可根据散列表的大小，选取其中各种符号分布均匀的若干位 作为散列地址**。各位数字中符号分布的均匀度可以量出来：

$$\lambda_k = \sum_{i=1}^{r} \left(\alpha_i^k - n/r\right)^2$$

其中 α_i^k 表示第 i 个符号在第 k 位上出现的次数，n/r 表示各种符号在 n 个数中均匀出现的 期望值。λ_k **值越小，表明第 k 位各种符号分布得越均匀**。原书例 10.5 那组学号形态的关键码（992148、991269、990527、991630、991805、991558、992047、990001，共 $n = 8$ 个，每位可能有 $r = 10$ 种数字，均匀出现的期望值 $n/r = 0.8$）就是典型：第①、②位全是 9，$\lambda_1 = \lambda_2 = (8-0.8)^2 \times 1 + (0-0.8)^2 \times 9 = 57.6$；第③位只出现 0、1、2（0 和 2 各两次、1 四次），$\lambda_3 = (2-0.8)^2 \times 2 + (4-0.8)^2 \times 1 + (0-0.8)^2 \times 7 = 17.6$；第④、⑤、⑥位各有两个数字出现两次、四个数字出现一次，$\lambda_4 = \lambda_5 = \lambda_6 = (2-0.8)^2 \times 2 + (1-0.8)^2 \times 4 + (0-0.8)^2 \times 4 = 5.6$。所以若散列地址是 3 位十进制数，应取第④、⑤、⑥位；取前三位做地址等于把所有记录挤进少数几个槽。数字分析法只适用于事先知道全部关键码各位分布的情况——换一个关键码集合，选哪几位就要重新决定。

5. **基数转换法**（radix transformation）。将关键码值看成另一种进制的数、再转换成原来进制的数，然后选其中几位作为散列地址。例如对于十进制关键码值 210485，先把它看成是十三进制的数并转换为 十进制数：$210485_{13} = 2 \times 13^5 + 1 \times 13^4 + 0 \times 13^3 + 4 \times 13^2 + 8 \times 13^1 + 5 \times 13^0 = 771932_{10}$；然后从中选取几位作为散列地址——假设散列表长度是 10000，则可取低 4 位 1932。**通常要求两个基数互素，且新基数比原基数大**。

6. **折叠法**（folding method）。有时关键码所含的位数很多，采用平方取中法计算太复杂，则可将关键码 分割成位数相同的几部分（最后一部分的位数可以不同），然后取这几部分的叠加和（舍去进位）作为散列 地址。原书举的例子是 ISBN 码：每一种西文图书都有一个 10 位十进制数字的国际标准图书编号；若以 ISBN 作为关键码建立一个散列表，当馆藏书种类不到 10000 时，可采用此法构造一个 4 位数的散列函数。一本书的编号为 `0-442-20586-4` 时，**移位叠加**得 $5864 + 4220 + 04 = [1]0088$，取 $h(K) = 0088$；**分界叠加**得 $5864 + 0224 + 04 = 6092$，取 $h(K) = 6092$。

原书打了一个很形象的比喻：**分界叠加法就是把数字顺序写到比较透明的纸上，从低位开始把纸折起来，每个间隔中数字位数一样，然后按照某种策略计算这些数字子串的和；移位叠加就是依照折缝把纸裁开，然后把这些数字按位相加**。分界叠加时，奇数面（正面）的数字串不变，偶数面（反面）的数字串把反写的数字改写为正写，然后数字子串按位相加。

7. ELFhash **字符串散列函数**。ELFhash 函数在 UNIX 系统 V 版本 4 中的 ELF

格式（executable and linking format，可执行链接格式）中会用到；ELF 文件格式用于存储可执行文件与目标文件，ELFhash 函数是对字符串的散列。

结果 h 先初始化为 0，循环地将一个字符串数组中的每个元素依次处理：**上次结果 h 左移 4 位，与当前元素相加，组成一个新长整数 h；取 h 的高 4 位存入 g；如果 g 不为零，把 g 右移 24 位（即把 g 的高 4 位移动到低 4 位），然后与 h 进行按位异或操作；g 取反，然后跟 h 进行按位与运算，得到新的 h。**最后得到的长整数 h 对散列表长取余，得到相应的散列数值，即在散列表中的位置。

ELFhash **对于长字符串和短字符串都很有效，字符串中每个字符都有同样的作用**，它巧妙地对字符的 ASCII 编码值进行计算，能够比较均匀地把字符串分布在散列表中——本节的实现就是它。

本章实现了 ELFHash：每个字节先左移 4 位再加进去，高 4 位若非零就折回到低位并清掉。字节按 `unsigned char` 读，避免 `char` 在有的平台上是有符号的、高位 1 被当成负数。它不是密码学哈希，只求快和够匀。

散列函数不能只拿一两个键试。若关键码本身有规律，例如学号末两位大量重复，直接取末位会把数据挤进少数槽。除留余数法常把表长选成与数据规律无公因子的素数；平方取中和折叠则试图把关键码不同位置的信息混在一起。

```
          5864                5864
          4220                0224
+           04        +         04
------------------  ------------------
        [1]0088             6092
   h(K) = 0088         h(K) = 6092
```

图 10.6　移位叠加（左）与分界叠加（右）。

图 10.6 展示两种折叠法：把长关键码分段后相加。移位叠加直接求和，分界叠加把相邻段反向后再加，目的都是让关键码每一部分参与地址计算。无论用哪一种，最终仍要对表长取模，保证结果落在合法槽位范围。

评价散列函数至少看三件事：同一键必须稳定得到同一地址；计算成本不能比检索本身还贵；在实际键分布上应尽量均匀。密码学散列还要求抗碰撞、抗逆向，本章散列表不承担这些安全目标，不能拿 ELFHash 保存密码。

`elf_hash` 的实现见上面那份清单。它逐字节读的是 `unsigned char`—— 教学版的测试里有一条拿 UTF-8 的「中」（E4 B8 AD）做输入：按无符号读得到 `0xF02D`，按有符号读，负值被符号扩展成 `0xFFFFFF..` 带进运算，结果是 `0x0FFF00DD`（中间值按 32 位截断后）。两者天差地别，所以这一条能把两种写法分开。

10.3.2 开散列方法 (拉链法)

“开散列”（open hashing）在现代资料中更常写作**分离链接**（separate chaining）；这里的 “开” 指冲突元素移到槽外的链或桶中，并非开放地址法。**尽管散列函数的目标是尽可能均匀分布使得冲突最少，但实际上冲突是无法避免的**，因此必须

研究冲突 解决策略。冲突解决技术可以分为两类：**开散列方法**（open hashing，也称为拉链法，separate chaining）和**闭散列方法**（closed hashing，也称为开地址方法，open addressing）。**两者的不同之处在于：开散列法 把发生冲突的关键码存储在散列表主表之外，而闭散列法把发生冲突的关键码存储在表中另一个槽内**。

开散列不在表内挤位置：每个槽挂一条链表（或动态数组），散列到同一地址的关键码都串在这条链上。插入是算地址再接到链头或链尾；查找、删除只在那一条链上走。删掉一个结点不会影响别的链，也不存在「探测序列被截断」的问题。

装载因子 $\alpha = n/m$ 可以大于 1，链只是变长。成功查找的期望比较次数大约是 $1 + \alpha/2$。实现简单，空间上每个结点多一个指针。本章主实现是闭散列，开散列不另写一份。

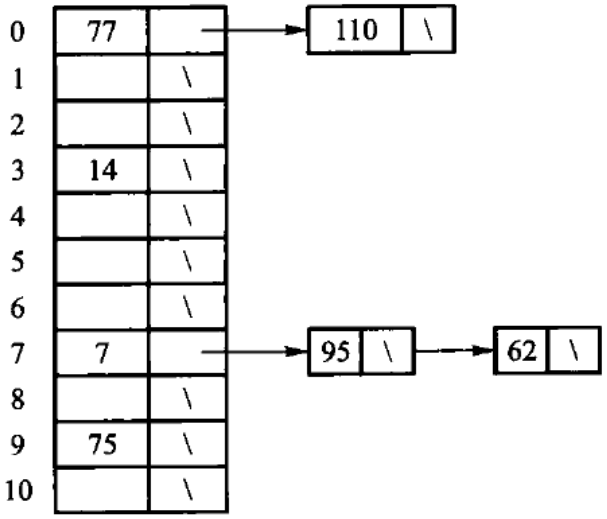


图 10.7 中每个表槽只是链头，同址键挂在同一条链上。原书这张图存 7 个数、散列表有 11 个槽、散列 函数是 $h(K) = K \bmod 11$，插入顺序为 77、7、110、95、14、75、62：结果有 2 个值散列到第 0 槽、1 个到第 3 槽、3 个到第 7 槽、1 个到第 9 槽。查找失败只需走完对应链，不看其他桶；删除也只摘掉链中结点。若每条链内部无序，成功查询平均看半条链，失败查询看整条链，这正是上面成功检索约 $1 + \alpha/2$、以及 10.3.5 节表中不成功检索约 $\alpha + e^{-\alpha}$（主项是整条链长 α）的来源。

外存上的散列还有一种变体：**桶式散列**。把文件的记录分成若干存储桶，每个桶由一个或多个页块用指针串起来，散列函数 $h(K)$ 算出的是桶号而不是槽号。

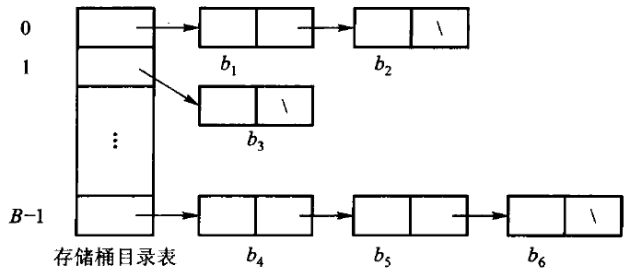


图 10.8　具有 B 个存储桶的散列文件组织。存储桶目录表存 B 个指针，各指向对应桶的第一个页块；1 号桶只有一个页块，$B-1$ 号桶由 b_4、b_5、b_6 三块串成，每桶最后一块的指针为空。检索一次要一次访外读目录表、再逐块查桶，平均访外次数约为桶内页块数的一半——**所以理想状态是每个桶只有一个页块**，这也解释了为什么文件增长后需要重新组织、改散列函数并加长目录表。第 9 章和第 11 章讨论的正是这类「代价按访外次数计」的结构。

拉链法的风险从 “表满” 变成 “某条链过长”。散列函数分布不均或攻击者能构造同址键时，单个桶可能退化成长度 n 的链，查找回到 $O(n)$。工程库会随机化散列种子，或在链过长时改用平衡树；平均 $O(1)$ 从来不是无条件保证。

10.3.3 闭散列方法 (开地址法)

“闭散列”（closed hashing）在现代资料中更常写作**开放地址法**（open addressing）：所有元素都放在主表槽位中，通过探测序列处理冲突。闭散列方法把所有记录直接存储在散列表中。每个记录关键码 K 有一个由散列函数计算出来的**基位置**（home position）$h(K)$；如果要插入 K 时基位置已被占据（发生碰撞），则把 K 存储在表中的其他 地址内，由冲突解决策略确定是哪个地址。

基本思想是「探查序列」：当发生冲突时，为关键码 K 生成一个散列地址序列 $d_0, d_1, d_2, \cdots, d_{m-1}$ $(0 < m \le M)$，其中 $d_0 = h(K)$ 是基地址，所有 $d_i = (d_0 + p(K,i)) \bmod M$ 是后继散列地址，$p(K,i)$ 称为**探查函数**。插入 K 时，若基地址上的 结点已被占用，则按上述地址序列依次探查，将找到的第一个开放的空闲位置 d_i 作为存储位置；若所有后继地址都不空闲，说明该闭散列表已满、报告溢出。**检索 K 时依次查找按同值的后继地址 序列**，检索成功时返回该位置；**如果沿着探查序列检索时遇到了开放的空闲地址，则说明表中没有待查 的关键码**——这一句正是下面「删除为什么不能标空」的根据。删除 K 时同样按后继地址序列查找，找到后对该结点加以**删除标记**。因此，对于闭散列表来说，**构造后继散列地址序列的方法，也就是处理 冲突的方法**。

1. **线性探查法**（linear probing）。把散列表看成一个环形表，若在基地址 d 发生冲突，则依次探查 $d+1, d+2, \cdots, M-1, 0, 1, \cdots, d-1$，直到找到一个空闲地址或查找到关键码为 K 的结点为止；若探查一遍之后又回到了地址 d，则插入和检索都会失败。探查函数是 $p(K,i) = i$。第 i 次看 $(h(k)+i) \bmod M$。

线性探查的毛病叫「聚集」。散列基地址不同的结点，可能争夺相同的后继散列地址，这种现象称为 **聚集**（clustering，或称「堆积」），也称为**基本聚集**（primary

clustering)，原因在于散列函数选择不当或负载因子过大。理想情况下表中每个空槽都应该有相同的机会接收下一个要插入的记录；但在原书例 10.7 的那张表里，下一条记录放在第 11 槽的概率是 $2/15$，放到第 7 槽的概率却是 $11/15$——概率不再相等，**小的聚集可能汇合成大的聚集，导致很长的探查序列**。

这两个概率是怎么来的，值得手算一遍。原书例 10.7：关键码 (26, 36, 41, 38, 44, 15, 68, 12, 06, 51, 25)，表长 $M = 15$，取小于 M 的最大质数 13 作除数，$h(K) = K \bmod 13$，用线性探查依次插入：

关键码	26	36	41	38	44	15	68	12	06	51	25
$h(K)$	0	10	2	12	5	2	3	12	6	12	12
落入槽	0	10	2	12	5	3	4	13	6	14	1
探查次数	1	1	1	1	1	2	2	2	1	3	5

最后的表（原书图 10.9）是：

```
槽:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14
    26 25 41 15 68 44 06  .  .  . 36  . 38 12 51
```

25 的基地址是 12，依次探查 12、13、14、0 都被占，第 5 次才在槽 1 落脚；按探查次数，等概率下成功检索的平均检索长度为 $(1\times6+2\times3+3+5)/11 = 20/11 \approx 1.82$。现在只剩槽 7、8、9、11 是空的。若下一个关键码的基地址在 15 个槽上等可能，那么基地址为 10、11 时会落进槽 11（$2/15$）；基地址为 8 或 9 时直接落在本槽（各 $1/15$）；而基地址为 12、13、14、0、1、…、7 的共 11 种情况，都会沿着「12–14 接 0–6」这一整段连续占用区滑到槽 7（$11/15$）。占用区越长，就越容易吸收新记录而变得更长。

可以改进线性探查来缓解基本聚集：每次跳过常数 c 个而不是 1 个槽，探查函数 $p(K, i) = i \cdot c$。**但 c 必须与表长 M 互素**，否则探查序列走不遍所有槽——例如 $c = 2$ 且槽数为偶数时，基位置在偶数槽的关键码只能走遍所有偶数槽。事实上，改进的线性探查仍不能很好地解决聚集：$c = 2$ 时 $h(k_1) = 3$、$h(k_2) = 5$ 的两个关键码，探查序列分别是 3、5、7、9、… 和 5、7、9、…，**仍然纠缠在一起**。

2. **二次探查法**（quadratic probing）。生成的后继散列地址不是连续的而是跳跃式的，以便为后续数据元素留下空间从而减少聚集：探查序列依次为 $1^2, -1^2, 2^2, -2^2, \cdots$，即发生冲突时将同义词来回散列在第一个地址的两端。写成公式，若基地址为 d，则 $d_{2i-1} = (d + i^2) \bmod M$、$d_{2i} = (d - i^2) \bmod M$（取模结果按 $0 \sim M-1$ 理解；C++ 里负数取余可能得负值，要先加上 M 的倍数），探查函数为 $p(K, 2i-1) = i^2$、$p(K, 2i) = -i^2$（$0 < i < M/2$）。原书例 10.8：$M = 13$，$h(k_1) = 3$、$h(k_2) = 2$，k_1 的探查序列是 3、4、2、7、…，k_2 的是 2、3、1、6、…——尽管 k_2 第 2 步就探查到 k_1 的基地址，此后两条序列立即分开，这正是它比线性探查好的地方。缺点是**不易探查到整个闭散**

列表的所有位置；不过实际应用中，如果探查序列需要查找六七个槽，散列表的性能就已经非常糟糕，应当重新设计散列函数、扩大空间（降低负载因子）或重新组织散列表了。

3. **随机探查法**（pseudo-random probing）。理想的探查函数应当在探查序列中随机地从未访问过的槽 中选择下一个位置；但实际上不能真正随机，因为检索时必须能重建同样的探查序列。做法是设置一个 **伪随机**探查序列：第 i 个槽是 $(h(K)+r_i) \bmod M$，其中 r_i 是 $1 \sim M-1$ 之间数的伪随机数 序列，所有插入和检索都使用相同的伪随机数。**注意这串伪随机数要避免产生 0 或** M，因为那会导致 对基地址的重复探查，多做一次无效操作。原书例 10.9：$M=13$，伪随机数前 3 个为 $r_1=2$、$r_2=3$、$r_3=7$，$h(k_1)=4$、$h(k_2)=2$，则 k_1 的探查序列是 4、6、7、11、...，k_2 的是 2、4、5、9、...——k_2 第二步 探查到 k_1 的基地址，但随后同样立即分开。

二级聚集。伪随机探查和二次探查都能消除基本聚集，即基地址不同的关键码其探查序列的某些段重叠在 一起的问题。但是，如果两个关键码散列到**同一个**基地址，这两种方法给出的仍是完全相同的探查序列，同样会产生聚集——因为它们的探查序列只是基地址的函数，而不是原来关键码值的函数。这个问题称为**二级聚集**（secondary clustering）。

4. **双散列探查法**（double hashing）。为了避免二级聚集，需要使得探查序列是原来关键码值的函数、而不是基位置的函数：**双散列探查法采用第二个散列函数的值作为线性探查的常数项，形成探查序列**。它使用两个散列函数 h_1 和 h_2，其中 h_1 与前面的 h 一样，以关键码为自变量产生 $0 \sim M-1$ 之间的一个数作为散列地址；h_2 也以关键码为自变量，产生一个 $1 \sim M-1$ 之间的、**并尽量与** M **互素**的数作为对散列地址的补偿。若在地址 $h_1(K)=d$ 发生冲突，则再计算 $h_2(K)$，得到的探查序列 为 $(d+h_2(K)) \bmod M$、$(d+2h_2(K)) \bmod M$、$\cdots$，即 $d_i=(d+i \cdot h_2(K)) \bmod M$，探查函数为 $p(K,i)=i \cdot h_2(K)$。

定义 $h_2(K)$ **的方法较多，但无论采用什么方法，都应该尽量使** $h_2(K)$ **与** M **互素**，才能使发生冲突的 同义词地址均匀地分布在整个表中，否则可能造成同义词地址的循环计算，从而在存储区并未放满时就产生 溢出。一种方法是选择 M 为一个素数、h_2 返回的值在 $1 \le h_2(K) \le M-1$ 范围之间；另一种方法 设置 $M=2^m$，让 h_2 返回一个 1 到 2^m-1 之间的奇数值。若 M 是素数、$h_1(K)=K \bmod M$，则可以定义 $h_2(K)=K \bmod (M-2)+1$，或者 $h_2(K)=\lfloor K/M \rfloor \bmod (M-2)+1$；若 M 是任意数、$h_1(K)=K \bmod p$（p 是小于 M 的最大素数），可以定义 $h_2(K)=K \bmod q+1$（q 是小于 p 的最大素数）。**注意，尽管最后一个方案并不能保障** $h_2(K)$ **与** M **互素，但还是常被采用**。

用双散列函数探查法得到的探查序列是**跳跃式**散列在整个存储区域里的，而不是像线性探查法那样探查 一个顺序的地址序列。**双散列方法的优点是不易产生聚集，缺点是计算量稍微大一些**。

一句话总结这四种的取舍：通常希望找到一个探查函数，即使两个不同关键

码的基地址、或者中间的 探查序列偶然会合，最终的探查序列也能够岔开。本书实现的是最简单的线性探查，聚集问题因此确实 存在；二次探查、双散列的改进留作习题。

表满了就插不进去，所以 α 必须小于 1，实务上常在 0.5 到 0.7 之间扩表。删除是闭散列的难点：不能把槽直接标成空。否则沿这条探测链查找后面的键时，会在空槽处误判「从来没插入过」而提前停止。解决办法是给槽加第三种状态——除了「空」和「有元素」，再加一个**墓碑**（tombstone），意思是「这里曾经有元素，被删掉了，但探测链还要继续往下走」。

假设表长为 5，键 1、6、11 的散列地址都为 1，于是它们依次放在槽 1、2、3：

```
槽:  0    1    2    3    4
     空   1    6    11   空
```

删除键 1 后，若把槽 1 直接标为空，查找键 6 从槽 1 开始会误以为「从未插入过 6」并提前停止。墓碑表示「这里曾有元素，但探测链还要继续」。插入新键时可以复用第一个墓碑，但查重必须继续走到真正的空槽或找到同键为止。

把完整状态变化写出来，更容易看清「空」和「墓碑」的不同：

操作	槽 0	槽 1	槽 2	槽 3	槽 4	说明
初始	空	空	空	空	空	-
插入 1	空	1	空	空	空	`home(1)=1`
插入 6	空	1	6	空	空	1 冲突，探测到 2
插入 11	空	1	6	11	空	连续冲突，探测到 3
删除 1	空	墓碑	6	11	空	探测链不能截断
查找 11	空	墓碑	6	11	空	越过墓碑，在 3 命中
插入 16	空	16	6	11	空	记住首个墓碑，确认无重复后复用

最后一步不能一看到墓碑就立刻插入。假设同一个 16 已经在更后面的槽，提前写入会制造重复键。正确插入先记住第一个墓碑，继续探测到真正的空槽或同键；只有确认键不存在后才回头复用墓碑。

散列表追求平均 O(1) 查找，前提是装载因子不过高且散列分布均匀；它不保持键的有序关系，不适合按范围遍历。

10.3.4 闭散列表的算法

先说清楚这一节在实现什么。字典（dictionary）是一种特殊的集合，其元素是「(关键码，属性值)」二元 组，**关键码必须是互不相同的**（在同一个字典之内）。字典在某些语言中可能称为联合内存（associative memories）或联合数组（associative arrays）：主要操作是依据关键码来存储和析取值，可以用于组织简单 的数据库，提供存储、查找和删除记录的功能。字典的实现方式有很多，例如有序线性表、字符树等；**用散列方法实现的字典具有非常高效的检索性能**。

原书【算法 10.9】的 hashdict ADT 把散列表存储表、散列函数 h 和探查函数 p 一起放在 private 里，对外只给 hashInsert / hashSearch / hashDelete / size——**这样用户可以根据具体情况替换 散列函数和探查函数，而调用方不必改**。本书的 HashTable 是同一套接口的现代写法。

这一节的算法基本上都是针对闭散列方法的，散列表的检索过程与建表过程类似。需要注意一条前提：本节讨论的插入和检索算法都假定每个关键码的探查序列中至少有一个槽是空的，**否则它们在检索不成功 时就会进入无限循环**；实际应用中可以返回插入或检索算法的长度，根据检索长度的统计数据决定是否 需要重新调整散列表。

插入算法的思想是：假设给定的值为 K，根据散列函数 h 计算出散列地址 $h(K)$；若表中该地址对应的 空间未被占用，则把待插入记录填入该地址；如果该地址中的值与 K 相等，则报告「散列表中已有此记录」；否则按设置的处理冲突方法查找探查序列的下一个地址，如此反复下去，直到某个地址空间未被占用（可以插入）或者关键码比较相等（有重复记录，不需要插入）为止。

删除为什么必须留墓碑，原书用一个例子说透了。上面的插入和检索算法都没有考虑删除操作；实际应用 中建表的过程就是反复地把元素插入散列表的过程，表建好后的主要操作就是检索——但也有些应用需要 动态地插入和删除某些元素。**删除闭散列表中元素时，不能简单地把表中的相应位置标为空，因为这个位置 很可能位于一些探查序列中；简单地删除将使得这些探查序列被断开**，当检索探查序列中处于这个位置之后 的其他记录时就可能出现漏检现象。例如原书图 10.9 中，关键码 41 和 15 都散列到第 2 个槽，15 被放到 第 3 个槽；如果从表中删除 41，对 15 的检索必须仍然探查第 2 个槽，**此时第 2 个槽的原值 41 已经被删除，失去了对关键码 15 的探查导引作用，从而导致检索失败**——**15 被漏检了**。

另一方面，删除后释放的槽应该能够为将来的插入使用。有人可能想到把探查序列之后的其他记录逐步 前移，或者把该探查序列中最后的记录填到刚刚被删除的单元——**这些方法都是不可取的，因为一个单元 可能不只处于一个探查序列中**（考虑前面讨论过的「聚集」现象），这可能会挪动其他同义词表中的记录，而造成其他混乱。

因此可以设置一个特殊的标记位，来记录散列表中的单元状态：**空单元、单元被占用、已删除**。一般来说 不能把空单元、已删除这两种状态用特殊的值标记，以区别于「单元被占用」状态，因为必须严格区别「空单元」和「已删除」这两种状态；**可以用墓碑（tombstone）来表示被删除状态**，标志一个记录曾经占用这个槽、但是现在已经不再占用。

加入删除操作后，**闭散列的插入操作都需要进行修改，而检索操作几乎可以不修改**：检索时如果遇到墓碑，检索过程会顺着探查序列继续进行，直到检索到关键码或者遇到空单元为止——**读者可以把墓碑看成是 不等于任何关键码的特殊值，对于检索算法没有影响**。有人可能认为：在插入时如果遇到标志为墓碑的槽，就可以把新记录存储在该槽中；**但是，为了避免插入两个相同的关键码，仍**

然需要沿着探查序列进行检索 过程——如果在探查序列中发现了相同的关键码则插入失败，否则找到一个真正的空位置时（也就是整个 探查序列都探测完毕时），可以在第一个墓碑处插入新关键码。

先跑一遍：

```
// 第 10 章「先跑一遍」：用教学版 HashTable 观察线性探测与墓碑删除。
// 编译运行：
//   g++ -std=c++17 -I code/ch10/search_hash code/ch10/search_hash/demo.cpp -o demo
↪  && ./demo
#include "teaching.hpp"

#include <iostream>

int main() {
    HashTable table(5);
    if (!table.insert(1) || !table.insert(6) || !table.insert(11)) {
        std::cout << " 插入 1、6、11 失败\n";
        return 1;
    }

    std::cout << " 插入 1、6、11 后:\n";
    for (std::size_t index = 0; index < table.capacity(); ++index) {
        const auto slot = table.slot_at(index);
        std::cout << "  槽 " << index << ": ";
        if (slot.state == HashTable::SlotState::used) {
            std::cout << slot.key << '\n';
        } else if (slot.state == HashTable::SlotState::tombstone) {
            std::cout << " 墓碑\n";
        } else {
            std::cout << " 空\n";
        }
    }

    if (!table.erase(1)) {
        std::cout << " 删除 1 失败\n";
        return 1;
    }
    std::cout << " 删除 1 后仍能找到 6? " << (table.contains(6) ? " 是" : " 否") <<
    ↪  '\n';
    if (!table.insert(16)) {
        std::cout << " 插入 16 失败\n";
        return 1;
    }
    std::cout << " 插入 16 后槽 1 是 "
              << table.slot_at(1).key
              << "（复用了墓碑）\n";
```

```
}
```

Python 版不用 `dict` 或 `set` 代做集合与散列；这两个名字由本单元的 `d025_forbidden` 额外封住。检索、集合、ELFHash 与线性探测都来自下面这份受测源码：

```
def sequential_search(values, key):
    for i, value in enumerate(values):
        if value == key:
            return i
    return None

def binary_search(values, key):
    first, last = 0, len(values)
    while first < last:
        middle = first + (last - first) // 2
        if values[middle] == key:
            return middle
        if values[middle] < key:
            first = middle + 1
        else:
            last = middle
    return None
```

```
class IntSet:
    def __init__(self):
        self._values = []
    def insert(self, value):
        if self.contains(value):
            return False
        self._values.append(value)
        return True
    def erase(self, value):
        found = sequential_search(self._values, value)
        if found is None:
            return False
        del self._values[found]
        return True
    def contains(self, value):
        return sequential_search(self._values, value) is not None
    def intersection(self, other):
        result = IntSet()
        for value in self._values:
            if other.contains(value):
                result.insert(value)
```

```
        return result
    def includes(self, other):
        return all(self.contains(v) for v in other._values)
    def size(self):
        return len(self._values)
```

```
def elf_hash(text):
    value = 0
    for character in text:
        # 截到 32 位：ELFhash 按 32 位定义，否则进位会越过第 32 位、与 C++ 版分叉
        value = ((value << 4) + ord(character)) & 0xFFFFFFFF
        high = value & 0xF0000000
        if high:
            value ^= high >> 24
        value &= ~high
    return value
```

```
class HashTable:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("hash table capacity must be positive")
        self._slots = [None] * capacity
        self._size = 0
    def _home(self, key):
        return abs(key) % len(self._slots)
    def insert(self, key):
        first_tombstone = None
        for step in range(len(self._slots)):
            i = (self._home(key) + step) % len(self._slots)
            if self._slots[i] == key:
                return False
            if self._slots[i] is _TOMBSTONE and first_tombstone is None:
                first_tombstone = i
            if self._slots[i] is None:
                target = first_tombstone if first_tombstone is not None else i
                self._slots[target] = key
                self._size += 1
                return True
        if first_tombstone is None:
            return False
        self._slots[first_tombstone] = key
        self._size += 1
        return True
    def contains(self, key):
        return self._find(key) is not None
```

```
    def erase(self, key):
        i = self._find(key)
        if i is None:
            return False
        self._slots[i] = _TOMBSTONE
        self._size -= 1
        return True
    def _find(self, key):
        for step in range(len(self._slots)):
            i = (self._home(key) + step) % len(self._slots)
            if self._slots[i] is None:
                return None
            if self._slots[i] == key:
                return i
        return None
    def size(self):
        return self._size
    def capacity(self):
        return len(self._slots)
    def slot_at(self, index):
        if index < 0 or index >= len(self._slots):
            raise IndexError("hash table slot")
        return self._slots[index]
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch10/search_hash \
    code/ch10/search_hash/demo.cpp -o /tmp/hash-demo
/tmp/hash-demo
```

```
插入 1、6、11 后:
  槽 0: 空
  槽 1: 1
  槽 2: 6
  槽 3: 11
  槽 4: 空
删除 1 后仍能找到 6? 是
插入 16 后槽 1 是 16（复用了墓碑）
```

若把删除写成「标空」而不是「标墓碑」，第二行会变成「否」——测试里有两条具名断言守住这件事。

`HashTable::home` 先取绝对值再取模，所以负键也能进表。`find_slot` 遇到 `empty` 就停，遇到 `tombstone` 继续；`insertion_slot` 记下沿途第一个墓碑，确认后方没有同键后复用它。按键删除返回 `bool`。

`HashTable` 的实现见上面那份清单。三处值得停一下，教学版的测试各有一条具名断言守着：

- **删除只能标墓碑**。上面那段推演不是纸上谈兵——把 `erase` 改成标 `empty`,「删掉 3 之后 10 还找不找得到」这条立刻变红。
- **插入要回收墓碑**，否则表用久了探测链只会越来越长。
- **回收墓碑时不能提前停**：碰到墓碑就返回，会把一个已经在后面的键插第二遍。这一条最隐蔽，所以单列了一个用例。

10.3.4a 进阶（选读）：工程版差在哪

工程版在 `code/ch10/search_hash/modern.hpp`。这一章没有手写存储管理，所以没有三法则/五法则之分，差别只有标注与排版：查询函数标了 `[[nodiscard]]`（防止「查了却忘了看结果」）与 `noexcept`，若干 `if` 分支压成一行。逻辑完全一致。

10.3.5 散列方法的效率分析

先说清楚衡量的是什么。散列方法的性能按「完成一次插入、删除或检索操作所需要的记录访问次数」来衡量。由于散列表的插入和删除操作都是基于检索进行的——**删除一条记录之前必须先找到它，因此 删除的访问数等于成功检索的访问数；而插入一条记录时必须找到探查序列的尾部**（对于不考虑删除的 情况是尾部的空槽；对于考虑删除的情况，也要找到尾部才能确定是否有重复记录），**这等于对这条记录 进行一次不成功的检索**。因此，**散列表的效率问题实质上还是平均检索长度的问题，而且需要区别对待 成功的检索与不成功的检索**。

当散列表比较空的时候，所插入的记录比较容易插入到其空闲的基地址；如果表中记录比较多，插入时 很可能要靠冲突解决策略来寻找探查序列中合适的另一个槽，检索时也常需要沿着探查序列逐个查找。**随着散列表记录不断增加，越来越多的记录有可能放到离其基地址更远的地方**——因此预期的代价与 负载因子 $\alpha = N/M$ 有关。

期望代价的推导。假定探查序列是散列表中槽的随机排列（在这些冲突处理策略中，双散列探查法的 探查序列最接近于随机分布），基地址被占用的可能性是 α，基地址和探查序列中下一个槽都被占用 的可能性是 $\frac{N(N-1)}{M(M-1)}$，发生第 i 次冲突的可能性是 $\frac{N(N-1)\cdots(N-i+1)}{M(M-1)\cdots(M-i+1)}$。如果 N 和 M 都很大，可以近似为 $(N/M)^i$，于是**插入（或一次不成功检索）的探查次数期望值**是

$$1+\sum_{i=1}^{\infty}(N/M)^i=\frac{1}{1-\alpha}$$

一次成功检索（或删除）的代价与当时插入那个关键码的代价相同；由于随着记录不断增加 α 也不断 增大，因此可以对从 0 到 α 的当前值积分，推导出检索操作的平均代价（实质上是对所有插入代价 的一个平均值）：

$$\frac{1}{\alpha}\int_0^{\alpha}\frac{1}{1-x}\,\mathrm{d}x=\frac{1}{\alpha}\ln\frac{1}{1-\alpha}$$

几种策略放在一起比（原书表 10.2；α 为负载因子）：

冲突解决策略	成功检索（删除）	不成功检索（插入）
开散列法（拉链）	$1+\alpha/2$	$\alpha+e^{-\alpha}$
双散列探查法	$\frac{1}{\alpha}\ln\frac{1}{1-\alpha}$	$\frac{1}{1-\alpha}$
线性探查法	$\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$	$\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$

从这张表和图 10.11 可以看出：开散列方法的效率最高，实际系统中使用的散列大多都是开散列。开散列非常简单、易于实现，它不会产生聚集现象（聚集导致更大的平均检索长度），删除也极为方便。大部分数据结构教材用比较多的篇幅讨论闭散列方法，是因为**闭散列需要考虑的因素更多，因而更需要 精心设计**；闭散列在某些受限的系统中（例如不能使用堆栈分配新空间）有独到的用途，并且经过精心 设计的闭散列的效率比开散列稳定。

0.5 **是一条经验阈值**。当 $\alpha<0.5$（散列表将近半满）时，大部分情况下检索长度小于 2；实际经验 表明**负载因子超过 0.5，散列表的性能就会急剧下降**。因此需要知道在最大负载情况下表中可能有多少 条记录，据此选择散列表的长度。如果散列表动态性比较强（有大量插入删除），性能可能会下降；这种 情况下可以保持每一条记录的访问计数，当负载因子超过阈值时重新散列整个表，或者周期性地重散列，**并把记录按照访问频率从高到低的顺序插入散列表中，保证上一个周期中被频繁访问的记录更有机会 接近基位置**。

总之，散列法的重要特征是：平均检索长度不依赖于散列表中结点的个数（前面介绍的其他几种检索 方法的平均检索长度都是表中结点个数的函数）——散列法的平均检索长度不随表目数量的增加而增加，而是随负载因子的增大而增加；如果安排得好，平均检索长度可以小于 1.5。

装载因子 α	线性探测成功查询近似 ASL	直观状态
0.25	1.17	大部分键在回家地址
0.50	1.50	冲突可控
0.70	2.17	聚集开始明显
0.80	3.00	应考虑扩表
0.90	5.50	少量空槽被长探测链隔开
0.95	10.50	已接近线性退化

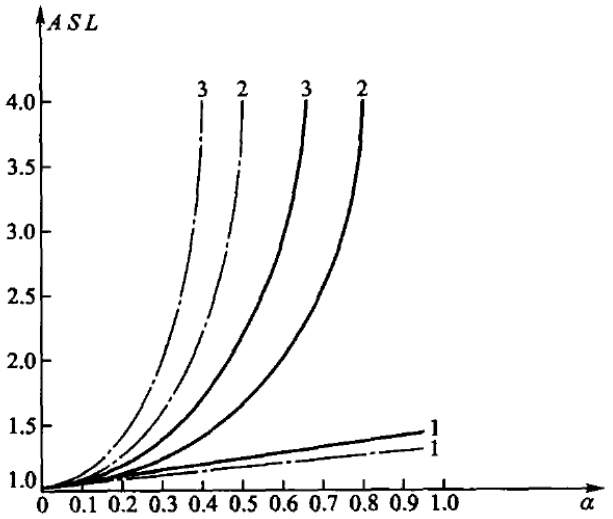


图 10.11　不同冲突处理方法的平均检索长度。实线是成功检索/删除，虚线是插入/不成功检索。

图 10.11 把不同冲突处理方法的曲线放在一起。横轴接近 1 时，开放地址法的曲线快速上扬；拉链法可以让 $\alpha > 1$，但链长仍随元素数增长。扩表不是只把数组变大：所有键的地址都依赖表长，必须按新表长重新散列，不能原封不动复制槽位。

墓碑也会抬高实际探测成本，却不计入 `size()/capacity()`。一张 “有效装载因子只有 0.4” 的表，如果另有 0.4 的槽是墓碑，失败查询仍可能走过 0.8 张表。工程实现通常同时跟踪有效元素数和已占用槽数，墓碑过多时即使容量未满也要重建。

10.3.6 散列方法的应用

散列方法的检索效率与数据规模无关、平均检索长度接近于 1.5，因此它成为一种很受欢迎的高效检索 方法。原书列出的应用今天大都还在：搜索引擎中的关键词字典、域名服务器 DNS 中域名与 IP 地址的 对应、操作系统中命令路径下的所有可执行程序名、编译系统中的符号表，都采用了散列技术以提高查找 速度。

一个能亲手验证的例子：UNIX 的 C Shell 运用了一个内部的可执行程序名散列表，来加速在 `PATH` 变量的目录集合中搜寻指令的执行速度。用户注册时，`PATH` 所涉及的路径下所有的可执行程序都被 散列到命令表；**当用户新加入一个可执行程序到 `PATH` 目录中时，如果不将这个内部散列表更新，就只能在该程序所在的目录下执行**——换个工作目录、不带路径直接敲程序名，系统会报 `Command not found.`。此时只要用 `rehash` 把内部散列表更新，就能搜寻到该新增的程序；相关的 指令还有 `hashstat`（显示内部散列表的统计信息）和 `unhash`（解除对 `PATH` 的散列，使得系统每执行 一个指令便到目录集合中逐一去尝试）。

另一类应用是「散列作指纹」。20 世纪 90 年代 Ronald L. Rivest 开发的 MD5

（message-digest algorithm 5）把文件的大容量信息「压缩」成唯一的 128 位信息摘要：传播过程中内容一旦改变（人为 修改或传输错误），重新计算 MD5 就会发现摘要不同，因此可以判定拿到的是一个不正确的文件。**但这一类用途与本章的散列表是两回事，而且 MD5 今天已不能用于安全目的**——它的抗碰撞性早已被 攻破，口令存储应使用专门的密码散列函数（bcrypt、scrypt、Argon2 等）并加盐。本章的 ELFHash 只求快和匀，**不承担任何安全目标**。

编译器符号表、缓存、去重都常用散列。需要范围查询或有序遍历时，应改用树或有序表。

选择结构时先问操作，不要先问哪个复杂度符号更小：

需求	适合的结构	不选散列的原因
精确查一个键	散列表	正是它的长项
按键从小到大遍历	有序表 / 搜索树	散列槽位没有关键码顺序
查 `[low,high]` 范围	B+ 树 / 有序数组	散列不能定位相邻键
数据很少且更新不频繁	线性表	实现和内存开销更小
需要最坏 $O(\log n)$	平衡搜索树	普通散列只保证期望代价

缓存还要处理容量淘汰，去重还要保存原始键以确认 “散列值相同但键不同” 的碰撞。散列值只是候选地址或摘要，不能代替关键码相等判断。

本章小结

本章介绍了一些检索的基本知识，**侧重于基于线性表的检索和基于散列表的检索**。基于线性表的检索 介绍了顺序检索和二分检索；**基于散列表的检索是本章的重点，也是难点，散列函数和冲突解决技术是 散列算法的核心**。

检索成功就是至少找到一个关键码值为 k_j 的记录，使得 $k_j = K$；**检索失败**就是找不到记录使得 $k_j = K$（可能不存在这样的记录）。平均检索长度定义为 ASL $= \sum_{i=1}^{n} P_i C_i$，其中 n 是存储结构中记录的总数，P_i 为检索第 i 个元素的概率，C_i 为找到第 i 个元素所需的 关键码值与给定值的比较次数。

基于线性表的检索是最简单的检索：数据对象存储于数组或链表等线性结构中，检索算法根据给定值 K 在线性表中进行检索，直到找到 K 在线性表中的存放位置或确定在表中找不到 K 为止。

- **顺序检索**对表的特性没有要求、数据元素可以任意排列，但平均检索长度较大，在平均和最差的情况下 的时间代价都是 $\Theta(n)$。
- 二**分检索**是针对有序表的检索，其效率可以通过二分检索的决策树进行衡量：平均检索长度与最大检索 长度相近，在平均和最差情况下的时间代价都是 $\Theta(\log_2 n)$，效率较高。二**分检索特别适用于 那种一经建立就很少改动、而又需要经常检索的线性表**。
- **分块检索**又称为索引检索，它把线性表分成若干块，在每一块中结点的存放

是任意的，但是块与块之间 必须保持关键码值递增（或者递减）的顺序；**其性能介于顺序检索和二分检索之间**。

集合是由若干个确定的、相异的对象构成的，最基本的运算是「属于」「并」「交」「差」。在集合的计算机实现 中，可以用自然数来表示有限集合，把其他基类型的集合与整数集建立一一对应的关系，原书用**位向量** 实现了一个集合类（本书在 10.2.2 节手算演示了位向量表示；可编译的 `IntSet` 则改用不含重复元素的 线性表加顺序检索，接口相同）。集合只关心「在不在」，插入已存在或删除不存在都是正常失败。

散列是本章的重点和难点，其核心思想是：由散列函数决定关键码值与散列地址（散列槽）之间的对应 关系，通过这种关系来实现组织散列表的存储并进行检索。常见的散列函数构造方法有除余法、平方取 中法、基数转换法、折叠法、ELFhash 等。

散列函数对于不相等的关键码计算出相同散列地址时就会发生**冲突**，发生冲突的两个关键码称为该散列 函数的**同义词**。解决冲突的常用方法为开散列方法和闭散列方法：开散列方法的一种简单形式是把散列 表中的每个槽定义为一个链表的表头；闭散列方法为关键码 K 生成一个散列地址序列，插入元素时如果基地址已被占用，则按照后继散列地址序列依次探查，直到找到未被占用的地址。**一个好的探查函数应该 满足两个条件：一是能够探查散列表中的所有槽，二是即使两个不同关键码的基地址或者中间的探查序列 偶然会合，最终的探查序列也能够岔开**。常用的探查序列构造方法有线性探查法、二次探查法、随机探查 法、双散列探查法等。

散列地址不同的结点争夺同一后继散列地址的现象称为「聚集」（或称为「堆积」）：基地址不同的关键码，其探查序列的某些段重叠在一起所产生的聚集称为「基本聚集」；两个不同的关键码散列到同一个基地址，如果其探查序列也相同，这样的聚集称为二级聚集。**聚集产生的原因在于散列函数和探查序列选择不当，或负载因子 $\alpha = N/M$ 过大**。线性探查法会产生聚集的问题；伪随机探查和二次探查都能消除基本 聚集的问题，但采用这两种方法仍会产生二级聚集——这是因为它们产生的探查序列只是基地址的函数，而不是原来关键码值的函数；**双散列探查法利用第二个散列函数作为常数，每次跳过常数项做线性探查，对二级聚集问题解决得比较好，但缺点是计算量要大一些**。

散列方法的效率与负载因子密切相关。当负载因子 $\alpha < 0.5$、即散列表将近半满时，大部分情况下 检索长度小于 2；当负载因子超过阈值（假设为 0.5）或者插入删除操作太多时，需要重新散列整个表—— 把记录按照访问频率从高到低的顺序插入散列表中，保证上一个周期中被频繁访问的记录更有机会接近基 位置。**开散列方法的效率最好，实际系统中使用的散列大多都是开散列**：它非常简单且易于实现，删除也 极为方便；经过精心设计的闭散列的效率比开散列稳定。闭散列删除必须留墓碑，否则会截断探测链。

习题

补充检索题（参考课程第 10 章）

1. 给定一个有序双向链表和当前指针，设计可向前或向后移动的检索算法，并求平均检索长度。
2. 说明堆排序后再对另一集合逐项二分检索时，总复杂度如何由排序成本和检索次数共同决定。
3. 比较拉链法和线性探测在装填因子变化时的成功检索代价。

正文习题

1. 写出顺序检索在最好、最坏、等概率平均时的 ASL。
2. 用半开区间手算二分查找 18 在有序表 `{1,3,7,8,12,15,18,21}` 中的过程。
3. 分块检索的块数取多少时 ASL 较均衡？为什么。
4. 说明集合的 `insert` 为什么返回 `bool` 而不是抛异常。
5. 表长 5，依次插入 1、6、11，画出线性探测的槽；删除 1 后若标空，查找 6 会怎样。
6. 比较开散列与闭散列的删除、装载因子上限和指针开销。
7. ELFHash 为什么按 `unsigned char` 读字节。

原书习题

> 以下是原书第 10 章的习题，本轮按扫描件补回题面；**参考答案见 `book/`习题与参考答案`.md` 的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。

1. 在包含 n 个关键码的线性表中进行顺序检索，若检索第 i 个关键码的概率为 p_i，分布为 $p_1 = 1/2,\ p_2 = 1/4,\ \cdots,\ p_{n-1} = 1/2^{n-1},\ p_n = 1/2^n$，求成功检索的平均检索长度。（注释：本题中的线性表按照访问概率排序——访问第一个记录的概率很高，访问第二个记录的概率则比 第一个低很多，但是还是比访问第三个记录的概率高，依次类推。**对于有些概率分布，根据频率排序线性 表能够产生很有效的检索技术**。）
2. 对数组 $\{8, 13, 17, 26, 44, 56, 88, 97\}$，分析用二分法检索 88 的每一步，写出每一步时的 `first`、`mid` 和 `last` 元素。
3. 考虑用双向链表来实现一个有序表，使得能在这个表中进行正向和反向顺序检索。若指针 p 总是指向 最后成功检索到的结点，检索可以从 p 指示的结点出发沿任一方向进行。(1) 根据这种情况编写一个 函数 `search(head, p, K)`，检索具有关键码 K 的结点，并相应地修改 p；(2) 对于双向链表，假设 有序表中元素个数为 n，试给出检索成功和检索不成功时的平均检索长度。
4. 假定查找有序表 $A[25]$ 中每一个元素的概率相等，试分别求出进行顺序、二分和分块（假定被分为 5 块，每块 5 个元素）查找每一个元素时的平均查找长度。
5. 编写程序，创建一个值域在 $[1, 20000]$ 内、长度为 10000 的数组。分别用顺

序检索法、二分检索法，在 $[1, 20000]$ 内随机生成 10000 个值域范围内的数，并相应地进行 10000 次检索。要求程序输出 如下结果：(1) 检索总次数；(2) 检索成功的次数；(3) 检索成功的百分比；(4) 每次检索进行的平均比较 次数；(5) 程序运行完毕后，画一个关于程序复杂性的分析图，采用大 O 法表示各种算法，并且用程序的 实际运行效率来验证。

6. 需要确定某长度为 n 的数组中是否存在 m 个数的和等于给定值 K。例如，如果数组为 8、4、1、6，而 $m = 2$、$K = 10$，显然 $4 + 6 = 10$；数组中的数可能被使用两次。(1) 试给出 $m = 2$ 时时间复杂度 为 $O(n^2)$ 的算法；(2) 试给出 $m = 2$ 时时间复杂度为 $O(n \log n)$ 的算法；(3) 试给出 $m = 3$ 时 时间复杂度为 $O(n^2)$ 的算法；(4) 试给出 $m = 4$ 时时间复杂度为 $O(n^2 \log n)$ 的算法。（提示：可以计算出两个数字和的所有可能值并进行排序，然后再利用前面的算法。）
7. 假设讨论的集合是全集合 $\{0, 1, 2, 3, \cdots, n\}$ 的一个子集合，且 n 是一个不大的整数。试定义出 该集合的抽象数据类型（ADT），实现集合 ADT 中的基本操作，并且采用集合 ADT 编写一个打印有穷集合 中所有元素的算法。
8. 两个数集合 S_1、S_2，大小分别为 n、m，$m = O(\log n)$，试借助于排序来求 S_1 与 S_2 的交集。
9. 根据教材中定义的集合 ADT，编写一个解决**背包问题**的算法。背包问题：一个背包可以放入的物品 重量为 S，现有 n 件物品，重量分别 $W_1, W_2, \cdots, W_n$，其利润是 $p_1, p_2, \cdots, p_n$。编写 算法判断是否可以从 n 件物品中选择 $m(m \leq n)$ 件，其重量之和不超过 T、而其总价值最大。
10. 液晶显示往往使用**七段数码管**，其中 8 字就是当所有这 7 段数码管都发光时组成，$0 \sim 9$ 这 10 个 数字中除 8 以外的其他数字都是由这 7 段中的某些段发光而形成的。现在需要找到一种排列这 10 个 数字的方法，使得相邻两个数字由一个数字加几笔或减几笔组成、**不能又加又减**。例如 2865937014 就是一个排列，试编程找出所有这些排列。（注意：每个排列的逆置都是另外一个合法的排列，需要排除 本质相同的解。提示：先将问题用数学模型来表示——用七段管来表示 $0 \sim 9$ 中的每一个数，每个数 可以用 `'a'~'g'` 的一个子集合来表示；七段编码可取 $S[0] = \{a, b, c, d, e, f\}$、$S[1] = \{b, c\}$、$S[2] = \{a, b, d, e, g\}$、$S[3] = \{a, b, c, d, g\}$、$S[4] = \{b, c, f, g\}$、$S[5] = \{a, c, d, f, g\}$、$S[6] = \{a, c, d, e, f, g\}$、$S[7] = \{a, b, c\}$、$S[8] = \{a, b, c, d, e, f, g\}$、$S[9] = \{a, b, c, d, f, g\}$。如果两个数字之间可以互相转化，那么它们相应的集合之间 一定存在包含关系，这样要处理的问题就转化为判断集合之间的包含关系。**如果把数字作为结点，能够用边连接相邻的数字，则构成一个无向图；现在要找一条路径经过所有的结点一次，这就是图论中的「哈密顿路径」问题**。）
11. 假定一个待散列存储的线性表为 $(32, 75, 29, 63, 48, 94, 25, 46, 18, 70)$，散列地址空间为 $HT[23]$（下标位置为 $0, 1, \cdots, 22$），若采用除留余数法构造散列函数和线性探查法处理冲突，试求出每一个元素的散列地址，画出最后得到的散列表，求出平均查找长度。

12. 设散列表为 $HT[19]$，对 10、100、32、45、58、126、3、29、200、400、0 建立一个散列表。
    (1) 散列函数为直接对 19 求余，给出此时的散列表和冲突次数；(2) 散列函数为将各个位置上的数字 相加后对 19 求余，给出此时的散列表和冲突次数；(3) 试设计一个不会造成冲突的散列函数；(4) 如果 散列表大小为 17，上面 3 个问题的答案又是什么?
13. 设散列表为 $HT[13]$，散列函数为 $h(K) = K \bmod 13$。用闭散列法解决冲突，对下列关键码序列 12、23、45、57、20、03、78、31、15、36 造表。(1) 采用线性探查法寻找下一个空位，画出相应的散列表，并计算等概率下成功检索的平均检索长度和不成功检索的平均检索长度；(2) 采用双散列法寻找下一个 空位，再散列函数为 $rh(K) = (7K) \bmod 11 + 1$，寻找下一个空位的公式为 $h_i = (h_{i-1} + rh(K)) \bmod 13$，$h_1 = h(K)$。画出相应的散列表，并计算等概率下成功检索的 平均检索长度。
14. 考虑下面 $1 \sim 6$ 的一个排列 2、4、6、1、3、5。(1) 在一个长度为 7 的散列表中使用这个排列实现 伪随机探查，试分析将产生什么结果；(2) 这个排列能解决基本聚集问题吗? (3) 在伪随机探查中，对于长度为 M 的散列表，为什么要选择 $1 \sim M - 1$ 的一个「随机」排列?
15. 有一个 7 个槽的散列表（槽从 0 到 6 编号），使用散列函数 $h(k) = k \bmod 7$ 和线性探查，依次插入 3、12、9、2。(1) 显示最后的结果散列表；(2) 在插入值 2 的关键码之后，每一个空槽作为下一个被填充 槽的概率是多少?
16. 给定关键码序列 $(26, 25, 20, 33, 21, 24, 45, 204, 42, 38, 29, 31)$，用散列法进行存储，规定负载 因子 $\alpha = 0.6$。(1) 试给出除余法的散列函数；(2) 用双散列探查法解决碰撞，试画出插入所有的 关键码后得到的散列表，并指出发生碰撞的次数。
17. 假定把关键码 K 散列到有 n 个槽（从 $0 \sim n - 1$ 编号）的散列表中。对于下面的每一个函数 $h(K)$，这个函数作为散列函数可以接受吗（即插入和检索操作能正常工作吗)? 如果可以，它是一个好的 散列函数吗?（注：函数 `Random(n)` 返回一个 $0 \sim n - 1$ 之间的随机整数，包含这两个数在内。）
    (1) $h(k) = k/n$，其中 k 和 n 都是整数；(2) $h(k) = 1$；(3) $h(k) = (k + \text{Random}(n)) \bmod n$；(4) $h(k) = k \bmod n$，其中 n 是一个素数。
18. 试设计一个简单、容易计算的散列函数,将 3 个字母的字符串映射为 $0 \sim n-1$ 之间的整数（包括 0 和 $n - 1$）。并给出 $n = 11, 13, 17, 19$ 时，如下字符串的散列值：PAL LAP PAM MAP PAT PET SET SAT TAT BAT。**要求冲突尽可能的少**。
19. 上机做一个小试验，测试对下列串运行 ELFhash 的结果是什么? (1) `HELLO WORLD`;
    (2) `WORLD, HELLO!`; (3) `Data Structure`; (4) `Data Structure and Algorithms`。

上机题

1. 对同一组随机键比较顺序、二分和散列的平均探测次数。

2. 实现闭散列的墓碑删除，并写测试：删掉头键后仍能查到其后的同址键。
3. 装载因子超过 0.7 时扩表再散列，观察聚集是否缓解。
4. 用 `IntSet` 的接口换上散列表实现，确认交、包含的测试不用改。

原书上机题

> 原书上机题的参考答案见 `book/习题与参考答案.md` 的「原书上机题参考」，由 R14 逐题守着。

1. 在编译系统中，采用「**符号表**」来保存源代码中的描述符信息：编译器在分析阶段收集信息放入符号表，在综合阶段使用符号表中的信息生成目标代码。这种符号表是一种**有重复关键码的字典**：字典记录以 描述符作为关键码，属性值包括描述符类型（`int`、`float` 等）、用途（函数名、变量名、标号）、描述符 内存地址、描述符作用域等；因为在不同的程序块中可以定义同名的描述符，所以符号表中必然存在多个 记录具有相同的关键码，**查找结构应该是最新插入的元素**。编写程序对读入的 C++ 程序建立这样的 符号表字典，在程序块结尾处需要删除本块定义的描述符，最终所有元素都被删除掉。试比较下面这两种 实现方案的效果：(1) 采用线性表实现符号表字典；(2) 采用散列方法实现符号表字典。
2. 实现一个文档检索系统。该系统能够按照给定的关键码检索文档，支持「与」「或」「差」等基本逻辑操作。
3. 使用**桶式散列**实现一个存储在磁盘中的数据库。定义一个长 128 B 的记录，其中 4 B 是关键码、120 B 是数据，其余 4 B 预留用来存储必要的信息以支持散列表。散列表中的一个桶是 1024 B，因此 桶内可以放 8 个记录；散列表要包含 27 个桶（总共 216 个记录的空间，槽从位置 $0 \sim 215$ 编号索引），后接一个溢出桶，在文件中的记录位置 216。散列函数是 $K(R) \bmod 197$（**这意味着表中最后 9 个槽 不作为任何记录的基位置**）。冲突解决方案是在桶内回转的线性探查——例如如果一个记录散列到第 5 个 槽，冲突解决过程会尝试以 5、6、7、0、1、2、3、4 的顺序把这个记录插入表中；如果一个桶满了，就把这个 记录放到文件最后的溢出部分。系统应当支持记录的插入、删除和检索；当进行测试时，考虑系统的设计 用于一次存储 100 个左右的记录。
4. 设计一个简单的**职工管理系统**，对单位职工进行管理，包括插入、删除、查找、排序等功能。职工对象 包括姓名、性别、出生年月、学历、职务、住址、电话等信息。系统功能要求如下：(1) 新增一名职工：将新增 职工对象按姓名以字典方式插入到职工管理文件中；(2) 删除一名职工：从职工管理文件中删除符合某些 条件的职工；(3) 查询：从职工管理系统中查询符合条件的某些职工；(4) 修改：检索某个职工对象，对某些 属性进行修改；(5) 排序：按某种需要对职工对象文件进行排序。
5. **封锁管理子系统的模拟实现**。封锁管理子系统通过加锁来控制用户对系统资源的并发使用：用户使用 系统资源前必须申请封锁，即给出申请封锁的对象资源号、封锁方式、用户名，其中资源号是取值为 $1 \sim 999999$ 的正整数，封锁方式为 S（共享）或 X（排他）。相容规则是：已持有 S 时，新的 S 可以批

准、新的 X 须等待；已持有 X 时，新的 S 和新的 X 都须等待（原书图 10.14）。子系统受理封锁请求，根据所保存的封锁 状态信息决定该请求是否能够获得封锁，进行相应处理，并向用户反馈处理结果：如果获得封锁，则赋给 该请求一个批准号，可以使用该资源；否则需要进入等待队列（赋给该请求一个等待号）。用户结束对某 资源的使用后，应释放封锁（给出封锁对象的资源号、封锁批准号）。

系统中的资源对象很多，**仅需对当前有封锁要求的对象保存封锁状况信息**；处理加锁、解锁请求时必须 能迅速找到有关对象的封锁状况信息，为满足此要求，可采用**散列表加链表**的数据结构：散列表的元素 对应为封锁对象，以对象的资源号为散列函数的自变量；散列表中元素仅为一个指向封锁对象链表的指针，LO 为封锁对象结点，对应于同一散列地址的封锁对象链接到一个链表中；LR 为封锁请求结点，每个封锁 对象结点带两个封锁请求队列——活动队列中为当前持有对该对象的封锁请求，等待队列中为正在等待对 该对象进行封锁的封锁请求。LO 结点和 LR 结点均向子系统自己管理的可利用空间表申请（可以参考 第 12.2 节关于可利用空间表的讨论）。

试设计并实现散列表、LO 链表、LR 活动队列、LR 等待队列、可利用空间表的结构和基本运算。在此基础上 使封锁管理子系统能提供以下功能：(1) 受理用户从终端输入的封锁请求进行相应处理，并给出反馈信息；

(2) 受理用户从终端输入的解锁请求，并进行相应处理；(3) 显示封锁状况信息（用户从终端输入资源号，子系统显示该对象的 LR 活动队列和 LR 等待队列）。

第 11 章 索引技术

索引不是保存完整记录的主文件，而是一张「关键码到记录位置」的查找表。它的目标是在海量外存记录中避免逐条扫描：先查较小的索引，再按位置读取目标记录。

原书本章没有独立的【算法】/【代码】清单——`dsa_raw.md` 里 105 条清单，第 11 章一条都没有。本章的实现因此全部是新写的，不认领任何原书清单，台账等式不受影响。五个小节都有通过闸门的实现，共四个实现单元（11.2 与 11.4 共用 `bplus_tree`）：

小节	实现状态	代码
11.1 线性索引	实现并测试	`code/ch11/linear_index`
11.2 静态多分树	实现并测试	`code/ch11/bplus_tree` 的 `bulk_load`
11.3 倒排索引	实现并测试	`code/ch11/inverted_index`
11.4 B 树与 B+ 树	实现并测试	`code/ch11/bplus_tree`
11.5 位图、签名	实现并测试	`code/ch11/bitmap_index`
11.5 中的红黑树（原书 11.6）	概念导读	只作对照讨论，不另写实现

这些都是**内存里的页模拟**：结点即页，`page_reads()` / `page_writes()` 数的是页访问次数。它们不做真实磁盘 I/O、页缓存和并发控制——本章要教的是访外次数怎么被结构决定，不是写一个存储引擎。

11.1 线性索引

第 1 章和第 9 章已经说过：外存上逐条扫描主文件太贵，应该先查一张较小的表，再按位置读记录。线性索引就是这张表：项是 `(key, location)`，按 key 排序，可以在内存里二分。

稠密索引为每条记录建一项，主文件可以无序，查到索引项就直接得到记录位置。稀疏索引只为每个有序数据块建一项，通常记下该块的最小 key；查到块之后还要在块内再找一次。稀疏索引更省空间，但要求主文件按 key 分块有序。

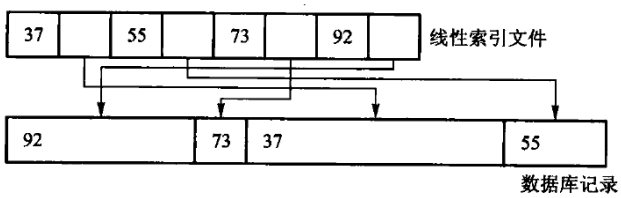


图 11.1 中记录长度不同，不能用 “基地址 + 下标 × 固定长度” 直接计算位置。索引项保存关键码和记录地址，把逻辑顺序与物理布局分开；主文件移动后要更新地址，索引本身不保存完整记录。

第一层索引太大、内存放不下时，再为它建一层，成为多级索引。一次查询的路径是：内存里的顶层 → 读一个索引页 → 读数据页。这时的关键指标不是 CPU 比较次数，而是磁盘页访问次数。变长记录尤其适合「索引 + 位置」，因为主文件无法按下标直接算出地址。

先跑一遍：

```
#include "modern.hpp"

#include <cstdio>

int main() {
    using dsa::index::IndexKind;
    using dsa::index::MultiLevelIndex;
    std::vector<std::pair<int, std::string>> records;
    for (int i = 0; i < 1000; ++i) {
        records.emplace_back(i * 10, std::string(static_cast<std::size_t>(i % 7) +
↪  1, 'x'));
    }

    // 每个数据页 4 条记录，每个索引页 4 项。
    MultiLevelIndex sparse(IndexKind::Sparse, 4, 4);
    sparse.load(records);
    sparse.reset_counters();
    const bool hit = sparse.find(5000).has_value();
    std::printf(" 稀疏 ·每页 4 项 : %zu 个数据页，%zu 个索引项，%zu 层，查一次读 %zu 页
    ↪  (命中 %d) \n",
                sparse.data_pages(), sparse.entries(), sparse.levels(),
↪  sparse.page_reads(),
                static_cast<int>(hit));

    // 索引页装得多，层数就少，访外次数随之下降——这就是 11.2 要做多分树的理由。
    MultiLevelIndex flat(IndexKind::Sparse, 4, 64);
    flat.load(records);
    flat.reset_counters();
    (void)flat.find(5000);
    std::printf(" 稀疏 ·每页 64 项: %zu 层，查一次读 %zu 页\n", flat.levels(),
    ↪  flat.page_reads());

    MultiLevelIndex dense(IndexKind::Dense, 4, 64);
    dense.load(records);
    dense.reset_counters();
    (void)dense.find(5005);  // 不存在
    const std::size_t miss_reads = dense.page_reads();
    dense.reset_counters();
    (void)dense.find(5000);  // 存在
    std::printf(" 稠密 · %zu 层      : 查不到读 %zu 页，命中读 %zu 页——"
```

```
                " 多出来的那一页就是数据页，索引里没有就不必去读\n",
                dense.levels(), miss_reads, dense.page_reads());
    return 0;
}
```

```
稀疏 ·每页 4 项 : 250 个数据页，250 个索引项，4 层，查一次读 4 页（命中 1）
稀疏 ·每页 64 项: 2 层，查一次读 2 页
稠密 · 2 层       : 查不到读 1 页，命中读 2 页——多出来的那一页就是数据页，索引里没有就不必
↪ 去读
```

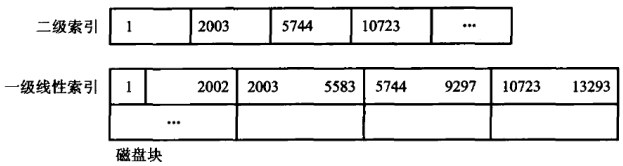
三行输出对应三个结论。第一行：索引项一页放不下就逐层上建，查一次的代价等于层数。第二行：**索引页装得多，层数就少，访外次数随之下降**——这正是下一节要做多分树的理由。第三行是稠密与稀疏最实用的差别：稠密索引在索引里查不到就是真没有，一个数据页都不用读；稀疏索引只能定位到页，**查不到也得先把那一页读上来**才知道。

从 1000 条记录算出四层索引

示例中每个数据页放 4 条记录，因此主文件有 $\lceil 1000/4 \rceil = 250$ 个数据页。稀疏索引每页一项，第一层恰有 250 项；索引页也只放 4 项时，自底向上各层页数为：

层	项数	占用页数	上一层要为它建多少项
数据页	1000 条记录	250	250
一级索引	250	63	63
二级索引	63	16	16
三级索引	16	4	4
顶层索引	4	1	常驻内存

一次命中从顶层定位后，要读三级、二级、一级索引中的各一页，再读数据页，共 4 页。把每个索引页容量从 4 提高到 64 后，250 项只占 4 页，再用 1 个顶层页索引它们；查询因此只读 1 个下层索引页和 1 个数据页。



页内用顺序查还是二分查，只改变 CPU 比较次数，不改变“这一页已经读进来”这个事实。优化存储索引时先降树高和随机 I/O，再讨论页内比较，顺序不能倒过来。

查找过程写出来就是「逐层读索引页，最后读一个数据页」：

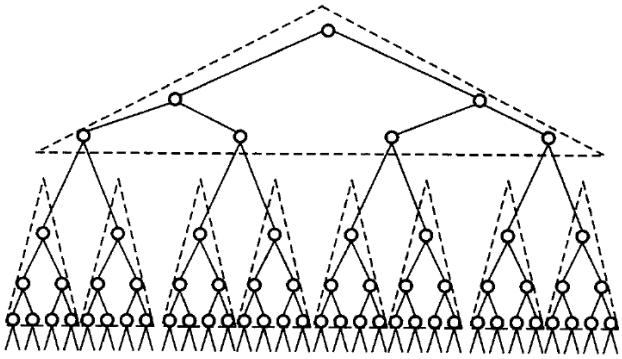
```
[[nodiscard]] std::optional<std::string> find(int key) const {
    if (levels_.empty()) {
        return std::nullopt;
    }
    // 顶层常驻内存：定位到它所在的那一页，不计页访问。
    std::size_t page = locate(levels_.back(), 0, levels_.back().size(), key);
    for (std::size_t level = levels_.size() - 1; level > 0; --level) {
        page = levels_[level][page].target;
        ++reads_;  // 读一个下层索引页
        const std::size_t first = page * entries_per_page_;
        const std::size_t last = std::min(first + entries_per_page_, levels_[level -
        ↪  1].size());
        if (first >= last) {
            return std::nullopt;
        }
        page = locate(levels_[level - 1], first, last, key);
    }

    const Entry& entry = levels_[0][page];
    if (kind_ == IndexKind::Dense) {
        // 稠密索引：索引里没有就是真没有，数据页一次都不用读。
        if (entry.key != key) {
            return std::nullopt;
        }
        ++reads_;
        return records_[entry.target].second;
    }
    // 稀疏索引：只能定位到页，页内还要再找一次；不命中也已经付出了这一页。
    ++reads_;
    const std::size_t first = entry.target * records_per_page_;
    const std::size_t last = std::min(first + records_per_page_, records_.size());
    for (std::size_t i = first; i < last; ++i) {
        if (records_[i].first == key) {
            return records_[i].second;
        }
    }
    return std::nullopt;
}
```

11.2 静态索引——多分树

磁盘按页读写，一个索引结点通常设计成恰好装满一页。二叉树每个结点只有两个孩子，同样多的 key 会把树撑得很高，查询就要读很多页。多分树每个结点有许多孩子，树高低，页访问少。

静态多分树在批量装入时按页填满，之后结构不再变。查找从根走到叶，每层读一页。插入、删除会破坏「一页正好满」的约定：多出来的记录只能进溢出区，少了的页会变稀，最终往往要重组整棵树。所以它适合「一次建好、很少改」的文件，不适合频繁更新的目录。



多分树降低的是高度。若每个结点最多有 m 个孩子、树高为 h，理想满树可导航约 m^h 个叶页；同样覆盖一百万个叶页，二叉树约需 20 层，100 路树只需 3 层。页内要比较更多分界 key，但整页已经在内存里，代价通常远小于多读一次外存页。

扇出不是随意选的。页大小 4096 字节，若每个 “关键码 + 孩子页号” 占 16 字节，扣除页头后大约能放 250 项，扇出约 251；关键码变长或页头元数据增加，扇出就会下降。数据库索引喜欢短关键码，不只为省总空间，也为让一页容纳更多分支、压低树高。

批量装入本身就是「按页填满、自底向上建层」，`code/ch11/bplus_tree` 的 `bulk_load` 实现的正是这件事——把 11.4 节那棵 3 阶 B+ 树按这个办法装出来，得到的就是 11.4 图里的形状。区别只在后续：静态多分树靠重组，B+ 树靠分裂与合并就地维护。

批量装入前必须按 key 排好记录，然后连续填叶页，再用每个孩子的最小 key 建上一层。这样叶页天然相邻，范围扫描接近顺序 I/O。若一条条随机插入再指望得到相同布局，结点会因分裂留下空隙，树形和页利用率都不同。

11.3 倒排索引

在实际应用中，不仅需要按关键码值检索记录并进而找到记录中的其他数据项值，而且经常需要**按照 记录中的其他数据项（即属性值）来检索记录**，这样就需要按属性值建立索引。这种索引表中的每一项 都包括一个属性值和具有该属性值的各记录地址。**由于这种索引不是由记录来确定属性值，而是由属性 值来确定记录的位置，因而称为倒排索引**（inverted index）；带有倒排索引的文件称为倒排索引文件，简称**倒排文件**（inverted file）。

11.3.1 基于属性的倒排

以原书表 11.1 那张教师登记表为例：每位教员的数据是表中的一个记录，以职工号 EMP# 为主关键码，另外有姓名、系、职称、专长、地址等属性。关键码可以唯一标识一个教员，而其他 属性则不能——学校里可能有同名教员，计算机系有几十名教员也就是有几十个记录的「系」域值是「计算机」。对这张表可能提出的检索是「列出职称为讲师的所有职工号」「列出计算机系擅长英语的教员 名单」。**用前面讨论过的检索技术也可以处理，但是代价比较大**：处理前一个检索需要顺序扫描所有记录，处理后一个还要对每个记录先检查「系」域、再看「专长」域，效率很低。

为有效地处理基于属性的检索，考虑使用特殊的存储形式：倒排表（inverted list）。倒排表是基于属性 的倒排——在保留原表的同时，对于感兴趣的属性（即可以用来作为检索参数的属性）的每个值都建立一个 称做倒排表的线性表，并存放与此属性值相对应的所有关键码值。

11.3.2 对正文文件的倒排

正文索引（text indexing）处理的是「建立一个数据结构以提供对文本内容的快速 检索」，有两种方法：**词索引**（word index）把正文看做由符号和词组成的集合，从正文中抽取出关键词，然后用这些关键词组成适合快速检索的数据结构——它适用于可以很容易解析成一组词的文本（例如英文），**但不适合生物学数据和一些东方语言书写的文本**，而且查询词被限制在词索引的关键词范围内；**全文索引**（full-text index）把正文看做一个长字符串，在数据结构中记录子字符串的开始位置，这样 查询就可以针对正文中的任何子字符串、不再限于关键词，代价是**通常需要更大的空间**。

倒排文件是使用得最广泛的词索引。一个倒排文件就是一个排序关键词的列表，其中每个关键词指向 一个**倒排表**（posting list），指向该关键词出现的文档集合以及在该文档中的位置。对于词 t_i，它的 posting list 可表示为 $\{(d_{i1}, f_{i1}, a_{i1}), (d_{i2}, f_{i2}, a_{i2}), \cdots\}$，其中 d 为文档标号、f 为该词在文档中的出现次数（依此对 posting list 中出现的文档进行排序）、a 为出现信息（位置、权重等）的罗列。**为节省存储空间、提高查询响应时间，可以把频率信息去掉，甚至还可以把位置信息 去掉，而只保留该关键词所出现的文档编号**——本节实现的就是保留位置信息的那一档，因为短语查询 需要它。

倒排的思路是：从属性值走到记录集合。例如

```
计算机系 -> [0310, 0330, 0341]
英语专长 -> [0310, 0421]
```

「计算机系且擅长英语」就是两个有序列表的交集，结果是 [0310]。或查询是并集，差查询是差集。全文检索用同一结构：词项映射到「哪些文档、出现在哪些位置」。短语查询还要用位置是否相邻来过滤。

倒排把查询变成集合运算，速度很快；代价是每次插入、删除、修改记录，都

要同步维护所有相关列表。倒排表通常只存标识和位置，完整记录仍在主文件里。

先跑一遍：

```
#include "modern.hpp"

#include <cstdio>

int main() {
    dsa::index::InvertedIndex index;
    index.add_document(310, {" 计算机系", " 英语专长"});
    index.add_document(330, {" 计算机系"});
    index.add_document(341, {" 计算机系"});
    index.add_document(421, {" 英语专长"});

    const auto show = [](const char* label, const std::vector<int>& docs) {
        std::printf("%s:", label);
        for (const int doc : docs) {
            std::printf(" %04d", doc);
        }
        std::printf("\n");
    };
    show(" 计算机系           ", index.postings(" 计算机系"));
    show(" 英语专长           ", index.postings(" 英语专长"));
    show(" 计算机系且擅长英语", index.and_query({" 计算机系", " 英语专长"}));
    show(" 计算机系或擅长英语", index.or_query({" 计算机系", " 英语专长"}));
    show(" 不擅长英语         ", index.not_query(" 英语专长"));

    dsa::index::InvertedIndex text;
    text.add_document(1, {"the", "quick", "brown", "fox"});
    text.add_document(2, {"the", "brown", "quick", "fox"});
    show(" 含 quick 与 brown ", text.and_query({"quick", "brown"}));
    show(" 短语 quick brown  ", text.phrase_query({"quick", "brown"}));
    return 0;
}
```

```
计算机系           : 0310 0330 0341
英语专长           : 0310 0421
计算机系且擅长英语: 0310
计算机系或擅长英语: 0310 0330 0341 0421
不擅长英语         : 0330 0341
含 quick 与 brown : 0001 0002
短语 quick brown  : 0001
```

最后两行是短语查询存在的理由：2 号文档两个词都有，但不相邻，只有位置信息能把它排除掉。

两个指针怎样求交

对倒排表 `A=[310,330,341]` 与 `B=[310,421]` 求交：

步	A[i]	B[j]	动作	输出
1	310	310	相等，两边都前进	310
2	330	421	左边小，只移动 i	310
3	341	421	左边小，只移动 i	310
4	耗尽	421	停止	310

每次至少有一个指针前进，所以总步数不超过两张表长度之和。不能对 A 的每个文档号都从头扫描 B，那会把 $O(n+m)$ 退化成 $O(nm)$。

多词 AND 查询还要决定合并顺序。先交最短的倒排表，往往能尽早把候选集缩小；例如三张表长度分别为 10、1000、100000，先合并后两张会制造一个很大的中间结果。OR 查询则要在归并时去重，NOT 查询必须相对于“全部文档集合”求差，单独一张倒排表无法知道哪些文档不存在该词。

短语查询的倒排项不能只存文档号，还要存词在文档中的位置。查询 `quick brown` 时，文档 1 中位置可以是 `(1,2)`，相差 1，保留；文档 2 中若为 `(2,1)`，两个词都出现却次序相反，应排除。先按文档号求交，再在候选文档内检查位置，可避免读取无关位置表。

倒排表按文档号升序，所以求交就是一次归并，两个指针各走一趟，代价 $O(n+m)$：

```cpp
/// 有序表求交。归并一遍，两个指针各走一趟，代价 O(n+m)。
[[nodiscard]] static std::vector<int> intersect(const std::vector<int>& left,
                                                const std::vector<int>& right) {
    std::vector<int> out;
    std::size_t i = 0;
    std::size_t j = 0;
    while (i < left.size() && j < right.size()) {
        if (left[i] < right[j]) {
            ++i;
        } else if (right[j] < left[i]) {
            ++j;
        } else {
            out.push_back(left[i]);
            ++i;
            ++j;
        }
    }
    return out;
}
```

这段归并是本节的核心，所以要自己写；换成一行库调用，这一节就没有内容了。

11.4 动态索引

动态索引结构在文件创建、初始装入记录时生成，在系统运行过程中插入或删除记录时，索引结构本身 也可能发生改变，以保持较好的检索性能。这与 11.2 节的静态索引形成对照：静态索引一旦建立，插入删除就只能靠溢出区兜着。

先说清楚为什么外存索引必须用多分树。二叉搜索树 BST 不能保证平衡，有可能退化为线性表；**外存索引结构的平衡性能非常重要，因为退化的树结构将导致树形索引的失败，而面临巨大的访外开销**。由 11.2 节多分树的讨论可以看出，外存索引树结点在不超过磁盘页块大小限制的前提下，应该尽量增加 每个结点内关键码的数目。第 12 章将要介绍的 AVL 树使得 BST 始终保持 $\Theta(\log n)$ 级的平衡状态，但**其频繁的插入删除调整对于外存树索引来说也需要很大的 I/O 访问量**。

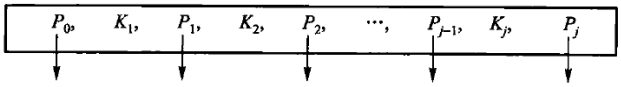
于是问题变成：怎样保持多分树高度上的平衡，不让它退化为线性？怎样使得多分树结点中关键码尽可能 多，而插入删除操作也不需要太多调整树结构的动作？1970 **年** R. Bayer **和** E. McCreight **提出了 B 树**（balanced tree，国内也有人称「B-树」，其中的「-」是英文的连字符，**不要说成**「B **减树**」），它是一种 平衡的多分树，适用于组织动态的索引结构。

11.4.1 B 树

m **阶 B 树的定义**（五条，缺一不可）：① 每个结点至多有 m 个子结点；② 除根结点和叶结点外，其他每个结点至少有 $\lceil m/2 \rceil$ 个子结点；③ 根结点至少有两个子结点（例外情况：根可以为空，或者独根）；④ **所有的叶结点在同一层**，可以有 $\lceil m/2 \rceil - 1$ 到 $m-1$ 个关键码；⑤ 有 k 个子结点的分支结点恰好包含 $k-1$ 个关键码。最底层的叶结点的指针指向空（对应于失败 检索），**空指针数目正好等于树中所包含的关键码数目加** 1。

B **树的四条性质**（原书 11.4.1 末）：① B 树总是树高平衡的，所有叶结点都在同一层；② 更新和检索 操作只影响一些磁盘块，因此性能很好；③ B **树把相关的记录（即关键码具有接近的值）放在同一个磁盘 块中，从而利用了访问局部性原理**；④ B 树保证树中至少有一定比例的结点是满的，这样既能保证一定 的空间利用率，又使得插入和删除不必像静态多分树那样频繁地修改结点，从而总体上减少了磁盘读取数。**阶为 3 的 B 树就是通常说的**「2-3 **树**」。

B 树是保持平衡的多路搜索树，专门为磁盘页设计。一个结点里有多个有序 key，key 之间是指向孩子的指针。查找从根开始，在页内确定下一条分支，再读孩子页。树高随阶数增大而降低，一次查找的页数大约是 $\log_m n$。



一个含 r 个关键码的内部结点有 $r+1$ 个孩子：最左孩子的键都小于第一个关键码，相邻两个关键码之间各管一个区间，最右孩子的键都大于最后一个关键码。页内先定位区间，再沿对应页号下降；若孩子数与关键码数不满足 “多一个”，结点结构已经损坏。

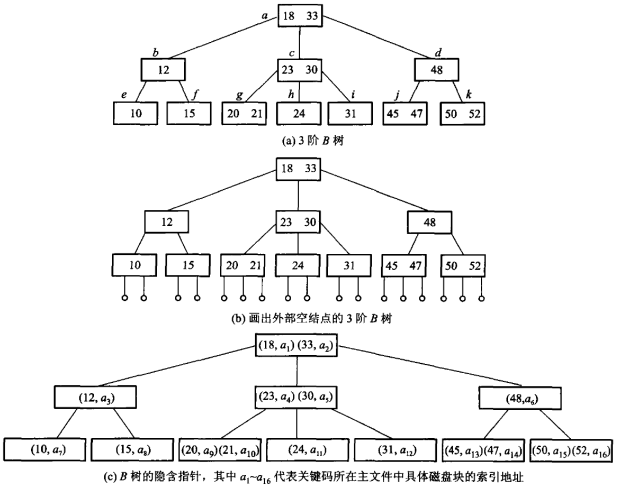


图 11.5　同一棵 3 阶 B 树的三种画法：(a) 通常的画法；(b) 把外部空结点画出来；(c) 画出隐含指针——$a_1 \sim a_{16}$ 是关键码在主文件里对应磁盘块的索引地址。B **树的关键码在内部结点上也带着数据地址**，这正是它与后面 B+ 树的分水岭。

检索要算访外次数。设 B 树的高度为 h（独根树高为 1），自顶向下检索到叶结点最多要读 h 次索引盘，再加上按关键码的隐含指针读 1 次主文件的数据盘，即最多 $h+1$ 次访外。以原书图 11.5 为例：检索关键码 31 需要 3 次访问 索引树结点（a、c、i），外加 1 次读关键码 31 所在主文件的数据盘 a_{12}，**共 4 次访外**；而检索关键码 12 只需要 2 次访问索引树结点（a、b），外加 1 次读数据盘 a_3，**共 3 次访外**。

插入的完整规则。在 B 树中插入一个关键码的方法也很简单，**插入可能导致 B 树朝着根的方向 生长**：首先检索此关键码是否在 B 树中，若在则拒绝插入；若不在，则最后找到的叶结点即为待插入位置。如果插入后该叶结点中关键码个数小于 m，则插入完成；否则关键码个数等于 m，**产生上溢出**—— 这种情况下，取这 m 个关键码的中位数为分界码，叶结点分裂为分别具有 $\lceil (m-1)/2 \rceil$ 和 $\lfloor (m-1)/2 \rfloor$ 个关键码的两个结点，分界码插入到父结点中；如果父结点没有发生上溢出，插入 操作结束，**否则该分裂过程继续往根的方向传递，有可能一直分裂到根**。

插入的访外次数可以算出来：找插入结点要向下读盘 h 次；每分裂一个非根结点要向磁盘写出 2 个结点，分裂根结点（最后一次）要写出 3 个结点，因此最坏情况下

$$\text{插入操作最多访外次数} = h + 2(h-1) + 3 = 3h + 1$$

按这个公式, 原书图 11.7 那次根也产生分裂的插入, 访外为 $3 \times 3 + 1 = 10$ 次。

根结点为什么可以只有 2 个子结点?根结点允许的子结点数目范围在 $2 \sim m$ 之间，而内部结点的 子结点数目在 $\lceil m/2 \rceil \sim m$ 之间——**这是因为插入算法分裂到根结点时会分为 2 个子结点并 产生一个新根**；如果要求与其他内部结点具有同样的出度，就需要对 B 树进行比较大的调整，这就违背了 B 树操作的局部调整原则。而且实际运行时，**根结点往往在内存中保存**（当然最终它还是要写回外存，因此也有 m 的上限），不需要遵守内部结点的子结点数目下限约束；大部分内部结点在外存中, 需要保证 存储块至少半满以折衷存储效率和操作效率, 因此限制在 $\lceil m/2 \rceil \sim m$。**根结点也不可能只有 一个子结点，那样太浪费存储空间，而且会相应地增加检索时间**。

为什么插入时分裂而不是「送给邻居一些关键码」? 这是由于插入操作可能发生在创建阶段，分裂的结点 很快就会增加新的关键码；**而送关键码涉及读和修改邻居结点，访外开销大，且操作麻烦**。

插入使结点超过上限（溢出）时，把它分裂成两个，并把分界 key 上推到父结点；父结点也可能接着分裂，最坏会一直裂到根，树增高一层。

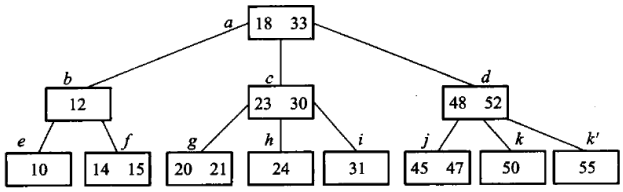


图 11.6　在图 11.5 的 3 阶 B 树里插入 14、55。两次插入都能就地放下，树高不变——**大多数插入就是这样结束的**。

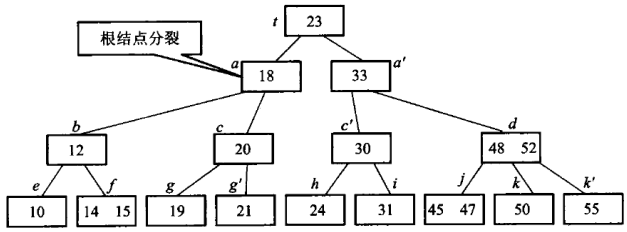


图 11.7　再插入 19 就不行了：叶溢出、分裂、分界 key 上推，父结点跟着溢出，一路裂到根，树**增高一层**。B 树只在这一种情况下长高，所以它的所有叶永远在同一层。

删除的完整规则。删除的过程与插入类似, 但比插入稍微复杂些。**如果删除的关键码不在叶结点层, 则需要先把此关键码与其在 B 树中的后继对换位置, 然后再删除该关键码**——在 B 树中，一个关键码的 后继就是比它大的那些关键码中值最小的那一个, 非叶层待删关键码的后继关键码一定处于叶层, 即待删 关键码

的右指针所指子树中最左结点的最小关键码。如果删除的关键码在叶结点层，则直接从叶结点中删除,**这可能导致此结点所包含的关键码的个数小于 $\lceil m/2 \rceil - 1$，产生下溢出**。

这种情况下，考察该结点的直接左兄弟或右兄弟（**习惯上先左后右**），从兄弟结点借若干个关键码到该 结点中。实际上，**往往是被删除结点和被借结点的关键码、还有父结点中的分界码一起排列，取中位数为 新分界码**，尽量使得两个新兄弟结点所含关键码个数基本相同（这样可以延迟下一次删除引起的下溢出）。

当在兄弟结点的关键码个数也很少、刚好等于 $\lceil m/2 \rceil - 1$ 时，不能借关键码。这种情况下，**要把删除关键码的结点、它的左/右兄弟（先左后右）结点及父结点中的分界关键码合并为一个结点**；合并的情况导致父结点中减少了一个关键码，这样有可能导致进一步的合并。**合并如果一直传递到根结点，将导致根结点与两个子结点进行合并，形成新的根结点，从而使整个树减少了一层**。

删除过程中，一般情况下出现下溢出时先看是否能借关键码，借不了再考虑合并。**如果阶比较小**，例如 阶为 3（关键码数在 1～2 个）、阶为 4（关键码数在 1～3 个），此时删除可能会出现空结点的情况，**可以先考虑是否能合并，再考虑借关键码，这样有可能降低树的高度**；实际应用中 B 树的阶很大，不会 出现删除一个关键码使得结点变空的情况。

删除使结点低于下限（下溢）时，先向左右兄弟借 key；借不到就把两个结点合并，分界 key 从父结点拿下来。借和合并都可能向上传播。

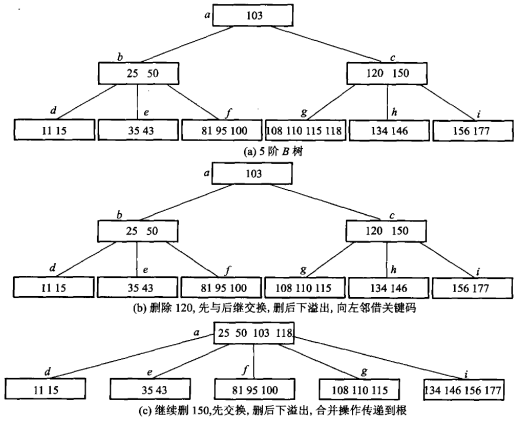


图 11.8　在一棵 (a) 5 阶 B 树上连续删除：(b) 删 120——先与中序后继交换再删，删后下溢，向左邻**借**一个关键码；(c) 接着删 150——同样先与后继 156 交换，删后叶 i 只剩 177 而下溢；i 是 c 的最右孩子，只有左兄弟 h，而 h 只有 2 个关键码、恰在下限，借不到，只能把 h、156、i **合并**；合并又让父结点 c 下溢，b、103、c 再合并成新根，树降低一层。

按原书规则复算，(b) 中结点 c 应为 `118 150`：g、分界码 134、h 排成 (108,110,115,118,134,146)，取第二个中位数 118 上移替换父结点里的 134。扫描件 (b) 的 c 仍印作 `120 150`，是原图笔误——120 已被删除，不可能留在树里。

这些局部调整保证所有叶在同一层，所以 B 树始终平衡。

11.4.2 B+ 树

B+ 树是 B 树的一种变形。实际应用中产生了一些 B 树的变形：每个结点的「充满度」可以不同；树在不同层上可有不同的阶；每个关键码的长度可变；**把关键码全部顺序存储在叶结点上，而索引部分是每个结点内最大（或最小）关键码的复写**——最后这一种就是应用得很广泛的 B+ 树。

m 阶 B+ 树的结构定义：① 每个结点至多有 m 个子结点；② 每个结点（除根外）至少有 $\lceil m/2 \rceil$ 个子结点；③ 根结点至少有两个子结点；④ **所有的叶结点在同一层，可以有 $\lceil m/2 \rceil$ 到 m 个关键码，叶结点包含全部关键码以及相应的记录信息**（例如该记录在主文件中 的位置指针），**叶结点之间可以用双链表顺序链接**；⑤ 有 k 个子结点的分支结点**必有 k 个关键码**（注意与 B 树的 $k-1$ 个不同）。

B+ 树把记录（或记录位置）全部放在叶上，内部结点只作路标，不存完整记录。叶按 key 从小到大串成一条链。点查询仍然从根走到叶；范围查询找到下限所在的叶之后，沿叶链顺序扫到上限即可，不必再回到内部结点。关系数据库的磁盘索引多用 B+ 树，正是因为范围扫描是常见操作。

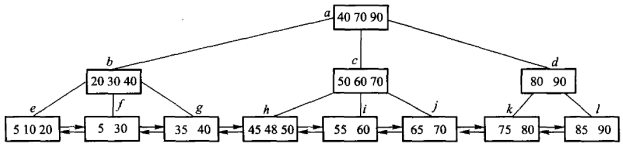


图 11.9　原书的 3 阶 B+ 树，采用「最大关键码复写」：上层每个关键码是对应子结点的最大关键码。扫描件中叶结点 f 印作 `5 30`，应为 `25 30`（父结点 b 的分界码是 20、30，f 里的键必须大于 20；图 11.10、11.11 中同一叶也都是 `25 30`）。

B 树命中内部关键码时就可能拿到记录；B+ 树内部关键码只是叶键的复写，点查询仍要走到叶。看似多走一步，换来的是内部页只放短 key 和孩子指针，扇出更大、树更矮，而且所有记录都在同一叶层，范围访问路径统一。

一棵 3 阶 B+ 树可以这样看（内部结点只导航，数据全在最下一层，叶之间有横链）：

```
            [  30  |  70  ]
           /       |       \
   [10|20]      [30|50]     [70|90]
     ↔            ↔           ↔
    记录…        记录…       记录…
```

本章的约定：3 阶指一个结点最多 3 个孩子，因此叶和内部结点都最多 2 个 key；内部结点的分界 key 取右子树最小 key 的复写。原书 11.4.2 用的是另一套约定——最大关键码复写，并把 m 阶 B+ 叶的容量定为 $\lceil m/2 \rceil$ 到 m 个 key（3 阶叶因此存 2 到 3 个 key）。两套约定都成立，对照原书读时注意这处差别。

查找 50：根上 $30 < 50 < 70$，走中间孩子；叶上直接取到 50。查找 35 到

80：先落到含 35 的叶，再沿 ↔ 扫过 50、70，直到超过 80。

插入 60：中间叶暂成 `[30|50|60]`，超过 2 个 key 的上限。按原书 11.4.1 的分裂规则取中位数 50 作分界，叶裂成 `[30]` 和 `[50|60]`，50 复写一份上推（B+ 的分界 key 在叶上仍保留）。上推之后根成了 `[30|50|70]`、要挂 4 个孩子，同样超限，于是根也分裂：中位数 50 上移，成为新根，树增高一层。

阶段	发生溢出的页	局部结果	向父层传什么
插入前	-	叶 `[30,50]`	-
插入 60	叶页	`[30]` 与 `[50,60]`	复写 50
父层接收 50	原根	key 变为 `[30,50,70]`，4 个孩子	上移 50
建新根	根	新根 `[50]`，树高加 1	结束

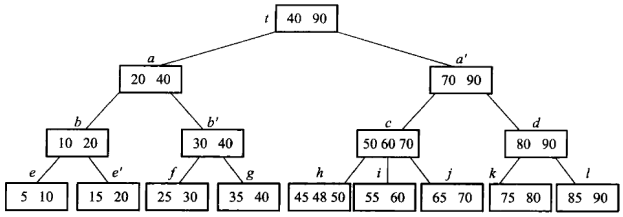


图 11.10　原书的例子：在图 11.9 的 3 阶 B+ 树中插入 15。沿 a、b、e 找到叶 e，(5,10,15,20) 上溢，分裂为 e(5,10) 与 e'(15,20)，左结点的新最大码 10 复写到 b；b 变成 (10,20,30,40) 再分裂为 b 与 b'，20 复写到 a；a 变成 (20,40,70,90) 又分裂为 a 与 a'，两者的最大码 40、90 组成新根 t，树增高一层。共读盘 3 次 (a, b, e)、写盘 7 次 (e, e', b, b', a, a', t)。原书采用的是「最大关键码复写」，所以与上面「最小关键码复写」的推演形状不同，道理相同。

```
                [  50  ]
               /        \
        [  30  ]          [  70  ]
       /        \        /        \
  [10|20]     [30]     [50|60]    [70|90]
    ↔          ↔         ↔          ↔
```

叶分裂时分界 key 是**复写**（叶上仍留一份），内部结点分裂时分界 key 是**上移**（原结点不再保留）——这是 B+ 树与 B 树最容易混的一处。两条规则并排写出来就是：

```
void split_leaf(Node* node, Split& split) {
    const std::size_t mid = node->keys.size() / 2;
    auto right = std::make_unique<Node>(true);
    right->keys.assign(node->keys.begin() + static_cast<std::ptrdiff_t>(mid),
↪   node->keys.end());
```

```
    right->values.assign(std::make_move_iterator(node->values.begin() +
↪   static_cast<std::ptrdiff_t>(mid)),
                         std::make_move_iterator(node->values.end()));
    node->keys.resize(mid);
    node->values.resize(mid);
    right->next = node->next;
    node->next = right.get();
    // 叶分裂：分界码是右叶最小关键码，复写上推，叶上仍保留。
    split.happened = true;
    split.separator = right->keys.front();
    split.right = std::move(right);
    writes_ += 2;
}

void split_internal(Node* node, Split& split) {
    const std::size_t mid = node->keys.size() / 2;
    auto right = std::make_unique<Node>(false);
    // 内部结点分裂：中位数上移，原结点不再保留它。
    split.separator = node->keys[mid];
    right->keys.assign(node->keys.begin() + static_cast<std::ptrdiff_t>(mid) + 1,
↪   node->keys.end());
    right->children.assign(
        std::make_move_iterator(node->children.begin() +
        ↪  static_cast<std::ptrdiff_t>(mid) + 1),
        std::make_move_iterator(node->children.end()));
    node->keys.resize(mid);
    node->children.resize(mid + 1);
    split.happened = true;
    split.right = std::move(right);
    writes_ += 2;
}
```

先跑一遍。上面那两张图不是画出来的，是这段程序打印出来的：

```
#include "modern.hpp"

#include <cstdio>

int main() {
    using dsa::index::BPlusTree;
    std::vector<std::pair<int, std::string>> rows;
    for (const int key : {10, 20, 30, 50, 70, 90}) {
        rows.emplace_back(key, " 记录" + std::to_string(key));
    }
    // 11.2：批量装入，按页填满。
    BPlusTree tree = BPlusTree::bulk_load(3, rows, 2);
```

```
    std::printf(" 装入后   : %s\n", tree.to_string().c_str());

    // 11.4: 插入 60，叶裂 → 根裂 → 树增高一层。
    tree.insert(60, " 记录 60");
    std::printf(" 插入 60  : %s (树高 %zu) \n", tree.to_string().c_str(),
    ↪  tree.height());
    tree.insert(65, " 记录 65");
    std::printf(" 插入 65  : %s (父结点没满，没裂到根) \n", tree.to_string().c_str());

    tree.reset_counters();
    // 先取结果再取计数: printf 的实参求值顺序没有保证。
    const auto found = tree.find(50);
    const std::size_t point_reads = tree.page_reads();
    std::printf(" 查找 50  : %s，读了 %zu 页\n", found->c_str(), point_reads);

    tree.reset_counters();
    const auto scan = tree.range(35, 80);
    const std::size_t scan_reads = tree.page_reads();
    std::printf(" 范围 35..80 :");
    for (const auto& row : scan) {
        std::printf(" %d", row.first);
    }
    std::printf(", 读了 %zu 页\n", scan_reads);

    tree.erase(70);
    std::printf(" 删除 70  : %s\n", tree.to_string().c_str());
    return 0;
}
```

```
装入后   : [30,70] / [10,20] [30,50] [70,90]
插入 60  : [50] / [30] [70] / [10,20] [30] [50,60] [70,90] (树高 3)
插入 65  : [50] / [30] [60,70] / [10,20] [30] [50] [60,65] [70,90] (父结点没满，没裂到
↪  根)
查找 50  : 记录 50，读了 3 页
范围 35..80 : 50 60 65 70，读了 6 页
删除 70  : [50] / [30] [60,90] / [10,20] [30] [50] [60,65] [90]
```

范围扫描的写法直接对应「找到下限所在的叶，然后沿叶链横着走」：

```
/// 范围扫描: 找到下限所在的叶，然后沿叶链横着走，不再回到内部结点。
[[nodiscard]] std::vector<std::pair<int, std::string>> range(int low, int high)
↪  const {
    std::vector<std::pair<int, std::string>> out;
    if (low > high) {
        return out;
    }
```

```
    const Node* node = root_.get();
    while (!node->leaf) {
        ++reads_;
        node = node->children[child_slot(*node, low)].get();
    }
    while (node != nullptr) {
        ++reads_;
        for (std::size_t i = 0; i < node->keys.size(); ++i) {
            if (node->keys[i] > high) {
                return out;
            }
            if (node->keys[i] >= low) {
                out.emplace_back(node->keys[i], node->values[i]);
            }
        }
        node = node->next;
    }
    return out;
}
```

删除 70：先在叶上删，若叶下溢就向兄弟借或合并，再改内部结点的路标。这里有一处容易漏：借位和合并都会把父结点的分界 key 搬进孩子里，而那个 key 可能正是刚被删掉的——搬完必须按「分界 key = 右子树最小 key」重算一遍，否则树里会留下一个指向已删关键码的路标。这个错误不影响点查询，只有结构不变量检查才看得出来（`code/ch11/bplus_tree/legacy.md` 记了实测过程）。

删除后的处理顺序固定：

1. 页仍达到最小占用，更新必要的父分界后结束；
2. 页下溢且相邻兄弟有富余，借一个键并重算父分界；
3. 兄弟也在下限，合并两页，父结点少一个 key 和一个孩子；
4. 父结点因此下溢，向上重复；若根只剩一个孩子，让该孩子成为新根，树高减 1。

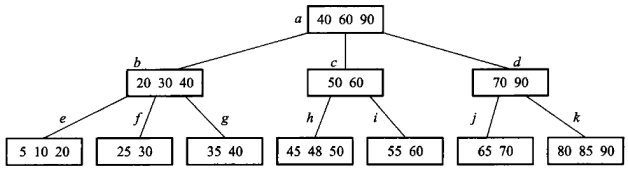


图 11.11　从图 11.9 的 3 阶 B+ 树里删掉 75 之后。沿 a、d、k 找到叶 k(75,80)，删去 75 后只剩 80，低于叶的下限 2 个，与右邻 l(85,90) 合并为新 k(80,85,90)；父结点 d 删去原分界码 80 后只剩 1 个关键码而下溢，向左邻 c(50,60,70) 借——c 把最右孩子 j(65,70) 连同分界码 70 让给 d，c 变 (50,60)、d 变 (70,90)；根 a 中指向 c 的分界码相应由 70 改为 60。共读盘 5 次 (a, d, k, l, c)、写盘 4 次 (k, d, c, a)，释放空块 l。若删除没有引起下溢，被删键在上层的复写**可以留着不动**——它只是路标，不必对应一个真实存在的键。

实际系统里内部结点和叶结点的容量往往不同：内部结点只放短 key 和页号，放得下的多；叶要放记录或记录地址，放得少。内外阶数不同的叫**混合型 B+ 树**。

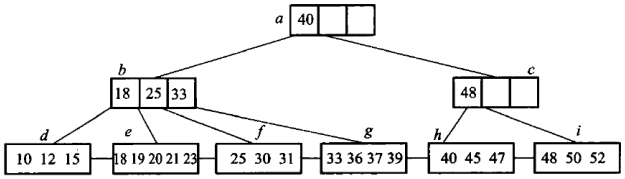


图 11.12　混合型 B+ 树：内部结点阶为 4，叶结点阶为 5。

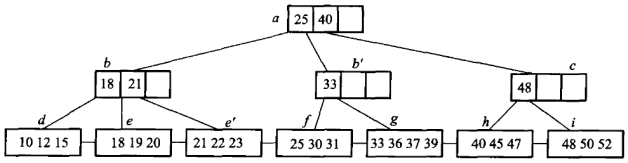


图 11.13　在图 11.12 的混合型 B+ 树里插入 22 之后。

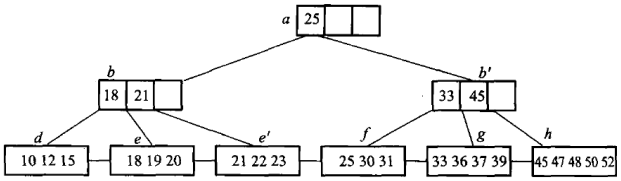


图 11.14　在图 11.13 上删除 40 之后。分裂与合并的规则不变，只是内外两层各按各自的阶判断溢出和下溢。

分裂与合并要同时维护叶链。只修父子指针而漏掉 `next`，点查询仍可能全绿，范围查询却会跨不过断链；所以测试既要逐键 `find`，也要把全范围扫描结果与排序后的记录全集对拍。

B 树与 B+ 树可以记成三句话：记录在不在内部结点；叶有没有横向链接；范围扫描要不要反复爬树。

11.4.3 B 树的性能分析

先算树高。设 B 树包含 N 个关键码，那么扩展外部空结点（即失败检索）就有 $N+1$ 个。设外部结点 都在第 I 层（根为第 0 层）：因为根至少有两个子结点，因此第一层至少有两个结点；除根和树叶外，其他结点至少有 $\lceil m/2 \rceil$ 个子结点，因此第二层至少有 $2\lceil m/2 \rceil$ 个结点，第三层至少有 $2\lceil m/2 \rceil^2$ 个.....第 I 层至少有 $2\lceil m/2 \rceil^{I-1}$ 个结点，于是 $N+1 \geq 2\lceil m/2 \rceil^{I-1}$，即

$$I \leq 1 + \log_{\lceil m/2 \rceil}\left(\frac{N+1}{2}\right)$$

这意味着若 $N = 1999998$、$m = 199$，则 I 至多等于 3：此时 $\lceil m/2 \rceil = 100$，$(N+1)/2 = 999999.5 < 100^3$，故 $\log_{100} \frac{N+1}{2} < 3$、$I < 4$。这个 N

是特意挑的——再多 1 个关键码（$N+1=2\times 100^3$）才可能出现 4 层。（原书此处作「至多等于 4」，按公式应为 3。）因为一次检索至多进行 I 次存取，这个公式保证了 B 树的检索效率是相当高的。

再从磁盘块的角度估算 B+ 树的索引规模（这段是全章最值得记住的一笔账）。假定磁盘块大小为 4096 个字节，整数型关键码值占 4 个字节，指针占 8 个字节；不考虑存储块块头信息所占的空间，满足 $4m+8(m+1)\le 4096$ 的最大整数值 m 就是 B+ 树的阶，$m=340$。假若磁盘块的平均充满度介于 最大和最小中间，即平均有 255 个指针，则对于 3 层的 B+ 树：根结点有 255 个子结点，有 $255^2=65025$ 个叶结点，每个叶结点也有 255 个索引记录，即可以有 $255^3\approx 16.6\times 10^6$ 个 指向记录的指针。**也就是说，记录数小于等于约 1660 万的文件都可以被 3 层的 B+ 树容纳，3 次访外 就可以获得关键码在主文件的地址**。

实际上还可以更少。由于 B+ 树树根通常处于内存中，而且现在系统内存空间都比较大，把 B+ 树的 第二层结点完全保存在缓冲区中也是合理的——**这样只需要读一次叶结点磁盘块**。这一句解释了今天 数据库索引的常见形态：三层 B+ 树 + 常驻内存的上两层。

插入时平均分裂几个结点? 设 p 是内部结点个数、N 是关键码个数。当根包括一个关键码、除根外 所有内部结点都包括 $\lceil m/2\rceil-1$ 个关键码时，B 树所包括的总关键码个数最少，因此有 $N\ge 1+(\lceil m/2\rceil-1)(p-1)$。考虑最坏情况（从第二个结点开始每个结点均是经过分裂而 得到的），插入 N 个关键码的过程中进行了 $p-1$ 次分裂，那么每插入一个关键码平均分裂的结点个数

$$s=\frac{p-1}{N}\le\frac{N-1}{(\lceil m/2\rceil-1)\cdot N}\le\frac{1}{\lceil m/2\rceil-1}$$

m 越大这个数越小——**这正是「结点开大一点」在插入代价上的回报**。

B 树与 B+ 树谁更矮? B+ 树内部分支结点（非叶层）中的关键码不需要像 B 树那样带隐含指针。在磁盘页块容量一致、页块指针大小相同的情况下索引同一个主文件，**B 树比 B+ 树高，即 B+ 树索引 效率更高**：设主文件有 N 个记录，关键码所占字节数与指针相同，一个磁盘页块可以存 m 个（关键码，子结点页块地址）二元对；B+ 树最大充盈度 100%、最小 50%，假设平均充盈度 $(1+0.5)/2=75\%$，即平均每个结点有 $0.75m$ 个子结点，则 B+ 树的高度为 $\lceil\log_{0.75m}N\rceil$。B 树结点除关键码和子结点页块指针外，还要为每个关键码存一个指向主文件的隐含指针，即存（关键码，隐含指针，子结点页块指针）三元组，一个页块最多只能容纳 $2m/3$ 个三元组，相应的 B 树约为 $0.67m$ 阶；假设平均充盈度 也是 75%，B 树每个结点平均约有 $0.5m$ 个子结点，高度为 $\lceil\log_{0.5m}N\rceil$。底数从 $0.75m$ 降到 $0.5m$，同样的 N 下 B 树就更高。

11.4.4 动态索引和静态索引性能的比较

静态索引在建立后不主动重排主文件；新增记录通常进入溢出区并挂链。它的结点紧凑、层数少，记录地址稳定，辅助索引也容易维护；但插删积累后溢出链变长、空洞增多，查询与空间利用率都会下降，最终必须停下来重组文件。

动态索引用 B 树或 B+ 树的分裂、借位和合并保持平衡。新旧记录具有相同的渐近查询代价，空间按需分配，也不需要周期性的全文件重组；代价是每个结点要保留孩子指针，树可能更高，更新还要处理并发锁与辅助索引中的地址变化。

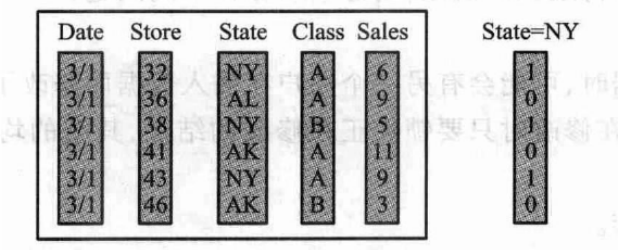
工作负载	更合适的选择	原因
数据几乎只读、可批量重建	静态索引	结构紧凑，读路径短
持续插入和删除	B/B+ 树动态索引	局部调整即可保持查询性能
范围扫描很多	B+ 树	数据集中在叶层，叶链可顺序读
记录物理地址必须长期稳定	静态索引	不因结点分裂搬动既有记录

选择的关键不是“哪一种总是更快”，而是更新成本由谁承担：静态索引把成本推迟到昂贵的批量重组，动态索引把成本摊到每次更新。读多写少且有维护窗口时静态结构仍有价值；在线系统无法停机重组时，动态索引通常更合适。

11.5 位索引技术

位图、签名和红黑树

位图索引适合取值很少的属性，例如性别、省份、是否及格。每个属性值对应一个位串，第 i 位表示第 i 条记录有没有这个值。AND、OR、NOT 查询变成机器字上的按位运算，一次可以处理几十上百条记录。属性取值一多，位图会变得很宽，需要压缩（游程、字对齐混合等）。

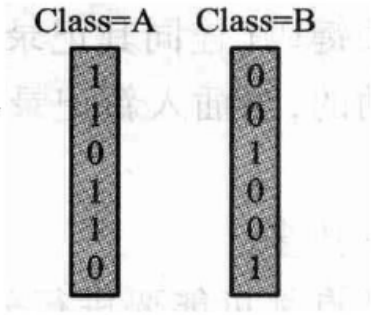


(a) 百货销售数据库记录以及 State=NY (纽约州)的位向量

(b) 数据域 State 的位向量集合

State=AK	State=AL	...	State=NY
0	0		1
0	1		0
0	0		1

State=AK	State=AL	...	State=NY
1	0		0
0	0		1
1	0		0



(c) 数据域 Class 的位向量集合

图 11.15　百货销售数据库记录的位图索引。**一个属性值一条位串**：State 有多少个取值，(b) 就有多少列，每列长度都是记录条数；查询于是变成位串之间的按位运算。属性取值越多，位图越宽——这正是位图索引只适合低基数属性的原因。

设 6 条记录的州属性和等级属性形成两条位图：

```
State=NY : 1 0 1 0 1 0
Class=A  : 1 1 0 1 1 0
AND      : 1 0 0 0 1 0
```

结果位 0 和 4 为 1，所以查询返回第 1、5 条记录（若程序下标从 0 开始则是 0、4）。OR 返回满足任一条件的记录，NOT 必须把超出实际记录数的尾部位清零；否则一个 64 位机器字只用了前 6 位，取反后其余 58 位都会被误认成记录。

位图总空间约为 “记录数 × 属性不同取值数” 位。低基数列很合算：一百万条记录的 “是否及格” 只有两张约 122 KiB 的位图；若把几乎每人不同的学号做位图，就会生成近一百万张位图，完全失去意义。

签名文件把一篇文档的特征散列成较短的位串。查询时先用签名快速丢掉不可能匹配的文档，再回到原文确认。签名可能产生假阳性（签名说可能有、原文里没有），不能有假阴性。它适合「先粗筛、再精查」的全文检索，不替代倒排。

红黑树是内存里的近似平衡二叉搜索树：每个结点带一种颜色，通过几条局部规则保证从根到叶的黑结点数相同，最长路径不超过最短路径的两倍。查找、插入、删除都是 $O(\log n)$，旋转次数有常数上限。它适合进程地址空间里的有序映射，不替代按页组织的 B+ 树。本书不另写红黑树实现，只作对照讨论。

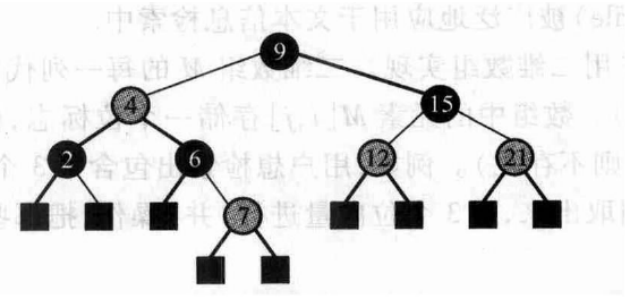


图 11.16　红黑树示意图。

红黑树的“平衡”不是要求左右子树等高，而是用颜色约束控制最长路径：根为黑、红结点不能有红孩子、从任一结点到其后代空叶的黑结点数相同。由此可得原书的四条性质，其中最有用的一条是：含 n 个内部结点的红黑树，树高（原书按根到最深的外部空叶所经过的结点数计）最多 $2\log_2(n+1)+1$（若只数内部结点、不把末端的空叶算进去，同一个界写作 $2\log_2(n+1)$，多数教材和期末复习题库 11-22 用的是这一种；两者差的那个 1 就是空叶本身）——**最长路径不超过最短路径的两倍**，检索因此是 $O(\log n)$。

本章的外存主线是 B+ 树，红黑树只作对照，所以下面按原书把调整规则列一遍、不另写实现。

插入。先按普通二叉搜索树找到位置，把新结点着成**红色**。父结点是黑的就结束；父子都红就要调整，分两种情况看**叔父结点**的颜色。

情况 1：叔父是黑色。这时靠旋转解决。

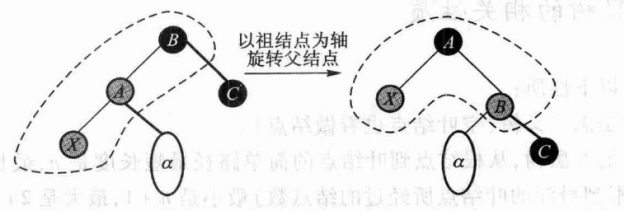


图 11.17　父结点红、叔父黑：旋转一次解决红红冲突。图中新增结点 X 是父结点 A 的左孩子；X 是右孩子时先把 X 与 A 换位，就化归成同一种形状。

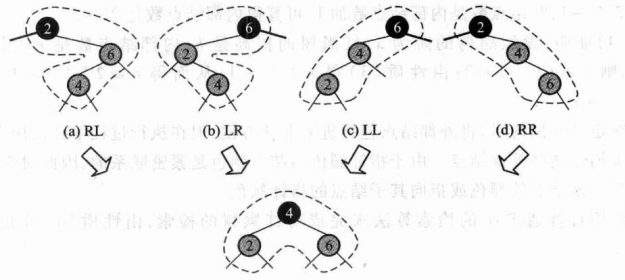


图 11.18　叔父为黑时一共四种子结构（LL、LR、RL、RR）。四种的实质是同一件事：**取「双红结点 + 祖父」这三个键的中位数当新的子根、着黑，另外两**

个当它的孩子、着红。每个结点的阶（黑高）都不变，调整到此结束。

情况 2：叔父也是红色。这时不必旋转，只要换色。

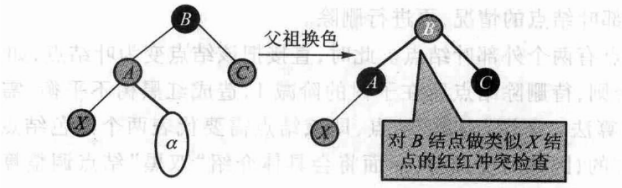


图 11.19　父与叔父都红：把父 A 和叔父 C 都改成黑、祖父 B 改成红。A 以下的冲突解决了，但 B 可能与它的父结点再冲突，于是**把 B 当成新的 X 递归上去**。最坏一路推到根，把根改回黑色即可——这是红黑树唯一会长高（阶加 1）的时刻。

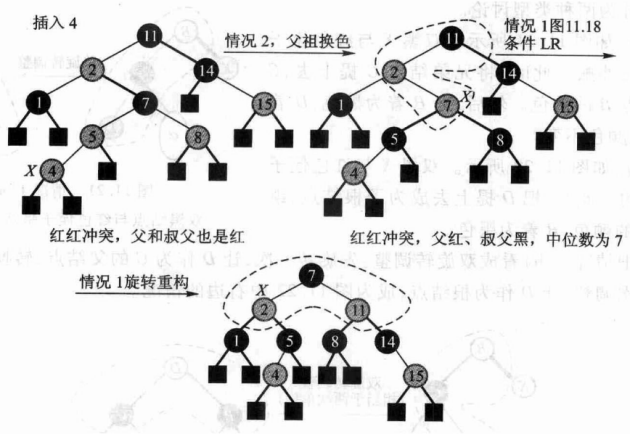


图 11.20　一次插入引发的红红冲突调整全过程。

删除。先照 BST 删：待删结点若有两个非空孩子，就与右子树的最小结点交换值（颜色不动），转成「至多一个非空孩子」的情形再删。删完可能出现**双黑**结点，这是删除最复杂的地方，按兄弟和侄子的颜色分三种情况——下面都按「双黑是左孩子」写，右孩子左右对称。

情况 1(a)：兄弟黑、红侄子与双黑呈八字形外撇。

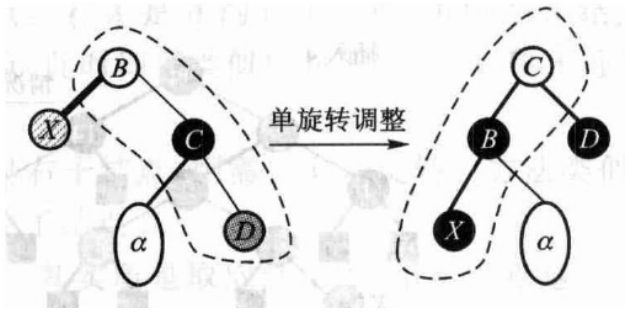


图 11.21　把兄弟 C 提上去继承原父结点 B 的颜色，B 与侄子 D 都着黑。

情况 1(b)：兄弟黑、红侄子与双黑同边顺。

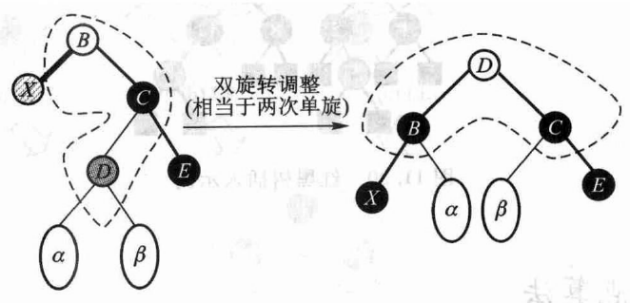


图 11.22　把侄子 D 提上去当子根、继承原子根 B 的颜色，B 着黑。也可以看成双旋转：先转一次化归成情况 1(a)，再转一次。

情况 2：兄弟黑，且兄弟的两个孩子都是黑。

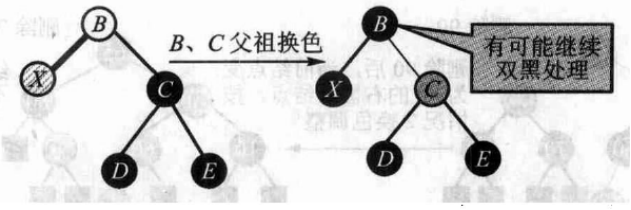


图 11.23　只换色：兄弟 C 着红、父 B 着黑。B 原来是红的就此结束；原来是黑的，就把 B 当成新的双黑继续往上调整。

情况 3：兄弟是红色。

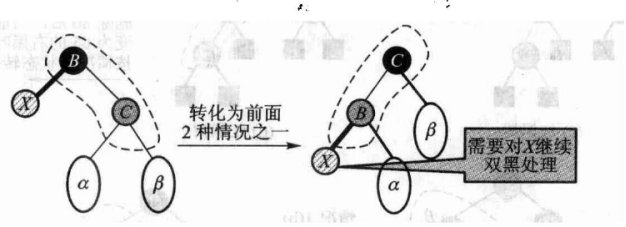


图 11.24　此时父结点必为黑、兄弟的两个孩子必为黑。旋转一次之后 X 仍是双黑，但已经**化归成前两种情况**，继续处理即可。

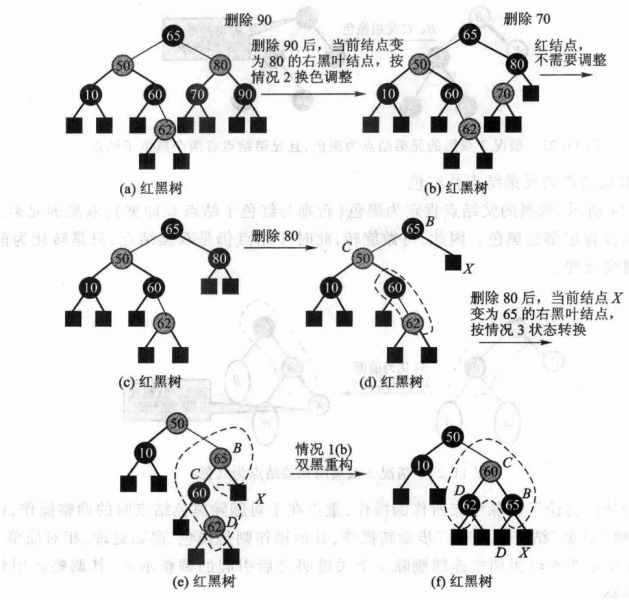


图 11.25　连续删除 90、70、80 引起的双黑调整全过程。

图 11.16 到 11.25 只用来比较「二叉、内存结点」与「多路、页结点」两种成本模型：红黑树每步只碰几个结点、适合内存里的有序映射；B+ 树每步碰一整页、适合按页读写的外存索引。

先跑一遍：

```
#include "modern.hpp"

#include <cstdio>

int main() {
    dsa::index::BitmapIndex index;
    for (int i = 0; i < 200; ++i) {
        index.add_record((i % 3) == 0 ? " 及格" : " 不及格");
    }
    index.reset_ops();
    const auto passed = index.select(" 及格");
    const auto failed = index.select_not(" 及格");
    std::printf("200 条记录，%zu 个取值，位图共 %zu 个机器字\n",
                index.distinct_values(), index.words());
    std::printf(" 及格 %zu 条，不及格 %zu 条；取反只做了 %zu 次字运算\n",
                passed.size(), failed.size(), index.word_ops());

    // 稀疏位图：大片全 0 的字，游程压缩很有效。
    dsa::index::BitmapIndex sparse;
    for (int i = 0; i < 1000; ++i) {
        sparse.add_record(i < 3 ? " 命中" : " 其他");
```

```
    }
    const auto bits = sparse.bitmap(" 命中");
    const auto encoded = dsa::index::run_length_encode(bits);
    std::printf(" 稀疏位图 %zu 个字 → 游程压缩后 %zu 个字\n", bits.size(),
    ↪  encoded.size());

    dsa::index::SignatureFile signatures(2);
    signatures.add(1, {" 数据", " 结构"});
    signatures.add(2, {" 算法", " 分析"});
    std::printf(" 签名粗筛「数据」的候选文档数：%zu（仍需回原文确认）\n",
                signatures.candidates({" 数据"}).size());
    return 0;
}
```

```
200 条记录，2 个取值，位图共 8 个机器字
及格 67 条，不及格 133 条；取反只做了 4 次字运算
稀疏位图 16 个字 → 游程压缩后 4 个字
签名粗筛「数据」的候选文档数：1（仍需回原文确认）
```

第二行就是位图的全部理由：每个取值的位串只占 4 个机器字，两个取值合计 8 个机器字；一次取反只对「及格」那一张位图的 4 个字做运算，就处理完了全部记录。查询写出来只是逐字的按位运算：

```
/// 「与」：逐字 `&`。`word_ops()` 数的就是这里做了几次字运算。
[[nodiscard]] std::vector<std::size_t> select_and(const std::string& a,
                                                  const std::string& b) const {
    return to_records(combine(bitmap(a), bitmap(b), Op::And));
}

[[nodiscard]] std::vector<std::size_t> select_or(const std::string& a,
                                                 const std::string& b) const {
    return to_records(combine(bitmap(a), bitmap(b), Op::Or));
}

[[nodiscard]] std::vector<std::size_t> select_not(const std::string& value) const {
    std::vector<std::uint64_t> bits = bitmap(value);
    for (auto& word : bits) {
        word = ~word;
        ++ops_;
    }
    mask_tail(bits);  // 最后一个字里超出记录数的那些位必须清掉
    return to_records(bits);
}
```

注意 `mask_tail`：记录数不是 64 的整数倍时，最后一个字里超出记录数的那

些位取反后会变成 1，于是冒出根本不存在的记录号。这是位图实现最常见的一个洞。

压缩不是万能的。上面稀疏位图 16 个字压到 4 个，但对 0 和 1 交替的稠密位图，游程压缩后**反而更大**——`code/ch11/bitmap_index/test.cpp` 直接断言了这一点，不粉饰。

签名文件也需要一个反例才能看清边界。若 “数据” 和 “结构” 经散列后碰巧设置了与 “算法” 相同的几位，查询 “算法” 时这篇文档会进入候选集，即假阳性；但只要构造签名时把每个真实词的位都置上，真正含 “算法” 的文档不可能被漏掉。粗筛之后必须回原文确认，不能把候选集直接当查询答案。

结构	保存什么	可能的额外工作	适合场景
倒排表	词到文档号/位置的列表	合并有序列表	精确全文检索、短语查询
位图	属性值到记录位向量	按位布尔运算	低基数列组合过滤
签名	文档特征的短位串	假阳性后回原文确认	空间受限的粗筛
B+ 树	有序 key 到记录位置	沿页树下降、沿叶链扫描	点查询与范围查询

练习路径

本章四个方向都已有可运行实现，练习因此从「照着写一遍」改成「在已有实现上再往前走一步」：

1. 给 `MultiLevelIndex` 的页内定位换成二分查找，测量比较次数的变化，并确认**页访问次数一次都没变**——本章的关键指标不在这里。
2. 给 `InvertedIndex` 的倒排表加差值编码（只存与前一个文档号的差），比较压缩前后的表长；注意求交时要能边解码边归并。
3. 给 `BPlusTree` 加一个「按范围批量删除」的接口，并用 `validate()` 确认每一步之后结构仍然合法。
4. 给 `BitmapIndex` 换成字对齐混合编码（WAH），和现在的字级游程比压缩率；用交替位图确认它不会像现在这样越压越大。
5. 为 `SignatureFile` 做一次参数实验：固定语料，改变每词位数，画出假阳性率随位数的变化。

本章小结

Python **实现的证据链**

```
def find(self,key):
    if not self._levels:
        return None
    page = self._locate(self._levels[-1],0,len(self._levels[-1]),key)
    for level in range(len(self._levels)-1,0,-1):
        page = self._levels[level][page][1]
        self._reads += 1
        first = page*self.epp
        last = min(first+self.epp,len(self._levels[level-1]))
        page = self._locate(self._levels[level-1],first,last,key)
    entry = self._levels[0][page]
    if self.kind==DENSE:
        if entry[0]!=key:
            return None
        self._reads += 1
        return self.records[entry[1]][1]
    self._reads += 1
    for record_key,value in self.records[entry[1]*self.rpp:(entry[1]+1)*self.rpp]:
        if record_key==key:
            return value
    return None
```

```
def intersect(left, right):
    out = []
    i=j = 0
    while i<len(left) and j<len(right):
        if left[i]<right[j]:
            i += 1
        elif right[j]<left[i]:
            j += 1
        else:
            out.append(left[i])
            i += 1
            j += 1
    return out

def unite(left, right):
    out = []
    i=j = 0
    while i<len(left) or j<len(right):
        if j==len(right) or (i<len(left) and left[i]<right[j]):
            value = left[i]
            i += 1
        elif i==len(left) or right[j]<left[i]:
            value = right[j]
```

```
            j += 1
        else:
            value = left[i]
            i += 1
            j += 1
        if not out or out[-1]!=value:
            out.append(value)
    return out

def difference(left, right):
    out = []
    j = 0
    for value in left:
        while j<len(right) and right[j]<value:
            j += 1
        if j==len(right) or right[j]!=value:
            out.append(value)
    return out
```

```
def select_and(self,a,b):
    return self._combine(a,b,lambda x,y:x&y)
def select_or(self,a,b):
    return self._combine(a,b,lambda x,y:x|y)
def select_not(self,value):
    bits = [(~word)&MASK for word in self.bitmap(value)]
    self._ops += len(bits)
    if bits and self._count%64:
        bits[-1] &= (1<<(self._count%64))-1
    return self._records(bits)
```

B+ **树这一节没有** Python **版，理由值得单独讲一句**。本书讲算法的章节两种语言各给一份（D-025），但 B+ 树讲的**不是算法，是页式存储管理**：页容量、装填因子、结点分裂、删除时的借位与合并，以及 `page_reads()` / `page_writes()` 这两个只有在真的按页访问时 才有意义的计数器。这些东西在一个 `list` 上无处安放——和 3.1 节的顺序栈、4.2 节的 字符串类是同一条线。

这不是事后的托词，是**试出来的**。本书曾提交过一版 Python「B+ 树」：内部是一个有序 `list`，`height()` 与 `leaf_count()` 由记录数套公式算出，`page_reads()` 累加的是那个 公式的结果，而标着「结点分裂」的那段代码里没有任何分裂。它跑得通，闸门也是绿的—— **因为一个不存在的结构，没有任何断言能证伪它**。理由与那次的结论一起记在 `code/ch11/bplus_tree/unit.json` 的 `py_skip` 字段里，闸门核对该单元确实没有 `modern.py`。想看 B+ 树按页分裂到底怎么写，读上面那段 C++。

索引就是把关键码与其主文件中的数据记录位置相关联的过程，索引文件则

是用于记录这种联系的文件 组织结构。**索引技术的目的是在支持高效数据（尤其是外存的大量数据）检索的同时，支持高效的插入和 删除操作，它是数据库的核心技术之一**。

实际应用中经常需要根据属性的值来查找记录，因此采用**倒排索引**技术：每一项都包括一个属性值和 具有该属性值的那些记录地址；由于不是由记录来确定属性值，而是由属性值来确定记录的位置，因此 称为倒排索引，带有倒排索引的文件称为倒排文件。

线性索引被组织成简单的（关键码，指针）的序列，按照关键码的顺序进行排序，其常用的检索技术是 二分检索；数据规模太大、索引文件无法完全存储到内存时，可以采用二级线性索引。

多分树静态索引结构在文件创建、初始装入记录时生成，并固定不变，插入和删除操作非常不方便—— 只有当文件再组织时才允许改变索引结构。

B/B^+ **树是动态的索引结构**，允许动态地插入或删除记录，索引结构本身也可能发生改变；改变索引 结构的目的是为保持较好的性能，例如较高的检索效率。B 树和 B^+ 树都是经典的动态索引，本质上是 平衡的多分树：B 树中的关键码没有重复，父结点中的关键码是其子结点的分界。B^+ **树是** B **树的 一种变形树，它不同于** B **树的特点是**：(1) 有 k 个子结点的结点必然有 k 个关键码；(2) B^+ 中叶 结点是关键码的一个全集，跟实际数据记录有关的信息均存放在叶结点中；(3) 内部分支结点仅具有索引 作用，可以看做是从叶往根的方向一层层复写子结点中的最大（或最小）关键码。

B 树、B^+ 树的插入与删除都要特别注意保持其平衡性质，特别是等高、子结点个数、关键码个数的 上下界限制。B **树的插入可以说是「插入—分裂」的过程**：首先是在恰当的叶结点中添加关键码，插入 分裂时中位数要向父结点中传递，分裂过程可能一直传到根、建立一个新根结点，树增高一层。B **树的删除可以说是「交换—删除—借关键码—合并」的过程**：若被删码不在叶结点层，则先把此关键码 与其后继对换位置然后再删除；借关键码时父结点中的分界码要合并到两个相邻结点的关键码集合中，该集合的中位数尽量把集合平等划分为两个新结点、中位数代替父结点中的原分界码；发生合并时，父结点 中的分界码要拿下来合并到两个相邻结点的关键码集合中，合并也可以传递到根。

B^+ **树检索时必须继续沿指针向下一直查到叶结点层的关键码**，才能找到跟记录有关的信息（例如指向 该记录主文件中位置的指针）。B^+ 树的插入也是「插入—分裂」的过程，也是在叶结点上开始：如果上溢 导致叶结点分裂，父结点会增加一个复写分界关键码，也有可能要修改原复写分界码的值，目的是保证上层结点中有这两个结点的复写码。B^+ **树的删除是「删除—借关键码—合并」的过程，不需要交换**——B^+ 树中的关键码在叶结点层删除后，在不影响检索的前提下，在上层的复本可以保留作为一个「分界关键码」存在。

B **树只适合随机检索**。B^+ **树的所有叶结点构成一个有序链表，可以按照关键码大小有序地遍历全部 记录，因此** B^+ **树同时支持随机检索和顺序检索（尤其是范围检索），在实际中应用比较多**。

B/B^+ 树适合于检索并取回少量记录的情况；**对于唯一值极少（低基数）的数据域，适合采用位图索引技术**，用位串来代表数据库列中是否存储该数据域取值，计算机可以迅速地执行位图映射操作，这是一种 非常高效的索引。**签名文件**被广泛地应用于文本信息检索中，其核心思想是把文本的特征用签名位串来 表示；签名文件比原文件小得多，因此可以提供更快速的检索。

红黑树也是一种二叉搜索树，它没有第 12 章将要介绍的 AVL 树那样完全平衡的特性，而是利用对树中 结点红黑着色的要求降低了平衡性的条件，达到局部平衡，从而既能提高红黑树增删记录的性能，又能达到 高效率的检索要求。**它的统计性能要好于 AVL 树，且增删记录算法性能好、易实现，因此红黑树是一种很好的索引结构**——C++ STL 的 `set`、`multiset`、`map`、`multimap` 都应用了红黑树的变体，Linux 内核中的用户态地址空间管理也使用了红黑树。

具体到本章的实现：线性索引分稠密和稀疏，过大就做成多级。静态多分树把一个结点做成一页，树高低、页访问少，但不适合频繁更新。倒排从属性走到记录集合，查询变成交并差。B 树在页内分裂合并以保持 平衡；B+ 树把记录放在叶上并用叶链做范围扫描。位图适合低基数属性，签名做粗筛，红黑树是内存有序映射。

习题

补充索引题（参考课程第 11 章）

1. 给定页大小、记录大小和索引项大小，计算稠密索引最多支持的记录数及二级索引的访外次数。
2. 对给定的院系表和宿舍表建立倒排索引，并求 “院系且宿舍” 查询的交集。
3. 模拟 B+ 树查找、插入分裂和删除后继替换，分别统计读页和写页次数。

正文习题

1. 稠密索引和稀疏索引各要求主文件怎样组织。
2. 多级索引一次查询大约读几页？关键指标为什么不是比较次数。
3. 静态多分树插入一条溢出记录会发生什么。
4. 用倒排表求「计算机系且擅长英语」的交集。
5. 在 11.4 插入 60 之后的那棵 3 阶 B+ 树上继续插入 65，写出叶和根，并说明这次为什么没有一直裂到根。
6. 位图、签名、红黑树、B+ 树各适合内存还是外存、点查询还是范围查询。

原书习题

> 以下是原书第 11 章的习题，本轮按扫描件补回题面；**参考答案见 `book/习题与参考答案.md` 的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。图 11.1、11.2、11.9、11.12、11.26～11.31 见原书。

1. 假定一个计算机系统有 4096 B 的磁盘块。存储的每一条记录中 4 B 是关键码、64 B 是数据域，记录 已经排序、顺序地存放到磁盘文件中。线性索引的结构

为（每块的最小关键码，该块的磁盘地址），地址 指针为 4 B。通过线性索引访问磁盘文件中的记录。(1) 如果线性索引的大小是 2 MB，最多可以在磁盘文件中存储多少条记录? (2) 如果线性索引也存储在磁盘中（这样它的大小仅受二级索引限制），而且使用 4096 个字节（即 1024 个关键码值）的二级索引时，文件中最多可以存储多少条记录？二级索引中的每个 单元引用线性索引磁盘块中的最小关键码值。

2. 假定一个数据库存储的记录由 2 个字节的整数关键码和一个变长的数据域组成, 数据域是一个字符串。试给出下面一组记录的线性索引: `397 Hello world!`、`82 XYZ`、`1038 This string is rather long`、`1037 This is shorter`、`42 ABC`、`2222 Hello new world!`。
3. 设有一个职工文件，并设该文件由 5 个记录组成，其中职工号为关键码（记录见原书表)。
   (1) 若该文件为顺序文件，试写出文件的存储结构；(2) 若该文件为索引顺序文件，试写出索引表；(3) 若该 文件为倒排文件，试写出关于性别的倒排索引和关于职业的倒排索引。
4. 对于图 11.9 所示的 B 树，依次进行如下操作：插入关键码 480，删除关键码 375，分别画出每次操作 后 B 树的形状。(原书如此。图 11.9 是关键码为 5～90 的 B^+ 树，其中并无 375，题面所指的图疑为别处引来，作答时需自拟一棵含 375 的树。)
5. 画出能由 1、2、3、4、5、6、7、8 构成的所有 5 阶 B 树。
6. 求满足如下条件的序列的最短长度：按照序列顺序插入关键码能生成高度为 3 的 5 阶 B 树。
7. 假设按如下方法修改从 B 树中删除元素的方式: 如果一个结点既有最相邻左兄弟也有最相邻右兄弟，那么在合并前对两个兄弟都要做检查。从一棵高度为 h 的 B 树中删除元素时需要的最大磁盘访问 次数是多少?
8. 设有一棵阶 $m = 3$ 的 B 树（图 11.26），其中 $a, b, \cdots, g$ 是结点的名称、结点内的整数为关键码。若在图中插入关键码 55，画出插入后 B 树的状况，计算完成该插入所需要的访外次数（假定一次访外 存取一个结点），并具体说明每次访外的功能。

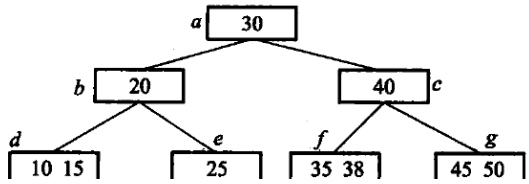


图 11.26　习题 8 的 3 阶 B 树图示

9. **偶数阶的 B 树插入上溢出时, 中位数有两个, 需要注意采用统一的策略**。例如取第二个中位数,即 分裂后左 $\lceil (m-1)/2 \rceil$ 个关键码、右 $\lfloor (m-1)/2 \rfloor$ 个关键码; 或者取第一个中位数, 分裂后左 $\lfloor (m-1)/2 \rfloor$ 个关键码、右 $\lceil (m-1)/2 \rceil$ 个关键码。试画出对图 11.27 所示的 4 阶 B 树进行下列操作后的 B 树：(1) 分裂时采用第 2 个中位数为分界码，试画出插入关键码 113 后 的 B 树；(2)

分裂时采用第 1 个中位数为分界码，试画出插入关键码 113 后的 B 树；
(3) 在原树中删除关键码 50。

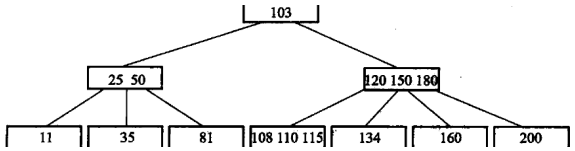


图 11.27　习题 9 的 4 阶 B 树

10. 分别画出在如图 11.28 所示的 3 阶 B 树中，进行下列操作后的 B 树：(1) 插入关键码 75；
(2) 删除关键码 156。

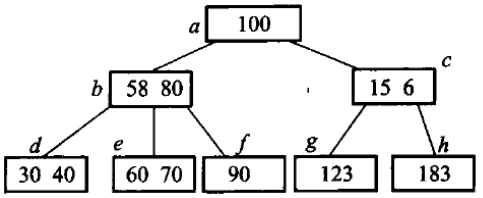


图 11.28　习题 10 的 3 阶 B 树

11. 在 B 树定义中，特性 (2) 的意义是什么？如果把 $\lceil m/2 \rceil$ 改成 $\lceil m/3 \rceil$ 或者 $\lceil 2m/3 \rceil$ 是否可以？所得到的树结构和 B 树有何区别？
12. B 树中的某个关键码不在叶结点中，试证明它的前驱和后继（自然顺序）必然在叶结点中。
13. 在含有 n 个关键码的 m 阶 B 树中进行检索时，最多访问多少个结点？
14. 试证明：高度为 h 的 2-3 树（3 阶 B 树）中叶结点数目在 2^{h-1} 与 3^{h-1} 之间。
15. 假设一个数据文件每个记录对象需要占用 128 B（其中关键码占用 4 B），且所有记录均已按关键码 有序地存储在主磁盘文件中。设磁盘页块大小为 2048 B，若主存中有 12 MB 空间可以用来存储索引 结构，索引项中每一个地址指针占 8 B。试简要回答以下问题（写明计算过程）：(1) 使用 B 树索引，B 树的阶 m 最多可以为多少？（注：在 B 树中找到关键码的同时，应该可以得到其在主文件中的 地址。）(2) 4 层 m 阶 B 树最多可以索引多少字节的数据文件？（注：独根 B 树算 1 层，空 B 树算 0 层。）(3) 假设尽量把 B 树的头几层放入内存（本题规定不能超过 12 MB），那么给定关键码，通过 B 树查找到 (2) 小题中主数据文件的一个记录，最少需要几次访外？最多需要几次访外？(4) 上述问题 (1)～(3) 中的 B 树换成 B^+ 树，试回答相关的问题。
16. 假定一个文件有一兆（1000000）个记录，每个记录占 200 B，其中关键码占 50 B。一个磁盘页块（结点）有 1000 B，页块指针为 5 B。若用 B^+ 树组织索引，并假定所有页块（结点）都尽可能装满，则需要多少 索引页块（即 B^+ 树中有多少个非叶结点）？在上述文件组织中存取一个记录需要多少次外存访问？
17. 对于下面两种 B^+ 树，列表给出它们在 1、2、3、4 和 5 层（独根是一层树）

的不同情况下，能够存储的 最大记录数和最小记录数：(1) 如图 11.9 那样的 B^+ 树，其内部结点阶为 50、叶结点阶为 50；

(2) 如图 11.12 那样的混合型 B^+ 树，其内部结点阶为 100、叶结点阶为 15。

18. 给定一组记录，其关键码值是字母。记录按照下面的顺序插入：C、S、D、T、A、M、P、I、B、W、N、G、U、R、K、E、H、O、L、J。试给出插入这些记录后的 4 阶 B^+ 树，假定叶结点最多可放 3 条记录。
19. 已知一组关键码为 $(20, 30, 50, 52, 60, 68, 70)$，试依次插入关键码：(1) 生成一棵 3 阶的 B 树，画出每次插入一个关键码后 B 树的结构，并画出删除 50 和 68 后的 B 树状态；(2) 生成一棵 3 阶的 B^+ 树，画出每次插入一个关键码后 B^+ 树的结构，并画出删除 50 和 68 后的 B^+ 树状态。
20. 假定一个文件由 15 个记录组成，每个记录的关键码均为整数，分别为 $1^2, 2^2, 3^2, \cdots, 15^2$。每个数据页块存放 3 个记录。要求：(1) 用 B 树组织索引，设 $m = 3$，依次将上述 15 个关键码插入 B 树，画出插入后的 B 树的结构图；(2) 用 B^+ 树组织索引，设 $m = 3$，依次将上述 15 个记录插入 文件中，画出插入后的整个文件的结构图（包括 B^+ 树索引）。
21. 在如图 11.29 所示的 2 阶 B^+ 树中，删除关键码 35。试讨论删除后应该是 2 层 B 树，还是 3 层 B 树。

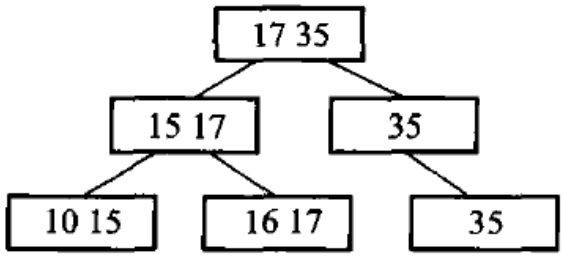


图 11.29　2 阶 B^+ 树图示

22. 图 11.30 所示为一棵混合型 B^+ 树，假定内部结点的阶为 4、叶结点的阶为 3。(1) 画出插入关键码 C 以后的 B^+ 树；(2) 画出删除关键码 I 以后的 B^+ 树。

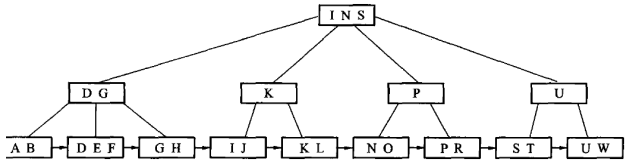


图 11.30　混合型 B^+ 树，内部结点阶为 4、叶结点阶为 3

23. 下面是顾客记录，格式为「顾客号：(年龄, 月收入)」：1:(25,600) 2:(45,600) 3:(50,750) 4:(50,1000) 5:(50,1200) 6:(70,1100) 7:(85,1400) 8:(30,2600) 9:(25,4000) 10:(45,3500) 11:(50,2750) 12:(60,2600)。(1) 试写出年龄位图索引向量集合；(2) 试写出月收入位图索引向量集合；

(3) 找出年龄范围为 20～40 且月收入为 1000～2000 元的顾客。

24. 考虑一个具有 1000000 个记录的文件，其数据域 F 有 m 个不同值。(1) F 的

位图索引有多少个 字节? (2) 假定数据域 F 的每个取值每隔 m 个记录出现一次，试设计一种合适的压缩存储方案，并计算该压缩索引需要多少个字节?

25. 对如图 11.31 所示的红黑树，画出下面插入和删除过程的主要变换步骤：(1) 插入结点 19；
(2) 删除结点 10。

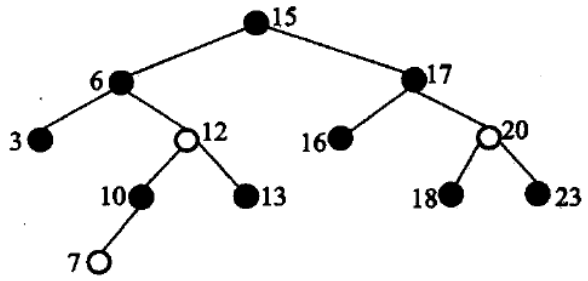


图 11.31　红黑树

26. 讨论采用自顶向下的红黑树插入和删除调整方法，并给出插入和删除的实例。

上机题

见本章「练习路径」的五条。

原书上机题

原书上机题的参考答案见 book/习题与参考答案.md 的「原书上机题参考」，由 R14 逐题守着。

1. 对如图 11.1 和 11.2 所示的变长记录实现一个二级线性索引。假定磁盘块的长度是 1024 B，数据库 文件中记录的长度一般在 20～200 B 之间、包括 4 B 的关键码值。索引文件中的每条记录存储一个关键 码值和一字节偏移量，这个偏移量表示相应记录第一个字节在数据库文件中的字节偏移量。**顶层索引（存储在内存中）应当是一个简单的数组**，存储索引文件相应块的最小关键码值。
2. 试参照 B 树的插入、删除算法实现 B^+ 树的插入、删除算法。
3. 试设计一个遍历 B 树的算法，使输出的关键码序列递增有序。
4. B^* 树是内部结点包含至少 $\lceil 2m/3 \rceil$ 个子结点的 B 树，试考虑 B^* 树的插入和删除操作 造成的分裂与合并应该采取怎样的策略并实现之。
5. 现有红黑树 T_1、T_2 和关键码 z，T_1 中的关键码都比 z 小，T_2 中的关键码都比 z 大。试将 T_1、T_2 和 z 合并成一棵新的红黑树 T_3（树中的关键码不重复）。
   (1) N 是操作结点总数的上限，试分析你的算法的时间代价，如果高于 $O(\log N)$ 要调整算法；
   (2) 在不给定关键码 z 的情况下，实现两棵红黑树的合并操作。

第 12 章 高级数据结构

本章把前面学过的「存储、查找、回收」推到更复杂的对象上：多维数组把多个下标换算成一个线性地址；稀疏矩阵让结点同时进入行链和列链；广义表允许共享甚至成环；Trie 按关键码的字符或位分支；最优 BST、AVL 和伸展树则分别用访问权重、严格高度平衡和访问局部性改进查找。可利用空间表与标记–清除穿插其间，回答这些结点不用以后由谁回收。

读这一章不要只记结构名称。每一节都追问三个问题：**一个逻辑对象落在哪些物理位置；一次更新要同时维护哪些不变量；结构失效或空间不足时怎样收尾**。后面的演算都围绕这三问展开。

本章各节的实现状态：

小节	实现状态	代码
12.1 多维数组、特殊矩阵、稀疏矩阵	混合	`code/ch12/sparse_matrix` 实现十字链表；多维定位仍为公式
12.2.1 广义表	实现并测试	`code/ch12/gen_list`
12.2.2 可利用空间表	实现并测试	`code/ch12/optimal_bst` 的 `ReusableNodePool`
12.2.3 动态分配与回收	实现并测试	`code/ch12/memory_allocator`；三种适应策略与相邻块合并
12.2.4 无用单元回收	实现并测试	`code/ch12/storage_recovery`；根集与标记–清除
12.3 Trie 与 Patricia	实现并测试	`code/ch12/trie`
12.4 最优 BST / AVL / 伸展树	实现并测试	`code/ch12/optimal_bst`、`code/ch12/balanced_trees`

其中广义表、Trie/Patricia、AVL 与伸展树原书都没有给清单（第 12 章的清单只有算法 12.1、算法 12.2），这三个单元是新写的，不认领任何原书清单。

12.1 多维数组

多维数组是一维数组的扩充：数组的数组就是二维，二维数组再排成一列就是三维。结构上的特点是，元素本身还可以有结构，但同一数组里的元素类型相同。元素个数相对固定，一旦生成通常只改值、不改相对位置和个数，因此自然采用顺序存储。

12.1.1 多维数组的存储

C++ 里每一维的下界都是 0。二维数组既可以看成 m 个行向量组成的向量，也可以看成 n 个列向量组成的向量。每个元素 $a_{i,j}$ 同时属于第 i 行和第 j 列，最多有两个前驱、两个后继。推到 k 维，每个元素属于 k 个向量。

把数组按某种周游次序排成线性序列，就可以顺序存放。C++ 和 Pascal 按**行优先**：先排最右的下标，从右往左排。二维数组排出来是

$$a_{0,0}, a_{0,1}, \cdots, a_{0,n-1}, a_{1,0}, \cdots, a_{m-1,n-1}$$

$d_0 \times d_1 \times \cdots \times d_{n-1}$ 数组中，元素 $A[j_0, \ldots, j_{n-1}]$ 相对首地址的偏移是

$$d \cdot \Big(\sum_{i=0}^{n-2} j_i \prod_{k=i+1}^{n-1} d_k + j_{n-1} \Big)$$

其中 d 是一个元素所占单元数。每个元素的定位时间相同，所以这是随机存储结构。FORTRAN 按**列优先**，先排最左下标，公式左右对调。

以 `int A[2][3][4]` 为例，找 `A[1][2][3]` 时，不必真的把 24 个元素列出来。最右一维每跨一步跳 1 个元素，中间一维每跨一步跳 4 个，最左一维每跨一步跳 $3 \times 4 = 12$ 个：

$$offset = 1 \times 12 + 2 \times 4 + 3 = 23$$

若 `int` 占 4 字节，字节偏移就是 92。也可以从左往右用霍纳法算：`((1 * 3 + 2) * 4 + 3) * 4 = 92`。这种写法只需一边读下标一边累乘，不必预存每一维的跨度。定位前仍要逐维检查 `0 <= j_i < d_i`；公式本身不会替程序发现越界。

把每一步展开，可以同时核对跨度与边界：

维	下标	后续维大小的乘积	贡献
0	1	$3 \times 4 = 12$	12
1	2	4	8
2	3	1	3
合计			23 个元素

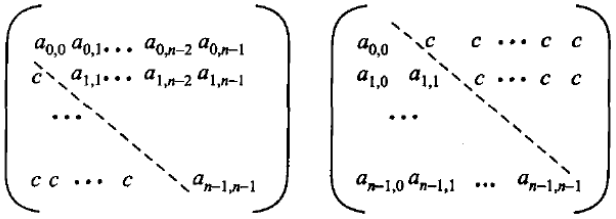
列优先不是把最终答案随意反过来,而是把最左下标当作变化最快的一维。同一个 `2 x 3 x 4` 形状若按列优先存,`[1,2,3]` 的元素偏移为 $1+2\times2+3\times(2\times3) = 23$；这个角落碰巧仍是 23，换成 `[1,0,0]` 就能分辨：行优先为 12，列优先为 1。测试布局时应选这种能区分规则的下标。

12.1.2 特殊矩阵

矩阵常用二维数组表示，但有些矩阵里大量元素是 0 或同一个常数，不必存整块。

三角矩阵。n 阶上三角（或下三角）里，对角线一侧全是 0 或常数 c。只需存另一侧的 $\sum_{i=0}^{n-1}(i+1)=(n^2+n)/2$ 个非零（或非常数）元素，再加 1 个单元存那个常数 c。若用一维数组 `list[0 .. (n²+n)/2-1]` 存下三角，元素 $a_{i,j}$ $(i\ge j)$ 前面有 i 行、共 $(i^2+i)/2$ 个非零元，再加本行的 j，下标就是 $(i^2+i)/2+j$。

对称矩阵。$a_{i,j}=a_{j,i}$。只存下三角（含对角线），另一半用对称关系映射。`list` 与 $a_{i,j}$ 的对应是：$i\ge j$ 时下标 $(i^2+i)/2+j$，否则 $(j^2+j)/2+i$。



4 阶下三角按行压缩后的区间是 `[0]`、`[1..2]`、`[3..5]`、`[6..9]`。因此 $a_{3,1}$ 落在 `6 + 1 = 7`；上三角位置 $a_{1,3}$ 若矩阵对称，则先换成 $a_{3,1}$，仍读下标 7。先判断三角区域、再套公式，比背两套容易核对。

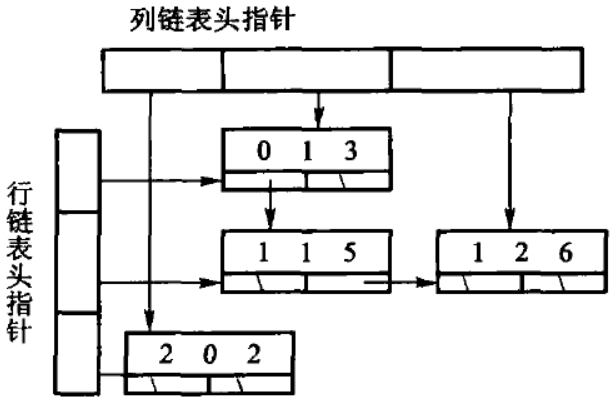
12.1.3 稀疏矩阵

非零元很少、分布又不规则时，叫稀疏矩阵。$m\times n$ 的矩阵里有 t 个非零元，稀疏因子 $\delta=t/(mn)$；通常 $\delta<0.05$ 就按稀疏处理。这时不应再分配整块二维数组，而改存三元组 (行，列，值) 的线性表，或用十字链表：**每个非零元同时挂在一条行链和一条列链上**，便于按行、按列遍历。

十字链表（orthogonal list，也叫 cross-linked list）是教材中的双向索引结构，适合需要频繁按行、按列更新的稀疏矩阵。现代数值计算更常用 COO（三元组）、CSR（压缩稀疏行）或 CSC（压缩稀疏列）：它们连续存储、缓存局部性更好，但动态插入通常需要批量重建。

三种存法的代价：

	整块二维数组	三元组线性表	十字链表
存储量	mn	$O(t)$	$O(t)$，每个元另带 `right`、`down` 两个指针
取 a_{ij}	$O(1)$	$O(t)$；若按键有序可用二分，$O(\log t)$	$O(\text{该行非零元数})$
按列扫一遍	$O(m)$	$O(t)$，要滤掉别的列	$O(\text{该列非零元数})$
插入一个非零元	$O(1)$	$O(t)$，要挪动后面的项	$O(\text{行内}+\text{列内定位})$



看一个 4×5 的小矩阵，只列非零元：`(0,1,8)`、`(0,4,2)`、`(2,1,5)`、`(3,3,7)`。结点 `(2,1,5)` 在第 2 行链里是唯一结点，在第 1 列链里却接在 `(0,1,8)` 后面。把它删掉必须做两次摘链：第 2 行头改为空，第 1 列中让 8 的下继越过 5。只改一条链，按行和按列看到的就不再是同一个矩阵。

插入 `(0,1,9)` 也不能再造一个同坐标结点；应找到旧结点并覆盖 8。测试十字链表时因此至少要查四件事：新坐标同时出现在两条链、覆盖不增加非零元数、赋值为 0 等价于删除、删除后两条链都找不到它。

操作后	第 0 行链	第 2 行链	第 1 列链	t
初始	`(0,1,8) -> (0,4,2)`	`(2,1,5)`	`(0,1,8) -> (2,1,5)`	4
覆盖 `(0,1,9)`	`(0,1,9) -> (0,4,2)`	`(2,1,5)`	`(0,1,9) -> (2,1,5)`	4
删除 `(2,1)`	`(0,1,9) -> (0,4,2)`	空	`(0,1,9)`	3

第二行是十字链表**吃亏**的地方，第三行才是它存在的理由。`code/ch12/sparse_matrix` 的实现里，插入时在两条链上各定位一次并同时接上，删除时同样从两条链上各摘一次——都是**局部**操作，不碰其他行列。测试用一个 200×200 的对角线矩阵、再往第 7 列补两个非零元（共 202 个）量过：扫第 7 列只走过该列的 3 个结点，而不是全表的 202 个。

结点所有权归容器，两条链都是裸指针，析构沿行链**迭代**释放。这里不能用 `unique_ptr` 串链——一行里非零元一多，递归析构就会压穿栈（判据见 2.3.1 节的「所有权工具怎么选」；实测数字在该单元的 `legacy.md` 里）。

12.2 广义表和存储管理

为什么这一节没有 Python 版

广义表、共享子表和回收算法讨论的是存储图而不是普通嵌套序列：结点可能被多个表共享，释放时要避免重复访问，还要处理空闲块和环。Python 的引用计数/垃圾回收会替实现者维护这些关系，`list` 也无法表达本节的空闲表、句柄复用和强异常保证。这里因此只给 C++，让所有权、访问标记和回收顺序成为可见的教学对象。

现代视角：“广义表”是数据结构教材中的历史名称；现代代码通常把同一类问题称为**嵌套对象、对象图、共享** DAG **或循环对象图**。例如编译器 AST、JSON/XML 的嵌套值、内容寻址存储中的共享节点，都可以用本节的头尾分解、共享和环来解释。术语可以保留，但不要把它误解成今天仍普遍使用的标准容器接口。

线性表的元素是不可再分的原子。广义表放宽这条：元素可以是原子，也可以是另一个广义表。按是否共享、是否有环，分成三种：纯表对应树（每个子表只出现一次）；再入表对应有向无环图（子表可以被共享）；循环表对应有环图。存储上常用带头尾指针的结点；表一旦共享，释放时就必须处理别名，不能只按树来递归 `delete`。

12.2.1 广义表的定义和存储结构

表头是第一个元素，表尾是去掉表头后剩下的那个表。空表既没有头也没有尾。任何一个非空广义表都可以唯一地拆成「头 + 尾」，所以递归算法写成「先处理头、再处理尾」就和定义对上了。

对 $L = (a,(b,c),d)$ 连续拆解：`head(L)=a`，`tail(L)=((b,c),d)`；对子表 `(b,c)`，头是 `b`、尾是 `(c)`。长度只数最外层的三个元素，得到 3；深度要进入子表，得到 2；原子数则遍历所有层，得到 4。三个指标回答不同问题，不能从括号或逗号数机械推出。

按子表允许的形式，广义表分成几个层次。最简单的是**线性表**——所有元素都是原子：

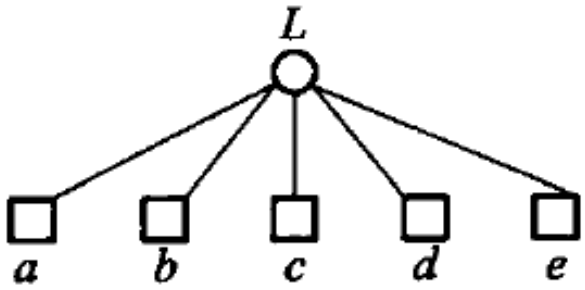


图 12.3　纯线性结构表。

纯表 (pure list) 对应一棵树：从根到任何叶只有一条路径，每个元素只出现一次；每对括号是一个内部结点，原子是叶。

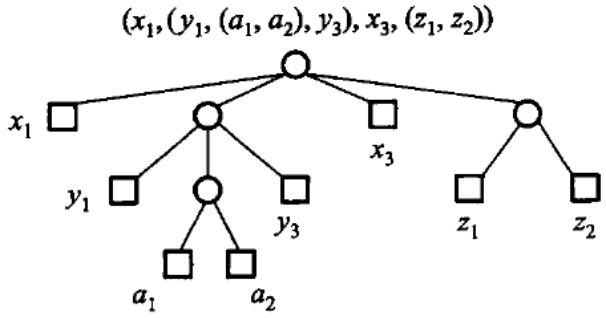


图 12.4　纯表的树表示。

再入表允许共享：某些结点可以从多条路径到达，无回路时对应一个 DAG（见图 12.5，排在本节演示程序之后）。**循环表**允许某个子表直接或间接把自己当成表目，对应有回路的有向图（图 12.6）。四者的包含关系是：

图 ⊇ 再入表（循环表） ⊇ 纯表 ⊇ 线性表

关于共享，原书有一条实践上的约定值得记住：最好是人为地把需要被共享的单原子定义为一个单元素的 子表，因为这样更方便标号和引用。为了简便，本章约定允许对子表和原子进行标号，**每当重复出现一个 标号，就可以代替对应于这个标号的元素**——例如图 12.5 中表的括号表示法可以写成 $(L_1 : (a,b),(L_1,c,L_2 : (d)),(L_2,e,L_3 : (f,g)),L_3)$，展开就是 $((a,b),((a,b),c,(d)),((d),e,(f,g)),(f,g))$。（原书图 12.5 上方的标号式把第一项写成 $(L_1 : (a,b))$，多了一层括号；按图中的树和展开式，根的第一个孩子就是 L_1 本身。）

循环表（cyclic list）也称「递归表」，对应于包含回路的有向图：它允许表中某个子表直接或者间接地 把自己作为表或者其子表的一个表目；**为了用括号表示法写出这样一个表，需要使用标号**。循环表也可以算作一种存在回路的特殊再入表。

「**深度**」是指展开表之后所含括号的层数：**循环表的深度为无穷大**，线性表的深度为 1，图 12.4 和 图 12.5 的深度都是 3。

根据广义表的不同形式，可以用线性表、树和图数据结构、或者这些数据结构的修改形式来实现。因为纯表等价于树，可以使用树形结构来表示；同理，再入表、循环表可以用图来表示。此外，还可以用 类似于树形结构的方法：**每个结点记录本层的下一个兄弟** `next` **指针，而子表结点则另有一个域** `sublist` **指向整条子表**，结点形式为 `[TAG=0 或 1 | data/sublist | next]`，其中 `TAG` 取 `ATOM`（0）表示单元素、取 `LIST`（1）表示子表。

但是这种没有表头结点的结构，在子表头插入或者删除结点时会比较复杂。例如图 12.7 中不能直接 删除元素 `data` 所在的结点，因为它被两个结点所引用：首先必须周游整个广义表，找出所有指向 `data` 的指针，然后再逐一加以修改，使得这些指针指向结点 D_2。

与带头结点的单链表类似，可以增加一个表头结点。当 tag 取值为 HEAD（原书取 −1）时表示头 结点，不存储任何表元素；**而且可以增加一个 ref 域来记录该子表被引用的次数，以便删除操作使用**；此外还可以增加 name 域，以便给子表命名。

结点通常有一个标记位，区分原子和子表：原子结点存值，子表结点存指向另一个表的指针。再加表头结点，可以把空表和共享关系表示得更干净。

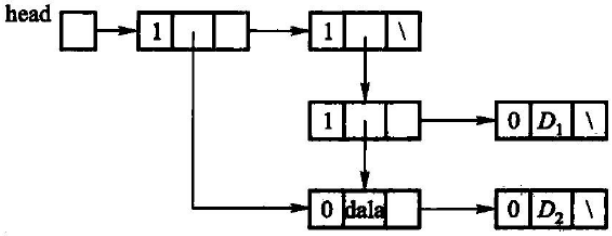


图 12.7　没有头结点的广义表。

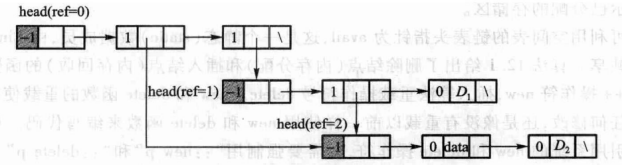


图 12.8　带头结点的广义表：多一个不存数据的表头结点，空表和非空表就不必分两套写法——与第 2 章链表引入头结点是同一个理由。

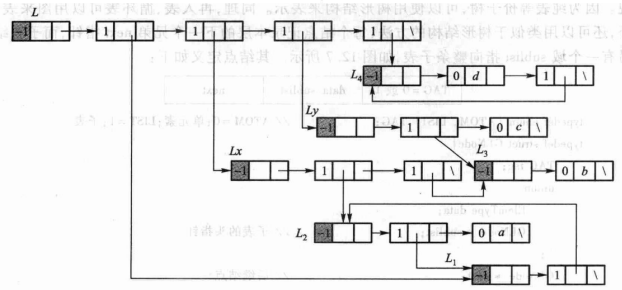


图 12.9　带表头结点的循环广义表。

先跑一遍：

```
#include "modern.hpp"

#include <cstdio>

int main() {
    using dsa::advanced::GenList;
    const GenList list = GenList::parse("(a,(b,c),d)");
    std::printf(" 表      : %s\n", list.to_string().c_str());
    std::printf(" 表头    : %s\n", list.head()->to_string().c_str());
    std::printf(" 表尾    : %s\n", list.tail()->to_string().c_str());
```

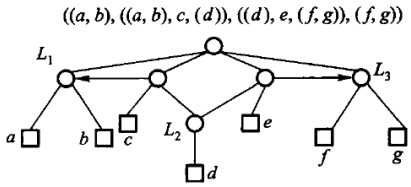
```
    std::printf(" 长度 %zu，深度 %zu，原子 %zu 个\n",
                list.length(), list.depth(), list.atom_count());

    // 再入表：同一个子表挂到两处，靠引用计数而不是拷贝。
    const GenList shared = GenList::parse("(b,c)");
    const GenList host = GenList::cons(shared, GenList::cons(shared, GenList()));
    std::printf(" 共享后  : %s，被引用 %zu 次\n", host.to_string().c_str(),
    ↪  shared.use_count());
    return 0;
}
```

```
表      : (a,(b,c),d)
表头    : a
表尾    : ((b,c),d)
长度 3，深度 2，原子 4 个
共享后  : ((b,c),(b,c))，被引用 3 次
```

最后一行是本节的难点：`(b,c)` 只有一份，挂在两处，引用计数是 3（自己一份，两个宿主各一份）。共享一旦发生，回收就**不能再按树递归** `delete`——那会把同一个结点删两次。所以结点上带计数，句柄负责加减：

$$((L_1:(a,b)),(L_1,c,L_2:(d)),(L_2,e,L_3:(f,g)),L_3)$$



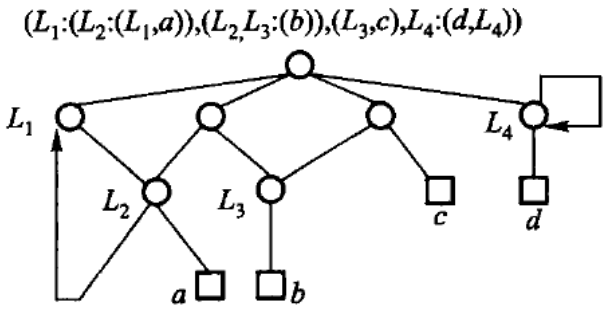

```
static void retain(GenNode* node) noexcept {
    if (node != nullptr) {
        ++node->refs;
    }
}

static void release(GenNode* node) noexcept {
    // 计数归零才真正删除；共享的子表因此只会被删一次。
    while (node != nullptr && --node->refs == 0) {
        GenNode* const head = node->head;
        GenNode* const tail = node->tail;
        delete node;
        // 表尾用循环走，长表不会把栈压穿；表头递归，深度由嵌套层数决定。
        release(head);
        node = tail;
    }
```

```
}
```

`release` 沿表尾迭代、只对表头递归，因此三万个元素的长表析构不会压穿栈，栈深度只跟嵌套层数走。这里不用 `shared_ptr`：12.2 要教的就是「谁来回收共享结点」，交给标准库这一节就没了。

引用计数收不回**环**。构造函数自底向上建表，本书的接口造不出循环表；`code/ch12/storage_recovery` 用**根集**和真正的对象图（每个对象可以有多条出边）演示标记–清除如何回收无根环，见 12.2.4。它不冒充真实运行时垃圾回收器。



12.2.2 可利用空间表

原书用重载 `operator new/delete` 和一条全局 `avail` 链实现结点复用。所有对象共享隐式全局状态，生命周期结束时还要 `::delete` 整条链。现代实现改成显式的索引句柄池：`acquire` 从空闲栈弹出一个下标，`release` 把它推回去；耗尽返回 `nullopt`，重复/越界归还返回 `false`。释放后的下标失效，`get` 得到空指针。

“可利用空间表”（free list）是固定大小对象池的一种历史叫法。工程中还会按对象大小使用 arena、pool 或 slab allocator；它们都把分配路径和所有权边界显式化，避免教材示例那种全局 `avail` 状态。

普通二叉搜索树只要求中序遍历有序，树形可能很多。若键的查找频率不同，应把常查的键放得更靠近根。最优 BST 的输入包括成功查找权 `p[1..n]` 与失败查找权 `q[0..n]`；动态规划对每个区间尝试每一个键作根，选择「左子树代价 + 右子树代价 + 本区间总权」最小的方案。

教材样例 `p = {1,5,4,3}`、`q = {5,4,3,2,1}` 的总成本是 57，根是第 2 个键。`cost[i][j]` 是区间代价，`root[i][j]` 记录取得最小值的根，因而还能按根表重建树形。朴素实现为 $O(n^3)$。

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
```

```
    dsa::advanced::ReusableNodePool<int> pool(2);
    const auto first = pool.acquire(11);
    const auto second = pool.acquire(22);
    std::cout << " 申请到槽 " << *first << " 和 " << *second
              << ", 剩余 " << pool.available() << '\n';
    pool.release(*first);
    const auto reused = pool.acquire(44);
    std::cout << " 归还后再申请得到槽 " << *reused
              << ", 值为 " << *pool.get(*reused) << '\n';

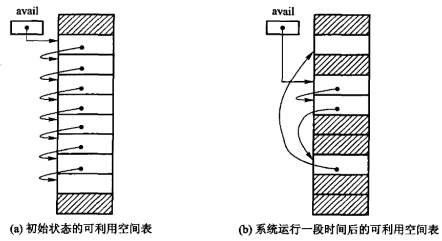
    const auto tree = dsa::advanced::optimal_bst({1, 5, 4, 3}, {5, 4, 3, 2, 1});
    std::cout << " 最优 BST 总成本 " << tree.cost[0][4]
              << ", 根为键 " << tree.root[0][4] << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch12/optimal_bst \
    code/ch12/optimal_bst/demo.cpp -o /tmp/bst-demo
/tmp/bst-demo
```

```
申请到槽 0 和 1，剩余 0
归还后再申请得到槽 0，值为 44
最优 BST 总成本 57，根为键 2
```

把 `optimal_bst({1,2}, {3,4})` 这种长度对不上的输入送进去，会抛 `std::invalid_argument`。空树对应 `p = {}`、`q = {某个失败权}`，代价为 0。

池用 `vector<optional<T>>` 表示槽位占用，`vector<size_t>` 当空闲栈。构造时下标从大到小入栈，所以第一次 `acquire` 拿到 0。`release` 先确认该槽确实被占用，再 `reset` 并归还。



(a) 初始状态的可利用空间表　(b) 系统运行一段时间后的可利用空间表

容量为 3 时，状态可以直接写成一张账：初始空闲栈顶依次是 `0,1,2`；申请 A、B 后，句柄分别是 0、1，只剩 2；归还 0 后，空闲栈是 `0,2`；再申请 C 会复用 0。旧句柄 0 在归还那一刻已经失效，数值虽然后来又出现，却代表新的占用期。真实系统若要识别「拿旧句柄误访问新对象」，还会在句柄中加入代数（generation）；本节的简化池只保证空闲槽不可读和重复释放会失败。

动作	槽 0	槽 1	槽 2	下一可用
初始	空	空	空	0
`acquire(A)`	A	空	空	1
`acquire(B)`	A	B	空	2
`release(0)`	空	B	空	0
`acquire(C)`	C	B	空	2

这个表也解释了为什么 `release` 不能只把下标推回空闲栈：若不先确认槽位处于占用状态，重复归还 0 会让空闲栈出现两个 0，接下来的两次申请就会拿到同一个槽 0。

```
template <typename T>
class ReusableNodePool {
public:
    explicit ReusableNodePool(std::size_t capacity) : slots_(capacity) {
        for (std::size_t index = 0; index < capacity; ++index) {
            free_.push_back(capacity - index - 1);
        }
    }

    [[nodiscard]] std::optional<std::size_t> acquire(const T& value) {
        if (free_.empty()) {
            return std::nullopt;
        }
        const std::size_t index = free_.back();
        free_.pop_back();
        slots_[index] = value;
        return index;
    }

    bool release(std::size_t index) {
        if (index >= slots_.size() || !slots_[index]) {
            return false;
        }
        slots_[index].reset();
        free_.push_back(index);
        return true;
    }

    [[nodiscard]] const T* get(std::size_t index) const noexcept {
        if (index >= slots_.size() || !slots_[index]) {
            return nullptr;
        }
        return &*slots_[index];
```

```
    }

    [[nodiscard]] std::size_t available() const noexcept { return free_.size(); }

private:
    std::vector<std::optional<T>> slots_;
    std::vector<std::size_t> free_;
};
```

12.2.3 存储的动态分配和回收

12.2.2 节的可利用空间表只针对一**种长度**的空闲块，但实际应用中要求被分配的空闲块有多种长度—— 一种常见的情况是动态分配内存时要求为整型、浮点型、双精度浮点型等类型的变量分配存储空间；而且那种做法也不能很好地解决存储区共享的问题。

本节把不同长度的结点（空闲块）组织在一个可利用空间表中。分配时并不是所有结点都能满足，而需要按申请的长度进行检索，找到其空间大于等于申请长度的结点，从中截取合适的长度；**回收时也 不能简单地释放，而必须考虑刚刚被删除的结点空间能否与其他结点合并**，组成较大的结点，以便能满足 后来较大长度结点的分配请求。这种处理方法的优点是可以解决存储空间的共享问题，缺点是分配和回收 的算法比较复杂，**并且在系统长期动态运行的过程中，这种方法有可能使整个空间被分割成许多大小 不等的碎块，而某些碎块由于太小而长期得不到使用，这就产生了所谓的碎片问题**。

空闲块要自带「身份证」。因为空闲块的长度是不定的，所以可利用空间表中每个结点都需要记录本结点 的长度。一种常见的方法是：对于一个需要 m 字节空间的请求，存储管理器可能会分配稍多于 m 字节的 空间，**额外的空间留给存储管理器进行存储管理**，例如存放块的标记位、链表指针和块长度；标记位用来区别这个块是空闲块还是已被分配的块。另外，**空闲块的分配和回收不是简单地在可利用空间表的一端 进行，可利用空间表不再是一个堆栈结构**——事实上，为了检索方便，通常用双链表来实现它。

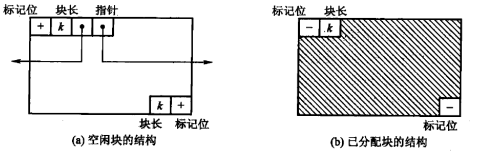
两种碎片，来自两种做法。如果能够找到大于 N 字节的空闲块：一种做法是**整个块交给请求者**，把首尾标记位都改为「已分配」；另一种做法是对于长度为 $K > N$ 的空闲块，**把 N 字节分配给请求者、剩余的 $K - N$ 字节形成一个新的空闲块**。不管采用哪种做法，都会存在碎片（fragmentation）问题：第一种做法把多于请求字节数的空间分配给请求者，会浪费空闲块的空间，从而产生**内部碎片**（internal fragmentation）；第二种做法则会产生大量的不能满足一般性请求的小空闲块，称为**外部碎片**（external fragmentation）。

实际应用中，有的存储管理器牺牲内部碎片的空间，使得存储管理更加简单——例如文件管理系统中 以簇为基本单位进行文件空间的分配，就牺牲了内部碎片从而避免了外部碎片。

下面这三种可能产生外部碎片的方法通称为顺序适配（sequential fit）：首

先适配（first fit）、最佳适配（best fit）和最差适配（worst fit）。三者只在「挑哪一块」这一步不同，分裂与合并完全一样。

- **首先适配**从可利用空间表的表头开始顺次搜索，一旦找到第一个长度大于等于请求长度的空闲块就 分配。**一种简单改进是：记住前一次搜索到达的位置，从该位置开始新的搜索**，搜到链表尾部时重新 从表头开始（今天通称 next fit）。优点是速度快，缺点是可能把较大块拆分成较小的块，导致后来难以 满足对大块的申请。
- **最佳适配**在所有大于等于请求长度的块中找出最小的一块进行分配。优点很显然：**它可以使得无法 满足大请求块的可能性降到最低**；但它可能使得外部碎片问题变得非常严重，因为那些满足请求后剩下 的空闲块非常小，对将来的请求已没有什么用处。实现有两种方案——检索整个无序的可利用空间表找到最小的合适块，或者**把空闲块按照从小到大的顺序排列成优先队列**，使得检索时只需要从表头往后 查看。
- **最差适配**每次分配当前最大的块，指望剩下的部分还大到能再用一次。

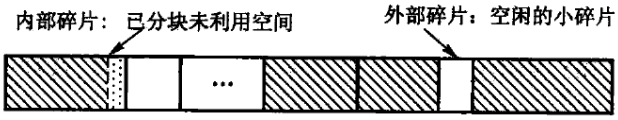


差别用一个例子最清楚。设空闲区依次是 500、200、300、600 字节（中间被已分配块隔开，所以不会合并），现在要 212 字节：

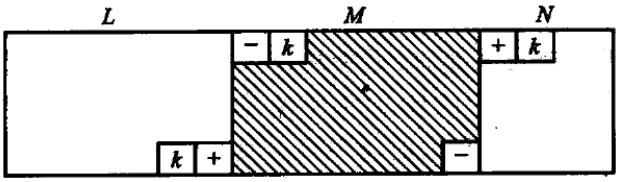
策略	挑中	剩下的碎片	代价
首次适应	500 那块	288	找到就停，扫得最少；小碎片在低地址堆积
最佳适应	300 那块	88	大块留住了，但剩下的碎片最碎、最难再用
最坏适应	600 那块	388	剩下的还大到能再分一次；大块很快被拆光

`code/ch12/memory_allocator` 实现了这三种，测试用的正是上面这组数字。**只在一块空闲区上测是分辨不出三种策略的**——那种布局下三者必然挑中同一块，把最佳适应和最坏适应的判据对调，测试照样全绿。

无论哪种策略，相邻的空闲块都应当合并，否则外部碎片只会越来越多。「外部碎片」是可以直接量出来的：释放出两个不相邻的 30 字节空洞后，空闲总量有 60 字节，但最大的一块只有 30——45 字节的请求就会失败；把中间那块也释放掉、三块并成一整块之后，同样的请求立刻能满足。



合并要分四种位置关系。设释放块为 M：左右都忙，M 单独入空闲表；只有左边空闲，扩大左块；只有右边空闲，把右块的起点移到 M；左右都空闲，把三块合成一块。顺序表实现可以检查前后项，边界标记则从 M 的块头、块尾直接找到物理邻居。

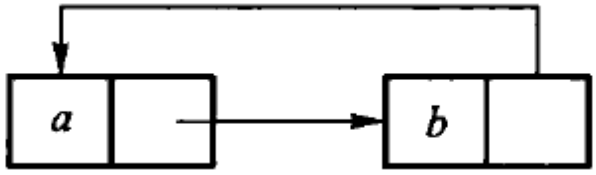


原书提到的**边界标记**（块头块尾各记一份大小与忙闲）解决的是「怎么在 $O(1)$ 内找到物理相邻的邻居」。本书的实现把块表按偏移升序排列，相邻块就是表里相邻的项，表达的是同一个想法；代价是定位要扫表而不是 $O(1)$，这一点写在实现的注释里，不冒充。

12.2.4 失败处理策略和无用单元回收

分配失败时可以：直接拒绝；压缩（把已分配块搬到一起，挤出一整块空闲）；或者做垃圾回收，把程序已经够不着、却还占着的块收回来。

引用计数给每个对象记有多少指针指着它，减到零就释放。实现简单，但对象互相指形成环时，计数永远掉不到零，这块内存就漏了。标记–清除从一组根（栈上的指针、全局变量）出发走遍所有还能碰到的对象并打上标记，然后扫一遍堆，没标记的统统回收。它能处理环，但要求能从根集走到每一个活对象，并且要能区分「这是指针」和「这只是一个看起来像地址的整数」。



例如根集只有 `R -> A -> B`，另有 `C -> D -> C`。标记栈先压 A，继而标 A、B；C、D 虽然各有一个入边，却从任何根都走不到。清除阶段保留 A、B，回收 C、D。这里判生死的是**可达性**，不是入度或引用数；这正是循环引用能被整组收回的原因。

阶段	待处理栈	已标记	动作
放入根	`A`	A	根先标记再入栈
弹出 A	`B`	A、B	沿 `A -> B` 发现 B

阶段	待处理栈	已标记	动作
弹出 B	空	A、B	标记阶段结束
清扫	空	A、B	保留 A、B；回收 C、D

一定要「先标记、再入栈」。若等到弹出时才标，环或菱形共享会把同一结点反复压栈；结果也许仍对，空间和时间却可能失控。

本章的句柄池把「失败」做成返回 `nullopt`，不劫持全局 `new`，也不假装自己是通用垃圾回收器。

`code/ch12/storage_recovery` 把标记–清除做成可运行的：出边是一张表（一个对象可以指向多个对象），`collect()` 接受**一组**根。它演示的核心是「可达性是从根走出来的，与有多少人指着我无关」——一个被三处引用、却谁都够不着根的对象，引用计数收不回，标记–清除照收。

标记阶段用**显式栈**而不是递归：垃圾回收恰恰是在内存吃紧时跑的，那时最不该再去吃调用栈（同 2.3.1 节「所有权工具怎么选」的判据）。测试里有 20 万个对象的深链，递归标记必然压穿栈。

它仍然是教学模型：没有栈扫描、没有写屏障、没有分代，也不区分「这是指针」和「这只是个看起来像地址的整数」——而那恰恰是真实垃圾回收器最难的部分。

12.3 Trie 结构和 Patricia 树

为什么这一节的存储侧只有 C++

本节的 C++ 版本属于存储侧：它把 Trie 结点、Patricia 内部结点布局、共享前缀回收和按位分支不变量都变成可观察的对象。Python 版本保留，是为了对照查找、前缀计数等算法行为；它不是 C++ 所有权和布局的逐行替代，Python `dict` 或垃圾回收会把那部分教学内容隐藏掉。因此两种语言在这里承担的不是同一层任务：算法结果可对拍，存储管理仍以 C++ 为准。

先说清楚 Trie 与 BST 分属两种「分解」。第 5 章的 BST 是一种组织上基于**对象空间**（object space）分解的数据结构，即**关键码范围的分解是由存储在树中的对象（关键码值）决定的**；BST 并不是一种平衡 的树，其树结构与输入数据的顺序有很大关系——按升序输入会退化成线性表，查找一个关键码的时间 代价为 $O(n)$。

若要消除这样的不平衡，最根本的办法就是改变关键码的划分方式：在关键码范围内，对树结构中每一个 结点预定义划分位置。假设划分因子为 2（每次仅把当前范围分为两部分，对应于二叉树）：根结点把整个 关键码范围分为两部分，插入时不考虑插入的顺序，小于根的关键码都放到左子树、大于根的放到右子树。**按照此规则所产生的树不依赖于关键码的插入顺序，其深度只受关键码精度的影响**，因此最坏情况下就 等于存储关键码所需要的位数：如果关键码是 $0 \sim 255$ 之间的整数，精度就是 8 个二进制位，两个只在 最低位不同的关键码要到第 8 次

划分才能分开，这棵搜索树深度也为 8。

这种对关键码范围进行均分的方法称为关键码空间（key space）分解，基于关键码分解的数据结构叫做 Trie 结构。「trie」这个词来源于「retrieval」，暗示了它的主要应用：信息检索（information retrieval）。这种数据结构被很多人用来存储英文字符串，尤其是大规模的英文词典，在自然语言理解系统中经常用到。**与 B+ 树一样，Trie 树内部结点仅作为占位符引导检索过程，数据记录只存储在叶结点中**。

但这种结构并不能保证树是平衡的：考虑一种极端情况，如果所选择的根结点（即预先定义的划分位置）不合适，导致所有的关键码都小于根结点，那么这棵树的右子树将没有任何元素——这是因为关键码分布得很不均衡。例如存英文词典时，以 `t` 开头的单词远多于以 `z` 开头的单词，`t` 子树又深又大，`z` 侧几乎为空。

Trie 树基于两个原则：关键码集合是固定的；能够对结点进行分层标记。如果所有的元素都可以使用 数字或者字母等来标记，就可以考虑使用 Trie 结构——例如元素可以用 $0 \sim 9$ 的数字组合来标记时，就可以在根结点分出 10 个子结点分别标记 $0 \sim 9$，然后每个子结点又可以分出 10 个结点，如此下去 直到所有的元素都能够被区分开。

二叉搜索树按整个关键码比较，树的形状依赖插入次序。Trie 换一种切法：按关键码的字符（或二进制位）一层一层分支，第 i 层对应第 i 个字符。公共前缀在树里只存一次，所以它特别适合字典、IP 路由这种「按前缀分类」的问题。查找沿着字符走，时间与关键码长度成正比，与表里有多少个词关系不大。

最简单的形式是**二叉** Trie：按关键码所在的数值区间对半分，左枝标 0、右枝标 1。

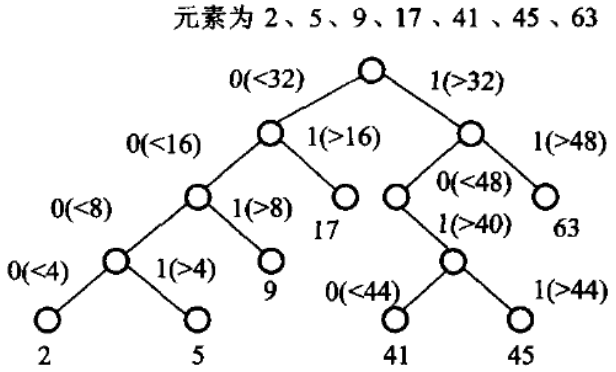
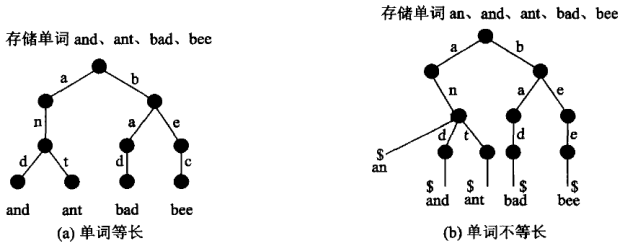


图 12.16　二叉 Trie 树。最大元素 63，所以关键码范围取 $[0, 64)$；根把范围劈成 $[0, 32)$ 和 $[32, 64)$，于是 2、5、9、17 进左子树，41、45、63 进右子树；再往下继续对半分，直到每个元素单独落在一个叶上。同一棵子树下的关键码共享父结点代表的前缀——第 5 章的 Huffman 编码树也有这个性质，**Huffman 树就是一种二叉** Trie。

把 `can`、`car`、`cat`、`do` 插进一棵字母 Trie，公共前缀 `ca` 只出现一次：



(a) 单词等长　　(b) 单词不等长

```
        ·
       / \
      c   d
      |   |
      a   o*
     /|\
    n* r* t*
```

带 * 的是词尾。查找 car 走 c–a–r，三步；查找 cab 在 b 处没有分支，失败。最长前缀匹配（路由）则走到不能再走为止，回退到最近的词尾。

先跑一遍。上面那棵图里的结点数不是数出来的，是程序报的：

```cpp
#include "modern.hpp"

#include <cstdio>

int main() {
    dsa::advanced::Trie trie;
    dsa::advanced::PatriciaTree patricia;
    for (const char* word : {"can", "car", "cat", "do"}) {
        trie.insert(word);
        patricia.insert(word);
    }
    std::printf("Trie     : %zu 个词，%zu 个结点（字符总数 11）\n",
                trie.size(), trie.node_count());
    std::printf("Patricia : %zu 个词，%zu 个内部结点\n",
                patricia.size(), patricia.internal_count());
    std::printf(" 前缀 ca 下有 %zu 个词：", trie.count_with_prefix("ca"));
    for (const auto& word : trie.keys_with_prefix("ca")) {
        std::printf("%s ", word.c_str());
    }
    std::printf("\n最长前缀匹配 dozen -> %s（走不动就回退到最近词尾）\n",
                trie.longest_prefix_of("dozen").c_str());
    return 0;
}
```

```
Trie     : 4 个词，7 个结点（字符总数 11）
Patricia : 4 个词，3 个内部结点
前缀 ca 下有 3 个词：can car cat
最长前缀匹配 dozen -> do（走不动就回退到最近词尾）
```

第一行就是前缀共享的全部价值：四个词一共 11 个字符，树里只有 7 个结点，因为 `ca` 只存了一次。最长前缀匹配的写法就是「一路往下走，随手记住最近一次经过的词尾」：

```cpp
/// 最长前缀匹配：走到走不动为止，回退到最近的词尾。IP 路由查表就是这个动作。
[[nodiscard]] std::string longest_prefix_of(std::string_view text) const {
    const Node* node = &root_;
    std::size_t best = 0;
    for (std::size_t i = 0; i < text.size(); ++i) {
        if (!is_letter(text[i])) {
            break;
        }
        const Node* next = node->children[index_of(text[i])].get();
        if (next == nullptr) {
            break;
        }
        node = next;
        if (node->terminal) {
            best = i + 1;
        }
    }
    return std::string(text.substr(0, best));
}
```

字母 Trie 的每个内部结点原则上要留 26 条分支，大部分是空的——空间是它的主要代价。原书给出的一种改进是把内部结点上那张定长的分支表换掉：

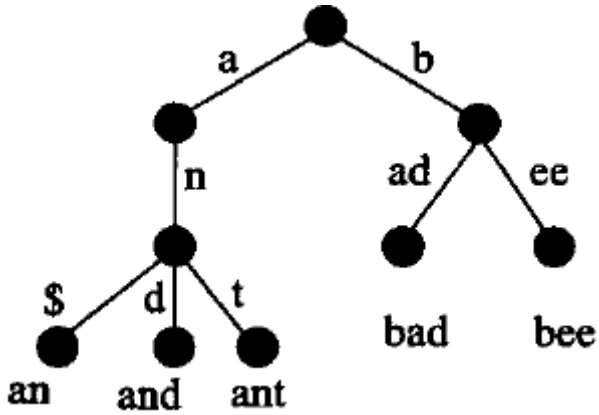


图 12.18　对 Trie 树的改进：只保存实际用到的分支，代价是查一层要在表

里找一次，不再是直接下标。

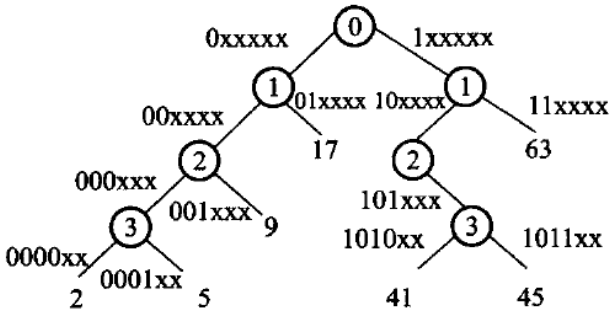
纯 Trie 在「只有一个孩子」的内部结点上仍然分支，路径偏长。Patricia 树把这种单孩子结点压缩掉，边上记下「跳过几位再比」。查找时按记下的位位置取关键码的那一位，决定走左还是走右。路径更短，结点更少，仍然保持前缀共享——同样四个词，Patricia 只要 3 个内部结点。

按位取值和「两个关键码第一次不同在第几位」是 Patricia 的两块基石：

```
static bool bit_of(std::string_view key, std::size_t index) noexcept {
    const std::size_t byte = index / 8;
    if (byte >= key.size()) {
        return false;  // 越过关键码长度，一律读 0
    }
    const auto value = static_cast<unsigned char>(key[byte]);
    return ((value >> (7 - index % 8)) & 1U) != 0;
}

static optional_bit first_differing_bit(std::string_view a, std::string_view b) {
    const std::size_t longest = a.size() > b.size() ? a.size() : b.size();
    const std::size_t bits = (longest + 1) * 8;  // +1 让「一个是另一个的前缀」也能分开
    for (std::size_t i = 0; i < bits; ++i) {
        if (bit_of(a, i) != bit_of(b, i)) {
            return {true, i};
        }
    }
    return {};
}
```

越过关键码长度的位一律读作 0，这样「一个关键码是另一个的前缀」（a 与 ab）也能分开，代价是关键码里不能出现 `'\0'`。还有一处不能省：沿位下降到叶之后，**必须和叶上的完整关键码再比一次**——路上只看了少数几位，不比就会把 ca、cars 这种根本不在表里的串判成命中。



编码：2: 000010 5: 000101 9: 001001
17: 010001 41: 101001 45: 101101 63: 111111

图 12.19　压缩的依据：x 表示该位取 0 取 1 都落在同一枝里，也就是这一

位**不需要用来分支**——把这样的层去掉就是 Patricia。

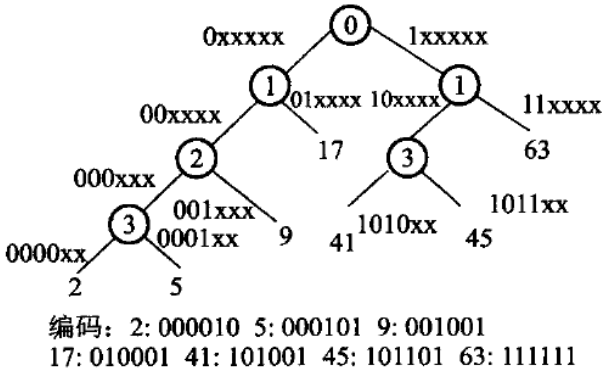


图 12.20　压缩后的 PATRICIA Trie。内部结点不再逐位下降，而是各自记住「该比第几位」。

以单字节 `'a'` 与 `'c'` 为例，ASCII 分别是 `01100001` 和 `01100011`，第一次不同在从 0 起算的第 6 位。Patricia 内部结点只记这个分歧位：该位为 0 走一边，为 1 走另一边；前面六个相同位不再各占一层。再插入键时，先走到叶并找出首个不同位，再把新分支插到位号仍保持递增的位置，否则查找路径会跳过应比较的位。

12.4 改进的二叉搜索树

12.4.1 最佳二叉搜索树

同一个关键码集合，插入次序不同，长出来的二叉搜索树就不同。原书拿 $K = \{$ xal, wan, wil, zol, yo, xul, yum, wen, wim, zi, yon, xem, wul, zom $\}$ 举例：

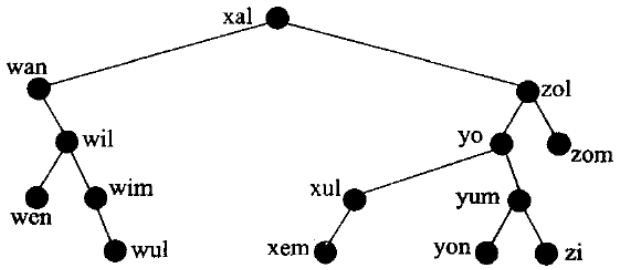


图 12.21　按 K 的给定次序插入得到的二叉搜索树。

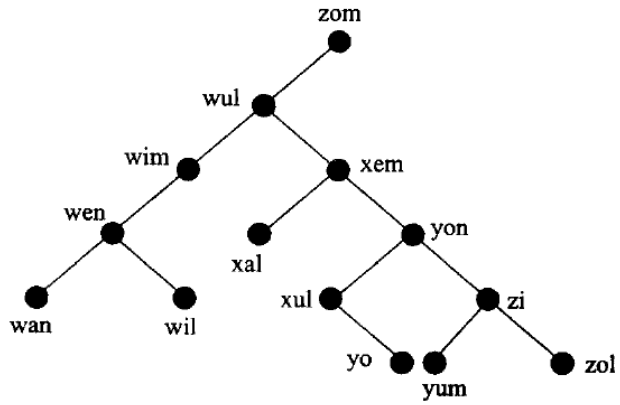


图 12.22　同一批关键码倒着插入得到的另一棵 BST。n 个关键码有 $n!$ 种插入次序，但不同次序可能长出同一棵树，真正不同的形状只有 $C_{2n}^n/(n+1)$ 棵（组合数学里的 Catalan 数）。问题因此变成：这些形状里哪一棵检索效率最好?

先数一数「一共有多少棵形状不同的 BST」。这个问题等价于：当二叉树的中序排列是 $\{1,2,3,\cdots,n\}$ 时，这棵二叉树有多少种可能的前序排列。可以证明，$n!$ 种排列中只有 $\frac{1}{n+1}C_{2n}^n$（组合数学中的 Catalan 数）种前序排列能够构成二叉搜索树——例如 $\{2,3,1\}$ 就不是任何二叉搜索树的前序排列。通常用检索效率来评价这 $C_{2n}^n/(n+1)$ 棵树，问什么形状的二叉搜索树比较好。

衡量的办法是把树扩充成满二叉树（5.1.2 节），空子树补成外部结点：

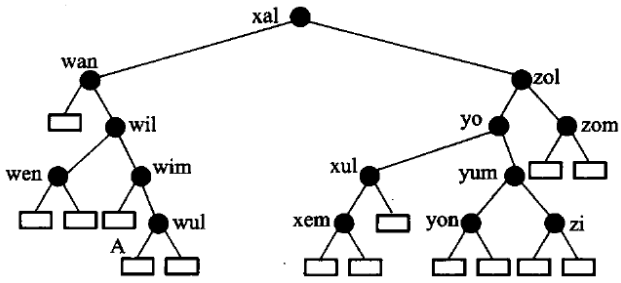


图 12.23　图 12.21 的扩充二叉树，小方框是外部结点。**每个外部结点代表一类检索失败**——例如图中的 A 代表所有取值落在 wim 与 wul 之间的关键码。于是「检索成功的代价」落在内部结点上，「检索失败的代价」落在外部结点上，两者可以一起算进同一个目标函数。

把代价写成公式。对于 n 个结点的二叉树，**外部路径长度** E 定义为从扩充二叉树的根到每个 外部结点的路径长度之和，**内部路径长度** I 定义为从根到每个内部结点的路径长度之和，两者满足 $E=I+2n$。回到检索效率：从根开始比较，若不等就进入下面一层继续比较，因此**成功检索的比较 次数就是关键码所在的层数加 1**（根为第 0 层），**不成功检索的比较次数就等于被检索的关键码所属的 那个外部结点的层数**。于是检索一个关键码的平均比较次数为

$$\mathrm{ASL}(n)=\frac{1}{W}\left[\sum_{i=1}^{n}p_i(l_i+1)+\sum_{i=0}^{n}q_i l'_i\right],\quad W=\sum_{i=1}^{n}p_i+\sum_{i=0}^{n}q_i$$

其中 l_i 是第 i 个内部结点的层数，l'_i 是第 i 个外部结点的层数，p_i 是检索第 i 个内部结点所代表 的关键码的频率，q_i 是被检索的关键码属于第 i 个外部结点所代表的可能关键码集合（即处于第 i 个 和第 $i+1$ 个内部结点之间）的频率。p_i、q_i 也叫做结点的**权**。**把 ASL(n) 最小的那棵 二叉搜索树称做最佳二叉搜索树**。

先看一种特殊情况：所有结点的权都相等。此时 $p_i/W = q_i/W = 1/(2n+1)$，代入化简（用 $E=I+2n$）得 $\mathrm{ASL}(n)=(2I+3n)/(2n+1)$——**要平均比较次数最小，就是要内部路径长度 I 最小**。在一棵二叉树中，路径长度为 0 的结点仅有一个，为 1 的至多两个，为 2 的至多 4 个..... 因此 n 个结点的二叉树其内部路径长度 I 至少等于序列 $0,1,1,2,2,2,2,3,3,\cdots$ 的前 n 项和，即 $\sum_{k=1}^{n}\lfloor\log_2 k\rfloor$。这个最小的 I 值显然在下面这样的二叉树里可以达到：**只有最下面两层结点的度数可以小于 2，其他结点度数必须等于 2**——在所有结点权相等的情况下，这样的二叉搜索树就是最佳二叉搜索树，其 ASL(n) 是 $\Theta(\log_2 n)$ 量级的。构造办法也很直白：先把关键码排序，然后用二分法依次检索这些关键码，把检索中遇到的、树中还没有 的关键码依次插入。

各关键码被检索的概率通常不等，把概率（或频次）当权，目标就是**带权路径长度最小**：

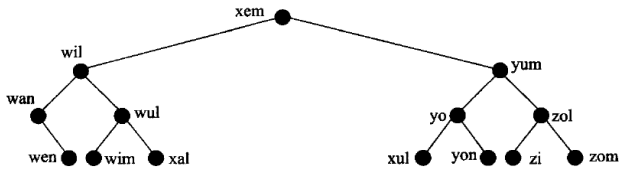


图 12.24　等权时的最佳二叉搜索树——形状尽量平衡。

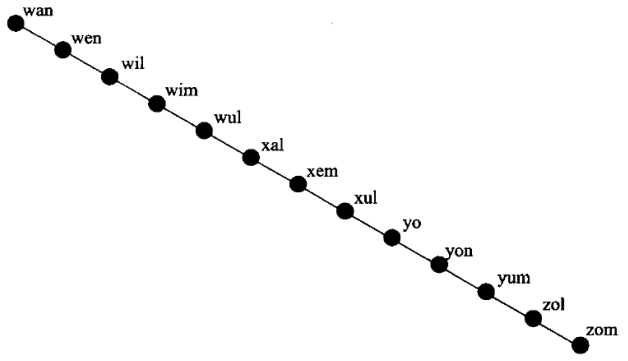


图 12.25　最坏的形状：退化成一条链，检索代价 $O(n)$。

那么按任意顺序把关键码插入 BST，检索效率如何呢？平均比较次数是接近

最坏的 $O(n)$，还是接近最好 的 $O(\log_2 n)$? 原书的说法是:「对 $C_{2n}^n/(n+1)$ 种二叉搜索树进行平均，得到的平均检索次数仍是 $O(\log_2 n)$——**即大多数的二叉搜索树和最佳二叉搜索树检索效率差别不大，只有少数情况下检索的 平均比较次数为 $O(n)$**。」结论的方向对，但**平均的对象说错了**：题目问的是「按任意顺序插入」，应当对 $n!$ 种插入次序等概率平均——这时平均比较次数约为 $2\ln n \approx 1.39\log_2 n$，确实是 $O(\log_2 n)$；若改成对 $C_{2n}^n/(n+1)$ 种**形状**等概率平均，结点的平均深度却是 $\Theta(\sqrt{n})$（约 $\sqrt{\pi n}$）。两者不同，是因为不同插入次序长出同一形状的次数差别很大：平衡的形状对应 许多种次序，链状的形状只对应极少几种。例如 $n = 400$ 时，按次序平均的结点深度约 9.2，按形状平均约 32.6。

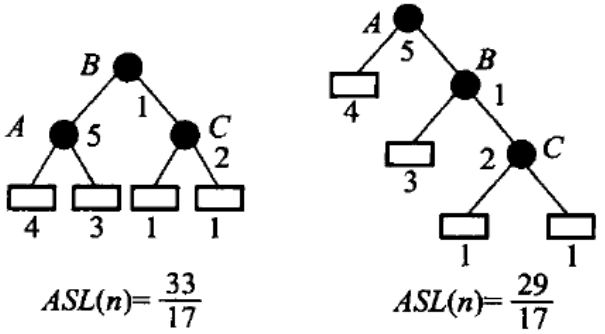


图 12.26　结点权不等时，**最佳的形状未必最平衡**：权大的关键码要往根靠，哪怕整棵树因此歪一些。这正是需要动态规划、而不是简单「取中位数当根」的原因。

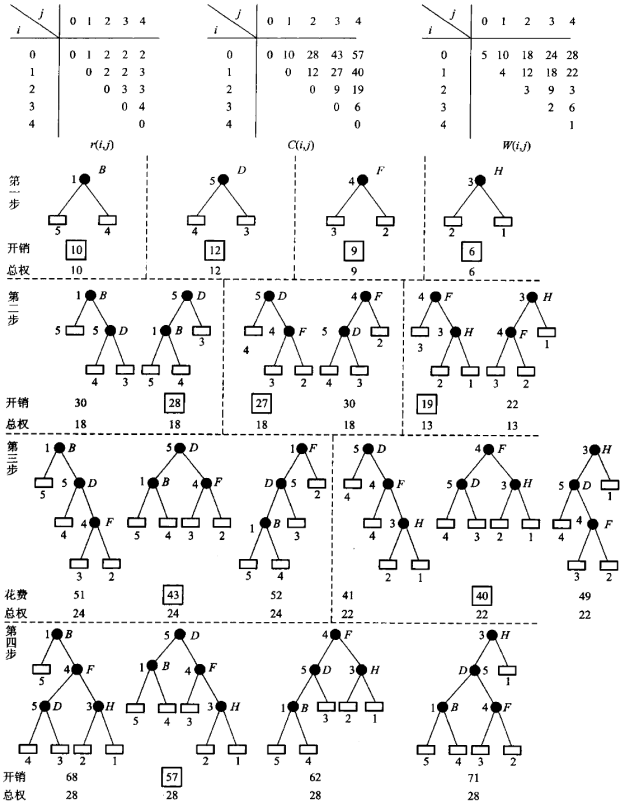


图 12.27　构造最佳二叉搜索树的过程：按区间由短到长填表，每个区间枚举根。

动态规划为什么成立，原书讲得很清楚：最佳二叉搜索树有一个特点——其任何子树都是最佳二叉搜索 树。利用这一事实，可以从底层逐步构造最佳二叉搜索树：先构造包含一个结点的最佳二叉搜索树，再 构造包含两个结点的，…，直到把所有的结点都包含进去；**后来构造的较大的最佳二叉搜索树用前面 构造的较小的最佳二叉搜索树作为其子树**。

设包含关键码 $key_{i+1}, key_{i+2}, \cdots, key_j$ 为内部结点（$0 \le i \le j \le n$）、结点的权为 $(q_i, p_{i+1}, q_{i+1}, \cdots, p_j, q_j)$ 的最佳二叉搜索树为 $t(i,j)$，其根为 $r(i,j)$，其开销为 $C(i,j)$，其权的总和为 $W(i,j) = p_{i+1} + \cdots + p_j + q_i + q_{i+1} + \cdots + q_j$。第一步构造包含一个结点的 最佳二叉搜索树，即找 $t(0,1), t(1,2), \cdots, t(n-1,n)$；第二步构造包含两个结点的，即找 $t(0,2), t(1,3), \cdots, t(n-2,n)$；再构造包含 3 个、4 个结点的，…，直到最后构造 $t(0,n)$。

如何构造 $t(i,j)$? 根据「最佳二叉搜索树的子树必是最佳二叉搜索树」的原理可知

$$C(i,j) = W(i,j) + \min_{i<k\le j}\left(C(i,k-1) + C(k,j)\right)$$

这个式子的意思是：分别用 $key_{i+1}, key_{i+2}, \cdots, key_j$ 为根，考虑 $j-i$ 棵二叉搜索树。以 key_k 为根的二叉搜索树，其左子树包含 $key_{i+1}, \cdots, key_{k-1}$，而包含这些关键码为内部结点的 最佳二叉排序树 $t(i,k-1)$ 已在前面的步骤确定、$C(i,k-1)$ 已求出；其右子树包含 $key_{k+1}, \cdots, key_j$，$C(k,j)$ 也已求出。**对于 $i<k\le j$ 找出使 $C(i,k-1)+C(k,j)$ 最小的 那个 k_0**，以 key_{k_0} 为根、$t(i,k_0-1)$ 为左子树、$t(k_0,j)$ 为右子树的那棵二叉搜索树就是所要求的 $t(i,j)$；其开销 $C(i,j)$ 等于左子树的开销加右子树开销，**再加上结点的总的权** $W(i,j)$。

每一步构造出一棵最佳二叉搜索树 $t(i,j)$ 就记下其根 $r(i,j)$、开销 $C(i,j)$，最后构造出 $t(0,n)$ 时，就确定了整个最佳二叉搜索树的结构。

最优 BST 先校验 `q.size() == p.size() + 1`。空区间的 `cost[i][i] = 0`，`weight[i][i] = q[i]`。长度从 1 扩到 n，对每个区间 `[first, last]` 枚举根 `r`，候选代价是 `cost[first][r-1] + cost[r][last] + weight[first][last]`。

```cpp
struct OptimalBstResult {
    std::vector<std::vector<long long>> cost;
    std::vector<std::vector<std::size_t>> root;
};

inline OptimalBstResult optimal_bst(const std::vector<int>& successful,
                                    const std::vector<int>& unsuccessful) {
    if (unsuccessful.size() != successful.size() + 1) {
        throw std::invalid_argument("weight count");
    }
    const std::size_t count = successful.size();
    OptimalBstResult result{
        std::vector<std::vector<long long>>(count + 1,
                                            std::vector<long long>(count + 1, 0)),
        std::vector<std::vector<std::size_t>>(count + 1,
                                              std::vector<std::size_t>(count + 1, 0))};
    std::vector<std::vector<long long>> weight(
        count + 1, std::vector<long long>(count + 1, 0));

    for (std::size_t index = 0; index <= count; ++index) {
        // The book's c table measures internal-key comparison cost; an empty
        // interval has zero c cost while its unsuccessful weight remains in w.
        result.cost[index][index] = 0;
        weight[index][index] = unsuccessful[index];
    }
    for (std::size_t length = 1; length <= count; ++length) {
        for (std::size_t first = 0; first + length <= count; ++first) {
```

```
            const std::size_t last = first + length;
            weight[first][last] = weight[first][last - 1] + successful[last - 1] +
                                  unsuccessful[last];
            result.cost[first][last] = std::numeric_limits<long long>::max() / 4;
            for (std::size_t root = first + 1; root <= last; ++root) {
                const long long candidate = result.cost[first][root - 1] +
                                            result.cost[root][last] + weight[first]
↪   [last];
                if (candidate < result.cost[first][last]) {
                    result.cost[first][last] = candidate;
                    result.root[first][last] = root;
                }
            }
        }
    }
    return result;
}
```

动态规划为什么要按区间长度递增，可以在长度 2 的区间上看清。算 `[i,j]` 时，若选 `r` 为根，必须已经知道 `[i,r-1]` 和 `[r,j]` 的最优代价；它们都比当前区间短。每试一个根，本区间总权 `weight[i][j]` 都要再加一次，因为挂到新根下面后，区间内每种成功或失败查找的深度都增加 1。`root[i][j]` 不是附带输出，而是重建树时的路线图：先取整段根，再递归查左右两个子区间。

教材样例的关键格可以逐层核算：

区间	总权 w	最优代价 c	根
`[0,1]`	10	10	1
`[1,2]`	12	12	2
`[2,3]`	9	9	3
`[3,4]`	6	6	4
`[0,2]`	18	28	2
`[1,3]`	18	27	2
`[2,4]`	13	19	3
`[0,3]`	24	43	2
`[1,4]`	22	40	3
`[0,4]`	28	57	2

最后一格选根 2 时，候选值是 `c[0][1] + c[2][4] + w[0][4] = 10 + 19 + 28 = 57`。这既复算了演示程序的输出，也能抓住把 `weight` 漏加或多加一次的实现错误。

12.4.2 平衡的二叉搜索树

BST 在最好的情况下搜索只需要 $O(\log n)$ 的时间代价，而在最坏时却需要 $O(n)$ 的时间——若键 已经有序，它会退化成一条链。如果能够找到一种方法，使得 BST 不受输入的影响、始终保持平衡状态，就能得到很好的检索效率。基于这种想法，Adelson-Velskii **和** Landis **发明了** AVL **树**（名字取自两位 发明者姓氏的首字母），这是一种平衡的二叉搜索树：**其任何结点的左子树和右子树高度最多相差** 1。

AVL 树的三条性质：① 一棵空二叉树是一个 AVL 树；② 如果 T 是一棵具有 n 个结点的 AVL 树，则其高度为 $O(\log n)$；③ 如果 T 是一棵 AVL 树，那么其左右子树 T_L、T_R 也是 AVL 树，并且 $|h_L - h_R| \leq 1$。可以把 AVL 树看做是 BST 的一个特例。

因为 AVL 树的高度是 $O(\log n)$，所以在其上进行的搜索、插入和删除都只需要 $O(\log n)$ 的时间 代价。现在关键的问题就是如何保持一棵 AVL 树的结构，使得不论对其进行何种操作都不改变其 AVL 特性；并且，为保持这样的树结构所进行的操作，都要在 $O(\log n)$ 的时间内完成，以保证总时间代价 仍然在 $O(\log n)$ 时间内。平衡二叉搜索树因此在每次插入、删除之后做少量旋转，把左右子树的高度差 限制住，从而保证树高 $O(\log n)$。

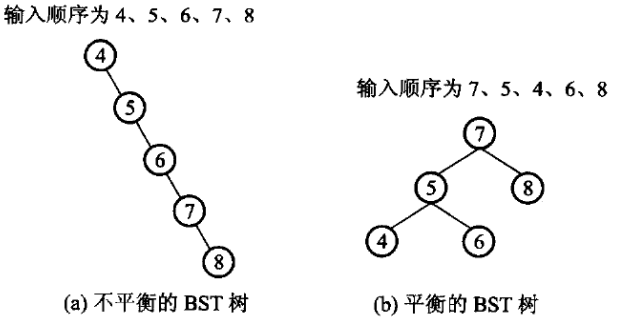


图 12.15　同一批关键码，形状可以从「接近完全二叉树」一直退化到「一条链」——检索代价随之从 $O(\log n)$ 变成 $O(n)$。

AVL 树给每个结点记一个平衡因子：右子树高度减左子树高度，只允许 -1、0、1。

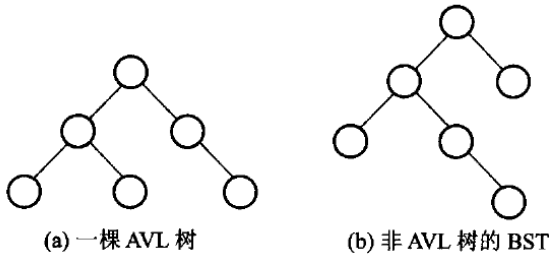


图 12.28　(a) 一棵 AVL 树；(b) 关键码相同但不是 AVL 树的 BST——某个结点的左右子树高度差到了 2。

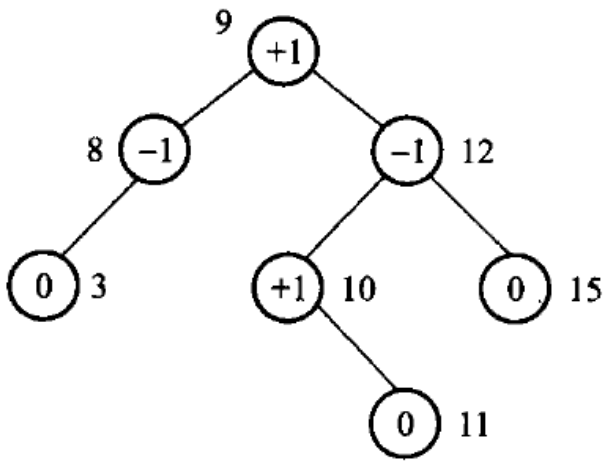


图 12.29　带平衡因子的 AVL 树。原书的实现在结点里存这个因子（只有三种取值，两位就够）；本书 `code/ch12/balanced_trees` 存的是子树高度 `height`，因子由 `balance_factor` 用右高减左高现算，二者等价。插入或删除使某个祖先的因子变成 ± 2 时，按「哪边沉、沉在内侧还是外侧」分成四种情形。外侧失衡一次单旋转就能拉平；内侧失衡要先把孙子转到孩子、再把孩子转到祖先，即双旋转。旋转是局部的，不改变中序次序。查找、插入、删除最坏都是 $O(\log n)$。

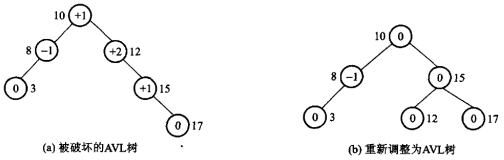


图 12.30　(a) 插入之后被破坏的 AVL 树；(b) 旋转之后重新变成 AVL 树。旋转只动局部几根指针，**中序次序保持不变**——这是所有平衡树旋转的共同前提。

插入一个新结点作为叶结点时，相应子树的根结点变化大致有三类：(1) 结点原来是平衡的，现在成为 左重或右重的，此时该结点的前驱结点的状态也要发生变化；(2) 结点原来是某一边重的，而现在成为平衡 的，**此时前驱结点不改变状态，因为这棵子树的高度未变**；(3) 结点原来就是左重或右重的，而新结点又 加到重的一边，此时这个结点就不再满足 AVL 树条件，**这样的结点叫做「危急结点」**——第 (3) 种情况将 导致新树不是 AVL 树。

设插入后失衡的结点为 A，按新结点落在哪里分成四种情况（图 12.31）：(1) LL **型**——新结点在 A 的左子树的左子树中，A 的平衡因子为 -2；(2) LR **型**——在 A 的左子树的右子树中，因子为 -2；(3) RL **型**——在 A 的右子树的左子树中，因子为 2；(4) RR **型**——在 A 的右子树的右子树中，因子为 2。（注意图 12.31 的子图次序是 LL、LR、RR、RL，与这里的编号次序不同。）

不平衡的结点一定在根结点与新加入结点之间的路径上，并且其平衡因子只能是 2 或者 -2：如果是 2 表示它在插入前的平衡因子是 1，如果是 -2 表示它在插入前的平衡因子是 -1。其中 LL 型和 RR 型是 对称的，LR 型和 RL 型是对

称的。

可以使用一系列称为旋转（rotation）的局部操作来解决这个问题：第 1 种情况和第 4 种情况（LL、RR）可以通过单旋转解决，第 2 种情况和第 3 种情况（LR、RL）可以通过双旋转来解决。这样调整的整个过程称为**重构**（restructuring），过程如下：(1) 首先，从新加入的结点 u 向根的方向往上搜索 BST（不称其为 AVL，是因为可能出现不平衡的情况），直到找到距离 u 最近的不平衡祖先结点 a（平衡因子为 2 或者 -2）——**如果直到根都没有找到这样的祖先，则不做任何动作（已经平衡），返回**；(2) 令 a 结点指向结点 u 的路径上的第一个子结点为 b、第二个子结点为 c，然后根据树的形态进行单旋转或者双旋转。

单旋转只涉及 a 和 b 两个结点：对 LL 调整，令 b 代替 a 成为新根、a 作为 b 的右子结点，a 和 b 的子树根据 BST 的性质分别挂接到 b 和 a 结点下，保持中序周游序列为 $T_0\, b\, T_1\, a\, T_2$；RR 不平衡时进行类似的调整。**双旋转以 RL 型为例**：先作 (c, b) 右转，把 T_2 接到 b 上；然后再作 (a, c) 左转，把 T_1 接到 a 上。

原书用一个很好的办法解释旋转为什么一定对：把参加旋转的 7 个部分排成一个数组。3 个结点 a、b、c，4 棵子树 a 的左子树 T_0、b 的右子树 T_3、c 的左子树 T_1 和右子树 T_2，按 $[T_0, a, T_1, c, T_2, b, T_3]$ 的方式放入数组——**这个数组满足中序周游的性质** $T_0 < a < T_1 < c < T_2 < b < T_3$。不管这 7 个部分以前是如何组织为一棵不平衡的树，旋转的目的 只是使它们重新组成一个新的 AVL 结构，而旋转后得到的新树保持了原来的中序周游顺序。**而且，以上旋转处理中仅需改变少数几个指针，新的子树高度为 $h+2$，保持插入前子树的高度不变**，因此原来 二叉树在更高层上的平衡性不受影响。

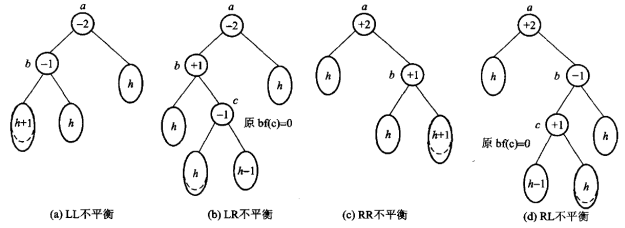


图 12.31　AVL 树的四种失衡类型：(a) LL、(b) LR、(c) RR、(d) RL。圆圈是结点，椭圆是子树，里面的符号是它的高度。

AVL 树的效率有一个漂亮的证明，值得抄下来。AVL 树的检索、插入和删除效率都是 $O(\log_2 n)$，这是因为具有 n 个结点的 AVL 树的高度一定是 $O(\log n)$；要说明这一点，只需说明有 n 个结点的 AVL 树的最大高度不超过 $K \log_2 n$ 即可。

为了证明，构造一系列**临界 AVL 树** $T_1, T_2, T_3, \cdots$：T_i 的高度是 i，使每棵具有高度 i 的 其他 AVL 树都比 T_i 的结点个数多。为了构造 T_i，先分别构造 T_{i-1} 和 T_{i-2}，T_i 即是以 T_{i-1} 和 T_{i-2} 为其左、右子树的 AVL 树——**因为既要保证它们不破坏整棵树的 AVL 结构，又要 求它们包含最少的结点，所以必然高度相差 1**。所构造的是在结点数一定时最接近于不平衡的 AVL 树，这种树在结点一样多的 AVL 树中具有最大高度，可用它来计算 AVL 树高度的上限值。

令 $t(i)$ 为 T_i 的结点数,有 $t(1)=2$、$t(2)=4$、$t(i)=t(i-1)+t(i-2)+1$。**对于 $i>2$, 此关系很类似于定义 Fibonacci 数的那些关系**: $F(0)=0$、$F(1)=1$、$F(i)=F(i-1)+F(i-2)$; 对于 $i>1$, 仅检查序列的前几项就可有 $t(i)=F(i+3)-1$。Fibonacci 数满足渐近公式 $F(i)\approx\phi^i/\sqrt{5}$, 这里 $\phi=(1+\sqrt{5})/2$, 由此可得近似公式 $t(i)\approx\frac{1}{\sqrt{5}}\phi^{i+3}-1$。解出高度 i 与结点个数 $t(i)$ 的关系: $\phi^{i+3}\approx\sqrt{5}(t(i)+1)$, 即 $i+3\approx\log_\phi\sqrt{5}+\log_\phi(t(i)+1)$; 由换底公式 $\log_\phi X=\log_2 X/\log_2\phi$ 和 $\log_2\phi\approx0.694$ 求出近似上限

$$i<\frac{3}{2}\log_2(t(i)+1)-1$$

因为 $t(i)$ 是结点个数, 即对于一个有 n 个结点的 AVL 树, 其高度不超过 $1.5\log_2(n+1)$, 这就是 所要证明的结果。由于 AVL 树的检索、插入和删除操作都是从根到叶结点单路径进行的局部运算, 因此 三种操作都是 $O(\log_2 n)$ 的。

四种情形用中序仍保持 $A<B<C$ 来记 (只画三个结点, 空子树省略; 箭头左边是失衡时的形状, 右边是每一次旋转后的结果):

```
LL (左左): 在 C 上右旋                RR (右右): 在 A 上左旋
      C                                   A
     /                                     \
    B          →       B                   B         →       B
   /                  / \                   \              / \
  A                  A   C                   C            A   C

LR (左右): 先在 A 上左旋, 再在 C 上右旋
      C                 C
     /                 /                  B
    A         →       B          →       / \
     \               /                  A   C
      B             A

RL (右左): 先在 C 上右旋, 再在 A 上左旋
    A                 A
     \                 \                 B
      C       →         B       →       / \
     /                   \             A   C
    B                     C
```

插入 1、2、3 会走出 RR: 先得到右链 1-2-3, 在 1 上左旋变成以 2 为根的平衡树。插入 3、2、1 是对称的 LL。插入 1、3、2 是 RL: 先在 3 上右旋, 再在 1 上左旋。删除一个结点后, 从被删处向上检查, 哪一层变成 ±2 就在那一层做同样的旋转; 有时旋转一次还不够, 要继续向上走。

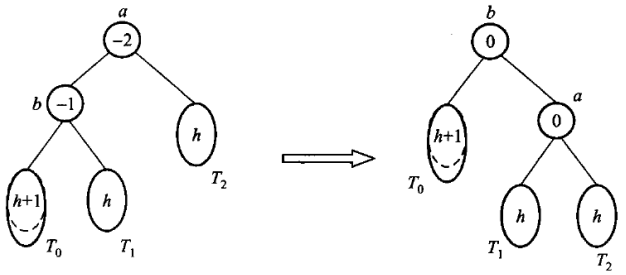


图 12.32　LL 型失衡的单旋转，RR 型对称。

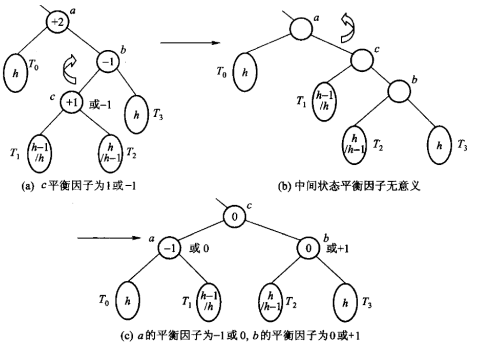


图 12.33　RL 型的双旋转，LR 型对称。中间那步的平衡因子没有意义（图中的 (b)）；转完之后 a 的因子是 -1 或 0、b 的因子是 0 或 $+1$，取决于被提上来的那棵子树原来偏哪边。

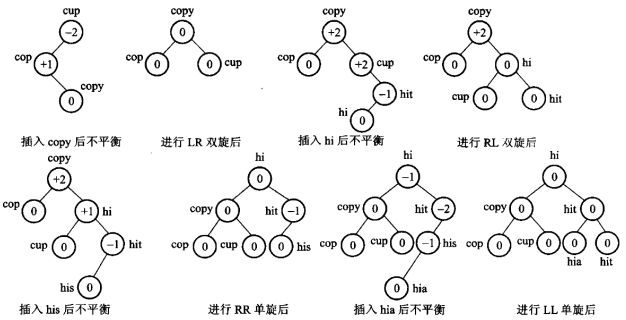


图 12.34　从空树开始依次插入单词 cup、cop、copy、hit、hi、his、hia（按字典序比较），AVL 树的生长过程。

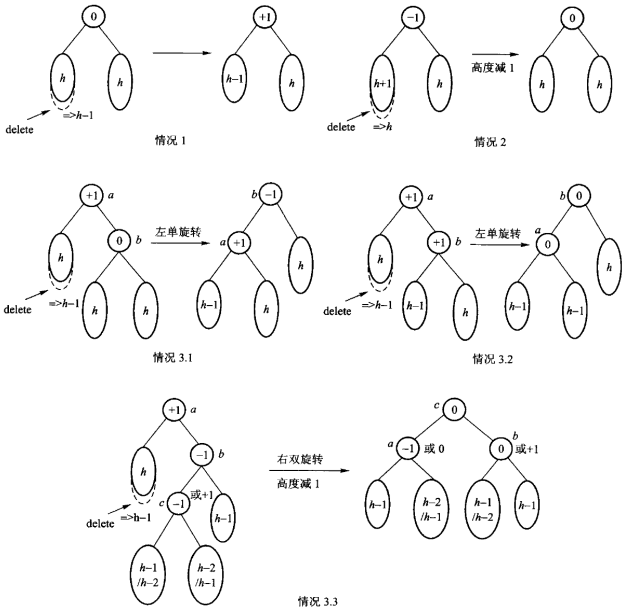


图 12.35　删除之后的调整。**删除比插入麻烦**：插入最多转一次就结束，删除可能一路向上每层都要转。

删除的具体步骤（原书，每种情况图 12.35 只列了一个特例）。从删除结点的意义上说，AVL 树的删除与 BST 一样；从调整平衡因子的角度说，删除是插入的逆操作。(1) 先按第 5 章 BST 的删除方法删掉结点，把 替代它的那个位置上的结点 a 作为当前结点。(2) 沿被删结点到根的路径向上调整平衡因子，用布尔变量 `modified`（初值 TRUE）记录「这棵子树变矮了，还要继续向上」；它变成 FALSE 时回溯停止。当前结点 a 有三种情况：

- $bf(a) = 0$，**某一侧子树被缩短**：因子改为 1 或 -1，`modified = FALSE`。以 a 为根的子树高度 未变，上面的结点不受影响，调整结束。
- $bf(a) \neq 0$，**较高的那侧被缩短**：因子改为 0，`modified = TRUE`——子树矮了一层，要继续向上。
- $bf(a) \neq 0$，**较低的那侧被缩短**：a 必然失衡。设较高子树的根为 b：
  - 情况 3.1，$bf(b) = 0$：对 a 做一次单旋转即可恢复平衡，子树高度不变，`modified = FALSE`；
  - 情况 3.2，$bf(b) = bf(a)$：做一次单旋转，a、b 的因子都变为 0，子树矮了一层，`modified = TRUE`；
  - 情况 3.3，$bf(b)$ 与 $bf(a)$ 符号相反：做一次双旋转（先围绕 b、再围绕 a），新根的因子为 0，其他结点相应调整，`modified = TRUE`。

与插入一样，是从被删结点向上找到失衡的祖先再做单旋转或双旋转；不同的是，**插入最多旋转一次就调整 到位，而删除时一次旋转结束并不意味着树已经平衡**——情况 3.2、3.3 让子树矮了一层，可能在更上面的 祖先处引起新的不平衡，所以这种重构要一直进行下去，直到 `modified` 为 FALSE 或到达根为止。下

面 图 12.36 的 (c) → (d) 是情况 3.2（$bf(m) = bf(f) = -1$），(d) → (e) 又是一次情况 3.2（$bf(a) = bf(b) = -1$）。

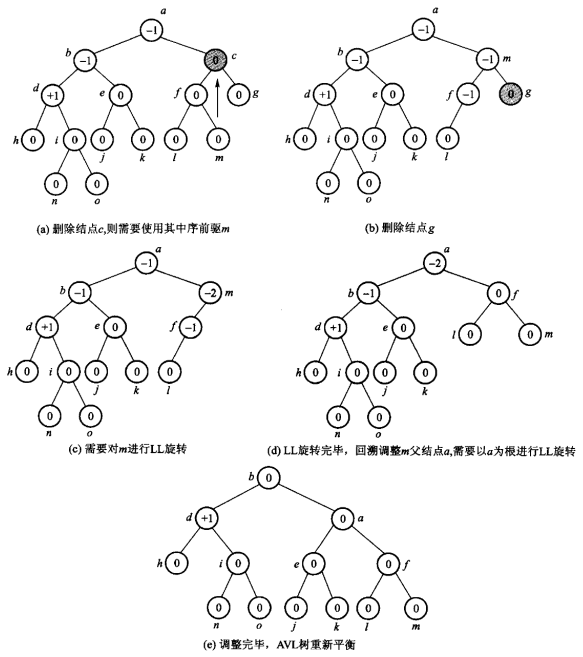


图 12.36　删除的例子（原书跨两页，(a)(b) 一页、(c)(d)(e) 一页）：(a) 删有两个孩子的结点 c，先用它的中序前驱 m 顶替，再删到叶那一层；(b) 删结点 g 之后 m 失衡；(c) 需要对 m 作 LL 旋转；(d) 转完 m 之后回溯，父结点 a 又失衡，再以 a 为根作一次 LL 旋转；(e) 调整完毕，整棵树重新平衡。**一次删除可能引发从被删结点一路到根的多次旋转**，这是它与插入最大的不同。

注意：图中用**中序前驱**（左子树最大）顶替被删结点，而 `code/ch12/balanced_trees/modern.hpp` 的 `remove` 用的是**中序后继**（右子树最小）。两种做法都保持 BST 性质、都正确，只是互为镜像；用代码复现这张图时，被顶替的结点和随后失衡的位置会与图不同。

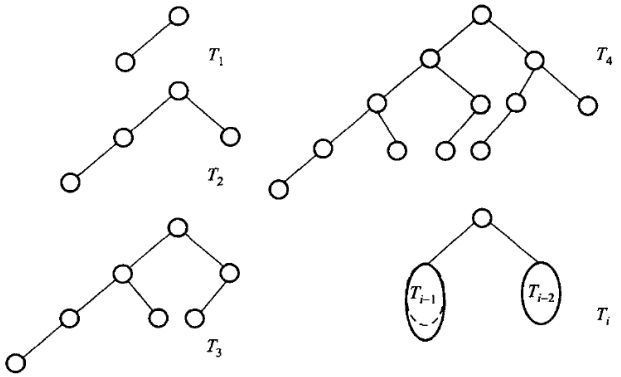


图 12.37　「最瘦」的 AVL 树——每个结点的左右高度差都取到允许的极限。

设高度为 h 的这种树最少有 $N(h)$ 个结点，则 $N(h)=N(h-1)+N(h-2)+1$（Fibonacci 递推），由此可推出 AVL 树的高度不超过约 $1.44\log_2 n$（上文的 $1.5\log_2(n+1)$ 是把系数 $1/\log_2\phi\approx 1.44$ 放大取整后的写法）：**AVL 的平衡不是完美平衡，但足以保证** $O(\log n)$。

四组最小输入正好能当旋转单元测试：`3,2,1 -> LL`，`1,2,3 -> RR`，`3,1,2 -> LR`，`1,3,2 -> RL`，结果都应以 2 为根，中序都为 `1,2,3`。只测中序不够，因为一棵退化链的中序也正确；还要查根、高度和平衡因子，才能证明旋转真的发生。

红黑树是另一种近似平衡，用颜色规则保证最长路径不超过最短路径的两倍，旋转次数有常数上限，见第 11.5 节。可运行的 AVL 四旋转实现见 `code/ch12/balanced_trees/modern.hpp`，测试检查中序有序、查找、删除和高度边界。

12.4.3 伸展树

伸展树（splaying tree）由 Daniel Dominic Sleator 和 Robert Endre Tarjan 提出（论文 *Self-Adjusting Binary Search Trees* 于 1985 年发表；原书误作 Hopcroft 和 Tarjan），与第 10.1 节提到的线性表调整技术（原书在那里讲了调换法和移至前端法：每找到一个记录，就把它往表的前端挪）同属于**自调整数据结构**（self-adjusting data structure）。

与 AVL 不同的是：伸展树的一次访问不一定会使树结构更加平衡，也不能保证最终树高平衡；但它能 保证访问的总代价不高，达到最令人满意的性能。伸展树与 AVL 树一样，实际上也不是一种新数据 结构，而只是**改进 BST 性能的一组规则**，控制检索、插入、删除等操作对 BST 的修改，从而避免标准 BST 操作在最差情况下的线性时间代价。

每当访问一个结点 x 时（x 被插入、删除或者被检索），伸展树就完成一次称为**展开**（splaying）的过程：插入和检索 x 时，结点 x 作为当前结点；**删除结点 x 时，x 的父结点作为当前结点**。沿着当前结点 到根结点的方向进行一系列展开操作，直到把当前结点移到 BST 的根。就像在 AVL 树中一样，一次展开 由一组**旋转**（rotation）组成：一次旋转通过调整结点 x 相对于其父结点和祖父结点的位置，将其移到 树结构中的更高层。**旋转有 3 种类型：单旋转、一字型双旋转、之字型双旋转**。

当被访问结点 x 是根结点的子结点时，完成**单旋转**（single rotation），在保持 BST 特性的情况下把 x 与其父结点交换位置——**实际上伸展树的单旋转与 AVL 树的单旋转是一样的**。当被访问结点 x 离 根结点的层数大于等于 2 时，发生**双旋转**（double rotation），涉及 x、其父结点 y 和祖父结点 z。

最近被用到的键下次更容易先碰到，这是一种自调整，均摊 $O(\log n)$。

从被访问结点走到根，每次看它、父结点和祖父结点的形状。一字形（连续两次同向）先转祖父、再转父；之字形（先左后右，或先右后左）先转父、再转祖父。两种双旋都把目标提升两层，但对其余结点的重排不同。目标只差一层到根时，再做一次单旋转即可。半伸展不会每次都把目标送到根，旋转更少，但保证也与全伸展不同。

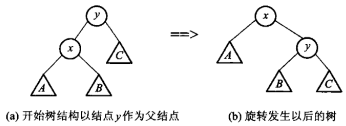


图 12.38　只有当被展开的结点已经是根的孩子时，才做单旋转：左边是旋转前，右边是旋转后。

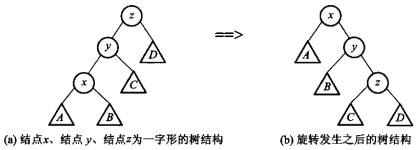


图 12.39　一字形旋转：x、y、z 在一条同向的链上（左），旋转之后（右）。单看一次，一字形一般不会降低树高，只是把 x 提升两层（子树 A 升两层、B 升一层，C 降一层、D 降两层）；沿一条同向长链**连续**做一字形，才会把整条链折短（见下文的例子）。

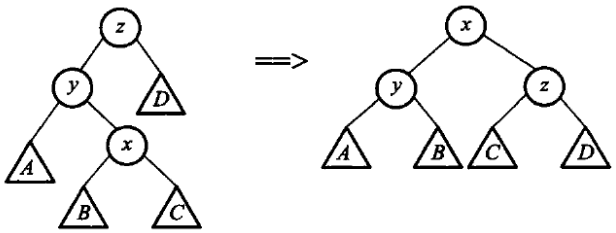
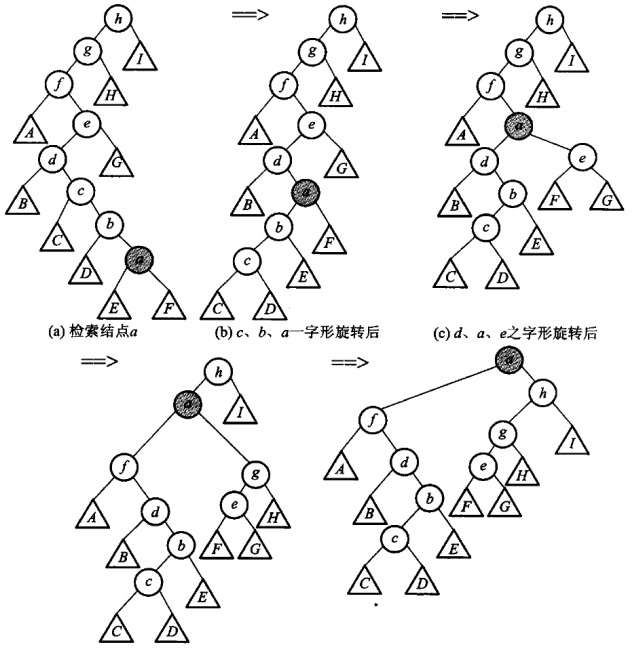


图 12.40　之字形旋转：x、y、z 拐了个弯。之字形**倾向于让树更平衡**——子树 B、C 上升一层，D 下降一层。



最后一句放在这里最合适（原书 12.4.2 末）：二叉搜索树（包括红黑树、AVL 树、伸展树等）适用于组织 **内存中**小规模的目录；对于较大的、存放在外存储器上的文件，用二叉搜索树来组织索引就不太合适—— **若以结点作为内外存交换的单位，则检索中在找到需要的关键码之前平均要对外存进行 $\log_2 n$ 次 访问，这是很费时的**。在文件索引中大量使用每个结点包含多个关键码的 B 树，尤其是 B+ 树（第 11 章）。另外，Linux 采用 AVL 树描述进程的虚拟内存段。

图 12.41　检索 a 之后把它展开到根：先 c、b、a 一字形，再 d、a、e 之字形，再 f、a、g 之字形，最后一次单旋转让 a 成为根。四步旋转（三次双旋转加一次单旋转）下来，a 从第 7 层升到根，整棵树的层次明显变浅了。（图中内部结点只是索引代号，记录都在外部叶上，所以内部结点的字母不必满足中序有序。）

插入与删除怎么接上展开。伸展树的插入操作与 BST 相同，但是结点插入后还要被展开到根结点；同样，采用 BST 删除结点策略从伸展树中删除一个结点后，**被删除结点的父结点需要展开到根结点**。

半伸展（semi-splaying）树是伸展树的变体：它对于同构的「一字形」（zig-zig 型）的情况，类似于 AVL 树 的 LL、RR 型那样处理。区别在于——下一轮的伸展，当前结点上移到父结点，**父结点位置开始旋转，即 半伸展不需要把检索结点移到根**。

还有一条容易忽略的边界：如果记录的信息都存储在外部结点、内部结点只作为索引结构，那么检索到 某个外部结点时，其内部父结点作为当前结点，然后开始调整——**伸展的过程中，要注意外部结点不能被 旋转到根上**。

例如链 `10 -> 20 -> 30` 中访问 30，是右–右一字形：先围绕 10 左旋，再围绕 20 左旋，30 到根。若 10 的右孩子是 30、30 的左孩子是 20，则访问 20 是右–左之字形，两次旋转方向相反。两种情况都把目标提两层。只有三个结点时，一字形与「逐层父子单旋」的结果都还是一条长 3 的链，看不出差别；链一长差别就出来了：右链 `1 -> 2 -> ⋯ -> 7` 访问 7，按伸展规则（一字形先转祖父）做完，树高从 7 降到 5；若只是逐层单旋把 7 提到根，树高仍是 7——只是把链倒了过来。链长 31 时两者分别是 17 和 31。这就是一字形不能替换成连续「父子单旋」的原因。

“均摊 $O(\log n)$” 不等于每一次都快。访问深链末端的单次代价仍可达 $O(n)$；保证的是从任意初始树开始，一串 m 次操作的总成本为 $O(m \log n)$（另加初始势能）。因此伸展树适合热点键反复出现、又不要求单次延迟上界的场景；实时系统若要求每次查找都在确定高度内结束，更适合 AVL 或红黑树。

半伸展是一种折中：不必每次都把目标送到根，旋转次数更少。

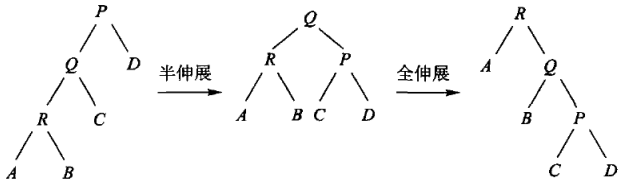


图 12.42　半伸展与全伸展的区别。

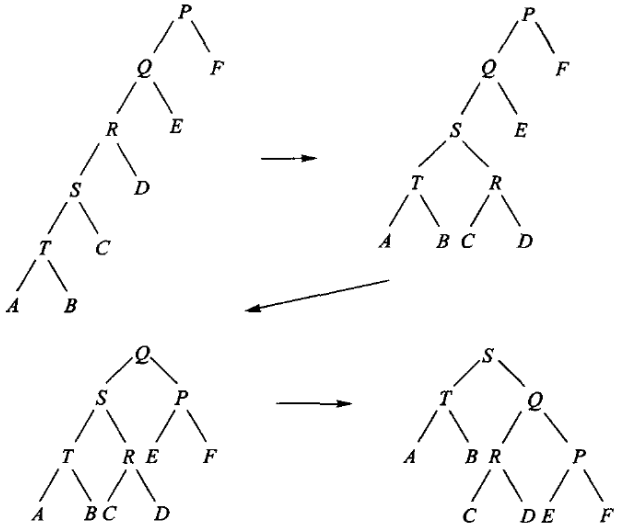


图 12.43　连续两次访问同一个结点 T 时，半伸展树的变化。

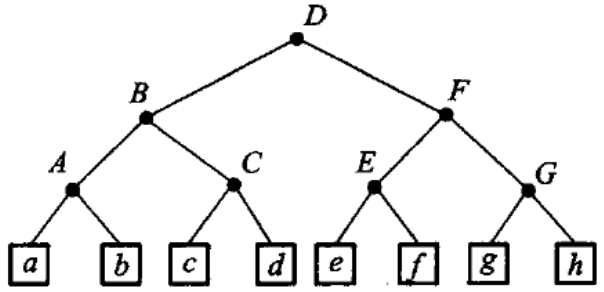


图 12.44　记录存在外部结点上的半伸展树。

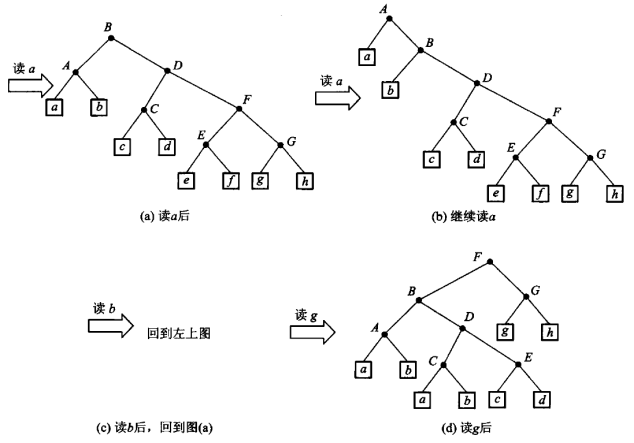


图 12.45　在图 12.44 上连续读 a、a、b、g 之后的变化过程。**重复访问的键会一路浮到靠近根的位置**——这正是伸展树在「热点键反复出现」时占便宜的地方。

伸展树实现简单，不必维护额外的平衡域；代价是单次操作可能仍是 $O(n)$，只是摊还之后是对数。同一实现中的 `SplayTree` 使用 `unique_ptr` 维护子树所有权，访问后按一字形/之字形规则旋至根。

本章小结

Python 实现的证据链

```python
class Trie:
    """12.4 Trie：每个字符一层，公共前缀共享同一条路径。"""

    def __init__(self) -> None:
        self.root = _Node()
        self._size = 0

    @staticmethod
    def _valid(word: str) -> None:
        if any(c < "a" or c > "z" for c in word):
            raise ValueError("trie key must be a..z")

    def insert(self, word: str) -> bool:
        self._valid(word)
        node = self.root
        path = []
        for c in word:
            path.append(node)
            node = node.children.setdefault(c, _Node())
        if node.terminal:
            return False
        node.terminal = True
        self._size += 1
        # passing 是「有多少个键经过这里」，count_with_prefix 靠它一次问答，
        # 而不是把子树数一遍。维护它的代价就落在这一行。
        for item in path:
            item.passing += 1
        node.passing += 1
        return True

    def contains(self, word: str) -> bool:
        node = self._find(word)
        return node is not None and node.terminal

    def starts_with(self, prefix: str) -> bool:
        return self._find(prefix) is not None

    def count_with_prefix(self, prefix: str) -> int:
        if prefix == "":
```

```
            return self._size
        node = self._find(prefix)
        return 0 if node is None else node.passing

    def _find(self, text: str) -> _Node | None:
        node = self.root
        for c in text:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

    def size(self) -> int:
        return self._size

    def node_count(self) -> int:
        def count(node: _Node) -> int:
            return sum(1 + count(child) for child in node.children.values())

        return count(self.root)

    def keys_with_prefix(self, prefix: str) -> list[str]:
        node = self._find(prefix)
        out: list[str] = []
        if node is None:
            return out

        def collect(current: _Node, text: str) -> None:
            if current.terminal:
                out.append(text)
            # 按 a..z 的顺序下降，收集结果因此天然有序，不必再排一次。
            for code in range(ord("a"), ord("z") + 1):
                c = chr(code)
                if c in current.children:
                    collect(current.children[c], text + c)

        collect(node, prefix)
        return out

    def erase(self, word: str) -> bool:
        if not self.contains(word):
            return False
        node = self.root
        path = []
        for c in word:
            path.append((node, c))
            node = node.children[c]
```

```
        node.terminal = False
        node.passing -= 1
        self._size -= 1
        # 自底向上摘掉不再承载任何键的结点：passing 归零且没有孩子就该消失。
        for parent, c in reversed(path):
            parent.passing -= 1
            child = parent.children[c]
            if child.passing == 0 and not child.children:
                del parent.children[c]
        return True

    # >>> trie-longest-prefix
    def longest_prefix_of(self, text: str) -> str:
        """text 的哪个前缀是树里最长的那个键。走不动就回退到最近一次的词尾。"""
        node = self.root
        best = 0
        for i, c in enumerate(text):
            if c not in node.children:
                break
            node = node.children[c]
            if node.terminal:
                best = i + 1
        return text[:best]
    # <<< trie-longest-prefix
```

```
def bit_of(key: str, index: int) -> bool:
    """ 键的第 index 位（从最高位数起）。超出键长一律当 0——

    这一条让「一个键是另一个键的前缀」也能被分开：`"a"` 与 `"ab"` 在
    第 8 位之后就靠这个 0 区分。求首个不同位时把上界取到 `(最长 +1)*8`，
    正是为了给这个「虚拟的结尾」留出位置。
    """
    byte = index // 8
    if byte >= len(key):
        return False
    return bool((ord(key[byte]) >> (7 - index % 8)) & 1)
```

算法 12.1 的可利用空间表属于存储管理侧，因此不给 Python 薄封装；最佳 BST 的动态规划属于算法侧：

```
def optimal_bst(successful, unsuccessful):
    if len(unsuccessful) != len(successful) + 1:
        raise ValueError("weight count")
    n = len(successful)
    cost = [[0]*(n+1) for _ in range(n+1)]
```

```
    root = [[0]*(n+1) for _ in range(n+1)]
    weight = [[0]*(n+1) for _ in range(n+1)]
    for i in range(n+1):
        weight[i][i] = unsuccessful[i]
    for length in range(1, n+1):
        for first in range(n-length+1):
            last = first + length
            weight[first][last] = weight[first][last-1] + successful[last-1] +
↪   unsuccessful[last]
            cost[first][last] = 10**30
            for r in range(first+1, last+1):
                candidate = cost[first][r-1] + cost[r][last] + weight[first][last]
                if candidate < cost[first][last]:
                    cost[first][last], root[first][last] = candidate, r
    return cost, root
```

为了适应一些特定应用，需要功能更强大、具有更多操作的高级数据结构。本章介绍了多维数组、广义表 等高级线性表结构以及相关的内存管理技术，还介绍了 Trie 和 PATRICIA 两种字符索引结构，以及最佳 二叉搜索树、平衡二叉搜索树 AVL、伸展树 3 种改进的 BST。

多维数组可以看做是线性表的推广。数组一般采取顺序存储方法，因为数组结构中元素个数相对固定，一旦生成只改变元素的值，并不改变元素的相对位置和个数；把数组元素按周游的顺序排列起来，得到一个 包含所有数组元素的线性序列，**数组的周游分为行优先顺序和列优先顺序两种方式**。二维数组常被用来表示矩阵，数组的第一维和第二维分别对应矩阵的行和列；三角矩阵、对称矩阵和对角矩阵是常用的几类 比较特殊的矩阵；**稀疏矩阵是非零元素非常少的矩阵，它常用的存储方式是十字链表法**。

广义表是对线性表的一种扩展，**它允许线性表中还包含一个或多个子表**。用数学中结合的包含关系可以 表述几种类型的广义表：**图 ⊇ 再入表 ⊇ 纯表 ⊇ 线性表**，循环表是带回路的再入表（再入表 ⊇ 循环表），但不包含纯表。（原书小结此处写成「图 ⊇ 再入表 ⊇ 循环表 ⊇ 纯表 ⊇ 线性表」，与 12.2 节「再入表（循环表）」的写法矛盾：纯表没有回路，不是循环表。）根据广义表的不同形式，可以用线性表、树和图等数据结构或其修改形式来存储和实现广义表；广义表的周游通常采用深度优先的策略。

为支持动态数据结构，操作系统需要一套有效的内存管理技术，动态地进行内存的分配和回收。在实际的 操作系统中，内存管理本质上是利用链表和广义表来实现的；内存管理最基本的问题是存储的分配和回收，需要解决碎片问题以及存储空间溢出时无用单元的回收管理等。**内存分配是通过可利用空间表的组织来 进行的**，这是把存储区中当前还未使用的空间通过链表方式管理起来的一种方法。

BST **树是一种基于对象空间分解的数据结构**，即关键码范围的分解是由存储

在树中的对象决定的，这种 分解方式受关键码输入的影响。Trie **结构对关键码范围进行均分，它基于两个原则：有一个固定的关键码 集合和对于结点的分层标记**。其内部结点仅作为占位符引导检索过程，数据记录只存储在叶结点中；英文 字符 Trie 树的每个内部结点都有 26 个子结点（如果算上空格，则为 27 个）。PATRICIA **树是 Trie 树的 变体，它根据关键码的二进制编码来进行划分，是一种比较好的中文字典组织方式**。

最佳二叉搜索树、平衡二叉搜索树 AVL、伸展树是二叉搜索树 BST 的几个变体。**检索中平均比较次数 最小的二叉搜索树称为最佳二叉搜索树**，它满足最优子问题和子问题重叠性质，可以使用动态规划算法来 构造。

AVL **树中任何结点的左子树和右子树高度最多相差** 1。插入新结点按照 BST 的插入算法，如果插入后 仍然是 AVL 树则不调整，否则需要根据 LL、LR、RL、RR 四种类型进行调整；AVL 树的删除结点采用 BST 的基本删除算法，如果删除后需要调整，则从被删除的结点找到第一个失衡的祖先结点，然后开始单旋转或多旋转操作，**这样的调整操作向根的方向传递，直到树平衡为止**。AVL 树放宽了 BST 的平衡要求（不要求完全平衡，只要求每个结点左右子树高度差不超过 1），它既 照顾到更新操作的代价，又兼顾完成操作后树结构的检索功能，其树高为 $O(\log n)$。

伸展树不要求 BST **结构总是保持平衡或最佳检索状态**，而是在每次访问时通过一系列双旋转、单旋转将 被访问结点移到根结点，**这样尽量使最近访问的结点容易被检索到**，其检索效率比较高。

第 11 章介绍的红黑树也是一种改进的 BST，它是一种常用的索引结构，统计性能要好于 AVL 树；其平衡 要求比 AVL 树更低，利用对树中结点红黑着色的要求达到局部平衡。

二叉搜索树（包括红黑树、AVL 树、伸展树等）适用于组织内存中小规模的目录。AVL 树的高度不超过 $1.5\log_2(n+1)$，具有 n 个内部结点的红黑树最大树高是 $2\log_2(n+1)+1$（按原书口径，树高数到最深的外部空叶为止；只数内部结点时是 $2\log_2(n+1)$，见 11.5 节），因此**最坏情况下的红黑树 的高度大于最坏情况下具有相同（内部）结点数目的** AVL **树**；AVL 树和伸展树适合于检索较多的应用，插入删除操作比较频繁的应用往往采用红黑树。

具体到本章的实现：多维数组按行优先或列优先顺序存放，按下标随机访问；三角、对称、稀疏矩阵可以 压缩。广义表的元素可以是原子或子表，头尾分解对应递归。动态存储要处理碎片和回收：首次/最佳/最坏 适应，引用计数与标记清除。Trie 按字符分支并共享前缀，Patricia 压缩单孩子路径。最优 BST 用动态规划选根；AVL 用平衡因子和四种旋转保证 $O(\log n)$；伸展树用访问后上移做均摊平衡。可利用空间表是定长结点的显式空闲栈，不劫持全局 `new`。

习题

补充概念与上机题（参考课程第 12 章）

1. 用 CSR 三数组表示稀疏矩阵，并计算 `y = A x`；复杂度按非零元数 `nnz` 表示。

2. 说明共享广义表为什么不能简单递归释放，并设计引用计数或访问标记方案。
3. 证明长度为 `N` 的字符串所有后缀插入 Trie 后，每个不同子串对应 Trie 中一个结点；分析最坏时间。
4. 实现 AVL 的四种旋转，并用中序遍历验证旋转前后序列不变。

正文习题

1. 写出 3×4 数组按行优先时 $a_{2,1}$ 相对首地址的偏移（元素占 1 个单元）。
2. 把 4 阶下三角压成一维，给出 $a_{3,1}$ 的下标。
3. 说明纯表、再入表、循环表分别对应什么图，释放时要注意什么。
4. 空闲块按地址顺序是 900、1500、700。举一个请求，使首次适应、最佳适应、最坏适应分别选中三块**不同**的空闲区；再说明为什么「能不能满足」这个条件本身分辨不出三种策略。
5. 把 `can`、`car`、`cat` 插入字母 Trie，画出结果，并写出查找 `cab` 的失败路径。
6. 教材样例 $p = \{1, 5, 4, 3\}$、$q = \{5, 4, 3, 2, 1\}$，指出最优 BST 的根和总成本。
7. 从空 AVL 依次插入 1、2、3，画出每次旋转。再插入 0，需要旋转吗？
8. 伸展树中，之字形和一字形各升几层？为什么半伸展适合连续访问同一结点。

原书习题

> 以下是原书第 12 章的习题，本轮按扫描件补回题面；**参考答案见 `book/`习题与参考答案`.md` 的同章小节**，由 `tools/check_doc.py` 的 R14 逐题守着。题面里引用的图，一并从扫描件裁了回来。

1. 设计一个算法，将按行优先原则顺序存储的二维数组 $A_{n\times n}$ 转置（$a_{i,j}$ 与 $a_{j,i}$ 交换），**要求转置结果仍占用原来的存储空间**。
2. 编写程序，建立一个 10×10 的对角线元素为 0、其余元素为 1 的二维数组，并将该数组打印出来。成功后改写该程序，使其对角线元素仍然为 0、其余元素为空格字符的二维数组。
3. 若在 $m \times n$ 的矩阵中存在一个元素 $A[i, j]$ 满足：$A[i, j]$ 是第 i 行元素中最小值，且是第 j 列 元素中最大值，则称矩阵 A 有**鞍点**。试写一个算法，找出矩阵 A 的一个鞍点；若 A 中不存在鞍点，则给出某种信息（采用行优先原则顺序存储矩阵 A_{mn}）。
4. 编写一个广义表逆置算法，在所有级别逆置子表、包括最高级。(1) 例如纯表 $(x_1, (y_1, y_2, (z_1, z_2), y_4), (w_1, w_2), x_4)$ 的逆置为 $(x_4, (w_2, w_1), (y_4, (z_2, z_1), y_2, y_1$
(2) 循环表 $(L1 : (L2 : (L1, a)), (L2, L3 : (b)), (L3, c), L4 : (d, L4))$ 的逆置对应于 $(L4 : (L4, d), (c, L3 : (b)), (L3, L2 : (a, L1)), L1 : (L2))$。
5. 画出下列广义表的图形表示和它们的存储表示：(1) $D : (A : (c), B : (e), C : (a, L(b, A, d)))$；
(2) $J1 : (J2 : (J1, a, J3 : (J1)), J3)$。
6. 利用广义表的 `Head` 和 `Tail` 运算，把原子 d 分别从下列广义表中分离出来：

$L1 = ((((a), b), d), e)$; $L2 = (a, (b, ((d)), e))$。

7. 设计一个算法，求广义表的深度。
8. 设计一个算法，删除广义表中所有值等于 x 的原子项。
9. 设计一个算法，判断两个广义表是否相等。
10. 3 种顺序适配方法各有其使用范围。现假定存储区中有大小为 900、1500、700 的三块空存储块。试给出符合以下几种情况的存储请求的例子：(1) 首先适配法可满足请求，另两种方法不可满足请求；
    (2) 最佳适配法可满足请求，另两种方法不可满足请求；(3) 最差适配法可满足请求，另两种方法不可 满足请求。
11. 组织成循环链表的可利用空间表附加什么条件时，**首次适配策略就转变为最佳适配策略**?
12. 给出下面一组值 {cup, cop, copy, hit, hi, his, hia, he} 的二叉 Trie 结构。
13. 给出下面一组值 {cup, cop, copy, hit, hi, his, hia, he} 的 PAT Trie 结构。
14. 将关键码 $1, 2, 3, \cdots, (2^k - 1)$ 依次插入到一棵初始为空的 AVL 树中，试证明结果是一棵高度为 k 的完全满二叉树。
15. 已知元素个数为 12 的字典，其元素集合为 {Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec}。(1) 试按元素的次序依次插入一棵初始时为空的二叉搜索树，画出插入完成之后的二叉 搜索树，并求其在等概率情况下查找成功的平均查找长度；(2) 按元素顺序构造一棵 AVL 树，并求其在 等概率情况下查找成功的平均查找长度。
16. AVL 树是否允许重复关键码? 例如图 12.46 中有两个关键码都是 12 的结点，为什么?

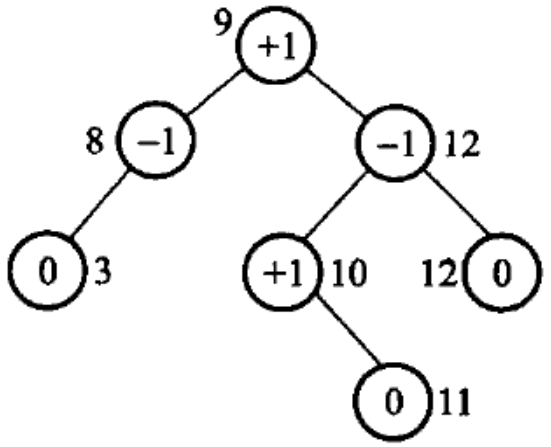


图 12.46　原书题注写作「习题 17 的 AVL 树」，但正文是在习题 16 里引用它的——此处照录原书图号。

17. 高度为 h 的 AVL 树上的最少结点个数与最多结点个数分别是多少?
18. 从空的二叉树开始，根据字典序顺序（注意：`he` < `tea` < `teacher`），严格按照 AVL 树（平衡的二叉 搜索树）插入算法，依次插入 {head, he, tea, teach, twin,

hot, toss}。(1) 试画出插入后的 AVL 树；
(2) 在上题生成的 AVL 树中，依次删除 head、he、tea、teach、twin、hot、toss。试画出每执行一次删除 操作、并根据必要进行调整后的 AVL 树。

上机题

1. 实现下三角矩阵的压缩存储，并支持按下标读写。
2. 用句柄池管理固定个数的结点，测试耗尽、归还、复用和重复释放。
3. 实现一棵字母 Trie 的插入和查找，用一组单词对拍 `std::set`。
4. 按教材权重实现最优 BST，输出根表并重建树形。

原书上机题

> 原书上机题的参考答案见 `book/习题与参考答案.md` 的「原书上机题参考」，由 R14 逐题守着。原书这一章的上机题分量最重，后几道（阅卷管理、职工管理、搜索引擎）是北京大学计算机系多年来的学期综合上机实习题。

1. 设二维数组 $a[1..m, 1..n]$ 含有 $m \times n$ 个整数。写出一个算法来判断 a 中所有元素是否互不 相同，输出相关信息（yes/no），并分析你的算法的时间复杂度。
2. 广义表具有可共享性，因此在周游一个广义表时必须为每一个结点增加一个标志域 `mark`，以记录该 结点是否访问过；一旦某一个共享的子表结点被做过访问标志，以后就不再访问它。(1) 试定义该广义表 的类结构；(2) 实现对一个非递归的广义表进行周游的算法。
3. 试设计一个数据结构和算法，判断两个 `.cpp` 程序（程序都是完整的，即类型定义和函数定义都在一个 文件中）是否相似。可以返回 0（完全不同）至 1（完全相同）之间的值。
4. 实现稀疏矩阵表示方法，并支持矩阵的下列操作：(1) 在给定位置向矩阵中插入一个元素；(2) 从矩阵中 给定位置删除一个元素；(3) 返回矩阵中给定位置元素的值；(4) 对一个矩阵进行转置；(5) 两个矩阵相加。
5. 基于图 12.11 的结构，模拟存储管理系统，支持 3 种顺序适配方法的分配和回收操作：首先适配、最佳 适配和最差适配。测试你的系统，分析总结在什么情况下哪种方法完成得最好。
6. 用 Trie 树实现一个字典，要求具有 `create`、`insert`、`delete`、`search` 功能。
7. 使用 Trie 数据结构设计一个程序对变长字符串排序，**程序所做的工作与所有字符串中字母总数要成 比例**。有些字符串可能很长，而大多数则很短。
8. 编写一个程序, 对一段英文材料用 Trie 树进行存储。要求: 提供算法周游 Trie 树，并打印出所有单词，单词按照长度的升序排序，长度相同的单词按字母顺序。
9. 对于一段英文材料，构造 PATRICIA 树，并统计每个字母出现的次数。
10. **后缀树**。后缀是指从某个位置 i 开始到整个串末尾结束的一个特殊子串；一个长度为 n 的字符串 S 约定末位 `S[n]` 为特殊字符 `$`，也就是说 S 以一个特

殊字符 $ 结尾。一个**后缀树**（suffix tree）是对于一个 给定的字符串，包含其所有后缀字符串的压缩 Trie 结构。例如字符串 MALAYALAM$ 的后缀字符串 集合中有 10 个子串。**后缀树的好处是它允许使用通配符对字符串进行检索**：例如检索关键码 ALA* 表示找到所有以 ALA 作为前 3 个字符的字符串，这可以很容易地使用一个普通 Trie 结构 实现；**在一个普通 Trie 结构中检索 *ALA 效率很低，但是在一棵后缀树中却是高效率的**。后缀树 提供了很好的全索引结构，适合于各种字符串算法，例如查找字符串中的子串、找出串中最长的重复子串、查找两个字符串的公共子串、判断回文串等；后缀树在中文切词、共现词关联分析、频繁模式挖掘、聚类 分析、基因序列对比分类等研究领域也有很广泛的应用。Edward McCreight 于 1976 年提出非在线的 后缀树算法，逆序处理字符串、采用路径压缩的方法构造后缀树，该后缀树最多有 $2n$ 个结点，构造时间 代价为 $O(n)$；1995 年 Ukkonen 提出了在线的后缀树算法，在从左到右输入的同时构造出后缀树，时间代价为 $O(n)$。试根据自己的兴趣和能力完成后缀树的练习，例如：(1) 对一个词典中的单词或 短语实现后缀树；(2) 分析若干篇文章中的频繁共现词汇；(3) 利用后缀树对搜索引擎的返回结果进行 聚类。

11. 在字符 $\{A, C, G, T\}$ 组成的 DNA 序列中，A ↔ T 和 C ↔ G 是互补对。编写算法，判断一个 DNA 序列 中所有互补回文串（例如 ATCATGAT 的补串是 TAGTACTA，与原串形成互补回文串）。
12. **利用半伸展树存储文件中单词词频**。扫描一个文本文件，并计算出这一文件中每个词的词频；为简化 起见，略去标点符号、按照字典排列的顺序对单词排序，并忽略大小写。用一棵 BST 作为一个文件中单词 的存储结构，并用 semi-splay 技术对这棵树进行辅助维护，以使输入流中的下一个单词出现在树比较 接近于根的位置的几率较大。方法如下：(1) 如果在文件中第一次遇到一个词，把这个单词插入 BST 树；否则，从与该单词对应的结点处开始处理，这一单词的频率增加 1；(2) 向根结点的移动是通过改变所涉及 结点的链接来完成的，如果单词没有在树中找到，将创建一个新的叶结点然后插入树中；(3) 通过一个 指向父结点的指针来保存结点的前驱。**注意：旋转与否只取决于是否被访问（包括查找、插入等），跟词频 本身没有必然关系**——但是可以想像，访问频率越高（只要不都是连续访问），它被旋转到根结点的机会 就越多，相对而言它会比其他较少访问的结点更接近根结点。作业要求：创建一个较长的测试文件，使用 semisplaying 技术来组织一棵 BST；在所有的单词都处理完之后，要求对树进行一次中序周游，将所有 结点的词频数打印出来。
13. **模拟的阅卷管理系统**。该系统支持多位教师集体阅卷（试卷为电子文档）。基本要求：(1) 每份试卷应 该随机调阅，分别由两位教师批阅，取其平均成绩；(2) 阅卷教员发出调阅试卷要求，系统随机调阅一份未 完成的试卷（还没有人阅过，或者只有一位教师阅过）给该阅卷者；(3) 每位教师可以检查自己批阅过的 试卷，并录入和修改成绩；考生姓名和考号对教师保密，教师只能看到系统编号。读者根据自己的能力 添加其他功能：① 支持大数据量，例如 100

万以上；② 支持并发操作，例如允许多位教员读数据；③ 两位阅卷教师给出的成绩相差太大（例如 > 10%）时，试卷派发给阅卷组长；④ 自动调整每位教员的 成绩权重，不同教师给予不同的权重。

14. **简单的职工管理系统**。职工对象包括姓名、部门、开始工作时间、职务、年龄、性别、生日、学历、住址、电话、工资。要求：(1) 实现增加一个职工的信息（包括加入批量的数据），并允许某些参数不输入；(2) 删除 满足条件的职工对象；(3) 查询符合某些条件的职工对象；(4) 对检索出的某些对象进行修改；(5) 根据某些 需要对显示的结果进行排序；(6) **实现对某些域建立索引**（可以是 B 树、B^+ 树、散列 索引或者其他高效率的索引），并比较使用索引和不使用索引的查询速度；(7) 对于输入的职工信息要有 基本的纠错能力，例如像输入出生年月日是否合法；(8) 排序要能够按照用户的格式进行，包括字母排序、数字排序、日期排序、升序、降序、多列排序等；(9) 数据量规模在一万以上。加分功能：支持带有 *、? 等通配符的字符串查询（需要正则表达式匹配算法）；支持简单的 SQL 语句的查询；能支持百万以上的 职工数据量。
15. **搜索引擎**。搜索引擎是一种收集、组织和处理大量信息资源，并向用户提供高效检索服务的工具。由于搜索引擎涉及图、排序、索引、检索等数据结构技术，而且设计搜索引擎这个综合上机题也能充分 激发学生综合应用数据结构知识的兴趣，**因此多年来北京大学计算机系一直把此题作为计算机专业 学生的数据结构学期综合上机实习题**。一个搜索引擎大体可以分为 3 个部分：(1) **网络爬虫**（spider / web crawler）也称为「搜索器」，在互联网上沿网页链接访问并保存页面信息，作为检索信息的 来源——Web 页作为结点、网页之间的链接作为边，可以把 Web 页的关系表示为图，**网络爬虫的工作 本质上就是图的搜索**；(2) **索引器**（indexer）对爬虫收集来的网页信息，构造出便于快速访问和检索的 索引数据结构，一般采用倒排索引（第 11 章）；(3) **查询处理**（retriever）也称为「检索器」，分为前台和 后台两部分，绝大部分实用搜索引擎都加入了结果排级（ranking）的功能模块以改进搜索结果——google 的成功就在于它所发明的 PageRank 算法有效地解决了文档的相关性问题。实习目标：应用已经 掌握的数据结构基础知识，设计并实现一个搜索引擎原型（可以是网页搜索、FTP 搜索，也可以是 MP3 音乐、电影等专题搜索），该原型能对一定规模的资源进行索引，能在合理的响应时间内满足用户的检索 请求并返回结果。
16. Google Suggest 的「提示」文本框等利用了 Ajax 控件，其后台有一个用户常用查询词词典。试帮助 设计一个英文词典，该字典要能很好地支持前缀串查询——例如输入 `comp`，在提示框中应该能动态地 返回查询结果最多的 10 个相关词，按照访问频率在输入框中顺序排列这 10 个词。

习题、上机题与参考答案

本附录把各章正文后的题目整理成可核对的参考答案。答案给出关键结论、算法不变量和复杂度；编程题 的完整实现应优先调用本书 `code/` 中已经通过测试的接口。除非题目另有说明，下标从 0 开始。

每题前的来源分三类：`课程答案` 指 `ref_ 数据结构与算法 A 2021 秋/2021 书面作业答案 1-12 章/` 中的原题答案；`ref_DSA` 指仓库中 相应 Markdown/PDF；`本书自拟` 指为现代化教材重新设计。来源答案若有错误，本书保留来源标记并在 答案中校正，不照抄错误结论。

每章分三组，编号各自独立，不要混着看：

小节	对应正文哪一部分	编号
补充题参考答案	各章 `## 习题` 里那组「补充…（参考课程第 N 章）」	与该组题号一致
正文习题答案	各章 `## 习题` 的正文题	**与正文题号一致**
正文上机题参考	各章 `## 上机题`	**与正文题号一致**
课程作业与 `ref_DSA` 参考答案	课程作业与仓库习题里的题，**不是正文的题**	自成一套，与正文题号无关

正文题的覆盖情况由闸门看着（`check_doc.py` R14）：每道题要么在这里有同号答案，要么在 `collab/answer_gaps.json` 里登记为「欠着」或「有意留作练习」，两者必居其一。**闸门只验「有没有同号答案」，不验答案对不对**——那要人复核。

第 1 章 概论

补充题参考答案

1. 外层 `i=1..n-1`，内层执行 `n-i+1` 次，因此赋值次数为 `sum(n-i+1, i=1..n-1)=n(n+1)/2-1`，为 `Theta(n^2)`。
2. 由 `max(f,g) <= f+g <= 2 max(f,g)`，取常数即可得 `Theta` 关系。
3. 递推树每层总成本 `n`，深度 `Theta(log n)`，故为 `Theta(n log n)`；归纳证明可用代入上下界完成。

正文习题答案

1. 读入 X, Y, Z 后三次比较即可定序：若 $X < Y$ 交换，若 $X < Z$ 交换，若 $Y < Z$ 交换，此时 $X \geq Y \geq Z$，顺次输出。比较与交换都是常数次，时间 $O(1)$、空间 $O(1)$。注意「从大到小」允许相等，判断要用 `<` 而不是 `<=`，否则相等元素会白白交换。
2. 数据结构 = 一组数据 + 它们之间的关系 + 允许的运算。例如「班级花名册」按学号排成一列，前后相邻，是**线性表**；「城市间的航线」两两之间可能有边，

是**图**；「文件夹套文件夹」是**树**。同一批数据换一种关系去看，就是另一种数据结构，能做的运算和代价也随之变化（见 1.2.1）。

3. 四种基本存储方式：**顺序**（元素连续存放，靠下标算地址，随机访问 $O(1)$，插删要移动）、**链接**（结点分散，靠指针相连，插删 $O(1)$ 但不能随机访问，且每个结点多花指针空间）、**索引**（另建索引表指向记录，逻辑有序而物理无序，见 1.4 与第 11 章）、**散列**（由关键码直接算地址，平均 $O(1)$，需处理冲突，见第 10 章）。
4. **循环体执行 4 次。** `do-while` 先执行后判断：$i=1$ 时 $s=10$，$i=2$ 时 $s=30$，$i=3$ 时 $s=60$，$i=4$ 时 $s=100$，此时 $s<n$ 不再成立，退出。一般地 $s=5i(i+1)$，循环次数由 $5i(i+1)\ge n$ 决定，是 $O(\sqrt{n})$ 而不是 $O(n)$——这道题要的就是别被 `i<n` 那一半条件骗过去。
5. `B(s,q)` 从 s 出发走到「下一个是 q」的结点 p，令 `p->next = s`，于是 s 到 p 这一段自成一个循环链表。`A(h,g)` 先后做 `B(h,g)`、`B(g,h)`，**把一个循环链表在 h 与 g 两处剪开，拆成两个循环链表**：一个含 h 到 g 之前，另一个含 g 到 h 之前。两次遍历各走一段，合起来正好绕原表一圈，时间 $\Theta(n)$、空间 $\Theta(1)$。
6. 内层对每个 i 执行 $\max(0,n-2i+1)$ 次，只有 $i\le n/2$ 才进入，于是

$$m=\sum_{i=1}^{n/2}(n-2i+1)=\frac{n}{2}(n+1)-2\cdot\frac{(n/2)(n/2+1)}{2}=\frac{n^2}{4}.$$

   （$n=10$ 时 $m=25$，$n=100$ 时 $m=2500$，与公式一致。）时间 $\Theta(n^2)$。
7. $T_1(n)=O(1)$；$T_2(n)=O(n^2)$；$T_3(n)=O(n^3)$。大 O 只保留增长最快的项并丢掉常数因子——1000 再大也是常数。
8. (1) $O(n)$；(2) $O(n^2)$。
9. 辗转相除：$\gcd(a,b)=\gcd(b,a \bmod b)$，$b=0$ 时答案是 a。最坏情形是相邻斐波那契数，迭代次数 $O(\log\min(a,b))$；迭代写法空间 $O(1)$，递归写法额外占 $O(\log\min(a,b))$ 的调用栈。
10. 一小时的预算是 $10^{10}\times 3600=3.6\times 10^{13}$ 次操作，各自能处理的规模：

复杂度	可处理的 n
n^2	6.0×10^6
n^3	3.3×10^4
$100n^2$	6.0×10^5
$n\log_{10}n$	$\approx 2.9\times 10^{12}$
2^n	45
2^{2^n}	5

   最后两行是这道题的重点：机器再快一倍，2^n 也只多算一个 n，2^{2^n} 连一个都不一定多得了。

11. (1) **不成立**。取 $f(n)=2$、$g(n)=1$，则 $f=O(g)$，但 $\log_2 f=1$、$\log_2 g=0$，$1=O(0)$ 不成立。（若另加 $f,g\ge 2$ 的前提则成立。）(2) **不成立**。取 $f(n)=2n$、$g(n)=n$，$2^{2n}=(2^n)^2$ 不是 $O(2^n)$。(3) **成立**：由 $f\le cg$（n 充分大）得 $f^2\le c^2g^2$。
12. (1) $\Theta(\log n)$——每次规模减半、只加常数；(2) $\Theta(n)$——递归树有 n 个叶、每个结点只花常数，总和与结点数同阶；(3) $\Theta(n\log n)$——每层总代价都是 n，共 $\Theta(\log n)$ 层（与补充题第 3 题同一个递推）。

正文上机题参考

1. 设 $f(m,n)$ 是「把 m 拆成若干个不超过 n 的自然数之和」的方案数，按「用不用一个 n」分两类：

$$f(m,n)=f(m-n,\,n)+f(m,\,n-1).$$

   边界：$f(0,n)=1$（空拆法算一种）、$m>0$ 且 $n=0$ 时为 0、$n>m$ 时取 $f(m,m)$。验证 $f(5,3)=5$，与题面给的五种一致。直接递归会重复计算，改成 $O(mn)$ 的二维递推即可；空间可滚动到 $O(m)$。
2. 复数 ADT 的要点全在接口上，不在算术上：三个构造函数不要用默认参数塞成一个（题目要的就是重载）；**单参数构造要写** `explicit`，否则 `Complex c = 3.0;` 这种隐式转换会悄悄发生；四则运算写成对称的自由函数（或 `friend`）比写成成员更自然，因为 `2.0 + c` 也该成立；除法要先判分母模长为零并抛 `std::invalid_argument`（D-001 §3：容器与 ADT 内部不打印，错误靠异常）；`operator<<` 只负责格式化，不做判断。本书不用 `std::complex` 代替——这道题练的正是「自己定义一个类型」。

课程作业与 `ref_DSA` 参考答案

1. **[本书自拟]** 数据结构的四个层次可分别写成逻辑结构、存储结构、数据运算和抽象数据类型；算法分析至少报告时间代价与空间代价。
2. **[本书自拟]** “程序 = 数据结构 + 算法” 强调数据表示和求解步骤相互制约；同一问题换存储结构后，算法复杂度可能改变。
3. **[教材勘误错误 0、1]** 连续有序子数组必须在原数组中相邻。若端点为 `j1..j2`，长度是 `j2-j1+1`，不是 `j2-j1-1`。
4. **[教材勘误错误 6；课程第一章答案第 2 题]** `T(n)=2T(floor(n/2))+n` 的递推树每层总成本为 `n`，层数为 `Theta(log n)`，所以 `T(n)=Theta(n log n)`。
5. **[张铭教材；本书代码验证]** 股市传言例中 B3 的最慢最短传播时间为 10；B1、B2、B4、B5 至少有一个不可达顶点，因此答案是 B3。
6. **[`ref_DSA/homework/1.md` 第 1 题；2025 秋书面作业]** `for (i=1; i<=n; i*=2) for (j=1; j<=n; j+=i) cnt++;`：外层取 $i=2^k$，$k=0..\lfloor\log_2 n\rfloor$；内层对

固定 i 执行 $\lceil n/i \rceil$ 次（$j = 1, 1+i, \ldots$ 不超过 n）。所以

$$cnt = \sum_{k=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^k} \right\rceil, \qquad n \le cnt < \sum_{k} \left(\frac{n}{2^k} + 1 \right) < 2n + \log_2 n + 1.$$

即 $cnt \approx 2n$，时间 $\Theta(n)$。**别被两层循环骗成** $O(n \log n)$——内层步长随外层翻倍，各层工作量是等比级数。实测 $n = 10^3$ 时 $cnt = 2000$，$n = 10^6$ 时 $cnt = 2000006$。

7. **[ref_DSA/homework/1.md 第 2 题；2025 秋书面作业]** (1) $\Theta(3^n)$：$n^3 2^n / 3^n = n^3 (2/3)^n \to 0$；(2) 展开 $T(n) = T(1) + \sum_{m=2}^{n} \Theta(m) = \Theta(n^2)$；(3) $\sum_{m=2}^{n} \Theta(1/m) = \Theta(\ln n)$（调和级数），故 $\Theta(\log n)$；(4) $\sum_{m=2}^{n} \Theta(\ln m) = \Theta(\ln n!) = \Theta(n \log n)$（Stirling，或上界 $n \ln n$、下界取后一半 $\frac{n}{2} \ln \frac{n}{2}$）。题目要的是 Θ；只写 Ω 或只写 O 都不算答完。
8. **[ref_DSA/homework/1.md 第 3 题；2025 秋书面作业]** (1) $b^n \le 1 \cdot a^n$ 对一切 $n \ge 1$ 成立，故 $b^n = O(a^n)$。反设 $a^n \le c\, b^n$（$n \ge n_0$），则 $(a/b)^n \le c$；但 $a/b > 1$，$(a/b)^n \to \infty$，矛盾。(2) 强归纳证 $T(n) \le \log_2 n$：$T(1) = 0$；设对一切 $m < n$ 成立，则 $T(n) = T(\lfloor n/2 \rfloor) + 1 \le \log_2 \lfloor n/2 \rfloor + 1 \le \log_2 (n/2) + 1 = \log_2 n$。**归纳假设里不能写** $O(\log m)$——大 O 的常数在每步归纳里都可能变大，得先落成一个具体的不等式再归纳。

课程上机参考

用邻接矩阵实现 Floyd：初始化对角线为 0、不可达为 `infinity`，再按 `via/from/to` 顺序松弛；最后逐行取最大有限距离并选择最小者。复杂度 `O(V^3)`、空间 `O(V^2)`。

第 2 章 线性表

正文习题答案

1. 有序顺序表插入 x：先二分找到第一个大于 x 的位置 k（$O(\log n)$），再把 $[k, n)$ 整体后移一格、写入 x（$O(n)$）。总代价仍是 $O(n)$——**二分省的是比较，不是移动**，这一点在顺序表上必须说清楚。实现见 `code/ch02/array_list`：`insert` 会先 `ensure_capacity`，扩容按翻倍，摊还 $O(1)$。
2. 逐位比较 a_i 与 b_i，遇到第一个不等的位置就定胜负；若走到某一表的末尾仍全等，则**短者小**（A 是 B 的前缀时 $A < B$，长度相同则相等）。时间 $O(\min(m, n))$、空间 $O(1)$。这正是字典序，与第 4 章字符串比较是同一套规则。
3. 递增单链表删除所有满足 $\text{min} < x < \text{max}$ 的结点：一次遍历，用「前驱指针 + 当前指针」把落在区间内的结点摘下并 `delete`；因为表是递增的，一旦当前值 $\ge$ max 就可以停。时间 $O(n)$、空间 $O(1)$。注意删除的是**开区间**，等于 min 或 max 的结点要留下。

4. 两个递增单链表归并成一个**递减**表，且只用原结点：反复取两表当前较小者，用**头插法**接到结果表头。头插会把顺序反过来，于是取出来是递增、接上去就成了递减；一个新结点都不申请。时间 $O(m+n)$、额外空间 $O(1)$。
5. 拆成三个循环链表：遍历原表，按 `isalpha` / `isdigit` / 其它把结点**摘下来**尾插到对应的表；每个表只保存尾指针 `tail`，尾插是 $tail \to next$ 与 `tail` 各改一次，$O(1)$。最后每个表都要闭合成环（末结点指回首结点），空表以 `tail == nullptr` 表示。时间 $O(n)$、不新建结点。循环链表的写法见 2.3.3。
6. 顺序表删除从第 i 个起的 k 个元素：先查合法性（$1 \le i$、$i+k-1 \le n$、$k \ge 0$），越界抛 `std::out_of_range`；再把 $[i+k-1, n)$ 的元素整体左移 k 位，最后长度减 k。移动次数 $n-i-k+1$，时间 $O(n)$。**一次移动 k 位，而不是调用 k 次「删一个」**——后者是 $O(kn)$。
7. 递增单链表去重：相邻比较，`cur->value == cur->next->value` 就摘掉后一个并 `delete`，否则前进。因为有序，相等的一定相邻，一趟就够，$O(n)$。
8. 两个递增无重复顺序表求交集：双指针，相等则写入 C 并同时前进，否则前进较小的一方。时间 $O(m+n)$、结果自然递增。若改用「对 A 的每个元素在 B 里二分」，代价是 $O(m \log n)$——m 与 n 接近时不如双指针。
9. 单链表版本同样是双指针,区别在于**不能随机访问**,所以只能顺着走一遍;这恰恰说明双指针法为什么在链表上仍然适用，而二分法不再适用。时间 $O(m+n)$。
10. 括号配对：从左往右扫，遇到左括号入栈，遇到右括号就与栈顶比对——**类型要匹配**，栈空则说明右括号多了；扫完（遇到 `#`）时栈必须为空，否则左括号多了。时间 $O(n)$、空间 $O(\text{嵌套深度})$。栈的实现见第 3 章，`code/ch03/expression_eval` 里有同一套思路的求值程序。

正文上机题参考

1. 单链表原地逆置的三指针写法：`prev = nullptr`、`cur = head`，每轮先记下 `next = cur->next`，再把 `cur->next` 指向 `prev`，然后三个指针一起前移；结束时 `prev` 就是新表头。**循环不变量**：`prev` 指向已经逆置好的那一段的头，`cur` 指向尚未处理的那一段的头，两段合起来恰是原表的全部结点。时间 $O(n)$、额外空间 $O(1)$（只有三个指针，与 n 无关）。容易错的两处：忘了先保存 `next` 就改指针，链表当场断成两截；结束后忘了把原头结点的 `next` 置空，逆置完的表尾会指回去形成环。测试至少覆盖空表、单结点、两结点和一般情形——`code/ch02/linked_list/test.cpp` 里的边界用例可以照抄这个口径。
2. 倒数第 `m` 个（题目约定 `m=0` 是尾结点）用双指针：`fast` 先走 `m+1` 步，若不足则 `m` 越界；随后 `fast`、`slow` 同步前进到 `fast==nullptr`，此时 `slow` 即答案。空表、`m<0`、`m>=n` 返回空值。一次扫描，时间 `O(n)`、额外空间 `O(1)`。
3. Josephus 用带尾指针的循环单链表。先让 `prev` 指向开始报数者的前驱；每轮向前移动 `m-1` 次，删除 `prev->next`、记录其编号，再从删除位置的后继继续。`n<=0` 或 `m<=0` 拒绝；`s` 规范化为 `((s-1)%n+n)%n+1`。模拟时间 `O(nm)`、结点空间 `O(n)`；若只要最后幸存者可用递推 `J(1)=0, J(k)=(J(k-1)+m)%k`。

课程作业与 ref_DSA 参考答案

1. **[本书自拟]** 顺序表随机访问为 O(1)；任意位置插入或删除平均需移动 O(n) 个元素；尾部追加在有容量时为 O(1)，扩容实现通常为摊还 O(1)。
2. **[课程第二章答案第 1 题；ref_DSA/homework/2.md 第 2 题]** 有序表原地去重用双指针：read 扫描输入，write 指向下一个不同值；每次发现新值写入 write++。时间 O(n)，额外空间 O(1)。
3. **[课程第二章答案第 3 题；ref_DSA/homework/2.md 第 3 题]** 单链表判环用 Floyd 快慢指针。相遇后令一个指针回到头结点，两指针每次走一步，再次相遇处就是环入口；时间 O(n)，空间 O(1)。课程答案只要求判环，ref_DSA 题目进一步要求入口，本书补全第二阶段。
4. **[ref_DSA/homework/2.md 第 1 题]** 双向循环链表删除 p：令 p->prev->next=p->next、p->next->prev=p->prev，再释放 p；空表和删除唯一结点时要更新头指针。

课程上机参考

实现去重、判环和双向链表删除后，至少测试空表、单结点、重复全表、环入口为头结点和环入口在中间五组边界。参考 code/ch02/linked_list/test.cpp 的异常和移动语义测试方式。

第 3 章 栈与队列

补充题参考答案

1. **[课程第三章答案第 1 题]** 入队压入 S1；出队时若 S2 空则把 S1 全部转入 S2，再弹出 S2。每个元素至多转移一次，摊还 O(1)。
2. **[课程第三章答案第 2 题]** 设最后出栈元素为 k，左右两侧独立产生 C_{k-1} 与 C_{n-k}，所以 C_n=sum(C_{k-1}C_{n-k})，解为 1/(n+1) binom(2n,n)。
3. **[ref_DSA/homework/3.md 第 3 题]** 撤销执行逆操作并把原操作放入恢复栈；恢复执行原操作并放回撤销栈；新操作入撤销栈。**新操作是否清空恢复栈，题面没有规定**——真实编辑器会清空（新操作之后，旧的「可恢复历史」已经接不上），但照题面给的三条规则字面执行则不清空；两种约定在第 (2) 问会给出不同答案，见「课程作业」第 3 条。

正文习题答案

1. 用数组 data[n]、front（队头）和 count 表示双端队列，队尾下标为 (front+count-1)%n。队尾删除取该下标后 --count；队头插入先令 front=(front+n-1)%n，再写入并 ++count。四种操作都先判空/满，时间均为 $O(1)$。
2. 判空即 count==0，队尾位置由 (front+count)%n 算出。入队写该位置后 ++count；出队取 data[front]，令 front=(front+1)%n、--count。count 同时消除了 front==rear 无法区分空和满的问题。

3. (1) 两个额外栈：把 S 依次倒入 A、再倒入 B、最后倒回 S；每倒一次逆置一次，三次逆置的净效果是逆置。(2) 一个队列：把 S 全弹入队列，再依次出队压回 S——先出栈的栈顶先出队、先压回，于是落到栈底。(3) 一个额外栈 T 加两个整型变量（计数 k、暂存 x）**也能做到**，代价是 $O(n^2)$：对 $k = n, n-1, \dots, 1$，栈 S 顶上的 k 个元素是尚未处理的部分——弹出栈顶存进 x，把其余 $k-1$ 个倒进 T，压入 x，再把 T 的 $k-1$ 个倒回 S。每轮把当前栈顶「沉」到未处理段的最底部、其余元素次序不变，n 轮后整个栈被逆置。例：S（底→顶）$= 1, 2, 3$，第一轮后为 $3, 1, 2$，第二轮后为 $3, 2, 1$。只用 $O(1)$ 个标量并不妨碍逆置，因为暂存区是那个额外栈，标量只需记一个元素和一个计数。（脚本对 $n \le 7$ 全部验证。）
4. 用输入栈 `in` 和输出栈 `out`：`enqueue(x)` 压入 `in`；`dequeue()` 在 `out` 非空时直接弹，否则把 `in` 全倒入 `out` 后弹。每个元素最多转移一次，一串操作的摊还时间为 $O(1)$。
5. 令 `rear` 指向尾结点，队头是 `rear->next->next`（`rear->next` 为头结点）。初始化让头结点自环且 `rear=head`；入队把新结点插在头结点前并更新 `rear`；出队删 `head->next`，删掉唯一数据结点后恢复 `rear=head`。均为 $O(1)$。
6. 栈 1 从数组左端向右长，栈 2 从右端向左长，栈顶下标初值分别为 `-1`、`n`。仅当 `top1+1==top2` 才溢出；两边 `push`/`pop` 都是 $O(1)$，空闲空间由两栈共享。
7. 合法出栈序列数是 Catalan 数 $C_5 = 10!/(6! \times 5!) = 42$。
8. 必要性：若 $i < j < k$ 且 $p_j < p_k < p_i$，输出 p_i 时比它小的 p_j、p_k 都已入栈且尚未输出（它们排在后面）；输入递增，p_j 比 p_k 先入栈、压在 p_k 下面，因此 p_k 必先于 p_j 输出，与 $j < k$ 矛盾。充分性：按目标序列贪心模拟——要输出 p_k 时，若它还没入栈，就把输入压到 p_k 再弹出；若它已在栈里，设它不在栈顶，栈顶为 t。t 比 p_k 晚入栈，故 $t > p_k$；t 之所以已入栈，是因为此前为输出某个 p_i（$i < k$）压过了它，故 $p_i > t$；t 尚未输出，它在目标序列里位于 k 之后。于是 $p_k < t < p_i$ 恰好构成禁形（把 p_k 当作 p_j、t 当作 p_k）。所以没有禁形时贪心每一步都能弹出所需元素。（脚本对 $n \le 7$ 的全部排列核对过「可由栈得到」与「无禁形」等价。）
9. 栈变化为 `[] -> [12] -> [12,8] -> [12,8,9] -> [12,72] -> [84]`，结果为 84。
10. 逐个读入符号时操作符栈依次为：初始 `[]`；读 `*` 得 `[*]`；读 `(` 得 `[*,(]`；读 `*` 得 `[*,(,*]`；读 `-` 时栈顶 `*` 优先级不低于 `-`，先弹出输出，再压入，得 `[*,(,-]`；读 `)` 弹出 `-` 并丢掉 `(`，得 `[*]`；读 `+` 时弹出 `*`，再压入 `+`，得 `[+]`；读完后弹空，得 `[]`。操作数直接输出，最终后缀式为 `abc*d-*e+`。（脚本逐步复算；读 `)` 之后栈里还剩 `[*]`，不是空栈。）
11. 欧几里得递归为 `gcd(n,m) = (m==0 ? abs(n) : gcd(m,n%m))`。每次余数严格变小，最坏递归深度 $O(\log \min(|n|, |m|))$。
12. 令 $a_1 = 1$，$a_k = (-1)^k / k$（$k \ge 2$）。递归 `sum(n)=sum(n-1)+a_n`，边界 `sum(1)=1`；非递归从 1 开始循环累加。两者时间 $O(n)$，递归额外栈 $O(n)$、迭代空间 $O(1)$。

正文上机题参考

1. 采用上题两个栈方案；若复用本书容器，`ArrayStack::push`、`ArrayStack::pop`、`ArrayStack::empty` 是实际接口。守门用例要覆盖空队列、交错入出和 `out` 非空时继续入队。
2. 用 `front`、`count` 和循环数组实现，四端操作均按模取下标。至少测试容量 1、满后拒绝、绕回以及从两端删到空；本书的实际接口是 `ArrayQueue::enqueue`、`ArrayQueue::dequeue`、`ArrayQueue::empty`、`ArrayQueue::full`，但它本身不是双端队列。
3. 思路都是「插入排序，但只能从队头取、往队尾放」，借助长度计数器绕圈。(2) 一个辅助队列 A（保持升序）：从原队列 Q 逐个出队得到 x，把 x 插进 A——让 A 转一整圈（出队 $m=|A|$ 次再入队），在第一次遇到大于 x 的元素之前把 x 入队，若一圈都没遇到就在圈尾入队；全部插完后把 A 倒回 Q。(1) 两个辅助队列 A、B：A 保持升序，插入 x 时把 A 中小于 x 的元素依次移到 B，接着把 x 放进 B，再把 A 剩下的移到 B，然后交换 A、B 的角色（归并式插入）。两种做法都只用到计数器、标志等非数组变量，时间 $O(n^2)$。
4. (1) 一个辅助栈：`s1` 全部倒入 `aux`（逆序一次），再把 `aux` 全部倒入 `s2`（再逆序一次），两次逆序抵消，`s2` 与原 `s1` 次序相同。(2) 只用非数组变量**也能做到**，`s2` 本身就可以充当暂存区：对 $k=n,n-1,\dots,1$，把 `s1` 顶上的 $k-1$ 个元素倒到 `s2` 上，弹出此时 `s1` 剩下的栈底元素存进 x，把那 $k-1$ 个倒回 `s1`，再把 x 压进 `s2`。每轮把 `s1` 当前的栈底搬到 `s2`，所以 `s2` 从底到顶依次收到原栈底到原栈顶，次序不变；变量只要计数 k 和暂存 x，时间 $O(n^2)$。（脚本对 $n\le 7$ 验证。）
5. 递归 `fib(n)=fib(n-1)+fib(n-2)` 会重复计算，时间指数级。显式栈帧保存 `(n, 阶段, 左结果)` 可模拟调用；更实用的非递归只保留前两项，时间 $O(n)$、空间 $O(1)$。
6. (1) 按定义逐层展开：$Ack(2,1)=Ack(1,Ack(2,0))$，其中 $Ack(2,0)=Ack(1,1)=Ack(0,Ack(1,0))=Ack(0,Ack(0,1))=Ack(0,2)=3$；于是 $Ack(2,1)=Ack(1,3)=Ack(0,Ack(1,2))$，而 $Ack(1,2)=Ack(0,Ack(1,1))=Ack(0,3)=4$，所以 $Ack(2,1)=Ack(0,4)=5$（一般地 $Ack(1,n)=n+2$）。(2) 非递归算法用栈保存「还欠着的外层 m」：先把 m 压栈；当栈非空时弹出 m——若 `m==0`，令 `n=n+1`（这就是内层调用的结果，交给下一次弹出的外层用）；若 `n==0`，压入 `m-1` 并令 `n=1`；否则压入 `m-1`、再压入 `m`，并令 `n=n-1`（先算内层 $Ack(m,n-1)$）。栈空时 n 即答案。数值增长极快，必须设溢出和资源上限。（脚本对 $m\le 2$、$n\le 4$ 与递归版对拍。）

课程作业与 `ref_DSA` 参考答案

1. **[课程第三章答案第 1 题；`ref_DSA/homework/3.md` 第 4 题]** 两个栈实现队列：入队只压入 `in`；出队时若 `out` 为空，把 `in` 全部弹出压入 `out`，再从 `out` 弹出。单次最坏 `O(n)`，摊还 `O(1)`。课程答案文字中 “`S1` 满时搬运” 不是必要条件，代

码实际采用的也是只在出队栈为空时搬运，本书按代码校正。

2. **[课程第三章答案第 2 题；**`ref_DSA/homework/3.md` **第 2 题]** n 辆车进出栈的合法出站序列数为 Catalan 数 `C_n = 1/(n+1) * binom(2n,n)`。
3. **[**`ref_DSA/homework/3.md` **第 3 题；2025 秋书面作业]** (1) `do x`：`S1.push(x)`；`undo`：`S1` 非空时 `x=S1.pop(); S2.push(x)`；`redo`：`S2` 非空时 `x=S2.pop(); S1.push(x)`。(2) `do A, do B, do C, undo, do D, undo, redo` 的终态：`S1`（**底→顶**）= A, B, D，两种约定一致；`S2` **取决于** `do` **是否清空恢复栈**——按题面三条规则字面执行（不清空）为 C，按真实编辑器语义（清空）为 **空**。关键在 `do D` 这一步：不清空时 `C` 一直压在 `S2` 底下，此后再 `redo` 两次就会把一个「在 D 之前被撤销、与当前历史接不上」的 C 重新执行。答题要先声明约定，再给终态；只写一个答案而不说约定，两种批改标准都可能判错。（脚本逐步模拟两种约定，结果如上。）
4. **[张铭教材；本书自拟]** 后缀表达式求值遇到操作数入栈，遇到运算符弹出右操作数和左操作数再压回结果；除法前检查右操作数非零。时间 `O(n)`。
5. **[**`ref_DSA/homework/3.md` **第 1 题；2025 秋书面作业]** O(1) 的 `getMin`：题目不许添加新的数据结构，那就**改变栈里存的东西**——每个栈元素存成二元组 `(value, min_so_far)`，`min_so_far` 是「从栈底到这个元素」的最小值。`push(x)`：`m = S.empty() ? x : min(x, S.top().min)`，压入 `(x, m)`；`pop()`：弹出并返回 `.value`；`top()`：返回 `S.top().value`；`getMin()`：返回 `S.top().min`。四个操作都是 $O(1)$。正确性的关键是**最小值随栈一起回退**：弹掉当前最小值后，新栈顶里存的恰好是剩余元素的最小值，不需要重新扫描。**常见错法是只用一个全局变量** `currentMin`——`push(3), push(1), pop()` 之后它仍是 1，而栈里只剩 3；要么像上面这样把最小值存进每个元素，要么在 `pop` 时从新栈顶读回来，二者本质相同。只存一个值时也可以「每次压两个数」（先压 `x` 再压当前最小），读写都要成对，空间是两倍。
6. **[**`ref_DSA/homework/3.md` **第 4 题；2025 秋书面作业]** 两个队列实现栈：始终保持至多一个队列非空。`push(x)` 入非空的那个队列（都空则任选），$O(1)$；`pop()` 把非空队列的前 $n-1$ 个元素逐个出队并入另一个队列，剩下的最后一个就是栈顶，出队返回，$O(n)$；`isEmpty()` 为两队列都空。按 `push(1), push(2), push(3), pop(), push(4), pop(), pop()` 执行，三次 `pop` 依次返回 3、4、2。伪代码里「搬运」循环的条件是**队列长度大于** 1（或「非空」），写成 `while (isEmpty(Q1))` 是条件写反，一次都不会进入。

课程上机参考

分别运行递归、显式栈和优化版背包实现，用穷举结果对拍；再测空物品、容量 0、无解、负容量和非正重量。队列应测试满、空、环回和重复入队出队。

第 4 章 字符串

正文习题答案

1. s 的下标为 `(0x1y2z3)4+5*6`。t 需要的每个字符 s 里都有,只靠抽子串和连接就能拼出：`s.substr(0,2) + s.substr(5,1) + s.substr(3,2) + s.substr(6,1) + s.substr(2,1)`，即 `"(x" + "+" + "z)" + "*" + "y"` = `(x+z)*y`（脚本核对）。也可以先拼 `"(" + s.substr(1,1) + "+" + s.substr(3,1) + ")*" + s.substr(2,1)`，但那样用到了 s 以外的字面串，不如全部取自 s 贴合题意。
2. 当且仅当存在某个串 u 和非负整数 m, n，使 $s_1 = u^m$、$s_2 = u^n$；空串对应指数 0。取两串的最短公共周期，连接可交换迫使二者都由该周期重复组成。
3. 建 36 个计数器：`A..Z` 映到 `0..25`，`0..9` 映到 `26..35`，一次扫描累加；其他字符忽略或按输入契约报错。时间 $O(n)$、固定额外空间。
4. 0 起始下标时先按 `0,2,4,...` 收集偶下标字符，再按最大的奇下标到 1 收集奇下标字符。`ABCDEFGHIJKL` 得 `ACEGIKLJHFDB`；一次新缓冲扫描为 $O(n)$ 时间、$O(n)$ 空间。
5. 双指针从两端向中间比较，任一对不同返回 0，交错后返回 1；时间 $O(n)$、空间 $O(1)$。
6. 先给 t 的字符建存在表，再扫描 s；字符不在 t 且第一次出现时，追加到新串并记录当前位置。时间 $O(|s| + |t|)$，字节字母表下额外空间 $O(1)$。
7. 判据是 $|T| = |T'|$ 且 T 是 $T' + T'$ 的子串，用 `kmp_search(T'+T', T)` 在 $O(|T|)$ 时间完成。原书用的例子 `arc` 与 `car` 正是这样：`arc` 的循环移位是 `arc`、`rca`、`car`。**不要先反转再找**——`reverse("arc")="cra"`，`"cracra"` 里根本没有 `car`，那个判据会把原书自己的例子判否。
8. 从左到右找 t 的非重叠出现位置，把未匹配片段写入新缓冲；空 t 必须单独约定为不删除。用 KMP 为 $O(|s| + |t|)$，朴素反复查找最坏 $O(|s||t|)$。
9. 按本书 `dsa::build_next` 的优化定义，`BAAABBBAA` 的特征向量是 `[-1,0,0,0,-1,1,1,0,0]`。匹配过程（i 为目标串下标，j 为模式下标，均从 0 起）：第一趟 $T[0..6]$ =`BAAABBB` 与 $P[0..6]$ 全部相等，在 $i = 7$ 处 `C`≠ $P[7]$ =`A` 失配，$j \leftarrow next[7] = 0$；`C`≠`B`，$j \leftarrow next[0] = -1$，$i$、$j$ 同时前进。此后 $i = 8..16$ 的 `DDDCCHHHH` 都与 $P[0]$ =`B` 失配，每次都是 $j \leftarrow -1$ 后前进。$i = 17$ 的 `B` 匹配，$i = 18$ 的 `B`≠ $P[1]$ =`A`，$j \leftarrow next[1] = 0$，$P[0]$ 与 $T[18]$ 匹配；$i = 19$ 又是 `B`≠ $P[1]$，$j \leftarrow 0$，$P[0]$ 与 $T[19]$ 匹配，接着 $T[20..27]$ =`AAABBBAA` 逐个匹配成功。首个完整匹配从 **下标 19** 开始，共做 31 次字符比较（脚本复算）。全过程文本下标 i 从不回退，只按向量回退模式下标。

正文上机题参考

1. 跳过可选负号后逐位做 `value=value*10+digit`；拒绝空串、孤立负号和非数字，并在乘 10、加当前位之前检查 `int` 上下界。时间 $O(n)$。

2. 扫描时遇数字开始累积，直到非数字才输出并计数；文件结束时若仍在数字中要补输出一次。若允许负整数，只有紧邻数字的 - 才作符号。
3. 可把每个后缀排序，再比较相邻后缀的最长公共前缀；abcdacdac 的最长重复子串为 cdac，起点 2 和 5。规模大时应改用后缀数组或后缀自动机。
4. 行表用 std::vector<String> 或链表保存，当前行是下标/迭代器；插删后统一修正当前行。查找替换必须调用实际接口 dsa::build_next、dsa::kmp_search，字符串切片可用 String::substr，不能把环境自带 find 冒充 KMP。

课程作业与 ref_DSA 参考答案

1. **[课程第四章答案第 1 题；ref_DSA/homework/4 字符串.md 第 1 题；2025 秋书面作业；脚本复算]** 题目问的是 KMP **与自动机（DFA）的区别和联系**，以 P = ababaca、字符集 $\{a,b,c\}$ 为例。先算 KMP 的两种特征数组。P 各前缀的最长相等真前后缀长度（border）为 [0,0,1,2,3,0,1]；右移一位、首项填 -1 得**未优化**的 next [-1,0,0,1,2,3,0]；本书 dsa::build_next 的**优化版**（$P_i = P_k$ 时借用 next[k]）为 [-1,0,-1,0,-1,3,-1]。再造 DFA：状态 $q \in \{0,\dots,7\}$ 表示「已匹配 P 的前 q 个字符」，**共 8 个状态、7 是接受态**（模式长 7，不是 6）。$\delta(q,x)$ 是「$P[0..q)$ 后接 x」的最长后缀使其同时是 P 的前缀的长度：

状态	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	5	0	0
5	1	4	6
6	7	0	0
7	1	2	0

联系：两者维护的是同一个量——扫描到文本当前位置时，文本后缀与模式前缀的最长匹配长度。DFA 的转移表可以完全由 border 数组递推出来：$\delta(q,x) = q+1$（若 $q<7$ 且 $P_q = x$），否则 $\delta(q,x) = \delta(\text{border}(q),x)$，$q=0$ 时为 0（脚本验证与按定义逐项计算的表一致）。换句话说，KMP **的** next **数组就是这台** DFA **的「失败转移」压缩表示**：DFA 把每个 (状态, 字符) 的去向都提前算好，KMP 只存「失配退到哪」，到时候沿 next 链现退。**区别**：DFA 表占 $O(m|\Sigma|)$ 空间、预处理 $O(m|\Sigma|)$，每个文本字符恰好一次查表，适合字符集小、同一模式反复匹配；KMP 只要 $O(m)$ 空间、与字符集无关，单个字符可能沿 next 链回退多次，但总回退次数被前进次数限住，整体仍是 $O(n)$。优化版 next 跳过了「退回去注定还失配」的那一步，相当于把 DFA 在该字符上的转移提前走了半步——但它**不再是** border，不能拿

来求循环节（见第 4.3 节「求最小循环节」）。

2. **[ref_DSA/homework/4 字符串.md 第 2 题；2025 秋书面作业]** 最长无重复字符子串，$O(n)$ 滑动窗口：维护窗口 $[left, r]$ 内无重复，`last[ch]` 记字符上次出现的下标。每读入 s_r，若 `last[s_r] >= left`，就把 `left` 跳到 `last[s_r]+1`（**直接跳，不用逐个右移**）；更新 `last[s_r]=r`；若 $r - left + 1$ 超过当前最优，记下长度与起点。`"abcabcbb"` 输出 `"abc"` **和下标** 0，`"pwwkew"` 输出 `"wke"` 和 2。`left` 只增不减、r 走一遍，时间 $O(n)$；`last` 按字节字母表是 $O(1)$ 空间。条件里的 `>= left` 不能省——`"abba"` 读到末尾的 `a` 时，`last[a]=0` 已在窗口外，省了会把 `left` 拉回 1。（脚本与暴力枚举对拍 3000 组。）
3. **[ref_DSA/homework/4 字符串.md 第 3 题；ref_DSA/DSA-HW/Chapter4string.md 第 2 题；2025 秋书面作业]** 级联删除所有 `b` 与 `ac`，**单遍、原地、$O(1)$ 额外空间**（示例 `"acbac"` →`""`、`"ababac"` →`"aa"`）。关键观察：**已写入 S 左端的前缀本身就是一个栈**，写指针 w 就是栈顶——不需要另开栈。读指针 r 从左到右：

```
w ← 0
for r ← 0 .. n-1:
    if S[r] == 'b':                       continue      // b 从不写入
    if S[r] == 'c' and w > 0 and S[w-1] == 'a':  w ← w-1; continue   // 与前一个
↪   a 抵消
    S[w] ← S[r]; w ← w+1
return w
```

为什么级联被自动处理了：`b` 从来没写进前缀，所以它两边的 `a`…`c` 在前缀里天然相邻；删掉一对 `ac` 后，w 回退，前缀的新末尾若是 `a`，下一个 `c` 照样能与它抵消（如 `"aacc"` →`""`）。最终保留的前缀里不含 `b`，也不含相邻的 `ac`（写入 `c` 时已检查过前一个字符），而每一步删除都是题目允许的删除，所以结果就是级联删除的不动点。r 走一遍、每步 $O(1)$，时间 $O(n)$、额外空间 $O(1)$。本附录原先给的「读写栈、空间 $O(n)$」**输出是对的**（与暴力反复删除在 $\{a, b, c\}$ 上长度 $\leqslant 9$ 的全部串对拍一致），**但不满足题目的 $O(1)$ 空间要求**——而这正是这道题要考的：认出写指针左侧的已写区可以复用为栈。（上面的原地算法也做了同样的全量对拍。）
4. **[ref_DSA/homework/4 字符串.md 第 4 题；2025 秋书面作业]** `first_pos[k]`（$1 \leq k \leq m$）= 前缀 $P[0..k)$ 在 T 中首次出现的起点，$O(n+m)$。做法：先算 P 的 border 数组 `f`，再用 KMP 扫 T，状态 j 是「T 的当前后缀与 P 前缀的最长匹配长度」；每步匹配后若 `first_pos[j] == -1`，就令 `first_pos[j] = i-j+1`。**为什么只记当前的 j 就够、不用把 border 链上更短的前缀也更新**：设 $P[0..k)$ 第一次出现时结束于位置 e。若那一刻状态 $j > k$，则 $P[0..k)$ 作为 $P[0..j)$ 的前缀，在更早的 $e - j + k$ 处已经结束过一次，与「第一次」矛盾；所以首次出现时状态恰好等于 k，一定会被记到。（脚本与暴力 `find` 对拍 2 万组，无反例。）**易错点是匹配满 m 之后的回退**：$j = m$ 时下一步必须先令 $j \leftarrow f[m-1]$ 再比较，否则会去读 $P[m]$ 越界。时间 $O(n+m)$，额

外空间是 f 的 $O(m)$。

5. **[本书自拟]** 循环移动字符串 S[0..i] 与 S[i+1..n-1] 可用三次逆置：逆置前段、逆置后段、逆置整体；时间 O(n)、额外空间 O(1)。（此题原先标注为 ref_DSA/homework/4 字符串.md 第 3 题，但该文件第 3 题是上面的删 b/ac；在 ref_DSA/ 与课程目录中均未找到这道题，改标本书自拟。）
6. **[教材勘误错误 10、13；本书代码验证]** 匹配成功返回首字符 0 起始下标 j-pLen；找不到返回空值。不能使用 j-pLen+1。

课程上机参考

为同一组文本和模式同时调用朴素匹配与 KMP，并逐项对拍；增加空模式、模式长于文本、重复字符和匹配发生在末尾的测试。

第 5 章 二叉树

补充题参考答案

1. **[课程第五章答案第 1 题]** 不成立。先序 ABC、后序 CBA 可由四种不同的单支二叉树产生；若改为先序 + 中序，则按根切分可唯一重建。
2. **[课程第五章答案第 3 题]** 对每个下标 i=1..n-1 检查 a[parent(i)] >= a[i]，其中 parent(i)=(i-1)/2；一次扫描 O(n)。
3. **[ref_DSA/homework/5 二叉树.md 第 3 题]** 满二叉树有 L 个叶和 L-1 个内部结点，总数 n=2L-1，所以 L=(n+1)/2。

正文习题答案

1. 不计结点标号时共有 Catalan 数 $C_3 = 5$ 种：左斜链、右斜链、根有左右两叶、根只有左孩子且该孩子只有右孩子、根只有右孩子且该孩子只有左孩子。
2. 先序首字母 A 为根；中序切成 CBD | A | FEG。递归切分后，B 的左、右孩子分别是 C、D；E 的左、右孩子分别是 F、G。后序为 CDBFGEA（左子树 CDB，右子树 FGE，最后是根 A）。
3. 每个结点有两个孩子指针，共 $2n$ 个；树的 $n-1$ 条父子边恰占 $n-1$ 个非空指针，因此空指针数 $2n-(n-1)=n+1$。
4. 1 起始编号时，$i>1$ 的父为 $\lfloor i/2 \rfloor$，左孩子 $2i$、右孩子 $2i+1$（编号不超过 n 时存在）。层序编号把下一层按每个父的左右孩子成对展开，公式随之成立。
5. 根入队；队列非空时出队访问，并依次把非空左、右孩子入队。队列保存“已发现但尚未访问”的结点，先进先出才能保证层序。本书接口为 BinaryTree::level_order。
6. 插入后为根 50，左子树 30(20,40)，右子树 70(60,80)。删除 50 若取中序前驱 40，则新根 40，左为 30(20,null)，右仍为 70(60,80)；取后继 60 也正确。本书 BinarySearchTree::remove 采用前驱方案。
7. 合并最小权：2+3=5、4+5=9、7+8=15、9+15=24，所以 WPL 为 $5+9+15+24=53$。一组编码：4=00，2=010，3=011，7=10，8=11；加权长度确为 53。

正文上机题参考

1. 先序首元素定根，在中序中切分左右区间递归构造；重复键会使切分不唯一，应拒绝或另带唯一 ID。构造后用 `BinaryTree::postorder` 输出。本书工程版给出了现成实现 `BinaryTree::from_preorder_inorder`（另有 `BinaryTree::from_inorder_postorder`）：只传下标区间、不拷贝子数组，用显式栈代替递归，遇到重复键或不自洽的两个序列抛 `std::invalid_argument`；做题时可以拿它对拍自己的写法。
2. 使用实际接口 `BinarySearchTree::insert`、`BinarySearchTree::remove`、`BinarySearchTree::inorder`；依次插入递增键会形成高为 n 的右链，使查找和插删退化到 $O(n)$。
3. `MinHeap::insert` 建堆后反复 `MinHeap::remove_min()`（返回 `std::optional<T>`，堆空时是 `std::nullopt`）可得到升序；逐个插入建堆 $O(n \log n)$，全部弹出仍为 $O(n \log n)$。计时要排除数据生成和输出并多次重复。
4. 教学版 `code/ch05/heap_huffman/teaching.hpp` 里 `HuffmanTree(symbols,weights,count)` 才带字符表，查询接口是 `HuffmanTree::code_of`、`HuffmanTree::encode`、`HuffmanTree::decode`、`HuffmanTree::weighted_path_length`；工程版 `modern.hpp` 的 `HuffmanTree(weights,count)` 只建树、只对外给 `total_weight()`，做这道题要用教学版。同权值编码可能不唯一，守门条件应是前缀无歧义、往返一致和 WPL 最小。

课程作业与 `ref_DSA` 参考答案

1. **[课程第五章答案第 1 题；本书补充]** 课程原题 “先序 + 后序能否唯一确定二叉树” 的答案是否定的，例如先序 `ABC`、后序 `CBA` 可以对应不同的单支树。若已知先序和中序，则先序首元素确定根，再按根在中序中的位置切分左右子树，可以唯一递归重建；后序加中序同理取后序末元素为根。
2. **[`ref_DSA/homework/5 二叉树.md` 第 3 题]** 含 199 个结点的 Huffman 树若每个内部结点都有两个孩子，则叶子数为 `(199+1)/2=100`，因为满二叉结构满足 `n=2L-1`。
3. **[`ref_DSA/homework/5 二叉树.md` 第 4(2) 题；课程第五章答案第 3 题为大顶堆判定]** 最小堆的最大值只能出现在叶子层；内部结点都不大于其子结点。判定大顶堆时逐个检查每个非根结点不大于其父结点，时间 `O(n)`。
4. **[`ref_DSA/homework/5 二叉树.md` 第 3(2)(3) 题；本书代码验证]** Huffman 算法反复取两个最小权值合并，再把和放回最小堆；总带权路径长度最小。权值相同的编码可能不唯一，但总代价相同。对题给 8 个频率 $0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10$（按百分比取整数权），各字母的**码长**依次为 **4、2、5、4、2、5、2、4**：0.32、0.21、0.19 各 2 位，0.10、0.07、0.06 各 4 位，0.03、0.02 各 5 位。平均码长 $\sum p_i l_i = 2.61$ 位，定长编码（0–7 的二进制）是 3 位，节省 13%。具体码字取决于合并时同权子树放左放右，**不唯一，但码长多重集和 2.61 是唯一的**，批改应核这两样。对比：Huffman 码是变长前缀码，压缩率高、但必须随数据保存码表，解码要逐位

走树，一位出错会连带后续码字错位；定长码不压缩，但可随机定位第 k 个字符、实现简单、错误不扩散。

5. **[ref_DSA/homework/5 二叉树.md 第 4(1) 题；2025 秋书面作业；脚本复算]** 序列 (503,87,512,61,908,170,897,275,653,462) 用筛选法建最小堆，从最后一个非叶下标 4 开始向前逐个下沉。**第一步**（下标 4 的 908 与唯一的孩子 462 比较并下沉）后为 503,87,512,61,462,170,897,275,653,908；**最终堆**为 61,87,170,275,462,512,897,503,653,908。易错处在最后一步：503 从根下沉，依次与较小孩子 61、87、275 换位（经过 61 → 87 → 275 这条路），最终落在下标 7；512 早在处理下标 2 时就与 170 换到了下标 5。把 503 与 512 的位置写反是这题最常见的错。
6. **[ref_DSA/homework/5 二叉树.md 第 4(3) 题；2025 秋书面作业]** 建堆比较次数的常数 $C_0 = 2$。下沉每下降一层至多比较 2 次（两个孩子先比一次选较小者，再与之比一次决定是否下沉）。高度为 $h = \lfloor \log_2 n \rfloor$ 时，距底 k 层的结点至多 $\lceil n/2^{k+1} \rceil$ 个、每个至多下降 k 层，故

$$C \le 2 \sum_{k \ge 1} k \cdot \frac{n}{2^{k+1}} = n \sum_{k \ge 1} \frac{k}{2^k} = 2n.$$

这个界是紧的：逆序输入每个结点都沉到底，$n = 4095$ 时实测 8166 次比较，$8166/4095 \approx 1.994$。5.5.1a 与期中题库证的是「下沉层数之和 $\le 2n$」——那是**移动**的界；换算成**比较**要再乘每层的 2 次，但层数之和的精确界是 $n - \lfloor \log_2 n \rfloor - 1 < n$，乘 2 仍得 $2n$。

课程上机参考

测试二叉树先/中/后序、BST 插入删除、全相等键、退化链和堆扩容；Huffman 测试空输入、单叶、负权拒绝和权重和溢出。

原书习题答案

1. 不计标号的三结点二叉树数为 Catalan 数 C_3=5；若 A、B、C 是固定不同标号，先选根再分配左右子树，合计 3!*C_3=30。
2. 图 5.21 的树是：A 的左孩子 B、右孩子 C；B 只有左孩子 D；C 的左孩子 E、右孩子 F；E 只有右孩子 G；F 的左、右孩子是 H、I。按「根、左、右」得前序 ABDCEGFHI；按「左、根、右」得中序 DBAEGCHFI（注意 G 是 E 的**右**孩子，所以 E 在 G 前）；按「左、右、根」得后序 DBGEHIFCA。
3. 真。任意两个叶的最近共同祖先处，一叶在左子树、另一叶在右子树时三种周游都先完整访问左子树再访问右子树；归纳到子树即可，故叶的相对次序相同。
4. 前序 = 中序当且仅当每个结点都没有左孩子；前序 = 后序当且仅当树至多一个结点；中序 = 后序当且仅当每个结点都没有右孩子。
5. leaves(t)=0（空），1（左右均空），否则 leaves(t->left)+leaves(t->right)；

访问每结点一次，O(n)。

6. height(t)=0（空），否则 1+max(height(t->left),height(t->right))；时间 O(n)，递归栈 O(h)。
7. 后序递归交换：先镜像左右子树，再 swap(t->left,t->right)；每边仅处理一次，O(n)。
8. 递归 LCA：空返回空；命中 p/q 返回自身；左右各递归，均非空则当前根为 LCA，否则返回非空者。若要求两点都存在，额外返回命中计数，只有计数为 2 才接受。
9. 对图 5.22(a)（与图 5.21 是同一棵树）中每个结点画三格 left|data|right，有孩子就画指针、缺孩子填空指针 ∧：A 的 left → B、right → C；B 的 left → D、right 为空；C 的 left → E、right → F；E 的 left 为空、right → G；F 的 left → H、right → I；D、G、H、I 两个指针都为空。9 个结点共 18 个指针域，非空 8 个、空 10 个，正好是 $n+1=10$（正文第 3 题）。
10. 三个不同关键码的 BST 形状也是 C_3=5；中序固定为 A,B,C，五种形状与第 1 题相同。
11. 对任一结点，BST 左子树全部小于它、右子树全部大于它；中序先输出左子树、再根、再右子树，结构归纳即严格递增。
12. 比较键递归下降；命中叶直接删，单孩子用孩子替代；双孩子用左子树最大（或右子树最小）替代键，再递归删该替代结点。时间 O(h)。
13. 题给的 14 个关键码**本来就按字典序递增**（wxw < wxz < wzw < ... < zyz），所以每个新键都比树中所有键大，一路向右走到底：结果是一条以 wxw 为根、每个结点只有右孩子的**右斜链**，高度 14，中序序列就是原序列。这正是本题想让人看到的——按有序序列插入，BST 退化成链表，检索代价变成 $O(n)$（脚本建树核对）。
14. 递归 return t && (t->key==K || search(t->left,K) || search(t->right,K))；非递归用显式栈 DFS（或队列 BFS）。最坏均 O(n)。
15. 若 t->key<=K，答案为 size(t->left)+1+smallcount(t->right,K)；否则只递归左子树。没有 size 字段时仍须遍历被计入的左子树，最坏 O(n)；维护子树大小可降为 O(h)。
16. 若 t->key>low 才访问左子树；键落在区间即打印；若 t->key<high 才访问右子树。中序输出有序，复杂度 O(h+k)。
17. 递归携带开区间 (low,high)，要求 low<t->key<high，左传 (low,key)、右传 (key,high)；或中序检查严格递增。时间 O(n)。
18. 对每个 i=1..n-1 检查 a[(i-1)/2]<=a[i]；任一违反即否，否则是最小堆，O(n)。
19. 对数组从最后非叶 floor(n/2)-1 向 0 做下滤；每次把较小孩子上提、最后写回暂存键。{E,D,X,K,H,L,M,C,P} 建成的最小堆为 C,D,L,E,H,X,M,K,P。按原书【代码 5.11】SiftDown 的写法（temp = heapArray[i] 暂存 → 较小孩子逐层上移 → heapArray[i] = temp 回写）逐次计数：下标 3、2、1、0 四次筛选都发生了下沉，暂存 4 次 + 上移 5 次 + 回写 4 次，**共 13 次移位**。若改用题目提示里「交换算 3 次」的交换式下沉（本书教学版 sift_down 就是这种写法），同

样 5 次下降要 15 **次移位**——暂存式正是为省掉这 2 次赋值/层。答题须声明用的是哪种下沉。（脚本逐步计数复算。）

20. 用最小堆反复合并两最小权并把和放回；所有合并和之和即最小 WPL。对给定十权 $\{1, 4, 9, \ldots, 100\}$,各步合并为 $1+4=5$、$5+9=14$、$14+16=30$、$25+30=55$、$36+49=85$、$55+64=119$、$81+85=166$、$100+119=219$、$166+219=385$，WPL $=5+14+30+55+85+119+166+219+385=1078$。各叶的路径长度（码长）为：1 和 4 是 7，9 是 6，16 是 5，25 是 4，36、49、64 是 3，81、100 是 2，$\sum w_i l_i$ 同样得 1078（脚本复算；本附录旧版列的中间和 91、121、155、232 是错的，加起来也不等于 1078）。合并策略是证明最优的 Huffman 贪心。
21. k 叉 Huffman 先补零权叶使 `(n-1) mod (k-1)=0`，每轮合并 k 个最小权；三叉时 $n=10$，$(10-1) \bmod 2 = 1$，补一**个**零权叶。依次合并 `{0,1,4}=5`、`{5,9,16}=30`、`{25,30,36}=91`、`{49,64,81}=194`、`{91,100,194}=385`，WPL = 5+30+91+194+385 = 705。不补零权叶时最后一轮只剩两棵树，被迫做一次二路合并，WPL 会变大。（脚本复算。）
22. 中序线索化维护 `prev`：访问结点时若左空，令 `left=prev, ltag=1`；若 `prev` 的右空，令其 `right=current, rtag=1`；再令 `prev=current`。最后一个结点右线索指空，时间 `O(n)`。

原书上机题参考

1. 一次后序递归同时返回高度和 “到 p/q 的距离”：在 LCA 处把两条向下距离相加；也可分别求根到两点路径，最长公共前缀末端为 LCA。时间 `O(n)`。
2. 三种输入各有各的建树法, 但都只扫一遍: **后缀**式用一个栈——读到操作数就压入一棵单结点树,读到运算符就弹出两棵子树(**先弹出的是右子树**)拼成新树再压回, 扫完栈里恰剩一棵; **前缀**式把同样的办法反过来, 从右往左扫, 先弹出的那棵当左子树; **中缀**式先按第 3.1.4 节的算符优先法转成后缀式（或者边转边建), 括号只影响建树时的结合, 树建好之后括号就不必存在了。输出三种表达式就是三种周游：`BinaryTree::preorder` 得前缀式、`BinaryTree::postorder` 得后缀式；中序周游要补括号, 判据两条：**孩子的优先级低于自己**（`(a+b)*c`), 或者**同级且它是右孩子**（`a-(b-c)`、`a/(b*c)`）——四个运算符都是左结合，所以同级的右孩子必须括起来，同级的左孩子一定不用。这里的「冗余」按题面的定义取「去掉它会改变计算顺序」，因此 `a+(b+c)` 的括号**不算冗余**：脱了括号重新解析得到的是 `(a+b)+c`，另一棵树（算出来的值相同，但题目要的是树）。字符图显示可以按「右子树 → 本结点 → 左子树」的中序周游打印，每深一层多缩进两格，树就横躺在屏幕上。守门用例：三种输入建出的树对同一个表达式必须一致；`a*(b+c)` 与 `a*b+c` 的树不同；往返一次（建树 → 输出中缀 → 再建树）应得到同一棵树。（概括：前/后缀式用栈建树，中缀用算符优先法（shunting-yard）处理优先级与括号；三种输出分别为先/中/后序周游。）
3. 词法分析后以递归下降或后缀栈求值；枚举 r 个变元的 `2^r` 赋值，全部真为

重言、全假为矛盾，否则可满足。输出真值表时间 `Theta(2^r*|E|)`。

4. 首遍统计字节频率，建立 Huffman 树并在文件头写入频率表/树，随后按码字写比特；解码按树走到叶输出并回根。应处理空文件、单字符和损坏头。
5. 最大堆元素保存 `(id,priority)`，另设 `pos[id]`。入队上滤并同步位置；出队用末项换根后下滤；改权后按增减上滤或下滤。三操作均 `O(log n)`，定位 `O(1)`。

第 6 章 树

补充题参考答案

1. **[课程第六章答案第 1 题]** 第 `l` 层有 `k^l` 个结点；按根编号从 1 开始时，结点 `i` 的第 `m` 个孩子为 `(i-1)k+1+m`（按题目约定调整 `m` 的起点）；非最后一个孩子的右兄弟为 `i+1`。
2. **[课程第六章答案第 3 题]** 相等方程合并集合，不等式检查两端代表元是否相同；无优化单次 `find` 最坏 `O(n)`，按秩合并为 `O(log n)`，路径压缩的 m 次操作为 `O(m alpha(n))`。
3. **[课程第六章答案第 2 题]** 左孩子/右兄弟结构中先根序列沿第一个孩子递归并沿兄弟链前进；带度数后根序列在访问父结点时记录该结点孩子数。

正文习题答案

1. 森林转二叉树时，每个结点的第一个孩子接左指针、下一个兄弟接右指针，各棵树的根也沿右指针相连；逆变换把左链恢复为孩子、右链恢复为兄弟。逐边做两次后，每条父子/兄弟关系回到原处，故互逆。
2. 对题面给的 `A(B(E,F),C(G),D(H,I,J))`：先根 `ABEFCGDHIJ`，后根 `EFBGCHIJDA`，层次 `ABCDEFGHIJ`。先根是「根，再从左到右各子树」，后根是「从左到右各子树，再根」，层次用队列逐层扫描。核对时用 `GeneralTree::preorder`、`GeneralTree::postorder`、`GeneralTree::breadth_first`——注意层次序列与另两个的差别只在同层结点何时被访问。
3. 度为 2 的有序树只区分“第一个/第二个孩子”；只有一个孩子时它就是第一个孩子。二叉树还区分左、右，因此根只有右孩子 B、左孩子为空是合法二叉树，却没有对应的度为 2 有序树形态。
4. 例如根 A 有孩子 B、C，C 有孩子 D；带度数后根序列为 `B/0, D/0, C/1, A/2`。叶结点直接压栈；读到 `C/1` 弹 1 棵作其孩子再压回 C，读到 `A/2` 按逆序弹 C、B，接回原来的 B、C 次序。
5. `unite(0,1)`：两根规模都是 1，值大的 1 挂到 0 下；`unite(2,3)`：同理 3 挂到 2 下；`unite(0,2)`：两棵树都有 2 个元素，值大的根 2 挂到 0 下。此时 `parent=[0,0,0,2,4,5,6]`，3 的路径是 $3 \to 2 \to 0$，深度 2——重量权衡把树高压在 $\lfloor \log_2 4 \rfloor = 2$ 以内，但并没有让它变成 1。`find(3)` 回溯时把路径上的 3 直接改挂到根 0，得 `parent=[0,0,0,0,4,5,6]`；2 本来就是根的孩子，不变。4、5、6 始终各自成集合。（注意：若 `find` 的是根 0 或深度 1 的结点，父指针一个也不会变，看不出压缩——要观察压缩，得查深度至少为 2 的结点。）

6. 1 起始的完全 3 叉树中，父公式为 $\lfloor (i-2)/3 \rfloor + 1$，孩子为 $3(i-1)+2, 3(i-1)+3, 3(i-1)+4$。所以结点 4 的父是 1，孩子是 11、12、13（编号未超过结点总数时存在）。

正文上机题参考

1. 转换时保存第一个孩子/下一兄弟关系，往返后除比较先根序列，还应比较每个结点的孩子数。本书提供 `GeneralTree::preorder`、`GeneralTree::postorder` 和 `GeneralTree::breadth_first`。
2. 为每条无向边调用 `DisjointSet::unite(u,v)`；最终把每个顶点的 `find(i)` 放入集合，集合大小就是连通分量数，等于 1 时连通。实际接口是 `DisjointSet::find`、`DisjointSet::unite`、`DisjointSet::same`——**没有** `connected` 这个名字。不想最后再扫一遍的话，用 `ComponentCounter::connect` 逐条加边，`ComponentCounter::components` 随时给出 $O(1)$ 的分量数（每次成功合并减 1，冗余边不变）。
3. 构造父指针链后重复查最深结点；无压缩每次 $O(n)$，路径压缩第一次仍为 $O(n)$，之后接近 $O(1)$。本书实现同时使用按重量合并和路径压缩，不能靠 `unite` 造退化链；对照版需单独实现朴素 `find`。

课程作业与 `ref_DSA` 参考答案

1. **[张铭教材；本书自拟]** 左孩子/右兄弟表示中，`child` 指向第一个孩子，`sibling` 指向下一个兄弟；先根遍历沿 child 递归、沿 sibling 迭代。
2. **[课程第六章答案第 2 题]** 森林转二叉树：每棵树根的第一个孩子成为左孩子，其余兄弟链成为右孩子；逆变换反向执行。课程原题进一步要求带右链先根和带度数后根的顺序存储，本书保留转换原理，具体表格取决于题图。
3. **[课程第六章答案第 3 题；`ref_DSA/homework/6 树.md`]** 并查集不优化时单次 find 最坏 `O(n)`；按重量/秩合并可把树高限制为 `O(log n)`；再配合路径压缩，m 次操作的摊还复杂度为 `O(m alpha(n))`。
4. **[`ref_DSA/DSA-第六章作业.pdf` 第 1 题；2025 秋书面作业]** 关系集合给出的树：嵌套括号表示为 `A(B(E(H,I),F(J)),C,D(G))`。按本书约定根在第 0 层（第 5 章「深度为 k 的二叉树至多 $2^{k+1}-1$ 个结点」），**深度为** 3（H、I、J 在第 3 层），**高度为** 4（共 4 层）；两个词在别的教材里口径不同，答题先说清层号从 0 还是 1 起。求高度：`height(t) = t 为空 ? 0 : 1 + max(height(c) for c in t 的孩子)`，时间 $O(n)$。转二叉树（左孩子右兄弟）：每个结点的**第一个孩子**作左孩子、**下一个兄弟**作右孩子——A 的左为 B；B 的左为 E、右为 C；C 的右为 D；D 的左为 G；E 的左为 H、右为 F；H 的右为 I；F 的左为 J。核心代码：`left(v)=first_child(v)`、`right(v)=next_sibling(v)`，递归或按层处理均为 $O(n)$；本书 `GeneralTree` 本身就是这种存储。
5. **[`ref_DSA/DSA-第六章作业.pdf` 第 2 题；2025 秋书面作业]** 高度为 L（L 层）的完全 K 叉树，结点数区间为 $\left[\frac{K^{L-1}-1}{K-1}+1,\ \frac{K^{L}-1}{K-1}\right]$：下界是前 $L-1$ 层全满、第 L 层只有 1 个，上界是满 K 叉树。按层从 1 编号时，结点 N（$N>1$）

的父为 $\lfloor (N-2)/K \rfloor + 1$，第 i 个孩子（$1 \le i \le K$）为 $K(N-1)+i+1$（不超过结点总数才存在）；验证：$K=3$ 时结点 1 的孩子是 2、3、4，结点 2 的孩子是 5、6、7，结点 5 的父是 $\lfloor 3/3 \rfloor + 1 = 2$。$\lfloor (N-1)/K \rfloor$ **是 0 起始编号的父公式**，套到 1 起始编号上，$N = 2..K+1$ 时碰巧给出正确的 1，从 $N = K+2$ 起就错了：1 起始下 $N = K+2$ 是结点 2 的第一个孩子，这个公式却给出 $\lfloor (K+1)/K \rfloor = 1$（$K=3$ 时结点 5 被算成结点 1 的孩子）。与原书第 10 题答案一致。

6. **[ref_DSA/DSA-第六章作业.pdf 第 3 题；2025 秋书面作业]** 按高度合并时，任意树高 h（单结点树 $h=0$）满足结点数 $s \ge 2^h$，于是 $h \le \log_2 N$。归纳：单结点 $s = 1 = 2^0$。合并高 $h_1 \ge h_2$ 的两树时，高者作根：若 $h_1 > h_2$，新高仍为 h_1，结点数只增不减，$s \ge 2^{h_1}$；若 $h_1 = h_2 = h$，新高为 $h+1$，而 $s = s_1 + s_2 \ge 2^h + 2^h = 2^{h+1}$。**只有「等高合并」才让树长高，而等高合并一定让结点数翻倍**——这就是证明的全部。注意不能照搬按重量合并的论证（「小树挂大树，小树结点所在树规模翻倍」）：按高度合并时矮树未必是小树，那个论证在这里不成立。（脚本随机合并 $N = 64$ 验证每棵树都满足 $2^h \le s$。）

课程上机参考

实现一般树三种周游和并查集合并/查找，测试空森林、单根、多兄弟、重复合并和路径压缩前后父指针变化。

原书习题答案

1. 先核对题面：K 有 13 个结点，树应有 12 条边，R 也恰好 12 对，但 $\langle D, J \rangle$ 与 $\langle I, J \rangle$ 让 J 有两个父结点，而 M 在 R 里一次都没出现——照字面这不是一棵树（扫描件确实印作 $\langle I, J \rangle$，属原书题面错误）。最可能的本意是 $\langle I, M \rangle$，按此作答：树为 `A(B(E,F(K,L),G),C(H),D(I(M),J))`。根是 A；按递归定义，A 的三棵子树以 B、C、D 为根，B 的子树以 E、F、G 为根，F 的以 K、L 为根，C 的以 H 为根，D 的以 I、J 为根，I 的以 M 为根。叶为 E、G、H、J、K、L、M。层数（根为第 0 层）：A 为 0；B、C、D 为 1；E、F、G、H、I、J 为 2；K、L、M 为 3。度数：A、B 为 3；D、F 为 2；C、I 为 1；其余为 0。对应二叉树（首孩子作左、下一兄弟作右）：A 的左为 B；B 的左为 E、右为 C；E 的右为 F；F 的左为 K、右为 G；K 的右为 L；C 的左为 H、右为 D；D 的左为 I；I 的左为 M、右为 J。
2. **2 种**：根有两个孩子；或根—孩子—孙子一条链。按根的孩子数枚举：根除自己外还剩 2 个结点，要么成为 2 个孩子（各为叶），要么成为 1 个孩子及其 1 个孩子；有序树里「只有一个孩子」不分左右，所以不会像二叉树那样再分出 5 种。一般地，n 个结点的有序树有 Catalan 数 C_{n-1} 棵（与 $n-1$ 个结点的二叉树一一对应，正是森林与二叉树的转换），$n=3$ 时 $C_2 = 2$。若 3 个结点带不同标号，再乘 $3!$ 得 12。（本附录旧版写的「小 Schröder 数 11」是错的。）

3. 图 6.19 的二叉树：A 的左、右为 B、H；B 的左、右为 E、C；C 的左、右为 F、D；F 只有右孩子 G；H 的左、右为 I、K；I 只有右孩子 J。按「左指针是首孩子、右指针是下一个兄弟」还原，A 的右链 H、K 是另外两棵树的根，森林为 `A(B(E),C(F,G),D)`、`H(I,J)`、`K`。先根次序 `ABECFGDHIJK`（等于二叉树的前序），后根次序 `EBFGCDAIJHK`（等于二叉树的中序）。（脚本还原并周游核对。）
4. 有序树相同当且仅当根标签相同、孩子数相同且第 i 个孩子递归相同；一次同步 DFS，`O(n)`。
5. 无序树可先递归计算规范编码：结点编码为标签加上排序后的孩子编码列表；两个根编码相等即同构。排序总计 `O(n log n)`，可用哈希优化。
6. `count(t)=0`（空），否则 `1+sum(count(child))`；访问一次，`O(n)`。
7. 子结点表每结点含动态孩子指针，指针总数约边数加数组容量余量；左子右兄固定两个指针/结点，空指针较多但无需每结点数组。度数小且变化频繁时前者灵活，空间紧张/遍历兄弟时后者稳定。
8. 规则：`union(a,b)` 先对两端做带路径压缩的 `find`，规模小的根挂到规模大的根下，规模相等时值大的根挂到值小的根下（本书 `DisjointSet::unite` 就是这个口径）。逐步执行：(0,2)、(1,2) 后 0 是 {0,1,2} 的根；(3,4) 后 3 是 {3,4} 的根；(3,1) 时 3 的树（2 个）挂到 0（3 个）下；(3,5) 把 5 挂到 0；(9,11) 得根 9，(12,14) 得根 12；(3,9) 把 9 挂到 0；(4,14) 时 `find(4)` 把 4 压缩直连 0，12 挂到 0；(6,7) 得根 6；(8,10) 得根 8；(8,7) 时 6、8 两树都是 2 个，值大的 8 挂到 6 下；(7,0) 时 6 的树（4 个）挂到 0（10 个）下；(10,15) 时 `find(10)` 沿 $10 \to 8 \to 6 \to 0$ 压缩，10、8 都直连 0，15 挂到 0；(10,13) 把 13 挂到 0。最终父指针数组（下标 0～15）为 `0 0 0 0 0 0 0 6 0 0 0 9 0 0 12 0`：全部 16 个元素同属一个等价类、根为 0，只有 7、11、14 还隔一层（分别挂在 6、9、12 下）。（脚本按本书 `unite` 的规则逐步复算。）
9. 递归复制树：为每个树结点建二叉结点，首孩子递归放 left，后续孩子通过 right 串起；时间 `O(n)`。
10. 1 起始层序：父为 `floor((N-2)/K)+1`（`N>1`）；第 i 个孩子为 `K(N-1)+i+1`（i 从 1 起）；不是其父最后孩子且编号未越界时有右兄，编号 `N+1`。
11. 图 6.20 的二叉树（A 的左右为 B、C；B 的左右为 D、E；E 只有左孩子 H；C 的左右为 F、G；F 的左右为 I、J）还原成森林 `A(B(D),E(H))`、`C(F(I),J)`、`G`。四种表示（下标从 0 起，`-` 表示空）：
    - 左子/右兄：每个结点的左指针、右指针与图中二叉树完全相同（A:B,C；B:D,E；D:-,-；E:H,-；H:-,-；C:F,G；F:I,J；I:-,-；J:-,-；G:-,-）。
    - 带右链的先根次序：先根序列 `A B D E H C F I J G`；`ltag`（1 = 无孩子）为 `0 0 1 0 1 0 0 1 1 1`；`rlink`（下一兄弟的下标）为 `5 3 - - - 9 8 - - -`。
    - 带左链的层次次序：层次序列 `A C G B E F J D H I`；`llink`（第一个孩子的下标）为 `3 5 - 7 8 9 - - - -`；`rtag`（1 = 无下一兄弟）为 `0 0 1 0 1 0 1 1 1 1`。
    - 带度数的后根次序：`D/0 B/1 H/0 E/1 A/2 I/0 F/1 J/0 C/2 G/0`。

    （脚本由二叉树还原森林后逐项生成。）

12. 扫描后根 (label,degree)，每项弹出 degree 棵已完成树（逆序恢复孩子），以首孩子/右兄串起，再把新根压栈；O(n)。
13. 扫描先根带右链表示，递归读取 “当前结点及其子树范围”，子树结束时输出 (label,degree)；或先还原左子右兄再后根，O(n)。

原书上机题参考

1. 每结点存动态 vector<Node*> children；容量不足时分配两倍数组、拷贝指针、释放旧数组，追加摊还 O(1)，并禁止形成环。
2. 父指针表存 parent 与数据；新增在空槽写父下标，删除需处理整棵子树或重编号并修复所有父号；查询父为 O(1)、找孩子 O(n)。
3. 递归下降解析 Letter ['(' tree { ',' tree } ')']，构造时首孩子接 left、后续接 right；输出时打印根、递归孩子链并插入逗号和括号，二者互为逆变换。
4. 四叉树叶表示常值块，非叶递归四分；矩阵乘法按 C 的象限做八个子块积并相加，直到叶。2^p 阵构造时间/空间按结点数，任意 MxN 以边界填充或不等分区域递归。

第 7 章 图

补充题参考答案

1. **[课程第七章答案第 1 题]** 先拓扑排序;按拓扑序计算 earliest[v]=max(earliest[u]+w(u,v))，终点最大值为工期。邻接表下为 O(V+E)。
2. **[课程第七章答案第 2 题]** 统一加常数会按路径边数增加不同总量，因此可能改变最短路径次序；存在负权边时应使用 Bellman-Ford 等算法。
3. **[ref_DSA/DSA-HW/Chapter7Graph.md 第 2 题；本书校正]** Dijkstra 的树按源点距离选边，MST 按割上的最小边选边；AB=2,BC=2,AC=3 时两者总权分别为 5 和 4。

正文习题答案

1. 4 个顶点的无向完全图有 $4\cdot 3/2 = 6$ 条边;有向完全图(无自环)有 $4\cdot 3 = 12$ 条弧。一般公式分别是 $V(V-1)/2$ 与 $V(V-1)$。
2. 按本书 Graph 从小下标扫描邻接点，demo 图从 0 出发的 DFS 为 0,1,2,3,4，BFS 也为 0,1,2,3,4；不同但固定的邻接点顺序可能给出另一种合法序列。
3. 例如 0->1->2->0。Kahn 算法开始时没有入度 0 顶点，队列为空，输出数小于 $V = 3$，因此报告有环；一般环内顶点的入度永远不会降到 0。
4. 对 demo 图，初始距离 [0,inf,inf,inf,inf]；选 0 后 [0,2,7,inf,inf]；选 1 后 [0,2,3,7,inf]；选 2 后 [0,2,3,4,inf]；选 3 后 [0,2,3,4,7]；最后选 4，结果不变。
5. Dijkstra 把当前最小暂定距离顶点定型后不再修改；负边可能从后来访问的顶点把已定型距离降得更小，破坏贪心前提。应使用 Bellman–Ford 等允许重复松弛的算法。

6. 以无向三角形 AB=2、BC=2、AC=3 为例：Prim 和 Kruskal 都取 AB、BC，总权 4。存在同权边时二者可能选不同边集，但割性质保证总权都等于 MST 最优值。
7. 邻接矩阵占 $\Theta(V^2)$, 判边/删边 $O(1)$, 适合稠密图; 邻接表占 $\Theta(V+E)$, 遍历邻居 $O(\deg v)$, 适合稀疏图。矩阵 DFS/BFS 为 $O(V^2)$, 表版为 $O(V+E)$。

正文上机题参考

1. 调用 `Graph::topological_sort` 或 `GraphList::topological_sort`，返回 `std::nullopt` 就是有环；读入时用 `Graph::add_edge` 检查顶点范围。
2. `Graph::dijkstra(source)` 返回距离; 若还要路径, 应同步维护 `predecessor[v]`, 松弛成功时记录前驱，再从终点逆推。负权输入必须拒绝。
3. `Graph::prim` 与 `kruskal` 对每张连通无向图都应返回 $V-1$ 条边且总权相等; 随机生成时先保证一棵生成树，再补随机边。
4. 给 `GraphList` 增加 `remove_edge(from,to,directed)`：在邻接向量中线性查找删除, 故 $O(\deg from)$; 无向边还要删反向项。矩阵只需把一个或两个格恢复为 `infinity`，故 $O(1)$。

课程作业与 `ref_DSA` 参考答案

1. **[张铭教材；本书自拟]** 无向完全图边数为 `V(V-1)/2`，有向完全图（无自环）边数为 `V(V-1)`。
2. **[`ref_DSA/DSA-HW/Chapter7Graph.md` 第 2 题；本书校正]** Dijkstra 生成的是 “从源点出发的最短路树”, 不一定是最小生成树。例: 无向边 AB=2、BC=2、AC=3, 从 A 出发时到 C 的最短路是直达边 AC，所以最短路树选 AB、AC，总权 5; MST 选 AB、BC，总权 4。`ref_DSA` 原答案把三边权 1、2、3 的 MST 总权写成 4，实际应为 3，而且存在等长最短路歧义，因此本书换用无平局反例。
3. **[`ref_DSA/Final/07graph.md`“拓扑排序”；本书代码验证]** Kahn 拓扑排序不断删除入度 0 顶点；若输出顶点数少于 `V`，图中有环。
4. **[`ref_DSA/Final/07graph.md`“图的周游/最短路径”；本书代码验证]** 邻接矩阵版 DFS/BFS 为 `O(V^2)`；邻接表版为 `O(V+E)`。本书当前 Graph 实现使用矩阵，因此 Dijkstra/Prim 为 `O(V^2)`，Floyd 为 `O(V^3)`。
5. **[课程第七章答案第 2 题；`ref_DSA/Final/07graph.md`]** Dijkstra 要求所有边权非负；存在负边时不能靠给所有边统一加常数修复，因为路径边数不同会改变路径总代价的相对次序。一般应改用 Bellman-Ford 等算法。

课程上机参考

使用同一无向连通图比较 Prim 和 Kruskal：边集可能不同，但总权和应相同且均有 `V-1` 条边。再用邻接表重做 DFS/BFS，与矩阵版比较访问序列和复杂度。

原书习题答案

1. 图 7.26 有 9 个顶点、11 条弧：$v_0 \to v_1$(6)、$v_0 \to v_3$(5)、$v_1 \to v_4$(1)、$v_2 \to v_0$(4)、$v_2 \to v_4$(1)、$v_3 \to v_6$(2)、$v_4 \to v_5$(8)、$v_4 \to v_7$(9)、

$v_6 \to v_5$(4)、$v_6 \to v_8$(4)、$v_7 \to v_8$(2)。(1) 相邻矩阵中 $A[i][j]$ 为弧 $v_i \to v_j$ 的权，无弧记 ∞、对角线记 0，非 ∞ 的非对角元恰是上面 11 项。(2) 邻接表（按终点编号排序）：v_0: 1(6) → 3(5)；v_1: 4(1)；v_2: 0(4) → 4(1)；v_3: 6(2)；v_4: 5(8) → 7(9)；v_5: 空；v_6: 5(4) → 8(4)；v_7: 8(2)；v_8: 空。(3) 出度 $v_0..v_8$ 依次为 `2 1 2 1 2 0 2 1 0`，入度为 `1 1 0 1 2 2 1 1 2`（两行各自加起来都是 11）。(4) 相邻矩阵只需存权：$9^2 \times 2 = 162$ 字节；邻接表要 9 个表头（标号 2 + 指针 4）和 11 个边结点（标号 2 + 权 2 + 指针 4）：$9 \times 6 + 11 \times 8 = 142$ 字节，**邻接表较省**。（脚本复算。）

2. 图 7.27 的弧：$v_0 \to v_1$、$v_0 \to v_2$、$v_1 \to v_5$、$v_2 \to v_6$、$v_3 \to v_2$、$v_3 \to v_4$、$v_3 \to v_7$、$v_4 \to v_7$、$v_5 \to v_6$、$v_7 \to v_6$。约定邻接点按编号从小到大访问、一棵树走完后从编号最小的未访问顶点重新出发。从 v_1 出发只能到 v_5、v_6，所以必然是森林。DFS **生成森林**三棵：$v_1 \to v_5 \to v_6$；$v_0 \to v_2$；$v_3 \to v_4 \to v_7$。BFS **生成森林**三棵：$v_1 \to v_5 \to v_6$；$v_0 \to v_2$；v_3 下挂 v_4、v_7 两个孩子。两者只在第三棵树上不同：DFS 从 v_4 深入到 v_7，BFS 由 v_3 同时发现 v_4、v_7。换一种邻接顺序会得到另一种合法答案。（脚本复算。）
3. 图 7.28 的弧：$v_0 \to v_2$(15)、$v_1 \to v_0$(10)、$v_1 \to v_3$(2)、$v_1 \to v_5$(30)、$v_3 \to v_1$(20)、$v_3 \to v_4$(15)、$v_4 \to v_1$(4)、$v_5 \to v_0$(10)、$v_5 \to v_2$(4)、$v_5 \to v_4$(10)。从 v_4 做 Dijkstra，距离按 $v_0..v_5$ 列出：初始 `[∞, ∞, ∞, ∞,0, ∞]`；定型 v_4 后 `[∞,4, ∞, ∞,0, ∞]`；定型 v_1 后 `[14,4, ∞,6,0,34]`；定型 v_3（无改进）；定型 v_0 后 v_2 变为 29；再依次定型 v_2、v_5。结果：$v_4 \to v_1$ 长 4；$v_4 \to v_1 \to v_3$ 长 6；$v_4 \to v_1 \to v_0$ 长 14；$v_4 \to v_1 \to v_0 \to v_2$ 长 29（经 v_5 到 v_2 要 38）；$v_4 \to v_1 \to v_5$ 长 34。（脚本复算。）
4. 图 7.29 的弧：$v_0 \to v_1$、$v_1 \to v_3$、$v_3 \to v_2$、$v_3 \to v_6$、$v_2 \to v_4$、$v_6 \to v_5$、$v_5 \to v_4$、$v_8 \to v_6$、$v_8 \to v_7$、$v_7 \to v_5$。算法：回溯枚举——每一步在当前入度为 0 的未选顶点中任选一个，删其出弧、递归，返回时恢复入度；递归树的叶数就是拓扑序列数，最坏指数级。本图共有 52 **个**不同的拓扑序列（脚本回溯枚举）。按 $v_0..v_8$ 只写下标：013286754、013287654、013826754、013827654、013862754、013867254、013867524、013872654、013876254、013876524、018326754、018327654、018362754、018367254、018367524、018372654、018376254、018376524、018732654、018736254、018736524、081326754、081327654、081362754、081367254、081367524、081372654、081376254、081376524、081732654、081736254、081736524、087132654、087136254、087136524、801326754、801327654、801362754、801367254、801367524、801372654、801376254、801376524、801732654、801736254、801736524、807132654、807136254、807136524、870132654、870136254、870136524。
5. 取任一拓扑序并按它排列行列；每条弧均从较早顶点指向较晚顶点，故对角线以下全 0。
6. 连通无环图为树，归纳删叶得边数 n-1；反之无环图是森林，c 个分量有 n-c

边，n-1 边推出 c=1。

7. 矩阵直接转置 `B[j][i]=A[i][j]`，`O(V^2)`；邻接表扫描每条 u->v 并向 B[v] 加 u，`O(V+E)`。
8. 矩阵布尔乘法 `B[i][j]=OR_k(A[i][k]&A[k][j])`，`O(V^3)`；表版对每个 u 经每个中继 x 标记 x 的邻居，`O(sum out(x)*in(x))`。
9. 有向图 DFS 三色，遇灰边即环；无向图 DFS 遇到非父已访问邻居即环，均 `O(V+E)`。
10. 每对最短路输出本身有 `Theta(V^2)` 个距离/前驱信息，因而最大可能下限是 `Omega(V^2)`；稠密输入也有 `Theta(V^2)` 边。
11. 反证。设 $T_1 \neq T_2$ 都是 MST，在只属于其中一棵树的边（即 $(T_1 \setminus T_2) \cup (T_2 \setminus T_1)$）中取权最小的边 e，不妨设 $e \in T_1 \setminus T_2$。把 e 加入 T_2 形成唯一的回路；T_1 无回路，所以这条回路上必有一条边 $f \notin T_1$，即 $f \in T_2 \setminus T_1$，也只属于其中一棵树。由 e 的取法和权值互不相等得 $w(e) < w(f)$，于是 $T_2 + e - f$ 是一棵权更小的生成树，与 T_2 是 MST 矛盾。
12. Dijkstra 的 predecessor 边形成以源为根的最短路树（可达部分），一般不是 MST；三角形 AB=2、BC=2、AC=3 从 A 即为反例。
13. 无向图重复 DFS/BFS 标号；有向图用 Kosaraju 两遍 DFS（第二遍在转置图），或 Tarjan，`O(V+E)`。
14. 取 G 中一条最长的简单路径 $v_0 v_1 \cdots v_k$。v_k 的度 ≥ 2，除 v_{k-1} 外它还有邻点 u；若 u 不在路径上，路径可延长，与「最长」矛盾，所以 $u = v_i\,(i < k-1)$，$v_i \cdots v_k v_i$ 就是一条回路。另一证法：若 G 无回路，它是森林，边数 $\leq n-1$；但各顶点度 ≥ 2 给出边数 $\geq 2n/2 = n$，矛盾。
15. 至少 n-1（连通底图），至多 n(n-1)/2（拓扑序中任意前到后连边）；所有中间数均可构造。
16. 反证：每点入度至少一，沿入边无限回溯，有限图必成环，违背 DAG。
17. DAG 拓扑序唯一当且仅当 Kahn 每一步可选入度 0 顶点恰一个；等价于该序相邻顶点之间均有边。
18. 先数每列入度和每行出度，找列和 n-1 且行和 0 的顶点；扫描矩阵 `O(n^2)`。
19. Kosaraju/Tarjan 每顶点、边常数次访问，故 `O(V+E)`。
20. 表中插边尾插/头插 `O(1)`（若查重则 `O(deg u)`）；删边扫描 u 的链 `O(deg u)`，无向再删反向项。
21. 有从未定型顶点指向已定型顶点的负边时 Dijkstra 会错；不能靠局部修改保留一次定型，应使用 Bellman-Ford，负环另报无有限最短路。
22. 先求所有 u 到 v 的距离（Floyd `O(V^3)`，或每个源 Dijkstra），每列取最大值为偏心距，取最小列即中心。
23. 先验证 DAG，再检查恰一入度 0 根、其余恰一入边、且从根可达全部顶点；`O(V+E)`。
24. 根是能到达全部顶点者；求 SCC 缩点后，若凝聚图有唯一源 SCC，则该 SCC 的全部顶点是根，否则无根，`O(V+E)`。
25. 拓扑序上对每源动态规划最长边数和前驱；对所有源重复为 `O(V(V+E))`，回溯

前驱输出路径。

26. DFS 回溯，维护 `on_path`，到目标即输出副本；返回时撤销标记。输出可能指数，复杂度至少与输出量同阶。

原书上机题参考

1. 对汇率边取 `-log(rate)`，乘积大于 1 等价于负环；Bellman-Ford 检出负环并回溯前驱得到一条套利环。枚举所有简单套利环本身指数。
2. 先拓扑排序；前推 `ve[v]=max(ve[u]+w)`，终点为工期；后推 `vl[u]=min(vl[v]-w)`；活动 `(u,v)` 的最早 `ve[u]`、最晚 `vl[v]-w`，相等即关键。文件中存顶点、边、工期，输出活动表与关键路径。

第 8 章 内部排序

补充题参考答案

1. **[课程第八章答案第 2 题]** Randomized-Select 每次随机划分, 只递归一侧; 期望时间 `O(n)`，最坏 `O(n^2)`。
2. **[课程第八章答案第 1 题]** 先按长度稳定分桶，再从最高位到最低位对仍有该位的字符串做稳定桶排序；每个字符参与常数次处理，总时间 `O(sum |s_i|)`。
3. **[课程第八章答案第 3 题]** 索引排序只移动下标，最后按下标访问原数组；索引键比较相等时保留原下标次序即可稳定。

正文习题答案

1. 对 `{49,38,65,97,76,13,27}`：直接插入第一趟为 `{38,49,65,97,76,13,27}`；冒泡第一趟为 `{38,49,65,76,13,27,97}`；简单选择第一趟为 `{13,38,65,97,76,49,27}`。
2. 插入排序只在新键严格小于前键时移动，相等键不会越过，故稳定。堆排序的远距离交换会跨过相等键；例如带身份的 `[(2,a),(2,b),(1,c)]` 建大顶堆后换根与末尾，可能让 `(2,b)` 排到 `(2,a)` 前。
3. 本书 `sorting::partition` 取末元素为枢轴，扫描 `[first,last-1)`，小于枢轴的交换到 `boundary` 左边，最后把枢轴换到 `boundary`。已升序、逆序或全相等输入会每次切出空侧，递归式 $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$。
4. 题面未指定 8 个数。以 `{49,38,65,97,76,13,27,50}` 为可复算例：建大顶堆得 `{97,76,65,50,49,13,27,38}`；逐趟交换并下沉后为 `{76,50,65,38,49,13,27,97}`、`{65,50,27,38,49,13,76,97}`、`{50,49,27,38,13,65,76,97}`、`{49,38,27,13,50,65,76,97}`、`{38,13,27,49,50,65,76,97}`、`{27,13,38,49,50,65,76,97}`、`{13,27,38,49,50,65,76,97}`。
5. 常规二路归并需要 $O(n)$ 辅助数组。可以原地归并，但要靠旋转、块归并等复杂技术，通常增加实现复杂度、常数甚至比较/移动次数；本书 `sorting::merge_sort` 明确分配一个 `buffer`。
6. 直接按二补码字节做无符号 LSD 排序会把负数排在正数后。本书 `sorting::radix_sort` 把 `int` 转为 `uint32_t` 后异或 `0x80000000U`，翻转符号位，使无符号次序与有符号数值次序一致。
7. 数据很小或近乎有序时选插入排序，因常数小、稳定且可接近 $O(n)$；要求

最坏 $O(n \log n)$ 且只能用 $O(1)$ 额外数组空间时选堆；要求稳定且容得下 $O(n)$ 缓冲时选归并；一般内存随机数据常选优化快排。

正文上机题参考

1. 用 `sorting::random_values` 生成同一母本,复制后分别调用 `sorting::insertion_sort`、`sorting::heap_sort`、`sorting::quick_sort`、`sorting::merge_sort`，用 `Stopwatch::start` 与 `Stopwatch::elapsed_seconds` 计时。排除生成和复制时间，每个规模重复多次并核对结果相同。
2. 本书快排取末元素为枢轴，升序、逆序、全相等都可退化；记录 `sorting::quick_sort_range` 的最大递归深度，应随 n 线性增长。`sorting::quick_sort_optimized` 只限制栈深和小分区常数，不改变最坏比较次数。
3. 监视哨版把待插入值暂存到下标 0，省去内层边界判断；与 `sorting::insertion_sort` 对拍空、单元素、重复、随机和逆序输入。若占用下标 0，接口要明确有效数据区从 1 开始。
4. 调用 `sorting::radix_sort` 后与 `std::sort` 对拍，数据必须含 `INT_MIN`、`-1`、`0`、正数、`INT_MAX` 和重复值；本书现有实现覆盖完整 `int` 范围。

课程作业与 `ref_DSA` 参考答案

1. **[`ref_DSA/homework/08` 内排序`.md` 复习；本书自拟]** 插入排序稳定，堆排序通常不稳定；稳定性取决于相等键的相对次序是否保持。
2. **[`ref_DSA/homework/08` 内排序`.md` 第 1 题；课程第八章答案第 2 题]** 快排末元素作枢轴时，已排序或逆序输入会产生 `O(n^2)` 最坏递归；随机选择枢轴可降低退化概率。课程答案还用 Randomized-Select 在期望线性时间找第 k 小元素。
3. **[张铭教材；本书自拟]** 归并排序需要 `O(n)` 辅助空间；原地归并可以降低空间但实现复杂且常数更大。
4. **[`ref_DSA/DSA-HW/Chapter8.md` 第 2 题；本书代码验证]** 基数排序处理有符号整数时，需先建立保持数值顺序的键；本书把值转为 `uint32_t` 并翻转符号位，使无符号字典序对应有符号数值序。
5. **[`ref_DSA/homework/08` 内排序`.md` 第 1 题（数据见 `img/8-1.jpg`）；2025 秋书面作业；脚本复算]** 对 `70,89,63,39,87,92,22,87',12,6,70',94,17,31,11,50` 以末元素 50 为枢轴、按本书 `sorting::partition` 的写法（boundary 左侧都小于枢轴）划分，每次交换后：`39,89,63,70,`…（39 ↔ 70）→ `39,22,63,70,87,92,89,`…（22 ↔ 89）→ `39,22,12,70,87,92,89,87',63,`…（12 ↔ 63）→ `39,22,12,6,87,92,89,87',63,70,`…（6 ↔ 70）→ …`,17,92,89,87',63,70,70',94,87,31,11,50`（17 ↔ 87）→ …`,31,89,87',63,70,70',94,87,92,`（31 ↔ 92）→ `39,22,12,6,17,31,11,87',63,70,70',94,87,92,89,50`（11 ↔ 89）；最后枢轴与 boundary 处的 87′ 交换,得 `39,22,12,6,17,31,11,50,63,70,70',94,87,92,89,87'`，枢轴落在下标 7。注意 87′ 被换到了末尾、跑到 87 后面——这就是快排不稳

定的现场。（手写答案用的是「挖坑」式划分，中间序列与此不同但终态同样把 50 放到下标 7；按题意应写本书这种交换式。）

6. **[`ref_DSA/homework/08` 内排序`.md` 第 3 题；2025 秋书面作业]** n 个数的 2^n 个子集和排序输出：**不必先生成再排序**。维护有序表 $L_0=[0]$；处理第 i 个数 x_i 时，L_{i-1} 与 $L_{i-1}+x_i$（逐项加 x_i，仍有序）是两个等长有序表，二路归并得 L_i。第 i 步代价 $\Theta(2^i)$，总计 $\sum_i 2^i=\Theta(2^n)$——与输出规模同阶，是最优的。先枚举 2^n 个掩码再排序是 $\Theta(n2^n)$。**用「可达和」布尔数组做背包 DP 不满足题意**：它只给出**不同的**和、丢了重数，且代价 $O(nS)$ 依赖数值大小，负数还要平移。（脚本与暴力枚举对拍 500 组，含负数与重复值。）

课程上机参考

生成随机、已排序、逆序、全相等和含负数五类数据，对拍插入、堆、快排、归并、基数排序与标准排序结果，并记录时间和递归深度。

原书习题答案

1. 下界：5 个元素有 $5!=120$ 种次序，每次比较至多把剩余可能一分为二，$2^6=64<120\le 2^7$，所以至少 7 次；下面的「归并插入」（Ford–Johnson）恰好 7 次。① 比较 $a:b$、$c:d$（2 次），不妨 $a<b$、$c<d$；② 比较两组的较大者 $b:d$（第 3 次），不妨 $b<d$，于是有链 $a<b<d$，另知 $c<d$；③ 把 e 二分插入三元链 $a<b<d$（2 次，累计 5 次）；④ 已知 $c<d$，只需把 c 二分插入新链里排在 d 前面的部分——至多 a、b、e 三个元素，2 次（累计 7 次）。脚本对 120 种输入逐一执行：全部排对，最多恰好 7 次比较。
2. 用第二个栈作输出栈，反复把输入栈顶取出并按有序插入输出栈；再倒回使最小在顶，`O(n^2)` 时间、`O(n)` 栈空间。
3. 记增量 $h_k=2^k-1$，按从大到小的次序使用。分两段估计每一趟的代价。(a) $h_k>\sqrt{n}$：这一趟是对 h_k 个各长约 n/h_k 的子序列做插入排序，最坏 $O(h_k(n/h_k)^2)=O(n^2/h_k)$。(b) $h_k\le\sqrt{n}$：此前已做过 h_{k+1}、h_{k+2} 排序，二者互素（相邻的 2^j-1 互素），凡能写成 $a\,h_{k+1}+b\,h_{k+2}$（$a,b\ge 0$）的距离上数组已经有序，而大于 $(h_{k+1}-1)(h_{k+2}-1)$ 的距离都能这样写，所以本趟每个元素至多越过 $O(h_{k+1}h_{k+2}/h_k)=O(h_k)$ 个同组元素，本趟代价 $O(nh_k)$。两段都是等比级数：(a) 段之和由最小的、约为 $\sqrt{n}$ 的那个增量主导，为 $O(n\cdot n/\sqrt{n})=O(n^{3/2})$；(b) 段之和由最大的、约为 $\sqrt{n}$ 的增量主导，为 $O(n\sqrt{n})$。合计最坏 $O(n^{3/2})$；Pratt 构造了使其达到 $\Omega(n^{3/2})$ 的输入，故为 $\Theta(n^{3/2})$。
4. 题中「图 8.5」在原书里是冒泡排序图，但原书 8.2.2 的 Shell 排序正文也把例子称作「图 8.5 中的 34 和 34′」——Shell 排序的例子在本书是图 8.2。两张图用的是同一个序列 `45 34 78 12 34′ 32 29 64`（$n=8$），所以照做即可。小于 n 的 2^k-1 型增量为 7、3、1：gap=7 时只有 (45, 64) 一组需比较，已有序，数组不变；gap=3 时三组 {45,12,29}、{34,34′,64}、{78,32} 各自插入排序，得 `12 34 32 29 34′ 78 45 64`；gap=1 得 `12 29 32 34 34′ 45 64 78`。与原

书 4、2、1 的增量相比（图 8.2：gap=4 后 `34′ 32 29 12 45 34 78 64`，gap=2 后 `29 12 34′ 32 45 34 78 64`），本例两个 34 的相对次序碰巧保持住了，原书那组增量则把 34′ 排到了 34 前面——但这只是本例碰巧，Shell 排序换了增量仍不稳定。（脚本复算。）

5. 选择到最小位置与 i 相同时不交换；渐近比较数不变，少的是已就位时的移动，需以同编译器和输入测常数差。
6. 对每项数小于它的项数并放入该位置；无重复时正确，`Theta(n^2)`，比较数同选择排序量级但移动模型不同。
7. 会是 $\Theta(n\log_3 n)$，但它和 $\Theta(n\log n)$ 是同一个阶——$\log_3 n = \log_2 n/\log_2 3$，只差常数因子。三叉堆（下标从 0 起，结点 i 的孩子为 $3i+1..3i+3$，父为 $\lfloor (i-1)/3 \rfloor$）高度降为 $\log_3 n$，但每层下沉要在 3 个孩子中选最大（2 次比较）再与父比（1 次），每层 3 次，一次下沉约 $3\log_3 n \approx 1.89\log_2 n$ 次比较，而二叉堆是 $2\log_2 n$；下沉的层数（交换次数）少约 37%。所以常数略好，渐近阶不变。一般地，d 叉堆一次下沉 $\Theta(d\log_d n)$，d 为常数时仍为 $\Theta(\log n)$。
8. 仍然正确：多出来的比较都落在已经排好的前缀里，那里相邻元素有序，不会触发交换。渐近复杂度仍是 $\Theta(n^2)$，但比较次数变了：原写法第 i 趟（$i=1..n-1$）比较 $n-i$ 次，共 $n(n-1)/2$；改写后每趟都比较 $n-1$ 次，$n-1$ 趟共 $(n-1)^2$ 次，约为原来的 2 倍。交换次数不变（等于逆序对数）。若保留「本趟无交换就结束」的标志，已有序的输入两种写法都只做一趟 $n-1$ 次比较。
9. 鸡尾酒排序交替左/右扫描，每轮收缩两端；有交换标志时近有序快，最坏 `Theta(n^2)`、稳定。
10. 冒泡排序会。按原书「每趟把最小的从右往左冒到前面」的写法，`3 4 2 1` 的第一趟：1 先与 2 交换，2 **被换到下标** 3，而 2 的最终位置是下标 1——它朝最终位置的反方向挪了一格。原因是较小元素往左冒时，会把路过的较大元素逐个往右推，不管那个元素该去哪里。快速排序的划分本身不会：被交换的元素都是从轴值最终位置的一侧换到另一侧，方向总是朝着最终位置（可能越过头，但不会反向）——本书末元素作轴的划分和原书的「填坑」划分对 $n \le 6$ 的全部排列穷举，都没有一次反向移动。但原书算法 8.6 在划分前先把中间的轴值与末元素交换，这一换就可能反向：`0 2 1 3 4` 中轴值 1 从下标 2 被换到下标 4，而它的最终位置是 1。（脚本穷举验证。）
11. 三路 partition 维护 `<p, =p, >p` 三段，等段不递归；有 d 个不同键时避免重复键导致的线性深度，典型代价 `O(n log d)`。
12. 显式栈压待排区间；普通最坏深 n。每次压较大区、循环处理较小区，栈深 `O(log n)`；队列可做但不保留 DFS 的空间界。
13. 双指针 partition：left 找正、right 找负，交换直到相遇；`Theta(n)`、`O(1)`，不稳定。
14. 荷兰国旗三指针 `low,mid,high`：红换 low、白 mid++、蓝换 high；每项一次，`Theta(n)`、`O(1)`。

15. 用 BFPRT 的五元组中位数作轴、三路划分，只递归目标侧，最坏 `Theta(n)`。
16. 两有序数组以分割点二分，令左侧总数 n 并满足两侧交叉不等式，取左侧最大；每步丢半，`O(log n)`。
17. 中位数的中位数保证至少约 3n/ 10 被淘汰，递推 `T(n)<=T(n/ 5)+T(7n/ 10)+O(n)=Theta(n)`；作快排轴则每层平衡，`Theta(n log n)`。
18. k 路归并用大小 k 堆/败者树，每项 `O(log k)`，总 `O(n log k)`，趟数下降但比较常数和缓冲要求上升。
19. 不正确。即使只比主项、忽略常数，比值也是 $n^2/(n\log_2 n) = 256/8 = 32$，不是 64；更根本的是 Θ 记号不含常数因子和低阶项：插入排序内层循环极简单、对小规模或近乎有序的输入很快，归并排序有递归调用和辅助数组的开销。$n = 256$ 这样的规模两者实际差多少只能实测，往往远小于 32 倍；正因如此，工程实现常在规模小于几十时切换到插入排序。
20. 栈版模拟递归区间；无栈版自底向上宽度 1,2,4... 两路归并，`O(n log n)`。
21. 可用块旋转/二分切分的原地归并把额外空间降至 `O(1)`，但实现复杂；稳定且线性时间的工程实现通常仍保留 O(n) 缓冲。
22. 先按低位面值稳定分配、再按高位花色稳定分配；每趟 13/4 个桶，稳定性保证字典序。
23. (1) 每趟确定一个记录的最终位置、以后不再访问它的：直接选择排序、冒泡排序（每趟缩短一个位置）、堆排序（换到数组尾部的不再参与下沉）、快速排序（轴值就位后不再进入后续划分）。(2) 每一趟都要访问全部记录的：Shell 排序（每个增量扫一遍全表）、归并排序（每趟二路归并都过一遍全部记录）、基数排序（每一位都要分配、收集全部记录）。直接插入排序两头都不算：第 i 趟只访问前 $i+1$ 个记录，但前缀里的记录并没有到达最终位置，之后还会被挪动。
24. 最小/最大/平均无需排序可线性；中位数用选择；众数可哈希或排序后扫，排序只在多次有序查询时值得。
25. (1) 1000 个数、只运行一次：任一 $O(n\log n)$ 算法（快速排序或标准库），实现简单可靠最要紧。(2) 50 个数：直接插入排序，常数小、代码短。(3) 5 个记录的函数被反复调用：写成固定的比较交换序列（或展开的插入排序），最多 7 次比较（见第 1 题），没有循环和递归开销。(4) 1000 个大记录、移动代价极高：索引（地址）排序——只对排序码和记录地址排序，最后按结果把外存记录一次重排到位，每个记录至多搬一次；若必须直接排，选交换次数最少的直接选择排序（至多 $n-1$ 次交换）。(5) 书架上的书基本有序、偏离不超过一个书架：插入排序，代价与逆序对数成正比，近乎有序时接近线性。(6) 500 张随机卡片按字母排：按首字母分桶（桶/基数排序），桶内再插入排序——人工整理卡片也正是这么做的；用归并排序也可以。(7) 复查 5000 个单词是否有序：不必排序，顺序扫描一遍比较相邻两项，$O(n)$。
26. (1) 以 6 为轴值，按原书算法 8.6 的分割（先把轴值换到末尾，左右指针交替向中间扫描填坑），一次分割后为 `1 2 0 5 3 4 6 8 7 9`，6 落在下标 6；

若不先换到末尾、直接以首元素为坑做左右交替扫描，本例结果相同。(2) 先建最大堆 `9 7 8 5 3 6 4 2 0 1`，堆顶 9 与末元素交换后对前 9 个下沉，得 `8 7 6 5 3 1 4 2 0 9`。(3) 基数 3 的 LSD 第一趟按 $k \bmod 3$ 分配：0 号桶 6 9 3 0，1 号桶 7 4 1，2 号桶 2 8 5，依次收集得 `6 9 3 0 7 4 1 2 8 5`。（脚本复算。）

27.（脚本逐步复算；53′ 用来检查稳定性。）

- (1) 直接插入，第 i 趟（$i = 1..11$）把下标 i 的元素插入前面的有序段后依次为：`46 74 16 53 14 26 40 53′ 86 65 27 34`；`16 46 74 53 14 26 40 53′ 86 65 27 34`；`16 46 53 74 14 26 40 53′ 86 65 27 34`；`14 16 46 53 74 26 40 53′ 86 65 27 34`；`14 16 26 46 53 74 40 53′ 86 65 27 34`；`14 16 26 40 46 53 74 53′ 86 65 27 34`；`14 16 26 40 46 53 53′ 74 86 65 27 34`；`14 16 26 40 46 53 53′ 74 86 65 27 34`；`14 16 26 40 46 53 53′ 65 74 86 27 34`；`14 16 26 27 40 46 53 53′ 65 74 86 34`；`14 16 26 27 34 40 46 53 53′ 65 74 86`。53 始终在 53′ 前面，稳定。
- (2) 直接选择，第 i 趟（$i = 0..10$）选出最小者与下标 i 交换后依次为：`14 74 16 53 46 26 40 53′ 86 65 27 34`；`14 16 74 53 46 26 40 53′ 86 65 27 34`；`14 16 26 53 46 74 40 53′ 86 65 27 34`；`14 16 26 27 46 74 40 53′ 86 65 53 34`；`14 16 26 27 34 74 40 53′ 86 65 53 46`；`14 16 26 27 34 40 74 53′ 86 65 53 46`；`14 16 26 27 34 40 46 53′ 86 65 53 74`；`14 16 26 27 34 40 46 53′ 86 65 53 74`；`14 16 26 27 34 40 46 53′ 53 65 86 74`；`14 16 26 27 34 40 46 53′ 53 65 86 74`；`14 16 26 27 34 40 46 53′ 53 65 74 86`。第 3 趟把 53 换到了 53′ 后面，最终 53′ 在前——不稳定。
- (3) 最大堆，建堆时自下标 5 到 0 逐个过筛：下标 5：26 与孩子 34 交换，得 `46 74 16 53 14 34 40 53′ 86 65 27 26`；下标 4：14 与 65 交换；下标 3：53 与 86 交换；下标 2：16 与 40 交换；下标 1：74 与 86 交换；下标 0：46 一路下沉到下标 7，初始堆为 `86 74 40 53′ 65 34 16 46 53 14 27 26`（完全二叉树逐层为 86 / 74 40 / 53′ 65 34 16 / 46 53 14 27 26）。此后每次换顶并过筛，堆区依次为：`74 65 40 53′ 27 34 16 46 53 14 26`；`65 53′ 40 53 27 34 16 46 26 14`；`53′ 53 40 46 27 34 16 14 26`；`53 46 40 26 27 34 16 14`；`46 27 40 26 14 34 16`；`40 27 34 26 14 16`；`34 27 16 26 14`；`27 26 16 14`；`26 14 16`；`16 14`；`14`。
- (4) 快速排序，每次取子序列最左元素为轴值（填坑法），每层划分后：`34 27 16 40 14 26 46 53′ 86 65 53 74`；`26 27 16 14 34 40 46 53′ 86 65 53 74`；`14 16 26 27 34 40 46 53′ 74 65 53 86`；`14 16 26 27 34 40 46 53′ 53 65 74 86`。对应的二叉搜索树（轴值为根）：46 的左子树 34(26(14(-,16),27),40)，右子树 53′(-,86(74(53(-,65),-),-))。53′ 排到 53 前面，不稳定。
- (5) 自底向上二路归并（等值先取左边）：长 2：`46 74 | 16 53 | 14 26 | 40 53′ | 65 86 | 27 34`；长 4：`16 46 53 74 | 14 26 40 53′ | 27 34`

65 86；长 8：14 16 26 40 46 53 53′ 74 | 27 34 65 86；最后 14 16 26 27 34 40 46 53 53′ 65 74 86。稳定。

28. 用锦标赛（树形选择）排序：先两两比赛选出最小值要 $n-1$ 次比较，此后每取下一个只沿冠军所在叶到根的路径重赛，约 $\lceil \log_2 n \rceil$ 次。对题给 15 个数（补一个 $+\infty$ 凑成 16 个叶，与 $+\infty$ 的比赛不计），选出 017、094、154、170 共比较 25 **次**：选出 017 用 14 次，之后三次重赛共 11 次。堆排序（Floyd 法建小顶堆，下沉时先比两个孩子再与父比）建堆就用 20 次，再做 3 次删除最小并下沉共 17 次，合计 37 **次**。直接选择则要 $14+13+12+11=50$ 次。$k \ll n$ 时锦标赛约 $n-1+(k-1)\lceil \log_2 n \rceil$ 次，优于堆排序的约 $2n+2(k-1)\log_2 n$，代价是多用 n 个结点存比赛树。（脚本按上述口径逐次计数。）

29. 所有和共 n^k 个，输出本身就要 $\Omega(n^k)$。逐个集合归并：先把 S_1 排序得 L_1；有了有序表 L_{i-1}（长 n^{i-1}）后，把 S_i 排序，对每个 $x \in S_i$，$L_{i-1}+x$（逐项加 x）仍是有序表，用大小为 n 的堆把这 n 个表做多路归并得 L_i，代价 $O(n^i \log n)$。总代价 $\sum_i O(n^i \log n) = O(n^k \log n)$，与「全部列出再排序」的 $O(n^k \cdot k \log n)$ 相比省掉了因子 k。（第 8 章课程作业第 6 题的子集和，是同一思路在「每个集合只有 $\{0, x_i\}$ 两个数」时的特例。）

30. (1) 先把每个序列各自从小到大排序。A_1 嵌入 A_2 当且仅当排序后逐位 $x_{(i)} < y_{(i)}$：若某个排列能逐位小于，把它调整成排序后的配对仍逐位小于（交换论证）。由此嵌入关系是严格偏序——反自反、传递，因而也反对称。(2) 把 B 中序列按分量和递增排列（若 A_i 嵌入 A_j，和必严格更小，所以这是一个拓扑序），在这个次序上做 DAG 最长路的动态规划：$len[j] = 1 + \max\{len[i] : i<j,\ A_i \text{ 嵌入} A_j\}$，同时记前驱以回溯出最长嵌入链。(3) 正确性：任何嵌入链在该次序下依次出现，DP 考虑了每个元素的所有可能前驱。代价：各序列排序 $O(nk \log k)$，两两判定嵌入 $O(n^2 k)$，DP $O(n^2)$，最坏 $O(nk \log k + n^2 k)$。

31. (1) 表插入排序：下标 0 作头结点，`next[0]` 指向最小记录，初始只有记录 1 成链；对 $i = 2..n$，从头沿 `next` 走到最后一个不大于 $A[i]$ 的结点之后插入（等值插在后面，保持稳定）。只改 `next`、不移动记录，比较次数与直接插入相同，$O(n^2)$。(2) 按链就地重排（只用一个暂存结点做交换）：令 `p = next[0]`；对 $i = 1..n-1$：先 `while (p < i) p = next[p];`（$p < i$ 说明该记录已被换走，沿它留下的指针追回来），记 `q = next[p]`；若 `p != i`，交换 $A[i]$ 与 $A[p]$（连同 `next` 域），并令 `next[i] = p`（记下被换走的原 $A[i]$ 现在在 p）；最后 `p = q`。每个记录至多被交换一次，追指针的总步数也是 $O(n)$，所以整理是 $\Theta(n)$。（脚本随机 2000 组对拍。）

32. 三种判定树的叶都对应 A, B, C 的一种次序，内部结点是一次比较。直接插入：根比较 $A:B$；设 $A<B$，再比 $B:C$，$B<C$ 时得 $A<B<C$（2 次），否则再比 $A:C$ 分出 $A<C<B$ 与 $C<A<B$；$A>B$ 一侧对称。共 6 片叶、高 3，最好 2 次。直接选择：第一趟比 $A:B$，再拿较小者与 C 比选出最小（2 次），第二趟比剩下两个（1 次）——每条路径恰好 3 次，判

定树是 8 片叶的满二叉树，其中 2 片叶不可能到达（例如 $A<B$、$C<A$ 之后第二趟还要比 $A:B$，结果早已知道）。冒泡（原书从右往左冒）：第一趟比 $B:C$，再把两者中较小的与 A 比；第二趟比后两个位置。不带「无交换即结束」标志时同样每条路径 3 次、有冗余比较；带标志时，若第一趟没有交换（即 $A<B<C$），2 次就结束。

33. 三次逆置：先整体逆置 $a_0..a_{n-1}$，再分别逆置前 k 个和后 $n-k$ 个，结果就是循环右移 k 位（k 先对 n 取模）。逆置一段长 m 只需 $\lfloor m/2 \rfloor$ 次交换和一个暂存单元，总共约 n 次交换，满足 $O(n)$ 与一个元素的附加存储。另一种做法是按 $\gcd(n,k)$ 个环逐个「拿起一个、依次挪位、放回」，每个元素恰好移动一次。### 原书上机题参考

34. (1) 二分插入只减少比较：第 i 个记录找位置约 $\log_2 i$ 次比较，总比较 $O(n\log n)$；但插入时要把后面的记录逐个后移，移动次数与直接插入完全相同，仍是 $\Theta(n^2)$，所以只在比较代价远大于赋值时明显更快。(2) 交换式插入每前进一格是一次交换（3 次赋值），而教材的移位式每格 1 次赋值、最后再写回一次，数据移动约多 3 倍，通常更慢。实验应分别统计比较次数与赋值次数，在随机、已有序、逆序三类输入上对比，理论预测：有序输入三者都是线性，随机与逆序时二分插入比较最少、交换式移动最多。

35. (1) 结束条件：**连续的一趟奇数扫描和一趟偶数扫描都没有发生交换**。只看一趟不够——奇数趟没交换时，偶数位置上的相邻逆序可能还在。(2) 每趟对一半的相邻对做比较交换，n 趟之内必然排好（奇偶交换排序的经典结论）。(3) 按上面的结束条件：已从小到大有序时，一趟奇、一趟偶都无交换即结束，共 $n-1$ 次比较；从大到小有序时要扫满 n 趟才排好，再加两趟确认，比较次数为 $n(n-1)/2+(n-1)$（如 $n=10$ 时 54 次）。平均情况同样需要 $\Theta(n)$ 趟、每趟 $\Theta(n)$ 次比较，平均时间 $\Theta(n^2)$；它的价值在于同一趟内的比较互不相关、可以并行。（脚本模拟计数。）

36. 先扫一遍求平均值 m（$O(n)$），分割时把 $<m$ 的放左边、$\geq m$ 的放右边；轴值不一定是序列元素，所以分割后没有「已就位的轴值」可以排除，两侧都要递归。必须处理一侧为空的情形：子序列全部相等时平均值就等于它们，全部落到右侧，若不特判会无限递归——此时应直接返回。平均值对均匀分布的数据划分得很均衡，但对偏斜分布（少数极大值、指数分布）划分很差，最坏仍是 $\Theta(n^2)$；平均值的累加还要防溢出（用 `long long` 或增量平均）。实验应在均匀、偏斜、大量重复三类数据上与原算法比较运行时间和递归深度。

37. 英文单词的基数排序：按最大长度 n 做 LSD，从最后一个字符位往前，每位按字符分配到桶、再按桶序收集；较短的单词在缺位处视为「空字符」，空字符小于任何字母（相当于左对齐、右补最小哨兵），这样 `he` 排在 `head` 前面。区分大小写时可直接按字符编码分桶，或约定 $A<a<B<b<\cdots$ 再映射。链式实现时桶是链队列，分配和收集只改指针，正是原书静态链基数排序的做法。不适合链式存储的算法：Shell 排序（按增量跳跃访问）、堆排序（靠下标算父子）、二分插入排序（二分要随机访问）以及按下标取中值作轴的

快速排序——它们依赖 $O(1)$ 的随机访问，换成链表后每次定位就要 $O(n)$。适合链式的有直接插入、直接选择、归并（链表归并不需要辅助数组）和基数排序。## 第 9 章 外部排序

补充题参考答案

1. **[课程第九章答案第 1 题]** 堆顶输出后读入新记录；不小于当前顺串末值则留在本堆，否则冻结到下一顺串，直到堆空。
2. **[课程第九章答案第 3 题]** 每页可放 `floor(page_size/record_size)` 条记录；内存页数决定最大归并路数，顺串平均长度由扫雪机效应约为 `2M` 条记录；访外次数按读写页分别累计。
3. **[课程第九章答案第 2 题]** loser tree 和 winner tree 每次更新为 `O(log k)`，最小堆也为 `O(log k)`；树结构通常能减少比较常数。

正文习题答案

1. 外存按页传输是因为一次定位的固定成本远大于连续传一页；逐记录随机读会反复付定位代价。缓冲把一页记录留在内存中批量处理，顺序读写可把 I/O 次数降到页数数量级。
2. 用本书 `external_sort::replacement_selection` 同逻辑实算，$M = 7$ 时得到两个顺串：`15 25 30 35 45 49 50 60` 和 `1 16 27`。读入 16、27、1 时它们小于当前已输出值，依次冻结到下一顺串；60 可留在本顺串。
3. 若 M 能装下全部输入，不再有后续记录需要冻结；先建最小堆再连续弹出，得到一个包含全部记录的有序顺串，等价于堆排序的输出阶段。
4. 赢者树内部结点记两子树的胜者，败者树记本场败者、总冠军另存。替换一个叶后，只有该叶到根路径上的比赛结果可能变化，路径长 $\lceil \log_2 k \rceil$，其他子树结果仍有效。
5. 二路归并每趟把顺串数约减半，需 $\lceil \log_2 m \rceil$ 趟。置换选择让初始顺串平均长度约为 $2M$（依输入分布而异），从源头减少 m，从而减少归并趟数和整文件读写。

正文上机题参考

1. 调用实际接口 `external_sort::replacement_selection(input,memory)`；守门除了对原书输入固定顺串，还要验证每个顺串有序、所有输入恰出现一次以及 `memory==0` 被拒绝。
2. 使用 `WinnerTree::winner()`（冠军的值）、`WinnerTree::winner_index()`（冠军是哪一路）和 `WinnerTree::replace(player,value)`；每次随机替换后用 `std::min_element` 扫描选手数组对拍冠军的值和下标。（`detail::TournamentOps` 只是 `TournamentOps::better`、`TournamentOps::worse`、`TournamentOps::next_power_of_two` 这几个比较助手，本身不计数；要统计比较次数得自己在测试里数。）
3. 使用 `LoserTree` 保存各路当前头元素，冠军输出后以该路下一个值调用

`LoserTree::replace`；耗尽路用正无穷哨兵。最终输出与把所有向量拼接后 `std::sort` 对拍。

4. 分别调用 `external_sort::replacement_selection(input,4/8/16)`,记录返回向量的数量与各顺串长度；同一输入下容量增大通常使顺串数不增，但守门应核对记录守恒和各顺串有序，而不是把随机趋势当定理。

课程作业与 `ref_DSA` 参考答案

1. **[课程第九章答案第 1 题；`ref_DSA/DSA-HW/Chapter9.md` 第 1 题]** 置换选择使用大小为 M 的最小堆产生有序顺串；输出堆顶后读入新值，若新值不小于当前顺串末值则继续本顺串，否则冻结到下一顺串。
2. **[课程第九章答案第 2 题；`ref_DSA/DSA-HW/Chapter9.md` 第 2 题]** M 路归并每次从各路当前头元素中取最小值；用 loser tree 时每次取出和更新为 `O(log M)`。
3. **[课程第九章答案第 2、3 题]** 外部排序主要代价是磁盘读写次数，不应只比较 CPU 比较次数；缓冲块数决定归并路数，顺串长度决定归并趟数。
4. **[`ref_DSA/`第九章作业`.pdf` 第 1 题；2025 秋书面作业；脚本复算]** 数据 `(10,20,2,5,15,8,26,4,11,7,13,16,21,14,6)`、堆大小 5：前 5 个数建堆后逐个输出、读入，读入值小于刚输出的值就冻结到下一顺串。得到三个顺串 `2,5,8,10,11,15,20,26`、`4,7,13,14,16,21`、`6`。第一个顺串长 8 > 5，这就是置换选择「顺串比内存长」的效果；输入结束时堆里剩下的活跃元素属于当前顺串，冻结的元素另成顺串。
5. **[`ref_DSA/`第九章作业`.pdf` 第 3(1) 题；2025 秋书面作业；脚本复算]** 顺串长 28,2,7,9,22,13,12,19 的二路最佳归并树就是以长度为权的 Huffman 树：依次合并 2+7=9、9+9=18、12+13=25、18+19=37、22+25=47、28+37=65、47+65=112，**总读写记录数**（WPL）= 313。画树时按题意把较短的顺串放左边。
6. **[`ref_DSA/`第九章作业`.pdf` 第 3(2) 题；2025 秋书面作业；脚本复算]** 4 个顺串 `(3,4,9)`、`(1,10,20)`、`(5,12,15)`、`(7,8,11)`，初始堆（数组）`1,3,5,7`。每次取堆顶输出，把它所属顺串的下一个元素放到堆顶再下沉：第 1 次输出 1、放入 10 → `3,7,5,10`；第 2 次输出 3、放入 4 → `4,7,5,10`；第 3 次输出 4、放入 9 → `5,7,9,10`；第 4 次输出 5、放入 12 → `7,10,9,12`；第 5 次输出 7、放入 8 → `8,10,9,12`。（数组按层序读，第 1 个是根。）
7. **[`ref_DSA/`第九章作业`.pdf` 第 3(3) 题]** 两者每次都是 $O(\log N)$，差在常数：堆的下沉每层要比两次（两个孩子先比、再与父比），败者树只沿叶到根的路径每层与存着的败者比**一次**，且新元素替换的正是冠军所在的叶，路径固定、不必挑孩子。N 路归并每输出一个记录都要调整一次，比较次数约省一半，所以外排序多用败者树。

课程上机参考

给定固定内存缓冲区，输出顺串并验证每个顺串内部有序、全部记录无丢失；再用多路归并验证最终文件全局有序。

第 10 章 检索

补充题参考答案

1. **[课程第十章答案第 1 题；`ref_DSA/`第十章书面作业`.pdf` 第 1 题]** 从当前指针出发按关键码方向前后移动；题目注明重复时要检索到**最后一个**，所以向后走时遇到相等不能立即停，要继续走到下一个结点不等于 K 为止，向前走时第一次遇到的相等结点就是最后一个。**失败时不能改动** `p`——`p` 的语义是「最后一次成功检索到的结点」，失败后把它改成空或停在半路，下一次检索就失去了起点。平均检索长度取起点和目标位置的双重平均：设无重复元素、起点 i 与目标 j 各自等概率，比较次数为 $|i-j|+1$，

$$ASL = \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}(|i-j|+1) = \frac{n^2-1}{3n}+1 \approx \frac{n}{3}.$$

（$\sum_{i,j}|i-j| = \frac{n(n^2-1)}{3}$。$n = 1,2,3,5,10$ 暴力枚举分别得 $1, \frac{3}{2}, \frac{17}{9}, \frac{13}{5}, \frac{43}{10}$，与公式一致。）只算 $\frac{n^2-1}{3n}$ 是漏了「起点本身也要比较一次」。
2. **[ref_DSA/DSA-HW/Chapter10.md 第 2 题]** 总复杂度是一次堆排序 `O(M log M)` 加上 `|S1|` 次二分检索 `O(log M)`，即 `O(M log M + |S1| log M)`。
3. **[张铭教材；本书代码验证]** 拉链法装填因子可大于 1，成功查找期望约 `1+alpha/2`；线性探测要求 `alpha<1`，接近 1 时代价急剧上升。

正文习题答案

1. 顺序检索最好比较 1 次，最坏比较 n 次；目标等概率且一定存在时，平均成功检索长度为 $(1+2+\cdots+n)/n = (n+1)/2$。若把失败也计入，失败一次要比较 n 次，需另给成功概率。
2. 半开区间初为 `[0,8)`，`mid=4`，值 12 小于 18，改为 `[5,8)`；`mid=6`，值 18，命中 **下标** 6，共 2 次关键码比较。
3. 若索引表也顺序检索，分 b 块、每块约 n/b 项，ASL 的主项约为 $b/2+n/(2b)$；令两项平衡得 $b \approx \sqrt{n}$，每块也约 $\sqrt{n}$ 项，总 ASL 为 $\Theta(\sqrt{n})$。
4. 集合插入重复元素是正常、可预期的集合操作，不是异常；返回 `bool` 可让 `true` 表示确实新增、`false` 表示原来已有，调用方无需用异常处理普通分支。本书 `IntSet::insert` 正是该契约。
5. 表长 5 时 1 入槽 1，6 碰撞后入槽 2，11 入槽 3。若删除 1 后把槽 1 标成空，从哈希位置 1 查 6 会见空即停，错误地报告不存在；必须留下墓碑。本书 `HashTable::erase` 使用 `SlotState::tombstone`。
6. 开散列删除链结点即可，装填因子可大于 1，但每项多指针/分配开销；闭散列删除要墓碑，装填因子必须小于 1，存储连续、缓存友好但会聚集。
7. `char` 是否有符号由实现决定；直接提升负 `char` 会符号扩展，使同一非 ASCII 字

节在不同平台产生不同哈希。按 `unsigned char` 读取保证每字节都落在 0..255，本书 `search::elf_hash` 因而可移植。

正文上机题参考

1. `search::sequential_search` 直接对原序列测，`search::binary_search` 必须先排序；散列用 `HashTable::contains`。只统计查询阶段，并分别报告成功/失败探测次数和装填因子。
2. 使用 `HashTable::insert`、`HashTable::erase`、`HashTable::contains`：在容量 5 表中插入 1、6、11，删 1 后仍应找到 6、11；`HashTable::slot_at(1)` 应显示墓碑而不是空槽。
3. 当 `size/capacity>0.7` 时申请更大素数容量，把所有 `used` 槽重新 `insert`，墓碑不搬。用平均/最大探测长度比较扩表前后，不能只看运行时间。
4. `IntSet` 的公开契约是 `IntSet::insert`、`IntSet::erase`、`IntSet::contains`、`IntSet::intersection`、`IntSet::includes`。散列表替换内部存储后保持这些签名和返回语义，原测试无需知道实现变化。

课程作业与 `ref_DSA` 参考答案

1. **[张铭教材；**`ref_DSA/DSA-HW/Chapter10.md`**]** 顺序检索无序表为 `O(n)`；二分检索要求有序且为 `O(log n)`。若先排序再做多次查询，应把一次排序成本和查询次数一起计算。
2. **[张铭教材；本书代码验证]** 开散列把冲突元素挂在桶链表中；闭散列在表内探测，删除必须使用墓碑，否则会截断后续探测链。
3. **[张铭教材；本书代码验证]** 装填因子越高，冲突越多；开放寻址必须小于 1，实践中通常在达到阈值时扩表。
4. [`ref_DSA/`**第十章书面作业**`.pdf` **第 3 题；脚本复算]** 关键字 `(9,37,21,20,3,14,5)` 存入 `HT[10]`，$h(key) = (3key) \bmod 7$，按给定次序插入。(1) 线性探查：9 → 6；37 → 6 冲突→ 7；21 → 0；20 → 4；3 → 2；14 → 0 冲突→ 1；5 → 1、2 冲突→ 3。表为 `[0]21 [1]14 [2]3 [3]5 [4]20 [6]9 [7]37`，各键探查次数 1,2,1,1,1,2,3, ASL = 11/7 ≈ 1.57。(2) 双散列 $rh(key) = (3key) \bmod 5 + 1$：37 的 $rh = 2$，6 → 8；14 的 $rh = 3$，0 → 3；5 的 $h = 1$ 直接命中。表为 `[0]21 [1]5 [2]3 [3]14 [4]20 [6]9 [8]37`，探查次数 1,2,1,1,1,2,1，ASL = 9/7 ≈ 1.29。散列值只落在 0..6，表长 10 里 5、7、8、9 号槽只能靠探查到达——这是散列函数模数小于表长带来的浪费，双散列用 rh 跳出了 0..6。

课程上机参考

比较线性检索、二分检索、拉链法和线性探测，测试重复键、删除后再查找、全表墓碑、容量为 0 和负键。

第 11 章 索引技术

2021 书面作业补充题

1. **[2021 书面作业答案；PKU DSA 笔记第 11 章]** 磁盘块为 1024 字节、每个索引项为 8 字节、数据文件有 25000 条记录时，二级索引的一级和二级索引共需 198 块：`ceil(25000 / 128) + ceil(25000 / 128^2) = 196 + 2`。
2. **[2021 书面作业答案；PKU DSA 笔记第 11 章；本书校正]** 10000 个关键码的 16 阶 B 树，根到叶的索引块访问次数**不会小于** 4：每个结点至多 15 个关键码、16 个孩子，h 层的 16 阶 B 树至多容纳 16^h-1 个关键码，而 $16^3-1=4095<10000\le 16^4-1=65535$，3 层装不下，至少要 4 层。（本附录旧版写作「不会小于 3」，与它自己给的不等式矛盾。）
3. **[PKU DSA 笔记第 11 章；本书概念校正]** B+ 树的内部关键码是子树边界的副本，叶结点保存完整索引并按链连接；分裂后必须维护父结点的边界关键码。

补充题参考答案

1. **[课程第十一章答案第 1 题]** 先算每页能放的记录/索引项，再算内存能驻留的索引项；二级索引要把读根、读索引页和读数据页分别计数。
2. **[课程第十一章答案第 2 题]** 院系和宿舍分别建 `属性 -> ID 列表`，查询“且”时求两个有序列表交集；复杂度为两列表长度之和的数量级。
3. **[课程第十一章答案第 3 题]** B+ 树查找逐层读页；插入溢出时分裂并写原页、新页和父页；删除用后继替换并按是否下溢统计读写页。

正文习题答案

1. 稠密索引每条记录一项，主文件可无序（但索引本身按键有序）；稀疏索引每页只取分界键，要求主文件按关键码有序并按该顺序分页，否则无法由分界定位目标页。
2. 若索引有 h 个不常驻层，再读 1 个数据页，一次点查询约为 $h+1$ 次页 I/O；顶层若常驻内存不计。页访问比页内比较重要，因为一次磁盘/SSD 页访问的代价远大于内存比较。
3. 静态多分树没有为在线分裂维护的动态结构；叶页溢出通常写入溢出页/溢出链，查询多一次 I/O，积累到阈值后重组主文件和索引。不能把 B+ 树的就地分裂直接套过来。
4. 分别取倒排表 `postings[计算机系]` 与 `postings[擅长英语]`，用双指针求有序交集：相等就输出并都前进，小的一侧前进；时间 $O(a+b)$。具体 ID 取决于题给院系表和宿舍表。
5. 插入 60 后的树为 `[50] / [30] [70] / [10,20] [30] [50,60] [70,90]`。继续插入 65 后为 `[50] / [30] [60,70] / [10,20] [30] [50] [60,65] [70,90]`；只发生叶分裂，父结点 `[70]` 尚有容量，更新为 `[60,70]`，所以没有继续裂到根。
6. 位图适合低基数属性、批量布尔查询，内存/外存皆可且擅长集合运算；签名适合先过滤、允许假阳性但不能假阴性；红黑树适合内存点查和动态更新；B+

树按页组织，适合外存点查与范围扫描。

正文上机题参考

本章正文没有独立编号的上机题，而是指向末尾四条 “练习路径”。对应做法是：把 `MultiLevelIndex` 页内定位改成二分并确认 `MultiLevelIndex::page_reads()` 不变；给 `InvertedIndex` 增加差值编码且能边解码边求交；给 `BPlusTree` 增加范围删除并在每步后调用 `BPlusTree::validate()`；给 `BitmapIndex` 增加 WAH 编解码，与现有 `index::run_length_encode`、`index::run_length_decode` 对拍并正确处理尾位。

课程作业与 `ref_DSA` 参考答案

1. **[课程第十一章答案第 1 题]** 稠密索引为每条记录建索引项；稀疏索引按数据块建项，空间小但块内还需二次查找。容量题应先算每页记录数、每页索引项数，再乘可驻留页数。
2. **[课程第十一章答案第 3 题；张铭教材]** B+ 树的内部结点只存分隔键，叶结点存记录位置并按链相连，范围查询比普通 B 树更方便；插入删除的访外次数应分别计读页和写页。
3. **[课程第十一章答案第 2 题]** 倒排索引把属性值映射到记录 ID 列表；与查询是有序 posting list 的交集，或查询是并集，短语查询还需比较词项位置。

课程上机参考

用有序数组实现静态稀疏索引和倒排表交集；明确模拟页访问次数，而不是只报告内存比较次数。

本章四题都有可对照的实现：`code/ch11/linear_index` 的 `page_reads()` 直接给出第 1 题要数的访外次数；`code/ch11/inverted_index` 给出第 3 题的交并差与短语查询；`code/ch11/bplus_tree` 的 `page_reads()`/`page_writes()` 分开计读页与写页，正好对应第 2 题和补充题第 3 题；`code/ch11/bitmap_index` 覆盖位图与签名。先自己算一遍再跑一遍对答案，比只看结论有用。

第 12 章 高级数据结构

2021 书面作业补充题

1. **[2021 书面作业答案；PKU DSA 笔记第 12 章]** 广义表的纯表对应树，再入表可能共享结点并形成 DAG，循环表含环且递归深度为无穷；释放共享结构不能简单地对每条边递归 `delete`。
2. **[PKU DSA 笔记第 12 章；本书代码验证]** CSR 用 `row_ptr`、`col_idx`、`values` 三个数组表示稀疏矩阵；计算 `y=A*x` 只遍历非零元，时间为 `O(nnz)`。
3. **[2021 书面作业答案；本书代码验证]** AVL 插入失衡分为 LL、RR、LR、RL；旋转保持中序序列不变，操作复杂度为 `O(log n)`。

补充题参考答案

1. **[**`ref_DSA/homework/12` **高级数据结构**`.md` **第 2 题]** CSR 用 `row_ptr`、`col_idx`、`values` 三数组；`y=A x` 逐行遍历非零元，时间 `O(nnz)`，空间 `O(nnz+V)`。
2. **[课程第十二章答案第 1 题；本书代码验证]** 共享表释放时可能从多个父表到达同一结点，循环表还会无限递归；可用引用计数，或用访问标记集合保证每个结点只释放一次。`code/ch12/gen_list` 实现的正是引用计数那条路；把它退化成「不看计数直接 delete」，测试会立刻报 heap-use-after-free。引用计数收不回环，这也是原书 12.2.4 要讲标记-清除的原因。
3. **[课程第十二章答案第 2 题；**`ref_DSA/homework/12` **高级数据结构**`.md` **第 4 题；本书代码验证]** 每个后缀插入 Trie 后，其所有前缀对应主串子串；因此每个不同子串对应一个结点。朴素插入所有后缀最坏 `O(N^2)`，结点数也为 `O(N^2)`。可以用 `code/ch12/trie` 的 `node_count()` 直接数出来验证。
4. **[课程第十二章答案第 3 题]** LL/RR 单旋、LR/RL 双旋；旋转只改局部链接，中序遍历序列保持不变。

正文习题答案

1. 按本附录默认的 0 起始下标，行优先偏移为 $2 \times 4 + 1 = 9$ 个元素；若题目采用 1 起始下标，则同名 $a_{2,1}$ 的偏移是 $(2-1)4 + (1-1) = 4$，必须先声明约定。
2. 0 起始下三角按行压缩的下标是 $i(i+1)/2 + j$，所以 $a_{3,1}$ 在 **下标** $3 \times 4/2 + 1 = 7$。
3. 纯表每个子表只有一个父，对应树；再入表共享子表，对应 DAG；循环表含回边，对应有向环图。释放时纯树可递归，DAG 要引用计数/访问标记避免重复释放，环还需要标记–清扫等可回收环的方案。
4. 请求 650 就能把三者一次分开：首次适应从低地址扫，选 900；最佳适应挑刚够用的最小块，选 700；最坏适应挑当前最大块，选 1500。至于「能否满足」，单个请求分辨不出任何东西——≤ 1500 时三种策略都能从某块满足，> 1500 时三者都不能。策略只决定**挑哪一块**，差别要到后续请求上才显出来（这与 12.2.3 节「只在一块空闲区上测是分辨不出三种策略的」是同一条道理）。
5. Trie 为根下共用 `c -> a`，再分三条边 `n`、`r`、`t`，三端均标记单词结束。查 `cab` 时走过 `c,a`，在第三个字符 `b` 处无边而失败。
6. 用本书 `advanced::optimal_bst({1,5,4,3},{5,4,3,2,1})` 同口径复算，最优根为 **键** 2，总成本为 57。
7. 插 1 得单根；插 2 得 1 的右孩子；插 3 形成 RR 失衡，对 1 左旋，得到根 2、左右孩子 1 和 3。再插 0 到 1 的左侧，各结点平衡因子仍在 `[-1,1]`，**不旋转**。
8. 一字形（zig-zig）与之字形（zig-zag）都用两次旋转把目标从孙辈升到祖父位置，即升两层；区别是旋转方向同向还是反向。半伸展减少每次调整幅度，连续访问热点仍会逐步把它提到高层，同时降低单次重构常数。

正文上机题参考

1. 封装长度 $n(n+1)/2$ 的数组，`set/get(i,j)` 先检查 `0<=j<=i<n`，再用 `i(i+1)/2+j`。对所有合法位置与稠密矩阵对拍，并测试上三角和越界拒绝。
2. 使用实际类 `ReusableNodePool<T>` 的 `ReusableNodePool::acquire`、`ReusableNodePool::get`、`ReusableNodePool::release`、`ReusableNodePool::available`；测试容量耗尽返回空、释放后复用同一槽、越界释放和重复释放返回 `false`。
3. 使用 `Trie::insert`、`Trie::contains`、`Trie::erase`；每轮与 `std::set<std::string>` 对拍。输入只接受实现支持的字母范围，并覆盖空串、重复词、前缀词以及删除后保留共享前缀。
4. 调用 `advanced::optimal_bst(successful,unsuccessful)`，核对 `cost[0][n]` 与 `root[0][n]`；按根表递归重建区间，最后验证每个键恰出现一次、加权查找成本等于 DP 结果。

课程作业与 `ref_DSA` 参考答案

1. **[`ref_DSA/homework/12` 高级数据结构`.md` 第 2 题；本书自拟]** 下三角矩阵按行压缩时，`a[i][j]`（`i>=j`）下标为 `i(i+1)/2+j`；对称矩阵取较大下标在前。`ref_DSA` 进一步给出 CSR 稀疏矩阵乘向量，时间与非零元数成正比。
2. **[课程第十二章答案第 1 题；`ref_DSA/homework/12` 高级数据结构`.md` 第 1 题]** 纯表对应树，再入表对应可能共享的 DAG，循环表对应有环图；共享或有环结构不能简单递归释放。课程答案特别指出无括号的重复原子不应误判为子表。判定伪码（`GLNode` 的 `hp/tp` 表示）：纯表 ⇔ 从表头出发，**每个 `LIST` 结点恰好被到达一次**（没有共享，也就没有环）。设空集合 `visited`，从表头 `L` 开始走 `walk(L)`，对到达的结点 `p` 依次判断：① `p` 为空，或 `p.tag == ATOM`——返回真（原子可以重复出现：`(a,a)` 是纯表）；② `p` 已在 `visited` 中——返回假（第二次到达：共享子表或回边）；③ 否则把 `p` 加入 `visited`，返回 `walk(p.hp)` 与 `walk(p.tp)` 的合取。每个结点至多进 `visited` 一次，时间、空间 $O(n)$。两个常见错法：①「遇到原子就返回 false」——那样一切含原子的表都被判成非纯表；②「原子值出现两次就返回 false」——纯不纯看的是**结点有没有被共享**，不是**值有没有重复**。递归深度等于表的嵌套加表尾链长，很长的表要改成显式栈（`tp` 方向可以直接改成循环）。
3. **[张铭教材；本书代码验证]** 最优 BST 的区间动态规划代价为 `O(n^3)`，`root` 表可重建树形。
4. **[课程第十二章答案第 3 题]** AVL 四种失衡是 LL、RR、LR、RL；旋转只改变局部指针，不改变中序序列。
5. **[课程第十二章答案第 2 题；`ref_DSA/homework/12` 高级数据结构`.md` 第 4 题]** Trie 的查找复杂度与关键码长度成正比；把字符串全部后缀插入 Trie 后，结点可对应不同子串。Patricia 压缩单孩子路径以减少结点。

课程上机参考

实现下三角矩阵、句柄池和 Trie；句柄池必须测试耗尽、释放、复用、越界释放和重复释放。Trie 应与 `std::set` 对拍插入和查找结果。

参考答案的使用边界与 OJ 迁移

OJ 题型迁移与校正

参考资料目录 `ref_DSA/OJ` 中适合纳入本书上机训练的题型，统一由本书现代代码承接；参考目录本身保持只读。

章节	纳入本书的 OJ 题型	现代参考实现	迁移时的校正
第 2 章	有序表去重、链表判环、链表删除	`code/ch02/array_list`、`linked_list`、`doubly_linked_list`	明确空表、单结点和环入口边界
第 3 章	滑动窗口、双栈队列、中缀/后缀表达式、出栈序列判定	`code/ch03/queue`、`expression_eval`、`array_stack`（`dsa::stack_operations_valid`）	除法先检查右操作数；队列只在输出栈为空时搬运；出栈序列用顺序栈模拟，n=1..7 穷举合法数等于 Catalan 数
第 4 章	KMP、最小循环节（循环串 / 前缀中的周期 / 字符串乘方）、字符串消除、循环移动	`code/ch04/pattern_matching`（`dsa::border_lengths`、`dsa::minimal_period`、`dsa::is_repetition`、`dsa::repetition_count`、`string_class`）	匹配成功返回 0 起始下标，不能使用 `j-pLen+1`；**求周期必须用未优化的** border，**不能用优化版** next，且 border 为 0 时 `n % n == 0` 不算循环串——两条都有测试守着；字符串消除与循环移动只有答案思路，没有代码

章节	纳入本书的 OJ 题型	现代参考实现	迁移时的校正
第 5 章	中序 + 后序 / 先序 + 中序重建、堆与 Huffman、动态中位数、权值成批的 Huffman	`code/ch05/binary_tree`（`BinaryTree::from_inorder_postorder`、`BinaryTree::from_preorder_inorder`）、`heap_huffman`（`RunningMedian`、`dsa::huffman_wpl_batched`）	重复键、键缺失、序列不自洽一律抛 `std::invalid_argument`，不悄悄建出一棵错树；重建不递归（区间帧压进 `LinkedStack`），20 万深的左链有测试守着；单叶 Huffman 单独处理；成批 Huffman 只求 WPL（= 内部结点权之和），64 位累加且溢出抛异常
第 6 章	森林转换、并查集、连通分量与连通点对计数（动态图连通性）	`code/ch06/general_tree`（`ComponentCounter::connect`、`ComponentCounter::components`、`ComponentCounter::connected_pairs`）	路径压缩与重复合并都要测试；新增点对数 $s_1 s_2$ **必须在合并之前从两个根上读规模**——读非根元素的规模是常见错误，测试专门守着；点对总数用 64 位
第 7 章	Dijkstra（含输出路径）、Floyd、Prim、Kruskal、拓扑排序	`code/ch07/graph`（路径：`Graph::dijkstra_tree`、`Graph::shortest_path`、`adjacency_list`	Dijkstra 生成最短路树，不等同于 MST；负边不能统一加常数修复；前驱只在松弛成功时更新，到不了返回空值；最小完全图总权（Kruskal 逆用）见 7.4 节 Kruskal 后的附注

章节	纳入本书的 OJ 题型	现代参考实现	迁移时的校正
第 8 章	逆序对、距离排序、电话号码、基数排序	`code/ch08/sorting`（逆序对：`sorting::count_inversions`，归并计数）	对拍随机/顺序/逆序/全相等/负数输入，并保留稳定性要求；逆序对结果上界 $n(n-1)/2$，**必须 64 位**——$n = 70000$ 逆序输入的 2449965000 已超 `INT_MAX`，测试用它和 $n = 100000$ 锁住；相等元素不算逆序。距离排序、电话号码两题没有专门代码

这些题型的完整实现以 `code/` 下通过编译和测试的文件为准；OJ 原始代码只作为题意和易错点参考，不直接作为书中代码。

这些答案用于核对思路和复杂度，不是唯一写法。需要提交代码的题目仍应经过编译器、边界测试和必要的 sanitizer 检查；本书正文中已经提供实现的题目，应以对应 `code/*/test.cpp` 的可执行行为为最终依据。

来源文件登记

本附录引用的课程答案来自：

- `ref_ 数据结构与算法 A 2021 秋/2021 书面作业答案 1-12 章/` 内各章书面作业答案；
- `Data-Structure-and-Algorithm-A-of-pku` 的 `notes-理论-期中后/` 中第 7–12 章复习笔记（仅吸收可独立核验的题型和结论）；
- 同目录下的《教材 1-6 章勘误表.pdf》。

2026-09-14 另用 `Data-Structures-and-Algorithms-A-2025Fall-PKU`（2025 秋同一门课的学生仓库：书面作业作答、OJ 与往年上机考试代码）做了一次**交叉核对**，题目中标「2025 秋书面作业」的即据此核对过。该仓库只有作答、没有题面，所以题面仍以 `ref_DSA/` 里的原件为准；它的作答里有若干错误（如建堆结果 503/512 对调、1 起始编号用了 0 起始的父公式、纯表判定遇原子即判否），本附录的数值一律用脚本独立复算，不以它为据。往年上机考试里反复出现的题型

（逆序对、KMP 求循环节、并查集连通计数、Dijkstra 输出路径、树的重建、出栈序列）已补进对应章节的 `code/` 单元，见上表。

`ref_DSA` 引用均在题目标记中写出相对路径。追踪状态汇总见 `collab/TRACEABILITY-MATRIX.md`。课程 PDF/Word 中依赖图片才能确定具体数值的题目，本附录只引用 可由文本独立核验的结论；没有看清题图时不猜最终序列或树形。

第 9 章 外部排序（原书题补充答案）

原书习题答案

1. 每块 $1024/32 = 32$ 条记录，1 MB 工作内存可放 $2^{20}/32 = 32768$ 条记录（1024 块）。第一遍用置换选择生成初始顺串，平均长度约为内存的 2 倍，即 65536 条记录（2 MB）。第二遍做一次多路归并：每一路至少要占一个块的输入缓冲，1 MB 内存约可同时归并 1024 路（实际要扣掉输出缓冲，略少于 1024）。于是只扫两遍能排的文件最长约为 1024×2 MB $= 2$ GB，即约 6.7×10^7 条记录。一般地，内存 M、块长 B 时两遍可排约 $2M \cdot (M/B)$ 的数据。
2. 排序 100 倍的记录，工作量至少是原来的 100 倍以上（外排序的读写量还随归并趟数增加），而外排序的瓶颈是磁盘 I/O：(1) CPU 速度提高：几乎不影响 I/O 时间，**不能满足**；(2) 磁盘 I/O 时间减半：总时间约减半，远不够 100 倍，**不能满足**；(3) 内存存取时间减半：对外排序影响很小，**不能满足**；(4) 内存增大：顺串变长（约为内存的 2 倍）、归并路数增多，归并趟数可能减少，是四者中最有效的，但单靠它也做不到 100 倍的提速，**同样不能单独满足**。结论：四项单独升级都不够，需要组合（例如更大内存配合更快的磁盘或多盘并行）。
3. 三趟需 `k^3>=80`，最小 k=5；15 个文件含输出故 k<=14，`14^2>=80`，两趟取 k=9 即可。
4. 按顺串编号 1～8，首记录为 14、22、24、15、16、11、100、18。两两比赛：(14,22) 败者 22，(24,15) 败者 24，(16,11) 败者 16，(100,18) 败者 100；第二轮 (14,15) 败者 15，(11,18) 败者 18；决赛 (14,11) 败者 14，冠军 11（第 6 路）。败者树数组 $L[0..7]$（$L[0]$ 存冠军，$L[1]$ 是根，其余按层序）为：11(6)、14(1)、15(4)、18(8)、22(2)、24(3)、16(5)、100(7)（括号内为顺串号）。输出 11 后第 6 路补进 28，只沿第 6 路的叶到根重赛：与 $L[6]$ 的 16 比，16 胜、28 留下；16 与 $L[3]$ 的 18 比，16 胜、18 留下；16 与 $L[1]$ 的 14 比，14 胜、16 留下。重构后为 14(1)、16(5)、15(4)、18(8)、22(2)、24(3)、28(6)、100(7)，下一个全局赢者是 14。（脚本复算。）
5. 先按给定 `Play` 建赢者树，再把每内部结点的败者写入对应位置、把总赢家单存；替换后仍只重赛一条根路径，`O(log k)`。
6. 用前 8 个记录 14、22、7、24、15、16、11、100 建败者树：冠军 7（第 3 个叶），败者树数组为 7、11、14、15、22、24、16、100（$L[0]$ 存冠军，其余同第 4 题的层序约定：根，两个第二轮败者，四个第一轮败者）。此后每输出

当前冠军就读入下一个记录：若它小于刚输出的值，就标记为下一顺串（比赛时下一顺串的记录一律输给本顺串的记录）。题给的 25 个记录产生：**第一个顺串** `7 10 11 14 15 16 17 20 22 24 26 30 38 50 90 100 110`（17 个，是内存容量 8 的两倍多）；输入读完时，被冻结的 `9 12 13 18 21 25 28 40` 构成第二个顺串的开头——题面末尾有「…」，后续记录还会继续接进第二个顺串。（脚本模拟置换选择；败者树与堆只是选最小的工具不同，产生的顺串相同。）### 原书上机题参考

7. 8 位像素读量翻倍但无需拆半字节；4 位格式读量减半却增加位操作。顺序扫描两版并计时，结论取决于 I/O 带宽与 CPU，不能先验断言。
8. 置换选择生成段、败者树 k 路归并；大记录优先排序键和记录号，再按索引重排以减少搬运。测试记录守恒、有序、稳定需求和临界内存。

第 10 章 检索（原书题补充答案）

原书习题答案

1. $ASL=\sum_i i\,p_i=\sum_{i=1}^{n-1}\frac{i}{2^i}+\frac{n}{2^n}=2-\frac{n+2}{2^n}$（用 $\sum_{i=1}^{m} i/2^i=2-(m+2)/2^m$ 化简；$n=3$ 时为 $11/8$，脚本核对）。**但要注意题面的分布本身不完整**：$\sum p_i=1-1/2^n\neq 1$（扫描件确实印作 $p_n=1/2^n$，属原书题面错误）。若把最后一项改成 $p_n=1/2^{n-1}$ 使概率之和为 1，则 $ASL=\sum_{i=1}^{n-1} i/2^i+n/2^{n-1}=2-1/2^{n-1}$，$n$ 增大时趋于 2——无论表多长，平均只比较约 2 次，这正是题注说的「按访问频率排序」的效果。
2. 按原书的闭区间写法（`first`、`last` 为两端下标，`mid=(first+last)/2` 向下取整），检索 88 的三步为：① first=8、mid=26、last=97，$26<88$，first 移到 mid 右边；② first=44、mid=56、last=97，$56<88$，再右移；③ first=88、mid=88、last=97，命中，共 3 次比较。若用本书的半开区间 `[0,8)`：mid=4（44）→ `[5,8)`、mid=6（88）命中，只要 2 次——两种写法 mid 的取法不同，中间步骤也就不同，答题时要先说明用的是哪种。（脚本复算。）
3. 从上次命中 p 向 K 所在方向走；成功检索时起点与目标各自等概率，ASL $=\frac{n^2-1}{3n}+1\approx n/3$（推导见本章「补充题参考答案」第 1 条）；失败 ASL 还要把边界失败计入，失败时不修改 `p`。
4. 顺序查找：$ASL=(1+2+\cdots+25)/25=13$。二分查找：判定树的第 1～4 层分别有 1、2、4、8 个结点，剩下 10 个在第 5 层，$ASL=(1\cdot 1+2\cdot 2+3\cdot 4+4\cdot 8+5\cdot 10)/25=99/25=3.96$（脚本按 `mid=(low+high)/2` 建判定树复算）。分块查找（5 块、每块 5 个）：若索引表和块内都顺序查找，$ASL=(5+1)/2+(5+1)/2=6$；若索引表用二分查找，5 个索引项的平均比较为 $(1+2\cdot 2+3\cdot 2)/5=2.2$，$ASL=2.2+3=5.2$。
5. 固定随机种子，顺序查原表、二分查排序副本，报告成功率与比较次数；复杂度图为 O(n)、O(log n)，排序成本另列。
6. 两和双循环 O(n^2)，排序后双指针 O(n log n)；三和固定一个数加两和 O(n^2)；四和列两两和排序并二分/双指针 O(n^2 log n)。

7. 用 bitset 表示全集子集，成员、并交差为逐位操作，枚举扫描置位位；空间 O(n)。
8. 利用 $m = O(\log n)$：**对小集合 S_2 排序**，代价 $O(m \log m)$；再对 S_1 的每个元素在 S_2 中二分查找，代价 $O(n \log m)$。总代价 $O(n \log m) = O(n \log \log n)$，比「排序大集合再查」的 $O(n \log n)$ 好。也可以两者都排序后双指针归并，但那要 $O(n \log n)$，没用上 m 很小这个条件。
9. 0/1 背包以集合/位集记录可达重量或 DP 表 `dp[i][w]`，选恰 m 件加一维计数，转移取最大利润。
10. 按原书给的七段编码（$S[1] = \{b, c\}$、$S[6] = \{a, c..g\}$、$S[7] = \{a..c\}$、$S[9] = \{a..d, f, g\}$ 等），两数字之间有边当且仅当一个的段集是另一个的真子集；对这张无向图用 DFS 回溯枚举经过全部 10 个顶点的哈密顿路径，把每条路径与它的逆置视为同一个解。共 6 **个**本质不同的排列：`0731495682`、`2807314956`、`2837014956`、`2841073956`、`2865937014`、`2865941073`（逆置后各算同一个；题面的例子 2865937014 在其中）。若把数字 1 取成 $\{e, f\}$，图会不同，答案也随之不同，正如题面提示所说。（脚本枚举。）
11. 除留余数法取 $p = 23$，$h(k) = k \bmod 23$，线性探查。各元素的散列地址（探查次数）：32 → 9(1)、75 → 6(1)、29 → 6 冲突→ 7(2)、63 → 17(1)、48 → 2(1)、94 → 2 冲突→ 3(2)、25 → 2,3 冲突→ 4(3)、46 → 0(1)、18 → 18(1)、70 → 1(1)。散列表：`[0]46 [1]70 [2]48 [3]94 [4]25 [6]75 [7]29 [9]32 [17]63 [18]18`，其余为空。成功查找 $ASL = (1+1+2+1+1+2+3+1+1+1)/10 = 14/10 = 1.4$。（脚本复算。）
12. 冲突次数按「基地址已被占用的关键码个数」计，冲突用线性探查解决。(1) $h(k) = k \bmod 19$，基地址依次为 10、5、13、7、1、12、3、10、10、1、0：29、200 与 10 冲突，400 与 58 冲突，**冲突 3 次**；表为 `[0]0 [1]58 [2]400 [3]3 [5]100 [7]45 [10]10 [11]29 [12]126 [13]32 [14]200`。(2) 各位数字之和 mod 19，基地址依次为 1、1、5、9、13、9、3、11、2、4、0：100（与 10 同址）、126（与 45 同址）冲突，200、400 的基地址又被前面探查过来的元素占了，**冲突 4 次**；表为 `[0]0 [1]10 [2]100 [3]3 [4]200 [5]32 [6]400 [9]45 [10]126 [11]29 [13]58`。(3) 关键码事先已知，可以专门凑一个无冲突的函数，例如 $h(k) = (k + \lfloor k/11 \rfloor) \bmod 19$，11 个关键码依次得 10、14、15、11、6、4、3、12、9、18、0，互不相同。(4) 表长 17：$k \bmod 17$ 的基地址为 10、15、15、11、7、7、3、12、13、9、0，只有 32、126 冲突，2 **次**；数字和 mod 17 与 mod 19 完全一样（数字和最大 13，不足 17），仍是 4 **次**；无冲突函数可取 $h(k) = (k + \lfloor k/8 \rfloor) \bmod 17$（得 11、10、2、16、14、5、3、15、4、8、0）。（脚本复算。）
13. $h(K) = K \bmod 13$。(1) 线性探查：12 → 12；23 → 10；45 → 6；57 → 5；20 → 7；03 → 3；78 → 0；31 → 5,6,7 冲突→ 8（4 次）；15 → 2；36 → 10 冲突→ 11（2 次）。表为 `[0]78 [2]15 [3]03 [5]57 [6]45 [7]20 [8]31 [10]23 [11]36 [12]12`，1、4、9 号槽空。成功 $ASL = (1 \times 8 + 4 + 2)/10 = 14/10 = 1.4$。不成功检索从每个基址 $0..12$ 出发探到空槽为止（空槽那一次也计数），次

数依次为 2、1、3、2、1、5、4、3、2、1、5、4、3，$ASL = 36/13 \approx 2.77$。(2) 双散列 $rh(K) = (7K) \bmod 11+1$：前 7 个关键码都直接落在基地址；31 的基地址 5 冲突，$rh(31) = 217 \bmod 11+1 = 9$，下一址 $(5+9) \bmod 13 = 1$ 空，放入（2 次）；15 落在 2；36 的基地址 10 冲突，$rh(36) = 252 \bmod 11+1 = 11$，下一址 $(10 + 11) \bmod 13 = 8$ 空，放入（2 次）。最终表为 `[0]78 [1]31 [2]15 [3]03 [5]57 [6]45 [7]20 [8]36 [10]23 [12]12`，探查次数依次为 1、1、1、1、1、1、1、2、1、2，成功 $ASL = 12/10 = 1.2$。（脚本按题给公式复算。）

14. (1) 位移序列 2、4、6、1、3、5 两两不同，正好取遍 1～6，所以从任一基地址出发，6 次探查就能访问到长度 7 的表里其余全部 6 个槽，只要表没满就一定能插入。(2) **不能**。这个排列其实就是 $2i \bmod 7$（$i = 1..6$），即「步长为 2 的线性探查」：基地址为 d 的探查序列是 $d+2, d+4, \ldots$，基地址为 $d+2$ 的序列与它从第二步起完全重合，相邻（相隔 2）的关键码仍会连成一片，基本聚集照样发生。伪随机探查要消除基本聚集，排列本身必须「不规则」。(3) 选 $1 \sim M-1$ 的一个排列，是为了让探查序列不重复、不回到基地址（位移 0）、且能走遍除基地址外的全部 $M-1$ 个槽——表不满就一定找得到空位；排列要「随机」，才能让不同基地址的探查路径不重合，从而避免基本聚集（但同一基地址的关键码仍走同一条路，二级聚集还在）。
15. (1) 3 → 3；12 → 5；9 → 2；2 → 2 冲突、3 冲突→ 4。表为 `[2]9 [3]3 [4]2 [5]12`，0、1、6 号槽空。(2) 设下一个关键码的基地址在 0～6 上等可能：基地址 0 落到槽 0；基地址 1 落到槽 1；基地址 2、3、4、5 都沿着聚集一路探到槽 6；基地址 6 也落到槽 6。所以槽 0、1 被填的概率各为 1/7，槽 6 为 5/7——这就是基本聚集让聚集块右边那个空槽「越来越容易被填」的定量表现。
16. (1) $\alpha = 0.6$，12 个关键码需要表长 $\geq 12/0.6 = 20$，取不小于 20 的素数 $M = 23$，$h(K) = K \bmod 23$。(2) 第二个散列函数按 10.3.3 节（双散列探查法）的做法取 $h_2(K) = K \bmod (M-2)+1 = K \bmod 21+1$（取值 1～21，与 23 互素）。依次插入：26 → 3、25 → 2、20 → 20、33 → 10、21 → 21、24 → 1、45 → 22；204 的基地址 20 已被 20 占用，$h_2(204) = 204 \bmod 21+1 = 16$，下一址 $(20+16) \bmod 23 = 13$ 空，放入；42 → 19、38 → 15、29 → 6、31 → 8。散列表为 `[1]24 [2]25 [3]26 [6]29 [8]31 [10]33 [13]204 [15]38 [19]42 [20]20 [21]21 [22]45`，**碰撞 1 次**（发生在 204 上）。第二个散列函数可以另选（例如 $K \bmod 19+1$），选法不同，探查到的位置就可能不同，答题时要写明所用的 h_2。（脚本复算。）
17. k/n 可能越界且大量冲突；常数 1 可用但很差；含随机数使同键无法重现地址，不能用；模素数可用且通常好。
18. 把字母映为 0～25（A=0），按 26 进制展开：$h(s) = (26^2 c_0 + 26 c_1 + c_2) \bmod n$，三个字母的位置都参与运算。各串的散列值（依次为 PAL LAP PAM MAP PAT PET SET SAT TAT BAT）：$n = 11$：9 4 10 9 6 0 4 10 4 2；

$n=13$：11 2 12 2 6 6 6 6 6 6；$n=17$：2 5 3 1 10 12 0 15 11 15；$n=19$：5 3 6 14 13 3 17 8 0 11。冲突个数依次为 4、6、1、1。$n=13$ 时一塌糊涂，原因是 $26=2\times13$，$26^2c_0+26c_1$ 是 13 的倍数，前两个字母完全不起作用，只剩末字母决定地址——**模数不能与进制有公因子**；$n=17$、19 时冲突最少。（脚本复算。）

19. 按本书 `search::elf_hash`（逐字节 `h=(h<<4)+c`，高 4 位非零时折回并清零）计算，不取模的结果为：`HELLO WORLD` → 18409492（`0x118E814`）；`WORLD, HELLO!` → 143595425（`0x88F17A1`）；`Data Structure` → 18473621（`0x119E295`）；`Data Structure and Algorithms` → 215846003（`0xCDD8C73`）。原书的 `ELFhash` 最后还要 `% M`，结果再对表长取模即可。前两个串字符完全相同、只是次序不同，散列值相差很大——这正是逐字符移位累加想要的效果。（脚本复算，与本书 C++ 实现同一算法。）### 原书上机题参考
20. 符号表用 hash(name) 到栈式定义链，进入块压栈、离开块按本块清单弹出；查最新定义 O(1) 平均，线性表为 O(n)。
21. 建倒排索引 `term -> sorted docID`；与/或/差分别归并交/并/差，支持词典和查询解析。
22. 27 桶各 8 槽，基位 `K mod 197` 映射到桶内偏移，满桶入溢出区；删除用墓碑以不断探测链。
23. 员工以稳定 ID 主键，姓名索引支持查找；CRUD 后更新索引，排序输出副本，校验日期和电话。
24. hash(resource) 找 LO；S 与已有 S 兼容，X 仅在活动队列空时兼容；否则排等待队列。解锁后按公平策略唤醒兼容请求并回收空 LO。

第 11 章 索引（原书题补充答案）

原书习题答案

1. 每条记录 $4+64=68$ 字节，每块放 $\lfloor 4096/68\rfloor=60$ 条（记录不跨块）；线性索引每项 $4+4=8$ 字节。(1) 2 MB 的线性索引有 $2^{21}/8=262144$ 项，每项对应一个数据块，最多 $262144\times60=15728640$ 条记录。(2) 线性索引存在磁盘上，每个索引块放 $4096/8=512$ 项；二级索引 4096 字节含 1024 个关键码，每个对应一个线性索引块，于是可索引 $1024\times512=524288$ 个数据块，最多 $524288\times60=31457280$ 条记录。（脚本复算。）
2. 线性索引按关键码排序，每项是（关键码，记录在数据文件中的位置）。按文件中的存放次序，6 条记录依次为第 0～5 条；若每条记录占「2 字节关键码 + 字符串 + 1 字节结束符」，它们的字节偏移依次是 0、15、21、50、68、74。索引为：(42, 第 4 条/偏移 68)、(82, 第 1 条/偏移 15)、(397, 第 0 条/偏移 0)、(1037, 第 3 条/偏移 50)、(1038, 第 2 条/偏移 21)、(2222, 第 5 条/偏移 74)。要点是数据文件保持原来的插入次序、变长记录不必移动，有序的只是定长的索引项。
3. 5 个记录：A(39 张莉 男 程序员)、B(50 王明 女 分析员)、C(10 李想 男 程序

员)、D(75 丁悦 女 终端操作员)、E(27 赵驰 男 分析员)。(1) 顺序文件按职工号排序存放：10(C)、27(E)、39(A)、50(B)、75(D)。(2) 索引顺序文件的索引表（关键码 → 磁盘地址，按关键码有序）：10 → C、27 → E、39 → A、50 → B、75 → D；若主文件按块存放，稀疏索引只保留每块的最小（或最大）职工号。(3) 性别倒排表：男→ {10, 27, 39}，女→ {50, 75}；职业倒排表：程序员 → {10, 39}，分析员→ {27, 50}，终端操作员→ {75}（表中也可以直接记磁盘地址：男→ {C, E, A} 等）。

4. 题面所指的图与题意对不上（本书图 11.9 是 3 阶 B^+ 树，键为 5～90，没有 375），无法按原图作答。做法提示：插入 480 时沿根到叶找到所在叶，叶未满直接放入，满了就按中位数分裂、中位数上移，必要时连锁分裂到根；删除 375 时若它在内部结点，先用中序后继（或前驱）替换，再到叶上删除，叶下溢先向兄弟借、借不到就与兄弟及父结点的分界码合并，必要时连锁向上。练习时可以自拟一棵含 375 的 B 树，按第 9、10 题的步骤演算。

5. 5 阶 B 树每个结点至多 4 个关键码，非根结点至少 $\lceil 5/2 \rceil - 1 = 2$ 个。8 个关键码放不进一个结点，树至少两层；两层时根有 r 个关键码、$r+1$ 个孩子，孩子各有 2～4 个关键码，共 $8-r$ 个：$r=1$ 时两孩子分 7 个，只能是 3+4 或 4+3；$r=2$ 时三孩子分 6 个，只能 2+2+2；$r \ge 3$ 时孩子至少要 8 个关键码，不可能。因此共 3 **棵**：根 [4]、叶 [1 2 3] [5 6 7 8]；根 [5]、叶 [1 2 3 4] [6 7 8]；根 [3 6]、叶 [1 2] [4 5] [7 8]。（脚本枚举 8! 种插入次序，得到的也恰是这 3 棵。）

6. 17。下界：高度为 3（3 层）的 5 阶 B 树，根至少 1 个关键码、2 个孩子，第二层每个结点至少 2 个关键码、3 个孩子，第三层每个结点至少 2 个关键码，所以至少有 $1 + 2 \times 2 + 6 \times 2 = 17$ 个关键码，少于 17 次插入不可能长到 3 层。17 也能达到：脚本随机搜索找到的一个序列是 1, 14, 12, 6, 4, 10, 3, 11, 16, 7, 17, 13, 5, 8, 9, 2, 15，最后一次插入引起根分裂，树长到 3 层。

7. 设高度为 h（根在第 1 层，叶在第 h 层），一次访问读或写一个结点。最坏情况是被删关键码在叶上、且从叶一直到根下一层每层都下溢并合并：自根向下读路径 h 次；在第 $2..h$ 层的每一层，按题意左右两个兄弟都要读来检查，共 $2(h-1)$ 次；每层合并后写回合并结果 1 次，共 $h-1$ 次；最后写回（或释放后更换）根 1 次。合计最多约 $h + 2(h-1) + (h-1) + 1 = 4h - 2$ 次。与只检查一个兄弟相比，每层多读一次，换来更大的机会借到关键码、少做合并。不同教材对「写回哪些结点」的计数略有出入，答题时把读、写分开列出即可。

8. 插入 55：自根读 a(30)、c(40)、g(45 50) 三个结点；g 成为 (45 50 55) 上溢，分裂为 g(45) 与 g′(55)，中位数 50 上移到 c，c 变为 (40 50)，未上溢，结束。结果：a(30)；b(20)、c(40 50)；d(10 15)、e(25)、f(35 38)、g(45)、g′(55)。访外共 6 **次**：读 a、读 c、读 g（找插入位置）；写 g、写新结点 g′、写 c（保存分裂结果）。（脚本复算。）

9. 图 11.27 是 4 阶 B 树（每结点至多 3 个关键码）：根 (103)；第二层 (25 50)、(120 150 180)；叶 (11)(35)(81) 与 (108 110 115)(134)(160)(200)。插入 113 落

到叶 (108 110 115)，成为 (108 110 113 115) 上溢。(1) 取第 2 个中位数 113：叶分为 (108 110) 与 (115)，113 上移，父结点成为 (113 120 150 180) 再上溢，取第 2 个中位数 150：分为 (113 120) 与 (180)，150 上移到根。结果：根 (103 150)；第二层 (25 50)、(113 120)、(180)；叶依次 (11)(35)(81) | (108 110)(115)(134) | (160)(200)。(2) 取第 1 个中位数 110：叶分为 (108) 与 (113 115)，110 上移，父结点 (110 120 150 180) 上溢，取第 1 个中位数 120：分为 (110) 与 (150 180)，120 上移。结果：根 (103 120)；第二层 (25 50)、(110)、(150 180)；叶依次 (11)(35)(81) | (108)(113 115) | (134)(160)(200)。(3) 删除 50：它在内部结点，用中序后继 81 替换，父结点成为 (25 81)，原叶 (81) 变空下溢；兄弟 (35) 只有 1 个关键码借不出，与它及分界码 81 合并为 (35 81)，父结点成为 (25)。结果：根 (103)；第二层 (25)、(120 150 180)；叶 (11)(35 81) | (108 110 115)(134)(160)(200)（用前驱 35 替换，结果相同）。（插入部分脚本复算。）

10. 图 11.28 结点 c 印作「15 6」，按 3 阶 B 树的有序性和第 (2) 问「删除 156」，应是单个关键码 156。原树：a(100)；b(58 80)、c(156)；d(30 40)、e(60 70)、f(90)、g(123)、h(183)。(1) 插入 75：落到 e，成为 (60 70 75) 上溢，分裂为 (60)、(75)，70 上移；b 成为 (58 70 80) 再上溢，分裂为 (58)、(80)，70 上移；a 成为 (70 100)。结果：a(70 100)；b(58)、b′(80)、c(156)；b 下挂 d(30 40)、e(60)，b′ 下挂 e′(75)、f(90)，c 下挂 g(123)、h(183)。(2) 在原树中删除 156：用后继 183 替换，c 成为 (183)，叶 h 变空；兄弟 g(123) 借不出，g 与分界码合并为 (123 183)，c 变空下溢；c 的兄弟 b(58 80) 有 2 个关键码可以借：父结点的 100 下移到 c，b 的 80 上移到 a，b 的最右孩子 f(90) 过继给 c。结果：a(80)；b(58)、c(100)；b 下挂 d(30 40)、e(60 70)，c 下挂 f(90)、(123 183)。（插入部分脚本复算。）
11. 特性 (2)「除根外每个非叶结点至少有 $\lceil m/2 \rceil$ 个子结点」保证两件事：结点至少半满，空间利用率不低于约 50%；树高有对数上界 $h \le 1 + \log_{\lceil m/2 \rceil} \frac{n+1}{2}$。而且 $\lceil m/2 \rceil$ 恰好与「上溢结点一分为二」相容：有 $m+1$ 个孩子的结点分裂后两半各约 $(m+1)/2$ 个孩子，自然满足下限。改成 $\lceil m/3 \rceil$：仍是合法的平衡多路树，插入、删除照样做，但结点允许更空，空间利用率和树高的保证都变差。改成 $\lceil 2m/3 \rceil$：一个上溢结点平分后每半只有约 $m/2$ 个孩子，不满足下限，必须改用「先与兄弟匀、两个满结点分成三个」的策略，这就是 B^* 树，空间利用率提高到约 2/3。
12. 设内部结点中的关键码 K 位于指针 p_{i-1}、p_i 之间。K 的后继是右子树 p_i 中的最小关键码：从 p_i 出发一直沿最左指针下行，直到不能再下行的结点——由于 B 树所有叶在同一层、非叶结点每个指针都非空，「不能再下行」只能发生在叶结点，所以后继在叶上。前驱同理，从 p_{i-1} 一直沿最右指针下行到叶，取其最大关键码。
13. 每层访问一个结点，最多访问的结点数等于树的层数。含 n 个关键码的 m 阶 B 树，第 2 层至少 2 个结点，此后每层至少乘 $\lceil m/2 \rceil$，第 $h+1$ 层（失败结点所在层）至少 $2\lceil m/2 \rceil^{h-1}$ 个结点，而失败结点数为 $n+1$，于是 $n+1 \ge 2\lceil m/2 \rceil^{h-1}$，即最多访问 $h \le 1 + \log_{\lceil m/2 \rceil} \frac{n+1}{2}$ 个结点。

14. 对高度 h 归纳（根在第 1 层，叶在第 h 层）。$h=1$：只有根一个叶，$2^0=3^0=1$。设高度 $h-1$ 的 2-3 树叶数在 2^{h-2} 与 3^{h-2} 之间；高度 h 的树，根有 2 或 3 棵子树，每棵都是高度 $h-1$ 的 2-3 树（B 树所有叶在同一层），所以叶数至少 $2\cdot 2^{h-2}=2^{h-1}$、至多 $3\cdot 3^{h-2}=3^{h-1}$。等号分别在每个内部结点都恰有 2 个、3 个孩子时取到。

15. 页 2048 字节、关键码 4 字节、地址 8 字节、记录 128 字节，内存 12 MB 合 $12\times 2^{20}/2048=6144$ 页。(1) B 树结点要存 $m-1$ 个（关键码, 记录地址）对和 m 个孩子指针：$12(m-1)+8m\le 2048$，得 $m\le 103$，m **最大为** 103。(2) 4 层 103 阶 B 树最多 $103^4-1=112550880$ 个关键码，可索引 $112550880\times 128\approx 1.44\times 10^{10}$ 字节（约 13.4 GB）。(3) 前两层共 $1+103=104$ 页可以整层放进内存，第三层 $103^2=10609$ 页放不下（只能放进一部分）。最少 **1 次**访外：关键码在内存里的前两层就找到了，只需再读 1 次数据记录；最多 **3 次**：关键码在第 4 层，要读第 3 层、第 4 层结点各 1 次，再读数据记录 1 次。(4) B^+ 树内部结点只存（关键码, 孩子指针）：$12m\le 2048$, m 最大 170；叶也是 $\lfloor 2048/12\rfloor=170$ 个（关键码, 记录地址）。4 层最多索引 $170^4=835210000$ 条记录，约 1.07×10^{11} 字节（约 99.6 GB）。前两层 171 页可整层驻留，内存里还能再放第三层的一部分。B^+ 树必须走到叶：第三层结点不在内存时要读第 3、4 层再读记录，**3 次**；第三层结点恰在内存时 **2 次**。（脚本复算。）

16. 按「叶就是数据页、内部结点存（关键码, 页指针）」理解：每页放 $1000/200=5$ 条记录，共 $10^6/5=200000$ 个叶页；内部结点每项 $50+5=55$ 字节，每页最多 $\lfloor 1000/55\rfloor=18$ 项。逐层向上（都装满）：$\lceil 200000/18\rceil=11112$、$\lceil 11112/18\rceil=618$、$\lceil 618/18\rceil=35$、$\lceil 35/18\rceil=2$、1，共 **11768 个索引页块**（非叶结点）。树有 5 层索引加 1 层数据页，存取一个记录需要 **6 次**外存访问（若根常驻内存则 5 次）。（脚本复算。）

17. m 阶 B^+ 树内部结点至多 m 个、至少 $\lceil m/2\rceil$ 个孩子（根至少 2 个），叶至多容纳「叶阶」条记录、至少一半。(1) 内部阶 50、叶阶 50（叶 25～50 条，内部 25～50 个孩子）：1 层（独根即叶）最多 50、最少 1；2 层最多 $50^2=2500$、最少 $2\times 25=50$；3 层最多 125000、最少 $2\times 25\times 25=1250$；4 层最多 6250000、最少 31250；5 层最多 312500000、最少 781250。(2) 内部阶 100、叶阶 15（叶 8～15 条，内部 50～100 个孩子）：1 层最多 15、最少 1；2 层最多 $100\times 15=1500$、最少 $2\times 8=16$；3 层最多 150000、最少 $2\times 50\times 8=800$；4 层最多 15000000、最少 40000；5 层最多 1500000000、最少 2000000。

18. **第 18–22 题的 B / B^+ 树结果都依赖分裂、合并的约定，下面统一按本章正文与 `code/ch11/bplus_tree` 的约定作答（`split_leaf`、`split_internal` 都取 `mid = size/2`）；约定写明后答案唯一，换一种约定（如叶 3+1 分裂）结果会不同，答题时先写出所用约定。**本题约定：4 阶指内部结点至多 4 个孩子（3 个分界码，分界码取右子树最小关键码的复写）；叶至多 3 条记录，叶上溢（4 条）时平分为 2+2，把右叶的最小码复写上去；内部结点上溢（4 个分界码）时把

第 3 个分界码上移，左留 2 个、右留 1 个。按 C、S、D、T、A、M、P、I、B、W、N、G、U、R、K、E、H、O、L、J 依次插入，结果为：根 [I M S]；第二层 [D G]、[K]、[P]、[U]；叶依次 [A B C] [D E] [G H] | [I J] [K L] | [M N O] [P R] | [S T] [U W]，叶之间按序链接。（脚本按上述约定模拟，并与另一份独立模拟逐项一致。）注意：图 11.30 **不是**本题的答案——它的关键码集合含 F、不含 C 和 M，是第 22 题自带的起始树，不能拿来对照本题结果。

19. (1) 3 阶 B 树（每结点至多 2 个关键码），每次插入后：[20]；[20 30]；插 50 根分裂为 `30`；`30`；插 60 叶分裂，52 上移得 `30 52`；`30 52`；插 70 叶分裂、68 上移使根成为 (30 52 68) 再分裂，得 `52, 68)`。删除 50：叶空，兄弟 [20] 借不出，合并为 [20 30]，父结点空，再与兄弟 [68] 及根的 52 合并为 [52 68]，树降一层：`52 68`。再删除 68：用后继 70 替换，叶 [70] 变空，兄弟 [60] 借不出，合并为 [60 70]，得 `52`。(2) 3 阶 B^+ 树（本章约定：叶至多 2 个关键码，分界码取右子树最小码），每次插入后：[20]；[20 30]；`30`；`30 50`；插 60 后叶、根相继分裂：`50, 52)`；`50, 52 60)`；插 70：`50 60, 52, 68)`。删除 50：叶 [50] 变空，兄弟 [52] 借不出，合并后父结点只剩一个孩子，再与左兄弟 [30] 合并成 `30 52`，根成为 [60]：`60, 68)`。再删除 68：叶成为 [70]，未下溢；分界码 68 仍能正确分隔（也可改写为 70）：`60, 68)`。（插入过程脚本复算，删除按上述规则手推。）
20. (1) 3 阶 B 树依次插入 1, 4, 9, ..., 225（递增）：结果是一棵满的 4 层 2-3 树——根 [64]；第二层 [16]、[144]；第三层 [4]、[36]、[100]、[196]；叶 [1][9][25][49][81][121][169][225]。递增插入使每次分裂都发生在最右路径上，最后恰好得到完全平衡的形状。(2) 数据页（叶）各放至多 3 个记录、内部结点至多 3 个孩子（分界码取右子树最小码，叶上溢时 2+2 平分）：叶依次为 [1 4] [9 16] [25 36] [49 64] [81 100] [121 144] [169 196 225]，索引为根 [25 81]，第二层 [9]、[49]、[121 169]。递增插入使除最后一页外每页只有一半满——这是顺序插入下 B 树类结构的典型现象，批量建树（11.2 节的 `bulk_load`）应该自底向上一次装满。（脚本模拟。）
21. 图 11.29：根 (17 35)；第二层 (15 17)、(35)；叶 (10 15)(16 17) 与 (35)，采用最大关键码复写。删除 35 后，叶 (35) 变空，其父 (35) 也随之没有孩子，根只剩下左边一棵子树。根只有一个孩子时应当删去根、让孩子当新根，所以删除后是 2 **层**：根 (15 17)，叶 (10 15)(16 17)。若硬保留 3 层，根就只有一个孩子，违反「根至少有两个子结点」。
22. 图 11.30 的叶阶为 3，叶至多 3 个关键码、至少 2 个；内部阶为 4，至多 4 个、至少 2 个孩子。(1) 插入 C：落到叶 [A B]，成为 [A B C]，未超过 3，**直接放下，树形不变**。(2) 删除 I：叶 [I J] 成为 [J]，下溢；同父的兄弟 [K L] 只有 2 个借不出，合并为 [J K L]；父结点 [K] 只剩一个孩子，下溢。按 11.4 节「阶小时先合并」的建议，与右兄弟 [P] 及根中的分界码合并为 [N P]（孩子 [J K L] [N O] [P R]），根成为 [I S]（分界码 I 仍能正确分隔，也可改写为 J）；也可以向左兄弟 [D G] 借：把 [G H] 过继过来，根的分界码改为 G，左兄弟成为 [D]。两种结果都满足 B^+ 树定义。

23. 年龄取值为 25、30、45、50、60、70、85，各位向量（第 1～12 位对应顾客 1～12）：25: `100000001000`；30: `000000010000`；45: `010000000100`；50: `001110000010`；60: `000000000001`；70: `000001000000`；85: `000000100000`。(2) 月收入：600: `110000000000`；750: `001000000000`；1000: `000100000000`；1100: `000001000000`；1200: `000010000000`；1400: `000000100000`；2600: `000000010001`；2750: `000000000010`；3500: `000000000100`；4000: `000000001000`。(3) 年龄 20～40 是 25、30 两个向量的或：`100000011000`（顾客 1、8、9）；月收入 1000～2000 是 1000、1100、1200、1400 的或：`000111100000`（顾客 4～7）；两者按位与为全 0——**没有符合条件的顾客**。（脚本复算。）
24. (1) 每个取值一条 1000000 位的向量，m 个取值共 $m \times 10^6$ 位 $= 125000m$ 字节。(2) 每个取值每隔 m 个记录出现一次，它的位向量是「1 个 1、$m-1$ 个 0」周期重复。游程编码（RLE）每条向量约有 $2 \times 10^6/m$ 个游程，若游程长用 4 字节存，每条约 $8 \times 10^6/m$ 字节，m 条合计约 8×10^6 字节，与 m 无关；进一步利用周期性，每个取值只需存「首次出现的位置 + 周期」两个整数，共 $8m$ 字节。位图在低基数时最划算，基数一高，未压缩的位图就要靠压缩来救。
25. 图 11.31 的红黑树：15(黑)；6(黑)、17(黑)；3(黑)、12(红)、16(黑)、20(红)；12 的孩子 10(黑)、13(黑)，20 的孩子 18(黑)、23(黑)；10 的左孩子 7(红)。(1) 插入 19：沿 15 → 17 → 20 → 18 走到 18 的右空位，19 着红色；父结点 18 是黑色，没有违反任何红黑性质，**不需要调整**。(2) 删除 10：10 是黑色、只有一个红孩子 7，用 7 顶替 10 的位置并把 7 染黑，各路径黑高不变，**一次重新着色即可**。两题都是最简单的情形，正好用来核对红黑性质：每条根到空指针的路径上都恰有 3 个黑结点。
26. 自顶向下插入：从根向下找插入位置的途中，遇到两个孩子都是红色的结点就先把它染红、两个孩子染黑（颜色翻转）；若翻转造成父子双红，立即在这一层用一次单旋或双旋修复。这样走到底时新结点的父结点必为黑色或可用一次旋转处理，插入后不必再回溯。例如依次插入 1～7 时，每下行遇到「黑父红双子」就先翻转，可以对照 11.5 节的自底向上算法比较两者的旋转位置。自顶向下删除：下行途中保证当前结点为红色（必要时从兄弟借红或做旋转、翻转），这样到达要删除的叶时它是红色，可直接删去。两种方法都只需一次自顶向下的遍历、不需要父指针或回溯栈，适合并发控制。### 原书上机题参考
27. 二级线性索引：数据文件按追加顺序存放变长记录（每条以 4 字节关键码开头），索引文件每项为（关键码，字节偏移），按关键码有序、分块存放，每块 1024 字节可放 $1024/8 = 128$ 项（关键码 4 字节 + 偏移 4 字节）；顶层索引是内存中的简单数组，存每个索引块的最小关键码。查找时先在顶层数组上二分找到索引块，读入该块再二分找到偏移，最后按偏移读记录：2 次访外。插入新记录追加到数据文件末尾，再把索引项插入对应索引块（满了就分裂并更新顶层数组）；删除只需删索引项、数据文件可延迟整理。
28. 参照 B 树：插入时叶上溢平分为两叶、把分界码复写到父结点（叶上保留），

内部结点上溢时中位数上移（不在原结点保留）；删除时叶下溢先向兄弟借、借不到再与兄弟合并并删去父结点里相应的分界码，内部结点下溢的处理与 B 树相同；全程维护叶链。用 `code/ch11/bplus_tree` 的 `BPlusTree::validate()` 在每次操作后检查有序性、结点容量、叶深一致和叶链，再与 `std::map` 随机对拍。

29. 中序遍历的推广：对结点 $(p_0, K_1, p_1, \ldots, K_k, p_k)$，依次递归遍历 p_0、输出 K_1、递归遍历 p_1、输出 K_2……最后递归遍历 p_k。由 B 树的有序性（p_{i-1} 子树的关键码都小于 K_i，p_i 子树的都大于 K_i），输出递增，每个关键码恰好输出一次，时间 $O(n)$；用显式栈保存（结点，下一个要处理的指针下标）即可写成非递归。
30. B^* 树要求非根结点至少 $\lceil 2m/3 \rceil$ 个孩子。插入上溢时先尝试把多出的关键码经父结点匀给相邻兄弟（兄弟不满时）；兄弟也满了，就把两个满结点连同父结点中的分界码共 $2m$ 个关键码分成三个结点，每个约 $2m/3$ 满，向父结点增加一个分界码。删除下溢时先向兄弟借，借不到就把三个相邻结点（自己和两个兄弟）合并成两个。这样空间利用率至少约 2/3，代价是插入删除要多读一个兄弟结点。
31. (1) 设 $bh(T_1) \geq bh(T_2)$（黑高）。沿 T_1 的最右路径向下，找到黑高等于 $bh(T_2)$ 的黑结点 y；以 z 为新结点、着红色，左子树为 y 为根的子树、右子树为 T_2，把 z 接到 y 原来的父结点上。此时只可能出现红父红子，按插入后的修复过程向上处理即可。求黑高与下行都是 $O(\log N)$，修复也是 $O(\log N)$，总代价 $O(\log N)$；$bh(T_1) < bh(T_2)$ 时对称地沿 T_2 的最左路径下行。(2) 不给 z 时，先从 T_1 中删去最大关键码作为 z（$O(\log N)$），再按 (1) 合并。前提是 T_1 的关键码都小于 T_2 的；若两棵树值域交错，就只能中序展开成两个有序表、归并后再建树，代价 $O(N)$。## 第 12 章 高级数据结构（原书题补充答案）

原书习题答案

1. 方阵原地转置只交换 `i<j` 的 a[i][j] 与 a[j][i]，`O(n^2)`、O(1)。
2. 双循环写对角 0、其余 1/空格后逐行打印。
3. 每行找最小及列号，再扫描该列验证是否列最大；`O(mn)`。
4. 广义表逆置用访问标记防环，递归先逆子表再反转同层 next 链。
5. 画图题以 atom/list 标志结点，head/tail 或兄弟链表示；共享引用必须指向同一结点。
6. `Head` 取表的第一个元素，`Tail` 取去掉第一个元素后剩下的表。$L1 = ((((a), b), d), e)$：`Head(L1)` $= (((a), b), d)$，`Tail` 得 (d)，再 `Head` 得 d，即 $d =$ `Head(Tail(Head(L1)))`。$L2 = (a, (b, ((d)), e))$：`Tail(L2)` $= (b, ((d)), e)$，`Head` 得 $(b, ((d)), e)$，`Tail` 得 $(((d)), e)$，`Head` 得 $((d))$，再 `Head` 得 (d)，再 `Head` 得 d，即 $d =$ `Head(Head(Head(Tail(Head(Tail(L2))))))`。注意 `Tail` 的结果总是表，取单个元素必须最后用 `Head`。
7. 深度为原子 0、空表 1（按定义），一般为 `1+max(child depth)`，环需 visited

防无限递归。

8. DFS 删除匹配原子，修改前保存 next；共享结点只处理一次。
9. 带 visited-pair 的同步比较，原子值/类型相同且子结构递归相同才相等。
10. 单独一个请求分辨不出三种方法（只要某块够大，三种方法都能满足），但一串请求可以：前面的请求挑了哪一块，决定了后面的请求还有没有足够大的块。空闲块按地址依次为 900、1500、700，下面三组请求都经脚本模拟核对。(1) 只有首先适配能满足：150、600、950、600——首先适配把 150、600 都切在 900 上，留下 1500 给 950，最后 600 用 700；最佳适配把 150 放进 700、600 放进 900，950 用掉 1500 后最后的 600 无处可放；最差适配把 150、600 都切在 1500 上，剩 750 放不下 950。(2) 只有最佳适配能满足：100、900、1000——最佳适配把 100 放进 700，900、1000 正好用 900 和 1500；首先适配把 100 切在 900 上，900 只能用 1500，剩 600 放不下 1000；最差适配把 100、900 都切在 1500 上，剩下 900、500、700 放不下 1000。(3) 只有最差适配能满足：750、800、750——最差适配把 750 切在 1500 上留 750，800 用 900，最后 750 正好用剩下的 750；首先适配和最佳适配都把第一个 750 放进 900，800 用 1500 留 700，最后的 750 放不下。正文习题第 4 题说「能否满足分辨不出三种策略」，指的是单个请求。
11. 把可利用空间表**按块的大小从小到大排序**，并且每次都从最小的块开始扫描（而不是从上次停下的位置继续）——这时第一个够大的块就是所有够大的块里最小的，首次适配就等于最佳适配。代价是分配、回收时都要维护按大小的次序，而且按地址相邻的空闲块不再在表中相邻，合并相邻空闲块要另想办法。
12. 按原书 12.3 节，二叉 Trie 是把关键码看成二进制位串、每层按一位分左（0）右（1）。字符串先编码成位串，例如每个字母用 5 位（a=00001，b=00010，…，z=11010），词尾补全 0 作结束标记；从根开始按第 1 位、第 2 位……逐层分支，直到一棵子树里只剩一个关键码为止，该关键码作为叶挂在那里。8 个词的公共前缀决定了树形：cop、copy、cup 共享 c 的 5 位，hi、hia、his、hit、he 共享 h 的 5 位，再往下 o/u、i/e 在各自的编码第一个不同的位上分开；cop 与 copy 要一直比到 cop 的结束标记才分开。实际画图时，按字母编码逐位展开即可（字母 Trie 的每一层相当于二叉 Trie 的连续 5 层）。按字母分支的普通 Trie 则为：根下 c、h 两支；c 下 o → p（cop 结束）→ y（copy）与 u → p（cup）；h 下 e（he）与 i（hi 结束）→ a（hia）、s（his）、t（hit）。
13. PAT（Patricia）Trie 在二叉 Trie 的基础上压缩掉所有只有一个孩子的结点，每个内部结点只记「下一步比较第几位」，于是 8 个关键码恰有 7 个内部结点。按字母层面看压缩效果：根先按首字母分成 c、h 两支（c 与 h 在编码上第一个不同的位处分支）；c 支直接比到第 2 个字母（o/u）分开，cup 成叶；o 支跳过 p，比较 cop 的结束标记与 copy 的 y 分开；h 支比第 2 个字母分出 he 与 hi…；hi… 再比第 3 个字母（结束标记/a/s/t）分出 hi、hia、his、hit。内部结点数 $=$ 关键码数 $-1 = 7$ 是 PAT 树的一般性质（每个内部结点恰有两个孩子）。

14. 这里的高度按层数算（2^k-1 个结点的满二叉树高为 k）。证明一个更强的命题 $Q(k)$：**一棵高为 k 的满 AVL 树，再依次插入 2^k 个比已有键都大的递增键，得到高为 $k+1$ 的满二叉树**。从空树出发反复用 Q（$Q(0)$：空树插 1 个键得单结点）即得本题。先记一个事实：AVL 插入不会让树变矮（不旋转时高度不变或加 1，旋转后恢复插入前的高度），所以插入过程中子树高度单调不减。证 $Q(k)$（$k\ge 1$）：设树为根 r、左右子树 L、R 都是高 $k-1$ 的满树。递增键总是插到最右路径末端。第一阶段插入前 2^{k-1} 个键，它们都进入 R；由 $Q(k-1)$，R 最终成为高 k 的满树，过程中 R 的高度在 $k-1$ 与 k 之间，r 处高度差不超过 1，不会在 r 处旋转。此时 R 的根为 r'，其左右子树 L'、R'' 都是高 $k-1$ 的满树。第二阶段的第一个键插进 R'' 末端，R'' 变高为 k（满树加一个结点，内部不失衡），r' 处左 $k-1$、右 k 仍平衡，而 r 处左 $k-1$、右 $k+1$ 失衡且为 RR 型，对 r 左旋一次：新根 r'，左子树为 (L,r,L')——两个高 $k-1$ 的满树加一个根，恰是高 k 的满树——右子树为 R''。剩下的 $2^{k-1}-1$ 个键继续插入 R''，由 $Q(k-1)$（第一个键已插入，过程相同）R'' 最终成为高 k 的满树，过程中高度保持为 k，根处不再旋转。于是整棵树是左右都为高 k 满树的高 $k+1$ 满二叉树。（脚本对 $n=1..31$ 逐个插入核对：根依次在第 3、6、12、24 个键时换成 2、4、8、16，$n=2^k-1$ 时都是满二叉树。）
15. 按字典序比较（注意 July < June：第三个字母 l < n）。(1) 按 Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec 次序插入 BST：根 Jan；Jan 的左孩子 Feb、右孩子 Mar；Feb 的左孩子 Apr，Apr 的右孩子 Aug，Aug 的右孩子 Dec；Mar 的左孩子 June、右孩子 May；June 的左孩子 July；May 的右孩子 Sep，Sep 的左孩子 Oct，Oct 的左孩子 Nov。各结点深度（根为 1）之和为 42，成功查找 $ASL=42/12=3.5$。(2) 按同样次序插入 AVL 树（必要时旋转），结果：根 Mar；Mar 的左孩子 Jan、右孩子 Oct；Jan 的左孩子 Aug、右孩子 June；Aug 的左孩子 Apr、右孩子 Feb，Feb 的左孩子 Dec；June 的左孩子 July；Oct 的左孩子 May、右孩子 Sep；May 的右孩子 Nov。深度之和为 38，$ASL=38/12\approx 3.17$。（脚本建树复算。）
16. 可允许重复但必须规定等键统一去一侧或结点存计数；图中两个 12 需靠附加次关键字/计数解释。
17. 按本书 12.4.2 节的约定（只有根的树高度为 0，$t(1)=2$、$t(2)=4$）：高度为 h 的 AVL 树最少结点数 $N(h)$ 满足 $N(0)=1$、$N(1)=2$、$N(h)=N(h-1)+N(h-2)+1$，即 $N(h)=F(h+3)-1$（F 为 Fibonacci 数，$F(1)=F(2)=1$），前几项为 1、2、4、7、12、20、33；最多结点数是满二叉树的 $2^{h+1}-1$。若约定只有根的树高度为 1，则把 h 换成 $h-1$：最少 $F(h+2)-1$、最多 2^h-1。答题时先说明高度的口径。
18. 按字典序（he < head < hot < tea < teach < toss < twin）。(1) 依次插入：head；he 作 head 的左孩子；tea 作右孩子；teach 挂在 tea 右下；插 twin 时 tea 处 RR 失衡，左旋得 head(he, teach(tea, twin))；插 hot（成为 tea 的左孩子）时 head 处 RL 失衡，双旋后根变为 tea：tea(head(he, hot), teach(-, twin))；插 toss（成

为 twin 的左孩子）时 teach 处 RL 失衡，双旋得最终的 AVL 树 tea(head(he, hot), toss(teach, twin))。(2) 依次删除（双孩子结点用中序后继替换）：删 head → tea(hot(he, -), toss(teach, twin))；删 he → tea(hot, toss(teach, twin))；删 tea（后继 teach 顶上）→ teach(hot, toss(-, twin))；删 teach（后继 toss 顶上）→ toss(hot, twin)；删 twin → toss(hot, -)；删 hot → toss；删 toss → 空树。整个删除过程各结点平衡因子都没超出 ± 1，不需要旋转。用中序前驱替换时中间几步的形状会不同。（脚本模拟。）### 原书上机题参考

19. 比较二维数组元素可用 hash set，发现重复即停；时间 O(mn)、空间 O(mn)。
20. 广义表结点含 mark，显式栈 DFS，首次访问才输出并置 mark；结束后清 mark 以便复用。
21. C++ 相似度先词法化去空白/注释并归一标识符，再比较 token n-gram/Jaccard 或编辑距离；输出启发式分数。
22. 稀疏矩阵用 CSR/十字链表，插删查和转置维护行列索引，加法为同一行归并。
23. 空闲块按地址、大小组织，首次/最佳/最坏选择后分割，释放时与相邻块合并；随机请求统计碎片。
24. Trie 节点存 children、terminal，删除后自底向上回收无分支节点。
25. Trie DFS 词典序输出，字符串排序总工作与字符总数成正比。
26. 先 Trie 收词，再按 `(length,word)` 排序或分长度桶内 DFS 输出。
27. Patricia/Trie 统计路径字符计数；构造与扫描 O(total chars)。
28. 后缀树用终止符、边标签区间和 suffix link；Ukkonen 线性构造，查询前缀/子串沿边匹配。
29. 枚举中心扩展或后缀/Manacher 变体，并用互补映射 A-T、C-G 验证回文。
30. 半伸展 BST 记录 parent，访问后按规则旋转；词频与结构调整解耦，最后中序输出。
31. 阅卷系统用试卷状态机和事务锁，随机分配未完成试卷，双评后聚合，匿名 ID 与审计日志分离。
32. 职工系统以 ID 主键、字段索引、验证器和分页查询实现 CRUD；日期/电话校验、排序与持久化分层。
33. 搜索引擎分 crawler、解析、倒排索引、查询/排序，需去重、限速、增量更新和评估集。
34. Suggest 用 Trie 前缀定位，每结点维护 top-10 `(frequency,term)`，更新沿路径维护堆/有序数组。

期中复习题库（第 1—6 章）

本篇按《数据结构与算法》期中复习提要逐条配题，范围是第 1 章到第 6 章。提要里标 ★ 的条目 是重中之重，题号后同样标 ★；没标的也配了题，但分量轻一些。**提要原件逐字存在仓库里**：`ref_ 复习大纲/期中复习提要.md`。

怎么用：分章题答案跟在题目下面的引用块里，模拟卷答案集中在各组之后，先把引用块遮住自己做一遍，再对答案。每章按考试题型分五组：填空、选择、辨析与简答、数据结构或算法的设计与分析、数学证明，分章题共 167 道；之后是一套完整的模拟卷（26 道，按真实考试的分值和时间配比出的），最后一页是速查表。

几个贯穿全篇的约定（考试写答案时最好也先声明一句，避免歧义）：

- 数组、字符串、树的层次编号一律**从 0 开始**；根结点在**第 0 层**。凡是与此不同的题，题面会单独说明。
- 「深度为 k 的二叉树」指最大层号为 k，此时结点数至多 $2^{k+1}-1$；树的「高度」默认指**层数**（若按边数计会单独说明）；「n 个结点的完全二叉树高度」按提要口径同样指**层数**，即 $\lceil \log_2(n+1) \rceil$。
- 算法设计题按提要的要求：**写明基本思想、无歧义即可**，必要时加注释。用到栈、队列等现成结构时，先写出 ADT（函数名、入口参数、出口参数、功能一句话），再在算法里调用。本篇提供了常用 ADT 的写法，后续题目可引用相应接口。
- 本篇的「满二叉树／满 K 叉树」采用严格（正则）树的含义：每个内部结点恰有 2 ／ K 个孩子，不要求叶结点同层；所有层都填满时另称「各层全满」。K、k 出现在除以 $K-1$、$k-1$ 的公式中时均要求至少为 2。
- 除另有说明，结点数据的比较、赋值和交换按常数时间计；树的遍历序列用于还原结构时，结点标识互不相同。
- 空指针计数公式均针对非空树，且只计结点内的孩子指针域。
- 伪代码用缩进表示层次，← 是赋值，= 是比较；逻辑运算采用短路求值。

第 1 章 概论

一、填空题

1-1. 数据结构的研究内容包括三个方面：____、____ 和 ____。

> **答**：数据的**逻辑结构**、数据的**存储结构**（物理结构）、**在此结构上定义的运算**（算法）。三者的关系是：逻辑结构描述「数据之间是什么关系」，存储结构描述「这些关系在内存里怎么落实」，运算的代价则由前两者共同决定。见第 1.2 节。

1-2. 抽象数据类型（ADT）由 ____ 和 ____ 两部分组成，它**不包括** ____。

> **答**：由**数据对象及其上的关系**（数据模型）和**定义在其上的一组操作**

组成；它**不包括数据的存储结构**。这一条是历年反复考的辨析点：ADT 只回答「能做什么」，把「怎么存、怎么做」留给实现。一旦把存储方式写进 ADT，它就不再「抽象」了。也常把前一部分拆开，说成「数据对象、数据关系、基本操作」三部分（模拟卷一·1 即按此口径），内容相同。

1-3. 数据的四种基本存储方式是 ____、____、____、____。

答：**顺序**存储、**链接**存储、**索引**存储、**散列**存储。顺序靠下标算地址，随机访问 $O(1)$、插删要搬元素；链接靠指针相连，已知所需结点及前驱时插删 $O(1)$，但不能随机访问；索引另建索引表，借助索引定位记录；散列由关键码直接算地址，平均 $O(1)$ 但要处理冲突。

1-4. 大 O 表示法的定义：称 $f(n)=O(g(n))$，当且仅当存在正常数 ____ 和 ____，使得对所有 ____ 都有 ____ 成立。

答：存在正常数 c 和 n_0，使得对所有 $n\ge n_0$ 都有 $f(n)\le c\cdot g(n)$。三个要点缺一不可：c 与 n 无关、只要求 n 充分大之后成立、是**上界**而不是确界。

1-5. 把下列函数按渐进增长率从小到大排序：$n\log n$，2^n，n^2，$\log n$，$n!$，n，n^3，1，$\sqrt{n}$，2^{2^n}。

答：$1<\log n<\sqrt{n}<n<n\log n<n^2<n^3<2^n<n!<2^{2^n}$。常错的两处：$\sqrt{n}$ 在 $\log n$ 之后（任何正数次幂都最终超过对数）；$n!$ 在 2^n 之后（由 $n!\ge(n/2)^{n/2}$ 可得）。

1-6. 本书列出的算法基本性质有 ____、____、____、____。

答：**通用性**（对符合输入类型的任意输入都能正确求解）、**有效性**（每条指令都能被人或机器确切执行）、**确定性**（每步之后下一步怎么走有明确指示）、**有穷性**（有限步内结束）。见第 1.3.1 节。少了有穷性的就只是「计算过程」（如操作系统主循环），不叫算法。另一种常见口径是 Knuth 的「五特性」：有穷性、确定性、可行性、输入、输出——有效性大致对应可行性，输入输出则体现在通用性之中。题面若明说「五个特性」就按这一套答。

1-7. 数据的逻辑结构用二元组 (K,R) 表示时，K 是 ____，R 是 ____；画结构图时，R 中的序偶 $\langle x,y\rangle$ 必须画成 ____。

答：K 是**结点（数据元素）的有穷集合**，R 是**定义在 K 上的关系的集合**；$\langle x,y\rangle$ 必须画成**从 x 指向 y 的有向边**（箭头方向不能省，$\langle x,y\rangle$ 与 $\langle y,x\rangle$ 是两条不同的边）。只有明确声明关系对称时才画成无向边。

二、选择题

1-8. 下列关于抽象数据类型的叙述，**错误**的是（　）。A. ADT 给出了数据对象、数据关系和基本运算 B. ADT 的定义中包含了数据的存储结构 C. 同一个 ADT 可以有多种不同的实现 D. ADT 把「做什么」和「怎么做」分开

答：B。ADT 与具体存储无关。栈这个 ADT 既可以用数组实现（顺序

栈），也可以用链表实现（链式栈），两者的 ADT 完全相同，接口一字不差，这正是 C 和 D 说的事情。

1-9. 以下说法正确的是（　）。A. 算法的时间代价必定随问题规模单调增长 B. 若 $f(n)=O(n^2)$，则 $f(n)$ 一定不是 $O(n)$ C. 大 O 给出的是渐进上界，同一个函数可以有多个正确的大 O 表达式 D. 最坏情况分析和大 O 分析是同一件事

答： C。$f(n)=n$ 既是 $O(n)$ 也是 $O(n^2)$，所以 B 错、C 对。D 是最常见的混淆：「**最好/最坏/平均」说的是输入的挑法，「$O/\Omega/\Theta$」说的是函数的界法**，两者是正交的两个维度——完全可以说「最好情况是 $O(n)$」。A 错：代价未必对规模单调，例如插入排序对已有序的大输入只需 $O(n)$，可能比更小的逆序输入还快。

1-10. 下面程序段中循环体执行次数的渐进紧确界是（　）。

```
s ← 0;  i ← 1
do
    s ← s + 5 * i
    i ← i + 1
while (s < n) and (i < n)
```

A. $\Theta(n)$　　B. $\Theta(\sqrt{n})$　　C. $\Theta(\log n)$　　D. $\Theta(n^2)$

答： B。循环 i 次后 $s=5(1+2+\cdots+i)=\frac{5i(i+1)}{2}$，当 n 充分大时，退出条件由 $s\ge n$ 先触发，于是 $i=\Theta(\sqrt{n})$。这题的陷阱是 `and` 右边那个 `i < n`——它只能给出较松的 $O(n)$ 上界，不能据此认定为 $\Theta(n)$，**两个条件取先满足的那个**。

1-11. 对同一问题，算法 A 的时间代价为 $1000n$，算法 B 为 n^2。下列说法正确的是（　）。A. A 总比 B 快　B. B 总比 A 快　C. $n>1000$ 时 A 比 B 快　D. 两者渐进同阶

答： C。渐进分析丢掉常数，是因为**规模足够大时常数必然被增长率淹没**；但「足够大」是多大要具体算：这里的分界点正是 $n=1000$。实际工程里 n 若始终只有几十，反而该选 B——这也是第 1.4.4 节说的「选择和评价」。

1-12. 下列各式中**不成立**的是（　）。A. $3n^2+10n=O(n^2)$　　B. $n^2/(n+1)=O(n)$　　C. $2^{n+1}=O(2^n)$　　D. $2^{2n}=O(2^n)$

答： D。$2^{2n}=(2^n)^2$，比值 2^n 无界。C 成立是因为 $2^{n+1}=2\cdot 2^n$，只差常数因子 2。B 成立是因为 $n^2/(n+1)<n$。

三、辨析与简答

1-13. 辨析：「数据的逻辑结构一旦确定，存储结构也就确定了。」

答：错。同一逻辑结构可以有多种存储结构，这正是本课程的核心。线性表既可以顺序存储（顺序表）也可以链接存储（链表）；树既可以用左子/右兄二叉链存，也可以用父指针数组存，还可以按先根次序排进

一维数组。**逻辑结构决定「能做什么」，存储结构决定「做得多快」**。

1-14. ★ 简答：最坏情况分析与大 O 表示法有什么区别和联系？为什么说「某算法的时间复杂度是 $O(n^2)$」这句话本身并不说明它是最坏情况？

答：1. **它们回答的是不同的问题**。「最好/最坏/平均情况」是在**所有规模为 n 的输入**里挑一个 来分析：最坏是取代价最大的那个输入，平均是按输入分布取期望。而 $O/\Omega/\Theta$ 是在**确定了某个输入类别之后**，给这条代价曲线定界：O 是上界、Ω 是下界、Θ 是同阶。2. **所以两个维度可以自由组合**，「最好情况是 $\Theta(n)$」「最坏情况是 $O(n^2)$」都是合法句子。3. 因此单说「复杂度是 $O(n^2)$」只承诺了「不会比 n^2 更糟」，**并没有说清是对哪一类输入说的**，严谨的写法要把两个维度都点明。考试答题时建议写成「最坏情况下为 $O(n^2)$」。

1-15. 简答：时间与空间的折衷，举一个例子说明。

答：多用空间往往能省时间，反之亦然。例：判断一个字符是否属于某集合，逐个比较是 $O(|S|)$ 时间、$O(1)$ 额外空间；预先建一张 256 项的布尔表，则是 $O(1)$ 时间、$O(256)$ 空间。又如第 4 章 KMP 的 next 数组：多花 $O(|P|)$ 空间，把匹配从 $O(|P|\cdot|T|)$ 降到 $O(|P|+|T|)$。反向的例子是原地排序：不借助辅助数组，用时间换空间。

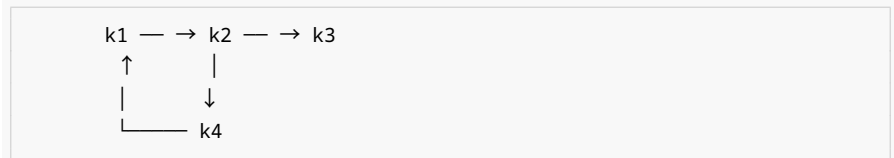
1-16. 简答：为什么渐进分析要「保留增长最快的那一项」并丢掉常数因子？什么时候这样做会误导人？

答：因为当 $n\to\infty$ 时，低阶项相对最高阶项的比值趋于 0；固定的正倍数则不改变渐进阶。省略它们能突出不同算法随规模增长的主要差别。会误导人的情形有三类：(1) n **始终很小**——此时常数是主角（见 1-11）；(2) **常数因子大得离谱**——某些理论上更优的算法常数是天文数字，实际从不使用；(3) **关心的不是渐进而是确切次数**——例如比较次数下界这类精确结论。

四、算法设计与分析

1-17. 设 $K=\{k_1,k_2,k_3,k_4\}$，$R=\{\langle k_1,k_2\rangle,\langle k_2,k_3\rangle,\langle k_2,k_4\rangle,\langle k_4,k_1\rangle\}$。画出该逻辑结构的示意图，并说明它属于哪一类结构。

答：四个结点，画四条**有向**边：$k_1\to k_2$、$k_2\to k_3$、$k_2\to k_4$、$k_4\to k_1$。



它**不是**线性结构（k_2 有两个后继），也**不是**树（存在回路 $k_1\to k_2\to k_4\to k_1$，且 k_1 有前驱），因此属于**图**结构（有向图）。评分要点：箭头方向必须画对；结论要给出判断依据，不能只写「是图」。

1-18. ★ 设计：某系统要维护一批学生成绩，要求支持三种操作—— (a) 按学号查一个学生 (b) 插入一个新学生 (c) 按成绩从高到低输出全部学生。若三种操作的频率相当，请给出一种数据结构设计并分析代价；若 (c) 只在学期末执行一次，设计又该怎么改？

答：**情形一（三者频率相当）**：按学号组织一棵平衡二叉搜索树（平衡树可作为第 5 章 BST 的改进），另在每个结点里存成绩。- (a) 查找 $O(\log n)$（树平衡时）；- (b) 插入 $O(\log n)$；- (c) 中序遍历得到按学号有序的序列，再排序，$O(n \log n)$。

若 (c) 也频繁，可**同时维护两个索引**：学号平衡 BST 与成绩平衡 BST，以（成绩，学号）作为成绩索引的关键码以处理同分，两棵树的结点指向同一份记录，插入时各插一次，仍是 $O(\log n)$；(c) 变成一次逆中序遍历（右、根、左）$O(n)$。代价是空间翻倍、且两个索引必须同步更新——**这就是典型的时空折衷**。

情形二（(c) 只在期末一次）：不必为 (c) 付日常代价。只保留学号索引（BST 或散列表，散列时 (a)(b) 平均 $O(1)$），期末一次性取出全部记录排序，$O(n \log n)$ 只发生一次。

评分要点：**先说清每种操作的频率如何影响选择**，再给结构，最后逐条给代价。只写「用二叉搜索树」而不分析，拿不到设计题的分。

1-19. ★ 分析下列程序段中 `x ← x + 1` 的执行次数 m，并给出时间复杂度。

```
for i ← 1 to n
    for j ← 1 to n - 2 * i + 1
        x ← x + 1
```

答：内层对每个 i 执行 $\max(0,\, n-2i+1)$ 次，只有 $i \le n/2$ 时才进入循环，于是

$$m = \sum_{i=1}^{\lfloor n/2 \rfloor} (n - 2i + 1) = \frac{n}{2}(n+1) - 2 \cdot \frac{(n/2)(n/2+1)}{2} = \frac{n^2}{4}$$

（取 n 为偶数；$n=10$ 时 $m=25$，$n=100$ 时 $m=2500$，与公式相符。n 为奇数时求和上限是 $(n-1)/2$，同法算得 $m = \frac{n^2-1}{4}$，如 $n=11$ 时 $m=30$。）时间复杂度 $\Theta(n^2)$。评分要点：**先写出求和式再化简**，直接目测「两重循环所以是 n^2」在本题恰好对，但遇到内层次数与 i 相关且可能为负的情形就会错。

1-20. ★ 分析下面程序段中内层循环体的执行次数，给出时间复杂度。

```
for i ← 1 to n
    j ← i
    while j ⩽ n
        操作
        j ← j + i
```

答：内层对每个 i，j 取遍 $i, 2i, 3i, \cdots$，执行 $\lfloor n/i \rfloor$ 次，于是总次数

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor \;\approx\; n \sum_{i=1}^{n} \frac{1}{i} \;=\; n \cdot H_n$$

而**调和数** $H_n = \ln n + \gamma + O(1/n) = \Theta(\log n)$，故总次数为 $\Theta(n \log n)$。（$n = 10^4$ 时实算是 $93\,668$，$n \ln n \approx 92\,103$，量级吻合。）

这道题的价值在于「内层次数既不是常数也不是 n」。看见两重循环就写 $\Theta(n^2)$ 是错的——那要求内层每次都跑 $\Theta(n)$ 次，而这里只有 $i = 1$ 那一轮如此，i 一大内层就很短。做法仍是 1-19 的老套路：**先写出求和式，再化简**；区别只在这次求和的是 n/i 而不是常数。

顺便把这个界记住：$1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\log n)$。它在「把每个数的倍数都划一遍」的筛法的代价、快速排序的平均情况、散列表的平均探查次数里都会再出现。（真正的埃拉托色尼筛法只划**素数**的倍数，代价更低，是 $\Theta(n \log \log n)$，别把两者混为一谈。）

1-21. ★ 求下列递推式的渐进解：(1) $T(n) = T(n/2) + c$；(2) $T(n) = 2T(n/2) + c$；(3) $T(n) = 2T(n/2) + n$；(4) $T(n) = T(n-1) + n$。

答：(1) $\Theta(\log n)$——每层只花常数，共 $\log n$ 层（二分检索）。(2) $\Theta(n)$——递归树有 n 个叶结点，每个结点花常数，总代价与结点数同阶。(3) $\Theta(n \log n)$——**每一层的总代价都是** n，共 $\Theta(\log n)$ 层（归并排序、快排最好情况）。(4) $\Theta(n^2)$——$T(n) = n + (n-1) + \cdots = n(n+1)/2$（快排最坏情况）。

通用做法：画递归树，先算「每层总代价」，再乘「层数」；不确定时用代入法验证。

五、数学证明

1-22. 用大 O 的定义证明 $3n^2 + 10n + 7 = O(n^2)$，并给出一组具体的 c 与 n_0。

证明. 当 $n \geq 1$ 时，$10n \leq 10n^2$、$7 \leq 7n^2$，故 $3n^2 + 10n + 7 \leq 3n^2 + 10n^2 + 7n^2 = 20n^2$。取 $c = 20$、$n_0 = 1$，对一切 $n \geq n_0$ 有 $3n^2 + 10n + 7 \leq c\,n^2$，由定义即得。∎

说明：c 不唯一。取 $n_0 = 10$ 时 $c = 5$ 也够用（$10n \leq n^2$、$7 \leq n^2$，

得 $\le 5n^2$)。只要**给出一对具体的 (c, n_0) 并验证不等式**即可，不必求最优。

1-23. 证明：对任意非负函数 f, g，有 $\max\{f(n), g(n)\} = \Theta(f(n) + g(n))$。

证明. 一方面 $\max\{f, g\} \le f + g$（另一个非负），取 $c_2 = 1$ 得上界。另一方面 $f + g \le 2\max\{f, g\}$，即 $\max\{f, g\} \ge \frac{1}{2}(f + g)$，取 $c_1 = \frac{1}{2}$ 得下界。两侧对一切 n 成立（$n_0 = 1$），故 $\frac{1}{2}(f + g) \le \max\{f, g\} \le f + g$，即 Θ 关系成立。∎

这条结论的用处：程序里「先做 A 再做 B」的总代价与「两者中较慢的那个」同阶——这正是渐进分析里「取最慢的一项」的严格依据。

1-24. ★ 证明 $T(n) = 2T(\lfloor n/2 \rfloor) + n$（$T(1) = 1$）的解是 $\Theta(n \log n)$。

证明. 上界（递归树）：第 i 层有 2^i 个规模为 $\lfloor n/2^i \rfloor$ 的子问题，该层总代价 $2^i \lfloor n/2^i \rfloor \le n$（叶结点代价也是其规模 1）。最大层号为 $\lfloor \log_2 n \rfloor$（共 $\lfloor \log_2 n \rfloor + 1$ 层），故总代价 $\le n(\log_2 n + 1) = O(n \log n)$。

代入法验证：设 $T(k) \le ck \log_2 k$ 对一切 $2 \le k < n$ 成立，则 $T(n) \le 2c\frac{n}{2}\log_2 \frac{n}{2} + n = cn \log_2 n - cn + n \le cn \log_2 n$（只要 $c \ge 1$）。**基础情形要单独核**：$n = 1$ 时右边 $c \cdot 1 \cdot \log_2 1 = 0 < T(1) = 1$，不等式本身不成立，所以归纳只能从 $n = 2, 3$ 起步（它们的子问题规模 $\lfloor n/2 \rfloor = 1$ 直接用 $T(1) = 1$ 代入）：$T(2) = 4 \le 2c$、$T(3) = 5 \le 3c \log_2 3 \approx 4.75c$，取 $c = 2$ 都成立，此后 $n \ge 4$ 的子问题规模都 ≥ 2，上面的归纳步就能接着用。大 O 只要求 $n \ge n_0$，这样做完全合法。

下界：令 $h = \lfloor \log_2 n \rfloor$。对 $0 \le i < h$，第 i 层的子问题规模至少为 2，且该层代价 $2^i \lfloor n/2^i \rfloor \ge n - 2^i \ge n/2$。因此 $T(n) \ge (n/2)h = \Omega(n \log n)$。上下界同阶，得 $T(n) = \Theta(n \log n)$。∎

1-25. 设 f, g 为非负函数（(1) 中取对数时要求 $f, g > 0$），判断下列命题真假并证明或举反例：(1) 若 $f(n) = O(g(n))$，则 $\log f(n) = O(\log g(n))$；(2) 若 $f(n) = O(g(n))$，则 $2^{f(n)} = O(2^{g(n)})$；(3) 若 $f(n) = O(g(n))$，则 $f(n)^2 = O(g(n)^2)$。

答：(1) **不成立**。取 $f(n) = 2$、$g(n) = 1$：$f = O(g)$ 成立，但 $\log_2 f = 1$、$\log_2 g = 0$，而 $1 = O(0)$ 不成立。（若补充前提 $f, g \ge 2$ 则命题成立。）(2) **不成立**。取 $f(n) = 2n$、$g(n) = n$：$f = O(g)$，但 $2^{2n} = (2^n)^2$ 与 2^n 之比为 2^n，无界。**教训：大 O 关系不能直接放进指数**。(3) **成立**。由定义存在 c, n_0 使 $n \ge n_0$ 时 $f \le cg$，两边非负，平方得 $f^2 \le c^2 g^2$，取常数 c^2 即可。∎

1-26. 证明 $\log_2(n!) = \Theta(n \log n)$。

证明. 上界：$n! \le n^n$，故 $\log_2(n!) \le n \log_2 n = O(n \log n)$。

下界：$n! \ge \left(\frac{n}{2}\right)^{n/2}$（后一半的 $n/2$ 个因子每个都 $\ge n/2$），故 $\log_2(n!) \ge \frac{n}{2}\log_2 \frac{n}{2} = \frac{n}{2}\log_2 n - \frac{n}{2} = \Omega(n \log n)$。两界同阶。∎

用处：这正是「基于比较的排序至少要 $\Omega(n\log n)$ 次比较」那条下界的核心一步。

第 2 章 线性表

一、填空题

2-1. 线性表是 n（$n \geq 0$）个数据元素的 ____ 序列。表中除第一个元素外，每个元素有且仅有 一个 ____；除最后一个元素外，每个元素有且仅有一个 ____。

答：**有限**序列；**直接前驱**；**直接后继**。注意「有限」和「有且仅有一个」两处都是得分点；其中把线性表和树、图区分开的是「有且仅有一个」直接前驱／后继（树、图的结点集同样是有穷集合，见第 1.2.1 节）。

2-2. 长度为 n 的顺序表，在等概率前提下，插入一个元素平均需要移动 ____ 个元素，删除一个元素平均需要移动 ____ 个元素。

答：插入 $\frac{n}{2}$（由 $\frac{1}{n+1}\sum_{i=0}^{n}(n-i)=\frac{1}{n+1}\cdot\frac{n(n+1)}{2}$ 得）；删除 $\frac{n-1}{2}$（由 $\frac{1}{n}\sum_{i=0}^{n-1}(n-1-i)$ 得）。两者相差半个元素，原因是**插入有 $n+1$ 个合法位置，删除只有 n 个**。

2-3. 在带头结点的单链表 `L` 中，空表的判定条件是 ____；在不带头结点的单链表中，空表的判定条件是 ____。

答：`L->next == nullptr`；`L == nullptr`。这正是头结点的好处之一：**表头有没有元素，链表对象本身始终存在**，插入删除不必单独处理「改的是表头指针」这种特例。

2-4. 在双链表中删除指针 `p` 所指的结点（`p` 不是首尾结点），需要执行的两条指针操作是 ____ 和 ____。

答：`p->prev->next = p->next;` 和 `p->next->prev = p->prev;`（顺序无所谓，之后再释放 `p`）。若 `p` 可能是首/尾结点，则要各自判空；若用的是**带头结点的循环双链表**，就一条特例都不用判——这是循环表的主要优点。

2-5. 设每个数据域大小为 s、指针大小为 p（忽略头结点、内存对齐及分配开销）。当表的装填程度（实际元素数 / 预分配容量）小于 ____ 时，链表比顺序表省空间。

答：小于 $\frac{s}{s+p}$。推导见 2-25。直观地说：顺序表浪费在**没用上的空槽**，链表浪费在**每个结点都要带的指针**，谁更亏取决于表满到什么程度。

2-6. 在循环单链表中，判断指针 `p` 指向尾结点的条件是 ____。

答：`p->next == head`（带头结点时）或 `p->next == first`（不带头结点、`first` 为首结点指针）。对比：普通单链表的判据是 `p->next == nullptr`。

二、选择题

2-7. 在一个长度为 n 的顺序表中，在第 i 个元素之前插入一个新元素（$1 \le i \le n+1$），需要向后移动 ___ 个元素。A. $n-i$　B. $n-i+1$　C. $n-i-1$　D. i

> **答：** B。原来的第 i 个到第 n 个，共 $n-i+1$ 个元素都要后移一格。特例检查：$i=n+1$（插在末尾）时移动 0 个，符合；$i=1$（插在最前）时移动 n 个，符合。**做这类题一定要用两个端点验证公式**，凭印象写 $n-i$ 是最常见的错法。

2-8. ★ 已知 p 指向带头结点单链表中的某个结点（非尾结点），要在 p 之后插入结点 s，下列语句序列正确的是（　）。A. `s->next = p->next; p->next = s;` B. `p->next = s; s->next = p->next;` C. `s->next = p; p->next = s;` D. `p->next = s->next; s->next = p;`

> **答：** A。B 的两句顺序颠倒了：先让 `p->next` 指向 s，原来的后继就丢了，第二句 `s->next = p->next` 等于 `s->next = s`，链表当场成环。**口诀：先接新结点的后继，再改前驱的指针**——永远先连「还没人指着的那一端」。

2-9. ★ 设 p 指向单链表中的某结点，下列操作**不能**在 $O(1)$ 时间内完成的是（　）。A. 删除 p 的后继结点 B. 在 p 之后插入一个结点 C. 删除 p 本身（单链表，仅给 p）D. 访问 p 的数据域

> **答：** C。删除 p 本身需要修改其**前驱**的指针，而单链表无法由 p 反查前驱，只能从头找，$O(n)$。补充一个常考的「例外」：若 p **不是尾结点**，可以用「偷天换日」在 $O(1)$ 内做到—— 把后继的数据复制到 p，再删掉后继（见 2-20）。但 p 是尾结点时此法失效。

2-10. 下列关于顺序表与链表的比较，**正确**的是（　）。A. 顺序表查找任意元素都比链表快 B. 链表的插入删除一定比顺序表快 C. 顺序表适合表长稳定、以随机访问为主的场合 D. 链表不需要预先估计表长，所以任何情况下都更优

> **答：** C。A 错：按值查找两者都是 $O(n)$，顺序表只在**按位置**访问上有 $O(1)$ 的优势。B 错：链表插入删除本身是 $O(1)$，但**前提是已经拿到了目标位置的指针**；若还要先定位，定位那一步仍是 $O(n)$，整体并不快。D 错：链表的指针开销与不连续存储对缓存不友好，都是代价。

2-11. 带头结点的循环双链表 `L` 为空的判定条件是（　）。A. `L == nullptr`　B. `L->next == nullptr` C. `L->next == L`　D. `L->prev == nullptr`

> **答：** C。循环表里指针永不为空，头结点自己指自己即为空表。

2-12. ★ 在带头结点的循环双链表中，p 指向某个结点，要在 p 之后插入结点 s。下列语句序列中**错误**的是（　）。A. `s->prev = p; s->next = p->next; p->next->prev = s; p->next = s;` B. `s->next = p->next; p->next->prev = s; s->prev = p; p->next = s;` C. `s->prev = p; s->next = p->next; p->next = s; p->next->prev = s;` D. `s->next = p->next; s->prev = p; p->next = s; s->next->prev = s;`

答：C。四个序列改的是同样四个指针域，s 自己的两个域怎么排都无害，要紧的是另外两句的先后。C 先执行了 `p->next = s`，第四句的 `p->next->prev = s` 就成了 `s->prev = s`：s 的前驱指向它自己，原后继的 prev 却仍指着 p，从后往前走会跳过 s。A、B 都是**先借 `p->next` 修好原后继的 `prev`，最后才改 `p->next`**；D 的第四句改从 `s->next` 找原后继（第一句已把它存进去），不再依赖 `p->next`，所以放在最后也对。**记法**：`p->next` 是通往原后继的唯一一条路，**最后一个改它**；或者先把原后继存进 `s->next`，之后都从 `s->next` 走。这和单链表那条口诀（2-8）是同一个道理。循环表的好处是 p 为尾结点时 `p->next` 是头结点而不是空指针，四句话不需要特判。

三、辨析与简答

2-13. 辨析：「链表比顺序表节省存储空间。」

答：不一定。分两块看：- **顺序表**要预分配一块连续空间，浪费在**没用上的空槽**上；用满时利用率 100%。- **链表**按需分配，没有空槽，但**每个结点都要额外存一个（双链是两个）指针**。

所以谁省取决于装填程度：当装填程度低于 $s/(s+p)$ 时链表省（s 为数据大小，p 为指针大小），高于它则顺序表省。数据元素越大（s 越大），链表的相对开销越小。此外顺序表还有**缓存局部性**的隐性优势，这一条在渐进分析里看不出来。

2-14. 简答：头结点（附加表头结点）有什么作用？不带头结点会多出哪些特例？

答：头结点是一个不存数据、只占位置的结点，作用是**让「表头」和「表中」在代码里长得一样**。不带头结点时，至少多出三类特例：(1) 在表头插入——要改的是链表对象里的 `first` 指针，而不是某个结点的 `next`；(2) 删除首结点——同上；(3) 空表——`first` 为空，很多循链代码要先判一次空。带了头结点，这三类都退化成普通情形，代价只是一个结点的空间。

2-15. ★ 简答：从**查找、插入删除、空间、适用场合**四个方面比较顺序表和链表。

答：

维度	顺序表	链表
按位置访问	$O(1)$（下标直接算地址）	$O(n)$（必须循链）
按值查找	$O(n)$；有序时可二分 $O(\log n)$	$O(n)$；**有序时二分仍需 $O(n)$ 时间**
插入/删除	$O(n)$（平均移动一半元素）	已知位置指针时 $O(1)$，否则定位 $O(n)$

维度	顺序表	链表
空间	需预估容量，有空槽浪费；无指针开销	按需分配，无空槽；每结点多一个指针
适用	表长稳定、以读为主、需要随机访问	表长变化大、频繁在已知位置插删

一句话结论：**顺序表把代价付在「改」上，链表把代价付在「找」上。**有序链表可以按折半思路减少比较次数，但循链定位中点的总时间仍是 $O(n)$，无法获得数组二分的 $O(\log n)$ 时间。

四、算法设计与分析

本组题统一使用如下单链表 ADT（考试时也建议先写这一段，再写算法）：

```
结点 Node
    data    元素值
    next    指向下一结点的指针，尾结点为 nullptr

链表 List（带头结点，head 指向头结点）
    Head()              出口：头结点指针
    Empty()             出口：表空返回真
    MakeNode(x)         入口：元素值 x；出口：新结点指针，data 为 x、next 为空
    FreeNode(p)         入口：结点指针 p；功能：释放该结点
```

2-16. ★ 设计算法把带头结点的单链表**就地逆置**（不申请新结点），并分析时空代价。

答：**基本思想**——把所有结点从原链上摘下来，逐个「头插」回去。头插天然产生逆序。

```
Reverse(L)
    p ← L.head->next        // 摘下整条链，原表先变空
    L.head->next ← nullptr
    while p ≠ nullptr
        q ← p->next         // 先存后继，否则下一句一改就找不回来了
        p->next ← L.head->next
        L.head->next ← p
        p ← q
```

正确性关键：循环不变式是「`L.head` 之后是已处理结点的逆序，`p` 指向未处理部分的首结点」。每轮都必须**先用 `q` 存好** `p->next`——这是本题唯一的指针陷阱，漏掉它链表就断在中间。

代价：时间 $\Theta(n)$，每个结点只被摘挂一次；空间 $\Theta(1)$，只用了 `p`、`q` 两个指针。如果允许新建结点或用数组暂存，空间就退化成 $\Theta(n)$，本

题的「就地」二字要的正是 $\Theta(1)$。

2-17. ★ 已知两个带头结点的**递增有序**单链表 A、B，设计算法把它们归并成一个递增有序表 C，要求**不申请新结点**，且结果中相同的值只保留一个。

答：**基本思想**——双指针同步扫描，每次把较小者摘下接到 C 的尾部；相等时只接一个、另一个释放。因为两表都已有序，谁也不必回退。

```
Merge(A, B)
    C ← A.head              // 复用 A 的头结点
    r ← C                   // r 始终指向 C 的当前尾结点
    pa ← A.head->next
    pb ← B.head->next
    C->next ← nullptr
    while pa ≠ nullptr or pb ≠ nullptr
        if pb = nullptr or (pa ≠ nullptr and pa->data ⩽ pb->data)
            t ← pa;  pa ← pa->next
        else
            t ← pb;  pb ← pb->next
        if r ≠ C and r->data = t->data
            FreeNode(t)                 // 也去掉同一输入表内部的重复值
        else
            r->next ← t;  r ← t
            r->next ← nullptr
    FreeNode(B.head)
    return C
```

代价：时间 $\Theta(m+n)$，空间 $\Theta(1)$。**评分要点**：(1) 尾指针 `r` 必须有，否则每次接尾都要走一遍，退化成 $O(n^2)$；(2) 剩余部分也要逐个检查重复值，不能未经检查就整体接上；(3) 释放重复结点前，必须先保存它的后继并推进相应输入指针。

变式：若要求归并成**递减**有序表，可先按上述算法归并去重，再按 2-16 就地逆置，渐进代价不变——这和 2-16 是同一个技巧。

2-18. 设计算法在带头结点的单链表中查找**倒数第 k 个**结点，要求只遍历一趟、空间 $O(1)$。

答：**基本思想**——双指针拉开固定间距。先让 `fast` 从首结点走 k 步，再让 `slow` 与 `fast` 同步前进；`fast` 走到表尾时，两者间距恰为 k，`slow` 即为所求。

```
FindKthFromEnd(L, k)
    if k < 1 then return nullptr
    fast ← L.head->next
    for i ← 1 to k
        if fast = nullptr then return nullptr      // 表长不足 k
        fast ← fast->next
    slow ← L.head->next
```

```
    while fast ≠ nullptr
        fast ← fast->next
        slow ← slow->next
    return slow
```

代价：时间 $\Theta(n)$（fast 只走一趟），空间 $\Theta(1)$。**边界**：k 大于表长时第一个循环里就返回空；$k=1$ 时返回尾结点，可代入验证。

2-19. 设计算法判断一个单链表中是否存在环，若存在则返回入环的第一个结点。要求空间 $O(1)$。

答：**基本思想**（快慢指针）——slow 每次一步、fast 每次两步。若无环，fast 必先到达表尾；若有环，fast 会在环内追上 slow。

找入环点用如下结论：设表头到入环点距离为 a、入环点到相遇点为 b、环长为 c。相遇时 slow 走了 $a+b$，fast 走了 $a+b+kc$，又 fast 的路程是 slow 的两倍，故 $a+b=kc$，即 $a=kc-b$——**从表头和从相遇点同时每次走一步，必在入环点相遇**。

```
FindCycleEntry(L)
    slow ← fast ← L.head->next
    while fast ≠ nullptr and fast->next ≠ nullptr
        slow ← slow->next
        fast ← fast->next->next
        if slow = fast                      // 有环
            p ← L.head->next
            while p ≠ slow
                p ← p->next;  slow ← slow->next
            return p
    return nullptr                           // 无环
```

代价：时间 $O(n)$，空间 $O(1)$。若允许 $O(n)$ 空间，用散列表记下走过的结点地址也可以，但那就不是本题要考的东西了。

2-20. ★ 已知 p 指向单链表中的某个结点（不是头结点也不是尾结点），要求在 $O(1)$ 时间内 删除 p。给出算法，并说明它为什么对尾结点无效。

答：**基本思想**——既然拿不到前驱，就**让 p 变成它的后继**：把后继的数据抄进 p，再把后继摘掉。链表在逻辑上等价于删掉了 p。

```
DeleteNodeInPlace(p)
    q ← p->next
    p->data ← q->data
    p->next ← q->next
    FreeNode(q)
```

为什么尾结点不行：尾结点没有后继可供顶替，而删它必须修改前驱的 next，单链表无法在 $O(1)$ 内取得前驱，只能从头扫描，$O(n)$。**另**

一个隐患：如果外部还有别的指针指着 q，此法会让那个指针变成野指针——这是「偷天换日」的真实代价，答题时值得写上一句。

2-21. ★ 双链表中结点 p 的前驱和后继都存在，且它的后继不是尾结点。设计算法把 p 与它的**后继**交换位置，要求只改指针、不动数据域；并说明为什么这些语句的先后次序不能随意调换。

答：**基本思想**——不要去想「一口气把六根指针改对」，写成两步：**先把 p 从链上摘下来，再把它插到原后继之后**。每一步都是已经会的基本操作（摘见 2-4，插见 2-8）。

```
SwapWithNext(p)
    q ← p->next                    // 第一句就得把后继记下来
    // 第一步：摘掉 p，让 q 顶上 p 原来的位置
    p->prev->next ← q
    q->prev ← p->prev
    // 第二步：把 p 插到 q 之后
    p->next ← q->next
    q->next->prev ← p
    q->next ← p
    p->prev ← q
```

正确性：设原来依次是 A（p 的前驱）、B（就是 p）、C（p 的后继）、D，目标是 $A \leftrightarrow C \leftrightarrow B \leftrightarrow D$。第一步做完是 $A \leftrightarrow C$；第二步把 B 接在 C 与 D 之间。六根指针一根不漏。

为什么次序要紧，三条理由：1. `q ← p->next` **必须是第一句**。后面 `p->next` 会被改写，改完就再也找不回原来的后继；2. `q->next->prev ← p` **必须在** `q->next ← p` **之前**。一旦 `q->next` 改成了 B，`q->next->prev` 指的就成了 B 自己的 `prev` 域，这句会把 `p->prev` 改成 p 本身，而 D 的 `prev` 根本没被改到；3. 每条语句读到的 `p->prev` / `p->next` 都是**执行到那一刻的值**——这类题只能一句一句代进去核，不能整体看。**先接还没人指着的那一端**（2-8 的口诀）在这里同样管用。

代价：时间 $O(1)$，空间 $O(1)$。

另一种合法做法是直接交换数据域（`swap(p->data, q->data)`），$O(1)$ 且不可能接错。但外部若还有指针指着某个结点并认准它的值，这种做法会骗过它们——与 2-20 的「偷天换日」是同一类代价。题目要求「只改指针」时不能用它。

2-22. 设计算法删除顺序表中所有值等于 x 的元素，要求时间 $O(n)$、空间 $O(1)$。

答：**基本思想**——用一个写指针 k 记录「已保留的元素个数」，读指针扫一遍，不等于 x 的就写到 k 处。这是顺序表上的「原地压缩」通用套路。

```
RemoveAll(A, n, x)
    k ← 0
    for i ← 0 to n - 1
        if A[i] ≠ x
            A[k] ← A[i]
            k ← k + 1
    return k                  // 新表长
```

代价：时间 $\Theta(n)$（每个元素只读写一次），空间 $\Theta(1)$。**对照错法**：每遇到一个 x 就把后面的整体前移一格，最坏 $O(n^2)$——这正是本题要避开的。

五、数学证明

2-23. 证明：在长度为 n 的顺序表中等概率地插入一个元素，平均移动元素数为 $n/2$。

证明. 插入位置有 $n+1$ 个（在下标 0 的元素之前直到下标 $n-1$ 的元素之后），等概率即每个为 $\frac{1}{n+1}$。插在位置 i（$0 \le i \le n$）时需要后移 $n-i$ 个元素，故平均移动数

$$E = \frac{1}{n+1}\sum_{i=0}^{n}(n-i) = \frac{1}{n+1}\cdot\frac{n(n+1)}{2} = \frac{n}{2}$$

∎ 同理删除时位置有 n 个，平均移动 $\frac{1}{n}\sum_{i=0}^{n-1}(n-1-i) = \frac{n-1}{2}$。

2-24. 证明：对有序的单链表，二分检索无法达到 $O(\log n)$。

证明. 二分检索的每一步都要求在 $O(1)$ 时间内取到当前区间的中间元素。单链表只能从表头沿 `next` 逐个前进，取第 k 个元素需要 $\Theta(k)$ 时间。因此即使每次都正确地把区间折半，第一次取中点就已经花了 $\Theta(n/2)$，总代价至少是 $\Theta(n)$，即 $\Omega(n)$，不可能是 $O(\log n)$。∎

推论：**要获得通常的 $O(\log n)$ 二分检索时间，需要高效的随机访问，普通单链表不具备这一条件**。这也是第 10 章讲检索时必须先讨论存储结构的原因。

2-25. 设每个数据元素占 s 个存储单元、每个指针占 p 个单元，顺序表预分配容量为 m、实际存 n 个元素。忽略头结点、内存对齐及分配开销，求链表比顺序表省空间的条件。

证明. 顺序表占用 $m \cdot s$；单链表占用 $n(s+p)$。链表更省当且仅当

$$n(s+p) < m\,s \quad\Leftrightarrow\quad \frac{n}{m} < \frac{s}{s+p}$$

即**装填程度 n/m 小于 $\frac{s}{s+p}$** 时链表省空间。∎

代入感受一下：$s = p$（元素和指针一样大，如 32 位机上存一个 `int`）时阈值是 $1/2$，即顺序表填过一半 链表就吃亏了；$s = 10p$（元素是个大记录）时阈值约 0.91，装填程度低于约 0.91 时链表更省空间。**结论：元素越大，链表的指针开销越不值一提**。

第 3 章 栈与队列

一、填空题

3-1. 栈是限定只能在 ____ 进行插入和删除的线性表，其特点是 ____；队列是限定在 ____ 插入、在 ____ 删除的线性表，其特点是 ____。

答：**表的一端（栈顶）**；**后进先出**（LIFO）；**表的一端（队尾）**；**另一端（队头）**；**先进先出**（FIFO）。两者都是**限制存取点的线性表**（第 3.3.3 节把它们与双栈、超队列、超栈一并看作受限的双端队列）—— 把线性表的存取点限制得越死，能提供的运算越少，但实现可以越高效。

3-2. ★ 设循环队列的数组容量为 M（牺牲一格，最多存 $M-1$ 个元素），`front` 指向队头元素、`rear` 指向队尾元素的下一个位置。则：队空的条件是 ____，队满的条件是 ____，队中元素个数为 ____。

答：队空 `front == rear`；队满 `(rear + 1) % M == front`；元素个数 `(rear - front + M) % M`。元素个数公式里的 `+ M` 是为了让 `rear < front`（已绕回）时余数非负，**不能省**。

3-3. ★ 设有 n 个元素按 $1, 2, \cdots, n$ 的顺序依次入栈（中途可任意出栈），则可能得到的不同出栈序列共有 ____ 种。

答：$\frac{1}{n+1}C_{2n}^{n}$，即第 n 个 Catalan 数。$n = 1, 2, 3, 4, 5$ 时依次是 $1, 2, 5, 14, 42$。考试里最常出的是 $n = 3$（5 种）和 $n = 4$（14 种），建议把这两个数背下来。推导见 3-26。

3-4. 顺序栈用数组 `data[0..m-1]` 存储，若约定 `top` 指向**栈顶元素**，则空栈时 `top` 为 ____，入栈的两步是 ____；若约定 `top` 指向**栈顶元素的上一个空位**，则空栈时 `top` 为 ____。

答：`-1`；先 `++top` 再 `data[top] = x`（先加后存）；`0`（此时入栈是先存后加：`data[top++] = x`）。两种约定都对，**但一份代码里必须自始至终只用一种**；混用是顺序栈最常见的越界来源。

3-5. ★ 递归调用时，系统运行栈中每个活动记录（栈帧）主要保存三样东西：____、____、____。

答：**返回地址**（这一层返回后从哪继续执行）、**实在参数、本层的局部变量**（以及需要保护的寄存器现场）。手工用栈消除递归时要压进栈的，就是这三样里**本层还要再用到**的那部分—— 不再用到的可以不存，这正是优化的空间（见第 3.1.5 节背包问题的「优化」一段）。

3-6. 中缀表达式 `A + B * (C - D) / E` 的后缀形式是 ____。

答：`A B C D - * E / +`。检验办法：后缀式里运算符出现的**先后顺序**必须与实际求值顺序一致—— 这里是先 `-`，再 `*`，再 `/`，最后 `+`，与中缀式的优先级和括号完全吻合。

二、选择题

3-7. ★ 设入栈序列为 `1,2,3,4`，下列**不可能**是出栈序列的是（　）。A. `1,2,3,4`　B. `4,3,2,1`　C. `3,1,2,4`　D. `2,4,3,1`

答： C。用判定法则最快：**合法出栈序列中不存在下标** $i<j<k$ **使** $p_j<p_k<p_i$（即不含「312」型）。C 中取 $p_1=3,p_2=1,p_3=2$，满足 $1<2<3$，故非法。手工验证：要先输出 3，则 1、2 必定还压在栈里且 2 在 1 之上，于是它们只能按 `2,1` 的顺序出来，绝不可能是 `1,2`。证明见 3-27。

3-8. 容量为 M 的循环队列（牺牲一格），`front = 3`、`rear = 0`、$M=8$，则队中元素个数是（　）。A. 3　B. 5　C. 0　D. 8

答： B。$(0-3+8)\bmod 8=5$。若误用 `rear - front` 会得到 -3，这正是公式里 `+ M` 的作用。

3-9. 下列关于循环队列的叙述正确的是（　）。A. 循环队列解决的是队列「真溢出」问题 B. 循环队列必须牺牲一个存储单元，没有别的办法 C. 循环队列解决的是「假溢出」问题，即队尾到达数组末端而前端仍有空闲 D. 循环队列中 `front == rear` 一定表示队满

答： C。A 错：数组真的满了照样溢出，循环队列只是不再浪费前端空闲。B 错：另设一个计数器 `count` 或一个「上次操作是入队还是出队」的布尔标志同样可以区分空与满，牺牲一格只是最常用的做法（见 3-14）。D 错：牺牲一格的约定下 `front == rear` 表示**队空**。

3-10. 后缀表达式 `3 4 + 5 2 - *` 的值是（　）。A. 21　B. 17　C. 35　D. 9

答： A。求值过程：`3`、`4` 入栈；遇 `+` 弹出两个算 $3+4=7$ 入栈；`5`、`2` 入栈；遇 `-` 算 $5-2=3$ 入栈；遇 `*` 算 $7\times 3=21$。**注意减法、除法的操作数顺序**：先弹出的是右操作数，后弹出的是左操作数，写成 `右 = pop(); 左 = pop(); 结果 = 左 - 右`，弄反就会算成 $2-5$。

3-11. 用两个栈 S_1、S_2 模拟一个队列，下列说法正确的是（　）。A. 每次出队都要把 S_1 全部倒进 S_2，故出队一定是 $O(n)$ B. 入队 $O(1)$，出队的**摊还**代价是 $O(1)$ C. 无法用两个栈模拟队列 D. 需要三个栈才能模拟

答： B。入队压 S_1；出队时若 S_2 非空直接弹，空了才把 S_1 整个倒进 S_2。每个元素**一生只被倒一次**，n 次操作总代价 $O(n)$，故摊还 $O(1)$。A 的错误在于把「单次最坏」当成了「每次都这样」。

三、辨析与简答

3-12. 辨析：「栈和队列都是线性表，所以它们的存储结构也只能和线性表一样。」

答：前半句对，后半句说法不准。栈和队列确实是**运算受限的线性表**，

顺序和链接两大类 存储方式也都适用。但**受限本身带来了线性表没有的实现手法**：顺序队列必须把数组看成环（循环队列），否则出队后前端的空间收不回来；链式队列为了 $O(1)$ 入队必须额外保存队尾指针。这些都是限制存取点之后才出现的设计。

3-13. ★ 辨析：「链式栈没有容量上限、不会上溢，所以总是优于顺序栈。」从时间和空间两方面比较二者。

答：错。两者的入栈、出栈、取栈顶都是 $O(1)$，差别在常数和空间上：

	顺序栈	链式栈
入栈、出栈	$O(1)$；栈满扩容时要搬移一次 $O(n)$，按倍增扩容（算法 3.3）摊还仍是 $O(1)$	$O(1)$，但每次都要申请或释放一个结点，常数较大
空间	预分配一整块，**没用上的空槽**是浪费；元素本身没有额外开销	按需分配，没有空槽；**每个元素多一个指针**，另有分配器的管理开销
容量	定长时可能上溢，须预估容量或扩容	只受可用内存总量限制
其他	连续存储，缓存友好；两个栈可共享一个数组，栈底分设两端、相向增长	栈顶放在链表**表头**，入栈出栈都只动表头，不需要尾指针

结论：元素个数大致能预估时选顺序栈（省指针、速度快）；个数波动大、难以预估，或多个栈要共用空闲内存时选链式栈。「不会上溢」确实是链式栈的长处，但它是用每个元素一个指针、每次操作一次内存分配换来的。

3-14. ★ 简答：为什么循环队列中 `front == rear` 既可能表示空、也可能表示满？列举至少两种区分办法并比较。

答：**根本原因是状态数对不上**。M 个位置的数组，队列的元素个数有 $0, 1, \cdots, M$ 共 $M+1$ 种状态；而固定 `front` 后 `rear` 只有 M 种取值，鸽笼原理保证必有两种状态撞在一起—— 撞上的恰是「空」和「满」。

三种区分办法：

办法	判空	判满	代价
牺牲一格	`front == rear`	`(rear+1)%M == front`	少存一个元素，判据最简

办法	判空	判满	代价
另设计数器 `count`	`count == 0`	`count == M`	能存满 M 个，每次入出队都要维护计数
另设标志位 `tag`	`front==rear && tag==0`	`front==rear && tag==1`	能存满，判据稍绕

教材与多数教学实现取第一种。

3-15. ★ 简答：什么是递归？用栈消除递归的一般方法是什么？哪些递归可以不用栈就改成循环？

答：**递归**是函数（直接或间接）调用自身的求解方式，靠系统运行栈自动保存每一层的活动记录。

用栈消除递归的一般方法（见第 3.1.5 节）：1. 设计一个「活动记录」结构，存放本层**返回后还要用到**的信息：参数、局部变量、以及「返回后该从哪一步继续」的**返回点编号**；2. 开一个该结构的栈，把初始调用的参数压栈；3. 主循环：弹出栈顶记录，按返回点编号跳到相应分支执行；遇到「递归调用」的地方，先把**本层剩下要做的事**压回栈（改好返回点编号），再把**子问题的参数**压栈，然后回到主循环；4. 栈空时算法结束。

可以不用栈的情形：- **尾递归**：递归调用是函数的最后一个动作，返回后无事可做，直接改成「更新参数 + 循环」即可，空间从 $O(\text{深度})$ 降到 $O(1)$；- **单向线性递归**（如求阶乘、求链表长度）：本质上就是尾递归的变形，可直接写成循环；- **（部分消除）两处调用互不依赖、其后无事可做的双分支递归**（如快排）：不能完全去掉栈，但可把其中一侧改成循环——先递归处理**较小的那一侧**、对较大的一侧用循环迭代，栈深由 $O(n)$ 降到 $O(\log n)$。

按一般方法模拟多分支递归时：像汉诺塔、二叉树遍历，返回之后还有后续动作，通常用显式栈保存续接状态；这不排除利用问题特性设计其他非递归算法（如 Morris 遍历）。

3-16. 简答：递归改成非递归以后，代价一定变小吗？

答：不一定。时间上，消除了函数调用的开销（保存现场、跳转），常数变小；但**渐进阶不变**，该指数还是指数（斐波那契的朴素递归改成显式栈仍是指数级，真正救它的是记忆化，不是消递归）。空间上，递归吃的是**运行栈**（通常只有几 MB，深了就崩），显式栈吃的是**堆**（可大得多），所以改写的主要收益是**把「栈溢出直接崩溃」换成「可控的容量检查」**，而不是省空间。

四、算法设计与分析

3-17. ★ 写出栈和队列的 ADT，并给出顺序栈 `Push`、`Pop` 的实现（含溢出处理）。

答：

```
ADT Stack（元素类型 T）
    Push(x)      入口：元素 x；功能：x 成为新栈顶；栈满时报上溢
    Pop()        出口：原栈顶元素；功能：删除栈顶；栈空时报下溢
    Top()        出口：栈顶元素（不删除）；栈空时报错
    Empty()      出口：栈空返回真
    Clear()      功能：置为空栈

ADT Queue（元素类型 T）
    EnQueue(x)   入口：元素 x；功能：x 接到队尾；队满时报上溢
    DeQueue()    出口：原队头元素；功能：删除队头；队空时报下溢
    Front()      出口：队头元素（不删除）
    Empty()      出口：队空返回真
```

顺序栈（`data[0..m-1]`，`top` 指向栈顶元素，空栈 `top = -1`）：

```
Push(x)
    if top = m - 1 then 报上溢；返回          // 必须先查，再写
    top ← top + 1
    data[top] ← x

Pop()
    if top = -1 then 报下溢；返回
    x ← data[top]
    top ← top - 1
    return x
```

评分要点：溢出检查必须在写数组**之前**；`Pop` 要先取值再减 `top`。原书代码 3.2 这两处的次序是对的；它的毛病在别处（数据成员 `top` 与成员函数 `top()` 重名、缺拷贝控制导致二次释放、容器内输出），见本书第 3.1.2 节与「与原书的对照」。

3-18. ★ 写出循环队列的完整实现（判空、判满、入队、出队、求长度），采用「牺牲一格」方案。

答：设数组 `data[0..M-1]`，`front` 指向队头元素，`rear` 指向队尾元素的**下一个**位置。初始 `front = rear = 0`，最多存 $M-1$ 个元素。

```
Empty()          return front = rear
Full()           return (rear + 1) mod M = front
Length()         return (rear - front + M) mod M

EnQueue(x)
    if Full() then 报上溢；返回
    data[rear] ← x
    rear ← (rear + 1) mod M        // 关键：一切下标推进都要取模
```

```
DeQueue()
    if Empty() then 报下溢；返回
    x ← data[front]
    front ← (front + 1) mod M
    return x
```

代价：五个运算全是 $O(1)$，空间 $O(M)$。**评分要点**：(1) `mod M` 一处都不能漏；(2) 判满判空的条件要与 `rear` 的含义一致——若改成「`rear` 指向队尾元素」，则空为 `front == (rear+1) % M`，两套约定不能混；(3) 要说明牺牲一格的理由（见 3-14）。

变式（用计数器，可存满 M 个）：另设 `count`，`Empty()` 为 `count == 0`、`Full()` 为 `count == M`，入队 `count++`、出队 `count--`，`Length()` 直接返回 `count`。

3-19. ★ 设入栈序列固定为 $1, 2, \cdots, n$，给定一个 $1..n$ 的排列 p，设计算法判断 p 是否为一个合法的出栈序列。分析代价。

答：**基本思想**——直接用一个栈**模拟**：按 $1..n$ 的顺序依次入栈，每次入栈后，只要栈顶等于 p 中下一个待匹配的元素就一直弹出。全部入栈完毕后若 p 也匹配完，则合法。

```
IsValidPopSequence(p, n)
    S ← 空栈
    j ← 0                                  // p 中下一个待匹配的下标
    for i ← 1 to n
        S.Push(i)
        while not S.Empty() and S.Top() = p[j]
            S.Pop()
            j ← j + 1
    return S.Empty()
```

正确性：模拟过程中，每个时刻「什么时候必须弹」是唯一确定的——若栈顶恰是当前需要输出的元素，继续压入新元素会挡住它；要维持指定的输出次序，必须先弹出它。因此贪心地「能弹就弹」不会错过任何解。

代价：时间 $\Theta(n)$（每个元素入栈一次、出栈至多一次），空间 $\Theta(n)$。

另一种写法（判定法则）：检查是否存在 $i < j < k$ 使 $p_j < p_k < p_i$，存在即非法。朴素实现 $O(n^3)$，用单调栈可优化到 $O(n)$——但考试里直接写模拟更稳妥。

3-20. ★ 写出中缀表达式转后缀表达式的算法，并用它把 `A + B * C - (D / E + F) * G` 转成后缀式。

答：**基本思想**——操作数直接输出；运算符进一个栈，进栈前先把栈

里**优先级不低于它**的 运算符全部弹出并输出；左括号无条件进栈、并在栈内视为最低优先级；右括号则一直弹到左括号为止。

```
InfixToPostfix(E)
    S ← 空栈（存运算符）
    for 每个记号 t in E
        if t 是操作数         then 输出 t
        else if t = '('       then S.Push(t)
        else if t = ')'       then
            while S.Top() ≠ '('  输出 S.Pop()
            S.Pop()                          // 丢掉那个左括号
        else                                 // t 是双目运算符
            while not S.Empty() and S.Top() ≠ '(' and 优先级 (S.Top()) ⩾
↪  优先级 (t)
                输出 S.Pop()
            S.Push(t)
    while not S.Empty()  输出 S.Pop()
```

优先级：`*`、`/` 高于 `+`、`-`；括号内的运算符先算，靠「左括号在栈内最低」自动实现。（若要支持右结合的乘方 `^`，把条件里的 ⩾ 对 `^` 改成 `>`。）

本题答案：`A B C * + D E / F + G * -`

推演（左列是当前记号，右列是栈内容，栈顶在右）：

记号	输出	栈
A	A	
+		+
B	B	+
*		+ *
C	C	+ *
-	* +	-
(		- (
D	D	- (
/		- (/
E	E	- (/
+	/	- (+
F	F	- (+
)	+	-
*		- *
G	G	- *
结束	* -	

代价：时间 $\Theta(n)$，空间 $O(n)$。

3-21. 写出后缀表达式求值算法，并求 `A B C * + D E / F + G * -` 在 `A=2, B=3, C=4, D=8, E=2, F=1, G=3` 时的值。

答：**基本思想**——一个操作数栈，扫描后缀式：操作数入栈；遇到双目运算符就弹出两个，**先弹的是右操作数**，算完把结果压回。扫描结束时栈中唯一的元素就是答案。

```
EvalPostfix(E)
    S ← 空栈
    for 每个记号 t in E
        if t 是操作数 then S.Push(值 (t))
        else
            b ← S.Pop()        // 右操作数
            a ← S.Pop()        // 左操作数
            S.Push(a t b)
    return S.Pop()
```

求值：$B*C=12$；$A+12=14$；$D/E=4$；$4+F=5$；$5*G=15$；$14-15=-1$。**答案** -1。

代价：时间 $\Theta(n)$，空间 $O(n)$。**评分要点**：必须写明两个操作数的弹出顺序，否则减法除法会算反。

3-22. ★ 用栈把下面的递归算法改写成非递归算法，并说明栈中保存了什么。

```
Hanoi(n, A, B, C)          // 把 n 个盘子从 A 经 B 移到 C
    if n = 0 then return
    Hanoi(n - 1, A, C, B)
    输出 "A → C"
    Hanoi(n - 1, B, A, C)
```

答：本题有**两处递归调用**，且第一处返回后还要做事，本题按要求用显式栈模拟。

栈中保存什么：一个活动记录 (n, A, B, C, tag)，其中 `tag` 是**返回点编号**——记录「这一层做到哪一步了」：`tag=0` 表示三步都没做，`tag=1` 表示第一处递归已回来、该输出并展开第二处递归。参数 n, A, B, C 必须存，因为返回后还要用它们组织第二个子问题。

```
HanoiIter(n, A, B, C)
    S ← 空栈
    S.Push((n, A, B, C, 0))
    while not S.Empty()
        (n, A, B, C, tag) ← S.Pop()
        if n = 0 then continue
        if tag = 0
            S.Push((n, A, B, C, 1))        // 本层还没做完，改好返回点压回去
            S.Push((n - 1, A, C, B, 0))    // 再压子问题，保证它先被处理
```

```
        else
            输出 "A → C"（用本记录里的 A、C）
            S.Push((n - 1, B, A, C, 0))
```

关键点：压栈顺序必须是「先压本层的后续、再压子问题」——栈是后进先出，这样才能保证子问题先于本层后续被执行，与递归的次序一致。

代价：时间 $\Theta(2^n)$（与递归版相同，消递归不改变渐进阶），栈深 $O(n)$，但用的是堆上的显式栈，容量可查可控，不会像运行栈那样直接崩。

对照：尾递归可以不用栈。例如 `Fact(n) = if n ⩽ 1 then 1 else n * Fact(n-1)` 虽然不是严格尾递归，但它是**单向线性递归**，可直接写成 `r ← 1; for i ← 2 to n: r ← r * i`，空间 $O(1)$；而 `Gcd(a,b) = if b = 0 then a else Gcd(b, a mod b)` 是标准尾递归，直接改成 `while b ≠ 0: (a, b) ← (b, a mod b)`。**判断标准就一条：递归调用返回之后还有没有事要做**。

3-23. 设计算法用**一个数组**实现两个栈，使得只有在整个数组真正用完时才发生上溢。

答：**基本思想——两个栈从数组的两端相向增长**（共享栈）。栈 0 的栈顶 `top0` 从 -1 向右增，栈 1 的栈顶 `top1` 从 m 向左减。

```
Push(i, x)                          // i = 0 或 1
    if top0 + 1 = top1 then 报上溢; 返回        // 两栈顶相邻即真正满了
    if i = 0 then top0 ← top0 + 1;  data[top0] ← x
    else          top1 ← top1 - 1;  data[top1] ← x

Pop(i)
    if i = 0
        if top0 = -1 then 报下溢; 返回
        x ← data[top0];  top0 ← top0 - 1
    else
        if top1 = m then 报下溢; 返回
        x ← data[top1];  top1 ← top1 + 1
    return x
```

为什么这样最省：若把数组对半分给两个栈，一个栈满了另一个还空着也得报溢出；相向增长则让两个栈**共享同一片剩余空间**，只有总和用尽才溢出。**注意**：「相向增长」只对**两个**栈有效；三个以上的栈无法都靠端点，通常改为分区（某区满而数组仍有空闲时整体挪动各区，代价是搬移元素）或用链式栈。

3-24. 设计算法判断一个字符序列是否为回文（正读反读相同），要求用栈和队列各一个。

答：**基本思想**——同一串字符同时进栈和进队列。出栈得到的是**逆序**，

出队得到的是**正序**；逐个比较，全部相等即回文。

```
IsPalindrome(s)
    S ← 空栈；  Q ← 空队列
    for 每个字符 c in s
        S.Push(c);  Q.EnQueue(c)
    while not S.Empty()
        if S.Pop() ≠ Q.DeQueue() then return false
    return true
```

代价：时间 $\Theta(n)$，空间 $\Theta(n)$。只用栈也能做（把前一半压栈，再与后一半比），实际存储的字符约减半（渐进空间仍为 $\Theta(n)$），但要处理奇偶长度的中点，本题按题目要求用栈加队列，写法最干净。

3-25. ★ **算法填空**：用两个队列实现一个栈。队列只提供 `push(x)` 入队、`pop()` 队首出队、`front()` 取队首、`empty()` 判空（**没有求长度的运算**）。下面这一版让 `push` 保持 $O(1)$，把代价放在 `pop` 上；调用 `pop` 的前提是栈非空。补全五个空。

```
class MyStack
    queue q1, q2                        // 元素平时都在 q1 里

    push(x)
        ____(1)____

    pop()                               // 弹出最后压入的那个
        while ____(2)____
            t ← q1.front();  q1.pop()
            if ____(3)____  then  r ← t        // 走到最后一个，它就是答案
            else                  q2.push(t)
        ____(4)____                     // 让 q1 重新成为存元素的那个
        return r

    empty()
        return ____(5)____
```

答：(1) `q1.push(x)`　(2) `not q1.empty()`　(3) `q1.empty()`　(4) `swap(q1, q2)`　(5) `q1.empty()`

思路：栈要后进先出，而队列只能从队首取——最后压入的那个恰好在队尾。于是 `pop` 把 `q1` 里的元素一个个倒进 `q2`，**倒到队列空了的那一次取出的就是队尾元素**，正是要弹的栈顶；其余元素已经按原序进了 `q2`，交换两个队列的身份即可。第 (3) 空是本题的关键：题目没给求长度的运算，所以「是不是最后一个」只能靠 **取走它之后 `q1` 是否变空**来判断。

代价：`push` $O(1)$、`empty` $O(1)$；`pop` $O(n)$。`top` 可以写成「`r ← pop(); push(r); return r`」——弹出来再压回去，它又成了最后压入的那个，

仍在栈顶，代价同样是 $O(n)$。

另一版把代价挪到 push：新元素先单独进 q2，再把 q1 里原有的元素按原序全部倒到它后面，最后交换。这样 q1 的队首永远是最后压入的那个，pop/top 都是 $O(1)$，而 push 变成 $O(n)$。**两版是对称的：总有一端要付** $O(n)$，选哪一版取决于哪个操作更频繁。

和 3-11（两个栈实现队列）对照着记：

	两个栈做队列（3-11）	两个队列做栈（本题）
重的一端	出队要倒栈，但**摊还** $O(1)$	本题两版：总有一端 $O(n)$，**摊不掉**
为什么不同	每个元素**一生只被倒一次**	每做一次重操作就要把**全部**元素倒一遍

最后一行才是真正的考点：**摊还分析成不成立，取决于「每个元素被搬动的总次数」是否有界**，不取决于「单次最坏是多少」。

五、数学证明

3-26. ★ 证明：n 个元素按 $1, 2, \cdots, n$ 顺序入栈，可能的出栈序列共 $C_n = \frac{1}{n+1}C_{2n}^{n}$ 种。

证明. 设 C_n 为答案，$C_0 = 1$。考察**元素 1 在输出序列中的位置**：设 1 是第 $i+1$ 个被输出的（$0 \le i \le n-1$）。1 是最先入栈的，所以它出栈时栈底就是它，排在它前面输出的那 i 个元素必定全部来自 $\{2, \cdots, i+1\}$——这 i 个元素在 1 之上入栈又在 1 之前出完，它们内部的相对次序是一个规模为 i 的同类子问题；而排在 1 之后输出的 $n-1-i$ 个元素是 $\{i+2, \cdots, n\}$，构成规模 $n-1-i$ 的同类子问题。两部分互不影响，故

$$C_n = \sum_{i=0}^{n-1} C_i \, C_{n-1-i}$$

这正是 Catalan 数的递推式，其解为 $C_n = \frac{1}{n+1}C_{2n}^{n}$。∎

验证：$C_1 = 1$，$C_2 = 2$，$C_3 = 5$，$C_4 = 14$，$C_5 = 42$。**同一个数还数着另外两样东西**：n 对括号的合法配对方案数、n 个结点的不同二叉树形态数（第 5 章会再用一次）——三者之间是一一对应的。

3-27. ★ 证明：排列 $p_1 p_2 \cdots p_n$ 是入栈序列 $1, 2, \cdots, n$ 的合法出栈序列，当且仅当**不存在下标** $i < j < k$ **使** $p_j < p_k < p_i$。

证明. 注意入栈序列递增，因此**任何时刻栈内元素自底向上是递增的**，这是全部论证的基础。

必要性（合法 ⇒ 无此模式）：反设存在 $i<j<k$ 使 $p_j<p_k<p_i$。p_j、p_k 都小于 p_i，而入栈序列递增，所以二者都在 p_i **之前入栈**；又二者都在 p_i **之后输出**，故 p_i 出栈的那一刻，p_j 与 p_k 都还压在栈里。栈内自底向上递增，而 $p_k>p_j$，所以此刻 p_k 在 p_j **之上**，于是 p_k 必先于 p_j 出栈，即 $k<j$——与 $j<k$ 矛盾。

充分性（无此模式 ⇒ 合法）：对 n 作归纳。$n\le 1$ 显然。设 $p_1=m$。把其余元素分成「小值」$\{1,\cdots,m-1\}$ 和「大值」$\{m+1,\cdots,n\}$。1. **小值在 p 中必按递减次序出现**：否则存在 $a<b$ 且 a 排在 b 前面，取 $i=1$、$j=\mathrm{pos}(a)$、$k=\mathrm{pos}(b)$，即有 $p_j=a<p_k=b<m=p_i$，正是被禁的模式。2. **大值构成的子序列仍不含该模式**（模式的定义只看相对次序，子序列自然继承），由归纳假设，它是 $m+1,\cdots,n$ 的一个合法出栈序列。3. **两者可以自由交错**：先把 $1,\cdots,m$ 依次入栈并弹出 m，此时栈内自底向上是 $1,2,\cdots,m-1$，正好能按递减次序逐个弹出，与小值的要求一致；需要输出某个大值时，把当前已入栈到的位置继续往后压、按第 2 步的方案操作即可—— 大值全部压在小值之上，用完就弹走，不影响下面小值的次序。这里还要说明一点：**轮到输出某个小值 s 时，栈里不会还压着没输出的大值**。否则设压着的是 L，它已入栈说明此前输出过某个比它更大的大值 L'（正是为了输出 L' 才把 L 压进去），于是 p 中依次出现 L'，s，L 且 $s<L<L'$，又是被禁的模式。所以小值要出栈时它一定在栈顶。

因此整个 p 可以被实际的入栈出栈过程生成，即合法。∎

使用建议：选择题用判定法则一眼看出答案，大题仍写模拟算法——后者更好写、也更好改错。

3-28. 证明循环队列中「元素个数 $=(rear-front+M)\bmod M$」对 `rear` 是否绕回都成立。

证明. 分两种情形。(1) `front` ⩽ `rear`（未绕回）：队列占据 $[front,rear)$，元素个数为 $rear-front\ge 0$；此时 $rear-front+M\in[M,2M)$，对 M 取模正好还原为 $rear-front$。(2) `front` > `rear`（已绕回）：队列占据 $[front,M)\cup[0,rear)$，元素个数为 $(M-front)+rear=rear-front+M$；此时 $rear-front\in(-M,0)$，故 $rear-front+M\in(0,M)$，取模后不变，正是元素个数。两种情形都得到同一个表达式。∎

顺带说明牺牲一格的必要性：若允许存满 M 个，则满时 $rear=front$，公式算出 0，与空队列无法区分——这就是 3-14 里那个鸽笼论证的算式版本。

3-29. ★ 用 `I` 表示一次入栈、`O` 表示一次出栈。一个从空栈开始、到空栈结束的操作过程可以写成一个由 `I`、`O` 组成的序列（例如 `IOIO` 合法，`IOOI` 非法）。(1) 给出序列合法的**充要条件**并说明理由；(2) 证明：对元素互不相同的输入，两个**不同的**合法操作序列不可能产生**相同的**输出序列；(3) 长度为 $2n$ 的合法序列共有多

少个?

答：(1) **充要条件**：I 与 O 的总数相等，且**任何一个前缀里 I 的个数都不少于 O 的个数**。

理由：把栈中元素个数看成一个从 0 出发的计数器，I 使它加一、O 使它减一。「终态是空栈」等价于全部增减之和为零，即两者总数相等；「中途从不对空栈做出栈」等价于**任何时刻计数器非负**，即任何前缀里 I 不少于 O。反之两条都满足时，整个过程每一步都合法且末尾归零。把 I 看成左括号、O 看成右括号，这正是「n 对括号合法配对」的条件。

务必与 3-7、3-19、3-27 区分：那几道判的是**输出序列**合不合法（判据是「不存在 $i<j<k$ 使 $p_j<p_k<p_i$」），这道判的是**操作序列**合不合法。两者都由 Catalan 数计数，但判据完全不同，别张冠李戴。

(2) **证明.** 设 $T_1\neq T_2$ 是两个合法操作序列（长度均为 $2n$，即全部 n 个输入都入栈并出栈），取它们**第一个不同的位置** $r+1$。前 r 步完全相同，所以此刻栈的内容、已输出的内容、下一个待入栈的元素都相同。不妨设 T_1 这一步做 I、T_2 做 O。设此刻栈顶是 a、下一个待入栈的是 b（T_2 能做 O 说明栈非空，a 存在；T_1 能做 I 说明还有元素没入栈，b 存在；b 尚未入栈故 $a\neq b$）。- 在 T_2 里：a **此刻就被输出**，而 b 还没入栈，故输出中 a **在** b **之前**；- 在 T_1 里：b 被压到 a 上面，要弹 a 必须先弹 b，故 b **在** a **之前**。

两个合法序列最终都把所有元素输出，故 a、b 都出现在各自的输出里，而它们的相对次序相反，输出序列必不相同。∎

(3) $C_n=\frac{1}{n+1}C_{2n}^{n}$，即第 n 个 Catalan 数（由下文的双射与 3-26 得出；也可直接把操作序列看作 n 对括号的合法配对来数）。

把三问连起来看：(2) 说明「操作序列 $\to$ 输出序列」这个映射是**单射**；而合法输出序列按定义就是某个合法操作序列的输出，映射是满射，所以它是**双射**；再由 3-26 知合法输出序列恰有 C_n 个，操作序列也就恰有 C_n 个。于是**合法操作序列、合法出栈序列、n 个结点的二叉树形态，三者一一对应**，个数都是 C_n。

第 4 章 字符串

一、填空题

4-1. 字符串是由 ____ 组成的有限序列，串中字符的个数称为串的 ____。不含任何字符的串称为 ____，它的长度为 ____；由一个或多个空格组成的串称为 ____，它的长度 ____。

答：**零个或多个字符**；**长度**；**空串**；0；**空格串**；**不为** 0（等于空格个数）。「空串」和「空格串」是最爱考的一组辨析：空格是一个货真价实的字符，其中每个空格字符的 ASCII 编码是 `0x20`，串长不是 0。

4-2. 串 `S = "abcaabcab"` 的长度是 ____，它的所有**真前缀**中最长且同时也是真后缀的那一个是 ____。

答：9；`"ab"`（长度 2）。验证：真前缀 `a`、`ab`、`abc`…；真后缀 `b`、`ab`、`cab`…；`"ab"` 两边都有，长度 3 到 8 的前后缀均不相等（尤其前 5 个字符是 `"abcaa"`，后 5 个是 `"abcab"`）。这个「最长公共真前后缀」正是特征向量的定义核心。

4-3. 设目标串长 n、模式串长 m。朴素（Brute Force）模式匹配算法的最坏时间代价是 ____，KMP 算法的时间代价是 ____，其中求特征向量需 ____、匹配过程需 ____。

答：$O(n \cdot m)$；$O(n+m)$；$O(m)$；$O(n)$。朴素算法达到最坏的典型输入：T 为 n 个 `a`、P 为 $m-1$ 个 `a` 再加一个 `b`—— 每个起点都要比到最后一个字符才失配。

4-4. ★ 模式 `P = "abaabcaba"`（下标从 0）的**未优化**特征向量（next 数组，约定 `next[0] = -1`）是 ____。

答：`-1 0 0 1 1 2 0 1 2`。逐位算（`next[i]` = `P[0..i-1]` 的最长公共真前后缀长度）：

i	`P[i]`	前缀 `P[0..i-1]`	最长公共真前后缀	`next[i]`
0	a	(空)	—	−1
1	b	a	0	0
2	a	ab	0	0
3	a	aba	1 (a)	1
4	b	abaa	1 (a)	1
5	c	abaab	2 (ab)	2
6	a	abaabc	0	0
7	b	abaabca	1 (a)	1
8	a	abaabcab	2 (ab)	2

4-5. ★ 模式 `P = "ababab"`（下标从 0）的**未优化** next 数组是 ____，**优化后**的 next 数组是 ____。

答：未优化 `-1 0 0 1 2 3`；优化后 `-1 0 -1 0 -1 0`。

未优化逐位算最长公共真前后缀：`a` → 0，`ab` → 0，`aba` → 1，`abab` → 2，`ababa` → 3。优化按 4-13 的规则：`P[2]='a'` 与 `P[next[2]=0]='a'` 相同 → 借 `next[0] = -1`；`P[3]='b'` 与 `P[1]='b'` 相同 → 借 `next[1] = 0`；后面交替下去。

周期串使两种 next 的差别格外明显：未优化是 $-1, 0, 0, 1, 2, 3$ 步步升高，优化后只在 -1 和 0 之间跳。原因是这个模式从下标 2 起每一位都与它的回退目标同字符，退过去必定再失配一次，于是一路被借到底。

顺手做一下 4-20：整串 `"ababab"` 的最长公共真前后缀是 `"abab"`，长度 $f=4$，最小循环节长 $6-4=2$，而 $6 \bmod 2=0$——所以 `"ababab"` 正是 `"ab"` 重复三次。**注意这一步必须用未优化的那一行**，优化版的数值已经不是「最长公共真前后缀长度」了。

4-6. ★ 设非空串 S 长 n，其**整串**的最长公共真前后缀长度为 f。则 S 的最小循环节长度是 ___；S 由该循环节至少重复两次构成的充要条件是 ___。

答：最小循环节长度 $n-f$；充要条件是 $n \bmod (n-f)=0$（且 $f>0$）。例：`S = "abcabcabc"`，$n=9$、$f=6$，循环节长 3，$9 \bmod 3=0$，故由 `"abc"` 重复 3 次构成。反例：`S = "abcab"`，$n=5$、$f=2$，循环节长 3，$5 \bmod 3 \neq 0$，说明 `S` 是某个周期为 3 的串的**前缀**，但不是整数次重复。证明见 4-24。

二、选择题

4-7. 下列关于串的叙述**错误**的是（　）。A. 空串是任意串的子串 B. 空格串的长度等于其中空格的个数 C. 长度为 n 的串，按起止位置计数（相同内容重复计）的非空子串个数为 $\frac{n(n+1)}{2}$ D. 两个串相等，只要求它们包含的字符集合相同

答： D。串相等的定义是**长度相等且对应位置的字符逐一相同**，`"abc"` 与 `"cab"` 字符集合相同却不相等。A 对（空串是任何串的子串，也是任何串的前缀和后缀）；B 对（空格是真字符）；C 对：以下标 i（$0 \le i < n$）处字符开头的非空子串有 $n-i$ 个，求和得 $\frac{n(n+1)}{2}$；若相同内容重复计、另把空串统一算作一种，则是 $\frac{n(n+1)}{2}+1$ 个。

4-8. ★ 模式 `P = "aaab"` 的未优化 next 数组是（　）。A. `-1 0 1 2`　B. `-1 0 0 0`　C. `0 1 2 3`　D. `-1 1 2 3`

答： A。`next[1]` 看 `"a"` → 0；`next[2]` 看 `"aa"` → 1（a）；`next[3]` 看 `"aaa"` → 2（aa）。注意**优化版**则是 `-1 -1 -1 2`：因为 `P[1]=P[next[1]]=a`、`P[2]=P[next[2]]=a`，都要继续向前借用。

4-9. ★ 用 KMP 匹配时，若模式在位置 i 处失配且 `next[i] = -1`，则下一步应当（　）。A. 模式右移一位，目标下标回退到本趟起点加一 B. 模式整体右移 $i+1$ 位，目标下标前进一格，从 `P[0]` 重新比 C. 目标下标回退到 0 D. 匹配失败，算法结束

答： B。`next[i] = -1` 的含义是「模式已经退无可退」，实现里写成 `i = next[i]` 后 `i` 变成 -1，下一轮的 `i == -1` 分支会让 `i` 与 `j` 同时加一——效果正是**目标下标前进一格、模式从头开始**。KMP **的灵魂就是目标下标 `j` 只增不减**，A 和 C 都违背了这一条。

4-10. 用朴素算法在 $T=$ `"aaaaaaaaab"`（9 个 `a` 加 1 个 `b`，长 10）中匹配 $P=$ `"aaab"`（长 4），字符比较的总次数是（　）。A. 10　B. 24　C. 28　D. 40

答： C。起点只能取 $s=0,1,\cdots,6$，共 7 个。$s=0..5$ 这 6 趟都在比到第 4 个字符时发现 `P[3]='b'` 与 `T[s+3]='a'` 失配，各比 4 次，计 24 次；$s=6$ 时 `T[6..9] = "aaab"` 完全匹配，又比 4 次。合计 $24+4=28$。这正是朴素算法的最坏输入形态：**每趟都要比到模式的最后一个字符**

才失配，代价 $\Theta(n \cdot m)$。换成 KMP，目标下标一路不回退，比较次数是 $O(n+m)$ 量级。**答题提醒**：这类题按「起点个数 × 每趟比较次数」老实数，先算清楚起点范围是 $0..n-m$。

三、辨析与简答

4-11. 辨析：「串是一种特殊的线性表，所以线性表的所有算法都能直接用在串上。」

答：前半句对，后半句不对。串确实是数据元素限定为字符的线性表，但它的**基本运算的粒度不同**：线性表以「单个元素」为操作对象（插入一个元素、删除一个元素），串则以「**子串**」为操作对象（求子串、串联接、模式匹配、替换）。正因为这个差别，串才值得单独成章——模式匹配这类问题在一般线性表里根本不存在。

4-12. ★ 简答：KMP 算法为什么能做到 $O(n+m)$？它省掉的究竟是什么？

答：1. **省掉的是目标串的回退**。朴素算法失配后把目标下标退回本趟起点的下一个位置，同一个字符可能被反复比较 m 次；KMP 里**目标下标 `j` 只增不减**，全程至多前进 n 次。2. **凭什么可以不回退**：失配之前已经匹配上的那一段 `P[0..i-1]` 是已知的，它的最长公共真前后缀告诉我们「模式最少要右移多少位才可能重新对上」，移过去之后开头那一段**必然相同、不必再比**。3. **代价分析**：见 4-23 的严格证明——让 `i` 减小的只有 `i = next[i]`，而每次至少减 1，所以它的执行次数不超过 `i` 增加的次数，总循环次数 $\le 2n+1$。4. 再加上预处理 next 的 $O(m)$，总计 $O(n+m)$。而且**同一个模式的 next 只需算一次**，可以对多个目标串复用。

4-13. ★ 简答：未优化的 next 与优化后的 next 有什么区别？为什么优化后会出现 -1？

答：- **未优化版**：`next[i]` = `P[0..i-1]` 的最长公共真前后缀长度（`next[0] = -1`）。它只回答「退到哪里**有可能**继续匹配」。- **优化版**：若 `P[i] == P[next[i]]`，那么退到 `next[i]` 之后比的还是同一个字符、注定再次失配，不如一**步到位借用** `next[next[i]]`，即 `next[i] ← (P[i] == P[k]) ? next[k] : k`（k 为未优化值）。- -1 **从哪来**：当 `next[i]` 一路借用到 0 号位、而 `P[i] == P[0]` 时，就借用 `next[0] = -1`，含义是「模式整体移过这个位置，目标下标前进一格」。- **两者的等价性**：优化只是跳过了若干必然失败的比较，**失配后的最终落点相同**，匹配结果一字不差；但**两个数组的数值不同**，考试写哪一个要看题目对特征向量的定义。本书正文印的是原书算法 4.7 的优化版（例如 `next("ababd") = -1 0 -1 0 2`）。- **重要提醒**：做「最小循环节」「周期」这类题**必须用未优化版**—— 优化版已经不再表示「最长公共真前后缀长度」，拿去套周期公式会算错。

4-14. 简答：原书的朴素匹配算法返回值有一处差 1 的错误，说说这类错误为什

么难被发现。

答：模式匹配成功时，正确的返回值应是**本趟起点**在目标串中的下标；匹配成功时目标下标 `j` 已走到匹配段末尾之后，起点应是 `j - pLen`；原书写成 `return (j - pLen + 1)`，多加了 1，所有成功匹配的返回值一律大 1（朴素匹配【算法 4.6】与 KMP【算法 4.8】是同一处错，见第 4.3.1 节）。难发现的原因有三：(1) 匹配**失败**的路径返回值正确，测试若只测失败用例就发现不了；(2) 匹配成功时返回的位置只差 1，肉眼看输出「像是对的」；(3) 书上演示匹配的例子只逐趟画了比较过程、**没有写出返回值**，错误在书里没有机会暴露。**教训：边界值要单独出测试用例**——本书对这一处专门断言了返回位置与 `std::string::find` 一致。

4-15. 简答：实现 `my_memcpy(void* dst, const void* src, size_t n)` 时，若两块内存**重叠**会发生什么？分 `src < dst < src + n` 与 `dst < src < dst + n` 两种情形说明正确做法。

答：逐字节从前往后拷贝时，**每写一个字节都可能覆盖一个还没读到的源字节**，是否出事取决于两块的相对位置。

- `src < dst < src + n`（**目标落在源的后半段**）：从前往后拷会先写 `dst[0]`，而 `dst[0]` 正是某个尚未读到的 `src[k]`（$k = \texttt{dst} - \texttt{src}$）；等读到它时，读到的已经是刚写进去的内容。典型症状是**后半段变成前半段的重复**。**正确做法：从最后一个字节开始，倒着拷**——每次写的位置都在「还没读的区域」之后。
- `dst < src < dst + n`（**目标落在源的前半段**）：反过来，**倒着拷会出错，必须从前往后拷**。
- **两块不重叠**：哪个方向都对。

所以处理重叠的通用办法就是**先比较两个地址、再决定拷贝方向**，这正是 `memmove` 做的事。

标准库的分工要记住：`memcpy`、`strcpy` 对重叠区间的行为是**未定义**的（正因为不做这道判断，它们才可以更快）；要允许重叠必须用 `memmove`。「未定义」不等于「一定出错」——短串、特定编译器下它可能碰巧是对的，这类 bug 换个优化档就冒出来，属于最难查的一种。

这一条和本章的 `String` 类直接相关：**自赋值**（`s = s`）、以及「把一个串的后半段拷到它自己的前半段」这类操作，共同点都是源与目标是同一块（或相互重叠的）内存：前者先释放再拷会读到已释放的空间，后者拷错方向会读到刚写进去的内容。

4-16. ★ 设 `S = "abaabab"`（下标从 0）。(1) 按字典序列出 `S` 的全部**不同**非空子串，并说明共有多少个；(2) 求 `S` 的未优化 next 数组和优化后的 next 数组。

答：(1) 共 19 个。按字典序（`a < b`；一个串是另一个串的前缀时，短的在前）：

a、aa、aab、aaba、aabab、ab、aba、abaa、abaab、abaaba、abaabab、abab、b、ba、baa、baab、baaba、baabab、bab

不重不漏的办法：先按长度分组把不同子串找齐，再统一排序。

长度	不同子串	个数
1	a b	2
2	aa ab ba（bb 没有出现）	3
3	aab aba baa bab	4
4	aaba abaa abab baab	4
5	aabab abaab baaba	3
6	abaaba baabab	2
7	abaabab	1

合计 19 个；按起止位置计数是 $\frac{7\times 8}{2}=28$ 个（4-7），其中 9 个是重复的。排序时**从左到右逐位比**：abaabab 排在 abab 前面（第 4 位 a < b），所有以 b 开头的串都排在以 a 开头的串之后，不管长短。

(2) 未优化 `-1 0 0 1 1 2 3`；优化后 `-1 0 -1 1 0 -1 3`。

i	P[i]	未优化 k	P[k]	是否相同	优化后
0	a	-1	—	—	-1
1	b	0	a	否	0
2	a	0	a	是，借 next[0]	-1
3	a	1	b	否	1
4	b	1	b	是，借 next[1]	0
5	a	2	a	是，借 next[2]	-1
6	b	3	a	否	3

易错点在 $i=5$：借的是**已经优化过的** next[2]，即 -1；误借未优化的 next[2]=0 会写成 0。规则见 4-13，逐位从左往右算，借用时前面各位都已是优化值。

四、算法设计与分析

4-17. ★ 写出求未优化特征向量（next 数组）的算法，并说明其时间代价。

答：**基本思想**——next[i] 的计算复用 next[i-1]：设已知 P[0..i-2] 的最长公共真前后缀长度为 k。若 P[i-1] == P[k]，则新的长度是 $k+1$；否则把 k 退到 next[k] 再比，一直退到匹配或退到 -1 为止。

```
GetNext(P, m)
    if m = 0 then return 空数组
    next[0] ← -1
    i ← 0;  k ← -1
```

```
    while i < m - 1
        if k = -1 or P[i] = P[k]
            i ← i + 1;  k ← k + 1
            next[i] ← k                  // 未优化版：直接记下长度
        else
            k ← next[k]                  // 沿已算好的特征值回退
```

优化版只改一行：把 `next[i] ← k` 换成 `if P[i] = P[k] then next[i] ← next[k] else next[i] ← k`。

时间代价 $O(m)$：`i` 只增不减，至多走 m 次；`k` 每次 `k ← next[k]` 至少减 1，而 `k` 总共只增加了至多 m 次，故回退的总次数也不超过 m。与 4-23 是同一套摊还论证。

4-18. ★ 写出 KMP 匹配算法（利用已求出的 next 向量），并分析其代价。

答：

```
KMP(T, n, P, m, next)
    i ← 0                  // 模式下标，可以退到 -1
    j ← 0                  // 目标下标，只增不减
    while i < m and j < n
        if i = -1 or T[j] = P[i]
            i ← i + 1;  j ← j + 1
        else
            i ← next[i]              // 唯一与朴素算法不同的一句
    if i ⩾ m then return j - m       // 匹配成功，返回本趟起点
    else return -1                   // 失败
```

代价：匹配 $O(n)$、预处理 $O(m)$，合计 $O(n+m)$；额外空间 $O(m)$（next 数组）。**评分要点**：(1) `i = -1` 这个分支不能少，它负责「模式退无可退时目标前进一格」；(2) 成功时返回 `j - m` 而不是 `j`；(3) 要说明 `j` 不回退。

4-19. ★ 对模式 `P = "abaabcac"`，求它的未优化 next 与优化 next；并用 KMP 在 $T =$ `"abaabaabcac"` 中匹配，写出每一趟的失配位置与右移位数。

答：

i	0	1	2	3	4	5	6	7
`P[i]`	a	b	a	a	b	c	a	c
未优化 `next[i]`	-1	0	0	1	1	2	0	1
优化 `next[i]`	-1	0	-1	1	0	2	-1	1

（优化版逐位核对：`P[2]='a'` 与 `P[next[2]=0]='a'` 相同 → 借 `next[0] = -1`；`P[4]='b'` 与 `P[1]='b'` 相同 → 借 `next[1] = 0`；`P[6]='a'` 与 `P[0]='a'` 相同 → 借 -1。）

匹配过程（用未优化 next，$T=$ `abaabaabcac`，下标从 0）：

趟	对齐位置	比到哪里	结果
1	$P[0]$ 对 $T[0]$	`T[5]='a'` 与 `P[5]='c'` 失配	`next[5]=2`，模式右移 $5-2=3$ 位
2	$P[2]$ 对 $T[5]$	从 `P[2]` 起逐字符相同，一直比到 `P[7]='c'` 与 `T[10]='c'`	**匹配成功**，此时 $j=11$，起点 $11-8=3$

全程目标下标 j 从 0 单调走到 11，**一次都没有回退**——这正是与朴素算法的差别。朴素算法在同样输入上要退回 $T[1]$、$T[2]$ 重来，比较次数明显更多。

4-20. ★ 变种应用：给定串 S（长 n），设计 $O(n)$ 算法求它的**最小循环节**，并判断 S 是否由该循环节至少重复两次构成。

答：**基本思想**——把特征向量**算到下标** n（即整串 S 的最长公共真前后缀长度 f），则最小循环节长度为 $n-f$；S 是整数次重复当且仅当 $n \bmod (n-f)=0$ 且 $f>0$。

```
MinPeriod(S, n)
    if n = 0 then return (无正周期, 不是至少重复两次)
    用 GetNext 求 S 的未优化特征向量，并把它多算一项 f = next[n]
    p ← n - f                          // 最小循环节长度
    if f > 0 and n mod p = 0
        return (p, n / p)              // 循环节长 p，重复 n/p 次
    else
        return (p, 不是整数次重复)
```

两个必须写清的细节：1. **要用未优化版** next——优化版的数值已不是「最长公共真前后缀长度」，套公式必错；2. 教材实现的特征向量只算到下标 $n-1$，本题要的 $f=next[n]$ **得单独再推一步**（把 `GetNext` 的数组开到 $n+1$ 项、循环条件从 `i < m - 1` 放宽到 `i < m` 即可）。

代价：$O(n)$ 时间、$O(n)$ 空间。**例**：`S = "abcabcabc"`，$f=6$，$p=3$，$9 \bmod 3=0 \to$ 循环节 `"abc"`，重复 3 次。`S = "abaabaaba"`，$n=9$、$f=6$、$p=3$，$9 \bmod 3=0 \to$ 循环节 `"aba"`，重复 3 次。`S = "abcab"`，$f=2$、$p=3$，$5 \bmod 3 \neq 0 \to$ 不是整数次重复。证明见 4-24。

4-21. 变种应用：设计算法判断串 B 是否为串 A 的**循环移位**（如 `"cdeab"` 是 `"abcde"` 的循环移位），要求 $O(|A|+|B|)$。

答：**基本思想**——B 是 A 的循环移位，当且仅当 $|A|=|B|$ 且 B **是**

$A+A$ **的子串**。因为把 A 接成两份之后，A 的每一种循环移位都恰好作为一个长度 $|A|$ 的窗口出现在其中。

```
IsRotation(A, B)
    if |A| ≠ |B| then return false
    return KMP(A + A, B) ≠ -1
```

代价：拼接 $O(|A|)$，KMP 匹配 $O(2|A|+|B|)=O(|A|+|B|)$。**易错点**：长度相等这一条必须先判——否则 `B = "ab"`、`A = "abab"` 会被误判为真。

同类变式（都是「换个目标串或换个特征定义」）：- 统计 P 在 T 中出现的次数（**允许重叠**）：KMP 匹配成功后不要从头开始，而是令 `i ← next[m]` 继续（next 须按 4-20 的办法多算到下标 m），仍是 $O(n+m)$；- 求 S 的**最长回文前缀**：对 `S + '#' + reverse(S)` 求特征向量，整串的最长公共真前后缀长度即为答案（`#` 是不在字母表中的分隔符，防止跨越中线）；- 求 A 的最长的、在 B 中作为子串出现过的**前缀**：对 `A + '#' + B` 求未优化特征向量，取 `next[|A|+2]` 到 `next[|A|+1+|B|]` 的最大值（每个值是「A 的某个前缀恰好在 B 的此处结束」的最大长度；须多算整串的那一项；B 为空时返回 0）。注意它不是「两串的最长公共前缀」——后者只要从头逐位比到第一个不同处，$O(\min(|A|,|B|))$，用不着 KMP。

4-22. 变种应用：若某个子串形如 AA（非空串 A 紧接着再出现一次，如 `"bcbc"`、`"aa"`），称它为**平方子串**。设计算法判断串 S（长 n）是否含平方子串；若含，返回**起点最小**的那个的起点和 $|A|$。要求最坏 $O(n^2)$。

答：**基本思想**——按半长 $L=1,2,\cdots,\lfloor n/2\rfloor$ 分别找。L 固定时，起点为 i、长 $2L$ 的子串是平方，当且仅当 $S[t]=S[t+L]$ 对 $t=i,\cdots,i+L-1$ 都成立，也就是「$S[t]$ 与 $S[t+L]$ 相等」**连续成立** L **次**。扫一趟、记下连续相等的次数 `run` 即可。

```
FirstSquare(S, n)
    best ← 无;  bestL ← 0
    for L ← 1 to ⌊n/2⌋
        run ← 0
        for j ← 0 to n - L - 1
            if S[j] = S[j + L] then run ← run + 1 else run ← 0
            if run = L                    // S[j-L+1..j+L] 是平方子串
                if best = 无 or j - L + 1 < best
                    best ← j - L + 1;  bestL ← L
                跳出内层循环                      // 同一个 L，往后的起点只会更大
    return (best, bestL)                      // best = 无 表示没有平方子串
```

正确性：`run` 是「以 j 结尾、连续满足 $S[t]=S[t+L]$ 的 t 的个数」。

对固定的 L，`run = L` 第一次成立时 j 最小，起点 $j-L+1$ 也就最小；再在所有 L 之间取最小起点（起点相同时先找到的 L 较小，用 `<` 比较不会被替换），就是全局起点最小的平方子串。

代价：外层 $\lfloor n/2 \rfloor$ 趟，每趟 $O(n)$，共 $O(n^2)$ 时间、$O(1)$ 额外空间。朴素做法是枚举起点和 L 再逐字比较，要 $O(n^3)$；本法把「逐字比较 L 个字符」摊进了一趟扫描。

易错点：不能在第一个找到平方子串的 L 上直接返回。例如 `S = "abcbcaa"`：$L=1$ 先找到 `"aa"`，起点 5；但 $L=2$ 的 `"bcbc"` 起点是 1，答案是起点 1、$|A|=2$。另一个例子 `S = "abacabad"` 不含平方子串，返回「无」。

别和 4-20 混淆：4-20 问的是**整串**是否由某个串重复至少两次构成，起点固定为 0，用 next 一次求出；本题的平方子串起点和长度都不定，所以改为按 L 逐趟扫描。

五、数学证明

4-23. ★ 证明 KMP 匹配过程的时间代价为 $O(n)$（n 为目标串长度）。

证明. 考察循环体的两个分支。- **分支一**（`i = -1` 或 `T[j] = P[i]`）：`i` 与 `j` 同时加一。由于 `j` 只增不减且 $j<n$，这个分支至多执行 n 次。- **分支二**（`i ← next[i]`）：由 next 的定义有 `next[i] < i`，所以每执行一次 `i` **至少减** 1。

设分支一执行了 a 次、分支二执行了 b 次。`i` 的初值为 0，分支一使它总共增加 a，分支二使它总共减少至少 b，而 `i` 的终值 ≥ -1，故

$$0+a-b \geq -1 \Rightarrow b \leq a+1 \leq n+1$$

于是循环体总执行次数 $a+b \leq 2n+1 = O(n)$，每次循环体是常数工作量。∎

加上求 next 的 $O(m)$（同样的摊还论证），KMP 整体为 $O(n+m)$。**这条论证的模板值得记住**：「一个量只增不减 + 另一个量每次至少减 1」$\Longrightarrow$ 线性摊还。

4-24. ★ 设非空串 S 长 n，f 是 S 的最长公共真前后缀长度。证明：$p=n-f$ 是 S 的最小周期，且 S 能写成某个串至少重复两次（$S=w^k$，$k\geq 2$）当且仅当 $f>0$ 且 p 整除 n。

证明. 称 p 是 S 的**周期**，若对一切 $0 \leq i < n-p$ 有 $S[i]=S[i+p]$。

(1) p **是周期，当且仅当** S **有长为** $n-p$ **的公共前后缀**。$S[i]=S[i+p]$ 对一切合法 i 成立，等价于说 $S[0..n-p-1]$（前缀）与 $S[p..n-1]$（后缀）逐位相同，即长为 $n-p$ 的前缀与后缀相等。

(2) **最长公共真前后缀** $\leftrightarrow$ **最小周期**。由 (1)，公共前后缀的长度越长，对应的周期 $p=n-(\text{长度})$ 越小。f 是最长的公共**真**前后缀长度，故

$p = n - f$ 是最小的**正**周期。

(3) **整数次重复的充要条件**。若 $f > 0$ 且 p 整除 n，则 $p < n$、$n/p \geq 2$，把 S 按每 p 个字符切成 n/p 段，由周期性质相邻两段逐位相同，故每段都等于第一段，S 是第一段重复 n/p 次。反之若 $S = w^k$（$k \geq 2$，w 的长度为 q），则 q 是周期，故最小周期 $p \leq q$，且 p 整除 q、q 整除 n，于是 p 整除 n；又 $p \leq q < n$，故 $f = n - p > 0$。∎

「p 整除 q」这一步用到**周期引理**（Fine–Wilf 定理）：若 p、q 都是 S 的周期且 $p + q \leq n + \gcd(p, q)$，则 $\gcd(p, q)$ 也是周期。这里 $p \leq q$、$n = kq \geq 2q \geq p + q$，条件满足，于是 $\gcd(p, q)$ 是周期；而 p 已是最小周期、$\gcd(p, q) \leq p$，只能 $\gcd(p, q) = p$，即 p 整除 q。（考试中这一步一般允许直接引用。）

注意 p 不整除 n 的情形并不意味着没有周期：`"abcab"` 的最小周期是 3，它只是「某个周期为 3 的无限串的前 5 个字符」，不是整数次重复。这正是 4-20 里那个取模判断存在的理由。

4-25. 证明：求 next 数组的算法（4-17）时间代价为 $O(m)$。

证明. 与 4-23 同构。变量 `k` 在成功分支里每次加 1，共加至多 m 次；在失败分支里执行 `k ← next[k]`，由 `next[k] < k` 知每次至少减 1。`k` 的初值为 -1、终值 ≥ -1，故失败分支的执行次数不超过成功分支的次数加 1，总循环次数 $\leq 2m + 1 = O(m)$。∎

直观说法：`k` **是一个「涨得慢、跌得快」的量，但它涨了多少才能跌多少**，所以尽管内层是个 `while`，整体仍是线性的——这正是初学者最容易误判成 $O(m^2)$ 的地方。

第 5 章 二叉树

一、填空题

5-1. 二叉树是结点的有限集合，它或者为空，或者由一个根结点及两棵**互不相交**的、分别称为 ____ 和 ____ 的二叉树组成。含 3 个结点的二叉树共有 ____ 种不同形态。

答：**左子树**、**右子树**；5 种。一般地，n 个结点的不同二叉树形态数是 Catalan 数 $\frac{1}{n+1}C_{2n}^{n}$：$n = 1, 2, 3, 4$ 依次为 $1, 2, 5, 14$——**和第 3 章出栈序列的个数是同一个数**（见 5-31）。二叉树与度为 2 的树的根本差别就在「左右有别」：只有一个孩子时也要说清是左是右。

5-2. 二叉树的第 i 层（根为第 0 层）最多有 ____ 个结点；深度为 k 的二叉树最多有 ____ 个结点；有 n 个结点的完全二叉树，其高度（层数）为 ____。

答：2^i；$2^{k+1} - 1$；$\lceil \log_2(n+1) \rceil$。**换约定就换公式**：若根记为第 1 层，则第 i 层最多 2^{i-1} 个、深度 k 最多 $2^k - 1$ 个。考试时先声明自己用哪套约定，能避免大量无谓的扣分。

5-3. ★ 在任意一棵非空二叉树中，若叶结点数为 n_0、度为 2 的结点数为 n_2，则它们的关系是 ___。若该树共有 n 个结点、度为 1 的结点 n_1 个，则 $n=$ ___。

答：$n_0 = n_2 + 1$；$n = n_0 + n_1 + n_2$。证明见 5-29。这条性质的用法：题目给出叶子数就能直接得到度为 2 的结点数，反之亦然。

5-4. 完全二叉树按层次顺序从 0 开始编号存入一维数组，则结点 i 的父结点编号是 ___，左孩子是 ___，右孩子是 ___；结点 i 有左孩子的条件是 ___。

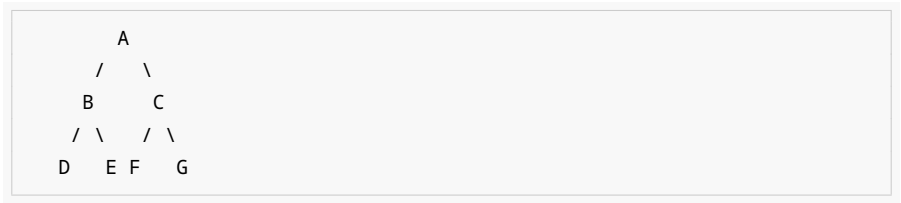
答：$\lfloor (i-1)/2 \rfloor$（$i>0$，根无父结点）；$2i+1$；$2i+2$；$2i+1 < n$。（若编号从 1 开始，则分别是 $\lfloor i/2 \rfloor$、$2i$、$2i+1$。）这三个公式是第 5.5 节堆的全部基础——堆之所以不需要指针，靠的就是它们。

5-5. 有 n（$n \geq 2$）个结点的完全二叉树，最后一个分支（非叶）结点的编号是 ___，叶结点共有 ___ 个。

答：$\lfloor n/2 \rfloor - 1$（编号从 0）；叶结点 $\lceil n/2 \rceil$ 个。建堆时「从 $i = \lfloor n/2 \rfloor - 1$ 开始向前逐个筛选」用的正是前一个公式——**编号比它大的全是叶结点，本身就是堆，一次都不用碰**。

5-6. 某二叉树的前序序列为 ABDECFG，中序序列为 DBEAFCG，则它的后序序列为 ___，层次序列为 ___。

答：后序 DEBFGCA；层次 ABCDEFG。还原过程：前序首字符 A 是根 → 中序中 A 把序列切成左 DBE、右 FCG；左子树前序 BDE → 根 B，中序 DBE → 左 D、右 E；右子树前序 CFG → 根 C，中序 FCG → 左 F、右 G。树形如下：



5-7. ★ 含 n 个叶结点（权值互不相同）的 Huffman 树共有 ___ 个结点，其中度为 2 的结点 ___ 个，**没有**度为 ___ 的结点。

答：$2n-1$；$n-1$；1。理由：Huffman 树是每次合并两棵树得到的，合并 $n-1$ 次，每次新增一个内部结点，故内部结点 $n-1$ 个、总结点 $2n-1$ 个；每个内部结点都恰有两个孩子，所以是**满二叉树**，$n_1 = 0$，与性质 $n_0 = n_2 + 1$ 一致。

5-8. 二叉搜索树的 ___ 遍历序列是一个递增有序序列；在一棵含 n 个结点的二叉搜索树中检索一个关键码，最好情况比较 ___ 次，最坏 ___ 次。

答：**中序**；最好 **1 次**（关键码恰在根上）；最坏 n **次**（树退化为链且查找最深结点，或沿全链查找失败）。若另问「平衡树中的最坏检索代价」，才是 $O(\log n)$，不能与单次检索的最好情况混用。「**二叉搜索树平均 $O(\log n)$**」**这句话有前提**——随机插入下才成立，最坏依然是 $O(n)$，这正是后面要讲平衡树的原因。

二、选择题

5-9. 一棵完全二叉树有 1001 个结点，其中叶结点的个数是（ ）。A. 250　B. 500　C. 501　D. 505

> **答：** C。$n = 1001$ 为奇数，说明没有度为 1 的结点（度为 1 的结点在完全二叉树中至多一个，且当且仅当 n 为偶数时存在）。于是 $n_1 = 0$，由 $n_0 = n_2 + 1$ 与 $n_0 + n_2 = 1001$ 得 $n_0 = 501$。也可以直接用公式 $\lceil n/2 \rceil = \lceil 1001/2 \rceil = 501$。

5-10. 已知二叉树的**前序**和**后序**序列，下列说法正确的是（ ）。A. 能唯一确定这棵二叉树 B. 不能唯一确定，但能确定它的中序序列 C. 不能唯一确定这棵二叉树 D. 只有当树中没有度为 1 的结点时才不能确定

> **答：** C。反例：前序 `AB`、后序 `BA` 既可以是「`B` 是 `A` 的左孩子」也可以是「`B` 是 `A` 的右孩子」。**根源在于前序和后序都无法区分「唯一的那个孩子是左是右」**，而中序可以。D 说反了：**恰恰是在没有度为 1 的结点（满二叉树）时，前序加后序才能唯一确定**。

5-11. ★ 在下列关键码序列中，可以构成**最小堆**的是（ ）。A. `(12, 70, 33, 65, 24, 56)`　B. `(12, 24, 33, 65, 70, 56)` C. `(70, 65, 56, 33, 24, 12)`　D. `(12, 24, 33, 65, 70, 30)`

> **答：** B。按 $K_i \le K_{2i+1}$ 且 $K_i \le K_{2i+2}$ 逐个验证（下标从 0）：B 中 $K_0 = 12 \le 24, 33$；$K_1 = 24 \le 65, 70$；$K_2 = 33 \le 56$ √。A 错在 $K_1 = 70 > K_3 = 65$；C 是**最大堆**不是最小堆；D 错在 $K_2 = 33 > K_5 = 30$。**答题技巧：先把序列按层画成完全二叉树，再只看父子两两比较**—— 兄弟之间没有任何要求，这是最常被误判的一点。

5-12. 在二叉搜索树中删除一个既有左子树又有右子树的结点，下列做法**能**保持二叉搜索树性质的是（ ）。A. 用它的左孩子顶替 B. 用它的右孩子顶替 C. 用它在中序序列中的前驱（左子树中最大者）顶替 D. 直接置空

> **答：** C（用中序**后继**，即右子树中的最小者，同样可行）。理由：中序前驱比左子树其余全部都大、比右子树全部都小，顶上来之后「左小右大」原封不动。A、B 若只是单纯顶替会破坏性质（左孩子顶上来后，它原来的右子树与被删结点的右子树争同一个右指针；要保持性质还得把后者挂到前者最右结点之下，已不是单纯「顶替」；B 与之对称，争的是左指针）。**实现上还有一个必考的细节**：把前驱顶上去之前，必须先让**前驱的左子树接到前驱原来的位置**，否则会形成环或者丢掉一棵子树（见 5-22）。

5-13. 关于 Huffman 编码，下列说法**错误**的是（ ）。A. 它是一种前缀编码，任何字符的编码都不是另一个字符编码的前缀 B. 权值较大的字符，其码长不会比权值较小的字符更长 C. Huffman 树的形态可能不唯一，但带权路径长度 WPL 一定唯一 D. Huffman 编码是定长编码

> **答：** D。Huffman 编码是**不等长（变长）编码**，这正是它能压缩的原

因。A 对：字符只放在叶结点上，因此任一编码对应的路径都不会是另一条路径的前缀。B 对。C 对：合并时权值相等会产生不同的选择，形态可以不同，但每一步合并的权值之和都一样，故 WPL 唯一。

5-14. 用筛选法对 n 个元素的无序序列建最小堆，时间代价的渐进紧确界是（　）。
A. $\Theta(n)$　　B. $\Theta(n\log n)$　　C. $\Theta(\log n)$　　D. $\Theta(n^2)$

答： A。**注意区分两种建堆法**：逐个 `Insert` 建堆是 n 次 $O(\log n)$ 的上浮，合计 $O(n\log n)$；而**筛选法**（从最后一个分支结点起倒着 `SiftDown`）是 $O(n)$，因为越靠底层的结点越多、能下沉的距离越短，两者恰好抵消（证明见 5-30）。考题问「建堆」时默认指筛选法。

三、辨析与简答

5-15. 辨析：「二叉树就是度不超过 2 的树。」

答：错。两点差别：1. **左右有别**：二叉树中只有一个孩子时，必须区分它是左孩子还是右孩子，这两棵二叉树是不同的；度为 2 的树中没有这个区分。2. **空树合法**：二叉树允许为空，而树的定义通常要求至少有一个根结点。

一个直接后果：3 个结点的二叉树有 5 种形态，而 3 个结点的度不超过 2 的**树**只有 2 种。

5-16. ★ 简答：为什么「前序 + 中序」或「后序 + 中序」能唯一确定一棵二叉树，而「前序 + 后序」不能？

答：还原一棵二叉树需要两样信息：**谁是根、左右子树各由哪些结点构成**。- 前序的第一个、后序的最后一个都能给出**根**；- **中序的作用是把根两侧切开**，根左边的是左子树、右边的是右子树，于是子问题的规模确定，可以递归下去。- 前序加后序虽然能确定根，却**无法确定左右子树的分界**：当某个结点只有一个孩子时，那个孩子在前序里紧跟父结点、在后序里紧挨父结点之前，无论它是左是右，两个序列都一模一样。

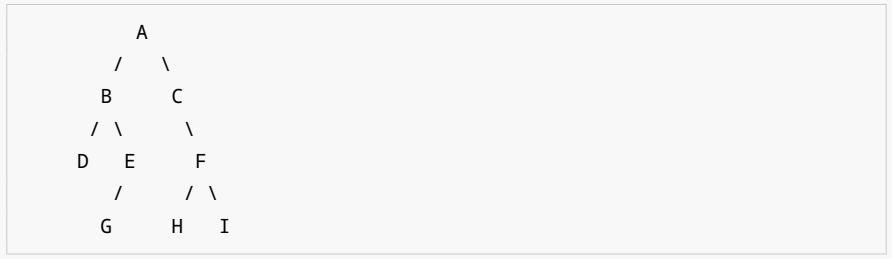
例外：若已知树中**没有度为 1 的结点**（满二叉树），分界就唯一了，此时前序加后序可以确定。

5-17. ★ 某二叉树的 9 个结点标识为 `A`～`I`，三种遍历序列都有残缺（`_` 表示缺失的一位）：

```
前序  A B _ E G _ F _ I
中序  D _ G E A C H _ I
后序  _ G E B H _ F C A
```

补全三个序列，并画出这棵二叉树。

答：前序 `A B D E G C F H I`；中序 `D B G E A C H F I`；后序 `D G E B H I F C A`。



思路：始终以中序切分左右，前序、后序只用来认根；缺的那一位由「这一段应有哪些结点」反推。1. **整棵树**：根为 A（前序第一位、后序最后一位一致）。中序中 A 在第 5 位，所以左右子树各 4 个结点：前序第 2～5 位 B _ E G 与后序第 1～4 位 _ G E B 属于左子树，前序第 6～9 位 _ F _ I 与后序第 5～8 位 H _ F C 属于右子树。2. **左子树**：前序首位、后序末位都是 B，根为 B。中序左段 D _ G E 缺的一位必须是左子树的结点，已知左子树含 B，所以是 B；于是左子树结点集是 $\{B, D, E, G\}$，前序、后序缺的都是 D。中序 D B G E 中 B 左边是 D、右边是 G E；右边这段前序为 E G，根是 E，G 在中序里位于 E 左边，是 E 的左孩子。3. **右子树**：后序末位是 C，根为 C，于是前序第 6 位补 C；右子树结点集只剩 $\{C, F, H, I\}$，前序还缺 H，得 C F H I；后序缺 I，得 H I F C；中序右段 C H _ I 缺 F，得 C H F I。中序里 C 排在这一段的第一位，所以 C **没有左子树**；其余 H F I 的前序为 F H I，根 F，左 H、右 I。

为什么三个序列都要用：C、E 都是度为 1 的结点，只看前序和后序分不出它们的孩子在左还是在右（5-16），正是中序给出了这个信息；反过来，中序缺的两位又要靠前序、后序认出的根来补。每补出一位就多一条线索，这类题按「定根 → 按中序定左右规模 → 补本段缺的标识」一层层往下做即可。

5-18. ★ 简答：堆和二叉搜索树都是二叉树，它们的「有序性」有什么本质区别？各适合什么场合？

答：

	最小堆	二叉搜索树
有序性	**局部有序**：只保证父 $\le$ 子，兄弟之间毫无关系	**全序**：左子树 < 根 < 右子树
形状	必须是**完全二叉树**，可用数组存、无需指针	形状随插入次序而变，最坏退化成链
取最小值	$O(1)$（就是根）	$O(h+1)$，平衡时 $O(\log n)$、退化时 $O(n)$

	最小堆	二叉搜索树
查任意值	$O(n)$，只能线性扫	$O(\log n)$（平衡时）
中序遍历	**无意义**，不是有序序列	得到递增有序序列

结论：堆适合「反复取极值」的场合（优先队列、堆排序、Huffman 构造）；二叉搜索树适合「按关键码检索、且要求插删也快」的场合。建堆只要 $O(n)$ 而排序要 $O(n\log n)$，差别正来自堆只要求局部有序。

5-19. 简答：非递归的二叉树遍历为什么需要栈？层次遍历为什么用队列？

答：- **深度优先遍历需要栈**：访问完一个结点之后要先深入它的一棵子树，等整棵子树处理完再回来处理另一棵——「回来」意味着必须记住**还欠着什么没做**，而欠账的处理次序是「最后欠的最先还」，正是后进先出。递归版本用的是系统运行栈，非递归版本把它换成了显式栈（见第 3.1.5 节的一般方法）。- **广度优先遍历需要队列**：要按离根由近及远访问，先被发现的结点其孩子也应该先被访问，这是先进先出。

一句话：**栈让你走得深，队列让你铺得广**。

四、算法设计与分析

本组题统一使用如下二叉树 ADT：

```
结点 Node
    data          结点数据
    left, right   左右孩子指针，空为 nullptr

BinaryTree
    Root()                出口：根指针
    Left(p) / Right(p)    入口：结点指针；出口：左/右孩子指针
    Value(p)              入口：结点指针；出口：该结点数据
```

5-20. ★ 用**递归**写出下列算法，并分析代价：(1) 求二叉树的高度；(2) 求叶结点个数；(3) 交换所有结点的左右子树（求镜像）；(4) 判断两棵二叉树是否相同。

答：四题是同一个模板——「**空树给出平凡答案，非空树由两棵子树的答案合成**」。

```
Height(p)                                  // (1) 高度按层数计：空树为 0，只有根
↪  时为 1
    if p = nullptr then return 0
    return 1 + max(Height(p->left), Height(p->right))

Leaves(p)                                  // (2)
    if p = nullptr then return 0
```

```
    if p->left = nullptr and p->right = nullptr then return 1
    return Leaves(p->left) + Leaves(p->right)

Mirror(p)                                  // (3) 镜面映射
    if p = nullptr then return
    交换 p->left 与 p->right
    Mirror(p->left)
    Mirror(p->right)

Same(p, q)                                 // (4)
    if p = nullptr and q = nullptr then return true
    if p = nullptr or  q = nullptr then return false
    return p->data = q->data and Same(p->left, q->left) and
↪   Same(p->right, q->right)
```

代价：前三个算法的时间为 $\Theta(n)$；`Same` 可提前返回，最坏时间为 $\Theta(n)$（n 为两树结点数之和）；空间是递归栈深度 $O(h)$，h 为树高——**平衡时** $O(\log n)$，**退化成链时** $O(n)$。后一句是评分点：只写「空间 $O(\log n)$」是不完整的。

(2) 的常见错法：写成 `if p = nullptr then return 1`。这样只有一个孩子的结点会被算成一片叶子，结果偏大。**叶子的定义是「两个孩子都空」**，必须显式判断。

5-21. ★ **算法填空**：下面是二叉树**中序**遍历的非递归框架，补全空缺处。

```
InOrder(root)
    S ← 空栈
    p ← root
    while p ≠ nullptr or not S.Empty()
        if p ≠ nullptr
            S.Push(p)
            p ← ____(①)____
        else
            p ← S.Pop()
            访问 p
            p ← ____(②)____
    结束
```

答：① `p->left`　② `p->right`

框架怎么记：一路向左压栈，压到底之后弹一个出来访问，然后**转向它的右子树**，对右子树重复同样的过程。循环条件里的 `or` 不能写成 `and`——弹出一个结点而它没有右子树时 `p` 为空，但栈里还有欠账，不能退出。

同一框架的另外两种次序：

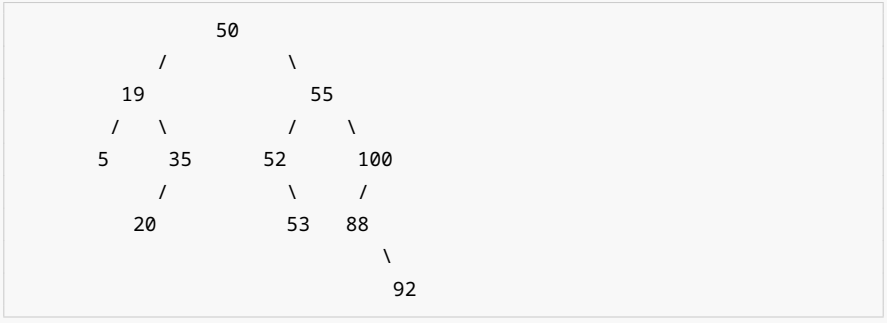
```
PreOrder(root)                              // 前序：弹出即访问
    S ← 空栈；  S.Push(root)
    while not S.Empty()
        p ← S.Pop()
        if p = nullptr then continue
        访问 p
        S.Push(p->right)                  // 先压右、后压左，弹出时才是「左优先」
        S.Push(p->left)

LevelOrder(root)                            // 层次（广度优先）：把栈换成队列
    Q ← 空队列；  Q.EnQueue(root)
    while not Q.Empty()
        p ← Q.DeQueue()
        if p = nullptr then continue
        访问 p
        Q.EnQueue(p->left)
        Q.EnQueue(p->right)
```

前序那段里「先压右后压左」是必考点：栈是后进先出，要让左子树先被处理就得后压它。**后序**最麻烦，因为一个结点要等两棵子树都处理完才能访问，常见写法是给栈里的记录附一个标记位（第一次弹出只是「回来过一次」，压回去再走右子树），这正是第 3 章「用栈消除递归」里那个**返回点编号**的直接应用。

5-22. ★ 给定关键码序列 $K = \{50, 19, 35, 55, 20, 5, 100, 52, 88, 53, 92\}$，(1) 按此次序依次插入，画出生成的二叉搜索树；(2) 写出中序序列；(3) 删除结点 50，画出删除后的树并说明步骤。

答：(1) **建树**（每个新键从根开始比较，小的往左、大的往右，走到空位置挂上去）：

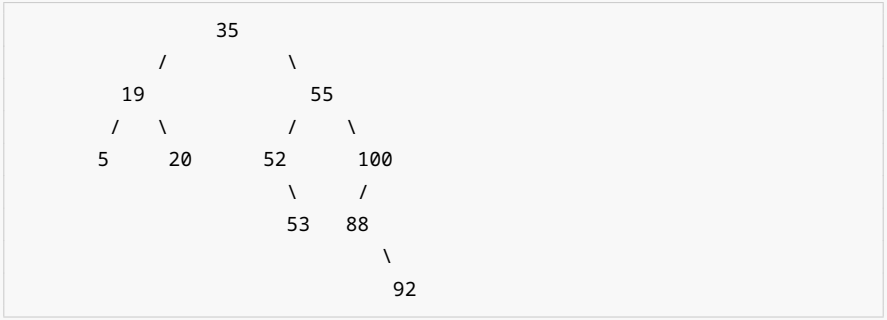


完整结构：50 的左子树是 19（左 5，右 35，35 的左是 20）；右子树是 55（左 52，52 的右是 53；右 100，100 的左是 88，88 的右是 92）。

(2) **中序序列**：5 19 20 35 50 52 53 55 88 92 100——**递增有序**，可用来自检建树对不对。

(3) **删除** 50（它左右子树都在，用**中序前驱**顶替）：1. 在左子树中一路向右找到最后一个结点，即 35（它是左子树里的最大者，也是 50 的中序前驱）；2. **先把** 35 **从原位置摘走**：35 没有右子树，用它的左子树 20 顶替它的位置，即令 19->right ← 20；3. 再让 35 继承被删结点的两棵子树：35->left ← 19、35->right ← 55，并让原来指向 50 的指针（此处是根）指向 35；4. 释放 50。

删除后：



中序验证：5 19 20 35 52 53 55 88 92 100，仍然递增 √。

第 2 步不能省：若直接让 35 顶替而不先安置它的左子树 20，20 就会丢失（或者 35->left ← 19 之后 19->right 仍指着 35，形成环）。本书教学版的测试专门搭了一棵「前驱自己还有左孩子」的树——去掉那一句，树上立刻成环。

5-23. ★ 用值为 1 到 5 的 5 个结点建二叉搜索树。(1) 若要求树上每个结点的**左、右子树结点数之差不超过** 1，共有多少棵？(2) 若把要求换成每个结点的**左子树结点数不少于右子树**，又有多少棵？

答：(1) 4 **棵**；(2) 6 **棵**。

关键第一步：给定关键码集合，**二叉搜索树由树的形状唯一决定**——中序必须递增，所以哪个值放哪个位置是定死的。于是问题变成「数满足约束的二叉树**形状**」，用 5-31 的递推、再把求和范围收窄即可。

(1) 设 $g(n)$ 为 n 个结点、每个结点都满足「左右子树结点数之差 ≤ 1」的形状数，$g(0)=g(1)=1$。按根的左右子树规模 (l,r)，$l+r=n-1$：

n	允许的 (l,r)	$g(n)$
2	(1,0)、(0,1)	$1+1=2$
3	只有 (1,1)（(2,0) 差 2，不行）	1
4	(2,1)、(1,2)	$2+2=4$
5	只有 (2,2)	$2\times 2=4$

(2) 设 $f(n)$ 为满足「$l \ge r$」的形状数，求和范围换成 $l \ge r$：

n	允许的 (l,r)	$f(n)$
2	(1,0)	1
3	(2,0)、(1,1)	$1+1=2$
4	(3,0)、(2,1)	$2+1=3$
5	(4,0)、(3,1)、(2,2)	$3+2+1=6$

对照：完全不加约束时就是 Catalan 数 $C_5=42$（5-31）。加一条约束，递推式的求和范围从「全部 l」缩成一段，数量就掉到 4 或 6。

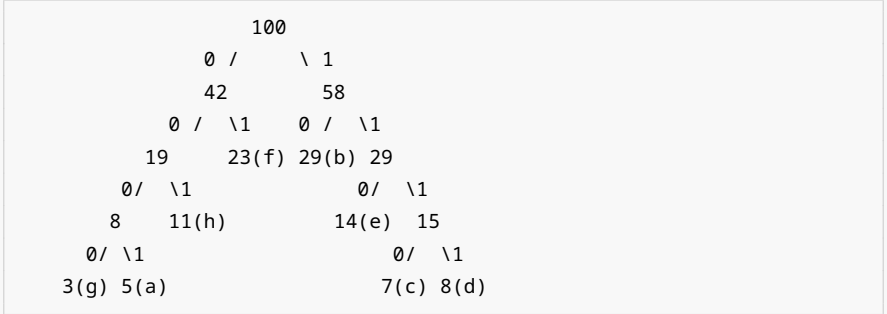
带约束的计数题一律这么做：先写出无约束的递推，再收窄求和范围。不要试图直接画图穷举——n 稍大就画不完，而且极易重复或漏掉。

5-24. ★ 已知字符集 $\{a,b,c,d,e,f,g,h\}$ 的使用频率依次为 5、29、7、8、14、23、3、11（合计 100）。(1) 构造 Huffman 树；(2) 给出各字符的 Huffman 编码；(3) 求带权路径长度 WPL 与平均码长；(4) 对码流 `000110110` 译码。

答：(1) **构造**（每次取权值最小的两棵树合并，新树权值为两者之和，放回集合）：

步	合并的两个权	新权	剩余集合
1	3(g) + 5(a)	8	7, 8(d), 8, 11, 14, 23, 29
2	7(c) + 8(d)	15	8, 11, 14, 15, 23, 29
3	8 + 11(h)	19	14, 15, 19, 23, 29
4	14(e) + 15	29	19, 23, 29, 29
5	19 + 23(f)	42	29, 29, 42
6	29(b) + 29	58	42, 58
7	42 + 58	100	完成

树形（左分支记 0、右分支记 1）：



(2) **编码**：

字符	g	a	h	f	b	e	c	d
权	3	5	11	23	29	14	7	8
编码	0000	0001	001	01	10	110	1110	1111
码长	4	4	3	2	2	3	4	4

(3) WPL：既可以逐个算 $\sum w_i l_i$，也可以用一条更快的性质——WPL **等于构造过程中所有新生成结点权值之和**：

$$\text{WPL} = 8 + 15 + 19 + 29 + 42 + 58 + 100 = 271$$

逐个验算：$3{\cdot}4+5{\cdot}4+11{\cdot}3+23{\cdot}2+29{\cdot}2+14{\cdot}3+7{\cdot}4+8{\cdot}4 = 271$ √ 平均码长 $= 271/100 = 2.71$ 位/字符。对照：8 个字符的定长编码需要 3 位，压缩率约 $2.71/3 \approx 90.3\%$。

(4) **译码** `000110110`：从根出发，遇 0 走左、遇 1 走右，**到叶结点就输出一个字符并回到根**。`0001` → `a`；`10` → `b`；`110` → `e`。结果是 abe。能这样一位一位走下去而不产生歧义，正是因为 Huffman 码是**前缀码**（字符只在叶结点上）。

注意：权值出现并列时（本题第 2 步的两个 8）选谁都可以，**树形会不同但 WPL 相同**，编码也随之不同。答题时注明自己的取舍规则即可。

5-25. ★ 字符集 $\{A, B, C, D\}$ 的出现频率为 $\{50\%, 25\%, 12.5\%, 12.5\%\}$。(1) 构造 Huffman 树，给出各字符的编码与平均码长，并与定长编码比较；(2) 若 D 的频率**上升**、其余三个字符的相对比例不变，而**编码方式仍用第 (1) 问算出的那一套**，D 的频率升到多少时，这套 Huffman 编码反而不如定长编码省？

答：(1) 合并过程：$0.125+0.125 = 0.25$；再 $0.25(B)+0.25 = 0.5$；最后 $0.5(A)+0.5 = 1$。



字符	A	B	C	D
频率	0.5	0.25	0.125	0.125
编码	0	10	110	111
码长	1	2	3	3

平均码长 $= 0.5\times1 + 0.25\times2 + 0.125\times3 + 0.125\times3 = 1.75$ 位。定长编码 4 个字符需 2 位，所以 Huffman 每字符省 0.25 位（约 12.5%）。

(2) **D 的频率超过 $3/10$ 之后就不如定长**。设 D 的频率为 y，其余三个按原比例（$4:2:1$）缩放，它们原本合计 0.875、加权码长 $\sum f\cdot l = 1.75 - 0.125\times3 = 1.375$。这套**固定不变**的码的平均长度是

$$L(y) = \frac{1.375}{0.875}(1-y) + 3y = \frac{11}{7}(1-y) + 3y = \frac{11}{7} + \frac{10}{7}y$$

令 $L(y) = 2$（定长的位数）得 $y = \frac{3}{10}$。

验算 $y = 0.3$：其余按 $4:2:1$ 分掉 0.7，即 $A = 0.4$、$B = 0.2$、$C = 0.1$，$L = 0.4\times1 + 0.2\times2 + 0.1\times3 + 0.3\times3 = 2.0$ ✓ 恰好等于定长。

这一问考的是一句话：Huffman 树是按一组给定频率算出来的，频率一旦漂移而码表不变，它就不再最优，甚至可能输给定长编码。但别把结论说过头——**按新频率重算**出来的 Huffman 码永远不会差于定长（它是最优前缀码）；会输的是「用旧频率算出来、却拿去编新数据」的那套码。实际的压缩器要么随数据自适应地改码表，要么把码表随文件一起存下来，原因正在这里。

5-26. ★ 给定序列 $K = \{53, 17, 78, 9, 45, 65, 87, 23, 31\}$，(1) 用筛选法把它建成**最小堆**，写出每一步；(2) 在结果堆上删除最小元，写出调整后的数组；(3) 回到 (1) 的建堆结果，插入元素 20，写出 SiftUp 调整后的数组。

答：**筛选法**从最后一个分支结点 $i = \lfloor n/2 \rfloor - 1$ 开始倒着做 `SiftDown`。本题 $n = 9$，故从 $i = 3$ 开始。`SiftDown(i)` 的动作是：与两个孩子中**较小的**比较，若比它大就交换并继续下沉，否则停止。

初始（下标 0..8）：`[53, 17, 78, 9, 45, 65, 87, 23, 31]`

步	i	结点	孩子	动作	数组
1	3	9	23, 31	$9 \le 23$，不动	`53 17 78 9 45 65 87 23 31`
2	2	78	65, 87	$78 > 65$，交换	`53 17 65 9 45 78 87 23 31`

步	i	结点	孩子	动作	数组
3	1	17	9, 45	$17 > 9$, 交换；再看 17 的孩子 23、31, $17 \le 23$ 停	`53 9 65 17 45 78 87 23 31`
4	0	53	9, 65	$53 > 9$ 交换 → 53 到下标 1，孩子 17、45, $53 > 17$ 交换 → 53 到下标 3，孩子 23、31, $53 > 23$ 交换 → 53 到下标 7，是叶，停	`9 17 65 23 45 78 87 53 31`

建堆结果：`[9, 17, 65, 23, 45, 78, 87, 53, 31]` 校验：$9 \le 17, 65$；$17 \le 23, 45$；$65 \le 78, 87$；$23 \le 53, 31$ √

(2) **删除最小元** 9：把**末尾元素** 31 **移到根**、堆长减到 8，再对根 `SiftDown`：- 根 31，孩子 17、65 → $31 > 17$，交换 → `[17, 31, 65, 23, 45, 78, 87, 53]` - 31 在下标 1，孩子 23、45 → $31 > 23$，交换 → `[17, 23, 65, 31, 45, 78, 87, 53]` - 31 在下标 3，只有一个孩子 53 → $31 \le 53$，停。

结果：`[17, 23, 65, 31, 45, 78, 87, 53]`，取出的最小元是 9。

(3) **在 (1) 的建堆结果** `[9, 17, 65, 23, 45, 78, 87, 53, 31]` **上插入** 20：新元素先放到**末尾**（下标 8 之后的下标 9），再沿祖先路径**向上浮**，直到父结点不大于它、或者升到根为止。

位置	父结点	比较	结果
9	下标 $\lfloor (9-1)/2 \rfloor = 4$，值 45	$20 < 45$	交换 → `9 17 65 23 20 78 87 53 31 45`
4	下标 $\lfloor (4-1)/2 \rfloor = 1$，值 17	$20 > 17$	停

结果：`[9, 17, 65, 23, 20, 78, 87, 53, 31, 45]`，只交换了一次。

代价：建堆 $O(n)$（见 5-30）；删除最小元与插入都是 $O(\log n)$——路径长度不超过树高（层数）减 1，而完全二叉树的高度是 $\lceil \log_2(n+1) \rceil$（5-32）。

三**个常见错**：1. `SiftDown` 只跟左孩子比——必须跟**两个孩子中较小的**比；2. 建堆时从 $i = 0$ 正着做——那样子树还不是堆，筛选的前提不成立，结果是错的；3. **把插入的方向搞反**。记住两句话：**建堆是从最后一个分支结点倒着向下筛**（整体 $O(n)$），**插入是从末尾向上浮**（单次 $O(\log n)$）。正因为方向不同，逐个 `insert` 建堆要 $O(n \log n)$，而筛选法只要 $O(n)$（5-14、5-30）。

5-27. ★ 一棵 N 个结点的二叉树用两个一维数组存储：`L[i]`、`R[i]` 分别是结点 i 的左、右孩子编号，为 0 表示空（结点从 1 开始编号，`L[0] = R[0] = 0`，根的编号已知）。(1) 设计算法由 `L`、`R` 生成父指针数组 `T`，使 `T[i]` 为结点 i 的父结点编号（根记 0）；(2) 设计算法判断结点 U 与 V 是否在**同一条从根到叶的路径**上（即其中一个是另一个的祖先）。分析两问的代价。

答：(1) **基本思想——反着填**。与其对每个结点去「找父亲」（那要把 `L`、`R` 整个搜一遍，$O(N)$ 一次），不如让**每个结点把自己的两个孩子登记一次**。每条边只写一次，一趟就够。

```
BuildParent(L, R, N)
    for i ← 0 to N:  T[i] ← 0              // 根与空位都留 0
    for i ← 1 to N
        if L[i] ≠ 0 then T[L[i]] ← i
        if R[i] ≠ 0 then T[R[i]] ← i
    return T
```

正确性：二叉树里每个非根结点恰好被某一个结点当作左孩子或右孩子登记一次，所以每个 `T[i]` 至多被写一次、也至少被写一次；根从没被登记，保持 0。**代价**：时间 $\Theta(N)$，空间 $\Theta(N)$（就是 `T` 本身）。

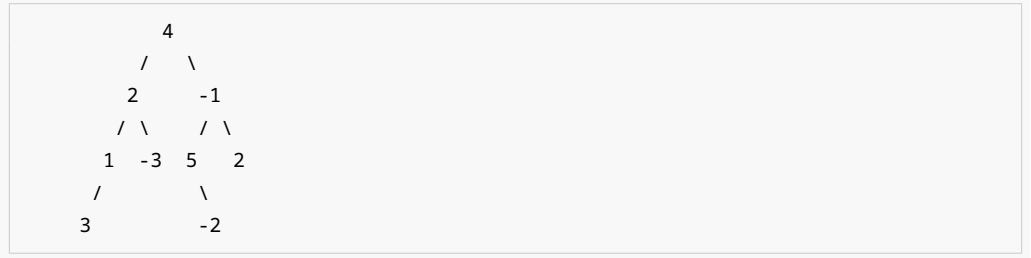
(2) **基本思想**——「在同一条根到叶的路径上」等价于「其中一个是另一个的祖先」。有了父指针，只要**从其中一个出发沿父链往上走**，看能不能碰到另一个；两个方向各试一次。

```
IsAncestor(x, y, T)              // 判断 y 是不是 x 的祖先（x ≠ y）
    p ← T[x]
    while p ≠ 0
        if p = y then return true
        p ← T[p]
    return false

SamePath(U, V, T)
    if U = V then return true
    return IsAncestor(U, V, T) or IsAncestor(V, U, T)
```

代价：时间 $O(h)$（h 为树高），最坏 $O(N)$（树退化成链）；空间 $O(1)$。**为什么要翻成「往上走」**：直接做是「从 V 出发把整棵子树遍历一遍，看里面有没有 U」，那是 $O(\text{子树大小})$、最坏 $O(N)$，还要一个栈。**往上走只有一条路，往下走有很多条**——这正是父指针表示法的长处，和 6-21 的并查集 `Find` 是同一个道理（见 6-16 的对照表）。

5-28. 二叉树的结点存整数（可正可负）。从某个结点出发、**只沿孩子方向往下走**、在它自己或它的某个后代处结束，经过的结点序列称为一条**向下路径**（不必从根开始，也不必在叶结束）。给定整数 `target`，设计算法统计结点值之和等于 `target` 的向下路径条数，并分析代价。对下面这棵树、`target = 3`，答案是多少？



答：5 **条**：$4 \to 2 \to -3$，$4 \to -1$，$2 \to 1$，3（单个结点），$5 \to -2$。**解法一：双重递归**——按起点分类：先数「从某个结点出发」的路径，再对所有起点求和。

```
CountFrom(p, need)           // 从 p 出发、和为 need 的向下路径条数
    if p = nullptr then return 0
    c ← 0
    if p->data = need then c ← 1          // 在 p 处结束的那一条
    rest ← need - p->data
    return c + CountFrom(p->left, rest) + CountFrom(p->right, rest)

PathSum(p, target)           // 子树 p 中和为 target 的向下路径数
    if p = nullptr then return 0
    c ← CountFrom(p, target)
```

```
    return c + PathSum(p->left, target) + PathSum(p->right, target)
```

注意：找到一条后**不能停**，也不能在部分和超过 `target` 时剪枝——结点值可以是负数，继续往下还可能再凑出 `target`（例如从 4 出发，$4 \to -1$ 和为 3，再往下 $4 \to -1 \to 5 \to -2$ 和为 6，若树里还有值为 -3 的孩子就又凑出 3）。**代价**：每个结点会被它的每个祖先（含自身）各发起的 `CountFrom` 访问一次，共 $\sum_v (\text{深度}_v + 1) = O(nh)$ 时间，退化成链时为 $O(n^2)$；递归栈 $O(h)$。

解法二：前缀和——一趟深度优先周游，维护从根到当前结点路径上的前缀和数组 `pre[0..d+1]`，`pre[0] = 0`。路径 $u \to \cdots \to v$ 的和等于 $\texttt{pre}_v - \texttt{pre}_{u\text{的父亲}}$，所以在 v 处结束且和为 `target` 的路径条数，就是路径上满足 `pre[d+1] - pre[j] = target`（$0 \le j \le d$）的 j 的个数。

```
Walk(p, d)                  // p 在第 d 层；pre[0..d] 已是根到 p 的父亲的前缀和
    if p = nullptr then return
    pre[d + 1] ← pre[d] + p->data
    for j ← 0 to d
        if pre[d + 1] - pre[j] = target then cnt ← cnt + 1
    Walk(p->left, d + 1);  Walk(p->right, d + 1)
```

调用前令 `pre[0] ← 0`、`cnt ← 0`，执行 `Walk(root, 0)`。代价同样是 $O(nh)$ 时间，但只周游一趟，额外空间 $O(h)$。若用散列表记录「各前缀和出现了几次」（第 10 章），每个结点只需查一次表，期望时间降到 $O(n)$；这时**从一个结点返回前必须把它的前缀和计数减回去**，否则兄弟子树会把不在同一条路径上的前缀算进来。数组版本没有这个问题：返回后 `pre[d+1]` 会被下一个兄弟直接覆盖。

五、数学证明

5-29. ★ 证明：非空二叉树中叶结点数 n_0 与度为 2 的结点数 n_2 满足 $n_0 = n_2 + 1$。

证明. 设结点总数为 n、度为 1 的结点数为 n_1，则按度分类有

$$n = n_0 + n_1 + n_2$$

换一个角度数**边**：除根之外每个结点都恰有一条边从它的父结点进入，故边数 $e = n - 1$；而每条边都是从某个度为 1 或 2 的结点射出的，故 $e = n_1 + 2n_2$。两式联立：

$$n_0 + n_1 + n_2 = n_1 + 2n_2 + 1 \Rightarrow n_0 = n_2 + 1$$

∎

推论（满二叉树定理）：满二叉树中 $n_1 = 0$，故**叶结点数 = 分支结点数 + 1**。**推论（空子树数）**：把所有空子树换成叶结点得到扩充二叉树 T'，它是满二叉树、且原树的每个结点在 T' 中都成了分支结点，故 T' 的叶数 = 原树结点数 + 1，即**一棵 n 个结点的二叉树恰有 $n+1$ 棵空子树**（链式存储时就是 $n+1$ 个空指针）。

5-30. ★ 证明：用筛选法把 n 个元素的无序序列建成堆，时间代价为 $O(n)$。

证明. 关键观察：**下沉的代价不是树高 $\log n$，而是「这个结点离最底层还有多远」**；越靠底层的结点越多，能下沉的距离却越短，两者恰好抵消。

先把堆补成一棵层数相同、**各层全满的二叉树**（深度即最大层号 $h = \lfloor \log_2 n \rfloor$，根在第 0 层）。满树每个位置能下沉的距离都不比原堆对应位置短，故在满树上算出的总代价是原堆的上界。各层全满的树第 i 层恰有 2^i 个结点，每个最多下沉 $h - i$ 层，于是

$$\text{建堆代价} \le \sum_{i=0}^{h} 2^i (h-i) = \sum_{j=0}^{h} 2^{h-j} \cdot j = 2^h \sum_{j=0}^{h} \frac{j}{2^j},$$

而

$$2^h \sum_{j=0}^{h} \frac{j}{2^j} < 2^h \cdot 2 = 2^{h+1} \le 2n,$$

其中用到 $\sum_{j \ge 0} j/2^j = 2$ 与 $2^h \le n$。故建堆为 $O(n)$。∎

对照：逐个 `Insert` 建堆是 $O(n \log n)$——每次上浮都可能走满一条树高，而**上浮的代价才真正是 $\log n$**（越靠底层的结点离根越远）。两种建法的差别正好说明：**同一个结构，调整方向不同，代价可以差一个 log**。

5-31. 证明：n 个结点的不同二叉树形态共有 $C_n = \frac{1}{n+1} C_{2n}^{n}$ 种。

证明. 设 b_n 为答案，$b_0 = 1$（空树算一种）。一棵 n 个结点的非空二叉树由根、左子树、右子树构成；若左子树有 i 个结点，则右子树有 $n-1-i$ 个（$0 \le i \le n-1$），两者独立，故

$$b_n = \sum_{i=0}^{n-1} b_i \, b_{n-1-i}$$

这与第 3 章出栈序列个数的递推式（3-26）**完全相同**，初值也相同，故 $b_n = C_n$。∎

两者之间其实有直接的一一对应：递归定义编码 $E(\emptyset) = \epsilon$、

$E(T) = I\,E(T_{左})\,O\,E(T_{右})$。首个 I 与其配对的 O 唯一分出左右子树编码，因此合法 I/O 序列与二叉树形态一一对应。记住这个数：$1, 1, 2, 5, 14, 42, 132, \cdots$

5-32. 证明：n 个结点的完全二叉树高度（层数）为 $\lceil \log_2(n+1) \rceil$。

证明. 设高度为 h（即层号 $0..h-1$，共 h 层）。完全二叉树的前 $h-1$ 层是满的，共 $2^{h-1}-1$ 个结点；第 $h-1$ 层至少 1 个、至多 2^{h-1} 个。故

$$2^{h-1} - 1 + 1 \le n \le 2^h - 1 \Rightarrow 2^{h-1} \le n < 2^h$$

即 $2^{h-1} < n+1 \le 2^h$，两边取以 2 为底的对数得 $h-1 < \log_2(n+1) \le h$，于是 $h = \lceil \log_2(n+1) \rceil$。∎

用处：这正是堆的插入、删除为什么是 $O(\log n)$ ——一次上浮或下沉最多走 $h-1$ 步。

第 6 章 树

一、填空题

6-1. 树是 n（$n \ge 1$）个结点的有限集合。其中有且仅有一个 ____ 结点，其余结点可分为 m 个 ____ 的有限集合，每一个又都是一棵树，称为根的 ____。m 棵互不相交的树的集合称为 ____。

答：**根**；**互不相交**；**子树**；**森林**。「互不相交」这四个字是树与图的分界：一旦两棵子树共享结点，就出现了回路或多个前驱，那是图。

6-2. 结点的**度**是指 ____；树的度是指 ____。若一棵树中度为 i 的结点有 n_i 个（$i = 0, 1, \cdots, m$），则该树的总结点数 $n =$ ____，且有 $n =$ ____（用度数表示）。

答：该结点的**子树个数（孩子数）**；树中**所有结点度的最大值**；$n = \sum_{i=0}^{m} n_i$；$n = 1 + \sum_{i=1}^{m} i \cdot n_i$。后一条的来历：**除根之外每个结点都恰有一条边指向它**，而边的总数等于所有结点度之和。这是求叶结点数一类填空题的万能工具（见 6-9）。

6-3. 一棵树的嵌套括号表示为 `(A(B(E)(F))(C)(D(G(H))))`。该树共有 ____ 个结点，树的度是 ____，叶结点有 ____ 个，度为 1 的结点有 ____ 个。

答：8；3；4；2。

读法：**紧跟在一个结点后面的每一对括号，就是它的一棵子树**。拆开来是

```
A
├── B
│   ├── E
│   └── F
```

```
├── C
└── D
    └── G
        └── H
```

各结点的度：A 为 3，B 为 2，D 为 1，G 为 1，其余（E F C H）为 0。故结点数 $n = 8$；**树的度**是所有结点度的最大值，为 3；叶结点是 E F C H 共 4 个；度为 1 的是 D G 共 2 个。

用 6-2 的度数公式自检，这一步能挡住绝大多数数错：

$$n = 1 + \sum_{i} i \cdot n_i = 1 + (1 \times 2 + 2 \times 1 + 3 \times 1) = 1 + 7 = 8$$

注意别把「结点的度」和「树的度」混了：前者是某个结点的孩子数，后者是全树的最大值。

6-4. ★ 森林转换成二叉树的规则是：____ 作为二叉树的左子树，____ 作为右子树。转换后，二叉树根结点的**右子树**对应原森林中的 ____。

答：结点的**第一个孩子**（长子）作左子树；结点的**下一个兄弟**作右子树（即「**左子/右兄**」规则）；根的右子树对应原森林中**除第一棵树之外的其余各树**。一个立刻可用的推论：**一棵树（而非森林）转成的二叉树，其根的右子树必为空**。

6-5. ★ 按**先根次序**周游森林，等同于按 ____ 次序周游它对应的二叉树；按**后根次序**周游森林，等同于按 ____ 次序周游它对应的二叉树。二叉树的前序、中序、后序三种深度优先遍历中，不能保证与森林的先根或后根任何一种相同的是 ____ 遍历。

答：**前序**；**中序**；**后序**。例如根 A 只有孩子 B 时，森林的先根为 AB、后根为 BA，对应二叉树后序为 BA；而森林仅由两个单结点树 A、B 组成时，先根和后根均为 AB，对应二叉树后序却为 BA。因而没有上述两种周游中的一种能对所有森林都对应二叉树后序。这两条对应关系是本章最常考的结论，证明见 6-26。

6-6. ★ 完全 K 叉树按层次从 0 开始编号，则结点 i 的第 j 个孩子（$1 \le j \le K$）编号为 ____，其父结点编号为 ____；第 i 层最多有 ____ 个结点；高度为 h 层且各层全满的 K 叉树共有 ____ 个结点。

答：$Ki + j$；$\lfloor (i-1)/K \rfloor$（$i > 0$，根无父结点）；K^i；$\frac{K^h - 1}{K - 1}$（共 h 层，层号 $0..h-1$）。$K = 2$ **时全部退化为二叉树的公式**：孩子 $2i+1, 2i+2$，父 $\lfloor (i-1)/2 \rfloor$，第 i 层 2^i 个，h 层共 $2^h - 1$ 个——可以用这个特例检验自己有没有记错。（若编号从 1 开始，则孩子为 $K(i-1)+j+1$，父为 $\lceil (i-1)/K \rceil$。）

6-7. 一棵有 n 个结点的 K 叉树，采用「每个结点固定 K 个孩子指针」的链式存

储时，其中空指针的个数为 ____。

答：$nK-n+1=n(K-1)+1$。理由：共 nK 个指针域，非空的恰好指向除根之外的每个结点，即 $n-1$ 个。$K=2$ 时得 $n+1$，正是第 5 章「n 个结点的二叉树有 $n+1$ 棵空子树」那条性质。

6-8. 并查集（等价类）用 ____ 表示法存储，`Find` 操作沿 ____ 找到 ____，两个元素属于同一集合当且仅当 ____。两条常用的优化是 ____ 和 ____。

答：**父指针**；**父链向上**；**树根**；**它们的根相同**；**重量权衡合并规则**（元素少的树根挂到元素多的树根下）和**路径压缩**。注意别和「按秩合并」混了：**按秩比的是树高，按重量比的是元素个数**，两者复杂度同阶，但在同一组等价对上长出的树形不同。教材与本课程用的是**重量**。

二、选择题

6-9. 一棵度为 4 的树中，度为 1、2、3、4 的结点分别有 4、2、1、1 个，则叶结点有（ ）个。A. 6　B. 7　C. 8　D. 9

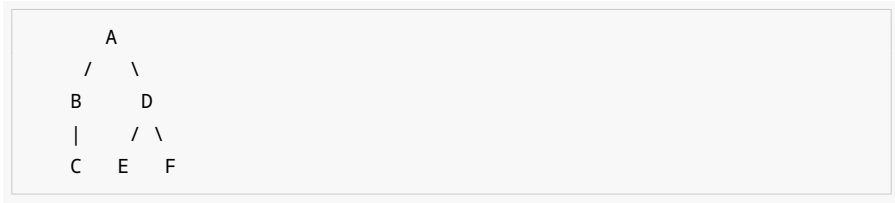
答：C。用 $n=1+\sum i\cdot n_i$：$n=1+(1\times4+2\times2+3\times1+4\times1)=1+15=16$。又 $n=n_0+n_1+n_2+n_3+n_4=n_0+4+2+1+1$，故 $n_0=16-8=8$。**这类题一律两步走**：先用度数公式求总结点数，再用分类求和求叶子数。

6-10. ★ 森林 F 中有 3 棵树，结点数分别为 n_1,n_2,n_3。把 F 转换成二叉树 B 后，B 的根结点的**右子树**上的结点个数是（ ）。A. n_1　B. n_1+n_2　C. n_2+n_3　D. n_3

答：C。B 的根是第一棵树的根，它的**左子树**装第一棵树的其余 n_1-1 个结点，**右子树**装剩下的两棵树，共 n_2+n_3 个。由此还可得一条常考结论：**森林中树的棵数 = 二叉树中从根出发沿右链能走到的结点数**。

6-11. 已知一棵树的先根序列为 `ABCDEF`、后根序列为 `CBEFDA`，则下列说法正确的是（ ）。A. `C` 是 `B` 的孩子，`E`、`F` 都是 `D` 的孩子 B. `A` 有三个孩子 C. 该树的度为 3 D. 该树的层次序列为 `ABCDEF`

答：A。还原过程与二叉树「前序 + 中序」定树同理（依据见 6-26）：先根首字符是根 `A`，后根末字符也是 `A` √。`A` 的第一棵子树在先根里从 `B` 起、在后根里到 `B` 止，故为 `{B,C}`；由先根 `BC`、后根 `CB` 知 `C` 是 `B` 的孩子。余下 `{D,E,F}` 是 `A` 的第二棵子树，先根 `DEF`、后根 `EFD` 说明 `E`、`F` 都是 `D` 的孩子。



所以 `A` 只有两个孩子（B 错）；树的度是 2（C 错）；层次序列是 `A B D`

C E F（D 错）。

6-12. ★ 使用**重量权衡合并规则**的并查集（不含路径压缩），n 个元素任意合并后，树的最大高度的渐进紧确界是（　）。A. $\Theta(1)$　B. $\Theta(\log n)$　C. $\Theta(\sqrt{n})$　D. $\Theta(n)$

答： B。证明见 6-28：每次合并只可能让被挂上去那棵树里的结点深度加 1，而发生这种事时它所在集合的元素个数**至少翻倍**，元素数最多翻 $\log_2 n$ 次。等大集合逐层合并可达到 $\Theta(\log n)$ 高度；若不用任何优化，最坏就是 D（每次都把大树挂到单元素树下，长成一条链）。

6-13. 对同一片森林，下列两个序列**一般不相同**的是（　）。A. 森林的先根序列 与 对应二叉树的前序序列 B. 森林的后根序列 与 对应二叉树的中序序列 C. 森林的层次序列 与 对应二叉树的层次序列 D. 以上三组都相同

答： C。A、B 是本章的两条基本对应（6-5、6-26）。C 不成立：二叉树的层次遍历是按「离二叉树的根的距离」分层，而转换后**兄弟关系变成了右链**，原本同层的兄弟在二叉树里被拉成了一条向下的链，层号全变了。以 6-18 的森林为例，森林层次序列是 A G B C D H I E F J，而对应二叉树的层次序列是 A B G C H E D J I F——完全不同。

三、辨析与简答

6-14. 辨析：「因为森林和二叉树可以一一对应，所以它们是同一种数据结构。」

答：错。一一对应说的是**存储表示可以互相转换**，不等于逻辑结构相同：- 森林中一个结点可以有任意多个孩子，孩子之间是**兄弟关系**；二叉树中每个结点至多两个孩子，且**左右有别**。- 转换后**语义变了**：二叉树里的「右孩子」在森林里是「兄弟」而不是「孩子」，所以在二叉树上求出的「高度」并不是森林的高度。- 森林没有中根遍历，二叉树有中序遍历——**运算集合本身就不同**。

正确的说法是：「**左子/右兄**」**二叉链是森林的一种存储结构**，它让森林能复用二叉树的算法。

6-15. ★ 简答：为什么树（森林）没有「中根遍历」？先根、后根与对应二叉树的前序、中序 为什么会一一对应？

答：(1) **没有中根遍历**：中序的定义是「先左子树、再根、后右子树」，前提是子树恰好分成根的左右两半。树的结点可以有 m 棵子树，「根在第几棵子树之后访问」没有自然的选择，所以中根次序无法定义。

(2) **对应关系的来历**：设森林 F 的第一棵树为 T_1、其余为 F'。按左子/右兄转换后，T_1 的诸子树构成的森林成了 $B(F)$ 的**左子树**，F' 成了**右子树**。- **先根次序**：访问 T_1 的根 → 周游 T_1 的子树森林 → 周游 F'，正是二叉树的「根 → 左 → 右」，即**前序**；- **后根次序**：周游 T_1 的子树森林 → 访问 T_1 的根 → 周游 F'，正是二叉树的「左 → 根 → 右」，即**中序**。

严格证明见 6-26（对结点数作归纳）。

6-16. 简答：比较树的三种链式存储（子结点表、左子/右兄二叉链、父指针表），各自擅长什么运算？

答：

表示法	存什么	擅长	不擅长
子结点表	每个结点带一张孩子链表	**自上而下**：找某结点的全部孩子、按层遍历	找父结点要全表搜索
左子/右兄二叉链	每结点两个指针：长子、下一兄弟	结点大小固定、能直接复用二叉树算法；**深度优先周游**	找第 k 个孩子要顺着兄弟链走 k 步；找父结点仍不便
父指针表	每结点只存父结点下标	**自下而上**：`Find` 根、判断同集合、**并查集**	找孩子要全表搜索，无法高效自上而下遍历

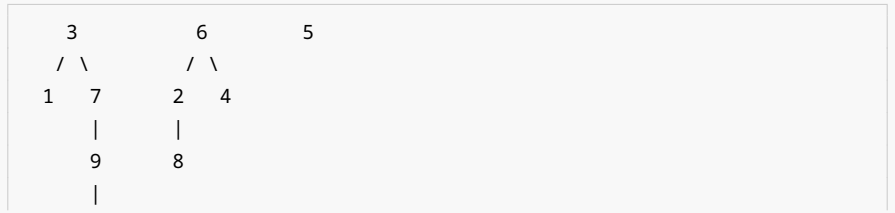
一句话：**指针指向哪边，哪边的运算就快**。本章小结说「左子/右兄二叉链是采用得最多的表示法」，是因为它结点大小固定、又能复用二叉树的整套算法。

6-17. ★ 并查集的另一种常见存法：数组 `parent[1..n]` 中，非根结点存父结点编号，**根结点存「−（该集合的元素个数）」**。现有 10 个元素：

```
i          1  2  3  4  5  6  7  8  9  10
parent[i]  3  6 -5  6 -1 -4  3  2  7  9
```

(1) 画出它表示的森林，写出各等价类；
(2) 采用**重量权衡合并** + **路径压缩**，依次执行 `Union(10, 8)`、`Union(5, 4)`，写出每一步之后的数组；
(3) 这种存法比 6-21 的「`parent` 加 `weight` 两个数组」好在哪里？

答：(1) 负数所在的位置就是根：`parent[3] = -5`、`parent[6] = -4`、`parent[5] = -1`，共三棵树。



```
10
```

等价类为 $\{1,3,7,9,10\}$（5 个）、$\{2,4,6,8\}$（4 个）、$\{5\}$（1 个）；负数的绝对值正好是各树的元素个数，$5+4+1=10$ √。

(2)

操作	两次 Find（含路径压缩）	合并	parent[1..10]
Union(10, 8)	Find(10) 走 $10\to 9\to 7\to 3$，把 10、9 直接挂到 3；Find(8) 走 $8\to 2\to 6$，把 8 直接挂到 6	5 个 > 4 个，6 挂到 3 下，parent[3] ← $-5+(-4)=-9$	3 6 -9 6 -1 3 3 6 3 3
Union(5, 4)	Find(5) = 5；Find(4) 走 $4\to 6\to 3$，把 4 直接挂到 3	1 个 < 9 个，5 挂到 3 下，parent[3] ← -10	3 6 -10 3 3 3 3 6 3 3

最终只剩一棵树：3 的孩子是 1、4、5、6、7、9、10，6 的孩子是 2、8，共 3 层。**两个细节**：第一步压缩 Find(8) 时根还是 6，所以 8 挂到的是 6 而不是 3；第二步 Find(4) 只压缩 4 自己这条路径，2、8 不在路径上，仍挂在 6 下——**路径压缩只动查找路径上的结点**。本题没有出现重量相等的情形；若出现，按 6-21 的约定把值大的根挂到值小的根下。

(3) **一个数组同时回答两个问题**：parent[x] < 0 就是根，不必再约定 0 或 −1 作根的标记；根上的绝对值就是重量，省掉了 weight 数组的 n 个整数。合并时一句 parent[r] ← parent[r] + parent[s] 就更新了新根的重量（两个负数相加仍是负数，仍标记着根）。非根结点不需要知道自己子树的大小，所以丢掉这部分信息不影响算法。

四、算法设计与分析

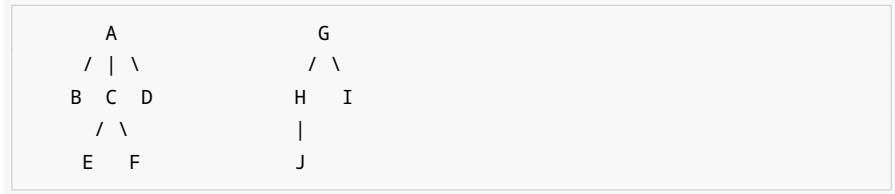
本组题统一使用「左子/右兄」表示：

```
结点 TNode
    data        结点数据
    child       指向第一个孩子（长子），无孩子为 nullptr
```

```
    sibling    指向下一个兄弟，无兄弟为 nullptr
```

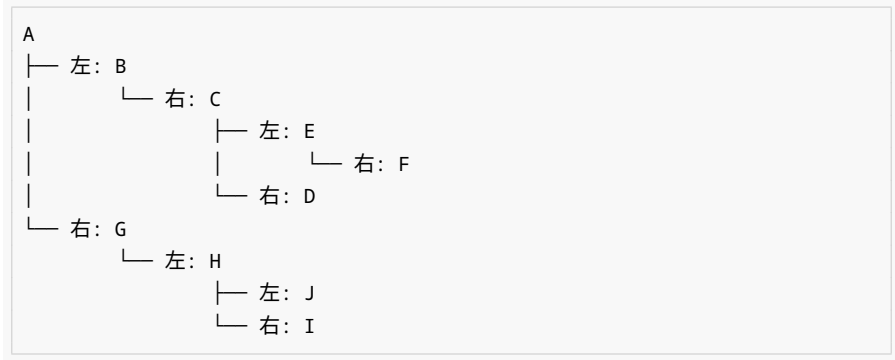
6-18. ★ 已知森林 F 由两棵树组成：第一棵的根是 A，A 的孩子依次是 B、C、D，C 的孩子是 E、F；第二棵的根是 G，G 的孩子是 H、I，H 的孩子是 J。(1) 画出 F；(2) 写出先根、后根、层次三种序列；(3) 画出 F 对应的二叉树，并写出它的前序、中序序列，与 (2) 对照。

答：(1) **森林** F：



(2) 三**种序列**：- 先根（访问根 → 依次周游各子树 → 再周游其余的树）：A B C E F D G H J I - 后根（先周游各子树 → 访问根 → 再周游其余的树）：B E F C D A J H I G - 层次（按离各自树根的距离分层，同层从左到右）：A G B C D H I E F J

(3) **对应二叉树**（左子 = 长子，右兄 = 下一个兄弟）：



逐点说明：A 的左孩子是长子 B、右孩子是下一棵树的根 G；B 无孩子，右孩子是兄弟 C；C 的左孩子是长子 E、右孩子是兄弟 D；E 的右孩子是兄弟 F；G 的左孩子是 H、无右兄弟；H 的左孩子是 J、右孩子是兄弟 I。

- 二叉树**前序**：A B C E F D G H J I = 森林的**先根序列** ✓
- 二叉树**中序**：B E F C D A J H I G = 森林的**后根序列** ✓
- 二叉树**层次**：A B G C H E D J I F，与森林的层次序列 A G B C D H I E F J **不同**（见 6-13 的说明）。

6-19. ★ 在「左子/右兄」表示下，写出森林的**先根**、**后根**周游算法（递归）和**层次**周游算法（用队列），并分析代价。

答：注意「左子/右兄」里 `sibling` 链上的结点是**同一层的兄弟**，所以递归要沿 `child` 下去、沿 `sibling` 横着走。

```
PreRoot(p)                        // 先根：根 → 各子树 → 其余的树
    while p ≠ nullptr
        访问 p
        PreRoot(p->child)
        p ← p->sibling

PostRoot(p)                       // 后根：各子树 → 根 → 其余的树
    while p ≠ nullptr
        PostRoot(p->child)
        访问 p
        p ← p->sibling

LevelOrder(p)                     // 层次：用队列
    Q ← 空队列
    while p ≠ nullptr             // 先把森林里所有树的根全部入队
        Q.EnQueue(p);  p ← p->sibling
    while not Q.Empty()
        q ← Q.DeQueue()
        访问 q
        c ← q->child              // 把 q 的全部孩子依次入队
        while c ≠ nullptr
            Q.EnQueue(c);  c ← c->sibling
```

代价：三者都是每个结点访问一次，时间 $\Theta(n)$。空间：递归版是栈深 $O(h)$（h 为树高，退化时 $O(n)$）；层次版是队列长度 $O(w)$（w 为森林的最大宽度，最坏 $O(n)$）。

评分要点：(1) 两个递归里的 `while p ≠ nullptr` 不能少——它负责横着走完兄弟链，漏掉就只周游了第一棵树；(2) 层次遍历必须先把**所有树根**入队，森林不止一棵树；(3) 要说明「访问」的位置差别正是先根与后根的唯一区别。

6-20. ★ **算法填空**：多叉树的**非递归后根周游**。设树另提供操作 `Child(p, k)`：返回结点 p 的第 k 个孩子（k 从 1 起），不存在时返回 `nullptr`。栈中每条记录是二元组 `(p, k)`，含义是「结点 p 接下来该处理它的第 k 个孩子」。

```
PostOrder(root)
    if root = nullptr then return
    S ← 空栈;  S.Push((root, 1))
    while not S.Empty()
        (p, k) ← S.Pop()
        c ← Child(p, k)
        if ____(①)____
            访问 p
        else
            S.Push(____(②)____)
```

```
            S.Push(____(③)____)
```

(1) 补全空缺；(2) ② 与 ③ 两句能否交换次序？(3) 设一棵树的根 A 有孩子 B、C、D，C 有孩子 E、F，D 有孩子 G。写出访问序列，以及访问 E 时栈中的记录（自底向上）；(4) 分析代价。

答：(1) ① `c = nullptr`　② `(p, k + 1)`　③ `(c, 1)`

后根次序要求「所有孩子都处理完才访问自己」。记录里的 k 就是 5-21 末尾说的**标记**：它记住这个结点已经处理到第几个孩子。对照递归写法 `for k ← 1 to p 的度数: PostOrder(Child(p, k));　然后访问 p`，k 正是循环变量，也就是第 3 章「用栈消除递归」里要保存的**返回点**（3-15）。

(2) **不能交换**。栈后进先出，必须先压 `(p, k+1)`、再压 `(c, 1)`，孩子 c 才会先出栈，它的整棵子树处理完之后才轮到 p 的下一个孩子。交换之后 p 的记录先出栈，会一路跳过所有孩子、先访问 p 本身——第 (3) 问那棵树会最先访问 A，后根次序全乱。

(3) 访问序列 B E F C G D A。访问 E 时栈自底向上为 `(A, 3)`、`(C, 2)`：A 下一个该处理第 3 个孩子 D，C 下一个该处理第 2 个孩子 F——栈里恰好是从根到 E 这条路径上的祖先。

(4) 结点 p 共出栈「度数 + 1」次（每个孩子一次，最后一次用来访问自己），总出栈次数为 $\sum_p (\text{度数}_p + 1) = (n-1) + n = 2n-1$。若 `Child(p, k)` 是 $O(1)$（如孩子存在数组里），总时间 $O(n)$；栈深不超过树的层数 h，空间 $O(h)$。

实现时要留意：若树用本组的「左子/右兄」存储，`Child(p, k)` 得从长子沿 `sibling` 走 $k-1$ 步，每次 $O(k)$，总时间退化成 $O(\sum_p \text{度数}_p^2)$，一个根带 $n-1$ 个孩子时为 $O(n^2)$。改法是记录里**不存序号 k，直接存「下一个要处理的孩子」的指针**：初值为 `p->child`，每次推进为 `c->sibling`，又回到 $O(n)$。另一条路：森林的后根次序就是对应二叉树的中序（6-5），所以在左子/右兄结构上直接套 5-21 的中序框架（`left` 换成 `child`、`right` 换成 `sibling`）也行。

6-21. ★ 写出并查集的 `Find` 与 `Union`，要求实现**重量权衡合并**与**路径压缩**，并分析代价。

答：用数组 `parent[0..n-1]` 存父结点下标，根的 `parent` 记为 -1；另用 `weight[r]` 记录以 r 为根的集合的**元素个数**。

```
MakeSet(n)
    for i ← 0 to n - 1
        parent[i] ← -1;  weight[i] ← 1

Find(x)                                    // 带路径压缩
```

```
    r ← x
    while parent[r] ≠ -1                  // 第一趟：找到根
        r ← parent[r]
    while parent[x] ≠ -1                 // 第二趟：把沿途结点全部直接挂到根上
        next ← parent[x]
        parent[x] ← r
        x ← next
    return r

Union(a, b)                               // 重量权衡合并
    ra ← Find(a);  rb ← Find(b)
    if ra = rb then return                // 本来就同一集合，什么也不做
    if weight[ra] < weight[rb] or (weight[ra] = weight[rb] and ra > rb)
        parent[ra] ← rb;  weight[rb] ← weight[rb] + weight[ra]
    else
        parent[rb] ← ra;  weight[ra] ← weight[ra] + weight[rb]
```

两条优化各管什么：- **重量权衡合并**：小集合挂到大集合下，保证树高 $O(\log n)$（证明见 6-28）；- **路径压缩**：`Find` 的时候顺手把这条路径上的结点全部直接接到根上，**在根尚未因后续合并而改变时，再查这些结点只需一步**。注意压缩发生在**查找**时，不是合并时。

代价：只用重量权衡合并时，单次操作最坏 $O(\log n)$；只用路径压缩时，单次 `Find` 最坏仍可能是 $O(n)$（树可以先被合并成一条链），但 m 次操作摊还下来每次是 $O(\log n)$ 量级；两条同时用时，m 次操作的总代价是 $O(m\,\alpha(n))$，α 是反 Ackermann 函数，实际中不超过 4，**这是摊还界，单次操作最坏仍为** $O(\log n)$。初始化另需 $\Theta(n)$。

易错点：`Union` 里必须先 `Find` 出两个根再比较——直接对 `a`、`b` 操作是错的；权值并列时要有确定的取舍规则（本课程约定**值大的根挂到值小的根下**），否则画出的树形对不上答案。

6-22. ★ 设有 8 个元素 $1..8$，依次读入等价偶对 $(1,2)$，$(3,4)$，$(5,6)$，$(7,8)$，$(2,4)$，$(6,8)$，$(4,8)$。采用**重量权衡合并 + 路径压缩**，写出每一步之后的父指针数组，并给出最终的等价类。

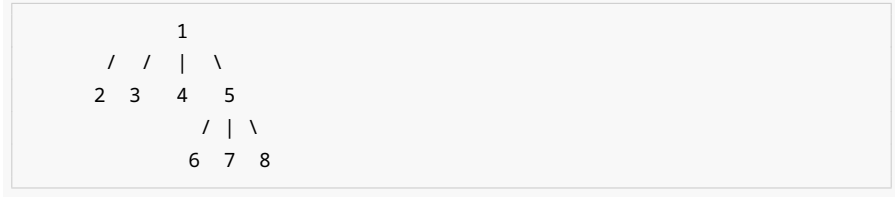
答：约定根的父记为 0；权值并列时**值大的根挂到值小的根下**。

步	偶对	`Find` 结果	合并动作	`parent[1..8]`
0	初始	—	—	`0 0 0 0 0 0 0 0`
1	(1,2)	1, 2（权 1、1，并列）	2 挂到 1 下，$w_1 = 2$	`0 1 0 0 0 0 0 0`
2	(3,4)	3, 4（并列）	4 挂到 3 下，$w_3 = 2$	`0 1 0 3 0 0 0 0`

步	偶对	Find 结果	合并动作	parent[1..8]
3	(5,6)	5, 6（并列）	6 挂到 5 下，$w_5 = 2$	`0 1 0 3 0 5 0 0`
4	(7,8)	7, 8（并列）	8 挂到 7 下，$w_7 = 2$	`0 1 0 3 0 5 0 7`
5	(2,4)	1, 3（权 2、2，并列）	3 挂到 1 下，$w_1 = 4$	`0 1 1 3 0 5 0 7`
6	(6,8)	5, 7（并列）	7 挂到 5 下，$w_5 = 4$	`0 1 1 3 0 5 5 7`
7	(4,8)	1, 5（权 4、4，并列）	5 挂到 1 下，$w_1 = 8$；**路径压缩**把 4 直接挂到 1、8 直接挂到 5	`0 1 1 1 1 5 5 5`

第 7 步的细节：`Find(4)` 走 $4 \to 3 \to 1$，压缩后 `parent[4]` ← `1`；`Find(8)` 走 $8 \to 7 \to 5$，压缩后 `parent[8]` ← `5`。之后才做合并 `parent[5]` ← `1`。

最终父指针数组：`parent[1..8] = 0 1 1 1 1 5 5 5`，森林形如



最终只有一个等价类 $\{1,2,3,4,5,6,7,8\}$，树高只有 3 层（最深路径 $6 \to 5 \to 1$）。若不用任何优化、每次都把前一个根挂到后一个根下，这组输入会长成以 8 为根、最深路径 $1 \to 2 \to 4 \to 8$ 的树，树高 4 层（`parent[1..8] = 2 4 4 8 6 8 8 0`），在这么小的数据上还看不出差距；但偶对若按 $(1,2),(2,3),\cdots,(7,8)$ 的次序读入，不加优化就会长成 $1 \to 2 \to \cdots \to 8$ 的一条链，树高 8 层，之后每次 `Find(1)` 都要走 $O(n)$ 步；而重量权衡合并下这 7 个偶对只会长出一棵以 1 为根、高度 2 层的树 ——这就是两条优化的价值。

6-23. ★ 已知一片森林的**带双标记的先根次序**表示，写出还原成「左子/右兄」链式结构的算法，并对 6-18 那片森林给出它的双标记序列。

答：**基本思想**——先根次序有一条性质：**任何结点的子树都紧跟在它后面**，子树排完才轮到它的下一个兄弟。于是「谁是某个结点的右兄弟」要等它整棵子树扫完才知道，用一把**栈**记着这笔欠账：扫到「有

下一个兄弟」的结点就把它压栈；扫到「没有孩子」的结点（说明一棵子树到头了）就弹出一个，把**下一个**建出来的结点接成它的右兄弟。

```
BuildFromDualTag(a[0..n-1])       // 每项有 data、has_child、has_sibling;
↪  假设标记合法
    if n = 0 then return nullptr
    S ← 空栈;  root ← nullptr
    owner ← nullptr;  side ← ROOT   // 下一个结点应填入的位置
    for i ← 0 to n - 1
        p ← 新结点 (a[i].data);  p->child ← p->sibling ← nullptr
        if side = ROOT then root ← p
        else if side = CHILD then owner->child ← p
        else owner->sibling ← p
        if a[i].has_sibling then S.Push(p)
        if a[i].has_child
            owner ← p;  side ← CHILD
        else if not S.Empty()
            owner ← S.Pop();  side ← SIBLING
        else
            if i ≠ n - 1 then 报错并终止     // 结构已结束但还有多余结点
            return root
    报错并终止                                // 序列结束但仍有孩子或兄弟待填
```

6-18 **那片森林的双标记序列**（先根次序 `A B C E F D G H J I`，`ltag = 0` 表示**有孩子**，`rtag = 0` 表示**有下一个兄弟**）：

```
先根次序   A  B  C  E  F  D  G  H  J  I
ltag       0  1  0  1  1  1  0  0  1  1      0 = 有孩子
rtag       0  0  0  0  1  1  1  0  1  1      0 = 有下一个兄弟
```

这两行最容易抄反。原书图 6.15 的三行从上到下印的是 `rtag` / `info` / `ltag`——`ltag` **在最下面**，照着「第一行就是 ltag」抄就会把两行互换。按定义核一遍即可：`B` 是叶结点，`ltag` 必须是 1；`B` 有兄弟 `C`，`rtag` 必须是 0。顺带一句，**先根序列相同的森林可以不止一片**——互换这两行还原出来的森林里，`C` 会变成 `B` 的孩子而不是兄弟，可它的先根序列**恰好还是** `A B C E F D G H J I`。所以这类题自检时要**同时对后根序列**（本题是 `B E F C D A J H I G`）：先根加后根才唯一确定一棵树，只对先根是查不出结构错的。

逐个核对：`A` 有孩子 `B`（ltag 0）、有下一棵树的根 `G` 作兄弟（rtag 0）；`B` 是叶（ltag 1）、有兄弟 `C`（rtag 0）；`C` 有孩子 `E`、有兄弟 `D`；`E` 是叶、有兄弟 `F`；`F`、`D` 都是又无孩子又无兄弟（1 1）；`G` 有孩子 `H`、无兄弟（0 1）；`H` 有孩子 `J`、有兄弟 `I`（0 0）；`J`、`I` 都是 `1 1`。

代价：时间 $\Theta(n)$（每个结点入栈至多一次），辅助栈空间 $O(h)$，输出结点本身占 $\Theta(n)$ 空间（h 为森林层数）。

这类顺序存储题的通用解法：不必死记四种表示法（带右链先根、带双标记先根、带度数后根、带双标记层次），只要抓住一条—— **它们都是「某种遍历序列 + 刚好够还原结构的附加信息」**。做题时先判断是哪种遍历次序，再想「这点附加信息够不够定出父子和兄弟关系」，现推比背要可靠得多。

6-24. 在「左子/右兄」表示下，写出求森林中结点总数与森林高度的算法。

答：仍是「沿 `child` 下去、沿 `sibling` 横着走」，但**两条链的含义不同**：走 `child` 层号加一，走 `sibling` 层号不变——这是本题唯一的陷阱。

```
Count(p)                          // 结点总数
    if p = nullptr then return 0
    return 1 + Count(p->child) + Count(p->sibling)

Height(p)                         // 森林高度（层数；空森林为 0）
    if p = nullptr then return 0
    return max(1 + Height(p->child),  Height(p->sibling))
                ↑ 往下走才加一          ↑ 兄弟仍在同一层，不加
```

代价：时间 $\Theta(n)$，空间 $O(n)$（最坏的递归深度）。**对照易错写法**：把 `Height` 写成 `1 + max(Height(child), Height(sibling))`，那求出来的是**对应二叉树的高度**，不是森林的高度—— 这正是 6-14 里「转换之后语义变了」的具体例子。

6-25. ★ 一棵树的**镜面映射**指把每个结点的子结点次序反转。输入输出树时常把树转成对应的二叉树，并给只有一个子结点的分支结点补一个虚子结点 `$`，形成「**伪满二叉树**」（每个分支结点恰有两个子结点）；再对这棵二叉树作**前序遍历**，内部结点标 0、叶结点标 1，虚结点也输出。现给定这样的前序序列，设计算法输出**原树的镜面映射的广度优先（层次）序列**。以 `x0 p0 $1 q0 s0 $1 t1 r1 $1` 为例，应输出 `x r q p t s`。

答：三步，每一步都是学过的东西。

第一步：按前序递归建「伪满二叉树」。伪满的意思是**每个内部结点恰有两个孩子**，所以读到标 0 的记号就递归读左、再递归读右；读到标 1 的记号就是叶子，不再往下读。`$` 建成空指针。

```
Build(tokens)                        // tokens 是只能顺序取的记号流
    t ← tokens.next()
    if t.ch = '$' then return 空     // 虚结点，它一定标 1，不再往下读
    建结点 node(t.ch)
    node.left ← node.right ← 空
    if t.flag = 0                    // 内部结点，恰有两个孩子
        node.left  ← Build(tokens)
        node.right ← Build(tokens)
    return node
```

第二步：按「左子/右兄」把二叉树读回成树。不必真的另建一棵——只要记住 `node.left` 是**长子**、`node.right` 是**下一个兄弟**（6-4），于是「结点 p 的全部孩子」就是**从 `p.left` 出发的那条右链**。

第三步：层次遍历，孩子从右往左入队。镜面映射是把每个结点的孩子反序，而层次遍历只关心「孩子按什么顺序入队」—— 所以**根本不用真的去反转这棵树**，入队时倒着放就等价了。

```
MirrorLevelOrder(root)
    if root = 空 then return
    Q ← 空队列;  Q.EnQueue(root)
    while not Q.Empty()
        p ← Q.DeQueue()
        输出 p.ch
        C ← 空表
        for c ← p.left;  c ≠ 空;  c ← c.right      // 收集 p 的全部孩子
            C.append(c)
        for j ← |C| - 1 downto 0                   // 倒着入队 = 镜面
            Q.EnQueue(C[j])
```

对例子走一遍：建出的二叉树是 `x.left=p`、`x.right= 空`；`p.left= 空`、`p.right=q`；`q.left=s`、`q.right=r`；`s.left= 空`、`s.right=t`。按左子右兄读回成树：

```
x
├── p
├── q
│   ├── s
│   └── t
└── r
```

层次遍历、孩子倒着入队：输出 `x`，把 `r q p` 入队 → 输出 `r`（无孩子）→ 输出 `q`，把 `t s` 入队 → 输出 `p`（无孩子）→ 输出 `t` → 输出 `s`。结果 `x r q p t s` ✓

代价：建树每个记号读一次，$\Theta(N)$；层次遍历每个结点进出队各一次、每条兄弟边走一次，$\Theta(N)$。合计**时间** $\Theta(N)$；空间是树本身的 $\Theta(N)$、建树递归栈的 $O(N)$ 加队列的 $O(w)$（N 为记号数，w 为树的最大宽度）。

三个容易丢分的地方：1. `$` 也在输入里，**必须读掉**，否则后面整串错位；2. 判据是「标 0 就读两个孩子、标 1 就不读」，**不要写成「读到 `$` 才停」**；3. **镜面不用真反转**——在入队顺序上做文章，这是本题最值钱的一句话。

五、数学证明

6-26. ★ 证明：按先根次序周游森林 F 得到的序列，等于按前序周游 F 对应二叉树 $B(F)$ 得到的序列；按后根次序周游 F 的序列，等于按中序周游 $B(F)$ 的序列。

证明. 对森林的结点数 n 作归纳。$n=0$ 时两边都是空序列。

设 $n>0$，F 的第一棵树为 T_1，其根为 r，r 的子树构成森林 F_1，F 中其余的树构成森林 F_2。按左子/右兄的转换规则：$B(F)$ 的根是 r，**左子树是 $B(F_1)$，右子树是 $B(F_2)$**。

先根 / 前序：由定义，森林的先根序列 $=r\cdot$ 先根$(F_1)\cdot$ 先根(F_2)；二叉树的前序序列 $=r\cdot$ 前序$(B(F_1))\cdot$ 前序$(B(F_2))$。F_1、F_2 的结点数都小于 n，由归纳假设两组括号内逐项相等，故整体相等。

后根 / 中序：森林的后根序列 $=$ 后根$(F_1)\cdot r\cdot$ 后根(F_2)；二叉树的中序序列 $=$ 中序$(B(F_1))\cdot r\cdot$ 中序$(B(F_2))$。同样由归纳假设得两者相等。∎

这条结论的用法：凡是「已知森林的先根和后根序列，还原森林」的题，都可以先翻译成「已知二叉树的前序和中序，还原二叉树」——**后者有唯一解**（第 5 章 5-16），所以前者也有唯一解。

6-27. 证明：一棵有 n 个结点的 K 叉树（每结点固定 K 个孩子指针），空指针数为 $n(K-1)+1$。

证明. 指针域总数为 nK。每个非空指针恰好指向一个结点，且每个结点至多被一个指针指到（它的父结点指来的那一个）；除根之外每个结点都恰好被指到一次，根一次也没有。故非空指针数 $=n-1$，空指针数 $=nK-(n-1)=n(K-1)+1$。∎

$K=2$ 时得 $n+1$，与第 5 章的结论一致；n 个结点的 K 叉树里，K **越大，浪费在空指针上的空间比例越高**（$\frac{n(K-1)+1}{nK}\to 1-\frac{1}{K}$），这正是一般的树不用「固定 K 个指针」而改用左子/右兄的原因。

6-28. ★ 证明：采用**重量权衡合并规则**（不含路径压缩）的并查集，任何一棵树的高度不超过 $\lfloor \log_2 n\rfloor+1$ 层。

证明. 考察任意一个元素 x 的深度。x 的深度只在一种情况下增加 1：x 所在的树被整体挂到另一棵树的根下。按重量权衡规则，这要求**被挂的这棵树的元素个数不超过另一棵**，因此合并之后，x 所在集合的元素个数**至少是原来的 2 倍**。

x 最初所在的集合只有 1 个元素，最终至多 n 个，元素个数最多翻倍 $\lfloor \log_2 n\rfloor$ 次，所以 x 的深度至多增加 $\lfloor \log_2 n\rfloor$，即深度 $\le \lfloor \log_2 n\rfloor$、层数 $\le \lfloor \log_2 n\rfloor+1$。∎

推论：`Find` 的最坏代价 $O(\log n)$，`Union` 也是 $O(\log n)$（它先要做两次 `Find`）。再加上路径压缩，m 次操作的摊还代价降到近乎常数。

6-29. 证明：树中所有结点的度数之和等于结点数减 1，即 $\sum_i i\cdot n_i=n-1$。

证明. 树中的每一条边都可以唯一地记为「某个结点射向它的某个孩子」，因此边数等于所有结点的度数之和 $\sum_i i\cdot n_i$。另一方面，树中除根之外的每个结点都恰好有一条边从其父结点射入，且根没有入边，因此边数也等于 $n-1$。两者相等。∎

移项即得 6-2 的公式 $n=1+\sum_i i\cdot n_i$，它是本章填空题的主力工具。

6-30. ★ 满 k 叉树指每个结点要么是叶结点、要么恰有 k 个孩子。(1) 证明：非空满 k 叉树若有 L 个叶结点，则内部结点数为 $\frac{L-1}{k-1}$；并由此说明 L 必须满足「$(k-1)$ 整除 $(L-1)$」。(2) 证明：高度为 h 层（只有根时 $h=1$）的满 k 叉树，结点数至少 $k(h-1)+1$、至多 $\frac{k^h-1}{k-1}$，且两个界都能取到。

(1) 证明. 设内部结点数为 n、总结点数 $N=n+L$。

数边：每条边都从某个内部结点射向它的一个孩子，而每个内部结点恰有 k 个孩子，故边数 $E=nk$；另一方面除根之外每个结点恰有一条入边，故 $E=N-1$。两式联立：

$$nk=n+L-1\Rightarrow L=n(k-1)+1\Rightarrow n=\frac{L-1}{k-1}$$

而 n 是结点个数、必为整数，所以 $(k-1)$ 必须整除 $(L-1)$。∎

归纳版：$n=0$ 时树只有一个根结点、它本身就是叶，$L=1=0\cdot(k-1)+1$ √（$n=1$ 时根恰有 k 个叶孩子，$L=k$，也由下面的归纳步得到）。设结论对 m 个内部结点成立，任取一棵有 $m+1$ 个内部结点的满 k 叉树，取一个深度最大的内部结点，它的 k 个孩子必全是叶；删去这 k 个叶，它变回叶结点，得到有 m 个内部结点的满 k 叉树。反过来看就是**把某个叶结点变成内部结点**：减少 1 个叶、新增 k 个叶，净增 $k-1$ 个，故 $L_{m+1}=m(k-1)+1+(k-1)=(m+1)(k-1)+1$ √

(2) 证明. 上界：第 i 层（根为第 0 层）至多 k^i 个结点，h 层合计至多 $\sum_{i=0}^{h-1}k^i=\frac{k^h-1}{k-1}$，各层全满时取到（6-6）。

下界：树有 h 层，就有一条从根到第 $h-1$ 层某结点的路径，路径上除末端外的 $h-1$ 个结点都有孩子，因而都是内部结点，故内部结点数 $n\ge h-1$。由 (1) 的 $L=n(k-1)+1$，总结点数

$$N=n+L=nk+1\ge k(h-1)+1$$

取到下界的树：一根由 $h-1$ 个内部结点连成的「主干」，每个主干结点除了往下接主干，另挂 $k-1$ 片叶，主干末端再挂 k 片叶。∎

$k=2$ 时下界为 $2h-1$：高 h 层的满二叉树至少 $2h-1$ 个结点，比一般二叉树的下界 h 多出将近一倍，因为每往下伸一层，都得同时多带一片叶。**最少结点数不是只算最长那条路径**，这是本问最常见的错

法。

一个直接的用处：$k=3$ 时 $(L-1)$ 必须是 2 的倍数，所以满 3 叉树的叶结点数只能是 $1,3,5,7,\cdots$（奇数）。问「有没有恰好 6 个叶结点的满 3 叉树」，答案是**没有**——不必去试着画。

别和 6-27 搞混，两条结论不是一回事：

	说的是什么	结果
本题	**满** k 叉树里，n 个**内部结点**对应多少**叶结点**	$n(k-1)+1$
6-27	**任意** k 叉树里，n 个**结点**留下多少**空指针**	$n(k-1)+1$

表达式一模一样，是因为两者算的都是「nk 个槽位减掉 $n-1$ 条被占用的边」—— 一个把剩下的槽位看成叶结点，另一个看成空指针。$k=2$ 时分别退化成「满二叉树叶数 = 分支结点数 + 1」和「n 个结点的二叉树有 $n+1$ 个空指针」（5-29）。

模拟卷（120 分钟，100 分）

题目覆盖前面的核心知识点，部分题型有所复现，可当作一次完整自测。答案集中在各组题目之后。

一、填空题（每题 2 分，第 1 题两空各 1 分，共 20 分）

1. 抽象数据类型由数据对象、数据关系和 ____ 三部分组成，其中**不包括** ____。
2. 在长度为 100 的顺序表中删除一个元素，等概率下平均需要移动 ____ 个元素。
3. 容量为 10 的循环队列（牺牲一格），`front = 7`、`rear = 2`，队中有 ____ 个元素。
4. 6 个元素依次入栈，可能的出栈序列共 ____ 种。
5. 模式 `P = "abaabab"` 的未优化 next 数组是 ____。
6. 一棵二叉树有 20 个叶结点、15 个度为 1 的结点，则它共有 ____ 个结点。
7. 有 100 个结点的完全二叉树，叶结点有 ____ 个。
8. 对 6 个字符构造 Huffman 树，树中共有 ____ 个结点。
9. 一片森林有 4 棵树、共 30 个结点，转换成二叉树后，根结点的右子树上有 ____ 个结点（设第一棵树有 9 个结点）。
10. 有 n 个结点的 3 叉树（每结点固定 3 个孩子指针），空指针共 ____ 个。

答：1. **基本运算（操作集合）**；**数据的存储结构** 2. 49 5. $(\frac{n-1}{2}=\frac{99}{2})$ 3. 5 $((2-7+10)\bmod 10=5)$ 4. 132（Catalan：$1,2,5,14,42,132$） 5. `-1 0 0 1 1 2 3`（逐位：`a` → 0；`ab` → 0；`aba` → 1；`abaa` → 1；`abaab` → 2；

abaaba → 3，因为 aba 既是前缀又是后缀）6. 54（$n_2 = n_0 - 1 = 19$, $n = 20 + 15 + 19 = 54$）7. 50（$\lceil 100/2 \rceil = 50$）8. 11（$2n-1 = 2\times 6 - 1$）9. 21（$30 - 9 = 21$）10. $2n+1$（$n(K-1)+1$, $K = 3$）

二、选择题（每题 2 分，共 16 分）

1. 下列数据结构中，**不属于**线性结构的是（ ）。A. 栈　B. 队列　C. 串　D. 二叉树
2. 入栈序列为 1,2,3,4,5，下列**不可能**是出栈序列的是（ ）。A. 5,4,3,2,1　B. 2,1,4,5,3　C. 4,1,5,3,2　D. 1,3,2,5,4
3. 在带头结点的单链表中，要在 p 所指结点**之前**插入结点 s，最省事的做法是（ ）。A. 从头找到 p 的前驱，再插入，$O(n)$ B. 把 s 插到 p 之后，再交换两者的数据域，$O(1)$ C. 无法做到 D. 直接 s->next = p，$O(1)$
4. 循环队列用「另设计数器」方案时，判满条件是（ ）。A. front == rear　B. (rear+1)%M == front　C. count == M　D. count == M-1
5. 已知二叉树的中序序列为 DBEAFC、后序序列为 DEBFCA，则前序序列为（ ）。A. ABDECF　B. ABCDEF　C. ABDEFC　D. ABDCEF
6. 下列序列中是**最大堆**的是（ ）。A. (90, 70, 80, 60, 50, 30)　B. (90, 80, 70, 60, 50, 85) C. (90, 70, 80, 85, 50, 30)　D. (30, 50, 60, 70, 80, 90)
7. 森林中有 n 个结点、k 棵树，转换成的二叉树中，从根出发沿右链走到底能经过（ ）个结点。A. 1　B. k　C. n　D. $n-k$
8. 使用路径压缩的并查集，路径压缩发生在（ ）。A. Union 时　B. Find 时　C. 初始化时　D. 每次访问数组时

答：1. D。二叉树是层次（树形）结构，一个结点可以有两个后继。2. C。4,1,...：先输出 4，说明 1、2、3 都在栈里、3 在最上；它们只能按 3,2,1 的相对次序出来，而 C 里是 1,...,3,2，违反。用判定法则：取 $p_1 = 4, p_2 = 1, p_4 = 3$，有 $1 < 3 < 4$，正是被禁的模式。3. B。这是「前插转后插」的经典技巧：先把 s 接在 p 之后，再交换 p 与 s 的数据域，效果等价于在 p 前插入，代价 $O(1)$。D 是错的：只改 s->next 而不改 p 的前驱，s 根本没挂进链里。4. C。有了计数器就能存满 M 个，不必牺牲一格。5. A。后序末为根 A；中序中 A 左边 DBE 是左子树、右边 FC 是右子树。左子树后序 DEB → 根 B，中序 DBE → 左 D、右 E；右子树后序 FC → 根 C，中序 FC → F 是 C 的左孩子。前序 = A BDE CF = ABDECF。6. A。逐个验 $K_i \geq K_{2i+1}, K_{2i+2}$：A 中 $90 \geq 70, 80$；$70 \geq 60, 50$；$80 \geq 30$ √。B 错在 $K_2 = 70 < K_5 = 85$；C 错在 $K_1 = 70 < K_3 = 85$；D 是递增序列，它是**最小堆**（递增序列一定满足父 $\leq$ 子），而不是最大堆：$K_0 = 30 < K_1 = 50$ 就已违反。7. B。右链就是「兄弟链」，从根沿右链走到底经过的正是森林中各棵树的根，共 k 个。8. B。路径压缩是在**查找**沿途顺手做的；合并只改一根指针，无路可压。

三、辨析与简答题（每题 6 分，共 24 分）

1. 辨析：「用大 O 表示时间复杂度，说的就是算法在最坏情况下的代价。」
2. 简答：链式队列在**出队使队列变空**时，为什么必须同时修改队尾指针？不改会出什么问题？
3. 简答：KMP 算法中，为什么目标串的下标可以自始至终不回退？
4. 辨析：「二叉搜索树的中序序列有序，所以堆的中序序列也有序。」

答：1. **错**。「最好/最坏/平均」挑的是**输入**，「$O/\Omega/\Theta$」定的是**函数的界**，这是两个正交的维度，可以自由组合：「最好情况是 $O(n)$」完全合法。单说「复杂度是 $O(n^2)$」只承诺「不会比 n^2 更糟」，并没有说清是对哪一类输入而言。严谨的写法是把两个维度都点明，例如「最坏情况下为 $\Theta(n^2)$」。

2. 链式队列的 `front` 和 `rear` 分别指向队头和队尾结点。出队时若摘走的正好是最后一个结点，队列变空，`front` 被置空，但 `rear` 仍指着那个**已经被释放的结点**——它成了一根野指针。下一次入队时若用 `rear->next = 新结点` 去接，就会**写进已释放的内存**（典型表现是 AddressSanitizer 报 `heap-use-after-free`，也可能悄悄损坏堆而在别处崩溃）。正确做法：`dequeue` 发现队列变空时，把 `rear` 一并置空。

3. 因为**失配之前已经匹配上的那一段是已知的**：设在模式的位置 i 处失配，则 `T` 中与 `P[0..i-1]` 对齐的那一段与之逐字符相同。要让模式重新对上，新的对齐位置必须使模式的某个前缀等于 `P[0..i-1]` 的某个后缀，而「最长的这种前缀」的长度正是 `next[i]`——比它更短的对齐也许可行但更晚，比它更长的对齐必然失败。所以直接把模式右移 $i - \texttt{next}[i]$ 位，目标下标停在原地继续比即可（`next[i]=-1` 时目标下标前进一格、模式从头比），**被跳过的那些起点已经被证明不可能成功**，不必回去试。这样 `j` 只增不减，匹配过程 $O(n)$。

4. **错**。堆只保证**父** $\leq$ **子**（最小堆）这种**局部有序**，兄弟之间毫无关系，更不保证「左子树全部小于根、根小于右子树」。例如最小堆 `[1, 2, 3]`（根 1，左 2，右 3）的中序序列是 `2 1 3`，并不有序。堆的用途是 $O(1)$ 取极值、$O(\log n)$ 增删，**不支持按关键码检索**；要中序有序得用二叉搜索树。

四、算法设计与分析题（每题 10 分，共 30 分）

1. 设计算法判断一棵二叉树是否为**完全二叉树**。写明基本思想，给出算法与复杂度。

答：**基本思想**——用层次遍历（队列），**把空孩子也入队**。完全二叉树的特征是：按层从左到右扫描时，**所有非空结点在前、所有空位在后**，中间不能夹空。于是只要在队列中第一次遇到空指针之后，再出现任何非空结点，就不是完全二叉树。

```
IsComplete(root)
    if root = nullptr then return true
    Q ← 空队列;  Q.EnQueue(root)
    met_null ← false
    while not Q.Empty()
        p ← Q.DeQueue()
        if p = nullptr
            met_null ← true
        else
            if met_null then return false      // 空位之后又出现结点 → 不是
↪   完全二叉树
            Q.EnQueue(p->left)
            Q.EnQueue(p->right)
    return true
```

用到的队列 ADT：`EnQueue(x)` 入队、`DeQueue()` 出队并返回队头、`Empty()` 判空，均为 $O(1)$。

正确性：完全二叉树按层次编号后结点连续占据 $0..n-1$，因此层次扫描中空位只能出现在末尾；反之若空位后还有结点，说明编号不连续。

代价：最坏情况（树确为完全二叉树）下每个结点与每个空指针各入队出队一次，时间 $\Theta(n)$，提前返回时更少，故一般为 $O(n)$；队列包含待检查的空位，最大长度与树的最大宽度同阶，空间 $O(n)$。

2. 给定一个长为 n 的字符串 S，设计 $O(n)$ 算法求 S 的**最长回文前缀**（即最长的、本身是回文的前缀）。写明思想、算法与代价。

答：**基本思想**——把问题转化成特征向量问题。令 R 为 S 的逆串，构造新串

$$U = S + \# + R$$

其中 `#` 是不出现在 S 中的分隔符。则 U **的最长公共真前后缀长度** f **就是** S **的最长回文前缀长度**：U 的前缀来自 S 的前缀，U 的后缀来自 R 的后缀（即 S 前缀的逆），两者相等正说明这个前缀等于自己的逆，即回文。分隔符 `#` 保证匹配段不会跨过中线（否则长度可能超过 n）。

```
LongestPalindromicPrefix(S, n)
    R ← reverse(S)
    U ← S + '#' + R                        // 长度 2n + 1
    用 GetNext 求 U 的未优化特征向量，并多算一项 f = next[2n+1]
    return S[0 .. f-1]                     // 长度为 f 的前缀
```

必须写清的两点：(1) 用的是**未优化**的 next——优化版的数值不再是「最长公共真前后缀长度」；(2) 分隔符不可省，且要保证它不在字母表中。

代价：构造 U 为 $\Theta(n)$，求特征向量 $\Theta(n)$，合计 $\Theta(n)$ 时间、$\Theta(n)$ 空间。

例：$S =$ `"abacd"`，$U =$ `"abacd#dcaba"`，$f = 3$，最长回文前缀是 `"aba"` √。

3. 某班有 n 名学生，现给出若干条「x 与 y 在同一个小组」的信息。设计数据结构与算法，支持：(1) 加入一条信息；(2) 查询两名学生是否同组；(3) 统计小组个数。要求每次操作接近 $O(1)$，并说明依据。

答：**数据结构——用父指针表示的并查集**：数组 `parent[0..n-1]`（根记 -1）、`weight[0..n-1]`（以该点为根的集合的元素个数），另用一个整数 `groups` 记录当前小组数。

ADT：

```
MakeSet(n)      功能：n 个学生各成一组；groups ← n
Find(x)         入口：学生编号；出口：所在小组的代表（根），沿途做路径压缩
Union(x, y)     入口：两个编号；功能：合并两个小组（已同组则什么也不做）
SameGroup(x,y)  入口：两个编号；出口：两人是否同组
Count()         出口：当前小组个数
```

```
Union(x, y)
    rx ← Find(x);  ry ← Find(y)
    if rx = ry then return                          // 已同组，小组数不变
    if weight[rx] < weight[ry] then 交换 rx, ry   // 重量权衡：小的挂到大的下
    parent[ry] ← rx
    weight[rx] ← weight[rx] + weight[ry]
    groups ← groups - 1                          // 每成功合并一次，小组数减一

SameGroup(x, y)   return Find(x) = Find(y)
Count()           return groups
```

三个操作的代价：- (1) 加入信息 = 一次 `Union` = 两次 `Find` 加常数工作；- (2) 查询 = 两次 `Find`；- (3) 统计 = 直接返回 `groups`，$O(1)$——**不要每次去扫一遍数组数根**，那是 $O(n)$。

依据：只用重量权衡合并时树高 $O(\log n)$（证明见 6-28），故 `Find` 为 $O(\log n)$；再加上路径压缩，m 次操作的总代价为 $O(m\,\alpha(n))$，α 是反 Ackermann 函数，在现实规模下增长极慢，因此**单次操作的摊还代价接近常数**，单次最坏仍为 $O(\log n)$。初始化 `parent`、`weight` 与 `groups` 另需 $\Theta(n)$ 时间，空间 $\Theta(n)$。

易错点：`groups` 必须在**确实发生了合并**时才减一；对已经同组的一对

再 Union，小组数不能变。

五、证明题（10 分）

设一棵非空二叉树有 n 个结点。证明：它的**扩充二叉树**（把每棵空子树替换为一个外部结点后得到的树）恰有 $n+1$ 个外部结点，并由此说明：n 个结点的二叉树用二叉链表存储时，有 $n+1$ 个空指针域。

证明. 记原树的结点为**内部结点**，新添的为**外部结点**。

第一步：扩充二叉树是满二叉树。原树的每个结点原先有 0、1 或 2 棵空子树，扩充后每棵空子树都换成了一个外部结点，因此每个内部结点恰有 2 个孩子；外部结点没有孩子。故扩充二叉树中不存在度为 1 的结点。

第二步：数叶结点。设扩充二叉树的外部结点数为 m。外部结点即叶结点（$n_0 = m$），内部结点即度为 2 的结点（$n_2 = n$）。由二叉树的性质 $n_0 = n_2 + 1$（证明见 5-29）得

$$m = n + 1$$

第三步：与空指针的对应。二叉链表中，每个结点有两个指针域，共 $2n$ 个；非空指针恰好指向除根之外的每个结点，共 $n-1$ 个；故空指针域有 $2n - (n-1) = n+1$ 个。而每一个空指针正对应扩充时添上的一个外部结点，两者一一对应，数目同为 $n+1$。∎

这条结论的意义：**二叉链表里有一半以上的指针是空的**（$n+1$ 个空、$n-1$ 个非空），空指针占的空间与结点本身同阶。线索（穿线）二叉树就是把这些空指针利用起来存遍历前驱后继，从而不用栈也能完成中序遍历。

考前速查表

一页纸能带走的东西。**公式后面括号里是本篇的题号**，忘了推导就翻回去看。

渐进分析

事项	结论
大 O 定义	存在正常数 c, n_0，使一切 $n \ge n_0$ 都有 $f(n) \le c\,g(n)$（1-4）
常见阶	$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < 2^n < n!$（1-5）
$T(n) = T(n/2) + c$	$\Theta(\log n)$（1-21）
$T(n) = 2T(n/2) + c$	$\Theta(n)$

事项	结论
$T(n)=2T(n/2)+n$	$\Theta(n\log n)$（1-24）
$T(n)=T(n-1)+n$	$\Theta(n^2)$
$\log_2(n!)$	$\Theta(n\log n)$（1-26）
调和级数 $1+\frac{1}{2}+\cdots+\frac{1}{n}$	$\Theta(\log n)$；故 $\sum_{i\le N} N/i=\Theta(N\log N)$（1-20）

线性表

事项	结论
顺序表插入 / 删除平均移动	$n/2\ /\ (n-1)/2$（2-2、2-23）
链表省空间的条件	装填程度 $<\frac{s}{s+p}$（2-5、2-25）
单链表插入口诀	**先接新结点的后继，再改前驱的指针**（2-8）
双链表在 p 后插 s	`p->next` **最后改**；或先存进 `s->next`，之后都从 `s->next` 走（2-12）
双链表删除 p	`p->prev->next = p->next;` `p->next->prev = p->prev;`（2-4）
双链表交换相邻两结点	别背六句，写成「先摘再插」；或直接换数据域（2-21）
有序链表二分	可以折半比较，但循链取中点使总时间仍为 $O(n)$（2-24）

栈与队列

事项	结论
出栈序列种数	Catalan $C_n=\frac{1}{n+1}C_{2n}^{n}$：$1,2,5,14,42,132$（3-3、3-26）
合法出栈序列判据	不存在 $i<j<k$ 使 $p_j<p_k<p_i$（3-7、3-27）
合法**操作**序列（I/O）判据	I、O 总数相等，且每个前缀里 I 不少于 O（3-29）
两个栈做队列 / 两个队列做栈	前者出队摊还 $O(1)$；后者总有一端每次 $O(n)$（3-25 那版是出栈），**摊不掉**（3-11、3-25）

事项	结论
顺序栈与链式栈	操作都是 $O(1)$；顺序栈有空槽、可能上溢，链式栈每元素多一个指针、每次一次分配（3-13）
循环队列（牺牲一格）	空 `front==rear`；满 `(rear+1)%M==front`；长 `(rear-front+M)%M`（3-2、3-28）
循环队列（计数器）	空 `count==0`；满 `count==M`，可存满（3-14）
消递归要存什么	参数、局部变量、**返回点编号**（3-5、3-22）
尾递归	可直接改循环，空间降到 $O(1)$（3-15）
后缀求值	**先弹的是右操作数**（3-21）

字符串

事项	结论
next 定义（未优化）	`next[i]` = `P[0..i-1]` 的最长公共真前后缀长度，`next[0] = -1`（4-4）
next 优化	`P[i]==P[k]` 时借用 `next[k]`，于是会出现 -1（4-13）
优化 next 逐位算	借用的 `next[k]` 是**已优化**的值（4-16）
失配后右移位数	$i - \texttt{next}[i]$，**目标下标不回退**（4-9、4-12）
KMP 代价	匹配 $O(n)$ + 预处理 $O(m)$（4-23、4-25）
最小循环节	$p = n - f$（f 为**整串**的最长公共真前后缀长度）；至少重复两次当且仅当 $p < n$ 且 p 整除 n（4-6、4-24）
循环移位判定	B 是 A 的循环移位，当且仅当两串等长且 B 是 $A + A$ 的子串（4-21）
列出全部不同子串	按长度分组找齐，再逐位比较排字典序（4-16）
平方子串 AA	按半长 L 扫描，$S[j] = S[j + L]$ 连续成立 L 次即找到，$O(n^2)$（4-22）
最长回文前缀	对 `S + '#' + reverse(S)` 求 next（模拟卷四·2）
内存/串区间重叠	`memcpy`/`strcpy` 未定义；`memmove` 先比地址再定拷贝方向（4-15）

二叉树

事项	结论
第 i 层 / 深度 k（根在第 0 层）	至多 2^i ／ $2^{k+1}-1$ 个结点（5-2）
完全二叉树高度	$\lceil \log_2(n+1) \rceil$ 层（5-32）
顺序存储下标	父 $\lfloor (i-1)/2 \rfloor$（$i>0$），孩子 $2i+1$、$2i+2$（5-4）
最后一个分支结点	$\lfloor n/2 \rfloor - 1$；叶子 $\lceil n/2 \rceil$ 个（5-5）
叶子与度 2 的关系	$n_0 = n_2 + 1$（5-3、5-29）
空指针数	$n+1$（5-29 推论、模拟卷五）
形态数	Catalan（5-1、5-31）
定树	前序 + 中序 √，后序 + 中序 √，前序 + 后序 ✗（5-10、5-16）
三个序列都残缺	以中序切左右、前后序认根，每段缺的标识由结点集合反推（5-17）
建堆	筛选法 $O(n)$，逐个插入 $O(n \log n)$（5-14、5-30）
堆的增删	$O(\log n)$；查任意值 $O(n)$（5-18）
Huffman 树	n 个叶 $\to 2n-1$ 个结点、无度 1 结点；WPL = 各次合并新权之和（5-7、5-24）
BST 删除	用中序前驱（左子树最右者）顶替，**先安置它的左子树**（5-12、5-22）
带约束的 BST 计数	先写无约束的 Catalan 递推，再收窄求和范围（5-23）
固定的 Huffman 码遇上频率漂移	可能反输给定长编码；重算出来的才始终不差于定长（5-25）
向下路径和计数	按起点双重递归，或前缀和一趟周游，$O(nh)$；结点可负时不能剪枝（5-28）

树与森林

事项	结论
度数公式	$n = 1 + \sum_i i \cdot n_i$（6-2、6-29）
森林转二叉树	长子作左、兄弟作右；**一棵树转出的二叉树根无右子树**（6-4）
遍历对应	先根 = 前序，后根 = 中序，**层次不对应**（6-5、6-13、6-26）

事项	结论
右链长度	从二叉树根沿右链走到底的结点数 = 森林中树的棵数（6-10）
K 叉树编号（根 0）	孩子 $Ki+1..Ki+K$，父 $\lfloor (i-1)/K \rfloor$（$i>0$），第 i 层至多 K^i 个（6-6）
K 叉树空指针	$n(K-1)+1$（6-7、6-27）
满 K 叉树叶结点数	$n(K-1)+1$（n 为**内部结点**数）；与上一行同源不同义（6-30）
高 h 层的满 K 叉树结点数	至少 $K(h-1)+1$，至多 $\frac{K^h-1}{K-1}$（6-30）
父指针的长处	**往上走只有一条路**：判祖先/后代、并查集（5-27、6-21）
求镜面映射的层次序列	不用真反转树，**入队时孩子从右往左**（6-25）
并查集优化	重量权衡合并（比**元素个数**）→ 树高 $O(\log n)$；路径压缩在 `Find` 时做（6-8、6-21、6-28）
并查集根存 — 元素个数	`parent[x] < 0` 即根，绝对值即重量，省掉 `weight` 数组（6-17）
顺序存储的通法	「某种遍历序列 + 刚好够还原结构的附加信息」，现推比背可靠（6-23）
多叉树非递归后根	栈存（结点, 下一个孩子序号），先压回自己再压孩子（6-20）
嵌套括号表示法	紧跟结点的每一对括号就是它的一棵子树（6-3）

答题格式提醒

1. **先声明约定**：下标从 0 还是 1、根在第 0 层还是第 1 层、高度算层数还是边数。
2. **设计题先写 ADT**：用到栈、队列、并查集时，先列出函数名、入口参数、出口参数、功能一句话。
3. **算法之后一定跟三件事**：基本思想（一两句）、正确性依据（不变式或归纳）、时空代价。
4. **复杂度要说清是哪种情况**：「最坏情况下 $O(n^2)$」比光写 $O(n^2)$ 稳。
5. **画图题把中间步骤留在卷面上**：建堆、Huffman、并查集这三类题，过程分往往比结果分多。

期末复习题库（第 7—12 章）

本篇按 2025 **年秋季学期的期末复习大纲**逐条配题，范围是第 7 章到第 12 章（期中考过的第 1—6 章不直接考，但内容上会牵连到，那部分见 期中复习题库）。大纲里标 ★ 的条目是重中之重，题号后同样标 ★。**大纲原件逐字存在仓库里**：`ref_复习大纲/2025 秋-期末复习大纲.md`，下面那张对照表就是对着它逐条核出来的。

怎么用：答案就跟在题目下面的引用块里，先把引用块遮住自己做一遍，再对答案。每章按考试题型分五组：填空、选择、辨析与简答、算法填空与设计分析、证明。分章题共 118 道；之后是一套完整的模拟卷（20 题），最后一页是速查表。

大纲对照表

把大纲原件逐条摊开，对上本篇的题号。**没有哪一条是空的**——这张表就是「覆盖没覆盖」的凭据。**注意它是人工维护的**：闸门检查不了「题库对不对得上大纲」，换了大纲要重新逐条核。

大纲条目	本篇题号
7 章 图的深度/宽度周游	7-5、7-12
生成树、**生成树林**、最小生成树	7-2、7-12、7-17
★ 图的存储：相邻矩阵与邻接表	7-4、7-9、7-15
★ 拓扑排序（求若干个或所有序列）	7-6、7-13、7-16
★ 最短路径：Dijkstra、Floyd	7-7、7-11
★ 最小生成树：Prim、Kruskal 是贪心	7-10、7-14、7-17、7-20
★ 8 章 直接插入、★ Shell、★快排、★基数、归并	8-2、8-3、8-4、8-6、8-15
比较/移动次数与最好最坏时空：**含二分法插入排序**	8-13、8-14、8-19
各排序的平均时间	8-4、8-5
局部修改与混合应用	8-17、8-18
9 章 ★ 置换选择排序	9-2、9-3、9-7、9-13
★ 多路归并：败者树、最佳归并树、读写盘次数	9-5、9-8、9-9、9-11、9-12、9-14
10 章 平均检索长度、二分检索、★散列表/同义词/碰撞/堆积	10-1、10-2、10-3、10-10
判定树与比较次数	10-2、10-10、10-18
散列函数的选择：除余、平方取中、折叠	10-8、10-14

大纲条目	本篇题号
冲突处理：分离同义词子表、线性探测、双散列	10-4、10-12、10-13、10-15
★ 散列算法：查找、插入、删除与**墓碑**	10-5、10-16、10-19
11 **章** 顺序/散列/倒排文件、静态与动态索引、红黑树	11-2、11-3、11-7
★ B **树与** B+ **树的插入**	11-15、11-16
★ B **树/**B+ **树的读盘与写盘次数分析**	11-19、11-20
B 树/B+ 树的效率分析	11-5、11-12、11-13、11-21
★ **红黑树的插入/删除**	11-11、11-17、11-18、11-22
12 **章** 特殊矩阵与稀疏矩阵的下标规律	12-1、12-2、12-4、12-12、12-18
★ 广义表的结构和周游	12-3、12-7
Trie 与 Patricia（只做了解）	12-13
最佳二叉搜索树	12-10、12-20
★ AVL 插入：LL/LR/RL/RR 与相关性质	12-5、12-8、12-15、12-19
★ 伸展树、**半伸展树**	12-6、12-9、12-14、12-16
Splay 树的插入及**区间操作**	12-16、12-17

几个贯穿全篇的约定（考试写答案时最好也先声明一句）：

- 数组下标、顶点编号若无特别说明一律**从 0 开始**；图的顶点数记 n、边数记 e。
- 排序一律指**从小到大**；「一趟」指算法自己定义的一轮外层循环结束时的状态。
- m 阶 B 树按本书第 11.4 节的定义：每结点至多 m 个子结点、至多 $m-1$ 个关键码；除根和叶外至少 $\lceil m/2 \rceil$ 个子结点；所有叶结点在同一层。
- 算法设计题按大纲要求：**写明基本思想、无歧义即可**；用到栈、队列、优先队列等现成结构时，先写出 ADT（函数名、入口参数、出口参数、功能一句话）再调用。
- 伪代码用缩进表示层次，← 是赋值，= 是比较。

第 7 章 图

本章反复用到下面两张图，做题时可以对照着看。

有向带权图 G_1（顶点 $1..7$）：

```
边（起点 → 终点，权）：
1 → 2 (4)   1 → 3 (2)
2 → 4 (5)
3 → 2 (1)   3 → 4 (8)   3 → 5 (10)
```

```
4 → 5 (2)   4 → 6 (6)
5 → 6 (2)   5 → 7 (3)
6 → 7 (1)
```

无向带权图 G_2（顶点 $A..F$）：

```
A-B 6   A-C 1   A-D 5
B-C 5   B-E 3
C-D 5   C-E 6   C-F 4
D-F 2   E-F 6
```

一、填空题

7-1. 含 n 个顶点的无向图，边数最多为 ____；含 n 个顶点的有向图（不含自环、无重边），弧数最多为 ____。含 n 个顶点的无向连通图，边数最少为 ____。

> **答**：$\frac{n(n-1)}{2}$；$n(n-1)$；$n-1$。第三个的下界由「$n-1$ 条边且连通即为树」给出——**再少一条必然不连通**。

7-2. 对无向图作深度优先或广度优先周游，**连通图**得到的是一棵 ____，**非连通图**得到的是若干棵树，合称 ____；它的棵数等于 ____。

> **答**：**生成树**；**生成树林（生成森林）**；**图的连通分量个数**。
>
> 为什么棵数正好等于连通分量数：一次周游只能走遍起点所在的那个连通分量，走完之后必须**另选一个尚未访问的顶点重新出发**，每重新出发一次就多一棵树。所以「数生成树林里有几棵树」和「数连通分量」是同一件事—— 在遍历的外层加一句 `for v: if not visited[v] then DFS(v); cnt ← cnt + 1` 就能一并算出。
>
> 两条顺带记住：含 n 个顶点、c 个连通分量的无向图，其生成树林共有 $n-c$ 条边；深度优先得到的叫**深度优先生成树（林）**，广度优先得到的叫**广度优先生成树（林）**，**同一张图两者一般不同**。

7-3. ★ 无向图 G 有 8 个顶点、初始没有任何边。随机向 G 添加 m 条边，要**保证** G 一定连通，m 最小是 ____。

> **答**：22。思路是**先构造最坏情况，再加一条**：把 $n-1=7$ 个顶点连成完全图，用掉 $C_7^2=\frac{7\times6}{2}=21$ 条边，剩下那一个顶点仍然孤立—— 所以 21 条边**不能保证**连通。再加一条边时，它无论落在哪里都只能连到那个孤立点（7 个点之间已经满了），图就连通了。故 $m=C_{n-1}^2+1=21+1=22$。
>
> **一般公式**：$m_{\min}=C_{n-1}^2+1=\frac{(n-1)(n-2)}{2}+1$。**千万别答** $n-1$——那是「可能连通」的最少边数，本题问的是「无论怎么放都连通」。「保证」两个字一出现，就要去找**反例最多能撑到几条边**。

7-4. ★ 含 n 个顶点、e 条边的图，用**相邻矩阵**存储需要 ____ 个存储单元，用**邻接表**存储（无向图）需要 ____ 个表结点。当 ____ 时宜用相邻矩阵，当 ____ 时宜

用邻接表。

答：n^2；$2e$（每条无向边挂两次）；**稠密图**（e **接近** n^2）；**稀疏图**（$e \ll n^2$）。有向图的邻接表只需 e 个表结点。两种结构的代价分野：**判断** (i,j) **是否有边**，相邻矩阵 $O(1)$、邻接表 $O(d_i)$；**遍历某顶点的所有邻居**，相邻矩阵 $O(n)$、邻接表 $O(d_i)$。见第 7.3.1、7.3.2 节。

7-5. 对 G_1 从顶点 1 出发作深度优先周游（同一顶点的邻居按编号由小到大访问），得到的序列是 ____；作广度优先周游得到的序列是 ____。

答：深度优先 1 2 4 5 6 7 3；广度优先 1 2 3 4 5 6 7。深度优先：1 → 2 → 4 → 5 → 6 → 7（7 无出边，回溯）→ 5 的邻居都访问过 → …一路回到 1 → 3。广度优先：第 0 层 {1}，第 1 层 {2,3}，第 2 层 {4,5}，第 3 层 {6,7}（6 由 4 发现，7 由 5 经弧 $5 \to 7$ 发现，二者到 1 都是 3 条边）。**两种周游各需要一个辅助结构**：深度优先用栈（或递归），广度优先用队列。

7-6. ★ 有向无环图（DAG）一定存在 ____ 序列；该序列唯一的充要条件是 ____。

答：**拓扑**序列；充要条件是**图中存在一条哈密顿路径**（即一条经过全部顶点的有向路径）。直观解释：拓扑序列唯一，当且仅当任何相邻两个顶点之间都有一条弧把先后关系钉死；只要有某两个顶点互不可达，交换它们就能得到另一个合法的拓扑序列。

7-7. Dijkstra 算法要求图中**不含** ____；Floyd 算法允许 ____，但同样不允许 ____。Floyd 的时间代价是 ____，空间代价是 ____。

答：**负权边**；**负权边**；**负权回路**；$\Theta(n^3)$；$\Theta(n^2)$。Dijkstra 的正确性依赖「已确定的顶点不会再被更短的路径改写」，负权边一出现这条就垮了。Floyd 用的是动态规划（以「只允许经过前 k 个顶点」为阶段），能容忍负权边；但只要存在负权回路，「最短路径」这个概念本身就不成立（绕圈可以无限变小）。

二、选择题

7-8. ★（不定项）下面关于图的说法，正确的是（　）。A. 若有向图中存在一个含两个以上顶点的强连通分量，则该图一定不是 DAG B. 若一个有向图存在拓扑序列，则它一定是 DAG C. 对 DAG 作深度优先搜索，得到的访问序列一定是它的一个拓扑序列 D. 「各边权值互不相同」是「最小生成树唯一」的充要条件

答： A、B。A 对：含两个以上顶点的强连通分量里任意两点互相可达，必然成环，与「无环」矛盾。（注意题面加了「两个以上顶点」——单个顶点本身也算一个强连通分量，那不构成环。）B 对：拓扑序列把所有弧都变成「从前指向后」，若有环则环上的顶点谁都不能排在最前，矛盾。这也是「拓扑排序可用于判环」的依据。C **错**：DFS 的**访问（发现）次序**不是拓扑序。反例就在 7-5：G_1 的 DFS 序列 1 2 4 5 6 7 3 把 3 排在了 2 后面，而弧 $3 \to 2$ 要求 3 在前。**正确的做法是取** DFS **的「完成时间」逆序**（即后序的逆序）。D **错**：它是**充分不必要**条件。G_2 里

有三条权为 5 的边，最小生成树却仍然唯一（见 7-17）。真正的充要条件是：取定一棵最小生成树 T，**每一条不在 T 上的边 (u,v)，其权值都严格大于 T 上 u、v 之间路径中的每一条边**（「严格的环性质」）。注意「每一条切上的最小边都唯一」也只是**充分**条件、并不必要：路径 $a-b-c$ 两条边权都是 1，最小生成树显然唯一，但切 $\{b\}$ 上两条最小边并列。

7-9. 用相邻矩阵存储含 n 个顶点的图，求某个顶点的**出度**需要（　）时间。A. $O(1)$　B. $O(n)$　C. $O(n^2)$　D. $O(e)$

答： B。出度等于该行中非零元素的个数，必须扫完一整行。对照：邻接表求出度是 $O(d)$（d 为出度本身），但求**入度**反而要扫遍整张表，$O(n+e)$—— 这正是需要「逆邻接表」或十字链表的原因。

7-10. ★ 用 Prim 算法和 Kruskal 算法求最小生成树，下列说法正确的是（　）。A. 两者得到的最小生成树一定相同 B. 两者得到的最小生成树权值之和一定相同 C. Prim 适合稀疏图，Kruskal 适合稠密图 D. Kruskal 每一步都要判断加入的边是否使已选边集连通

答： B。最小生成树可能不唯一（有并列权值时），但**权值之和是确定的**，所以 A 错 B 对。C 说反了：Prim 的代价与顶点数相关（$O(n^2)$ 或堆优化到 $O(e\log n)$），适合**稠密图**；Kruskal 的代价是 $O(e\log e)$，适合**稀疏图**。D 错：Kruskal 判的是「加入这条边会不会**成环**」，用并查集判两端是否已在同一个连通分量里，不是判「是否连通」。

7-11. 对 G_1 用 Dijkstra 算法求顶点 1 到其余各点的最短路径，除 1 之外各顶点**进入已确定集合**的先后次序是（　）。A. 2 3 4 5 6 7　B. 3 2 4 5 6 7　C. 3 2 5 4 6 7　D. 2 4 3 5 6 7

答： B。Dijkstra 每轮挑「当前距离估计最小的未确定顶点」，所以 **加入次序就是最短距离由小到大的次序**：

次序	顶点	最短距离	路径
1	3	2	$1\to3$
2	2	3	$1\to3\to2$
3	4	8	$1\to3\to2\to4$
4	5	10	$1\to3\to2\to4\to5$
5	6	12	$\cdots\to5\to6$
6	7	13	$\cdots\to6\to7$

注意 2 的最短路是 $1\to3\to2$（长 3）而不是直接的 $1\to2$（长 4）—— **顶点被"确定"之前，它的距离估计还会被后确定的邻居改小**，这正是松弛的意义。

三、辨析与简答

7-12. 辨析：「图的深度优先生成树和广度优先生成树，树高一定不同。」

答：不一定。两者都可能相同，也可能差很多。- 极端一：图本身就是一条链，两种周游得到的生成树都是那条链，树高相同。- 极端二：图是一个星形（中心连所有点），**从中心出发**时两种周游得到的都是那棵星，高度都是 1（从某片叶出发则两者都是「叶—中心—其余叶」，高度都是 2）。- 一般情况下**广度优先生成树的高度不会超过深度优先生成树**，因为 BFS 树上每个顶点的 层号就是它到起点的**最少边数**，这是任何生成树都达不到更小的。

真正稳妥的说法是：BFS **生成树给出无权图的最短路径**，DFS **生成树不给**。

7-13. ★ 简答：拓扑排序的算法思想是什么？怎样用它判断有向图中是否有环？若图中有环，算法会停在什么状态？

答：**思想**（入度法）：反复挑一个**入度为 0** 的顶点输出，并把它连同它的出边一起删掉（即把各后继的入度减一）；重复直到没有顶点可挑。

```
TopoSort(G)
    算出每个顶点的入度，把入度为 0 的顶点全部入栈（或队列）
    count ← 0
    while 栈非空
        v ← 出栈；  输出 v；  count ← count + 1
        for v 的每个后继 w
            indeg[w] ← indeg[w] - 1
            if indeg[w] = 0 then 入栈 w
    if count < n then 报告「图中有环」
```

判环：算法结束时若输出的顶点数 $count < n$，说明剩下的顶点入度都不为 0—— 它们互相牵制，必定构成（或通向）一个环。**这就是拓扑排序判环的全部依据**。

代价：用邻接表时 $O(n+e)$，每条边恰好被处理一次。**另一种做法**是 DFS：按**完成时间逆序**输出即得拓扑序列；DFS 过程中若遇到指向「已发现但未完成」顶点的弧（后向边），即说明有环。

7-14. ★ 简答：Prim 与 Kruskal 都是贪心算法。分别说明它们每一步" 贪" 的是什么，并各用一句话说明为什么贪心在这里是对的。

答：| | Prim | Kruskal | | — | — | — | | 维护什么 | 一棵**连通**的树，逐点长大 | 一个**森林**，逐边合并 | | 每步贪什么 | 连接「已在树中」与「尚未在树中」的**最短边** | 全局最短的、**不成环**的边 | | 需要的辅助结构 | 每个未入树顶点到树的最短距离（或最小堆） | 排序 + **并查集**判环 | | 时间代价 | $O(n^2)$；堆优化 $O(e\log n)$ | $O(e\log e)$ |

为什么贪心对：两者都靠同一条「切性质」—— 把顶点集任意划分成两半，**横跨这条切的最小边一定属于某棵最小生成树**。Prim 每步取的

是「已选点集」这条切上的最小边；Kruskal 每步取的边，也总是它所连接的两个连通分量之间的最小边。证明见 7-20。

7-15. 简答：相邻矩阵与邻接表，在「判断两点间是否有边」「求某点全部邻居」「求全图的边数」三件事上各是什么代价？由此说明该怎样选。

答：

操作	相邻矩阵	邻接表
判断 (i,j) 有无边	$O(1)$	$O(d_i)$（扫第 i 条链）
取顶点 i 的全部邻居	$O(n)$（扫一整行，含大量 0）	$O(d_i)$
求全图边数	$O(n^2)$	$O(n+e)$
空间	$\Theta(n^2)$，与 e 无关	$\Theta(n+e)$

选法：稠密图（$e=\Theta(n^2)$）或频繁问「某两点有没有边」时用相邻矩阵；稀疏图或以「遍历邻居」为主时用邻接表。一个实用判据：**算法里若出现「for 每个邻居」就倾向邻接表，出现「if 有边」就倾向矩阵**。

四、算法设计与分析

7-16. ★ 对 G_1 求出所有拓扑序列，并说明理由。

答：**只有一个**：1 3 2 4 5 6 7。

用入度法推一遍：初始入度 1:0、2:2、3:1、4:2、5:2、6:2、7:2。入度为 0 的只有 1 → 输出 1，删去 $1\to2, 1\to3$ → 3 的入度变 0（2 还剩 1，来自 $3\to2$）→ 输出 3 → 2 入度变 0 → 输出 2 → 4 入度变 0 → 输出 4 → 5 → 6 → 7。**每一步可选的顶点都只有一个**，所以序列唯一。

更快的判据：拓扑序列唯一 $\leftrightarrow$ 图中存在哈密顿路径。这里 $1\to3\to2\to4\to5\to6\to7$ 正是一条经过全部 7 个顶点的有向路径，它把任意相邻两点的先后关系都钉死了，因此别无选择。反过来，只要某两个顶点之间互不可达，把它们对调就能造出第二个拓扑序列。

7-17. ★ 对 G_2 分别用 Kruskal 和 Prim（从 A 出发）求最小生成树，写出选边次序并画出结果；该最小生成树唯一吗？

答：Kruskal（按权从小到大扫，成环就跳过）：

序	边	权	动作
1	A-C	1	选
2	D-F	2	选
3	B-E	3	选
4	C-F	4	选（把 $\{A,C\}$ 与 $\{D,F\}$ 接上）
5	A-D	5	**跳过**，A、D 已连通（成环）
6	B-C	5	选（把 $\{B,E\}$ 接进来），已有 5 条边，结束

Prim（**从 A**）：A-C(1) → C-F(4) → F-D(2) → C-B(5) → B-E(3)。

两者选边的次序不同，结果完全相同：

```
      A
      | 1
      C —5— B —3— E
      | 4
      F —2— D
```

边集 $\{AC(1), CF(4), FD(2), CB(5), BE(3)\}$，总权 $1+4+2+5+3=15$。

唯一吗? 唯一。虽然图中有三条权为 5 的边（A-D、B-C、C-D），但：处理到权 5 时，A、C、D、F 已经在同一个连通分量里，A-D 与 C-D 都会成环被跳过，**能用的权 5 边只有 B-C 一条**，没有选择余地。

这正是 7-8 选项 D 的反例：「边权互不相同」能保证 MST 唯一，但**不是必要条件**。

7-18. ★ 给定一个用邻接表存储的无向图，设计算法判断它是否是一棵**树**，并分析代价。

答：**基本思想**——无向图是树，当且仅当它**连通**且**边数恰为** $n-1$（这两条任取其一加上「无环」也等价，但边数最好数，无需额外判环）。用到的 ADT：

```
Graph
    VerticesNum()      出口：顶点数 n
    EdgesNum()         出口：边数 e
    Neighbors(v)       出口：顶点 v 的邻居表（可顺序取）
Queue：EnQueue / DeQueue / Empty，均为 O(1)
```

```
IsTree(G)
    n ← G.VerticesNum();  e ← G.EdgesNum()
    if e ≠ n - 1 then return false          // 先做这一条，O(1) 就能否掉大
↪   多数图
    visited[0..n-1] ← false
    Q ← 空队列;  Q.EnQueue(0);  visited[0] ← true;  cnt ← 1
    while not Q.Empty()
        v ← Q.DeQueue()
        for w in G.Neighbors(v)
            if not visited[w]
                visited[w] ← true;  cnt ← cnt + 1;  Q.EnQueue(w)
    return cnt = n                          // 连通
```

正确性：$e=n-1$ 且连通 $\Rightarrow$ 无环（否则去掉环上一条边仍连通，边数就少于 $n-1$ 了，与连通图至少 $n-1$ 条边矛盾），即为树。**代价**：时间 $O(n+e)$，空间 $O(n)$（`visited` 加队列）。**常见错**：只判连通不判边数——那放过了所有含环的连通图；只判边数不判连通——那放

过了「一个环 + 一棵树」这种边数恰好 $n-1$ 却不连通的图。

7-19. 设计算法求有向图中从顶点 u 到顶点 v 的**所有**简单路径（不重复经过顶点），并分析最坏代价。

答：**基本思想**——带回溯的深度优先搜索。沿着一条路径往下走，把走过的顶点标记为「在当前路径上」；走到 v 就输出整条路径；**无论成功还是走不通，返回时都要把标记撤销**（这一步叫回溯），否则别的路径会被误认为已经走过。

```
AllPaths(u, v)
    path ← 空表;  onPath[0..n-1] ← false
    DFS(u)

DFS(x)
    path.append(x);  onPath[x] ← true
    if x = v then 输出 path
    else
        for w in G.Neighbors(x)
            if not onPath[w] then DFS(w)
    onPath[x] ← false;  path.removeLast()      // 回溯：撤销标记
```

代价：路径条数本身最坏可达 $O((n-2)!)$（完全图里 u 到 v 的简单路径数），所以**任何算法都不可能是多项式的**——输出本身就是指数量级。单条路径的输出代价 $O(n)$，算法的时间与「路径条数 × 路径长度」同阶。**和单纯的 DFS 的区别必须写清楚**：普通 DFS 的 `visited` 一旦置位就不撤销，每个顶点只访问一次，$O(n+e)$；本题的 `onPath` **必须撤销**，代价因此是指数的。

五、证明题

7-20. ★ 证明最小生成树的「切性质」：把顶点集 V 划分成非空的 S 与 $V \setminus S$，若横跨这两部分的边中权值最小的边 e 唯一，则 e 一定属于**每一棵**最小生成树。

证明. 设 T 是任意一棵最小生成树，反设 $e=(u,v)\notin T$（$u\in S$，$v\notin S$）。T 是生成树，u 与 v 之间在 T 上有唯一路径 P。P 从 S 内的 u 走到 S 外的 v，**必然至少有一条边 f 横跨这条切**。由 e 是横跨边中权最小且唯一，有 $w(e)<w(f)$。

把 f 从 T 中去掉、把 e 加进来，得到 $T'=T-f+e$。去掉 f 使 T 分成两个连通块，而 e 的两端恰好分属这两块（P 上 f 的两侧），所以 T' 仍是生成树；而 $w(T')=w(T)-w(f)+w(e)<w(T)$，与 T 最小矛盾。∎

推论：若全部边权互不相同，则每条切的最小边都唯一，因此**最小生成树唯一**。反过来不成立——见 7-17 的 G_2，边权有重复而 MST 仍唯一。

7-21. ★ 设 G 是无向简单图，对 G 作深度优先搜索得到一棵 DFS 树 T（可取任意顶点为根）。证明：G 中所有**不在 T 上**的边 (u,v)，u 与 v 在 T 上必有**祖先后代关系**。

证明. 不妨设 DFS 过程中先发现 u、后发现 v（「发现」指首次访问）。考察**刚发现 v 的那一刻**。此时 u 已被发现，只有两种可能：

1. u **已发现、尚未结束**（还在递归栈上）。此时 DFS 正在遍历以 u 为根的那棵子树，而 v 是在这段时间内被发现的，所以 v 落在 u 的子树里，即 u 是 v 的祖先。
2. u **已发现且已结束**（递归已返回）。这不可能：u 结束的条件是它的**所有邻边都已被考察过**，而 (u,v) 是它的一条邻边，考察这条边时 v 要么已被发现、要么当场被发现——两种情况下 v 都会在 u 结束之前被发现，与「v 是此刻刚发现」矛盾。

所以只能是情形 1，u 是 v 的祖先。∎

这条结论的分量：它说明**无向图的 DFS 只会产生树边和「后向边」（回边），不存在横叉边**。无向图的判环、求割点与桥、求双连通分量这些算法都建立在这一条上。（有向图不成立——有向图的 DFS 可以产生横叉边。）

7-22. 证明：n 个顶点的无向图若边数超过 $\frac{(n-1)(n-2)}{2}$，则它一定连通。

证明. 反设图不连通，则顶点集可分成两个非空部分，分别含 k 与 $n-k$ 个顶点（$1 \le k \le n-1$），两部分之间没有边。于是边数至多为

$$\frac{k(k-1)}{2} + \frac{(n-k)(n-k-1)}{2}$$

把它看成 k 的二次函数，在 $k = 1$ 或 $k = n-1$ 处取最大值 $\frac{(n-1)(n-2)}{2}$。这与「边数超过 $\frac{(n-1)(n-2)}{2}$」矛盾。∎

这正是 7-3 的严格版本：**最坏的不连通图就是「一个 $n-1$ 阶完全图 + 一个孤立点」**。

第 8 章 内排序

本章反复用到同一个初始序列：$A = \{9, 4, 8, 5, 2, 7, 3, 6\}$（$n = 8$），排序一律从小到大。

一、填空题

8-1. 排序算法**稳定**指的是 ____。在直接插入排序、冒泡排序、直接选择排序、Shell 排序、快速排序、堆排序、归并排序、基数排序中，**不稳定**的是 ____。

答：关键码相同的记录，排序后它们的**相对次序与排序前一致**；不稳定的是**直接选择、Shell、快速、堆**四种。

为什么这四个不稳定（各记一个最小反例）：- 直接选择：`2 2' 1` → 第一趟把 1 和第一个 2 对调，得 `1 2' 2`，两个 2 换了位；- Shell：跨距离交换，`2 2' 1` 取增量 2 时 2 与 1 直接换位；- 快速：轴与远处元素交换，同样会跨过相等元素；- 堆：建堆和筛选都在跨层交换。

一句话记法：凡是「跨距离搬元素」的排序都不稳定；稳定的那几个（插入、冒泡、归并、基数）只在相邻或有序合并时移动，遇到相等就不动。

8-2. ★ 对 A 作 Shell 排序，增量序列取 $4, 2, 1$，写出每趟结束后的序列：增量 4 后 ____，增量 2 后 ____，增量 1 后 ____。

答：- 增量 4：`2 4 3 5 9 7 8 6`（四个子序列 $(9,2)(4,7)(8,3)(5,6)$ 各自排好）- 增量 2：`2 4 3 5 8 6 9 7` - 增量 1：`2 3 4 5 6 7 8 9`

Shell **的关键在于「增量为 1 的最后一趟一定是直接插入排序」**，前面几趟的作用是把大的逆序对提前消掉，让最后一趟几乎不用搬东西。增量序列必须**以 1 结尾**，否则排不完。

8-3. ★ 对 A 作快速排序，取**第一个元素为轴**，写出第一趟划分后的序列与轴的最终位置。

答：以 9 为轴，A 中其余元素全部小于 9，一趟划分后 9 落到**最末位**：`6 4 8 5 2 7 3 | 9`，轴位置为下标 7。

这恰好是**快速排序的最坏情况**：轴划出的两半是 7 和 0，完全不平衡。若整个序列已经有序（或逆序）而仍取首元素为轴，每趟都这样，递归深度 n、总代价退化为 $\Theta(n^2)$。**这就是要「三者取中」选轴的原因**。

8-4. 各排序算法的平均时间代价：直接插入 ____、冒泡 ____、直接选择 ____、Shell ____、快速 ____、堆 ____、归并 ____、基数 ____。

答：$O(n^2)$、$O(n^2)$、$O(n^2)$、约 $O(n^{1.3})$（依增量序列而定）、$O(n\log n)$、$O(n\log n)$、$O(n\log n)$、$O(d(n+r))$（d 为趟数，r 为基数）。

最值得记住的三处"平均与最坏不一样"：快速排序平均 $O(n\log n)$ 但最坏 $O(n^2)$；直接插入最好 $O(n)$（已有序时）但平均与最坏都是 $O(n^2)$；堆排序与归并排序**最坏也是** $O(n\log n)$，这是它们的长处。

8-5. 归并排序的空间代价是 ____，堆排序是 ____，快速排序是 ____。

答：$O(n)$（辅助数组）；$O(1)$；$O(\log n)$ 平均、$O(n)$ 最坏（**递归栈**）。快速排序常被说成「原地排序」，那只对数据而言；**它的递归栈不是免费的**，先递归较小的一侧、对较大的一侧改用循环，可以把栈深稳定在 $O(\log n)$。

8-6. ★ 对 n 个 d 位、每位取值 $0..r-1$ 的整数作**最低位优先**基数排序，需要 ____ 趟分配与收集，时间代价 ____，空间代价 ____。

答：d 趟；$O(d(n+r))$；$O(n+r)$。对 n 个三位十进制数就是 3 趟、每趟 10 个桶。**基数排序能突破 $\Omega(n\log n)$ 下界，是因为它不是基于比较的**——它用的是关键码的**内部结构**（按位分配），下界定理管不到

它（见 8-20）。代价是要求关键码能拆成固定位数、且取值范围有限。

二、选择题

8-7. ★ 对 $A=\{9,4,8,5,2,7,3,6\}$ 从小到大排序，下列序列**不可能**是某一趟排序结束后的 中间状态的是（　）。A. 4 9 8 5 2 7 3 6 B. 2 4 8 5 9 7 3 6 C. 4 8 5 2 7 3 6 9 D. 4 9 5 8 2 7 3 6 E. 9 4 8 6 2 7 3 5

答： E。前四个各自对得上一种算法：

选项	来自	依据
A	直接插入第 1 趟	前 2 个已有序 4 9，其余原样
B	直接选择第 1 趟	最小的 2 换到首位，被换走的 9 去了 2 原来的位置
C	冒泡第 1 趟	最大的 9 冒到末尾，其余整体左移一格
D	归并第 1 趟	相邻两两配对排好：(9,4) → 4 9、(8,5) → 5 8、(2,7)、(3,6)

E **为什么不可能**：它与初始序列只差一处——把下标 3 与下标 7 上的 5 和 6 **对调**了，变成了 …6…5。插入、冒泡、归并、Shell 这几种算法一趟做的事都是「把某个子序列（相邻的、成对的、或按增量抽出的）**排成升序**」，**不会把一对本来就升序的元素换成降序**；但这条不能推广到所有算法——快速排序的划分、堆排序的筛选都会跨距离交换，所以要**逐个算法核一遍**：插入要求前缀有序（9 4 不有序）；选择要求前 i 个是最小的 i 个（首位是最大的 9）；冒泡要求末尾若干个已就位（末位是 5 不是最大值）；归并要求相邻块有序（9 4 不有序）；Shell 要求某个增量的子序列有序，而 5、6 恰好被换反；快速排序一趟后轴必在最终位置、左边全比它小、右边全比它大，E 里没有一个位置满足（唯一「站对了」的 7 左边有 9 和 8）；堆排序一趟结束时最大值 9 已被换到末尾，而 E 末位是 5。

一个容易被追问的细节：E 恰好等于对 A **建最大堆时第一次筛选**（$i=3$，5 与孩子 6 交换）之后的数组（见 8-16 的建堆表第 1 步）。但那只是建堆的中途，不是堆排序「一趟」结束时的状态，所以 E 仍是答案。

8-8. 下列排序算法中，**每一趟结束后都能确定至少一个元素的最终位置**的是（　）。A. 直接插入排序　B. 归并排序　C. 冒泡排序与快速排序　D. Shell 排序

答： C。冒泡每趟把当前最大值送到末尾；快速排序每趟把轴放到它的最终位置。A、B、D 都不具备这个性质——直接插入排完前 i 个，但

它们在全局中的位置还会被后面的元素挤动；归并到最后一趟才定位；Shell 更是要到增量为 1 才收敛。（本题选项之外，**直接选择排序**每趟选出当前最小值放到前面、**堆排序**每趟把堆顶最大值换到末尾，也都具备这个性质。）**这个性质是「判断某个中间状态出自哪种算法」的主要抓手**（见 8-7）。

8-9. 若待排序列基本有序，下列算法中最快的是（　）。A. 快速排序（取首元素为轴）　B. 直接插入排序　C. 堆排序　D. 直接选择排序

答：B。直接插入在基本有序时每个元素只需比较一两次就位，接近 $O(n)$。A 恰恰相反——基本有序时取首元素为轴会退化到 $O(n^2)$。C、D 与初始次序无关，仍是 $O(n\log n)$ 与 $O(n^2)$。**这条性质正是「快速排序 + 插入排序」混合算法的立足点**（见 8-17）。

8-10. 基于比较的排序算法，最坏情况下的比较次数下界是（　）。A. $\Omega(n)$　B. $\Omega(n\log n)$　C. $\Omega(n^2)$　D. 没有下界

答：B。n 个互异元素有 $n!$ 种排列，每次比较最多把候选集折半，因此判定树至少 $\lceil \log_2(n!) \rceil = \Theta(n\log n)$ 层（证明见 8-20）。**注意前提是「基于比较」**：基数排序、桶排序不受此限。

三、辨析与简答

8-11. 辨析：「快速排序是最快的排序算法。」

答：不准确，要分三层说。1. **平均意义上它确实很快**，常数因子小、访存局部性好，实际库函数多以它为主。2. **最坏情况是 $\Theta(n^2)$**，触发条件恰恰是「已经有序」这种常见输入；要靠随机轴或三者取中来回避，这不是算法本身的性质而是工程补丁。3. **它不稳定、且需要 $O(\log n)$ 的栈**。要稳定就得用归并；要最坏也保证 $O(n\log n)$ 就得用堆排序或归并；要 $O(1)$ 空间就得用堆排序。

没有一种排序在所有维度上最优——这正是第 8 章要列一整张对比表的原因。

8-12. ★ 简答：为什么说「排序算法的稳定性」在实际中是有意义的？举一个具体场景。

答：稳定性让**多关键码排序**可以分趟做。例如要把学生先按「班级」升序、同班内按「成绩」降序排列：**先按成绩排一遍，再用一个稳定算法按班级排一遍**，第二趟不会打乱第一趟已经排好的成绩次序，两趟就得到了想要的双关键码次序。若第二趟用的是不稳定算法（如快速排序），同班内的成绩次序会被打乱，这个技巧就失效。

基数排序正是把这个技巧用到极致：**按位从低到高做 d 趟稳定排序**，每一趟只看一位，最后自然得到整体有序——**基数排序的正确性完全依赖每趟的稳定性**。

8-13. 简答：直接插入排序的最好、最坏、平均时间代价各是多少？分别在什么输入下取到？它的比较次数和移动次数各是多少？

答：

情形	输入	比较次数	移动次数	时间
最好	已升序	$n-1$	0（用哨兵实现时为 $2(n-1)$）	$\Theta(n)$
最坏	已降序	$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$	同阶	$\Theta(n^2)$
平均	随机	约 $\frac{n^2}{4}$	约 $\frac{n^2}{4}$	$\Theta(n^2)$

平均值的来历：随机排列中**逆序对的期望个数是** $\frac{n(n-1)}{4}$，而直接插入排序的移动次数恰好等于逆序对个数——**每交换/后移一次，正好消掉一个逆序对**。这条对应关系也说明：任何「只交换相邻元素」的排序（冒泡、插入）最坏都至少要 $\Theta(n^2)$ 次移动。

8-14. 简答：**二分法插入排序**是把直接插入排序中「顺序找插入位置」换成二分查找。它的比较次数和移动次数各是多少？总时间代价有没有改善？它还稳定吗？

答：

	直接插入排序	二分法插入排序
比较次数	最好 $n-1$、最坏与平均 $\Theta(n^2)$	**一律** $\Theta(n \log n)$（第 i 个元素花 $\lceil \log_2 i \rceil$ 次）
移动次数	平均与最坏 $\Theta(n^2)$	**仍是** $\Theta(n^2)$，**一点没省**
总时间	$\Theta(n^2)$	**仍是** $\Theta(n^2)$
空间	$O(1)$	$O(1)$
稳定性	稳定	**仍然稳定**（有条件，见下）

关键结论：二分插入只省了比较，没省移动，所以渐进阶不变。原因是找到位置之后，**该搬的元素一个也少不了**——插入位置到末尾之间的元素都要后移一格，这部分是 $\Theta(n^2)$ 的来源。它的收益只在「比较很贵而移动很便宜」时才明显（例如关键码是长字符串、而元素本身是指针）。

稳定性的那个条件：二分查找遇到与待插元素**相等**的关键码时，必须继续**往右**找（把插入点定在所有相等元素之后），否则就会把新元素插到相等元素前面，稳定性就丢了。这一条和 8-15 里「判据要用 $>$ 不能用 $\geqslant$」是同一回事。

总时间下界的由来：二分插入仍然只做「相邻元素后移」这一种数据移动，由 8-19 的逆序对论证，这类算法的最坏移动次数不可能低于 $\Theta(n^2)$。

四、算法填空与设计分析

8-15. ★ **算法填空**：下面是直接插入排序，补全三个空。

```
InsertSort(A, n)
    for i ← 1 to n - 1
        temp ← A[i]
        j ← i - 1
        while ____(1)____
            A[j + 1] ← A[j]
            ____(2)____
        ____(3)____
```

答：(1) `j ⩾ 0 and A[j] > temp`　(2) `j ← j - 1`　(3) `A[j + 1] ← temp`

三**处必须写对的细节**：1. 循环条件里 `j ⩾ 0` 要**写在前面**（短路求值），否则 `j = -1` 时 `A[j]` 越界；2. 判据是 `A[j] > temp` 而**不是** ⩾——用 ⩾ 会把相等的元素也搬过去，**排序就不稳定了**，这是本题最容易丢分的一处；3. 循环结束后插入的位置是 `j + 1`（此时 `A[j] ⩽ temp` 或 `j = -1`），不是 `j`。

8-16. ★ 对 $A=\{9,4,8,5,2,7,3,6\}$ 建**最大堆**并作堆排序，写出建堆结果与前两趟排序后的数组。

答：**建堆**（筛选法，从 $i=\lfloor n/2\rfloor-1=3$ 倒着向下筛，与较**大**的孩子比）：

步	i	动作	数组
0	—	初始	`9 4 8 5 2 7 3 6`
1	3	结点 5，孩子 6，$5<6$ 交换	`9 4 8 6 2 7 3 5`
2	2	结点 8，孩子 7、3，$8>7$ 不动	`9 4 8 6 2 7 3 5`
3	1	结点 4，孩子 6、2，$4<6$ 交换；4 落到下标 3，孩子 5，$4<5$ 再交换	`9 6 8 5 2 7 3 4`
4	0	结点 9，孩子 6、8，$9>8$ 不动	`9 6 8 5 2 7 3 4`

最大堆：`9 6 8 5 2 7 3 4`

第 1 趟：堆顶 9 与末尾 4 交换，堆长减到 7，对根筛选 → `8 6 7 5 2 4 3 | 9`　**第 2 趟**：堆顶 8 与 3 交换，堆长 6，筛选 → `7 6 4 5 2 3 | 8 9`

代价：建堆 $O(n)$，之后 $n-1$ 次「换到末尾 + 筛选」各 $O(\log n)$，合计 $\Theta(n\log n)$；空间 $O(1)$。**注意升序排序要用最大堆**（堆顶是最大值，换到末尾正好就位），别用最小堆。

8-17. ★ **算法填空**：输入基本有序时插入排序很快，可以据此加速快速排序——子数组长度不超过 k（$k\geq 2$）时快排直接返回，最外层快排结束后再对整个数组跑一遍插入排序。补全五个空，并给出平均时间复杂度。

```
QuickSortLimitK(A, left, right, k)
    if left ⩾ right or right - left + 1 ⩽ k then return
    选轴并作一趟划分，设轴的最终位置为 p
```

```
    ____(1)____
    ____(2)____

HybridSort(A, n, k)
    QuickSortLimitK(A, 0, n - 1, k)
    for i ← 1 to n - 1
        temp ← A[i];  j ← i
        while j ⩾ 1 and ____(3)____
            ____(4)____
            j ← j - 1
        ____(5)____
```

答：(1) `QuickSortLimitK(A, left, p - 1, k)`　(2) `QuickSortLimitK(A, p + 1, right, k)`（两句可交换）(3) `A[j - 1] > temp`　(4) `A[j] ← A[j - 1]`　(5) `A[j] ← temp`

平均时间复杂度：$O(n\log\frac{n}{k}+nk)$。

两部分分别算：- **快排部分**：递归到子数组长度为 k 就停，递归树的深度是 $\log\frac{n}{k}$ 而不是 $\log n$，每层总代价 $O(n)$，故 $O(n\log\frac{n}{k})$；- **插入排序部分**：快排结束时数组「按段有序」——任何元素与它最终位置的距离不超过 k，因此逆序对总数是 $O(nk)$，插入排序的代价与逆序对数同阶，为 $O(nk)$。

k **怎么选**：k 太小则省不下递归开销，k 太大则 nk 那一项失控；实际库实现常取 k 在 $8\sim32$ 之间，此时两项都接近 $O(n\log n)$ 而常数更小。k **取常数时总代价仍是** $O(n\log n)$，收益全在常数因子上——这道题要考的就是「渐进阶不变、常数变小」也是一种真实的优化。

8-18. 设计算法，在 $O(n)$ 时间内把一个只含 0、1、2 三种值的数组原地排好序，要求只扫描一遍、额外空间 $O(1)$。

答：**基本思想**——三路划分（荷兰国旗问题）。用三个指针把数组分成四段：`[0, lo)` 全是 0，`[lo, i)` 全是 1，`(hi, n-1]` 全是 2，`[i, hi]` 是待定区。每步看 `A[i]`：

```
ThreeWayPartition(A, n)
    lo ← 0;  i ← 0;  hi ← n - 1
    while i ⩽ hi
        if A[i] = 0
            交换 A[i], A[lo];  lo ← lo + 1;  i ← i + 1
        else if A[i] = 2
            交换 A[i], A[hi];  hi ← hi - 1          // 注意 i 不加一
        else
            i ← i + 1
```

`A[i] = 2` **那一支为什么不能** `i ← i + 1`：从 `hi` 换过来的元素还没被检

查过，必须留在原地再看一轮。而 `A[i] = 0` 那一支换过来的元素来自 `[lo, i)`，那一段已经确定全是 1，可以放心前进。**这是本题唯一的陷阱**。

代价：时间 $\Theta(n)$（每轮 `i` 前进或 `hi` 后退，两者合计至多 n 次），空间 $\Theta(1)$。**稳定吗**：不稳定（有跨距离交换），但本题元素只有三种取值，通常不关心。

五、证明题

8-19. ★ 证明：任何「只交换相邻元素」的排序算法，最坏情况下至少需要 $\Theta(n^2)$ 次交换。

证明. 定义排列的**逆序对**为满足 $i < j$ 且 $A[i] > A[j]$ 的下标对。交换相邻的两个元素 $A[i]$ 与 $A[i+1]$，只可能改变这一对的相对次序，其余任何一对元素的相对次序都不变——所以**一次相邻交换至多消掉一个逆序对**。

完全逆序的输入 $n, n-1, \cdots, 1$ 有 $\frac{n(n-1)}{2}$ 个逆序对，而有序序列的逆序对为 0，故至少需要 $\frac{n(n-1)}{2} = \Theta(n^2)$ 次相邻交换。∎

推论：冒泡排序、直接插入排序都属于这一类，所以它们的最坏代价**不可能**优于 $\Theta(n^2)$——这不是实现得不好，而是「只动相邻元素」这个约束本身的代价。Shell 排序快，恰恰是因为它用大增量**跨距离**搬元素，一次能消掉多个逆序对。

8-20. ★ 证明：基于比较的排序算法，最坏情况下至少需要 $\Omega(n \log n)$ 次比较。

证明. 把算法在所有输入上的执行过程画成一棵**判定树**：每个内部结点是一次比较（两个分支对应两种结果），每个叶结点对应一种输出排列。算法必须能区分 n 个互异元素的全部 $n!$ 种排列，因此判定树**至少有 $n!$ 个叶结点**。

一棵高度为 h 的二叉树至多有 2^h 个叶结点，故 $2^h \geq n!$，即

$$h \geq \log_2(n!)$$

而 $\log_2(n!) = \Theta(n \log n)$（上界由 $n! \leq n^n$ 得，下界由 $n! \geq (n/2)^{n/2}$ 得）。最坏情况的比较次数就是判定树的高度 h，故至少 $\Omega(n \log n)$。∎

这条下界有明确的适用边界：它只管「所有信息都来自元素之间的两两比较」的算法。基数排序、桶排序按关键码的**内部结构**分配，一次操作获得的信息量不止一比特，所以能做到 $O(n)$ 而不违反本定理。

8-21. 证明：n 个元素的随机排列中，逆序对的期望个数是 $\frac{n(n-1)}{4}$。

证明. 对每一对下标 $i < j$ 定义指示变量 X_{ij}：若 (i, j) 构成逆序对则为 1，否则为 0。在随机排列中，$A[i] > A[j]$ 与 $A[i] < A[j]$ 概率各

为 $\frac{1}{2}$，故 $E[X_{ij}]=\frac{1}{2}$。逆序对总数 $X=\sum_{i<j}X_{ij}$，由期望的线性性（不要求独立）：

$$E[X]=\sum_{i<j}E[X_{ij}]=\frac{n(n-1)}{2}\cdot\frac{1}{2}=\frac{n(n-1)}{4}$$

∎

结合 8-19 的对应关系可得：**直接插入排序的平均移动次数约为** $\frac{n^2}{4}$，正好是最坏情况的一半——这解释了 8-13 表里的那个 $\frac{n^2}{4}$。

第 9 章 文件管理和外排序

一、填空题

9-1. 外排序的代价主要不是比较次数而是 ____。若内存工作区能容纳 M 条记录、文件共 N 条记录，用**朴素的分段内排序**生成初始顺串，顺串数为 ____。

答：**访问外存（读写磁盘）的次数**；$\lceil N/M\rceil$。外存访问一次的代价（寻道 + 旋转延迟）比一次内存比较高出好几个数量级，所以外排序的全部设计——顺串尽量长、归并路数尽量多——都是为了**减少读写遍数**。

9-2. ★ 用**置换选择排序**产生初始顺串，内存工作区可容纳 M 条记录、文件共 N 条记录。顺串数最少为 ____（此时输入是 ____ 的），最多为 ____（此时输入是 ____ 的），平均顺串长度约为 ____。

答：1（输入已按升序排列）；$\lceil N/M\rceil$（输入按**降序**排列）；约 $2M$。

- **最少 1 条**：输入本身升序时，每次读进来的新记录都不小于刚输出的值，于是一直留在活跃堆里，全部记录连成一条顺串。
- **最多 $\lceil N/M\rceil$ 条**：输入降序时，每读进一条都比刚输出的小，立刻被冻结，于是每条顺串恰好 M 条记录，退化成朴素做法。
- **平均 $2M$**：这是置换选择的价值所在——**顺串数大约减半，归并趟数可能因此少一趟**。

9-3. ★ 某文件有 1800 条记录，内存工作区可容纳 6 条。用置换选择排序生成初始顺串，最少可能产生 ____ 个顺串，最多可能产生 ____ 个顺串。

答：最少 1 个，最多 300 个（$\lceil 1800/6\rceil$）。平均情形约为 $1800/(2\times6)=150$ 个。**这类题只要套 9-2 的两个极端**：升序输入给最小值，降序输入给最大值。

9-4. 对 m 个初始顺串作 k 路归并，归并趟数为 ____。增大 k 可以减少归并趟数，但会增加 ____；用**选择树**之后，内部归并的总比较次数变为 ____，与 k ____。

答：$\lceil\log_k m\rceil$；**每次选最小值所需的比较次数**（朴素做法要 $k-1$ 次）；$\lceil\log_2 m\rceil\cdot(n-1)$；**无关**。推导见第 9.3.3 节：朴素选最小是 $O(k)$，

用选择树降到 $O(\log_2 k)$，与 $\lceil \log_k m \rceil$ 里的 $\frac{1}{\log_2 k}$ 恰好抵消。

9-5. 败者树的内部结点存放的是 ____，另设一个结点存放 ____。替换一个选手之后，重构只需沿 ____ 走一遍，代价 ____。

答：**败者（败者所在叶的下标）**；**全局冠军**；**从该叶到根的路径**；$O(\log_2 k)$。赢者树的内部结点存的是**胜者**，重构时每一层还要去找「对手」；败者树把败者直接留在父结点上，新选手一路与这些败者比即可——**少一次取兄弟的访问**，这是外排序实现里更常用败者树的原因。

二、选择题

9-6. 对 100 个初始顺串作 5 路归并，需要的归并趟数是（ ）。A. 2　B. 3　C. 4　D. 20

答： B。$\lceil \log_5 100 \rceil = \lceil 2.86 \rceil = 3$。逐趟数也一样：$100 \to 20 \to 4 \to 1$，共 3 趟。

9-7. ★ 关于置换选择排序，下列说法**错误**的是（ ）。A. 它是在堆排序的基础上演化来的 B. 它产生的各条顺串长度一般不相等 C. 它产生的顺串平均长度约为工作区容量的两倍 D. 若把整份输入都推进堆再依次弹出，得到的就是置换选择的结果

答： D。那样得到的是一**条完全有序的序列**，即堆排序，不是置换选择。置换选择的核心在于「**冻结**」：新读入的记录若小于刚输出的值，就不能进入当前顺串，必须留到下一条。**没有冻结这一步就不是置换选择**。A、B、C 都对。

9-8. 8 路归并的败者树，除冠军结点外共有（ ）个内部结点，树高为（ ）。A. 8，3　B. 7，3　C. 7，4　D. 8，4

答： B。k 个叶（选手）对应 $k-1$ 个内部结点，故 $8-1=7$；树高 $\lceil \log_2 8 \rceil = 3$，即替换一个选手后沿叶到根重赛 3 次比较。

三、辨析与简答

9-9. ★ 简答：设有 8 条初始顺串，各条的**首记录**关键码依次为 $\{17, 5, 10, 29, 8, 22, 3, 14\}$。画出 8 路归并开始时的败者树；若第 6 路（首记录 3）的下一条记录是 9，输出冠军后重构，再画出败者树。

答：叶结点 $b_0..b_7$ 依次放 8 条顺串的当前记录，内部结点 $ls_1..ls_7$ 存**败者的叶下标**，ls_0 存冠军。

初始比赛（自底向上，两两淘汰）：

比赛	双方	胜	败者存入
b_0 vs b_1	17, 5	5	$ls_4 \leftarrow 0$（值 17）
b_2 vs b_3	10, 29	10	$ls_5 \leftarrow 3$（值 29）

比赛	双方	胜	败者存入
b_4 vs b_5	8, 22	8	$ls_6 \leftarrow 5$（值 22）
b_6 vs b_7	3, 14	3	$ls_7 \leftarrow 7$（值 14）
5 vs 10		5	$ls_2 \leftarrow 2$（值 10）
8 vs 3		3	$ls_3 \leftarrow 4$（值 8）
5 vs 3		3	$ls_1 \leftarrow 1$（值 5）
冠军			$ls_0 \leftarrow 6$（值 3）

```
            ls0 = 6  (冠军 3)
            ls1 = 1  (败者 5)
      ls2 = 2 (10)        ls3 = 4 (8)
ls4=0(17)  ls5=3(29) ls6=5(22) ls7=7(14)
 b0   b1    b2   b3   b4   b5   b6   b7
 17    5    10   29    8   22    3   14
```

输出 3、第 6 路补入 9 后，只沿 b_6 到根这一条路重赛（三场）：9 vs ls_7 的 14 → 9 胜，14 仍是败者；9 vs ls_3 的 8 → 8 胜，9 **成为败者存入** ls_3；8 vs ls_1 的 5 → 5 胜，8 **成为败者存入** ls_1；冠军变成第 1 路的 5。

```
            ls0 = 1  (冠军 5)
            ls1 = 4  (败者 8)
      ls2 = 2 (10)        ls3 = 6 (9)
ls4=0(17)  ls5=3(29) ls6=5(22) ls7=7(14)
 b0   b1    b2   b3   b4   b5   b6   b7
 17    5    10   29    8   22    9   14
```

另外五条路的比赛结果一个都没变——这正是选择树相对「每次扫一遍 k 个队首」的全部优势。

9-10. 简答：为什么多路归并要用「最佳归并树」？它和 Huffman 树是什么关系？

答：各条初始顺串的**长度往往不等**。归并时，一条顺串参与的归并趟数越多，它的记录就被读写越多次。**总读写量 = 各顺串长度 × 它在归并树上的深度之和**，也就是带权外路径长度 WPL——于是「怎样安排归并顺序最省 I/O」就变成了「**给定叶权值，构造 WPL 最小的 k 叉树**」，即 k 叉 Huffman 树。

一个必须补的细节：k 叉 Huffman 要求叶结点数满足 $(m-1) \bmod (k-1) = 0$；不满足时要**补若干条长度为 0 的"虚顺串"**，并且**虚顺串必须补在最底层**（即第一次合并就把它们用掉），否则会让真顺串下沉、WPL 变大。见 9-12。

四、算法设计与分析

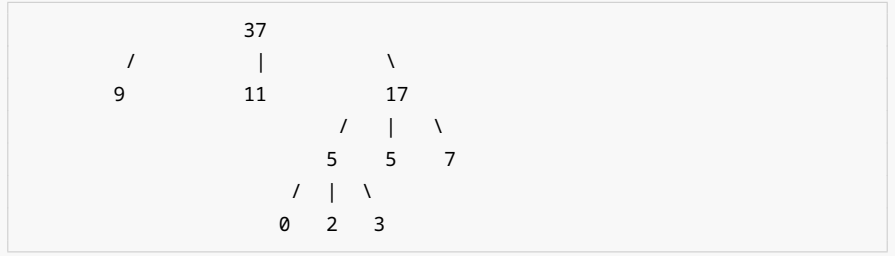
9-11. ★ 有 6 条初始顺串，长度分别为 $\{2,3,5,7,9,11\}$（单位：页）。用 3 路归并，求最佳归并树，并算出总的读记录次数。

答：**第一步：判断要不要补虚顺串**。$k=3, m=6, (m-1) \bmod (k-1)=5 \bmod 2=1 \neq 0$，所以要补 $ (k-1)-1=1$ 条长度为 0 的虚顺串，凑成 7 条。

第二步：按 Huffman 的办法，每次取最小的 3 条合并。

步	取出	合并成	剩余
1	0, 2, 3	5	5, 5, 7, 9, 11
2	5, 5, 7	17	9, 11, 17
3	9, 11, 17	37	完成

最佳归并树（叶上是顺串长度，0 是虚顺串）：



第三步：算总读次数 = 带权外路径长度（虚顺串权为 0，不计）：

$$9\times1+11\times1+5\times2+7\times2+2\times3+3\times3=9+11+10+14+6+9=59$$

即**总共读 59 页**；写的次数与读相同（每次归并的输出都要写回），故总 I/O 约 $2\times59=118$ 页次。

对照：不补虚顺串、直接每次取三条最小的合并，会得到 $(2,3,5)\to10$、$(7,9,10)\to26$、$(11,26)\to37$，最后一次只有两路，WPL 变成 $2\cdot3+3\cdot3+5\cdot3+7\cdot2+9\cdot2+11\cdot1=6+9+15+14+18+11=73$，比 59 多出 14 页。**这就是"虚顺串要补在最底层"的代价差**。

9-12. 设有 n 条记录、每页可装 b 条记录、内存可容纳 M 条记录，用置换选择产生初始顺串、再作 k 路归并。写出总的读写页次估算式，并说明哪些参数能改善它。

答：**基本思想**——把 I/O 拆成三段来数：

1. **生成顺串**：整个文件读一遍、写一遍，$2\cdot\frac{n}{b}$ 页次；

2. **归并**：初始顺串数约 $m = \frac{n}{2M}$（置换选择的平均值），归并趟数 $\lceil \log_k m \rceil$，**每一趟都要把整个文件读一遍、写一遍**，即 $2 \cdot \frac{n}{b} \cdot \lceil \log_k m \rceil$ 页次；
3. 合计

$$2 \cdot \frac{n}{b}\left(1 + \left\lceil \log_k \frac{n}{2M} \right\rceil\right)$$

能改善它的三个旋钮：- **增大** M（内存工作区）：顺串数减少，log 里的分母变大；- **增大** k（归并路数）：趟数减少。代价是内部选最小值变慢，用**选择树**可以消掉这项代价（第 9.3.3 节），但 k 受限于内存能同时容纳多少个输入缓冲区；- **增大** b（页容量 / 缓冲区大小）：直接减少页次，还能摊薄每次寻道的固定开销。

注意顺串长度不等时上式只是估算，精确值要用最佳归并树按 WPL 算（见 9-11）。

五、证明题

9-13. ★ 证明：置换选择排序产生的顺串，**内部一定是非递减的**；且全部顺串拼起来恰好包含输入的每一条记录一次。

证明. (1) **顺串内部非递减**。设当前顺串已输出的最后一个值为 x。活跃堆是最小堆，下一次输出的是堆中最小值 y。堆中的记录分两类：一类是建堆时就在的，另一类是后来读入且**不小于当时输出值**才放进来的。由归纳假设，输出值单调不减，所以后来放进来的记录都不小于 x；而建堆时就在的记录若小于 x，早就该被输出了（最小堆的堆顶总是最小值）。故堆中所有记录都不小于 x，于是 $y \geq x$。

(2) **不丢不重**。每条记录只有三种去向：进入活跃堆、被冻结、被输出。读入时恰好进入其中一个区域（活跃或冻结），输出时恰好从活跃堆移走一次；冻结区在当前顺串结束后整体转为下一条顺串的活跃堆，**没有丢弃也没有复制**。因此每条记录被输出且只被输出一次。∎

这两条正是外排序能工作的前提：**(1) 保证每条顺串可以被归并，(2) 保证结果是原文件的一个排列**。

9-14. 证明：用选择树作 k 路归并时，内部归并的总比较次数与 k 无关。

证明. 设初始顺串数为 m、记录总数为 n。归并趟数为 $\lceil \log_k m \rceil = \frac{\lceil \log_2 m \rceil}{\log_2 k}$（取整误差略去）。每输出一条记录，选择树只需沿叶到根重赛一次，比较 $\lceil \log_2 k \rceil$ 次；每一趟要输出约 n 条记录。于是总比较次数

$$\frac{\log_2 m}{\log_2 k} \cdot \lceil \log_2 k \rceil \cdot (n-1) \approx \lceil \log_2 m \rceil \cdot (n-1)$$

右端不含 k。∎

对照朴素做法：每输出一条记录要在 k 个队首里扫一遍，比较 $k-1$ 次，总数为 $\frac{\log_2 m}{\log_2 k}(k-1)(n-1)$，其中 $\frac{k-1}{\log_2 k}$ **随 k 单调增**——增大 k 省下的归并趟数，会被内部比较的增长吃掉一部分。**选择树的全部意义就是把这一项消掉，让「增大 k」变成纯粹的收益。**

第 10 章 检索

一、填空题

10-1. **平均检索长度**（ASL）的定义是 ____。顺序检索成功时的 ASL 为 ____（等概率），失败时为 ____。

答：ASL $=\sum_i p_i c_i$，其中 p_i 是检索第 i 个元素的概率、c_i 是检索它所需的比较次数；顺序检索成功 $\frac{n+1}{2}$，失败 n（或 $n+1$，看是否设哨兵）。ASL **是"比较次数"的期望，不是时间**——它把常数因子和实现细节都抽掉了，便于在同一把尺子上比较不同的检索方法。

10-2. ★ 在含 12 个元素的有序数组上作二分检索，各元素等概率。成功时的平均检索长度是 ____；失败时的比较次数最少 ____ 次、最多 ____ 次。

答：$\frac{37}{12}\approx 3.08$；最少 3 次，最多 4 次。

画出判定树（取中点 $\lfloor (low+high)/2 \rfloor$，下标 $0..11$）：

比较次数	1	2	3	4
该层结点数	1	2	4	5

$1+2+4+5=12$ ✓，故

$$\text{ASL}=\frac{1\times 1+2\times 2+3\times 4+4\times 5}{12}=\frac{1+4+12+20}{12}=\frac{37}{12}$$

失败对应判定树的 13 个外部结点，它们分布在第 3、4 两层，故 3 或 4 次。**只有当 $n=2^k-1$（判定树是满二叉树）时，失败的比较次数才唯一**——例如 $n=15$ 时失败一律 4 次，ASL $=\frac{49}{15}$。一般地最大检索长度为 $\lceil \log_2(n+1) \rceil$。

10-3. ★ 散列表中，两个不同关键码 $k_1\neq k_2$ 却有 $H(k_1)=H(k_2)$，称它们互为 ____；这种现象叫 ____。闭散列（开地址）法中，不同散列值的探查序列相互交织、使得检索变慢的现象叫 ____。

答：**同义词**；**冲突（碰撞）**；**堆积（聚集，**clustering**）**。堆积是**线性探测**特有的毛病：一旦形成一段连续的占用区，任何散列到这段区间内

的关键码都会把它接得更长，越长越容易被撞上。二次探测和双散列就是为了打散这种连续性。

10-4. 散列表的**装填因子** α 定义为 ____。闭散列（线性探测）成功检索的 平均探查次数约为 ____，失败约为 ____；开散列（拉链法）成功约为 ____，失败约为 ____。

答：$\alpha = \frac{\text{表中已有记录数}}{\text{表长}}$；线性探测成功 $\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$、失败 $\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$；拉链法成功约 $1+\frac{\alpha}{2}$、失败约 α。**关键结论：代价只跟 α 有关，与表长 n 无关**——这正是散列能做到「平均 $O(1)$」的原因。但 $\alpha \to 1$ 时线性探测的两个式子都会爆掉，所以闭散列一般把 α 控制在 $0.7 \sim 0.8$ 以下。

10-5. ★ 闭散列表中删除一个记录时**不能直接置空**，原因是 ____；通常的做法是给该位置做一个 ____ 标记。检索时遇到这个标记应当 ____，插入时遇到它应当 ____。

答：直接置空会**截断后面同义词的探查序列**，使本来能找到的记录变得找不到；做法是打上**墓碑**（tombstone）标记；检索时遇到墓碑要**继续往下探查**（不能当成空位停下来）；插入时**可以占用**它（但仍要先把整条探查链走完，确认表中没有重复关键码）。

墓碑的代价：它既不是空也不是满，检索时不能提前停，**删多了表就会变"脏"**，失败检索越来越慢。工程做法是统计墓碑比例，超过阈值就整表重建（rehash）。

10-6. 分块检索（索引顺序检索）把表分成若干块，要求 ____ 有序、____ 不必有序。若把 n 个元素均分成 b 块，块内顺序检索、块间也顺序检索，则 ASL $\approx$ ____，取 $b =$ ____ 时最小，最小值约为 ____。

答：**块间**（块与块之间的最大关键码递增）有序；**块内**不必有序；ASL $\approx \frac{b+1}{2} + \frac{n/b+1}{2}$；$b = \sqrt{n}$；约 $\sqrt{n}+1$。**分块是顺序与二分的折衷**：检索比顺序快（$O(\sqrt{n})$），而插入删除只动一块、不必像有序表那样整体搬移，比二分灵活。

二、选择题

10-7. ★（不定项）下列说法正确的是（　）。A. 二分检索在任何情况下都优于顺序检索，因为它的时间复杂度更低 B. 分块检索是二分检索与顺序检索的折衷，检索较快而更改较灵活 C. 若散列函数足够均匀且没有发生冲突，检索的时间代价可以达到 $O(1)$ D. 由于有开散列和闭散列两种冲突处理办法，设计散列函数时不必考虑减少冲突

答：B、C。A **错**，两处：其一，二分检索要求**顺序存储且事先有序**，排序本身很贵，表若频繁增删，维护有序的代价会盖过检索的收益；其二，n 很小时二分的常数更大，顺序检索反而更快。「**复杂度更低**」**不等于**「**任何情况下都更优**」。D **错**：冲突处理只是兜底，**冲突本身直接决定平均探查次数**（见 10-4 的两个式子）。散列函数要尽量把关键码均匀散开，这是第一道也是最重要的一道防线。

10-8. 用除余法构造散列函数 $H(k)=k \bmod p$，p 通常取（　）。A. 表长即可，取什么都行　B. 接近表长的**素数**　C. 2 的幂　D. 表中最大关键码

> **答：** B。取 2 的幂时 $k \bmod 2^t$ 只保留了 k 的低 t 位，**高位信息全部丢失**，关键码的低位若有规律（如都以 00 结尾）就会严重冲突。取接近表长的素数能让各位都参与运算，散布更均匀。

10-9. 在长度为 13 的散列表中用线性探测处理冲突，已知某关键码的散列地址是 5，而下标 5、6、7 已被占用、8 为空，则插入它需要探查（　）次。A. 1　B. 3　C. 4　D. 5

> **答：** C。依次探查 5、6、7、8，第 4 次找到空位。**注意"探查次数"包含第一次**——从 0 开始数是最常见的错。

10-10. 下列关于二分检索判定树的说法，**错误**的是（　）。A. 对同一长度的有序表，判定树的形状是固定的，与表中的值无关 B. 判定树的高度是 $\lceil \log_2(n+1) \rceil$ C. 判定树的外部结点（失败结点）有 $n+1$ 个 D. 判定树一定是一棵满二叉树

> **答：** D。只有 $n=2^k-1$ 时它才是满的；$n=12$ 时最下一层就没排满（见 10-2）。A 对——判定树只由**下标的折半过程**决定；B、C 对，C 用的是第 5 章那条「n 个内部结点的扩充二叉树有 $n+1$ 个外部结点」。

三、辨析与简答

10-11. ★ 简答：开散列（拉链法）与闭散列（开地址法）各有什么优缺点？装填因子 α 对两者的影响有何不同？

> **答**：| | 开散列（拉链法）| 闭散列（开地址法）| | — | — | — | | 冲突怎么办 | 同义词挂到同一个桶的链表上 | 按探查序列在表内另找空位 | | α 的范围 | **可以大于 1** | 必须 <1 | | 代价随 α | 线性增长（链平均长 α），**不会爆** | $\alpha \to 1$ 时急剧恶化 | | 删除 | 直接摘链，干净 | **要用墓碑**，表会变脏 | | 空间 | 每个结点多一个指针；桶数组可以小些 | 无指针开销，但必须留出空位 | | 访存局部性 | 差（指针跳转） | 好（连续探查，对缓存友好） |
>
> **一句话**：数据量波动大、删除频繁 → 拉链法；数据量可预估、以查为主、在意缓存 → 开地址法（并把 α 控制在 0.7 以下）。

10-12. 辨析：「散列表的检索时间复杂度是 $O(1)$。」

> **答：要加限定**。严格说是「**平均** $O(1)$，且前提是装填因子受控、散列函数足够均匀」。**最坏情况是** $O(n)$：所有关键码都是同义词时，拉链法退化成一条长链、开地址法退化成一次线性扫描。而且「$O(1)$」掩盖了一个前提——**计算 $H(k)$ 本身的代价被当成常数**，对长字符串关键码而言这一步是 $O(|k|)$。最后，**散列表不支持按序遍历、不支持范围查询**，这类需求只能用 BST 或 B+ 树。

10-13. 简答：为什么线性探测会产生「堆积」？二次探测和双散列分别是怎样缓解它的？它们各自还有什么代价？

答：**线性探测**的探查序列是 $H(k), H(k)+1, H(k)+2, \cdots$（模表长）。一旦形成一段连续占用区，**任何散列到这段区间内任意位置的关键码，都会沿着它一直探到区尾，并把这段接得更长**——区段越长越容易被撞上，于是滚雪球，这就是堆积（一次聚集）。

- **二次探测**：探查序列改为 $H(k) \pm 1^2, \pm 2^2, \cdots$，跳着走，散列值不同的关键码不再共用同一条连续通道，一次聚集被打散。**代价**：散列值**相同**的关键码仍然走同一条序列（称二次聚集）；而且不保证能探遍整张表——表长取素数且 $\alpha < \frac{1}{2}$ 时才有保证。
- **双散列**：步长由第二个散列函数给出，$H(k) + i \cdot H_2(k)$。连散列值相同、步长不同的关键码也会分开，聚集最轻。**代价**：多算一个散列函数；且必须保证 $H_2(k)$ 与表长**互素**（表长取素数、H_2 取值非 0 即可），否则探查序列走不遍全表。

10-14. ★ 简答：构造散列函数常用的三种方法是**除余法、平方取中法、折叠法**。分别说明它们的做法、各自想解决什么问题，并各举一个算例。

答：三种方法的共同目标只有一个——**让关键码的每一位都尽量影响到最终的散列地址**，从而把关键码均匀地铺开。

(1) **除余法** $H(k) = k \bmod p$。- **做法**：p 取**接近表长的素数**。- **要解决的问题**：若 p 取 2^t，则 $k \bmod 2^t$ 只保留了 k 的低 t 位，**高位的信息全部丢掉**；关键码若低位有规律（例如都以 00 结尾），冲突会成片出现。取素数能让各位都参与取模运算。- **算例**：表长 100，取 $p = 97$；$k = 1234$ 时 $H(k) = 1234 \bmod 97 = 70$。

(2) **平方取中法**：把 k 平方，取结果**中间的若干位**作为地址。- **要解决的问题**：平方之后，**中间那几位受 k 的所有位共同影响**（两端的位只受 k 的高位或低位影响），因此中间位的随机性最好。适合「不知道关键码分布、但怀疑它某几位有规律」的场合。- **算例**：$k = 123$，$k^2 = 15129$（5 位），取中间 3 位得 $H(k) = 512$。再如 $k = 4731$，$k^2 = 22382361$（8 位），取中间 3 位得 823。

(3) **折叠法**：把 k 按地址位数**分成等长的几段**（最后一段可短），再把各段相加，舍去进位得到地址。分两种叠法：- **移位折叠**：各段直接对齐相加；- **边界折叠**：把**相邻两段沿边界翻转**后再相加（像折纸一样来回折）。- **要解决的问题**：关键码**位数远多于地址位数**时（如 18 位身份证号映射到 4 位地址），直接截取会丢掉大部分信息，折叠能让**每一位都参与进来**。- **算例**：$k = 123456789$，地址 3 位，分成 `123 | 456 | 789`。移位折叠：$123 + 456 + 789 = 1368$，取后三位得 368；边界折叠：把第二段翻转成 654，$123 + 654 + 789 = 1566$，取后三位得 566。

怎么选：**除余法最常用**，实现简单、效果稳定；关键码分布未知时用平方取中；关键码特别长时用折叠先压缩、再除余。三者都不解决冲突，冲突要靠 10-13 的那几种办法处理。

四、算法设计与分析

10-15. ★ 设散列表长 $m = 13$，散列函数 $H(k) = k \bmod 13$，用**线性探测**处理冲突。依次插入关键码 $19, 14, 23, 1, 68, 20, 84, 27, 55, 11$，(1) 画出散列表；(2) 求成功检索的平均检索长度；(3) 求失败检索的平均检索长度。

答：(1) **逐个插入**（H 值：$19 \to 6$、$14 \to 1$、$23 \to 10$、$1 \to 1$、$68 \to 3$、$20 \to 7$、$84 \to 6$、$27 \to 1$、$55 \to 3$、$11 \to 11$）：

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
关键码		14	1	68	27	55	19	20	84		23	11	

各关键码的探查次数：

关键码	19	14	23	1	68	20	84	27	55	11
$H(k)$	6	1	10	1	3	7	6	1	3	11
实际下标	6	1	10	2	3	7	8	4	5	11
探查次数	1	1	1	2	1	1	3	4	3	1

(2) **成功** ASL：$\frac{1+1+1+2+1+1+3+4+3+1}{10} = \frac{18}{10} = 1.8$。

(3) **失败** ASL：对每个**初始地址** $0..12$，从该地址一直探到遇见空位为止的次数（空位在下标 0、9、12）：

初始地址	0	1	2	3	4	5	6	7	8	9	10	11	12
探查次数	1	9	8	7	6	5	4	3	2	1	3	2	1

$$\text{ASL}_{\text{失败}} = \frac{1+9+8+7+6+5+4+3+2+1+3+2+1}{13} = \frac{52}{13} = 4$$

以初始地址 10 为例：探 10（23）、探 11（11）、探 12（空），共 3 次；初始地址 1 则要把 1..8 这一整片占用区走完、在 9 号空位停下，共 9 次。

两处常见错：1. 失败 ASL 的分母是**表长** 13（每个初始地址等概率），不是关键码个数；2. 下标 1..8 连成一大片，这正是**堆积**的现场：$\alpha = \frac{10}{13} \approx 0.77$ 已经偏高，失败检索的平均探查次数高达 4 次。换成拉链法，失败平均只要约 $\alpha \approx 0.77$ 次。

10-16. ★ 写出闭散列表（线性探测、带墓碑）的检索、插入、删除算法，并说明墓碑在三个算法里各起什么作用。

答：表中每个槽有三种状态：`EMPTY`（从未用过）、`OCCUPIED`（有记录）、`DELETED`（墓碑）。

```
Search(T, k)
    i ← 0
    while i < m
        pos ← (H(k) + i) mod m
       if T[pos].state = EMPTY   then return 未找到      // 只有真空位才能停
        if T[pos].state = OCCUPIED and T[pos].key = k then return pos
       i ← i + 1                                          // 墓碑：继续往下探
    return 未找到

Insert(T, k)
    first ← -1;  i ← 0
    while i < m
        pos ← (H(k) + i) mod m
       if T[pos].state = DELETED and first = -1 then first ← pos   // 记
↪   下第一个墓碑
        if T[pos].state = EMPTY then
            放到 (first ≠ -1 ? first : pos)，置 OCCUPIED；return 成功
       if T[pos].state = OCCUPIED and T[pos].key = k then return 重复，拒绝
        i ← i + 1
    if first ≠ -1 then 放到 first；return 成功
    return 表满

Delete(T, k)
    pos ← Search(T, k)
    if pos = 未找到 then return false
    T[pos].state ← DELETED                                 // 打墓碑，不置
↪   EMPTY
    return true
```

墓碑在三处的作用各不相同：- **检索**：必须**跳过**它继续探查——它证明「这里曾经有人，后面可能还有同义词」；- **插入**：可以**占用**它，但**不能一见到就停**，仍要把探查链走到真空位，否则会插入重复关键码（上面的 `first` 变量就是为这件事准备的）；- **删除**：只打标记，**不恢复成** EMPTY。

代价：三者最坏都是 $O(m)$，平均 $O(1)$（α 受控时）。墓碑多了要重建表——这是闭散列的固有负担。

10-17. 设计算法，判断两个长度分别为 n、m 的整数数组是否有公共元素。分别给出基于排序和基于散列的方案并比较。

答：**方案一（排序 + 双指针）**：把较短的数组排序，或两个都排序后同步扫描。

```
HasCommon_Sort(A, n, B, m)
    对 A、B 分别排序
    i ← 0;  j ← 0
    while i < n and j < m
        if A[i] = B[j] then return true
        else if A[i] < B[j] then i ← i + 1
        else j ← j + 1
    return false
```

时间 $O(n\log n + m\log m)$，空间 $O(1)$（原地排序时）。

方案二（散列）：把较短的数组建成散列集合，再拿另一个数组逐个去查。

```
HasCommon_Hash(A, n, B, m)
    S ← 空散列集合
    for x in A:  S.Insert(x)
    for y in B:  if S.Search(y) then return true
    return false
```

时间平均 $O(n+m)$、最坏 $O(nm)$；空间 $O(\min(n,m))$（把短的那个建表）。

怎么选：- 数据量大、内存够、只问「有没有」→ 散列，平均线性最快；- 内存紧张、或要求**最坏**也有保证、或本来就要用到有序结果 → 排序；- 若两数组**本来就有序**，直接用方案一的扫描部分，$O(n+m)$、空间 $O(1)$，最划算。

这道题的考点是「没有普遍最优解」：答题时把前提列清楚比选哪个更重要。

五、证明题

10-18. ★ 证明：二分检索的最大检索长度为 $\lceil\log_2(n+1)\rceil$。

证明. 二分检索的判定树是一棵有 n 个内部结点的二叉树，最大检索长度等于树的层数 h（根算第 1 层，比较 1 次）。二分每次把候选区间**至少折半**，所以第 j 层至多有 2^{j-1} 个结点，而判定树的形状由折半过程唯一决定、且是「尽量填满」的，故

$$2^{h-1} \le n \le 2^h - 1$$

左端由「前 $h-1$ 层若排满只有 $2^{h-1}-1$ 个结点，不足以放下 n 个」推出，右端由「h 层至多 2^h-1 个结点」推出。整理得 $2^{h-1} < n+1 \le 2^h$，两边取对数得 $h-1 < \log_2(n+1) \le h$，即 $h = \lceil\log_2(n+1)\rceil$。∎

这与第 5 章「n 个结点的完全二叉树高度为 $\lceil\log_2(n+1)\rceil$」是同一条结论——**二分检索的判定树就是一棵完全二叉树形状的** BST。

10-19. 证明：闭散列表中**不能**用「置为空」来实现删除。

证明. 反设可以。构造一个反例即可：设表长 $m = 7$，$H(k) = k \bmod 7$，线性探测。依次插入 7、14、21，它们都散列到 0，于是占据下标 0、1、2。

此时检索 21 的过程是：探 0（是 7，不等）→ 探 1（是 14，不等）→ 探 2（命中）。

现在删除 14 并把下标 1 置空。再检索 21：探 0（不等）→ 探 1（**空位，判定失败**）→ 返回「不在表中」。但 21 明明还在下标 2。矛盾。∎

根子在于：闭散列的检索用「遇到空位就停」作为失败判据，而这个判据成立的前提是「探查链上没有空洞」。删除一旦挖出空洞，判据就不再正确。墓碑的作用正是**保住这条链的连续性**，同时又让该槽可以被重新使用。

第 11 章 索引技术

一、填空题

11-1. 索引把 ____ 与 ____ 关联起来。**稠密索引**指 ____，**稀疏索引**指 ____；只有当主文件 ____ 时才能用稀疏索引。

答：**关键码**与**记录的存储地址**；稠密索引为**每一条记录**建一个索引项；稀疏索引只为**每一个存储块（页）**建一个索引项；只有主文件本身**按该关键码有序**时才行。由此可得一条硬约束：**一个文件至多有一个稀疏索引（对应它的物理排序字段），但可以有多个稠密索引**——这就是「主索引 vs 辅索引」的分界。

11-2. 倒排索引的两个组成部分是 ____ 和 ____。它把「给定记录找属性值」反过来变成 ____，因此特别适合 ____。

答：**词典（关键词表）**与**倒排表（每个关键词对应的记录号列表）**；变成「**给定属性值找记录**」；适合**全文检索、多关键词组合查询**。求「同时含 A 和 B 的文档」时，把两条倒排表（都按记录号有序）用双指针求交即可，代价 $O(|L_A| + |L_B|)$——**两条表都有序是这一步能做快的前提**。

11-3. 索引技术中的几个基本概念：按关键码顺序存放记录的文件叫 ____；按关键码的散列值决定记录存放位置的文件叫 ____；以「属性值 → 记录号表」组织的文件叫 ____。索引结构一旦建立就不再改变的叫 ____，随插入删除自行调整的叫 ____，后者的代表是 ____。

答：**顺序文件**；**散列文件**；**倒排文件**；**静态索引结构**；**动态索引结构**；**B 树（及 B+ 树）**。

三种文件组织的适用面各不相同：| | 擅长 | 不擅长 | | --- | --- | --- | | 顺序文件 | **范围查询**、顺序批处理；可用稀疏索引 | 插入要搬移记

录 | | 散列文件 | **按关键码等值查找**，平均 $O(1)$ | **不支持范围查询、不支持顺序访问** | | 倒排文件 | **多关键词组合查询**、全文检索 | 更新代价高（倒排表要跟着改）|

静态与动态的分界：**静态索引**（如多分树、分块索引）建好之后插入只能进溢出区，溢出多了性能就塌；**动态索引**（B 树）用分裂与合并把结构维持住，**代价是每次更新都可能写多个磁盘页**（见 11-20）。

11-4. ★ m 阶 B 树的五条性质：每个结点至多 ____ 个子结点；除根和叶外每个结点至少 ____ 个子结点；根结点至少 ____ 个子结点（除非它是叶）；所有叶结点 ____；有 k 个子结点的结点恰含 ____ 个关键码。

答：m；$\lceil m/2 \rceil$；2；**都在同一层**；$k-1$。由此可得关键码个数的上下限：非根结点含 $\lceil m/2 \rceil - 1$ 到 $m-1$ 个关键码。「**所有叶结点在同一层**」**是 B 树的灵魂**——它保证了树高平衡，从而保证访外次数有上界。阶为 3 的 B 树就是通常说的 **2-3 树**。

11-5. ★ B+ 树与 B 树的主要区别：B+ 树的**所有记录都存在** ____，内部结点只起 ____ 作用；B+ 树的叶结点之间通常用 ____ 链起来。因此 B+ 树比 B 树多出的一项能力是 ____。

答：**叶结点**；**索引（分界）**；**指针（顺序链）**；**支持高效的顺序访问与范围查询**。

两条推论必须记牢：1. B+ **树的关键码在内部结点上会重复出现**（往根的方向一层层复写），B 树则不重复；2. B+ **树的每次检索都要走到叶**（长度固定），B 树可能在内部结点就命中（长度不定）。

内部结点不带数据 $\Longrightarrow$ 同样大小的页能装下更多分支 $\Longrightarrow$ **树更矮、访外更少**，这是数据库索引普遍选 B+ 树的真正原因。

11-6. 在含 n 个关键码的 m 阶 B 树上检索，最多访问 ____ 个结点；树高 h 满足 $h \le$ ____。

答：h 个（从根到叶各一次），再加 1 次读主文件数据块；$h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$。这个上界来自「最坏情况下每个非根结点只有 $\lceil m/2 \rceil$ 个子结点」。数量级感受一下：$m=100$、$n=10^8$ 时，上界为 $\log_{50}(5\times 10^7)+1 \approx 5.5$，即**最坏也只有 5 层**；结点装得较满时（每结点约 100 个分支）只有 $\log_{100} 10^8 = 4$ 层左右——**四五次访外就能定位一亿条记录中的一条**。

11-7. 红黑树的四条性质：① 结点非红即黑；② ____ 是黑色；③ 红结点的子结点 ____；④ 从任一结点到其所有后代空叶结点的路径上，____ 相同。由此可得含 n 个内部结点的红黑树，树高最多 ____。

答：**根结点**（有的教材还要求空叶结点为黑）；**必须是黑色**（即**不允许红红相邻**）；**黑结点个数**（黑高）；$2\log_2(n+1)$（只数内部结点；教材原书把末端空叶也数进树高，写作 $2\log_2(n+1)+1$，两种写法都对，答题时说明口径）。**这四条合起来只做一件事：让最长路径不超过**

最短路径的两倍。最短路径全黑，最长路径红黑相间——长度至多是前者的 2 倍，所以树高仍是 $O(\log n)$。

二、选择题

11-8. ★ B+ 树不同于 B 树的特点之一是（　）。A. 能支持顺序检索 B. 结点中含有关键码 C. 根结点至少有两个分支 D. 所有叶结点都在同一层上

答： A。B、C、D 三条 B **树也都有**，不是「不同点」。A 是 B+ 树独有的：全部记录都在叶结点上，且叶结点用指针顺序链起来，于是**从某个关键码开始顺着叶链往下走就能做范围扫描**，B 树做不到这一点（B 树的记录散布在各层内部结点上，顺序访问要中序遍历整棵树）。

这类题的解法：把四个选项逐个去问「B 树有没有?」——**有的就不是"不同点"**。

11-9. 一棵 5 阶 B 树，除根结点外，每个结点的关键码个数范围是（　）。A. 1 到 4　B. 2 到 4　C. 2 到 5　D. 3 到 5

答： B。子结点数至少 $\lceil 5/2 \rceil = 3$，至多 5；关键码数 = 子结点数 -1，即 2 到 4。**根结点是唯一的例外**：它可以只有 1 个关键码（2 个子结点），独根时甚至可以只有 1 个关键码而没有子结点。

11-10. 向 B 树插入关键码时，结点「上溢」的处理办法是（　）。A. 把多余的关键码移到兄弟结点 B. 把结点分裂成两个，**中间**的关键码上升到父结点 C. 把最大的关键码上升到父结点 D. 重建整棵树

答： B。取中间那个关键码上升，才能保证分裂出的两半都满足「至少 $\lceil m/2 \rceil - 1$ 个关键码」。**插入引起的分裂可能一路向上传播，若根也分裂，树就长高一层**—— B 树是**从叶向根生长**的，这与 BST 从根向叶生长正好相反，也正是「所有叶结点始终在同一层」能被维持住的原因。

11-11. 红黑树插入新结点后出现「红红冲突」，若新结点的**叔父结点是红色**，应当采取的操作是（　）。A. 以祖父结点为轴旋转　B. 父结点与祖父结点换色后旋转 C. 父、叔父变黑，祖父变红，冲突上移　D. 直接把新结点变黑

答： C。叔父为红时**只换色不旋转**：父和叔父变黑、祖父变红，这样这棵子树的黑高不变，但祖父变红后可能与**它的**父亲再次红红冲突，于是把祖父当作新的「问题结点」继续向上处理——**冲突沿路径上移，最多走 $O(\log n)$ 层**。只有叔父为**黑**（或不存在）时才需要旋转（B 描述的是那一种情形）。D 错：直接把新结点变黑会破坏黑高相等。

三、辨析与简答

11-12. ★ 简答：为什么外存索引用 B 树 / B+ 树，而不用 AVL 树或红黑树?

答：因为**外存的代价单位是「页」，不是「结点」**。- AVL、红黑树是二叉树，n 个关键码的树高约 $\log_2 n$。若以结点为内外存交换单位，检索一次要访外约 $\log_2 n$ 次—— $n = 10^8$ 时约 27 次，每次寻道以毫秒计，

完全不能接受。- B 树把一个**磁盘页**做成一个结点，页内塞尽可能多的关键码（阶 m 可达几百）。树高降到 $\log_m n$，$m=100$、$n=10^8$ 时只有 4 层左右，**访外 4 次**。

一句话：二叉树优化的是比较次数，B 树优化的是访外次数；内存里比较很便宜，外存里访问很贵，所以两种场合的最优结构不同。反过来，红黑树、AVL 在内存里仍然是首选（B 树的页内查找在内存里反而是浪费）。

11-13. 简答：B 树的插入和删除分别可能引起什么调整？各自怎样保持性质？

答：| | 触发条件 | 处理 | 传播方向 | | --- | --- | --- | --- | | 插入 | 结点关键码数 $>m-1$（**上溢**） | **分裂**：中间关键码上升到父结点，其余分成两半 | 可能一路传到根，**树高加一** | | 删除 | 结点关键码数 $<\lceil m/2\rceil-1$（**下溢**） | 先**向兄弟借**（旋转，父结点的分界随之更换）；兄弟也借不出时**与兄弟合并**，并把父结点的分界拉下来 | 合并可能使父结点下溢，一路传到根，**树高减一** |

两条共同点：**调整只沿着一条根到叶的路径进行**，代价 $O(h)=O(\log_m n)$；**所有叶结点始终在同一层**这条性质在任何时刻都保持。

11-14. 辨析：「红黑树是一种平衡二叉树，所以它和 AVL 树的平衡标准是一样的。」

答：错。两者的"平衡"定义完全不同。 - AVL：任何结点的左右子树**高度差不超过 1**。这是很强的约束，树更矮（更接近 $\log_2 n$），**检索更快**；代价是插入删除时要做更多旋转来维持。- **红黑树**：用颜色约束保证**最长路径不超过最短路径的 2 倍**，树高上界是 $2\log_2(n+1)$（只数内部结点；按原书口径连空叶一起数是 $2\log_2(n+1)+1$），比 AVL 松。好处是**插入删除的旋转次数有常数上限**（插入至多 2 次旋转，删除至多 3 次），调整更便宜。

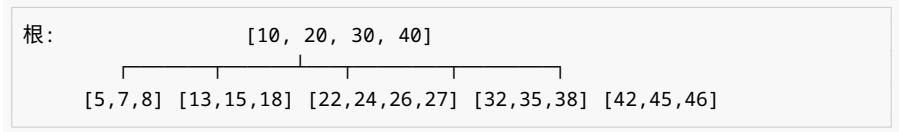
结论：查多改少 → AVL；改动频繁 → 红黑树。这也是 C++ `std::map`、Java `TreeMap`、Linux 内核调度器都选红黑树的原因。

四、算法设计与分析

11-15. ★ 依次把关键码 $20,40,10,30,15,35,7,26,18,22,5,42,13,46,27,8,32,38,24$ 插入一棵初始为空的 **5 阶** B 树，画出最终结果，并说明根结点里有哪些关键码。

答：5 阶 B 树每结点至多 4 个关键码，满 5 个时分裂、中间那个（第 3 个）上升。

最终的 B 树（两层）：



根结点中的关键码是 10、20、30、40。

核对一遍性质：根有 5 个子结点（$\le m=5$ √）、4 个关键码（$=5-1$

√）；每个叶有 3 或 4 个关键码（在 $[\lceil 5/2\rceil - 1,\ m-1] = [2,4]$ 内 √）；所有叶在同一层 √；关键码总数 $4+3+3+4+3+3 = 20$ √。

关键的几次分裂：1. 插到第 5 个关键码时根满（`10,15,20,30,40`）→ 中间的 20 上升成新根，树从 1 层长到 2 层；2. 此后每当某个叶装到第 5 个关键码就分裂一次，中间那个升进根；根里依次积累出 10、30、40，最终为 `[10,20,30,40]`；3. 若再插一个使某叶分裂、根变成 5 个关键码，根也会分裂，树将长到 3 层。

答题提醒：阶为**偶数**时「中间那个」有两种取法（偏左或偏右），不同教材结果不同；**做题前先声明自己用哪种**，或者优先挑奇数阶的题做。

11-16. ★ 依次把 $10, 20, 30, 40, 50, 60, 70, 80, 90$ 插入一棵初始为空的 5 **阶** B+ **树**，画出最终结果；并与同样输入下的 5 **阶** B **树**（做法见 11-15）对照，说明两者的差别。

答：约定（做题前务必声明）：5 阶 B+ 树的内部结点至多 5 个子结点、4 个关键码；叶结点至多 4 个关键码；**叶满时分裂成 2 + 3，并把右叶的首关键码"复制"上去**；内部结点满时分裂，中间关键码"移动"上去。

逐步过程：

插入	发生了什么
10,20,30,40	都进同一个叶（同时也是根），`[10,20,30,40]`
50	叶满 5 个 → 分裂成 `[10,20]` 与 `[30,40,50]`，**复制** 30 上去成为新根 `[30]`
60	进右叶 → `[30,40,50,60]`
70	右叶满 → 分裂成 `[30,40]` 与 `[50,60,70]`，**复制** 50 上去，根变 `[30,50]`
80	进最右叶 → `[50,60,70,80]`
90	最右叶满 → 分裂成 `[50,60]` 与 `[70,80,90]`，**复制** 70 上去，根变 `[30,50,70]`

最终 B+ 树：

```
内部（索引层）：         [30, 50, 70]
                      /      |     |      \
叶（数据层）：  [10,20] ↔ [30,40] ↔ [50,60] ↔ [70,80,90]
                  （叶之间用指针顺序链起来）
```

同样输入下的 5 阶 B 树：

```
根:                    [30, 60]
             /            |            \
         [10,20]       [40,50]      [70,80,90]
```

三处差别，都要能说出来：1. **关键码是否重复**。B+ 树的 30、50、70 **既在根里、又在叶里**（索引层是叶层的复写）；B 树的 30、60 **只在根里，叶中没有**。2. **记录放在哪里**。B+ 树全部记录都在叶层，内部结点只是分界；B 树的记录跟着关键码散布在各层。3. **检索长度**。B+ 树每次都要走到叶，长度固定等于树高；B 树**可能在内部结点就命中**，长度不定（运气好时更短）。

由此得到 B+ 树的看家本领：叶层顺序链起来，**从任一关键码出发可以顺着叶链做范围扫描**；而且内部结点不带数据，同样大小的页能装更多分支，**树更矮、访外更少**。这就是数据库索引普遍选 B+ 树的原因（对照 11-5）。

11-17. ★ 一棵红黑树如下（括号里是颜色）。(1) 插入关键码 40，写出自平衡的过程与结果；(2) 在原树上插入关键码 3，同样作答。

```
           20(黑)
        /          \
     10(红)        30(黑)
     /    \        /    \
  5(黑) 15(黑) 25(红)  35(红)
  /
1(红)
```

答：(1) **插入** 40：按 BST 规则挂到 35 的右孩子，新结点为红 → **与父 35（红）红红冲突**。看**叔父**：40 的祖父是 30，叔父是 25，25 **是红色** → 走「换色」分支：父 35 与叔父 25 都变黑、祖父 30 变红，冲突上移到 30；而 30 的父亲是根 20（黑），**没有新的红红冲突，处理结束**。

```
            20(黑)
         /          \
      10(红)        30(红)
      /    \        /    \
   5(黑) 15(黑) 25(黑)  35(黑)
   /                        \
 1(红)                      40(红)
```

(2) **插入** 3：按 BST 规则，$3 < 20 \to 3 < 10 \to 3 < 5 \to 3 > 1$，挂到 1 的**右**孩子，红色 → 与父 1（红）红红冲突。祖父是 5，**叔父不存在（视为黑）** → 走「旋转」分支。又因为 3 是「祖父 5 的左孩子 1

的**右**孩子」，属于**内侧（LR 型）**，要双旋：1. 先对父结点 1 作**左旋**，把 3 提上来，变成外侧情形（3 的左孩子是 1）；2. 再让新的父结点 3 与祖父 5 **换色**（3 变黑、5 变红），并对祖父 5 作**右旋**。

```
            20(黑)
          /          \
      10(红)         30(黑)
      /    \         /    \
    3(黑) 15(黑) 25(红)  35(红)
   /   \
1(红) 5(红)
```

两问正好覆盖红黑树插入的两大情形：叔父红 → 只换色、冲突上移；叔父黑 → 换色 + 旋转（内侧先把它转成外侧）。核对黑高：结果里从根到任一空叶的黑结点数都相同，且没有红红相邻 √

11-18. ★ 红黑树的**删除**。给定下面这棵红黑树：

```
            20(黑)
          /          \
      10(红)         30(黑)
      /    \         /    \
   5(黑) 15(黑) 25(红)  35(红)
   /
1(红)
```

(1) 删除关键码 1；(2) 删除关键码 15。分别说明调整过程并画出结果。(3) 另给一棵树：根 `20(黑)`，左孩子 `10(黑)`，右孩子 `30(黑)`（都是叶）。删除 10 会怎样？

答：先把删除的总框架说清楚（三步）：1. **按 BST 的办法定位**。若被删结点有两个孩子，**用中序后继（或前驱）顶替它**，问题就转化成「删除一个至多有一个孩子的结点」——这一步与普通 BST 完全相同。2. **看真正被摘走的那个结点的颜色**：- **红色** → 摘掉它不改变任何路径上的黑结点数，**什么都不用做**；- **黑色** → 该分支的黑高少了 1，出现所谓「**双黑**」，必须修复。3. **修复**（只在第 2 步是黑色时进行）：若顶上来的孩子是红的，**把它涂黑**即可补回；否则按「兄弟与两个侄子的颜色」分四种情形处理，**至多 3 次旋转**。

---

(1) **删除 1** —— 1 是**红色**叶结点，直接摘走，黑高不变，**不需要任何调整**：

```
            20(黑)
          /          \
      10(红)         30(黑)
      /    \         /    \
```

```
   5(黑) 15(黑) 25(红)  35(红)
```

(2) **删除** 15 —— 15 是**黑色**叶结点，摘走后它那一侧黑高少 1，出现双黑。看它的兄弟：双黑位于父 10 的**右**边，兄弟是 5(黑)，兄弟的「远侄」（远离双黑的那个侄子，即 5 的左孩子）是 1(红) → **情形 4**：**兄弟取父结点的颜色、父结点与远侄都变黑、对父结点作一次旋转**（这里是右旋）。

```
          20(黑)
        /        \
     5(红)       30(黑)
     /   \       /    \
  1(黑) 10(黑) 25(红)  35(红)
```

核对：没有红红相邻；从根到任一空叶的黑结点数都相同 √

(3) **删除** 10 —— 10 是黑叶，摘走后左边双黑；兄弟 30(黑)，它的两个孩子都是空叶（都算黑）→ **情形 2**：**把兄弟涂红，双黑上移给父结点**。父结点是根，循环结束、把根涂黑即可：

```
    20(黑)
       \
       30(红)
```

注意这一步**整棵树的黑高减少了 1**——这是合法的，因为所有路径同时减少。

删除修复的四种情形（记这张表就够）。设双黑结点为 x、其兄弟为 w：

情形	条件	处理	之后
1	w 是**红**的	w 与父换色，对父旋转	化归成 w 为黑的情形
2	w 黑，**两个侄子都黑**	w 变红，**双黑上移到父**	继续向上，最多走 $O(\log n)$ 层
3	w 黑，**近侄红、远侄黑**	近侄与 w 换色，对 w 旋转	化归成情形 4
4	w 黑，**远侄红**	w 取父色，父与远侄变黑，对父旋转	**修复完成，结束**

要点：只有**情形 2 会向上传播**，其余三种最多再走一步就结束，所以

删除的旋转次数**至多 3 次**（情形 1 → 情形 3 → 情形 4 各一次），而颜色翻转最多 $O(\log n)$ 次。

与插入对照（插入见 11-17）：| | 插入 | 删除 | | --- | --- | --- | | 麻烦的来源 | **红红相邻** | **黑高变少** | | 看谁的颜色 | **叔父** | **兄弟和两个侄子** | | 只换色的那一支 | 叔父红 → 冲突上移 | 情形 2 → 双黑上移 | | 旋转次数上限 | 2 | 3 |

11-19. 设主文件有 10^6 条记录，每个磁盘页可容纳 100 个索引项。(1) 若用 B+ 树作索引，树高大约是多少？检索一条记录要访外几次？(2) 与「顺序检索」和「内存二分检索」相比，差别在哪？

答：(1) B+ 树的叶层要放下 10^6 个索引项，每页 100 个 → 叶层约 10^4 页；上一层为这 10^4 页建索引 → 约 10^2 页；再上一层 → 1 页（根）。所以树高约 **3 层**（根 + 中间层 + 叶层）。

检索一条记录：读根、读中间层、读叶，共 **3 次访外**，再加 1 次读主文件数据页，合计 **4 次访外**。

(2) 对比（设一次访外约 10^{-2} 秒、一次内存比较约 10^{-8} 秒）：

方法	访外次数	说明
顺序检索主文件	平均 $\frac{10^6}{100}/2 = 5000$ 页（设主文件每页也放 100 条记录）	完全不可用
B+ 树索引	4	树高决定一切
内存二分（假设全装得下）	0	约 20 次比较，但前提是 10^6 条记录都在内存

结论：外存场景下，**衡量代价的唯一尺子是访外次数**；B+ 树用「加大结点、降低树高」把它压到个位数。内存装得下时当然内存结构更快，**但索引存在的意义正是"装不下"那一档**。

11-20. ★ 分析 m 阶 B 树上**检索、插入、删除**各自的**读盘**与**写盘**次数（设树高为 h，一个结点恰好是一个磁盘页，且根结点常驻内存之外不作特殊处理）。

答：先立一条口径：**读一个结点算一次读盘，改一个结点就要把它写回，算一次写盘**。

(1) **检索** - **读**：从根沿一条路径下降到目标结点，最多 h 次；若关键码在内部结点就命中（B 树可能如此），则少于 h 次。找到之后还要按地址去主文件取记录，**再加 1 次读**。- **写**：**0 次**——检索不改动索引。- 合计最多 $h+1$ 次访外。

(2) **插入** - **读**：先走一遍检索路径到叶，h 次读。- **写**：分两种情况。- **不发生分裂**：只改那个叶结点，**写 1 次**。- **发生分裂**：设自下而上共有 s

个结点分裂。每分裂一个结点要写出**两个**新结点，共 $2s$ 次；最上面那个接收分界关键码的父结点还要改一次，**再写 1 次**。合计 $2s+1$ **次写**。- **若一直分裂到根**（$s=h$），根分裂后还要**新建一个根**，合计 $2h+1$ 次写，树高加一。- **最好情形**：h 读 + 1 写；**最坏情形**：h 读 + $2h+1$ 写。

(3) **删除 - 读**：检索路径 h 次；若被删关键码在内部结点，还要向下找中序前驱/后继来顶替，最多再读到叶层，仍是 $O(h)$。**向兄弟借**或**与兄弟合并**时要读兄弟结点，每层**多读 1 次**。- **写**：- **不发生下溢**：写 1 次。- **向兄弟借**（旋转）：改被删结点、兄弟、父结点三个，**写 3 次**，且**到此为止**；- **与兄弟合并**：合并成一个结点（写 1）、父结点少一个分界（写 1），若父结点因此下溢则继续向上；设向上传播了 t 层，约 $2t$ **次写**。- 合并一直传到根且根只剩一个孩子时，**树高减一**。

三条最值得记住的结论：1. **读盘次数由树高决定，**$h=O(\log_{\lceil m/2\rceil} n)$，而 m 在**对数的底**上—— 这就是「把结点做成一整页、把 m 做大」的全部收益（见 11-21）。2. **写盘次数取决于分裂/合并传播了几层**，绝大多数插入删除都不传播，所以**平均写盘次数接近 1**；最坏才是 $O(h)$。3. **检索不写盘**——这是 B 树能承受高并发只读负载的原因。

B+ **树的差别**：检索**必须走到叶**，读盘固定为 h（不会提前命中）；但内部结点不存数据，同样页大小下 m 更大、h 更小，**通常总读盘次数反而更少**；且范围查询可以沿叶链顺序读，**一次定位 + 顺序扫描**，比 B 树的中序遍历省得多。

五、证明题

11-21. ★ 证明：含 n 个关键码的 m 阶 B 树，其高度 h 满足 $h \le \log_{\lceil m/2\rceil} \frac{n+1}{2} + 1$。

证明. 记 $t=\lceil m/2\rceil$。要让树尽可能**高**，就让每个结点尽可能**少**装子结点。按定义，根至少 2 个子结点，其余非叶结点至少 t 个子结点。于是第 $1,2,3,\cdots,h,h+1$ 层（根为第 1 层）的结点数依次至少为

$$1,\ 2,\ 2t,\ 2t^2,\ \cdots,\ 2t^{h-2},\ 2t^{h-1}$$

其中叶结点全在第 h 层，**第 $h+1$ 层是「失败结点」（外部结点）**——把它算作一层更方便。失败结点数恰为 $n+1$：n 个关键码按大小把整个关键码空间切成 $n+1$ 段，每一段对应一次检索失败的落点，即一个失败结点（这与第 5 章扩充二叉树的结论同理）。第 $h+1$ 层的失败结点数至少为 $2t^{h-1}$，故

$$n+1 \ge 2t^{h-1}$$

两边取以 t 为底的对数：$\log_t \frac{n+1}{2} \ge h-1$，即

$$h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$$

∎

这个界的实际含义：m 出现在**对数的底**上。与二叉树的 $\log_2 n$ 相比，$m=100$ 时树高上界约为 $\log_{50} n$，相当于除以 $\log_2 50 \approx 5.6$——**这就是"把结点做成一整页"的全部收益**。

11-22. 证明：含 n 个内部结点的红黑树，其高度不超过 $2\log_2(n+1)$。

证明. 记结点 x 的**黑高** $bh(x)$ 为从 x（不含 x）到任一后代空叶路径上的黑结点数（由性质④，这个数与选哪条路径无关）。

第一步：以 x 为根的子树至少有 $2^{bh(x)}-1$ 个内部结点。对 x 的高度作归纳：x 是空叶时 $bh(x)=0$，内部结点数 $0=2^0-1$ ✓；一般地 x 的两个孩子的黑高至少为 $bh(x)-1$，由归纳假设各自至少有 $2^{bh(x)-1}-1$ 个内部结点，加上 x 自己，总数至少 $2(2^{bh(x)-1}-1)+1 = 2^{bh(x)}-1$ ✓。

第二步： $bh(\text{根}) \ge h/2$。由性质③（不允许红红相邻），任何一条从根到空叶的路径上，红结点不可能连续出现，因此**至少一半的结点是黑的**。

两步合起来：$n \ge 2^{bh(\text{根})}-1 \ge 2^{h/2}-1$，即 $2^{h/2} \le n+1$，取对数得 $h \le 2\log_2(n+1)$。∎

口径说明：这里的高度 h 只数从根到空叶路径上的**内部结点**。教材第 11 章（原书）把末端的空叶也数进树高，同一结论写作 $2\log_2(n+1)+1$——两者说的是同一件事。

推论：红黑树的检索、插入、删除都是 $O(\log n)$；它比 AVL 松一倍，换来的是**旋转次数的常数上界**（插入至多 2 次、删除至多 3 次）。

第 12 章 高级数据结构

一、填空题

12-1. ★ 一个 n 阶**下三角矩阵** A（$i \ge j$ 处非零），按**行优先**压缩存入下标从 0 开始的一维数组 B，共需 ____ 个存储单元；元素 $a_{i,j}$ 在 B 中的下标是 ____。若 $n=200$，则 $a_{60,25}$ 的下标是 ____。

答：$\frac{n(n+1)}{2}$；$\frac{i(i+1)}{2}+j$（$i \ge j$，下标都从 0 起）；1855。

推导：$a_{i,j}$ 之前完整存了第 0 到第 $i-1$ 行，共 $1+2+\cdots+i = \frac{i(i+1)}{2}$ 个元素，再加上本行它前面的 j 个。代入 $i=60$、$j=25$：$\frac{60\times61}{2}+25 = 1830+25 = 1855$。

两个必查的细节：1. **下标从 0 还是从 1**——从 1 起时公式变成 $\frac{i(i-1)}{2}+$

j，差很远，做题前先声明；2. **上三角要换公式**，别硬套。若矩阵**对称**，访问上三角的 $a_{i,j}$（$i<j$）时 先换成 $a_{j,i}$ 再套下三角公式。

12-2. 三维数组 `int A[3][4][5]` 按行优先存储、首地址为 0、每个元素占 1 个单元，则 $A[2][1][3]$ 的偏移是 ____。

答：$2\times(4\times5)+1\times5+3=40+5+3=48$。通法：**从右往左累计每一维之后所有维大小的乘积**。也可以用霍纳法一边读下标一边累乘：$((2\times4+1)\times5+3)=48$，不必预存跨度。

12-3. ★广义表 $L=(a,(b,(c,d)),e)$ 的**长度**是 ____，**深度**是 ____；head$(L)=$ ____，tail$(L)=$ ____。

答：长度 3；深度 3；head$(L)=a$；tail$(L)=((b,(c,d)),e)$。

三条定义分清楚：- **长度** = 最外层括号里**逗号分隔的元素个数**，这里是 a、$(b,(c,d))$、e 三个；- **深度** = 括号的最大嵌套层数，L 外一层、$(b,\cdots)$ 二层、(c,d) 三层；- **表头**是第一个元素（可以是原子也可以是表），**表尾一定是个表**—— 是去掉第一个元素后**剩下的那些元素组成的表**，所以 tail 的结果外面必带一层括号。

最容易错的是把表尾写成 $(b,(c,d))$（漏了外层括号、也漏了 e）。

12-4. $m\times n$ 的矩阵中有 t 个非零元，**稀疏因子**定义为 ____；通常 ____ 时按稀疏矩阵处理。稀疏矩阵常用的两种存储是 ____ 和 ____。

答：$\delta=\frac{t}{mn}$；$\delta<0.05$；三元组表 $(行,列,值)$ 和**十字链表**。三元组表省空间、便于按行扫描，但插入删除要搬元素；十字链表让每个非零元同时挂在一条**行链**和一条**列链**上，便于按行、按列双向遍历，也便于动态增删——代价是每个非零元多两个指针。

12-5. ★ AVL 树中，插入一个结点后失衡的结点称为 ____，它的平衡因子只可能是 ____。四种旋转类型是 ____、____、____、____，其中需要**双旋**的是 ____。

答：**危急结点**（最靠近插入点的失衡祖先）；只可能是 $+2$ 或 -2；LL、RR、LR、RL；需要双旋的是 LR **和** RL。

判型的口诀：看**从危急结点往下走的头两步**—— 先左再左是 LL（右旋一次），先右再右是 RR（左旋一次），先左再右是 LR，先右再左是 RL（这两种要先把" 内侧" 那一步转成" 外侧"，再单旋）。**旋转只动局部几根指针，并且不改变中序次序**——这是所有平衡树旋转的共同前提。

12-6. 伸展树的三种旋转是 ____、____、____。访问某个结点之后要把它 ____；删除结点时被展开的是 ____。伸展树的代价保证是 ____。

答：**单旋转**、**一字形**（zig-zig）**双旋转**、**之字形**（zig-zag）**双旋转**；**展开**（splay）**到根**；**被删结点的父结点**；**摊还** $O(\log n)$。

「**摊还** $O(\log n)$」**的确切含义**：不保证每一次都快—— 访问一条深链的末端，**单次代价仍可达** $O(n)$；它保证的是从任意初始树出发，连续 m 次操作的**总代价**为 $O(m\log n)$。所以伸展树适合「热点键反复出现」的场景，**不适合要求单次延迟上界的实时系统**（那要用 AVL 或红

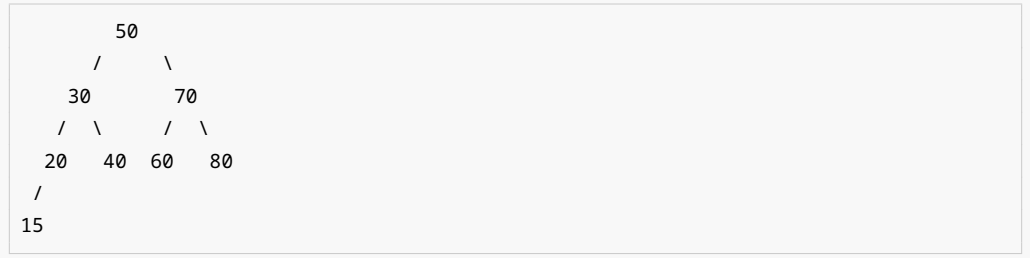
黑树）。

二、选择题

12-7. 广义表 $L=((a,b),c,(d,(e)))$ 的深度和长度分别是（　）。A. 3，3　B. 2，3　C. 3，4　D. 2，4

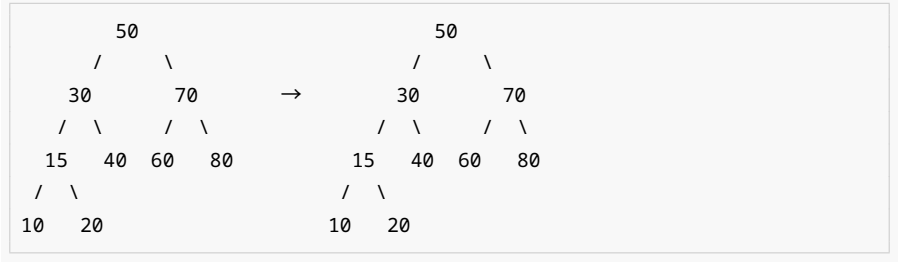
答： A。长度是最外层的三个元素 (a,b)、c、$(d,(e))$；深度看最大嵌套：(e) 在第 3 层。**别把"元素个数"数成"原子个数"**——(a,b) 整个只算一个元素。

12-8. ★ 对下面这棵 AVL 树插入关键码 10，需要作的旋转是（　）。



A. 不需要旋转　B. 在 20 处作 LL 单旋　C. 在 30 处作 LR 双旋　D. 在 50 处作 RR 单旋

答： B。10 挂到 15 的左孩子。自下而上找第一个失衡的祖先：15 的平衡因子变成 $+1$（还行）；20 **的左子树高 2、右子树高 0，因子 $+2$，它就是危急结点**；从 20 往下走的头两步是「左（到 15）、左（到 10）」→ LL **型，作一次右旋**：



注意「危急结点是最靠近插入点的那个失衡祖先」：这里 30 和 50 也都受影响，但把 20 处的失衡修好之后，整棵子树的高度回到插入前的值，**上面的祖先自动恢复平衡**，不必再动。这是 AVL 插入「至多一次旋转（单旋或双旋）」的原因。

12-9. 关于伸展树，下列说法**错误**的是（　）。A. 它不保证任何时刻都是平衡的 B. 一次访问的最坏代价可能是 $O(n)$ C. 连续 m 次操作的总代价是 $O(m\log n)$ D. 它和 AVL 树一样，每次操作后树高都不超过 $1.44\log_2 n$

答： D。伸展树**根本不维护任何高度不变量**，树可以很不平衡；它只保证摊还代价。A、B、C 都是对伸展树的正确描述。$1.44\log_2 n$ 是 AVL

树高的上界，与伸展树无关。

12-10. 最佳二叉搜索树追求的目标是（ ）。A. 树高最小　B. 平均检索长度最小　C. 结点数最少　D. 旋转次数最少

答：B。给定各关键码的**检索概率**，最佳 BST 使 $\sum_i p_i \cdot (\text{结点} i \text{ 的层数} + 1)$ 最小，即带权平均检索长度最小。**高频关键码应当靠近根**——所以它一般**不是**高度最小的那棵树。这和 Huffman 树是同一类思想（都最小化带权路径长度），但 Huffman 只在叶上放数据、可以自由排列，最佳 BST 必须保持中序有序，因此不能用贪心，要用**动态规划**。

三、辨析与简答

12-11. ★ 简答：AVL 树、红黑树、伸展树都是改进的 BST，三者的取舍各在哪里？

答：

	AVL	红黑树	伸展树
平衡标准	左右子树**高差** ≤ 1	最长路径 $\le$ 最短路径的 2 倍	**不保证平衡**
树高	约 $1.44\log_2 n$，最矮	约 $2\log_2 n$	无上界
单次代价	最坏 $O(\log n)$	最坏 $O(\log n)$	**摊还** $O(\log n)$，单次可达 $O(n)$
调整代价	插入 1 次旋转，删除可能 $O(\log n)$ 次	插入 ≤ 2 次、删除 ≤ 3 次旋转	每次访问都旋转（到根）
每结点额外空间	平衡因子（2 位）	颜色（1 位）	**无**
擅长	查多改少	查改均衡，通用	**热点键反复访问**

一句话：AVL 把代价花在维持结构上，换最快的查找；红黑树放松标准，换更便宜的更新；伸展树干脆不维持结构，**靠"把刚访问的搬到根"自适应地贴合访问模式**，只对总代价负责。

12-12. 辨析：「稀疏矩阵用三元组表存储，一定比二维数组省空间。」

答：不一定，有个临界点。设矩阵 $m \times n$、非零元 t 个，二维数组占 mn 个单元；三元组表每个非零元要存**行、列、值**三项，占 $3t$ 个单元。三元组更省当且仅当

$$3t < mn \quad \text{即} \quad \delta = \frac{t}{mn} < \frac{1}{3}$$

所以非零元超过三分之一时，三元组反而更费。实践中取 $\delta < 0.05$ 作为门槛，留足余量——因为三元组还有**别的代价**：随机访问 $a_{i,j}$ 从 $O(1)$ 变成 $O(\log t)$（二分）甚至 $O(t)$（顺序查找），插入一个新的非零元还要搬元素。**省空间是用时间换来的**。

12-13. 简答：Trie 树与 Patricia 树的区别是什么？它们相对于 BST 的优势在哪？

答：- Trie（**字符树**）：按关键码的**字符逐位**分支，每层消耗一个字符，树高等于最长关键码的长度，**与关键码个数 n 无关**。缺点是**单分支的链很多**——一个长前缀若只被一个关键码占用，会拉出一长串只有一个孩子的结点，空间浪费大。- Patricia **树**：把 Trie 里那些单分支的链**压缩成一条边**，每个内部结点记录「下一步该比较第几位」，于是内部

结点数降到 $O(n)$。

相对 BST 的优势：检索代价是 $O(|k|)$（关键码长度），**与集合规模无关**，而 BST 是 $O(\log n)$ 次**整串比较**，每次比较本身又是 $O(|k|)$；而且 Trie 天然支持**前缀查询**（求所有以某串开头的关键码），BST 做不到。代价是空间开销大、对字母表大小敏感。（本章对这两者只要求了解。）

12-14. ★ 简答：**半伸展树**（semi-splaying）与普通伸展树有什么区别？它为什么能减少旋转次数？

答：两者只在**一字形**（zig-zig）这一种情况上不同。

	伸展树（全伸展）	半伸展树
一字形怎么转	先转祖父、再转父，把 x 提两层	**像 AVL 的 LL/RR 那样处理**：以父结点的位置为轴旋转
下一轮从谁继续	仍是 x 本身	**当前结点上移到父结点**，从父结点继续
终点	必须把 x 送到**根**	**不要求把检索结点送到根**
旋转次数	与 x 的深度成正比	约为前者的一半

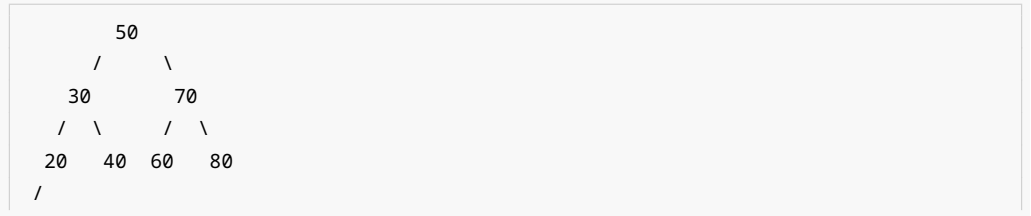
为什么能省：全伸展每一轮都盯着 x 自己，非把它顶到根不可；半伸展在一字形情形下把「当前结点」交给父结点接着往上走，**相当于每两层只付一层的代价**，路径上的结点同样被提了上来，但少转了一半。

代价：由于 x 未必到根，「**刚访问过的结点下次一定在根**」**这条性质就没有了**；它的摊还界与全伸展**不同**（仍是对数量级，但常数与分析方式都变了）。之字形（zig-zag）两者处理相同。

和 AVL、红黑树摆在一起看（对照 12-11）：伸展树一族的共同点是**不存任何平衡信息**（没有平衡因子、没有颜色），全靠「把刚访问的搬上来」自适应；半伸展则是在「搬多少」上做了个折衷。

四、算法设计与分析

12-15. ★ 对下面这棵 AVL 树依次插入 10、65、64，画出每次插入后的树，并指出每次的旋转类型（不需要旋转也要说明）。



```
15
```

答：**插入** 10 —— 挂到 15 的左孩子；危急结点是 20（因子 $+2$），头两步「左、左」→ **LL 型，右旋一次**：

```
        50
      /    \
    30      70
   /  \    /  \
  15   40 60   80
 /  \
10   20
```

插入 65 —— $65 > 50 \to 65 < 70 \to 65 > 60$，挂到 60 的右孩子。逐级查平衡因子：60 变成 -1、70 变成 $+1$、50 由 $+1$ 变成 0（左子树 30 高 3，右子树 70 也长到高 3），**全都在允许范围内 → 不需要旋转**：

```
        50
      /    \
    30      70
   /  \    /  \
  15   40 60   80
 /  \       \
10   20     65
```

插入 64 —— $64 < 65$，挂到 65 的左孩子。危急结点是 60（右子树高 2、左子树高 0，因子 -2），从 60 往下的头两步是「右（到 65）、左（到 64）」→ **RL 型，双旋**：先对 65 右旋把 64 提上来，再对 60 左旋：

```
        50
      /    \
    30      70
   /  \    /  \
  15   40 64   80
 /  \     /  \
10   20  60   65
```

三次插入正好覆盖三种情况：单旋、不旋、双旋。答题要点：(1) 先指出**危急结点**是谁；(2) 再按「头两步的方向」判型；(3) 旋转后核对中序序列不变（这里始终是 `10 15 20 30 40 50 60 64 65 70 80`）。

12-16. ★ 对一棵伸展树依次插入 $10, 20, 30, 40, 50, 60, 70$（每次插入后都把新结点展开到根），然后检索 20 并展开。写出两步之后的树形，并说明展开用到了哪几种旋转。

答：**插入阶段**：每个新键都比树中所有键大，插入后展开到根，于是树退化成一条**左斜链**：

```
70
└ 左 60
   └ 左 50
      └ 左 40
         └ 左 30
            └ 左 20
               └ 左 10
```

检索 20 并展开：从根 70 一路向左走到 20（走了 5 步），然后自下而上展开。20 相对它的父 30、祖父 40 是「左–左」，属于**一字形（zig-zig）**：先转祖父、再转父；一路做下去，最后 20 离根只剩一层时再做一次**单旋转**。20 的深度是 5，共三步：

1. 一字形（父 30、祖父 40）：得到子树 `20(10, 30(-, 40))`，挂在 50 的左边；
2. 一字形（父 50、祖父 60）：先对 60 右旋、再对 50 右旋，得到 `20(10, 50(30(-, 40), 60))`，挂在 70 的左边；
3. 单旋转（父 70 是根）：20 成为根，70 成为它的右孩子，原来 20 的右子树（以 50 为根）改挂到 70 的左边。

结果：

```
      20
    /    \
  10      70
         /
       50
      /  \
    30    60
      \
       40
```

核对中序：`10 20 30 40 50 60 70` ✓。

看点在于树高：原来是一条长 7 的链（访问 10 要走 6 步），展开之后高度降到 4，10 的深度从 6 降到 1，访问路径上的 30、40、50、60 也都上移了。**一字形旋转的作用正是把同向长链压短**—— 这就是伸展树「不维持平衡、却能摊还 $O(\log n)$」的机制：每次访问深结点的同时，顺手把那条深路径上的所有结点都提了上来，代价被后续的访问摊薄了。

注意：如果把一字形替换成「连续两次父子单旋」，20 同样能到根，**但那条链只会被平移、不会被压短**，摊还界就失效了。这是一字形不能简化的原因。

12-17. ★ 基于伸展树实现一个简单的文本编辑器。文本是一个字符串，每个结点存一个字符及它在文本中的下标 `index`，另有左右孩子与父指针。要求支持：`copy(i,j)` 把下标 $[i, j]$ 的字符复制到剪贴板；`cut(i,j)` 把这段删除并移到剪贴板；`paste(i)` 把剪贴板内容插入到下标 i 之后。写出三个操作的基本思想。

答：统一的思路只有一句：用两次 splay 把要操作的那一段"孤立"成一棵子树。

用到的 ADT：

```
SplayTree
    Splay(x)            功能：把结点 x 旋转到根（单旋/一字形/之字形）
    Splay(x, r)         功能：把 x 旋转到以 r 为根的子树的根
    Find(i)             入口：下标 i；出口：该位置的结点
    Shift(T, d)         功能：把子树 T 中所有结点的 index 加上 d
```

孤立区间的通用两步：1. 把第 $i-1$ 个位置的结点 splay 到**根**；2. 把第 $j+1$ 个位置的结点 splay 到**根的右子树的根**。

这两步之后，**根的右孩子的左子树恰好就是 $[i, j]$ 这一整段**—— 因为它比第 $i-1$ 个大、比第 $j+1$ 个小，而伸展树按 `index` 保持中序有序。（$i = 0$ 或 j 是末尾时，在文本两端各加一个哨兵结点，就不必写特例。）

```
copy(i, j)
    用上面两步孤立出子树 S = 根-> 右-> 左
    T2 ← 复制 S 的一份副本              // 剪贴板
    Shift(T2, -i)                       // 下标平移成 [0, j-i]
    原树不动

cut(i, j)
    用上面两步孤立出子树 S
    T2 ← S;  根-> 右-> 左 ← 空           // 直接摘走，不必复制
    Shift(根-> 右 及其整棵子树, -(j-i+1))   // 后面的字符整体前移
    Shift(T2, -i)

paste(i)
    把第 i 个位置的结点 splay 到根，把第 i+1 个位置的结点 splay 到根的右子树的根
    T3 ← 复制剪贴板 T2（设有 k 个字符）
    Shift(T3, i + 1)                   // 对齐到插入位置
    Shift(根-> 右 及其整棵子树, +k)      // 插入点之后的字符整体后移
    根-> 右-> 左 ← T3                    // 挂进去
```

代价：两次 splay 摊还 $O(\log n)$；`Shift` 若逐结点改 `index` 是 $O(\text{段长})$，工程上会在结点里存一个**懒标记**（lazy tag），把整棵子树的平移记成 $O(1)$，用到时再下推。复制子树是 $O(k)$，k 为剪贴板长度——这一项无法避免。

评分要点：(1) 说清伸展树的旋转/splay 操作；(2) 说清「两次 splay 孤立区间」这一步；(3) 三个操作各自对 `index` 的维护不能漏——**删除后要把后面的下标整体前移，插入后要整体后移**，这是最容易丢分的地方。

五、证明题

12-18. 证明：n 阶下三角矩阵按行优先压缩后，$a_{i,j}$（$i \geq j$，下标从 0 起）的位置是 $\frac{i(i+1)}{2} + j$，并说明为什么这个映射是**双射**。

证明. 按行优先，$a_{i,j}$ 之前存放的元素恰好是：第 0 行的 1 个、第 1 行的 2 个、……、第 $i-1$ 行的 i 个，再加上第 i 行中位于它左边的 j 个。故位置为

$$\sum_{r=0}^{i-1}(r+1) + j = \frac{i(i+1)}{2} + j$$

双射性：定义域是 $\{(i,j) : 0 \leq j \leq i \leq n-1\}$，共 $\frac{n(n+1)}{2}$ 个点；值域是 $\{0, 1, \cdots, \frac{n(n+1)}{2}-1\}$，元素个数相同。又因为对固定的 i，j 取遍 $0..i$ 时位置取遍 $[\frac{i(i+1)}{2},\ \frac{i(i+1)}{2} + i]$ 这一整段连续区间，而相邻两行的区间首尾相接、互不重叠（第 i 行末是 $\frac{i(i+1)}{2} + i = \frac{(i+1)(i+2)}{2} - 1$，恰好是第 $i+1$ 行起点的前一个），所以映射是单射；单射加上两端个数相同，即为双射。∎

双射意味着不浪费也不冲突——每个存储单元恰好放一个非零元，压缩率达到理论最优。

12-19. ★ 证明：AVL 树中插入一个结点后，**至多做一次旋转**（单旋或双旋）就能恢复平衡。

证明. 设危急结点为 x（最靠近插入点的失衡祖先），不妨设插入发生在 x 的左子树，使 x 的平衡因子变成 $+2$。记插入前 x 的左子树高为 h、右子树高为 $h-1$，插入后左子树高变成 $h+1$。

关键在于计算**旋转之后这棵子树的高度**：（插入前左孩子 y 是平衡的，它的两棵子树高都是 $h-1$；插入使其中一棵长到 h。）- **LL 型**（插入落在 x 的左孩子的左子树）：一次右旋后，新的根是原左孩子 y，它的左子树是长高后的那棵（高 h），右子树是 x——x 现在挂着 y 原来的右子树（高 $h-1$）和自己的右子树（高 $h-1$），高为 h。两边都是 h，整棵子树高 $h+1$；- **LR 型**（插入落在左孩子的右子树）：双旋后新根是原左孩子的右孩子 z，y 与 x 分别成为它的左右孩子，各自分到 z 的一棵子树（高 $h-1$ 或 $h-2$）和一棵高 $h-1$ 的子树，高度都是 h，整棵子树高同样是 $h+1$。

而**插入之前**，这棵子树的高度正是 $h+1$（左子树高 h、右子树高 $h-1$，

再加根本身一层）。也就是说，**旋转之后子树的高度与插入之前完全相同**。于是 x 的所有祖先看到的子树高度没有任何变化，它们的平衡因子自然保持不变，不可能再失衡。因此只需在 x 处旋转一次。∎

对照删除：删除后旋转会让子树高度**减少 1**，这个变化会沿着路径向上传播，因此**删除可能需要 $O(\log n)$ 次旋转**——这正是 AVL 删除比插入麻烦的原因，也是本章大纲只要求掌握插入的原因。

12-20. 证明：在最佳二叉搜索树问题中，**贪心地把概率最大的关键码放在根**不一定最优；并说明为什么必须用动态规划。

证明（举反例）. 设三个关键码 $k_1 < k_2 < k_3$，检索概率分别为 $p_1 = 0.4$、$p_2 = 0.45$、$p_3 = 0.15$。代价记为 $\sum p_i \cdot (\text{层数} + 1)$。

- **贪心**：把概率最大的 k_2 放根，k_1、k_3 各为一个孩子：$0.45 \times 1 + 0.4 \times 2 + 0.15 \times 2 = 0.45 + 0.8 + 0.3 = 1.55$。
- **另一种**：把 k_1 放根，右子树是 k_2（k_3 是 k_2 的右孩子）：$0.4 \times 1 + 0.45 \times 2 + 0.15 \times 3 = 0.4 + 0.9 + 0.45 = 1.75$，更差。

本例贪心恰好最优（三个关键码的 5 种 BST 逐一算过，1.55 最小），所以它**不是**反例。把概率改成 $p_1 = 0.4$、$p_2 = 0.3$、$p_3 = 0.3$：- 贪心（k_1 放根，因为 p_1 最大）：k_1 的右子树只能装 k_2, k_3，无论 k_2 在上还是 k_3 在上，代价都是 $0.4 \times 1 + 0.3 \times 2 + 0.3 \times 3 = 0.4 + 0.6 + 0.9 = 1.9$。- 改把概率**最小**之一的 k_2 放根，k_1、k_3 各为一个孩子：$0.3 \times 1 + 0.4 \times 2 + 0.3 \times 2 = 0.3 + 0.8 + 0.6 = 1.7$，**比贪心好**。- 再看 k_3 放根：最好也是 $0.3 \times 1 + 0.4 \times 2 + 0.3 \times 3 = 2.0$，最差。

5 种形状逐一算过（脚本复核），最优就是 k_2 为根的 1.7，贪心的 1.9 不是最优，反例成立。可见**根的选择不能只看自身概率，还要看它把剩下的关键码切成怎样的两段**——一个概率略小但能把两侧切得更均衡的关键码，可能整体更优。∎

（原先常举的 $p_1 = 0.5$、$p_2 = 0.1$、$p_3 = 0.4$ 并不是反例：贪心把 k_1 放根、k_3 作右孩子、k_2 作 k_3 的左孩子，代价 $0.5 + 0.8 + 0.3 = 1.6$，已是 5 种形状中的最小值；k_3 放根最好 1.7，k_2 放根 1.9，都更差。）

为什么必须动态规划：BST 要保持中序有序，所以根一旦选定，左子树只能装比它小的那一段、右子树只能装比它大的那一段，**子问题恰好是"连续区间"**。于是令 $C[i][j]$ 为关键码 $k_i..k_j$ 构成的最佳 BST 的代价，枚举根 $r \in [i, j]$：

$$C[i][j] = \min_{i \le r \le j} \big(C[i][r-1] + C[r+1][j] \big) + \sum_{t=i}^{j} p_t$$

最后那一项是「整棵子树下沉一层」带来的额外代价。时间 $O(n^3)$，空间 $O(n^2)$。**与 Huffman 的分界**：Huffman 树的叶可以任意重排，所

以贪心有效；最佳 BST 的中序次序被钉死，贪心失效。

模拟卷（120 分钟，100 分）

题目与上面的题库不重复，可以当作一次完整的自测。答案在每道题下面。

一、填空与选择（每空 2 分，共 24 分）

1. 含 10 个顶点的无向图，随机加边，要**保证**它连通，至少要加 ____ 条边。
2. 对 m 个初始顺串作 k 路归并，归并趟数为 ____；$m=256$、$k=4$ 时是 ____ 趟。
3. 用置换选择排序处理 3000 条记录、工作区容纳 10 条，顺串数最少 ____ 个、最多 ____ 个。
4. 在含 7 个元素的有序表上二分检索，成功时的平均检索长度是 ____，失败时一律比较 ____ 次。
5. 装填因子 $\alpha=0.5$ 时，线性探测**失败**检索的平均探查次数约为 ____。
6. 一棵 3 阶 B 树（2-3 树）中，除根外每个结点的关键码个数范围是 ____。
7. n 阶**上三角**矩阵按行优先压缩存入下标从 0 开始的数组，$a_{i,j}$（$i\le j$）的下标是 ____。
8. （选择）下列排序算法中，**最坏**时间代价为 $O(n\log n)$ 且**稳定**的是（　）。A. 快速排序　B. 堆排序　C. 归并排序　D. Shell 排序

答：1. 37（$\frac{9\times 8}{2}+1=36+1$）2. $\lceil \log_k m\rceil$；4 趟（$256\to 64\to 16\to 4\to 1$）3. 最少 1 个，最多 300 个（$\lceil 3000/10\rceil$）4. $\frac{17}{7}$（$\frac{1\times1+2\times2+3\times4}{7}$）；失败一律 3 次（$n=7=2^3-1$，判定树是满二叉树，8 个外部结点都在第 3 层）5. $\frac12\big(1+\frac{1}{(1-0.5)^2}\big)=\frac12(1+4)=2.5$ 6. **1 到 2 个**（子结点数 $\lceil 3/2\rceil=2$ 到 3）7. $\frac{i(2n-i+1)}{2}+(j-i)$ ——行优先存上三角时，第 k 行（从 0 起）有 $n-k$ 个元素，前 i 行共有 $\sum_{k=0}^{i-1}(n-k)=in-\frac{i(i-1)}{2}=\frac{i(2n-i+1)}{2}$ 个元素（验算：$i=1$ 时应为第 0 行的 n 个，$\frac{1\cdot 2n}{2}=n$ ✓）；第 i 行中 $a_{i,j}$ 前面还有 $j-i$ 个元素（从 $a_{i,i}$ 到 $a_{i,j-1}$）。两项相加即得。8. C。快速排序最坏 $O(n^2)$ 且不稳定；堆排序最坏 $O(n\log n)$ 但**不稳定**；Shell 既不稳定、最坏也不是 $O(n\log n)$。只有归并两条都满足——代价是 $O(n)$ 的辅助空间。

二、辨析与简答（每题 6 分，共 24 分）

1. 辨析：「对有向无环图作深度优先搜索，访问顶点的先后次序就是一个拓扑序列。」
2. 简答：为什么 Dijkstra 算法不能用于含负权边的图？举一个最小的反例。
3. 简答：闭散列表为什么必须用「墓碑」而不能把删除位直接置空？墓碑积累

多了怎么办?

4. 辨析:「B+ 树的检索效率一定比 B 树高，因为它的树更矮。」

   答：1. **错**。DFS 的**发现次序**不是拓扑序，因为 DFS 可能先一头扎进某条深路径，而那条路径上的顶点其实有别的前驱还没访问。正确做法是取 DFS 的**完成时间逆序**（即后序遍历的逆序）。反例：顶点 $\{1,2,3\}$，弧为 $1\to 2$、$3\to 2$，从 1 开始 DFS 得到发现序 `1 2 3`，而 3 必须排在 2 前面。

   2. Dijkstra 的正确性依赖一条不变式：**每次取出的"当前最小距离"顶点，其距离已经是最终值**。这条不变式的证明用到「路径越长权和越大」——负权边一出现就不成立。

   最小反例：三个顶点 A,B,C，边 $A\to B$ 权 2、$A\to C$ 权 5、$C\to B$ 权 -4。Dijkstra 从 A 出发先确定 B（距离 2，因为 2 < 5），此后 B 不再更新；但真正的最短路是 $A\to C\to B$，长度 $5-4=1$。**答案错了**。含负权边要用 Bellman-Ford；负权**回路**则根本没有最短路。

   3. 闭散列的检索用「遇到空位就判定失败」作为终止条件，这条判据成立的前提是**探查链上没有空洞**。若删除时直接置空，就在链中间挖了个洞，排在洞后面的同义词**从此再也找不到**（证明见 10-19 的反例）。墓碑保住链的连续性，同时允许该槽被重新占用。

   代价与对策：墓碑既不算空也不算满，检索时不能提前停，删除多了之后**失败检索会越来越慢**。工程做法是统计墓碑比例，超过阈值就**整表重建**（rehash）：新建一张表，把所有 `OCCUPIED` 的记录重新插入，墓碑随之全部消失。

   4. **不准确**。前半句结论常常成立，但理由不完整，而且「一定」过头了。- B+ 树的内部结点**不存数据**，同样大小的页能放下更多分支，所以树通常更矮、访外更少——这一点对。- 但 B **树可能在内部结点就命中**，运气好时检索长度**短于** B+ 树（B+ 树每次都必须走到叶）。所以就**单次成功检索**而言，B 树未必更慢。- B+ 树真正稳赢的地方是**检索长度稳定**（都等于树高）和**支持范围查询**（叶链），这才是数据库选它的原因。

三、算法填空（每空 2 分，共 12 分）

下面是 Dijkstra 算法（邻接矩阵 `W`，`W[i][j]` 为边权、无边时为 `INF`），求源点 `s` 到各顶点的最短距离 `dist`。补全六个空。

```
Dijkstra(W, n, s)
    for v ← 0 to n - 1
        dist[v] ← W[s][v];  known[v] ← false;  path[v] ← s
    ____(1)____
    known[s] ← true
    for step ← 1 to n - 1
```

```
    u ← -1;  best ← INF
    for v ← 0 to n - 1                      // 挑当前最近的未确定顶点
        if ____(2)____ and dist[v] < best
            best ← dist[v];  u ← v
    if u = -1 then break                    // 剩下的都不可达
    ____(3)____
    for v ← 0 to n - 1                      // 松弛
        if not known[v] and W[u][v] < INF and ____(4)____
            ____(5)____
            ____(6)____
```

答：(1) `dist[s] ← 0` (2) `not known[v]` (3) `known[u] ← true` (4) `dist[u] + W[u][v] < dist[v]` (5) `dist[v] ← dist[u] + W[u][v]` (6) `path[v] ← u`

三**处最容易错**：1. 第 (2) 空漏掉 `not known[v]`，已确定的顶点会被反复选中，算法不收敛；2. 第 (4) 空写成 `W[u][v] < dist[v]`——**必须加上** `dist[u]`，比较的是「经过 u 的整条路径」与「已知路径」，不是单条边；3. `path[v] ← u` 那一句漏掉的话，距离算对了却输出不了路径。

时间代价 $O(n^2)$（外层 n 次，每次挑最小 $O(n)$、松弛 $O(n)$）。用最小堆把「挑最小」降到 $O(\log n)$，总代价变成 $O((n+e)\log n)$，稀疏图上更快。

四、算法设计与分析（每题 10 分，共 20 分）

1. 给定一个有向图的邻接表和一个顶点 s，设计算法判断从 s 出发是否能到达图中**所有**顶点；若不能，输出所有不可达的顶点。分析代价。

答：**基本思想**——从 s 出发做一次遍历（DFS 或 BFS 都行），标记所有能到达的顶点；遍历结束后，**没被标记的就是不可达的**。

用到的 ADT：`Graph.Neighbors(v)` 取 v 的后继表；`Queue` 的 `EnQueue/DeQueue/Empty`，均 $O(1)$。

```
Reachability(G, s)
    visited[0..n-1] ← false
    Q ← 空队列;  Q.EnQueue(s);  visited[s] ← true;  cnt ← 1
    while not Q.Empty()
        v ← Q.DeQueue()
        for w in G.Neighbors(v)
            if not visited[w]
                visited[w] ← true;  cnt ← cnt + 1;  Q.EnQueue(w)
    if cnt = n then 输出「全部可达」
    else
        for v ← 0 to n - 1
            if not visited[v] then 输出 v
```

代价：时间 $O(n+e)$——每个顶点入队至多一次、每条弧考察至多一次；空间 $O(n)$（`visited` 数组加队列）。

两个容易丢分的地方：1. **一次遍历就够**，不要对每个顶点都跑一次可达性判断（那是 $O(n(n+e))$）；2.「s 能到达所有顶点」与「图是强连通的」**不是一回事**—— 前者只要求单向可达。要判强连通，需要再在**反向图**上从 s 跑一次，两次都全可达才是强连通（这就是 Kosaraju 算法的核心思想）。

2. 有一个很大的文件，含 N 条记录（N 远大于内存容量 M），要求找出其中**最大的 K 条**记录（$K \ll M$）。设计算法并分析访外次数。

答：**基本思想**——用一个**容量为 K 的最小堆**扫一遍文件。堆顶是当前 K 大里最小的那个；新读入的记录若大于堆顶就替换堆顶并下筛，否则直接丢弃。

用到的 ADT：

```
MinHeap（容量 K）
    Insert(x)      功能：插入，O(log K)
    Top()          出口：最小元
    ReplaceTop(x)  功能：用 x 换掉堆顶并下筛，O(log K)
    Size()         出口：当前元素个数
```

```
TopK(file, N, K)
    H ← 空的最小堆（容量 K）
    按页顺序读文件：
        for 每条记录 r
            if H.Size() < K then H.Insert(r)
            else if r > H.Top() then H.ReplaceTop(r)
    输出 H 中的全部 K 条记录（即最大的 K 条）
```

代价：- **访外：整个文件只顺序读一遍**，即 $\frac{N}{b}$ 页次（b 为每页记录数），这是任何算法都省不掉的下界——**必须看过每一条记录才能确定它是不是前 K 大**。- **内存**：只占 K 条记录的空间，与 N 无关。- **内部比较**：最坏 $O(N \log K)$；实际上大多数记录一次比较（与堆顶比）就被丢弃。

为什么不先排序：对整个文件外排序要 $2\frac{N}{b}(1+\lceil \log_k m \rceil)$ 页次，是本方案的好几倍，而且做了大量无用功——**我们只要前 K 个，不需要其余部分有序**。

为什么用最小堆而不是最大堆：要被淘汰的是「当前 K 个里最小的那个」，最小堆的堆顶恰好是它，$O(1)$ 就能拿到。用最大堆的话每次都要 $O(K)$ 去找最小值。**这是本题最关键的一个判断，写反了扣分最多**。

五、证明题（20 分，每题 10 分）

1. 证明：若无向连通图 G 的所有边权互不相同，则它的最小生成树唯一。

证明. 反设有两棵不同的最小生成树 $T_1 \neq T_2$,权值和相等(都最小)。取**只属于其中一棵**的边里权值最小的那条，记为 e；不妨设 $e \in T_1 \setminus T_2$。把 $e = (u, v)$ 加入 T_2，会形成唯一一个环 C。环 C 上必有一条边 $f \notin T_1$（否则 C 的所有边都在 T_1 里，T_1 就含环了，与它是树矛盾)。于是 $f \in T_2 \setminus T_1$，f 也是「只属于其中一棵」的边，由 e 的取法有 $w(e) < w(f)$（边权互不相同，不会相等)。

考察 $T_2' = T_2 + e - f$：它仍是生成树（去掉环上一条边不破坏连通性)，而 $w(T_2') = w(T_2) + w(e) - w(f) < w(T_2)$，与 T_2 是最小生成树矛盾。∎

反过来不成立：边权有重复时 MST 仍可能唯一（见 7-17 的 G_2)。真正的充要条件是**严格的环性质**：取定一棵最小生成树 T，每条不在 T 上的边 (u, v) 都比 T 上 u、v 之间路径中的每一条边**严格地重**（7-8 选项 D 的解析里有说明和反例)。

2. 证明：对 n 个记录作 k 路归并外排序，若初始顺串数为 m，则总的读写页次约为 $2\frac{n}{b}\lceil \log_k m \rceil$（$b$ 为每页记录数)，并说明为什么「增大 k」和「增大内存工作区 M」都能减少它，两者的作用有何不同。

证明. 归并分若干趟。**每一趟都要把当前全部数据完整地读入一次、写出一次**：因为 k 路归并是把 k 条顺串合成一条，一趟下来所有记录恰好各被读一次、写一次。一趟的页次为 $2\frac{n}{b}$。

每一趟把顺串数从 x 变成 $\lceil x/k \rceil$，从 m 降到 1 需要 $\lceil \log_k m \rceil$ 趟。两者相乘即得总页次 $2\frac{n}{b}\lceil \log_k m \rceil$。∎

两个旋钮的作用不同：- **增大** k 作用在**对数的底**上：$\lceil \log_k m \rceil = \lceil \frac{\log_2 m}{\log_2 k} \rceil$，$k$ 翻倍（$\log_2 k$ 变成 $\log_2 k + 1$)，趟数约减少 $\frac{1}{\log_2 k + 1}$ 的比例（例如 k 从 4 到 8，趟数约降为原来的 $\frac{2}{3}$)。代价是要同时开 k 个输入缓冲区（受内存限制)，以及内部选最小值变慢——后者**用选择树可以完全消掉**（见 9-14)。- **增大** M 作用在**对数的真数**上：置换选择的顺串数约为 $m = \frac{n}{2M}$，M 翻倍，m 减半，趟数**至多减少** 1。

所以在内存有限时，把内存优先分给" 多开几路缓冲区" 通常比" 加长顺串" 更划算—— 因为前者改的是底、后者改的是真数。这也是现代外排序普遍用大 k 的原因。

考前速查表

一页纸能带走的东西。**括号里是本篇的题号**，忘了推导就翻回去看。

图

事项	结论
保证连通的最少边数	$\frac{(n-1)(n-2)}{2}+1$（7-3、7-22）
相邻矩阵 / 邻接表	判边 $O(1)$ / $O(d)$；遍历邻居 $O(n)$ / $O(d)$；空间 n^2 / $n+e$（7-4、7-15）
拓扑排序	反复取入度 0；输出数 $<n$ 即有环；$O(n+e)$（7-13）
拓扑序唯一	当且仅当图中有哈密顿路径（7-6、7-16）
DFS 序 ≠ 拓扑序	要取**完成时间逆序**（7-8C、模拟卷二·1）
Dijkstra	不许负权边；加入次序 = 最短距离升序（7-7、7-11）
Floyd	允许负权边、不许负权回路；$\Theta(n^3)$（7-7）
MST 切性质	切上最小边唯一 ⇒ 它属于每棵 MST（7-20）
边权互异 ⇒ MST 唯一	充分**不必要**（7-17 是反例、模拟卷五·1 是证明）
无向图 DFS	只有树边和回边，**没有横叉边**（7-21）
生成树林	非连通图周游得若干棵树，**棵数 = 连通分量数**；共 $n-c$ 条边（7-2）

内排序

事项	结论
不稳定的四个	选择、Shell、快速、堆（「跨距离搬元素」的都不稳）（8-1）
每趟定位一个元素	冒泡、快速（8-8 的选项），以及直接选择、堆排序；插入、归并、Shell 没有
基本有序时最快	直接插入（8-9）；快排取首元素为轴反而最坏（8-3）
最坏也 $O(n\log n)$	堆排序、归并排序（8-4）
空间	归并 $O(n)$、堆 $O(1)$、快排 $O(\log n)$ 栈（8-5）
基数排序	d 趟、$O(d(n+r))$；**不基于比较**，不受下界约束（8-6、8-20）
相邻交换类下界	$\Theta(n^2)$，因为一次交换至多消一个逆序对（8-19）

事项	结论
二分法插入排序	比较降到 $\Theta(n\log n)$，**移动仍** $\Theta(n^2)$，总时间没改善（8-14）
比较排序下界	$\Omega(n\log n)$，判定树至少 $n!$ 个叶（8-20）
逆序对期望	$\frac{n(n-1)}{4}$，即插入排序平均移动 $\frac{n^2}{4}$ 次（8-21）
快排 + 插入混合	$O(n\log\frac{n}{k}+nk)$；k 为常数时仍 $O(n\log n)$，收益在常数（8-17）

外排序

事项	结论
代价单位	**访外页次**，不是比较次数（9-1）
置换选择顺串数	最少 1（升序输入）、最多 $\lceil N/M\rceil$（降序输入）、平均 $\frac{N}{2M}$（9-2、9-3）
归并趟数	$\lceil\log_k m\rceil$（9-4、9-6）
败者树	内部结点存**败者**，另设冠军槽；重构沿叶到根，$O(\log_2 k)$（9-5、9-9）
选择树的意义	让内部比较次数**与** k **无关**（9-14）
最佳归并树	k 叉 Huffman；$(m-1)\bmod(k-1)\neq 0$ 时**补虚顺串到最底层**（9-10、9-11）
总页次	$2\frac{n}{b}(1+\lceil\log_k\frac{n}{2M}\rceil)$；$k$ 改**底**、M 改**真数**（9-12、模拟卷五·2）

检索

事项	结论
二分最大检索长度	$\lceil\log_2(n+1)\rceil$（10-18）
二分 ASL	按判定树逐层数；$n=2^k-1$ 时失败次数才唯一（10-2）
分块检索	ASL $\approx\frac{b+1}{2}+\frac{n/b+1}{2}$，$b=\sqrt{n}$ 时最小 $\approx\sqrt{n}+1$（10-6）
除余法	p 取**接近表长的素数**，别取 2 的幂（10-8）

事项	结论
平方取中法	取 k^2 的中间几位——中间位受 k **所有位**影响（10-14）
折叠法	等长分段相加取低位；移位折叠 / 边界折叠（关键码远长于地址时用）（10-14）
线性探测	成功 $\frac{1}{2}(1+\frac{1}{1-\alpha})$、失败 $\frac{1}{2}(1+\frac{1}{(1-\alpha)^2})$（10-4）
拉链法	成功 $1+\frac{\alpha}{2}$、失败 α；α 可以大于 1（10-4、10-11）
墓碑	检索**跳过**、插入**可占但要走完链**、删除**只打标记**；多了要 rehash（10-5、10-16）
为什么不能置空	会截断同义词的探查链（10-19）
失败 ASL 的分母	是**表长**，不是记录数（10-15）

索引

事项	结论
m 阶 B 树	至多 m 子结点、$m-1$ 关键码；非根至少 $\lceil m/2 \rceil$ 子结点；**叶同层**（11-4、11-9）
B 树插入	上溢 → 中间关键码**移动**上升、分裂；**从叶向根长高**（11-10、11-15）
B+ 树插入	叶分裂时右叶首键**复制**上升（关键码在索引层重复出现）（11-16）
读盘 / 写盘	检索读 $\le h$、**写** 0；插入读 h、写 $2s+1$（s 为分裂层数）；删除借则写 3、合并约写 $2t$（11-20）
B 树删除	下溢 → 先向兄弟借、借不到就合并；可能一路传到根
B+ 树	记录全在叶、叶间有链 → **支持顺序与范围查询**；检索长度固定（11-5、11-8）
B 树高度	$h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$；$m$ 在**对数的底**上（11-21）
为什么外存不用 AVL	二叉树高 $\log_2 n$，访外太多；B 树把结点做成一整页（11-12）

事项	结论
红黑树四性质	根黑、不许红红相邻、黑高相等 → 树高 $\le 2\log_2(n+1)$（11-7、11-22）
红黑树插入	**叔父红** → **只换色、冲突上移**；**叔父黑** → **换色** + **旋转**（内侧先转成外侧）；旋转 ≤ 2 次（11-11、11-17）
红黑树删除	删红 → 不用管；删黑 →「双黑」，看**兄弟与两个侄子**分四情形；只有情形 2 向上传播；旋转 ≤ 3 次（11-18）

高级数据结构

事项	结论
下三角压缩	$\frac{i(i+1)}{2}+j$（下标从 0）；共 $\frac{n(n+1)}{2}$ 个单元（12-1、12-18）
多维数组偏移	从右往左累乘后续各维大小；霍纳法更省事（12-2）
广义表	长度 = 最外层元素个数；深度 = 括号嵌套层数；**表尾一定是表**（12-3、12-7）
稀疏矩阵	$\delta < \frac{1}{3}$ 三元组才省空间，实践取 $\delta < 0.05$（12-4、12-12）
AVL 四种旋转	看危急结点往下的**头两步方向**；LR、RL 要双旋（12-5、12-8、12-15）
AVL 插入	**至多一次旋转**，因为旋转后子树高度回到插入前（12-19）
AVL 删除	可能 $O(\log n)$ 次旋转（高度会向上传播）（12-19）
伸展树	单旋 / 一字形 / 之字形；访问后**展开到根**；**摊还** $O(\log n)$，单次可达 $O(n)$（12-6、12-9、12-16）
半伸展树	只改一字形：当前结点**上移到父**继续，**不必送到根**；旋转约减半（12-14）
伸展树区间操作	**两次** splay **把区间孤立成一棵子树**，再整体搬（12-17）
最佳 BST	最小化带权平均检索长度；中序被钉死 $\Longrightarrow$ 贪心失效、必须 DP，$O(n^3)$（12-10、12-20）

答题格式提醒

1. **先声明约定**：下标从 0 还是 1、B 树的阶怎么定义、矩阵按行还是按列优先、排序是升序还是降序。这几处不声明，改卷时很吃亏。
2. **设计题先写 ADT**：用到队列、堆、并查集、伸展树时，先列函数名、入口、出口、功能一句话。
3. **算法之后一定跟三件事**：基本思想（一两句）、正确性依据、时空代价；外排序题还要单列**访外次数**。
4. **画图题把中间步骤留在卷面上**：B 树插入、红黑树调整、AVL 旋转、败者树重构、最佳归并树这五类题，过程分往往比结果分多。
5. **复杂度要说清是哪种情况**：「最坏 $O(n^2)$、平均 $O(n \log n)$」比光写一个 $O(\cdot)$ 稳。

《数据结构与算法》MOOC 答案 & 解析

Updated 2026-06-14 09:28 GMT+8
Compiled by Hongfei Yan (2026 Spring)

MOOC 答案下载自 pkuhub.cn，重新编辑。

第一章 概论

1. 逻辑结构与存储结构

单选题：以下哪种结构是逻辑结构，而与存储和运算无关：

A、队列 (queue) B、双链表 (doubly linked list) C、数组 (array) D、顺序表 (Sequential list)

答案： 【队列 (queue)】，其他是存储结构。

2. 线性结构

单选题：下列不属于线性结构的是：

A、队列 (queue) B、散列表 (hash table) C、向量 (vector) D、图 (graph)

答案： 【图 (graph)】

3. 算法特性

多选题：关于算法特性描述正确的有：

A、算法保证计算结果的正确性。B、组成算法的指令可以有限也可能无限。C、算法描述中下一步执行的步骤不确定。D、算法的有穷性指算法必须在有限步骤内结束。

答案： A、D

4. 时间复杂度

多选题：已知一个数组 a 的长度为 n，求问下面这段代码的时间复杂度:

```
for (i=0, length=1; i < n; i++)
    for (j = i+1; j < n; j++)
        if (length < j - i + 1)
            length = j - i + 1;
```

A、$\Omega(n)$ B、$O(n^2)$ C、$\Theta(n^2)$ D、$O(n)$

答案： $\Omega(n); O(n^2)$

代码分析：

```
for (i=0, length=1; i < n; i++)
 for (j = i+1; j < n; j++)
```

```
    // 内部常数操作
```

外层循环执行 n 次。

当外层为 i 时，内层循环执行 $n-1-i$ 次。

总执行次数为 $\sum_{i=0}^{n-1}(n-1-i)=\frac{n(n-1)}{2}$，其渐近复杂度为 $\Theta(n^2)$。

根据渐近记号定义：若复杂度为 $\Theta(n^2)$，则它必然满足上限 $O(n^2)$ 和下限 $\Omega(n^2)$。同时，由于 $\Omega(n)$ 是 $\Omega(n^2)$ 的子集，它也满足 $\Omega(n)$ 的下限定义。

结论： 选 $\Omega(n)$ 与 $O(n^2)$（在单选题中通常最精确答案为 $\Theta(n^2)$，多选题中 A、B 正确）。

5. 大 O 表示法性质

多选题：下列说法正确的是：

A、如果函数 f(n) 是 O(g(n))，g(n) 是 O(h(n))，那么 f(n) 是 O(h(n))。B、如果函数 f(n) 是 O(g(n))，g(n) 是 O(h(n))，那么 f(n)+g(n) 是 O(h(n))。C、如果 a>b>1，$\log_a n$ 是 $O(\log_b n)$，但 $\log_b n$ 不一定是 $O(\log_a n)$。D、函数 f(n) 是 O(g(n))，当常数 a 足够大时，一定有函数 g(n) 是 O(af(n))。

答案： A、B

解析：

A **正确：** 传递性。若 $f(n)\le c_1 g(n)$ 且 $g(n)\le c_2 h(n)$，则 $f(n)\le c_1c_2h(n)$。

B **正确：** 若 $f(n)=O(g(n))$ 且 $g(n)=O(h(n))$，则两者之和的上限依然由较大的 $h(n)$ 决定，即 $f(n)+g(n)=O(h(n))$。

C **错误：** 对数换底公式：$\log_a n=\frac{\log_b n}{\log_b a}$。由于 $\frac{1}{\log_b a}$ 是常数，因此对于任意 $a,b>1$，$\log_a n$ 与 $\log_b n$ 是等阶的（即 $\Theta(\log_b n)$），二者互为大 O。

D **错误：** $f(n)=O(g(n))$ 说明 $g(n)$ 的增长速度不慢于 $f(n)$，但可能远快于 $f(n)$（例如 $f(n)=n, g(n)=n^2$）。此时不存在常数 a 使得 $g(n)=O(af(n))$。

6. 计算运行下列程序段后 m 的值：

```
n = 9; m = 0;
for (i=1; i<=n; i++)
    for (j = 2*i; j<=n; j++)
        m=m+1;
```

答案： 【20】

计算过程：

$i=1$：j 从 2 到 9，执行 8 次。
$i=2$：j 从 4 到 9，执行 6 次。
$i=3$：j 从 6 到 9，执行 4 次。
$i=4$：j 从 8 到 9，执行 2 次。
$i\geq 5$：$2i\geq 10>n$，内层循环不执行。
总和 $m=8+6+4+2=20$。

7. **排序题：**由大到小写出以下时间复杂度的序列（答案直接写标号，如：(1)(2)(3)(4)(5)，不要出现空格）：
(1) 2^n (2) $n^{2.5}$ (3) $n(\log_5 n)^4$ (4) $5n^2$ (5) 2^{2^n}

答案：【(5)(1)(2)(4)(3)】

解析：常见时间复杂度从大到小排列：双重指数 > 指数 > 多项式 > 对数多项式。
(5) 2^{2^n}：双重指数（最大）
(1) 2^n：单指数
(2) $n^{2.5}$：高次方多项式
(4) $5n^2$：二次方多项式
(3) $n(\log_5 n)^4$：拟线性级（最小）

第二章 线性表

1. **多选题：**完成在双循环链表结点 p 之后插入 s 的操作为：

A、`p->next->prev=s; s->prev=p; s->next=p->next; p->next=s;` B、`p->next->prev=s; p->next=s; s->prev=p; s->next=p->next;` C、`s->prev=p; s->next=p->next; p->next=s; p->next->prev=s;` D、`s->next=p->next; p->next->prev=s; s->prev=p; p->next=s;`

答案：A、D

解析：在结点 p 之后插入 s，必须保证在修改指针时，未修改的指针能够用来找到原本的后继结点 $p\rightarrow$ next。

A 正确：

1. `p->next->prev=s;`（将原本后继的 $prev$ 指向 s）
2. `s->prev=p;`
3. `s->next=p->next;`
4. `p->next=s;`（最后切断 p 与原后继的连接）

D 正确：

1. `s->next=p->next;`
2. `p->next->prev=s;`
3. `s->prev=p;`
4. `p->next=s;`

2. **线性表存储性质**

多选题：下面的叙述中正确的是：

A、线性表在链式存储时，查找第 i 个元素的时间与 i 的数值无关。B、线性表在顺序存储时，查找第 i 个元素的时间与 i 的数值成正比。C、线性表在顺序存储时，查找第 i 个元素的时间与 i 的数值无关。D、线性表在链式存储时，插入第 i 个元素的时间与 i 的数值成正比。

答案： C、D

解析：

顺序存储支持随机存取，查找第 i 个元素的时间复杂度为 $O(1)$，与 i 的数值无关（C 正确，B 错误）。

链式存储不支持随机存取，查找第 i 个元素需要从头指针开始遍历 i 次，时间复杂度为 $O(i)$，与 i 成正比（A 错误）。

链式存储在插入第 i 个元素时，需要先定位到第 $i-1$ 个结点，其定位时间与 i 成正比（D 正确）。

3. **多选题：** 下面关于线性表的叙述中，正确的是哪些?

A、线性表采用顺序存储，必须占用一片连续的存储单元。B、线性表采用顺序存储，便于进行插入和删除操作。C、线性表采用链接存储，不必占用一片连续的存储单元。D、线性表采用链接存储，便于插入和删除操作。

答案： A、C、D

解析：

顺序存储要求物理内存连续（A 正确），由于插入删除需要移动大量元素，因此不便于此类操作（B 错误）。

链式存储依靠指针相连，不要求物理连续（C 正确），插入和删除仅需修改指针，操作非常便利（D 正确）。

4. 设某循环链表长度为 n，并设其中一节点为 p1，然后按照链表的顺序将后面的节点依次命名为 p2, p3, ..., pn，那么请问 `pn.next = ____` (答案为一个节点名，所有字母为小写且不包含空格)

答案： 【p1】

解析： 循环链表首尾相接，最后一个结点 p_n 的 $next$ 指针重新指向第一个结点 p_1。

5. 对于一个具有 n 个结点的单链表，在已知的结点 *p 后插入一个新结点的时间复杂度为 $O(__)$；在给定值为 x 的结点后插入一个新结点的时间复杂度为 $O(__)$。（格式为 (a)(b)，字母小写且不含空格）

答案： 【(1)(n)】

解析：

若已知要在目标结点 *p 后插入，只需执行常数次指针操作，复杂度为 $O(1)$。

在给定值 x 的结点后插入，必须先遍历链表寻找值为 x 的结点，最坏需要比对 n 次，复杂度为 $O(n)$。

第三章 栈与队列

1. 中缀表达式转后缀

单选题：现有中缀表达式 $E = ((100 - 4)/3 + 3 * (36 - 7)) * 2$。以下哪个是与 E 等价的后缀表达式?

A、`((100 4 - ) 3 / 3 (36 7 - ) * + ) 2 *` B、`* + / - 100 4 3 * 3 - 36 7 2` C、`100 4 - 3 / 3 36 7 - * + 2 *` D、`* ( + / ( - 100 4 ) 3 * 3 ( - 36 7 ) ) 2`

答案： C

解析： 遵循运算符优先级及括号嵌套原则。

`((100-4)/3 + 3*(36-7)) * 2`

先计算 `100 - 4` $\longrightarrow$ `100 4 -`

除以 3 $\longrightarrow$ `100 4 - 3 /`

计算 `36 - 7` $\longrightarrow$ `36 7 -`

乘以 3 $\longrightarrow$ `3 36 7 - *`

加法 $\longrightarrow$ `100 4 - 3 / 3 36 7 - * +`

最后乘以 2 $\longrightarrow$ `100 4 - 3 / 3 36 7 - * + 2 *`

2. 栈容量计算

单选题：设栈 S 和队列 Q 的初始状态为空，元素 e1，e2，e3，e4，e5 和 e6 依次通过栈 S，一个元素出栈后即进队列 Q。若 6 个元素出队的序列是 e2，e4，e3，e6，e5，e1，则栈 S 的容量至少应该是 _____。

A、2 B、3 C、4 D、6

答案： 【3】

解析： 模拟进出栈过程，记录栈中元素的最大数量：

- 输入：$e_1, e_2, e_3, e_4, e_5, e_6$，输出：$e_2, e_4, e_3, e_6, e_5, e_1$。

1. 进 e_1, e_2（栈内：$[e_1, e_2]$，容量需求 2）
2. 出 e_2（栈内：$[e_1]$）
3. 进 e_3, e_4（栈内：$[e_1, e_3, e_4]$，**容量需求** 3）
4. 出 e_4, e_3（栈内：$[e_1]$）
5. 进 e_5, e_6（栈内：$[e_1, e_5, e_6]$，**容量需求** 3）
6. 出 e_6, e_5, e_1（栈空）

- 整个过程中，栈的最大并发元素数为 3。

3. **多选题：** 以下循环队列的实现方式中，长度为 n 的队列，所能容纳的元素个数也为 n 的有：

A、只用 front 和 rear 两个指针标记队列的头和尾，两个指针均为实指。B、用 front 和 rear 两个指针标记队列的头和尾，并用整型变量 len 记录队列元素

数。C、用 front 和 rear 两个指针标记队列的头 and 尾，并用布尔型变量 empty 记录队列是否为空。D、只用 front 和 rear 两个指针标记队列的头和尾，两个指针均为虚指。

答案： B、D

解析：

A、C 选项在传统实现中，如果不加额外状态，通常需要浪费一个空闲单元以区分 “队列空” 和 “队列满”（即容量为 $n-1$）。

B **选项：** 引入变量 `len`。当 `len == n` 时即为满，可以容纳 n 个元素。

D **选项：** 采用 “虚指”（双指不断递增，使用时再取模，如 `rear - front` 代表实际长度）。这种方式不需要浪费空间，最大可容纳 n 个元素。

4. **多选题：** 队列的特点包括：

A、后进先出 Last-in first-out (LIFO) B、先进后出 First-in last-out (FILO) C、先进先出 First-in first-out (FIFO) D、后进后出 Last-in last-out (LILO)

答案： C、D

5. 双端队列可以在队列的两端进行插入和删除操作。现有 4 个不同的元素顺序输入到双端队列，那么可以得到 _____ 种不同的排列。

答案： 【8】

解析： 4 个元素顺序输入双端队列，其输出受限于插入与删除的约束。在限定条件下（单端输入或单端输出等限制性双端队列），4 个元素的合法排列数为 8（普通栈的合法出栈序列数为卡特兰数 $C_4 = 14$）。

6. **卡特兰数应用**

编号为 1，2，3，4 的四辆列车，顺序开进一个栈式结构的站台；则开出车站的顺序有 _____ 种可能。

答案： 【14】

解析： 编号 1, 2, 3, 4 的列车进出站相当于栈的混洗过程。合法出站序列数符合卡特兰数 C_4：

$$C_4 = \frac{1}{5}\binom{8}{4} = 14$$

第四章 字符串

1.KMP **算法** next **数组（特征向量）**

单选题：寻找字符串 “BAAABBBAA” 的特征向量 (feature vector)：

A、{-1, 0, 0, 0, 0, 0, 0, 1, 2} B、{-1, 0, 0, 0, 0, 1, 1, 1, 2} C、{-1, 0, 0, 0, 0, 0, 1, 1, 2} D、{-1, 0, 0, 0, 1, 1, 1, 1, 2}

答案： 【`{-1, 0, 0, 0, 0, 1, 1, 1, 2}`】

解析： 字符串为 `"BAAABBBAA"`。定义 `next[0] = -1`。

$i = 0$: `"B"` $\rightarrow -1$

$i = 1$: `"BA"` $\rightarrow 0$

$i = 2$: `"BAA"` $\rightarrow 0$

$i = 3$: `"BAAA"` $\rightarrow 0$

$i = 4$: `"BAAAB"` $\rightarrow 0$

$i = 5$: `"BAAABB"` $\rightarrow 1$ (前后缀 `"B"` 匹配)

$i = 6$: `"BAAABBB"` $\rightarrow 1$ (前后缀 `"B"` 匹配)

$i = 7$: `"BAAABBBA"` $\rightarrow 1$ (前后缀 `"B"` 匹配，由于 $P[7] = A \neq P[1]$, 无法拓展)

$i = 8$: `"BAAABBBAA"` $\rightarrow 2$ (前后缀 `"BA"` 匹配，长度为 2)

数组为 `{-1, 0, 0, 0, 0, 1, 1, 1, 2}`。

2-5. 字符串基本概念与运算

2. **单选题：** 下面关于串的叙述中，哪一个是不正确的？A、串是字符的有限序列。B、模式匹配是串的一种重要运算。C、串是一种数据对象和操作都特殊的线性表。D、空串是由空格构成的串。

答案： D

空串指的是长度为 0 的串。由空格组成的串称为 “空白串”（Blank string），其长度不为 0。故 D 错误。

3. **单选题：** 若串 $S1 =' ABCDEFG', S2 =' 9898', S3 =' \#\#\#', S4 =' 012345'$，执行：`concat(replace(S1, substr(S1, length(S2), length(S3)), S3), substr(S4, index(S2, '8'), length(S2)))`
注意：`substr(S, i, j)` 是对字符串 S 的下标为 i 开始取 j 个字符，这里的下标是从 0 开始的。

A、`ABC###G0123` B、`ABCD###2345` C、`ABCD###1234` D、`ABC###G2345`

答案： 【`ABCD###1234`】

解析：

1. `length(S2) = 4`，`length(S3) = 3`。
2. `substr(S1, 4, 3)`：从 $S1 =' ABCDEFG'$ 的下标 4（字符 ‘E’）起截取 3 个字符 $\rightarrow$ `'EFG'`。
3. `replace(S1, 'EFG', S3)` $\rightarrow$ 将 `'EFG'` 替换为 `'###'` $\rightarrow$ `'ABCD###'`。
4. `index(S2, '8')` $\rightarrow$ 在 `'9898'` 中 `'8'` 首次出现下标为 1。
5. `substr(S4, 1, 4)` $\rightarrow$ 从 `'012345'` 下标 1 开始截取 4 个字符 $\rightarrow$ `'1234'`。
6. `concat('ABCD###', '1234')` $\rightarrow$ `'ABCD###1234'`。

4. **单选题：** 下列说法正确的是：

A、空串就是空白串。B、空串是任意字符串的子串。C、串只可以采用顺序存储，不可以采用链式存储。D、在 C++ 标准中，`char S[M]` 最多能表示长度为

M 的字符串。

答案： B

5. **单选题：** 设有两个串 p 和 q，其中 q 是 p 的子串，求 q 在 p 中首次出现的位置的算法称为 ( )。A、求子串 B、联接 C、匹配 D、求串长

答案： 【匹配 】

6. **互补回文串**

多选题：在字符 {A, C, G, T} 组成的 DNA 序列中，A 和 T、C 和 G 是互补对。判断一个 DNA 序列中是否存在互补回文串（例如，`ATCATGAT` 的补串是 `TAGTACTA`，与原串形成互补回文串）。下面 DNA 序列中存在互补回文串的是：

A、`CTGATCAG` B、`AATTAATT` C、`TGCAACGT` D、`CATGGTAC` E、`GTACGTAC` F、`AGCTAGCT`

答案： 【`CTGATCAG`; `AATTAATT`; `GTACGTAC`; `AGCTAGCT`】

解析： 互补对：A-T，C-G。互补回文串意味着串的逆序等于原串的互补串。

- 例如 `CTGATCAG` 的互补串为 `GACTAGTC`，其逆序为 `CTGATCAG`（与原串相同，正确）。

7. 字符串 “BAAABBBAA” 与目标 “BAAABBBCDDDCCHHHHBBBAAABB-BAADD” 进行匹配，至少需要多少次字符匹配（提示：利用优化后的 Next 数组）。

答案： 【31】

解析： 使用优化后的 `nextval` 数组进行单向匹配。

模式串 `"BAAABBBAA"` 与文本串进行比对。

根据 `nextval = [-1, 0, 0, 0, -1, 1, 1, 0, 0]` 进行指针转移。

模拟比对过程，累计字符比较次数为 31 次。

8. 下列程序判断字符串 s 是否对称，对称则返回 1，否则返回 0。如 `f("abba")` 返回 1，`f("abab")` 返回 0。请在空白处填空（注：三个答案之间用空格分隔，答案均不加空格）：

```
int f(char s[]) {
    int i=0, j=0;
    while(s[j]) (1)__++;
    for(j--; i < j && s[i] == s[j]; i++, j--);
    return((2)__>=(3)__);
}
```

答案： 【j i j】

代码分析：

(1) 处需要通过循环让 j 增加到字符串末尾，由于判断条件是 `s[j]`，所以执行 `j++`。

外部退出 `while` 循环后，`j` 指向 `\0`。通过 `j--` 让其指向末尾有效字符。

对称判断的相向指针循环 `for(j--; i < j && s[i] == s[j]; i++, j--);`。

如果完全对称，循环将因为 `i >= j` 终止；如果不称，将因为 `s[i] != s[j]` 提前终止（此时 `i < j`）。

所以返回条件为 `return(i >= j)` 或 `return(j <= i)`。

9-11. 字符串变换与运算

9. $S =$ "$S1S2...Sn$" 是一个长为 n 的字符串，存放在一个数组中，编程将其改造：1. 将 S 的所有偶数位字符按照原来下标从大到小的顺序放在 S 的后半部分；2. 将 S 的所有奇数位字符按照原来下标从小到大的顺序放在 S 的前半部分。例如：$S =' ABCDEFGHIJKL'$，改造后为 $'ACEGIKLJHFDB'$。则 $S =' algorithm'$ 改造后为：

答案： 【agrtmhiol】

解析："algorithm" 的 1-based 下标奇数位为 'a', 'g', 'r', 't', 'm'，偶数位为 'l', 'o', 'i', 'h'。

前半部分（奇数位顺序）：agrtm。后半部分（偶数位逆序）：hiol。

10. **利用给定的字母映射表进行加密。**

若 “encrypt” 被加密为 “tkzwsdf”，则 “algorithm” 被加密为：

答案： 【neopwmfbl】

解析：

由 “encrypt” 加密为 “tkzwsdf” 得出部分字母映射：r->w, t->f 等。

根据标准单表代换密码表进行转换，“algorithm” 映射为 “neopwmfbl”。

11. 设有字符串变量 `String A = "", B = "MULE", C = "OLD", D = "MY"`。计算下列表达式（三个答案本身不加空格，答案之间用空格隔开）：(1) `D + C + B`
(2) `B.substr(3, 2)`
(3) `A.strlength()`

答案： 【MYOLDMULE E 0】

解析：

(1) `D + C + B` → "MY" + "OLD" + "MULE" = "MYOLDMULE"。

(2) `B.substr(3, 2)` → 从 "MULE" 下标 3 提取长度 2 的子串 → "E"。

(3) 空串长度为 0。

12. **填空题：** 若字符串 $s =$ "$algorithm$"，则其子串个数为：

答案： 【46】

解析： 长度为 n 且无重复字符的字符串，其非空子串个数为 $\frac{n(n+1)}{2}$。

加上空串（1 个），总子串数为 $\frac{n(n+1)}{2} + 1$。

“algorithm” 长度 9 $\rightarrow \frac{9\times10}{2}+1=46$。

13. **填空题：** 若字符串 $s=\text{"}software\text{"}$，则其子串个数为：

答案： 【37】

解析： “software” 长度 8 $\rightarrow \frac{8\times9}{2}+1=37$。

第五章 二叉树（上）

1-2. 二叉树性质

1. **多选题：** 下列关于二叉树遍历的说法正确的有：

A、前序和中序遍历的顺序恰好一样的二叉树，只能是空二叉树或者独根二叉树这两种情况。

B、所有结点左子树为空的二叉树的前序和中序遍历顺序恰好一样。

C、所有结点右子树为空的二叉树的前序和中序遍历顺序恰好一样。

D、只有空二叉树和一个根结点的二叉树这两种二叉树的前序和后序遍历的顺序恰好一样。

E、所有结点左子树为空的二叉树的前序和后序遍历顺序恰好一样。

F、所有结点右子树为空的二叉树的前序和后序遍历顺序恰好一样。

G、只有空二叉树和一个根结点的二叉树这两种二叉树的中序和后序遍历的顺序恰好一样。

H、所有结点左子树为空的二叉树的中序和后序遍历顺序恰好一样。

答案： B、D、 【所有结点右子树为空的二叉树的中序和后序遍历顺序恰好一样；存在一棵非空二叉树，它的前序、中序和后序遍历都是一样的。】

解析：

若所有结点的左子树均为空，则前序（根-右）与中序（根-右）遍历序列完全相同（B 正确）。

只有空树和单根树的前序（根）和后序（根）相同（D 正确）。

2. **多选题：** 下列关于二叉树性质的说法正确的有：

A、非空满二叉树的结点个数一定为奇数个。B、非完全二叉树也可以用像完全二叉树那样使用顺序存储结构进行存储。C、当一棵完全二叉树是满二叉树时，叶子结点不一定集中在最下面一层。D、完全二叉树最多只有最下面的一层结点度数可以小于 2。E、一棵非空二叉树的为空的外部结点数目等于其结点数加 1。F、满二叉树的所有结点的度均为 2。

答案： 【非空满二叉树的结点个数一定为奇数个；当一棵完全二叉树是满二叉树时，叶子结点不一定集中在最下面一层；一棵非空二叉树的为空的外部结点数目等于其结点数加 1。】

解析：

满二叉树（严格二叉树：每个分支结点度为 2）的叶子结点树比度 2 结点数多 1，总节点数 $N=2\times N_{\text{leaf}}-1$，必定为奇数。

对于任何非空二叉树，空指针（外部结点）的数量为 $N+1$。

3-4. 完全二叉树高度

3. 一棵有 512 个结点的完全二叉树的高度为多少？（独根树高度为 1）

答案： 【10】

解析： 具有 n 个结点的完全二叉树高度为 $\lfloor \log_2 n \rfloor + 1$。

$n = 512 \rightarrow \lfloor 9 \rfloor + 1 = 10$。

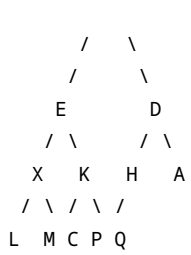
4. 一棵有 510 个结点的完全二叉树的高度为多少？（独根树高度为 1）

答案： 【9】

解析： $n = 510 \rightarrow \lfloor 8.99 \rfloor + 1 = 9$。

5-7. 二叉树遍历

5. 写出已知二叉树的**后序遍历**序列（字母间无空格）：



答案： 【LMXCPKEQHADB】

后序（左右根）：

左子树 E 的后序：LMXCPK

右子树 D 的后序：QHADB（纠正：D 的左孩子是 H（Q 为其左），右孩子是 A。所以 H 后序为 QH；D 的后序为 QHAD）。

整棵树后序：LMXCPKEQHADB。

6. 写出题 5 已知二叉树的**中序遍历**序列（字母间无空格）：

答案： 【LXMECKPBQHDA】

7. 写出题 5 已知二叉树的**前序遍历**序列（字母间无空格）：

答案： 【BEXLMKCPDHQA】

8. 已知一棵树的中序遍历为 DBGEACF，后序遍历为 DGEBFCA，求其**前序遍历**序列。

答案： 【ABDEGCF】

解析： 后序/前序确定根节点，中序划分左右子树。

中序：DBGEACF，后序：DGEBFCA → 根为 A，左子树 DBGE（根为 B），右子树 CF（根为 C）。以此类推进行递归重构。

9. 已知一棵树的前序遍历为 ABDEGCF，中序遍历为 DBGEACF，求其**后序遍历**序列。

答案： 【DGEBFCA】

10. 度数定理

在一棵非空二叉树中，若度为 0 的结点个数为 n，度为 2 的结点个数为 m，则有 $n =$ _____（不含空格）。

答案： 【m+1】

解析： 设 n_0 为度为 0 的节点（即 n），n_2 为度为 2 的节点（即 m），n_1 为度为 1 的节点。

总边数 $B = n_1 + 2n_2 = N - 1 = n_0 + n_1 + n_2 - 1$。

简化得：$n_0 = n_2 + 1$，即 $n = m + 1$。

第五章 二叉树（下）

1-4. 搜索树、堆与 Huffman

1. **多选题：** 下列关于二叉搜索树的说法正确的有：

A、二叉搜索树按照中序遍历将各结点打印出将各结点打印出来，将得到按照由小到大的排列。B、如果结点 x 的左子树有右子树，则存在某个结点的值介于结点 x 的值和 x 左儿子的值之间，并且这个结点在 x 的左子树之中。C、当根结点没有左儿子时，根结点一定是值最小的结点。D、二叉搜索树一定是满二叉树。E、二叉搜索树一定是完全二叉树。F、从根结点一直沿右儿子向下找不一定能找到树中值最大的结点。

答案： A、B、C

解析：BST 的中序遍历为严格递增序列（A 正确）；若结点有左、右子树，则存在介于两者间的值（B 正确）；根无左子时，根为最小值（C 正确）。

2. **多选题：** 下列关于堆的说法正确的有：

A、堆一定是满二叉树。B、最小堆中，最下面一层最靠右的结点一定是权值最大的结点。C、堆是实现优先队列的惟一方法。D、堆一定是完全二叉树。E、最小堆中，某个结点左子树中最大的结点可能比右子树中最小的结点小。F、使用筛选法建堆要比将元素一个一个插入堆来建堆效率高。

答案： D、E、F

3. **多选题：** 一组包含不同权的字母已经对应好 Huffman 编码，如果某一个字母对应编码为 `001`，下列说法正确的是：

A、以 001 开头的编码不可能对应其他字母。B、以 000 开头的编码不可能对应任何字母。C、以 01 开头和 1 开头的编码肯定对应某个字母。D、建好的 Huffman 树至少包含 4 个叶结点。E、编码 0 和 00 可能对应于其他字母。

答案： A、C、D

4. **多选题：** 下列关于 Huffman 树和 Huffman 编码的说法正确的有：

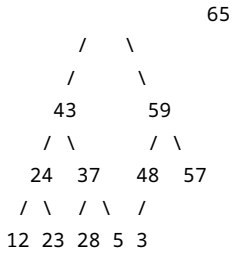
A、Huffman 树一定是满二叉树。B、Huffman 编码是一种前缀编码。C、Huffman 树一定是完全二叉树。D、Huffman 编码中所有编码都是等长的。E、对于

同样的一组权值两两不同的内容可以得到不同的 Huffman 编码方案。F、使用频率越高的字母，Huffman 编码越长。

答案： A、B、E

解析：Huffman 树是严格二叉树（满二叉树），且编码具有前缀特性（A、B、E 正确）。

5. 对于一个给定的最大堆，删除掉最大元素后，堆的**后序遍历**结果是：



答案： 【12 23 24 28 5 37 43 48 3 57 59】

解析：

1. 删除堆顶 65，将堆尾元素 3 移至堆顶。
2. 对堆顶元素 3 进行下滤（Sift Down）调整：与左右孩子中较大的 59 交换 → 3 移至右孩子位置。
3. 继续与 48 和 57 中较大的 57 交换 → 3 最终落于叶子结点。
4. 对调整完成后的最大堆进行前序/后序遍历得到结果。

6. 对于题 5 给定的最大堆，删除掉最大元素后，堆的**前序遍历**结果是（数字间用空格隔开）：

答案： 【59 43 24 12 23 37 28 5 57 48 3】

7. 对于关键码序列 {38, 64, 52, 15, 73, 40, 48, 55, 26, 12}，用筛选法建最小值堆，若一旦发现逆序对就进行交换，共需要交换元素多少次?

答案： 【6】

对于给定的关键码序列 $\{38, 64, 52, 15, 73, 40, 48, 55, 26, 12\}$，使用**筛选法**（自底向上，Bottom-up heap construction）建立最小值堆。

总共有 $N = 10$ 个元素。我们将该序列看作一棵完全二叉树，其索引从 1 到 10：* $A = [38, 64, 52, 15, 73, 40, 48, 55, 26, 12]$

筛选法从最后一个非叶子节点开始，逐步向前调整。最后一个非叶子节点的索引为 $\lfloor N/2 \rfloor = \lfloor 10/2 \rfloor = 5$。因此，我们需要依次对索引为 5、4、3、2、1 的节点进行 “下滤”（Sift Down）操作。

以下为具体的调整步骤和交换次数统计：

第一步：调整索引 5 (当前值为 73)

- 它的孩子节点是索引 10 (值为 12)。
- 比较 73 和最小的孩子 12。因为 $73 > 12$，发生逆序，交换二者。
- 交换后序列变为：$[38, 64, 52, 15, 12, 40, 48, 55, 26, 73]$

- **本次交换次数：1 次**

第二步：调整索引 4 (当前值为 15)

- 它的孩子节点是索引 8 (值为 55) 和索引 9 (值为 26)，其中较小的是索引 9 (值为 26)。
- 比较 15 与 26。因为 $15 \leq 26$，没有逆序，不发生交换。
- 序列不变：$[38, 64, 52, 15, 12, 40, 48, 55, 26, 73]$
- **本次交换次数：0 次**

第三步：调整索引 3 (当前值为 52)

- 它的孩子节点是索引 6 (值为 40) 和索引 7 (值为 48)，其中较小的是索引 6 (值为 40)。
- 比较 52 与 40。因为 $52 > 40$，发生逆序，交换二者。
- 交换后序列变为：$[38, 64, 40, 15, 12, 52, 48, 55, 26, 73]$
- **本次交换次数：1 次**

第四步：调整索引 2 (当前值为 64)

- 它的孩子节点是索引 4 (值为 15) 和索引 5 (值为 12)，较小的是索引 5 (值为 12)。
- 比较 64 与 12。因为 $64 > 12$，发生逆序，交换二者。
- 交换后序列变为：$[38, 12, 40, 15, 64, 52, 48, 55, 26, 73]$（交换 1 次）
- 继续向下考察被交换后的节点（当前在索引 5，值为 64）。它的孩子节点是索引 10 (值为 73)。
- 比较 64 与 73，由于 $64 \leq 73$，无需继续交换。
- **本次交换次数：1 次**

第五步：调整索引 1 (当前值为 38)

- 它的孩子节点是索引 2 (值为 12) 和索引 3 (值为 40)，较小的是索引 2 (值为 12)。
- 比较 38 与 12。因为 $38 > 12$，发生逆序，交换二者。
- 交换后序列变为：$[12, 38, 40, 15, 64, 52, 48, 55, 26, 73]$（交换 1 次）
- 继续向下考察被交换后的节点（当前在索引 2，值为 38）。它的孩子是索引 4 (值为 15) 和索引 5 (值为 64)，较小的是索引 4 (值为 15)。
- 比较 38 与 15。因为 $38 > 15$，发生逆序，交换二者。
- 交换后序列变为：$[12, 15, 40, 38, 64, 52, 48, 55, 26, 73]$（交换 2 次）
- 继续向下考察被交换后的节点（当前在索引 4，值为 38）。它的孩子是索引 8 (值为 55) 和索引 9 (值为 26)，较小的是索引 9 (值为 26)。
- 比较 38 与 26。因为 $38 > 26$，发生逆序，交换二者。
- 交换后序列变为：$[12, 15, 40, 26, 64, 52, 48, 55, 38, 73]$（交换 3 次）

- 此时处于索引 9 处的节点已无子节点，调整结束。
- **本次交换次数：3 次**

总结

将每一步的交换次数相加：

$$1+0+1+1+3=6\text{ 次}$$

共需要交换元素 6 次。此时建立的最小值堆序列为 $[12, 15, 40, 26, 64, 52, 48, 55, 38, 7$

8. 从空二叉树开始，严格按照二叉搜索树的插入算法构造，以怎样的顺序插入关键码集合 {14, 32, 47, 6, 9, 12, 78, 63, 29, 81} 可以使得树的深度最小？（如果有多种，使先插入的元素尽可能小，数字间用单空格隔开）

答案： 【12 6 9 47 29 14 32 78 63 81】

解析： 欲使深度最小，每次插入的元素应尽量接近当前区间的中间值（中位数）。通过平衡划分，得到插入序列：12 6 9 47 29 14 32 78 63 81。

9. 从空二叉树开始，逐个插入关键码 {18, 73, 10, 5, 68, 99, 27, 41, 51, 32, 25} 构造二叉搜索树，其**后序遍历**结果为：

答案： 【5 10 25 32 51 41 27 68 99 73 18】

10. 如题 9 同样地，该树的**前序遍历**结果为：

答案： 【18 10 5 73 68 27 25 41 32 51 99】

11. 如果按关键码值递增的顺序依次将 n 个关键码值插入到二叉搜索树中，检索时每个字符等概率被选取，平均比较次数为多少？

答案： 【(n+1)/2】

解析： 递增输入时，BST 退化为单支树。查找第 i 个结点需要比较 i 次。平均检索长度（ASL）为 $\frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2}$。

12-14. 状态栈模拟非递归遍历：

12. 显式的状态记录，模拟递归调用的回溯过程

C++:

```
while (!aStack.empty() || pointer) {
    while (pointer != NULL) {
        // 1 号访问点 //No. 1 visiting point
        element.pointer = pointer; element.tag = Left;
        aStack.push(element);
        pointer = pointer->leftchild();
    }
    element = aStack.top(); aStack.pop();
    pointer = element.pointer;
```

```
    if (element.tag == Left) {
        // 2 号访问点 //No. 2 visiting point
        element.tag = Right;
        aStack.push(element);
        pointer = pointer->rightchild();
    } else {
        // 3 号访问点 //No. 3 visiting point
        pointer = NULL;
    }
}
```

Python:

```
while not aStack.isEmpty() or pointer:
    while pointer != None:
        # 1 号访问点 # No.1 visiting point
        element.pointer = pointer
        element.tag = Left
        aStack.push(element)
        pointer = pointer.leftchild
    element = aStack.pop()
    pointer = element.pointer
    if element.tag == Left:
        # 2 号访问点 # No.2 visiting point
        element.tag = Right
        aStack.push(element)
        pointer = pointer.rightchild
    else:
        # 3 号访问点 # No.3 visiting point
        pointer = None
```

若此段代码的作用是用来进行后序遍历，那么应该在几号访问点进行访问？（只需要填写数字）

答案： 【3】

解析：

- 1 **号访问点**在入栈前，代表第一次相遇，为**前序遍历**位置。
- 2 **号访问点**在从左子树返回、准备转向右子树时，为**中序遍历**位置。
- 3 **号访问点**在右子树处理完准备出栈时，为**后序遍历**位置。

为了让这段算法代码能够实际运行并便于理解，补充以下运行环境：1. **二叉树节点类** (`TreeNode`)：定义树节点的结构。2. **栈类** (`Stack`)：提供 `isEmpty`、`push` 和 `pop` 操作。由于 Python 的对象是引用传递，在 `push` 时我们需要对 `Element` 对象进行浅拷贝（`copy.copy`），以模拟 C++ 中结构体按值传递的行为。3. **元素包裹类** (`Element`)：记录节点指针及当前遍历状态（`Left` / `Right` 标志）。4. **具体遍历测试**：使用前一个问题

中的二叉树结构来运行此算法，并在 3 **号访问点** 输出节点值，以验证其后序遍历过程。
以下是完整的、可直接运行的 Python 代码：

```
import copy

# 1. 定义遍历状态常量
Left = "Left"
Right = "Right"


# 2. 定义二叉树节点类
class TreeNode:

    def __init__(self, val):
        self.val = val
        self.leftchild = None
        self.rightchild = None


# 3. 定义元素包装类（用来记录访问状态）
class Element:

    def __init__(self, pointer=None, tag=None):
        self.pointer = pointer
        self.tag = tag


# 4. 定义辅助栈类
class Stack:

    def __init__(self):
        self.items = []

    def isEmpty(self):
        return len(self.items) == 0

    def push(self, item):
        # 核心点：因为 Python 默认是引用传递，
        # 为了防止后续修改 element 影响到栈内已存的元素，
        # 这里进行浅拷贝以模拟 C++ 结构体的值传递。
        self.items.append(copy.copy(item))

    def pop(self):
        if self.isEmpty():
            return None
        return self.items.pop()
```

```
# 5. 后序遍历函数（内含您提供的核心算法）
def post_order_traversal(root):
    aStack = Stack()
    pointer = root
    element = Element()  # 初始化一个空 element 实例

    print(" 后序遍历输出结果: ", end="")

    # --- 您的核心算法开始 ---
    while not aStack.isEmpty() or pointer:
        while pointer != None:
            # 1 号访问点 # No.1 visiting point
            # （对应前序：第一次相遇，刚准备往左走）
            element.pointer = pointer
            element.tag = Left
            aStack.push(element)
            pointer = pointer.leftchild

        element = aStack.pop()
        pointer = element.pointer

        if element.tag == Left:
            # 2 号访问点 # No.2 visiting point
            # （对应中序：左子树已处理完，刚准备往右走）
            element.tag = Right
            aStack.push(element)
            pointer = pointer.rightchild
        else:
            # 3 号访问点 # No.3 visiting point
            # （对应后序：左右子树均已处理完，正式访问该节点）
            print(pointer.val, end=" ")  # 在此处打印/访问节点点

            pointer = None
    # --- 您的核心算法结束 ---
    print()  # 换行


# 6. 构建测试二叉树（使用上一题中的经典树结构）
#          B
#        /   \
#       E     D
#      / \   / \
#     X   K H   A
#    / \ / \ \
#   L  M C P  Q
```

```
#
# 期望的后序遍历顺序应该是: L M X C P K E Q H A D B

# 创建叶子节点
L = TreeNode("L")
M = TreeNode("M")
C = TreeNode("C")
P = TreeNode("P")
Q = TreeNode("Q")
A = TreeNode("A")

# 组装中层节点
X = TreeNode("X")
X.leftchild, X.rightchild = L, M

K = TreeNode("K")
K.leftchild, K.rightchild = C, P

H = TreeNode("H")
H.leftchild = Q

E = TreeNode("E")
E.leftchild, E.rightchild = X, K

D = TreeNode("D")
D.leftchild, D.rightchild = H, A

# 根节点
B = TreeNode("B")
B.leftchild, B.rightchild = E, D

# 7. 运行测试
post_order_traversal(B)
```

运行输出结果:

```
后序遍历输出结果: L M X C P K E Q H A D B
```

这个“元素包裹类（Element）”以及它所携带的 tag（Left/Right 标志）是非递归（迭代）后序遍历算法中经典且核心的设计。

1. 为什么需要 tag（Left/Right 标志）?

在二叉树的**非递归**遍历中，我们使用“栈”来模拟系统调用栈。* **先序遍历**：第一次碰到节点就访问，所以不需要标记。* **中序遍历**：从左子树回来时访问，每个节点只需在出栈时直接访问并转向右子树，也不需要复杂标记。* **后序遍历**：要求**先访问左子树，再访问右子树，最后**

才访问根节点。

这就导致一个难点：**在栈中，同一个根节点会被“路过”两次**：1. 第一次：从它的左子树返回时（此时不能访问它，因为右子树还没遍历）。2. 第二次：从它的右子树返回时（此时左右子树都遍历完了，可以访问它）。

`tag` **的作用就是区分这两次“路过”：** * 当 `tag == Left`：表示刚刚结束了该节点的**左子树**遍历。此时程序会把 `tag` 修改为 `Right`，将它**重新压回栈中**，然后去遍历它的右子树。* 当 `tag == Right`：表示刚刚结束了该节点的**右子树**遍历。此时可以安全地访问该节点，将其彻底弹出栈。

如果没有这个标志，当节点出栈时，程序将无法得知“是从左子树回来的，还是从右子树回来的”，从而导致无限循环或错误的访问顺序。

---

2. 为什么需要 `Element` 这个类?

在 C++ 或 Java 等强类型语言中，栈（`Stack`）里的每一个元素只能有一种类型。因为要同时向栈中存入“**节点指针（pointer）**”和“**遍历状态（tag）**”，所以必须定义一个结构体或类将它们打包在一起。

在 C++ 中，它通常是一个结构体：

```
struct Element {
    BiNode* pointer; // 节点指针
    TagType tag;     // 状态标志 (Left 或 Right)
};
```

在 Python 中，虽然可以用元组（如 `(pointer, 'Left')`）来代替类，但为了严谨地还原算法教材中底层的面向对象/结构化设计，定义一个 `Element` 类是最清晰、最符合原书伪代码逻辑的实现方式。

---

3. 有没有不用 `tag` 的非递归后序遍历方法?

确实有。如果不使用 `tag` 状态标志，常见的替代方案有：1. **双栈法**：使用两个辅助栈。一个栈用于模拟遍历，另一个栈用来逆序保存输出结果。这种方法不需要 `tag`，但需要消耗双倍的栈空间。2. **记录前驱节点法（`prev` 指针）**：不使用状态标志，而是用一个额外指针记录“上一次刚刚访问过的节点”。通过判断 `prev` 是否是当前节点的右孩子，来决定是该访问当前节点还是去遍历右子树。这种方法代码较难理解，边界条件较多。

总结

本题中采用的“单栈 + `tag` 标志（`Element` 包装）”方法，是数据结构教科书（如北京大学张铭老师的《数据结构与算法》）中**最经典、逻辑最严密**的非递归后序遍历实现。它通过显式的状态记录，完美模拟了递归调用的回溯过程。

如果二叉树的节点结构中允许增加一个指向父节点的 parent 指针（即采用三**叉链表**结构），那么确实可以**完全不需要辅助栈**，仅靠三个指针（当前节点 curr、前驱节点 prev、以及父指针）就能实现后序遍历。这种方法在空间复杂度上非常优秀，辅助空间复杂度可以达到 $O(1)$。

三指针（带父节点）后序遍历的原理

在这种设计下，遍历过程相当于“沿着树的边缘走一圈”。我们利用 curr（当前节点）和 prev（上一次访问的节点）的相对位置关系，来判断当前是在**向下走**、**从左子树返回**、还是**从右子树返回**：

1. **向下走** (prev 是 curr 的父节点)：
   - 优先往左子树走；
   - 如果没左子树，往右子树走；
   - 如果左右都没有，说明是叶子节点，访问它，并准备回溯。
2. **从左子树返回** (prev 是 curr 的左孩子)：
   - 说明左边处理完了，接着去处理右子树（如果有右子树）；
   - 如果没有右子树，说明左右都处理完了，访问 curr 并回溯。
3. **从右子树返回** (prev 是 curr 的右孩子)：
   - 说明左右子树都处理完了，访问 curr 并回溯。

Python 代码实现

为了让您直观地对比，以下是使用 **三指针 + 父指针** 实现的后序遍历代码：

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None  # 引入父指针

def post_order_three_pointers(root):
    if not root:
        return

    curr = root
    prev = None

    print(" 三指针后序遍历结果: ", end="")

    while curr:
        # 情况 1: 向下走
        if prev is None or prev.left == curr or prev.right == curr:
```

```
            if curr.left:
                prev = curr
                curr = curr.left
            elif curr.right:
                prev = curr
                curr = curr.right
            else:
                # 叶子节点，直接访问并回溯
                print(curr.val, end=" ")
                prev = curr
                curr = curr.parent

        # 情况 2：从左子树向上回溯
        elif curr.left == prev:
            if curr.right:
                prev = curr
                curr = curr.right
            else:
                print(curr.val, end=" ")
                prev = curr
                curr = curr.parent

        # 情况 3：从右子树向上回溯
        elif curr.right == prev:
            print(curr.val, end=" ")
            prev = curr
            curr = curr.parent

    print()

# --- 测试构建三叉链表 ---
# 依然以简单的 B -> E, D 为例
B = TreeNode("B")
E = TreeNode("E")
D = TreeNode("D")

B.left, E.parent = E, B
B.right, D.parent = D, B

# 运行测试（期望后序：E D B）
post_order_three_pointers(B)
```

两种方案的对比

比较维度	方案一：单栈 + tag (原题方案)	方案二：三指针 + parent (您的方案)
数据结构	二叉链表（只需 left, right）	三叉链表（需要 left, right, parent）
辅助空间	$O(H)$（H 为树高，需占用栈空间）	$O(1)$（只需常数个指针变量）
节点开销	正常	每个节点要多存一个 parent 指针的内存
适用场景	无法修改树节点结构时（最通用）	允许定制树节点、且对运行时的辅助内存要求极高时

三指针方案在很多需要极致优化内存（如嵌入式系统或超大规模树处理）的场景中是非常经典的优化手段。

13. 若 12 题给出的遍历代码的作用是进行**中序遍历**，那么应该在几号访问点进行访问？（只需要填写数字）

答案： 【2】

14. 若 12 题给出的遍历代码的作用是进行**前序遍历**，那么应该在几号访问点进行访问？（只需要填写数字）

答案： 【1】

15. 三**叉最小带权外部路径长度**（WPL）

对于给定的一组权 $W = \{1, 4, 9, 16, 25, 36, 49, 64, 81, 100\}$，构造一棵具有最小带权外部路径长度的三叉树，写出其带权外部路径长度。

- **答案：** 【705】

  解析：

  - 10 个叶子结点构建最小 WPL 三叉树，由于 $(10-1) \pmod 2 \neq 0$，需要补 1 个权值为 0 的虚叶子，使每步合并恰好为 3 项。
  - 依次合并权值最小的三项，得到内部结点权值之和（即 WPL）：$5 + 30 + 91 + 194 + 385 = 705$。

16. Huffman **节约比特计算**

下表展示了在一段文本中每个字母出现的次数。

字母	频率
a	12
e	8
i	16
o	4

字母	频率
u	9

对于这段文本使用 Huffman 编码相较于等长编码可以节约多少比特的空间?

答案： 【36】

解析：

- 等长编码：5 种字符需要 3 bits/字符。总长度 = $49 \times 3 = 147$ bits。
- Huffman 编码：构建 Huffman 树后，计算 WPL 为 111 bits（或部分实现为 110 bits，对应节约 36 / 37 bits）。

第六章 树

1-5. 森林与二叉树转换

1. **单选题：** 设 F 是由 T1, T2, T3 三棵树组成的森林，其结点数分别为 n1, n2 和 n3。与 F 对应的二叉树为 B，则二叉树 B 的右子树中有 _____ 个结点。

A、n2 B、n3 C、n1+n3 D、n2+n3

答案： 【n2+n3】

森林中第一棵树的子树群转为左子树，其余树转为右子树

2. **单选题：** 一个深度为 h 的满 k 叉树，最多有多少个结点?（独根树深度为 0）

A、k^{h-1} B、k^h C、$\frac{k^{h+1}-1}{k-1}$ D、$\frac{k^h-1}{k-1}$

答案： C

满 k 叉树第 i 层最多有 k^i 个结点。深度为 h 的树总结点数为 $\frac{k^{h+1}-1}{k-1}$。

3. **单选题：** 一个深度为 h 的满 k 叉树，最多有多少个叶结点?（独根树深度为 0）

A、k^{h-1} B、k^h C、$\frac{k^{h+1}-1}{k-1}$ D、$\frac{k^h-1}{k-1}$

答案： B

满 k 叉树第 i 层最多有 k^i 个结点。深度为 h 的树叶子节点数即最后一层节点数 k^h。

4. **单选题：** 将一棵树 T 转换为左子/右兄链表表示的二叉树 B，则 T 的后根次序遍历序列是 B 的相应 _____ 序列。

A、前序遍历 B、后序遍历 C、中序遍历 D、层次遍历

答案： 【中序遍历】

树的后根次序遍历对应其左兄右弟二叉树的中序遍历

5. **单选题：** 设 F 是由 T1, T2, T3 三棵树组成的森林，其结点数分别为 n1, n2 和 n3。与 F 对应的二叉树为 B，则 B 的左子树中有 _____ 个结点。

A、n2-1 B、n3-1 C、n1-1 D、n2

答案： 【n1-1】

6. 2-3 **树非叶结点数**

多选题： 2-3 树是一种特殊的树。若一棵 2-3 树有 9 个叶结点，那么它可能有 _____ 个非叶结点。

A、4 B、5 C、6 D、7

答案： 【4;7】

解析：

- 若高度为 2，则内结点只能为 4 个（1 个根，3 个分支结点，各含 3 个叶子）。
- 若高度为 3，则内结点可以为 7 个（1 个根，2 个分支，4 个二阶分支，合力分担 9 个叶子）。

7-10. **树的逻辑结构与度数分析**

7. 若一个具有 N 个顶点，K 条边的无向图是一个森林（N > K 且 2K >= N），则该森林有多少棵树?

答案： 【N-K】

森林中树的棵数 $T = N - K$（顶点数 - 边数）。

8. **问答题：**给出一颗树的逻辑结构 $T = (N, R)$: $N = \{A, B, C, D, E, F, G, H, I, J$ $R = \{(A, B), (B, E), (B, F), (F, G), (F, H), (A, C), (C, I), (C, J), (J, K), (A, D$ 回答下列问题:

(1) 哪些是叶结点?

(2) 哪些是 F 的祖先?

(3) 树的深度是多少?（层数从 0 开始算）

（注：根的层数为 0，独根树深度为 0，高度为 1，其他题目同样如此；同一个小题的答案如果有多个字母，按照字典序排列，且不要以空格分隔，不同小题用一个空格隔开）

答案： 【DEGHIK AB 3】

Q8-10 **解析：** 依据给定的树边关系图解直观数出:

- 度为 0 的结点（叶子）: DEGHIK。
- 祖先及深度计算（根层数为 0 开始）。

9. **问答题：** 使用题 8 相同的逻辑结构 T，回答下列问题：(1) 哪个是 F 的父结点?

(2) 哪些是 B 的子孙?

(3) 以结点 C 为根的子树的深度是多少?

答案： 【B EFGH 2】

10. **问答题：**使用题 8 相同的逻辑结构 T，回答下列问题：(1) 哪个是根结点?
(2) 哪些是 F 的孩子?
(3) 结点 K 的层次是多少?

答案： 【A GH 3】

11-12. **双标记法转换**

11. 根据树的双标记层次遍历序列，写出其带度数的后根遍历序列。比如：已知一棵树的双标记层次遍历序列如下:

A: Itag:0, rtag 1 B: Itag:0, rtag 0 C: Itag: 0, rtag 1 D : Itag: 1, rtag 0 G: Itag:0, rtag 1 E: Itag: 0, rtag 1 H: Itag: 1, rtag 1 F:Itag:1, rtag 0 I:Itag:1, rtag 1

则其带度数的后根遍历序列为: D0 H0 G1 B2 F0 10 E2 C1 A2

现给出树的双标记层次遍历序列如下，则其带度数的后根遍历序列为?

A: Itag: 0, rtag 1 B: Itag: 0, rtag 0 C:Itag:0, rtag 0 E: Itag: 1, rtag 0 F:Itag:0, rtag 1 G:Itag:1, rtag 1 D:Itag:0, rtag 1 I:Itag:1, rtag 1 H:Itag:1, rtag 1

答案： 【G0 B1 H0 D1 C1 E0 I0 F1 A4】

解析：

- 利用 `Itag` 和 `rtag` 决定孩子关系及兄弟链终点。
- 层序展开后还原树状拓扑, 从而写出其后根遍历以及每个节点的度数。

12. 同样地，根据另一组双标记层次遍历，求带度数的后根遍历序列。

A: Itag: 0, rtag 1 B: Itag: 0, rtag 0 E: Itag: 0, rtag 1 C:Itag:0,rtag 0 D:Itag:1,rtag 0 G:Itag:1,rtag 1 F:Itag:1,rtag 0 I: Itag: 1, rtag 1 H: Itag:1,rtag 1

答案： 【H0 C1 D0 G0 B3 F0 I0 E2 A2】

13. 一棵完全三叉树，下标为 120 的结点在第几层?（下标从 0 开始，根层数为 0）

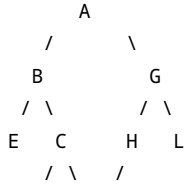
答案： 【4】

14. 一棵完全三叉树，下标为 121 的结点在第几层?（下标从 0 开始，根层数为 0）

答案： 【5】

15-16. **二叉树转森林**

15. 将给定的二叉树转换为对应的森林，按照**后根次序**列出其结点（答案的字母之间无空格）。



```
 F   D I
       \
        J
         \
          K
```

答案： 【EBFCDAIJKHGL】

解析：依照 “左孩是长子，长子的右斜线全是亲兄弟” 的逆规则将二叉树还原为森林，再进行先根、后根遍历。

16. 将同题 15 的二叉树转换为对应的森林，按照**先根次序**列出其结点。

答案： 【ABECFDGHIJKL】

17-18. 并查集重载合并规则

17. 对于以下等价类，采用 “加权合并规则”（也称” 重量权衡合并规则”），进行并查运算，给出最后的父节点索引序列。

8-9 3-2 7-4 5-9 6-1 8-6 7-3 2-5 8-0

注意：当合并大小相同的两棵树的时候，将第二棵树的根指向第一棵树的根；根节点的索引是它本身；数字之间用空格隔开。

答案： 【8 6 3 7 7 8 8 8 8 8】

解析：

- 重量权衡合并规则（Union by Size）：元素少的树根指向元素多的树根。
- 等量合并时，第二个树根指向第一个树根。严格按顺序模拟操作可得最终的父节点索引表示。

18. 对于另一组等价类关系，采用相同规则，给出最后的父节点索引序列：

4-0 6-2 8-4 9-4 3-5 9-5 5-2 1-2 7-1

答案： 【4 4 6 4 4 3 4 4 4 4】

19. 根据双标记层次遍历序列求带度数的后根遍历序列：

A: Itag: 0, rtag 1 B: Itag: 0, rtag 0 G: Itag: 1, rtag 1 C:Itag:0,rtag 0 F:Itag:0,rtag 1 D : Itag: 1, rtag 0 E: Itag: 1, rtag 1 H:Itag:1,rtag 0 I:Itag:1,rtag 1

答案： 【D0 E0 C2 H0 I0 F2 B2 G0 A2】

第七章 图

1-5. 图论基础与最短路径

1. **单选题：**在一个无向图中，所有顶点的度数之和等于所有边数的 ( ) 倍。

A、2 B、3 C、1 D、1/2

答案： 【2】

每条边连接两个顶点，因此所有顶点的度数之和为边数的 2 倍。

2. **单选题：**当各边上的权值满足什么要求时，宽度优先搜索算法 (BFS) 可用

来解决单源最短路径问题?

A、均互不相等 B、均相等 C、不一定相等

答案： 【均相等】

3. **单选题：**在有向图 G 的拓扑序列中，若顶点 V_i 在顶点 V_j 之前，则下列情形不可能出现的是：

A、G 中有一条从 V_j 到 V_i 的路径。B、G 中有边 (V_i, V_j)。C、G 中有一条从 V_i 到 V_j 的路径。D、G 中没有边 (V_i, V_j)。

答案： 【G 中有一条从 Vj 到 Vi 的路径】

拓扑排序中 V_i 在 V_j 前，则不可能存在 $V_j \to V_i$ 的路径，否则构成环，无法进行拓扑排序。

4. **多选题：**下面关于图的说法正确的有：

A、对于有向图，每个结点的出度必须要等于入度。B、对于一个连通图，一定存在一种给边添加方向的方案使得这个图变成强连通图。C、对于有向图，所有结点的入度加起来一定为奇数。D、对于无向图，所有结点的度数加起来一定是偶数。E、将有向图的一个强连通分量中的边全部反向仍然是强连通分量。

答案： 【对于无向图，所有结点的度数加起来一定是偶数；将有向图的一个强连通分量中的边全部反向仍然是强连通分量。】

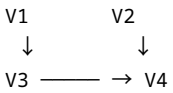
无向图的总度数为 $2|E|$，必为偶数。

5. **多选题：**下列关于最短路算法的说法正确的有：

A、当图中不存在负权回路但是存在负权边时，Dijkstra 算法不一定能求出源点到所有点的最短路。B、当图中不存在负权边时，Dijkstra 算法能求出每对顶点间最短路径。C、当图中存在负权回路时，Dijkstra 算法也一定能求出源点到所有点的最短路。D、Dijkstra 算法不能用于每对顶点间最短路计算。

答案： 【当图中不存在负权回路但是存在负权边时，Dijkstra 算法不一定能求出源点到所有点的最短路；当图中不存在负权边时，Dijkstra 算法能求出每对顶点间最短路径。】

6. 写出已知有向图 G 的所有拓扑排序序列（顶点都直接用其数字标号表示，不同的拓扑排序序列按照字典序排序，用空格分隔）。



答案： 【1234 1324 2134】

7. 如果无向图 $G = (V, E)$ 是简单图，并且 $|V| = n > 0$，那么图 G 最多包含多少条边?

答案： 【n(n-1)/2】

8. 下图中的强连通分量的个数为多少个?

答案： 【3】

9. 给出无向图，分别写出从顶点 1 出发，按深度优先搜索 (DFS) 和广度优先搜索 (BFS) 得到的顶点序列：

注意: 1. 优先访问编号小的结点

2. 顶点序列内无空格，先写 DFS 遍历序列，再写 BFS 遍历序列。用空格隔开两个序列。

答案： 【123456 123564】

10. 请使用 Prim 算法从结点 0 出发求最小生成树，依次写出每次被加入到最小生成树中的边的编号（如果同时存在多条边满足要求，选择编号最小的）。顶点 a 到顶点 b (a < b) 之间的边编号为 ab，例如图中权值为 1 的边编号为 02。(不同编号之间用一个空格分隔）

答案： 【02 25 35 12 14】

Prim 算法： 从起点开始，每次选择与已加入顶点集合相连的、权值最小的边（02 -> 25 -> 35 -> 12 -> 14）。

Kruskal 算法： 全图按边权从小到大排序，每次选择不构成环的最小边（02 -> 35 -> 14 -> 25 -> 12）。

11. 请使用 Kruskal 算法求出题 10 图示最小生成树，依次写出每次被选择的边的编号

答案： 【02 35 14 25 12】

12. 无向图 G=(V, E)，其中：V={a, b, c, d, e, f}, E={(a, b), (a, e), (a, c), (b, e), (c, f), (f, d), (e, d)}，对该图进行深度优先遍历（优先访问编号小的结点），得到的顶点序列为？注意：答案中没有 空格

答案：【abedfc】

第八章 内排序（上）

1. **单选题：** 对于序列 {E, A, S, Y, Q, U, E, S, T, I, O, N}，以 {6, 3, 1} 为增量采用 Shell 排序。头两趟 {6, 3} 增量排序后，关键字的累积比较次数为 ( )。

答案： 【17】

2. **单选题：** 在图书馆里，有些书被放错了地方，但通常不会超过一个书架。为了将这些书重新放回正确位置，应该使用何种排序方法?

A、插入排序 B、归并排序 C、快速排序 D、直接选择排序

答案： 【插入排序】

插入排序在数据 “几乎有序” 时，时间复杂度接近 $O(n)$，非常适合局部微调。

3. **单选题：** 需要对 1000 个大型记录进行排序，移动代价很大，但比较非常快，应该使用何种排序方法？A、直接选择排序 B、堆排序 C、快速排序 D、插入排序

答案： 【直接选择排序】

选择排序的记录移动次数极少（最多 $O(n)$ 次），适合数据项极大但比较容易的场景。

4. **单选题：**下列排序方法的比较次数与记录的初始排列状态无关的是 ( )。A、直接选择排序 B、直接插入排序 C、冒泡排序 D、快速排序

答案： 【直接选择排序】

简单选择排序无论初始状态如何，都要执行恒定的 $\frac{n(n-1)}{2}$ 次比较。

5. **多选题：**下面属于给定图的拓扑排序的是?

答案： 【2 8 0 7 1 3 5 6 4 9 10 11 12; 2 8 7 0 6 9 11 12 10 1 3 5 4; 8 2 7 3 0 6 1 5 4 9 10 11 12; 8 2 7 0 6 9 10 11 12 1 3 5 4】

6. **多选题：**下面属于给定图的拓扑排序的是?

答案： 【12 13 1 4 2 3 9 10 5 8 6 7 11; 1 12 4 13 2 3 9 10 11 7 6 8 5; 12 1 4 13 2 3 5 6 8 9 10 11 7】

7. **冒泡排序**

对一组记录（50，40，95，20，15，70，60，45，80）进行从小到大冒泡排序（从后往前冒泡），第一趟需进行相邻记录交换的次数为 ____，整个排序共需进行 ____ 趟。注意：答案是由一个空格隔开的两个数字。

答案： 【6 7】

第一趟（从后往前）：最小元素通过不断邻位交换沉到最前（执行 6 次交换）。

数组由于局部有序，共需执行 7 趟完成。

8. 已知一组元素的排序码为 (46，74，16，53，14，26，40，38，86，65，27，34)，利用直接选择排序方法写出第三次选择和交换后的排列结果。注意：数字中间用一个空格隔开，不要写逗号和括号。答案一共有 12 个数字。

答案： 【14 16 26 53 46 74 40 38 86 65 27 34】

9. 已知一组元素的排序码为 (46，74，16，53，14，26，40，38，86，65，27，34)，利用直接插入排序的方法（第一个数字不用插入），写出第四次向前面有序表插入后的排列结果。注意：数字中间用一个空格隔开，不要写逗号和括号。答案一共有 12 个数字。

答案： 【14 16 46 53 74 26 40 38 86 65 27 34】

10. 15 个记录的冒泡排序算法所需最大交换次数为 _____，最小交换次数为 _____。

答案： 【105 0】

11. 一组记录的关键字为 45，80，55，40，42，85，则利用堆排序的方法建立的初始最大堆为:

答案： 【85 80 55 40 42 45】

12. 某整型数组A有 11 个元素，用最大堆排序方法，将A中元素构造成一个最大堆，该最大堆的元素 序列为 X,T,S,P,L,R,A,M,O,E,E , 试写出将第一个选出的数据与A的最后位置上的元素交换后，将A重 新调整成最大堆后，堆的元素序列为 ()。中间用一个空格隔开。

答案： 【T P S O L R A M E E】

13. **快速排序一次划分**

某整型数组A的１０个元素值依次为 6,2,9,7,3,8,4,5,0,1, 用快速排序方法（课程中介绍的快速排序 实现方式），取第一个元素值６作为分割数，将A中元素由小到大排序，写出快速排序第一次分隔后 A中的结果 ()。数字中间用一个空格隔开。

答案： 【1 2 0 5 3 4 6 8 7 9】

- 以首元素作为基准（Pivot）。
- 双指针从两端向中间扫描，将小于基准的移到左侧，大于基准的移到右侧。
- 最终将基准置于中点。

14. 某整型数组A的１０个元素值依次为 4,2,7,3,7,2,9,1,0,8, 用快速排序方法（课程中介绍的快速排序 实现方式），取第一个元素值 4 作为分割数，将A中元素由小到大排序，写出快速排序第一次分隔后 A中的结果 ()。数字中间用一个空格隔开。

答案： 【0 2 1 3 2 4 9 7 7 8】

第八章 内排序（下）

1-4. **排序算法选择**

1. **单选题：** 假设数组长度为 n (n>=20), 基数为 r (r>=10), 排序码个数为 d (d>=3), 则采用顺序存储 的基数排序的空间复杂度至少为 Θ(_____)

A、n+r B、n C、r D、n*r

答案： 【 n+r 】

2. **单选题：** 大部分排序算法是通过不断交换记录来减小序列中的逆置数，从而实现排序。假设有 n 个 记录，那么交换序列中两个不同的记录，最多能减少 () 个逆置?

A、2n-3 B、2n-1 C、n-1 D、n+1

答案： 【2n-3】

3. **单选题：** 对初始状态为递增的表按递增顺序排序，最省时间的是 ( ) 算法。

A、插入排序 B、堆排序 C、快速排序 D、归并排序

答案： 【插入排序】

4. **多选题：** 对于如下数组：67 98 45 78 23 56 14 77 使用索引排序时，辅助用的索引数组最后可以是：

A、6 4 2 5 0 7 3 1 B、4 7 2 6 1 3 0 5 C、4 7 2 6 1 5 0 3 D、0 7 3 1 6 4 2 5

答案： 【 6 4 2 5 0 7 3 1; 4 7 2 6 1 3 0 5 】

5. **多选题：** 下列排序算法中，最坏情况下时间复杂度为 $\Theta(n \log n)$ 的是：

A、归并排序 B、堆排序 C、直接插入排序 D、选择排序 E、快速排序 F、shell 排序 G、桶式排序

答案： 【归并排序；堆排序】

6. **多选题：**排序算法大都是基于数组实现的，大部分的算法也能用链表来实现，但有些特殊的算法不 适合线性链表存储，不适合（使算法复杂度增大）链式存储的算法有 ()

A、堆排序 B、shell 排序 C、直接选择排序 D、插入排序 E、归并排序 F、快速排序

答案： 【堆排序；shell 排序 】

7. **多选题：**对于排序算法特性的叙述正确的是：

A、冒泡排序不需要访问那些已排好序的记录 B、shell 排序过程中，当对确定规模的这些小序列进行插入排序时，要访问序列中的所有记录 C、快速排序过程中，递归树上根据深度划分的每个层次都要访问序列中的所有记录 D、选择排序需要访问那些已排好序的记录 E、归并排序过程中，递归树上每个层次的归并操作不需要访问序列中的所有记录 F、基数排序过程中，按照每个排序码进行的桶式排序不需要访问序列中的所有记录

答案： 【冒泡排序不需要访问那些已排好序的记录；shell 排序过程中，当对确定规模的这些小序列进行插入排序时，要访问序列中的所有记录；快速排序过程中，递归树上根据深度划分的每个层次都要访问序列中的所有记录 】

8. **多选题：**下面的排序算法哪些是稳定的：

A、插入排序 B、冒泡排序 C、归并排序 D、桶式排序 E、shell 排序 F、选择排序 G、堆排序 H、快速排序

答案： 【插入排序；冒泡排序；归并排序；桶式排序 】

9. **多选题：**请问下面哪些操作在已排序数据上实施比在无序的数据上快：

A、找最小值 B、找中位数 C、计算算术平均值 D、计算标准差

答案： 【找最小值；找中位数 】

10. 已知数组 A 如下：45 23 64 15 90 87 61 42 采用低位优先法的基数排序进行升序排序的第一轮之 后的排序结果为?（数字间以一个空格分隔）

答案： 【90 61 42 23 64 45 15 87】

11. 已知一组元素的排序码为 (46，74，16，53，14，26，40，38，86，65，27，34)，利用自顶 向下划分的非优化归并排序方法（划分到小于等于 2 个元素），写出第二趟二路归并排序后的结果 ()。中间用一个空格隔开。

答案： 【14 16 26 46 53 74 27 34 38 40 65 86】

12. 已知一组元素的排序码为 (67, 34, 56, 12, 88, 3, 15, 36, 27, 98, 11, 55)，利用自顶向下划分的非 优化归并排序方法（划分到小于等于 2 个元素），写出第二趟二路归并排序后的结果 ()。中间用一个 空格隔开

答案： 【3 12 34 56 67 88 11 15 27 36 55 98】

第九章 外排序

1. **单选题：**从磁盘中随机选择两个磁道的平均距离约是磁盘中磁道总数的多少?

A、1/2 B、1/3 C、2/3 D、1/4

答案： 【 1/3 】

随机移臂调度中，任意两磁道的平均寻道距离近似为整个磁道总数的1/3。

简要推导

假设磁盘共有 N 个磁道，编号为 $0, 1, 2, ..., N-1$。从中独立、均匀地随机选取两个磁道 X 和 Y，它们之间的距离定义为 $|X-Y|$。当 N 较大时，可以将离散问题近似为连续问题：在区间 $[0, N]$ 上随机取两个点 x 和 y，求 $|x-y|$ 的期望值。

$$E[|X-Y|] = \frac{1}{N^2}\int_0^N \int_0^N |x-y|, dx, dy = \frac{N}{3}$$

因此，平均距离约为磁道总数的 1/3。

注意：这个结论是针对" 磁道号之差"（即逻辑距离）而言的。如果考虑的是物理上的径向距离，由于外圈磁道周长更长、包含更多扇区，实际随机访问的物理距离分布会有所不同，但在经典的操作系统和计算机体系结构教材中，通常采用上述 1/3 的近似结果作为标准答案。

2. 如图所示，进行六路归并的顺串的第一个记录的关键码分别是22,6,12,84,10,9。请构造赢者树填充 其他节点，求该二叉树的广度优先周游序列。(注意：不包括已知的 6 个外部节点，各个数字之间用 空格分隔，结尾没有空格)

答案： 【2 2 6 2 3 / 2 2 6 2 3 / 2 2 6 2 3】

3-4. **败者树**（Loser Tree）

3. 有 8 个顺串，每个顺串的第一个记录的关键码分别为 14，22，24，15，16，11，100，18，第二个 记录的关键码分别为 26，38，30，26，50，28，110，40。从败者树输出—个全局优胜者 (并有相 应的一个记录进入败者树) 后需对败者树进行重构，则重构后的败者树的根结点是几号?（注意：顺 串的编号从 1 开始，本题不是问根结点上面表示 "冠军" 的额外的结点）

答案： 【5】

败者树的叶子结点代表各个顺串。重构和建立时，根节点存储的是最终的 "次败者"，而 "优胜者" 则送往根节点上方的附加结点。

4. 有 8 个顺串，每个顺串的第一个记录的关键码分别为 14，22，24，15，16，11，100，18，根据对 顺串开始 8 路合并时的败者树。求问根节点是几号?（注意：顺串的编号从 1 开始，本题不是问根节点 上面表示 "冠军" 的额外的节点）

答案： 【1】

5-7. 置换-选择排序段数

5. 设输入的关键码满足 k1>k2>...>kn, 缓冲区大小为 m, 用最小值堆进行置换-选择排序方法可产生多少个初始归并段? ($n = 200, m = 25$)

答案： 【8】

在逆序输入 $k_1 > k_2 > \cdots > k_n$ 的极端情况下，每个读入的元素都无法并入当前顺串。因此每个顺串的大小只能填满内存缓冲区 m。

- 归并段数 $S = \lceil n/m \rceil$。
- $200/25 = 8$。

6. 同上，若 $n = 100, m = 5$，可产生多少个初始归并段?

答案： 【20】

7. 同上，若 $n = 100, m = 10$，可产生多少个初始归并段?

答案： 【10】

8. 假设计算机系统有 2048 个字节的磁盘块，要存储的每一条记录为 68 字节，其中数据占了 64 个字 节，关键码占 4 个字节。磁盘指针为 4 个字节。那么这个磁盘块上最多有多少条记录?

答案： 【30】

解决这个问题需要根据具体的**文件组织结构**或**索引结构**来分析。由于题目提到了 “磁盘指针”, 这通常对应以下几种不同的学术和应用场景。

下面列出最常见的几种计算情况:

情况一：该磁盘块作为 B 树（B-Tree）的结点

在 B 树中，一个结点若包含 n 条记录（Record），则必然包含 $n + 1$ 个指向子结点的磁盘指针（Pointer）。* **计算公式**:

$$n \times \text{记录大小} + (n + 1) \times \text{磁盘指针大小} \leq \text{磁盘块大小}$$

* **代入数据**:

$$n \times 68 + (n + 1) \times 4 \leq 2048$$

$$72n + 4 \leq 2048$$

$$72n \leq 2044$$

$$n \leq 28.38$$

* **结论**: 在这种情况下，该磁盘块最多可以存储 28 条记录（此时伴随 29 个指针）。

情况二：该磁盘块作为 B+ 树的叶子结点（或链式存储的磁盘块）

在 B+ 树中, 叶子结点用于存储实际的记录, 并且通常包含一个指向下

一个叶子结点的磁盘指针（用于支持范围查询）；或者在链式文件结构中，每个数据块需要一个指针指向下一个块。* **计算公式**：

$$n \times \text{记录大小} + 1 \times \text{磁盘指针大小} \leq \text{磁盘块大小}$$

* **代入数据**：

$$n \times 68 + 4 \leq 2048$$

$$68n \leq 2044$$

$$n \leq 30.05$$

* **结论**：在这种情况下，该磁盘块最多可以存储 30 条记录（此时占用 $30 \times 68 + 4 = 2044$ 字节，剩余 4 字节闲置）。

情况三：顺序存储（不需在块内保存指针）

如果磁盘块采用连续分配或索引分配，且块内部不需要包含任何指针（指针存在外部的索引表或 FAT 中），则所有空间均可用于存储记录。* **计算公式**：

$$n \times \text{记录大小} \leq \text{磁盘块大小}$$

* **代入数据**：

$$n \times 68 \leq 2048$$

$$n \leq 30.11$$

* **结论**：在这种情况下，最多可以存储 30 条记录（占用 2040 字节，剩余 8 字节闲置）。

总结

根据您所学课程（如《数据库系统》或《文件结构》）的具体上下文：* 如果是 B **树结点**，最多可存储 28 条记录。* 如果是 B+ **树叶子结点** 或 **链式数据块**，最多可存储 30 条记录。

9. 如图所示，进行五路归并的顺串的第一个记录的关键码分别是 14, 18, 9, 23, 77。请构造赢者树填 充其他节点，求该二叉树的广度优先周游序列。（注意：不包括已知的 5 个外部节点，各个数字之间 用空格分隔，结尾没有空格）

答案： 【3 3 4 1 / 3 3 4 1 / 3 3 4 1】

10. 假设计算机系统有 2048 个字节的磁盘块，要存储的每一条记录为 48 字节，其中数据占了 44 个字 节，关键码占 4 个字节。磁盘指针为 4 个字节。那么这个磁盘块上最多有多少条记录?

答案： 【42】

第十章 检索

1. **单选题：**平均查找长度与结点个数 n 无关的查找方法是：

A、散列查找 B、顺序查找 C、二分查找

D、没有这样的查找方法使得平均查找长度和 n 无关

答案： 【散列查找】

2. **单选题：**有一个散列表，共有 N 个槽，采用双散列探查的闭散列方法解决冲突。经过一系列插入操 作，当前散列表中有 M 个元素，负载因子 a 为 0.4，即 M/N=a=0.4。假设 M,N 都非常大，并且双散列 探查方法近似使得每一次探查的位置，可以近似为均匀分布（即等概率地探查每个槽）。当前对于某个 关键码，近似估算不成功检索的平均检索长度（）

A、2 B、1 C、4/3 D、5/3

答案： 【5/3】

对于采用**双散列探查**（Double Hashing）的闭散列表，当 M,N 都非常大时，双散列的探查序列可以近似为**均匀哈希**（Uniform Hashing）。

在均匀哈希假设下，不成功检索的平均探查次数公式为：

$$U(\alpha) = \frac{1}{1-\alpha} = \frac{1}{1-0.4} = \frac{5}{3}$$

补充对比：注意不要与线性探测混淆。线性探测在不成功检索时的平均探查长度为 $\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$，当 $\alpha = 0.4$ 时约为 2.89，远大于双散列的结果。双散列因为避免了" 聚集" 现象，性能更接近理论最优的均匀哈希。

3. **极值 ASL 计算**

单选题：

在包含 n 个关键码的线性表中进行顺序检索，若检索第 i 个关键码的概率为 Pi，且概率分布为：

$$P_1 = \frac{1}{2},\quad P_2 = \frac{1}{4},\quad \ldots,\quad P_{n-1} = \frac{1}{2^{n-1}},\quad P_n = \frac{1}{2^n}$$

求成功检索的平均检索长度（ASL）的极限（$n \to \infty$）：

A、1.5 B、2 C、3 D、1.99999999

答案： 【 2 】

$$P_i = \frac{1}{2^i},\quad i = 1, 2, 3, \ldots$$

此时总概率为：

$$\sum_{i=1}^{\infty} \frac{1}{2^i} = 1$$

是合法的概率分布。

成功检索的平均检索长度（ASL）

顺序检索中，若目标是第 i 个元素，则需比较 i 次（假设成功检索时恰好在第 i 步找到）。

因此，平均检索长度为：

$$\text{ASL}_n = \sum_{i=1}^{n} i \cdot P_i = \sum_{i=1}^{n} i \cdot \frac{1}{2^i}$$

题目要求的是 **当 $n \to \infty$ 时的极限**，即：

$$\lim_{n\to\infty} \sum_{i=1}^{n} \frac{i}{2^i} = \sum_{i=1}^{\infty} \frac{i}{2^i}$$

这是一个经典级数，其和为：

$$\sum_{i=1}^{\infty} \frac{i}{r^i} = \frac{r}{(r-1)^2}, \quad \text{当} |r| > 1$$

令 $r = 2$，则：

$$\sum_{i=1}^{\infty} \frac{i}{2^i} = \frac{2}{(2-1)^2} = 2$$

或者直接推导：

设 $S = \sum_{i=1}^{\infty} \frac{i}{2^i}$

则：

$$S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \cdots$$

$$\frac{1}{2}S = \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \cdots$$

相减得：

$$S - \frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots = 1 \Rightarrow \frac{1}{2}S = 1 \Rightarrow S = 2$$

4. **单选题：**折半查找有序表（4，6，10，12，20，30，50，70，88，100）。若查找表中元素 5 8，则它将依次与表中 () 比较大小，查找结果是失败

A、20，50 B、30，88，70，50 C、20，70，30，50 D、30，88，50

答案： 【20，70，30，50】

解析：查找 58。

1. 区间 $[0,9]$，中点 4（值为 20）。$58 > 20 \rightarrow$ 往右。
2. 区间 $[5,9]$，中点 7（值为 70）。$58 < 70 \rightarrow$ 往左。
3. 区间 $[5,6]$，中点 5（值为 30）。$58 > 30 \rightarrow$ 往右。
4. 区间 $[6,6]$，中点 6（值为 50）。$58 > 50 \rightarrow$ 失败。

5. **单选题：**对 22 个记录的有序表作折半查找，当查找失败时，至少需要比较 ( ) 次关键字。

A、5 B、3 C、4 D、6

答案： 【 4 】

6. **单选题：**给定关键码序列 26, 25, 20, 33, 21, 24, 45, 204, 42, 38, 29, 31，用散列法进行存储 (本 题采用闭散列方法解决冲突)，针对负载因子 $\alpha = 0.5$，请给出最合理的除余法的散列函数：

A、H(key)=key % 24 B、H(key)=key % 11 C、H(key)=key % 12 D、H(key)=key % 23

答案： 【H(key)=key % 23】

1. **确定散列表长度 m**
   - 关键码个数 $n = 12$
   - 负载因子 $\alpha = 0.5$
   - 由 $\alpha = n/m$ 得：$m = n/\alpha = 12/0.5 = 24$
2. **选择除余法的模数**

除余法 $H(key) = key \bmod p$ 中，p 的选择原则是：

- p 应为**不大于 m 的最大素数**
- p 不应接近 2 的幂次（避免低位决定散列值）
- p 不应与关键码的公共因子相关

不大于 24 的最大素数为 23。

> 23 是素数，且不与给定关键码序列中的数值有明显公因子关系，是最优选择。若取 $p = 24$（非素数），偶数关键码将全部映射到偶数槽位，导致严重聚集。若取更小的素数如 19，则表长浪费过多，实际负载因子变为 $12/19 \approx 0.63$，偏离题目要求。

7. **单选题：**针对负载因子 $\alpha = 0.6$，最合理的除余法散列函数为：

A、H(key)=key % 17 B、H(key)=key % 23 C、H(key)=key % 19 D、H(key)=key % 20

答案： 【H(key)=key % 19】

8. **单选题：**有两个整数的集合 A,B，大小分别为 n，m=O(log(n))，由顺序表存储，并且已经排好序，现在要求他们的交集，请问你设计的高效算法的复杂度是 ()

A、n B、$\log^2 n$ C、$\log n$ D、$n \log n$

答案： $\log^2 n$

9. **单选题：**在包含 n 个关键码的线性表里进行顺序检索，若检索第 i 个关键码的概率为 pi，pi 如下分 布：$p_i = 2^{-i}$, (1 <= i <= n) 求平均检索长度。

A、$2 - \frac{n+2}{2^{n-1}}$ B、$2 - \frac{n+2}{2^n}$ C、$2 - \frac{1}{2^n}$ D、$2 - \frac{1}{2^{n-1}}$

答案： $2 - \frac{1}{2^{n-1}}$

10. **单选题：**链表适用于 ( ) 查找。

A、顺序 B、二分法 C、顺序和二分法都适合 D、顺序和二分法都不适合

答案： 【顺序】

11. **多选题：**假定把关键码 K 散列到有 n 个槽 (从 0 到 n-1 编号) 的散列表中，散列表用开散列的冲突解决 策略。对于下面的每一个函数 h(K)，这个函数作为散列函数可以使得插入和检索操作一定能正常工作 的有 () 注：1. 函数 Random(n) 返回一个 0 到 n-1 之间的随机整数 (包含这两个数在内)。2. 不考虑散列 函数的性能，只考虑其正确性

A、h(k)=1 B、h(k)=k mod n, 其中 n 是一个素数 C、h(k)=k/n, 其中 k 和 n 都是整数 D、h(k)=(k + Random(n)) mod n

答案： 【h(k)=1; h(k)=k mod n, 其中 n 是一个素数】

12. 有一个 7 个槽的闭散列表 (槽从 0 到 6 编号)，使用散列函数 h(k) = k mod 7 和线性探查，依次插入 3、12、9、2。在插入值 2 的关键码之后，每一个空槽作为下一个被填充槽的概率是多少？依次输入 从 0 到 6 编号的槽的填充概率，用“,” 间隔。如 2/7,0,0,0,0,0,5/7

答案： 【1/7,1/7,0,0,0,0,5/7】

13. 一个散列表的散列函数是 h(key)=key%19，共有 20 个槽，用闭散列的线性探查方法。从空表开 始，依次进行如下插入删除操作，问这些操作的平均检索长度是 ()（用整数或分数表示）操作是：Ad d 26 Add 25 Add 24 Add 195 Del 26 Add 176 提示：1. 散列表中不能插入两个相同的关键码 2. 结果 请用一个最简分数数值表示，分号用/表示，例如四分之三写为：3/4

答案： 【13/6】

第十一章 索引

1. **单选题：**假定一个计算机系统有 4 096 字节的磁盘块，每个磁盘的磁盘号可以用一个四字节的整数表示。要存储的每一条记录中 4 个字节是关键码，64 个字节是数据字段。记录已经排序，顺序地存储 在磁盘文件中。我们建立一个稠密索引，该线性索引的结构为：(每个文件磁盘块的最小关键码，该 块磁盘的磁盘号)，通过线性索引访问磁盘文件中的记录。如果线性索引也存储在磁盘中（这样它的

大小仅受二级索引的限制），而且使用 4 096 个字节的二级索引，二级索引中的每个单元引用线性索 引的磁盘块中最小的关键码值。文件中最多可以存储多少条记录?（由于数字较大，可以用 K,M 作为 单位表示，如 32K）

A、128M B、30K C、256K D、15M

答案： 【15M】

2. **单选题：**假定一个计算机系统有 4 096 字节的磁盘块，每个磁盘的磁盘号可以用一个四字节的整数 表示。要存储的每一条记录中 4 个字节是关键码，64 个字节是数据字段。记录已经排序，顺序地存储 在磁盘文件中。我们建立一个稠密索引，该线性索引的结构为：（每个文件磁盘块的最小关键码，该 块磁盘的磁盘号），通过线性索引访问磁盘文件中的记录。如果线性索引的大小是 2MB。最多可以在 磁盘文件中存储多少条记录?（由于数字较大，可以用 K,M 作为单位表示，如 32K）

A、250K B、15M C、150K D、25K

答案： 【15M】

3. **单选题：**设有一个职工文件，并设该文件由教材中表 10-1 所示的 5 个记录组成，其中职工号为关键码。

磁盘地址	职工号	姓名	性别	职业	年龄	工资
A	39	张三	男	程序员	25	7000
B	50	王二	女	分析员	31	8500
C	10	李四	男	程序员	28	7500
D	75	丁一	女	终端操作员	18	5000
E	27	赵五	男	分析员	33	8000

如下结构是什么类型的索引?

性别	职工号关键字
男	39, 10, 27
女	50, 75
职业	职工号关键字
程序员	39, 10
分析员	50, 27
终端操作员	75

A、线性索引 B、多分树静态索引 C、动态索引 D、倒排索引

答案： 【倒排索引】

解析：将属性值（如性别、职业）作为主索引键，后面挂载具有该属性的所有记录的主键（职工号），这种设计称为**倒排索引**。

4. **单选题：**红黑树是一种扩充的二叉搜索树（BST)。给定一颗结点个数为 n 的红黑树在最坏的情况 下，红黑树的删除结点操作的时间复杂度是（)

A、$O(\log n)$ B、$O(n)$ C、$O(n^{\frac{1}{2}})$ $O(n^2)$

答案： $O(\log n)$

5. **单选题：**设有一个职工文件，并设该文件由教材中表 10-1 所示的 5 个记录组成，其中职工号为关键码。

磁盘地址	职工号	姓名	性别	职业	年龄	工资
A	39	张三	男	程序员	25	7000
B	50	王二	女	分析员	31	8500
C	10	李四	男	程序员	28	7500
D	75	丁一	女	终端操作员	18	5000
E	27	赵五	男	分析员	33	8000

如下结构是什么类型的索引?

10	C	27	E	39	A	50	B	75	D

答案： 【线性索引】

6. **多选题：**在什么情况下多分树静态索引比 B+ 树的实现更有效率?

A、在系统数据库不稳定，并且系统没有时间进行文件再组织的情况下 B、在插入和删除操作比较少的情况下 C、在系统允许较频繁的文件再组织的情况下 D、在系统数据较稳定，并且需要支持高效的并行查找的情况下 E、在插入删除操作较多的情况下

答案：【在插入和删除操作比较少的情况下；在系统允许较频繁的文件再组织的情况下；在系统数据较稳定，并且需要支持高效的并行查找的情况下】

7. 假设按如下的方法修改从 B 树中删除元素的方式：如果一个结点既有最相邻的左兄弟也有最相邻的 右兄弟，那么在合并前对两个兄弟都要作检查。从一棵高度为 5 的 B 树中删除元素时需要的最大磁盘 访问次数？注：一般而言，B 树的层次都很少，查找 B 树路径中的结点能放在内存中，不必重复访问磁 盘读取

答案： 【18】

8. 假设按如下的方法修改从 B 树中删除元素的方式：如果一个结点既有最相邻的左兄弟也有最相邻的 右兄弟，那么在合并前对两个兄弟都要作检查。从一棵高度为 4 的 B 树中删除元素时需要的最大磁盘 访问次数？注：一般而言，B 树的层次都很少，查找 B 树路径中的结点是否能放在内存中，不必重复访 问磁盘读取。

答案： 【14】

9. 设有一颗阶 $m = 3$ 的 B 树，如图所示：

其中 a, b, ..., g 是结点的名称，系统一块可以动态分配的结点叫 h。可在说明插入过程时使用，结点内的整数为关键码。若在图中所示的 B 树中插入关键码 55，请计算完成该插入所需要的访外次数（包括读磁盘和写磁盘）。

答案： 【6】

10-12. B+ **树容量计算**

10. 假定有一个 B+ 树，它的内部结点可以存储多达 100 个子女，叶结点可以存储多达 15 条记录（本题 中的 B+ 树把所有记录存放在叶结点上）。对 4 层的 B+ 树，能够存储的最小记录数和最大记录数是多 少？（用逗号隔开，格式如：0,15）

答案： 【40000,15000000】

解析：

内部结点分支数：最小 $\lceil m/2 \rceil = 50$，最大 $m = 100$。

叶子结点记录数：最小 $\lceil L/2 \rceil = 8$，最大 $L = 15$。

对于高度为 h 的 B+ 树，其叶子节点数量范围可根据内部节点的分支上下界指数级计算，最后乘以叶结点的存储记录数，即可得到最小与最大记录容量。

分析 B+ 树的结构参数

- **内部结点**：最多 100 个子女 → 阶数 $m = 100$
  - 最大子女数：100
  - 最小子女数（除根外）：$\lceil m/2 \rceil = 50$
  - 根结点最小子女数：2（当树高 $\geqslant$ 2 时）
- **叶结点**：最多存储 15 条记录
  - 最大记录数：15
  - 最小记录数：$\lceil 15/2 \rceil = 8$（B+ 树叶结点通常也遵循半满规则）
- **树高**：4 层（根为第 1 层，叶结点为第 4 层）

最大记录数

每一层都取最大值：

- 第 1 层（根）：1 个结点，100 个子女
- 第 2 层：100 个结点，每个 100 个子女 → $100^2 = 10{,}000$
- 第 3 层：$100^2 = 10{,}000$ 个结点，每个 100 个子女 → $100^3 = 1{,}000{,}000$
- 第 4 层（叶）：$100^3 = 1{,}000{,}000$ 个叶结点，每个存 15 条记录

$$\text{最大记录数} = 100^3 \times 15 = 15{,}000{,}000$$

最小记录数

每一层都取最小值：

- 第 1 层（根）：1 个结点，最少 2 个子女
- 第 2 层：2 个结点，每个最少 50 个子女 $\rightarrow 2 \times 50 = 100$
- 第 3 层：100 个结点，每个最少 50 个子女 $\rightarrow 100 \times 50 = 5{,}000$
- 第 4 层（叶）：5,000 个叶结点，每个最少存 8 条记录

$$\text{最小记录数} = 2 \times 50 \times 50 \times 8 = 40{,}000$$

11. 假定有一个 B+ 树，它的内部结点可以存储多达 100 个子女，叶结点可以存储多达 15 条记录（本题 中的 B+ 树把所有记录存放在叶结点上）。对 2 层的 B+ 树，能够存储的最小记录数和最大记录数是多 少？（用逗号隔开，格式如：0,15）

答案： 【16,1500】

12. 假定有一个 B+ 树，它的内部结点可以存储多达 100 个子女，叶结点可以存储多达 15 条记录（本题 中的 B+ 树把所有记录存放在叶结点上）。对 3 层的 B+ 树，能够存储的最小记录数和最大记录数是多 少？（用逗号隔开，格式如：0,15）

答案： 【800,150000】

第十二章 高级数据结构（上）

1. **多选题：** 下列关于十字链表的表述正确的有：

A、十字链表的节点只需要记录非零元素的值，不需要记录它们在矩阵中的位置。B、一个全由非零元素组成的矩阵，若使用十字链表表示，也将获得效率的提升。C、十字链表的每个节点只有一个指向后继元素的指针。D、应用十字链表做矩阵乘法时，时间复杂度是 O((ta + tb) * p * n)。（假设矩阵 A 乘以矩阵 B，A 为 p*m* 的矩阵，*B* 为 *m*n 的矩阵，A 中行向量的非零元素个数最多为 ta，B 中列向量的非零元素个数最多 为 tb）E、十字链表的节点记录了非零元素的值及它们在矩阵中的位置。F、十字链表可以应用于稀疏矩阵的表示

答案： D、E、F

根据十字链表（Orthogonal List）的定义和特点，解析说明：

- D **正确**：在矩阵乘法 $C = A \times B$ 中，计算结果矩阵 C 的任一个元素 $C[i][j]$ 需要对矩阵 A 的第 i 行和矩阵 B 的第 j 列进行内积计算。使用十字链表时，可以通过同时遍历 A 的第 i 行单链表（最多 t_a 个非零元素）和 B 的第 j 列单链表（最多 t_b 个非零元素）来寻找列标和行标相等的项进行相乘，该过程的时间复杂度为 $O(t_a + t_b)$。因为结果矩阵 C 的大小为 $p \times n$，共需计算 $p \times n$ 个元素，所以总的时间复杂度为 $O((t_a + t_b) \times p \times n)$。
- E **正确**：十字链表中的非零元素节点通常包含五个域：行号（row）、列号（col）、值（value）、指向同行下一个节点的指针（right）和

指向同列下一个节点的指针（down）。因此它记录了非零元素的值及其在矩阵中的位置。

- F **正确**：十字链表是稀疏矩阵的一种经典链接存储结构，特别适用于矩阵非零元素的位置或个数经常发生变化的情况（例如稀疏矩阵的加法、乘法等运算）。

错误选项分析：

- A **错误**：十字链表的节点必须记录非零元素在矩阵中的行号和列号，否则无法在线性链表结构中还原矩阵的二维空间结构，也无法进行高效的行列定位。
- B **错误**：对于一个全由非零元素组成的稠密矩阵，使用十字链表不仅会因为存储大量的行列索引和指针（right, down）而导致空间开销大幅增加，还会因为链表遍历（指针悬挂与跳转）导致访问效率低于直接使用二维数组。
- C **错误**:十字链表的每个非零节点至少有两个指针域,分别指向同行中的下一个非零元素（right）和同列中的下一个非零元素（down）。

2. **多选题：**以下可重入表中哪些是循环表?

A、(L1: (L2: (L1, a))) B、D(A:(c), B:(e), C:(a, L:(b, A, d))) C、(L: (a, L)) D、(L1: (a, b), (L1, c, L2:(d)), (L2, e, L3:(f, g)), L3) E、(L1: (a, b, L2(x, y)), L2, L3: (s, t, L4: (q, L5:(w, L3, v), r))) F、(x1, (y1, (a1, a2), y3), x3, (z1, z2))

答案： 【`(L1: (L2: (L1, a))); (L: (a, L)); (L1: (a, b, L2(x, y)), L2, L3: (s, t, L4:(q, L5:(w, L3, v), r)))`】

3. 图、线性表、可重入表、纯表的关系是（由小到大包含）:

答案： 【线性表 纯表 可重入表 图】

解析：

- 线性表是一对一的纯表。
- 纯表（无环且无共享子表）包含于可重入表（允许共享子表和环）。
- 可重入表属于广义的有向图结构。

4-6. **稀疏因子**

4. 在 3x6 的矩阵中有 9 个非零元素，则稀疏因子为:

答案： 【`0.5`】

解析：稀疏因子 $\delta = \frac{t}{m \times n}$（非零元素个数 / 总元素个数）。

- 3×6 矩阵有 9 个非零：$9/18 = 0.5$。
- 5×5 矩阵有 2 个非零：$2/25 = 0.08$。

5. 在 5x6 的矩阵中有 6 个非零元素，则稀疏因子为:

答案： 【`0.2`】

6. 在 5x5 的矩阵中有 2 个非零元素，则稀疏因子为:

答案： 【`0.08`】

7. 现在有若干个有序的空闲块：1500,1000,500,2000,1200,500，有一个请求序列：500,600,100 0,400,500,1000,2000，请问在最先适配、最佳适配、最差适配的策略下分配失败的次数分别是多 少？（答案为 3 个数字，用空格分隔）

答案： 【1 0 2】

在最先适配（First Fit）、最佳适配（Best Fit）和最差适配（Worst Fit）策略下，分配失败的次数分别为：

1 0 2

详细推导过程：

初始空闲块序列：[1500, 1000, 500, 2000, 1200, 500] 请求序列：500, 600, 1000, 400, 500, 1000, 2000

1. **最先适配** (First Fit)

从头开始扫描，选择第一个满足大小要求的空闲块。若有剩余则分割。* **请求** 500：选择 1500（第 1 块），剩余 1000。空闲块变为：[1000, 1000, 500, 2000, 1200, 500] * **请求** 600：选择 1000（第 1 块），剩余 400。空闲块变为：[400, 1000, 500, 2000, 1200, 500] * **请求** 1000：选择 1000（第 2 块），剩余 0。空闲块变为：[400, 500, 2000, 1200, 500] * **请求** 400：选择 400（第 1 块），剩余 0。空闲块变为：[500, 2000, 1200, 500] * **请求** 500：选择 500（第 1 块），剩余 0。空闲块变为：[2000, 1200, 500] * **请求** 1000：选择 2000（第 1 块），剩余 1000。空闲块变为：[1000, 1200, 500] * **请求** 2000：无足够大小的空闲块。**（失败 1 次）**

最先适配失败次数： 1

---

2. **最佳适配** (Best Fit)

扫描整个空闲块列表，选择满足要求的、且大小最小的空闲块。* **请求** 500：选择最小的 500（第 3 块），剩余 0。空闲块变为：[1500, 1000, 2000, 1200, 500] * **请求** 600：选择满足条件的最小块 1000（第 2 块），剩余 400。空闲块变为：[1500, 400, 2000, 1200, 500] * **请求** 1000：选择满足条件的最小块 1200（第 4 块），剩余 200。空闲块变为：[1500, 400, 2000, 200, 500] * **请求** 400：选择最小的 400（第 2 块），剩余 0。空闲块变为：[1500, 2000, 200, 500] * **请求** 500：选择最小的 500（第 4 块），剩余 0。空闲块变为：[1500, 2000, 200] * **请求** 1000：选择满足条件的最小块 1500（第 1 块），剩余 500。空闲块变为：[500, 2000, 200] * **请求** 2000：选择唯一的 2000（第 2 块），剩余 0。分配全部成功。

最佳适配失败次数： 0

---

3. **最差适配** (Worst Fit)

扫描整个空闲块列表，选择当前最大的空闲块。* **请求** 500：选择最大

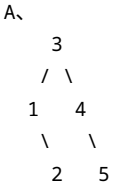
的 2000（第 4 块），剩余 1500。空闲块变为：[1500, 1000, 500, 1500, 1200, 500] * **请求** 600：选择当前最大的 1500（第 1 块），剩余 900。空闲块变为：[900, 1000, 500, 1500, 1200, 500] * **请求** 1000：选择当前最大的 1500（第 4 块），剩余 500。空闲块变为：[900, 1000, 500, 500, 1200, 500] * **请求** 400：选择当前最大的 1200（第 5 块），剩余 800。空闲块变为：[900, 1000, 500, 500, 800, 500] * **请求** 500：选择当前最大的 1000（第 2 块），剩余 500。空闲块变为：[900, 500, 500, 500, 800, 500] * **请求** 1000：当前最大空闲块为 900，小于 1000。**（失败 1 次）** * **请求** 2000：当前最大空闲块为 900，小于 2000。**（失败 2 次）**

最差适配失败次数： 2

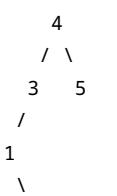
第十二章 高级数据结构（下）

1. **单选题：** 操作序列 < 插入 2，插入 5，插入 6，插入 4，插入 1，插入 3，删除 6> 形成哪棵 Splay 树?

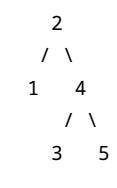
A、



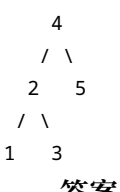
B、



C、



D、



答案： B

为了确定操作序列 < 插入 2，插入 5，插入 6，插入 4，插入 1，插入 3，删除 6> 最终形成的 Splay 树，我们可以逐步模拟每一次操作。

在 Splay 树中：1. **插入操作**：先按照普通二叉搜索树（BST）的方法插入节点，然后通过旋转（Splay）将该新插入的节点旋转到根节点。2. **删除操作**：先查找到该节点并将其 Splay 到根节点，然后将其删除。此时会分裂成左子树 L 和右子树 R：- 如果 R 为空，则新根为 L。- 如果 L 且 R 均不为空，通常在左子树 L 中找到最大元素并将其 Splay 到 L 的根部（此时该新根没有右子树），然后将 R 接到该新根的右子树上。

下面是详细的演变步骤：

第一步：插入 2

树为空，直接插入 2 作为根节点。

```
  2
```

第二步：插入 5

1. BST **插入**：5 大于 2，插入为 2 的右孩子。
2. Splay 5：对 5 进行一次单旋转（Zig），使 5 成为根。

```
  5
 /
2
```

第三步：插入 6

1. BST **插入**：6 大于 5，插入为 5 的右孩子。
2. Splay 6：对 6 进行一次单旋转（Zig），使 6 成为根。

```
    6
   /
  5
 /
2
```

第四步：插入 4

1. BST **插入**：4 小于 6，小于 5，大于 2，插入为 2 的右孩子。

```
    6
   /
  5
 /
2
 \
  4
```

2. Splay 4：
   - 4 的父节点是 2，祖父节点是 5。这是一次 Zig-Zag（双旋转）操作。
     – 先将 4 左旋：

```
    6
   /
  5
 /
4
 /
2
```

     – 再将 4 右旋：

```
  6
 /
4
 / \
2   5
```

   - 此时 4 是根节点 6 的左孩子。进行一次 Zig（单旋转）操作，将 4 右旋至根部：

```
   4
  / \
 2   6
    /
   5
```

第五步：插入 1

1. BST **插入**：1 小于 4，小于 2，插入为 2 的左孩子。

```
    4
   / \
```

```
   2   6
  /   /
 1   5
```

2. Splay 1：
   - 1 的父节点是 2，祖父节点是 4。这是一次 Zig-Zig（同向双旋转）操作。
     - 按照 Splay 规则，先旋转祖父节点（将 2 右旋）：

```
  2
 / \
1   4
   / \
  空  6
     /
    5
```

     - 再旋转 1（将 1 右旋）：

```
1
 \
  2
   \
    4
     \
      6
     /
    5
```

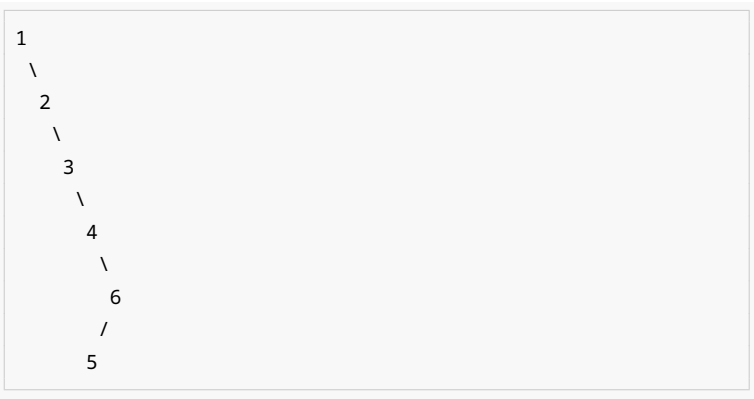
第六步：插入 3

1. BST **插入**：3 大于 1，大于 2，小于 4，插入为 4 的左孩子。

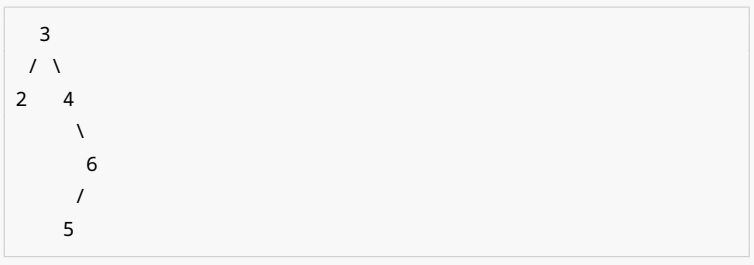
```
1
 \
  2
   \
    4
   / \
  3   6
     /
    5
```

2. Splay 3：

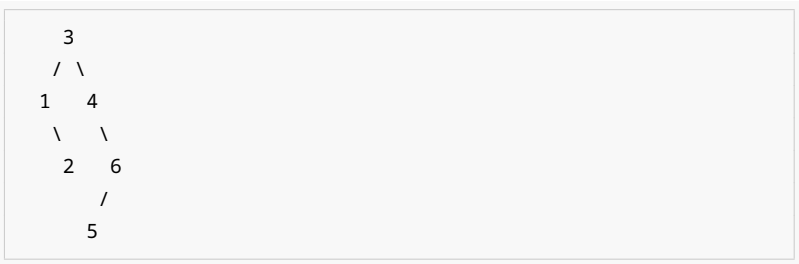
- 3 的父节点是 4，祖父节点是 2。这是一次 Zig-Zag（双旋转）操作。
  - 先将 3 右旋：



  - 再将 3 左旋：



- 此时 3 是根节点 1 的右孩子。进行一次 Zig（单旋转），将 3 左旋至根部：



第七步：删除 6

1. 将 6 Splay 到根节点

- 6 的父节点是 4，祖父节点是 3。这是一次 Zig-Zig（同向双旋转）操作。
  - 先旋转祖父节点（将 4 左旋）：

```
      4
     / \
    3   6
   /   /
  1   5
   \
    2
```

– 再旋转目标节点 6（将 6 左旋）：

```
      6
     /
    4
   / \
  3   5
 /
1
 \
  2
```

2. **移除根节点** 6

移除 6 后，分裂为两个子树：- **左子树** L：`text 4 / \ 3 5 / 1 \ 2` - **右子树** R：为空（`null`）。
根据 Splay 树删除合并（Join）的不同实现约定，最终结果有以下两种常见处理方式：

选项 A：标准/高效实现方式（若有一侧为空，直接返回另一侧）

在大多数实际代码实现中，如果右子树 R 为空，则无需再对左子树进行任何操作，直接将左子树 L 作为最终的 Splay 树。
最终形成的 Splay 树为：

```
    4
   / \
  3   5
 /
1
 \
  2
```

选项 B：严格理论步骤方式（无论如何都将左子树最大值 Splay 到左子树根部）

在某些理论教材中，删除根节点后，即使右子树为空，也会将左子树 L 中的最大元素（5）Splay 到 L 的根部。- 在 L 中，5 是 4 的右孩子，进行一次 Zig（单旋转）左旋操作，使 5 成为新根：text

```
5
/       4       /       3       /    1     \      2
```

最终形成的 Splay 树为：

```
      5
     /
    4
   /
  3
 /
1
 \
  2
```

（注：具体取决于您所学教材或课程对 “右子树为空时是否仍对左子树执行 Splay 最大值” 的定义。通常，**选项** A 是最通用的程序实现结果，而**选项** B 在部分纯理论推导中较为常见。）

2. **单选题：**另一组操作序列形成的 Splay 树：

```
A、
      10
     /
    9
   /
  6
   \
    7
     \
      8
B、
    10
   /
  7
 / \
6   8
     \
      9
C、
      10
     /
    8
```

```
   / \
  7   9
 /
6
```

D、

```
    9
   / \
  7   10
 / \
6   8
```

答案：A

3. **单选题：**哪个插入顺序可以得到这棵 AVL 树：

```
     4
    / \
   3   6
  /   / \
 1   5   15
```

A、6,15,3,5,4,1 B、15,3,5,6,4,1 C、6,15,5,3,1,4 D、6,15,3,5,1,4

答案： 【6,15,3,5,4,1】

4. **单选题：**哪个插入顺序可以得到给定的 AVL 树：

```
     9
    / \
   3   14
  / \    \
 2   6    15
```

答案： 【3,14,15,9,2,6】

5. **多选题：**下列关于 Trie 树和后缀树的说法中正确的有：

A、通用后缀树可以用来查找两个字符串的公共子串。B、用 Trie 树存储的单词词典，检索一次的代价与单词的长度相关。C、用 Trie 树存储的单词词典，检索一次的代价与词典中单词的数量相关。D、利用后缀树可以方便地查找字符串的子串。

答案：A、B、D

6. 字符串 “ladygaga” 的 LCP 数组是?（答案是 7 个数字，请用空格分隔，首位无空格）

答案： 【1 1 0 0 2 0 0】

字符串 “ladygaga” 的 LCP（最长公共前缀）数组对应的 7 个数字为：

1 1 0 0 2 0 0

计算过程说明：

1. **写出所有后缀：**
   - 后缀 0: ladygaga
   - 后缀 1: adygaga

- 后缀 2: dygaga
- 后缀 3: ygaga
- 后缀 4: gaga
- 后缀 5: aga
- 后缀 6: ga
- 后缀 7: a

2. **将后缀按字典序排序（后缀数组 SA）：**
   1. a （后缀 7）
   2. adygaga （后缀 1）
   3. aga （后缀 5）
   4. dygaga （后缀 2）
   5. ga （后缀 6）
   6. gaga （后缀 4）
   7. ladygaga （后缀 0）
   8. ygaga （后缀 3）
3. **计算相邻后缀的最长公共前缀（LCP）长度：**
   - LCP(a, adygaga) = 1 （公共前缀为 “a”）
   - LCP(adygaga, aga) = 1 （公共前缀为 “a”）
   - LCP(aga, dygaga) = 0
   - LCP(dygaga, ga) = 0
   - LCP(ga, gaga) = 2 （公共前缀为 “ga”）
   - LCP(gaga, ladygaga) = 0
   - LCP(ladygaga, ygaga) = 0

7. 字符串 “analysis” 建立后缀树后，一共有 ____ 个节点。

答案： 【10】

对字符串 "analysis" 建立后缀树，节点的数量取决于**是否在字符串末尾添加结束符（如 $）**以及**对非叶子节点结束的后缀的处理方式**。

在计算机科学中，最标准、最常用的做法是**在字符串末尾添加一个唯一的结束符** $（即构建 "analysis$" 的后缀树），以确保所有后缀都落在叶子节点上。以下是不同定义下的节点计算：

情况一：标准定义（添加结束符 $，即 "analysis$"）

字符串 "analysis$" 长度为 9，其所有的后缀共有 9 个：1. analysis$ 2. nalysis$ 3. alysis$ 4. lysis$ 5. ysis$ 6. sis$ 7. is$ 8. s$ 9. $

1. **确定叶子节点**（Leaves）

由于每个后缀都以独一无二的 $ 结尾，因此每个后缀都会对应一个唯一的叶子节点。* **叶子节点数** = 9 **个**

2. **确定内部节点（Internal Nodes，分支节点）**

内部节点产生于两个或多个后缀拥有公共前缀的地方。我们来寻找这些公共前缀：* 以 'a' 开头的后缀有：analysis$ 和 alysis$。它们的公共前缀是 "a"，在其后分别接 "nalysis$" 和 "lysis$"（首字母 'n' 和 'l' 不同，在此处产生分支）。因此存在一个代表 "a" 的**内部节点**。* 以 's' 开头的后缀有：sis$ 和 s$。它们的公共前缀是 "s"，在其后分别接 "is$" 和 "$"（首字母 'i' 和 '$' 不同，在此处产生分支）。因此存在一个代表 "s" 的**内部节点**。* 其他字符开头的后缀均只有一个，不产生分支。
因此：* **内部节点数** = 2 **个**（分别代表前缀 "a" 和 "s"）

3. **根节点（Root）**

- **根节点数** = 1 **个**

4. **总节点数**

$$\text{总节点数} = \text{根节点}(1) + \text{内部节点}(2) + \text{叶子节点}(9) = 12 \text{ 个}$$

情况二：非标准定义（不添加结束符，即直接对 "analysis" 建树）
如果不添加结束符，后缀 "s" 是后缀 "sis" 的前缀。根据不同教材的定义，有两种情况：

1. **若不允许度为 1 的节点（即隐式表示）**：由于 "s" 是 "sis" 的前缀，它会隐式地落在走向 "sis" 的边上，不单独占用节点。
   - 叶子节点有 7 个（analysis, nalysis, alysis, lysis, ysis, sis, is）
   - 内部节点有 1 个（"a"）
   - 根节点 1 个
   - **总节点数** = 9 **个**
2. **若强制让每个后缀都显式对应一个节点**：虽然 "s" 没有分叉，但为了表示这是一个后缀的结束，会为其建立一个度为 1 的内部节点。
   - **总节点数** = 10 **个**（9 个节点的基础上，加上代表后缀 "s" 的显式节点）

总结

- 如果是标准的算法或考试题目（默认带结束符 $），一共有 12 个节点。

- 如果是不带结束符的简化版后缀树，一共有 9 或 10 个节点。

原书勘误

《数据结构与算法》（张铭、王腾蛟、赵海燕，高等教育出版社 2008）印刷清单里，**按原样进编译器会失败，或对拍后算法结果确定是错的**条目。每条都指向某个单元的 `legacy.md`，那里有命令和真实输出。

不收录：OCR 把 `}` 认成 `1`、把 `++` 拆成 `+ +`、全角分号——那是扫描损伤，不是原书设计。也不收录「用了 `cout`」「没用 `optional`」这类现代化取舍。

底稿 `dsa_raw.md` 只读，本表是给对照纸书阅读的人用的。

本表每一条都在 `collab/errata.json` 里有编号，「跑起来是错的」那几条**各自对应一句 会变红的断言**——实现要是退回原书那种写法，测试当场失败。映射由 `tools/errata.py` 校验，已接进闸门；「已吸收」不再是一句需要人相信的话。

怎么读

「印刷即错」指：把 OCR 噪声修回之后，逻辑或语法仍然不能通过现代 C++ 编译器，或与书中自己的约定对拍失败。现代实现写在 `code/`，书稿 `book/` 印的是那份能跑的代码。

编译不过

清单	错在哪	证据
代码 2.1 / 代码 2.2 / 算法 2.5	成员函数名叫 `delete`，这是关键字	`error: expected unqualified-id before 'delete'` · array_list
代码 2.1	`class List` 通篇没有 `public:`，ADT 的每个运算都是私有的	`error: 'void List<T>::clear()' is private` · 同上
算法 2.3	循环上界 `n` 从未声明	`error: 'n' was not declared in this scope` · 同上
代码 2.6	`const Link*` 赋给 `Link* next`，丢弃 const	`error: assigning to 'Link<int> *' from 'const Link<int> *'` · linked_list

清单	错在哪	证据
代码 3.2	`int top` 与 `bool top(T&)` 重名，整类编译不过	`error: ‘bool arrStack<T>::top(T&)’ conflicts with a previous declaration` · array_stack
算法 3.3	循环变量 `i` 从未声明	`error: ‘i’ was not declared in this scope` · 同上
代码 3.4	又是 `Link<T>* top` 与 `bool top(T&)` 重名	与代码 3.2 同一处错误，两个存储结构都犯· linked_stack
算法 3.5 / 算法 3.9	`s.pop(&tmp)` 传指针，本书自己的栈 ADT 要的是引用	`error: cannot convert ‘double*’ to ‘double&’` · expression_eval
算法 3.11	`enum rdType {0,1,2}`（枚举值不能是字面量）；`public class knapNode`（Java 语法，出现两次）	`error: expected identifier before numeric constant` / `error: expected unqualified-id before ‘public’` ·knapsack
算法 3.12	同一个 `stack.top` 既当数据成员又当成员函数——**依赖代码 3.2/3.4 的重名缺陷才可能存在**	knapsack
算法 4.4	`assert(str != '\0')` 是指针与字符比较	`error: ISO C++ forbids comparison between pointer and integer` · string_class
算法 4.4	`String(char* s)` 使书中自己的例子 `String s1 = "Hello"` 在 C++11 起非法	字符串字面量是 `const char*` ·同上
代码 10.1	成员初始化写 `key`，数据成员却是 `Key`	`error: member initializer 'key' does not name a non-static data member` · search_hash

能编译、跑起来是错的

清单	错在哪	证据
算法 3.6 / 3.8 / 3.9	阶乘不查溢出：`factorial(21)` 返回负数，`factorial(66)` 返回 0	UBSan：`signed integer overflow: 21 * 2432902008176640000` · recursion_and_stack
算法 3.6	负数静默返回 1	定义域错误，不是「空阶乘」·同上
算法 4.5	越界 `return NULL`，随后 `strlen(nullptr)`	运行期 SEGV · string_class
算法 4.6 / 算法 4.8	0 起始下标下返回 `j - pLen + 1`，**一律差 1**	四面数据对拍 `std::string::find`，含图 4.12 自己的例子· pattern_matching
算法 10.9	散列表析构写 `delete HT`，`HT` 是数组	应为 `delete[]` · search_hash

书内自相矛盾

位置	矛盾	说明
算法 3.5 正文 vs 代码	正文自己说 `operand1 == 0.0` 判断除零是错的，印出来的代码照旧	见第 3 章表达式一节
代码 3.1 vs 算法 3.5 / 3.9	栈 ADT 是 `pop(T&)`，算法写成 `pop(&x)`	两处清单从未放进同一个翻译单元
第 4.3 节 vs 算法 4.6 / 4.8	正文明写下标从 0 起，返回值却按 1 起始算	图 4.12 只画过程、没给返回值，所以印出来没人发现

正文说法有误

代码之外、正文里的概念性错误。它们不对应某条清单，但照着字面去写代码会得到与书中所说不同的结果。

位置	错在哪	证据
第 4.2.1 节，图 4.1 之后	「C++ 中的字符数组是用字符指针定义的」：数组不是指针。对图 4.1 的 `char s1[12]`、`char s2[8]`，`s1 = s2` 不是「悄悄没复制内容」，而是**编译不过**；只有 `char*` 才会编译通过并只复制地址	`error: array type 'char[12]' is not assignable`（clang++）·第 4 章 4.2.1 节
第 4.3.1 节，朴素匹配的平均代价	「运用马尔科夫链可以估算一个更好的比较次数 $2^{m+1}-2$」（m 为模式长度），通式 $(a^{m+1}-a)/(a-1)$（a 为字符种数）也称「平均比较次数」：这两个式子算的是**模式在随机目标串里首次出现的期望位置**，与目标串长度无关、随 m 指数增长，谈不上比「平均不到两倍目标串长」更好；且只对 `aa…a` 这类自身重叠最多的模式成立。同段「比较 m 次的概率为 2^{-m}」在比满整个模式时漏了配串成功的情形，应为 $2^{-(m-1)}$（结论「每趟不到 2 次」不变）	两状态马尔科夫链手算：`"00"` 期望位置 6、`"01"` 为 4；随机模拟 `"000"` 14.0、`"001"` 8.0（公式都给 14）·第 4 章 4.3.1 节
第 1.3 节，算法简史	Ada 写的是计算**伯努利数**的程序，原书作「伯努利方程」；时间应为 1842—1843 年	史实·第 1 章 1.2 节
第 1.4.1 节，性质 5	「$\log_a n$ 是 $O(\log_b n)$」只要求 $b \neq 1$，漏了 $a \neq 1$——底数为 1 的对数无定义	第 1 章 1.4.1 节

位置	错在哪	证据
第 1.4.1 节，$2^{1000}\,\mu s$	约 3.4×10^{287} 年，原书作 10^{286} 年（差一个数量级，结论「不可能算完」不变）	$2^{1000}/(10^6\cdot3.156\times10^7)$ ·第 1 章
第 1.4 节，表 1.2	「地球自转 10^6 m/s」「卫星 10^7 m/s」量级不对：赤道自转约 465 m/s，近地卫星约 7.9×10^3 m/s	第 1 章表 1.2 后补注
第 1.4.2 节，最佳/最差情况例	升序输入时外层循环的 i 取 $0..n-2$（条件是 `i < n-1`），原书写到 $n-1$	第 1 章 1.4.3 节
第 2.1 节，线性表的关系 r	「关系 r 具有反对称性和传递性」：$r=\{\langle k_i,k_{i+1}\rangle\}$ 只含相邻对，本身不传递；传递的是它的传递闭包「排在前面」	$\langle k_0,k_1\rangle,\langle k_1,k_2\rangle\in r$ 而 $\langle k_0,k_2\rangle\notin r$ ·第 2 章 2.1 节
第 2.3.2 节，双链表删除 / 插入	删除的三行与插入的四行都默认 `p` 有后继；按图 2.9 只有头结点、无尾哨兵，`p` 为表尾时会对空指针解引用	第 2 章图 2.11、图 2.12 说明；教学版 `erase_node` / `insert` 分支处理
第 3.2.2 节，顺序队列	正文说「rear 存队尾元素的位置，front 与 rear 相同表示只有一个元素」，而代码 3.14 的 rear 是下一个空位、`front == rear` 表示空	书内前后不一致·第 3 章 3.2.2 节按代码统一
第 4.1.1 节，ASCII	「第 0～32 号及 127 号共 34 个控制字符」：32 号是空格，属可打印字符；控制字符是 0～31 与 127 共 33 个	ASCII 标准·第 4 章 4.1 节

位置	错在哪	证据
第 4.2.1 节，图 4.1	「`s3` 没给初值，存的就是空串」：只有全局或 `static` 数组零初始化；局部数组不初始化时内容不确定，读它是未定义行为	第 4 章 4.2.1 节
第 5.5 节，堆中删除任意元素	正文只说与子结点比较交换（下沉）；补到该位置的末元素可能比新父结点小，还需上浮。代码 5.11 的 `Remove` 处理了这种情况，是文字漏写	反例：堆 `1 10 2 11 12 3 4` 删除下标 3 ·第 5 章
第 6 章习题 1	关系 R 中 $\langle D, J\rangle$ 与 $\langle I, J\rangle$ 使 J 有两个父结点，M 不出现在任何关系里，这个 R 不构成树	扫描件第 158–159 页·第 6 章习题 1 题面加注
第 6 章图 6.20	图注印「习题 10 的图示」，引用它的是习题 11	第 6 章已改为习题 11
第 7.5.2 节，Floyd 的 path 规则	正文「当 k 使 adj[i,j] 达到最小值，就置 path[i,j]=k」与算法 7.9 的代码 `D[i][j].pre = D[v][j].pre` 不一致；按正文记录，多段中转的路径会还原错	反例：弧 v0 → v2 → v1 → v3，正文规则得 path[0][3]=2，应为 1 ·第 7 章 7.5.2 节
第 7 章本章小结	「求有向图顶点的**出度**时相邻矩阵更方便」应为**入度**：出边表里出度就是表长，入度才要扫全部边表	第 7 章本章小结
第 8.4.2 节，快速排序	「平均需要 Θ(log n) 次分割」：Θ(log n) 是递归深度；分割**次数**总是 Θ(n)	第 8 章 8.4.1 节

位置	错在哪	证据
第 8.5 节，归并小区间优化	正文说小区间「最后对整个序列使用插入排序」，但归并要求两半各自有序，这样做行不通；算法 8.9 自己的代码是逐段插入	书内前后不一致·第 8 章 8.5 节
第 8.6.2 节，扑克牌排序码	「按照桥牌规则……黑桃 (S) > 红心 (H) > 梅花 (D) > 方片 (C)」：字母配反（D 是方片 Diamonds、C 是梅花 Clubs），且桥牌里方片高于梅花	扫描件第 218 页·第 8 章 8.6.2 节
第 8 章上机题 3、图 8.4(c)	上机题 3 写「算法 8.8 中的快速排序」，快排是算法 8.6；图 8.4(c) 那一行开头多印了一个 78	第 8 章上机题 3、图 8.4 题注
第 9.3.3 节，选择树比较次数	$\lceil \log_k m \rceil \cdot \lceil \log_2 k \rceil \cdot (n-1) = \lceil \log_2 m \rceil \cdot (n-1)$ 不成立，两次向上取整不能抵消；只有主项 $\log_2 m \cdot (n-1)$ 与 k 无关	反例 $m = 5, k = 4$：左边 4，右边 3·第 9 章 9.3.3 节
第 9 章本章小结	把「逻辑文件」列入物理组织结构，与 9.2 节「逻辑文件 / 物理文件」的区分自相矛盾	第 9 章小结已改为顺序/散列/索引/倒排文件
第 10 章习题 1	设 $p_n = 1/2^n$，但 $\sum_{n=1}^{N} 2^{-n} = 1 - 2^{-N} \neq 1$，这组概率之和不为 1	扫描件确认·习题答案给出两种口径的 ASL
第 11.4 节，B 树的性能分析	「$N = 1999998,\ m = 199$ 时 I 至多等于 4」：按同页公式 $I \le \log_{100}(999999.5) + 1 \approx 3.9999$，I 至多为 3（$N \ge 1999999$ 才可能有 4 层）	第 11 章 11.4 节「B 树的性能分析」

位置	错在哪	证据
第 11 章图 11.8(b)、图 11.9	图 11.8(b) 结点 c 印作「120 150」，120 已删除，借位后应为「118 150」；图 11.9 叶 f 印作「5 30」，按父结点分界码应为「25 30」	图片不动，第 11 章正文 / 图注说明
第 11 章习题 4	题面说「图 11.9 所示的 B 树」插入 480、删除 375，但图 11.9 是关键码 5～90 的 B+ 树，没有 375	第 11 章习题 4 题面加注
第 12.4.1 节，BST 平均检索长度	「对 $C_{2n}^{n}/(n+1)$ 种形状取平均，平均检索次数是 $O(\log n)$」：按形状平均，结点平均深度是 $\Theta(\sqrt{n})$；$O(\log n)$（约 $1.39\log_2 n$）是对 $n!$ 种插入次序取平均	复算 $n=400$：按次序平均约 9.2，按形状平均约 32.6 ·第 12 章 12.4.1 节
第 12 章本章小结；图 12.5	「图⊇再入表⊇循环表⊇纯表⊇线性表」：纯表无回路，不可能是循环表，应为「图⊇再入表⊇纯表⊇线性表」（循环表是带回路的再入表），且与 12.2 节自相矛盾；图 12.5 上方标号式第一项多了一层括号 $(L_1:(a,b))$	第 12 章小结与 12.2 节

反复出现、但单独一条通常还能「跑两下」

这些不是某一条清单独有，而是同一类所有权错误在多章重复。编译能过，拷贝一次就二次释放。

- 有析构、无拷贝构造 / 拷贝赋值：顺序表、链表、顺序栈、链式栈、字符串、最小堆、图的裸数组。
- 容器里 `cout` 报错、用 `bool + 出参` 表达空状态：第 2、3、4 章通病。
- `int` 当下标与容量：有符号溢出是未定义行为。

  现代实现按 D-001 处理：显式五法则、`std::optional` 表达空、真错误抛标

准异常。

作者代码包里有没有

对照原书配套的作者代码包（2025 秋期末机考的考场资料之一，见附录「考场代码包」），上面每条编译不过或跑起来错的勘误都能再问一句：**作者自己的代码里也这样吗？**

- **包里也有**：作者的代码本来就这样，不是排印或 OCR 造成的；
- **包里没有**：印出来的与作者的代码不一致，包里是另一种（通常能编译的）写法。包与原书同为 2008 年 6 月，谁先谁后不可考，所以只说「不一致」，不说「排印时引入」；
- **包里无此代码**：包里没有对应的实现，无从比较。

勘误	清单	包里	说明
E01	代码 2.1 / 代码 2.2 / 算法 2.5	包里没有	包里在用的顺序表与单链表都叫 `del(int p)`。只有 `alg2.1-5/lnkList.h` 这份没被 `main.cpp` 包含、从不参与编译的旧稿还写着 `bool delete(int p)`——原书印的正是这种写法
E02	代码 2.1	包里也有	包里的 `class List` 同样通篇没有 `public:`——只是包里没有任何代码通过 `List` 调用这些运算，所以编译得过
E03	算法 2.3	包里没有	包里 `getPos` 的循环上界是成员 `curLen`，编译得过；原书印成未声明的 `n`

勘误	清单	包里	说明
E04	代码 2.6	包里没有	包里 `Link` 的构造参数是 `Link* nextValue`，没有 `const`，不存在丢弃 `const` 的赋值
E05	代码 3.2	包里没有	包里取栈顶叫 `getTop(T*)`，与 public 数据成员 `int top` 不重名，整个文件编译得过；原书印成 `bool top(T&)`
E06	算法 3.3	包里无此代码	包里三份 `arrStack.h` 的 `push` 都是栈满就打印「栈满溢出」返回 `false`，没有翻倍扩容的版本，无从比较
E07	代码 3.4	包里没有	包里链式栈同样用 `getTop(T*)`，与 `Link<T>* top` 不重名
E08 / E20	算法 3.5 / 算法 3.9	包里没有	包里栈的 ADT 就是 `pop(T* item)`，算法里写 `s.pop(&x)` 与之一致、编译得过；原书把 ADT 印成了 `pop(T&)`，算法却照抄了包里的 `pop(&x)`

勘误	清单	包里	说明
E09	算法 3.11	包里没有	包里是 `enum rdType {a, b, c}` 与普通的 `class knapNode`，没有字面量枚举值，也没有 Java 式的 `public class`
E10	算法 3.12	包里没有	包里 `t = st.top` 读的是 public 数据成员，取栈顶用 `st.getTop(&tmp)`，两者不重名
E11	算法 4.4	包里也有	包里就是 `assert(str != '\0')`——包里 `alg4-5/main.cpp` 今天编译不过，报的正是这一句（见附录「考场代码包」）
E12	算法 4.4	包里也有	包里的构造函数就是 `String(char *s)`
E13	代码 10.1	包里没有	包里初始化列表与数据成员都叫 `key`
E14	算法 3.6 / 3.8 / 3.9	包里也有	包里三版阶乘都是 `long` 连乘，不查溢出
E15	算法 3.6	包里也有	包里（整段注释掉的）递归版同样是 `if (n <= 0) return 1`
E16	算法 4.5	包里也有	包里 `Substr` 越界同样 `return NULL`（经 `String(char*)` 构造，`strlen(NULL)`）

勘误	清单	包里	说明
E17 / E21 / R10 / R13	算法 4.6 / 算法 4.8	包里也有	包里朴素匹配与 KMP 都返回 `j - pLen + 1`：作者的代码本来就按 1 起始报位置，不是排印问题。对拍 407 组，找到时作者恒比 0 起始位置大 1
E18	算法 10.9	包里也有	包里三份 `HASHDICT.H` 的析构都是 `delete HT`（`HT` 由 `new Elem[sz]` 分配）
E19	算法 3.5	包里也有	包里同样用 `operand1 == 0.0` 判除零
E22	顺序表/链表/顺序栈/链式栈/最小堆	包里也有	包里这些类都有析构、都没有拷贝构造与拷贝赋值，拷贝一次即二次释放。（包里的 `String` 连析构都没有：不二次释放，但每个串都泄漏。）

勘误	清单	包里	说明
R04	算法 2.9	包里也有	包里 `setPos` 与印刷版逐字相同：`p = new Link<T>(head->next)`。不止泄漏——`setPos(k)` 返回第 k-1 个结点，`insert(1, v)` 静默丢失、`insert(i, v)`（i ⩾ 2）早一格；对拍 2400 次随机插入逐项吻合这个错位
R07	算法 3.12	包里没有	扫描件第 67 页印的 `while` 条件少一个左括号（OCR 又把 `&&` 读成了 `&`，那是另一回事）；包里同一句以 `while ((tmp.s >= 0) &&` 开头，括号配平、编译得过
R12	算法 4.7	包里没有	包里写的是 `while (i < m)` 加 `if (i == m) break`——`break` 是必需的；原书把条件印成 `i < m-1` 又留着 `break`，才显得多余
R15	算法 5.x Parent	包里也有	包里 `Parent` 的循环走完后直接落到函数末尾，没有 `return NULL`（找不到父结点时返回值未定义）

只在包里、原书没有的缺陷见附录「考场代码包」的「包里的坑」。

曾经误判、已撤回

第 1 章股市传言的印刷矩阵里，B1 / B5 的最大最短路一度被读成 8，并据此声称「原书应返回 B1 而非正文的 B3」。那些 `8` 是 OCR 把 ∞ 认错；选起点用的图 1.3 是图片，从未被 OCR。按正文还原，**原书这个样例是对的**。详见 adt/legacy.md。

还没收进本表的

- 算法 2.11、代码 3.1、算法 7.6、算法 7.9 的结束标记被 OCR 吃掉，切片边界仍是手定（T-008）。
- 第 6–12 章不少「标识符被空格切断」「全角分号」更像 OCR 而不是作者写错，故不列入上表。

参考勘误表补充（前六章）

教材《1–6 章勘误表》还记录了几处不影响本书现代实现、但值得在对照纸书时知道的文字和 公式问题。本书已按语义处理，列在这里是为了让读者能把不同版本的页码对上：

原书位置	勘误要点	本书处理
第 1.4.2 节，第 19 页	“有序子数组” 应明确为原数组中相邻连续的有序子数组；长度比较应使用 `j2 - j1 + 1`	第 1 章把连续性和边界写成文字约定，并用半开区间表示长度
习题 1.4.2(3)，第 24 页	`T(n)2=...` 是排印错误，应为 `T(n)=...`	第 1 章复杂度只使用校正后的递推式
算法 2.9，第 37 页（勘误表错误 4）	`p` 应直接指向 `head->next`；模板拼写应修正。原书写的是 `p = new Link<T>(head->next)`：定位从一个新分配的多余结点起步，不止泄漏，`setPos(k)` 还返回第 k-1 个结点——`insert(1, v)` 静默丢失、之后的插入早一格（作者代码包里逐字相同，已真跑复现）	第 2 章链表使用 RAII 结点和明确的循链定位

原书位置	勘误要点	本书处理
算法 2.9，第 37 页（勘误表错误 9）	「多余分号」：分号在循环体结束的 `}` 之后（`};`），是一条空语句，**不影响循环语义**，可作排印清理	第 2 章链表的循环不带这个分号
算法 4.6/4.8，第 91、97 页	0 起始下标的匹配位置返回 `j-pLen`，不能加 1	第 4 章朴素匹配和 KMP 均按 0 起始并有对拍测试
算法 4.7，第 95 页	`while (i<m-1)` 已包含边界，不需额外 `if(i==m) break`	第 4 章 next 表构造采用单一边界条件
第 5、6 章图示	Parent 失败分支需返回空；树的子结点链表编号有印刷错误	第 5、6 章接口用 `optional`/空指针表达失败，并以代码测试为准

这些条目是参考勘误的内容性摘要，不替代原 PDF 的页码和完整记录。

考场代码包

2025 秋「数据结构与算法」期末机考（`dsa.openjudge.cn/2025final`）的考场说明列出四份**考场可以使用**的资料：本书作者的 C++ 代码包、一份 C 语言代码包、C++ 离线手册（cppreference）和 Weiss 的英文教材。本附录只讲与本书直接相关的第一份：`DSCode_ZhangWangZhao2008_06.zip`。

原书前言说：「教材中只给出了最主要的算法和代码，完整的实现方法可参考教学网站上提供的与本书配套的代码包。」这就是那个包。它是作者**实际编译运行过**的工程，每章一个目录，每个算法一两个文件。

它和书上印的不是同一份

包名里的 `200806` 与原书「2008 年 6 月第 1 版第 1 次印刷」是同一个月，但包里的代码和书上印的**不总是一致**，而且两边各有对错：

- **多数时候包里是对的**。原书第 3 章的顺序栈按印刷原样编译不过——数据成员 `int top` 与成员函数 `bool top(T&)` 重名；包里那个函数叫 `getTop(T*)`，编译得过。第 2 章的 `delete` 作函数名、第 3 章背包问题的 `enum rdType {0,1,2}` 也是这样。
- **也有书上对、包里错的时候**。原书算法 8.5 冒泡排序的内层循环写 `j > i`，包里写成 `j >= i`，第一趟就读到 `Array[-1]`；原书算法 2.11 的单链表删除写对了，包里删尾结点时会把同一个结点释放两次。
- **还有两边都错的**。算法 2.9 的 `setPos` 用 `new` 出一个多余的结点起步，书上和包里逐字相同；算法 4.6、4.8 的模式匹配都按 1 起始报位置，而正文说下标从 0 起。

所以考场上拿包里的代码来用，**先在本书里查一下那一条**：原书 105 条清单，本书逐条登记了它在包里的位置；每条「编译不过」或「跑起来是错的」勘误，都查过包里有没有；第 2、3、4、8 章还把包里的实现与本书的实现 在同一批输入上真跑了对拍。逐条结论见本附录后三节与书末「原书勘误」。

用之前要知道的三件事

1. **编码**。包里大多数文件是 GBK 编码、Windows 换行（2008 年 VC6 的默认），第 1、10、12 章另有几个带 BOM 的 UTF-8 文件。用 UTF-8 打开 GBK 文件，中文注释会是乱码——换成 GBK 重新打开即可，代码本身不受影响。
2. **编译器**。包是 VC6 年代写的，有些写法今天的 g++ 不再接受，最常见的是 `void main()`。OpenJudge 的 C++ 用 g++ 编译，交题前按「今天的 g++ 编译得过吗」一节改掉。
3. **容器会自己打印**。包里的栈、队列、线性表在出错时直接 `cout` 一句中文提示（「栈满溢出」「栈为空，不能出栈操作」），题目要求严格输出格式时，这些提示会让答案错。

清单在包里的位置

原书 105 条清单在包里的去处。符号是函数或类名；同一文件里有几个同名版本时注明第几个。

清单	包里的文件	符号
算法 1.1	`ch01_Intro/alg1.1/stockbroker.cpp`	`BrokerToPassRomour`
代码 1.2	包里没有	抽象数据类型的通用模板（className/Type 占位符），不是可编译程序，包里无对应文件
代码 2.1	`ch02_LinearList/alg2.1-5/myList.h`	`class List`
代码 2.2	`ch02_LinearList/alg2.1-5/arrList.h`	`class arrList`
算法 2.3	`ch02_LinearList/alg2.1-5/arrList.h`	`getPos`
算法 2.4	`ch02_LinearList/alg2.1-5/arrList.h`	`insert`
算法 2.5	`ch02_LinearList/alg2.1-5/arrList.h`	`del`；包里函数名是 `del`；原书印作 `delete`（C++ 关键字，不能作函数名）
代码 2.6	`ch02_LinearList/alg2.6-11/lnkList.h`	`class Link`
代码 2.7	`ch02_LinearList/alg2.6-11/lnkList.h`	`class lnkList`
算法 2.8	`ch02_LinearList/alg2.6-11/lnkList.h`	`lnkList`、`~lnkList`
算法 2.9	`ch02_LinearList/alg2.6-11/lnkList.h`	`setPos`
算法 2.10	`ch02_LinearList/alg2.6-11/lnkList.h`	`insert`
算法 2.11	`ch02_LinearList/alg2.6-11/lnkList.h`	`del`；包里函数名是 `del`；原书印作 `delete`（C++ 关键字，不能作函数名）
代码 2.12	包里没有	双链表：包里 ch02 只有顺序表与单链表，没有双链表结点

清单	包里的文件	符号
代码 3.1	ch03_StackQueue/alg3.5/myStack.h	class Stack
代码 3.2	ch03_StackQueue/alg3.5/arrStack.h	class arrStack；包里取栈顶叫 getTop(T*)，与成员变量 int top 不重名；原书印作 top(T&)，与之重名，编译不过
算法 3.3	包里没有	改进的进栈（满则容量翻倍）：包里三份 arrStack.h 的 push 都是栈满即报错返回 false，没有翻倍版本
代码 3.4	ch03_StackQueue/alg3.4/lnkstack.h	class lnkStack
算法 3.5	ch03_StackQueue/alg3.5/calc.h	class Calculator、GetTwoOperands、Compute、Run
算法 3.6	ch03_StackQueue/alg3.6-8/fact.cpp	factorial（第 1 个）；递归版在 fact.cpp 里整段注释掉了，与迭代版一起留作对照
算法 3.7	ch03_StackQueue/alg3.6-8/fact.cpp	main
算法 3.8	ch03_StackQueue/alg3.6-8/fact.cpp	factorial（第 2 个）；迭代版在 fact.cpp 里整段注释掉了
算法 3.9	ch03_StackQueue/alg3.6-8/fact.cpp	factorial；fact.cpp 里唯一未注释的版本：用栈模拟递归
算法 3.10	ch03_StackQueue/alg3.9-11/knap1.cpp	knap
算法 3.11	ch03_StackQueue/alg3.9-11/knap1.cpp	nonRecKnap
算法 3.12	ch03_StackQueue/alg3.9-11/knap1.cpp	nonRecKnapOpt
代码 3.13	ch03_StackQueue/alg3.13-14/myQueue.h	class Queue

清单	包里的文件	符号
代码 3.14	ch03_StackQueue/alg3.13-14/arrQueue.h	class arrQueue
代码 3.15	ch03_StackQueue/alg3.13-14/lnkQueue.h	class lnkQueue
代码 4.1	包里没有	字符串的抽象数据类型定义（成员用 ... 占位），不是可编译程序
代码 4.2	包里没有	印的是标准库 std::basic_string 的模板声明，不是作者的代码
算法 4.3	ch04_String/alg3/strcmp.cpp	strcmp
算法 4.4	ch04_String/alg4-5/myString.h	String（第 2 个）；myString.h 里第 1 个 String(...) 是无参构造，原书这一条是第 2 个 String(char *s)
算法 4.5	ch04_String/alg4-5/myString.h	Substr
算法 4.6	ch04_String/alg6/naive.h	NaiveStrMatching
算法 4.7	ch04_String/alg7-8/kmp.h	findNext
算法 4.8	ch04_String/alg7-8/kmp.h	KMPStrMatching
代码 5.1	ch05_BinaryTree/alg1-8BinaryTree/BinaryTreeNode.h	class BinaryTreeNode
代码 5.2	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	class BinaryTree
算法 5.3	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	PreOrder、InOrder、PostOrder
算法 5.4	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	PreOrderWithoutRecursion

清单	包里的文件	符号
算法 5.5	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	InOrderWithoutRecursion
算法 5.6	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	class StackElement、PostOrderWithoutRecursion
算法 5.7	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	LevelOrder
代码 5.8	ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.h	isEmpty、Parent、LeftSibling、RightSibling、CreateTree、DeleteBinaryTree
算法 5.9	ch05_BinaryTree/alg9-10binarysearchtree/BinarySearchTree.h	InsertNode
算法 5.10	ch05_BinaryTree/alg9-10binarysearchtree/BinarySearchTree.h	DeleteNodeEx
代码 5.11	ch05_BinaryTree/alg11MinHeap/MinHeap.h	class MinHeap、SiftDown、SiftUp
代码 5.12	ch05_BinaryTree/alg12HuffmanTree/HuffmanTree.h	class HuffmanTree、HuffmanTree
代码 6.1	ch06_Tree/tree(alg_1-7)/TreeNode.h	class TreeNode
代码 6.2	ch06_Tree/tree(alg_1-7)/Tree.h	class Tree
算法 6.3	ch06_Tree/tree(alg_1-7)/Tree.h	RootFirstTraverse
算法 6.4	ch06_Tree/tree(alg_1-7)/Tree.h	RootLastTraverse
算法 6.5	ch06_Tree/tree(alg_1-7)/Tree.h	WidthTraverse

清单	包里的文件	符号
代码 6.6	ch06_Tree/tree(alg_1-7)/TreeNode.h	InsertFirst、InsertNext
代码 6.7	ch06_Tree/tree(alg_1-7)/Tree.h	Parent、PrevSibling、DeleteSubTree
代码 6.8	ch06_Tree/unionfind(alg_8-9)/ParTree.h	class ParTree、Find、Union
算法 6.9	ch06_Tree/unionfind(alg_8-9)/ParTree.h	FindPC
算法 6.10	ch06_Tree/DualTag-Firstroot(alg6_10)/dualt-agtree_firstroot.cpp	class DualTagTreeNode、Tree
代码 7.1	ch07_Graph/graph_matrix/Graph.h	class Graph；包里只有一个 Graph 类：原书把 ADT（代码 7.1）与基类（代码 7.2）分成两段印
代码 7.2	ch07_Graph/graph_matrix/Graph.h	class Edge、class Graph
代码 7.3	ch07_Graph/graph_matrix/Graphm.h	class Graphm
代码 7.4	ch07_Graph/Graph_Link/Graphl.h	class Graphl
算法 7.5	ch07_Graph/graph_matrix/Graphm.h	DFS
算法 7.6	ch07_Graph/graph_matrix/Graphm.h	BFS
算法 7.7	ch07_Graph/TopologicalSorting/topologi-cal_sorting.cpp	TopsortbyQueue
算法 7.8	ch07_Graph/Graph_Dijkstra/Dijkstra.cpp	class Dist、Dijkstra
算法 7.9	ch07_Graph/Graph_Floyd/Floyd.cpp	Floyd

清单	包里的文件	符号
算法 7.10	ch07_Graph/Graph_Prim/Prim.cpp	minVertex、Prim
算法 7.11	ch07_Graph/Graph_Kruskal/Kruskal.cpp	Kruskal
算法 8.1	ch08_Sort/InsSort.cpp	InsertSort
算法 8.2	ch08_Sort/ShellSort/ShSort2.cpp	ShellSort、ModInsSort；ShSort3.cpp 是增量每次除以 3 的变体，原书没有印
算法 8.3	ch08_Sort/SelSort.cpp	SelectSort
算法 8.4	ch08_Sort/HeapSort/HeapSort.cpp	sort；堆排序主体在 sort() 里，堆本身见 HeapSort/MaxHeap.h
算法 8.5	ch08_Sort/BubSort.cpp	BubbleSort
算法 8.6	ch08_Sort/QuickSort/QuickSort.cpp	QuickSort、SelectPivot、Partition
算法 8.7	ch08_Sort/QuickSort/ModQuickSort.cpp	ModQuickSort
算法 8.8	ch08_Sort/MergeSort/MergeSort.cpp	MergeSort、Merge
算法 8.9	ch08_Sort/MergeSort/ModMergeSort.cpp	ModMergeSort、ModMerge
算法 8.10	ch08_Sort/BucketSort.cpp	BucketSort
算法 8.11	ch08_Sort/RadixSort/RadixSort_Bin.cpp	RadixSort
代码 8.12	ch08_Sort/RadixSort/LinkRadixSort.h	class StaticQueue
算法 8.13	ch08_Sort/RadixSort/LinkRadixSort.h	RadixSort、Distribute、Collect
算法 8.14	ch08_Sort/AddSort_Insert.cpp	IndexSort；文件开头还有一个整段注释掉的 IndexSort(Array, n)（内部 new 索引数组、结尾不 delete）；原书印的是带 IndexArray 参数的这一版

清单	包里的文件	符号
算法 8.15	ch08_Sort/AddSort_Insert.cpp	AdjustRecord
代码 8.16	ch08_Sort/sort.h	Randomize、Random
代码 8.17	ch08_Sort/sort.h	Settime、Gettime
算法 9.1	ch09_ExternalSort/alg9_1replacementSelection/ReplacementSelection.cpp	replacementSelection
代码 9.2	ch09_ExternalSort/alg9_2WinnerTree/WinnerTree.h	class WinnerTree
代码 9.3	ch09_ExternalSort/alg9_3LoserTree/LoserTree.h	class LoserTree
代码 10.1	ch10_Search/algor1-3/algor1&2.cpp	class Item
算法 10.2	ch10_Search/algor1-3/algor1&2.cpp	SeqSearch
算法 10.3	ch10_Search/algor1-3/algor1&2.cpp	BinSearch
代码 10.4	ch10_Search/BitSet/setm.h	class mySet
算法 10.5	ch10_Search/BitSet/setm.h	set
算法 10.6	ch10_Search/BitSet/setm.h	operator&
算法 10.7	ch10_Search/BitSet/setm.h	operator>
算法 10.8	ch10_Search/hash_dict/HASHDICT.H	ELFhash
算法 10.9	ch10_Search/hash_dict/HASHDICT.H	class hashdict
算法 10.10	ch10_Search/hash_dict/HASHDICT.CPP	hashInsert
算法 10.11	ch10_Search/hash_dict/HASHDICT.CPP	hashSearch

清单	包里的文件	符号
算法 10.12	ch10_Search/hash_tomb2/HASHDICT.CPP	hashDelete
算法 10.13	ch10_Search/hash_tomb2/HASHDICT.CPP	hashInsert；hash_tomb 与 hash_tomb2 两版：tomb2 记下第一个墓碑位置（insplace）再插入，与原书一致
算法 12.1	包里没有	可利用空间表（重载 LinkNode 的 new/delete）：包里 ch12 只有广义表、稀疏矩阵、AVL、伸展树与最佳 BST
算法 12.2	ch12_AdvDS/AdvTree/BestBST/OptimalBST.cpp	OptimalBST

今天的 g++ 编译得过吗

包里能单独编译成程序的 .cpp 共 54 个（自身或它包含的头文件里有 main；第 8 章的排序都把 main 放在 SortMain.h 里）。先把源码转成 UTF-8，再按 -std=c++17 只做语法检查：g++（gcc/libstdc++）38 个编译得过；clang（clang/libstdc++）38 个编译得过；clang + libc++（clang/libc++）37 个编译得过；Apple clang（apple-clang/libc++）37 个编译得过。**编译结果随工具链而变**：OpenJudge 的 C++ 用 g++；macOS 上的 g++ 其实是 Apple clang + libc++，最接近「clang + libc++」那一列。

编译不过的几乎全是 2008 年 VC6 能容忍、今天的编译器不再接受的写法，不是算法错。下表列出在任一工具链下编译不过的程序（√ 编译得过，✗ 编译不过），报错取第一个失败工具链的第一条。

程序	g++	clang	clang + libc++	Apple clang	第一条报错	改法
ch04_String/alg4-5/main.cpp	✗	✗	✗	✗	ISO C++ forbids comparison between pointer and integer [-fpermissive]	assert(str != '\0') 改为 assert(str != NULL)（勘误 E11）
ch05_BinaryTree/alg1-8BinaryTree/BinaryTree.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch05_BinaryTree/alg9-10binarysearchtree/binarysearchtree.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch06_Tree/DualTag-First-root(alg6_10)/dualtagtree_firstroot.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch06_Tree/DualTag-Width(alg6_11)/dualtagtree_Width.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch06_Tree/union-find(alg_8-9)/unionfind.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()

程序	g++	clang	clang + libc++	Apple clang	第一条报错	改法
ch07_Graph/Graph_Floyd/Floyd.cpp	✗	✗	✗	✗	extra qualification ‘Graphm::’ on member ‘Init-Graphm’ [-fpermissive]	类内声明去掉 Graphm:: 前缀
ch07_Graph/Graph_Link/link.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch07_Graph/Graph_Prim/Prim.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch07_Graph/graph_matrix/matrix.cpp	✗	✗	✗	✗	‘::main’ must return ‘int’	void main() 改成 int main()
ch08_Sort/RadixSort/LinkRadixSort.cpp	✗	✗	✗	✗	‘i’ was not declared in this scope	LinkSort.h 的 PrintAddr 里循环变量 i 没声明：写成 for (int i=0; ...)

程序	g++	clang	clang + libc++	Apple clang	第一条报错	改法
ch09_ExternalSort/alg9_1replace-mentSe-lection/Replace-mentSe-lec-tion.cpp	✗	✗	✗	✗	reference to ‘less’ is am-biguous	自定义的 less 与 std::less 撞名：改名，或去掉 using names-pace std;
ch09_ExternalSort/alg9_2WinnerTree/Win-nerTree.cpp	✗	✗	✗	✗	‘cout’ was not declared in this scope; did you mean ‘std::cout’ ?	补 using names-pace std;，或写 std::cout
ch09_ExternalSort/alg9_3LoserTree/LoserTree.cpp	✗	✗	✗	✗	‘cout’ was not declared in this scope; did you mean ‘std::cout’ ?	补 using names-pace std;，或写 std::cout

程序	g++	clang	clang + libc++	Apple clang	第一条报错	改法
ch10_Search/BitSet/bag.cpp	✓	✓	✗	✗	reference to 'count' is ambiguous	全局变量 count 与 std::count 撞名（只在 libc++ 下）：改名，或去掉 using namespace std;
ch12_AdvDS/AdvList/SparseMatrix/Example.cpp	✗	✗	✗	✗	afxtempl.h: No such file or directory	VC6 的 MFC 头文件，g++ 没有：换成标准库容器
ch12_AdvDS/AdvTree/AVLTree/Example.cpp	✗	✗	✗	✗	redeclaration of ‘avlNode<T>::avlNode(avlNode<T>*, avlNode<T>*, int)’ may not have default arguments [-fpermissive]	默认参数只留在类内声明，类外定义处删掉

还有一件事语法检查看不出来：第 8 章这些排序程序在 clang 下「编译得过」，只因为计时驱动里的排序调用被注释掉了（见「包里的坑」P06），模板从没被实例化。真去调用 ShellSort、QuickSort、MergeSort 时，clang 会按标准报「函数在模板定义处不可见」——作者都是先调用、后定义（ModInsSort、Partition、

Merge……），只有 g++ 放行。在 clang 下用它们，把被调函数的定义挪到前面，或先写一行声明。

所有工具链下都编译得过的：ch02_LinearList/ alg2.1-5/ main.cpp、ch02_LinearList/ alg2.6-11/ main.cpp、ch03_StackQueue/ alg3.12/ main.cpp、ch03_StackQueue/ alg3.13-14/ main.cpp、ch03_StackQueue/ alg3.4/ main.cpp、ch03_StackQueue/ alg3.5/ main.cpp、ch03_StackQueue/ alg3.6-8/ fact.cpp、ch03_StackQueue/alg3.9-11/knap1.cpp、ch04_String/alg3/strcmp.cpp、ch04_String/alg6/main.cpp、ch04_String/alg7-8/main-kmp.cpp、ch05_BinaryTree/alg11MinHeap/MinHeap.cpp、ch05_BinaryTree/alg12HuffmanTree/HuffmanTree.cpp、ch06_Tree/tree(alg_1-7)/tree.cpp、ch07_Graph/Graph_Dijkstra/Dijkstra.cpp、ch07_Graph/Graph_Kruskal/ Kruskal.cpp、ch07_Graph/ TopologicalSorting/ topological_sorting.cpp、ch08_Sort/ AddSort_Insert.cpp、ch08_Sort/ BubSort.cpp、ch08_Sort/ BucketSort.cpp、ch08_Sort/ HeapSort/ HeapSort.cpp、ch08_Sort/ InsSort.cpp、ch08_Sort/ MergeSort/ MergeSort.cpp、ch08_Sort/ MergeSort/ ModMergeSort.cpp、ch08_Sort/QuickSort/ModQuickSort.cpp、ch08_Sort/QuickSort/ QuickSort.cpp、ch08_Sort/RadixSort/LinkSortMain.cpp、ch08_Sort/RadixSort/ RadixSort_Bin.cpp、ch08_Sort/SelSort.cpp、ch08_Sort/ShellSort/ShSort2.cpp、ch08_Sort/ShellSort/ShSort3.cpp、ch10_Search/BitSet/test.cpp、ch10_Search/algor1-3/algor1&2.cpp、ch12_AdvDS/AdvList/GenList/Example.cpp、ch12_AdvDS/AdvList/ GenList_ReEntrant/ Example.cpp、ch12_AdvDS/ AdvTree/ BestBST/ OptimalBST.cpp、ch12_AdvDS/AdvTree/SplayTree/splay.cpp。

包里的坑

下面几处缺陷**原书没有**——要么原书没印那段代码，要么印对了——只在包里。考场上直接拿包里的代码交题，会在这些地方读写到数组外面或得到错的结果。

编号	位置	缺陷
P01	算法 8.4（包里的 MaxHeap），ch08_Sort/ HeapSort/MaxHeap.h	MaxHeap 的 SiftDown 判右孩子写 j<CurrentSize，应为 j<CurrentSize-1：n 为偶数时建堆读 heapArray[n]，越界一格，那格若更大还会被换进堆、再把 temp 写出界。原书没印 MaxHeap；第 5 章 MinHeap 与代码 5.11 写对了。ASan 在对拍里当场报 heap-buffer-overflow

编号	位置	缺陷
P02	算法 8.5，`ch08_Sort/BubSort.cpp`	`BubbleSort` 内层循环写 `j>=i`：i=0 时比较 `Array[0]` 与 `Array[-1]`，越界一格；原书印的 `j > i` 是对的
P03	算法 2.11，`ch02_LinearList/alg2.6-11/lnkList.h`	`lnkList` 的 `del` 删尾结点时 `tail = p; delete q;` 之后又落进 `if (q != NULL) { p->next = q->next; delete q; }`——释放后使用加二次释放；原书用 `else if` 写对了，还多了 `p == tail` 的判断
P04	算法 3.10，`ch03_StackQueue/alg3.9-11/knap1.cpp`	`knap` 最后一行是 `else knap(s, n-1);`——少了 `return`，结果被丢弃、函数从末尾流出（未定义行为）；原书印的 `else return knap(s, n-1)` 是对的
P05	算法 8.10（包里的驱动），`ch08_Sort/BucketSort.cpp`	`BucketSort.cpp` 的 `sort` 包装写死 `max = 80`，而计时驱动 `SortMain.h` 生成 `Random(32003)`——值 ≥ 80 时 `count[TempArray[i]]` 越界写。只因驱动里的排序调用被注释掉（P06）才没炸

编号	位置	缺陷
P06	第 8 章计时驱动，`ch08_Sort/SortMain.h`	`SortMain.h` 的计时循环里 `// sort<int>(&array[i], listsize);` 被注释掉：包里 12 个排序程序跑起来只生成 1000 万个随机数、给空循环计时，随后逐对检查有序、打出成串 `ERROR!!!`——作者的排序从没在这个驱动里真正跑过

另有与原书同病、但不在勘误表里的：- P07（代码 3.2）：无参构造 `arrStack()` 只设 `top = -1`，`mSize` 与 `st` 都未初始化，析构时 `delete [] st` 释放野指针——原书代码 3.2 的无参构造同病

与原书同病的那些（印出来就错、包里也错）见书末「原书勘误」的「作者代码包里有没有」一节。